

```

1  ; *****
2  ; IBMBIO7.S (PCDOS 7.1 IBMBIO.COM) - RETRO DOS 5.0 by ERDOGAN TAN - 12/09/2023
3  ; -----
4  ; Last Update: 04/02/2026 - Retro DOS v5.0 (Modified PCDOS 7.1)
5  ; -----
6  ; Beginning: 26/12/2018 (Retro DOS 4.0), 01/10/2022 (Retro DOS 4.2)
7  ; -----
8  ; Assembler: NASM version 2.15
9  ; -----
10 ; ((nasm ibmbio7.s -l ibmbio7.txt -o IBMBIO.COM -Z error.txt))
11 ; -o IBMBIO7.BIN
12 ; *****
13 ;
14 ; 12/09/2023 - Retro DOS v5.0 Kernel -dosbios- ('ibmbio7.s')
15 ; Modified from 'iosys6.s' (11/09/2023, Retro DOS v4.2 Kernel's IO.SYS) file
16 ; as below:
17 ;
18 ; 1) Retro DOS v4.2 IO.SYS is based on disassembled source code
19 ; of MSDOS 6.21 IO.SYS, derived using MSDOS 6.0 source code.
20 ;
21 ; 2) Labels, names, comments, explanations and structure definitions
22 ; about procedures and code details are almost entirely taken from
23 ; the original MSDOS 6.0 source code, except for the details that
24 ; Erdogan Tan personally experienced. Some of them are incompatible
25 ; with PCDOS 7.1 code. But they have not been deleted to preserve
26 ; the originality of the descriptions.)
27 ;
28 ; 3) 'ibmbio7.s' contains the BIOSLOADER (MSLOADER) section located in
29 ; the 1st 4 sectors of the IBMBIO.COM file on disk. This is a method
30 ; from older DOS versions (3 sectors for MSDOS 6.22).
31 ; The MSDOS/PCDOS boot sector code only reads these MSLOADER/BIOSLOADER
32 ; sectors and transfers control to the MSLOADER/BIOSLOADER code.
33 ; BUT!!! The Retro DOS v3 (& v5) boot sector code loads the entire
34 ; MSDOS.SYS/PCDOS.SYS -combined- kernel file into memory at once.
35 ; So, hence the Retro DOS boot sector code, 'retrodos5.s' file
36 ; contains slightly different IO.SYS/IBMBIO.COM INITIALIZATION code
37 ; than the original PCDOS/MSDOS. It does not include
38 ; the MSLOADER/BIOSLOADER section. The 'retrodos5.s' and 'ibmbio7.s'
39 ; files are almost identical except their INIT codes.)
40 ;
41 ; ('iosys6.s' has been converted to 'ibmbio7.s' and 'retrodos42.s' has been
42 ; converted to 'retrodos5.s'. 'ibmbio7.s' is IBMBIO.COM source code file
43 ; while 'retrodos5.s' is source code of Retro DOS v5 kernel file 'PCDOS.SYS'.
44 ; 'retrodos5.s' includes 'ibmdos7.bin' or IBMDOS.COM as binary file.)
45 ;
46 ; -----
47 ;
48 ; 09/12/2022 - Multisection binary file format (BIOSDATA & BIOSCODE sections)
49 ; 01/10/2022 - Erdogan Tan (Istanbul)
50 ;
51 ; Note: This code is a part of Retro DOS 4.0 kernel source code
52 ; (as included binary, 'IOSYS5.BIN')
53 ; Equivalent of MSDOS 5.0 IO.SYS, BIOSCODE and BIOSDATA and SYSINIT
54 ; (except MSLOAD code)
55 ;
56 ; ---- Retro DOS v2 (v3) boot sector loads RETRODOS.SYS (MSDOS.SYS)
57 ; at 1000h:0000h and loader (initialization) part of RETRODOS kernel
58 ; moves IO.SYS (DOSBIOSCODE & DOSBIOSDATA, 'IOSYS5.BIN') to 70h:0000h.
59 ; Then SYSINIT code to the next segment (46Dh for original MSDOS 5.0)..
60 ; SYSINIT code relocates itself and DOSBIOSCODE and MSDOS.SYS
61 ; (MSDOS5.BIN) according to request/setting in 'config.sys' file.
62 ;
63 ; -----
64 ;
65 ; -----
66 ;
67 ; +-----+
68 ; | This file has been generated by The Interactive Disassembler (IDA) |
69 ; | Copyright (c) 2013 Hex-Rays, <support@hex-rays.com> |
70 ; | Licensed to: Freeware version |
71 ; +-----+
72 ;
73 ; -----
74 ;
75 ; .386
76 ; .model flat
77 ;
78 ; =====
79 ;
80 ; 12/09/2023 - Erdogan Tan - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
81 ;
82 ; -----
83 ;
84 ; [[ Most of comments here are from the original MSDOS 6.0 source code ]]
85 ;
86 ; -----
87 ; Start of (PCDOS 7.1) IBMBIO.COM
88 ; -----
89 ;
90 ; [ORG 0] ; segment 0x0070h
91 ;
92 ; =====
93 ; IBMBIO.COM (IO.SYS) LOADER SECTION
94 ; =====
95 ; 12/09/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
96 ; 09/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
97 ; non-contiguous io.sys loader (msload) ((MSDOS 6.0 source: MSLOAD.ASM, 1991))
98 ;
99 section .MSLOAD ; vstart=0 ;; .BIOSLOAD
100 ;
101 ; =====
102 ;
103 ; 09/12/2022
104 ; Comments are from MSDOS 6.0 MSLOAD.ASM (1991) & HEX-RAYS IDA disasm output
105 ;
106 ; =====
107 ; NOTE: The boot loader should be verifying that the first
108 ; block of io.sys is, in fact, at cluster 2. This would be saving
109 ; a whole lot of time during system debugging.
110 ;
111 ; =====
112 ;
113 ; for dos 4.00, msload program has been changed to allow:
114 ; 1. 32 bit calculation,
115 ; 2. reading a fat sector when needed, instead of reading the whole
116 ; fat sectors at once. this will make the boot time faster,
117 ; and eliminate the memory size limitation problem,
118 ; 3. solving the limitation of the file size (29 kb) of io.sys0,
119 ; 4. adding the boot error message. show the same boot error message
120 ; and do the same behavior when the read operation of io.sys
121 ; fails as the msboot program, since msload program is the
122 ; extension of msboot program.
123 ;
124 ; =====

```

```

125
126
127
128
129
130
131
132
133
134
135
136 00000000 EB45
137 00000002 90
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184 00000003 07
185 00000004 0A
186 00000005 0000
187 00000007 0000
188 00000009 0000
189 0000000B 0000
190
191 0000000D FFFF
192 0000000F FFFF
193 00000011 0000
194
195
196
197 00000013 0000
198
199 00000015 0000
200
201
202 00000017 0000
203 00000019 0000
204
205 0000001B 0000
206 0000001D 0000
207
208 0000001F 0000
209 00000021 0000
210 00000023 0000
211 00000025 0000
212 00000027 0000
213 00000029 0000
214 0000002B 0000
215 0000002D 0000
216 0000002F 0000
217 00000031 0000
218 00000033 0000
219 00000035 0000
220 00000037 0000
221
222 00000039 0000
223 0000003B 0000
224 0000003D 00
225 0000003E 00
226 0000003F 00
227 00000040 00
228 00000041 00000000
229
230
231
232 00000045 0000
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248

```

```

;-----
; M056 : Added RPL support, so that RPL's fake INT 13 code can be safe from
;        SYSINIT & transient portion of COMMAND.COM
;-----
[ORG 0]                ; segment 0x0070h

START$:
    jmp     short SaveInputValues ; 13/09/2023
    nop     ; 13/09/2023

%if 0
; 20/12/2022
; 09/12/2022
;-----
SysVersion: dw 5          ; expected_version
;MyStacks:  db 256 dup(0) ; local stack
; 22/12/2022
; 20/12/2022
;MyStacks:  dw 102 dup(0) ; local stack
NumHeads:   dw 0          ; ...
ClusterSize: dw 0         ; ...
StartSecL:  dw 0          ; ...
StartSecH:  dw 0          ; ...
TempH:      dw 0          ; for 32 bit calculation
TempCluster: dw 2 dup(0)  ; temporary place for cluster number
LastFatSector: dw 2 dup(0FFh) ; fat sec # start from 1st FAT entry
SectorCount: dw 0         ; ...
SecPerFat:  dw 0          ; ...
HiddenSectorsL: dw 0      ; ...
HiddenSectorsH: dw 0      ; ...
BytesPerSec: dw 0         ; ...
ReservSectors: dw 2 dup(0) ; ...
CurrentCluster: dw 2 dup(0) ; ...
NextBioLocation: dw 2 dup(0) ; ...
FirstSectorL:  dw 0       ; ...
FirstSectorH:  dw 0       ; ...
TotalSectorsL: dw 0       ; max. number of sectors
TotalSectorsH: dw 0       ; ...
SecPerTrack:  db 2 dup(0) ; ...
BootDrive:    db 0        ; ...
Fatsize:      db 0        ; ...
MediaByte:    db 0        ; ...
EndOfFile:    db 0        ; ...
OrgDasdPtr:   db 4 dup(0) ; ...
FatSegment:   db 2 dup(0) ; ...
SecPerCluster: db 0       ; ...
;-----
%endif

; 13/09/2023 (Retro DOS v5)
; 24/12/2022 (Retro DOS v4)
; 23/12/2022
; 20/12/2022
; 09/12/2022
;-----
;SysVersion:      dw 5          ; expected_version
SysVersionMajor:  db 7          ; Retro DOS v5.0 (IBM PC DOS 7.1)
SysVersionMinor:  db 10
ClusterSize:      dw 0
StartSecL:        dw 0
StartSecH:        dw 0
TempH:            dw 0          ; for 32 bit calculation
;TempCluster:     dw 0
LastFatSectorL:   dw 0FFFFh     ; fat sec # start from 1st FAT entry
LastFatSectorH:   dw 0FFFFh     ; fat sec # start from 1st FAT entry
SectorCount:      dw 0
CurrentCluster:   ; 06/10/2023
CurrentClusterL:  dw 0
CurrentClusterH:  dw 0          ; 13/09/2023 - HW of FAT32 cluster number
; 27/12/2023
FirstCluster:     ; 06/10/2023
FirstClusterL:    dw 0
FirstClusterH:    dw 0
;;
BytesPerSec:      dw 0
SecPerCluster:    dw 0
; 13/09/2023
ReservSectors:    dw 0
NumFats:          dw 0
RootEntCnt:       dw 0
SecPerTrack:      dw 0
NumHeads:         dw 0
HiddenSectorsL:   dw 0
HiddenSectorsH:   dw 0
TotalSectorsL:    dw 0          ; max. number of sectors
TotalSectorsH:    dw 0
FATSectorsL:      dw 0
FATSectorsH:      dw 0
RootClusterL:     dw 0
RootClusterH:     dw 0
;;
FirstSectorL:     dw 0
FirstSectorH:     dw 0
BootDrive:        db 0
FatType:          db 0
MediaByte:        db 0
EndOfFile:        db 0
OrgDasdPtr:       dd 0
; 06/10/2023
;FatStartSecL:    dw 0
;FatStartSecH:    dw 0
FatSegment:        dw 0
; 05/10/2023
;NextBioLocation:
; 05/10/2023 (bp register will be used instead of [NextBioLocation])
;dw 0
; 13/09/2023

; SaveInputValues
;-----
; INPUT:          none
;
; dl = int 13 drive number we booted from
; ch = media byte
; bx = first data sector (low) on disk (0-based)
; ds:si = original rom bios diskette parameter table.
;

```

```

249 ; if an extended boot record, then ax will be the first data sector
250 ; high word. save ax and set FirstSectorH according to ax if it is an
251 ; extended boot record.
252 ;
253 ; ax = first data sector (high) on disk ;
254 ; OUTPUT:
255 ;
256 ; bx = first data sector on disk
257 ;
258 ; MediaByte = input ch
259 ; BootDrive = input dl
260 ; FirstSectorL = input bx
261 ; FirstSectorH = input ax, if an extended boot record.;j.k.
262 ; TotalSectorsL = maximum sector number in this media ;j.k.
263 ; TotalSectorsH = high word of the above
264 ; HiddenSectorsL = hidden sectors
265 ; HiddenSectorsH
266 ; ReservSectors = reserved sectors
267 ; SecPerTrack = sectors/track
268 ; NumHeads = heads/cylinder
269 ;
270 ; ds = 0
271 ; AX,DX,SI destroyed
272 ;
273 ; calls: none
274 ;-----
275 ;FUNCTION:
276 ; save input information and bpb informations from the boot record.
277 ;-----
278
279 Sec9 equ 522h
280 ; 20/12/2022
281 DskAddr equ 1Eh*4 ; 78h
282 ; 22/12/2022
283 ;StackPtr equ MyStacks+(NumHeads-MyStacks)
284 ;-----
285 ;
286 ;
287 ; 13/09/2023
288 ; (registers from PC DOS 7.1 boot sector)
289 ; ss = 0
290 ; sp = 7BE4h
291 ; [0:7BE4h] = ss:bx = 0:78h (1Eh vector)
292 ; [0:7BE8h] = ds:si = DSK_PARMS (INT 1Eh) table address
293 ; bp = 7BECh
294 ; ds = 0
295 ; ax:bx = absolute disk address for cluster 2 (data start)
296 ; = dword/far ptr [0:7BFCh]
297 ; es = ax
298 ; dl = [BootDrv] = [7C40h] ; !FAT32 BPB!
299 ; ch = [MediaByte] = [7C15h]
300 ; ds:si = rom bios disk(ette) params table address (INT 1Eh)
301 ; = [0:7BE8h] = 0:7BECh
302 ; (ds:si is also in stack, at [0:7BE8h])
303 ; 0:500h = root dir buffer (1st sector of the root dir)
304 ; [0:7Eh] = disk(ette) params table address = 0:7BECh
305 ; (head settle time = 15ms)
306 ;
307
308 SaveInputValues:
309 ; 13/09/2023 (Retro DOS v5 MSLOADER/BIOSLOADER)
310 mov di, ds ; DSK_PARMS (INT 1Eh) table segment
311 ;
312 ; 24/12/2022 (Retro DOS v4 MSLOADER)
313 push cs
314 pop ds
315 ;mov [cs:FirstSectorL], bx ; first data sector (low word)
316 ;mov [cs:MediaByte], ch
317 ;mov [cs:BootDrive], dl
318 ; 13/09/2023
319 mov [FirstSectorL], bx
320 mov [FirstSectorH], ax
321 mov [StartSecL], bx ; **!***
322 mov [StartSecH], ax ; **!***
323 mov [MediaByte], ch
324 mov [BootDrive], dl
325 ;
326 ; 13/09/2023
327 ; (PC DOS 7.1 MSLOAD:0058h)
328 ;pop si
329 ;pop ds
330 ; ; from BS code..
331 ; ; ss:sp = 0:7BE4h, bp = 7BECh
332 ; ; Clear stack and load disk parameters table in ds:si
333 ; ;
334 ; ; pop.. Original INT 1Eh vector address
335 ;pop si
336 ;pop ds
337 ; ; pop.. Original INT 1Eh disk table address
338 ;
339 ; 13/09/2023
340 ; Note: DS:SI -from BS- points to DSK_PARMS (INT 1Eh) tbl addr
341 ; (no need to pop/take address from stack)
342 ;
343 ;mov sp, bp ; sp = 7BECh
344 ;
345 ; sp = 7BE4h
346 ;
347 ; 13/09/2023
348 ; Save original (ROMBIOS) DSK_PARMS table address
349 mov [OrgDasdPtr], si ; DSK_PARMS (INT 1Eh) tbl offset
350 mov [OrgDasdPtr+2], di ; DSK_PARMS (INT 1Eh) tbl segment
351 ;
352 xor cx, cx ; segment 0 (obviously)
353 mov ds, cx ; ZERO
354 ; 13/09/2023
355 mov es, cx
356 push di
357 di, Sec9
358 mov [DskAddr], di ; mov [78h], di ; 522h
359 mov [DskAddr+2], cx ; mov [7Ah], cx ; 0
360 pop ds
361 mov cl, 14 ; (11+3 bytes for IBM rombios)
362 cld
363 rep movsb ; copy table
364 ; 20/12/2022
365 mov ds, cx ; 0
366 ; 23/12/2022
367 ; es = 0
368 ; ds = 0
369 ; ss = 0
370 ;
371 ; 13/09/2023
372 ;mov cx, [051Ah] ; LW of IBMBIO.COM (IO.SYS) first cluster
373 ;mov [cs:CurrentCluster], cx

```

```

373             ;mov     cx, [0514h]      ; HW of IBMBIO.COM (IO.SYS) first cluster
374             ;mov     [cs:CurrentCluster+2], cx
375 ; 24/12/2022
376 %if 0
377             mov     cx, [7C0Bh]      ; BootSector.ext_boot_bpb.BPB_bytespersector
378             mov     [cs:BytesPerSec], cx
379             mov     cl, [7C0Dh]      ; BootSector.ext_boot_bpb.BPB_sectorspercluster
380             mov     [cs:SecPerCluster], cl
381             mov     cx, [7C18h]      ; BootSector.ext_boot_bpb.BPB_sectorspertrack
382             mov     [cs:SecPerTrack], cx
383             mov     cx, [7C1Ah]      ; BootSector.ext_boot_bpb.BPB_heads
384             mov     [cs:NumHeads], cx
385             ;mov     cx, [7C16h]      ; BootSector.ext_boot_bpb.BPB_sectorsperfat
386             ;mov     [cs:SecPerFat], cx
387             ; 13/09/2023
388             mov     dx, [7C16h]      ; BootSector.ext_boot_bpb.BPB_sectorsperfat
389             ;mov     [cs:FATsectorsL], dx
390             mov     bl, [7C26h]      ; BS_BootSig ; (FAT12 and FAT16)
391             or      dx, dx ; **
392             jnz     short not_fat32
393             mov     bl, [7C42h]      ; BS_BootSig ; (FAT32)
394 not_fat32:
395             mov     cl, [7C10h]      ; BPB_NumFATS
396             mov     [cs:NumFats], cl
397             mov     cx, [7C11h]      ; BPB_RootEntCnt
398             mov     [cs:RootEntCnt], cx
399             ;
400             mov     cx, [7C0Eh]      ; BootSector.ext_boot_bpb.BPB_reservedsectors
401             mov     [cs:ReservSectors], cx
402             mov     cx, [7C1Ch]      ; BootSector.ext_boot_bpb.BPB_hiddensectors
403             mov     [cs:HiddenSectorsL], cx
404             mov     cx, [7C13h]      ; BootSector.ext_boot_bpb.BPB_totalsectors
405             mov     [cs:TotalSectorsL], cx
406
407             ; First of all, check if it the boot record is an extended one.
408             ; This is just a safe guard in case some user just "copy" the
409             ; 4.00 iosys.com to a media with a conventional boot record.
410
411             ; 22/12/2022
412             ;cmp     byte [7C26h], 29h ; ext_boot_signature
413             ; 13/09/2023
414             cmp     bl, 29h
415             jne     short Relocate ; old boot sector
416             ; no need to copy high words
417             mov     [cs:FirstSectorH], ax ; Start sector # of data, high word
418             mov     ax, [7C1Eh]      ; BPB_HiddSec+2
419             mov     [cs:HiddenSectorsH], ax
420             ; 10/12/2022
421             or      cx, cx
422             ;cmp     cx, 0           ; cx set already before (=totalsectors)
423             ; 22/12/2022
424             ;jnz     short Relocate
425             ; 13/09/2023
426             jnz     short not_big
427             mov     ax, [7C20h]      ; BootSector.ext_boot_bpb.BPB_bigtotalsectors
428             mov     [cs:TotalSectorsL], ax
429             mov     ax, [7C22h]      ; BootSector.ext_boot_bpb.BPB_bigtotalsectors+2
430             mov     [cs:TotalSectorsH], ax
431             ; 13/09/2023
432 not_big:
433             ;cmp     word [cs:FATsectorsL], 0
434             and     dx, dx ; **
435             jnz     short Relocate ; FAT12 or FAT16 fs
436
437             mov     cx, [7C24h]      ; BPB_FATSz32 ; FAT32 fs
438             mov     [cs:FATsectorsL], cx
439             mov     cx, [7C26h]      ; BPB_FATSz32+2
440             mov     [cs:FATsectorsH], cx
441             mov     cx, [7C2Ch]      ; BPB_RootClus
442             mov     [cs:RootClusterL], cx
443             mov     cx, [7C2Eh]      ; BPB_RootClus+2
444             mov     [cs:RootClusterH], cx
445 %endif
446
447             ; 13/09/2023 - Erdogan Tan - Istanbul
448             ;
449             ; Note: Boot signature check has been removed because
450             ; it is not possible to start/run IBMBIO.COM
451             ; if it would not be a valid FAT32 (or compatible) boot sector
452             ; (input parameters and register contents would be wrong)
453 00000083 0E      push     cs
454 00000084 07      pop      es
455
456             ; 13/09/2023
457 00000085 BF[1700] mov     di, FirstCluster
458 00000088 A1A05   mov     ax, [051Ah]      ; LW of IBMBIO.COM (IO.SYS) first cluster
459 0000008B AB      stosw      ; Initialize to this cluster
460 0000008C A1A05   mov     ax, [0514h]      ; HW of IBMBIO.COM (IO.SYS) first cluster
461 0000008F AB      stosw
462
463 00000090 BE0B7C  mov     si, 7C0Bh      ; boot sector's bpb, BytesPerSector
464             ;mov     di, BytesPerSec
465 00000093 A5      movsw     ; BytesPerSec
466 00000094 A4      movsb     ; SecPerCluster
467 00000095 47      inc     di ; skip high byte of SecPerCluster word (it is 0)
468 00000096 A5      movsw     ; ReservSectors
469             ; 13/09/2023
470 00000097 A4      movsb     ; NumFats
471 00000098 47      inc     di ; skip high byte of NumFats word (it is 0)
472 00000099 A5      movsw     ; RootEntCnt
473 0000009A AD      lodsw     ; TotalSectorsL
474 0000009B 50      push     ax ; save TotalSectorsL
475 0000009C AC      lodsb     ; skip MediaByte
476             ; 13/09/2023
477 0000009D AD      lodsw     ; FATsectorsL (Retro DOS 5) - SecPerFat (Retro DOS 4)
478 0000009E 89C2     mov     dx, ax ; save BPB_FATSz16 into dx (it is 0 for FAT32 fs)
479 000000A0 A5      movsw     ; SecPerTrack
480 000000A1 A5      movsw     ; NumHeads
481 000000A2 A5      movsw     ; HiddenSectorsL
482 000000A3 A5      movsw     ; HiddenSectorsH
483 000000A4 58      pop      ax ; restore TotalSectorsL
484             ; si = 7C20h
485             ; di = offset TotalSectorsL
486 000000A5 09C0     or      ax, ax ; 16 bit total sectors value
487 000000A7 7403     jz      short big_total_sectors
488 000000A9 AB      stosw     ; TotalSectorsL
489             ; TotalSectorsH = 0
490 000000AA EB02     jmp     short chk_fatsz_16
491
492 big_total_sectors:
493             ; BigTotalSecs - 32 bit total sectors value
494 000000AC A5      movsw     ; BPB_TotSec32 (lw) -> TotalSectorsL
495 000000AD A5      movsw     ; BPB_TotSec32 (hw) -> TotalSectorsH
496 chk_fatsz_16:

```

```

497 ; 13/09/2023
498 ; si = 7C24h
499 000000AE 09D2 or dx, dx ; **
500 000000B0 7405 jz short fat32_bs ; FAT32 boot sector
501 ; FAT (FAT12 or FAT16) boot sector
502 000000B2 47 inc di ; skip TotalSectorsH
503 000000B3 47 inc di
504 000000B4 92 xchg ax, dx ; mov ax, dx
505 ;stosw
506 000000B5 EB06 jmp short fat_bs
507
508 fat32_bs:
509 ; FAT32 boot sector
510 000000B7 A5 movsw ; BPB_FATSz32 (lw) -> FATSectorsL
511 000000B8 A5 movsw ; BPB_FATSz32 (hw) -> FATSectorsH
512 000000B9 AD lodsw ; skip BPB_ExtFlags
513 000000BA AD lodsw ; skip BPB_FSVer
514 000000BB A5 movsw ; RootClusterL
515 000000BC AD lodsw ; RootClusterH
516
517 000000BD AB fat_bs: stosw ; 13/09/2023
518
519 ; 13/09/2023
520 000000BE 0E push cs
521 000000BF 1F pop ds
522
523 ; 13/09/2023
524 ; (PCDOS 7.1 - IBMBIO.COM - MSLOAD:0151h)
525
526 ; Relocate
527 ; -----
528 ;
529 ; NOTES:
530 ;
531 ; Relocates the loader code to top-of-memory.
532 ;
533 ; INPUT: none
534 ;
535 ; OUTPUT: code and data relocated.
536 ; AX,CX,SI,DI destroyed
537 ;
538 ; calls: none
539 ; -----
540 ;
541 ; Determine the number of paragraphs (16 byte blocks) of memory.
542 ; this involves invoking the memory size determination interrupt,
543 ; which returns the number of 1k blocks of memory, and then
544 ; converting this to the number of paragraphs.
545 ; Find out whether RPL code is present at top of memory and modify the
546 ; available amount of memory in AX
547 ; leave the number of paragraphs of memory in ax.
548 ;
549 ; -----
550 ; copy code from start to top of memory.
551 ;
552 ; the length to copy is EndOfLoader
553 ;
554 ; jump to relocated code
555 ; -----
556
557 ; 14/09/2023 - Retro DOS v5.0 BIOSLOADER/MSLOADER
558 ; PC DOS 7.1
559
560 Relocate: ; 24/12/2022 - Retro DOS v4 (4.0 & 4.1 & 4.2) MSLOADER
561 ; MSDOS 5.0 & 5.0+ & 6.22 (6.21)
562
563 ;cld
564
565 000000C0 31F6 xor si, si
566 000000C2 89F7 mov di, si
567 000000C4 CD12 int 12h ; MEMORY SIZE -
568 ; Return: AX = number of contiguous 1K blocks of memory
569 000000C6 B106 mov cl, 6
570 000000C8 D3E0 shl ax, cl ; Memory size in paragraphs
571
572 ;----- Check if an RPL program is present at TOM and do not tromp over it
573
574 ; 10/12/2022
575 ; ds = 0
576 ; 24/12/2022
577 ; ds = cs
578 ;xor bx, bx
579 ;mov ds, bx ; ZERO
580 ; 14/09/2023
581 000000CA 8EDE mov ds, si ; 0
582
583 ; 10/12/2022
584 000000CC 8B1EBC00 mov bx, [2Fh*4] ; (Int 2Fh)
585 000000D0 8E1EBE00 mov ds, [2Fh*4+2]
586
587 ;cmp word ptr [bx+3], 'PR'
588 ; 09/12/2022
589 000000D4 817F035250 cmp word [bx+3], 'RP' ; 'RPL'
590 000000D9 750F jnz short skip_RPL
591 000000DB 807F054C cmp byte [bx+5], 'L'
592 000000DF 7509 jnz short skip_RPL
593 000000E1 89C2 mov dx, ax ; get TOM into DX
594 000000E3 88064A mov ax, 4A06h ; (multMULT shl 8) + multMULTRPLTOM
595 000000E6 CD2F int 2Fh ; Get new TOM from any RPL
596 000000E8 89D0 mov ax, dx
597
598 skip_RPL: ; 24/12/2022
599 000000EA 0E push cs
600 000000EB 1F pop ds ; 25/12/2022
601
602 000000EC B104 mov cl, 4
603 000000EE 8B16[1B00] mov dx, [BytesPerSec] ; 24/12/2022
604 ;mov dx, [cs:BytesPerSec]
605 000000F2 D3EA shr dx, cl
606 000000F4 42 inc dx
607 000000F5 29D0 sub ax, dx
608 000000F7 A3[4500] mov [FatSegment], ax ; 24/12/2022
609 ;mov [cs:FatSegment], ax ; This will be used for fat sector
610 ; 14/09/2023
611 ;mov dx, 5F0h ; 1520 (for PC DOS 7.1 IBMBIO.COM)
612 000000FA BA[8004] mov dx, EndOfLoader ; loader size = 1520
613 000000FD D3EA shr dx, cl
614 000000FF 42 inc dx
615 00000100 29D0 sub ax, dx
616 00000102 8EC0 mov es, ax ; ES:DI -> place be relocated.
617 ; 14/09/2023
618 ; 22/12/2022
619 ;dec dx
620 ;shl dx, cl ; convert paragraphs to bytes (*)

```

```

621                                     ; (stack pointer will be set to this offset)
622                                     ; 24/12/2022
623                                     ; push cs
624                                     ; pop ds ; DS:SI -> source
625
626                                     ; 14/09/2023
627 00000104 B9[8004] mov cx, EndOfLoader ; 1520 (for PC DOS 7.1 IBMBIO.COM)
628 00000107 F3A4 rep movsb
629
630 00000109 06 push es ; Far jump to relocated MSLOAD code
631                                     ; (via retf, far return)
632 0000010A B8[0F01] mov ax, SetupStack
633 0000010D 50 push ax ; Message stack for destin of CS:IP
634 0000010E CB retf
635
636 ; -----
637 ; Start of relocated code
638 ; -----
639 ;
640 ; Move the stack to just under the boot record and relocation area (0:7c00h)
641 ;
642 ;
643 SetupStack:
644 ; 22/12/2022
645 ; mov ax, cs ; Start of relocated code
646 ; mov ss, ax
647 ; mov sp, NumHeads ; StackPtr offset
648 ; 20/12/2022
649 ; mov sp, StackPtr ; StackPtr offset
650
651 ; 22/12/2022
652 ; (set a temporary stack just above the relocated loader code)
653 ; ((instead of using/reserving 256 bytes of stack space in 'IO.SYS' file))
654
655 ; 22/12/2022
656 ; cs = loader segment (relocated)
657 ; dx = loader size + stack space (*) -paragraph aligned-
658
659 ; 14/09/2023
660 ; cli
661 ; mov ax, cs
662 ; mov ds, ax ; 24/12/2022
663 ; cli
664 ; mov ss, ax
665 ; mov sp, dx ; (*)
666 ; sti
667
668 ; 14/09/2023
669 ; (PCDOS 7.1 IBMBIO.COM - MSLOAD:01A5h)
670 mov ax, cs
671 0000010F 8CC8 ; 27/12/2023
672 mov ds, ax
673 00000111 8ED8 ;
674 sub ax, 40h ; move ss to 400h backward for stack space
675 00000113 83E840 ; then set sp to the end of this stack space
676 mov ss, ax
677 00000116 8ED0 mov sp, 400h
678 00000118 BC0004 add ax, 40h ; ax = cs
679 mov ds, ax
680
681 ; FindClusterSize
682 ; -----
683 ;
684 ; INPUT: bpb information in loaded boot record at 0:7c00h
685 ;
686 ; OUTPUT:
687 ;
688 ; ds = 0
689 ; ax = bytes/cluster
690 ; bx = sectors/cluster
691 ; si destroyed
692 ; calls: none
693 ; -----
694 ;
695 ; get bytes/sector from bpb
696 ;
697 ; get sectors/cluster from bpb
698 ;
699 ; bytes/cluster = bytes/sector * sector/cluster
700 ; -----
701 ;
702 FindClusterSize:
703 ; for the time being just ASSUME the boot record is valid and the bpb is there.
704
705 ; 14/09/2023
706 ; 24/12/2022
707 ; ds = cs
708 mov ax, [BytesPerSec]
709 0000011B A1[1B00] ; xor bx, bx
710 ; mov bl, [SecPerCluster] ; get sectors/cluster
711 ; mul bx
712 0000011E F726[1D00] mul word [SecPerCluster]
713 ; 14/09/2023
714 or dx, dx
715 00000122 09D2 jz short CalcFatSize
716 00000124 7403 jmp ErrorOut
717 00000126 E98E01
718
719 ; CalcFatSize
720 ; -----
721 ;
722 ; NOTES:
723 ;
724 ; Determine if fat is 12 or 16 bit fat. 12 bit fat if floppy, read mbr
725 ; to find out what system id byte is.
726 ;
727 ; INPUT:
728 ;
729 ; OUTPUT:
730 ;
731 ; CS:FatSize = FAT_12_BIT or FAT_16_BIT
732 ; all other registers destroyed
733 ; -----
734 ;
735 CalcFatSize:
736 ; 14/09/2023 (Retro DOS v5, PC DOS 7.1 IBMBIO.COM LOADER)
737 ; 24/12/2022 (Retro DOS v4, MSDOS 5.0-6.22 IO.SYS LOADER)
738 mov [ClusterSize], ax ; cluster size in bytes
739
740 ; 24/12/2022
741 00000129 A3[0500] ; ds = cs
742
743
744

```



```

745             ;mov     byte [Fatsize], 1; FAT_12_BIT (assume)
746             ; 14/09/2023
747 0000012C C606[3E00]01     mov     byte [FatType],1 ; FAT12
748
749             ;mov     dx, [TotalSectorsH]
750             ;mov     ax, [TotalSectorsL] ; DX:AX = total disk sectors
751             ; 14/09/2023
752 00000131 A1[2F00]         mov     ax, [TotalSectorsH]
753 00000134 8B1E[2D00]       mov     bx, [TotalSectorsL] ; AX:BX = total disk sectors
754             ; 14/09/2023
755             %if 0
756             sub      ax, [ReservSectors]
757
758             sbb      dx, 0          ; DX:AX= Total available sectors
759
760             push     ax
761             push     dx
762
763             mov     bx, [FATSectorsL]
764
765             ;mov     cx, [FATSectorsH]
766             ;push     ax
767             ;push     dx
768             ;mov     al, [NumFats]
769             ;xor     ah, ah
770             ;mov     ax, [NumFats]
771             ;xchg    ax, cx
772             ;mul     cx
773
774             mov     ax, [FATSectorsH]
775             mov     cx, [NumFats] ; calculate total FAT sectors
776             mul     cx
777             xchg    ax, cx
778             mul     bx
779             add     cx, dx
780             mov     bx, ax
781
782             pop     dx
783             pop     ax
784
785             sub     ax, bx
786             sbb     dx, cx          ; DX:AX = Total sectors - FAT sectors
787
788             mov     bx, [RootEntCnt] ; Root directory entry count
789             mov     cl, 4
790             shr     bx, cl          ; BX = Total directory sectors
791             sub     ax, bx
792             sbb     dx, 0          ; DX:AX= Sectors in data area
793             %endif
794             ; 14/09/2023
795 00000138 8B16[3900]       mov     dx, [FirstSectorL]
796 0000013C 8B0E[3B00]       mov     cx, [FirstSectorH]
797             ; 04/10/2023
798 00000140 8916[0700]       mov     [StartSecL], dx
799 00000144 890E[0900]       mov     [StartSecH], cx
800             ;
801             ; ! here, cx:dx includes hidden sectors (partition start address) !
802 00000148 2B16[2900]       sub     dx, [HiddenSectorsL]
803 0000014C 1B0E[2B00]       sbb     cx, [HiddenSectorsH] ; cx:dx = start of data from boot sector
804
805             ; 14/09/2023
806 00000150 29D3             sub     bx, dx ; total secs - start of data
807 00000152 19C8             sbb     ax, cx
808                                     ; AX:BX= Sectors in data area
809
810             ; 24/12/2022
811 00000154 8B0E[1D00]       mov     cx, [SecPerCluster] ; *#*
812             ; 14/09/2023
813             ; bx = lw of data sector count
814             ; ax = hw of data sector count
815 00000158 31D2             xor     dx, dx
816 0000015A F7F1             div     cx ; *#*
817             ; 24/12/2022
818             ;mov     [cs:TempH], ax ; AX = Total number of clusters (hw)
819             ;mov     [TempH], ax
820             ; 14/09/2023
821 0000015C 93             xchg    ax, bx ; ax = lw of data sector count ; 06/10/2023
822             ; bx = hw of cluster count
823             div     cx ; *#*
824             ; 14/09/2023
825 0000015F C606[3E00]0B     mov     dx, [FirstCluster+2]
826             mov     byte [FatType], 0Bh ; set FAT type to FAT32 (CHS type disk R/W)
827             ;cmp     word [TempH], 0
828             ;jne     short ReadInFirstCluster
829 00000164 09DB             or     bx, bx ; is cluster count > 65535 ?
830             ;jnz     short ReadInFirstCluster ; yes, it is (it must be) FAT32 fs
831             ; 06/10/2023
832 00000168 83F8F6         cmp     ax, 0FFF6h ; FAT16 limit (65536-10)
833             ;jnb     short ReadInFirstCluster ; FAT32
834             ;
835 0000016D 891E[1900]       mov     [FirstCluster+2], bx ; 0 ; (clear HW of FirstCluster)
836             ;xor     dx, dx
837 00000171 C606[3E00]01     mov     byte [FatType], 1 ; set FAT type to FAT12
838             ; 06/10/2023
839 00000176 3DF60F         cmp     ax, 0FF6h
840             ;cmp     ax, 4086 ; 4096-10
841 0000017B 7205             jb     short ReadInFirstCluster ; 12 bit FAT
842             mov     byte [FatType], 4 ; set FAT type to FAT16
843
844             ; ReadInFirstCluster
845             ; -----
846             ; NOTES: read the start of the clusters that covers at least IbmLoadSize
847             ; fully. for example, if sector/cluster = 2, and IbmLoadSize=3
848             ; then we are going to re-read the second cluster to fully cover
849             ; msload program in the cluster boundary.
850
851             ; INPUT:
852             ; IbmLoadSize - make sure this value is the same as the one in
853             ; msboot program when you build the new version!!!!
854
855             ; SecPerCluster
856             ; ClusterSize
857             ; FirstSectorL
858             ; FirstSectorH
859
860             ; OUTPUT: msload program is fully covered in a cluster boundary.
861             ; ax = # of clusters we read in so far.
862
863             ; calls: ReadSectors
864             ; logic:
865             ; ax; dx = IbmLoadSize / # of sector in a cluster.
866             ; if dx = 0 then ok. (msload is in a cluster boundary.)
867             ; else (has to read (ax+1)th cluster to cover msload)
868             ; read (ax+1)th cluster into the address after the clusters we

```

```

869 ; read in so far.
870 ; -----
871
872 ; 09/12/2022
873 ; BiosStart equ 51Ah ; AX = IO.SYS starting cluster
874 ; IbmLoadSize equ 3 ; AX = Number sectors in MSLOAD
875 ; BiosOffset equ 700h ; Address where loader was read in
876
877 ReadInFirstCluster:
878 ; 14/09/2023
879 00000180 8B16[1900] mov dx, [FirstCluster+2]
880 00000184 A1[1700] mov ax, [FirstCluster]
881 ; IBMBIO.COM First Cluster
882 ; Root dir buffer at 500h (segment=0)
883 ; IBMBIO.COM first cluster ptr at 51Ah
884 ; high word of cluster is at 514h
885 ; 14/10/2023 (!*)
886 ;; a cluster number start from 2
887 ;; convert it to (correct) cluster index number
888 ;sub ax, 2
889 ;sbb dx, 0
890 ; DX:AX = zero based cluster number (cluster index)
891
892 00000187 8916[1500] mov [CurrentClusterH], dx
893 0000018B A3[1300] mov [CurrentClusterL], ax ; Initialize to this cluster
894
895 ; 24/12/2022
896 ; (PCDOS 7.1 IBMBIO.COM - MSLOAD:0255h) ; 04/10/2023 ('mov ax,3')
897 ;mov ax, IbmLoadSize
898 ;mov ax, 3 ; Load the 3rd and other IO.SYS sectors
899 ; 04/10/2023 ; **
900 ; (Windows ME IO.SYS - MSLOAD:01E4h)
901 0000018E B80400 mov ax, 4 ; ** ; Load the 4rd and other IO.SYS sectors
902
903 ; 14/09/2023
904 ;div byte [SecPerCluster]
905 00000191 F6F1 div cl ; **
906 ; AL = total cluster read in
907 ; AH = remaining sectors in last cluster
908
909 ; (Note: PCDOS 7.1 bs loads 1st 4 sectors of IBMBIO.COM)
910 ; If cluster size > 3, al = 0, ah <> 0
911 ; If cluster size = 2, al = 1, ah = 1
912 ; If cluster size = 1, al = 3, ah = 0
913 ; If ah = 0, nothing remaining in last cluster
914
915 ; 14/10/2023
916 00000193 BE7000 mov si, 70h ; ++*
917 00000196 8EC6 mov es, si ; ++ ; ES = BIOSDATA (IO.SYS DATA) segment
918
919 ; 14/09/2023
920 ;cmp ah, 0
921 ; 10/12/2022
922 00000198 20E4 and ah, ah
923 ;cmp ah, 0
924 0000019A 742F jz short SetNextClusterNum ; next cluster
925 ; 04/10/2023 ; **
926 ; If AH=0
927 ; and if CL=1, AL=4
928 ; and if CL=2, AL=2 (?)
929 ; and if CL=4, AL=1
930 ; If AH>0
931 ; AL=0 and AH=4
932
933 ; 04/10/2023 ; **
934 ; al = 0
935 ;xor ah, ah ; 0
936 ;push ax ; (*) ; AX = total clusters in the loader
937 ; already read in
938
939 ; 14/09/2023
940 ; 24/12/2022
941 %if 0
942 mov cx, [FirstSectorL] ; Put starting sector of disk data
943 mov [StartSecL], cx ; area in StartSecH:StartSecL
944 mov cx, [FirstSectorH]
945 mov [StartSecH], cx
946 mul byte [cs:SecPerCluster]
947 add [StartSecL], ax ; Add number of sectors already loaded
948 adc word [StartSecH], 0 ; to start sector
949 ;mov dx, [FirstCluster+2]
950 ;mov ax, [FirstCluster]
951 ;sub ax, 2
952 ;sbb dx, 0
953 mov dx, [CurrentClusterH] ; IBMBIO.COM 1st cluster (index)
954 mov ax, [CurrentClusterL] ; (zero based cluster number)
955
956 xor bx, bx
957 mov bl, [SecPerCluster]
958 mul bx ; DX:AX = logical start sector
959 add [StartSecL], ax
960 adc [StartSecH], dx
961 ; abs start sector for next read of
962 ; the rest of the last loader cluster
963 pop ax ; (*) number of clusters already loaded
964 ; (0 or 1)
965 ; (Note: if al=0, the 1st 4 sectors of the 1st cluster
966 ; will be loaded again! -PCDOS 7.1-)
967
968 push ax
969 mul word [ClusterSize]
970 ;mov di, BiosOffset
971 mov di, 700h ; IBMBIO.COM (IO.SYS) loading addr (segment = 0)
972 add di, ax
973 xor ax, ax
974 mov es, ax ; ES = segment 0
975 mov al, [SecPerCluster]
976 ; Read in the entire last cluster
977 mov [SectorCount], ax
978 call ReadSectors
979 pop ax ; AX = total clust read by boot loader
980 inc ax ; AX = total clust read in now
981 SetNextClusterNum:
982 ; ...
983 inc ax ; AX = total clusters read in based 2
984 add [CurrentClusterL], ax
985 adc [CurrentClusterH], 0
986 dec ax ; CurrentCluster = Last cluster read
987 ; AX = number of clusters loaded
988 %endif
989 ; 04/10/2023 ; **
990 ; ah=4 & al=0
991 0000019C 88C8 mov al, cl ; ** (SecPerCluster)
992 0000019E 28E0 sub al, ah ; ** (remain sectors to read in the cluster)
993 000001A0 A2[1100] mov [SectorCount], al ; ** (spc-4)
994
995 ; 24/12/2022

```



```

993 ; ds = cs
994 ;mov cx, [FirstSectorL] ; Put starting sector of disk data
995 ;mov [StartSecL], cx ; area in StartSecH:StartSecL
996 ;mov cx, [cs:FirstSectorH]
997 ;mov [StartSecH], cx
998 ; [StartSecL] = [FirstSectorL] ; **!
999 ; [StartSecH] = [FirstSectorH] ; **!
1000
1001 ; 24/12/2022
1002 ; cx = [SecPerCluster] ; **
1003
1004 ; 04/10/2023
1005 ; ax = 0 (cluster size > 3) or ax = 1 (cluster size = 2)
1006 ; al = 0 (cluster size > 4) and ah > 0 ; (as win ME IO.SYS)
1007 ; cx = sectors per cluster (ch = 0)
1008
1009 ; 04/10/2023 ; **
1010 ; cx = sectors per cluster (ch = 0)
1011
1012 ; or al, al ; *
1013 ; jz short rfc_1 ; al = 0 ; *
1014 ; al = 1
1015
1016 ; mul byte [SecPerCluster]
1017 ; mul cl ; **
1018 ; add [StartSecL], ax ; Add number of sectors already loaded
1019 ; adc word [StartSecH], 0 ; to start sector
1020 ; 04/10/2023
1021 ; add [StartSecL], cx ; * (AL=1, CL*AL=CL, CH=0)
1022 ; adc word [StartSecH], 0 ; *
1023
1024 ; rfc_1: ; * ; 04/10/2023
1025 ; mov ax, [51Ah] ; AX = [51Ah] = IO.SYS 1st clust
1026 ; dec ax
1027 ; dec ax
1028 ; 14/10/2023
1029 ; mov ax, [CurrentClusterL] ; ***
1030 ; ax = word [51Ah] - 2
1031 ; 04/10/2023
1032 ; mov dx, [CurrentClusterH]
1033 000001A3 A1[1500] mov ax, [CurrentClusterH] ; ****
1034
1035 ; xor bx, bx
1036 ; mov bl, [SecPerCluster]
1037 ; mov bx, [SecPerCluster]
1038 ; mul bx ; DX:AX = logical start sector
1039 ; 04/10/2023
1040 ; mul cx ; [SecPerCluster] ; **
1041 ; 32 bit multiplication (HLL*SPC)
1042 ; 14/10/2023
1043 ; push ax ; Current Cluster LW
1044 ; mov ax, dx ; Current Cluster HW (HH) ; ****
1045
1046 000001A6 F7E1 mul cx ; (HH*SPC) ; (result: dx is -must be- zero)
1047 000001A8 91 xchg ax, cx
1048 ; 14/10/2023
1049 ; pop dx ; Current Cluster LW (LL)
1050 ; mul dx ; LL*SPC
1051 000001A9 F726[1300] mul word [CurrentClusterL] ; ***
1052 000001AD 01CA add dx, cx ; (add lw of HH*SPC)
1053
1054 ; 04/10/2023 ; **
1055 000001AF 83C004 add ax, 4 ; ** ; IbmLoadSize (win ME BS's IO.SYS read count)
1056 000001B2 83D200 adc dx, 0 ; **
1057
1058 000001B5 0106[0700] add [StartSecL], ax
1059 000001B9 1116[0900] adc [StartSecH], dx
1060 ; abs start sector for next read of
1061 ; the rest of the last loader cluster
1062 ; 04/10/2023 ; **
1063 ; (number of clusters already -complete- loaded = 0)
1064 ; pop ax ; (*) number of clusters already loaded
1065 ; push ax
1066
1067 ; 04/10/2023 ; **
1068 ; mul word [Clustersize]
1069 ; 14/10/2023
1070 ; mov ax, [BytesPerSec]
1071 ; shl ax, 2 ; * 4 (4 sectors already loaded)
1072
1073 ; mov di, BiosOffset
1074 ; mov di, 700h ; IO.SYS offset (segment = 0)
1075 ; add di, ax
1076 ; 14/10/2023
1077 000001BD 8B3E[1B00] mov di, [BytesPerSec]
1078 000001C1 C1E702 shl di, 2 ; * 4 (4 sectors already loaded)
1079 ; add di, 700h ; ++
1080 ; di = buffer offset
1081
1082 ; 04/10/2023 ; **
1083 ; xor ax, ax
1084 ; mov es, ax ; ES = segment 0
1085 ; dx = 0 ; **
1086 ; mov es, dx ; 0
1087 ; 14/10/2023
1088 ; cx = 0
1089 ; mov es, cx ; 0 ; ++
1090 ; es = buffer segment = 0
1091
1092 ; 14/10/2023
1093 ; es = si = 70h ; ++
1094 ; mov si, 70h ; ++
1095 ; mov es, si ; ++
1096 ; es:di = 70h:800h
1097
1098 ; 24/12/2022
1099 ; mov al, [SecPerCluster]
1100 ; ; Read in the entire last cluster
1101 ; mov [SectorCount], ax
1102 ; 14/10/2023 ; **
1103 ; mov [SectorCount], cx ; [SecPerCluster] ; **
1104
1105 000001C4 E87F00 call ReadSectors
1106
1107 ; 04/10/2023 ; *
1108 ; pop ax ; AX = total clust read by boot loader
1109 ; inc ax ; AX = total clust read in now
1110 ; 04/10/2023
1111 ; mov ax, 1 ; 1 cluster loaded
1112 ; SetNextClusterNum:
1113 ; 14/10/2023
1114 000001C7 89F8 mov ax, di
1115 ; ax = loaded (IBMBIO.COM or IO.SYS) byte count
1116 000001C9 EB0F jmp short SaveLoadedBios2

```

```

1117 SetNextClusterNum:
1118 ; 14/10/2023 (!*)
1119 ;inc ax
1120 000001CB 48 dec ax ; (4 clusters -> +3, 1 cluster -> +0)
1121 000001CC 0106[1300] add [CurrentClusterL], ax ; ah = 0
1122 000001D0 8316[1500]00 adc word [CurrentClusterH], 0
1123 ; CurrentCluster = Last cluster (loaded)
1124 000001D5 40 inc ax
1125 ;dec ax
1126 ; AX = number of clusters loaded
1127
1128 ; SaveLoadedBios
1129 -----
1130 ;
1131 ; NOTES:
1132 ;
1133 ; Determine how much of iosys was loaded in when the loader was loaded
1134 ; by the boot record (only the portion that is guaranteed to be contiguous)
1135 ;
1136 ; INPUT:
1137 ; AX:Total cluster already read in (loader & bios)
1138 ; CS:CurrentCluster = number of clusters used for loader+2
1139 ;
1140 ; OUTPUT:
1141 ; ES = 70h
1142 ; DI = next offset to load iosys code
1143 ; AX,BX,CX,DX,SI destroyed
1144 ;
1145 ; CS:NextBioLocation = di on output
1146 ; CS:last_cluster = last cluster loaded
1147 ;
1148 ; calls: none
1149 -----
1150 ;
1151 ; Multiply cluster * cluster size in bytes to get total loaded for msload
1152 ;
1153 ; Subtract total_loaded - (EndOfLoader) to get loaded io.sys in last cluster
1154 ;
1155 ; Relocate this piece of iosys down to 70:0
1156 ;
1157 -----
1158 ;
1159 SaveLoadedBios:
1160 ; 14/10/2023
1161 ;push ds
1162 ;
1163 ; 24/12/2022
1164 ; ds = cs
1165 ; ax = number of loaded clusters
1166 mul word [ClusterSize]
1167 000001D6 F726[0500] mul word [cs:ClusterSize]
1168 ;
1169 ; Get total bytes loaded by
1170 ; this is always < 64k, so
1171 ; lower 16 bits ok
1172 ; 14/10/2023
1173 ; ax = [clusterSize] * (loaded cluster count)
1174 SaveLoadedBios2:
1175 ; 14/10/2023
1176 000001DA 1E push ds
1177 ;
1178 ; 14/10/2023
1179 ;sub ax, EndOfLoader ; (OFFSET EndOfLoader)-(OFFSET Start)
1180 000001DB BE[8004] mov si, EndOfLoader
1181 000001DE 29F0 sub ax, si
1182 000001E0 89C1 mov cx, ax
1183 ;
1184 ; 14/10/2023
1185 ;mov ax, 70h ; Segment at 70h
1186 ;mov ds, ax
1187 ;mov es, ax
1188 ; es = 70h
1189 000001E2 06 push es
1190 000001E3 1F pop ds
1191 ;
1192 ;mov si, EndOfLoader
1193 000001E4 31FF xor di, di
1194 000001E6 F3A4 rep movsb ; Relocate this code to 0070h:0000h
1195 ;
1196 ;mov [NextBioLocation], di
1197 ;mov [cs:NextBioLocation], di
1198 ; 05/10/2023
1199 ;mov [NextBioLocation], di
1200 000001E8 89FD mov bp, di
1201 ;
1202 ; es:di = (the next) buffer address for next read
1203 ;
1204 000001EA 1F pop ds ; Save where location for next read
1205 ;
1206 ; GetContigClusters
1207 -----
1208 ;
1209 ; NOTES: go find clusters as long as they are contiguous
1210 ;
1211 ;
1212 ; INPUT:
1213 ; CS:NextBioLocation
1214 ; CS:
1215 ;
1216 ; OUTPUT:
1217 ;
1218 ; calls: GetNextFatEntry
1219 -----
1220 ;
1221 ; Set CS:SectorCount to sectors per cluster
1222 ;
1223 ; Call GetNextFatEntry to get next cluster in file
1224 ;
1225 ; Call check_for_eof
1226 ;
1227 ; if (nc returned)
1228 ;
1229 ; {call GetNextFatEntry
1230 ;
1231 ; if (new cluster is contig to old cluster)
1232 ; {add sectors per cluster to CS:SectorCount
1233 ;
1234 ; call check_for_eof
1235 ;
1236 ; if (nc returned)
1237 ;
1238 ; -----
1239 ;
1240 ; 09/12/2022

```

```

1241 ; END_OF_FILE equ 0FFh
1242 ; DosLoadSeg equ 70h
1243
1244 GetContigClusters:
1245 ; 24/12/2022
1246 %if 0
1247
1248     xor     ah, ah
1249     mov     al, [cs:SecPerCluster]; Assume we will get one cluster
1250     mov     [cs:SectorCount], ax ; Sector count = sectors in 1 cluster
1251     push    word [cs:SectorCount]
1252     call    GetNextFatEntry ; Returns next cluster to read in AX
1253     pop     word [cs:SectorCount]
1254     mov     word [cs:CurrentCluster], ax ; Update the last one found
1255     cmp     byte [cs:EndOfFile], 0FFh ; END_OF_FILE
1256     jz      short GoToBioInit
1257     xor     dx, dx
1258     ;sub     ax, 2 ; Zero base the cluster
1259     ; 10/12/2022
1260     dec     ax
1261     dec     ax
1262     xor     ch, ch
1263     mov     cl, [cs:SecPerCluster]
1264     mul     cx ; How many sectors (before next cluster)
1265     add     ax, [cs:FirstSectorL] ; See where the data sector starts
1266     adc     dx, [cs:FirstSectorH]
1267     mov     [cs:StartSecL], ax ; Save it (used by ReadSectors)
1268     mov     [cs:StartSecH], dx
1269     mov     di, [cs:NextBioLocation] ; Get where to put code
1270     push    word [cs:SectorCount] ; Save how many sectors
1271     ;mov     ax, DosLoadSeg
1272     mov     ax, 70h
1273     mov     es, ax
1274     call    ReadSectors
1275     pop     ax ; Get back total sectors read in
1276     mul     word [cs:BytesPerSec] ; Get number of bytes we loaded
1277     add     [cs:NextBioLocation], ax ; Point to where to load next
1278     jmp     short GetContigClusters
1279
1280 %endif
1281 ; 24/12/2022
1282 ; ds = cs
1283
1284     mov     ax, [SecPerCluster] ; Assume we will get one cluster
1285     mov     [SectorCount], ax ; Sector count = sectors in 1 cluster
1286     ;push    word [SectorCount]
1287     push    ax
1288
1289     call    GetNextFatEntry ; Returns next cluster to read in AX
1290
1291     ;pop     word [SectorCount]
1292     ; 05/10/2023
1293     pop     cx ; sc = spc
1294
1295     mov     [CurrentClusterL], ax ; Update the last one found, lw
1296     mov     [CurrentClusterH], di ; hw
1297
1298     cmp     byte [EndOfFile], 0FFh ; END_OF_FILE
1299     je      short GoToBioInit ; 23/12/2022
1300
1301     ; 22/12/2022
1302     ;xor     dx, dx ; * (not required)
1303     ; 10/12/2022
1304     ;sub     ax, 2 ; Zero base the cluster
1305     ;dec     ax
1306     ;dec     ax
1307     ; 14/10/2023
1308     ;xor     dx, dx
1309     sub     ax, 2 ; Zero base the cluster (32 bit as di:ax)
1310     sbb     di, 0
1311
1312     ;; 24/12/2022
1313     ;; ax = cluster index
1314     ;;mov     cx, [SecPerCluster]
1315     ;;mul     cx, * ; How many sectors (before next cluster)
1316     ; 04/10/2023
1317     ;;mul     word [SecPerCluster]
1318     ;mul     cx
1319     ; 04/10/2023
1320     xchg     di, ax ; 32 bit multiplication
1321     mul     cx ; (dx:ax)*cx
1322     xchg     ax, di
1323     mul     cx
1324     add     dx, di
1325
1326     add     ax, [FirstSectorL] ; See where the data sector starts
1327     adc     dx, [FirstSectorH]
1328     mov     [StartSecL], ax ; Save it (used by ReadSectors)
1329     mov     [StartSecH], dx
1330
1331     ; 05/10/2023
1332     ;mov     di, [NextBioLocation]
1333     mov     di, bp
1334     mov     [SectorCount], cx
1335     ; es = 70h
1336
1337     ; es:di = (current) buffer address for (current) read
1338
1339     call    ReadSectors
1340     ; ES:DI = (the next) buffer address for next read
1341     ; 05/10/2023
1342     ;mov     [NextBioLocation], di
1343     mov     bp, di
1344
1345     jmp     short GetContigClusters
1346
1347 ; -----
1348 ; GoToBiosInit
1349 ; -----
1350
1351 ;
1352 ; NOTES:
1353 ;
1354 ; Set up required registers for iosys, then jump to it (70:0)
1355 ;
1356 ; INPUT: none
1357 ;
1358 ; CS:MediaByte = media byte
1359 ; CS:BootDrive = int 13 drive number we booted from
1360 ; CS:FirstSectorL = first data sector on disk (low) (0-based)
1361 ; CS:FirstSectorH = first data sector on disk (high)
1362 ;
1363 ; OUTPUT:
1364 ;

```

```

1365 ; required by msinit
1366 ; DL = int 13 drive number we booted from
1367 ; CH = media byte
1368 ; BX = first data sector on disk (0-based)
1369 ; AX = first data sector on disk (high)
1370 ; DI = sectors/fat for the boot media.
1371 ;
1372 ; calls: none
1373 ; -----
1374 ;
1375 ; set up registers for msinit then do far jmp
1376 ; -----
1377 ;
1378
1379 GoToBioInit:
1380
1381 ; 05/10/2023 - Retro DOS v5.0 IBMBIO.COM (PCDOS 7.1)
1382 %if 0
1383 ; 24/12/2022
1384 ; ds = cs
1385 ;mov ch, [cs:MediaByte]
1386 ;mov dl, [cs:BootDrive]
1387 ;mov bx, [cs:FirstSectorL]
1388 ;mov ax, [cs:FirstSectorH]
1389
1390 mov ch, [MediaByte] ; Restore regs required for msint
1391 mov dl, [BootDrive] ; Physical drv number we booted from.
1392 mov bx, [FirstSectorL] ; AX:BX = first data sector of disk
1393 mov ax, [FirstSectorH]
1394 %endif
1395
1396 ; 05/10/2023 - Retro DOS v5.0 IBMBIO.COM (PCDOS 7.1)
1397 %if 1
1398 ; 05/10/2023
1399 ; (PCDOS 7.1 IBMBIO.COM - MSLOAD:034Ah)
1400 0000022E 8A2E[3F00] mov ch, [MediaByte] ; Set up required registers for iosys,
1401 ; then jump to it (70:0)
1402 ;
1403 ; Restore regs required for msint
1404 00000232 8A16[3D00] mov dl, [BootDrive] ; Physical drv number we booted from
1405 00000236 8B1E[3900] mov bx, [FirstSectorL]
1406 0000023A A1[3B00] mov ax, [FirstSectorH]
1407 ; bx:ax = first data sector of disk
1408 0000023D C536[4100] lds si, [OrgDasdPtr]
1409 ; Set ds:si to Original INT 1Eh disk(ette)
1410 ; table address and then push disk table
1411 ; address and INT 1Eh vector to stack
1412 ; (set stack content just as at the start
1413 ; of MSLOAD)
1414 ; 05/10/2023
1415 ; following pushes are not necessary..
1416 ; PCDOS 7.1 BIOSDATA init procedure does not pop the pushed
1417 ; registers here and also it doesn't use the di value here
1418 ; (but ds:si is used)
1419 ;
1420 ;push ds ; INT 1Eh original table segment
1421 ;push si ; INT 1Eh original table offset
1422 ;xor di, di ; 0
1423 ;push di ; INT 1Eh vector segment
1424 ;mov di, 78h ; 1Eh*4 = 78h
1425 ;push di ; INT 1Eh vector offset
1426 %endif
1427 ; 05/10/2023
1428 ;mov bp, sp ; not necessary
1429 ; (BIOSDATA init doesn't use the bp value here)
1430
1431 00000241 EA00007000 jmp 70h:0 ; Far jump to IoSysAddr (DOSBIOS)
1432
1433 ; ===== S U B R O U T I N E =====
1434
1435 ; 05/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PCDOS 7.1)
1436 ; (PCDOS 7.1 IBMBIO.COM - MSLOAD:036Bh)
1437
1438 %if 0
1439 check_int13h_extensions:
1440 push ax
1441 push dx
1442 xor ax, ax
1443 push ax ; zero (buffer offset 24)
1444 ; (bytes per sector)
1445 mov bx, sp
1446 sub sp, 20
1447 push ax ; info flags
1448 mov ax, 26 ; Result buffer size
1449 push ax
1450 mov si, sp
1451 mov dl, [BootDrive]
1452 mov ah, 48h
1453 push ds
1454 push ss
1455 pop ds
1456 cmp dl, 0
1457 jge short not_hard_disk
1458 int 13h ; DISK - IBM/MS Extension - GET DRIVE PARAMETERS
1459 ; (DL - drive, DS:SI - buffer)
1460 not_hard_disk:
1461 pop ds
1462 mov sp, bx
1463 pop ax ; bytes per sector, buffer offset 24
1464 jc short int13h_ext_err
1465 cmp ax, 512
1466 je short int13h_ext_ok
1467 stc
1468 int13h_ext_err:
1469 int13h_ext_ok:
1470 pop dx
1471 pop ax
1472 retn
1473 %endif
1474
1475 ; ===== S U B R O U T I N E =====
1476
1477 ; ReadSectors
1478 ; -----
1479 ; notES:
1480 ;
1481 ; read in the CS:SectorCount number of sectors at ES:di
1482 ;
1483 ;
1484 ; INPUT:
1485 ;
1486 ; DI = OFFSET of start of read
1487 ; ES = segment of read
1488 ; CS:SectorCount = number of sectors to read

```

```

1489 ; CS:StartSecL = starting sector (low)
1490 ; CS:StartSecH = starting sector (high)
1491 ; following is bpb info that must be setup prior to call
1492 ; CS:NumHeads
1493 ; CS:number_of_sectors
1494 ; CS:BootDrive
1495 ; CS:SecPerTrack
1496
1497 ; OUTPUT:
1498 ;
1499 ; AX,BX,CX,DX,SI,DI destroyed
1500 -----
1501 ; divide start sector by sectors per track
1502 ; the remainder is the actual sector number, 0 based
1503
1504 ; increment actual sector number to get 1 based
1505
1506 ; the quotient is the number of tracks - divide by heads to get the cyl
1507
1508 ; the remainder is actual head, the quotient is cylinder
1509
1510 ; figure the number of sectors in that track, set al to this
1511
1512 ; do the read
1513
1514 ; if error, do reset, then redo the int 13h
1515
1516 ; if successful read, subtract # sectors read from SectorCount, add to logical
1517 ; sector, add #sectors read * BytesPerSec to bx;
1518
1519 ; if SectorCount <> 0 do next read
1520 -----
1521
1522 ; 05/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PC DOS 7.1)
1523
1524 ; 24/12/2022
1525 ; 22/12/2022
1526
1527 00000246 B90500 ReadSectors: mov cx, 5 ; 5 retries
1528
1529 ; Convert a logical sector into track/sector/head. AX has the
1530 ; logical sector number
1531 TryRead:
1532 ; 24/12/2022
1533 ; ds = cs
1534 00000249 51 push cx ; (*)
1535 ;mov ax, [cs:StartSecL] ; Get starting sector
1536 ;mov dx, [cs:StartSecH]
1537 0000024A A1[0700] mov ax, [StartSecL] ; Get starting sector
1538 0000024D 8B16[0900] mov dx, [StartSecH]
1539 00000251 50 push ax ; (**)
1540 ;;;
1541
1542 ; 05/10/2023 - Retro DOS v5.0 (PC DOS 7.1 IBMBIO.COM)
1543 ;call check_int13h_extensions
1544 ;jc short chs_read
1545 ;-----
1546 ; 06/10/2023
1547 check_int13h_extensions:
1548 00000252 8A1E[3D00] mov bl, [BootDrive]
1549 00000256 08DB or bl, bl
1550 00000258 7953 jns short chs_read ; not hard disk
1551 ; bl >= 80h
1552 0000025A 52 push dx ; ***
1553 ;mov dl, [BootDrive]
1554 0000025B 88DA mov dl, bl
1555 0000025D 50 push ax ; ****
1556 0000025E 31C0 xor ax, ax
1557 00000260 50 push ax ; zero (buffer offset 24)
1558 ; (bytes per sector)
1559 00000261 89E3 mov bx, sp
1560 00000263 83EC14 sub sp, 20
1561 00000266 50 push ax ; info flags
1562 00000267 B81A00 mov ax, 26 ; Result buffer size
1563 0000026A 50 push ax
1564 0000026B 89E6 mov si, sp
1565 0000026D B448 mov ah, 48h
1566 0000026F 1E push ds
1567 00000270 16 push ss
1568 00000271 1F pop ds
1569 00000272 CD13 int 13h ; DISK - IBM/MS Extension - GET DRIVE PARAMETERS
1570 ; (DL - drive, DS:SI - buffer)
1571 00000274 1F pop ds
1572 00000275 89DC mov sp, bx
1573 00000277 58 pop ax ; bytes per sector, buffer offset 24
1574 00000278 7203 jc short int13h_ext_err
1575 0000027A 3D0002 cmp ax, 512
1576 int13h_ext_err:
1577 0000027D 58 pop ax ; ****
1578 0000027E 5A pop dx ; ***
1579 0000027F 752C jne short chs_read
1580 ;-----
1581 lba_read:
1582 ;xor si, si ; LBA read
1583 ;push si ; 0
1584 ;push si ; 0
1585 00000281 31DB xor bx, bx ; LBA read
1586 00000283 53 push bx ; 0
1587 00000284 53 push bx ; 0
1588 00000285 52 push dx
1589 00000286 50 push ax ; 0:0:dx:ax = start sector (8 bytes)
1590 00000287 06 push es
1591 00000288 57 push di ; memory buffer address (seg:off)
1592 00000289 FF36[1100] push word [SectorCount]
1593 ; number of sectors to read
1594 ;mov bx, 16 ; size of DAP
1595 0000028D B310 mov bl, 16
1596 0000028F 53 push bx
1597 00000290 89E6 mov si, sp
1598 00000292 B442 mov ah, 42h
1599 00000294 52 push dx
1600 00000295 8A16[3D00] mov dl, [BootDrive]
1601 00000299 1E push ds
1602 0000029A 16 push ss
1603 0000029B 1F pop ds
1604
1605 0000029C CD13 int 13h ; DISK - IBM/MS Extension - EXTENDED READ
1606 ; (DL - drive, DS:SI - disk address packet)
1607 0000029E 1F pop ds
1608 0000029F 5A pop dx ; sector number, hw
1609 000002A0 7209 jc short lba_read_err
1610 000002A2 58 pop ax ; size of DAP (disk address packet) = 16
1611 000002A3 58 pop ax ; number of sectors to read
1612 000002A4 50 push ax ; (**) discard ax on stack (StartSecL)

```

```

1613 000002A5 01DC          add     sp, bx      ; sp points to cx (*)
1614 000002A7 59            pop      cx      ; (*) ; remaining retry count value
1615 000002A8 E98400        jmp      ReadOk
1616 lba_read_err:
1617 000002AB 01DC          add     sp, bx
1618 ;;;
1619 chs_read:
1620 000002AD 89D0          mov     ax, dx      ; start sector, hw
1621 000002AF 31D2          xor     dx, dx ; 0
1622 ;;;
1623 ; 05/10/2023 - Retro DOS v5.0
1624 ; (PCDOS 7.1 IBMBIO.COM - MSLOAD:03E3h
1625 000002B1 3906[2500]    cmp     [SecPerTrack], ax ; hw of disk (LBA) address
1626 000002B5 7326          jnb     short DoDivide ; (must not be > sectors per track)
1627
1628 ErrorOut:
1629 ReadError:
1630 ;push     cs
1631 ;pop      ds
1632 ; ds = cs
1633 000002B7 BE[2904]        mov     si, NonSystemDiskMsg ; "Non-System disk or disk error" ...
1634 WriteTTY:
1635 000002BA AC            lodsb
1636 000002BB 08C0          or      al, al
1637 000002BD 7408          jz      short wait_key_reboot
1638 000002BF B40E          mov     ah, 0Eh
1639 000002C1 B307          mov     bl, 7
1640 000002C3 CD10          int     10h
1641 000002C5 EBF3          jmp     short WriteTTY
1642 wait_key_reboot:
1643 000002C7 30E4          xor     ah, ah
1644 000002C9 CD16          int     16h
1645 ;xor     bx, bx
1646 ;mov     ds, bx ; 0
1647 000002CB C41E[4100]    les     bx, [cs:OrgDasdPtr]
1648 ; les     bx, [OrgDasdPtr] ; Restore ROMBIOS's INT 1Eh vector
1649 ; 06/10/2023
1650 000002CF 31F6          xor     si, si ; 0
1651 000002D1 8EDE          mov     ds, si
1652 000002D3 BE7800        mov     si, 78h ; 1Eh*4
1653 000002D6 891C          mov     [si], bx
1654 000002D8 8C4402        mov     [si+2], es
1655 000002DB CD19          int     19h
1656 ; ; DISK BOOT
1657 ; ; causes reboot of disk system
1658
1659 DoDivide:
1660 ;;;div     word [cs:SecPerTrack]
1661 ;div     word [SecPerTrack]
1662 ; 24/12/2022
1663 000002DD 8B1E[2500]    mov     bx, [SecPerTrack]
1664 000002E1 F7F3          div     bx
1665 000002E3 A3[0B00]        mov     [TempH], ax
1666 000002E6 58            mov     [cs:TempH], ax
1667 000002E7 F7F3          pop     ax ; (**) ; 05/10/2023; start sector, 1w
1668 div     bx
1669 ;div     word [SecPerTrack]
1670 ;div     word [cs:SecPerTrack] ; [TempH]:ax = track,
1671 ; ; dx = sector number
1672 ;mov     bx, [cs:SecPerTrack] ; Get number of sectors we can
1673 ; ; read in this track
1674 000002E9 29D3          sub     bx, dx      ; dx = start sector on (same) track
1675 ;mov     si, bx
1676 000002EB 8B36[1100]    mov     si, [SectorCount] ; sectors to read on (same) track (remain sectors)
1677
1678 000002EF 39DE          cmp     si, bx
1679 000002F1 7602          jna     short GotLength
1680 ;cmp     [SectorCount], si
1681 ; ;cmp     [cs:SectorCount], si ; Is possible sectors in track more
1682 ; ; jnb     short GotLength ; than what we need to read?
1683 000002F3 89DE          mov     si, bx
1684 ;mov     si, [SectorCount]
1685 ; ;mov     si, [cs:SectorCount] ; Yes, only read what we need to
1686 ;GotLength:
1687 ; 24/12/2022
1688 ; IO.SYS < 40KB (segment override is not possible)
1689 ; 700h+0F8FFh < 64KB address
1690 ; ; (there is not an override risk up to 63743 bytes)
1691 ;
1692 ; 24/12/2022
1693 %if 0
1694 ; 24/12/2022
1695 ; dma boundary check for >64KB reads
1696 ; Also, Segment Override risk !
1697 or      di, di
1698 jz      short dma_boundary_ok ; no problem for the 1st read
1699 ;cmp     byte [BootDrive], 80h
1700 ; ;cmp     byte [cs:BootDrive], 80h
1701 ; ; jnb     short dma_boundary_ok ; no problem for hard disks
1702 dma_boundary_chk:
1703 cmp     si, 1
1704 jna     short dma_boundary_ok
1705 ; ; 1 sector read will not cause a boundary error
1706 push     dx
1707 push     ax
1708 mov     ax, si
1709 sub     dx, dx
1710 mul     word [BytesPerSec]
1711 mov     bx, es
1712 mov     cl, 4
1713 shl     bx, cl ; convert paragraphs to bytes
1714 ; bx = segment start position (for 64K memory sections)
1715 add     bx, ax ; byte count to read
1716 pop     ax
1717 pop     dx
1718 add     bx, di ; add current buffer offset to byte count
1719 jnc     short dma_boundary_ok
1720 ;
1721 ; Sector count must be decreased to prevent
1722 ; DMA boundary error or segment override risk!
1723 dec     si
1724 jmp     short dma_boundary_chk
1725 dma_boundary_ok:
1726 %endif
1727 ; 24/12/2022
1728 GotLength:
1729 ;inc     dl
1730 ; ; 18/12/2022
1731 ; ; inc     dx
1732 000002F5 42            mov     bl, dl
1733 000002F6 88D3          mov     [bx], dl ; Start sector in BL
1734 ; 24/12/2022
1735 000002F8 8B16[0B00]    mov     dx, [TempH] ; DX:AX = Track
1736 000002FC 50            mov     dx, [cs:TempH] ; DX:AX = Track
1737 push     ax

```



```

1737 000002FD 89D0      mov     ax, dx
1738 000002FF 31D2      xor     dx, dx
1739                      ; 24/12/2022
1740 00000301 F736[2700] div     word [NumHeads]
1741                      ;div word [cs:NumHeads] ; start cyl in AX, head in dl
1742                      ;mov [TempH], ax
1743                      ;;mov [cs:TempH], ax
1744 00000305 58        pop     ax
1745 00000306 F736[2700] div     word [NumHeads]
1746                      ;div word [cs:NumHeads] ; [TempH]:AX = Cylinder, DX = Head
1747                      ; At this moment, we assume that TempH = 0,
1748                      ; ax <= 1024, dx <= 255
1749
1750
1751 0000030A 88D6      mov     dh, dl
1752
1753 0000030C B106      mov     cl, 6
1754 0000030E D2E4      shl     ah, cl ; Shift cyl high bits up
1755 00000310 08DC      or      ah, bl ; Mix in with sector bits
1756 00000312 88C5      mov     ch, al ; Setup cyl low
1757 00000314 88E1      mov     cl, ah ; Setup cyl/high - sector
1758 00000316 89FB      mov     bx, di ; Get back OFFSET
1759                      ; 24/12/2022
1760 00000318 8A16[3D00] mov     dl, [BootDrive] ; Get drive
1761                      ;mov dl, [cs:BootDrive] ; Get drive
1762 0000031C 89F0      mov     ax, si ; Get number of sectors to read (al)
1763 0000031E B402      mov     ah, 2 ; Read sectors
1764                      ; 23/12/2022
1765                      ;push ax
1766                      ;push di
1767
1768                      ; Issue one read request. ES:BX have the transfer address,
1769                      ; AL is the number of sectors.
1770
1771 00000320 CD13      int     13h ; DISK - READ SECTORS INTO MEMORY
1772                      ; AL = number of sectors to read, CH = track, CL = sector
1773                      ; DH = head, DL = drive, ES:BX -> buffer to fill
1774                      ; Return: CF set on error, AH = status, AL = number of sectors read
1775                      ; 23/12/2022
1776                      ;pop di
1777                      ;pop ax
1778
1779                      ; 09/12/2023
1780                      ; 23/12/2022
1781                      ;mov ah, 0
1782
1783 00000322 59        pop     cx ; (*) ; Get retry count back
1784 00000323 730A      jnc     short ReadOk ; 23/12/2022
1785
1786                      ; 23/12/2022
1787                      ;mov bx, di ; Get offset
1788                      ; 12/12/2023
1789 00000325 30E4      xor     ah, ah
1790                      ; ah = 0
1791                      ; 23/12/2022
1792                      ;push cx
1793                      ; 24/12/2022
1794                      ;mov dl, [BootDrive]
1795                      ;;mov dl, [cs:BootDrive]
1796                      ; 23/12/2022
1797                      ;push di
1798 00000327 CD13      int     13h ; DISK - RESET DISK SYSTEM
1799                      ; DL = drive (if bit 7 is set both hard disks and floppy disks reset)
1800                      ; 23/12/2022
1801                      ;pop di
1802                      ;pop cx
1803 00000329 49        dec     cx
1804 0000032A 748B      jz      short ReadError
1805 0000032C E91AFF      jmp     TryRead
1806
1807                      ; -----
1808                      ; 05/10/2023
1809                      ; ReadError:
1810                      ; jmp ErrorOut
1811                      ; -----
1812
1813 ReadOk:
1814                      ; 09/12/2023
1815                      ; al = sectors per cluster or sectors per track
1816                      ; (al <= 64, sectors per cluster <= 64) ; (!!!)
1817                      ; ((128*512 = 65536 -> ax=0, dx=1 is unexpected result here!))
1818                      ; 23/12/2022
1819                      ; ah = 0
1820                      ; 22/12/2022
1821                      ;xor ah, ah ; Mask out read command, just get # read
1822                      ; ch = 0
1823 0000032F 88C1      mov     cl, al
1824
1825                      ; 09/12/2023
1826                      ; 22/12/2022
1827                      ;; cx = ax = read (sector) count
1828                      ;;mov bx, [cs:BytesPerSec] ; Bytes per sector
1829                      ;;mul bx ; Get total bytes read
1830                      ; 24/12/2022
1831                      ;; ds = cs
1832                      ;mul word [BytesPerSec]
1833                      ;;mul word [cs:BytesPerSec]
1834                      ; 09/12/2023
1835 00000331 A1[1B00] mov     ax, [BytesPerSec]
1836 00000334 F7E1      mul     cx ; (!!!)
1837 00000336 01C7      add     di, ax ; Add it to OFFSET
1838                      ; 09/12/2023
1839                      ; dx = 0 (CL must be <= 64) ; (!!!)
1840
1841                      ; 24/12/2022
1842                      ; IO.SYS < 40KB (segment override is not possible)
1843                      ; 700h+0F8FFh < 64KB address
1844                      ; (there is not an override risk up to 63743 bytes)
1845                      ;
1846                      ;add di, ax
1847                      ;jnc short read_next_sector
1848                      ;mov bx, es
1849                      ;;add bx, 1000h
1850                      ;add bh, 10h
1851                      ;mov es, bx
1852                      ;read_next_sector:
1853                      ; 24/12/2022
1854                      ; ds = cs
1855                      ; 22/12/2022
1856 00000338 290E[1100] sub     [SectorCount], cx
1857                      ;sub [cs:SectorCount], cx
1858                      ;;sub [cs:SectorCount], ax ; Bump number down
1859 0000033C 7464      jz      short EndRead
1860 0000033E 010E[0700] add     [StartSecL], cx

```

```

1861             ;add [cs:StartSecL], cx
1862             ;add [cs:StartSecL], ax ; where to start next time
1863             ;adc word [StartSecH], 0
1864             ; 09/12/2023
1865 00000342 1116[0900]     adc [StartSecH], dx ; 0
1866             ;adc word [cs:StartSecH], 0
1867 00000346 E9FDFF     jmp ReadSectors
1868             ; -----
1869
1870             ; 09/12/2023
1871             ; 06/10/2023
1872             ; 24/12/2022
1873 ;EndRead:
1874             ;
1875             retn
1876
1877 ; ===== S U B R O U T I N E =====
1878
1879 ; GetNextFatEntry
1880 ; -----
1881 ;
1882 ; NOTES:
1883 ;
1884 ; given the last cluster found, this will return the next cluster of
1885 ; iosys. if the last cluster is (f)ff8 - (f)fff, then the final cluster
1886 ; of iosys has been loaded, and control is passed to goto_iosys
1887 ; msload can handle maximum fat area size of 128 kb.
1888 ;
1889 ; INPUT:
1890 ;
1891 ; CS:CurrentCluster
1892 ; CS:FatSize
1893 ;
1894 ; OUTPUT:
1895 ;
1896 ; CS:CurrentCluster (updated)
1897 ;
1898 ; calls: GetFatSector
1899 ; -----
1900 ; get CurrentCluster
1901 ;
1902 ; if (16 bit fat)
1903 ; {if (CurrentCluster = fff8 - ffff)
1904 ; {jmp goto_iosys}
1905 ; else
1906 ; {get OFFSET by multiply cluster by 2}
1907 ;
1908 ; else
1909 ; {if (CurrentCluster = ff8 - fff)
1910 ; {jmp goto_iosys}
1911 ; else
1912 ; {get OFFSET by multiply cluster by 3
1913 ;
1914 ; rotate right to divide by 2
1915 ;
1916 ; if (cy set - means odd number)
1917 ; {shr 4 times to keep high twelve bits}
1918 ;
1919 ; else
1920 ; {and with 0fffh to keep low 12 bits}
1921 ; }
1922 ; }
1923 ; -----
1924
1925 ; 09/12/2022
1926 ; FAT_12_BIT equ 1
1927 ; NOT_END_OF_FILE equ 0 ; ~END_OF_FILE ; END_OF_FILE equ 0FFh
1928
1929 GetNextFatEntry:
1930 ; 05/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PCDOS 7.1)
1931
1932 00000349 06             push es
1933             ; 24/12/2022
1934             ; ds = cs
1935             ;mov ax, [cs:FatSegment]
1936 0000034A A1[4500]     mov ax, [FatSegment]
1937 0000034D 8EC0     mov es, ax ; ES-> FAT area segment
1938             ; 09/12/2022
1939             ;mov byte [cs:EndOfFile], END_OF_FILE
1940             ;mov byte [cs:EndOfFile], 0FFh ; Assume last cluster
1941             ;mov ax, [cs:CurrentCluster] ; Get last cluster
1942             ; 24/12/2022
1943             ; ds = cs
1944 0000034F C606[4000]FF mov byte [EndOfFile], 0FFh ; Assume last cluster
1945             ;;;
1946             ;mov ax, [CurrentCluster] ; Get last cluster
1947             ; 05/10/2023 - Retro DOS v5.0 (PCDOS 7.1)
1948 00000354 A1[1300]     mov ax, [CurrentClusterL] ; Get last cluster
1949             ; 06/10/2023
1950             ;mov di, [CurrentClusterH]
1951
1952 chk_fat32_type:
1953             ; 05/10/2023
1954 00000357 803E[3E00]0B cmp byte [FatType], 0Bh ; FAT32 (CHS) fs ?
1955             ;jne short chk_fat_type ; no
1956
1957 Got32bit:
1958 0000035E 8B3E[1500]     mov di, [CurrentClusterH]
1959 00000362 01C0     add ax, ax
1960 00000364 11FF     adc di, di
1961 00000366 01C0     add ax, ax
1962 00000368 11FF     adc di, di ; Get the FAT offset (di:si)
1963 0000036A 89C6     mov si, ax
1964 0000036C E86700     call GetFatSector
1965 0000036F 268B07     mov ax, [es:bx]
1966 00000372 268B7F02     mov di, [es:bx+2]
1967             ; 06/10/2023
1968             ;and di, 0FFFh ; 28 bit cluster number
1969             ;cmp di, 0FFFh ; FAT32 cluster numbers are 28 bit numbers
1970             ; ; (higher 4 bits are -must be- zero)
1971             ;jne short GotFAT32ClusterDone
1972             ;jne short GotClusterDoneJ
1973             ; 06/10/2023
1974             ;cmp ax, 0FFF8h
1975
1976 ;GotFAT32ClusterDone:
1977             ;jmp short GotClusterDoneJ
1978             ; 09/12/2023
1979             ;mov dx, 0FFF8h
1980             ;jmp short isitlastcluster
1981             ;jmp short isitlastcluster_16_32
1982             ;;;
1983 chk_fat_type:
1984             ; 06/10/2023
1985 0000037E 29FF     sub di, di ; 0 ; hw of the cluster number must be 0
1986             ; ; (if it is not FAT32 cluster)
1987             ; 05/10/2023

```



```

1985 00000380 803E[3E00]01      cmp     byte [FatType], 1
1986                               ;cmp     byte [Fatsize], 1
1987                               ;;;cmp    byte [cs:FatSize], FAT_12_BIT
1988                               ;;cmp     byte [cs:Fatsize], 1
1989                               ;jne      short Got16Bit ; 23/12/2022
1990                               ; 09/12/2023
1991 00000385 741C              je       short Got12Bit
1992
1993                               ; -----
1994
1995                               ; 09/12/2023
1996
1997                               ; 23/12/2022
1998                               ;push    dx
1999                               ;xor     dx, dx
2000                               ; 05/10/2023
2001                               ;sub     dx, dx ; 23/12/2022
2002
2003                               ; 05/10/2023 - Retro DOS v5.0 IBMBIO.COM (MSLOAD)
2004                               ;shl     ax, 1          ; Multiply cluster by 2
2005                               ;adc     dx, 0
2006 00000387 D1E0              shl     ax, 1
2007 00000389 11FF              adc     di, di ; di = 0 ; 06/10/2023
2008
2009                               mov     si, ax          ; Get the final buffer OFFSET
2010 0000038D E84600            call    GetFatSector
2011                               ; 23/12/2022
2012                               ;pop     dx
2013
2014                               ; 05/10/2023
2015 00000390 31FF              xor     di, di          ; HW of cluster number is 0
2016
2017                               mov     ax, [es:bx]
2018
2019                               ; 09/12/2023
2020                               isitlastcluster_16_32:
2021                               ; 06/10/2023
2022 00000395 BAF8FF            mov     dx, 0FFF8h
2023                               ;jmp     short isitlastcluster
2024
2025                               ; 09/12/2023
2026                               ; 06/10/2023
2027                               %if 1
2028                               isitlastcluster:
2029                               cmp     ax, dx
2030                               ;cmp     ax, 0FFF8h
2031 0000039A 7305              GotClusterDoneJ: ; 05/10/2023
2032                               jnb     short GotClusterDone
2033                               NotLastCluster:
2034                               ; 24/12/2022
2035                               ; ds = cs
2036                               ;mov     byte [cs:EndOfFile], NOT_END_OF_FILE ; ~END_OF_FILE
2037 0000039C C606[4000]00      ;mov     byte [cs:EndOfFile], 0; Assume not last cluster
2038                               mov     byte [EndOfFile], 0 ; Assume not last cluster
2039 000003A1 07              GotClusterDone:
2040                               pop     es
2041                               ; 24/12/2022
2042 000003A2 C3              EndRead:
2043                               retn
2044                               %endif
2045
2046                               ; -----
2047
2048                               ; 09/12/2023
2049 000003A3 89C6              Got12Bit:
2050                               mov     si, ax
2051                               ;shr     ax, 1
2052                               ;add     si, ax          ; SI = AX * 1.5 = AX + AX/2
2053                               ; 05/10/2023
2054                               ;mov     dx, di
2055                               ;shr     dx, 1
2056                               ;rcr     ax, 1
2057                               ;add     si, ax
2058                               ;adc     di, dx          ; di:si = dx:ax * 1.5 = dx:ax + dx:ax/2
2059 000003A5 D1E8              ; 06/10/2023
2060 000003A7 01C6              shr     ax, 1
2061                               add     si, ax
2062                               ;sub     di, di ; 0 ; (di must be 0 for FAT12)
2063
2064                               ; 23/12/2022
2065                               ;push    dx
2066                               ;xor     dx, dx
2067                               ; 05/10/2023
2068 000003A9 E82A00            ;sub     dx, dx ; 23/12/2022
2069                               call    GetFatSector
2070                               ; 23/12/2022
2071 000003AC 7510              ;pop     dx
2072 000003AE 268A07            jnz     short ClusterOk
2073                               mov     al, [es:bx]
2074                               ; 22/12/2022
2075                               ;mov     [cs:TempCluster], al
2076 000003B1 50              ; 06/10/2023
2077                               push    ax ; (*)
2078                               ;inc     si
2079                               ;add     si, 1
2080                               ;adc     di, 0
2081 000003B2 46              ; 05/10/2023
2082                               inc     si
2083                               ; 06/10/2023
2084                               ;jnz     short Got12Bit_rnfs
2085                               ;inc     di
2086                               ;Got12Bit_rnfs:
2087                               ; 23/12/2022
2088                               ;push    dx
2089                               ; 05/10/2023
2090 000003B3 E82000            ;xor     dx, dx
2091                               call    GetFatSector ; Read next fat sector
2092                               ; 23/12/2022
2093                               ;pop     dx
2094                               ; 22/12/2022
2095                               ;mov     al, [es:0]
2096                               ;mov     [cs:TempCluster+1], al
2097                               ;mov     ax, [cs:TempCluster]
2098                               ; 06/10/2023
2099 000003B6 58              ; 22/12/2022
2100 000003B7 268A260000        pop     ax ; (*)
2101 000003BC EB03              mov     ah, [es:0]
2102                               jmp     short EvenOdd
2103
2104                               ; -----
2105 000003BE 268B07            ClusterOk:
2106                               mov     ax, [es:bx]
2107                               EvenOdd:
2108                               ; 24/12/2022
2109                               ; ds = cs

```

```

2109             ;test byte [CurrentCluster], 1
2110             ; 05/10/2023 - Retro DOS v5.0 IBMBIO.COM (MSLOAD)
2111 000003C1 F606[1300]01 test byte [CurrentClusterL], 1
2112             ; 10/12/2022
2113             ;test byte [cs:CurrentCluster], 1 ; 09/12/2022
2114             ;test word [cs:CurrentCluster], 1 ; was last cluster odd?
2115 000003C6 7505 jnz short OddResult ; If not zero it was odd
2116 000003C8 25FF0F and ax, 0FFFh ; Keep low 12 bits
2117 000003CB EB04 jmp short TestEOF
2118             ; -----
2119
2120 OddResult:
2121 000003CD B104 mov cl, 4 ; Keep high 12 bits for odd
2122 000003CF D3E8 shr ax, cl
2123
2124 TestEOF:
2125             ; 06/10/2023
2126             ; di = 0
2127             ;xor di, di ; HW of cluster number is 0
2128
2129             ; 06/10/2023
2130             ;cmp ax, 0FF8h ; Is it last cluster?
2131             ;jnb short GotClusterDone ; Yep, all done here
2132             ;jmp short NotLastCluster
2133 000003D1 BAF80F mov dx, 0FF8h
2134 000003D4 EBC2 jmp short isitlastcluster
2135             ; 09/12/2023
2136             ; 06/10/2023
2137             ;if 0
2138 isitlastcluster:
2139             ; 06/10/2023
2140             ;cmp ax, dx
2141 GotClusterDone:
2142             jnb short GotClusterDone
2143 NotLastCluster:
2144             ; 24/12/2022
2145             ; ds = cs
2146             ;mov byte [cs:EndOfFile], NOT_END_OF_FILE ; ~END_OF_FILE
2147             ;mov byte [cs:EndOfFile], 0; Assume not last cluster
2148             ;mov byte [EndOfFile], 0 ; Assume not last cluster
2149 GotClusterDone:
2150             pop es
2151             ; 06/10/2023
2152             ; 24/12/2022
2153             ;EndRead:
2154             retn
2155             %endif
2156
2157             ; GetFatSector
2158             ; -----
2159             ;function: find and read the corresponding fat sector into ES:0
2160             ;
2161             ;in). SI = offset value (starting from fat entry 0) of fat entry to find.
2162             ; ES = fat sector segment
2163             ; CS:BytesPerSec
2164             ;
2165             ;out). corresponding fat sector read in.
2166             ; BX = offset value of the corresponding fat entry in the fat sector.
2167             ; CX destroyed.
2168             ; zero flag set if the fat entry is splitted, i.e. when 12 bit fat entry
2169             ; starts at the last byte of the fat sector. in this case, the caller
2170             ; should save this byte, and read the next fat sector to get the rest
2171             ; of the fat entry value. (this will only happen with the 12 bit fat).
2172             ; -----
2173
2174             ; 09/12/2023
2175             ; 06/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PC DOS 7.1)
2176             ; (PC DOS 7.1 IBMBIO.COM - MSLOAD:054ch)
2177
2178             ; 24/12/2022
2179             ; 22/12/2022
2180
2181 GetFatSector:
2182             ;push ax ; 06/10/2023
2183             push si
2184             push di ; di:si = byte offset in (entire) FAT
2185             ;;;
2186             ; 06/10/2023
2187             mov ax, di ; 32 bit division (dx:ax/512)
2188             mov cx, [BytesPerSec]
2189             xor dx, dx
2190             div cx
2191             mov bx, ax
2192             ;;;
2193             mov ax, si
2194             ; 24/12/2022
2195             ; ds = cs
2196             ;mov cx, [cs:BytesPerSec]
2197             ; 06/10/2023
2198             div cx ; AX = Sector number, DX = Offset
2199             ;div word [BytesPerSec]
2200             ; dx = byte offset in the FAT sector
2201             ; ax = low word of the FAT sector number
2202             ; bx = high word of the FAT sector number
2203             ; 06/10/2023
2204             cmp bx, [LastFatSectorH] ; FAT32 (32 bit cluster numbers)
2205             jne short not_same_fat_sector
2206             cmp ax, [LastFatSectorL]
2207             ;cmp ax, [LastFatSector]
2208             ;cmp ax, [cs:LastFatSector]; The same fat sector?
2209             je short SplitChk ; Don't need to read it again.
2210 not_same_fat_sector:
2211             ; 06/10/2023
2212             mov [LastFatSectorL], ax
2213             mov [LastFatSectorH], bx
2214             ;
2215             ;mov [LastFatSector], ax
2216             ;mov [cs:LastFatSector], ax
2217             ; 06/10/2023
2218             push cx ; ** ; bytes per sector
2219             ;
2220             push dx ; *
2221             ; 24/12/2022
2222             ;xor dx, dx
2223             ; 06/10/2023
2224             mov dx, bx
2225             ;
2226             ;add ax, [cs:HiddenSectorsL]
2227             ;adc dx, [cs:HiddenSectorsH]
2228             ;add ax, [cs:ReservSectors]
2229             ;adc dx, 0
2230             ; 24/12/2022
2231             ; ds = cs
2232             ; 06/10/2023 - Retro DOS v5.0 (PC DOS 7.1 IBMBIO.COM)

```

```

2233             ;add ax, [FatStartSecL]
2234             ;adc dx, [FatStartSecH]
2235             ; 09/12/2023
2236 000003FF 31FF xor di, di ; 0
2237 00000401 0306[2900] add ax, [HiddenSectorsL]
2238 00000405 1316[2800] adc dx, [HiddenSectorsH]
2239 00000409 0306[1F00] add ax, [ReservSectors]
2240             ;adc dx, 0
2241             ; 09/12/2023
2242 0000040D 11FA adc dx, di ; 0
2243
2244 0000040F A3[0700] mov [StartSecL], ax
2245 00000412 8916[0900] mov [StartSecH], dx ; Set up for ReadSectors
2246             ;mov [cs:StartSecL], ax
2247             ;mov [cs:StartSecH], dx ; Set up for ReadSectors
2248
2249 00000416 C706[1100]0100 mov word [SectorCount], 1 ; 1 sector
2250             ;mov word [cs:SectorCount], 1 ; 1 sector
2251             ; 06/10/2023
2252             ; di = 0
2253             ;xor di, di ; 0
2254             ; es:di = FATSEGMENT:0000h
2255 0000041C E827FE call ReadSectors
2256 0000041F 5A pop dx ; *
2257             ; 06/10/2023
2258             ;mov cx, [BytesPerSec]
2259 00000420 59 pop cx ; **
2260             ; 24/12/2022
2261             ;mov cx, [cs:BytesPerSec]
2262 SplitChk:
2263             ; 06/10/2023
2264             ; cx = bytes per sector
2265             ; 24/12/2022
2266             ;mov cx, [BytesPerSec]
2267 00000421 49 dec cx ; CX = SECTOR SIZE - 1
2268 00000422 39CA cmp dx, cx ; If last byte of sector, splitted entry.
2269 00000424 89D3 mov bx, dx ; set bx to dx
2270 00000426 5F pop di
2271 00000427 5E pop si
2272             ;pop ax ; 06/10/2023
2273 EndWrite:             ; 10/12/2022
2274 00000428 C3 retn
2275
2276 ; -----
2277
2278 ; 05/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PC DOS 7.1)
2279 %if 0
2280
2281 ErrorOut:
2282             ; 24/12/2022
2283             ; ds = cs
2284             ;push cs
2285             ;pop ds
2286
2287 mov si, NonSystemDiskMsg ; "\r\nNon-System disk or disk error\r\nRe"...
2288 call WriteTTY
2289
2290             ; wait for a keypress on the keyboard.
2291             ; Use the bios keyboard interrupt.
2292
2293 xor ah, ah
2294 int 16h ; KEYBOARD - READ CHAR FROM BUFFER, WAIT IF EMPTY
2295             ; Return: AH = scan code, AL = character
2296
2297             ; We have to restore the address of the original rom disk
2298             ; parameter table to the location at [0:DskAddr]. The address
2299             ; of this original table has been saved previously in
2300             ; 0:OrgDasdPtr and 0:OrgDasdPtr+2. After this table address
2301             ; has been restored we can reboot by invoking the bootstrap
2302             ; loader bios interrupt.
2303
2304             ; 23/12/2022
2305             ;xor bx, bx
2306             ;mov ds, bx
2307             ;les bx, [OrgDasdPtr] ; wrong DS segment !
2308             ; ; (Erdogan Tan, 23/12/2022)
2309             ;les bx, [OrgDasdPtr] ; Correct DS segment = CS
2310
2311             ; 23/12/2022
2312             ;push ss ; 0
2313             ;pop ds
2314             ; ds = 0
2315
2316 mov si, DskAddr ; (Int 1Eh)
2317 mov [si], bx ; restore offset
2318 mov [si+2], es ; restore segment
2319 int 19h ; reboot
2320
2321 ; ===== S U B R O U T I N E =====
2322
2323 ; WriteTTY
2324 ; -----
2325 ; in) DS:si -> asciiz string.
2326 ;
2327 ; WriteTTY the character in al to the screen.
2328 ; use video service 'write teletype to active page' (ROM_TTY)
2329 ; use normal character attribute
2330 ; -----
2331
2332 WriteTTY:
2333 lodsb
2334 or al, al
2335 jz short EndWrite
2336 ;mov AH, ROM_TTY ; 09/12/2022
2337 mov ah, 0Eh
2338 mov bl, 7 ; "normal" attribute
2339 int 10h ; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
2340             ; AL = character, BH = display page (alpha modes)
2341             ; BL = foreground color (graphics modes)
2342 jmp short writeTTY
2343 ; -----
2344
2345 ; 10/12/2022
2346 ;EndWrite:
2347 ; retn
2348
2349 %endif ; 05/10/2023
2350
2351 ; -----
2352
2353 ; 06/10/2023 - Retro DOS v5.0 IBMBIO.COM (IO.SYS) ((Modified PC DOS 7.1))
2354 ; (PC DOS 7.1 IBMBIO.COM - MSLOAD:054Ch)
2355
2356 ; 09/12/2022

```

```

2357 ;include msbio.c11
2358
2359 ; 22/12/2022
2360 ; 20/12/2022
2361 ; 18/12/2022
2362 ;db 0 ; (word alignment)
2363 NonSystemDiskMsg:
2364 00000429 0D0A db 0Dh,0Ah ;...
2365 0000042B 4E6F6E2D5379737465- db 'Non-System disk or disk error',0Dh,0Ah
2366 00000434 6D206469736B206F72-
2367 0000043D 206469736B20657272-
2368 00000446 6F720D0A
2369 0000044A 5265706C6163652061- db 'Replace and press any key when ready',0Dh,0Ah,0
2370 00000453 6E6420707265737320-
2371 0000045C 616E79206B65792077-
2372 00000465 68656E207265616479-
2373 0000046E 0D0A00
2374
2375 ; 25/12/2022
2376 align 16
2377
2378 EndOfLoader: ; (PCDOS 7.1 IBMBIO.COM - MSLOAD:05F0h) ; 06/10/2023
2379 ;dw 01A1h ; 10/12/2022
2380
2381 ; -----
2382 ; =====
2383 ; DOS BIOS (IO.SYS) DATA SEGMENT
2384 ; =====
2385 ; 09/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
2386 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
2387 ; 02/10/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
2388
2389 section .BIOSDATA vstart=0
2390
2391 ;--- DOSBIOS data segment -----
2392 ;-----
2393 ;Bios_Data segment
2394
2395 BData_start:
2396 hdrv_pat: jmp init ; MSBIO1.ASM, MSSBDATA.INC
2397 ; -----
2398
2399 DosDataSg: dw 0
2400
2401 ; DOS's int 2f handler will exit via a jump through here.
2402 ; This is how the BIOS hooks int2f
2403
2404 ;BIOSDATA:0005h: ; 10/05/2023 (Note the 'bios_i2f equ 5' in 'msdos6.s')
2405
2406 bios_i2f: db 0EAh ; far jump to int_2f (segment may not be at 70h)
2407 ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
2408 ; PCDOS 7.1 IBMBIO.COM - BIODATA:0006h
2409 ;dw int_2f
2410 ;dw 70h ; BIOSDATA segment (KERNEL_SEGMENT)
2411 ;dw i2f_handler
2412 bios_i2f_seg: ; 10/08/2023
2413 dw DOSBIOCODESEG ; 02Cch for MSDOS 6.21 IO.SYS (25Ch+070h)
2414 ; 0364h PCDOS 7.1 IBMBIO.COM (2F4h+070h)
2415
2416 romstartaddr: dw 0 ; The start address for the romfind routines
2417 ; This is to maintain binary compatibility
2418 ; with DISK based DOS 5.0
2419
2420 ; This is a byte used for special key handling in the resident
2421 ; console device driver. It must be here so that it can be included
2422 ; in the WIN386 instance table (in INC\LMSTUB.ASM).
2423
2424 altah: db 0 ; special key handling
2425
2426 inHMA: db 0 ; flag indicates we're running from HMA
2427 xms: dd 0 ; entry point to xms if above is true
2428
2429 ; PTRSAV - pointer save
2430 ;
2431 ; This variable holds the pointer to the Request Header passed by a program
2432 ; wishing to use a device driver. When the strategy routine is called it
2433 ; puts the address of the Request header in this variable and returns.
2434
2435 ptrsav: dd 0
2436 auxbuf: db 0, 0, 0, 0 ; 19/10/2022 ; set of 1 byte buffers for com 1,2,3, and 4
2437
2438 zeroseg: dw 0 ; easy way to load segment registers with zero
2439 i13_ds: dw 0 ; ds register for int13 call through
2440 prevoper: dw 0 ; holds int 13 request (i.e. register ax).
2441 number_of_sec: db 0 ; holds number of secs. to read on an ecc error
2442 auxnum: dw 0 ; which aux device was requested
2443
2444 ;-----
2445 res_dev_list:
2446 ; Device Header for the CON Device Driver
2447
2448 CONHeader: ; HEADER FOR DEVICE "CON"
2449 dw auxdev2
2450 dw 70h
2451 word_727: dw 8013h
2452 dw strategy
2453 dw con_entry
2454 aCon: db 'CON'
2455 auxdev2: dw prndev2 ; HEADER FOR DEVICE "AUX"
2456 dw 70h
2457 dw 8000h
2458 dw strategy
2459 dw aux0_entry
2460 aAux: db 'AUX'
2461 prndev2: dw timdev ; HEADER FOR DEVICE "PRN"
2462 dw 70h
2463 word_74B: dw 0A0C0h
2464 dw strategy
2465 dw prn0_entry
2466 aPrn: db 'PRN' ; HEADER FOR DEVICE "CLOCK$"
2467 timdev: dw dskdev
2468 dw 70h
2469 dw 8008h
2470 dw strategy
2471 dw tim_entry
2472 aClock: db 'CLOCK$'
2473 dskdev: dw com1dev ; HEADER FOR DISK DEVICES
2474 dw 70h
2475 ;dw 8C2h
2476 ; 02/10/2023 - Retro DOS v5.0
2477 dw 48C2h ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:006Fh

```

```

2474             ;dw offset strategy
2475             ;dw offset dsk_entry
2476             ; 19/10/2022
2477 00000071 [1506] dw strategy
2478 00000073 [5E06] dw dsk_entry
2479
2480             ; maximum number of drives
2481
2482 00000075 04      drvmax:      db 4
2483 00000076 FE      step_drv:  db 0FEh ; -2          ; last drive accessed
2484 00000077 00      fhav96:    db 0          ; flag to indicate presence of
2485                                     ; 96tpi support
2486 00000078 00      single:    db 0          ; used to detect single drive systems
2487 00000079 00      fhav909:  db 0          ; indicates if this is a k09 or not
2488                                     ; used by console driver.
2489 0000007A 00      fsetowner: db 0          ; = 1 if we are setting the owner of a
2490                                     ; drive. (examined by checksingle).
2491
2492 0000007B [8D00]    com1dev:    dw lpt1dev      ; Device Header for device "COM1"
2493 0000007D 7000      dw 70h
2494 0000007F 0080      dw 8000h
2495 00000081 [1506]    dw strategy
2496 00000083 [4106]    dw aux0_entry
2497 00000085 434F4D3120202020 aCom1:    db 'COM1'
2498 0000008D [9F00]    lpt1dev:    dw lpt2dev      ; Device Header for device LPT1
2499 0000008F 7000      dw 70h
2500 00000091 C0A0      dw 0A0C0h
2501 00000093 [1506]    dw strategy
2502 00000095 [2C06]    dw prn1_entry
2503 00000097 4C50543120202020 aLpt1:    db 'LPT1'
2504 0000009F [B800]    lpt2dev:    dw lpt3dev      ; Device Header for device LPT2
2505 000000A1 7000      dw 70h
2506 000000A3 C0A0      dw 0A0C0h
2507 000000A5 [1506]    dw strategy
2508 000000A7 [3306]    dw prn2_entry
2509 000000A9 4C5054322020202000- aLpt2:    db 'LPT2' ',0,0,0'
2509 000000B2 0000
2510
2511             ;M058; Start of changes
2512             ; Orig13 needs to be at offset 0B4h for the CMS floppy driver to work.
2513             ; These guys patch Orig13 with their own int 13h hook and so this offset
2514             ; cannot change for them to work. Even ProComm does this.
2515
2516 000000B4 00000000    Orig13:    dd 0          ; to make Orig13 offset 0B4h
2517
2518 000000B8 [CA00]    lpt3dev:    dw com2dev      ; Device Header for device LPT3
2519 000000BA 7000      dw 70h
2520 000000BC C0A0      dw 0A0C0h
2521 000000BE [1506]    dw strategy
2522 000000C0 [3A06]    dw prn3_entry
2523 000000C2 4C50543320202020 aLpt3:    db 'LPT3'
2524 000000CA [DC00]    com2dev:    dw com3dev      ; Device Header for device "COM2"
2525 000000CC 7000      dw 70h
2526 000000CE 0080      dw 8000h
2527 000000D0 [1506]    dw strategy
2528 000000D2 [4706]    dw aux1_entry
2529                                     ; 19/10/2022
2530 000000D4 434F4D3220202020 aCom2:    db 'COM2'
2531 com3dev:    ;dw offset com4dev      ; Device Header for device "COM3"
2532 000000DC [EE00]    dw com4dev
2533 000000DE 7000      dw 70h
2534 000000E0 0080      dw 8000h
2535                                     ;dw offset strategy
2536                                     ;dw offset aux2_entry
2537 000000E2 [1506]    dw strategy
2538 000000E4 [4D06]    dw aux2_entry
2539 000000E6 434F4D3320202020 aCom3:    db 'COM3'
2540 000000EE FFFF      com4dev:    dw 0FFFFh      ; Device Header for device "COM4"
2541 000000F0 7000      dw 70h
2542 000000F2 0080      dw 8000h
2543 000000F4 [1506]    dw strategy
2544 000000F6 [5306]    dw aux3_entry
2545 000000F8 434F4D3420202020 db 'COM4'
2546
2547             ;-----
2548
2549 00000100 10      RomVectors: db 10h
2550 00000101 00000000    Old10:    dd 0
2551 00000105 13      db 13h
2552 00000106 00000000    Old13:    dd 0
2553 0000010A 15      db 15h
2554 0000010B 00000000    Old15:    dd 0
2555 0000010F 19      db 19h
2556 00000110 00000000    Old19:    dd 0
2557 00000114 1B      db 1Bh
2558 00000115 00000000    Old1B:    dd 0
2559
2560             ;EndRomVectors      equ $
2561
2562             ;NUMROMVECTORS      equ ((EndRomVectors - RomVectors)/5)
2563
2564             ;-----
2565
2566 00000119 [5203]    start_bds:  dw bds1          ; Start of linked list of BDS's
2567 0000011B 7000      dw 70h          ; KERNEL_SEGMENT
2568
2569             ; (MSDOS 3.3) NOTE:
2570             ; Some floppy drives do not have changeline support. The result is a
2571             ; large amount of inefficiency in the code. A media-check always returns
2572             ; "I don't know". This cause DOS to reread the FAT on every access and
2573             ; always discard any cached data.
2574             ; We get around this inefficiency by implementing a "Logical Door Latch".
2575             ; The following three items are used to do this. The logical door latch is
2576             ; based on the premise that it is not physically possible to change floppy
2577             ; disks in a drive in under two seconds (most people take about 10). The
2578             ; logical door latch is implemented by saving the time of the last successful
2579             ; disk operation (in the value TIM_DRV). When a new request is made the
2580             ; current time is compared to the saved time. If less than two seconds have
2581             ; passed then the value "No Change" is returned. If more than two seconds
2582             ; have passed the value "Don't Know" is returned.
2583             ; There is one complication to this algorithm. Some programs change the
2584             ; value of the timer. In this unfortunate case we have an invalid timer.
2585             ; This possibility is detected by counting the number of disk operations
2586             ; which occur without any time passing. If this count exceeds the value of
2587             ; "AccessMax" we assume the counter is invalid and always return "Don't
2588             ; know". The variable "AccessCount" is used to keep track of the number
2589             ; of disk operation which occur without the time changing.
2590
2591 0000011D 00      accesscount: db 0
2592 0000011E FF      tim_drv:   db 0FFh
2593 0000011F 00      medbyt:    db 0
2594 wrtverify:    ; 15/10/2022
2595 00000120 02      rflag:     db 2          ; 2 for read, 3 for write
2596 00000121 00      verify:    db 0          ; 1 if verify after write

```

```

2597 00000122 0000
2598 00000124 00
2599 00000125 01
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621 00000126 00
2622 00000127 00
2623 00000128 00
2624 00000129 00
2625 0000012A 00
2626 0000012B 00
2627 0000012C 09
2628 0000012D 00000000
2629 00000131 00
2630 00000132 00
2631 00000133 0000
2632 00000135 0000
2633 00000137 08
2634 00000138 00
2635 00000139 0000
2636 0000013B 50
2637
2638
2639
2640
2641
2642 0000013C CC
2643 0000013D 80
2644 0000013E 40
2645 0000013F 10
2646 00000140 08
2647 00000141 06
2648 00000142 04
2649 00000143 03
2650
2651 00000144 01
2652 00000145 B2
2653
2654 00000146 00
2655
2656
2657
2658 00000147 0A
2659 00000148 02
2660 00000149 06
2661 0000014A 04
2662 0000014B 04
2663 0000014C 0F
2664 0000014D 08
2665 0000014E 00
2666
2667 0000014F 03
2668 00000150 03
2669
2670 00000151 0C
2671
2672
2673
2674
2675
2676
2677
2678
2679 00000152 00<rep AEh>
2680
2681 00000200 4A
2682 00000201 42
2683 00000202 E9FBFD
2684
2685 00000205 402349424D3A31322E-
2685 0000020E 30312E323030332E62-
2685 00000217 75696C645F312E3332-
2685 00000220 23402049424D42494F-
2685 00000229 2E434F4D2855534129-
2685 00000232 00
2686
2687
2688 00000233 00<rep 11Fh>
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715

secnt:      dw 0
            db 0          ; -- pad where hardnum was
dsktnum:    db 1          ; number of diskette drives

; (MSDOS 3.3) NOTE:
; Some of the older versions of the IBM rom-bios always assumed a seek would
; have to be made to read the diskette. Consequently a large head settle
; time was always used in the I/O operations. To get around this problem
; we need to continually adjust the head settle time. The following
; algorithm is used:
;
;   Get the current head settle value.
;   If it is 1, then
;   set slow = 15
;   else
;   set slow = value
;   ...
;   if we are seeking and writing then
;   use slow
;   else
;   use fast
;   ...
;   restore current head settle value

motorstartup: db 0          ; value from table
settlecurrent: db 0          ; value from table
settlestart:  db 0          ; slow settle value
nextspeed:    db 0          ; value of speed to be used
save_head_sttl: db 0        ; used by read_sector routine
save_eot:     db 0          ; saved eot from the default DPT
eot:          db 9
dpt:          dd 0          ; pointer to Disk Parameter Table
cursec:       db 0          ; current sector
curhd:        db 0          ; current head
curtrk:       dw 0          ; current track
spsav:        dw 0          ; save the stack pointer
format_eot:   db 8          ; eot used for format
hdnum:        db 0          ; head number
trknum:       dw 0          ; track being manipulated
gap_patch:    db 50h        ; format gap patched into dpt

;-----
; disk errors returned from the IBM rom
errin:        db 0CCh        ; write fault (hard disk)
            db 80h          ; write fault (hard disk)
            db 40h          ; seek failed
            db 10h          ; uncorrectable CRC or ECC error on read
            db 8            ; dma overrun
            db 6            ; disk changed (floppy)
            db 4            ; sector not found/read error
            db 3            ; disk write-protected
            ; 02/10/2023 - Retro DOS 5.0 (PCDOS 7.1 IBMBIO.COM)
            db 1            ; invalid function in AH or invalid parameter
            db 0B2h         ; volume not removable
;
lsterr:       db 0          ; all other errors

; returned error codes corresponding to above
errout:       db 10         ; write fault error
            db 2            ; no response (timeout)
            db 6            ; seek failure
            db 4            ; bad crc
            db 4            ; dma overrun
            db 15           ; invalid media change
            db 8            ; sector not found
            db 0            ; write attempt to write-protect disk
            ; 02/10/2023
            db 3            ; unknown command error
            db 3            ; unknown command error
            ;
            db 12           ; general error

;-----
; PCDOS 7.1 IBMBIO.COM - BIOSDATA:0152h
; 02/10/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
%if 1
disksector:  times 174 db 0
NUM174 equ 512-$
JB_sign      times NUM174 db 0
            :             ; dw 424Ah          ; 'BJ' (nasm) ; 'JB' (masm)
            dec dx
            inc dx
            jmp BData_start ; db 0E9h, 0FBh, 0FDh

IBMBIOCOM$:  db '@#IBM:12.01.2003.build_1.32#@ IBMBIO.COM(USA)',0

            times 287 db 0
            times (disksector+512-$) db 0 ; 287
%endif

;-----
; 30/12/2018 - Retro DOS v4.0
; read in boot sector here, read done in readboot.
; also read sector for dma check for hard disk.
;
; This buffer is word aligned because certain AMI BIOSs have a bug
; in them which causes the byte after the buffer to be trashed
; on floppy reads to odd-byte boundaries. Although no general effort
; is made to enforce this in the bigger picture, this one small sacrifice
; makes that system more-or-less work.
; 02/10/2023
%if 0
disksector:  db 512 dup(0) ; read in boot sector here
            ; 19/10/2022
            times 512 db 0
%endif

;-----
; 02/10/2023 - Retro DOS v5.0
; 30/12/2018 - Retro DOS v4.0

```



```

2716 ;-----
2717 ; 25/05/2018 (04/04/2018)
2718 ;*****
2719 ; "bds" contains information for each drive in the system.
2720 ; various values are patched whenever actions are performed.
2721 ; sectors/alloc. unit in bpb initially set to -1 to signify that
2722 ; the bpb has not been filled. link also set to -1 to signify end
2723 ; of list. # of cylinders in maxparms initialized to -1 to indicate
2724 ; that the parameters have not been set.
2725
2726 bds1: ;dw offset bds2
2727 00000352 [E803] dw bds2; 19/10/2022
2728 00000354 7000 dw 70h ; dwordlink tonext structure
2729 00000356 00 db 0 ; int 13h drive number
2730 00000357 00 db 0 ; logical drive letter
2731 00000358 0002 fdrive1: dw 512
2732 ; physical sector size in bytes
2733 0000035A FF db 0FFh ; sectors/allocation unit
2734 0000035B 0100 dw 1 ; reserved sectors for dos
2735 0000035D 02 db 2 ; no of file allocation tables
2736 0000035E 4000 dw 64 ; number of root directory entries
2737 00000360 6801 dw 360 ; number sectors (at 512 bytes each)
2738 00000362 00 db 0 ; mediadescriptor, initially 0
2739 00000363 0200 dw 2 ; number of fat sectors
2740 00000365 0900 dw 9 ; sector limit (sectors per track)
2741 00000367 0100 dw 1 ; head limit (number of heads - 1)
2742 ;
2743 ; 02/10/2023
2744 ; MSDOS 5.0-6.22 (& PC DOS 7.0)
2745 ;dw 0 ; hidden sector count (low word)
2746 ;dw 0 ; hidden sector (high)
2747 ;dw 0 ; number sectors (low)
2748 ;dw 0 ; number sectors (high)
2749 ;db 0 ; true => large fats
2750 ; 02/10/2023
2751 ; PC DOS 7.1 (FAT32 support)
2752 00000369 00000000 dd 0 ; hidden sector count
2753 0000036D 00000000 dd 0 ; number of sectors (32 bit)
2754 00000371 00000000 dd 0 ; BPB_FATSz32 ; FAT32 FAT size in sectors ; 4 bytes
2755 ; BS_DrvNum ; FAT INT 13h drive number ; 1 byte
2756 ; BS_Reserved1 ; FAT reserved byte = 0 ; 1 byte
2757 ; BS_BootSig ; FAT Extended boot signature = 29h ; 1 byte
2758 ; BS_VOLID ; FAT Volume serial number ; 4 bytes
2759 00000375 0000 dw 0 ; BPB_ExtFlags ; FAT32 Extended Flags
2760 00000377 0000 dw 0 ; BPB_FSVer ; FAT32 fs/volume version
2761 00000379 00000000 dd 0 ; BPB_RootClus ; FAT32 root directory's first cluster number
2762 0000037D FFFF dw 0FFFFh ; BPB_FSInfo ; FAT32 FSINFO sector number = -1 (initial)
2763 0000037F FFFF dw 0FFFFh ; BPB_BkBootSec ; FAT32 backup boot sector number = -1 (initial)
2764 00000381 00<rep ch> times 12 db 0 ; BPB_Reserved ; FAT32 reserved field = 0, 12 bytes
2765 0000038D 00 db 0 ; true => large fats
2766 ;
2767 0000038E 0000 dw 0 ; open ref. count
2768 00000390 03 db 3 ; form factor
2769 00000391 2000 dw 20h ; various flags
2770 00000393 2800 dw 40 ; number of cylinders
2771 00000395 0002 recommended_bps: dw 512 ; recommended bps for this drive
2772 00000397 01 db 1
2773 00000398 0100 dw 1
2774 0000039A 02 db 2
2775 0000039B E000 dw 224 ; number of root directory entries
2776 0000039D 6801 dw 360
2777 0000039F F0 db 0F0h ; mediadescriptor, initially 0F0h
2778 000003A0 0200 dw 2
2779 000003A2 0900 dw 9
2780 000003A4 0200 dw 2
2781 ;
2782 ; 02/10/2023
2783 ;dw 0
2784 ;dw 0
2785 ;dw 0
2786 ;dw 0
2787 ;db 6 dup(0)
2788 ;times 6 db 0 ; 19/10/2022
2789 000003A6 00000000 dd 0
2790 000003AA 00000000 dd 0
2791 000003AE 00000000 dd 0
2792 000003B2 0000 dw 0
2793 000003B4 0000 dw 0
2794 000003B6 00000000 dd 0
2795 000003BA FFFF dw 0FFFFh
2796 000003BC FFFF dw 0FFFFh
2797 ;db 12 dup(0)
2798 000003BE 00<rep ch> times 12 db 0 ; 02/10/2023
2799 ;
2800 000003CA FF db 0FFh ; last track accessed on this drive
2801 000003CB FFFF dw 0FFFFh ; keep these two contiguous (?)
2802 000003CD FFFF dw 0FFFFh
2803 000003CF 4E4F204E414D452020- db 'NO NAME ',0 ; volume id for this disk
2804 000003D8 202000 dw 2000
2805 000003DB 00000000 dd 0 ; current volume serial from boot record
2806 000003DF 464154313220202000 db 'FAT12 ',0 ; current file system id from boot record
2807 ; ----
2808 ; 02/10/2023
2809 ; PC DOS 7.1
2810 %if 1
2811
2812 bds2: ; 02/10/2023 - Retro DOS v5.0 (Modified PC DOS 7.1)
2813 000003E8 FFFF dw 0FFFFh ; -1
2814 000003EA 7000 dw 70h
2815 000003EC 00 db 0
2816 000003ED 00 db 0
2817 000003EE 0002 fdrive2: dw 512
2818 000003F0 FF db 0FFh
2819 000003F1 0100 dw 1
2820 000003F3 02 db 2
2821 000003F4 4000 dw 64
2822 000003F6 6801 dw 360
2823 000003F8 00 db 0
2824 000003F9 0200 dw 2
2825 000003FB 0900 dw 9
2826 000003FD 0100 dw 1
2827 000003FF 00000000<rep 5h> times 5 dd 0
2828 00000413 FFFFFFFF dd 0FFFFFFFh
2829 00000417 00000000<rep 3h> times 3 dd 0
2830 00000423 00 db 0
2831 00000424 0000 dw 0
2832 00000426 03 db 3
2833 00000427 2000 dw 20h
2834 00000429 2800 dw 40
2835
2836 0000042B 0002 recbpb2: dw 512
2837 0000042D 01 db 1
2838 0000042E 0100 dw 1

```

```

2839 00000430 02                db 2
2840 00000431 E000              dw 224
2841 00000433 6801              dw 360
2842 00000435 F0                db 0F0h
2843 00000436 0200              dw 2
2844 00000438 0900              dw 9
2845 0000043A 0200              dw 2
2846 0000043C 00000000<rep 5h>  times 5 dd 0
2847 00000450 FFFFFFFF          dd 0FFFFFFFh
2848 00000454 00000000<rep 3h>  times 3 dd 0
2849 00000460 FF                db 0FFh
2850 00000461 FFFFFFFF          dd 0FFFFFFFh
2851 00000465 4E4F204E414D452020-  db 'NO NAME ',0
2851 0000046E 202000
2852 00000471 00000000          dd 0
2853 00000475 464154313220202000  db 'FAT12 ',0
2854
2855
2856
2857
2858 ; 02/10/2023
2859 ; MSDOS 5.0 - 6.22 (& PC DOS 7.0)
2860 %if 0
2861
2862 bds2:                dw bds3
2863                dw 70h
2864                db 0
2865                db 0
2866 fdrive2:            dw 512
2867                db 0FFh, 1, 0, 2, 40h, 0, 68h, 1, 0, 2, 0, 9, 0, 1, 0
2868                db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h, 0
2869                db 0, 2, 1, 1, 0, 2, 0E0h, 0, 68h, 1, 0F0h, 2, 0, 9, 0
2870                db 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh
2871                db 0FFh, 0FFh, 0FFh, 0FFh, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh
2872                db 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h, 54h
2873                db 31h, 32h, 20h, 20h, 20h, 0
2874
2875 bds3:                dw bds4
2876                dw 70h
2877                db 0
2878                db 0
2879 fdrive3:            dw 512
2880                db 0FFh, 1, 0, 2, 40h, 0, 68h, 1, 0, 2, 0, 9, 0, 1, 0
2881                db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h, 0
2882                db 0, 2, 1, 1, 0, 2, 0E0h, 0, 68h, 1, 0F0h, 2, 0, 9, 0
2883                db 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh
2884                db 0FFh, 0FFh, 0FFh, 0FFh, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh
2885                db 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h, 54h
2886                db 31h, 32h, 20h, 20h, 20h, 0
2887
2888 ; ----
2889 bds4:                dw 0FFFFh
2890                dw 70h
2891                db 0
2892                db 0
2893 fdrive4:            dw 512
2894                db 0FFh, 1, 0, 2, 40h, 0, 68h, 1, 0, 2, 0, 9, 0, 1, 0
2895                db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h, 0
2896                db 0, 2, 1, 1, 0, 2, 0E0h, 0, 68h, 1, 0F0h, 2, 0, 9, 0
2897                db 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh
2898                db 0FFh, 0FFh, 0FFh, 0FFh, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh
2899                db 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h, 54h
2900                db 31h, 32h, 20h, 20h, 20h, 0
2901
2902 ;-----
2903
2904 sm92:                db 3                ; .spf
2905                db 9                ; .spt
2906                dw 112 ; 70h          ; .cdire
2907                dw 1440 ; 2*9*80      ; .csec
2908                db 2                ; .spau
2909                db 2                ; .thead
2910
2911 %endif
2912
2913 0000047E 00                keyrd_func: db 0
2914 0000047F 01                keysts_func: db 1
2915 00000480 00                printdev: db 0                ; printer device index
2916
2917 wait_count: ;dw 4 dup(50h)        ; retry counts for printers
2918 00000481 5000<rep 4h>        times 4 dw 50h        ; 19/10/2022
2919
2920 00000489 0000                daycnt: dw 0
2921 0000048B 00                t_switch: db 0                ; flag for updating daycnt
2922 0000048C 00                havecmosclock: db 0
2923 0000048D 13                base_century: db 19
2924 0000048E 50                base_year: db 80
2925
2926 0000048F 1F                month_tab: db 31
2927 00000490 1C                february: db 28 ; 08/08/2023
2928 00000491 1F1E1F1E1F1E1F1E-  db 31, 30, 31, 30, 31, 31, 30, 31, 30, 31
2928 0000049A 1F
2929
2930 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
2931 %if 0
2932 bintobcd: dw bin_to_bcd        ; points to bin_to_bcd proc in msinit
2933                dw 70h ; 17/10/2022
2934 daycnttoday: dw daycnt_to_day    ; points to daycnt_to_day in msinit
2935                dw 70h ; 17/10/2022
2936 %endif
2937
2938 0000049B 00                set_id_flag: db 0                ; flag for getbp routine
2939
2940 ; 02/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PC DOS 7.1)
2941 ;fat_12_id: db 'FAT12 ',0
2942 ;fat_16_id: db 'FAT16 ',0
2943 ;vol_no_name: db 'NO NAME ',0
2944 ;temp_h: dw 0                ; temporary for 32 bit calculation
2945
2946 0000049C 0000                start_sec_h: dw 0                ; starting sector number high word
2947 0000049E 0000                saved_word: dw 0                ; tempory saving place for a word
2948 000004A0 0000                multrk_flag: dw 0
2949 000004A2 00                ec35flag: db 0                ; flags for 3.5 inch disk drives
2950 000004A3 0000                vretry_cnt: dw 0
2951 000004A5 0000                soft_ecc_cnt: dw 0
2952 000004A7 00                multitrk_format_flag: db 0        ; multi track format request flag
2953 000004A8 0000                xfer_seg: dw 0                ; temp for transfer segment
2954
2955 ; variables for msdioc1.asm module
2956
2957 ; tracktable contains a 4-tuples (c,h,r,n) for each sector in a track
2958 ; c = cylinder number,h = head number,r = sector id,n = bytes per sector
2959 ; n bytes per sector
2960 ; --- -----

```



```

2961 ; 0 128
2962 ; 1 256
2963 ; 2 512
2964 ; 3 1024
2965
2966 ;max_sectors_curr_sup equ 63 ; current maximum sec/trk that
2967 ; ; we support (was 40 in dos 3.2)
2968
2969 000004AA 2400 sectorspertrack: dw 36
2970 000004AC 00000102 tracktable: db 0, 0, 1, 2
2971 000004B0 00000202 db 0, 0, 2, 2
2972 000004B4 00000302 db 0, 0, 3, 2
2973 000004B8 00000402 db 0, 0, 4, 2
2974 000004BC 00000502 db 0, 0, 5, 2
2975 000004C0 00000602 db 0, 0, 6, 2
2976 000004C4 00000702 db 0, 0, 7, 2
2977 000004C8 00000802 db 0, 0, 8, 2
2978 000004CC 00000902 db 0, 0, 9, 2
2979 000004D0 00000A02 db 0, 0, 10, 2
2980 000004D4 00000B02 db 0, 0, 11, 2
2981 000004D8 00000C02 db 0, 0, 12, 2
2982 000004DC 00000D02 db 0, 0, 13, 2
2983 000004E0 00000E02 db 0, 0, 14, 2
2984 000004E4 00000F02 db 0, 0, 15, 2
2985 000004E8 00001002 db 0, 0, 16, 2
2986 000004EC 00001102 db 0, 0, 17, 2
2987 000004F0 00001202 db 0, 0, 18, 2
2988 000004F4 00001302 db 0, 0, 19, 2
2989 000004F8 00001402 db 0, 0, 20, 2
2990 000004FC 00001502 db 0, 0, 21, 2
2991 00000500 00001602 db 0, 0, 22, 2
2992 00000504 00001702 db 0, 0, 23, 2
2993 00000508 00001802 db 0, 0, 24, 2
2994 0000050C 00001902 db 0, 0, 25, 2
2995 00000510 00001A02 db 0, 0, 26, 2
2996 00000514 00001B02 db 0, 0, 27, 2
2997 00000518 00001C02 db 0, 0, 28, 2
2998 0000051C 00001D02 db 0, 0, 29, 2
2999 00000520 00001E02 db 0, 0, 30, 2
3000 00000524 00001F02 db 0, 0, 31, 2
3001 00000528 00002002 db 0, 0, 32, 2
3002 0000052C 00002102 db 0, 0, 33, 2
3003 00000530 00002202 db 0, 0, 34, 2
3004 00000534 00002302 db 0, 0, 35, 2
3005 00000538 00002402 db 0, 0, 36, 2
3006
3007 ; 02/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PCDOS 7.1)
3008 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:053Ch
3009
3010 ;times 108 db 0 ; 19/10/2022
3011 ;;db 108 dup(0) ; 4*max_sectors_curr_sup - ($ - tracktable) dup (0)
3012 ; times ((4*63)-144) db 0
3013
3014 dskdrvs: dw fdrive1
3015 0000053E [EE03] dw fdrive2
3016
3017 ;dw 52 dup(0)
3018 00000540 00<rep 68h> times 104 db 0 ; times (((4*63)-144)-4) db 0
3019 ; 4*max_sectors_curr_sup-($-tracktable)-4 dup (0)
3020
3021 ;-----
3022
3023 ; this is a real ugly place to put this
3024 ; it should really go in the bds
3025
3026 000005A8 00 mediatype: db 0
3027 000005A9 00 media_set_for_format: db 0 ; 1 if we have done an int 13h set media
3028 ; type for format call
3029 000005AA 00 had_format_error: db 0 ; 1 if the previous format operation
3030 ; failed.
3031
3032 ; temp disk base table. it holds the the current dpt which is then replaced by
3033 ; the one passed by "new roms" before we perform a format operation. the old
3034 ; dpt is restored in restoreolddpt. the first entry (disk_specify_1) is -1 if
3035 ; this table does not contain the previously saved dpt.
3036
3037 000005AB FFFFFFFF tempdpt: dd 0FFFFFFFFh ; -1 ; temp disk base table
3038 000005AF FF model_byte: db 0FFh ; model byte set at init time
3039 000005B0 00 secondary_model_byte: db 0
3040
3041 000005B1 00 int19sem: db 0 ; indicate that all int 19h
3042 ; initialization is complete
3043
3044 ;; we assume the following remain contiguous and their order doesn't change
3045 ;i19_lst:
3046 ; irp aa,<02,08,09,0a,0b,0c,0d,0e,70,72,73,74,76,77>
3047 ; public int19old&aa
3048 ; db aa&h ; store the number as a byte
3049 ;int19old&aa dd -1 ; original hardware int. vectors for int 19h.
3050 ; endm
3051
3052 ; 21/10/2022
3053
3054 000005B2 02 i19_lst: db 2
3055 ; Int19old&aa
3056 000005B3 FFFFFFFF int19old02: dd 0FFFFFFFFh ; -1
3057 000005B7 08 db 8
3058 000005B8 FFFFFFFF int19old08: dd 0FFFFFFFFh ; original hardware int. vectors for int 19h
3059 000005BC 09 db 9
3060 000005BD FFFFFFFF int19old09: dd 0FFFFFFFFh
3061 000005C1 0A db 0Ah
3062 000005C2 FFFFFFFF int19old0A: dd 0FFFFFFFFh
3063 000005C6 0B db 0Bh
3064 000005C7 FFFFFFFF int19old0B: dd 0FFFFFFFFh
3065 000005CB 0C db 0Ch
3066 000005CC FFFFFFFF int19old0C: dd 0FFFFFFFFh
3067 000005D0 0D db 0Dh
3068 000005D1 FFFFFFFF int19old0D: dd 0FFFFFFFFh
3069 000005D5 0E db 0Eh
3070 000005D6 FFFFFFFF int19old0E: dd 0FFFFFFFFh
3071 000005DA 70 db 70h
3072 000005DB FFFFFFFF int19old70: dd 0FFFFFFFFh
3073 000005DF 72 db 72h
3074 000005E0 FFFFFFFF int19old72: dd 0FFFFFFFFh
3075 000005E4 73 db 73h
3076 000005E5 FFFFFFFF int19old73: dd 0FFFFFFFFh
3077 000005E9 74 db 74h
3078 000005EA FFFFFFFF int19old74: dd 0FFFFFFFFh
3079 000005EE 76 db 76h
3080 000005EF FFFFFFFF int19old76: dd 0FFFFFFFFh
3081 000005F3 77 db 77h
3082 000005F4 FFFFFFFF int19old77: dd 0FFFFFFFFh
3083
3084 ;num_i19 equ ($ - i19_lst)/5 ; 18/03/2019

```

```

3085
3086
3087
3088 ; 02/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PC DOS 7.1)
3089
3090 ; dskdrvs: dw fdrive1
3091 ; dw fdrive2
3092 ; dw fdrive3
3093 ; dw fdrive4
3094
3095 ; M011 -- made all hard drive stuff variable
3096 ; dw 22 dup(0) ; up to 26 drives for mini disks
3097 ; times 22 dw 0 ; 19/10/2022
3098
3099
3100
3101 ; 01/10/2022 - Retro DOS v4.0 (MSDOS v5.0 -actual-)
3102 ; 30/12/2018 - Retro DOS v4.0 (MSDOS v6.21 -draft-)
3103 ; 01/06/2018 - Retro DOS v3.0 (MSDOS v3.3)
3104
3105 ; variables for dynamic relocatable modules
3106 ; these should be stay resident.
3107
3108 000005F8 00000000 int6c_ret_addr: dd 0 ; return address from int 6ch
3109 ; for p12 machine
3110
3111 ; data structures for real-time date and time
3112
3113 000005FC 00000000 bin_date_time: db 0, 0, 0, 0 ; century, year, month, day
3114
3115 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3116 %if 0
3117 month_table: dw 0 ; january
3118 dw 31 ; february
3119 dw 59
3120 dw 90
3121 dw 120
3122 dw 151
3123 dw 181
3124 dw 212
3125 dw 243
3126 dw 273
3127 dw 304
3128 dw 334 ; december
3129 %endif
3130
3131 00000600 0000 daycnt2: dw 0
3132 ; 08/08/2023
3133 ; feb29: db 0 ; february 29 in a leap year flag
3134
3135
3136
3137 ; 01/10/2022 - (New/Actual) Retro DOS v4.0 (will run as MSDOS 5.0)
3138 ; by Erdogan Tan (Istanbul) ! free source code !
3139 ; 31/12/2018 - (old/draft) Retro DOS v4.0 (will/would run as MSDOS 6.21)
3140
3141
3142
3143
3144
3145 ; *****
3146 ; * Entry points into Bios_Code routines. The segment values *
3147 ; * are plugged in by seg_reinit. *
3148 ; * *
3149 ; *****
3150
3151 ; 01/10/2022 - Retro DOS v4.0 - IO.SYS (MSDOS v5.0)
3152 ; BIOSCODE_SEGMENT equ 2C7h
3153 ; BIOSDATA_SEGMENT equ 70h ; KERNEL_SEGMENT equ 70h
3154
3155 ; 01/10/2022 - Erdogan Tan
3156 ; (disassembled MSDOS 5.0 IO.SYS code here with fixed function/routine
3157 ; addresses, they will be changed to table labels later)
3158
3159 ; 09/12/2022
3160 %if 0
3161 cdev: dw 43h, 2C7h ; chardev_entry
3162 ; at 2C7h:43h = 70h:25B3h
3163 ; time_to_ticks
3164 ; at 2C7h:396h = 70h:2906h
3165 bcode_i2f: dw 1302h, 2C7h ; i2f_handler
3166 ; at 2C7h:1302h = 70h:3872h
3167 i13x: dw 154Bh, 2C7h ; i13z
3168 ; at 2C7h:154Bh = 70h:3ABBh
3169 %endif
3170
3171 ; 30/12/2022
3172 ; (IOSYSCODESEG is 2CCh for MSDOS 6.21 IO.SYS)
3173
3174 ; 02/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PC DOS 7.1)
3175 ; PC DOS 7.1 IBMBIO.COM - BIOSDATA:0602h
3176 ; (IOSYSCODESEG is 364h for PC DOS 7.1 IBMBIO.COM)
3177
3178 ; 09/12/2022
3179 00000602 [4700]2D03 cdev: dw chardev_entry, IOSYSCODESEG
3180 00000606 [A003]2D03 tticks: dw time_to_ticks, IOSYSCODESEG
3181 ; 07/08/2023
3182 0000060A [5718]2D03 bcode_i2f: dw i2f_handler, IOSYSCODESEG
3183 i13x: dw i13z, IOSYSCODESEG
3184
3185 end_BC_entries: ; 15/10/2022
3186
3187 ; *****
3188 ; * cbreak - break key handling - simply set altah=3 and iret *
3189 ; * *
3190 ; *****
3191
3192 cbreak: mov byte [cs:altah], 3 ; break key handling
3193 0000060E 2EC606[0C00]03 ; indicate break key set
3194
3195 intret: iret
3196 00000614 CF
3197
3198 ; ===== S U B R O U T I N E =====
3199
3200
3201 ; *****
3202 ; * strategy - store es:bx (device driver request packet) *
3203 ; * away at [ptrsav] for next driver function call *
3204 ; * *
3205 ; *****
3206
3207 strategy: ; proc far
3208

```

```

3209 00000615 2E891E[1200]      mov     [cs:ptrsav], bx ; store es:bx (device driver request packet)
3210                                ; away at [ptrsav] for next driver function call
3211 0000061A 2E8C06[1400]      mov     [cs:ptrsav+2], es
3212 0000061F CB                retf
3213
3214 ; -----
3215 ; *****
3216 ; *
3217 ; * device driver entry points. these are the initial *
3218 ; * 'interrupt' hooks out of the device driver chain. *
3219 ; * in the case of our resident drivers, they'll just *
3220 ; * stick a fake return address on the stack which *
3221 ; * points to dispatch tables and possibly some unit *
3222 ; * numbers, and then call through a common entry point *
3223 ; * which can take care of a20 switching *
3224 ; *
3225 ; *****
3226 ;
3227 ; 01/10/2022 - Erdogan Tan
3228 ; (disassembled MSDOS 5.0 IO.SYS code here with fixed table
3229 ; addresses, they will be changed to table labels later)
3230
3231 ; 09/12/2022
3232
3233 ; 02/10/2023 - Retro DOS v5.0
3234 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:0620h, BIOSCODE = 0364h)
3235
3236 con_entry:
3237     call     cdev_entry
3238 00000620 E84000
3239 ; -----
3240 ;dw 0E4h ; con_table
3241 00000623 [E400] dw con_table ; 364h:0E4h (PCDOS 7.1)
3242 ; 2C7h:0E4h = 70h:2654h
3243 ; -----
3244
3245 prn0_entry:
3246     call     cdev_entry
3247 ; -----
3248 ;dw 0FBh ; prn_table
3249 00000628 [FB00] dw prn_table ; 2C7h:0FBh = 70h:266Bh
3250 ;
3251 0000062A 0000 db 0, 0
3252 ; -----
3253
3254 prn1_entry:
3255     call     cdev_entry
3256 ; -----
3257 ;dw 0FBh ; prn_table
3258 0000062F [FB00] dw prn_table ; 2C7h:0FBh = 70h:266Bh
3259 ;
3260 00000631 0001 db 0, 1
3261 ; -----
3262
3263 prn2_entry:
3264     call     cdev_entry
3265 ; -----
3266 ;dw 0FBh ; prn_table
3267 00000636 [FB00] dw prn_table ; 2C7h:0FBh = 70h:266Bh
3268 ;
3269 00000638 0102 db 1, 2
3270 ; -----
3271
3272 prn3_entry:
3273     call     cdev_entry
3274 ; -----
3275 ;dw 0FBh ; prn_table
3276 0000063D [FB00] dw prn_table ; 2C7h:0FBh = 70h:266Bh
3277 ;
3278 0000063F 0203 db 2, 3
3279 ; -----
3280
3281 aux0_entry:
3282     call     cdev_entry
3283 ; -----
3284 ;dw 130h ; aux_table
3285 00000644 [3001] dw aux_table ; 2C7h:130h = 70h:26A0h
3286 ;
3287 00000646 00 db 0
3288 ; -----
3289
3290 aux1_entry:
3291     call     cdev_entry
3292 ; -----
3293 ;dw 130h ; aux_table
3294 0000064A [3001] dw aux_table ; 364h:130h = 70h:3070h (PCDOS 7.1)
3295 ; 2C7h:130h = 70h:26A0h
3296 0000064C 01 db 1
3297 ; -----
3298
3299 aux2_entry:
3300     call     cdev_entry
3301 ; -----
3302 ;dw 130h ; aux_table
3303 00000650 [3001] dw aux_table ; 2C7h:130h = 70h:26A0h
3304 ;
3305 00000652 02 db 2
3306 ; -----
3307
3308 aux3_entry:
3309     call     cdev_entry
3310 ; -----
3311 ;dw 130h ; aux_table
3312 00000656 [3001] dw aux_table ; 2C7h:130h = 70h:26A0h
3313 ;
3314 00000658 03 db 3
3315 ; -----
3316
3317 tim_entry:
3318     call     cdev_entry
3319 ; -----
3320 ;dw 147h ; tim_table
3321 0000065C [4701] dw tim_table ; 364h:147h = 70h:3087h (PCDOS 7.1)
3322 ; 2C7h:147h = 70h:26B7h
3323 ; -----
3324
3325 ; 15/10/2022
3326 ; DSKTBL equ dsktbl - DOSBIOSEG_2C7h ; dsktbl - 2C70h
3327 ; 09/12/2022
3328 DSKTBL equ dsktbl
3329
3330 dsk_entry:
3331     call     cdev_entry
3332 ; -----

```

```

3333             ;dw 4A2h             ; dsktbl
3334 00000661 [6F05]             dw DSKTBL             ; 09/12/2022
3335                                     ; 2C7h:4A2h = 70h:2A12h
3336                                     ; 02/10/2023 (PCDOS 7.1 IBMBIO.COM)
3337                                     ; 364h:579h = 70h:34B9h
3338
3339 ; ===== S U B   R O U T I N E =====
3340
3341 ;*****
3342 ;*
3343 ;* Ensure A20 is enabled before jumping into code in HMA.
3344 ;* This code assumes that if Segment of Device request packet is
3345 ;* DOS DATA segment then the Device request came from DOS & that
3346 ;* A20 is already on.
3347 ;*
3348 ;*****
3349
3350 cdev_entry: ; proc near
3351             cmp     byte [cs:inHMA],0
3352             jz      short ce_enter_codeseg
3353             ; optimized for DOS in HMA
3354             push    ax
3355             mov     ax,[cs:DosDataSg]
3356             cmp     [cs:ptrsav+2],ax
3357             pop     ax
3358             jnz     short not_from_dos
3359             ; jump is coded this way to fall thru
3360             ; in 99.99% of the cases
3361
3362 ce_enter_codeseg:
3363             jmp     far [cs:cdev]
3364             jmp     dword ptr cs:cdev
3365
3366 ;-----
3367 not_from_dos:
3368             call    EnsureA20on
3369             jmp     short ce_enter_codeseg
3370
3371 ;*****
3372 ;*
3373 ;* outchr - this is our int 29h handler. it writes the
3374 ;* character in al on the display using int 10h ttywrite
3375 ;*
3376 ;*****
3377 ; 17/07/2024
3378 ; 02/10/2023
3379 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:0682h
3380 outchr:
3381             push    ax             ; int 29h handler
3382             push    si
3383             push    di
3384             push    bp
3385             push    bx
3386             ;;;
3387             ; 02/10/2023 - Retro DOS v5.0 (Modified PCODOS 7.1)
3388             mov     ah,0Eh
3389             mov     bx,7
3390             int     10h             ; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
3391             ;
3392             ; AL = character, BH = display page (alpha modes)
3393             ; BL = foreground color (graphics modes)
3394             ; 17/07/2024
3395             ; 02/10/2023
3396             push    ds ; *
3397             xor     bx,bx ; 0
3398             mov     ds,bx ; 0
3399             mov     ah,0Eh
3400             mov     bl,7
3401             cmp     [cs:Iswin386], bl ; (are we in) windows ?
3402             ; 17/07/2024
3403             jnz     short win_outchr ; *
3404             push    ds ; *
3405             mov     ds,bx ; 0
3406             mov     ah,0Eh
3407             mov     bl,7
3408             jnz     short win_outchr ; Running on windows
3409             pushf
3410             cli     ; disable interrupts
3411             call    far [40h] ; far call (simulate INT)
3412             ; 17/07/2024
3413             pop     ds ; *
3414             jmp     short outchr_ok
3415 win_outchr:
3416             int     10h
3417 outchr_ok:
3418             ; 17/07/2024
3419             pop     ds ; *
3420             ;;;
3421             pop     bx
3422             pop     bp
3423             pop     di
3424             pop     si
3425             pop     ax
3426             iret
3427
3428 ;-----
3429 ; 02/10/2023 - Retro DOS v5.0
3430 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:06A8h
3431
3432             db 50h ; P             ; 'PCI' signature
3433             db 43h ; C
3434             db 49h ; I
3435
3436 orig1A:             dd 0
3437
3438 ; ===== S U B   R O U T I N E =====
3439
3440 ; 02/10/2023 - Retro DOS v5.0
3441 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:06AFh
3442
3443 int1A:
3444             cmp     ah,4             ; (Y2K-fix)
3445             je      short int1a_1 ; Read the date from the computer's real-time clock
3446             jmp     far [cs:orig1A]
3447
3448 int1a_1:
3449             push    bp
3450 int1a_2:
3451             mov     bp,sp
3452             push    bp
3453             pushf
3454             call    far [cs:orig1A]
3455             jc      short int1a_4
3456             ;cmp     cl,0             ; Year (BCD)

```

```

3457             ; 02/10/2023
3458 000006C5 08C9      or      cl,cl
3459 000006C7 7515      jnz     short int1a_3
3460 000006C9 80FD19     cmp     ch,19h      ; Century (BCD)
3461 000006CC 7510      jne     short int1a_3
3462 000006CE B520      mov     ch,20h
3463 000006D0 B405      mov     ah,5        ; Set the date on the computer's real-time clock
3464 000006D2 51        push    cx
3465 000006D3 52        push    dx      ; dh = Month (BCD), dl = Day (BCD)
3466 000006D4 9C        pushf
3467 000006D5 2EFF1E[AB06] call    far [cs:Orig1A]
3468 000006DA 5A        pop     dx
3469 000006DB 59        pop     cx
3470 000006DC 7207      jc      short int1a_4
3471
3472 000006DE 5D        pop     bp
3473 000006DF 806606FE and     byte [bp+6],0FEh ; clear carry flag
3474 000006E3 EB05      jmp     short int1a_5
3475
3476 000006E5 5D        pop     bp
3477 000006E6 804E0601 or      byte [bp+6],1 ; set carry flag
3478
3479 000006EA 5D        pop     bp
3480 000006EB CF        iret
3481
3482             ; 02/10/2023
3483 000006EC 90        nop      ; (not necessary, i have used this 'nop' to locate 'block13:'
3484             ; at BIOSDATA:06EDh, just as in the original PC DOS 7.1 IBMBIO.COM)
3485
3486 ;-----
3487
3488 ;*****
3489 ;*
3490 ;* block13 - our int13 hooker
3491 ;*
3492 ;*****
3493
3494             ; 02/10/2023 - Retro DOS v5.0
3495             ; PC DOS 7.1 IBMBIO.COM - BIOSDATA:06EDh
3496
3497 block13:
3498 000006ED 2E803E[0D00]00 cmp     byte [cs:inHMA],0
3499 000006F3 7403      jz      short skipa20
3500
3501             ; call IsA20Off ; A20 Off?
3502             ; jnz short skipa20
3503             ; call EnableA20 ; assure a20 enabled
3504             ; 02/10/2023 - Retro DOS v5.0 (Modified PC DOS 7.1)
3505 000006F5 E83200     call    EnsureA20On ; assure a20 enabled
3506
3507 skipa20:
3508 000006F8 2E8C1E[1C00] mov     [cs:i13_ds],ds ; save caller's ds for call-through
3509 000006FD 9C        pushf    ; fake interrupt
3510 000006FE 2EFF1E[0A06] call    far [cs:i13x]
3511             ; call dword ptr cs:i13x ; call through Bios_Code entry table
3512 00000703 2E8E1E[1C00] mov     ds,[cs:i13_ds]
3513 00000708 CA0200     retf     2
3514
3515 ; ===== S U B R O U T I N E =====
3516
3517 ; the int13 hook calls back here to call-through to the ROM
3518 ; this is necessary because some people have extended their
3519 ; ROM BIOSs to use ds as a parameter/result register and
3520 ; our int13 hook relies heavily on ds to access Bios_Data
3521
3522 call_orig13:
3523             ; proc far
3524 0000070B 8E1E[1C00] mov     ds,[i13_ds] ; get caller's ds register
3525 0000070F 9C        pushf    ; simulate an int13
3526 00000710 2EFF1E[B400] call    far [cs:Orig13]
3527             ; call cs:Orig13
3528 00000715 2E8C1E[1C00] mov     [cs:i13_ds],ds
3529 0000071A 0E        push     cs
3530 0000071B 1F        pop      ds      ; restore ds -> Bios_Data before return
3531
3532             pushf
3533             ; 10/12/2022
3534             ; ds = cs
3535 0000071D 803E[0D00]00 cmp     byte [inHMA],0 ; 16/10/2022
3536 00000722 7403      jz      short corig13_popf_ret
3537             ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3538             ; PC DOS 7.1 IBMBIO.COM - BIOSDATA:0725h
3539             ; call IsA20Off
3540             ; jnz short corig13_popf_ret
3541             ; call EnableA20
3542 00000724 E80300     call    EnsureA20On ; 07/08/2023
3543
3544 corig13_popf_ret:
3545             popf
3546             ; 20/09/2023
3547 re_init:         ; 07/08/2023
3548             retf
3549
3550             ; 02/10/2023
3551 00000729 90        nop      ; (not necessary, i have used this 'nop' to locate 'EnsureA20On:'
3552             ; at BIOSDATA:072Ah, just as in the original PC DOS 7.1 IBMBIO.COM)
3553
3554 ;-----
3555
3556 ; BIOSDATA:07BBh (MSDOS 6.21, IO.SYS)
3557 ; BIOSDATA:07BBh (MSDOS 5.0, IO.SYS) ; 16/10/2022
3558
3559 ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3560 ; HiMem: dd 0FFFF0090h
3561 ; LoMem: dd 80h
3562
3563 ; -----
3564
3565 ; ===== S U B R O U T I N E =====
3566
3567 ;*****
3568 ;*
3569 ;* EnsureA20On - ensure that a20 is enabled if we're running
3570 ;* in the HMA before interrupt entry points into Bios_Code
3571 ;*
3572 ;*****
3573
3574 EnsureA20On:
3575 0000072A E80E00     ; proc near
3576             call    IsA20Off
3577             ; jz short EnableA20
3578             ; ret
3579             ; 18/12/2022
3580             jnz     short A20On_retn

```

```

3581 ; ===== S U B   R O U T I N E =====
3582
3583
3584 EnableA20: ; proc near
3585             push    ax
3586             push    bx
3587             mov     ah,5      ; local enable a20
3588             ;call   cs:xms
3589             call    far [cs:xms] ; 16/10/2022
3590             pop     bx
3591             pop     ax
3592 A20on_retn: ; 18/12/2022
3593             retn
3594
3595 ; ===== S U B   R O U T I N E =====
3596
3597
3598 IsA200ff:   ; proc near
3599             push    ds
3600             push    es
3601             push    cx
3602             push    si
3603             push    di
3604             ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3605             ;lds    si,[cs:HiMem]
3606             ;les    di,[cs:LoMem]
3607             ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:0740h
3608             xor     di,di
3609             mov     es,di
3610             dec     di
3611             mov     si,90h ; 0FFFFh:0090h ; HiMem
3612             mov     ds,di
3613             mov     di,80h ; 0000h:0080h ; LoMem
3614             ; 02/10/2023 - Retro DOS v5.0 IBMBIO.COM (PCDOS 7.1)
3615             ; (following cpu instructions will be modified by 'SYSIN'
3616             ; if the cpu is a 386/32bit, for checking A20 line faster)
3617 cpu386_cmpsd:
3618             nop
3619             mov     cx,8
3620             repe    cmpsw
3621
3622             ; zf = 0 -> A20 line is ON
3623             ; zf = 1 -> A20 line is OFF
3624             pop     di
3625             pop     si
3626             pop     cx
3627             pop     es
3628             pop     ds
3629             retn
3630
3631 ; -----
3632 ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3633 %if 0
3634 DisableA20:
3635             push    ax
3636             push    bx
3637             mov     ah,6      ; local disable A20
3638             call    far [cs:xms]
3639             ;call   cs:xms
3640             pop     bx
3641             pop     ax
3642             retn
3643 %endif
3644
3645 ; -----
3646
3647 ;*****
3648 ;*
3649 ;*  int19 - bootstrap interrupt -- we must restore a bunch of the
3650 ;*          interrupt vectors before resuming the original int19 code
3651 ;*
3652 ;*****
3653
3654             ; 02/10/2023 - Retro DOS v5.0
3655             ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:0759h
3656 int19:
3657             push    cs
3658             pop     ds
3659             ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3660             ;mov     es,[zeroseg] ; 16/10/2022
3661             ;mov     cx,5          ; NUMROMVECTORS
3662             xor     cx,cx
3663             mov     es,cx
3664             mov     cl,5
3665             ;mov     si,offset RomVectors
3666             mov     si,RomVectors ; 19/10/2022
3667 next_int:
3668             lodsb                    ; get int number
3669             cbw                     ; assume < 128
3670             shl     ax,1
3671             shl     ax,1             ; int * 4
3672             ; 07/08/2023
3673             ;mov     di,ax
3674             ;lodsw
3675             ;stosw
3676             ;lodsw
3677             ;stosw                    ; install the saved vector
3678             ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:076Ah
3679             xchg    ax,di
3680             movsw
3681             movsw
3682             loop    next_int
3683             ;cmp     byte [int19sem], 0 ; 19/10/2022
3684             cmp     [int19sem], cl ; 0 ; 07/08/2023
3685             jz      short doint19
3686             mov     si,i19_1st      ; stacks code has changed these hardware interrupt vectors
3687                                     ; stkinit in sysinit1 will initialize int19oldxx values
3688             ;mov     cx,14
3689             ; 07/08/2023
3690             mov     cl,14
3691 i19_restore_loop:
3692             lodsb                    ; get interrupt          number
3693             cbw                     ; assume < 128
3694             ;mov     di,ax
3695             ;lodsw
3696             ;mov     bx,ax            ; get original vector offset
3697             ;lodsw                    ; save it
3698             ; 07/08/2023
3699             xchg    ax,di
3700             lodsw
3701             xchg    ax,bx
3702             lodsw
3703             ;cmp     bx,0FFFFh      ; check for 0ffffh (unlikely segment)
3704             inc     bx ; 07/08/2023

```



```

3705 00000781 7409          jz      short i19_restor_1 ; opt no need to check selector too
3706                      ;cmp    ax,0FFFFh ; opt 0ffffh is unlikely offset
3707                      ;jz      short i19_restor_1
3708 00000783 4B             dec     bx ; 07/08/2023
3709 00000784 01FF          add     di,di
3710 00000786 01FF          add     di,di
3711 00000788 93             xchg    ax,bx
3712 00000789 AB             stosw
3713 0000078A 93             xchg    ax,bx
3714 0000078B AB             stosw ; put the vector back
3715
i19_restor_1:
3716 0000078C E2EC          loop    i19_restore_loop
3717
doint19:
3718                      ;cmp    byte [inhMA],0 ; ; Is dos running from HMA
3719 0000078E 380E[0D00]      cmp     [inhMA],cl ; 0 ; 07/08/2023
3720 00000792 7403          jz      short SkipVdDisk
3721 00000794 E82A00          call    EraseVdDiskHead ; Then erase our VDISK header at 1MB boundary
3722                      ; Some m/c's (AST 386 & HP QS/16 do not clear
3723                      ; the memory above 1MB during a warm boot.
3724
SkipVdDisk:
3725 00000797 CD19          int     19h ; DISK 800T
3726                      ; causes reboot of disk system
3727
; ===== S U B R O U T I N E =====
3728
; -----
3729
; procedure : int15
3730
;
; Int15 handler for recognizing ctrl-alt-del seq
3731
; If it recognizes ctrl-alt-del and if DOS was
3732
; is running high, it Erases the VDISK header
3733
; present at 1MB boundary
3734
; -----
3735
; 16/10/2022
3736
; DELKEY equ 53h
3737
; ROMDATASEG equ 40h
3738
; KBFLAG equ 17h
3739
; CTRLSTATE equ 04h
3740
; ALTSTATE equ 08h
3741
; 02/10/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
3742
Int15:
3743
;cmp    ax,4F00h+DELKEY
3744
;cmp    ax,4F53h ; del keystroke ?
3745
; 02/10/2023 - Retro DOS v5.0
3746
; 07/08/2023
3747
;jz      short int15_1
3748
;jnz     short Old15_j ; 07/08/2023
3749
Old15_j:
3750
;jmp     far [cs:Old15] ; 16/10/2022
3751
; -----
3752
; int15_1:
3753
; push    ds
3754
; push    ax
3755
; ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3756
; ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:07A5h
3757
; ;mov    ax,40h ; ROMDATASEG
3758
; ;mov    ds,ax
3759
; ;mov    al,ds:17h ; [KBFLAG]
3760
; ; 16/10/2022
3761
; ;mov    al,[KBFLAG]
3762
; xor     ax,ax
3763
; mov     ds,ax
3764
; mov     al,[0417h] ; KBFLAG = 0417h (PCDOS 7.1 IBMBIO.COM)
3765
; and     al,0Ch ; (CTRLSTATE | ALTSTATE)
3766
; cmp     al,0Ch ; (CTRLSTATE | ALTSTATE)
3767
; jnz     short int15_2
3768
; ; 07/08/2023
3769
; ;push    cs
3770
; ;pop     ds
3771
; ;cmp    byte [inhMA],0 ; is DOS running from HMA
3772
; ;cmp    byte [cs:inhMA],ah ; 0
3773
; ;jz      short int15_2
3774
; ;call    EraseVdDiskHead
3775
int15_2:
3776
; pop     ax
3777
; pop     ds
3778
; stc
3779
; ; 02/10/2023 - Retro DOS v5.0
3780
; ;jmp     short Old15_j
3781
; ; 02/10/2023
3782
; ; 07/08/2023
3783
; ;jmp     far [cs:Old15] ; 16/10/2022
3784
; ;jmp     cs:Old15
3785
; -----
3786
; ===== S U B R O U T I N E =====
3787
; -----
3788
; procedure : EraseVdDiskHead
3789
;
; Erases the VDisk Header present in the 1MB boundary
3790
; -----
3791
EraseVdDiskHead:
3792
; proc near
3793
; ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3794
; ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:07C1h
3795
; ;push    ax
3796
; ;push    cx
3797
; ;push    di
3798
; ;push    es
3799
; ;call    EnsureA20On
3800
; ;mov     ax,0FFFFh ; HMA seg
3801
; ;mov     es,ax
3802
; ; 03/10/2023 - Retro DOS v5.0
3803
; ;push    0FFFFh
3804
; ;pop     es
3805
; ;mov     di,10h ; point to VDISK header
3806
; ; 07/08/2023
3807
; ;mov     cx,10h ; size of vdisk header
3808
; ;mov     cx,di ; 16
3809
; ; 03/10/2023
3810
; xor     ax,ax
3811
; inc     ax ; ax = 0
3812
; rep stosw ; clear it
3813
; pop     es
3814
; pop     di
3815
; pop     cx

```

```

3829                                     ;pop    ax ; 07/08/2023
3830 000007D6 C3                       ;retn
3831
3832 ; -----
3833
3834 ; 03/10/2023 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMBIO.COM)
3835 ; 17/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
3836
3837 ; 09/12/2022
3838 ;SYSINITSEG equ 46Dh ; SYSINIT segment
3839 ;DOSLOADSEG equ 83Fh ; MSDOS.SYS (kernel) loading segment
3840 ; (followings are in sysinit segment)
3841 ;FtryToMovDOSHI equ 0A84h ; (procedure in SYSINIT segment)
3842 FTRYTOMOVDOshi equ FtryToMovDOSHI ; SYSINIT section
3843 ;DEVICELIST equ 273h
3844 DEVICELIST equ DEVICE_LIST ; SYSINIT section
3845 ;MEMORYSIZE equ 292h
3846 MEMORYSIZE equ MEMORY_SIZE ; SYSINIT section
3847 ;DEFAULTDRIVE equ 296h
3848 DEFAULTDRIVE equ DEFAULT_DRIVE ; SYSINIT section
3849 ;currentdoslocation equ 271h
3850 CURRENTDOSLOCATION equ 271h
3851 CURRENTDOSLOCATION equ CURRENT_DOS_LOCATION ; SYSINIT section
3852 ;SYSINITSTART equ 267h
3853 SYSINITSTART equ SYSINIT ; SYSINIT section
3854 ; 18/10/2022
3855 ;toomanydrivesflag equ 3FFh
3856 TOOMANYDRIVESFLAG equ toomanydrivesflag ; SYSINIT section
3857
3858 ; -----
3859
3860 ; 03/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PC DOS 7.1)
3861 ; PC DOS 7.1 IBMBIO.COM - BIOSDATA:07D7h
3862
3863 %if 1
3864
3865 000007D7 FFFF                       FreeHMAptr: dw 0FFFFh
3866                                     ;MovedOSIntoHMA: dd 46D0A84h ; FtryToMovDOSHI
3867                                     ; (procedure in SYSINIT segment)
3868
3869 ; 17/10/2022
3870 000007D9 [E20B]                     MovedOSIntoHMA: dw FTRYTOMOVDOshi ; 09/12/2022
3871 000007DB 0405                       dw SYSINITSEG ; 08/08/2023
3872                                     ; 0544h for PC DOS 7.1 IBMBIO.COM
3873                                     ; 0473h for MSDOS 6.21 IO.SYS
3874
3875 ;SR;
3876 ; A communication block has been setup between the DOS and the BIOS. All
3877 ; the data starting from SysinitPresent will be part of the data block.
3878 ; Right now, this is the only data being communicated. It can be expanded
3879 ; later to add more stuff
3880
3881 SysinitPresent: db 0
3882
3883 %endif
3884
3885 ; -----
3886
3887 ;*****
3888 ;*
3889 ;* the int2f handler chains up to Bios_Code through here.
3890 ;* it returns through one of the three functions that follow.
3891 ;* notice that we'll assume we're being entered from DOS, so
3892 ;* that we're guaranteed to be A20 enabled if needed
3893 ;*
3894 ;*****
3895
3896 ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3897 %if 0 ; 20/09/2023
3898 int_2f:
3899     jmp far [cs:bcode_i2f] ; 16/10/2022
3900     ;jmp dword ptr cs:bcode_i2f ; far [cs:bcode_i2f]
3901
3902 ; -----
3903
3904 ; re-enter here to transition out of hma mode and jmp to dsk_entry
3905 ; note: is it really necessary to transition out and then back in?
3906 ; It's not as if this is a really speed critical function.
3907 ; might as well do whatever's most compact.
3908
3909 i2f_dskentry:
3910     jmp dsk_entry
3911
3912 ; -----
3913
3914 ;*****
3915 ;*
3916 ;* re_init - called back by sysinit after a bunch of stuff
3917 ;* is done. presently does nothing. affects no
3918 ;* registers!
3919 ;*
3920 ;*****
3921
3922 ; 09/12/2022
3923 ; re_init_:
3924 re_init:
3925     retf ; called back by sysinit after
3926         ; a bunch of stuff is done.
3927         ; presently does nothing
3928
3929 %endif
3930
3931 ; -----
3932
3933 ;SR; WIN386 support
3934
3935 ; WIN386 instance data structure
3936
3937 ; Here is a win386 startup info structure which we set up and to which
3938 ; we return a pointer when win386 initializes.
3939
3940 win386_SI: db 3,0 ; SI_Version
3941             ; Startup Info for win386
3942 SI_Next: dd 0 ; pointer to next info structure
3943          dd 0 ; a field we don't need
3944          dd 0 ; another field we don't need
3945 SI_Instance: dw Instance_Table
3946             dw 70h ; Bios_Data ; far pointer to instance table
3947
3948 ; This table gives win386 the instance data in the BIOS and ROM-BIOS data
3949 ; areas. Note that the address and size of the hardware stacks must
3950 ; be calculated and inserted at boot time.
3951
3952 Instance_Table: dw 0,50h ; printscreen status...
3953                dw 2 ; ... 2 bytes
3954                dw 0Eh,50h ; ROM Basic data...
3955                dw 14h ; ... 14H bytes
3956                dw altah ; a condevice buffer...

```



```

3953 000007FE 7000          dw 70h          ; Bios_Data segment
3954 00000800 0100          dw 1           ; ... 1 byte
3955
3956 NextStack:
3957
3958 ; NOTE: If stacks are disabled by STACKS=0,0, the following
3959 ; instance items WILL NOT be filled in by SYSINIT.
3960 ; That's just fine as long as these are the last items
3961 ; in the instance list since the first item is initialized
3962 ; to 0000 at load time.
3963
3964 00000802 00000000          dw 0,0          ; pointer to next stack      to be used...
3965 00000806 0200          dw 2           ; ... 2 bytes
3966 00000808 00000000 IT_StackLoc: dd 0          ; location of hardware stacks
3967 0000080C 0000 IT_StackSize: dw 0          ; size of hardware stacks
3968 0000080E 00000000          dd 0           ; terminate the instance table
3969
3970 ;SR;
3971 00000812 00 Iswin386: db 0          ; Flag to indicate whether
3972 ; Win386 is running or not
3973 ;-----
3974
3975 ; 09/12/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PC DOS 7.1)
3976 ; PC DOS 7.1 IBMBIO.COM - BIOSDATA:0813h
3977
3978 ;This routine was originally in BIOS_CODE but this causes a lot of problems
3979 ;when we call it including checking of A20. The code being only about
3980 ;30 bytes, we might as well put it in BIOS_DATA
3981
3982 V86_Crit_SetFocus:
3983 00000813 57          push di
3984 00000814 06          push es
3985 00000815 53          push bx
3986 00000816 50          push ax
3987 00000817 31FF        xor di,di
3988 00000819 8EC7        mov es,di
3989 0000081B BB1500        mov bx,15h          ; Device ID of DOSMGR device
3990 0000081E B88416        mov ax,1684h         ; Get API entry point
3991 00000821 CD2F        int 2Fh          ; - Multiplex - MS WINDOWS - GET DEVICE API ENTRY POINT
3992 ; BX = virtual device (VxD) ID, ES:DI = 0000h:0000h
3993 ; Return: ES:DI -> VxD API entry point, or 0:0 if the VxD does not
support an API
3994 00000823 8CC0        mov ax, es
3995 00000825 09F8        or ax, di
3996 00000827 740A        jz short Skip      ; Here, es:di is address of API routine.
3997 ; Set up stack frame to simulate a call.
3998 00000829 0E          push cs
3999 ;mov ax,offset Skip
4000 ;mov ax,Skip
4001 ;push ax
4002 ; 03/10/2023 - Retro DOS v5.0
4003 0000082A 68[3308]    push Skip
4004 0000082D 06          push es
4005 0000082E 57          push di          ; API far call address
4006 0000082F B80100        mov ax,1          ; SetFocus function number
4007 00000832 CB          retf          ; do the call
4008 ;-----
4009
4010 Skip:
4011 00000833 58          pop ax
4012 00000834 5B          pop bx
4013 00000835 07          pop es
4014 00000836 5F          pop di
4015 00000837 CB          retf
4016
4017 ;End WIN386 support
4018
4019 ; -----
4020
4021 ; 03/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PC DOS 7.1)
4022 %if 0
4023
4024 FreeHMAPtr: dw 0FFFFh
4025 ;MoveDOSIntoHMA: dd 46D0A84h          ; FTryToMovDOSHi
4026 ; (procedure in SYSINIT segment)
4027
4028 ; 17/10/2022
4029 MoveDOSIntoHMA: dw FTRYTOMOVDOshi          ; 09/12/2022
4030 dw SYSINITSEG          ; 08/08/2023
4031 ; 0544h for PC DOS 7.1 IBMBIO.COM
4032 ; 0473h for MSDOS 6.21 IO.SYS
4033
4034 ;SR;
4035 ; A communication block has been setup between the DOS and the BIOS. All
4036 ; the data starting from SysinitPresent will be part of the data block.
4037 ; Right now, this is the only data being communicated. It can be expanded
4038 ; later to add more stuff
4039
4040 SysinitPresent: db 0
4041
4042 endfloppy: db 0, 0
4043
4044 %endif
4045
4046 ; 03/10/2023 - Retro DOS v5.0 IBMBIO.COM
4047 00000838 00          endfloppy:
4048 db 0
4049
4050 ; 03/10/2023
4051 numxdiv equ ($-BData_start)
4052 numxmod equ (numxdiv % 16)
4053
4054 %if (numxmod>0) & (numxmod<16)
4055 00000839 00<rep 7h> times (16-numxmod) db 0
4056 %endif
4057
4058 ; -----
4059
4060 ; Bios_Data ends
4061
4062 ; Possibly disposable BIOS data
4063 ; This data follows the regular BIOS data,
4064 ; and is part of the same group.
4065
4066 ; 03/10/2023 - Retro DOS v5.0 IBMBIO.COM
4067 nul_vid: db 'NO NAME',0          ; null volume id
4068 tmp_vid: db 'NO NAME',0          ; vid scratch buffer
4069
4070 ; 03/10/2023
4071 00000840 4E4F204E414D452020- tmp_vid: db 'NO NAME'
4071 00000849 2020
4072
4073 0000084B 80          harddrv: db 80h
4074

```

```

4075 end96tpi:
4076
4077 ; 03/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PC DOS 7.1)
4078 ; PC DOS 7.1 IBMBIO.COM - BIOS DATA:084Ch ('bdss:' address)
4079
4080 ;*****
4081 ;memory allocation for bdss
4082 ;*****
4083
4084 ;max_mini_dsk_num equ 23 ; max # of mini disk ibmbio can support
4085
4086 ;bdss BDS_STRUC (2+max_mini_dsk_num) dup (<>) ; currently max. 25
4087
4088 ;bdss: times BDS.size*(2+max_mini_dsk_num) db 0
4089
4090
4091 ; 09/12/2023
4092 %if 1
4093 ; Retro DOS v5.0 (PC DOS 7.1) IBMBIO.COM BDS structure (FAT32 adaptation)
4094
4095 0000084C FFFF bdss: dw 0FFFFh ; max_mini_dsk_num equ 23
4096 ; BDS_STRUC (2+max_mini_dsk_num) dup (<>)
4097 ; currently max. 25
4098 ; (MSDOS 6 BDS structure size = 100 bytes)
4099 ; (PC DOS 7.1 BDS structure size = 150 bytes)
4100 ; BDS.link
4101
4102 0000084E 0000 dw 0 ; BDS.drivenum
4103 00000850 50 db 80 ; BDS.drivelet
4104 00000851 03 db 3 ; BDS.BPB (BDS offset 6)
4105 00000852 0002 dw 512 ; 53 bytes BPB for FAT32 fs
4106 ; 25 bytes BPB for FAT16 and FAT12 fs
4107 ; .bytespersec
4108
4109 00000854 01 db 1 ; .secpersclus
4110 00000855 0100 dw 1 ; .resectors
4111 00000857 02 db 2 ; .fats
4112 00000858 1000 dw 16 ; .direntries
4113 0000085A 0000 dw 0 ; .totalsec16
4114 0000085C F8 db 0F8h ; .media
4115 0000085D 0100 dw 1 ; .fatsecs16
4116 0000085F 0000 dw 0 ; .secperstrack
4117 00000861 0000 dw 0 ; .heads
4118 00000863 00000000 dd 0 ; .hiddensectors
4119 00000867 00000000 dd 0 ; .totalsecs32
4120 ; (End of FAT12/FAT16 BPB)
4121
4122 ; FAT32 extensions to BDS
4123 0000086B 00000000 dd 0 ; .fatsecs32 ; BPB_FATSz32 (BDS offset 31)
4124 0000086F 0000 dw 0 ; .extflags ; BPB_ExtFlags
4125 00000871 0000 dw 0 ; .fsver ; BPB_FSVer
4126 00000873 00000000 dd 0 ; .rootdirclust ; BPB_RootClus (BDS offset 39)
4127 00000877 FFFF dw 0FFFFh ; .fsinfo ; BPB_FSInfo ; initialized to -1
4128 00000879 FFFF dw 0FFFFh ; .bkbootsec ; BPB_BkBootSec ; initialized to -1
4129 0000087B 00<rep ch> times 12 db 0 ; .reserved ; BPB_Reserved (12 zero bytes)
4130 00000887 00 db 0 ; BDS.fatsiz (BDS offset 59)
4131 00000888 0000 dw 0 ; BDS.opcnt
4132 0000088A 03 db 3
4133 0000088B 2000 dw 20h ; BDS.flags (BDS offset 63)
4134 0000088D 2800 dw 40
4135 0000088F 00<rep 25h> times 37 db 0
4136 000008B4 FFFFFFFF dd 0FFFFFFFh
4137 000008B8 00<rep ch> times 12 db 0
4138 000008C4 FF db -1 ; BDS.track (BDS offset 120)
4139 000008C5 0100 dw 1 ; BDS.tim_lo ; BDS.bdsminismini
4140 000008C7 0000 dw 0 ; BDS.tim_hi
4141 000008C9 4E4F204E414D452020- db 'NO NAME ',0 ; BDS.vol_id
4142 000008D2 202000
4143 000008D5 00000000 dd 0 ; BDS.vol_serial (BDS offset 137)
4144 000008D9 464154313220202000 db 'FAT12 ',0 ; BDS.filesys_id
4145 000008E2 FFFF
4146 000008E4 000050030002010100- bds_1: db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
4147 000008ED 02100000000F8
4148 000008F3 010000000000000000- db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4149 000008FC 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4150 00000905 0000000000000000FF- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4151 0000090E FFFFFFFF0000
4152 00000913 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
4153 0000091C 0000000003200028
4154 00000924 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4155 0000092D 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4156 00000936 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4157 0000093F 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4158 00000948 0000FFFFFFF0000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4159 00000951 000000000000
4160 00000956 00000000FF01000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
4161 0000095F 4E4F204E41
4162 00000964 4D4520202020000000- db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 0, 0, 46h, 41h
4163 0000096D 00004641
4164 00000971 5431322020202000 bds_2: db 54h, 31h, 32h, 20h, 20h, 20h, 0
4165 00000978 FFFF dw 0FFFFh
4166 0000097A 000050030002010100- db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
4167 00000983 02100000000F8
4168 00000989 010000000000000000- db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4169 00000992 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4170 0000099B 0000000000000000FF- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4171 000009A4 FFFFFFFF0000
4172 000009A9 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
4173 000009B2 0000000003200028
4174 000009BA 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4175 000009C3 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4176 000009CC 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4177 000009D5 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4178 000009DE 0000FFFFFFF0000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4179 000009E7 000000000000
4180 000009EC 00000000FF01000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
4181 000009F5 4E4F204E41
4182 000009FA 4D4520202020000000- db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 0, 0, 46h, 41h
4183 00000A03 00004641
4184 00000A07 5431322020202000 bds_3: db 54h, 31h, 32h, 20h, 20h, 20h, 0
4185 00000A0E FFFF dw 0FFFFh
4186 00000A10 000050030002010100- db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
4187 00000A19 02100000000F8
4188 00000A1F 010000000000000000- db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4189 00000A28 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4190 00000A31 0000000000000000FF- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4191 00000A3A FFFFFFFF0000
4192 00000A3F 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
4193 00000A48 0000000003200028
4194 00000A50 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4195 00000A59 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4196 00000A62 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4197 00000A6B 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4198 00000A74 0000FFFFFFF0000000-

```

```

4173 00000A7D 0000000000
4174 00000A82 00000000FF01000000-
4174 00000A8B 4E4F204E41
4175 00000A90 4D4520202020000000-
4175 00000A99 00004641
4176 00000A9D 54313220202000
4177 00000AA4 FFFF
4178 00000AA6 000050030002010100-
4178 00000AAF 0210000000F8
4179 00000AB5 010000000000000000-
4179 00000ABE 000000000000000000
4180 00000AC7 0000000000000000FF-
4180 00000AD0 FFFFFFF0000
4181 00000AD5 000000000000000000-
4181 00000ADE 0000000003200028
4182 00000AE6 000000000000000000-
4182 00000AEF 000000000000000000
4183 00000AF8 000000000000000000-
4183 00000B01 000000000000000000
4184 00000B0A 0000FFFFFFFFF0000000-
4184 00000B13 00000000000
4185 00000B18 00000000FF01000000-
4185 00000B21 4E4F204E41
4186 00000B26 4D4520202020000000-
4186 00000B2F 00004641
4187 00000B33 54313220202000
4188 00000B3A FFFF
4189 00000B3C 000050030002010100-
4189 00000B45 0210000000F8
4190 00000B48 010000000000000000-
4190 00000B54 000000000000000000
4191 00000B5D 0000000000000000FF-
4191 00000B66 FFFFFFF0000
4192 00000B6B 000000000000000000-
4192 00000B74 0000000003200028
4193 00000B7C 000000000000000000-
4193 00000B85 000000000000000000
4194 00000B8E 000000000000000000-
4194 00000B97 000000000000000000
4195 00000BA0 0000FFFFFFFFF0000000-
4195 00000BA9 00000000000
4196 00000BAE 00000000FF01000000-
4196 00000BB7 4E4F204E41
4197 00000BBC 4D4520202020000000-
4197 00000BC5 00004641
4198 00000BC9 54313220202000
4199 00000BD0 FFFF
4200 00000BD2 000050030002010100-
4200 00000BDB 0210000000F8
4201 00000BE1 010000000000000000-
4201 00000BEA 000000000000000000
4202 00000BF3 0000000000000000FF-
4202 00000BFC FFFFFFF0000
4203 00000C01 000000000000000000-
4203 00000C0A 0000000003200028
4204 00000C12 000000000000000000-
4204 00000C1B 000000000000000000
4205 00000C24 000000000000000000-
4205 00000C2D 000000000000000000
4206 00000C36 0000FFFFFFFFF0000000-
4206 00000C3F 00000000000
4207 00000C44 00000000FF01000000-
4207 00000C4D 4E4F204E41
4208 00000C52 4D4520202020000000-
4208 00000C5B 00004641
4209 00000C5F 54313220202000
4210 00000C66 FFFF
4211 00000C68 000050030002010100-
4211 00000C71 0210000000F8
4212 00000C77 010000000000000000-
4212 00000C80 000000000000000000
4213 00000C89 0000000000000000FF-
4213 00000C92 FFFFFFF0000
4214 00000C97 000000000000000000-
4214 00000CA0 0000000003200028
4215 00000CA8 000000000000000000-
4215 00000CB1 000000000000000000
4216 00000CBA 000000000000000000-
4216 00000CC3 000000000000000000
4217 00000CCC 0000FFFFFFFFF0000000-
4217 00000CD5 00000000000
4218 00000CDA 00000000FF01000000-
4218 00000CE3 4E4F204E41
4219 00000CE8 4D4520202020000000-
4219 00000CF1 00004641
4220 00000CF5 54313220202000
4221 00000CFC FFFF
4222 00000CFE 000050030002010100-
4222 00000D07 0210000000F8
4223 00000D0D 010000000000000000-
4223 00000D16 000000000000000000
4224 00000D1F 0000000000000000FF-
4224 00000D28 FFFFFFF0000
4225 00000D2D 000000000000000000-
4225 00000D36 0000000003200028
4226 00000D3E 000000000000000000-
4226 00000D47 000000000000000000
4227 00000D50 000000000000000000-
4227 00000D59 000000000000000000
4228 00000D62 0000FFFFFFFFF0000000-
4228 00000D6B 00000000000
4229 00000D70 00000000FF01000000-
4229 00000D79 4E4F204E41
4230 00000D7E 4D4520202020000000-
4230 00000D87 00004641
4231 00000D8B 54313220202000
4232 00000D92 FFFF
4233 00000D94 000050030002010100-
4233 00000D9D 0210000000F8
4234 00000DA3 010000000000000000-
4234 00000DAC 000000000000000000
4235 00000DB5 0000000000000000FF-
4235 00000DBE FFFFFFF0000
4236 00000DC3 000000000000000000-
4236 00000DCC 0000000003200028
4237 00000DD4 000000000000000000-
4237 00000DDD 000000000000000000
4238 00000DE6 000000000000000000-
4238 00000DEF 000000000000000000
4239 00000DF8 0000FFFFFFFFF0000000-
4239 00000E01 00000000000
4240 00000E06 00000000FF01000000-
4240 00000E0F 4E4F204E41
4241 00000E14 4D4520202020000000-

```

bds_4:

```

db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h

```

```

4241 00000E1D 00004641
4242 00000E21 54313220202000
4243 00000E28 FFFF
4244 00000E2A 000050030002010100-
4244 00000E33 02100000000F8
4245 00000E39 010000000000000000-
4245 00000E42 000000000000000000
4246 00000E4B 0000000000000000FF-
4246 00000E54 FFFFFFF0000
4247 00000E59 000000000000000000-
4247 00000E62 0000000003200028
4248 00000E6A 000000000000000000-
4248 00000E73 000000000000000000
4249 00000E7C 000000000000000000-
4249 00000E85 000000000000000000
4250 00000E8E 0000FFFFFFFFF000000-
4250 00000E97 00000000000
4251 00000E9C 00000000FF01000000-
4251 00000EA5 4E4F204E41
4252 00000EAA 4D4520202020000000-
4252 00000EB3 00004641
4253 00000EB7 54313220202000
4254 00000EBE FFFF
4255 00000EC0 000050030002010100-
4255 00000EC9 02100000000F8
4256 00000ECF 010000000000000000-
4256 00000ED8 000000000000000000
4257 00000EE1 0000000000000000FF-
4257 00000EEA FFFFFFF0000
4258 00000EEF 000000000000000000-
4258 00000EF8 0000000003200028
4259 00000F00 000000000000000000-
4259 00000F09 000000000000000000
4260 00000F12 000000000000000000-
4260 00000F1B 000000000000000000
4261 00000F24 0000FFFFFFFFF000000-
4261 00000F2D 00000000000
4262 00000F32 00000000FF01000000-
4262 00000F3B 4E4F204E41
4263 00000F40 4D4520202020000000-
4263 00000F49 00004641
4264 00000F4D 54313220202000
4265 00000F54 FFFF
4266 00000F56 000050030002010100-
4266 00000F5F 02100000000F8
4267 00000F65 010000000000000000-
4267 00000F6E 000000000000000000
4268 00000F77 0000000000000000FF-
4268 00000F80 FFFFFFF0000
4269 00000F85 000000000000000000-
4269 00000F8E 0000000003200028
4270 00000F96 000000000000000000-
4270 00000F9F 000000000000000000
4271 00000FA8 000000000000000000-
4271 00000FB1 000000000000000000
4272 00000FBA 0000FFFFFFFFF000000-
4272 00000FC3 00000000000
4273 00000FC8 00000000FF01000000-
4273 00000FD1 4E4F204E41
4274 00000FD6 4D4520202020000000-
4274 00000FDF 00004641
4275 00000FE3 54313220202000
4276 00000FEA FFFF
4277 00000FEC 000050030002010100-
4277 00000FF5 02100000000F8
4278 00000FFB 010000000000000000-
4278 00001004 000000000000000000
4279 0000100D 0000000000000000FF-
4279 00001016 FFFFFFF0000
4280 0000101B 000000000000000000-
4280 00001024 0000000003200028
4281 0000102C 000000000000000000-
4281 00001035 000000000000000000
4282 0000103E 000000000000000000-
4282 00001047 000000000000000000
4283 00001050 0000FFFFFFFFF000000-
4283 00001059 00000000000
4284 0000105E 00000000FF01000000-
4284 00001067 4E4F204E41
4285 0000106C 4D4520202020000000-
4285 00001075 00004641
4286 00001079 54313220202000
4287 00001080 FFFF
4288 00001082 000050030002010100-
4288 0000108B 02100000000F8
4289 00001091 010000000000000000-
4289 0000109A 000000000000000000
4290 000010A3 0000000000000000FF-
4290 000010AC FFFFFFF0000
4291 000010B1 000000000000000000-
4291 000010BA 0000000003200028
4292 000010C2 000000000000000000-
4292 000010CB 000000000000000000
4293 000010D4 000000000000000000-
4293 000010DD 000000000000000000
4294 000010E6 0000FFFFFFFFF000000-
4294 000010EF 00000000000
4295 000010F4 00000000FF01000000-
4295 000010FD 4E4F204E41
4296 00001102 4D4520202020000000-
4296 0000110B 00004641
4297 0000110F 54313220202000
4298 00001116 FFFF
4299 00001118 000050030002010100-
4299 00001121 02100000000F8
4300 00001127 010000000000000000-
4300 00001130 000000000000000000
4301 00001139 0000000000000000FF-
4301 00001142 FFFFFFF0000
4302 00001147 000000000000000000-
4302 00001150 0000000003200028
4303 00001158 000000000000000000-
4303 00001161 000000000000000000
4304 0000116A 000000000000000000-
4304 00001173 000000000000000000
4305 0000117C 0000FFFFFFFFF000000-
4305 00001185 00000000000
4306 0000118A 00000000FF01000000-
4306 00001193 4E4F204E41
4307 00001198 4D4520202020000000-
4307 000011A1 00004641
4308 000011A5 54313220202000
4309 000011AC FFFF
4310 000011AE 000050030002010100-

```

```

db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h

db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h

db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h

db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h

db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h

db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h

db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h

db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h

db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h

db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h

db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h

db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h

db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h

```

4310	000011B7	0210000000F8	
4311	000011BD	01000000000000000000-	db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4311	000011C6	00000000000000000000	
4312	000011CF	0000000000000000FF--	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
4312	000011D8	FFFFFFFF0000	
4313	000011DD	00000000000000000000-	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
4313	000011E6	00000000003200028	
4314	000011EE	00000000000000000000-	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4314	000011F7	00000000000000000000	
4315	00001200	00000000000000000000-	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4315	00001209	00000000000000000000	
4316	00001212	0000FFFFFFFFF0000000-	db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4316	0000121B	000000000000	
4317	00001220	00000000FF01000000-	db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
4317	00001229	4E4F204E41	
4318	0000122E	4d4520202020000000-	db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 0, 0, 46h, 41h
4318	00001237	00004641	
4319	0000123B	54313220202000	db 54h, 31h, 32h, 20h, 20h, 20h, 0
4320	00001242	FFFF	dw 0FFFFh
4321	00001244	000050030002010100-	db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0, 0F8h
4321	0000124D	0210000000F8	
4322	00001253	01000000000000000000-	db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4322	0000125C	00000000000000000000	
4323	00001265	000000000000000000FF-	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
4323	0000126E	FFFFFFFF0000	
4324	00001273	00000000000000000000-	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
4324	0000127C	00000000003200028	
4325	00001284	00000000000000000000-	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4325	0000128D	00000000000000000000	
4326	00001296	00000000000000000000-	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4326	0000129F	00000000000000000000	
4327	000012A8	0000FFFFFFFFF0000000-	db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4327	000012B1	000000000000	
4328	000012B6	00000000FF01000000-	db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
4328	000012BF	4E4F204E41	
4329	000012C4	4d4520202020000000-	db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 0, 0, 46h, 41h
4329	000012CD	00004641	
4330	000012D1	54313220202000	db 54h, 31h, 32h, 20h, 20h, 20h, 0
4331	000012D8	FFFF	dw 0FFFFh
4332	000012DA	000050030002010100-	db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0, 0F8h
4332	000012E3	0210000000F8	
4333	000012E9	01000000000000000000-	db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4333	000012F2	00000000000000000000	
4334	000012FB	000000000000000000FF-	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
4334	00001304	FFFFFFFF0000	
4335	00001309	00000000000000000000-	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
4335	00001312	00000000003200028	
4336	0000131A	00000000000000000000-	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4336	00001323	00000000000000000000	
4337	0000132C	00000000000000000000-	db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
4337	00001335	00000000000000000000	
4338	0000133E	0000FFFFFFFFF0000000-	db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

[illegible]

[illegible]

```

;-----
; Possibly disposable data, goes at end of data group
;*****
; Possibly disposable data, goes at end of data group
***ibm_disk_io - main routine, fixes at rom bug
;
; entry: (ah) = function, 02 or 0a for read.
;        (dl) = drive number (80h or 81h).
;        (dh) = head number.
;        (ch) = cylinder number.
;        (cl) = sector number (high 2 bits has cylinder number).
;        (al) = number of sectors.
;        (es:bx) = address of read buffer.
;        for more on register contents see rom bios listing.
;        stack set up for return by an iret.
;
; exit:   (ah) = status of current operation.
;        (cy) = 1 if failed, 0 if successful.
;        for other register contents see rom bios listing.
;
; uses:
;
; warning: uses old13 vector for non-read calls.
;         does direct calls to the at rom.
;         does segment arithmetic.
;
; effects: performs disk i/o operation.
; 16/10/2022
; 28/05/2019
cmd_block equ 42h ; ROMBIOS DATA segment (40h) offset 42h ; 13/12/2022
;* offsets into cmd_block for registers
pre_comp equ 0    ;write pre-compensation
sec_cnt   equ 1    ;sector count
sec_num   equ 2    ;sector number
cyl_low   equ 3    ;cylinder number, low part
cyl_high equ 4     ;cylinder number, high part
drv_head  equ 5    ;drive/head (bit 7 = ecc mode, bit 5 = 512 byte sectors,
                    ;          bit 4 = drive number, bits 3-0 have head number)
cmd_reg   equ 6    ;command register
; 01/10/2022
disk_status1 equ 74h
hf_num       equ 75h
control_byte equ 76h
ibm_disk_io:
cmp dl, 80h      ; main routine,           fixes at rom bug
jb short atd1    ; pass through floppy disk calls.
cmp ah, 2        ;
jz short atd2    ; intercept call 02 (read sectors).
cmp ah, 0Ah      ;
short atd2       ; and call 0Ah (read long).
atd1:
jmp far [cs:old13]
;jmp cs:old13    ; use rom int 13h handler
;-----
atd2:
push bx
push cx
push dx
push di
push ds
push es
push ax
mov ax, 40h      ; bioseg (rombios data segment)
                ; establish bios segment addressing
mov ds, ax
; 16/10/2022
mov byte [disk_status1], 0
mov byte ptr ds:74h, 0 ; [disk_status1]
                        ; initially no error code.
                        ; mask to hard disk number
and dl, 7Fh
cmp dl, [hf_num]
;jmp dl, ds:75h    ; [hf_num] ; 40h:75h
jb short atd3      ; disk number in range
;mov byte ptr ds:74h, 1 ; bad_disk
mov byte [disk_status1], 1
jmp short atd4      ; disk number out of range error,
                    ; return
; -----
atd3:
push bx
mov ax, es
shr bx, 4          ; make es:bx to seg:000x form.
add ax, bx
mov es, ax
pop bx
and bx, 0Fh
push cs
call check_dma
jb short atd4      ; abort if dma across segment boundary
pop ax
push ax
call setcmd        ; set up command block for disk op
mov dx, 3F6h       ; hf_reg_port
out dx, al          ; write out command modifier
call docmd         ; carry out command
; -----
atd4:
```



```

4726
4727 ; new code - let logical or clear carry and then set carry if ah!=0
4728 ; and save a couple bytes while were at it.
4729
4730 00001747 58          pop     ax
4731          ;mov     ah, ds:74h      ; [disk_status1]
4732 00001748 8A267400    mov     ah, [disk_status1]
4733 0000174C 08E4        or      ah, ah
4734 0000174E 7401        jz      short atd5
4735 00001750 F9          stc
4736
4737 00001751 07          atd5:    pop     es
4738 00001752 1F          pop     ds
4739 00001753 5F          pop     di
4740 00001754 5A          pop     dx
4741 00001755 59          pop     cx
4742 00001756 5B          pop     bx
4743 00001757 CA0200     retf     2          ; far return, dropping flags
4744
4745 ; ===== S U B   R O U T I N E =====
4746
4747 ;***setcmd - set up cmd_block for the disk operation
4748 ;
4749 ; entry: (ds) = bios data segment.
4750 ; (es:bx) in seg:000x form.
4751 ; other registers as in int 13h call
4752 ;
4753 ; exit: cmd_block set up for disk read call.
4754 ; control_byte set up for disk operation.
4755 ; (al) = control byte modifier
4756 ;
4757 ; sets the fields of cmd_block using the register contents
4758 ; and the contents of the disk parameter block for the given drive.
4759 ;
4760 ; warning: (ax) destroyed.
4761 ; does direct calls to the at rom.
4762
4763 setcmd:
4764         ; proc near
4765         ;mov     ds:43h, al      ; [cmd_block+sec_cnt]
4766         ; 16/10/2022
4767         mov     [cmd_block+sec_cnt], al
4768         ;mov     byte ptr ds:48h, 20h ; [cmd_block+cmd_reg]
4769         mov     byte [cmd_block+cmd_reg], 20h ; assume function 02h (read)
4770         cmp     ah, 2
4771         jz      short setc1      ; cmd_reg = 20h          if function 02h          (read)
4772         mov     byte [cmd_block+cmd_reg], 22h
4773         ;mov     byte ptr ds:48h, 22h ; [cmd_block+cmd_reg]
4774         ; cmd_reg = 22h          if function 0Ah          (read long)
4775
4776 setc1:
4777         mov     al, cl
4778         and     al, 3Fh          ; mask sector number
4779         ;mov     ds:44h, al      ; [cmd_block+sec_num]
4780         ;mov     ds:45h, ch      ; [cmd_block+cyl_low]
4781         mov     [cmd_block+sec_num], al ; mov [44h], al
4782         mov     [cmd_block+cyl_low], ch ; mov [45h], ch
4783         mov     al, cl
4784         shr     al, 6            ; get two high bits of cylinder      number
4785         ;mov     ds:46h, al      ; [cmd_block+cyl_high]
4786         mov     [cmd_block+cyl_high], al ; mov [46h], al
4787         mov     ax, dx
4788         shl     al, 4            ; drive number
4789         and     ah, 0Fh
4790         or      al, ah          ; head number
4791         or      al, 0A0h        ; set ecc and 512 bytes      per sector
4792         ;mov     ds:47h, al      ; [cmd_block+drv_head]
4793         mov     [cmd_block+drv_head], al ; mov [47h], al
4794         push    es
4795         push    bx
4796         push    cs
4797         call    get_vec
4798         mov     ax, [es:bx+5]    ; [es:bx+fdp_precomp]
4799         ; write pre-comp from disk parameters
4800         shr     ax, 2
4801         ;mov     ds:42h, al      ; [cmd_block+pre_comp]
4802         mov     [cmd_block+pre_comp], al ; mov [42h], al
4803         ; only use low part
4804         mov     al, [es:bx+8]    ; [es:bx+fdp_control]
4805         ; control byte modifier
4806         pop     bx
4807         pop     es
4808         ;mov     ah, ds:76h      ; [control_byte]
4809         mov     ah, [control_byte] ; mov ah, [76h]
4810         and     ah, 0C0h        ; keep disable retry bits
4811         or      ah, al
4812         ;mov     ds:76h, ah
4813         mov     [control_byte], ah ; mov [76h], al
4814         retn
4815
4816 ; ===== S U B   R O U T I N E =====
4817
4818 ;***docmd - carry out read operation to at hard disk
4819 ;
4820 ; entry: (es:bx) = address for read in data.
4821 ; cmd_block set up for disk read.
4822 ;
4823 ; exit: buffer at (es:bx) contains data read.
4824 ; disk_status1 set to error code (0 if success).
4825 ;
4826 ; warning: (ax), (bl), (cx), (dx), (di) destroyed.
4827 ; no check is made for dma boundary overrun.
4828 ;
4829 ; effects: programs disk controller.
4830 ; performs disk input.
4831
4832 docmd:
4833         ; proc near
4834         mov     di, bx
4835         push    cs
4836         call    command
4837         jnz     short doc3
4838
4839 doc1:
4840         push    cs
4841         call    waitt            ; wait for controller to complete read
4842         jnz     short doc3
4843         mov     cx, 256          ; 256 words per sector
4844         mov     dx, 1F0h         ; hf_port
4845         cld                     ; string op goes up
4846         cli                     ; disable interrupts
4847         ; (bug was forgetting this)
4848
4849 ; M062 -- some of these old machines have intermittent failures
4850 ; when the read is done at full speed. Instead of using
4851 ; a string rep instruction, we'll use a loop. There is
4852 ; a slight performance hit, but it only affects these

```

```

4850 ; very old machines with an exact date code match, and
4851 ; it makes said machines more reliable
4852 ;
4853 ;M062 repz insw ;read in sector
4854
4855 rsct_loop:
4856 insw
4857 loop rsct_loop
4858 sti
4859 ; 16/10/2022
4860 test byte [cmd_block+cmd_reg], 02h
4861 ;test byte ptr ds:48h, 2 ; [cmd_block+cmd_reg]
4862 ; (ds = 40h)
4863 jz short doc2 ; no ecc bytes to read.
4864 push cs
4865 call wait_drq ; wait for controller to complete read
4866 jnb short doc3
4867 mov cx, 4 ; 4 bytes of ecc
4868 mov dx, 1F0h ; hf_port
4869 cli
4870 rep insb ; read in ecc
4871 sti
4872
4873 doc2:
4874 push cs
4875 call check_status
4876 jnz short doc3 ; operation failed
4877 ;dec byte ptr ds:43h ; [cmd_block+sec_cnt]
4878 dec byte [cmd_block+sec_cnt]
4879 jnz short doc1 ; loop while more sectors to read
4880 doc3:
4881 retn
4882
4883 ; ===== S U B R O U T I N E =====
4884
4885 ;***define where the rom routines are actually located
4886 ; in the buggy old AT BIOS that we might need to
4887 ; install a special level of int13 handler for
4888
4889 ; 16/10/2022
4890 romsegment equ 0F000h ; segment
4891 romcommand equ 2E1Eh ; offset in romsegment
4892 romwait equ 2E7Fh ; offset in romsegment
4893 romwait_drq equ 2EE2h ; offset in romsegment
4894 romcheck_status equ 2EF8h ; offset in romsegment
4895 romcheck_dma equ 2F69h ; offset in romsegment
4896 romget_vec equ 2F8Eh ; offset in romsegment
4897 romfret equ 0FF65h ; far return in rom
4898
4899 ;***get_vec - get pointer to hard disk parameters.
4900 ;
4901 ; entry: (dl) = low bit has hard disk number (0 or 1).
4902 ;
4903 ; exit: (es:bx) = address of disk parameters table.
4904 ;
4905 ; uses: ax for segment computation.
4906 ;
4907 ; loads es:bx from interrupt table in low memory, vector 46h (disk 0)
4908 ; or 70h (disk 1).
4909 ;
4910 ; warning: (ax) destroyed.
4911 ; this does a direct call to the at rom.
4912
4913 get_vec: ; proc near
4914 ;push 0FF65h ; romfret ; far return in rom
4915 ;jmp far ptr 0F000h:2F8Eh
4916 ; 16/10/2022
4917 push romfret ; far return in rom
4918 jmp romsegment:romget_vec
4919
4920 ; ===== S U B R O U T I N E =====
4921
4922 ;***command - send contents of cmd_block to disk controller.
4923 ;
4924 ; entry: control_byte
4925 ; cmd_block - set up with values for hard disk controller.
4926 ;
4927 ; exit: disk_status1 = error code.
4928 ; nz if error, zr for no error.
4929 ;
4930 ;
4931 ; warning: (ax), (cx), (dx) destroyed.
4932 ; does a direct call to the at rom.
4933 ;
4934 ; effects: programs disk controller.
4935
4936 command: ; proc near
4937 ;push 0FF65h ; romfret ; far return in rom
4938 ;jmp far ptr 0F000h:2E1Eh
4939 ; 16/10/2022
4940 push romfret ; far return in rom
4941 jmp romsegment:romcommand
4942
4943 ; ===== S U B R O U T I N E =====
4944
4945 ;***waitt - wait for disk interrupt
4946 ;
4947 ; entry: nothing.
4948 ;
4949 ; exit: disk_status1 = error code.
4950 ; nz if error, zr if no error.
4951 ;
4952 ;
4953 ; warning: (ax), (bl), (cx) destroyed.
4954 ; does a direct call to the at rom.
4955 ;
4956 ; effects: calls int 15h, function 9000h.
4957
4958 waitt: ; proc near
4959 ;push 0FF65h ; romfret ; far return in rom
4960 ;jmp far ptr 0F000h:2E7Fh
4961 ; 16/10/2022
4962 push romfret ; far return in rom
4963 jmp romsegment:romwait
4964
4965 ; ===== S U B R O U T I N E =====
4966
4967 ;***wait_drq - wait for data request.
4968 ;
4969 ; entry: nothing.
4970 ;
4971 ; exit: disk_status1 = error code.
4972 ; cy if error, nc if no error.
4973 ;

```

```

4974 ; warning: (al), (cx), (dx) destroyed.
4975 ; does a direct call to the at rom.
4976
4977 wait_drq: ; proc near
4978 ; push 0FF65h ; romfret ; far return in rom
4979 ; jmp far ptr 0F000h:2EE2h
4980 ; 16/10/2022
4981 00001808 6865FF push romfret ; far return in rom
4982 0000180B EAE22E00F0 jmp romsegment:romwait_drq
4983
4984 ; ===== S U B R O U T I N E =====
4985
4986 ; ***check_status - check hard disk status.
4987 ;
4988 ; entry: nothing.
4989 ;
4990 ; exit: disk_status1 = error code.
4991 ; nz if error, zr if no error.
4992 ;
4993 ; warning: (ax), (cx), (dx) destroyed.
4994 ; does a direct call to the at rom.
4995
4996 check_status: ; proc near
4997 ; push 0FF65h ; romfret ; far return in rom
4998 ; jmp far ptr 0F000h:2EF8h
4999 ; 16/10/2022
5000 00001810 6865FF push romfret ; far return in rom
5001 00001813 EAF82E00F0 jmp romsegment:romcheck_status
5002
5003 ; ===== S U B R O U T I N E =====
5004
5005 ; ***check_dma - check for dma overrun 64k segment.
5006 ;
5007 ; entry: (es:bx) = addr. of memory buffer in seg:000x form.
5008 ; cmd_block set up for operation.
5009 ;
5010 ; exit: disk_status1 - error code.
5011 ; cy if error, nc if no error.
5012 ;
5013 ; warning: does a direct call to the at rom.
5014
5015 check_dma: ; proc near
5016 ; push 0FF65h ; romfret ; far return in rom
5017 ; jmp far ptr 0F000h:2F69h
5018 ; 16/10/2022
5019 00001818 6865FF push romfret ; far return in rom
5020 0000181B EA692F00F0 jmp romsegment:romcheck_dma
5021
5022 ; -----
5023
5024 endatrom:
5025 ; -----
5026
5027 ; M015 -- begin changes
5028 ;
5029 ;
5030 ; Certain old COMPAQ '286 machines have a bug in their ROM BIOS.
5031 ; When Int13 is done with AH > 15h and DL >= 80h, they trash
5032 ; the byte at DS:74h, assuming that DS points to ROM_DATA.
5033 ; If our init code detects this error, it will install this
5034 ; special Int13 hook through the same mechanism that was set
5035 ; up for the IBM patch above. This code is also dynamically
5036 ; relocated by MSINIT.
5037
5038 compaq_disk_io:
5039 00001820 80FC15 cmp ah, 15h ; compaq_disk_io proc far
5040 ;
5041 ; the following label defines the end of the at rom patch.
5042 ; this is used at configuration time.
5043 ;
5044 ; warning!!!
5045 ; this code will be dynamically relocated by msinit
5046 00001823 7705 ja short mebbe_hookit ; only deal with functions > 15h
5047
5048 no_hookit:
5049 ; jmp cs:old13
5050 ; 16/10/2022
5051 00001825 2EFF2E[0601] jmp far [cs:old13]
5052
5053 ; -----
5054
5055 mebbe_hookit:
5056 0000182A 80FA80 cmp dl, 80h
5057 0000182D 72F6 jb short no_hookit
5058 0000182F 1E push ds
5059 ;
5060 ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
5061 ; PC DOS 7.1 IBMBIO.COM - BIOSDATA:1830h
5062 ; push ax
5063 ; mov ax, 40h
5064 ; mov ds, ax
5065 ; pop ax
5066 00001830 6A40 push 40h
5067 00001832 1F pop ds
5068 00001833 9C pushf
5069 ; call cs:old13
5070 ; 16/10/2022
5071 00001834 2EFF1E[0601] call far [cs:old13]
5072 00001839 1F pop ds
5073 0000183A CA0200 retf 2
5074
5075 ; -----
5076
5077 0000183D 00 end_compaq_i13hook: db 0
5078
5079 ; ===== S U B R O U T I N E =====
5080
5081 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
5082 %if 0
5083
5084 ; CMOS Clock setting support routines used by MSCLOCK.
5085 ; warning!!! This code will be dynamically relocated by MSINIT.
5086
5087 daycnt_to_day: ; proc far
5088 ;
5089 ; entry: [daycnt] = number of days since 1-1-80
5090 ;
5091 ; return: ch - century in bcd
5092 ; cl - year in bcd
5093 ; dh - month in bcd
5094 ; dl - day in bcd
5095 ;
5096 ; 16/10/2022
5097 push word [cs:daycnt] ; save daycnt

```

```

5098             cmp     word [cs:daycnt], 7305; (365*20+(20/4))
5099             ; # days from 1-1-1980 to 1-1-2000
5100             jnb     short century20
5101             mov     byte [cs:base_century], 19
5102             mov     byte [cs:base_year], 80
5103             jmp     short years
5104             ; -----
5105
5106 century20:
5107             mov     byte [cs:base_century], 20
5108             mov     byte [cs:base_year], 0
5109             sub     word [cs:daycnt], 7305; (365*20+(20/4))
5110             ; adjust daycnt
5111
5112 years:
5113             xor     dx, dx
5114             mov     ax, [cs:daycnt]
5115             mov     bx, 1461             ; (366+365*3)
5116             ; # of days in a Leap year block
5117             div     bx                 ; AX = # of leap block, DX = daycnt
5118             mov     [cs:daycnt], dx    ; save daycnt left
5119             mov     bl, 4
5120             mul     bl                 ; AX = # of years. Less than 100
5121             add     [cs:base_year], al ; So, ah = 0. Adjust year
5122             inc     word [cs:daycnt]   ; set daycnt to 1 base
5123             cmp     word [cs:daycnt], 366 ; daycnt=remainder of leap year    bk
5124             jbe     short leapyear ; within 366+355+355+355 days.
5125             inc     byte [cs:base_year] ; if daycnt <= 366, then leap year
5126             sub     word [cs:daycnt], 366 ; else daycnt--, base_year++ ;
5127             mov     cx, 3             ; And next three years are normal
5128
5129 regularyear:
5130             cmp     word [cs:daycnt], 365 ; for(i=1; i>3 or daycnt <=365;    i++)
5131             jbe     short yeardone ; {if (daycnt > 365)
5132             inc     byte [cs:base_year] ; { daycnt -= 365
5133             sub     word [cs:daycnt], 365 ; }
5134             loop    regularyear       ; }
5135             ; should never fall through loop
5136
5137 leapyear:
5138             mov     byte [cs:month_tab+1], 29 ; leap year.
5139             ; change month table.
5140
5141 yeardone:
5142             xor     bx, bx
5143             xor     dx, dx
5144             mov     ax, [cs:daycnt]
5145             ;mov     si, offset month_tab
5146             mov     si, month_tab ; 19/10/2022
5147             mov     cx, 12
5148
5149 months:
5150             inc     bl
5151             ; !!! -- 16/10/2022 -- (if DS=CS, what for CS: prefixes are used !?)
5152             ;mov     dl, [cs:si]
5153             ; !!! -- 16/10/2022 -- (may be to keep code addrs as unchanged/fix!?)
5154             ; ds = cs !? ((ofcourse ds must be same with cs here))
5155             ;mov     dl, [si] ; 20/03/2019 (MSDOS 6.21 IO.SYS, BIOSDATA:14C0h)
5156             ;mov     dl, [si] ; 16/10/2022 (MSDOS 5.0 IO.SYS, BIOSDATA:14C0h)
5157
5158             mov     dl, [si] ; ? ; mov dl, [cs:si]
5159             cmp     ax, dx             ; cmp daycnt for each month till fit
5160             ; dh=0
5161             jbe     short month_done
5162             inc     si                 ; next month
5163             sub     ax, dx             ; adjust daycnt
5164             loop    months             ;
5165             ; should never fall through loop
5166
5167 month_done:
5168             mov     byte [cs:month_tab+1], 28
5169             ; restore month table value
5170
5171             mov     dl, bl
5172             mov     dh, [cs:base_year]
5173             mov     cl, [cs:base_century] ; al=day, dl=month, dh=year, cl=cntry
5174             call    far [cs:bintobcd]
5175             ;call    cs:bintobcd ; convert "day" to bcd
5176             ; dl = bcd day, al = month
5177
5178             xchg     dl, al
5179             call    far [cs:bintobcd]
5180             ;call    cs:bintobcd ; dh = bcd month, al = year
5181             xchg     dh, al
5182             call    far [cs:bintobcd]
5183             ;call    cs:bintobcd ; cl = bcd year, al = century
5184             xchg     cl, al
5185             call    far [cs:bintobcd]
5186             ;call    cs:bintobcd ; ch = bcd century
5187             mov     ch, al
5188             pop     word [cs:daycnt] ; restore original value
5189             retf
5190
5191 enddaycnttoday:
5192
5193 %endif
5194
5195 ; ===== S U B R O U T I N E =====
5196
5197 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
5198 %if 0
5199
5200 bin_to_bcd: ; proc far ; real time clock support
5201
5202 ;convert a binary input in al (less than 63h or 99 decimal)
5203 ;into a bcd value in al. ah destroyed.
5204
5205             push     cx
5206             aam                     ; al=high digit bcd, ah=low digit bcd
5207             mov     cl, 4
5208             shl     ah, cl           ; mov the high digit to high nibble
5209             or      al, ah
5210             pop     cx
5211             retf
5212
5213 %endif
5214
5215 ; -----
5216
5217 ; the k09 requires the routines for reading the clock because of the suspend/
5218 ; resume facility. the system clock needs to be reset after resume.
5219
5220 ; the following routine is executed at resume time when the system
5221 ; powered on after suspension. it reads the real time clock and
5222 ; resets the system time and date, and then irets.
5223
5224 ; warning!!! this code will be dynamically relocated by msinit.
5225
5226 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
5227 ; PC DOS 7.1 IBMBIO.COM - BIOSDATA:183Eh
5228 int_6Ch:

```

```

5222 0000183E 0E          push    cs
5223 0000183F 1F          pop     ds
5224                      ;cmp    byte [cs:inHMA], 0
5225 00001840 803E[0D00]00        cmp     byte [inHMA], 0
5226 00001845 7405          jz      short int6c
5227 00001847 BB[2A07]        mov     bx, EnsureA200n
5228 0000184A FFD3          call   bx
5229                      int6c:
5230                      ;push    cs
5231                      ;pop     ds
5232 0000184C 8F06[F805]        pop     word [int6c_ret_addr] ; pop off return address
5233 00001850 8F06[FA05]        pop     word [int6c_ret_addr+2]
5234 00001854 9D          popf
5235 00001855 E81300        call   read_real_date ; get the date from the clock
5236 00001858 FA          cli
5237 00001859 8936[8904]        mov     [daycnt], si ; update dos copy of date
5238 0000185D FB          sti
5239 0000185E E88B00        call   read_real_time ; get the time from the      rtc
5240 00001861 FA          cli
5241 00001862 B401        mov     ah, 1
5242 00001864 CD1A        int     1Ah          ; CLOCK - SET TIME OF DAY
5243                      ; CX:DX= clock count
5244                      ; Return: time of day set
5245 00001866 FB          sti
5246                      ;jmp     int6c_ret_addr ; long jump
5247                      ; 16/10/2022
5248 00001867 FF2E[F805]  jmp     far [int6c_ret_addr] ; long jump
5249
5250                      ; ===== S U B   R O U T I N E =====
5251
5252                      ; read_real_date reads real-time clock for date and returns the number
5253                      ; of days elapsed since 1-1-80 in si
5254
5255 read_real_date:        ; proc near
5256 0000186B 50          push    ax
5257 0000186C 51          push    cx
5258 0000186D 52          push    dx
5259 0000186E 30E4        xor     ah, ah          ; throwaway clock roll      over
5260 00001870 CD1A        int     1Ah          ; CLOCK - GET TIME OF DAY
5261                      ; Return: CX:DX = clock count
5262                      ; AL = 00h if clock was read or written (via AH=0,1) since the previous
5263                      ; midnight
5264                      ; Otherwise, AL > 0
5265 00001872 5A          pop     dx
5266 00001873 59          pop     cx
5267 00001874 58          pop     ax
5268 00001875 50          push    ax
5269 00001876 53          push    bx
5270 00001877 51          push    cx
5271 00001878 52          push    dx
5272                      ;mov     word [cs:daycnt2], 1
5273                      ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
5274                      ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:187Ah
5275 00001879 C706[0006]0100 mov     word [daycnt2], 1
5276                      ; REAL TIME CLOCK ERROR      FLAG (+1 DAY)
5277 0000187F B404        mov     ah, 4
5278 00001881 CD1A        int     1Ah          ; CLOCK - READ DATE FROM REAL TIME CLOCK (AT,XT286,CONV,PS)
5279                      ; Return: DL = day in BCD
5280                      ; DH = month in      BCD
5281                      ; CL = year in BCD
5282                      ; CH = century (19h or 20h)
5283 00001883 7303        jnb     short read_ok
5284 00001885 E98500        jmp     r_d_ret
5285
5286                      ;-----
5287
5288 read_ok:              mov     [bin_date_time], ch
5289 0000188C 880E[FD05]        mov     [bin_date_time+1], cl
5290 00001890 8836[FE05]        mov     [bin_date_time+2], dh
5291 00001894 8816[FF05]        mov     [bin_date_time+3], dl
5292                      ;mov     word [cs:daycnt2], 2 ; READ OF R-T CLOCK SUCCESSFUL
5293                      ; 08/08/2023
5294                      ;mov     byte [daycnt2], 2
5295 00001898 FE06[0006]  inc     byte [daycnt2] ; 2
5296 0000189C E83601        call   bcd_verify ; verify bcd values in range
5297 0000189F 726C        jb     short r_d_ret ; some value out of range
5298                      ;mov     word [cs:daycnt2], 3
5299                      ; 08/08/2023
5300                      ;mov     byte [daycnt2], 3
5301 000018A1 FE06[0006]  inc     byte [daycnt2] ; 3
5302 000018A5 E8DD00        call   date_verify
5303 000018A8 7263        jb     short r_d_ret
5304                      ;mov     word [cs:daycnt2], 0
5305                      ; 08/08/2023
5306 000018AA C606[0006]00  mov     byte [daycnt2], 0
5307 000018AF E8A300        call   in_bin
5308 000018B2 A0[FD05]        mov     al, [bin_date_time+1]
5309 000018B5 98          cbw
5310 000018B6 803E[FC05]14  cmp     byte [bin_date_time], 20 ; 20th century?
5311 000018BB 7503        jnz     short century_19 ; no
5312 000018BD 83C064        add     ax, 100          ; add in a century
5313
5314 century_19:          sub     ax, 80          ; subtract off 1-1-80
5315 000018C3 8104        mov     cl, 4          ; leap year every 4
5316 000018C5 F6F1        div     cl          ; al= #leap year blocks, ah= remainder
5317 000018C7 88E3        mov     bl, ah          ; save odd years
5318 000018C9 98          cbw
5319 000018CA B9B505        mov     cx, 1461        ; 366+(3*365)
5320                      ; # of days in leap year blocks
5321 000018CD F7E1        mul     cx
5322                      ;mov     [cs:daycnt2], ax ; SAVE COUNT OF DAYS
5323                      ; 08/08/2023
5324 000018CF A3[0006]        mov     [daycnt2], ax
5325 000018D2 88D8        mov     al, bl          ; get odd years      count
5326 000018D4 98          cbw
5327 000018D5 09C0        or      ax, ax
5328 000018D7 740B        jz      short leap_year
5329 000018D9 B96D01        mov     cx, 365          ; days in year
5330 000018DC F7E1        mul     cx
5331                      ;add     [cs:daycnt2], ax ; ADD ON DAYS IN ODD YEARS
5332                      ; 08/08/2023
5333 000018DE 0106[0006]  add     [daycnt2], ax
5334 000018E2 EB07        jmp     short leap_adjustment ; account for leap year
5335                      ; possibly account for a leap day
5336                      ;-----
5337
5338 leap_year:          cmp     byte [bin_date_time+2], 2 ; is      month february?
5339 000018E4 803E[FE05]02  jbe     short no_leap_adjustment ; jan or feb. no leap day yet.
5340 000018E9 7604
5341
5342 leap_adjustment:    ;inc     word [cs:daycnt2] ; account for leap day
5343                      ; 08/08/2023
5344 000018EB FF06[0006]  inc     word [daycnt2]
5345
5346 no_leap_adjustment:

```

```

5346 000018EF 8A0E[FF05]      mov     cl, [bin_date_time+3] ; get days of month
5347 000018F3 30ED          xor     ch, ch
5348 000018F5 49            dec     cx                      ; because of offset from day 1,      not day 0
5349                                ;add     [cs:daycnt2], cx ; GET DAYS IN MONTHS PRECEEDING
5350                                ; 08/08/2023
5351 000018F6 010E[0006]      add     [daycnt2], cx
5352 000018FA 8A0E[FE05]      mov     cl, [bin_date_time+2] ; get month
5353                                ; 08/08/2023
5354                                ;xor     ch, ch
5355 000018FE 49            dec     cx                      ; january starts at offset 0
5356                                ; 19/01/2026 (BugFix)
5357 000018FF 740C          jz      short r_d_ret
5358
5359                                ; 08/08/2023
5360                                ;shl     cx, 1                      ; word offset
5361                                ;mov     si, month_table
5362                                ;add     si, cx
5363                                ; 16/10/2022
5364                                ; ds must be same with cs here, if so..
5365                                ; what for cs: prefixes are used !?)
5366                                ; mov ax, [cs:si]
5367                                ; mov ax, [si] ; 16/10/2022 (MSDOS 5.0 IO.SYS - BIOSDATA:15D5h)
5368                                ;mov     ax, [si] ; mov ax, [cs:si]
5369                                ;
5370                                ;add     [cs:daycnt2], ax
5371
5372                                ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
5373                                ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:1907h
5374 00001901 B400          mov     ah, 0
5375 00001903 BE[8F04]      mov     si, month_tab
5376
5377 00001906 AC          r_d_sum_loop: lodsb
5378 00001907 0106[0006]      add     [daycnt2], ax
5379 0000190B E2F9          loop    r_d_sum_loop
5380
5381 0000190C 0000          r_d_ret:
5382                                ;mov     si, [cs:daycnt2]
5383                                ; 08/08/2023
5384 0000190D 8B36[0006]      mov     si, [daycnt2]
5385 00001911 5A          pop     dx
5386 00001912 59          pop     cx
5387 00001913 5B          pop     bx
5388 00001914 58          pop     ax
5389                                retn
5390
5391                                ;-----
5392
5393 00001916 31C9          r_t_retj: xor     cx, cx
5394 00001918 31D2          xor     dx, dx
5395 0000191A EB38          jmp     short r_t_ret
5396
5397                                ; ===== S U B   R O U T I N E =====
5398
5399                                ; read_real_time reads the time from the rtc. on exit, it has the number of
5400                                ; ticks (at 18.2 ticks per sec.) in cx:dx.
5401
5402                                read_real_time: ; proc near
5403 0000191C B402          mov     ah, 2
5404 0000191E CD1A          int     1Ah                      ; CLOCK - READ REAL TIME CLOCK (AT,XT286,CONV,PS)
5405                                ; Return: CH = hours in      BCD
5406                                ; CL = minutes in BCD
5407                                ; DH = seconds in BCD
5408 00001920 72F4          jb      short r_t_retj
5409 00001922 882E[FC05]      mov     [bin_date_time], ch ; hours
5410 00001926 880E[FD05]      mov     [bin_date_time+1], cl ; minutes
5411 0000192A 8836[FE05]      mov     [bin_date_time+2], dh ; seconds
5412 0000192E C606[FF05]00    mov     byte [bin_date_time+3], 0 ; unused for time
5413 00001933 E89F00          call    bcd_verify
5414 00001936 72DE          jb      short r_t_retj
5415 00001938 E88500          call    time_verify
5416 0000193B 72D9          jb      short r_t_retj
5417 0000193D E81500          call    in_bin ; from bcd to bin
5418 00001940 8A2E[FC05]      mov     ch, [bin_date_time]
5419 00001944 8A0E[FD05]      mov     cl, [bin_date_time+1]
5420 00001948 8A36[FE05]      mov     dh, [bin_date_time+2]
5421 0000194C 8A16[FF05]      mov     dl, [bin_date_time+3]
5422                                ; 16/10/2022
5423                                ; 17/09/2022
5424                                ; 31/05/2019
5425 00001950 FF1E[0606]      call    far [ttticks]
5426                                ;call    dword ptr tticks ; note: indirect far call
5427                                ; cx:dx= number of ticks
5428                                ; (at 18.2 ticks per sec.)
5429
5430 00001954 C3          r_t_ret: retn
5431
5432                                ; ===== S U B   R O U T I N E =====
5433
5434                                ; in_bin converts bin_date_time values from bcd to bin
5435
5436                                in_bin: ; proc near
5437 00001955 A0[FC05]          mov     al, [bin_date_time] ; century or hours
5438 00001958 E81F00          call    bcd_to_bin
5439 0000195B A2[FC05]          mov     [bin_date_time], al
5440 0000195E A0[FD05]          mov     al, [bin_date_time+1] ; years or minutes
5441 00001961 E81600          call    bcd_to_bin
5442 00001964 A2[FD05]          mov     [bin_date_time+1], al
5443 00001967 A0[FE05]          mov     al, [bin_date_time+2] ; months or seconds
5444 0000196A E80D00          call    bcd_to_bin
5445 0000196D A2[FE05]          mov     [bin_date_time+2], al
5446 00001970 A0[FF05]          mov     al, [bin_date_time+3] ; days (not used for time)
5447 00001973 E80400          call    bcd_to_bin
5448 00001976 A2[FF05]          mov     [bin_date_time+3], al
5449 00001979 C3          retn
5450
5451                                ; ===== S U B   R O U T I N E =====
5452
5453                                ; bcd_to_bin converts two bcd nibbles in al (value <= 99.) to
5454                                ; a binary representation in al
5455                                ; ah is destroyed
5456
5457                                bcd_to_bin: ; proc near
5458 0000197A 88C4          mov     ah, al
5459 0000197C 240F          and     al, 0Fh
5460 0000197E B104          mov     cl, 4
5461 00001980 D2EC          shr     ah, cl
5462 00001982 D50A          aad
5463 00001984 C3          retn
5464
5465                                ; ===== S U B   R O U T I N E =====
5466
5467                                ; date_verify loosely checks bcd date values to be in range
5468                                ; in bin_date_time
5469

```



```

5470
5471 00001985 803E[FC05]20
5472 0000198A 7732
5473 0000198C 740E
5474 0000198E 803E[FC05]19
5475
5476
5477 00001993 722A
5478 00001995 803E[FD05]80
5479
5480
5481 0000199A 7223
5482
5483 0000199C 803E[FD05]99
5484 000019A1 771B
5485 000019A3 803E[FE05]12
5486 000019A8 7714
5487 000019AA 803E[FE05]00
5488
5489 000019AF 760D
5490 000019B1 803E[FF05]31
5491 000019B6 7706
5492
5493
5494
5495
5496
5497
5498
5499 000019B8 803E[FF05]01
5500 000019BD C3
5501
5502
5503
5504 000019BE F9
5505
5506 000019BF C3
5507
5508
5509
5510
5511
5512
5513
5514 000019C0 803E[FC05]24
5515 000019C5 770C
5516 000019C7 803E[FD05]59
5517 000019CC 7705
5518
5519
5520
5521
5522
5523
5524 000019CE 803E[FE05]5A
5525
5526
5527 000019D3 F5
5528 000019D4 C3
5529
5530
5531
5532
5533
5534
5535
5536
5537
5538
5539
5540
5541
5542 000019D5 B90400
5543 000019D8 BB[FC05]
5544
5545 000019DB 8A07
5546 000019DD 88C4
5547 000019DF 250FF0
5548
5549 000019E2 3C0A
5550 000019E4 77ED
5551 000019E6 D0EC
5552 000019E8 D0EC
5553 000019EA D0EC
5554 000019EC D0EC
5555 000019EE 80E40F
5556 000019F1 80FC0A
5557 000019F4 77DD
5558 000019F6 43
5559 000019F7 49
5560 000019F8 75E1
5561 000019FA F8
5562 000019FB C3
5563
5564
5565
5566
5567
5568
5569
5570
5571
5572
5573
5574
5575
5576
5577
5578
5579
5580
5581
5582
5583
5584
5585
5586
5587
5588
5589
5590
5591
5592
5593

date_verify: ; proc near
    cmp byte [bin_date_time], 20h ; century check
    ja short date_error
    jz short century_20 ; jmp in 21th century
    cmp byte [bin_date_time], 19h ; century check
    jnb short date_error
    ; 12/12/2022
    jb short date_err2
    cmp byte [bin_date_time+1], 80h ; year check
    jnb short date_error
    ; 12/12/2022
    jb short date_err2
century_20:
    cmp byte [bin_date_time+1], 99h ; year check
    ja short date_error
    cmp byte [bin_date_time+2], 12h ; month check
    ja short date_error
    cmp byte [bin_date_time+2], 0
    jbe short date_error
    jna short date_error
    cmp byte [bin_date_time+3], 31h ; day check
    ja short date_error
    ;cmp byte [bin_date_time+3], 0 ; day check
    ;jbe short date_error
    ;jna short date_error
    ; 12/12/2022
    ; cf=0
    ;clc
    ; 12/12/2022
    cmp byte [bin_date_time+3], 1 ; day check
    retn
;-----
date_error:
    stc
date_err2:
    retn
; ===== S U B R O U T I N E =====
; time_verify very loosely checks bcd date values to be in range
; in bin_date_time
time_verify: ; proc near
    cmp byte [bin_date_time], 24h ; hour check
    ja short time_error
    cmp byte [bin_date_time+1], 59h ; minute check
    ja short time_error
    ; 12/12/2022h
    ;cmp byte [bin_date_time+2], 59h ; second check
    ;ja short time_error
    ;clc
    ;retn
    ; 12/12/2022
    cmp byte [bin_date_time+2], 5Ah
time_error:
bv_error:
    cmc ; cf=0 -> cf=1, cf=1 -> cf=0
    retn
; -----
;time_error:
;stc
;retn
; ===== S U B R O U T I N E =====
; bcd_verify checks values in bin_date_time to be valid
; bcd numerals. carry set if any nibble out of range
bcd_verify: ; proc near
    mov cx, 4 ; 4 bytes to check
    mov bx, bin_date_time
bv_loop:
    mov al, [bx] ; get abcd number (0..99)
    mov ah, al
    and ax, 0F00Fh ; 10's place in high ah, 1's in al
    ; is 1's place in range?
    cmp al, 10
    ja short bv_error ; jmp out of range
    shr ah, 1
    shr ah, 1
    shr ah, 1
    shr ah, 1
    and ah, 0Fh ; get rid of any erroneous bits
    cmp ah, 10 ; is 10's place in range
    ja short bv_error ; jmp out of range
    inc bx ; next byte
    dec cx
    jnz short bv_loop
    clc ; set success flag
    retn
; -----
; 12/12/2022
bv_error:
    stc ; set error flag
    retn
; -----
endk09:
; -----
;
; System initialization
;
; The entry conditions are established by the bootstrap loader and are considered unknown. The following jobs will be performed by this module:
;
; 1. All device initialization is performed
; 2. A local stack is set up and DS:SI are set to point to an initialization table. Then an inter-segment call is made to the first byte of the dos
; 3. Once the dos returns from this call the ds register has been set up to point to the start of free memory. The initialization will then load the command program into this area beginning at 100 hex and transfer control to
;

```

```

5594 ; this program. ;
5595 ; ;
5596 ; -----
5597 ; 01/10/2022
5598 ; 08/01/2018 - Retro DOS v4.0
5600 ;
5601 ; drvfat must be the first location of freeable space!
5602
5603 align 2
5604 ;db 90h
5605
5606 ; 09/12/2023 - Retro DOS v5.0 IO.SYS (IBMBIO.COM 7.1)
5607 ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1A0Ch)
5608
5609 ; 30/12/2022
5610 ; (MSDOS 6.21 IO.SYS, BIOSDATA:16D6h)
5611
5612 000019FC 0000 drvfat: dw 0 ; driveand fatid of dos
5613 ; 09/12/2023
5614 ;bios_l: dw 0 ; firstsector of data (low word)
5615 ;bios_h: dw 0 ; firstsector of data (high word)
5616 First_Data_Sector:
5617 000019FE 0000 dw 0
5618 00001A00 0000 dw 0
5619 00001A02 0000 doscnt: dw 0 ; how many sectors to read
5620 ;fbigfat: db 0 ; flags for drive
5621 00001A04 0000 fatloc: dw 0 ; seg addr of fat sector
5622 00001A06 0000 init_bootseg: dw 0 ; seg addr of buffer for reading boot record
5623 ; 09/12/2023
5624 00001A08 00 fbigfat: db 0 ; flags for drive
5625 00001A09 80 rom_drv_num: db 80h ; rom drive number
5626 00001A0A 0002 md_sectorsize: dw 200h ; used by get_fat_sector proc.
5627 ; 12/12/2023
5628 ;temp_cluster: dw 0 ; used by get_fat_sector proc.
5629 00001A0C FFFF last_fat_sec_num: dw 0FFFFh ; used by get_fat_sector proc.
5630
5631 ; the following two bytes are used to save the info returned by int 13, ah = 8
5632 ; call to determine drive parameters.
5633
5634 00001A0E 02 num_heads: db 2 ; dw 2 ; number of heads returned by rom
5635 00001A0F 00 db 0 ; 09/12/2023
5636 ;sec_trk: db 9 ; sec/trk returned by rom
5637 00001A10 28 num_cylin: db 40 ; dw 40 ; number of cylinders returned by rom
5638 00001A11 00 db 0 ; 09/12/2023
5639 ; 09/12/2023
5640 00001A12 09 sec_trk: db 9 ; sec/trk returned by rom
5641 00001A13 00 fakefloppydrv: db 0 ; if 1, then no diskette drives in the system.
5642
5643 ; 09/12/2023 - Retro DOS v5.0 IO.SYS/IBMBIO.COM
5644 Orig_Int1Eh_Table:
5645 00001A14 0000 dw 0
5646 00001A16 0000 dw 0
5647
5648 ; -----
5649
5650 ; 09/12/2023
5651 %if 0
5652
5653 disktable: dw 512, 0100h, 64, 0 ; warning !!! old values
5654 dw 2048, 0201h, 112, 0
5655 dw 8192, 0402h, 256, 0
5656 dw 32680, 0803h, 512, 0 ; warning !!! old values
5657 dw 65535, 1004h, 1024, 0
5658 ; default disktable under
5659 ; the assumption of total fat size <= 128 kb,
5660 ; and the maximum size of fat entry = 16 bit.
5661 %endif
5662
5663 ; 09/12/2023 - Retro DOS v5.0 IO.SYS (IBMBIO.COM 7.1)
5664 ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1A2Ah)
5665
5666 ; 09/12/2023
5667 ; 08/08/2023
5668 ; disktable.totalsectors: resw 1 ; high word
5669 ; resw 1 ; low word
5670 ; disktable.shiftcount: resb 1
5671 ; disktable.secpersclus: resb 1
5672 ; disktable.rdirentries: resw 1
5673 ; disktable.bigflag: resw 1
5674 00001A18 0000A87F0308000200- disktable2: dw 0, 32680, 0803h, 512, 0 ; for compatibility.
5675 00001A21 00
5676 00001A22 040000000204000240- dw 4, 0, 0402h, 512, 40h ; (32680 sectors, 16340 KB)
5677 00001A2B 00 ; covers upto 134 mb media.
5678 00001A2C 080000000308000240- dw 8, 0, 0803h, 512, 40h ; fbig = 40h ; (40000h sectors = 128 MB)
5679 00001A35 00 ; upto 268 mb ; (80000h sectors = 256 MB)
5680 00001A36 100000000410000240- dw 16, 0, 1004h, 512, 40h ; upto 536 mb ; (100000h sectors = 512 MB)
5681 00001A3F 00
5682 00001A40 200000000520000240- dw 32, 0, 2005h, 512, 40h ; upto 1072 mb ; (200000h sectors = 1024 MB)
5683 00001A49 00
5684 00001A4A 400000000640000240- dw 64, 0, 4006h, 512, 40h ; upto 2144 mb ; (400000h sectors = 2048 MB)
5685 00001A53 00
5686 ; 09/12/2023
5687 ;dw 128, 0, 8007h, 512, 40h ; upto 4288 mb ; (800000h sectors = 4096 MB)
5688 00001A54 FFFFFFFF0308000060- dw 0FFFFh, 0FFFFh, 0803h, 0, 60h ; FAT32 (> 2144MB)
5689 00001A5D 00
5690 ; (fbig and fbigbig flags are set)
5691
5692 ; -----
5693
5694 ; *****
5695 ; variables for mini disk initialization
5696 ; *****
5697
5698 ; 01/10/2022
5699 ; [ Note: Minidisk == logical dos drive (in extended dos partition) ]
5700
5701 rom_minidisk_num: db 0 ; temp variable for phys unit
5702 hnum: db 0 ; real number of hardfiles
5703 last_dskdrv_table: dw dskdrvs ; index into dskdrv table
5704 end_of_bdss: dw bdss ; offset value of the ending address
5705 ; of bds table. needed to figure out
5706 ; the dosdatasg address.
5707
5708 mini_hdlim: dw 0
5709 mini_seclim: dw 0
5710
5711 ; 19/12/2023
5712 ; 09/12/2023
5713 ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1A7Ah)
5714 ;ld_p_number: dw 2BADh ; (for 'find_mini_partition' proc)
5715
5716 ;end of mini disk init variables *****

```



```

5711 ; -----
5712 ;
5713 bios_date: db '01/10/84',0 ; used for checking at rom bios date.
5714 00001A68 30312F31302F383400
5715 ; 13/12/2022
5716 %if 0
5717
5718 ;align 2
5719 db 90h
5720
5721 ; the following are the recommended bpbs for the media that we know of so far.
5722
5723 ;struc bpbx
5724 ; resw 1 ; 512
5725 ; resb 1
5726 ; resw 1 ; 1
5727 ; resb 1 ; 2
5728 ; resw 1
5729 ; resw 1
5730 ; resb 1
5731 ; resw 1
5732 ; resw 1
5733 ; resw 1 ; 2
5734 ; resw 1
5735 ; resw 1 ; hidden sector high
5736 ; resd 1 ; extended total sectors
5737 ;.size:
5738 ;endstruc
5739
5740 ; 08/01/2019 - Retro DOS v4.0
5741
5742 ; 20/04/2019
5743
5744 ; 01/10/2022 - Retro DOS v4.0 (MSDOS 5.0) IO.SYS
5745
5746 ; 09/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
5747 ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1A86h)
5748
5749 ; 09/12/2022
5750 BPB48T:
5751 ;bpb48t: ; bpbx <512, 2, 1, 2, 112, 720, 0FDh, 2, 9, 2, 0, 0, 0, 0>
5752 ; 48 tpi diskettes ;
5753 dw 512 ; physical sector size in bytes
5754 db 2 ; sectors/allocation unit
5755 dw 1 ; reserved sectors for dos
5756 db 2 ; number of allocation tables
5757 dw 112 ; number of directory entries
5758 dw 720 ; 2*9*40 ; number of sectors (at 512 bytes each)
5759 db 0FDh ; media descriptor
5760 dw 2 ; number of fat sectors
5761 dw 9 ; sectors per track
5762 dw 2 ; heads
5763 dw 0 ; hidden sector count (low word)
5764 dw 0 ; hidden sector (high)
5765 dw 0 ; number of sectors (low)
5766 dw 0 ; number of sectors (high)
5767 ; 09/12/2023
5768 ; FAT32 extensions (to BDS)
5769 times 28 db 0
5770 ;
5771 ;
5772 db 90h
5773 ;align 2
5774 BPB96T:
5775 ;bpb96t: ; bpbx <512, 1, 1, 2, 224, 2400, 0F9h, 7, 15, 2, 0, 0, 0, 0>
5776 ; 96 tpi diskettes ;
5777 dw 512 ; physical sector size in bytes
5778 db 1 ; sectors/allocation unit
5779 dw 1 ; reserved sectors for dos
5780 db 2 ; number of allocation tables
5781 dw 224 ; number of directory entries
5782 dw 2400 ; 2*15*80 ; number of sectors (at 512 bytes each)
5783 db 0F9h ; media descriptor
5784 dw 7 ; number of fat sectors
5785 dw 15 ; sectors per track
5786 dw 2 ; heads
5787 dw 0 ; hidden sector count (low word)
5788 dw 0 ; hidden sector (high)
5789 dw 0 ; number of sectors (low)
5790 dw 0 ; number of sectors (high)
5791 ; 09/12/2023
5792 ; FAT32 extensions (to BDS)
5793 times 28 db 0
5794 ;
5795 ;
5796 db 90h
5797 ;align 2
5798 BPB35:
5799 ;bpb35: ; bpbx <512, 2, 1, 2, 112, 1440, 0F9h, 3, 9, 2, 0, 0, 0, 0>
5800 ; 3.5" diskettes - 720 KB ;
5801 dw 512 ; physical sector size in bytes
5802 db 2 ; sectors/allocation unit
5803 dw 1 ; reserved sectors for dos
5804 db 2 ; number of allocation tables
5805 dw 112 ; number of directory entries
5806 dw 1440 ; 2*9*80 ; number of sectors (at 512 bytes each)
5807 db 0F9h ; media descriptor
5808 dw 3 ; number of fat sectors
5809 dw 9 ; sectors per track
5810 dw 2 ; heads
5811 dw 0 ; hidden sector count (low word)
5812 dw 0 ; hidden sector (high)
5813 dw 0 ; number of sectors (low)
5814 dw 0 ; number of sectors (high)
5815 ; 09/12/2023
5816 ; FAT32 extensions (to BDS)
5817 times 28 db 0
5818 ;
5819 ;
5820 db 90h
5821 ;align 2
5822 ;BPB144:
5823 ;bpb144: ; Retro DOS v4.0 feature only ! ; 1.44MB diskettes
5824 ;
5825 ; dw 512 ; physical sector size in bytes
5826 ; db 1 ; sectors/allocation unit
5827 ; dw 1 ; reserved sectors for dos
5828 ; db 2 ; number of allocation tables
5829 ; dw 224 ; number of directory entries
5830 ; dw 2880 ; 2*18*80 ; number of sectors (at 512 bytes each)
5831 ; db 0F0h ; media descriptor
5832 ; dw 9 ; number of fat sectors
5833 ; dw 18 ; sectors per track
5834 ; dw 2 ; heads

```

```

5835 ; dw 0 ; hidden sector count (low word)
5836 ; dw 0 ; hidden sector (high)
5837 ; dw 0 ; number of sectors (low)
5838 ; dw 0 ; number of sectors (high)
5839 ;
5840 ; align 2 db 90h
5841 ;align 2
5842
5843 BPB288:
5844 ;bp288: ; bpbx <512, 2, 1, 2, 240, 5760, 0F0h, 9, 36, 2, 0, 0, 0, 0>
5845 ; 3.5" diskettes - 2.88 MB ;
5846 dw 512 ; physical sector size in bytes
5847 db 2 ; sectors/allocation unit
5848 dw 1 ; reserved sectors for dos
5849 db 2 ; number of allocation tables
5850 dw 240 ; number of directory entries
5851 dw 5760 ; 2*36*80 ; number of sectors (at 512 bytes each)
5852 db 0F0h ; media descriptor
5853 dw 3 ; number of fat sectors
5854 dw 9 ; sectors per track
5855 dw 2 ; heads
5856 dw 0 ; hidden sector count (low word)
5857 dw 0 ; hidden sector (high)
5858 dw 0 ; number of sectors (low)
5859 dw 0 ; number of sectors (high)
5860 ; 09/12/2023
5861 ; FAT32 extensions (to BDS)
5862 times 28 db 0
5863 ;
5864 ; align 2 db 90h
5865 ;align 2
5866
5867 %endif
5868
5869 ; -----
5870 ; align 2
5871 ; 09/12/2022
5872 %if 0
5873 bpbtable: dw bpb48t ; 48tpi drives
5874 dw bpb96t ; 96tpi drives
5875 dw bpb35 ; 3.5" drives
5876 dw bpb35 ; unused 8" diskette
5877 dw bpb35 ; unused 8" diskette
5878 dw bpb35 ; used for hard disk
5879 dw bpb35 ; used for tape drive
5880 dw bpb35 ; FFOTHER
5881 dw bpb35 ; ERIMO
5882 dw bpb288 ; 2.88MB drive
5883 ;
5884 ;dw bpb144 ; 1.44MB drive - Retro DOS v4.0 feature !
5885 %endif
5886
5887 ; 13/12/2022
5888 %if 0
5889 BPBTABLE: dw BPB48T ; 48tpi drives
5890 dw BPB96T ; 96tpi drives
5891 dw BPB35 ; 3.5" drives
5892 dw BPB35 ; unused 8" diskette
5893 dw BPB35 ; unused 8" diskette
5894 dw BPB35 ; used for hard disk
5895 dw BPB35 ; used for tape drive
5896 dw BPB35 ; FFOTHER
5897 dw BPB35 ; ERIMO
5898 dw BPB288 ; 2.88MB drive
5899 ;
5900 ;dw BPB144 ; 1.44MB drive - Retro DOS v4.0 feature !
5901 %endif
5902
5903 %endif
5904
5905 ; -----
5906 ; entry point to call utility functions in Bios_Code. At this time,
5907 ; we aren't doing any A20 switching. During MSINIT time Bios_Code
5908 ; will not yet be moved to its final resting place, so we know
5909 ; it'll be low.
5910 ;
5911 ; to use this function, do a "push cs" and load bp with the offset of
5912 ; the function you want to call in Bios_Code. This routine will
5913 ; push the address of a retf in Bios_Code onto the stack which
5914 ; will get executed when the utility function finishes. It will
5915 ; then transfer control to Bios_Code:bp using a couple of pushes
5916 ; and a retf
5917 ;
5918 ; 16/10/2022
5919 ;BC_RETf equ bc_ret f - DOSBIOSEG_2C7h
5920 ; 09/12/2022
5921 BC_RETf equ bc_ret f
5922
5923 ; 09/12/2023
5924 ;PCDOS 7.1 IBMBIO.COM bc_ret f offset = 0CAh (in BIOSCODE segment = 364h)
5925
5926 addr_of_bc ret f: ;dw 0C8h ; dw bc_ret f
5927 ; 2C7h:0C8h = 70h:2638h
5928 ; 09/12/2023
5929 ; 364h:0CAh = 70h:300Ah ; PCDOS 7.1
5930 00001A71 [CA00] dw BC_RETf ; dw 0CAh
5931 ;
5932 ; -----
5933
5934 call_bios_code: ; proc far
5935 00001A73 2EFF36[711A] push word [cs:addr_of_bc ret f]
5936 ; set up near return to far return
5937 00001A78 2EFF36[0406] push word [cs:cdev+2] ; push Bios_Code segment
5938 00001A7D 55 push bp ; save offset of utility function
5939 00001A7E CB retf ; far jump to (DOS)BIOS code
5940 ;
5941 ; -----
5942
5943 ; 09/12/2023 - Retro DOS v5.0 IBMBIO.COM/IO.SYS
5944 ; 20/12/2022
5945 00001A7F 00 flp_drvs: db 0
5946 ; 11/12/2023
5947 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:1B81h)
5948 firstcluster_hw:
5949 00001A80 0000 dw 0 ; 06/04/2024
5950 00001A82 00 Boot_Drv: db 0
5951 ;
5952 ; -----
5953
5954 ; 09/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
5955 ; -----
5956 ; PCDOS 7.1 CD BOOT option code
5957 ; -----
5958 ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1B84h)

```

5959					
5960		cd_boot_option:			
5961	00001A83	50	push	ax	
5962	00001A84	1E	push	ds	
5963	00001A85	06	push	es	
5964	00001A86	52	push	dx	
5965					
5966	00001A87	B401	mov	ah, 1	
5967	00001A89	CD16	int	16h	; KEYBOARD - status
5968	00001A8B	7406	jz	short cdbo_2	
5969	00001A8D	30E4	xor	ah, ah	
5970	00001A8F	CD16	int	16h	; KEYBOARD - READ CHAR FROM BUFFER, WAIT IF EMPTY
5971					; Return: AH = scan code, AL = character
5972	00001A91	EBF4	jmp	short cdbo_1	
5973					
5974	00001A93	0E	push	cs	
5975	00001A94	1F	pop	ds	
5976	00001A95	BE[6D1B]	mov	si, cd_boot_msg	; "Press the ENTER key to boot from CD"...
5977	00001A98	AC	lodsb		
5978					
5979	00001A99	BB0700	mov	bx, 7	
5980	00001A9C	B40E	mov	ah, 0Eh	
5981	00001A9E	CD10	int	10h	; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
5982					; AL = character, BH = display page (alpha modes)
5983					; BL = foreground color (graphics modes)
5984	00001AA0	AC	lodsb		
5985	00001AA1	08C0	or	al, al	
5986	00001AA3	75F4	jnz	short cdbo_3	
5987	00001AA5	B84000	mov	ax, 40h	
5988	00001AA8	8ED8	mov	ds, ax	
5989			;mov	bx, [6Ch]	; 0:46Ch = Daily timer counter (4 bytes)
5990				; 09/12/2023	
5991	00001AAA	8B166C00	mov	dx, [6Ch]	
5992	00001AAE	8B366E00	mov	si, [6Eh]	
5993			wait_for_key:		
5994			;push	bx	
5995			;mov	bx, 7	
5996			; bx = 7		
5997	00001AB2	B8080E	mov	ax, 0E08h	
5998	00001AB5	CD10	int	10h	; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
5999					; AL = character, BH = display page (alpha modes)
6000					; BL = foreground color (graphics modes)
6001	00001AB7	B8200E	mov	ax, 0E20h	
6002	00001ABA	CD10	int	10h	; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
6003					; AL = character, BH = display page (alpha modes)
6004					; BL = foreground color (graphics modes)
6005	00001ABC	B8080E	mov	ax, 0E08h	
6006	00001ABF	CD10	int	10h	; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
6007					; AL = character, BH = display page (alpha modes)
6008					; BL = foreground color (graphics modes)
6009			;pop	bx	
6010			;add	bx, 18	; 18.2 ticks per second
6011				; 09/12/2023	
6012	00001AC1	83C212	add	dx, 18	
6013	00001AC4	83D600	adc	si, 0	; next second (if carry flag is 1)
6014			continue_to_wait:		
6015	00001AC7	B401	mov	ah, 1	
6016	00001AC9	CD16	int	16h	; KEYBOARD - status
6017	00001ACB	741B	jz	short cdbo_5	
6018	00001ACD	B400	mov	ah, 0	
6019	00001ACF	CD16	int	16h	; KEYBOARD - READ CHAR FROM BUFFER, WAIT IF EMPTY
6020					; Return: AH = scan code, AL = character
6021					
6022				; 09/12/2023	
6023			;cmp	ax, 11Bh ; ESC key	
6024			;jz	short cdb0_7	
6025			;cdb0_4:		
6026			;push	ax ; *	
6027	00001AD1	89C2	mov	dx, ax ; *	
6028					
6029			; CRLF (next line)		
6030			;mov	bx, 7	
6031			; bx = 7		
6032	00001AD3	B80D0E	mov	ax, 0E0Dh	
6033	00001AD6	CD10	int	10h	; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
6034					; AL = character, BH = display page (alpha modes)
6035					; BL = foreground color (graphics modes)
6036	00001AD8	B80A0E	mov	ax, 0E0Ah	
6037	00001ADB	CD10	int	10h	; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
6038					; AL = character, BH = display page (alpha modes)
6039					; BL = foreground color (graphics modes)
6040			; 09/12/2023		
6041			;pop	ax ; *	
6042					
6043	00001ADD	81FA1B01			

```

6083
6084 00001AFF B8004B      mov     ax, 4B00h
6085                      ;xor     dl, dl
6086                      ; 09/12/2023
6087 00001B02 31D2      xor     dx, dx
6088                      ; dl = disk drive = 0 ; fd
6089                      ;mov     si, 1C93h
6090 00001B04 BE[591B]    mov     si, empty_dap_buff
6091 00001B07 CD13      int     13h          ; DISK - Bootable CD-ROM - AL = TERMINATE DISK EMULATION
6092                      ; DS:SI = Specification packet filled
6093
6094                      ;mov     dx, 80h
6095                      ;xor     ax, ax
6096                      ; 09/12/2023
6097 00001B09 B81300    mov     ax, 19
6098 00001B0C 89F7      mov     di, si
6099                      ;mov     byte [si], 13h
6100                      ;mov     [si+1], al
6101 00001B0E AB        stosw
6102                      ;mov     [si+2], dx
6103 00001B0F B080      mov     al, 80h
6104 00001B11 AB        stosw
6105 00001B12 89C2      mov     dx, ax
6106                      ;mov     [si+4], ax
6107                      ;mov     [si+6], ax
6108                      ;mov     [si+8], ax
6109                      ;mov     [si+0Ah], ax
6110                      ;mov     [si+0Ch], ax
6111                      ;mov     [si+0Eh], ax
6112                      ;mov     [si+10h], al
6113                      ;mov     [si+11h], al
6114                      ;mov     [si+12h], al
6115 00001B14 B90F00    mov     cx, 15
6116 00001B17 F3AA      rep     stosb
6117                      ; dl = disk drive = 80h ; hd
6118 00001B19 B8004B    mov     ax, 4B00h
6119 00001B1C CD13      int     13h          ; DISK - Bootable CD-ROM - AL = TERMINATE DISK EMULATION
6120 00001B1E 31C0      xor     ax, ax
6121                      ; 09/12/2023
6122                      ;mov     dx, 80h
6123                      ; dx = 80h
6124 00001B20 CD13      int     13h          ; DISK - RESET DISK SYSTEM
6125                      ; DL = drive (if bit 7 is set both hard disks and floppy disks reset)
6126                      ; 09/12/2023
6127                      ;push    cs
6128                      ;pop     es
6129                      ; es = ds = cs
6130
6131 00001B22 B80102    mov     ax, 201h
6132                      ;mov     bx, 152h
6133 00001B25 BB[5201]    mov     bx, disksector
6134                      ;mov     cx, 1
6135                      ; 09/12/2023
6136 00001B28 41        inc     cx ; cx = 1
6137                      ;mov     dx, 80h
6138                      ; dx = 80h
6139 00001B29 CD13      int     13h          ; DISK - READ SECTORS INTO MEMORY
6140                      ; AL = number of sectors to read, CH = track, CL = sector
6141                      ; DH = head, DL = drive, ES:BX -> buffer to fill
6142                      ; Return: CF set on error, AH = status, AL = number of sectors read
6143                      ;jc      short cdbo_8
6144                      ; 10/12/2023
6145 00001B2B 72B6      jc      short cdbo_4
6146
6147 00001B2D 2681BFFE0155AA cmp     word [es:bx+1FEh], 0AA55h
6148                      ;jz      short cdbo_9
6149                      ; 10/12/2023
6150 00001B34 75AD      jnz     short cdbo_4
6151                      ;cdbo_8:
6152                      ;cdbo_9:
6153                      ; 10/12/2023
6154                      ; (stack clearing -pop- is not necessary here,
6155                      ; PC DOS 7.1 boot sector will set stack pointer again)
6156                      ;pop     ax ; near call return address
6157                      ;pop     cx ; +++ ; ch = [MediaByte]
6158
6159                      ; 09/12/2023
6160                      ;push    cs
6161                      ;pop     ds
6162                      ; ds = cs
6163
6164 00001B36 31C0      xor     ax, ax ; 0
6165 00001B38 BF007C    mov     di, 7C00h
6166 00001B3B 8EC0      mov     es, ax
6167 00001B3D 89DE      mov     si, bx
6168 00001B3F 06        push    es
6169 00001B40 57        push    di
6170 00001B41 B90001    mov     cx, 100h ; 256
6171                      ; 10/12/2023
6172                      ;cld     ; not necessary (direction flag is already cleared)
6173 00001B44 F3A5      rep movsw
6174 00001B46 8ED8      mov     ds, ax
6175 00001B48 BE7800    mov     si, 78h
6176 00001B4B 2EA1[141A] mov     ax, [cs:Orig_Int1Eh_Table]
6177 00001B4F 8904      mov     [si], ax
6178 00001B51 2EA1[161A] mov     ax, [cs:Orig_Int1Eh_Table+2]
6179 00001B55 894402    mov     [si+2], ax
6180 00001B58 CB        retf
6181
6182                      ; -----
6183 dap_buffer: ; 16/12/2023
6184
6185 00001B59 13        empty_dap_buff: db 19
6186                      ;db 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0 ; db 18 dup(0)
6187 00001B5A 00<rep 12h>    times 18 db 0
6188 00001B6C 05        time_counter: db 5 ; 5 seconds
6189 00001B6D 0D0A      cd_boot_msg: db 0Dh,0Ah
6190                      ;db 'Press the ENTER key to boot from CD or DVD.....',0
6191                      ; 09/12/2023
6192 00001B6F 507265737320616E79- db 'Press any key to boot from CD or DVD ...',0
6192 00001B78 206B657920746F2062-
6192 00001B81 6F6F742066726F6D20-
6192 00001B8A 4344206F7220445644-
6192 00001B93 202E2E2E00
6193
6194                      ; -----
6195
6196                      ; 01/10/2022 - Retro DOS v4.0 (MSDOS 5.0)
6197
6198                      ; 10/12/2023 - Retro DOS v5.0 (PCDOS 7.1)
6199                      ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1CDAh)
6200
6201                      ; -----
6202                      ; entry point from boot sector

```

```

6203 ;-----
6204
6205 init: ; 27/12/2018
6206 ; MSDOS 6.0 (MSINIT.ASM)
6207 ;=====
6208 ;
6209 ; entry from boot sector. the register contents are:
6210 ;
6211 ; dl = int 13 drive number we booted from
6212 ; ch = media byte
6213 ; bx = first data sector on disk.
6214 ; ax = first data sector (high)
6215 ; di = sectors/fat for the boot media.
6216
6217 ; 07/04/2018
6218 ;=====
6219 ; Retro DOS v2.0 - registers from FD Boot Sector
6220 ; dl = [bsDriveNumber]
6221 ; DH = [bsMedia]
6222 ; AX = [bsSectors] ; Total sectors
6223 ; DS = 0, SS = 0
6224 ; BP = 7C00h
6225
6226 ; 10/12/2023
6227 ; Retro DOS v5.0 (IBMBIO.COM)
6228 ;=====
6229 ; PCDOS 7.1 IBMBIO.COM - registers from MSLOAD section
6230 ; DL = [BootDrive]
6231 ; CH = [MediaByte]
6232 ; AX:BX = First data Sector
6233 ; DS:SI = Original INT 1Eh table address
6234 ;
6235 ; Stack: INT 1Eh vector (0:78h) !not used! (dword [sp])
6236 ; INT 1Eh table address !not used! (dword [sp+4])
6237 ; DI = 78h !not used!
6238
6239 ; 11/12/2023
6240 ;cli ; not necessary at this stage
6241
6242 ; 10/12/2023
6243 ;mov [cs:Orig_Int1Eh_Table+2], ds
6244 ;mov [cs:Orig_Int1Eh_Table], si
6245 00001B98 1E push ds
6246 00001B99 07 pop es
6247 00001B9A 0E push cs
6248 00001B9B 1F pop ds
6249 00001B9C 8C06[161A] mov [Orig_Int1Eh_Table+2], es
6250 00001BA0 8936[141A] mov [Orig_Int1Eh_Table], si
6251
6252 ; 21/12/2022
6253 ; ds = 0 (?)
6254 ;push ax
6255 ;xor ax, ax
6256 ;mov ds, ax
6257 ;pop ax
6258
6259 ; 02/10/2022
6260 ;-----
6261 ; Note: Retro DOS v4.0 kernel does not use/contain MSLOAD part of IO.SYS (5.0)
6262 ; Because, Retro DOS v2 boot sector loads complete/entire MSDOS.SYS
6263 ; (RETRODOS.SYS) kernel file (IO.SYS & MSDOS.SYS together).
6264 ; As result of boot sector ve init differences, Retro DOS init code (here)
6265 ; moves kernel to segment 070h at first, then sets diskette parameters
6266 ; at segment 50h (while MSDOS 5.0 boot sector sets this).
6267 ;-----
6268
6269 ; msload will check the extended boot record and set ax, bx accordingly.
6270
6271 ; msload passes a 32 bit sector number hi word in ax and low in bx
6272 ; save this in cs:bios_h and cs:bios_l. this is for the start of
6273 ; data sector of the bios.
6274
6275 ;mov [cs:bios_h], ax ; (start of) dos bios (IO.SYS) data sector
6276 ;mov [cs:bios_l], bx
6277 ; 10/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
6278 ;mov [cs:First_Data_Sector+2], ax
6279 ;mov [cs:First_Data_Sector], bx
6280 ;mov [cs:Boot_Drv], dl
6281 ; ds = cs
6282 00001BA4 A3[001A] mov [First_Data_Sector+2], ax
6283 00001BA7 891E[FE19] mov [First_Data_Sector], bx
6284 00001BAB 8816[821A] mov [Boot_Drv], dl
6285
6286 ; with the following information from msload, we don't need the
6287 ; boot sector any more.-> this will solve the problem of 29 kb size
6288 ; limitation of msbio.com file.
6289
6290 00001BAF 0E push cs ; Save a peck of interrupt vectors...
6291 00001BB0 07 pop es
6292
6293 00001BB1 51 push cx ; +++ ; ch = [MediaByte]
6294 ;push di ; *! (not necessary) ; 10/12/2023
6295
6296 ; 10/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
6297 00001BB2 FC cld ; (may not be necessary)
6298
6299 xor ax, ax
6300 00001BB5 8ED8 mov ds, ax ; ds = 0
6301
6302 ; 06/04/2024
6303 00001BB7 50 push ax ; push ds ; 0
6304
6305 ;mov ax, 544h ; SYSINIT segment
6306 00001BB8 B80405 mov ax, SYSINITSEG
6307
6308 ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1D06h)
6309
6310 ; check (1st sector) of the root directory -of BOOT CD-
6311 ; for special names (as boot option signature)
6312
6313 00001BBB BE4005 mov si, 540h ; ROOT DIRECTORY BUFFER offset 40h
6314 ; (BOOT DRV's root directory the 3rd entry)
6315
6316 00001BBE 803C00 chk_boot_hdnos: cmp byte [si], 0
6317 00001BC1 7436 jz short chk_no_logo_nos
6318 00001BC3 813C5F42 cmp word [si], 425Fh ; '_BOOT_HDNOZ'
6319 00001BC7 7527 jnz short chk_next_1
6320 00001BC9 817C024F4F cmp word [si+2], 4F4Fh ; 'oo'
6321 00001BCE 7520 jnz short chk_next_1
6322 00001BD0 817C04545F cmp word [si+4], 5F54h
6323 00001BD5 7519 jnz short chk_next_1
6324 00001BD7 817C064844 cmp word [si+6], 4448h ; 'HD'
6325 00001BDC 7512 jnz short chk_next_1
6326 00001BDE 817C084E4F cmp word [si+8], 4F4Eh

```

```

6327 00001BE3 750B          jnz     short chk_next_1
6328 00001BE5 807C0A5A      cmp     byte [si+0Ah], 5Ah ; 'z'
6329 00001BE9 7505          jnz     short chk_next_1
6330 00001BEB E895FE      call    cd_boot_option
6331 00001BEE EB09          jmp     short chk_no_logo_noz
6332
chk_next_1:
6333 00001BF0 83C620      add     si, 32 ; (next entry)
6334 00001BF3 81FE0007    cmp     si, 700h
6335 00001BF7 72C5          jb      short chk_boot_hdnz
6336
chk_no_logo_noz:
6337 00001BF9 BE4005      mov     si, 540h ; (BOOT DRV's root directory the 3rd entry)
6338
chk_no_logo_noz2_nxt:
6339 00001BFC 803C00      cmp     byte [si], 0
6340 00001BFF 7431          jz      short write_start_msg
6341 00001C01 813C4E4F    cmp     word [si], 4F4Eh ; 'NO_LOGO NOZ'
6342 00001C05 7522          jnz     short chk_next_2
6343 00001C07 817C025F4C  cmp     word [si+2], 4C5Fh
6344 00001C0C 751B          jnz     short chk_next_2
6345 00001C0E 817C044F47  cmp     word [si+4], 474Fh
6346 00001C13 7514          jnz     short chk_next_2
6347 00001C15 817C064F20  cmp     word [si+6], 204Fh
6348 00001C1A 750D          jnz     short chk_next_2
6349 00001C1C 817C084E4F  cmp     word [si+8], 4F4Eh
6350 00001C21 7506          jnz     short chk_next_2
6351 00001C23 807C0A5A      cmp     byte [si+0Ah], 5Ah
6352 00001C27 741C          jz      short startmsg_ok
6353
chk_next_2:
6354 00001C29 83C620      add     si, 32 ; (next entry)
6355 00001C2C 81FE0007    cmp     si, 700h
6356 00001C30 72CA          jb      short chk_no_logo_noz2_nxt
6357
write_start_msg:
6358 00001C32 8ED8          mov     ds, ax ; SYSINIT segment
6359 00001C34 BE[C551]      mov     si, StartMsg ; "Starting PC DOS...\r\n\n"
6360
startmsg_nxt_chr:
6361 00001C37 AC          lodsb
6362 00001C38 08C0          or      al, al
6363 00001C3A 7409          jz      short startmsg_ok
6364 00001C3C B40E          mov     ah, 0Eh
6365 00001C3E BB0700      mov     bx, 7
6366 00001C41 CD10          int     10h
6367
; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
6368
; AL = character, BH = display page (alpha modes)
6369 00001C43 EBF2          jmp     short startmsg_nxt_chr
6370
startmsg_ok:
6371
; 06/04/2024
6372 00001C45 1F          pop     ds ; 0
6373
; 10/12/2023
6374
; ds = 0
6375
; 21/12/2022
6376
; ds = 0 (?)
6377
; 24/12/2022
6378
; ds = cs
6379
; xor cx, cx
6380
; mov ds, cx
6381
; ds = 0
6382
; mov cl, 5
6383
; 10/12/2023
6384
; mov cx, 5 ; NUMROMVECTORS
6385
; no. of rom vectors to be saved
6386
; mov si, offset RomVectors ; point to list of int vectors
6387
; mov si, RomVectors
6388
; 10/12/2023
6389
cli
6390 00001C49 BE[0001]
6391
next_int_:
6392
cs ; 16/10/2022
6393 00001C4C FA          lodsb
6394
; lods byte ptr cs:[si] ; cs lodsb
6395 00001C4D 2E          cbw     ; ax = interrupt number
6396 00001C4E AC          shl     ax, 1
6397
; shl ax, 1 ; int no * 4
6398 00001C4F 98          mov     di, ax ; interrupt vector address
6399 00001C50 D1E0      xchg    si, di ; rombios interrupt vector address in si
6400 00001C52 D1E0      ; saving address in di
6401 00001C54 89C7      ; movsw
6402 00001C56 87F7      ; stosw
6403
; ; movsw
6404
; ; stosw
6405
; ; save the vector
6406
; 21/04/2024
6407
movsw
6408
movsw
6409 00001C58 A5
6410 00001C59 A5
6411
xchg    si, di
6412 00001C5A 87F7      loop   next_int_
6413 00001C5C E2EF
6414
; 10/12/2023
6415
; pop di ; *!
6416
; pop cx ; +++ ; ch = [MediaByte]
6417
; we need to save int13 in two places in case we are running on an at.
6418
; on ats we install the ibm supplied rom_bios patch which hooks
6419
; int13 ahead of orig13. since int19 must unhook int13 to point to the
6420
; rom int13 routine, we must have that rom address also stored away.
6421
; 21/12/2022
6422
; mov ax, [cs:old13] ; save old13 in orig13 also
6423
; mov [cs:orig13], ax
6424
; mov ax, [cs:old13+2]
6425
; mov [cs:orig13+2], ax
6426
; 16/10/2022
6427
mov     word [13h*4], block13
6428
; mov word ptr ds:4Ch, offset block13 ; 13h*4
6429
; ; set up int 13 for newaction
6430
mov     [13h*4+2], cs
6431 00001C5E C7064C00[ED06]
6432
; mov word ptr ds:4Eh, cs ; 13h*4+2
6433
mov     word [15h*4], Int15
6434 00001C64 8C0E4E00
6435
; mov word ptr ds:54h, offset Int15 ; 15h*4
6436 00001C68 C7065400[9907]
6437
; ; set up int 15 for newaction
6438
mov     [15h*4+2], cs
6439 00001C6E 8C0E5600
6440
; mov word ptr ds:56h, cs ; 15h*4+2
6441 00001C72 C7066400[5907]
6442
mov     word [19h*4], Int19
6443
; mov word ptr ds:64h, offset int19 ; 19h*4
6444
; ; set up int 19 for newaction
6445
mov     [19h*4+2], cs
6446
; mov word ptr ds:66h, cs ; 19h*4+2
6447
; 10/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
6448 00001C7C A16800      mov     ax, [68h] ; 1Ah*4
6449 00001C7F 8B3E6A00      mov     di, [6Ah] ; 1Ah*4+2
6450 00001C83 C7066800[AF06]      mov     word [68h], Int1A

```



```

6451 00001C89 8C0E6A00      mov     [6Ah], cs
6452
6453      ; 21/12/2022
6454 00001C8D 0E      push    cs
6455 00001C8E 1F      pop     ds
6456
6457      ; 10/12/2023
6458 00001C8F A3[AB06]     mov     [Orig1A], ax
6459 00001C92 893E[AD06]   mov     [Orig1A+2], di
6460
6461 00001C96 A1[0601]     mov     ax, [old13]      ; save old13 in orig13 also
6462 00001C99 A3[B400]     mov     [Orig13], ax
6463 00001C9C A1[0801]     mov     ax, [old13+2]
6464 00001C9F A3[B600]     mov     [Orig13+2], ax
6465
6466 00001CA2 FB      sti
6467 00001CA3 CD11     int     11h      ; EQUIPMENT DETERMINATION
6468                                     ; Return: AX = equipment flag bits
6469
6470      ; 10/12/2023
6471      jmp     short chk_fd_count
6472      ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1DF7h) ; *!!*
6473      ; ((signature))
6474      push    dx      ; 52h ; 'R'
6475      push    ax      ; 50h ; 'P'
6476      push    bx      ; 53h ; 'S'
6477
6478      ; we have to support a system that does not have any diskette
6479      ; drives but only hardfiles. this system will ipl from the hardfile.
6480      ; if the equipment flag bit 0 is 1, then the system has diskette drive(s).
6481      ; otherwise, the system has only hardfiles.
6482
6483      ; important thing is that still, for compatibility reason, the drive letter
6484      ; for the hardfiles start from "c". so, we still need to allocate dummy bds
6485      ; drive a and drive b. at sysinit time, we are going to set cds table entry
6486      ; of dpb pointer for these drives to 0, so any user attempt to access this
6487      ; drives will get "invalid drive letter ..." message. we are going to
6488      ; establish "fakefloppydrv" flag. ***sysinit module should call int 11h to
6489      ; determine whether there are any diskette drivers in the system or not.!!!***
6490
6491      ; check the register returned by the equipment determination interrupt
6492      ; we have to handle the case of no diskettes in the system by faking
6493      ; two dummy drives.
6494
6495      ; if the register indicates that we do have floppy drives we don't need
6496      ; to do anything special.
6497
6498      ; if the register indicates that we don't have any floppy drives then
6499      ; what we need to do is set the fakefloppydrv variable, change the
6500      ; register to say that we do have floppy drives and then go to execute
6501      ; the code which starts at notsingle. this is because we can skip the
6502      ; code given below which tries to find if there are one or two drives
6503      ; since we already know about this.
6504
6505      chk_fd_count:      ; 10/12/2023
6506      ;or     ax, 1      ; *!!*
6507
6508      ; 12/12/2022
6509      test    al, 1
6510      ;test    ax, 1      ; floppy drives      present?
6511      jnz     short normalfloppydrv ; yes.
6512
6513      ; Some ROM BIOSs lie that there are no floppy drives. Lets find out
6514      ; whether it is an old ROM BIOS or a new one
6515
6516      ; WARNING !!!
6517
6518      ; This sequence of code is present in SYSINIT1.ASM also. Any modification
6519      ; here will require an equivalent modification in SYSINIT1.ASM also
6520
6521      ; 10/12/2023
6522      ; ((cx is already on top of the stack))
6523      push    cx      ; +++ ; ch = [MediaByte]
6524      push    bx      ; not necessary
6525      push    ax
6526      push    dx
6527      push    di      ; not necessary
6528      push    es
6529
6530      mov     ah, 8
6531      mov     dl, 0
6532      int     13h      ; DISK - DISK - GET CURRENT DRIVE PARAMETERS (XT,AT,XT286,CONV,PS)
6533                                     ; DL = drive number
6534                                     ; Return: CF set on error, AH = status code, BL = drivetype
6535                                     ; DL = number of consecutive drives
6536                                     ; DH = maximum value for head number, ES:DI -> drive parameter
6537
6538      jc      short _gdskp_error
6539      mov     [cs:flp_drvs], dl
6540      ; 21/12/2022
6541      ; ds = cs
6542      mov     [flp_drvs], dl
6543
6544      _gdskp_error:
6545      ; 10/12/2023
6546      pop     es
6547      pop     di
6548      pop     dx
6549      pop     ax
6550      pop     bx
6551      pop     cx      ; +++ ; ch = [MediaByte]
6552      jc      short normalfloppydrv
6553      ; if error it is an old ROM BIOS
6554      ; so, lets assume that ROM BIOS lied
6555
6556      ; 21/12/2022
6557      cmp     byte [cs:flp_drvs], 0 ; number of drvs == 0?
6558      jz      short _set_fake_flpdrv
6559      mov     al, [cs:flp_drvs]
6560      mov     al, [flp_drvs]
6561      or      al, al      ; number of drvs == 0?
6562      jz      short _set_fake_flpdrv
6563
6564      dec     al      ; make it zero based
6565      ; 18/12/2022
6566      dec     ax
6567      jmp     short got_num_flp_drvs
6568
6569      ; -----
6570
6571      _set_fake_flpdrv:
6572      ; 21/12/2022
6573      mov     ax, 1
6574      ; 10/12/2023
6575      inc     ax ; al = 1
6576      mov     [fakefloppydrv], al ; 1
6577      mov     byte [cs:fakefloppydrv], 1
6578      ;
6579      ; we don't have any floppy drives.
6580      mov     ax, 1

```

```

6575 00001CCC EB09          jmp     short settwodrive ; well then set it for two drives!
6576                      ; -----
6577
6578 normalfloppydrv:         ; yes, bit 0 is 1.
6579 00001CCE D0C0          rol     al, 1          ; there exist floppy drives.
6580 00001CD0 D0C0          rol     al, 1          ; put bits 6 & 7 into bits 0 & 1
6581
6582 got_num_flp_drvs:        ; and ax, 3          ; only look at bits 0 & 1
6583                      ; 18/12/2022
6584 00001CD2 2403          and     al, 3
6585 00001CD4 7505          jnz     short notsingle ; zero means single drive system
6586 00001CD6 40            inc     ax          ; pretend it's a two drive system
6587                      ; set this to two fakedrives
6588 settwodrive:            ; 21/12/2022
6589                      ; ds = cs
6590 00001CD7 FE06[7800]    inc     byte [single]
6591                      ; inc byte [cs:single] ; remember this
6592
6593 00001CDB 40            notsingle: inc     ax          ; ax has number of drives, 2-4
6594                      ; is also 0 indexed boot drive if we
6595                      ; booted off hard file
6596 00001CDC 88C1          mov     cl, al        ; ch is fat id, cl # floppies
6597
6598 ; 16/10/2022
6599 ; MSDOS 3.3 - "MSEQU.INC" (24/07/1987)
6600 INITSPOT EQU 534h      ; IBM wants 4 zeros here
6601 BRKADR EQU 1BH * 4 ; 6CH, 1BH break vector address
6602 TIMADR EQU 1CH * 4 ; 70H, 1CH timer interrupt
6603 DSKADR EQU 1EH * 4 ; address of ptr to disk parameters
6604 SEC9EQU EQU 522h      ; address of disk parameters
6605 CHROUT EQU 29h
6606 LSTDRV EQU 504h
6607
6608 ; determine whether we booted from floppy or hard disk...
6609
6610 00001CDE F6C280        test    dl, 80h      ; boot from floppy ?
6611 00001CE1 7505          jnz     short gothrd  ; no.
6612 00001CE3 31C0          xor     ax, ax      ; indicate boot from drive a
6613                      ; 10/12/2023
6614 00001CE5 A2[821A]      mov     [Boot_Drv], al
6615
6616 00001CE8 31D2          gothrd:  xor     dx, dx ; 0          ; ax = 0-based drive we booted from
6617                      ; bios_l, bios_h set.
6618                      ; cl = number of floppies including fake one
6619                      ; ch = media byte
6620 00001CEA FA            cli
6621 00001CEB 8ED2          mov     ss, dx      ; set stack segment and stack pointer
6622 00001CED BC0007        mov     sp, 700h
6623 00001CF0 FB            sti
6624
6625 00001CF1 51            push    cx ; *          ; save number of floppies and media byte
6626 00001CF2 88EC          mov     ah, ch      ; FAT ID to AH
6627 00001CF4 50            push    ax ; **         ; save boot drive number and media byte
6628
6629 ; let model_byte, secondary_model_byte be set here!!!
6630
6631 00001CF5 B4C0          mov     ah, 0C0h
6632 00001CF7 CD15          int     15h          ; SYSTEM - GET CONFIGURATION (XT after 1/10/86, AT mdl 3x9, CONV, XT286, PS)
6633 00001CF9 7215          jb     short no_rom_system_conf ; just use Model_Byte
6634 00001CFB 80FC00        cmp     ah, 0
6635 00001CFE 7510          jnz     short no_rom_system_conf
6636
6637                      ; 21/12/2022
6638                      ; ds = cs
6639 00001D00 268A4702      mov     al, [es:bx+2] ; [es:bx+ROMBIOS_DESC.bios_sd_modelbyte]
6640 00001D04 A2[AF05]      mov     [model_byte], al
6641                      ; mov [cs:model_byte], al
6642                      ; get/save model byte
6643 00001D07 268A4703      mov     al, [es:bx+3] ; [es:bx+ROMBIOS_DESC.bios_sd_scnd_modelbyte]
6644 00001D0B A2[B005]      mov     [secondary_model_byte], al
6645                      ; mov [cs:secondary_model_byte], al
6646                      ; get/save secondary model byte
6647 00001D0E EB0C          jmp     short turn_timer_on
6648
6649 ; -----
6650
6651 00001D10 BEFFFF        no_rom_system_conf: mov     si, 0FFFFh
6652 00001D13 8EC6          mov     es, si
6653                      ; 21/12/2022
6654 00001D15 26A00E00      mov     al, [es:0Eh] ; get model byte (from 0FFFFh:0Eh)
6655 00001D19 A2[AF05]      mov     [model_byte], al
6656                      ; mov [cs:model_byte], al ; save model byte
6657
6658 00001D1C B020          turn_timer_on:  mov     al, 20h ; ' ' ; turn on the timer
6659 00001D1E E620          out     20h, al      ; Interrupt controller, 8259A.
6660                      ; AKPORT
6661
6662 ; some Olivetti m24 machines have an 8530 serial communications
6663 ; chip installed at io address 50h and 52h. if we're running
6664 ; on one of those, we must inhibit the normal aux port initialization
6665
6666                      ; 21/12/2022
6667                      ; ds = cs
6668 00001D20 803E[AF05]00  cmp     byte [model_byte], 0
6669                      ; cmp byte [cs:model_byte], 0 ; next to last byte in rom bios
6670 00001D25 7510          jnz     short not_olivetti_m24 ; skip for all other machines
6671                      ; (except Olivetti m24)
6672 00001D27 E466          in     al, 66h      ; is 8530 installed?
6673 00001D29 A820          test    al, 20h
6674 00001D2B 740A          jz     short not_olivetti_m24 ; we're done if not
6675 00001D2D B00F          mov     al, 0Fh      ; double check
6676 00001D2F E650          out     50h, al
6677 00001D31 E450          in     al, 50h
6678 00001D33 A801          test    al, 1          ; this test was copied from Olivetti
6679 00001D35 7414          jz     short skip_aux_port_init ; take this branch if 8530 installed
6680
6681 not_olivetti_m24:
6682 00001D37 B003          mov     al, 3          ; init com4
6683 00001D39 E8F00A        call    aux_init
6684 00001D3C B002          mov     al, 2          ; init com3
6685 00001D3E E8EB0A        call    aux_init
6686 00001D41 B001          mov     al, 1          ; init com2
6687 00001D43 E8E60A        call    aux_init
6688 00001D46 30C0          xor     al, al          ; init com1
6689 00001D48 E8E10A        call    aux_init
6690
6691 skip_aux_port_init:
6692 00001D4B B002          mov     al, 2          ; init lpt3
6693 00001D4D E8D40A        call    print_init
6694 00001D50 B001          mov     al, 1          ; init lpt2
6695 00001D52 E8CF0A        call    print_init
6696 00001D55 30C0          xor     al, al          ; init lpt1
6697 00001D57 E8CA0A        call    print_init
6698

```



```

6699 00001D5A 31D2      xor     dx, dx ; 0
6700 00001D5C 8EDA      mov     ds, dx      ; to initialize      print screen vector
6701 00001D5E 8EC2      mov     es, dx
6702
6703      ; 11/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
6704 00001D60 BF3405     mov     di, 534h      ; offset INITSPOT
6705      ;mov    di, INITSPOT      ; 0534h
6706      ;          ; IBMDOS.COM's first cluster - high word
6707      ;          ; 520h (the 2nd entry of root dir) + 14h
6708 00001D63 8805      mov     ax, [di]
6709      ;mov    [firstcluster_hw], ax
6710      ; 06/04/2024
6711 00001D65 2EA3[801A]   mov     [cs:firstcluster_hw], ax
6712
6713 00001D69 31C0      xor     ax, ax
6714      ; 11/12/2023
6715      ; 16/10/2022
6716      ;mov    di, INITSPOT      ; 0534h
6717      ;mov    di, 534h          ; INITSPOT (0000h:0534h)
6718      ;          ; IBM wants 4 zeros here
6719 00001D6B AB      stosw
6720 00001D6C AB      stosw
6721 00001D6D 8CC8      mov     ax, cs      ; fetch segment
6722 00001D6F C7066C00[0E06]   mov     word [BRKADR], cbreak
6723      ;mov    word ptr ds:6Ch, offset      cbreak ; [BRKADR]
6724      ;          ; break entry point
6725 00001D75 A36E00     mov     [BRKADR+2], ax
6726      ;mov    ds:6Eh, ax      ; vector for break
6727 00001D78 C706A400[8206]   mov     word [CHROUT*4], outchr
6728      ;mov    word ptr ds:0A4h, offset outchr      ; [CHROUT*4]
6729 00001D7E A3A600     mov     [CHROUT*4+2], ax
6730      ;mov    ds:0A6h, ax      ; [CHROUT*4+2]
6731 00001D81 BF0400     mov     di, 4
6732 00001D84 BB[1406]     mov     bx, intret ; 19/10/2022
6733      ;mov    bx, offset intret ; intret (cs:intret)
6734      ;          ; will initialize rest of interrupts
6735 00001D87 93      xchg    ax, bx
6736 00001D88 AB      stosw
6737 00001D89 93      xchg    ax, bx      ; location 4
6738 00001D8A AB      stosw      ; cs:
6739 00001D8B 83C704     add     di, 4      ; int 1; location 6
6740 00001D8E 93      xchg    ax, bx      ; skip int 2
6741 00001D8F AB      stosw
6742 00001D90 93      xchg    ax, bx      ; location 12
6743 00001D91 AB      stosw      ; cs:
6744 00001D92 93      xchg    ax, bx      ; int 3; location 14
6745 00001D93 AB      stosw
6746 00001D94 93      xchg    ax, bx      ; location 16
6747 00001D95 AB      stosw      ; cs:
6748 00001D96 89160005     mov     [0500h], dx
6749      ;mov    ds:500h, dx      ; set print screen & break = 0
6750 00001D9A 89160405     mov     [LSTDRV], dx ; [0504h]
6751      ;mov    ds:504h, dx      ; cleanout last drive spec
6752
6753      ; we need to initialize the cs:motorstartup variable from the disk
6754      ; parameter table at sec9. the offsets in this table are defined in
6755      ; the disk_parms struc in msdiskprm.inc. 2 locs
6756
6757 00001D9E A02C05     mov     al, [SEC9+0Ah] ; 16/10/2022
6758      ;mov    al, ds:52Ch      ; [SEC9+DISK_PARMS.DISK_MOTOR_STRT]
6759      ;          ; [522h+0Ah]
6760      ; 21/12/2022
6761      ; ds = 0
6762
6763 00001DA1 2EA2[2601]   mov     [cs:motorstartup], al
6764 00001DA5 2E803E[AF05]FD   cmp     byte [cs:model_byte], 0FDh ; is this an old rom?
6765 00001DAB 720B      jb     short no_diddle ; no
6766 00001DAD C7062B050F02   mov     word [SEC9+09h], 20Fh
6767      ;mov    word ptr ds:52Bh, 20Fh ; [SEC9+DISK_PARMS.DISK_HEAD_STTL], 0200h+NORMSETTLE
6768      ;          ; set head settle and motor start on pc-1 pc-2 pc-xt hal0
6769 00001DB3 C6062205DF   mov     byte [SEC9+0], 0DFh
6770      ;mov    byte ptr ds:522h, 0DFh ; [SEC9+DISK_PARMS.DISK_SPECIFY_1]
6771      ;          ; set 1st specify byte      on pc-1pc-2 pc-xt hal0
6772      no_diddle:
6773 00001DB8 CD12      int     12h      ; MEMORY SIZE -
6774      ;          ; Return: AX = number of contiguous 1K blocks of memory
6775 00001DBA B106      mov     cl, 6
6776 00001DBC D3E0      shl     ax, cl      ; convert memory size to 16-byte blocks      (segment no.)
6777
6778      ; 21/12/2022
6779      ;pop    cx
6780      ;mov    [cs:drvfat], cx ; save drive to load dos, and fat id
6781
6782 00001DBE 50      push    ax ; ***      ; save real top      of memory
6783
6784      ;M068 - BEGIN
6785      ;----- Check if an RPL program is present at TOM and do not tromp over it
6786
6787      ; 21/12/2022
6788      ; ds = 0
6789      ;push    ds
6790      ;push    bx      ; pushes not required but since this
6791      ;          ; happens to be a last minute change
6792      ;          ; & since it is only init code.
6793      ;xor     bx, bx
6794      ;mov     ds, bx
6795
6796      ;mov     bx, ds:0BCh      ; [2Fh*4]
6797 00001DBF 8B1EBC00     mov     bx, [2Fh*4]
6798      ;mov     ds, word ptr ds:0BEh ; [2Fh*4+2]
6799 00001DC3 8E1EBE00     mov     ds, [2Fh*4+2]
6800 00001DC7 817F035250     cmp     word [bx+3], 'RP' ; 'RPL'
6801      ;cmp     word ptr [bx+3], 'PR' ; 'RPL'
6802 00001DCC 750F      jnz     short SkipRPL
6803 00001DCE 807F054C     cmp     byte [bx+5], 'L'
6804      ;cmp     byte ptr [bx+5], 'L'
6805 00001DD2 7509      jnz     short SkipRPL
6806 00001DD4 89C2      mov     dx, ax      ; get TOM into DX
6807 00001DD6 B8064A     mov     ax, 4A06h      ; (multMULT shl 8) + multMULTRPLTOM
6808 00001DD9 CD2F      int     2Fh      ; Get new TOM from any RPL
6809 00001ddb 89D0      mov     ax, dx
6810      SkipRPL:
6811      ; 21/12/2022
6812      ;pop     bx
6813      ;pop     ds
6814      ;M068 - END
6815      ; 21/12/2022
6816 00001DDD 0E      push    cs
6817 00001DDE 1F      pop     ds
6818
6819 00001DDF 83E840     sub     ax, 64      ; room for fatloc segment. (1 kb buffer)
6820      ; 21/12/2022
6821 00001DE2 A3[041A]     mov     [fatloc], ax
6822      ;mov     [cs:fatloc], ax      ; location to read fat

```

```

6823
6824 00001DE5 83E840          sub    ax, 64
6825 00001DE8 A3[061A]        mov     [init_bootseg], ax ; 21/12/2022
6826                          ;mov     [cs:init_bootseg], ax
6827 00001DEB 58               pop     ax ; ***          ; get back real top of memory for
6828
6829                          ; 21/12/2022
6830 00001DEC 59               pop     cx ; **
6831 00001DED 890E[FC19]        mov     [drvfat], cx ; save drive to load dos, and fat id
6832
6833                          ;;mov     dx, 46Dh          ; SYSINIT segment
6834                          ;;mov     dx, 544h          ; 10/12/2023 (PCDOS 7.1 IBMBIO.COM)
6835 00001DF1 BA0405          mov     dx, SYSINITSEG ; 17/10/2022
6836 00001DF4 8EDA            mov     ds, dx
6837
6838                          ; set pointer to resident device driver chain
6839
6840                          ; 17/10/2022
6841 00001DF6 C706[7502][2300]  mov     word [DEVICELIST], res_dev_list
6842                          ;mov     word [273h], res_dev_list
6843                          ;;mov     word ptr ds:273h, offset res_dev_list
6844                          ;;                          ; [SYSINIT+DEVICE_LIST]
6845 00001DFC 8C0E[7702]        mov     [DEVICELIST+2], cs
6846                          ;mov     [275h], cs
6847                          ;;mov     word ptr ds:275h, cs ; [SYSINIT+DEVICE_LIST+2]
6848
6849 00001E00 A3[9402]          mov     [MEMORYSIZE], ax
6850                          ;mov     [292h], ax
6851                          ;;mov     ds:292h, ax      ; [SYSINIT+MEMORY_SIZE]
6852
6853 00001E03 FEC1            inc     cl
6854 00001E05 880E[9802]        mov     [DEFAULTDRIVE], cl
6855                          ;mov     [296h], cl
6856                          ;;mov     ds:296h, cl      ; [SYSINIT+DEFAULT_DRIVE]
6857
6858                          ;mov     word [CURRENTDOSLOCATION], 0AF8h ; 10/12/2023
6859 00001E09 C706[7302]450A    mov     word [CURRENTDOSLOCATION], DOSLOADSEG
6860                          ;mov     word [271h], 83Fh ; (MSDOS.SYS segment)
6861                          ;;mov     word ptr ds:271h, 83Fh ; [SYSINIT+CURRENT_DOS_LOCATION]
6862                          ; dos_load_seg
6863
6864                          ; important: some old ibm hardware generates spurious int 0F's due to bogus
6865                          ; printer cards. we initialize this value to point to an iret only if
6866                          ;
6867                          ; 1) the original segment points to storage inside valid ram.
6868                          ;
6869                          ; 2) the original segment is 0F000:xxxx
6870
6871                          ;;mov     ax, 46Dh          ; SYSINIT segment
6872                          ;;mov     ax, 544h          ; 10/12/2023
6873                          ;mov     ax, SYSINITSEG ; 17/10/2022
6874                          ;mov     es, ax
6875                          ; 21/12/2022
6876 00001E0F 8EC2            mov     es, dx ; SYSINITSEG
6877 00001E11 31C9            xor     cx, cx ; 0
6878 00001E13 8ED9            mov     ds, cx      ; segment 0
6879                          ;mov     ax, ds:3Eh          ; [0Fh*4+2]
6880 00001E15 A13E00          mov     ax, [0Fh*4+2] ; segment for INT 0Fh
6881                          ; 18/10/2022
6882 00001E18 263B06[9402]    cmp     ax, [es:MEMORYSIZE] ; es:292h
6883                          ;cmp     ax, es:292h        ; [ES:SYSINIT+MEMORY_SIZE] ; (condition 1)
6884 00001E1D 7605            jbe     short resetintf
6885 00001E1F 3D00F0          cmp     ax, 0F000h ; (condition 2)
6886 00001E22 750A            jnz     short keepintf
6887
6888 00001E24 C7063C00[1406]    resetintf: mov     word [0Fh*4], intret
6889                          ;mov     word ptr ds:3Ch, offset intret ; [0Fh*4]
6890 00001E2A 8C0E3E00        mov     word [0Fh*4+2], cs
6891                          ;mov     word ptr ds:3Eh, cs ; [0Fh*4+2]
6892
6893                          keepintf:
6894                          ; end important
6895
6896                          ; 17/10/2022
6897                          ; 28/12/2018 - Retro DOS v4.0
6898
6899                          ; (MSDOS 6.0, MSINIT.ASM, 1991)
6900                          ;
6901                          ; we will check if the system has ibm extended keyboard by
6902                          ; looking at a byte at 40:96. if bit 4 is set, then extended keyboard
6903                          ; is installed, and we are going to set keyrd_func to 10h, keysts_func to 11h
6904                          ; for the extended keyboard function. use cx as the temporary register.
6905
6906                          ; 21/12/2022
6907                          ; ds = 0, cx = 0
6908                          ;xor     cx, cx
6909                          ;mov     ds, cx
6910 00001E2E 8A0E9604        mov     cl, [496h] ; get keyboard flag
6911
6912                          ; 21/12/2022
6913 00001E32 0E              push    cs
6914 00001E33 1F              pop     ds
6915
6916 00001E34 F6C110          test    cl, 10h ; extended keyboard ?
6917 00001E37 740A            jz      short org_key ; no, original keyboard
6918
6919                          ; 21/12/2022
6920                          ; ds = cs
6921 00001E39 C606[7E04]10    mov     byte [keyrd_func], 10h ; extended keyboard
6922 00001E3E C606[7F04]11    mov     byte [keysts_func], 11h
6923                          ;mov     byte [cs:keyrd_func], 10h ; extended keyboard
6924                          ;mov     byte [cs:keysts_func], 11h
6925                          ; change for extended keyboard functions
6926
6927                          org_key:
6928                          ; 02/06/2018 - Retro DOS v3.0
6929
6930                          ;*****
6931                          ; will initialize the number of drives
6932                          ; after the equipment call (int 11h) bits 6&7 will tell
6933                          ; the indications are as follows:
6934
6935                          ;
6936                          ; bits      7      6      drives
6937                          ; 0          0      1
6938                          ; 0          1      2
6939                          ; 1          0      3
6940                          ; 1          1      4
6941                          ;*****
6942
6943                          ; 21/12/2022
6944                          ; ds = cs
6945                          ;push     cs
6946                          ;pop      ds
6947                          ;push     cs

```

```

6947 ;pop es
6948
6949 00001E43 E8C70B call cmos_clock_read ; If cmos clock exists,
6950 ; then set the system time according to that.
6951 ; also, reset the cmos clock rate.
6952 ; 18/10/2022
6953 ;mov word ptr BData_start, offset harddrv ;
6954 ; set up pointer to hdrive
6955 ; 02/10/2022
6956 00001E46 C706[0000][4B08] mov word [hdrv_pat], harddrv
6957
6958 00001E4C 58 pop ax ; * ; number of floppies and FAT ID
6959 00001E4D 30E4 xor ah, ah ; chuck fat id byte
6960 00001E4F A2[7500] mov [drvmax], al ; remember which drive is hard disk
6961 00001E52 A2[2501] mov [dsktnum], al ; and set initial number of drives
6962 00001E55 D1E0 shl ax, 1
6963 00001E57 0106[601A] add [last_dskdrv_table], ax
6964
6965 ; 10/12/2023 - Retro DOS v5.0 IO.SYS (IBMBIO.COM)
6966 ; ((MSDOS 6.22 IO.SYS & PC DOS 7.1 IBMBIO.COM))
6967 ; .....
6968 00001E5B 1E push ds
6969 00001E5C B800F0 mov ax, 0F000h
6970 00001E5F 8ED8 mov ds, ax
6971
6972 00001E61 813EEAFF434F cmp word [0FFEAh], 'CO' ; 'COMPAQ'
6973 00001E67 751F jne short skip_mode2
6974 00001E69 813EECF4D50 cmp word [0FFECCh], 'MP'
6975 00001E6F 7517 jne short skip_mode2
6976 00001E71 813EEEF4151 cmp word [0FFEEh], 'AQ'
6977 00001E77 750F jne short skip_mode2
6978
6979 00001E79 B800E4 mov ax, 0E400h ; get advanced system info (COMPAQ ROMBIOS)
6980 00001E7C CD15 int 15h
6981 00001E7E 7208 jc short skip_mode2
6982 ; 10/12/2023
6983 ; PC DOS 7.1 IBMBIO.COM
6984 ; or bx, 0 ; or bx,40h ; enable mode 2
6985 ; (MSDOS 6.0)
6986 ; MSDOS 6.22 IO.SYS
6987 00001E80 83CB40 or bx, 40h ; enable mode 2 (dual harddisk controller)
6988 00001E83 B880E4 mov ax, 0E480h ; set advanced system info (COMPAQ ROMBIOS)
6989 00001E86 CD15 int 15h
6990 skip_mode2:
6991 00001E88 1F pop ds
6992 ; .....
6993
6994 00001E89 B280 mov dl, 80h
6995 00001E8B B408 mov ah, 8
6996 00001E8D CD13 int 13h
6997 ; DISK - DISK - GET CURRENT DRIVE PARAMETERS (XT,AT,XT286,CONV,PS)
6998 ; DL = drive number
6999 ; Return: CF set on error, AH = status code, BL = drivetype
7000 ; DL = number of consecutive drives
7001 ; DH = maximum value for head number, ES:DI -> drive parameter
7001 00001E8F 7204 jc short enddrv
7002 00001E91 8816[5F1A] mov [hnum], dl
7003 enddrv:
7004 ; 21/12/2022
7005 00001E95 0E push cs
7006 00001E96 07 pop es
7007
7008 ; scan the list of drives to determine their type. we have three flavors of
7009 ; diskette drives:
7010 ;
7011 ; 48tpi drives we do nothing special for them
7012 ; 96tpi drives mark the fact that they have changeline support.
7013 ; 3.5" drives mark changeline support and small.
7014 ;
7015 ; the following code uses registers for certain values:
7016 ;
7017 ; dl - physical drive
7018 ; ds:di - points to current bds
7019 ; cx - flag bits for bds
7020 ; dh - form factor for the drive (1 - 48tpi, 2 - 96tpi, 3 - 3.5" medium)
7021
7022 00001E97 30D2 xor dl, dl
7023
7024 ; 21/12/2022
7025 ; ds = cs
7026 ;push cs
7027 ;pop ds
7028
7029 00001E99 C606[2C01]09 mov byte [eot], 9
7030 00001E9E BF[1901] mov di, start_bds ; if we are faking floppy drives we need
7031 ; to set aside two bds for the two fake floppy drives
7032
7033 ; 02/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS)
7034 ; 28/12/2018 - Retro DOS v4.0 (MSDOS 6.0, MSINIT.ASM)
7035
7036 ; check to see if we are faking floppy drives. if not we don't
7037 ; do anything special. if we are faking floppy drives we need
7038 ; to set aside two bds for the two fake floppy drives. we
7039 ; don't need to initialise any fields though. so starting at start_bds
7040 ; use the link field in the bds structure to go to the second bds
7041 ; in the list and initialise it's link field to -1 to set the end of
7042 ; the list. then jump to the routine at dohard to allocate/initialise
7043 ; the bds for harddrives.
7044
7045 00001EA1 803E[131A]01 cmp byte [fakefloppydrv], 1
7046 00001EA6 750B jnz short loop_drive
7047 00001EA8 8B3D mov di, [di] ; [di+BDS.link]
7048 ; di <- first bds link
7049 00001EAA 8B3D mov di, [di] ; [di+BDS.link]
7050 ; di <- second bds link
7051 00001EAC C705FFFF mov word [di], 0FFFFh ; -1 ; set end of link
7052 00001EB0 E98801 jmp dohard ; allocate/initialise bds for harddrives
7053 ;-----
7054
7055 loop_drive:
7056 00001EB3 3A16[7500] cmp dl, [drvmax]
7057 00001EB7 7203 jb short got_more
7058 00001EB9 E97B01 jmp done_drives
7059 ;-----
7060
7061 got_more:
7062 ; 10/12/2023
7063 ;xor cx, cx ; zero all flags
7064 00001EBC 8B3D mov di, [di] ; [di+BDS.link]
7065 ; get next bds
7066 ; .....
7067 ; 10/12/2023 - Retro DOS v5.0
7068 ; (PCDOS 7.1 IBMBIO.COM BIOSDATA:2046h)
7069 00001EBE 83FFFF cmp di, 0FFFFh ; end of link ?
7070 00001EC1 7516 jne short not_last_bds

```

```

7071 00001EC3 88D0          mov     al, dl          ; drive number (0 based)
7072 00001EC5 98            cbw
7073 00001EC6 01C0          add     ax, ax
7074 00001EC8 05[3C05]      add     ax, dskdrvs
7075 00001ECB A3[601A]      mov     [last_dskdrv_table], ax
7076 00001ECE 8B3E[621A]    mov     di, [end_of_bdss]
7077 00001ED2 E8020B      call    xinstall_bds
7078 00001ED5 FE0E[7500]    dec     byte [drvmax]
7079 not_last_bds:
7080 ; .....
7081
7082 00001ED9 B600          mov     dh, 0          ; ff48tpi
7083 ; ..... ; set form factor to 48 tpi
7084 00001EDB C606[101A]28  mov     byte [num_cyl_n], 40 ; 40 tracks per side
7085
7086 ; 21/12/2022
7087 ;push ds
7088 00001EE0 57            push    di
7089 00001EE1 52            push    dx
7090 ;push cx ; not necessary (10/12/2023)
7091 00001EE2 06            push    es ; es=cs=ds ; 21/12/2022
7092
7093 ; .....
7094 ; 10/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
7095 ;xor bx, bx
7096 ;xor cx, cx
7097 00001EE3 52            push    dx ; dl = drive number
7098
7099 00001EE4 B408          mov     ah, 8
7100 00001EE6 CD13          int     13h          ; DISK - DISK - GET CURRENT DRIVE PARAMETERS (XT,AT,XT286,CONV,PS)
7101 ; ..... ; DL = drive number
7102 ; ..... ; Return: CF set on error, AH = status code, BL = drivetype
7103 ; ..... ; DL = number of consecutive drives
7104 ; ..... ; DH = maximum value for head number, ES:DI -> drive parameter
7105
7106 ;jc short noparmsfromrom
7107 00001EE8 58            pop     ax ; al = drive number
7108 00001EE9 7303          jnc     short chk_drv_type
7109 00001EEB E9E600        jmp     noparmsfromrom
7110
7111 chk_drv_type:
7112 ; 10/12/2023
7113 ; ch = low eight bits of maximum cylinder number
7114 ; cl = maximum sector number (bits 5-0)
7115 ; ..... high two bits of maximum cylinder number (bits 7-6)
7116 ; .....
7117 00001EEE 80FB10        cmp     bl, 10h          ; ATAPI Removable Media Device
7118 00001EF1 7554          jne     short not_atapi_removable
7119
7120 ; save ds:si
7121 00001EF3 1E            push    ds
7122 ;push si ; not necessary (10/12/2023)
7123
7124 00001EF4 88C2          mov     dl, al
7125 00001EF6 83EC1A        sub     sp, 26
7126 00001EF9 31C0          xor     ax, ax ; 0
7127 00001EFB 50            push    ax
7128 00001EFC 881E00        mov     ax, 30
7129 00001EFF 50            push    ax
7130 00001F00 89E6          mov     si, sp          ; DS:SI = segment:offset pointer to Result Buffer
7131 00001F02 16            push    ss
7132 00001F03 1F            pop     ds
7133 00001F04 B448          mov     ah, 48h
7134 00001F06 CD13          int     13h          ; DISK - IBM/MS Extension
7135 ; ..... ; GET DRIVE PARAMETERS (DL - drive, DS:SI - buffer)
7136 00001F08 7239          jc     short ext_gdp_err
7137 00001F0A 8B4408        mov     ax, [si+8]      ; physical number of heads
7138 00001F0D A3[0E1A]      mov     [num_heads], ax
7139 00001F10 8B4404        mov     ax, [si+4]      ; physical number of cylinders
7140 00001F13 A3[101A]      mov     [num_cyl_n], ax
7141 00001F16 8A440C        mov     al, [si+0Ch]    ; physical number of sectors per track
7142 00001F19 A2[121A]      mov     [sec_trk], al
7143 00001F1C 3A06[2C01]    cmp     al, [eot]
7144 00001F20 7603          jbe     short _eotok
7145 00001F22 A2[2C01]      mov     [eot], al
7146
7147 _eotok:
7148 ; 10/12/2023
7149 00001F25 31C9          xor     al, al
7150 00001F27 F6440210      xor     cx, cx ; 0
7151 test     byte [si+2], 10h ; information flags
7152 ; ..... ; bit 4 = Device has change line support
7153 jz     short not_chgline_sup
7154 00001F2D 80C902        or     al, 2          ; change line support
7155 or     cl, 2
7156 not_chgline_sup:
7157 add     sp, 30
7158 ;pop si ; (10/12/2023)
7159 pop     ds
7160 ;
7161 pop     es ; es=cs=ds (21/12/2022)
7162 ;pop cx ; (10/12/2023)
7163 pop     dx
7164 pop     di
7165 ;pop ds ; (21/12/2022)
7166
7167 ; 10/12/2023
7168 test    cl, 2
7169 ;test al, 2
7170 ;jz short gotother_j
7171 jz     short gotother
7172 ;or cl, al
7173 mov     byte [fhav96], 1 ; Device has change line support
7174 gotother_j:
7175 jmp     short gotother
7176 ext_gdp_err:
7177 add     sp, 30
7178 ;pop si ; (10/12/2023)
7179 pop     ds
7180 ; 10/12/2023
7181 not_atapi_removable:
7182 ; .....
7183
7184 ; if cmos is bad, it gives es,ax,bx,cx,dh,di=0. cy=0.
7185 ; in this case, we are going to put bogus informations to bds table.
7186 ; we are going to set ch=39,cl=9,dh=1 to avoid divide overflow when
7187 ; they are calculated at the later time. this is just for the diagnostic
7188 ; diskette which need msbio,msdos to boot up before it sets cmos.
7189 ; this should only happen with drive b.
7190
7191 00001F47 80FD00        cmp     ch, 0          ; if ch=0, then cl,dh=0 too.
7192 00001F4A 7505          jnz     short pfr_ok
7193
7194 ;mov ch, 39 ; rom gave wrong info.

```

```

7195             ;mov     cl, 9             ; let's default to 360k.
7196             ; 21/12/2022
7197 00001F4C B90927 mov     cx, 2709h
7198 00001F4F B601   mov     dh, 1
7199
7200 pfr_ok:
7201             ;inc     dh             ; make number of heads 1-based
7202             ;mov     [num_heads], dh ; save parms returned by rom
7203             ; 10/12/2023
7203 00001F51 86D6   xchg     dl, dh
7204 00001F53 30F6   xor      dh, dh
7205 00001F55 42     inc     dx             ; make number of heads 1-based
7206 00001F56 8916[0E1A] mov     [num_heads], dx
7207
7208             ;inc     ch             ; make number of cylinders 1-based
7209             ;and     cl, 3Fh
7210             ;mov     [sec_trk], cl
7211             ;mov     [num_cyl], ch ; assume less than 256 cylinders!!
7212             ; 10/12/2023
7213 00001F5A 88CA   mov     dl, cl
7214 00001F5C 80E23F and     dl, 3Fh
7215 00001F5F 8816[121A] mov     [sec_trk], dl
7216 00001F63 86CD   xchg     cl, ch
7217 00001F65 D0C5   rol      ch, 1
7218 00001F67 D0C5   rol      ch, 1
7219 00001F69 80E503 and     ch, 3
7220 00001F6C 41     inc     cx             ; make number of cylinders 1-based
7221 00001F6D 890E[101A] mov     [num_cyl], cx
7222
7223             ; make sure that eot contains the max number of sec/trk in system of floppies
7224
7225             ;mov     cl, [sec_trk] ; 10/12/2023
7226             ;cmp     cl, [eot] ; may set carry
7227             ;;jbe     short eot_ok
7228             ;; 09/12/2022
7229             ;;jne     short eotok ; wrong ! 14/08/2023
7230             ;; 14/08/2023
7231             ;jbe     short eotok
7232             ;mov     [eot], cl
7233             ; 10/12/2023
7234 00001F71 3A16[2C01] cmp     dl, [eot] ; dl = [sec_trk]
7235 00001F75 7604   jbe     short eotok
7236 00001F77 8816[2C01] mov     [eot], dl
7237
7238 eotok:
7239             ; 10/12/2023
7240             ; !!!
7241             ; (following pops are moved to 'chk_changeline' procedure)
7242             ;pop     es ; es=cs=ds ; 21/12/2022
7243             ;;pop     cx ; (10/12/2023)
7244             ;pop     dx
7245             ;pop     di
7246
7247             ; 21/12/2022
7248             ;pop     ds
7249
7250             ; check for presence of changeline
7251
7252             ; 10/12/2023
7253 %if 0
7254             ; 10/12/2023
7255             ;xor     cx, cx ; 0
7256             ;push    cx
7257             ;push    dx
7258
7259             mov     ah, 15h
7260             int     13h             ; DISK - DISK - GET TYPE (AT,XT2,XT286,CONV,PS)
7261                                     ; DL = drive ID
7262                                     ; Return: CF set on error, AH = disk type (3 = hard drive)
7263                                     ; CX:DX= number of sectors on the media
7264
7265             ; 10/12/2023
7266             pop     dx
7267             ;pop     cx
7268             mov     cx, 0 ; 12/12/2023
7269             jc      short changeline_done
7270             cmp     ah, 2 ; check for presence of changeline
7271             jnz     short changeline_done
7272
7273             ; we have a drive with change line support.
7274             or      cl, 2             ; fchangeline
7275             ; signal type
7276             mov     byte [fhave96], 1 ; remember that we have 96tpi disks
7277
7278 %endif
7279 00001F7B E83800 call     chk_changeline
7280             ;jc      short changeline_done
7281
7282             ; we now try to set up the form factor for the types of media that we know
7283             ; and can recognise. for the rest, we set the form factor as "other".
7284
7285 changeline_done:
7286 00001F7E 803E[101A]28 cmp     byte [num_cyl], 40
7287 00001F83 750B   jnz     short try_80
7288 00001F85 803E[121A]09 cmp     byte [sec_trk], 9
7289 00001F8A 765F   jbe     short nextdrive
7290
7291 gotother:
7292             ; 10/12/2023
7293             ; ch = 0, cl = 2 or 0
7294 00001F8C B607   mov     dh, 7             ; ffother
7295             ; we have a "strange" medium
7296 00001F8E EB5B   jmp     short nextdrive
7297
7298 ;-----
7299
7300             ; 80 cylinders and 9 sectors/track => 720 kb device
7301             ; 80 cylinders and 15 sec/trk => 96 tpi medium
7302
7303 try_80:
7304 00001F90 803E[101A]50 cmp     byte [num_cyl], 80
7305 00001F95 75F5   jnz     short gotother
7306 00001F97 B609   mov     dh, 9             ; ff288
7307                                     ; assume 2.88 MB drive
7308 00001F99 803E[121A]24 cmp     byte [sec_trk], 36 ; is it ?
7309 00001F9E 744B   jz      short nextdrive ; yeah, go update
7310
7311             ; 12/05/2019 (ff144 type will not be used -compatibility problem-)
7312             ; 08/01/2018 - Retro DOS v4.0 feature only ! for 1.44MB diskettes
7313             ;mov     dh, ff144
7314             ;cmp     byte [sec_trk], 18
7315             ;je      short nextdrive
7316 00001FA0 803E[121A]0F cmp     byte [sec_trk], 15
7317 00001FA5 740B   jz      short got96
7318

```

```

7319 00001FA7 803E[121A]09      cmp     byte [sec_trk], 9
7320 00001FAC 75DE              jnz     short gotother
7321
7322 00001FAE B602              mov     dh, 2          ; ffSmall
7323 00001FB0 EB39              jmp     short nextdrive
7324
; -----
7325
7326
7327 00001FB2 B601      got96:    mov     dh, 1          ; ff96tpi
7328 00001FB4 EB35      jmp     short nextdrive
7329
; -----
7330
7331
7332      ; 10/12/2023 - Retro DOS v5.0 IO.SYS/IBMBIO.COM
7333      ; check change line feature (and set fhav96 if there is)
7334      ; (common procedure for 'eotok:' and 'noparmsfromrom:')
7335
7336 00001FB6 59      chk_changel:  pop     cx ; near call return address
7337
7338      ; (pop es, dx, di for 'eotok' and 'noparmsfromrom' procs)
7339 00001FB7 07      pop     es ; es=cs=ds ; 21/12/2022
7340      ; pop     cx ; (10/12/2023)
7341 00001FB8 5A      pop     dx
7342 00001FB9 5F      pop     di ; BDS address/offset
7343
7344 00001FBA 51      push    cx ; near call return address
7345
7346      ;xor     cx, cx ; 0
7347      ;push    cx
7348 00001FBB 52      push    dx
7349
7350 00001FBC B415      mov     ah, 15h
7351 00001FBE CD13      int     13h          ; DISK - DISK - GET TYPE (AT,XT2,XT286,CONV,PS)
7352      ; DL = drive ID
7353      ; Return: CF set on error, AH = disk type (3 = hard drive)
7354      ; CX:DX= number of sectors on the media
7355 00001FC0 5A      pop     dx
7356      ;pop     cx
7357 00001FC1 B90000      mov     cx, 0
7358 00001FC4 720D      jc      short chk_chgl_1
7359
7360 00001FC6 80FC02      cmp     ah, 2          ; is there changeline?
7361 00001FC9 7508      jne     short chk_chgl_2 ; *
7362
7363 00001FCB 80C902      or      cl, 2
7364      ;or      cl, ah ; 2
7365 00001FCE C606[7700]01  mov     byte [fhav96], 1 ; fchangeline
7366      ; cf = 0
7367
7368      chk_chgl_1:
7369 00001FD3 C3      chk_chgl_2:  retn
7370
7371      ;chk_chgl_2:      ; *
7372      ; 10/12/2023
7373      ; ah = 1 ; harddisk type (ah = 3) return not possible here for floppies
7374      ;
7375      ; stc
7376      ; cf = 1
7377      ; retn
7378
; -----
7379
7380      ; we have an old rom, so we either have a 48tpi or 96tpi drive. if the drive
7381      ; has changeline, we assume it is a 96tpi, otherwise we treat it as a 48tpi.
7382
7383      noparmsfromrom:
7384      ; 10/12/2023
7385      ; !!!
7386      ; (following pops are moved to 'chk_changeline' procedure)
7387      ;pop     es ; es=cs=ds ; 21/12/2022
7388      ;pop     cx ; (10/12/2023)
7389      ;pop     dx
7390      ;pop     di
7391
7392      ; 21/12/2022
7393      ;pop     ds
7394      ; 10/12/2023
7395      %if 0
7396      ; 10/12/2023
7397      ;xor     cx, cx ; 0
7398      ;push    cx
7399      ;push    dx
7400
7401
7402      mov     ah, 15h
7403      int     13h          ; DISK - DISK - GET TYPE (AT,XT2,XT286,CONV,PS)
7404      ; DL = drive ID
7405      ; Return: CF set on error, AH = disk type (3 = hard drive)
7406      ; CX:DX= number of sectors on the media
7407
7408      ; 10/12/2023
7409      pop     dx
7410      ;pop     cx
7411      mov     cx, 0 ; 12/12/2023
7412      jc      short nextdrive
7413
7414      cmp     ah, 2          ; is there changeline?
7415      jnz     short nextdrive
7416
7417      or      cl, 2
7418      mov     byte [fhav96], 1 ; fchangeline
7419
7420      %endif
7421      ; 10/12/2023
7422      call    chk_changeline
7423      jc      short nextdrive
7424
7425      ; change line support, [fhav96] = 1
7426      mov     byte [num_cyln], 80
7427      mov     dh, 1          ; ff96tpi
7428      mov     al, 15
7429      cmp     al, [eot]
7430      jbe     short nextdrive
7431      mov     [eot], al
7432
; -----
7433
7434      nextdrive:
7435      ; 10/12/2023
7436      ; ch = 0, cl = 2 or 0
7437
7438      or      cl, 20h          ; fi_own_physical
7439      mov     bh, dl          ; set this true for all drives
7440      ; save int13 drive number
7441
7442      ; we need to do special things if we have a single drive system and are setting
7443      ; up a logical drive. it needs to have the same int13 drive number as its

```



```

7443 ; counterpart, but the next drive letter. also reset ownership flag.
7444 ; we detect the presence of this situation by examining the flag single for the
7445 ; value 2.
7446 00001FF0 803E[7800]02      cmp     byte [single], 2
7447 00001FF5 7505              jnz     short not_special
7448 00001FF7 FECF              dec     bh ; int13 drive number same for logical drive
7449 00001FF9 80F120            xor     cl, 20h ; reset ownership flag for logical drive
7450 not_special:
7451
7452 ; the values that we put in for BDS_RBPB.BPB_HEADS and
7453 ; BDS_RBPB.BPB_SECTORS PER TRACK will only remain if the
7454 ; form factor is of type "ffother".
7455
7456 ;xor     ax, ax ; fill BDS for drive
7457 ;mov     al, [num_heads]
7458 ; 10/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM) ; *
7459 00001FFC A1[0E1A]          mov     ax, [num_heads]
7460 ;mov     [di+36h], ax ; [di+BDS.rheads]
7461 00001FFF 894552            mov     [di+52h], ax ; [di+BDS.rheads] ; *
7462 00002002 31C0              xor     ax, ax ; *
7463 00002004 A0[121A]          mov     al, [sec_trk]
7464 ;mov     [di+34h], ax ; [di+BDS.rsecpertrack]
7465 00002007 894550            mov     [di+50h], ax ; [di+BDS.rsecpertrack] ; *
7466 ;mov     [di+23h], cx ; [di+BDS.flags]
7467 0000200A 894D3F            mov     [di+3Fh], cx ; [di+BDS.flags] ; *
7468 ;mov     [di+22h], dh ; [di+BDS.formfactor]
7469 0000200D 88753E            mov     [di+3Eh], dh ; [di+BDS.formfactor] ; *
7470 00002010 885505            mov     [di+5], dl ; [di+BDS.drivelet]
7471 00002013 887D04            mov     [di+4], bh ; [di+BDS.drivenum]
7472 ;mov     bl, [num_cyls]
7473 ;mov     [di+25h], bl ; [di+BDS.cylinders]
7474 ; 10/12/2023
7475 00002016 A1[101A]          mov     ax, [num_cyls]
7476 00002019 894541            mov     [di+41h], ax ; [di+BDS.cylinders] ; *
7477
7478 0000201C 803E[7800]01      cmp     byte [single], 1 ; Special case for single drive system
7479 00002021 7510              jnz     short no_single
7480 ;mov     byte [single], 2 ; Don't forget we have
7481 ; single drive system
7482 ; 10/12/2023
7483 00002023 FE06[7800]        inc     byte [single] ; [single] = 2
7484 ; 18/12/2022
7485 00002027 80C910            or     cl, 10h
7486 ;or     cx, 10h ; fi_am_mult
7487 ; set that this is one of several drives
7488 ;or     [di+23h], cx ; [di+BDS.flags]
7489 0000202A 094D3F            or     [di+3Fh], cx ; [di+BDS.flags] ; *
7490 ; save flags
7491 0000202D 8B3D              mov     di, [di] ; [di+BDS.link]
7492 ; move to next BDS in list
7493 0000202F FEC2              inc     dl ; add a number
7494 00002031 EBB8              jmp     short nextdrive ; Use same info for BDS as previous
7495 ; -----
7496
7497 no_single:
7498 ;inc     dl
7499 ; 18/12/2022
7500 00002033 42              inc     dx
7501 00002034 E97CFE            jmp     loop_drive
7502 ; -----
7503
7504 ; 11/12/2023 - Retro DOS v5.0 IO.SYS/IBMBIO.COM
7505 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:21E8h)
7506 done_drives:
7507 ;mov     word [di+BDS.link], -1
7508 00002037 C705FFFF            mov     word [di], -1 ; set link to null
7509
7510 ; set up all the hard drives in the system
7511
7512 dohard:
7513 0000203B 8A36[5F1A]        mov     dh, [hnum]
7514 0000203F 08F6              or     dh, dh ; done if no hardfiles
7515 00002041 7459              jz     short static_configure
7516 00002043 B280              mov     dl, 80h
7517
7518 dohard1:
7519 00002045 52              push    dx
7520 00002046 8B3E[621A]        mov     di, [end_of_bdss]
7521 0000204E B700              mov     bl, [drvmax]
7522 00002050 E89902        mov     bh, 0 ; first primary partition (or active)
7523 00002053 7208              call    sethard
7524 00002055 E86A09        jc     short hardfile_err
7525 00002058 7303              call    dmax_check ; error if already 26 drives
7526 0000205A E87A09        jnb     short hardfile_err
7527 ; call    xinstall_bds ; insert new bds into linked list
7528 0000205D 5A              hardfile_err:
7529 ;pop     dx
7530 ;inc     dl ; next hard drive
7531 ; 12/12/2023
7532 0000205E 42              inc     dx
7533 0000205F FECE              dec     dh
7534 00002061 75E2              jnz     short dohard1
7535
7536 ; end of physical drive initialization
7537
7538 ; *** do not change the position of the following statement.
7539 ; *** domini routine will use [drvmax] value for the start of the logical
7540 ; *** drive number of mini disk(s).
7541 00002063 E8CF07        call    domini ; for setting up mini disks, if found
7542
7543 ; -- begin added section
7544
7545 00002066 8A36[5F1A]        mov     dh, [hnum] ; we already know this is >0
7546 0000206A B280              mov     dl, 80h
7547
7548 dohardx1:
7549 0000206C B701              mov     bh, 1 ; do all subsequent primary partitions
7550
7551 dohardx2:
7552 0000206E 52              push    dx
7553 0000206F 53              push    bx
7554 00002070 8B3E[621A]        mov     di, [end_of_bdss]
7555 00002074 8A1E[7500]        mov     bl, [drvmax]
7556 00002078 E87102        call    sethard
7557 0000207B 720E              jc     short dohardx4 ; move to next hardfile if error
7558 0000207D E84209        call    dmax_check ; make sure <=26 drives
7559 00002080 7309              jnb     short dohardx4 ; skip if error
7560 00002082 E85209        call    xinstall_bds ; insert new bds into linked list
7561 00002085 5B              pop     bx ; get partition number
7562 00002086 5A              pop     dx ; restore physical drive counts
7563 00002087 FEC7              inc     bh
7564 00002089 EBE3              jmp     short dohardx2 ; keep looping until we fail
7565 ; -----
7566
7567 dohardx4:
7568 0000208B 5B              pop     bx ; unjunk partition number from stack

```

```

7567 0000208C 5A          pop     dx          ; restore physical drive counts
7568                      ;inc     dl          ; next hard drive
7569                      ; 12/12/2023
7570 0000208D 42          inc     dx
7571 0000208E FECE        dec     dh
7572 00002090 75DA        jnz     short dohardx1
7573
7574                      ; -- end changed section
7575
7576                      ;*****
7577                      ; if more than 2 diskette drives on the system, then it is necessary to remap
7578                      ; the bds chain to adjust the logical drive num (drive letter) with greater
7579                      ; than two diskette drives
7580
7581                      ; new scheme:      if more than 2 diskette drives, first map the bds structure
7582                      ; as usual and then rescan the bds chain to adjust the drive
7583                      ; letters. to do this, scan for disk drives and assign logical
7584                      ; drive number starting from 2 and then rescan diskette drives
7585                      ; and assign next to the last logical drive number of last disk
7586                      ; drive to the 3rd and 4th diskette drives.
7587                      ;*****
7588
7589 00002092 803E[2501]02  cmp     byte [dsktnum], 2 ; >2 diskette drives
7590                      ;jbe     short static_configure ; no - no need for remapping
7591 00002097 7603          jbe     short no_remap
7592 00002099 E8D401        call    remap          ; remapbds chain to adjust driver letters
7593 no_remap:
7594
7595                      ; End of drive initialization.
7596
7597                      ; -----
7598
7599                      ;we now decide, based on the configurations available so far, what
7600                      ;code or data we need to keep as a stay resident code. the following table
7601                      ;shows the configurations under consideration. they are listed in the order
7602                      ;of their current position memory.
7603                      ;
7604                      ;configuration will be done in two ways:
7605                      ;
7606                      ;first, we are going to set "static configuration". static configuration will
7607                      ;consider from basic configuration to endof96tpi configuration. the result
7608                      ;of static configuration will be the address the dynamic configuration will
7609                      ;use to start with.
7610                      ;
7611                      ;secondly, "dynamic configuration" will be performed. dynamic configuration
7612                      ;involves possible relocation of code or data. dynamic configuration routine
7613                      ;will take care of bds tables and at rom fix module thru k09 suspend/resume
7614                      ;code individually. after these operation, [dosdatasg] will be set.
7615                      ;this will be the place sysinit routine will relocate msdos module for good.
7616
7617                      ; -- begin changed section
7618
7619                      ; 1. basic configuration for msbio (endfloppy)
7620                      ; 2. end96tpi ; a system that supports "change line error"
7621                      ; 3. end of bdss ; end of bdss for hard disks
7622                      ; 4. endatrom ; some of at rom fix module.
7623                      ; 5. endcmosclockset; supporting program for cmos clock write.
7624                      ; 6. endk09 ; k09 cmos clock module to handle suspend/resume operation.
7625
7626
7627                      ; 02/10/2022 - Retro DOS v4.0 (MSDOS v5.0 IO.SYS)
7628
7629 static_configure:
7630 0000209C 8B3E[621A]    mov     di, [end_of_bdss]
7631 000020A0 81FF[4C08]    cmp     di, bdss ; 19/10/2022
7632                      ;cmp     di, offset bdss ; did we allocate any hard drive bdss?
7633 000020A4 750D          jnz     short dynamic_configure ; that's the end, then
7634                      ; 18/10/2022
7635 000020A6 BF[4C08]     mov     di, end96tpi
7636                      ;mov     di, offset harddrv ; end96tpi
7637                      ;keep everything up to end96tpi
7638 000020A9 803E[7700]00  cmp     byte [fhav96], 0
7639 000020AE 7503          jnz     short dynamic_configure
7640
7641 000020B0 BF[3808]     mov     di, endfloppy
7642 dynamic_configure:
7643                      ; 20/12/2022
7644                      ;push     cs
7645                      ;pop      es
7646
7647                      ; 10/12/2023
7648                      ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2268h)
7649                      ; (MSDOS 6.22 IO.SYS - BIOSDATA:1C34h)
7650 000020B3 FC          cld ; clear direction flag is not necessary here !?
7651                      ; because there will not be a running program
7652                      ; which will set direction flag as backward (std)
7653
7654                      ; -- end changed section
7655
7656                      ; 21/12/2022
7657                      ; ds = cs <=> es
7658                      ; ss = 0
7659                      ; sp = 700h
7660
7661                      ; 13/12/2023
7662 000020B4 BE00F0        mov     si, 0F000h
7663 000020B7 8EC6          mov     es, si ; ES -> ROM BIOS segment
7664
7665 000020B9 803E[AF05]FC  cmp     byte [model_byte], 0FCh ; AT ?
7666                      ;jnz     short checkcmosclock
7667                      ; 10/12/2023
7668 000020BE 751E          jnz     short checkcompaqbug ; no
7669 000020C0 803E[5F1A]00  cmp     byte [hnum], 0 ; No hard file?
7670                      ;jz     short checkcmosclock
7671                      ;jz     short checkcompaqbug
7672 000020C7 97           xchg     ax, di ; save allocation pointer in ax
7673                      ; 13/12/2023
7674                      ;mov     si, 0F000h
7675                      ;mov     es, si ; ES -> ROM BIOS segment
7676 000020C8 BE[681A]     mov     si, bios_date ; "01/10/84"
7677 000020CB BFF5FF        mov     di, 0FFF5h ; ROM BIOS string is at F000:FFF5
7678 000020CE B90900        mov     cx, 9 ; bdate_1
7679                      ; Only patch ROM for bios 01/10/84
7680 000020D1 F3A6          repe     cmpsb ; check for date + zero on end
7681 000020D3 97           xchg     ax, di ; restore allocation pointer
7682
7683                      ; M015 -- begin changes
7684
7685                      ;jnz     short checkcmosclock
7686                      ; 02/10/2022
7687 000020D4 7508          jnz     short checkcompaqbug
7688
7689                      ; install at rom fix
7690

```



```

7691                                     ; 19/10/2022
7692                                     ;mov    cx, offset endatrom
7693 000020D6 B9[2018]                   mov    cx, endatrom
7694                                     ;mov    si, offset ibm_disk_io
7695 000020D9 BE[F216]                   mov    si, ibm_disk_io
7696 000020DC EB46                       jmp     short install_int13_patch
7697                                     ; -----
7698
7699 ; M065 -- begin changes
7700 ;
7701 ; On certain systems with Western Digital disk controllers, the
7702 ; following detection scheme caused an unpredictable and serious
7703 ; failure. In particular, they've implemented a nonstandard
7704 ; int13(ah=16h) which reconfigures the hard drive, depending on
7705 ; what happens to be at es:[bx] and other memory locations indexed
7706 ; off of it.
7707 ;
7708 ; Compaq was unable to tell us exactly which kind of systems have
7709 ; the bug, except that they guarantee that the bug was fixed in
7710 ; ROM BIOSs dated 08/04/86 and later. We'll check for the COMPAQ
7711 ; string, and then look for date codes before 08/04/86 to decide
7712 ; when to install the hook.
7713
7714 ;checkcmosclock:
7715 ; 02/10/2022
7716 checkcompaqbug:
7717     ; 21/12/2022
7718     ; es = 0F000h
7719     ;mov    ax, 0F000h      ; point to ROM BIOS
7720     ;mov    es, ax
7721
7722     ; 19/10/2022
7723 000020DE 26813EEAFF434F               cmp     word [es:0FFEAh], 'CO'
7724                                     ;cmp     word ptr es:0FFEAh, 'OC' ; look for COMPAQ
7725 000020E5 754B                       jnz     short not_compaq_patch
7726 000020E7 26813EECF4D50               cmp     word [es:0FFEC], 'MP'
7727                                     ;cmp     word ptr es:0FFEC, 'PM'
7728 000020EE 7542                       jnz     short not_compaq_patch
7729 000020F0 26813EEEFF4151               cmp     word [es:0FFEEh], 'AQ'
7730                                     ;cmp     word ptr es:0FFEEh, 'QA'
7731 000020F7 7539                       jnz     short not_compaq_patch
7732
7733 ; We're running on a COMPAQ. Now look at the date code.
7734
7735 000020F9 26A1FBFF                   mov     ax, [es:0FFFBh]      ; get year
7736 000020FD 86E0                       xchg    ah, al
7737
7738 ; 11/12/2023
7739 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:22B9h)
7740 %if 0
7741     cmp     ax, 3836h      ; '68' (NASM syntax) (('86' in MASM syntax))
7742     ja      short checkk09
7743     jz      short chkcompaqbug1
7744     cmp     ax, 3739h      ; '97'
7745     jbe     short not_compaq_patch
7746     stc
7747 chkcompaqbug1:
7748     jb      short do_compaq_patch
7749     mov     ax, [es:0FFF5h]
7750     xchg    ah, al
7751     cmp     ax, 3038h      ; '80'
7752     ja      short not_compaq_patch
7753     jb      short do_compaq_patch
7754     mov     ax, [es:0FFF8h]
7755     xchg    ah, al
7756     cmp     ax, 3034h      ; '40'
7757     jnb     short not_compaq_patch
7758 do_compaq_patch:
7759 %endif
7760
7761     ; 11/12/2023
7762     ; (MSDOS 6.22 IO.SYS - BIOSDATA:1C85h)
7763 000020FF 3D3638                   cmp     ax, 3836h ; 02/10/2022 (NASM syntax)
7764                                     ;cmp     ax, '86'      ; 3836h
7765                                     ; is it 86?
7766 00002102 772E                   ja      short not_compaq_patch
7767 00002104 7218                   jb      short do_compaq_patch
7768 00002106 26A1F5FF               mov     ax, [es:0FFF5h]      ; get month
7769 0000210A 86E0                       xchg    ah, al
7770 0000210C 3D3830                   cmp     ax, 3038h ; 02/10/2022 (NASM syntax)
7771                                     ;cmp     ax, '08'      ; 3038h
7772                                     ; is it 08?
7773 0000210F 7721                   ja      short not_compaq_patch
7774 00002111 720B                   jb      short do_compaq_patch
7775 00002113 26A1F8FF               mov     ax, [es:0FFF8h]      ; get day
7776 00002117 86E0                       xchg    ah, al
7777 00002119 3D3430                   cmp     ax, 3034h ; 02/10/2022 (NASM syntax)
7778                                     ;cmp     ax, '04'      ; 3034h
7779                                     ; is it 04?
7780 0000211C 7314                   jnb     short not_compaq_patch
7781
7782 do_compaq_patch:
7783 0000211E B9[3D18]                   mov     cx, end_compaq_i13hook
7784                                     ;mov     si, endatrom
7785                                     ; 11/12/2023
7786 00002121 BE[2018]                   mov     si, compaq_disk_io ; endatrom
7787
7788 install_int13_patch:
7789 00002124 0E                       push    cs
7790 00002125 07                       pop     es
7791                                     ; 18/10/2022
7792 00002126 893E[B400]               mov     [Orig13], di      ; set new rom bios int 13 vector
7793 0000212A 8C0E[B600]               mov     [Orig13+2], cs
7794 0000212E 29F1                       sub     cx, si            ; size of rom fix module
7795 00002130 F3A4                       rep movsb                ; relocate it
7796
7797 ; M065 -- end changes
7798 ; -----
7799
7800 not_compaq_patch:
7801                                     ; M065
7802                                     ; 17/10/2022
7803 checkcmosclock:
7804                                     ; 18/10/2022
7805 00002132 0E                       push    cs
7806 00002133 07                       pop     es
7807
7808                                     ; 21/12/2022
7809                                     ; ds = cs = es
7810                                     ; ss = 0
7811                                     ; sp = 700h
7812
7813 ; 09/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
7814 %if 0
7815     cmp     byte [havecmosclock], 1 ; cmos clock exists?

```

```

7815             jnz     short checkk09 ; no
7816
7817             mov     word [daycnttoday], di
7818             ;mov    word ptr ds:daycnttoday, di ; set the address for mschar
7819             mov     cx, 209 ; enddaycnttoday - daycnt_to_day
7820             mov     si, daycnt_to_day
7821             rep movsb
7822             mov     word [bintobcd], di
7823             ;mov    word ptr ds:bintobcd, di ; set the address for msclock
7824             ;let original segment stay
7825             ;mov    cx, 11 ; endcmosclockset - bin_to_bcd
7826             ; 08/08/2023
7827             mov     cl, 11
7828             mov     si, bin_to_bcd
7829             rep movsb
7830             %endif
7831
7832             ; 09/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
7833             ; PC DOS 7.1 IBMBIO.COM - BIOSDATA:22F4h
7834             ;push    cs
7835             ;pop     es
7836
7837             checkk09: push    di ; ? ; save ? ; 21/12/2022
7838
7839             ; 13/12/2023 - Retro DOS v4.2 IO.SYS
7840             ; (MSDOS 6.22 IO.SYS - BIOSDATA:1CDAh)
7841             %if 0
7842
7843             mov     ax, 4101h ; wait for bh=es:[di]
7844             mov     bl, 1 ; wait for 1 clock tick
7845             mov     bh, [es:di]
7846             stc ; Assume we will fail
7847             int     15h ; SYSTEM - WAIT ON EXTERNAL EVENT (CONVERTIBLE)
7848             ; AL = condition type, BH = condition compare or mask value
7849             ; BL = timeout value times 55 milliseconds, 00h means no timeout
7850             ; DX = I/O port address if AL bit 4 set
7851             ; 11/12/2023
7852             ; ES:DI = user byte if AL bit 4 clear
7853             %endif
7854
7855             ; 13/12/2023 - Retro DOS v5.0 IBMBIO.COM/IO.SYS
7856             ; (PC DOS 7.1 IBMBIO.COM - BIOSDATA:1CDAh)
7857
7858             ; .....
7859             mov     ax, 4100h ; wait for any external event (al=0)
7860             mov     bl, 4 ; wait for 4 clock ticks
7861             stc ; Assume we will fail
7862             int     15h ; SYSTEM - WAIT ON EXTERNAL EVENT (CONVERTIBLE)
7863             ; AL = condition type, BH = condition compare or mask value
7864             ; BL = timeout value times 55 milliseconds, 00h means no timeout
7865             ; DX = I/O port address if AL bit 4 set
7866             ; .....
7867
7868             pop     di ; ?
7869             jc      short configdone ; 21/12/2022
7870
7871             mov     byte [fhavak09], 1
7872             ; remember we have a k09 type
7873             push    ds
7874             xor     ax, ax
7875             mov     ds, ax
7876
7877             mov     [6Ch*4], di
7878             ;mov    ds:1B0h, di ; [6Ch*4]
7879             ; new int 6Ch handler
7880             ;mov    word ptr ds:1B2h, cs ; [6Ch*4+2]
7881             mov     word [6Ch*4+2], cs
7882             pop     ds
7883             ; 20/12/2022
7884             ; ds = cs = es
7885             mov     si, int6c
7886             mov     cx, endk09-int6c ; 459
7887             ;mov    cx, 459 ; endk09 - int6c
7888             ; size of k09 routine
7889             ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
7890             ; PC DOS 7.1 IBMBIO.COM - BIOSDATA:2315h
7891             mov     si, int_6Ch
7892             mov     cx, endk09-int_6Ch ; 461 in PC DOS 7.1 IBMBIO.COM
7893             rep movsb
7894             ; set up config stuff for sysinit
7895
7896             ; -----
7897             ; Set up config stuff for SYSINIT
7898
7899             ; 17/10/2022
7900             ; SETDRIVE equ SetDrive - DOSBIOSEG_2C7h ; (4D7h for MSDOS 5.0 IO.SYS)
7901             ; GETBP equ GetBp - DOSBIOSEG_2C7h ; (606h for MSDOS 5.0 IO.SYS)
7902             ; 09/12/2022
7903             SETDRIVE equ SetDrive
7904             GETBP equ GetBp
7905
7906             ; 17/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
7907             configdone:
7908             ; 19/04/2024
7909             ;push    cs ; di is final ending address of msbio.
7910             ;pop     ds
7911             add     di, 15 ; round (up) to paragraph
7912             ; 10/12/2022
7913             shr     di, 1
7914             shr     di, 1
7915             shr     di, 1
7916             shr     di, 1
7917             mov     cl, 4
7918             shr     di, cl
7919             ; 10/12/2022
7920             add     di, 70h ; KERNEL_SEGMENT (in fact: IO.SYS loading segment)
7921             ; 19/10/2022 - Temporary !
7922             db      81h, 0C7h, 70h, 0 ; add di, 0070h
7923             mov     [DosDataSg], di ; where the dosdata segment will be
7924
7925             ; 11/12/2023 - Retro DOS v5.0 IBMBIO.COM/IO.SYS
7926             ; -----
7927             ; (PC DOS 7.1 IBMBIO.COM - BIOSDATA:2332h)
7928             ; -----
7929             ; ("IBMDOS.COM" kernel file reading code here, below...)
7930
7931             mov     ax, [drvfat] ; get drive and fat id
7932             ; 22/12/2022
7933             ; Note: SETDRIVES uses AL (drive number) only
7934             mov     bp, SETDRIVE
7935             mov     bp, 5AEh ; 11/12/2023 (PC DOS 7.1 IBMBIO.COM)
7936             ;mov    bp, 4D7h ; set_drive (in dosbios code segment)
7937             ; at 2C7h:4D7h = 70h:2A47h
7938             push    cs ; simulate far call

```

```

7939 00002170 E800F9          call    call_bios_code ; get bds for drive
7940                          ; 06/04/2024
7941                          ; es:di = BDS address
7942 00002173 BD[D606]        mov     bp, GETBP      ; ensure valid bpb is present
7943                          ;mov    bp, 6E4h ; 11/12/2023 (PCDOS 7.1 IBMBIO.COM)
7944                          ;;mov   bp, 606h      ; GetBp (2C7h:606h = 70h:2B76h)
7945 00002176 0E              push    cs
7946 00002177 E8F9F8          call    call_bios_code
7947
7948                          ; resort to funky old segment definitions for now
7949
7950                          ; 22/12/2022
7951                          ;push    es          ; copy bds to ds:di
7952                          ;pop     ds
7953
7954                          ; the following read of es:0000 was spurious anyway. Should look into it.
7955
7956                          ; hmmm. j.k. took out a call to getfat right here a while
7957                          ; back. Apparently it was what actually setup es: for the following
7958                          ; cas----
7959
7960                          ; 22/12/2022
7961                          ;xor     di, di
7962                          ;mov     al, [es:di]    ; get fat id byte
7963                          ;;mov    byte ptr es:drvfat+1, al ; save fat byte
7964                          ;mov     [es:drvfat+1], al
7965                          ;mov     ax, [es:drvfat]
7966
7967                          ; 22/12/2022
7968                          ; ds = cs
7969 ;;; mov     al, [drvfat]
7970
7971                          ; cas -- why do a SECOND setdrive here???
7972
7973                          ; 22/12/2022
7974                          ;push    es          ; save whatever's in es
7975                          ;push    ds          ; copy bds to es:di
7976                          ;pop     es
7977                          ;push    cs          ; copy Bios_Data to ds
7978                          ;pop     ds
7979
7980                          ; 22/12/2022
7981                          ;;; mov     bp, SETDRIVE
7982                          ;;; mov     bp, 4D7h      ; SetDrive (2C7h:4D7h = 70h:2A47h)
7983                          ;;; push    cs          ; simulate far call
7984                          ;;; call    call_bios_code ; get correct bds for this drive
7985
7986                          ; 22/12/2022
7987                          ;push    es          ; copy bds back to ds:di
7988                          ;pop     ds
7989                          ;pop     es          ; pop whatever was in es
7990
7991                          ; Now we load in the MSDOS.SYS file
7992
7993                          ; 22/12/2022
7994                          ; -----
7995                          ; mov     bx, [di+6]      ; [di+BDS.BDS_BPB.BPB_BYTESPERSECTOR]
7996                          ; mov     [cs:md_sectorsize], bx ; used by get_fat_sector proc.
7997                          ; mov     bl, [di+1Fh]    ; [di+BDS.fatsiz]
7998                          ;                          ; get size of fat on media
7999                          ; mov     es:16DEh, bl
8000                          ; mov     [es:fbigfat], bl
8001                          ; mov     cl, [di+8]
8002                          ; mov     ax, [di+17h]    ; [di+BDS.BDS_BPB.BPB_HIDDENSECTORS]
8003                          ; sub     es:16D8h, ax
8004                          ; sub     [es:bios_1], ax ; subtract hidden sectors since we
8005                          ;                          ; need a logical sector number that will
8006                          ;                          ; be used by getclus(diskrd procedure)
8007                          ; mov     ax, [di+19h]    ; [di+BDS.BDS_BPB.BPB_HIDDENSECTORS+2]
8008                          ; sbb     es:16DAh, ax
8009                          ; sbb     [es:bios_h], ax ; subtract upper 16 bits of sector num
8010                          ; -----
8011
8012                          ; 11/12/2023 - Retro DOS v5.0 IO.SYS/IBMBIO.COM
8013                          ; -----
8014 0000217A 268B5D06          mov     bx, [es:di+6] ; [di+BDS.BDS_BPB.BPB_BYTESPERSECTOR]
8015 0000217E 891E[0A1A]        mov     [md_sectorsize], bx ; used by get_fat_sector proc.
8016                          ; 11/12/2023 ; *
8017 00002182 268A5D3B          mov     bl, [es:di+3Bh] ; [di+BDS.fatsiz] ; *
8018                          ;mov     bl, [es:di+1Fh] ; [di+BDS.fatsiz]
8019                          ;                          ; get size of fat on media
8020                          ; mov     [fbigfat], bl
8021 0000218A 268A4D08          mov     cl, [es:di+8]
8022 0000218E 268B4517          mov     ax, [es:di+17h] ; [di+BDS.BDS_BPB.BPB_HIDDENSECTORS]
8023 00002192 2906[FE19]        sub     [First_Data_Sector], ax ; *
8024                          ;sub     [bios_1], ax ; subtract hidden sectors since we
8025                          ;                          ; need a logical sector number that will
8026                          ;                          ; be used by getclus(diskrd procedure)
8027 00002196 268B4519          mov     ax, [es:di+19h] ; [di+BDS.BDS_BPB.BPB_HIDDENSECTORS+2]
8028 0000219A 1906[001A]        sbb     [First_Data_Sector+2], ax ; *
8029                          ;sbb     [bios_h], ax ; subtract upper 16 bits of sector num
8030                          ; -----
8031
8032 0000219E 30ED              xor     ch, ch ; cx = sectors/cluster
8033
8034                          ; the boot program has left the directory at 0:500h
8035
8036                          ; 11/12/2023 - - Retro DOS v5.0 IBMBIO.COM/IO.SYS
8037                          ;push    di
8038 000021A0 1E              push    ds
8039                          ;xor     di, di
8040                          ;mov     ds, di
8041 000021A1 31DB              xor     bx, bx ; 0
8042 000021A3 8EDB              mov     ds, bx
8043 000021A5 8B1E3A05          mov     bx, [53Ah]
8044                          ;mov     bx, ds:53Ah ; (First cluster of the 2nd dir entry
8045                          ;                          ; of root directory in the buffer at 500h)
8046 000021A9 1F              pop     ds
8047 000021AA 8B36[801A]        mov     si, [firstcluster_hw] ; 11/12/2023
8048                          ;                          ; (32 bit cluster number for FAT32 fs)
8049                          ;pop     ds
8050                          ;pop     di
8051
8052                          ; 12/12/2023
8053                          ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2397h)
8054                          ; .....
8055                          ; ds = cs
8056 000021AE A0[081A]          mov     al, [fbigfat]
8057 000021B1 50              push    ax ; (*) save fbigfat flags
8058 000021B2 A0[FC19]          mov     al, [drvfat]
8059 000021B5 0A06[821A]        or      al, [Boot_Drv]
8060 000021B9 757B              jnz     short boot_drv_fixed ; hard disk
8061                          boot_drv_removable:
8062 000021BB 53              push    bx ; calculate cluster count and set fbig or fbigbig flag
                          ;                          ; for removable drives

```

```

8063 000021BC 51          push    cx
8064                      ; 28/12/2023
8065                      ;push    dx ; (not necessary)
8066
8067                      ; 12/12/2023
8068 000021BD 06          push    es
8069 000021BE 1F          pop     ds
8070
8071 000021BF 8B450E        mov     ax, [di+0Eh]    ; [di+BDS.totalsecs16]
8072 000021C2 31D2          xor     dx, dx
8073 000021C4 09C0          or      ax, ax
8074 000021C6 7506          jnz     short prep_totalsecs_ok
8075 000021C8 8B451B        mov     ax, [di+1Bh]    ; [di+BDS.totalsecs32]
8076 000021CB 8B551D        mov     dx, [di+1Dh]
8077
prep_totalsecs_ok:
8078 000021CE 2B4509        sub     ax, [di+9]      ; [di+BDS.resectors]
8079 000021D1 83DA00        sbb     dx, 0
8080 000021D4 50          push    ax
8081 000021D5 52          push    dx
8082 000021D6 8B5D11        mov     bx, [di+11h]    ; [di+BDS.fatsecs16]
8083 000021D9 31C0          xor     ax, ax
8084 000021DB 09DB          or      bx, bx
8085 000021DD 7506          jnz     short prep_fatsecs_ok
8086 000021DF 8B5D1F        mov     bx, [di+1Fh]    ; [di+BDS.fatsecs32]
8087 000021E2 8B4521        mov     ax, [di+21h]
8088
prep_fatsecs_ok:
8089 000021E5 8A4D0B        mov     cl, [di+0Bh]    ; ax:bx = 32 bit count of FAT sectors
8090                      ; [di+BDS.fats]
8091 000021E8 30ED          xor     ch, ch
8092 000021EA F7E1          mul     cx
8093 000021EC 91          xchg    ax, cx
8094 000021ED F7E3          mul     bx
8095 000021EF 01D1          add     cx, dx
8096 000021F1 89C3          mov     bx, ax          ; cx:bx = total (2*) fat sectors
8097 000021F3 5A          pop     dx
8098 000021F4 58          pop     ax              ; dx:ax = totals sectors - reserved sectors
8099 000021F5 29D8          sub     ax, bx
8100 000021F7 19CA          sbb     dx, cx          ; dx:ax = data sectors (includes root dir sectors)
8101 000021F9 8B5D0C        mov     bx, [di+0Ch]    ; [di+BDS.direntries]
8102 000021FC 83C30F        add     bx, 15          ; 16 directory entries per sector
8103                      ; (round up sector count by adding 15)
8104 000021FF B104          mov     cl, 4           ; (rounded) dir entries / 16
8105 00002201 D3EB          shr     bx, cl
8106 00002203 31C9          xor     cx, cx
8107 00002205 29D8          sub     ax, bx
8108 00002207 19CA          sbb     dx, cx          ; dx:ax = data sectors (except root directory sectors)
8109                      ; (will be used for cluster count calculation)
8110 00002209 8A4D08        mov     cl, [di+8]      ; [di+BDS.secpersclus]
8111
8112                      ; 12/12/2023
8113 0000220C 0E          push    cs
8114 0000220D 1F          pop     ds
8115
8116 0000220E 50          push    ax              ; 32 bit division (data sectors / sector per cluster)
8117 0000220F 89D0          mov     ax, dx
8118 00002211 31D2          xor     dx, dx
8119 00002213 F7F1          div     cx
8120 00002215 89C3          mov     bx, ax
8121 00002217 58          pop     ax
8122 00002218 F7F1          div     cx
8123 0000221A 09DB          or      bx, bx          ; 32 bit cluster count if bx > 0
8124 0000221C 7505          jnz     short set_fbigbig_flag ; too big cluster number
8125 0000221E 83F8F6        cmp     ax, 0FFF6h
8126 00002221 7207          jb      short set_fbig_flag
8127
set_fbigbig_flag:
8128 00002223 800E[081A]20      or      byte [fbigfat], 20h ; FAT32 ; fbigbig
8129 00002228 EB0A          jmp     short set_fbig_flag_ok
8130
; -----
8131
set_fbig_flag:
8132
8133 0000222A 3DF60F        cmp     ax, 0FF6h        ; 4096-10
8134                      ; is this 16-bit fat?
8135 0000222D 7205          jb      short set_fbig_flag_ok ; no, small fat
8136 0000222F 800E[081A]40      or      byte [fbigfat], 40h ; FAT16 ; fbig
8137
set_fbig_flag_ok:
8138                      ; 28/12/2023
8139                      ;pop     dx
8140 00002234 59          pop     cx
8141 00002235 5B          pop     bx
8142
boot_drv_fixed:
8143 00002236 31FF          xor     di, di
8144
8145                      ; cx = sectors/cluster
8146                      ; si:bx = first cluster
8147                      ; di = 0
8148
8149                      ; .....
8150
loadit:
8151 00002238 B80405        mov     ax, SYSINITSEG ; 46Dh
8152                      ;mov     ax, 544h    ; 11/12/2023 - PC DOS 7.1 IBMBIO.COM
8153                      ;mov     ax, 46Dh    ; sysinit segment
8154 0000223B 8EC0          mov     es, ax
8155 0000223D 268E06[7302]      mov     es, [es:CURRENTDOSLOCATION] ; 09/12/2022
8156                      ;mov     es, [es:271h]
8157
8158 00002242 E86008        call    getclus         ; read cluster at ES:DI (DI is updated)
8159
; -----
8160
8161                      ; 13/12/2023 - Retro DOS v5.0 (PC DOS 7.1) IBMBIO.COM
8162                      ; (PC DOS 7.1 IBMBIO.COM - BIOSDATA:2431h)
8163
8164                      ;test    byte [cs:fbigfat], 20h ; fbigbig ; FAT32 fs flag
8165                      test    byte [fbigfat], 20h ; fbigbig ; FAT32 fs flag
8166 00002245 F606[081A]20      test    byte [fbigfat], 20h ; fbigbig ; FAT32 fs flag
8167                      ;jz      short iseof
8168                      ; 06/04/2024
8169 0000224A 750D          jnz     short eofbigbig
8170
; -----
8171                      ; 06/04/2024
8172
%if 1
8173                      ; 13/12/2023
8174
iseof:
8175                      ;:test    byte [cs:fbigfat], fbig
8176                      ;test    byte [cs:fbigfat], 40h ; fbig
8177                      ; 12/12/2023
8178                      ; ds = cs
8179                      test    byte [fbigfat], 40h ; fbig
8180 0000224C F606[081A]40      test    byte [fbigfat], 40h ; fbig
8181 00002251 750C          jnz     short eofbig
8182 00002253 81FBF70F        cmp     bx, 0FF7h
8183 00002257 EB09          jmp     short iseofx
8184
%endif
8185
; -----
8186

```

```

8187 eofbigbig: ; si:bx = 32 bit cluster number
8188 cmp si, 0FFFh
8189 jnz short iseofx
8190 ;cmp bx, 0FFF7h
8191 ;jmp short iseofx
8192 ; 06/04/2024
8193 ;jmp short eofbig
8194
8195 ; -----
8196 ; 06/04/2024
8197 %if 0
8198 ; 13/12/2023
8199 iseof:
8200 ;;test byte [cs:fbigfat], fbig
8201 ;test byte [cs:fbigfat], 40h ; fbig
8202 ; 12/12/2023
8203 ; ds = cs
8204 test byte [fbigfat], 40h ; fbig
8205 jnz short eofbig
8206 cmp bx, 0FFF7h
8207 jmp short iseofx
8208 %endif
8209 ; -----
8210
8211 eofbig:
8212 cmp bx, 0FFF7h
8213 iseofx:
8214 jb short loadit ; keep loading until cluster = eof
8215 ; -----
8216
8217 ; 06/04/2024
8218 ;call setdrvparms ;
8219
8220 ; 28/12/2023
8221 pop ax ; (*) restore fbigfat flags
8222 ; (after loading DOS kernel)
8223 ; 06/04/2024
8224 ;mov [cs:fbigfat], al
8225 mov [fbigfat], al
8226
8227 call setdrvparms ; 06/04/2024
8228
8229 ;;jmp far ptr 46Dh:267h ; jmp SYSINIT_SEG:SYSINIT_START
8230 ;;jmp far 46Dh:267h
8231 ; 12/12/2023
8232 ;jmp far 544h:269h ; (PCDOS 7.1 IBMBIO.COM)
8233
8234 jmp SYSINITSEG:SYSINITSTART
8235
8236 ; ===== S U B R O U T I N E =====
8237
8238 ; Following are subroutines to support resident device driver initialization
8239 ;
8240 ;M011 -- note: deleted setup_bdsms and reset_bdsms here
8241
8242 ; M035 -- begin changed section
8243
8244 ;*****
8245 ; module name: remap
8246 ;
8247 ; descriptive name: all the code for himem that could be separated from msbio
8248 ;
8249 ; function: remap the bds chain to adjusted logical drive numbers (drive
8250 ; letters) if more than two diskette drives on the system.
8251 ;
8252 ; scheme: if more than 2 diskette drives, first map the bds structure
8253 ; as usual and then rescans the bds chain to adjust the drive
8254 ; letters. to do this, scan for disk drives and assign logical
8255 ; drive number starting from 2 and then rescans diskette drives
8256 ; and assign next to the last logical drive number of last disk
8257 ; drive to the 3rd and 4th diskette drives.
8258 ;
8259 ; input: none
8260 ; exit: drive letters have been remapped in bds chain
8261 ; exit error: none
8262 ; called from: msinit
8263 ;
8264 ; notes: this function will be called only if more than 2 diskettes are
8265 ; found in the system
8266 ; this function assumes that there are no more than 26 drives assigned
8267 ; this is guaranteed by the code that creates bdss for partitions
8268 ; this function assumes that the first entries in the chain are
8269 ; floppy drives, and all the rest are hard drives
8270 ; will alter the boot drive if necessary to reflect remapping
8271 ;
8272 ;*****
8273
8274 ; 17/10/2022
8275 ; 02/10/2022
8276 ; 15/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
8277 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2464h)
8278 ; (MSDOS 6.22 IO.SYS - BIOSDATA:1D9Eh)
8279
8280 remap: ; proc near
8281
8282 ; 15/12/2023
8283 ; ds = cs
8284 ;mov di, [cs:start_bds] ; get first bds
8285 mov di, [start_bds]
8286
8287 ; search for 1st fixed disk physical drive num
8288
8289 drive_loop:
8290 cmp byte [di+4], 80h ; [di+BDS.drivenum]
8291 ; first hard disk??
8292 jz short fdrv_found ; yes, continue
8293 mov di, [di] ; [di+BDS.link]
8294 ; get next bds, assume segment
8295 cmp di, -1 ; 0FFFFh ; last bds?
8296 jnz short drive_loop ; loop if not
8297 jmp short rmap_exit ; yes, no hard drive on system
8298
8299 ; -----
8300 ; first disk drive bds, now change the logical drive num to 2 and the subsequent
8301 ; logical drive nums to 3, 4, 5 etc.
8302 ; -----
8303
8304 fdrv_found:
8305 mov al, 2 ; start with logical drv num=2
8306
8307 fdrv_loop:
8308 mov [di+5], al ; [di+BDS.drivelet]
8309 mov di, [di] ; [di+BDS.link]
8310 ; ds:di--> next bds
8311 ; inc al ; set num for next drive

```

```

8311 ; 18/12/2022
8312 0000228A 40 inc ax
8313 0000228B 83FFFF cmp di, 0FFFFh ; last hard drive ?
8314 0000228E 75F5 jnz short fdrv_loop ; no - assign more disk drives
8315
8316 ; -----
8317 ; now, rescan and find bds of 3rd floppy drive and assign next drive letter
8318 ; in al to 3rd. if the current drive letter is past z, then do not allocate
8319 ; any more.
8320 ; -----
8321
8322 ;mov di, [cs:start_bds] ; [start_bds]
8323 ; 15/12/2023
8324 00002290 8B3E[1901] mov di, [start_bds] ; get first bds
8325 00002294 8B3D mov di, [di] ; [di+BDS.link]
8326 ; ; ds:di-->bds2
8327 ;mov ah, [cs:dsktnum] ; get number of floppies to remap
8328 00002296 8A26[2501] mov ah, [dsktnum]
8329 0000229A 80EC02 sub ah, 2 ; adjust for a: & b:
8330 remap_loop1:
8331 0000229D 8B3D mov di, [di] ; [di+BDS.link]
8332 ; ; set new num to next floppy
8333 0000229F 884505 mov [di+5], al ; [di+BDS.drivelet]
8334 000022A2 FEC0 inc al ; new number for next floppy
8335 000022A4 FECC dec ah ; count down extra floppies
8336 000022A6 75F5 jnz short remap_loop1
8337
8338 ; now we've got to adjust the boot drive if we reassigned it
8339
8340 ; 15/12/2023
8341 ;mov al, [cs:drvfat]
8342 000022A8 A0[FC19] mov al, [drvfat]
8343 000022AB 3C02 cmp al, 2 ; is ita: or b: ?
8344 000022AD 721D jb short rmap_exit
8345 ;sub al, [cs:dsktnum]
8346 000022AF 2A06[2501] sub al, [dsktnum] ; is it one of the other floppies?
8347 000022B3 7204 jb short remap_boot_flop ; brif so
8348
8349 ; we've got to remap the boot hard drive
8350 ; subtract the number of EXTRA floppies from it
8351
8352 000022B5 0402 add al, 2 ; bootdrv -= (dsktnum-2)
8353 000022B7 EB04 jmp short remap_change_boot_drv
8354 ; -----
8355
8356 ; we've got to remap the boot floppy.
8357 ; add the number of hard drive partitions to it
8358
8359 remap_boot_flop:
8360 ;add al, [cs:drvmax] ; bootdrv += (drvmax-dsktnum)
8361 ; 15/12/2023
8362 000022B9 0206[7500] add al, [drvmax]
8363 remap_change_boot_drv:
8364 ;mov [cs:drvfat], al ; alter msdos.sys load drive
8365 000022BD A2[FC19] mov [drvfat], al
8366 000022C0 FEC0 inc al
8367 000022C2 1E push ds
8368 000022C3 BF0405 mov di, SYSINITSEG ; 46Dh
8369 ;mov di, 544h ; PCDOS 7.1 IBMBIO.COM
8370 ; ;mov di, 46Dh ; SYSINIT segment
8371 000022C6 8EDF mov ds, di
8372 000022C8 A2[9802] mov [DEFAULTDRIVE], al
8373 ;mov ds:296h, al ; [SYSINIT+DEFAULT_DRIVE]
8374 ; ; pass it to sysinit as well
8375 000022CB 1F pop ds ; ds = cs
8376 rmap_exit:
8377 000022CC C3 retn
8378
8379 ; ===== S U B R O U T I N E =====
8380
8381 ; 17/10/2022
8382 ; 02/10/2022 - Retro DOS v4.0 (MSDOS 5.0 -actual-)
8383 ; 28/12/2018 - Retro DOS v4.0 (MSDOS 6.21 -draft-)
8384 ; 02/06/2018 - Retro DOS v3.0 (MSDOS 3.3)
8385 ; 19/03/2018 - Retro DOS v2.0 (MSDOS 2.11)
8386 ; *****
8387 ; getboot - get the boot sector for a hard disk
8388 ;
8389 ; Reads the boot sector from a specified drive into
8390 ; a buffer at the top of memory.
8391 ;
8392 ; dl = int13 drive number to read boot sector for
8393 ; *****
8394
8395 ; 17/10/2022
8396 bootbias equ 200h
8397
8398 getboot: ; proc near
8399
8400 ; 15/12/2023 - Retro DOS v5.0
8401 ; (Modified PCDOS 7.1) IBMBIO.COM/IO.SYS
8402 ; ds = cs
8403
8404 ; 08/04/2018
8405 ; Retro DOS v2.0 (IBMBIO.COM, IBMDOS 2.1)
8406 ; 28/03/2018 - MSDOS 6.0 - MSINIT.ASM, 1991
8407 ; 02/10/2022 - Retro DOS v4.0
8408 ; (disassembled IO.SYS code of MSDOS 5.0)
8409
8410 ;mov ax, [cs:init_bootseg] ; 17/10/2022
8411 ; 15/12/2023
8412 000022CD A1[061A] mov ax, [init_bootseg]
8413 000022D0 8EC0 mov es, ax
8414
8415 ; 17/10/2022
8416 000022D2 B80002 mov bx, bootbias ; 200h
8417 ;mov bx, 200h ; bootbias
8418 ; ; load BX, ES:BX is where sector goes
8419 000022D5 B80102 mov ax, 201h
8420 000022D8 30F6 xor dh, dh
8421 000022DA B90100 mov cx, 1
8422 000022DD CD13 int 13h ; DISK - READ SECTORS INTO MEMORY
8423 ; AL = number of sectors to read, CH = track, CL = sector
8424 ; DH = head, DL = drive, ES:BX -> buffer to fill
8425 ; Return: CF set on error, AH = status, AL = number of sectors read
8426 000022DF 7209 jc short erret
8427 ; 17/10/2022
8428 000022E1 26813EFE0355AA cmp word [es:bootbias+1FEh], 0AA55h
8429 ;cmp word ptr es:3FEh, 0AA55h ; [es:bootbias+1FEh]
8430 ; ; Dave Litton magic word?
8431 000022E8 7401 jz short norm_ret ; yes
8432 erret:
8433 000022EA F9 stc
8434 norm_ret:

```



```

8435 000022EB C3          retn
8436
8437 ; ===== S U B   R O U T I N E =====
8438
8439 ; 28/12/2018 - Retro DOS v4.0
8440
8441 ;*****
8442 ;   sethard - generate bpb for a variable sized hard file. ibm has a
8443 ;   partitioned hard file; we must read physical sector 0 to determine where
8444 ;   our own logical sectors start. we also read in our boot sector to
8445 ;   determine version number
8446 ;
8447 ;   inputs: dl is rom drive number (80...)
8448 ;           bh is partition number (0...)
8449 ;           ds:di points to bds
8450 ;   outputs: carry clear -> bpb is filled in
8451 ;            carry set  -> bpb is left uninitialized due to error
8452 ;   trashes (at least) si, cx
8453 ;   MUST PRESERVE ES!!!!
8454 ;*****
8455
8456 ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
8457 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:24E9h)
8458
8459 sethard: ; proc near
8460 ; 12/08/2023
8461 ; ds = cs = BIOSDATA
8462 000022EC 57      push    di
8463 000022ED 53      push    bx
8464          ;push    ds ; ds = cs = BIOSDATA ; 12/08/2023
8465 000022EE 06      push    es
8466 000022EF 885D05  mov     [di+5],bl ; [di+BDS.drivelet]
8467 000022F2 885504  mov     [di+4],dl ; [di+BDS.drivenum]
8468          ; 16/12/2023
8469 000022F5 804D3F01 or      byte [di+3Fh], 1 ; PCDOS 7.1
8470          ;or      byte [di+23h], 1 ; [di+BDS.flags]
8471          ;           ; fnon_removable
8472 000022F9 C6453E05 mov     byte [di+3Eh], 5 ; PCDOS 7.1
8473          ;mov     byte [di+22h], 5 ; [di+BDS.formfactor]
8474          ;           ; ffHardFile
8475 000022FD C606[081A]00 mov     byte [fbigfat], 0 ; assume 12 bit FAT
8476 00002302 88FE      mov     dh, bh ; partition number
8477 00002304 52      push    dx
8478 00002305 B408      mov     ah, 8
8479 00002307 CD13      int     13h
8480          ; DISK - DISK - GET CURRENT DRIVE PARAMETERS (XT,AT,XT286,CONV,PS)
8481          ; DL = drive number
8482          ; Return: CF set on error, AH = status code, BL = drivetype
8483          ; DL = number of consecutive drives
8484          ; DH = maximum value for head number, ES:DI -> drive parameter
8485
8486          ;inc     dh
8487          ; 16/12/2023 - Retro DOS v5.0
8488 00002309 88F2      mov     dl, dh
8489 0000230B B600      mov     dh, 0
8490 0000230D 42      inc     dx
8491          ;mov     [di+15h], dh ; [di+BDS.heads] ; get number of heads
8492 0000230E 895515  mov     [di+15h], dx
8493 00002311 5A      pop     dx
8494 00002312 7253      jc      short setret ; error if no hard disk
8495          ; 16/12/2023
8496          ;jc      short setret_j
8497
8498          ; 17/04/2024
8499          ;and     cx, 3Fh
8500 00002314 80E13F  and     [di+13h], cx
8501          ;and     cl, 3Fh
8502 00002317 884D13  mov     [di+13h], cl ; [di+BDS.secptrack]
8503
8504          push    dx ; save partition number
8505          call   getboot
8506          pop     dx ; restore partition number
8507          ; 16/12/2023
8508          ;jnc     short chk_act_part
8509 ;setret_j: ;jmp     setret
8510
8511          ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8512          ;chk_act_part:
8513 00002321 31DB      xor     bx, bx ; 0
8514          ;mov     [cs:ep_start_sector], bx
8515          ;mov     [cs:ep_start_sector+2], bx
8516          ;mov     [cs:ep_hidden_secs], bx
8517          ;mov     [cs:ep_hidden_secs+2], bx
8518          ; 16/12/2023
8519          ; ds = cs
8520          ; 20/12/2023
8521          ;mov     [ep_start_sector], bx
8522          ;mov     [ep_start_sector+2], bx
8523 00002323 891E[8923] mov     [ep_hidden_secs], bx
8524 00002327 891E[8B23] mov     [ep_hidden_secs+2], bx
8525
8526 0000232B B8C203      mov     bx, 3C2h ; 1C2h+bootbias
8527
8528 ; The first 'active' partition is 00, the second is 01....
8529 ; then the remainder of the 'primary' but non-active partitions
8530
8531 act_part: test     byte [es:bx-4], 80h ; is the partition active?
8532 0000232E 26F647FC80 jz      short no_act ; no
8533 00002333 740B
8534          ; 16/12/2023
8535          %if 0
8536          ; 16/12/2023
8537          ; reject if partitiontype != 1, 4, 6, 0Bh, 0Ch, 0Eh
8538          cmp     byte [es:bx], 1 ; FAT12
8539          jz      short got_good_act
8540          cmp     byte [es:bx], 4 ; FAT16 CHS (<= 32MB)
8541          jz      short got_good_act
8542
8543          ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8544          cmp     byte [es:bx], 0Bh ; FAT32 CHS
8545          jz      short got_good_act
8546          cmp     byte [es:bx], 0Ch ; FAT32 LBA
8547          jz      short got_good_act
8548          cmp     byte [es:bx], 0Eh ; FAT16 LBA
8549          jz      short got_good_act
8550
8551          cmp     byte [es:bx], 6 ; FAT16 BIG CHS (> 32MB)
8552          jnz     short no_act
8553 ;%else
8554          ; 16/12/2023
8555          mov     al, [es:bx] ; partition type
8556
8557          ; reject if partitiontype != 1, 4, 6, 0Bh, 0Ch, 0Eh
8558          cmp     al, 1 ; FAT12

```

```

8559             je      short got_good_act
8560             cmp     al, 4          ; FAT16 CHS (<= 32MB)
8561             je      short got_good_act
8562
8563             ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8564             cmp     al, 0Bh        ; FAT32 CHS
8565             je      short got_good_act
8566             cmp     al, 0Ch        ; FAT32 LBA
8567             je      short got_good_act
8568             cmp     al, 0Eh        ; FAT16 LBA
8569             je      short got_good_act
8570
8571             cmp     al, 6          ; FAT16 BIG CHS (> 32MB)
8572             jne     short no_act
8573         %endif
8574             ; 16/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
8575             ; check if it is a primary dos partition
8576
8577 00002335 E83300      call     chk_partition_type
8578 00002338 7506       jne     short no_act
8579
8580         got_good_act:
8581 0000233A 08F6       or      dh, dh          ; 11/08/2023
8582                                     ; is this our target partition #?
8583                                     ; (0 = first primary dos or active partition)
8584 0000233C 744F       jz      short set2      ; WE GOT THE ONE WANTED!!
8585 0000233E FECE       dec     dh          ; countdown
8586
8587         no_act:
8588 00002340 83C310     add     bx, 16
8589 00002343 81FB0204  cmp     bx, 402h        ; 202h+bootbias
8590                                     ; last entry done?
8591                                     ; no, process next entry
8592             jnz     short act_part
8593             mov     bx, 3C2h        ; 1C2h+bootbias
8594                                     ; restore original value of bx
8595
8596             ; Now scan the non-active partitions
8597         get_primary:
8598 0000234C 26F647FC80 test     byte [es:bx-4], 80h
8599             jnz     short not_prim ; we've already scanned
8600                                     ; the ACTIVE ones
8601             ; 16/12/2023
8602             %if 0
8603             ; 16/12/2023
8604             ; see if partitiontype == 1, 4, 6, 0Bh, 0Ch, 0Eh
8605             cmp     byte [es:bx], 1      ; FAT12
8606             jz      short got_prim
8607             cmp     byte [es:bx], 4      ; FAT16 CHS (<= 32MB)
8608             jz      short got_prim
8609
8610             ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8611             cmp     byte [es:bx], 0Bh    ; FAT32 CHS
8612             jz      short got_prim
8613             cmp     byte [es:bx], 0Ch    ; FAT32 LBA
8614             jz      short got_prim
8615             cmp     byte [es:bx], 0Eh    ; FAT16 LBA
8616             jz      short got_prim
8617
8618             cmp     byte [es:bx], 6      ; FAT16 BIG CHS (> 32MB)
8619             jnz     short not_prim
8620         %else
8621             ; 16/12/2023
8622             mov     al, [es:bx]          ; partition type
8623
8624             ; see if partitiontype == 1, 4, 6, 0Bh, 0Ch, 0Eh
8625             cmp     al, 1                ; FAT12
8626             je      short got_prim
8627             cmp     al, 4                ; FAT16 CHS (<= 32MB)
8628             je      short got_prim
8629
8630             ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8631             cmp     al, 0Bh              ; FAT32 CHS
8632             je      short got_prim
8633             cmp     al, 0Ch              ; FAT32 LBA
8634             je      short got_prim
8635             cmp     al, 0Eh              ; FAT16 LBA
8636             je      short got_prim
8637
8638             cmp     al, 6                ; FAT16 BIG CHS (> 32MB)
8639             jne     short not_prim
8640         %endif
8641             ; 16/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
8642             ; check if it is a primary dos partition
8643
8644 00002353 E81500      call     chk_partition_type
8645 00002356 7506       jne     short not_prim
8646
8647         got_prim:
8648 00002358 08F6       or      dh, dh          ; is this our target partition?
8649 0000235A 7431       jz      short set2
8650 0000235C FECE       dec     dh
8651
8652         not_prim:
8653 0000235E 83C310     add     bx, 16
8654 00002361 81FB0204  cmp     bx, 402h        ; 202h+bootbias
8655 00002365 75E5       jne     short get_primary ; loop till we've gone through table
8656
8657         setret:
8658 00002367 F9         stc
8659 00002368 E9C703     jmp     ret_hard_err ; error return
8660
8661             ; -----
8662             ; 16/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
8663
8664         chk_partition_type:
8665             ; 16/12/2023
8666             mov     al, [es:bx]          ; partition type
8667
8668             ; see if partitiontype == 1, 4, 6, 0Bh, 0Ch, 0Eh
8669             cmp     al, 1                ; FAT12
8670             je      short chk_ptype_retn
8671             cmp     al, 4                ; FAT16 CHS (<= 32MB)
8672             je      short chk_ptype_retn
8673
8674             ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8675             cmp     al, 0Bh              ; FAT32 CHS
8676             je      short chk_ptype_retn
8677             cmp     al, 0Ch              ; FAT32 LBA
8678             je      short chk_ptype_retn
8679             cmp     al, 0Eh              ; FAT16 LBA
8680             je      short chk_ptype_retn
8681
8682             cmp     al, 6                ; FAT16 BIG CHS (> 32MB)
8683             jne     short chk_ptype_retn
8684         chk_ptype_retn:
8685             ; zf = 1 -> primary DOS partition

```



```

8683                                     ; zf = 0 -> not a primary DOS partition
8684 00002384 c3                          retn
8685
8686                                     ; -----
8687
8688                                     ; 16/12/2023 - Retro DOS v5.0 (Modified PC DOS 7.1)
8689                                     ; (PC DOS 7.1 IBMBIO.COM - BIOS DATA:25B6h)
8690 ep_start_sector:
8691 00002385 00000000                    dd 0
8692 00002389 00000000                    ep_hidden_secs: dd 0
8693
8694                                     ; -----
8695
8696                                     ; until we get the real logical boot record and get the bpb,
8697                                     ; BDS_BPB.BPB_BIGTOTALSECTORS will be used instead of BDS_BPB.BPB_TOTALSECTORS
8698                                     ; for the convenience of the computation.
8699
8700                                     ; at the end of this procedure, if a bpb information is gotten from
8701                                     ; the valid boot record, then we are going to use those bpb information
8702                                     ; without change.
8703
8704                                     ; otherwise, if (hidden sectors + total sectors) <= a word, then we will move
8705                                     ; BDS_BPB.BPB_BIGTOTALSECTORS (low) to BDS_BPB.BPB_TOTALSECTORS and zero out
8706                                     ; BDS_BPB.BPB_BIGTOTALSECTORS entry to make it a conventional bpb format.
8707
8708 set2:
8709                                     ; 12/08/2023
8710                                     ; ds = cs = BIOS DATA segment (0070h)
8711 0000238D 8816[091A]                  mov     [rom_drv_num], dl
8712                                     ; mov     [cs:rom_drv_num], dl
8713                                     ; save the rom bios drive number we are handling now.
8714 00002391 268B4704                    mov     ax, [es:bx+4] ; hidden sectors (start sector)
8715 00002395 268B5706                    mov     dx, [es:bx+6]
8716
8717                                     ; decrement the sector count by 1 to make it zero based. exactly 64k
8718                                     ; sectors should be allowed
8719
8720                                     sub     ax, 1
8721 0000239C 83DA00                      sbb     dx, 0
8722 0000239F 26034708                    add     ax, [es:bx+8] ; sectors in partition
8723 000023A3 2613570A                    adc     dx, [es:bx+10]
8724 000023A7 7305                      jnc     short okdrive
8725 000023A9 800E[081A]80                or      byte [fbigfat], 80h ; ftoobig
8726
8727                                     ; 16/12/2023 - Retro DOS v5.0 (PC DOS 7.1) IBMBIO.COM
8728                                     ;;;
8729 okdrive:
8730                                     ; add     ax, [cs:ep_hidden_secs]
8731                                     ; adc     dx, [cs:ep_hidden_secs+2]
8732                                     ; ds = cs
8733 000023AE 0306[8923]                  add     ax, [ep_hidden_secs]
8734 000023B2 1316[8B23]                  adc     dx, [ep_hidden_secs+2]
8735 000023B6 7305                      jnc     short okdrive_1
8736 000023B8 800E[081A]80                or      byte [fbigfat], 80h ; ftoobig
8737
8738 okdrive_1:
8739 000023BD 26803F0C                    cmp     byte [es:bx], 0Ch ; FAT32 LBA partition ID
8740 000023C1 7418                      je      short set_lba_flag
8741 000023C3 26803F0E                    cmp     byte [es:bx], 0Eh ; FAT16 LBA partition ID
8742 000023C7 7412                      je      short set_lba_flag
8743 000023C9 3B5513                    cmp     dx, [di+13h] ; if dx > [di+BDS.secptrack] then
8744 000023CC 730D                      jnb     short set_lba_flag ; set LBA r/w flag
8745 000023CE F77513                    div     word [di+13h]
8746 000023D1 31D2                    xor     dx, dx
8747 000023D3 F77515                    div     word [di+15h]
8748 000023D6 3D0004                    cmp     ax, 400h ; if ax (cylinder number) >= 1024
8749 000023D9 7204                      jb      short set3
8750 set_lba_flag:
8751 000023DB 804D4004                    or      byte [di+40h], 4 ; fLBArw ; LBA r/w flag
8752                                     ;;;
8753 ;okdrive:
8754                                     ; 16/12/2023
8755 set3:
8756                                     ; mov     ax, [es:bx+4]
8757                                     ; mov     [di+17h], ax ; [di+BDS.hiddensecs]
8758                                     ; ;
8759                                     ; ;
8760                                     ; mov     ax, [es:bx+6] ; BPB_HIDDESECTORS = p->partitionbegin
8761                                     ; mov     [di+19h], ax ; [di+BDS.hiddensecs+2]
8762
8763                                     ; 16/12/2023 - Retro DOS v5.0 (PC DOS 7.1) IBMBIO.COM
8764                                     ;;;
8765 000023DF 268B4704                    mov     ax, [es:bx+4] ; start sector (LBA) of the partition
8766 000023E3 268B5706                    mov     dx, [es:bx+6]
8767                                     ; add     ax, [cs:ep_hidden_secs]
8768                                     ; adc     dx, [cs:ep_hidden_secs+2]
8769 000023E7 0306[8923]                  ; ds = cs
8770                                     ; add     ax, [ep_hidden_secs]
8771                                     ; ; + hidden secs of the extd dos partion
8772 000023EB 1316[8B23]                  adc     dx, [ep_hidden_secs+2]
8773 000023EF 894517                    mov     [di+17h], ax ; [di+BDS.hiddensecs]
8774 000023F2 895519                    mov     [di+19h], dx ; [di+BDS.hiddensecs+2]
8775 000023F5 31C0                    xor     ax, ax ; 0
8776 000023F7 89457B                    mov     [di+7Bh], ax ; [di+BDS.bds_hidden_trks]
8777 000023FA 89450E                    mov     [di+0Eh], ax ; [di+BDS.totalsec16]
8778                                     ;;;
8779 000023FD 268B570A                    mov     dx, [es:bx+10] ; # of sectors (high)
8780 00002401 268B4708                    mov     ax, [es:bx+8] ; # of sectors (low)
8781 00002405 89551D                    mov     [di+1Dh], dx ; [di+BDS.totalsecs32+2]
8782 00002408 89451B                    mov     [di+1Bh], ax ; [di+BDS.totalsecs32]
8783                                     ; bpb->maxsec = p->partitionlength
8784                                     ; cmp     dx, 0
8785                                     ; ja      short okdrive_1
8786                                     ; 16/12/2023
8787 0000240B 09D2                    or      dx, dx
8788 0000240D 7505                      jnz     short set3_read
8789 0000240F 83F840                    cmp     ax, 64 ; if (p->partitionlength < 64)
8790                                     ; jb      short setret ; return -1;
8791 00002412 7264                    jb      short set3_err
8792
8793 ;okdrive_1:
8794                                     ; 16/12/2023
8795 set3_read:
8796 00002414 8B5519                    mov     dx, [di+19h] ; [di+BDS.hiddensecs+2]
8797 00002417 8B4517                    mov     ax, [di+17h] ; [di+BDS.hiddensecs]
8798 0000241A 31DB                    xor     bx, bx
8799                                     ; boot sector number - for mini disk
8800 0000241C 8A5D13                    mov     bl, [di+13h] ; [di+BDS.secptrack]
8801 0000241F 50                    push    ax
8802 00002420 89D0                    mov     ax, dx
8803 00002422 31D2                    xor     dx, dx
8804 00002424 F7F3                    div     bx
8805                                     ; (sectors)dx:ax / (BDS.secptrack)bx =
8806                                     ; (track)temp_h:ax + (sector)dx
8807                                     ; 16/12/2023
8808 %if 0

```

```

8807 ; 17/10/2022
8808 ;mov [cs:temp_h], ax
8809 ; 12/08/2023 (ds=cs)
8810 mov [temp_h], ax
8811 pop ax
8812 div bx
8813 mov cl, dl
8814 inc cl
8815 xor bx, bx
8816 mov bl, [di+15h] ; [di+BDS.heads]
8817 push ax
8818 xor dx, dx
8819 ;mov ax, [cs:temp_h]
8820 mov ax, [temp_h] ; 12/08/2023
8821 div bx
8822 ;mov [cs:temp_h], ax
8823 mov [temp_h], ax ; 12/08/2023
8824 pop ax
8825 div bx ; dl is head, ax is cylinder
8826 ; 12/08/2023 (ds=cs)
8827 cmp word [temp_h], 0
8828 ;cmp word [cs:temp_h], 0
8829 ja short setret_brdg ; exceeds the limit of int 13h
8830 cmp ax, 1024
8831 ja short setret_brdg ; exceeds the limit of int 13h
8832 ; Retro DOS v3.2 note by Erdogan Tan - 28/07/2019
8833 ; **MSDOS code accepts if ax = 1024 but it is nonsense here
8834 ; ('ja' must be 'jnb')
8835 okdrive_2:
8836 ; 28/07/2019
8837 ; dl is head.
8838 ; ax is cylinder
8839 ; cl is sector number (assume less than 2**6 = 64 for int 13h)
8840
8841 ;*** for mini disks ***
8842
8843 cmp word [di+47h], 1 ; [di+BDS.bdsbm_ismini]
8844 ; check for mini disk
8845 jnz short oknotmini ; not mini disk.
8846 add ax, [di+49h] ; [di+BDS.bdsbm_hidden_trks]
8847 ; set the physical track number
8848 oknotmini:
8849 %endif
8850 ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
8851 ;;;
8852 ;mov [cs:saved_word], ax
8853 mov [saved_word], ax
8854 pop ax
8855 div bx
8856 mov cl, dl
8857 inc cl
8858 mov bx, [di+15h] ; [di+BDS.heads]
8859 push ax
8860 xor dx, dx
8861 ;mov ax, [cs:saved_word]
8862 mov ax, [saved_word]
8863 div bx
8864 ;mov [cs:saved_word], ax
8865 mov [saved_word], ax ; not necessary !? (ax must be 0)
8866 pop ax
8867 div bx ; dl is head, ax is cylinder
8868 ; 16/12/2023
8869 push cs
8870 pop es ; (*)
8871 mov bx, disksector ; (**)
8872 ;
8873 test byte [di+40h], 4 ; fLBArw ; LBA read/write flag
8874 jz short set3_chs_read
8875 set3_lba_read:
8876
8877 ; 16/12/2023
8878 %if 0
8879 ;push cs
8880 ;pop es ; (*)
8881 ;mov bx, disksector ; (**)
8882
8883 ;push ds
8884 ;push si
8885 xor ax, ax ; 0
8886 push ax
8887 push ax
8888 mov ax, [di+19h] ; [di+BDS.hiddensectors+2]
8889 push ax
8890 mov ax, [di+17h] ; [di+BDS.hiddensectors]
8891 push ax
8892 push es ; buffer address
8893 push bx
8894 mov ax, 1 ; sector (read) count
8895 push ax
8896 ;mov ax, 16 ; DAP size
8897 mov al, 16
8898 push ax
8899 mov dl, [rom_drv_num] ; ds = cs
8900 mov ax, ss
8901 mov ds, ax ; ds = ss
8902 mov si, sp
8903 ;mov dl, [cs:rom_drv_num]
8904 mov ah, 42h
8905 int 13h ; DISK - IBM/MS Extension
8906 ; EXTENDED READ (DL - drive, DS:SI - disk address packet)
8907 ;pop si
8908 ;pop ds
8909 jnc short set3_lba_read_ok
8910 add sp, 16
8911 ;pop si
8912 ;pop ds
8913 set3_err:
8914 ;jmp setret
8915 jmp ret_hard_err
8916
8917 set3_lba_read_ok
8918 add sp, 16
8919 ;pop si
8920 ;pop ds
8921 jmp short set3_read_ok
8922 %else
8923 ; 16/12/2023
8924 ;push si ; * ; (not necessary)
8925 ;mov si, empty_dap_buff ; dap_buffer
8926 mov si, dap_buffer ; empty_dap_buff
8927 push si
8928 xchg si, di
8929 ; si = BDS
8930 ; di = DAP buffer

```

```

8931 00002452 B81000      mov     ax, 16
8932 00002455 AB          stosw    ; DAP size
8933 00002456 B001      mov     al, 1
8934 00002458 AB          stosw    ; sector (read) count
8935                      ; buffer address
8936 00002459 89D8      mov     ax, bx ; offset disksector
8937 0000245B AB          stosw
8938 0000245C 8CC0      mov     ax, es ; es=ds=cs = BIOSDATA segment
8939 0000245E AB          stosw
8940                      ; sector address (bits 0 to 31)
8941 0000245F 8B4417     mov     ax, [si+17h] ; [di+BDS.hiddensectors]
8942 00002462 AB          stosw
8943 00002463 8B4419     mov     ax, [si+19h] ; [di+BDS.hiddensectors+2]
8944 00002466 AB          stosw
8945                      ; sector address bits 32 to 63 (0)
8946 00002467 31C0      xor     ax, ax ; 0
8947 00002469 AB          stosw
8948 0000246A AB          stosw
8949                      ;xchg di, si
8950 0000246B 89F7      mov     di, si
8951                      ; di = BDS
8952 0000246D 5E          pop     si ; DAP buffer address
8953
8954 0000246E 8A16[091A]  mov     dl, [rom_drv_num] ; ds = cs
8955 00002472 B442      mov     ah, 42h
8956 00002474 CD13      int     13h ; DISK - IBM/MS Extension
8957                      ; EXTENDED READ (DL - drive, DS:SI - disk address packet)
8958                      ;pop si ; *
8959 00002476 7324      jnc     short set3_read_ok
8960
8961                      ;jmp setret
8962 00002478 E9B702     jmp     ret_hard_err
8963
%endif
8964
set3_chs_read:
8965                      cmp     word [di+79h], 1 ; [di+BDS.bdsbm_ismini] ; check for mini disk
8966 0000247B 837D7901  jnz     short oknotmini
8967 0000247F 7503      add     ax, [di+7Bh] ; [di+BDS.bdsbm_hidden_trks]
8968 00002481 03457B     ;;;
8969
8970
8971 oknotmini:
8972 ;*** end of added logic for mini disk
8973
8974 00002484 D0CC      ror     ah, 1 ; move high two bits of cyl to high
8975 00002486 D0CC      ror     ah, 1 ; two bits of upper byte
8976 00002488 80E4C0     and     ah, 0C0h ; turn off remainder of bits
8977 0000248B 08E1      or     cl, ah ; move two bits to correct spot
8978 0000248D 88C5      mov     ch, al ; ch is cylinder (low 8 bits)
8979                      ; cl is sector + 2 high bits of cylinder
8980 0000248F 88D6      mov     dh, dl ; dh is head
8981
8982                      ; 12/08/2023 (ds=cs)
8983 00002491 8A16[091A]  mov     dl, [rom_drv_num]
8984                      ;mov dl, [cs:rom_drv_num] ; dl is drive number
8985
8986 ; cl is sector + 2 high bits of cylinder
8987 ; ch is low 8 bits of cylinder
8988 ; dh is head
8989 ; dl is drive
8990
8991 ; for convenience, we are going to read the logical boot sector
8992 ; into cs:disksector area.
8993
8994 ; read in boot sector using bios disk interrupt. the buffer where it
8995 ; is to be read in is cs:disksector.
8996
8997 ; 16/12/2023
8998 ; es=ds=cs = BIOSDATA segment
8999 ; bx = disksector ; (**)
9000
9001 ;push cs
9002 ;pop es ; (*)
9003
9004 ;mov bx, disksector ; for convenience,
9005 ; we are going to read the logical boot sector
9006 ; into cs:disksector area.
9007 00002495 B80102     mov     ax, 201h
9008 00002498 CD13      int     13h ; DISK - READ SECTORS INTO MEMORY
9009                      ; AL = number of sectors to read, CH = track, CL = sector
9010                      ; DH = head, DL = drive, ES:BX -> buffer to fill
9011                      ; Return: CF set on error, AH = status, AL = number of sectors read
9012
9013 ; 16/12/2023
9014 jnc     short set3_err
9015
9016 ; cs:disksec contains the boot sector. in theory, (ha ha) the bpb in this thing
9017 ; is correct. we can, therefore, suck out all the relevant statistics on the
9018 ; media if we recognize the version number.
9019
set3_read_ok:
9020 ; 11/08/2023
9021 ;mov bx, disksector ; BIOSDATA:014Eh ; MSDOS 6.21 ; 11/08/2023
9022 ; BIOSDATA:0152h ; PCDOS 7.1 IBMBIO.COM
9023
9024 ; 18/12/2023
9025 ;push bx ; +
9026 ;push ax ; (not necessary)
9027
9028 ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
9029 ;;;
9029 0000249C 81BFFE0155AA  cmp     word [bx+1FEh], 0AA55h
9030 000024A2 7541      jne     short invalid_boot_record
9031
9032 ; 16/12/2023
9033 ; 12/08/2023
9034 ; ds = cs = BIOSDATA segment ('disksector:' is in BIOSDATA)
9034 000024A4 803FE9     cmp     byte [bx], 0E9h ; is it a near jump?
9035 000024A7 740B      je      short check_1_ok ; yes
9036 000024A9 803FEB     cmp     byte [bx], 0EBh ; is it a short jump?
9037 000024AC 7537      jne     short invalid_boot_record ; no
9038 000024AE 807F0290     cmp     byte [bx+2], 90h ; yes, is the next one a nop?
9039 000024B2 7531      jne     short invalid_boot_record ; no, invalid bs ; 11/08/2023
9040
check_1_ok:
9041 000024B4 837F1600     cmp     word [bx+16h], 0 ; [bx+BPB_FATSz16]
9042                      ;jz short check_1 ; 16 bit FAT size is 0 if it is FAT32 bs
9043                      ; 16/12/2023
9044 000024B8 740E      jz      short check_2 ; FAT32 bs
9045
9046                      ; FAT16 or FAT12 bs
9047
9048 ;push ds
9049 ;push si ; (not necessary)
9050 000024BA 57          push    di
9051                      ; es=ds=cs = BIOSDATA segment
9052                      ;push es
9053                      ;pop ds
9054

```

```

9055             ;mov     cx, 28
9056 000024BB B90E00    mov     cx, 14 ; *
9057 000024BE 8D7724    lea     si, [bx+24h] ; move offset 36 to 63
9058             ;             ; to offset 64 (28 bytes)
9059 000024C1 8D7F40    lea     di, [bx+40h] ; boot sector offset 64
9060 000024C4 FC        cld     ; (not necessary, 'std' is not used before here)
9061             ;rep movsb
9062 000024C5 F3A5      rep movsw ; *
9063 000024C7 5F        pop     di
9064             ;pop     si
9065             ;pop     ds
9066             ;;;
9067 ; 16/12/2023
9068 %if 0
9069 ;check_1:
9070             ; 12/08/2023
9071             ; ds = cs = BIOSDATA segment ('disksector:' is in BIOSDATA)
9072 cmp     byte [bx], 0E9h
9073 ;cmp     byte [cs:bx], 0E9h ; is it a near jump?
9074 je      short check_1_ok ; yes
9075 cmp     byte [bx], 0EBh
9076 ;cmp     byte [cs:bx], 0EBh ; is it a short jump?
9077 jne     short invalid_boot_record ; no
9078 cmp     byte [bx+2], 90h
9079 ;cmp     byte [cs:bx+2], 90h ; yes, is the next one a nop?
9080 jne     short invalid_boot_record ; no, invalid bs ; 11/08/2023
9081 check_1_ok:
9082 %endif
9083
9084 ; 18/12/2023
9085 %if 0
9086             ; 14/08/2023
9087 check_2:
9088 mov     bx, disksector+11 ; disksector+EXT_BOOT.BPB
9089 ;mov     bx, 159h         ; disksector+EXT_BOOT.BPB
9090             ; point to the bpb in the boot record
9091 ;mov     al, [cs:bx+10] ; [bx+EBPB.MEDIADESCRIPTOR]
9092 mov     al, [bx+10] ; 12/08/2023
9093             ; get the mediadescriptor byte
9094 and     al, 0F0h         ; mask off low nibble
9095 cmp     al, 0F0h         ; is high nibble = 0Fh?
9096 jne     short invalid_boot_record ; no, invalid boot record
9097 ;cmp     word [cs:bx], 512 ; [bx+EBPB.BYTESPERSECTOR]
9098 cmp     word [bx], 512 ; 12/08/2023
9099 jne     short invalid_boot_record ; invalidate non 512 byte sectors
9100
9101 check2_ok: ; yes, mediadescriptor ok.
9102 mov     al, [bx+2] ; 12/08/2023
9103 ;mov     al, [cs:bx+2] ; now make sure that
9104             ; the sectorspercluster is
9105             ; a power of 2
9106             ;
9107             ; [bx+EBPB.SECTORSPERCLUSTER]
9108             ; get the sectorspercluster
9109 %endif
9110             ;;;
9111 check_2:
9112             ; 18/12/2023
9113             ; bx = disksector
9114 000024C8 8A4715    mov     al, [bx+21] ; [bx+EXT_BOOT.BPB+EBPB.MEDIADESCRIPTOR]
9115             ; get the mediadescriptor byte
9116 and     al, 0F0h         ; mask off low nibble
9117 cmp     al, 0F0h         ; is high nibble = 0Fh?
9118 jne     short invalid_boot_record ; no, invalid boot record
9119 cmp     word [bx+11], 512 ; [bx+EXT_BOOT.BPB+EBPB.BYTESPERSECTOR]
9120 jne     short invalid_boot_record ; invalidate non 512 byte sectors
9121
9122 check2_ok: ; yes, mediadescriptor ok.
9123 000024D8 8A470D    mov     al, [bx+13] ; now make sure that
9124             ; the sectorspercluster is
9125             ; a power of 2
9126             ;
9127             ; [bx+EXT_BOOT.BPB+EBPB.SECTORSPERCLUSTER]
9128             ; get the sectorspercluster
9129             ;;;
9130 or      al, al           ; is it zero?
9131 jz      short invalid_boot_record ; yes, invalid boot record
9132
9133 ck_power_of_two:
9134 shr     al, 1           ; shift until first bit emerges
9135 jnc     short ck_power_of_two
9136 jz      short valid_boot_record
9137
9138 invalid_boot_record:
9139             ; 18/12/2023
9140             ;pop     ax
9141             ;pop     bx ; +
9142 000024E5 E96001    jmp     unknown          ; jump to invalid boot record
9143             ; unformatted or illegal media.
9144
9145 ; 16/12/2023
9146 ; -----
9147 ;             ; 12/08/2023
9148 ;setret_brdg:
9149 ;             ; jmp     setret
9150 ; -----
9151
9152 unknown3_0_j:
9153 000024E8 E96101    jmp     unknown3_0       ; legally formatted media,
9154             ; although, content might be bad.
9155 ; -----
9156
9157 valid_boot_record:
9158             ; 18/12/2023
9159             ;pop     ax
9160             ;pop     bx ; +
9161
9162             ; 18/12/2023
9163             ; bx = offset disksector ; +
9164
9165 ; Signature found. Now check version.
9166
9167             ; 14/08/2023
9168 000024EB 817F08322E cmp     word [bx+8], '2.'
9169             ;cmp     word [cs:bx+8], '2.' ; 03/10/2022 (NASM syntax)
9170             ;cmp     word ptr cs:[bx+8], 2E32h ; '2.'
9171 jne     short try5
9172 cmp     byte [bx+10], '0'
9173 ;cmp     byte [cs:bx+0Ah], '0' ; 03/10/2022 (NASM syntax)
9174 ;cmp     byte ptr cs:[bx+0Ah], 30h ; '0'
9175             ; 12/08/2023
9176 ;jnz     short try5
9177 ;jmp     short copybpb
9178 je      short copybpb

```

```

9179
9180
9181
9182
9183
9184
9185
9186
9187
9188
9189
9190
9191
9192 000024F8 E83B02
9193
9194
9195
9196
9197
9198 000024FB 817F08302E
9199
9200
9201 00002500 750C
9202 00002502 8A4707
9203
9204 00002505 2C31
9205
9206 00002507 24FE
9207 00002509 7412
9208 0000250B E93A01
9209
9210
9211
9212
9213
9214
9215
9216 0000250E 817F08332E
9217
9218
9219 00002513 72D3
9220
9221 00002515 7506
9222
9223 00002517 807F0A31
9224
9225
9226 0000251B 72CB
9227
9228
9229
9230
9231
9232
9233
9234
9235
9236
9237
9238
9239
9240
9241
9242
9243
9244
9245
9246
9247 0000251D BE[5D01]
9248
9249 00002520 29C9
9250
9251 00002522 8A4C05
9252
9253
9254
9255
9256
9257
9258
9259
9260
9261
9262
9263
9264
9265
9266
9267
9268
9269 00002525 803E[9401]29
9270
9271
9272
9273
9274
9275 0000252A 7538
9276
9277
9278
9279
9280
9281
9282
9283
9284
9285
9286
9287
9288
9289
9290
9291
9292
9293
9294
9295
9296
9297
9298
9299
9300
9301
9302

```

```

;-----
; 12/08/2023
; setret_brdg:
; jmp setret
;-----
; unknown3_0_j:
; jmp unknown3_0 ; legally formatted media,
; ; although, content might be bad.
;-----

try5:
    call cover_fdisk_bug

; see if it is an os2 signature

; 12/08/2023
; ds = cs = BIOSDATA segment
cmp word [bx+8], '0.'
; cmp word [cs:bx+8], '0.' ; 03/10/2022 (NASM syntax)
; cmp word ptr cs:[bx+8], 2E30h ; '0.'
jne short no_os2
mov al, [bx+7] ; 12/08/2023
mov al, [cs:bx+7] ; 17/10/2022 (NASM syntax)
sub al, '1'
sub al, 31h ; '1'
and al, 0FEh
jz short copybpb ; accept either '1' or '2'
jmp unknown
;-----

; no os2 signature, this is to check for real dos versions

no_os2:
; 12/08/2023
; ds = cs = BIOSDATA
cmp word [bx+8], '3.'
; cmp word [cs:bx+8], '3.' ; 03/10/2022 (NASM syntax)
; cmp word ptr cs:[bx+8], 2E33h ; '3.'
jb short unknown3_0_j ; must be 2.1 boot record.
; do not trust it, but still legal.
jnz short copybpb ; honor os2 boot record
; or dos 4.0 version
cmp byte [bx+10], '1' ; 12/08/2023
; cmp byte [cs:bx+10], '1'
; cmp byte ptr cs:[bx+0Ah], 31h ; '1'
jb short unknown3_0_j ; if version >= 3.1, then o.k.

copybpb:
; 03/10/2022

; we have a valid boot sector. use the bpb in it to build the
; bpb in bios. it is assumed that only
; BDS_BPB.BPB_SECTORS PER CLUSTER
; BDS_BPB.BPB_ROOT ENTRIES, and
; BDS_BPB.BPB_SECTORS PER FAT
; need to be set (all other values in already). fbigfat is also set.

; if it is non fat based system, then just copy the bpb from the boot sector
; into the bpb in bds table, and also set the boot serial number, volume id,
; and system id according to the boot record.
; for the non_fat system, don't need to set the other value. so just do goodret.

; 17/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2787h)
; ;
; ; 17/12/2023
mov si, disksector+11
sub ch, ch ; ; (ch may be > 0)
sub cx, cx ; 0
; mov cl, [disksector+16] ; BPB_NumFATS
mov cl, [si+5] ; number of FATs

; NOTE: This check is not proper for FAT32 boot sector (standard spec)
; (after PCDOS 7.1). So, it is not existing in windows ME IO.SYS
; Erdogan Tan - 01/09/2023 ((IBMBIO.COM 7.1 disassembly note))

; cmp word ptr cs:disksector+4Dh, 0 ; ???
; cmp word [disksector+4Dh], 0
; jnz short check_3

; 17/12/2023
; check extended boot signature (0x29)
; ;
; ; (***) NOTE: 28 bytes of FAT16/FAT12 boot sector from offset 36
; ; have been moved to offset 64 (see label 'check_1_ok:' above)
; ; ((now, BS_BootSig is at offset 66 even if it was at offset 38))

; cmp cs:disksector+42h, 29h ; BS_BootSig (FAT32)
; cmp byte [disksector+42h], 29h ; BS_BootSig (***)
; jmp short check_4

check_3:
; cmp cs:disksector+26h, 29h ; BS_BootSig (FAT16/FAT12)
; cmp byte [disksector+26h], 29h ; (***)

check_4:
jnz short copybpb_fat ; conventional fat system

; 17/12/2023
%if 0
; 10/12/2022
; (number of FATs optimization)
mov si, disksector+11 ; disksector+0Bh
; mov cl, [cs:disksector+10h] ; Number of FATs (may be 2 or 1)
; mov cl, [cs:si+05h]
; 12/08/2023
; ds = cs = BIOSDATA segment (0070h)
mov cl, [si+05h] ; number of FATs

cmp byte [si+1Bh], 29h ; 12/08/2023
; cmp byte [cs:si+1Bh], 29h ; 10/12/2022
; cmp byte [cs:disksector+26h], 29h ; 17/10/2022
; ; [disksector+EXT_BOOT.SIG]
; ; EXT_BOOT_SIGNATURE
jnz short copybpb_fat ; conventional fat system

; 03/10/2022
; 29/12/2018 - Retro DOS v4.0 modification note:
; Regarding 'fat_big_small' part of this (MSDOS 6.0) code
; ; number of FATs must be 2 ; =*?=
; ; (Otherwise, '# of data sectors' would be calculated as wrong!!!)
; ;
; cmp byte [disksector+EXT_BOOT.BPB+EBPB.NUMBEROFFATS], 2 ; =*?=

```

```

9303 ; 10/12/2022
9304 ;cmp byte [cs:disksector+10h], 0
9305 ; ; [disksector+EXT_BOOT.BPB+EBPB.NUMBEROFFATS]
9306 ;jnz short copybpb_fat ; a fat system.
9307 or cl, cl ; [cs:disksector+10h]
9308 jnz short copybpb_fat ; a fat system.
9309 %endif
9310
9311 ; 17/12/2023 - Retro DOS v5.0
9312 ;cmp byte [cs:disksector+10h], 0 ; BPB.fats
9313 ;cmp byte [disksector+10h], 0 ; BPB_NumFATS
9314 ;jnz short copybpb_fat ; a fat system
9315 ; 17/12/2023
9316 ; cl = [disksector+10h]
9317 0000252C 20C9 and cl, cl ; 0 ?
9318 0000252E 7534 jnz short copybpb_fat ; a fat system
9319
9320 ; non fat based media.
9321
9322 00002530 57 push di ; BDS
9323 ; 12/08/2023
9324 ;push ds ; ds = cs = BIOSDATA segment
9325
9326 ; 17/12/2023
9327 ; es = ds = cs
9328 ;push ds
9329 ;pop es
9330
9331 ; 12/08/2023
9332 ; ds = cs
9333 ;push cs
9334 ;pop ds
9335
9336 ; 10/12/2022
9337 ; (number of FATs optimization)
9338 ; SI = disksector+11
9339 ; 17/10/2022
9340 ;mov si, 159h ; disksector+EXT_BOOT.BPB
9341 ;mov si, disksector+11
9342 00002531 83C706 add di, 6 ; add di,BDS.BPB
9343
9344 ; just for completeness, we'll make sure that total_sectors and
9345 ; big_total_sectors aren't both zero. I've seen examples of
9346 ; this on DOS 3.30 boot records. I don't know exactly how it
9347 ; got that way. If it occurs, then use the values from the
9348 ; partition table.
9349
9350 ; 17/12/2023
9351 ; cx = 0
9352 ; 18/12/2022
9353 ;sub cx, cx
9354
9355 ;cmp word [cs:si+8], 0 ; [cs:si+EBPB.TOTALSECTORS]
9356 ;jnz short already_nonz
9357 ; ; how about big_total?
9358 ;cmp word [cs:si+15h], 0 ; [cs:si+EBPB.BIGTOTALSECTORS]
9359 ;jnz short already_nonz ; we're okay if any are != 0
9360 ;cmp word [cs:si+17h], 0 ; [cs:si+EBPB.BIGTOTALSECTORS+2]
9361 ;jnz short already_nonz
9362
9363 ; 12/08/2023
9364 ; ds = cs = BIOSDATA segment (0070h)
9365
9366 ; 17/12/2023
9367 ; 12/08/2023
9368 00002534 394C08 cmp [si+8], cx ; 0 ; [si+EBPB.TOTALSECTORS]
9369 00002537 751C jnz short already_nonz
9370 ; ; how about big_total?
9371 00002539 394C15 cmp [si+15h], cx ; 0 ; [si+EBPB.BIGTOTALSECTORS]
9372 0000253C 7517 jnz short already_nonz ; we're okay if any are != 0
9373 0000253E 394C17 cmp [si+17h], cx ; 0 ; [si+EBPB.BIGTOTALSECTORS+2]
9374 00002541 7512 jnz short already_nonz
9375
9376 ; now let's copy the values from the partition table (now in the BDS)
9377 ; into the BPB in the boot sector buffer, before they get copied back.
9378
9379 00002543 8B4508 mov ax, [di+8] ; [di+BDS.totalsecs16]
9380 ; 12/08/2023
9381 ;mov [cs:si+8], ax ; [cs:si+EBPB.TOTALSECTORS]
9382 00002546 894408 mov [si+8], ax
9383 00002549 8B4515 mov ax, [di+15h] ; [di+BDS.totalsecs32]
9384 ;mov [cs:si+15h], ax ; [cs:si+EBPB.BIGTOTALSECTORS]
9385 0000254C 894415 mov [si+15h], ax
9386 0000254F 8B4517 mov ax, [di+17h] ; [di+BDS.totalsecs32+2]
9387 ;mov [cs:si+17h], ax ; [cs:si+EBPB.BIGTOTALSECTORS+2]
9388 00002552 894417 mov [si+17h], ax
9389
9390 already_nonz:
9391 ; 18/12/2022
9392 ; cx = 0
9393 ;mov cl, 25
9394 ;mov cx, 25 ; A_BPB.size - 6 ; Use SMALL version!
9395 ; 17/12/2023 - Retro DOS v5.0
9396 00002555 B135 mov cl, 53 ; PCDOS 7.1 IBMBIO.COM
9397 ; BDS.BPB size (25 + 28 for FAT32 parms)
9398 00002557 F3A4 rep movsb
9399 ;pop ds
9400 ; 12/08/2023
9401 ; ds = cs
9402 ;pop bp ; ds (on top of stack) = BIOSDATA
9403 00002559 5F pop di ; BDS
9404 ;push es
9405 ;push ds
9406 ;pop es
9407 ;push cs
9408 ;pop ds
9409 ; 12/08/2023
9410 ;mov es, bp
9411 ; ds = cs = es
9412
9413 ; 14/08/2023
9414 0000255A BD[4F08] mov bp, MOVMEDIAIDS ; mov_media_ids
9415 ; 18/12/2022
9416 ;mov bp, mov_media_ids
9417 ;mov bp, 751h ; mov_media_ids
9418 ; at 2C7h:751h = 70h:2CC1h
9419 ; set volume id, systemid, serial.
9420 0000255D 0E push cs ; simulate far call
9421 0000255E E812F5 call call_bios_code
9422 ; 12/08/2023
9423 ; ds = cs = es
9424 ;push es
9425 ;pop ds
9426 ;pop es

```



```

9427 00002561 E9C701          jmp     goodret
9428
9429          ; -----
9430
9431          ; ***** cas ---
9432          ; IBM DOS 3.30 doesn't seem to mind that the TOTAL_SECTORS and
9433          ; BIG_TOTAL_SECTORS field in the boot sector are 0000. This
9434          ; happens with some frequency -- perhaps through some OS/2 setup
9435          ; program. We haven't actually been COPYING the TOTAL_SECTORS
9436          ; from the boot sector into the DPB anyway, we've just been using
9437          ; it for calculating the fat size. Pretty scary, huh? For now,
9438          ; we'll go ahead and copy it into the DPB, except in the case
9439          ; that it equals zero, in which case we just use the values in
9440          ; the DPB from the partition table.
9441
9442          ; 17/10/2022
9443          ; MOV MEDIAIDS equ mov_media_ids - DOSBIOSEG_2C7h ; (751h for MSDOS 5.0 IO.SYS)
9444          ; CLEARIDS equ clear_ids - DOSBIOSEG_2C7h ; (5D9h for MSDOS 5.0 IO.SYS)
9445          ; 09/12/2022
9446          MOV MEDIAIDS equ mov_media_ids
9447          CLEARIDS equ clear_ids
9448          ; 11/09/2023
9449          CLEARIDS_X equ clear_ids_x
9450
9451          copybpb_fat:
9452          ; 17/12/2023
9453          ; ch = 0, cl = number of FATs
9454          ; 10/12/2022
9455          ; (number of FATs optimization)
9456          ; SI = disksector+11
9457          ; 17/10/2022
9458          ; mov si, disksector+11
9459          ; mov si, 159h ; disksector+EXT_BOOT.BPB
9460          ; cs:si -> bpb in boot
9461
9462          ; 17/12/2023
9463          ; dx = 0
9464          ; 18/04/2024 (BugFix)
9465          xor dx, dx
9466
9467          ; 12/08/2023
9468          ; ds = cs = BIOSDATA segment (0070h)
9469          mov ax, [si+8]
9470          ; mov ax, [cs:si+8] ; [cs:si+EBPB.TOTALSECTORS]
9471          ; get totsec from boot sec
9472          or ax, ax
9473          jnz short copy_totsec ; if non zero, use that
9474          mov ax, [si+15h] ; 12/08/2023
9475          ; mov ax, [cs:si+15h] ; [cs:si+EBPB.BIGTOTALSECTORS]
9476          ; get the big version
9477          ; (32 bit total sectors)
9478          mov dx, [si+17h] ; 12/08/2023
9479          ; mov dx, [cs:si+17h] ; [cs:si+EBPB.BIGTOTALSECTORS+2]
9480          ; 10/12/2022
9481          ; (number of FATs optimization)
9482          ; CL = number of FATs (2 or 1)
9483          mov bx, dx ; see if it is a big zero
9484          or bx, ax
9485          jnz short copy_totsec
9486          ; screw it. it was bogus.
9487          mov ax, [di+1Bh] ; [di+BDS.totalsecs32]
9488          mov dx, [di+1Dh] ; [di+BDS.totalsecs32+2]
9489          jmp short fat_big_small
9490
9491          ; mov cx, dx
9492          ; or cx, ax ; see if it is a big zero
9493          ; jz short totsec_already_set ; screw it. it was bogus.
9494          copy_totsec:
9495          mov [di+1Bh], ax ; [di+BDS.totalsecs32]
9496          ; make DPB match boot sec
9497          mov [di+1Dh], dx ; [di+BDS.totalsecs32+2]
9498
9499          ; 10/12/2022
9500          ; totsec_already_set:
9501          ; mov ax, [di+1Bh] ; [di+BDS.totalsecs32]
9502          ; mov dx, [di+1Dh] ; [di+BDS.totalsecs32+2]
9503
9504          ; determine fat entry size.
9505          fat_big_small:
9506
9507          ; at this moment dx;ax = total sector number
9508
9509          ; Do not assume 1 reserved sector. Update the reserved sector field in BDS
9510          ; from the BPB on the disk
9511
9512          ; 12/08/2023
9513          ; ds = cs = BIOSDATA segment (0070h)
9514
9515          mov bx, [si+3]
9516          ; mov bx, [cs:si+3] ; [cs:si+EBPB.RESERVEDSECTORS]
9517          ; get #reserved_sectors from BPB
9518          mov [di+9], bx ; [di+BDS.reseectors]
9519          ; update BDS field
9520          sub ax, bx
9521          sbb dx, 0 ; update the count
9522          ; 12/08/2023
9523          mov bx, [si+0Bh]
9524          ; mov bx, [cs:si+0Bh] ; [cs:si+EBPB.SECTORSPERFAT]
9525          ; bx = sectors/fat
9526          mov [di+11h], bx ; [di+BDS.fatsecs]
9527          ; set in bds bpb
9528          ; 17/12/2023 - Retro DOS v5.0
9529          ; (PCDOS 7.1 IBMBIO.COM)
9530          push bx ; FAT sectors
9531          or bx, bx
9532          jnz short fat_16bit
9533
9534          ; 17/12/2023
9535          %if 0
9536          sub ax, [si+19h] ; FAT32 file system (PCDOS 7.1 BUG!)
9537          ; BPB.FATSz32
9538          sbb dx, [si+1Bh] ; BPB.FATSz32+2 (PCDOS 7.1 BUG!)
9539          ; dx:ax = partition size - (one FAT sectors + reserved sects)
9540          mov bx, [si+19h] ; BPB.FATSz32
9541          mov [di+1Fh], bx ; [di+BDS.fatsecs32]
9542          mov bx, [si+1Bh] ; BPB.FATSz32+2
9543          mov [di+21h], bx ; [di+BDS.fatsecs32+2]
9544          mov bx, [si+1Dh] ; BPB.BPB_ExtFlags
9545          mov [di+23h], bx ; [di+BDS.extflags]
9546          mov bx, [si+1Fh] ; BPB.FSVer
9547          mov [di+25h], bx ; [di+BDS.fsver]
9548          mov bx, [si+21h] ; BPB.RootClus
9549          mov [di+27h], bx ; [di+BDS.rootdirclust]
9550          mov bx, [si+23h] ; BPB.RootClus+2

```

```

9551      mov     [di+29h], bx      ; [di+BDS.rootdirclust+2]
9552      mov     bx, [si+25h]      ; BPB.FSInfo
9553      mov     [di+2Bh], bx      ; [di+BDS.fsinfo]
9554      mov     bx, [si+27h]      ; BPB.FSInfo+2
9555      mov     [di+2Dh], bx      ; [di+BDS.fsinfo+2]
9556      jmp     short fat_32bit    ; PCDOS 7.1 BUG! Erdogan Tan - 8/8/2023
9557      ; correct code (would be):
9558      ;     mov cl, [cs:si+05h] ; BPB_NumFATS
9559      ;     sub_fat32_size:
9560      ;     sub ax, [cs:si+19h] ; BPB_FATSz32
9561      ;     sbb dx, [cs:si+1Bh] ; BPB_FATSz32+2
9562      ;     dec cl
9563      ;     jg short sub_fat32_size
9564      ;     jmp short fat_32bit
9565      %endif
9566      ; 17/12/2023
9567      ; cl = BPB_NumFATS (2 or 1)
9568      ; ch = 0
9569      mov     0000259D 8B5C19    mov     bx, [si+19h]      ; BPB.FATSz32
9570      sub_fat32_size:
9571      000025A0 29D8             sub     ax, bx
9572      000025A2 1B541B          sbb     dx, [si+1Bh]      ; BPB.FATSz32+2
9573      ;dec     cl
9574      000025A5 49              dec     cx
9575      000025A6 7FF8             jg      short sub_fat32_size
9576
9577      000025A8 895D1F          mov     [di+1Fh], bx      ; [di+BDS.fatsecs32]
9578      000025AB 8B5C1B          mov     bx, [si+1Bh]      ; BPB.FATSz32+2
9579      000025AE 895D21          mov     [di+21h], bx      ; [di+BDS.fatsecs32+2]
9580
9581      000025B1 8B5C1D          mov     bx, [si+1Dh]      ; BPB.BPB_ExtFlags
9582      000025B4 895D23          mov     [di+23h], bx      ; [di+BDS.extflags]
9583      000025B7 8B5C1F          mov     bx, [si+1Fh]      ; BPB.FSVer
9584      000025BA 895D25          mov     [di+25h], bx      ; [di+BDS.fsver]
9585      000025BD 8B5C21          mov     bx, [si+21h]      ; BPB.RootClus
9586      000025C0 895D27          mov     [di+27h], bx      ; [di+BDS.rootdirclust]
9587      000025C3 8B5C23          mov     bx, [si+23h]      ; BPB.RootClus+2
9588      000025C6 895D29          mov     [di+29h], bx      ; [di+BDS.rootdirclust+2]
9589      000025C9 8B5C25          mov     bx, [si+25h]      ; BPB.FSInfo
9590      000025CC 895D2B          mov     [di+2Bh], bx      ; [di+BDS.fsinfo]
9591      000025CF 8B5C27          mov     bx, [si+27h]      ; BPB.FSInfo+2
9592      000025D2 895D2D          mov     [di+2Dh], bx      ; [di+BDS.fsinfo+2]
9593      000025D5 EB08             jmp     short fat_32bit
9594
9595      fat_16bit:
9596      ; 17/12/2023 - Retro DOS v5.0
9597      ;     (PCDOS 7.1 IBMBIO.COM)
9598      ; 10/12/2022
9599      ; (number of FATS optimization)
9600      ; CL = number of FATS (2 or 1)
9601      ; CH = 0 ; 17/12/2023
9602      ;dec     cl ; *
9603      ; 18/12/2022
9604      000025D7 49              dec     cx ; *
9605      000025D8 D3E3             shl     bx, cl
9606      ;shl     bx, 1 ; *=? ; always 2 fats
9607
9608      000025DA 29D8             sub     ax, bx            ; sub # fat sectors
9609      000025DC 83DA00          sbb     dx, 0
9610      fat_32bit:
9611      ; 17/12/2023
9612      000025DF 8B5C06          mov     bx, [si+6] ; 12/08/2023
9613      ;mov     bx, [cs:si+6] ; [cs:si+EBPB.ROOTENTRIES]
9614      ; # root entries
9615      000025E2 895D0C          mov     [di+0Ch], bx      ; [di+BDS.direntries]
9616      ; set in bds bpb
9617      000025E5 B104             mov     cl, 4
9618      000025E7 D3EB             shr     bx, cl            ; div by 16 ents/sector
9619      000025E9 29D8             sub     ax, bx            ; sub #dir sectors
9620      000025EB 83DA00          sbb     dx, 0
9621      ; dx:ax now contains the
9622      ; # of data sectors
9623
9624      ; 17/12/2023
9625      ; ch = 0
9626      000025EE 8A4C02          xor     cx, cx ; *
9627      mov     cl, [si+2] ; 12/08/2023
9628      ;mov     cl, [cs:si+2] ; [cs:si+EBPB.SECTORSPERCLUSTER]
9629      000025F1 884D08          mov     [di+8], cl        ; sectors per cluster
9630      ; [di+BDS.secperclus]
9631      ; set in bios bpb
9632      000025F4 50              push    ax
9633      000025F5 89D0             mov     ax, dx
9634      000025F7 31D2             xor     dx, dx
9635      000025F9 F7F1             div     cx                ; cx = sectors per cluster
9636      ; 12/08/2023 (ds=cs)
9637      ;mov     [temp_h], ax
9638      ;mov     [cs:temp_h], ax ; [temp_h]:ax now contains the
9639      ; # clusters.
9640      000025FB A3[9E04]          mov     [saved_word], ax ; hw of cluster number
9641      000025FE 58              pop     ax
9642      000025FF F7F1             div     cx
9643      ; 17/12/2023
9644      ;cmp     word [cs:temp_h], 0
9645      ;cmp     word [temp_h], 0 ; 12/08/2023
9646      ;cmp     word [saved_word], 0 ; (*)
9647      ;ja      short toobig_ret ; too big cluster number
9648
9649      ; 17/12/2023
9650      ;;;
9651      00002601 5B              pop     bx ; FAT sectors (16 bit)
9652      ;and     bx, bx ; 0 ?
9653      00002602 09DB             or      bx, bx ; 0 ?
9654      00002604 751F             jnz     short chk_clnum_hw
9655      ; 16 bit fat sectors > 0 ; FAT12 or FAT16 fs
9656
9657      00002606 813E[9E04]FF0F      cmp     word [saved_word], 0FFFh
9658      0000260C 7503             jne     short fat32_clust_limit
9659      0000260E 83F8F6          cmp     ax, 0FFF6h        ; FAT32 cluster number limit: 0FFFFFF6h
9660      fat32_clust_limit:
9661      00002611 772D             ja      short short toobig_ret ; too big cluster number
9662      00002613 391E[9E04]          cmp     [saved_word], bx ; 0 ?
9663      ;jnz     short fat16_clust_limit
9664      00002617 7505             jnz     short set_bigbig_flag ; 17/12/2023
9665      fat16_clust_limit: ; 17/12/2023
9666      00002619 83F8F6          cmp     ax, 0FFF6h        ; FAT16 cluster number limit: 0FFF6h
9667      ;fat16_clust_limit:
9668      0000261C 760E             jna     short fat12_clust_limit ; jbe
9669      set_bigbig_flag: ; 17/12/2023
9670      0000261E 800E[081A]20          or      byte [fbigfat], 20h ; fbigbig ; FAT32 fs
9671      00002623 EB11             jmp     short copymediaid
9672      chk_clnum_hw:
9673      00002625 833E[9E04]00          cmp     word [saved_word], 0 ; (*)
9674      0000262A 7714             ja      short toobig_ret ; too big cluster number

```



```

9675 fat12_clust_limit:
9676 cmp ax, 0FF6h ; 4096-10
9677 0000262C 3DF60F ; is this 16-bit fat?
9678 ; 17/10/2022
9679 0000262F 7205 ; short copymediaid ; no, small fat
9680 ; 17/10/2022
9681 00002631 800E[081A]40 or byte [fbigfat], 40h ; fbig ; FAT16 fs
9682 ;or ds:fbigfat, 40h ; fbig
9683 ; 16 bit fat
9684 copymediaid:
9685 ; 17/12/2023
9686 ; es = ds = cs
9687
9688 ;push es
9689 ;push ds
9690 ;pop es
9691
9692 ; 12/08/2023
9693 ; ds = cs = BIOSDATA
9694 ;push cs
9695 ;pop ds
9696 ; 17/10/2022
9697 00002636 BD[4F08] mov bp, MOV MEDIAIDS
9698 ;mov bp, 865h ; (PCDOS 7.1 IBMBIO.COM)
9699 ;mov bp, 751h ; mov_media_ids
9700 ; at 2C7h:751h = 70h:2CC1h
9701 ; copy filesys_id, volume label
9702 00002639 0E push cs ; simulate far call
9703 0000263A E836F4 call call_bios_code
9704
9705 ; 12/08/2023
9706 ;push es
9707 ;pop ds
9708 ; 17/12/2023
9709 ;pop es
9710
9711 0000263D E9CD00 jmp message_bpb ; now final check for bpb info
9712 ; and return.
9713 ; -----
9714
9715 toobig_ret:
9716 ; 12/08/2023 (ds=cs=BIOSDATA)
9717 00002640 800E[081A]80 or byte [fbigfat], 80h ; ftoobig
9718 ;or byte [cs:fbigfat], 80h ; ftoobig
9719 ; too big (32 bit clust #) for FAT16
9720 00002645 E9E300 jmp goodret ; still drive letter is assigned
9721 ; but useless. to big for
9722 ; current pc dos fat file system
9723 ; -----
9724
9725 unknown:
9726 ; 17/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
9727 00002648 804D4002 or byte [di+40h], 2 ; [di+BDS.flags+1]
9728 ; unformatted_media
9729 ; 12/12/2022
9730 ;or byte [di+24h], 02h
9731 ;or word [di+23h], 200h ; [di+BDS.flags]
9732 ; unformatted_media
9733 ; Set unformatted media flag.
9734
9735 ; the boot signature may not be recognizable,
9736 ; but we should try and read it anyway.
9737
9738 unknown3_0:
9739 0000264C 8B551D mov dx, [di+1Dh] ; skip setting unformatted_media bit
9740 ; [di+BDS.totalsecs32+2]
9741 0000264F 8B451B mov ax, [di+1Bh] ; [di+BDS.totalsecs32]
9742 00002652 BE[181A] mov si, disktable2
9743
9744 scan:
9745 ; 08/08/2023
9746 ; 12/08/2023 (ds=cs)
9747 00002655 3B14 cmp dx, [si] ; total sectors hw
9748 00002657 720C jb short gotparm
9749 00002659 7705 ja short scan_next
9750 0000265B 3B4402 cmp ax, [cs:si+2] ; total sectors lw
9751 0000265E 7605 cmp ax, [si+2]
9752 jbe short gotparm
9753 scan_next:
9754 00002660 83C60A add si, 10 ; 5*2
9755 00002663 EBF0 jmp short scan ; covers upto 512 mb media
9756 ; -----
9757
9758 gotparm:
9759 00002665 8A4C08 mov cl, [si+8] ; fat size for fbigfat flag
9760 ;or ds:fbigfat, cl
9761 ; 17/10/2022
9762 00002668 080E[081A] or [fbigfat], cl ; (fbig flag, 40h or 0) ; 08/08/2023
9763 ; 12/08/2023
9764 0000266C 8B4C04 mov cx, [si+4]
9765 ;mov cx, [cs:si+4] ; ch = number of sectors per cluster
9766 ; cl = log base 2 of ch
9767 0000266F 8B5406 mov dx, [si+6]
9768 ;mov dx, [cs:si+6] ; dx = number of root dir entries
9769
9770 ; now calculate size of fat table
9771
9772 00002672 89550C mov [di+0Ch], dx ; [di+BDS.direntries]
9773 ; save number of (root) dir entries
9774 00002675 8B551D mov dx, [di+1Dh] ; [di+BDS.totalsecs32+2]
9775 00002678 8B451B mov ax, [di+1Bh] ; [di+BDS.totalsecs32]
9776 0000267B 886D08 mov [di+8], ch ; [di+BDS.secpclus]
9777 ; save sectors per cluster
9778
9779 ; 17/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
9780 0000267E F606[081A]60 test byte [fbigfat], 60h ; fbig+fbigbig ; FAT16 or FAT32
9781 ; 11/09/2023
9782 ; 17/10/2022
9783 ;test byte [fbigfat], 40h
9784 ;test ds:fbigfat, 40h ; fbig
9785 ; if (fbigfat)
9786 00002683 751E jnz short dobig ; goto dobig; (16 bit fat)
9787
9788 ; we don't need to change "small fat" logic since it is guaranteed
9789 ; that double word total sector will not use 12 bit fat (unless
9790 ; it's sectors/cluster >= 16 which will never be in this case.)
9791 ; so in this case we assume dx = 0 !!
9792
9793 00002685 31DB xor bx, bx ; 12 bit fat (FAT12 fs)
9794 00002687 88EB mov bl, ch
9795 00002689 4B dec bx
9796 0000268A 01C3 add bx, ax ; dx=0
9797 0000268C D3EB shr bx, cl ; bx = 1+(bpb->maxsec+BDS.secpclus-1)/
9798 0000268E 43 inc bx ; BDS.secpclus

```

```

9799 0000268F 80E3FE          and    bl, 0FEh          ; bx &= ~1; (=number of clusters)
9800 00002692 89DE          mov    si, bx
9801 00002694 D1EB          shr    bx, 1
9802 00002696 01F3          add    bx, si          ; number of FAT bytes ; 08/08/2023
9803 00002698 81C3FF01        add    bx, 511         ; bx += 511 + bx/2
9804 0000269C D0EF          shr    bh, 1          ; bh >= 1; (=bx/512)
9805 0000269E 887D11        mov    [di+11h], bh    ; [di+BDS.fatsecs]
9806                                     ; save number of fat sectors
9807 000026A1 EB6A          jmp    short message_bpb
9808                                     ; -----
9809
9810                                     ; for bigfat we do need to extend this logic to 32 bit sector calculation.
9811
9812 dobig:
9813 000026A3 B104          mov    cl, 4           ; 16 (2^4) directory entries per sector
9814 000026A5 52          push   dx            ; save total sectors (high)
9815 000026A6 8B550C        mov    dx, [di+0Ch]    ; [di+BDS.direntries]
9816 000026A9 D3EA          shr    dx, cl         ; root dir sectors = BDS.direntries / 16;
9817 000026AB 29D0          sub    ax, dx
9818 000026AD 5A          pop     dx
9819 000026AE 83DA00        sbb    dx, 0          ; dx:ax= total sectors - root dir sectors
9820 000026B1 83E801        sub    ax, 1
9821 000026B4 83DA00        sbb    dx, 0          ; dx:ax= t - r - d
9822                                     ; total secs - reserved      secs - root dir      secs
9823 000026B7 B302          mov    bl, 2
9824 000026B9 8A7D08        mov    bh, [di+8]     ; [di+BDS.secperclus]
9825                                     ; bx = 256 * BDS.secperclus + 2
9826
9827                                     ; I don't understand why to add bx here!!!
9828
9829                                     ; 29/12/2018 - Erdogan Tan (Retro DOS v4.0)
9830                                     ; 27/09/2022
9831                                     ; (Microsoft FAT32 File System Specification,
9832                                     ; December 2000, Page 21)
9833                                     ; TmpVal1 = DskSize - (BPB_ResvdSecCnt+RootDirSectors)
9834                                     ; TmpVal2 = (256*BPB_SecPerClus)+BPB_NumFATS
9835                                     ; 8/8/2023 (Retro DOS v5.0)
9836                                     ; If(FATType == FAT32)
9837                                     ;     TmpVal2 = TmpVal2 / 2;
9838                                     ; FATSz = (TmpVal1+(TmpVal2-1))/TmpVal2
9839                                     ; 8/8/2023 (Retro DOS v5.0)
9840                                     ; If(FATType == FAT32) {
9841                                     ;     BPB_FATSz16 = 0;
9842                                     ;     BPB_FATSz32 = FATSz;
9843                                     ; } else {
9844                                     ;     BPB_FATSz16 = LOWORD(FATSz);
9845                                     ; /* there is no BPB_FATSz32 in a FAT16 BPB */
9846                                     ; }
9847                                     ; dx:ax = TmpVal1, bx = TmpVal2
9848 000026BC 01D8          add    ax, bx
9849 000026BE 83D200        adc    dx, 0          ; dx:ax = TmpVal1+TmpVal2
9850 000026C1 83E801        sub    ax, 1
9851 000026C4 83DA00        sbb    dx, 0          ; dx:ax = TmpVal1+TmpVal2-1
9852
9853                                     ;;;
9854                                     ; 17/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
9855 000026C7 F606[081A]20    test   byte [fbigfat], 20h ; fbigbig (FAT32) flag
9856 000026CC 740D          jz     short dobig1
9857
9858 000026CE D1EB          shr    bx, 1          ; TmpVal2 = TmpVal2 / 2
9859                                     ; dx:ax = TmpVal1+(2*TmpVal2)-1
9860 000026D0 83E81F        sub    ax, 31         ; reserved sectors = 32 (for FAT32 fs) /// 1+31 = 32
9861 000026D3 83DA00        sbb    dx, 0
9862 000026D6 29D8          sub    ax, bx
9863 000026D8 83DA00        sbb    dx, 0          ; dx:ax = TmpVal1+(2*TmpVal2)-TmpVal2-1
9864                                     ; = TmpVal1+(TmpVal2-1)
9865 dobig1:
9866 000026DB 50          push   ax            ; save lw of dividend
9867 000026DC 89D0          mov    ax, dx         ; divide hw of dx:ax at first (as 1st stage)
9868 000026DE 31D2          xor    dx, dx
9869 000026E0 F7F3          div    bx            ; 32 bit division, dx:ax/bx
9870                                     ; remainder in dx is hw of 2nd stage dividend
9871 000026E2 89C5          mov    bp, ax         ; hw of quotient
9872 000026E4 58          pop     ax           ; restore lw of dividend (of 1st stage)
9873                                     ;;;
9874
9875                                     ; assuming dx in the table will never be bigger than bx.
9876
9877 000026E5 F7F3          div    bx            ; BDS.fatsecs =
9878                                     ; ceil((total-dir-res)/(256*BDS.secperclus+2))
9879 000026E7 894511        mov    [di+11h], ax    ; [di+BDS.fatsecs]
9880                                     ; number of fat      sectors
9881                                     ;;;
9882
9883                                     ; 18/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
9884 000026EA 8A1E[081A]    mov    bl, [fbigfat]
9885 000026EE 885D3B        mov    [di+3Bh], bl    ; [di+BDS.fatsiz] ; fat size flag
9886
9887 000026F1 F6C320        test   bl, 20h        ; fbigbig (FAT32) flag
9888 000026F4 7410          jz     short dobig2    ; not FAT32
9889
9890 000026F6 89451F        mov    [di+1Fh], ax    ; [di+BDS.fatsecs32]
9891 000026F9 896D21        mov    [di+21h], bp    ; [di+BDS.fatsecs32+2]
9892 000026FC C745110000    mov    word [di+11h], 0 ; [di+BDS.fatsecs] = 0
9893                                     ; clear 16 bit FAT size field
9894 00002701 C745092000    mov    word [di+9], 32 ; [di+BDS.resectors]
9895                                     ; set reserved sectors to 32 (FAT32 de facto)
9896 dobig2:
9897                                     ;;;
9898
9899                                     ; now, set the default filesystem_id, volume label, serial number
9900
9901                                     ; 05/08/2023
9902                                     ; [di+1Fh] = [fbigfat]
9903                                     ;
9904                                     ; mov bl, ds:fbigfat
9905                                     ; 17/10/2022
9906                                     ; mov bl, [fbigfat]
9907                                     ; mov [di+1Fh], bl ; [di+BDS.fatsiz] ; fat      size flag
9908
9909                                     ; 12/08/2023
9910                                     ; push ds ; ds = cs = BIOSDATA
9911
9912                                     ; 17/12/2023
9913                                     ; es = ds = cs
9914                                     ; push ds
9915                                     ; pop es
9916
9917                                     ; 12/08/2023
9918                                     ; ds = cs = BIOSDATA
9919                                     ; push cs
9920                                     ; pop ds
9921
9922                                     ; 18/12/2023 - Retro DOS v5.0

```

```

9923 ; b1 = [fbigfat] (clear_ids_x uses b1 value here)
9924 ; 11/09/2023
9925 ;mov al, [fbigfat]
9926 00002706 BD[A106] mov bp, CLEARIDS_X ; clear_ids_x (uses AL value here)
9927 ; 17/10/2022
9928 ;mov bp, CLEARIDS
9929 ;;mov bp, 5D9h ; clear_ids
9930 ; at 2C7h:5D9h = 70h:2B49h
9931 ; at BIOSCODE:06ABh
9932 ; in PCDOS 7.1 IBMBIO.COM
9933 00002709 0E push cs
9934 0000270A E866F3 call call_bios_code
9935
9936 ; 12/08/2023
9937 ;pop ds ; ds = cs = BIOSDATA
9938
9939 ; at this point, in bpb of bds table, BDS_BPB.BPB_BIGTOTALSECTORS which is
9940 ; set according to the partition information. we are going to
9941 ; see if (hidden sectors + total sectors) > a word. if it is true,
9942 ; then no change. otherwise, BDS_BPB.BPB_BIGTOTALSECTORS will be moved
9943 ; to BDS_BPB.BPB_TOTALSECTORS and BDS_BPB.BPB_BIGTOTALSECTORS will be set to 0.
9944 ; we don't do this for the bpb information from the boot record. we
9945 ; are not going to change the bpb information from the boot record.
9946
9947 message_bpb:
9948 ; 05/08/2023
9949 ; [di+1Fh] = [fbigfat]
9950 ;
9951 ;; 12/12/2022
9952 ;mov b1, [fbigfat]
9953 ;mov [di+1Fh], b1 ; [di+BDS.fatsiz]
9954 ; ; set size of fat on media
9955 ;
9956 0000270D 8B551D mov dx, [di+1Dh] ; [di+BDS.totalsecs32+2]
9957 00002710 8B451B mov ax, [di+1Bh] ; [di+BDS.totalsecs32]
9958 ; 11/09/2023
9959 00002713 09D2 or dx, dx
9960 00002715 7514 jnz short goodret
9961 ;cmp dx, 0 ; double word total sectors?
9962 ;;ja short goodret ; don't have to change it.
9963 ;; 12/12/2022
9964 ;ja short short goodret2
9965 ;cmp word [di+19h], 0 ; [di+BDS.hiddensecs+2]
9966 ;ja short goodret ; don't have to change it.
9967 ; 12/12/2022
9968 00002717 395519 cmp [di+19h], dx ; 0
9969 ;ja short goodret2
9970 0000271A 770F ja short goodret ; 11/09/2023
9971 0000271C 034517 add ax, [di+17h] ; [di+BDS.hiddensecs]
9972 ;jb short goodret
9973 ; 12/12/2022
9974 ;jc short goodret
9975 0000271F 7209 jc short goodret_c1c ; 11/09/2023
9976 00002721 8B451B mov ax, [di+1Bh] ; [di+BDS.totalsecs32]
9977 00002724 89450E mov [di+0Eh], ax ; [di+BDS.totalsecs16]
9978 ;mov word [di+1Bh], 0 ; [di+BDS.totalsecs32]
9979 ; 12/12/2022
9980 00002727 89551B mov [di+1Bh], dx ; 0
9981
9982 goodret_c1c:
9983 ; 11/09/2023
9984 c1c
9985
9986 goodret:
9987 ;mov b1, ds:fbigfat
9988 ; 11/09/2023
9989 ; 12/12/2022
9990 ; 17/10/2022
9991 0000272B 8A1E[081A] mov b1, [fbigfat]
9992 ; 17/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
9993 0000272F 885D3B mov [di+3Bh], b1 ; [di+BDS.fatsiz]
9994 ;mov [di+1Fh], b1 ; [di+BDS.fatsiz]
9995 ; ; set size of fat on media
9996 ; 11/09/2023
9997 ;clc
9998
9999 ret_hard_err:
10000 ; 12/12/2022
10001
10002 goodret2:
10003 pop es
10004 ;pop ds ; ds = cs = BIOSDATA ; 14/08/2023
10005 pop bx
10006 pop di
10007 retn
10008
10009 ; ===== S U B R O U T I N E =====
10010
10011 ; 15/10/2022
10012
10013 ;fdisk of pc dos 3.3 and below, os2 1.0 has a bug. the maximum number of
10014 ;sector that can be handled by pc dos 3.3 ibmbio should be 0ffffh.
10015 ;instead, sometimes fdisk use 10000h to calculate the maximum number.
10016 ;so, we are going to check that if BPB_TOTALSECTORS + hidden sector = 10000h
10017 ;then subtract 1 from BPB_TOTALSECTORS.
10018 ; 17/10/2022
10019 cover_fdisk_bug:
10020 ; 12/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
10021 ; ds = cs
10022 ;
10023 ; 18/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
10024 ; (optimization)
10025 ;push ax
10026 ;push dx
10027 ;push si
10028 ;
10029 ; 18/12/2023
10030 ; bx = offset disksector
10031 ;
10032 ; 18/04/2024 - Retrodos v5.0 (PCDOS 7.1 IBMBIO.COM)
10033 ;;cmp word [cs:disksector+16h],0
10034 ;cmp word [disksector+16h],0 ; BPB_FATSz16
10035 cmp word [bx+16h],0
10036 jz short cfb_retit ; FAT32 boot sector
10037 ;
10038 ; 18/12/2023
10039 cmp byte [bx+26h], 29h
10040 ; 12/08/2023
10041 ;cmp byte [disksector+26h], 29h
10042 ;;cmp byte [cs:disksector+26h], 29h
10043 ; ; [disksector+EXT_BOOT.SIG],
10044 ; EXT_BOOT_SIGNATURE
10045 je short cfb_retit ; if extended bpb, then >= pc dos 4.00
10046
10047 cmp word [bx+7], 3031h
10048 ;cmp word [cs:bx+7], 3031h ; '10' ; os2 1.0 = ibm 10.0
10049 jne short cfb_chk_totalsecs ; 11/08/2023
10050

```

```

10047 00002749 807F0A30      cmp     byte [bx+10], '0'
10048                        ;cmp     byte [cs:bx+10], '0'
10049 0000274D 7519          jne     short cfb_retit
10050
10051 cfb_chk_totalsecs:
10052                        ; 11/08/2023
10053 ; 18/12/2023
10054 %if 0
10055                        ; 17/10/2022
10056 mov     si, disksector+11 ; 14Eh+08h
10057 ;mov     si, 159h          ; disksector+EXT_BOOT.BPB
10058 ; 12/08/2023
10059 cmp     word [si+8], 0
10060 ;cmp     word [cs:si+8], 0 ; [cs:si+EBPB.TOTALSECTORS]
10061                        ; just to make sure.
10062 jz      short cfb_retit
10063 ;mov     ax, [cs:si+8] ; [cs:si+EBPB.TOTALSECTORS]
10064 ;add     ax, [cs:si+11h] ; [cs:si+EBPB.HIDDENSECTORS]
10065 ; 12/08/2023
10066 mov     ax, [si+8]
10067 add     ax, [si+11h]
10068
10069 jnb     short cfb_retit
10070 jnz     short cfb_retit
10071                        ; if carry set and ax=0
10072 dec     word [si+8]
10073 ;dec     word [cs:si+8] ; 0 -> 0FFFFh
10074                        ; then decrease          BPB_TOTALSECTORS by 1
10075 %endif
10076                        ; 18/12/2023
10077 ;cmp     word [bx+19], 0
10078 0000274F 8B4713      mov     ax, [bx+19] ; [bx+EBPB.TOTALSECTORS]
10079 00002752 21C0      and     ax, ax ; 0 ?
10080 00002754 7412      jz      short cfb_retit
10081
10082 ;mov     ax, [bx+19]
10083 00002756 03471C      add     ax, [bx+28] ; [bx+EBPB.HIDDENSECTORS]
10084 00002759 730D      jnc     short cfb_retit
10085 0000275B 750B      jnz     short cfb_retit
10086                        ; ax = 0
10087 0000275D FF4F13      dec     word [bx+19] ; then decrease          BPB_TOTALSECTORS by 1
10088
10089 00002760 836D1B01     sub     word [di+1Bh], 1 ; [di+BDS.totalsecs32]
10090 00002764 835D1D00     sbb     word [di+1Dh], 0 ; [di+BDS.totalsecs32+2]
10091 cfb_retit:
10092                        ; 18/12/2023
10093 ;pop     si
10094 ;pop     dx
10095 ;pop     ax
10096
10097 00002768 C3          retn
10098
10099 ; -----
10100
10101                        ; 18/12/2023 - Retro DOS v5.0
10102                        ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2A3Dh)
10103                        ; ((MSDOS 6.22 IO.SYS - BIOSDATA:21DCh))
10104
10105 word2:                dw 2
10106 word3:                dw 3
10107 word512:              dw 512
10108
10109 ; ===== S U B   R O U T I N E =====
10110
10111 ; 15/10/2022
10112
10113 ; setdrvparms sets up the recommended bpb in each bds in the system based on
10114 ; the form factor. it is assumed that the bpbs for the various form factors
10115 ; are present in the bpbtable. for hard files, the recommended bpb is the same
10116 ; as the bpb on the drive.
10117 ;
10118 ; no attempt is made to preserve registers since we are going to jump to
10119 ; sysinit straight after this routine.
10120
10121                        ; 18/12/2023 - Retro DOS v5.0
10122                        ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2A43h)
10123 setdrvparms:
10124                        ; 12/12/2023
10125                        ; ds = cs
10126 0000276F 31DB      xor     bx, bx
10127                        ; 18/10/2022
10128 00002771 C43E[1901]  les     di, [start_bds] ; get first bds in list
10129
10130 _next_bds:
10131 00002775 06          push    es
10132 00002776 57          push    di
10133
10134 ; 18/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
10135 00002777 268A5D3E     mov     bl, [es:di+3Eh] ; [es:di+BDS.formfactor]
10136                        ;mov     bl, [es:di+22h] ; [es:di+BDS.formfactor]
10137 0000277B 80FB05      cmp     bl, 5 ; ffHardFile
10138 0000277E 753A      jnz     short nothardff
10139 00002780 31D2      xor     dx, dx
10140 00002782 268B450E     mov     ax, [es:di+0Eh] ; [es:di+BDS.totalsecs16]
10141 00002786 09C0      or      ax, ax
10142 00002788 7508      jnz     short get_ccyl
10143 0000278A 268B551D     mov     dx, [es:di+1Dh] ; [es:di+BDS.totalsecs32+2]
10144 0000278E 268B451B     mov     ax, [es:di+1Bh] ; [es:di+BDS.totalsecs32]
10145
10146 get_ccyl:
10147 00002792 52          push    dx
10148 00002793 50          push    ax
10149 00002794 268B4515     mov     ax, [es:di+15h] ; [es:di+BDS.heads]
10150 00002798 26F76513     mul     word [es:di+13h] ; [es:di+BDS.secptrack]
10151                        ; assume sectors per cyl. < 64k.
10152 0000279C 89C1      mov     cx, ax ; cx has # sectors per cylinder
10153 0000279E 58          pop     ax
10154 0000279F 5A          pop     dx ; dx:ax = total sectors
10155 000027A0 50          push    ax
10156 000027A1 89D0      mov     ax, dx
10157 000027A3 31D2      xor     dx, dx
10158 000027A5 F7F1      div     cx
10159                        ; 12/12/2023 ; !*!
10160                        ; (data segment may not be same with code segment here)
10161 ;mov     [cs:temp_h], ax ; ax be0 here.
10162 ; 18/12/2023 - Retro DOS v5.0
10163 000027A7 58          ;mov     [cs:saved_word], ax
10164 000027A8 F7F1      pop     ax
10165 000027AA 09D2      div     cx ; div #sec by sec/cyl to get # cyl.
10166 000027AC 7401      or      dx, dx
10167 000027AE 40          jz      short no_cyl_rnd ; came out even
10168                        ; inc     ax ; round up
10169 no_cyl_rnd:
10170 000027AF 26894541     ; 18/12/2023 - Retro DOS v5.0
10171                        ;mov     [es:di+41h], ax ; [es:di+BDS.cylinders]

```

```

10171             ;mov     [es:di+25h], ax          ; [es:di+BDS.cylinders]
10172
10173 000027B3 06      push     es
10174 000027B4 1F      pop      ds ; !! ; 12/12/2023
10175
10176 000027B5 8D7506   lea      si, [di+6]          ; [di+BDS.bytespersec]
10177             ; ds:si -> bpb for hard      file
10178 000027B8 EB55     jmp      short set_recbpb
10179 ; -----
10180
10181 nothardff:
10182 000027BA 0E      push     cs
10183 000027BB 1F      pop      ds
10184
10185 ; if fake floppy drive variable is set then we don't have to handle this bds.
10186 ; we can just go and deal with the next bds at label go_to_next_bds.
10187
10188             ; 10/12/2022
10189             ; ds = cs
10190             ; 17/10/2022 (ds=cs)
10191 000027BC 803E[131A]01 cmp     byte [fakefloppydrv], 1
10192             ; cmp     byte [cs:fakefloppydrv], 1
10193 000027C1 7454     jz       short go_to_next_bds
10194 000027C3 80FB07   cmp     bl, 7              ; ffother
10195             ; special case "other" type of medium
10196 000027C6 753D     jnz      short not_process_other
10197 process_other:
10198 000027C8 31D2     xor      dx, dx
10199
10200             ;mov     ax, [di+25h] ; [di+BDS.cylinders]
10201             ;mul     word [di+36h] ; [di+BDS.rheads]
10202             ;mul     word [di+34h] ; [di+BDS.rsecptrack]
10203             ;mov     [di+2Fh], ax ; [di+BDS.rtotalsecs16]
10204             ; have the total number of sectors
10205             ; 18/12/2023 - Retro DOS v5.0
10206 000027CA 8B4541   mov     ax, [di+41h] ; [di+BDS.cylinders]
10207 000027CD F76552   mul     word [di+52h] ; [di+BDS.rheads]
10208 000027D0 F76550   mul     word [di+50h] ; [di+BDS.rsecptrack]
10209 000027D3 89454B   mov     [di+4Bh], ax ; [di+BDS.rtotalsecs16]
10210             ; have the total number of sectors
10211 000027D6 48       dec     ax
10212 000027D7 B201     mov     dl, 1
10213
10214 000027D9 3DF60F   _again: cmp     ax, 0FF6h      ; 4096-10
10215 000027DC 7206     jb      short _@@
10216 000027DE D1E8     shr     ax, 1
10217 000027E0 D0E2     shl     dl, 1
10218 000027E2 EBF5     jmp     short _again
10219 ; -----
10220
10221 _@@:
10222 000027E4 80FA01   cmp     dl, 1          ; is it a small disk ?
10223 000027E7 7405     jz      short __@@     ; yes, 224 root      entries is enuf
10224
10225             ; 18/12/2023 - Retro DOS v5.0
10226 000027E9 C74549F000 mov     word [di+49h], 240 ; [di+BDS.rdirentries]
10227             ;mov     word [di+2Dh], 240 ; [di+BDS.rdirentries]
10228
10229 __@@:
10230 000027EE 885545   ; 18/12/2023 - Retro DOS v5.0
10231             mov     [di+45h], dl ; [di+BDS.rsecpclus]
10232             ;mov     [di+29h], dl ; [di+BDS.rsecpclus]
10233
10234 ; logic to get the sectors/fat area.
10235 ; fat entry is assumed to be 1.5 bytes!!!
10236
10237             ; 10/12/2022
10238             ; ds = cs
10239             ; 17/10/2022 (ds=cs)
10240 000027F1 F726[6B27] mul     word [word3] ; * 3
10241 000027F5 F736[6927] div     word [word2] ; / 2
10242 000027F9 31D2     xor     dx, dx
10243 000027FB F736[6D27] div     word [word512] ; / 512
10244
10245             ; 10/12/2022
10246             ;mul     word [cs:word3] ; * 3
10247             ;div     word [cs:word2] ; / 2
10248             ;xor     dx, dx
10249             ;div     word [cs:word512] ; / 512
10250 000027FF 40       inc     ax ; + 1
10251 no_round_up:
10252             ; 18/12/2023 - Retro DOS v5.0
10253 00002800 89454E   mov     [di+4Eh], ax ; [di+BDS.rfatsecs]
10254             ;mov     [di+32h], ax ; [di+BDS.rfatsecs]
10255
10256 00002803 EB12     jmp     short go_to_next_bds
10257 ; -----
10258
10259 not_process_other:
10260 00002805 D1E3     shl     bx, 1          ; bx is word index into      table of bpbs
10261
10262             ;mov     si, bpbtable
10263             ;mov     si, [bpbtable+bx] ; 15/10/2022
10264             ; 09/12/2022
10265             ;mov     si, BPBTABLE
10266             ;mov     si, [bx+si] ; get address of bpb
10267             ; 10/12/2022
10268             ;mov     si, [BPBTABLE+bx]
10269             ; 13/12/2022
10270             ;mov     si, [SYSINITOFFSET+bpbtable+bx] ; wrong ! 14/08/2023
10271
10272             ; 14/08/2023
10273             SYSINIT_OFFSET equ (SYSINITSEG-DOSBIODATASEG<<4)
10274             ; correct offset
10275 00002807 8B87[6C99] mov     si, [bx+SYSINIT_OFFSET+bpbtable]
10276
10277             ; 18/12/2023
10278             ; si = address of the requested disk(ette) parameter block
10279             ; ! as offset from SYSINIT segment !
10280
10281             ; 28/08/2023
10282 0000280B 81C64049 add     si, SYSINIT_OFFSET
10283             ; + displacement from BIOSDATA segment ; 18/12/2023
10284 set_recbpb:
10285             ; 18/12/2023
10286             ;lea     di, [di+27h] ; [di+BDS.R_BPB]
10287             ; es:di -> recbpb
10288             ;mov     cx, 25 ; bpbx.size
10289             ;rep movsb ; move (size bpbx) bytes
10290
10291             ; 18/12/2023 - Retro DOS v5.0
10292 0000280F 8D7D43   lea     di, [di+43h] ; [di+BDS.R_BPB]
10293             ; es:di -> recbpb
10294 00002812 B93500   mov     cx, 53 ; bpbx.size

```

```

10295 00002815 F3A4      rep movsb          ; move (size bpbx) byte
10296
10297 00002817 5F      go_to_next_bds:
10298 00002818 07          pop     di
10299 00002819 26C43D      pop     es          ; restore pointer to bds
10300 0000281C 83FFFF      les     di, [es:di]    ; [es:di+BDS.link]
10301 0000281F 740A      cmp     di, 0FFFFh    ; -1
10302 00002821 E951FF      jz      short got_end_of_bds_chain
10303                          jmp     _next_bds
10304
10305                          ; -----
10306                          ; 18/12/2022
10307 got_end_of_bds_chain:
10308                          ;retn
10309
10310                          ; ===== S U B   R O U T I N E =====
10311
10312                          ; 15/10/2022
10313                          ; 30/12/2018 - Retro DOS v4.0
10314
10315                          ; a1 = device number
10316
10317 print_init:
10318 00002824 98          cbw
10319 00002825 89C2      mov     dx, ax
10320 00002827 B401      mov     ah, 1
10321 00002829 CD17      int     17h          ; PRINTER - INITIALIZE
10322                          ; DX = printer port (0-3)
10323                          ; Return: AH = status
10324
10325 0000282B C3      got_end_of_bds_chain:    ; 18/12/2022
10326                          ;retn
10327
10328                          ; ===== S U B   R O U T I N E =====
10329
10330                          ; a1 = device number
10331
10332 aux_init:
10333 0000282C 98          cbw
10334 0000282D 89C2      mov     dx, ax
10335                          ;mov     a1, 0A3h          ; RSINIT ; 0A3h
10336                          ;                ; 2400,n,1,8 (msequ.inc)
10337                          ;mov     ah, 0
10338 0000282F B8A300      ; 10/12/2022
10339 00002832 CD14      mov     ax, 00A3h
10340                          int     14h          ; SERIAL I/O - INITIALIZE USART
10341                          ;                ; AL = initializing parameters,
10342                          ;                ; DX = port number (0-3)
10343                          ; Return: AH = RS-232 status code bits,
10344 00002834 C3      ;                ; AL = modem status bits
10345                          ;retn
10346
10347                          ; ===== S U B   R O U T I N E =====
10348
10349                          ; 18/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
10350                          ; 08/08/2023 - Retro DOS v4.2 (Modified MSDOS 6.22 IO.SYS)
10351                          ; 15/10/2022 (Modified MSDOS 5.0 IO.SYS) -Retro DOS v4 2022- (MSDOS 5.0-6.21)
10352                          ; 30/12/2018 - Retro DOS v4.0
10353                          ; 03/06/2018 - Retro DOS v3.0
10354                          ; (19/03/2018 - Retro DOS v2.0)
10355
10356                          ; domini *****
10357
10358                          ;mini disk initialization routine. called right after dohard
10359                          ;modified for >2 hardfile support
10360
10361                          ; **cs=ds=es=datagrp
10362
10363                          ; **domini will search for every extended partition in the system, and
10364                          ; initialize it.
10365
10366                          ; **bdsbm stands for bds table for mini disk and located right after the label
10367                          ; end96tpi. end_of_bdsbm will have the offset value of the ending
10368                          ; address of bdsbm table.
10369
10370                          ; **bdsbm is the same as usual bds structure except that tim_lo, tim_hi entries
10371                          ; are overlapped and used to identify mini disk and the number of hidden_trks.
10372                          ; right now, they are called as ismini, hidden_trks respectively.
10373
10374                          ; **domini will use the same routine in sethard routine after label set2 to
10375                          ; save coding.
10376
10377                          ; **drvmax determined in dohard routine will be used for the next
10378                          ; available logical mini disk drive number.
10379
10380                          ; input: drvmax, dskdrvs
10381
10382                          ; output: minidisk installed. bdsbm table established and installed to bds.
10383                          ; end_of_bdsbm - ending offset address of bdsbm.
10384
10385                          ; called modules:
10386                          ; getboot
10387                          ; find_mini_partition (new), xinstall_bds (new), M038
10388                          ;
10389                          ; setmini (new, it will use set2 routine)
10390
10391                          ; variables used: end_of_bdsbm
10392                          ; rom_minidisk_num
10393                          ; mini_hdlim, mini_seclim
10394                          ; BDS_STRUC, start_bds
10395
10396                          ; *****
10397
10398                          ; 18/12/2023 - Retro DOS v5.0 IO.SYS/IBMBIO.COM
10399                          ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2B10h)
10400
10401                          ; 19/10/2022
10402
10403 domini:
10404 00002835 8A36[5F1A]      mov     dh, [hnum]          ; get number of hardfiles
10405                          ; 10/12/2022
10406 00002839 20F6      and     dh, dh
10407 0000283B 743C      cmp     dh, 0
10408 0000283D B280      jz      short dominiret    ; no hard file?          then exit.
10409                          mov     dl, 80h          ; startwith hardfile 80h
10410
10411 domini_loop:
10412 0000283F 31C0      ; 18/12/2023 - Retro DOS v5.0
10413                          xor     ax, ax ; 0
10414                          ; ds = cs
10415                          ;mov     [cs:ep_start_sector], ax
10416                          ;mov     [cs:ep_start_sector+2], ax
10417                          ;mov     [cs:ep_hidden_secs], ax
10418                          ;mov     [cs:ep_hidden_secs+2], ax
10419                          ;mov     [ep_start_sector], ax
10420                          ;mov     [ep_start_sector+2], ax
10421                          ;mov     [ep_hidden_secs], ax

```



```

10419 0000284A A3[8B23]      mov     [ep_hidden_secs+2], ax
10420                          ;
10421 0000284D 52              push    dx
10422 0000284E 8816[5E1A]      mov     [rom_minidisk_num], dl
10423 00002852 B408          mov     ah, 8
10424 00002854 CD13          int      13h
10425                          ; DISK - DISK - GET CURRENT DRIVE PARAMETERS (XT,AT,XT286,CONV,PS)
10426                          ; DL = drive number
10427                          ; Return: CF set on error, AH = status code, BL = drive type
10428                          ; DL = number of consecutive drives
10429                          ; DH = maximum value for head number, ES:DI -> drive parameter
10430
10431                          ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
10432                          ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:2B36h
10433                          ;inc     dh
10434                          ;xor     ax, ax
10435 00002856 31C0          xor     ax, ax
10436 00002858 88F0          mov     al, dh ; <= 255
10437 0000285A 40          inc     ax ; (0FFh -> 100h)
10438 0000285B A3[641A]      mov     [mini_hdlim], ax ; # of heads
10439                          ;and     cl, 3Fh
10440                          ;mov     al, cl
10441                          ; 08/08/2023
10442 0000285E 88C8          mov     al, cl
10443 00002860 83E03F      and     ax, 3Fh
10444 00002863 A3[661A]      mov     [mini_seclim], ax ; # of sectors/track
10445
10446                          ; 18/12/2023
10447                          ;push     es ; * ; not necessary
10448 00002866 8A16[5E1A]      mov     dl, [rom_minidisk_num]
10449 0000286A E860FA      call    getboot ; read master boot record into
10450                          ; initbootsegment:bootbias
10451 0000286D 7203          jc      short domininext
10452 0000286F E80800      call    find_mini_partition
10453 domininext:
10454                          ;pop      es ; *
10455 00002872 5A          pop     dx
10456 00002873 FEC2          inc     dl ; next hard file
10457 00002875 FECE          dec     dh
10458 00002877 75C6          jnz     short domini_loop
10459 dominiret:
10460 00002879 C3          retn
10461
10462 ; ===== S U B R O U T I N E =====
10463
10464 ; 15/10/2022 (Modified MSDOS 5.0 IO.SYS)
10465 ; 30/12/2018 - Retro DOS v4.0
10466
10467 ;find_mini_partition tries to find every extended partition on a disk.
10468 ;at entry: di -> bdsm entry
10469 ;          es:bx -> 07c0:bootbias - master boot record
10470 ;          rom_minidisk_num - rom drive number
10471 ;          drvmax - logical drive number
10472 ;          mini_hdlim, mini_seclim
10473 ;
10474 ;called routine: setmini which uses set2 (in sethard routine)
10475 ;variables & equates used from original bios - flags, fnon_removable, fbigfat
10476
10477 ; 19/12/2023 - Retro DOS v5.0
10478 ; (Modified PCDOS 7.1 IBMBIO.COM)
10479 ; (PCDOS 7.1 IBMBIO.COM - BIOSADATA:2BFCh)
10480
10481 find_mini_partition:
10482 0000287A 81C3C201      add     bx, 1C2h ; bx -> file system id
10483
10484 ; 19/12/2023
10485 ; PCDOS 7.1 IBMBIO.COM
10486 ;mov     word [ld_p_number], 26
10487 fmpnext:
10488 ;add     word [ld_p_number], 16
10489 ;cmp     word [ld_p_number], 4122
10490 ;          ; 64 logical disk partitions (64 EBRs)
10491 ;          ; (64*4 = 256 pte's, 256*16 = 4096, + 26 = 4122)
10492 ;jg      short fmpnextfound
10493
10494 0000287E 26803F05      cmp     byte [es:bx], 5 ; 05h = extended partition id.
10495 00002882 7410          je      short fmpgot ; Extended DOS CHS
10496
10497 ; 19/12/2023 - Retro DOS v5.0
10498 00002884 26803F0F      cmp     byte [es:bx], 0Fh ; Extended DOS LBA
10499 00002888 740A          je      short fmpgot
10500
10501 add     bx, 16
10502 0000288D 81FB0204      cmp     bx, 402h ; 202h+bootbias
10503 00002891 75EB          jne     short fmpnext
10504 ;jmp     short fmpnextfound ; extended partition not found
10505 ; 18/12/2022
10506 fmpnextfound:
10507 00002893 C3          retn
10508
10509 ; 30/07/2019 - Retro DOS v3.2
10510 ;jb      short fmpnext
10511 ;fmpret:
10512 ;retn    ; 29/05/2019
10513
10514 ; -----
10515
10516 ; 19/10/2022
10517 fmpgot:
10518 00002894 E82B01      call    dmax_check ; check for drvmax already 26
10519 00002897 73FA          jnb     short fmpnextfound ; done if too many
10520
10521 00002899 8B3E[621A]      mov     di, [end_of_bdss] ; get next free bds
10522
10523 ; 19/12/2023
10524 ;mov     word [di+47h], 1 ; [di+BDS.bdsminimini]
10525 ;; 10/12/2022
10526 ;or      byte [di+23h], 1
10527 ;;or     word [di+23h], 1 ; [di+BDS.flags]
10528 ;          ; fNon_Removable
10529 ;mov     byte [di+22h], 5 ; [di+BDS.formfactor]
10530 ;          ; ffHardFile
10531 ;
10532 ; 19/12/2023 - Retro DOS v5.0
10533 0000289D C745790100      mov     word [di+79h], 1 ; [di+BDS.bdsminimini]
10534 000028A2 804D3F01      or      byte [di+3Fh], 1 ; [di+BDS.flags], fNon_Removable
10535 000028A6 C6453E05      mov     byte [di+3Eh], 5 ; [di+BDS.formfactor], ffHardFile
10536
10537 000028AA C606[081A]00      mov     byte [fbigfat], 0 ; assume 12 bit fat.
10538 000028AF A1[641A]      mov     ax, [mini_hdlim]
10539 000028B2 894515      mov     [di+15h], ax ; [di+BDS.heads]
10540 000028B5 A1[661A]      mov     ax, [mini_seclim]
10541 000028B8 894513      mov     [di+13h], ax ; [di+BDS.secptrack]
10542 000028BB A0[5E1A]      mov     al, [rom_minidisk_num]
10543 000028BE 884504      mov     [di+4], al ; [di+BDS.drivenum]

```

```

10543                                     ; set physical number
10544 000028C1 A0[7500]                mov     al, [drvmax]
10545 000028C4 884505                  mov     [di+5], al
10546                                     ; [di+BDS.drivelet]
10547 000028C7 26837F0A00              cmp     word [es:bx+10], 0
10548                                     ;ja     short fmpgot_cont
10549 000028CC 7707                    ja     short fmpgot1 ; 19/12/2023
10550 000028CE 26837F0840              cmp     word [es:bx+8], 64 ; with current bpb,
10551                                     ; only lower word is meaningful.
10552 000028D3 72BE                    jnb     short fmpnextfound
10553                                     ; should be bigger than 64 sectors at least
10554 fmpgot1: ; 19/12/2023
10555 ;fmpgot_cont:
10556 000028D5 83EB04                  sub     bx, 4 ; let bx point to the start of the entry
10557 000028D8 268A7702              mov     dh, [es:bx+2] ; cylinder
10558 000028DC 80E6C0                and     dh, 0C0h ; get higher bits of cyl
10559 000028DF D0C6                  rol     dh, 1
10560 000028E1 D0C6                  rol     dh, 1
10561 000028E3 268A5703              mov     dl, [es:bx+3] ; cyl byte
10562                                     ; 19/12/2023 - Retro DOS v5.0
10563 000028E7 89557B                mov     [di+7Bh], dx ; [di+BDS.bds_hidden_trks]
10564                                     ;mov     [di+49h], dx ; [di+BDS.bds_hidden_trks]
10565                                     ; set hidden trks
10566                                     ; 19/12/2023
10567 ;push     bx ; * ; PCDOS 7.1
10568 ;;;
10569 000028EA 268B4F08              mov     cx, [es:bx+8] ; partition size, lw
10570 000028EE 268B470A              mov     ax, [es:bx+10] ; partition size, hw
10571 000028F2 030E[8523]            add     cx, [ep_start_sector]
10572 000028F6 1306[8723]            adc     ax, [ep_start_sector+2]
10573 000028FA 31D2                  xor     dx, dx ; 19/12/2023
10574 000028FC 3916[8523]            cmp     [ep_start_sector], dx ; 0
10575                                     ;cmp     word [ep_start_sector], 0
10576 00002900 750D                  jnz     short fmpgot2
10577 00002902 3916[8723]            cmp     [ep_start_sector+2], dx ; 0
10578                                     ;cmp     word [ep_start_sector+2], 0
10579 00002906 7507                  jnz     short fmpgot2
10580 00002908 890E[8523]            mov     [ep_start_sector], cx
10581 0000290C A3[8723]              mov     [ep_start_sector+2], ax
10582 fmpgot2:
10583 0000290F 890E[8923]            mov     [ep_hidden_secs], cx
10584 00002913 A3[8B23]              mov     [ep_hidden_secs+2], ax
10585                                     ; convert start sector address to CHS
10586                                     ; 19/12/2023
10587                                     ; dx = 0
10588                                     ;push     bx ; * ; not necessary
10589                                     ;
10590                                     ;mov     bx, [di+13h] ; [di+BDS.secptrack]
10591 00002916 8B7513                mov     si, [di+13h] ; [di+BDS.secptrack]
10592                                     ;xor     dx, dx ; dx = 0
10593                                     ;div     bx
10594                                     ;div     si
10595 00002919 F7F6                  xchg     ax, cx
10596 0000291B 91                    ;div     bx
10597                                     ;div     si
10598 0000291C F7F6                  ;mov     bx, [di+15h] ; [di+BDS.heads]
10599                                     ; 17/04/2024 (BugFix)
10600                                     ;mov     si, [di+15h] ; [di+BDS.heads]
10601 0000291E 8B7515                mov     si, [di+15h] ; [di+BDS.heads]
10602 00002921 91                    xchg     ax, cx
10603 00002922 31D2                  xor     dx, dx
10604                                     ;div     bx
10605 00002924 F7F6                  div     si
10606 00002926 91                    xchg     ax, cx
10607                                     ;div     bx
10608 00002927 F7F6                  div     si
10609                                     ;pop     bx ; *
10610                                     ;
10611                                     ;or     cx, cx
10612 00002929 09C9                  jnz     short fmpgot_lba_rd
10613 0000292B 7505                  cmp     ax, 1024 ; cylinder number < 1024, CHS read is proper
10614 0000292D 3B0004              jnb     short fmpgot_chs_rd
10615 fmpgot_lba_rd:
10616 00002930 7235                  or     byte [di+40h], 4 ; set fLBArw flag ; LBA read/write ok/ready
10617 00002932 804D4004              mov     dl, [rom_minidisk_num]
10618 00002936 8A16[5E1A]            push     ds
10619 0000293A 1E                    ; 19/12/2023
10620                                     ;push     si ; ** ; not necessary
10621 0000293B 31C0                  xor     ax, ax ; push bp
10622                                     ; mov bp, sp ; (*)
10623                                     ;
10624 0000293D 50                    push     ax ; 0
10625 0000293E 50                    push     ax ; 0
10626 0000293F FF36[8B23]            push     word [ep_hidden_secs+2]
10627 00002943 FF36[8923]            push     word [ep_hidden_secs]
10628 00002947 B80002              mov     ax, bootbias ; 200h
10629 00002949 B80002              ;mov     ax, 200h ; bootbias (buffer offset)
10630                                     ;push     es ; buffer segment
10631 0000294A 06                    push     ax
10632 0000294B 50                    push     ax
10633 0000294C B80100              mov     ax, 1
10634 0000294F 50                    push     ax ; read count
10635 00002950 B81000              mov     ax, 10h ; DAP size = 16
10636 00002953 50                    push     ax
10637 00002954 8CD0              mov     ax, ss
10638 00002956 8ED8              mov     ds, ax
10639 00002958 89E6              mov     si, sp ; ds:si = Disk Address Packet
10640                                     ;
10641 0000295A B442              mov     ah, 42h ; LBA read
10642 0000295C CD13              int     13h ; DISK - IBM/MS Extension
10643                                     ; EXTENDED READ (DL - drive, DS:SI - disk address packet)
10644                                     ; 19/12/2023
10645 0000295E 9F                    ;pushf ; PCDOS 7.1 IBMBIO.COM BUG! Erdogan Tan - 08/08/2023
10646 0000295F 83C410              add     sp, 16
10647 00002962 9E                    ;popf ; BUG!
10648                                     ; mov sp, bp ; (*)
10649                                     ; pop bp
10650                                     ; 19/12/2023
10651 0000295E 9F                    lahf ; load status flags into AH
10652 0000295F 83C410              add     sp, 16
10653 00002962 9E                    sahlf ; store AH into flags
10654                                     ;
10655                                     ;pop     si ; ** ; 19/12/2023
10656 00002963 1F                    pop     ds
10657 00002964 7317              jnc     short fmpgot3
10658 fmpnotfound: ; 19/12/2023
10659 00002966 C3                    retn
10660                                     ; jmp     short fmpgot3
10661                                     ;;;
10662                                     ; 19/12/2023
10663 fmpgot_chs_rd:
10664 00002967 268B4F02              mov     cx, [es:bx+2] ; cylinder, cylinder/sector
10665 0000296B 268A7701              mov     dh, [es:bx+1] ; head

```



```

10667 0000296F 8A16[5E1A]      mov     dl, [rom_minidisk_num]
10668 00002973 B80002      mov     bx, 200h      ; bootbias
10669 00002976 B80102      mov     ax, 201h
10670 00002979 CD13          int     13h          ; DISK - READ SECTORS INTO MEMORY
10671                          ; AL = number of sectors to read, CH = track, CL = sector
10672                          ; DH = head, DL = drive, ES:BX -> buffer to fill
10673                          ; Return: CF set on error, AH = status, AL = number of sectors read
10674
10675 ;fmpgot3:      ; 19/12/2023
10676 0000297B 72E9      ;jc      short fmpnextfound
10677                          jc      short fmpnotfound
10678 0000297D BBC203      fmpgot3:      mov     bx, 3C2h      ; 1C2h+bootbias
10679
10680                          ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
10681                          ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:2C7Ch
10682 00002980 26817F3C55AA      cmp     word [es:bx+3Ch], 0AA55h ; 03C2h+03Ch = 3FEh
10683                          ;jne      short fmpnextfound ; not a valid boot sector !
10684                          ; 19/12/2023
10685 00002986 75DE      ;jne      short fmpnotfound ; not a valid boot sector !
10686
10687                          ; 13/08/2023
10688                          ;push     es
10689 00002988 E80800      call    setmini      ; install a mini disk.
10690                          ;bx value saved.
10691                          ;pop      es ; 13/08/2023
10692 0000298B 7203      ;jc      short fmpnextchain
10693 0000298D E84700      call    xinstall_bds ; -- install the bdsm into table
10694
10695 00002990 E9EBFE      fmpnextchain:  jmp     fmpnext      ; let's find out
10696                          ; if we have any chained partition
10697
10698 ; -----
10699
10700 ;fmpnextfound:      ; 18/12/2022
10701                          ;retn
10702
10703 ; ===== S U B   R O U T I N E =====
10704
10705 ; 15/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
10706 ; 28/12/2018 - Retro DOS v4.0 (MSDOS 6.21)
10707
10708 ; 19/12/2022 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
10709 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2C92h)
10710
10711 setmini:      ; 'setmini' is called from 'find_mini_partition' procedure
10712
10713 00002993 57      push     di
10714 00002994 53      push     bx
10715                          ; 12/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
10716                          ; ds = cs = BIOSDATA segment
10717                          ;push     ds
10718 00002995 06      push     es
10719
10720 setmini_1:
10721                          ;cmp     byte [es:bx], 1 ; FAT12 partition
10722                          ;je      short setmini_2
10723                          ;cmp     byte [es:bx], 4 ; FAT16 (CHS) partition
10724                          ;je      short setmini_2
10725                          ;cmp     byte [es:bx], 6 ; FAT16 BIG (CHS) partition
10726                          ;je      short setmini_2
10727                          ;
10728                          ; 19/12/2023 - Retro DOS v5.0
10729                          ;cmp     byte [es:bx], 0Bh ; FAT32 (CHS) partition
10730                          ;je      short setmini_2
10731                          ;cmp     byte [es:bx], 0Ch ; FAT32 (LBA) partition
10732                          ;je      short setmini_2
10733                          ;cmp     byte [es:bx], 0Eh ; FAT16 (LBA) partition
10734                          ;je      short setmini_2
10735
10736                          ; 19/12/2023
10737 00002996 268A07      mov     al, [es:bx]
10738 00002999 3C01      cmp     al, 1      ; FAT12 partition
10739 0000299B 7422      je      short setmini_2
10740 0000299D 3C04      cmp     al, 4      ; FAT16 (CHS) partition
10741 0000299F 741E      je      short setmini_2
10742 000029A1 3C06      cmp     al, 6      ; FAT16 BIG (CHS) partition
10743 000029A3 741A      je      short setmini_2
10744 000029A5 3C0B      cmp     al, 0Bh    ; FAT32 (CHS) partition
10745 000029A7 7416      je      short setmini_2
10746 000029A9 3C0C      cmp     al, 0Ch    ; FAT32 (LBA) partition
10747 000029AB 7412      je      short setmini_2
10748 000029AD 3C0E      cmp     al, 0Eh    ; FAT16 (LBA) partition
10749 000029AF 740E      je      short setmini_2
10750
10751 000029B1 83C310      add     bx, 16
10752 000029B4 81FB0204      cmp     bx, 402h    ; 202h+bootbias
10753                          ;jne      short setmini_1
10754 000029B8 72DC      jb      short setmini_1 ; 19/12/2023
10755 000029BA F9      stc
10756 000029BB 07      pop     es
10757                          ; 12/08/2023
10758 000029BC 5B      ;pop     ds
10759 000029BD 5F      pop     bx
10760 000029BE C3      pop     di
10761                          retn
10762
10763 ; -----
10764 000029BF E9CBF9      setmini_2:      jmp     set2      ; branch into middle of sethard
10765
10766 ; ===== S U B   R O U T I N E =====
10767
10768 ; 30/12/2022 - Retro DOS v4.2
10769 ; (SYSINITSEG is 473h for MSDOS 6.21 IO.SYS)
10770
10771 ; 15/10/2022
10772 ; 28/12/2018 - Retro DOS v4.0
10773
10774 ; dmax_check -- call this when we want to install a new drive.
10775 ; it checks for drvmax < 26 to see if there is
10776 ; a drive letter left.
10777
10778 ; drvmax < 26 : carry SET!
10779 ; drvmax >=26 : carry RESET!, error flag set for message later
10780 ; trash ax
10781
10782 ; 19/12/2023 - Retro DOS v5.0
10783
10784 000029C2 803E[7500]1A      dmax_check:      cmp     byte [drvmax], 26 ; checks for drvmax < 26
10785 000029C7 720D      jb      short dmax_ok ; return with carry if okay
10786 000029C9 06      push     es
10787                          ;mov     ax, 46Dh      ; SYSINIT_SEG (SYSINIT segment)
10788                          ;mov     ax, 544h      ; 19/12/2023 (PCDOS 7.1)
10789 000029CA B80405      mov     ax, SYSINITSEG ; 17/10/2022
10790 000029CD 8EC0      mov     es, ax

```

```

10791 ; 18/10/2022
10792 000029CF 26C606[8803]01 mov byte [es:TOOMANYDRIVESFLAG], 1 ; 09/12/2022
10793 ;mov byte ptr es:3FFh, 1 ; [es:toomanydrivesflag]
10794 ; set message flag
10795 ; [SYSINIT+toomanydrivesflag]
10796 000029D5 07 pop es
10797
10798 ;;push es
10799 ;;mov ax,SYSINIT_SEG
10800 ;;mov es,ax
10801 ;;mov byte [es:toomanydrivesflag],1
10802 ; set message flag
10803 ;;pop es
10804 ;
10805 ;mov byte [SYSINIT+toomanydrivesflag],1
10806 dmax_ok:
10807 000029D6 C3 retn
10808
10809 ; ===== S U B R O U T I N E =====
10810
10811 ; 18/10/2022
10812 ; 15/10/2022
10813 ; 28/12/2018 - Retro DOS v4.0
10814 ;
10815 ; link next bds (at ds:di) into the chain. assume that the
10816 ; chain is entirely within ds == datagrp. also update drvmax,
10817 ; dskdrv_table, and end_of_bdss.
10818
10819 ; 19/12/2023 - Retro DOS v5.0
10820 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2CE1h)
10821 xinstall_bds:
10822 000029D7 56 push si
10823 000029D8 53 push bx
10824 000029D9 8B36[1901] mov si, [start_bds] ; get first bds
10825
10826 000029DD 833CFF xinstall_bds_1: cmp word [si], 0FFFFh ; is this the last one?
10827 000029E0 7404 jz short xinstall_bds_2 ; skip ahead if so
10828 ;mov si, [si+BDS.link]
10829 000029E2 8B34 mov si, [si] ; chain through list
10830 000029E4 EBF7 jmp short xinstall_bds_1
10831
10832 xinstall_bds_2:
10833 ;mov [si+BDS.link], di
10834 000029E6 893C mov [si], di
10835 ;mov [si+BDS.link+2], ds
10836 000029E8 8C5C02 mov [si+2], ds
10837 ;mov word [di+BDS.link], -1
10838 000029EB C705FFFF mov word [di], 0FFFFh ; make sure it is a null ptr.
10839 ;mov [di+BDS.link+2], ds
10840 000029EF 8C5D02 mov [di+2], ds ; might as well plug segment
10841 ; 20/03/2019 - Retro DOS v4.0
10842 ;lea bx, [di+BDS.BPB]
10843 000029F2 8D5D06 lea bx, [di+6]
10844 000029F5 8B36[601A] mov si, [last_dskdrv_table]
10845 000029F9 891C mov [si], bx
10846 000029FB 8306[601A]02 add word [last_dskdrv_table], 2
10847 00002A00 FE06[7500] inc byte [drvmax]
10848 ;add word [end_of_bdss], 100 ; BDS.size = 100
10849 ; 19/12/2023 - Retro DOS v5.0
10850 00002A04 8106[621A]9600 add word [end_of_bdss], 150 ; BDS.size = 150
10851 00002A0A 5B pop bx
10852 00002A0B 5E pop si
10853 00002A0C C3 retn
10854
10855 ; ===== S U B R O U T I N E =====
10856
10857 ; 17/10/2022
10858 ; 15/10/2022
10859 ; 28/12/2018 - Retro DOS v4.0
10860 ; 03/06/2018 - Retro DOS v3.0
10861
10862 ; 19/12/2023 - Retro DOS v5.0
10863 cmos_clock_read:
10864 00002A0D 50 push ax
10865 00002A0E 51 push cx
10866 00002A0F 52 push dx
10867 00002A10 55 push bp
10868 00002A11 31ED xor bp, bp
10869
10870 00002A13 31C9 loop_clock: xor cx, cx
10871 00002A15 31D2 xor dx, dx
10872 00002A17 B402 mov ah, 2
10873 00002A19 CD1A int 1Ah ; CLOCK - READ REAL TIME CLOCK (AT,XT286,CONV,PS)
10874 ; Return: CH = hours in BCD
10875 ; CL = minutes in BCD
10876 ; DH = seconds in BCD
10877
10878 ; 19/12/2023
10879 00002A1B 21C9 ;cmp cx, 0
10880 00002A1D 750F and cx, cx
10881 jnz short clock_present
10882 00002A1F 09D2 ;cmp dx, 0
10883 00002A21 750B or dx, dx
10884 jnz short clock_present
10885 ;cmp bp, 1 ; read again after a slight delay, in case clock
10886 ;je short no_readdate ; was at zero setting.
10887 00002A23 21ED and bp, bp
10888 00002A25 751A jnz short no_readdate
10889 00002A27 45 inc bp ; only perform delay once.
10890 ;mov cx, 4000h ; 16384
10891 00002A28 B540 ; 19/12/2023
10892 mov ch, 40h ; cx = 4000h ; 16384
10893 00002A2A E2FE delay: loop delay
10894 00002A2C EBE5 jmp short loop_clock
10895
10896 ; -----
10897 clock_present:
10898 ;mov byte [cs:havecmosclock], 1 ; set the flag for cmos clock
10899 ; 19/12/2023
10900 ; ds = cs
10901 00002A2E C606[8C04]01 mov byte [havecmosclock], 1 ; set the flag for cmos clock
10902
10903 00002A33 E81000 call cmosck ; reset cmos clock rate that may be
10904 ; possibly destroyed by cp dos and
10905 ; post routine did not restore that.
10906 00002A36 56 push si
10907 00002A37 E831EE call read_real_date ; read real-time clock for date
10908 00002A3A FA cli
10909 ;mov ds:daycnt, si ; set system date
10910 00002A3B 8936[8904] mov [daycnt], si
10911 00002A3F FB sti
10912 00002A40 5E pop si
10913 no_readdate:
10914 00002A41 5D pop bp

```

```

10915 00002A42 5A          pop     dx
10916 00002A43 59          pop     cx
10917 00002A44 58          pop     ax
10918 cmosck9:          ; 19/12/2023
10919 00002A45 C3          retn
10920
10921 ; -----
10922 ; the following code is written by jack gully in engineering group.
10923 ; cp dos (CP/DOS, OS/2) is changing cmos clock rate for its own purposes
10924 ; and if the use cold boot the system to use pc dos while running cp dos,
10925 ; the cmos clock rate are still slow which slow down disk operations
10926 ; of pc dos which uses cmos clock. pc dos is put this code in msinit
10927 ; to fix this problem at the request of cp dos.
10928
10929 ; the program is modified to be run on msinit. equates are defined
10930 ; in cmosequ.inc. this program will be called by cmos_clock_read procedure.
10931
10932 ; the following code cmosck is used to insure that the cmos has not
10933 ; had its rate controls left in an invalid state on older at's.
10934
10935 ; it checks for an at model byte "fc" with a submodel type of
10936 ; 00, 01, 02, 03 or 06 and resets the periodic interrupt rate
10937 ; bits in case post has not done it. this initialization routine
10938 ; is only needed once when dos loads. it should be run as soon
10939 ; as possible to prevent slow diskette access.
10940
10941 ; this code exposes one to dos clearing cmos setup done by a
10942 ; resident program that hides and re-boots the system.
10943
10944 cmosck:              ; check and reset rtc rate bits
10945
10946 ;model byte and submodel byte were already determined in msinit.
10947
10948 ; 16/06/2018 - Retro DOS v3.0
10949 ; 19/03/2018 (Model: 0FCh, Sub Model: 01h, REF: AMIBIOS Prog. Guide)
10950
10951 ; 19/12/2023 - Retro DOS v5.0
10952
10953 ; 19/12/2023
10954 ; ds = cs
10955 ;push ax ; not necessary ; 19/12/2023
10956
10957 cmp     byte [model_byte], 0FCh
10958 00002A46 803E[AF05]FC  ;cmp     byte [cs:model_byte], 0FCh
10959 ;cmp     short cmosck9 ; Exit if not an AT model
10960 00002A4B 75F8          jnz     short cmosck9
10961 00002A4D 803E[B005]06  ;cmp     byte [secondary_model_byte], 6 ; 21/04/2024
10962 ;cmp     byte [cs:secondary_model_byte], 6
10963 ; Is it 06 for the industrial AT ?
10964 00002A52 7407          jz      short cmosck4 ; Go reset CMOS periodic rate if 06
10965 00002A54 803E[B005]04  ;cmp     byte [secondary_model_byte], 4
10966 ;cmp     byte [cs:secondary_model_byte], 4
10967 ; Is it 00, 01, 02, or 03 ?
10968 00002A59 73EA          jnb     short cmosck9 ; EXIT if problem fixed by POST
10969 ; Also, Secondary_model_byte = 0
10970 ; when AH=0C0h, int 15h failed.
10971 ; RESET THE CMOS PERIODIC RATE
10972 ; Model=FC submodel=00,01,02,03 or 06
10973 cmosck4:
10974 00002A5B B08A          mov     al, 8Ah ; cmos_reg_a|nmi
10975 ; NMI disabled on return
10976 00002A5D B426          mov     ah, 26h ; 00100110b
10977 ; Set divider & rate selection
10978 00002A5F E80B00        call    cmos_write
10979 00002A62 B08B          mov     al, 8Bh ; cmos_reg_b|nmi
10980 ; NMI disabled on return
10981 00002A64 E82000        call    cmos_read
10982 00002A67 2407          and     al, 7 ; 00000111b
10983 ; clear SET,PIE,AIE,UIE,SQWE
10984 00002A69 88C4          mov     ah, al
10985 00002A6B B00B          mov     al, 0Bh ; cmos_reg_b
10986 ; NMI enabled on return
10987 ; 19/12/2023
10988 ;call    cmos_write
10989 ;cmosck9:
10990 ;pop     ax ; 19/12/2023
10991 ;retn
10992
10993 ; 19/12/2023
10994 ;jmp     short cmos_write
10995
10996 ; ===== S U B R O U T I N E =====
10997
10998 ;--- cmos_write -----
10999 ; write byte to cmos system clock configuration table :
11000 ; :
11001 ; input: (al)= cmos table address to be written to :
11002 ; bit 7 = 0 for nmi enabled and 1 for nmi disabled on exit :
11003 ; bits 6-0 = address of table location to write :
11004 ; (ah)= new value to be placed in the addressed table location :
11005 ; :
11006 ; output: value in (ah) placed in location (al) with nmi left disabled :
11007 ; if bit 7 of (al) is on. during the cmos update both nmi and :
11008 ; normal interrupts are disabled to protect cmos data integrity. :
11009 ; the cmos address register is pointed to a default value and :
11010 ; the interrupt flag restored to the entry state on return. :
11011 ; only the cmos location and the nmi state is changed. :
11012 ;-----
11013
11014 cmos_write:          ; write (ah) to location (al)
11015 00002A6D 9C          pushf
11016 00002A6E 50          push     ax
11017 00002A6F FA          cli
11018 00002A70 50          push     ax
11019 00002A71 0C80          or      al, 80h ; save user nmi state
11020 00002A73 E670          out     70h, al ; disable nmi for us
11021 ; CMOS Memory/RTC Index Register:
11022 ; RTC Seconds
11023 00002A75 90          nop
11024 00002A76 88E0          mov     al, ah
11025 00002A78 E671          out     71h, al ; CMOS Memory/RTC Data Register
11026 00002A7A 58          pop     ax ; get user nmi
11027 00002A7B 2480          and     al, 80h
11028 00002A7D 0C0F          or      al, 0Fh
11029 00002A7F E670          out     70h, al ; CMOS Memory/RTC Index Register:
11030 ; RTC Seconds
11031 00002A81 90          nop
11032 00002A82 E471          in      al, 71h ; CMOS Memory/RTC Data Register
11033 00002A84 58          pop     ax ; restore work registers
11034
11035 ; 19/12/2023
11036 ;push     cs ; *place code segment in stack and
11037 ;call     cmos_popf ; *handle popf for b- level 80286
11038 00002A85 EB18          ;retn
11039 ;jmp     short cmos_rw_popf

```

```

11039
11040
11041 ; ===== S U B   R O U T I N E =====
11042
11043 ;--- CMOS_READ ---
11044 ; read byte from cmos system clock configuration table :
11045 ; input: (al)= cmos table address to be read :
11046 ; bit 7 = 0 for nmi enabled and 1 for nmi disabled on exit :
11047 ; bits 6-0 = address of table location to read :
11048 ;
11049 ; output: (al) value at location (al) moved into (al). if bit 7 of (al) was :
11050 ; on then nmi left disabled. during the cmos read both nmi and :
11051 ; normal interrupts are disabled to protect cmos data integrity. :
11052 ; the cmos address register is pointed to a default value and :
11053 ; the interrupt flag restored to the entry state on return. :
11054 ; only the (al) register and the nmi state is changed. :
11055 ;-----
11056
11057 cmos_read: ; read location (al) into (al)
11058 pushf
11059 cli
11060 push bx
11061 ;push ax ; * ; AL = cmos table address to be read
11062 ; 19/12/2023
11063 mov bx, ax ; * ; input
11064 or al, 80h
11065 out 70h, al ; CMOS Memory/RTC Index Register:
11066 ; RTC Seconds
11067 nop ; (undocumented delay needed)
11068 in al, 71h ; CMOS Memory/RTC Data Register
11069
11070 ;mov bx, ax ; output
11071 ;pop ax ; * ; input
11072
11073 ; 19/12/2023
11074 ; al = output, bl = input
11075 xchg ax, bx ; *
11076 ; bl = output, al = input
11077
11078 and al, 80h
11079 or al, 0Fh
11080 out 70h, al ; CMOS Memory/RTC Index Register:
11081 ; RTC Seconds
11082 nop
11083 in al, 71h ; CMOS Memory/RTC Data Register
11084 ;mov ax, bx ; * ; output
11085 ; 19/12/2023
11086 xchg ax, bx
11087 pop bx
11088
11089 ; 19/12/2023
11090 cmos_rw_popf:
11091 push cs ; *place code segment in stack and
11092 call cmos_popf ; *handle popf for b- level 80286
11093 retn ; return with flags restored
11094
11095 ; -----
11096
11097 cmos_popf:
11098 iret ; popf for level b- parts
11099 ; return far and restore flags
11100
11101 ; -----
11102 ; MSINIT.ASM (MSDOS 6.0, 1991)
11103 ; -----
11104 ; The following routines provide support for reading in the file MSDOS.SYS.
11105 ; -----
11106
11107 ; 15/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
11108 ;
11109 ; (For Retro DOS, 'IO.SYS' and 'MSDOS.SYS' are already loaded together
11110 ; at once -as single kernel file- by the Retro DOS boot sector code.
11111 ; So, following disk reads -MSDOS.SYS loading- is not needed!
11112 ; Only needing is to move MSDOS Kernel to it's final memory location.)
11113
11114 ; 20/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
11115 ; -----
11116
11117 ;ClusterH: dw 0 ; 20/12/2023
11118
11119 ; ===== S U B   R O U T I N E =====
11120
11121 ; GetClus, read in a cluster at a specified address
11122 ;
11123 ; bx = cluster to read
11124 ; cx = sectors per cluster
11125 ; es:di = load location
11126
11127 ; 17/10/2022
11128 ;DISKRD equ diskrd - DOSBIOSEG_2C7h ; (8E5h for MSDOS 5.0 IO.SYS)
11129 ; 09/12/2022
11130 DISKRD equ diskrd
11131
11132 ; 29/12/2023
11133 ; 20/12/2023 - Retro DOS v5.0
11134 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2DC4h)
11135
11136 ; si:bx = (32 bit) cluster to read
11137 ; cx = sectors per cluster
11138 ; es:di = load location
11139
11140 ; 06/04/2024
11141 %if 1
11142 ; 17/10/2022
11143 getclus:
11144 ; 12/12/2023
11145 ; ds = cs
11146
11147 push cx ; 1*
11148 push di ; 2*
11149 ;mov [cs:doscnt], cx
11150 mov [doscnt], cx ; 12/12/2023
11151
11152 ; 20/12/2023
11153 ;mov [cs:ClusterH], si ; high word of cluster number
11154 ;mov [ClusterH], si ; high word of cluster number
11155 mov bp, si
11156
11157 mov ax, bx
11158
11159 ;dec ax
11160 ;dec ax
11161 ; 20/12/2023
11162 sub ax, 2

```

```

11163
11164
11165
11166 00002AB2 83DD00
11167
11168
11169
11170
11171 00002AB5 95
11172
11173 00002AB6 F7E1
11174
11175
11176
11177 00002AB8 95
11178
11179 00002AB9 F7E1
11180
11181
11182
11183
11184
11185
11186
11187
11188
11189
11190
11191
11192 00002ABB 01EA
11193
11194
11195
11196 00002ABD 0306[FE19]
11197 00002AC1 1316[001A]
11198
11199
11200
11201 00002AC5 1E
11202 00002AC6 52
11203 00002AC7 50
11204
11205 00002AC8 56
11206 00002AC9 53
11207
11208
11209
11210
11211
11212
11213
11214 00002ACA 53
11215 00002ACB FF36[041A]
11216
11217
11218 00002ACF F606[081A]20
11219 00002AD4 1F
11220 00002AD5 7415
11221
11222
11223 00002AD7 89F2
11224
11225 00002AD9 5E
11226 00002ADA 01F6
11227 00002ADC 11D2
11228 00002ADE 01F6
11229 00002AE0 11D2
11230
11231 00002AE2 E89600
11232 00002AE5 8B7702
11233 00002AE8 8B1F
11234
11235 00002AEA EB45
11236
11237
11238
11239 00002AEC 5E
11240 00002AED 2EF606[081A]40
11241
11242 00002AF3 752F
11243
11244 00002AF5 D1EE
11245
11246 00002AF7 01DE
11247
11248
11249 00002AF9 31D2
11250
11251
11252 00002AFB E87D00
11253
11254
11255 00002AFE 8B07
11256 00002B00 750C
11257
11258
11259
11260
11261
11262 00002B02 50
11263
11264 00002B03 46
11265
11266
11267 00002B04 31D2
11268 00002B06 E87200
11269
11270
11271
11272
11273
11274
11275
11276
11277
11278
11279 00002B09 58
11280 00002B0A 8A260000
11281
11282
11283 00002B0E 5B
11284 00002B0F 53
11285 00002B10 D1EB
11286 00002B12 7308

;;sbb [cs:ClusterH], 0
;;sbb [ClusterH], 0
sbb bp, 0

; 20/12/2023
;;xchg ax, [cs:ClusterH]
;xchg ax, [ClusterH]
xchg ax, bp

mul cx

;;xchg ax, [cs:ClusterH]
;xchg ax, [ClusterH]
xchg ax, bp ; (+)
;
mul cx ; convert to logical sector
; dx:ax = matching logical sector number
; starting from the data sector
;;add ax, [cs:bios_l]
;;adc dx, [cs:bios_h] ; dx:ax = first logical sector to read
; 12/12/2023
;add ax, [bios_l]
;adc dx, [bios_h] ; dx:ax = first logical sector to read

; 20/12/2023
;;add dx, [cs:ClusterH]
;add ax, [cs:First_Data_Sector]
;adc dx, [cs:First_Data_Sector+2]
add dx, bp ; (+)
;add dx, [ClusterH] ; convert to logical sector
; dx:ax = matching logical sector number
; starting from the data sector
add ax, [First_Data_Sector]
adc dx, [First_Data_Sector+2]
; dx:ax = first logical sector to read

unpack:
; 20/12/2023
push ds ; 3* ; ds = cs ; 12/12/2023
push dx ; 4* ; * ; 12/12/2023
push ax ; 5*
; 29/12/2023
push si ; 6*
push bx ; 7*

;;mov si, [cs:fatloc]
;mov si, [fatloc] ; 12/12/2023
;mov ds, si
; 20/12/2023
;mov ax, [fatloc]
;mov ds, ax
push bx ; 8*
push word [fatloc] ; 9*

;test byte [cs:fbigfat], 20h
test byte [fbigfat], 20h ; fbigbig FAT32 ?
pop ds ; 9* ; ds = [fatloc]
jz short not_32bit_cluster ; no

unpack32:
;push dx
mov dx, si
;mov si, bx
pop si ; 8* ; si = bx
add si, si
adc dx, dx
add si, si
adc dx, dx
; dx:si = 4*(si:bx) ; clust num offset from FAT entry 0
call get_fat_sector
mov si, [bx+2] ; high word of the FAT32 cluster number
mov bx, [bx] ; low word of the FAT32 cluster number
;pop dx
jmp short getc11

not_32bit_cluster:
;mov si, bx ; next cluster
pop si ; 8* ; si = bx
test byte [cs:fbigfat], 40h; fbig
; 16 bit fat?
jnz short unpack16 ; yes

unpack12:
shr si, 1 ; 12 bit fat. si = si/2
; si = clus + clus/2
add si, bx ; (si = byte offset of the cluster in the FAT)
;push dx ; 12/12/2023
xor dx, dx
; 12/12/2023
; ds = FAT buffer segment
call get_fat_sector
;pop dx ; 12/12/2023

mov ax, [bx] ; save it into ax
jnz short even_odd ; if not a splitted fat, check even-odd.
; 25/06/2023
;mov al, [bx] ; splitted fat

; 12/12/2023
;mov [cs:temp_cluster], al
push ax ; ** ; al = low 8 bits of 12 bits cluster number

inc si ; (next byte)

;push dx ; 12/12/2023
xor dx, dx
call get_fat_sector
;pop dx ; 12/12/2023

;mov al, ds:0
; 12/12/2023
; ds = FAT buffer segment
;mov al, [0] ; 19/10/2022
;mov [cs:temp_cluster+1], al
;mov ax, [cs:temp_cluster]
; 12/12/2023
;mov al, [cs:temp_cluster]
pop ax ; ** ; al = low 8 bits of 12 bits cluster number
mov ah, [0] ; high 4 bits (bits 7 to 11) of 12 bits cluster num

even_odd:
; 29/12/2023
pop bx ; 7* ; restore old fat entry value
push bx ; save it right away.
shr bx, 1 ; was it even or odd?
jnc short havclus ; it was even.

```

```

11287 00002B14 D1E8          shr     ax, 1          ; odd. message fat value and keep
11288                                ; the highest 12 bits.
11289 00002B16 D1E8          shr     ax, 1
11290 00002B18 D1E8          shr     ax, 1
11291 00002B1A D1E8          shr     ax, 1
11292 havclus:
11293 00002B1C 89C3          mov     bx, ax          ; now bx = new fat entry.
11294 00002B1E 81E3FF0F      and     bx, 0FFFh       ; keep low 12 bits.
11295 00002B22 EB0B          jmp     short unpackx
11296                                ; -----
11297
11298 unpack16:
11299                                ; push dx ; 12/12/2023
11300 00002B24 31D2          xor     dx, dx ; 0
11301 00002B26 D1E6          shl     si, 1          ; extend to 32 bit offset
11302                                ; adc dx, 0
11303                                ; 12/12/2023
11304 00002B28 D1D2          rcl     dx, 1
11305                                ; 12/12/2023
11306                                ; ds = FAT buffer segment
11307                                call    get_fat_sector
11308 00002B2A E84E00      call    get_fat_sector
11309                                ; pop dx ; 12/12/2023
11310 00002B2D 8B1F          mov     bx, [bx]
11311                                ; bx = new fat entry.
11312
11313 unpackx:
11314 00002B2F 31F6          xor     si, si          ; high word of cluster number = 0
11315                                ; (FAT12 or FAT16)
11316
11317 getc11:
11318 00002B31 58          pop     ax          ; 7* - cluster number lw
11319                                pop     word [cs:ClusterH]
11320 00002B32 5A          pop     dx          ; 6* - cluster number hw
11321                                ; 20/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
11322                                ; (this is a fast kernel loading method by the MSDOS programmer)
11323                                ; ((consecutive clusters --> consecutive sectors))
11324
11325 sub     ax, bx ; previous - current (or current - new)
11326 00002B33 29D8          sbb     [cs:ClusterH], si
11327                                sbb     dx, si
11328 00002B35 19F2          ; cmp [cs:ClusterH], -1 ; one apart? (current = previous+1)
11329                                cmp     dx, -1
11330                                ; 29/12/2023
11331                                inc     dx ; -1 -> 0
11332 00002B37 42          jnz     short not_consequential
11333 00002B38 7501          cmp     ax, -1          ; 0FFFFh ; is [clusterH]:ax = -1 ?
11334                                inc     ax ; -1 -> 0
11335 00002B3A 40          not_consequential:
11336                                pop     ax ; 5* ; restore logical sector (low)
11337 00002B3B 58          pop     dx ; 4* ; * ; 12/12/2023
11338 00002B3C 5A          pop     ds ; 3*
11339 00002B3D 1F
11340                                ; 12/12/2023
11341                                ; (this is a fast kernel loading method by the MSDOS programmer)
11342                                ; ((consecutive clusters --> consecutive sectors))
11343                                ; ds = cs
11344                                sub     si, bx
11345                                cmp     si, -1          ; one apart? (consecutive?)
11346                                ; (current = previous+1)
11347                                jnz     short getc12 ; no, read [doscnt] sectors
11348
11349 00002B3E 7507          ; add [cs:doscnt], cx ; (cx = sectors per cluster)
11350                                add     [doscnt], cx ; 12/12/2023 ; add to read count
11351                                jmp     unpack
11352                                ; -----
11353
11354 getc12:
11355                                push     si ; 20/12/2023
11356                                push     bx
11357 00002B47 56          ; bx = low word of the new cluster number
11358 00002B48 53          ; 20/12/2023 - Retro DOS v5.0 (32 bit cluster numbers)
11359                                ; si = high word of the new cluster number
11360                                push     dx ; sector to read (high word)
11361                                push     ax ; sector to read (low word)
11362 00002B49 52
11363 00002B4A 50
11364                                ; 12/12/2023
11365                                ; ds = cs
11366                                mov     ax, [cs:drvfat] ; get drive and fat spec
11367                                mov     cx, [cs:doscnt]
11368                                mov     ax, [drvfat] ; get drive and fat spec
11369 00002B4B A1[FC19]
11370                                ; 20/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
11371                                ; dma and segment (64K boundary) overrun precaution
11372                                ; (sector count will be decreased if it is required)
11373                                mov     cx, di
11374                                not     cx ; cx = 65535 - cx
11375                                shr     cx, 1 ; cx = cx/2
11376 00002B4E 89F9          xor     cl, cl
11377 00002B50 F7D1          xchg    cl, ch ; cx = cx/256
11378 00002B52 D1E9
11379 00002B54 30C9
11380 00002B56 86CD
11381                                ; cmp cx, [cs:doscnt]
11382                                ; if sector read count > cx, decrease it to cx
11383                                cmp     cx, [doscnt]
11384 00002B58 3B0E[021A]    jbe     short getc13
11385 00002B5C 7604
11386                                ; 12/12/2023
11387                                ; mov cx, [cs:doscnt]
11388 00002B5E 8B0E[021A]    mov     cx, [doscnt]
11389
11390 00002B62 5A          getc13:
11391                                pop     dx ; sector to read for diskrd (low)
11392                                pop     word [cs:start_sec_h]
11393 00002B63 8F06[9C04]    ; 12/12/2023
11394                                pop     word [start_sec_h]
11395                                ; sector to read for diskrd (high)
11396                                ; 06/04/2024
11397 00002B67 51          push     cx ; +*
11398                                ; 12/12/2023
11399                                ; ds = cs
11400                                push     ds
11401                                push     cs
11402                                pop     ds
11403                                push     cs
11404                                pop     ds
11405                                ; simulate far call
11406 00002B68 0E          push     cs
11407                                ; 20/12/2023
11408                                ; 17/10/2022
11409                                mov     bp, DISKRD ; offset diskrd
11410 00002B69 BD[0C0A]

```



```

11411      ;mov     bp, 0A2Bh      ; 20/12/2023
11412      ;      (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0A2Bh ; 364h:0A2Bh)
11413      ;mov     bp, 8E5h      ; 17/10/2022
11414      ;      ; 2C7h:8E5h = 70h:2E55h
11415
11416 00002B6C E804EF      call     call_bios_code ; read the clusters
11417
11418      ;pop     ds
11419      ; 12/12/2023
11420      ; ds = cs
11421
11422      ; 06/04/2024
11423      ;;;
11424 00002B6F 58      pop     ax ; +*      ; sector count
11425      ;;;
11426
11427      pop     bx      ; lw of the new cluster number
11428 00002B70 5E      pop     si ; 20/12/2023 ; hw of the new cluster number
11429
11430 00002B72 5F      pop     di ; 2* - (kernel) load location (es:di)
11431
11432      ; 06/04/2024
11433      ;mov     ax, [cs:doscnt]      ; get number of      sectors read
11434      ; 12/12/2023
11435      ;mov     ax, [doscnt]
11436
11437 00002B73 86E0      xchg     ah, al      ; multiply by 256
11438 00002B75 D1E0      shl     ax, 1      ; times 2 equal 512
11439 00002B77 01C7      add     di, ax      ; update load location
11440
11441 00002B79 59      pop     cx ; 1*      ; restore sectors/cluster
11442
11443 00002B7A C3      retn
11444
11445      ; ===== S U B   R O U T I N E =====
11446
11447      ;function: find and read the corresponding fat sector into ds:0
11448      ;
11449      ;in). dx:si - offset value (starting from fat entry 0) of fat entry to find. M054
11450      ;      ds - fatloc segment
11451      ;      cs:drvfat - logical drive number, fat id
11452      ;      cs:md_sectorsize
11453      ;      cs:last_fat_secnum - last fat sector number read in.
11454      ;
11455      ;out). corresponding fat sector read in.
11456      ;      bx = offset value from fatlog segment.
11457      ;      other registers are saved.
11458      ;      zero flag set if the fat entry is splitted, i.e., when 12 bit fat entry
11459      ;      starts at the last byte of the fat sector. in this case, the caller
11460      ;      should save this byte, and read the next fat sector to get the rest
11461      ;      of the fat entry value. (this will only happen with the 12 bit fat.)
11462
11463      ; 17/10/2022
11464      get_fat_sector:
11465      ; 20/12/2023
11466      ; 12/12/2023
11467      ; ds = fat buffer segment
11468
11469      ; 12/12/2023
11470      ;push     ax ; (not necessary)
11471 00002B7B 51      push     cx ; read count (sectors per cluster)
11472 00002B7C 57      push     di ; IBMDOS.COM/MSDOS.SYS load offset
11473 00002B7D 56      push     si ; FAT offset value (from fat entry 0)
11474 00002B7E 06      push     es ; IBMDOS.COM/MSDOS.SYS load segment
11475 00002B7F 1E      push     ds ; FAT buffer segment
11476
11477      ; 12/12/2023
11478 00002B80 0E      push     cs
11479 00002B81 1F      pop      ds
11480
11481      ; 06/04/2024
11482      ; dx:si = offset value (starting from fat entry 0)
11483      ;      of fat entry to find
11484
11485 00002B82 89F0      mov      ax, si
11486      ;mov     cx, [cs:md_sectorsize] ; 512
11487      ; 12/12/2023
11488      ;mov     cx, [md_sectorsize] ; 512
11489      ;div     cx      ; ax = sector number, dx = offset
11490      ; 12/12/2023
11491      ;nop
11492
11493      ; 12/12/2023
11494 00002B84 F736[0A1A]      div     word [md_sectorsize] ; 512
11495
11496      ; ax = FAT sector (sequence/index) number
11497      ; dx = cluster number offset
11498
11499      ; Get rid of the assumption that
11500      ; there is only one reserved sector
11501
11502      ; 12/12/2023 ; *
11503      ;push     es ; *
11504      ;push     ds ; *
11505      ;push     di ; *
11506 00002B88 50      push     ax
11507      ;push     cs ; *
11508      ;pop      ds ; *
11509
11510      ;mov     ax, [cs:drvfat]      ; get drive # and FAT id
11511      ; 12/12/2023
11512 00002B89 A1[FC19]      mov      ax, [drvfat]      ; get drive # and FAT id
11513 00002B8C BD[A405]      mov      bp, SETDRIVE
11514      ;mov     bp, 5AEh ; PCDOS 7.1 IBMBIO.COM
11515      ;mov     bp, 4D7h      ; setdrive
11516      ;      ; at 2C7h:4D7h = 70h:2A47h
11517 00002B8F 0E      push     cs      ; simulate far call
11518 00002B90 E8E0EE      call     call_bios_code ; get bds for drive
11519 00002B93 58      pop      ax      ; (sector number -without reserved and hidden sectors-)
11520 00002B94 26034509      add     ax, [es:di+9] ; [es:di+BDS.resectors]
11521      ;      ; add #reserved_sectors
11522      ; 12/12/2023
11523      ;pop     di ; *
11524      ;pop     ds ; *
11525      ;pop     es ; *
11526
11527      ; 12/12/2023
11528      ; ds = cs
11529 00002B98 3B06[0C1A]      cmp      ax, [last_fat_sec_num]
11530      ;cmp     ax, [cs:last_fat_sec_num]
11531 00002B9C 741C      jz      short gfs_split_chk ; don't need to read it again.
11532 00002B9E A3[0C1A]      mov      [last_fat_sec_num], ax
11533      ;mov     [cs:last_fat_sec_num], ax
11534      ;      ; sector number

```

```

11535                                     ; (in the partition, without hidden sectors)
11536                                     ; 13/12/2023
11537 00002BA1 07 pop     es ; FAT buffer segment (DS on top of the stack)
11538 00002BA2 06 push    es ; (put it on top of the stack again)
11539
11540 00002BA3 52 push    dx ; cluster number offset
11541
11542                                     ; 12/12/2023
11543 00002BA4 31C9 xor     cx, cx
11544 00002BA6 890E[9C04] mov     [start_sec_h], cx ; 0
11545                                     ; mov     word [cs:start_sec_h], 0
11546                                     ; prepare to read the fat sector
11547                                     ; start_sec_h is always 0 for fat sector.
11548 00002BAA 89C2 mov     dx, ax
11549                                     ; 12/12/2023
11550 00002BAC 41 inc     cx ; cx = 1
11551                                     ; mov     cx, 1 ; 1 sector read
11552                                     ; mov     ax, [cs:drvfat]
11553 00002BAD A1[FC19] mov     ax, [drvfat]
11554                                     ; push    ds
11555                                     ; pop     es
11556
11557 00002BB0 31FF xor     di, di ; 0 ; es:di -> fatloc segment:0
11558
11559                                     ; 12/12/2023
11560                                     ; push    ds
11561                                     ; push    cs
11562                                     ; pop     ds
11563
11564 00002BB2 0E push    cs ; simulate far call
11565
11566                                     ; 20/12/2023
11567                                     ; 17/10/2022
11568 00002BB3 BD[0C0A] mov     bp, DISKRD ; offset diskrd
11569                                     ; mov     bp, 0A2Bh ; 20/12/2023
11570                                     ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0A2Bh ; 364h:0A2Bh)
11571                                     ; mov     bp, 8E5h ; 17/10/2022
11572                                     ; 2C7h:8E5h = 70h:2E55h
11573
11574 00002BB6 E8BAEE call   call_bios_code ; read the clusters
11575
11576                                     ; 12/12/2023
11577                                     ; pop     ds
11578                                     ; ds = cs = biosdata segment
11579
11580 00002BB9 5A pop     dx ; cluster number offset
11581
11582 gfs_split_chk:
11583                                     ; 13/12/2023
11584                                     ; mov     cx, [cs:md_sectorsize] ; 512
11585 00002BBA 8B0E[0A1A] mov     cx, [md_sectorsize]
11586 ;gfs_split_chk:
11587 00002BBE 49 dec     cx ; 511
11588 00002BBF 39CA cmp     dx, cx ; if offset points to the
11589                                     ; last byte of this sector,
11590                                     ; then splitted entry.
11591 00002BC1 89D3 mov     bx, dx ; set bx to dx
11592
11593                                     ; 12/12/2023
11594                                     ; bx = dx = cluster number offset in the FAT buffer
11595 00002BC3 1F pop     ds ; FAT buffer segment
11596 00002BC4 07 pop     es ; IBMDOS.COM/MSDOS.SYS load segment
11597 00002BC5 5E pop     si ; FAT offset value (from fat entry 0)
11598 00002BC6 5F pop     di ; IBMDOS.COM/MSDOS.SYS load offset
11599 00002BC7 59 pop     cx ; read count (sectors per cluster)
11600 pop     ax
11601
11602 00002BC8 C3 retn
11603
11604 %else
11605                                     ; 06/04/2024 - temporary
11606 temp_cluster: dw 0
11607
11608 ClusterH: dw 0
11609
11610 ; ===== S U B R O U T I N E =====
11611
11612 getclus:
11613                                     push    cx ; 1*
11614                                     ; si:bx = (32 bit) cluster to read
11615                                     ; cx = sectors per cluster
11616                                     ; es:di = load location
11617                                     ; 2*
11618                                     push    di
11619                                     mov     [cs:doscnt], cx
11620                                     mov     ax, bx
11621                                     mov     [cs:ClusterH], si ; high word of cluster number
11622                                     sub     ax, 2
11623                                     sbb     word [cs:ClusterH], 0
11624                                     xchg    ax, [cs:ClusterH]
11625                                     mul     cx
11626                                     xchg    ax, [cs:ClusterH]
11627                                     mul     cx
11628                                     add     dx, [cs:ClusterH] ; convert to logical sector
11629                                     ; dx:ax = matching logical sector number
11630                                     ; starting from the data sector
11631                                     add     ax, [cs:First_Data_Sector]
11632                                     adc     dx, [cs:First_Data_Sector+2]
11633                                     ; dx:ax = first logical sector to read
11634
11635 unpack:
11636                                     push    ds ; 3*
11637                                     push    ax ; 4*
11638                                     push    si ; 5*
11639                                     push    bx ; 6*
11640                                     mov     ax, [cs:fatloc]
11641                                     mov     ds, ax
11642                                     test    byte [cs:fbigfat], 20h ; fbigbig
11643                                     ; FAT32 ?
11644                                     jz     short not_32bit_cluster ; no
11645                                     ; yes
11646                                     ; 7*
11647                                     push    dx
11648                                     mov     dx, si
11649                                     mov     si, bx
11650                                     add     si, si
11651                                     adc     dx, dx
11652                                     add     si, si ; dx:si = 4*(si:bx)
11653                                     call   get_fat_sector
11654                                     mov     si, [bx+2] ; byte 16-31 of the FAT32 cluster number
11655                                     mov     bx, [bx] ; byte 0-15 of the FAT32 cluster number
11656                                     pop     dx ; 7*
11657                                     jmp     short getc11
11658 ; -----

```



```

11659 not_32bit_cluster:
11660     mov     si, bx           ; next cluster
11661     test    byte [cs:fbigfat], 40h ; fbig
11662     ; FAT16 ?
11663     jnz     short unpack16 ; yes
11664
11665 unpack12:
11666     shr     si, 1
11667     add     si, bx           ; 12 bit fat. si=si/2
11668     ; si = clus + clus/2
11669     ; (si = byte offset of the cluster in the FAT)
11670     push    dx
11671     xor     dx, dx
11672     call    get_fat_sector
11673     pop     dx
11674     mov     ax, [bx]         ; save cluster number into ax
11675     jnz     short even_odd  ; if not a splitted fat, check even-odd
11676     mov     al, [bx]         ; (not needed!) Erdogan Tan - 2023
11677     mov     byte [cs:temp_cluster], al ; splitted fat
11678     inc     si               ; (next byte)
11679     push    dx
11680     xor     dx, dx
11681     call    get_fat_sector
11682     pop     dx
11683     mov     al, [0]          ; mov ah,[0]
11684     mov     byte [cs:temp_cluster+1], al
11685     mov     ax, [cs:temp_cluster] ; mov al,[cs:temp_cluster]
11686
11687 even_odd:
11688     pop     bx               ; restore old fat entry value
11689     push    bx               ; 6*
11690     shr     bx, 1            ; was it even or odd?
11691     jnb     short havclus    ; it was even
11692     shr     ax, 1            ; odd. massage fat value and keep
11693     ; the highest 12 bits.
11694     shr     ax, 1
11695     shr     ax, 1
11696
11697 havclus:
11698     mov     bx, ax           ; now bx = new fat entry
11699     and     bx, 0FFFh        ; keep low 12 bits
11700     jmp     short unpackx
11701
11702 ; -----
11703
11704 unpack16:
11705     push    dx
11706     xor     dx, dx           ; extend to 32 bit offset
11707     shl     si, 1            ; cluster number * 2
11708     adc     dx, 0
11709     call    get_fat_sector
11710     pop     dx
11711     mov     bx, [bx]         ; bx = new fat entry
11712
11713 unpackx:
11714     xor     si, si           ; high word of cluster number = 0
11715     ; (FAT12 or FAT16)
11716
11717 getc11:
11718     pop     ax               ; 6* - cluster number lw
11719     pop     word [cs:ClusterH] ; 5* - cluster number hw
11720     sub     ax, bx           ; previous - current (or current - new)
11721     sbb     [cs:ClusterH], si
11722     cmp     word [cs:ClusterH], -1 ; one apart? (current = previous+1)
11723     jnz     short not_consequential
11724     cmp     ax, -1           ; 0FFFFh ; is [ClusterH]:ax = -1 ?
11725
11726 not_consequential:
11727     pop     ax               ; 4* - low word of first logical sector
11728     pop     ds               ; 3*
11729     short getc12
11730     add     [cs:doscnt], cx ; consequential cluster read, +1 cluster sectors
11731     ; (cx = sectors per cluster)
11732     jmp     unpack
11733
11734 ; -----
11735
11736 getc12:
11737     push    bx
11738     push    si
11739     push    dx               ; sector to read (high)
11740     push    ax               ; sector to read (low)
11741     mov     ax, [cs:drvfat] ; get drive and fat spec
11742     mov     cx, di           ; dma and segment (64K boundary) overrun precaution
11743     ; (sector count will be decreased if it is required)
11744     not     cx               ; cx = 65535 - cx
11745     shr     cx, 1            ; cx = cx/2
11746     xor     cl, cl
11747     xchg    cl, ch           ; cx = cx/256
11748     cmp     cx, [cs:doscnt] ; if sector read count > cx, decrease it to cx
11749     jbe     short getc13
11750     mov     cx, [cs:doscnt]
11751
11752 getc13:
11753     pop     dx               ; sector to read for diskrd (low)
11754     pop     word [cs:start_sec_h] ; sector to read for diskrd (high)
11755     push    cx
11756     push    ds
11757     push    cs
11758     pop     ds
11759     push    cs               ; simulate far call
11760     mov     bp, DISKRD ; BIOSCODE:0A2Bh ; 364h:0A2Bh
11761     call    call_bios_code
11762     pop     ds
11763     pop     ax               ; sector count
11764     pop     si
11765     pop     bx
11766     pop     di               ; 2* - load location (es:di)
11767     xchg    ah, al
11768     shl     ax, 1            ; ax = ax * 512 ; byte count
11769     add     di, ax           ; update load location
11770     pop     cx               ; 1* - restore sectors/cluster
11771     retn
11772
11773 ; ===== S U B R O U T I N E =====
11774
11775 get_fat_sector:
11776     push    ax               ; dx:si = offset value (starting from fat entry 0)
11777     ; of fat entry to find
11778     push    cx
11779     push    di
11780     push    si
11781     push    es
11782     push    ds
11783     mov     ax, si
11784     mov     cx, [cs:md_sectorsize] ; 512
11785     div     cx
11786     ; ax = sector number, dx = offset
11787     push    es
11788     push    ds
11789     push    di
11790     push    ax

```

```

11783         push    cs
11784         pop     ds
11785         mov     ax, [cs:drvfat] ; get drive # and FAT id
11786         mov     bp, SetDrive ; BIOSCODE:05AEh
11787         push    cs ; simulate far call
11788         call    call_bios_code ; get bds for drive
11789         pop     ax ; (sector number -without reserved and hidden sectors-)
11790         add     ax, [es:di+9] ; [es:di+BDS.resectors]
11791                     ; add #reserved_sectors
11792         pop     di
11793         pop     ds
11794         pop     es
11795         cmp     ax, [cs:last_fat_sec_num]
11796         jz      short gfs_split_chk ; don't need to read it again
11797         mov     [cs:last_fat_sec_num], ax ; sector number
11798                     ; (in the partition, without hidden sectors)
11799         push    dx
11800         mov     word [cs:start_sec_h], 0 ; prepare to read the fat sector
11801                     ; start_sec_h is always 0 for fat sector
11802         mov     dx, ax
11803         mov     cx, 1 ; 1 sector read
11804         mov     ax, [cs:drvfat]
11805         push    ds
11806         pop     es
11807         xor     di, di ; es:di -> fatloc segment:0
11808         push    ds
11809         push    cs
11810         pop     ds
11811         push    cs ; simulate far call
11812         mov     bp, DISKRD ; BIOSCODE:0A2Bh ; 364h:0A2Bh
11813         call    call_bios_code
11814         pop     ds
11815         pop     dx
11816         mov     cx, [cs:md_sectorsize] ; 512
11817 gfs_split_chk:
11818         dec     cx ; 511
11819         cmp     dx, cx ; if offset points to the last byte of this sector,
11820                     ; then splitted entry.
11821         mov     bx, dx ; offset value from fatloc segment
11822         pop     ds
11823         pop     es
11824         pop     si
11825         pop     di
11826         pop     cx
11827         pop     ax
11828         retn
11829
11830 %endif
11831
11832 ; 15/10/2022
11833 ;Bios_Data_Init ends
11834
11835 ; -----
11836
11837 ; 09/12/2022
11838 ;db 0
11839
11840 numbertodiv equ ($-BData_start)
11841 numbertomod equ (numbertodiv % 16)
11842
11843 %if numbertomod>0 & numbertomod<16
11844         times (16-numbertomod) db 0
11845 %endif
11846
11847 ;align 16
11848
11849 ; 09/12/2022
11850 IOSYSCODESEGOFF equ $ - BData_start
11851 IOSYSCODESEG equ (IOSYSCODESEGOFF>>4)+(700h>>4)
11852
11853 ;--- End of DOSBIOS data segment -----
11854 ; -----
11855 ;db 4 dup(0)
11856 ; 09/12/2022
11857 ; times 4 db 0 ; 19/10/2022
11858 ; -----
11859
11860 ;=====
11861 ; DOS BIOS (IO.SYS) CODE SEGMENT
11862 ;=====
11863 ; 09/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
11864 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
11865
11866 section .BIOSCODE vstart=0
11867
11868 ; 30/12/2022
11869 ; (BIOSCODE SEGMENT is 2CCh for MSDOS 6.21 IO.SYS) -- ((25C0h+700h)>>4) --
11870
11871 BCode_start: ; 09/12/2022
11872
11873 ; 02/10/2022
11874
11875 ;--- DOSBIOS code segment -----
11876 ; -----
11877 ; MSBIO1.ASM (MSDOS 6.0, 1991)
11878 ; -----
11879
11880 DOSBIOSEG_2C7h: ;db 30h dup(0) ; SEGMENT 2C7h (2C70h-700h=2570h)
11881         times 48 db 0 ; 19/10/2022
11882 BiosDataWord: dw 70h
11883
11884 ; 15/10/2022
11885 ;BIOSDATAWORD equ BiosDataWord - DOSBIOSEG_2C7h
11886 ; 09/12/2022
11887 BIOSDATAWORD equ BiosDataWord
11888
11889 ; -----
11890
11891 ; 15/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
11892 ; 20/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
11893
11894 ;*****
11895 ;*
11896 ;* seg_reinit is called with ax = our new code segment value, *
11897 ;* trashes di, cx, es *
11898 ;* *
11899 ;* cas -- should be made disposable! *
11900 ;* *
11901 ;*****
11902
11903 ; 20/09/2023
11904 ; 10/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
11905 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0032h
11906

```

```

11907 _seg_reinit:
11908     mov     es, [cs:BIOSDATAWORD]
11909           ; at 2C7h:30h or 70h:25A0h
11910           ;mov    di, (offset cdev+2)
11911     00000037 BF[0406]     mov    di, cdev+2      ; 19/10/2022
11912           ;mov    cx, 4      ; (end_BC_entries - cdev)/4
11913           ; 10/08/2023
11914     0000003A B90300     mov    cx, 3 ; (PCDOS 7.1)
11915 _seg_reinit_1:
11916     stosw      ; modify Bios_Code entry points
11917     inc    di
11918     inc    di
11919     loop    _seg_reinit_1
11920           ; 10/08/2023 (PCDOS 7.1)
11921           ; (direct jump to i2f_handler from BIOSDATA:bios_i2f)
11922           ; (instead of 'bcode_i2f: dw i2f_handler, IOSYSCODESEG')
11923     00000042 26A3[0800]   mov    [es:bios_i2f_seg], ax ; actual BIOSCODE segment
11924
11925     retf
11926
11927 ; -----
11928
11929 ; 15/10/2022
11930
11931 ;*****
11932 ;*
11933 ;* chardev_entry - main device driver dispatch routine
11934 ;* called with a dummy parameter block on the stack
11935 ;* dw dispatch_table, dw prn/aux numbers (optional)
11936 ;*
11937 ;* will eventually take care of doing the transitions in
11938 ;* out of Bios_Code
11939 ;*
11940 ;*****
11941
11942           ; 20/09/2023
11943 chardev_entry:
11944     push    si
11945     push    ax
11946     push    cx
11947     push    dx
11948     push    di
11949     push    bp
11950     push    ds
11951     push    es
11952     push    bx
11953     mov     bp, sp
11954     mov     si, [bp+18] ; get return address (dispatch table)
11955     ;mov    ds, word [cs:0030h]
11956     ;mov    ds, word ptr cs:BIOSDATAWORD ; [cs:0030h]
11957     00000055 2E8E1E[3000]   mov    ds, [cs:BIOSDATAWORD] ; 17/10/2022
11958           ; 20/09/2023 (PCDOS 7.1 - IBMBIO.COM - BIOSCODE:005Ah)
11959     0000005A C434         les     si, [si]
11960     ;mov    ax, [si+2] ; get the device number if present
11961     mov     ax, es
11962     mov     [auxnum], al
11963     mov     [printdev], ah
11964     ;mov    si, [si] ; point to the device dispatch table
11965     les     bx, [ptrsav] ; get pointer to i/o packet
11966     mov     al, [es:bx+1] ; [es:bx+unit] ; al = unit code
11967     mov     ah, [es:bx+13] ; [es:bx+media] ; ah = media descrip
11968     mov     cx, [es:bx+18] ; [es:bx+count] ; cx = count
11969     mov     dx, [es:bx+20] ; [es:bx+start] ; dx = start sector
11970           ; 17/10/2022
11971     cmp     si, DSKTBL
11972     ;cmp    si, 579h ; (PCDOS 7.1 IBMBIO.COM)
11973     ;cmp    si, 4A2h ; dsktbl
11974           ; at 2C7h:4A2h = 70h:2A12h
11975     0000007D 7517         jnz     short no_sector32_mapping
11976
11977 ; Special case for 32-bit start sector number:
11978 ; if (si==dsktbl) /* if this is a disk device call */
11979 ; set high 16 bits of secnum to 0
11980 ; if (secnum == 0xffff) fetch 32 bit sector number
11981 ;
11982 ; pass high word of sector number in start_sec_h, low word in dx
11983 ;
11984 ; note: start_l and start_h are the offsets within the io_request packet
11985 ; which contain the low and hi words of the 32 bit start sector if
11986 ; it has been used.
11987 ;
11988 ; note: remember not to destroy the registers which have been set up before
11989
11990           ; 20/09/2023
11991     ;mov    ds:start_sec_h, 0 ; initialize to 0
11992     mov     word [start_sec_h], 0
11993     cmp     dx, 0FFFFh
11994     jnz     short no_sector32_mapping
11995     mov     dx, [es:bx+28] ; [es:bx+start_h]
11996           ; 32 bits dsk req
11997     ;mov    ds:start_sec_h, dx ; start_sec_h = packet.start_h
11998     mov     [start_sec_h], dx
11999     mov     dx, [es:bx+26] ; [es:bx+start_l]
12000           ; dx = packet.start_l
12001 no_sector32_mapping:
12002     xchg    ax, di
12003     mov     al, [es:bx+2] ; [es:bx+cmd]
12004     cmp     al, [cs:si]
12005     jnb     short command_error
12006     cbw
12007           ; note that al <= 15 means ok
12008     shl     ax, 1
12009     add     si, ax
12010     xchg    ax, di
12011     les     di, [es:bx+14] ; [es:bx+trans]
12012     cld
12013           ; 17/10/2022
12014     call    near [cs:si+1]
12015     ;call   word ptr cs:si+1
12016     jb     short already_got_ah_status
12017     mov     ah, 1
12018 already_got_ah_status:
12019     ;mov    ds, [cs:0030h] ; 15/10/2022
12020     ;mov    ds, word ptr cs:BIOSDATAWORD ; [cs:0030h]
12021           ; cas note: shouldn't be needed!
12022     mov     ds, [cs:BIOSDATAWORD] ; 17/10/2022
12023     lds     bx, ds:ptrsav
12024     lds     bx, [ptrsav]
12025     mov     [bx+3], ax ; [bx+status]
12026           ; mark operation complete
12027     pop     bx
12028     pop     es
12029     pop     ds
12030     pop     bp
12031     pop     di

```

```

12031 000000C4 5A          pop     dx
12032 000000C5 59          pop     cx
12033 000000C6 58          pop     ax
12034 000000C7 5E          pop     si
12035          ;add     sp, 2          ; get rid of fake return address
12036          ; 20/09/2023 (PCDOS 7.1 - IBMBIO.COM - BIOSCODE:00C8h)
12037 000000C8 44          inc     sp
12038 000000C9 44          inc     sp
12039
12040          ; fall through into bc_retfn
12041
12042          ; -----
12043 000000CA CB          bc_retfn: retfn
12044          ; -----
12045
12046          command_error:
12047 000000CB E80700        call    bc_cmderr
12048 000000CE EBE3          jmp     short already_got_ah_status
12049          ; 15/10/2022
12050          ; 01/05/2019
12051
12052          ; -----
12053          ; The following piece of hack is for supporting CP/M compatibility
12054          ; Basically at offset 5 we have a far call into 0:c0. But this does not call
12055          ; 0:c0 directly instead it call f01d:feF0, because it needs to support 'lhld 6'
12056          ; The following hack has to reside at ffff:d0 (= f01d:feF0) if BIOS is loaded
12057          ; high.
12058          ; -----
12059
12060          ;db 7 dup(0)
12061
12062          ; 20/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
12063          ; (PCDOS 7.1 - IBMBIO.COM - BIOSCODE:0D0h)
12064          ; 15/10/2022
12065          ;dw 0          ; pad to bring offset to 0D0h
12066
12067 000000D0 00<rep 5h>    off_d0: times 5 db 0 ; 5 bytes from 0:c0 will be copied onto here
12068          ; which is the CP/M call 5 entry point
12069
12070
12071          ; -----
12072
12073          ; exit - all routines return through this path
12074
12075          bc_cmderr:
12076 000000D5 B003          mov     al, 3          ; 2C7h:D5h = 70h:2645h
12077          ; unknown command error
12078
12079          ; ===== S U B R O U T I N E =====
12080
12081          ; now zero the count field by subtracting its current value,
12082          ; which is still in cx, from itself.
12083
12084          ; subtract the number of i/o's NOT YET COMPLETED from total
12085          ; in order to return the number actually complete
12086
12087          bc_err_cnt:
12088          ;les     bx, ds:ptrsav
12089          ; 19/10/2022
12090 000000D7 C41E[1200]    les     bx, [ptrsav]
12091 000000DB 26294F12      sub     [es:bx+18], cx ; [es:bx+count]
12092          ; # of successful i/o's
12093 000000DF B481          mov     ah, 81h          ; mark error return
12094 000000E1 F9          stc          ; indicate abnormal end
12095 000000E2 C3          retfn
12096
12097          ; 15/10/2022
12098
12099          ;Bios_Code ends
12100
12101          ; -----
12102          ; MSCHAR.ASM - MSDOS 6.0 - 1991
12103          ; -----
12104          ; 15/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
12105          ; 10/01/2019 - Retro DOS v4.0
12106
12107          ; 30/04/2019
12108
12109          ;title     mschar - character and clock devices
12110
12111          ;MODE_CTRLBRK equ     0FFh
12112
12113          ; BIOSCODE:00E4h (MSDOS 6.21, IO.SYS)
12114
12115          ;*****
12116          ;*
12117          ;* device driver dispatch tables
12118          ;*
12119          ;* each table starts with a byte which lists the number of
12120          ;* legal functions, followed by that number of words. Each
12121          ;* word represents an offset of a routine in Bios_Code which
12122          ;* handles the function. The functions are terminated with
12123          ;* a near return. If carry is reset, a 'done' code is returned
12124          ;* to the caller. If carry is set, the ah/al registers are
12125          ;* returned as abnormal completion status. Notice that ds
12126          ;* is assumed to point to the Bios_Data segment throughout.
12127          ;*
12128          ;*****
12129
12130          ; 20/09/2023
12131          ; (PCDOS 7.1 - IBMBIO.COM - BIOSCODE:00E3h)
12132          ; 13/12/2022
12133 000000E3 00          db 0
12134
12135          ; 13/12/2022
12136 000000E4 0B          con_table: db ((con_table_end - con_table)-1)/2 ; 11
12137
12138 000000E5 [FA01]      dw bc_exvec ; 1FBh ; 2C7h:0E4h = 70h:2654h
12139          ; bc_exvec at 2C7h:1FBh = 70h:276Bh
12140          ; 00 init
12141 000000E7 [FA01]      dw bc_exvec ; 1FBh ; 01
12142 000000E9 [FA01]      dw bc_exvec ; 1FBh ; 02
12143 000000EB [D500]      dw bc_cmderr ; 0D5h ; bc_exvec at 2C7h:D5h = 70h:2645h
12144          ; 03
12145 000000ED [5C01]      dw con_read ; 15Ch ; con_read at 2C7h:15Ch = 70h:26CCh
12146          ; 04
12147 000000EF [9F01]      dw con_rdnd ; 19Fh ; con_rdnd at 2C7h:19Fh = 70h:270Fh
12148          ; 05
12149 000000F1 [FA01]      dw bc_exvec ; 1FBh ; 06
12150 000000F3 [0802]      dw con_flush ; 209h ; con_flush at 2C7h:209h = 70h:2779h
12151          ; 07
12152 000000F5 [FC01]      dw con_writ ; 1FDh ; con_writ at 2C7h:1FDh = 70h:276Dh
12153          ; 08
12154 000000F7 [FC01]      dw con_writ ; 1FDh ; 09
12155 000000F9 [FA01]      dw bc_exvec ; 1FBh ; 0A

```

```

12155
12156 000000FB 1A
12157
12158 000000FC [FA01]
12159 000000FE [FA01]
12160 00000100 [FA01]
12161 00000102 [D500]
12162 00000104 [1902]
12163
12164 00000106 [C701]
12165
12166 00000108 [FA01]
12167 0000010A [FA01]
12168 0000010C [1E02]
12169 0000010E [1E02]
12170 00000110 [4F02]
12171 00000112 [FA01]
12172 00000114 [FA01]
12173 00000116 [FA01]
12174 00000118 [FA01]
12175 0000011A [FA01]
12176 0000011C [9402]
12177 0000011E [FA01]
12178 00000120 [FA01]
12179 00000122 [C202]
12180 00000124 [FA01]
12181 00000126 [FA01]
12182 00000128 [FA01]
12183 0000012A [FA01]
12184 0000012C [FA01]
12185 0000012E [F702]
12186
12187 00000130 0B
12188
12189 00000131 [FA01]
12190 00000133 [FA01]
12191 00000135 [FA01]
12192 00000137 [D500]
12193 00000139 [1203]
12194 0000013B [3703]
12195 0000013D [FA01]
12196 0000013F [7803]
12197 00000141 [7F03]
12198 00000143 [7F03]
12199 00000145 [5703]
12200
12201 00000147 0A
12202
12203 00000148 [FA01]
12204 0000014A [FA01]
12205 0000014C [FA01]
12206 0000014E [D500]
12207 00000150 [E404]
12208 00000152 [C701]
12209 00000154 [FA01]
12210 00000156 [FA01]
12211 00000158 [E503]
12212 0000015A [E503]
12213
12214
12215
12216
12217
12218
12219
12220
12221
12222
12223
12224
12225 0000015C E306
12226
12227 0000015E E80500
12228 00000161 AA
12229 00000162 E2FA
12230
12231 00000164 F8
12232 00000165 C3
12233
12234
12235
12236
12237
12238
12239
12240
12241
12242
12243
12244
12245
12246
12247
12248
12249
12250
12251
12252
12253
12254
12255
12256
12257
12258
12259
12260
12261
12262
12263
12264 00000166 8A26[7E04]
12265 0000016A 30C0
12266 0000016C 8606[0C00]
12267 00000170 08C0
12268 00000172 752A
12269 00000174 CD16
12270 00000176 09C0
12271 00000178 74EC
12272 0000017A 3D0072
12273 0000017D 7504
12274 0000017F B010
12275 00000181 EB1B
12276
12277
12278

con_table_end:
prn_table: db ((prn_table_end - prn_table)-1)/2 ; 26
            ; 2C7h:0FBh = 70h:266Bh
            dw bc_exvec ; 1FBh ; bc_exvec
            dw bc_exvec ; 1FBh ; 01
            dw bc_exvec ; 1FBh ; 02
            dw bc_cmderr ; 0D5h ; bc_cmderr
            dw prn_input ; 21Ah ; prn_input
            ; 04 indicate zero chars read
            dw z_bus_exit ; 1C8h ; z_bus_exit
            ; 05 read non-destructive
            dw bc_exvec ; 1FBh ; 06
            dw bc_exvec ; 1FBh ; 07
            dw prn_writ ; 21Fh ; prn_writ
            dw prn_writ ; 21Fh ; 09
            dw prn_stat ; 251h ; prn_stat
            dw bc_exvec ; 1FBh ; 0B
            dw bc_exvec ; 1FBh ; 0C
            dw bc_exvec ; 1FBh ; 0D
            dw bc_exvec ; 1FBh ; 0E
            dw bc_exvec ; 1FBh ; 0F
            dw prn_tilbusy ; 28Bh ; prn_tilbusy
            dw bc_exvec ; 1FBh ; 11
            dw bc_exvec ; 1FBh ; 12
            dw prn_genioct1 ; 2BAh ; prn_genioct1
            dw bc_exvec ; 1FBh ; 14
            dw bc_exvec ; 1FBh ; 15
            dw bc_exvec ; 1FBh ; 16
            dw bc_exvec ; 1FBh ; 17
            dw bc_exvec ; 1FBh ; 18
            dw prn_ioctl_query ; 2F0h ; prn_ioctl_query
prn_table_end:
aux_table: db ((aux_table_end - aux_table)-1)/2 ; 11
            ; 2C7h:130h = 70h:26A0h
            dw bc_exvec ; 1FBh ; 00 - init
            dw bc_exvec ; 1FBh ; 01
            dw bc_exvec ; 1FBh ; 02
            dw bc_cmderr ; 0D5h ; 03
            dw aux_read ; 30Dh ; aux_read ; 04 - read
            dw aux_rdnd ; 335h ; aux_rdnd - 05 - read non-destructive
            dw bc_exvec ; 1FBh ; 06
            dw aux_flsh ; 36Ch ; aux_flsh
            dw aux_writ ; 374h ; aux_writ
            dw aux_writ ; 374h ; 09
            dw aux_wrst ; 355h ; aux_wrst
aux_table_end:
tim_table: db ((tim_table_end - tim_table)-1)/2 ; 10
            ; 2C7h:147h = 70h:26B7h
            dw bc_exvec ; 1FBh ; 00
            dw bc_exvec ; 1FBh ; 01
            dw bc_exvec ; 1FBh ; 02
            dw bc_cmderr ; 0D5h ; 03
            dw tim_read ; 435h ; tim_read
            dw z_bus_exit ; 1C8h ; z_bus_exit
            dw bc_exvec ; 1FBh ; 06
            dw bc_exvec ; 1FBh ; 07
            dw tim_writ ; 3DBh ; tim_writ
            dw tim_writ ; 3DBh ; 09
tim_table_end:
; -----
; *****
; *
; * con_read - read cx bytes from keyboard into buffer at es:di *
; *
; *****
con_read: ; 2C7h:15Ch = 70h:26CCh
        jcxz short con_exit ; read cx bytes from keyboard into buffer
        jcxz con_exit ; 19/10/2022
con_loop: call chrin ; get char in al
        stosb ; store char at es:di
        loop con_loop
con_exit: cll
        retn
; ===== S U B R O U T I N E =====
; *****
; *
; * chrin - input single char from keyboard into al *
; *
; * we are going to issue extended keyboard function, if *
; * supported. the returning value of the extended keystroke *
; * of the extended keyboard function uses 0E0h in al *
; * instead of 00h as in the conventional keyboard function. *
; * this creates a conflict when the user entered real *
; * greek alpha charater (= 0E0h) to distinguish the extended *
; * keystroke and the greek alpha. this case will be handled *
; * in the following manner: *
; *
; * ah = 16h *
; * int 16h *
; * if al == 0, then extended code (in ah) *
; * else if al == 0E0h, then *
; * if ah <> 0, then extended code (in ah) *
; * else greek_alpha character. *
; *
; * also, for compatibility reason, if an extended code is *
; * detected, then we are going to change the value in al *
; * from 0E0h to 00h. *
; *
; *****
; 19/10/2022
chrin: mov ah, [keyrd_func] ; set by msinit. 0 or 10h
        xor al, al
        xchg al, [altah] ; get character & zero altah
        or al, al
        jnz short keyret
        int 16h ; KEYBOARD -
        or ax, ax
        jz short chrin
        cmp ax, 7200h ; check for ctrl-prtsc
        jnz short alt_ext_chk
        mov al, 10h
        jmp short keyret
; -----
; if operation was extended function (i.e. keyrd_func != 0) then

```

```

12279 ; if character read was 0E0h then
12280 ; if extended byte was zero (i.e. ah == 0) then
12281 ; goto keyret
12282 ; else
12283 ; set al to zero
12284 ; goto alt_save
12285 ; endif
12286 ; endif
12287 ; endif
12288
12289 alt_ext_chk:
12290 00000183 803E[7E04]00 cmp byte [keyrd_func], 0
12291 00000188 740C jz short not_ext
12292 0000018A 3CE0 cmp al, 0E0h
12293 0000018C 7508 jnz short not_ext
12294 0000018E 08E4 or ah, ah
12295 00000190 740C jz short keyret
12296 00000192 30C0 xor al, al
12297 00000194 EB04 jmp short alt_save
12298 ; -----
12299
12300 not_ext:
12301 00000196 08C0 or al, al ; special case?
12302 00000198 7504 jnz short keyret
12303
12304 0000019A 8826[0C00] alt_save: mov [altah], ah ; store special key
12305
12306 0000019E C3 keyret: retn
12307 ; -----
12308
12309 ; *****
12310 ; * con_rdnd - keyboard non destructive read, no wait *
12311 ; * *
12312 ; * pc-convertible-type machine: if bit 10 is set by the dos *
12313 ; * in the status word of the request packet, and there is no *
12314 ; * character in the input buffer, the driver issues a system *
12315 ; * wait request to the rom. on return from the rom, it returns *
12316 ; * a 'char-not-found' to the dos. *
12317 ; * *
12318 ; *****
12319
12320 ; 19/10/2022
12321
12322 con_rdnd:
12323 mov al, [altah]
12324 0000019F A0[0C00] or al, al
12325 000001A2 08C0 jnz short rdexit
12326 000001A4 754C mov ah, [keysts_func]
12327 000001A6 8A26[7F04] int 16h ; KEYBOARD -
12328 000001AA CD16 jnz short gotchr
12329 000001AC 751D cmp byte [fhavok09], 0
12330 000001AE 803E[7900]00 jz short z_bus_exit
12331 000001B3 7412 les bx, [ptrsav]
12332 000001B5 C41E[1200] ; 12/12/2022
12333 test byte [es:bx+4], 04h
12334 000001B9 26F6470404 ; test word [es:bx+3], 400h ; [es:bx+status]
12335 jnz short z_bus_exit
12336 000001BE 7407 mov ax, 4100h
12337 000001C0 B80041 xor bl, bl
12338 000001C3 30DB int 15h ; SYSTEM - WAIT ON EXTERNAL EVENT (CONVERTIBLE)
12339 000001C5 CD15 ; AL = condition type, BH = condition compare or mask value
12340 ; BL = timeout value times 55 milliseconds, 00h means no timeout
12341 ; DX = I/O port address if AL bit 4 set
12342
12343 z_bus_exit:
12344 000001C7 F9 stc
12345 000001C8 B403 mov ah, 3
12346 000001CA C3 ; indicate busy status
12347 ; -----
12348
12349 gotchr:
12350 000001CB 09C0 or ax, ax
12351 000001CD 7508 jnz short notbrk ; check for null after break
12352 000001CF 8A26[7E04] mov ah, [keyrd_func] ; issue keyboard read function
12353 000001D3 CD16 int 16h ; KEYBOARD -
12354 000001D5 EBC8 jmp short con_rdnd ; get a real status
12355 ; -----
12356
12357 notbrk:
12358 000001D7 3D0072 cmp ax, 7200h ; check for ctrl-prtsc
12359 000001DA 7504 jnz short rd_ext_chk
12360 000001DC B010 mov al, 10h ; ('P' & 1Fh) ; return control p
12361 000001DE EB12 jmp short rdexit
12362 ; -----
12363
12364 rd_ext_chk:
12365 000001E0 803E[7E04]00 cmp byte [keyrd_func], 0 ; extended keyboard function?
12366 000001E5 740B jz short rdexit
12367 000001E7 3CE0 cmp al, 0E0h ; extended key value or greek alpha?
12368 000001E9 7507 jnz short rdexit
12369 000001EB 80FC00 cmp ah, 0 ; scan code exist?
12370 000001EE 7402 jz short rdexit ; yes. greek alpha char.
12371 000001F0 B000 mov al, 0 ; no. extended key stroke.
12372 ; change it for compatibility
12373
12374 000001F2 C41E[1200] rdexit: les bx, [ptrsav]
12375 000001F6 2688470D mov [es:bx+13], al ; [es:bx+media]
12376 ; return keyboard character here
12377
12378 000001FA F8 bc_exvec: cll ; bc_exvec at 2C7h:1FBh = 70h:276Bh
12379 ; indicate normal termination
12380 000001FB C3 retn
12381 ; -----
12382
12383 ; *****
12384 ; * con_write - console write routine *
12385 ; * *
12386 ; * entry: es:di -> buffer *
12387 ; * cx = count *
12388 ; * *
12389 ; *****
12390
12391 con_writ:
12392 ; jcxz short bc_exvec
12393 jcxz bc_exvec ; 19/10/2022
12394 000001FC E3FC ; 12/12/2022
12395 ; jcxz cc_ret
12396
12397 con_lp:
12398 000001FE 268A05 mov al, [es:di]
12399 00000201 47 inc di
12400 00000202 CD29 int 29h ; DOS 2+ internal - FAST PUTCHAR
12401 ; AL = character to display
12402 00000204 E2F8 loop con_lp

```



```

12403
12404 00000206 F8
12405 00000207 C3
12406
12407
12408
12409
12410
12411
12412
12413
12414
12415
12416 00000208 C606[0C00]00
12417
12418 0000020D B401
12419 0000020F CD16
12420
12421
12422
12423 00000211 74F3
12424 00000213 30E4
12425 00000215 CD16
12426
12427 00000217 EBF4
12428
12429
12430
12431
12432
12433
12434
12435
12436
12437
12438
12439
12440
12441
12442
12443
12444
12445
12446
12447
12448
12449
12450
12451
12452
12453
12454
12455
12456
12457
12458
12459
12460
12461
12462
12463
12464
12465 00000219 E8BBFE
12466
12467
12468
12469 0000021C F8
12470 0000021D C3
12471
12472
12473
12474
12475
12476
12477
12478
12479
12480
12481
12482
12483 0000021E E3FC
12484
12485 00000220 BB0200
12486
12487 00000223 E83600
12488 00000226 751D
12489 00000228 268A05
12490 0000022B 30E4
12491 0000022D E82E00
12492 00000230 7419
12493 00000232 80FCFF
12494 00000235 7509
12495 00000237 B00C
12496 00000239 C606[0C00]00
12497 0000023E EB08
12498
12499
12500
12501 00000240 F6C401
12502 00000243 7406
12503
12504 00000245 4B
12505 00000246 75DB
12506
12507 00000248 E98CFE
12508
12509
12510
12511 0000024B 47
12512 0000024C E2D2
12513
12514
12515
12516
12517
12518 0000024E C3
12519
12520
12521
12522
12523
12524
12525
12526

cc_ret:
    clc
    retn

; ===== S U B   R O U T I N E =====
; *****
; *
; * con_flush - flush out keyboard queue
; *
; *
; *****
con_flush:
    mov     byte [altah], 0        ; clear out holding buffer
floop:     ; while(charavail()) charread();
    mov     ah, 1
    int     16h                   ; KEYBOARD - CHECK BUFFER, DO NOT CLEAR
    ; Return: ZF clear if character in buffer
    ; AH = scan code, AL = character
    ; ZF set if no character in buffer
    jz      short cc_ret
    xor     ah, ah
    int     16h                   ; KEYBOARD - READ CHAR FROM BUFFER, WAIT IF EMPTY
    ; Return: AH = scan code, AL = character
    jmp     short floop

; -----
; 15/10/2022
; *****
; *
; * some equates for rom bios printer i/o
; *
; *
; *****
; ibm rom status bits (i don't trust them, neither should you)
; warning!!! the ibm rom does not return just one bit. it returns a
; whole slew of bits, only one of which is correct.
;notbusystatus equ 10000000b      ; not busy
;nopaperstatus  equ 00100000b      ; no more paper
;prnselected    equ 00010000b      ; printer selected
;ioerrstatus    equ 00001000b      ; some kinda error
;timeoutstatus  equ 00000001b      ; time out.
;
;noprnter       equ 00110000b      ; no printer attached

; 18/03/2019 - Retro DOS v4.0
;error_I24_out_of_paper equ 9 ; MSDOS 6.0, ERR.INC, 1991

; -----
; *****
; *
; * prn_input - return with no error but zero chars read
; *
; *
; * enter with cx = number of characters requested
; *
; *
; *****
prn_input:
    call    bc_err_cnt            ; 2C7h:21Ah = 70h:278Ah
    ; reset count to zero
    ; (sub reqpkt.count,cx)
    ; 12/12/2022
prn_done:
    clc
    retn                          ; but return with carry      reset for no error

; -----
; *****
; *
; * prn_writ - write cx bytes from es:di to printer device
; *
; *
; * auxnum has printer number
; *
; *
; *****
prn_writ:
    jcxz    short prn_done        ; 2C7h:21Fh = 70h:278Fh
    jcxz    short prn_done        ; no chars to output
    ; 19/10/2022
prn_loop:
    mov     bx, 2                 ; retry count
prn_out:
    call    prnstat               ; get status
    jnz     short TestPrnError    ; short TestPrnError
    mov     al, [es:di]           ; get character      to print
    xor     ah, ah
    call    prnop                 ; print to printer
    jz      short prn_con         ; no error - continue
    cmp     ah, 0FFh              ; MODE_CTRLBRK
    jnz     short _prnwf          ;
    mov     al, 0Ch               ; error_I24_gen_failure
    mov     byte [altah], 0
    jmp     short pmessg

; -----
_prnwf:
    test    ah, 1                 ; timeoutstatus
    jz      short prn_con
TestPrnError:
    dec     bx                    ; retry until count is exhausted.
    jnz     short prn_out
pmessg:
    jmp     bc_err_cnt

; -----
prn_con:
    inc     di                    ; point to next char and continue
    loop    prn_loop
;prn_done:
; 12/12/2022
prn_done2:
    clc
    ; cf=0
    retn

; -----
; *****
; *
; * prn_stat - device driver entry to return printer status
; *
; *
; *****

```

```

12527 prn_stat:      call    prnstat      ; 2C7h:251h = 70h:27C1h
12528 0000024F E80A00      jnz      short pmsgg      ; device in dx
12529 00000252 75F4      test     ah, 80h      ; notbusystatus
12530 00000254 F6C480      ;jnz     short prn_done
12531      ; 12/12/2022
12532      jnz      short prn_done2 ; cf=0
12533 00000257 75F5      jmp      z_bus_exit
12534 00000259 E96BFF
12535
12536 ; -----
12537 ;*****
12538 ;*
12539 ;* prnstat - utility function to call ROM BIOS to check
12540 ;* printer status. Return meaningful error code
12541 ;*
12542 ;*****
12543
12544 prnstat:      mov     ah, 2      ; set command for get status
12545 0000025C B402      ; PRINTER - GET STATUS
12546      ; DX = printer port (0-3)
12547      ; Return: AH = status
12548
12549 ; ===== S U B   R O U T I N E =====
12550
12551 ;*****
12552 ;*
12553 ;* prnop - call ROM BIOS printer function in ah
12554 ;* return zero true if no error
12555 ;* return zero false if error, al = error code
12556 ;*
12557 ;*****
12558
12559 prnop:      ; 20/12/2023 - Retro DOS v5.0
12560      ; PCDOS 7.1 IBMBIO.COM
12561
12562      ;mov    dx, [auxnum] ; get printer number
12563      ;int    17h
12564
12565      push    ds
12566      push    word [auxnum]
12567 0000025E 1E      xor     dx, dx ; 0
12568 0000025F FF36[2100]  mov     ds, dx
12569 00000263 31D2      pop     dx
12570 00000265 8EDA      pushf   ; simulate int 17h
12571 00000267 5A      cli
12572 00000268 9C      ;call    dword ptr ds:5ghghCh
12573 00000269 FA      call    far [17h*4] ; 0:5Ch = INT 17h vector
12574      pop     ds
12575 0000026A FF1E5C00
12576 0000026E 1F
12577
12578      ; This check was added to see if this is a case of no
12579      ; printer being installed. This tests checks to be sure
12580      ; the error is noprinter (30h)
12581
12582      push    ax
12583 00000270 80E430      and     ah, 30h
12584 00000273 80FC30      cmp     ah, 30h ; noprinter
12585 00000276 58      pop     ax
12586 00000277 7506      jnz     short NextTest
12587 00000279 80E4DF      and     ah, 0DFh ; ~nopaperstatus
12588 0000027C 80CC08      or      ah, 8 ; ioerrstatus
12589
12590      ; examine the status bits to see if an error occurred. unfortunately, several
12591      ; of the bits are set so we have to pick and choose. we must be extremely
12592      ; careful about breaking basic.
12593
12594 NextTest:
12595 0000027F F6C428      test     ah, 28h ; (ioerrstatus+nopaperstatus)
12596      ; i/o error?
12597 00000282 740A      jz      short checknotready ; no, try not ready
12598
12599      ; at this point, we know we have an error. the converse is not true
12600
12601      mov     al, 9 ; error_I24_out_of_paper
12602      ; first, assume out of paper
12603 00000286 F6C420      test     ah, 20h ; out of paper set?
12604 00000289 7502      jnz     short ret1 ; yes, error is set
12605 0000028B FEC0      inc     al ; return al=10 (i/o error)
12606
12607 ret1:      retn
12608
12609 ; -----
12610
12611 checknotready:
12612 0000028E B002      mov     al, 2 ; assume not-ready
12613 00000290 F6C401      test     ah, 1
12614      retn
12615
12616 ; -----
12617 ;*****
12618 ;*
12619 ;* prn_tilbusy - output until busy. Used by print spooler.
12620 ;* this entry point should never block waiting for
12621 ;* device to come ready.
12622 ;*
12623 ;* inputs: cx = count, es:di -> buffer
12624 ;* outputs: set the number of bytes transferred in the
12625 ;* device driver request packet
12626 ;*
12627 ;*****
12628
12629 ; 19/10/2022
12630 prn_tilbusy:      ; 2C7h:28Bh = 70h:27FBh
12631 00000294 89FE      mov     si, di ; everything is set for lodsb
12632
12633 prn_tilbloop:
12634      push    cx
12635      push    bx
12636 00000296 51      xor     bh, bh
12637 00000297 53      mov     bl, [printdev]
12638 00000298 30FF      shl     bx, 1
12639 0000029A 8A1E[8004]  ;mov     cx, ds:wait_count[bx] ; wait count times to come ready
12640 0000029E D1E3      mov     cx, [wait_count+bx]
12641      pop     bx
12642 prn_getstat:
12643      call    prnstat ; get status
12644 000002A5 E8B4FF      jnz     short prn_bperr ; error
12645 000002A8 7514      test     ah, 80h ; ready yet?
12646 000002AA F6C480      loope   prn_getstat ; no, go for more
12647 000002AD E1F6      pop     cx ; get original count
12648 000002AF 59      jz      short prn_berr ; still not ready => done
12649 000002B0 740D      es
12650 000002B2 26      lodsb
12651 000002B3 AC      ;lods  byte ptr es:[si] ; es

```



```

12651                                     ; lodsb
12652 000002B4 30E4                xor     ah, ah
12653 000002B6 E8A5FF             call    prnop
12654 000002B9 7504                jnz     short prn_berr ; error
12655 000002BB E2D9                loop    prn_tilbloop
12656                                     ; 12/12/2022
12657                                     ; cf=0 (prnop)
12658                                     ; clc
12659 000002BD C3                   retn     ; normal no-error return
12660                                     ; from device driver
12661 ; -----
12662
12663 prn_bperr:
12664 000002BE 59                   pop     cx          ; restore transfer count from stack
12665 prn_berr:
12666 000002BF E915FE              jmp     bc_err_cnt
12667 ; -----
12668
12669 ; 15/10/2022
12670
12671 ;*****
12672 ;*
12673 ;* prn_genioctl - get/set printer retry count
12674 ;*
12675 ;*****
12676
12677 ; IOCTL.INC (MSDOS 6.0, 1991)
12678 ; 11/01/2019
12679
12680 ;*****;*
12681 ; CHARACTER DEVICES (PRINTERS)
12682 ;*****;*
12683
12684 ;;RAWIO SUB-FUNCTIONS
12685 ;;get_retry_count equ 65h
12686 ;;set_retry_count equ 45h
12687
12688 ;;struc A_RETRYCOUNT
12689 ;;.rc_count: resw 1
12690 ;;endstruc
12691
12692 ;ioc_pc equ 5
12693
12694 ; -----
12695
12696 ; 19/10/2022
12697 prn_genioctl:
12698 000002C2 C43E[1200]          les     di, [ptrsav]          ; 2C7h:2BAh = 70h:282Ah
12699 000002C6 26807D0D05         cmp     byte [es:di+13], 5 ; [es:di+IOCTL_REQ.MAJORFUNCTION]
12700                                     ; ioc_pc
12701 000002CB 7403                jz      short prnfunc_ok
12702
12703 prnfuncerr:
12704 000002CD E905FE              jmp     bc_cmderr
12705 ; -----
12706
12707 prnfunc_ok:
12708 000002D0 268A450E           mov     al, [es:di+14] ; [es:di+IOCTL_REQ.MINORFUNCTION]
12709 000002D4 26C47D13           les     di, [es:di+19] ; [es:di+IOCTL_REQ.GENERICIOCTL_PACKET]
12710 000002D8 30FF              xor     bh, bh
12711                                     ; mov     bl, ds:printdev ; get index into retry counts
12712 000002DA 8A1E[8004]         mov     bl, [printdev]
12713 000002DE D1E3              shl     bx, 1
12714                                     ; mov     cx, ds:wait_count[bx] ; pull out retry count for device
12715 000002E0 8B8F[8104]         mov     cx, [wait_count+bx]
12716 000002E4 3C65              cmp     al, 65h ; get_retry_count
12717 000002E6 7407              jz      short prngetcount
12718 000002E8 3C45              cmp     al, 45h ; set_retry_count
12719 000002EA 75E1              jnz     short prnfuncerr
12720 000002EC 268B0D           mov     cx, [es:di]
12721
12722 prngetcount:
12723 000002EF 898F[8104]         mov     ds:wait_count[bx], cx
12724 000002F3 26890D           mov     [wait_count+bx], cx
12725                                     mov     [es:di], cx ; [es:di+A_RETRYCOUNT.RC_COUNT]
12726                                     ; return current retry count
12727                                     ; 12/12/2022
12728                                     ; cf=0
12729 000002F6 C3                   retn
12730 ; -----
12731
12732 ;*****
12733 ;*
12734 ;* prn_ioctl_query
12735 ;*
12736 ;* Added for 5.00
12737 ;*****
12738
12739 prn_ioctl_query:
12740 000002F7 C43E[1200]          les     di, [ptrsav]          ; 2C7h:2F0h = 70h:2860h
12741 000002FB 26807D0D05         cmp     byte [es:di+13], 5 ; [es:di+IOCTL_REQ.MAJORFUNCTION]
12742                                     ; ioc_pc
12743 00000300 750D              jnz     short prn_query_err
12744 00000302 268A450E           mov     di, [es:di+14] ; [es:di+IOCTL_REQ.MINORFUNCTION]
12745 00000306 3C65              cmp     al, 65h ; GET_RETRY_COUNT
12746 00000308 7404              jz      short IoctlSupported
12747 0000030A 3C45              cmp     al, 45h ; SET_RETRY_COUNT
12748 0000030C 7501              jnz     short prn_query_err
12749
12750 IoctlSupported:
12751                                     ; 12/12/2022
12752                                     ; cf=0
12753 0000030E C3                   retn
12754 ; -----
12755
12756 prn_query_err:
12757                                     ; 12/12/2022
12758                                     ; stc
12759 0000030F E9C3FD              jmp     bc_cmderr ; (bc_cmderr sets cf to 1)
12760 ; -----
12761
12762 ;*****
12763 ;*
12764 ;* aux port driver code -- "aux" == "com1"
12765 ;*
12766 ;* the device driver entry/dispatch code sets up auxnum to
12767 ;* give the com port number to use (0=com1, 1=com2, 2=com3...)
12768 ;*
12769 ;*****
12770
12771 ; values in ah, requesting function of int 14h in rom bios
12772
12773 ;auxfunc_send equ 1 ;transmit
12774 ;auxfunc_receive equ 2 ;read

```

```

12775 ;auxfunc_status equ 3 ;request status
12776
12777 ; error flags, reported by int 14h, reported in ah:
12778
12779 ;flag_data_ready equ 01h ;data ready
12780 ;flag_overrun equ 02h ;overrun error
12781 ;flag_parity equ 04h ;parity error
12782 ;flag_frame equ 08h ;framing error
12783 ;flag_break equ 10h ;break detect
12784 ;flag_tranhol_emp equ 20h ;transmit holding register empty
12785 ;flag_timeout equ 80h ;timeout
12786
12787 ; these flags reported in al:
12788
12789 ;flag_cts equ 10h ;clear to send
12790 ;flag_dsr equ 20h ;data set ready
12791 ;flag_rec_sig equ 80h ;receive line signal detect
12792
12793 ; -----
12794 ;*****
12795 ;*
12796 ;*
12797 ;* aux_read - read cx bytes from [auxnum] aux port to buffer *
12798 ;* at es:di *
12799 ;*
12800 ;*****
12801
12802 aux_read: ; 2C7h:30Dh = 70h:287Dh
12803 ;jcxz short exvec2
12804 ;jcxz exvec2 ; 19/10/2022
12805 call getbx ; put address of auxbuf in bx
12806 xor al, al
12807 xchg al, [bx]
12808 or al, al
12809 jnz short aux2
12810
12811 aux1: call auxin ; get character from port
12812 ; won't return if error
12813
12814 aux2: stosb
12815 loop aux1 ; if more characters, go around again
12816
12817 exvec2: cld ; all done, successful exit
12818
12819 auxin_retn: ; 18/12/2022
12820 retn
12821 ; -----
12822 ;*****
12823 ;*
12824 ;* auxin - call rom bios to read character from aux port *
12825 ;* if error occurs, map the error and return one *
12826 ;* level up to device driver exit code, setting *
12827 ;* the number of bytes transferred appropriately *
12828 ;*
12829 ;*****
12830
12831 auxin: mov ah, 2 ; auxfunc_receive
12832 call auxop
12833 test ah, 0Eh ; flag_frame|flag_parity|flag_overrun
12834 ;jnz short arbad ; skip if any error bits set
12835 ;retn
12836 ; 25/06/2023 (BugFix)
12837 jz short auxin_retn
12838 ; -----
12839
12840 arbad: pop ax ; remove return address (near call)
12841 ;xor al, al
12842 ;or al, 0B0h ; flag_rec_sig|flag_dsr|flag_cts
12843 ; 11/08/2023
12844 mov al, 0B0h ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0334h
12845 jmp bc_err_cnt
12846 ; -----
12847 ;*****
12848 ;*
12849 ;* aux_rdnd - non-destructive aux port read *
12850 ;*
12851 ;*****
12852
12853 aux_rdnd: ; 2C7h:335h = 70h:28A5h
12854 call getbx ; have bx point to auxbuf
12855 mov al, [bx] ; if al is non-zero (char in buffer)
12856 or al, al
12857 jnz short auxdrx ; then return character
12858 call auxstat ; if not, get status of aux device
12859 test ah, 1 ; flag_data_ready - test data ready
12860 jz short auxbus ; then device is busy (not ready)
12861 test al, 20h ; flag_dsr - test data set ready
12862 jz short auxbus ; then device is busy (not ready)
12863 call auxin ; else aux is ready, get character
12864 mov [bx], al
12865
12866 auxdrx: jmp rdexit ; return busy status
12867 ; -----
12868
12869 auxbus: jmp z_bus_exit
12870 ; -----
12871 ;*****
12872 ;*
12873 ;* aux_wrst - return aux port write status *
12874 ;*
12875 ;*****
12876
12877 aux_wrst: ; 2C7h:355h = 70h:28C5h
12878 call auxstat ; get status of aux in ax
12879 test al, 20h ; test data set ready
12880 jz short auxbus ; then device is busy (not ready)
12881 test ah, 20h ; flag_tranhol_emp - test transmit hold reg empty
12882 jz short auxbus ; then device is busy (not ready)
12883 ; 12/12/2022
12884 ; cf=0 ; (test instruction resets cf)
12885 cld
12886 retn
12887 ; -----
12888 ;*****
12889 ;*
12890 ;* auxstat - call rom bios to determine aux port status *
12891 ;*
12892 ;*****

```

```

12899 ;* exit: ax = status ;
12900 ;* dx = [auxnum] ;
12901 ;* ;
12902 ;*****
12903
12904 auxstat:
12905 00000364 B403 mov ah, 3 ; auxfunc_status
12906 ; fall into auxop
12907
12908 ; ===== S U B R O U T I N E =====
12909
12910 ;*****
12911 ;*
12912 ;* auxop - perform rom-biox aux port interrupt ;
12913 ;* ;
12914 ;* entry: ah = int 14h function number ;
12915 ;* exit: ax = results ;
12916 ;* dx = [auxnum] ;
12917 ;* ;
12918 ;*****
12919
12920 auxop:
12921 ; proc near
12922 ; 20/12/2023 - Retro DOS v5.0
12923 ; mov dx, [auxnum] ; ah=function code
12924 ; ; 0=init, 1=send, 2=receive, 3=status
12925 ; ; get port number
12926 ;
12927 ; int 14h ; SERIAL I/O - GET USART STATUS
12928 ; ; DX = port number (0-3)
12929 ; ; Return: AX = port status code
12930 ; (PCDOS 7.1 IBMBIO.COM)
12931 00000366 1E push ds
12932 00000367 FF36[2100] push word [auxnum]
12933 00000368 31D2 xor dx, dx ; 0
12934 0000036D 8EDA mov ds, dx
12935 0000036F 5A pop dx
12936 00000370 9C pushf ; simulate INT 14h
12937 00000371 FA cli
12938 ; call dword ptr ds:50h
12939 00000372 FF1E5000 call far [14h*4] ; INT 14h vector (14h*4 = 50h)
12940 00000376 1F pop ds
12941 00000377 C3 retn
12942
12943 ; -----
12944 ;*****
12945 ;*
12946 ;* aux_flnh - flush aux input buffer - set contents of ;
12947 ;* auxbuf [auxnum] to zero ;
12948 ;* ;
12949 ;* cas - shouldn't this code call the rom bios input function ;
12950 ;* repeatedly until it isn't ready? to flush out any ;
12951 ;* pending serial input queue if there's a tsr like MODE *
12952 ;* which is providing interrupt-buffering of aux port? *
12953 ;* ;
12954 ;* ;
12955 ;*****
12956
12957 aux_flnh:
12958 00000378 E81C00 call getbx ; 2c7h:36ch = 70h:28pch
12959 0000037B C60700 mov byte [bx], 0 ; flush aux input buffer
12960 ; ; get bx to point to auxbuf
12961 ; ; zero out buffer
12962 ; ; all done, successful return
12963 ; 12/12/2022
12964 0000037E C3 ; cf=0 ('add' instruction in 'getbx')
12965 retn
12966 ; -----
12967 ;*****
12968 ;*
12969 ;* aux_writ - write to aux device ;
12970 ;* ;
12971 ;*****
12972
12973 aux_writ:
12974 ; jcxz short exvec2 ; 2c7h:374h = 70h:28E4h
12975 0000037F E3A4 jcxz exvec2 ; write to aux device (if cx > 0)
12976 ; 19/10/2022
12977 00000381 268A05 mov al, [es:di] ; get character to be written
12978 ; ; move di pointer to next character
12979 00000384 47 inc di
12980 00000385 B401 mov ah, 1 ; auxfunc_send - indicates a write
12981 00000387 E8DCFF call auxop ; send character over aux port
12982 0000038A F6C480 test ah, 80h ; check for error
12983 0000038D 7405 jz short awok ; then no error
12984 0000038F B00A mov al, 10 ; else indicate write fault
12985 00000391 E943FD jmp bc_err_cnt ; call error routines
12986 ; -----
12987
12988 awok:
12989 00000394 E2EB loop aux_loop ; if cx is non-zero,
12990 ; ; still more character to print
12991 ; ; all done, successful return
12992 ; 12/12/2022
12993 ; cf=0 (test instruction above)
12994 00000396 C3 retn
12995
12996 ; ===== S U B R O U T I N E =====
12997
12998 ;*****
12999 ;*
13000 ;* getbx - return bx -> single byte input buffer for ;
13001 ;* selected aux port ([auxnum]) ;
13002 ;* ;
13003 ;*****
13004
13005 getbx:
13006 00000397 8B1E[2100] mov bx, [auxnum] ; return bx -> single byte input buffer
13007 ; ; for selected aux port ([auxnum])
13008 ; add bx, offset auxbuf
13009 0000039B 81C3[1600] add bx, auxbuf ; 19/10/2022
13010 ; 12/12/2022
13011 ; cf=0 (if [auxnum] is valid number)
13012 0000039F C3 retn
13013
13014 ; -----
13015
13016 ; 15/10/2022
13017
13018 ; -----
13019 ;
13020 ; clock device driver ;
13021 ; ;
13022 ;

```

```

13023 ; this file contains the clock device driver. ;
13024 ; ;
13025 ; the routines in this files are: ;
13026 ; routine function ;
13027 ; ----- ;
13028 ; tim_writ set the current time ;
13029 ; tim_read read the current time ;
13030 ; time_to_ticks convert time to corresponding ;
13031 ; number of clock ticks ;
13032 ;
13033 ; the clock ticks at the rate of: ;
13034 ; ;
13035 ; 1193180/65536 ticks/second (about 18.2 ticks per second): ;
13036 ; see each routine for information on the use. ;
13037 ; ;
13038 ; ----- ;
13039 ;
13040 ; convert time to ticks
13041 ; input : time in cx and dx
13042 ; ticks returned in cx:dx
13043
13044 ;19/07/2019
13045 ;09/03/2019
13046
13047 time_to_ticks: ; 0070h:2906h = 02C7h:0396h
13048
13049 ; first convert from hour,min,sec,hund. to
13050 ; total number of 100th of seconds
13051
13052 mov al, 60
13053 mul ch ; hours to minutes
13054 mov ch, 0
13055 add ax, cx ; total minutes
13056 mov cx, 6000 ; 60*100
13057 mov bx, dx ; get out of the way of the multiply
13058 mul cx ; convert to 1/100 sec
13059 mov cx, ax
13060 mov al, 100
13061 mul bh ; convert seconds to 1/100 sec
13062 add cx, ax ; combine seconds with hours and min
13063 adc dx, 0 ; ripple carry
13064 mov bh, 0
13065 add cx, bx ; combine 1/100 sec
13066 adc dx, 0
13067 ; dx:cx is time in 1/100 sec
13068
13069 xchg ax, dx
13070 xchg ax, cx ; now time is in cx:ax
13071 mov bx, 59659
13072 mul bx ; multiply low half
13073 xchg dx, cx
13074 xchg ax, dx ; cx->ax, ax->dx, dx->cx
13075 mul bx ; multiply high half
13076 add ax, cx ; combine overlapping products
13077 adc dx, 0
13078 xchg ax, dx ; ax:dx=time*59659
13079 mov bx, 5
13080 div bl ; divide high half by 5
13081 mov cl, al
13082 mov ch, 0
13083 mov al, ah ; remainder of divide-by-5
13084 cbw
13085 xchg ax, dx ; use it to extend low half
13086 div bx ; divide low half by 5
13087 mov dx, ax ; cx:dx is now number of ticks in time
13088 retf ; far return
13089
13090 ; -----
13091 ;
13092 ; 17/10/2022
13093 ; 15/10/2022
13094
13095 ; -----
13096 ;
13097 ; tim_writ sets the current time
13098 ;
13099 ; on entry es:[di] has the current time:
13100 ;
13101 ; number of days since 1-1-80 (word)
13102 ; minutes (0-59) (byte)
13103 ; hours (0-23) (byte)
13104 ; hundredths of seconds (0-99) (byte)
13105 ; seconds (0-59) (byte)
13106 ;
13107 ; each number has been checked for the correct range.
13108 ;
13109 ; NOTE: Any changes in this routine probably require corresponding
13110 ; changes in the version that is built with the power manager driver.
13111 ; See ptime.asm.
13112 ;
13113 ; -----
13114 ;
13115 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
13116 ; PC DOS 7.1 IBMBIO.COM - BIOS CODE:03EAh
13117 tim_writ: ; 2C7h:3DBh = 70h:294Bh
13118 mov ax, [es:di]
13119 push ax ; daycnt. we need to set this at the very
13120 ; end to avoid tick windows.
13121 cmp byte [havecmosclock], 0
13122 jz short no_cmos_1
13123 mov al, [es:di+3] ; near indirect calls
13124 ; get binary hours
13125 ; convert to bcd
13126 ; call far [bintobcd]
13127 ; call ds:bintobcd ; call far [bintobcd]
13128 ; 08/08/2023
13129 call bintobcd
13130 mov ch, al ; ch = bcd hours
13131 mov al, [es:di+2] ; get binary minutes
13132 ; call far [bintobcd]
13133 ; call ds:bintobcd ; convert to bcd
13134 call bintobcd
13135 mov cl, al ; cl = bcd minutes
13136 mov al, [es:di+5] ; get binary seconds
13137 ; call far [bintobcd]
13138 ; call ds:bintobcd
13139 call bintobcd
13140 mov dh, al ; dh = bcd seconds
13141 mov dl, 0 ; dl = 0 (st) or 1 (dst)
13142 cli
13143
13144 000003A0 B03C
13145 000003A2 F6E5
13146 000003A4 B500
13147 000003A6 01C8
13148 000003A8 B97017
13149 000003AB 89D3
13150 000003AD F7E1
13151 000003AF 89C1
13152 000003B1 B064
13153 000003B3 F6E7
13154 000003B5 01C1
13155 000003B7 83D200
13156 000003BA B700
13157 000003BC 01D9
13158 000003BE 83D200
13159
13160
13161
13162
13163
13164
13165
13166
13167
13168
13169
13170
13171
13172
13173
13174
13175
13176
13177
13178
13179
13180
13181
13182
13183
13184
13185
13186
13187
13188
13189
13190
13191
13192
13193
13194
13195
13196
13197
13198
13199
13200
13201
13202
13203
13204
13205
13206
13207
13208
13209
13210
13211
13212
13213
13214
13215
13216
13217
13218
13219
13220
13221
13222
13223
13224
13225
13226
13227
13228
13229
13230
13231
13232
13233
13234
13235
13236
13237
13238
13239
13240
13241
13242
13243
13244
13245
13246
13247
13248
13249
13250
13251
13252
13253
13254
13255
13256
13257
13258
13259
13260
13261
13262
13263
13264
13265
13266
13267
13268
13269
13270
13271
13272
13273
13274
13275
13276
13277
13278
13279
13280
13281
13282
13283
13284
13285
13286
13287
13288
13289
13290
13291
13292
13293
13294
13295
13296
13297
13298
13299
13300
13301
13302
13303
13304
13305
13306
13307
13308
13309
13310
13311
13312
13313
13314
13315
13316
13317
13318
13319
13320
13321
13322
13323
13324
13325
13326
13327
13328
13329
13330
13331
13332
13333
13334
13335
13336
13337
13338
13339
13340
13341
13342
13343
13344
13345
13346
13347
13348
13349
13350
13351
13352
13353
13354
13355
13356
13357
13358
13359
13360
13361
13362
13363
13364
13365
13366
13367
13368
13369
13370
13371
13372
13373
13374
13375
13376
13377
13378
13379
13380
13381
13382
13383
13384
13385
13386
13387
13388
13389
13390
13391
13392
13393
13394
13395
13396
13397
13398
13399
13400
13401
13402
13403
13404
13405
13406
13407
13408
13409
13410
13411
13412
13413
13414
13415
13416

```

```

13147 0000040E B403          mov     ah, 3
13148 00000410 CD1A          int      1Ah          ; CLOCK - SET REAL TIME      CLOCK (AT,XT286,CONV,PS)
13149                                ; CH = hours in      BCD, CL = minutes in BCD
13150                                ; DH = seconds in BCD
13151                                ; DL = 01h if daylight savings, 00h if standard time
13152                                ; Return: CMOS clock set
13153 00000412 FB          sti
13154 no_cmos_1:
13155 00000413 268B4D02        mov     cx, [es:di+2]
13156 00000417 268B5504        mov     dx, [es:di+4]
13157                                ; 17/10/2022
13158 0000041B FF1E[0606]      call    far [ttticks]
13159                                ; call dword ptr ds:ttticks ; call far [ttticks]
13160                                ; convert time to ticks
13161                                ; cx:dx now has time in ticks
13162 0000041F FA          cli          ; turn off timer
13163 00000420 B401          mov     ah, 1
13164 00000422 CD1A          int      1Ah          ; CLOCK - SET TIME OF DAY
13165                                ; CX:DX = clock count
13166                                ; Return: time of day set
13167                                ; pop ds:daycnt
13168 00000424 8F06[8904]      pop     word [daycnt]
13169 00000428 FB          sti
13170                                ; cmp ds:havecmosclock, 0
13171 00000429 803E[8C04]00    cmp     byte [havecmosclock], 0
13172 0000042E 7409          jz      short no_cmos_2
13173                                ; 08/08/2023
13174                                ; call far [daycnttoday]
13175                                ; call ds:daycnttoday ; call far [daycnttoday]
13176                                ; convert to bcd format
13177                                call    daycnttoday
13178 00000430 E80700          call    daycnttoday
13179                                cli
13180 00000433 FA          mov     ah, 5
13181 00000434 B405          int      1Ah          ; CLOCK - SET DATE IN REAL TIME      CLOCK (AT,XT286,CONV,PS)
13182 00000436 CD1A          ; DL = day in BCD, DH =      month in BCD, CL = year      in BCD
13183                                ; CH = century (19h or 20h)
13184                                ; Return: CMOS clock set
13185                                sti
13186 00000438 FB          no_cmos_2:
13187                                ; 12/12/2022
13188                                ; cf=0
13189                                ; clc
13190                                ; ret
13191 00000439 C3          ret
13192
13193 ; -----
13194 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
13195 ; PC DOS 7.1 IBMBIO.COM - BIOS CODE:0440h
13196 %if 1
13197 ; CMOS clock setting support routines used by MSCLOCK.
13198 ; Warning!!! This code will be dynamically relocated by MSINIT.
13199
13200 daycnttoday:          ; proc near
13201
13202 ; entry: [daycnt] = number of days since 1-1-80
13203 ;
13204 ; return: ch - century in bcd
13205 ;         cl - year in bcd
13206 ;         dh - month in bcd
13207 ;         dl - day in bcd
13208 ;
13209 ; 20/12/2023 - Retro DOS v5.0
13210 ; 08/08/2023 (ds:) (near proc)
13211 ; 16/10/2022 (cs:) (far proc)
13212 push     word [daycnt] ; save daycnt
13213 cmp     word [daycnt], 7305 ; (365*20+(20/4))
13214                                ; # days from 1-1-1980 to 1-1-2000
13215 jnb     short century20
13216 ; mov byte [base_century], 19
13217 ; mov byte [base_year], 80
13218 ; 08/08/2023
13219 mov     word [base_century], 5013h
13220 jmp     short years
13221
13222 ; -----
13223 century20:
13224 ; mov byte [base_century], 20
13225 ; mov byte [base_year], 0
13226 ; 08/08/2023
13227 mov     word [base_century], 20
13228 sub     word [daycnt], 7305 ; (365*20+(20/4))
13229                                ; adjust daycnt
13230 years:
13231 xor     dx, dx
13232 mov     ax, [daycnt]
13233 mov     bx, 1461          ; (366+365*3)
13234                                ; # of days in a Leap year block
13235 div     bx                ; AX = # of leap block, DX = daycnt
13236 mov     [daycnt], dx      ; save daycnt left
13237 mov     bl, 4
13238 mul     bl                ; AX = # of years. Less than 100
13239 add     [base_year], al   ; So, ah = 0. Adjust year
13240 inc     word [daycnt]    ; set daycnt to 1 base
13241 ; 08/08/2023
13242 mov     bx, 366
13243 mov     cx, 3
13244 ; cmp word [daycnt], 366 ; daycnt=remainder of leap year
13245 cmp     [daycnt], bx     ; 366
13246 jbe     short leapyear  ; within 366+355+355+355 days.
13247 inc     byte [base_year] ; if daycnt <= 366, then leap year
13248 ; sub word [daycnt], 366 ; else daycnt--, base_year++ ;
13249 sub     [daycnt], bx     ; 366 ; 08/08/2023
13250 ; mov cx, 3 ; And next three years are normal
13251 ; 08/08/2023
13252 dec     bx ; 365
13253 regularyear:
13254 ; cmp word [daycnt], 365 ; for(i=1; i>3 or daycnt <=365; i++)
13255 cmp     [daycnt], bx     ; 365 ; 08/08/2023
13256 jbe     short yeardone  ; if (daycnt > 365)
13257 inc     byte [base_year] ; { daycnt -= 365
13258 ; sub word [daycnt], 365 ; }
13259 sub     [daycnt], bx     ; 365 ; 08/08/2023
13260 loop    regularyear      ; }
13261                                ;
13262                                ; should never fall through loop
13263 leapyear:
13264 mov     byte [february], 29 ; 08/08/2023
13265 ; mov byte [month_tab+1], 29 ; leap year.
13266                                ; change month table.
13267 yeardone:
13268
13269
13270

```

```

13271 0000049E 31DB          xor     bx, bx
13272 000004A0 31D2          xor     dx, dx
13273 000004A2 A1[8904]        mov     ax, [daycnt]
13274                      ;mov     si, offset month_tab
13275 000004A5 BE[8F04]        mov     si, month_tab ; 19/10/2022
13276                      ;mov     cx, 12
13277                      ; 08/08/2023
13278 000004A8 B10C          mov     cl, 12
13279 months:
13280 000004AA FEC3          inc     bl
13281                      ; 08/08/2023
13282 000004AC 8A14          mov     dl, [si]      ; PC DOS 7.1 IBMBIO.COM - BIOS CODE:04B7h
13283 000004AE 39D0          cmp     ax, dx      ; cmp daycnt for each month till fit
13284                      ; dh=0
13285 000004B0 7605          jbe     short month_done
13286 000004B2 46             inc     si      ; next month
13287 000004B3 29D0          sub     ax, dx      ; adjust daycnt
13288 000004B5 E2F3          loop    months    ;
13289                      ; should never fall through loop
13290 month_done:
13291 000004B7 C606[9004]1C    mov     byte [february], 28 ; 08/08/2023
13292                      ;mov     byte [month_tab+1], 28
13293                      ; restore month table value
13294 000004BC 88DA          mov     dl, bl
13295 000004BE 8A36[8E04]    mov     dh, [base_year]
13296 000004C2 8A0E[8D04]    mov     cl, [base_century] ; al=day, dl=month, dh=year, cl=cntry
13297 000004C6 E81600        call    bintobcd      ; convert "day" to bcd
13298                      ; dl = bcd day, al = month
13299 000004C9 86D0          xchg    dl, al
13300 000004CB E81100        call    bintobcd      ; dh = bcd month, al = year
13301 000004CE 86F0          xchg    dh, al
13302 000004D0 E80C00        call    bintobcd      ; cl = bcd year, al = century
13303 000004D3 86C8          xchg    cl, al
13304 000004D5 E80700        call    bintobcd      ; ch = bcd century
13305 000004D8 88C5          mov     ch, al
13306 000004DA 8F06[8904]    pop     word [daycnt] ; restore original value
13307 000004DE C3             retn
13308
13309 ;-----
13310 bintobcd: ; proc near ; real time clock support
13311
13312 ;convert a binary input in al (less than 63h or 99 decimal)
13313 ;into a bcd value in al. ah destroyed.
13314
13315 aam             ; AH = AL/10, AL = AL MOD 10
13316 000004DF D40A          aad     10h         ; db 0D5h,10h
13317 000004E1 D510          ; AL = (AH*10H)+AL, AH = 0
13318 retn
13319 000004E3 C3             %endif
13320
13321 ;-----
13322 ; 15/10/2022
13323
13324 ;-----
13325 ;
13326 ; gettime reads date and time
13327 ; and returns the following information:
13328 ;
13329 ; es:[di] =count of days since 1-1-80
13330 ; es:[di+2]=hours
13331 ; es:[di+3]=minutes
13332 ; es:[di+4]=seconds
13333 ; es:[di+5]=hundredths of seconds
13334 ;
13335 ; NOTE: Any changes in this routine probably require corresponding
13336 ; changes in the version that is built with the power manager driver.
13337 ; See ptime.asm.
13338 ;-----
13339
13340 tim_read: ; 2C7h:435h = 70h:29A5h
13341          call    GetTickCnt
13342 000004E4 E84A00        mov     si, [daycnt]
13343 000004E7 8B36[8904]
13344
13345 ; we now need to convert the time in tick to the time in 100th of
13346 ; seconds. the relation between tick and seconds is:
13347 ;
13348 ; 65,536 seconds
13349 ; -----
13350 ; 1,193,180 tick
13351 ;
13352 ; to get to 100th of second we need to multiply by 100. the equation is:
13353 ;
13354 ; ticks from clock * 65,536 * 100
13355 ; ----- = time in 100th of seconds
13356 ; 1,193,180
13357 ;
13358 ; fortunately this formula simplifies to:
13359 ;
13360 ; ticks from clock * 5 * 65,536
13361 ; ----- = time in 100th of seconds
13362 ; 59,659
13363 ;
13364 ; the calculation is done by first multiplying tick by 5. next we divide by
13365 ; 59,659. in this division we multiply by 65,536 by shifting the dividend
13366 ; my 16 bits to the left.
13367 ;
13368 ; start with ticks in cx:dx
13369 ; multiply by 5
13370
13371 000004EB 89C8          mov     ax, cx
13372 000004ED 89D3          mov     bx, dx
13373                      ; startwith ticks in cx:dx
13374                      ; multiply by 5
13375 000004EF D1E2          shl     dx, 1
13376 000004F1 D1D1          rcl     cx, 1      ; times 2
13377 000004F3 D1E2          shl     dx, 1
13378 000004F5 D1D1          rcl     cx, 1      ; times 4
13379 000004F7 01DA          add     dx, bx
13380 000004F9 11C8          adc     ax, cx      ; times 5
13381 000004FB 92             xchg    ax, dx
13382
13383 ; now have ticks * 5 in dx:ax
13384 ; we now need to multiply by 65536 and divide by 59659 d.
13385 000004FC B90BE9        mov     cx, 59659    ; get divisor
13386 000004FF F7F1          div     cx          ; dx now has remainder
13387                      ; ax has high word of final quotient
13388
13389 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
13390 ;mov     bx, ax ; put high word in safeplace
13391 00000501 93             xchg    bx, ax
13392 00000502 31C0          xor     ax, ax      ; this is the multiply by 65536
13393 00000504 F7F1          div     cx          ; bx:ax now has time in 100th of seconds
13394

```



```

13395 ; rounding based on the remainder may be added here
13396 ; the result in bx:ax is time in 1/100 second.
13397
13398 00000506 89DA mov dx, bx
13399 00000508 B9C800 mov cx, 200 ; extract 1/100's
13400
13401 ; division by 200 is necessary to ensure no overflow--max result
13402 ; is number of seconds in a day/2 = 43200.
13403
13404 0000050B F7F1 div cx
13405 0000050D 80FA64 cmp dl, 100 ; remainder over 100?
13406 00000510 7203 jb short noadj
13407 00000512 80EA64 sub dl, 100 ; keep 1/100's less than 100
13408 noadj:
13409 00000515 F5 cmc bl, dl ; if we subtracted 100, carry is now set
13410 00000516 88D3 mov ; save 1/100's
13411
13412 ; to compensate for dividing by 200 instead of 100, we now multiply
13413 ; by two, shifting a one in if the remainder had exceeded 100.
13414
13415 00000518 D1D0 rcl ax, 1
13416 0000051A B200 mov dl, 0
13417 0000051C D1D2 rcl dx, 1
13418 ;mov cx, 60 ; divide out seconds
13419 ; 20/12/2023
13420 0000051E B13C mov cl, 60
13421 00000520 F7F1 div cx
13422 00000522 88D7 mov bh, dl ; save the seconds
13423 00000524 F6F1 div cl ; break into hours and minutes
13424 00000526 86C4 xchg al, ah
13425
13426 ; time is now in ax:bx (hours, minutes, seconds, 1/100 sec)
13427
13428 ; 08/08/2023
13429 ;push ax
13430 ;mov ax, si ; daycnt
13431 00000528 96 xchg ax, si
13432 00000529 AB stosw
13433 ;pop ax
13434 0000052A 96 xchg ax, si ; al = hours, ah = minutes
13435 0000052B AB stosw
13436 0000052C 89D8 mov ax, bx
13437 0000052E AB stosw
13438 0000052F F8 clc ; [es:di] = count of days since 1-1-80
13439 ; [es:di+2] = hours
13440 ; [es:di+3] = minutes
13441 ; [es:di+4] = seconds
13442 ; [es:di+5] = hundredths of seconds
13443 00000530 C3 retn
13444
13445 ; ===== S U B R O U T I N E =====
13446
13447 ; 15/10/2022
13448
13449 ;-----
13450 ;
13451 ; procedure : GetTickCnt
13452 ;
13453 ; Returns the tick count in CX:DX. Takes care of DayCnt in case
13454 ; of rollover [except when power management driver is in use].
13455 ; Uses the following logic for updating Daycnt
13456 ;
13457 ; if ( rollover ) {
13458 ; if ( t_switch )
13459 ; daycnt++ ;
13460 ; else
13461 ; daycnt += rollover ;
13462 ; }
13463 ;
13464 ; USES : AX
13465 ;
13466 ; RETURNS : CX:DX - tick count
13467 ; MODIFIES : daycnt
13468 ;
13469 ;-----
13470
13471 ; 17/10/2022
13472 GetTickCnt:
13473 00000531 30E4 xor ah, ah
13474 00000533 CD1A int 1Ah ; CLOCK - GET TIME OF DAY
13475 ; Return: CX:DX = clock count
13476 ; AL = 00h if clock was read or written (via AH=0,1) since the previous
13477 ; midnight
13478 ; Otherwise, AL > 0
13479
13480 ; 20/12/2023
13481 00000535 30E4 xor ah, ah
13482 00000537 3826[8B04] cmp byte [t_switch], ah ; 0
13483 ;cmp byte [t_switch], 0 ; use old method ? (>0 is yes)
13484 0000053B 7505 jnz short inc_case ; old method assumes that Int 1Ah returns rollover flag
13485 ;xor ah, ah ; new method assumes that Int 1Ah returns roll over count
13486 ; and not flag
13487 0000053D 0106[8904] add [daycnt], ax
13488 00000541 C3 retn
13489
13490 ; -----
13491 inc_case:
13492 00000542 08C0 or al, al
13493 00000544 7404 jz short no_rollover
13494 00000546 FF06[8904] inc word [daycnt]
13495 no_rollover:
13496 0000054A C3 retn
13497
13498 ; -----
13499 ; 03/10/2023 - Retro DOS v5.0 (Modified PC DOS 7.1)
13500 ; PC DOS 7.1 IBMBIO.COM - BIOS CODE:0556h
13501
13502 %if 1
13503
13504 0000054B 4641543132202020 fat_12_id: db 'FAT12 '
13505 00000553 4641543136202020 fat_16_id: db 'FAT16 '
13506 0000055B 4641543332202020 fat_32_id: db 'FAT32 '
13507 00000563 4E4F204E414D452020- nul_vid: db 'NO NAME '
13508 0000056C 2020
13509
13510 %endif
13511
13512 ;-----
13513 ; MSDISK.ASM - MSDOS 6.0 - 1991
13514 ;
13515 ; 15/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
13516 ; 09/03/2019 - Retro DOS v4.0
13517
13518 ; MSDISK.ASM - MSDOS 3.3 - 02/02/1988

```

```

13518 ; 26/05/2018 - Retro DOS v3.0
13519 ; 23/03/2018 - Retro DOS v2.0
13520
13521 ;error_unknown_media equ 7 ; for use in BUILD BPB call
13522
13523 ;struc BPB_TYPE
13524 ;.SECSIZE: resw 1
13525 ;.SECALL: resb 1
13526 ;.RESNUM: resw 1
13527 ;.FATNUM: resb 1
13528 ;.DIRNUM: resw 1
13529 ;.SECNUM: resw 1
13530 ;.FATID: resb 1
13531 ;.FATSIZE: resw 1
13532 ;.SLIM: resw 1
13533 ;.HLIM: resw 1
13534 ;.HIDDEN: resw 1
13535 ;.size:
13536 ;endstruc
13537
13538 ;-----
13539 ; disk interface routines
13540 ;-----
13541
13542 ; device attribute bits:
13543 ; bit 6 - get/set map for logical drives and generic ioctl.
13544
13545 ;MAXERR equ 5
13546 ;MAX_HD_FMT_ERR equ 2
13547
13548 ;LSTDRV equ 504h
13549
13550 ; some floppies do not have changeline. as a result, media-check would
13551 ; normally return i-don't-know, the dos would continually reread the fat and
13552 ; discard cached data. we optimize this by implementing a logical door-latch:
13553 ; it is physically impossible to change a disk in under 2 seconds. we retain
13554 ; the time of the last successful disk operation and compare it with the current
13555 ; time during media-check. if < 2 seconds and at least 1 timer tick has passed,
13556 ; the we say no change. if > 2 seconds then we say i-don't-know. finally,
13557 ; since we cannot trust the timer to be always available, we record the number
13558 ; of media checks that have occurred when no apparent time has elapsed. while
13559 ; this number is < a given threshold, we say no change. when it exceeds that
13560 ; threshold, we say i-don't-know and reset the counter to 0. when we store
13561 ; the time of last successful access, if we see that time has passed too,
13562 ; we reset the counter.
13563
13564 accessmax equ 5
13565
13566 ; due to various bogosities, we need to continually adjust what the head
13567 ; settle time is. the following algorithm is used:
13568 ;
13569 ; get the current head settle value.
13570 ; if it is 0, then
13571 ; set slow = 15
13572 ; else
13573 ; set slow = value
13574 ; ...
13575 ;***** old algorithm *****
13576 ;* if we are seeking and writing then
13577 ;* use slow
13578 ;* else
13579 ;* use fast
13580 ;*****
13581 ;***** ibm's requested logic *****
13582 ; if we are seeking and writing and not on an at then
13583 ; use slow
13584 ; else
13585 ; use fast
13586 ; ...
13587 ; restore current head settle value
13588 ;
13589 ;
13590 ;-----
13591
13592 multrk_on equ 10000000b ;user spcified mutitrack=on, or system turns
13593 ; it on after handling config.sys file as a
13594 ; default value, if multrk_flag = multrk_off1.
13595 multrk_off1 equ 00000000b ;initial value. no "multitrack=" command entered.
13596 multrk_off2 equ 00000001b ;user specified multitrack=off.
13597
13598 ; close data segment, open Bios_Code segment
13599
13600 ; 15/10/2022
13601
13602 ; BIOSCODE:04A2h (MSDOS 6.21, IO.SYS)
13603
13604 ;-----
13605 ; command jump table
13606 ;-----
13607
13608 0000056E 00 db 0
13609
13610 ; 11/12/2022
13611 %if 0
13612
13613 dsktbl: db 26 ; 2C7h:4A2h = 70h:2A12h
13614 ; ((dtbl_siz-1)/2) ; this is the size of the table ; 26
13615 dw 1742h ; dsk_init
13616 dw 4EBh ; media_chk
13617 dw 592h ; get_bpb
13618 dw 0D5h ; bc_cmderr
13619 dw 857h ; dsk_read
13620 dw 83Dh ; x_bus_exit
13621 dw 558h ; ret_carry_clear
13622 dw 558h ; ret_carry_clear
13623 dw 849h ; dsk_writ
13624 dw 841h ; dsk_writv
13625 dw 558h ; ret_carry_clear
13626 dw 558h ; ret_carry_clear
13627 dw 0D5h ; bc_cmderr
13628 dw 80Ah ; dsk_open
13629 dw 81Ah ; dsk_close
13630 dw 831h ; dsk_rem
13631 dw 558h ; ret_carry_clear
13632 dw 558h ; ret_carry_clear
13633 dw 558h ; ret_carry_clear
13634 dw 0C6Bh ; do_generic_ioctl
13635 dw 558h ; ret_carry_clear
13636 dw 558h ; ret_carry_clear
13637 dw 558h ; ret_carry_clear
13638 dw 1124h ; ioctl_getown
13639 dw 1142h ; ioctl_setown
13640 dw 129Ah ; ioctl_support_query
13641

```



```

13642 ;dtbl_siz equ $-dsktbl
13643
13644 %endif
13645
13646 ; 21/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
13647 ; PC DOS 7.1 IBMBIO.COM - BIOSCODE:0579h
13648
13649 ; 21/12/2023 - Retro DOS v5.0
13650 ; 11/12/2022
13651 dsktbl: db (dtbl_siz-1)/2 ; 26 ; this is the size of the table
13652 dw dsk_init
13653 dw media_chk
13654 dw get_bpb
13655 ;dw bc_cmderr
13656 dw ioctl_input ; PC DOS 7 ; 21/12/2023
13657 dw dsk_read
13658 dw x_bus_exit
13659 dw ret_carry_clear
13660 dw ret_carry_clear
13661 dw dsk_writ
13662 dw dsk_writv
13663 dw ret_carry_clear
13664 dw ret_carry_clear
13665 ;dw bc_cmderr
13666 dw ioctl_output ; PC DOS 7 ; 21/12/2023
13667 dw dsk_open
13668 dw dsk_close
13669 dw dsk_rem
13670 dw ret_carry_clear
13671 dw ret_carry_clear
13672 dw ret_carry_clear
13673 dw do_generic_ioctl
13674 dw ret_carry_clear
13675 dw ret_carry_clear
13676 dw ret_carry_clear
13677 dw ioctl_getown
13678 dw ioctl_setown
13679 dw ioctl_support_query
13680
13681 dtbl_siz equ $-dsktbl
13682
13683 ; ===== S U B R O U T I N E =====
13684
13685 ; -----
13686 ; setdrive scans through the data structure of bdss, and returns a pointer to
13687 ; the one that belongs to the drive specified. carry is set if none exists
13688 ; for the drive. Pointer is returned in es:[di]
13689 ;
13690 ; AL contains the logical drive number.
13691 ; -----
13692
13693 SetDrive:
13694 ;les di, dword ptr ds:start_bds ; Point es:di to first bds
13695 ;les di, [start_bds] ; 19/10/2022
13696
13697 X_Scan_Loop:
13698 cmp [es:di+5], al
13699 jz short X_SetDrv
13700 les di, [es:di] ; [es:di+BDS.link] ; Go to nextbds
13701 cmp di, 0FFFFh
13702 jnz short X_Scan_Loop
13703 stc
13704
13705 X_SetDrv:
13706 retn
13707
13708 ; -----
13709 ; 15/10/2022
13710
13711 ; 21/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
13712 ; PC DOS 7.1 IBMBIO.COM - BIOSCODE:05C2h
13713
13714 ; -----
13715 ; if id is f9, have a 96tpi disk else
13716 ; if bit 2 is 0 then media is not removable and could not have changed
13717 ; otherwise if within 2 secs of last disk operation media could not
13718 ; have changed, otherwise dont know if media has changed
13719 ; -----
13720
13721 media_chk: ; 2C7h:4EBh = 70h:2A5Bh
13722 call SetDrive
13723 mov si, 1
13724 ; 21/12/2023
13725 test byte [es:di+40h], 1
13726 test byte [es:di+24h], 1 ; [es:di+BDS.flags+1]
13727 ; fchanged_by_format
13728 jz short weAreNotFakingIt
13729 ; 21/12/2023
13730 and byte [es:di+40h], 0FEh
13731 ; 12/12/2022
13732 and byte [es:di+24h], 0FEh ; ~fchanged_by_format
13733 ; and word [es:di+23h], 0FEFFh ; [es:di+BDS.flags]
13734 ; ~fchanged_by_format ; reset flag
13735 mov byte [tim_drv], 0FFh ; -1
13736 ; Ensure that we ask the rom if media has changed
13737 ; 21/12/2023
13738 test byte [es:di+3Fh], 1
13739 test byte [es:di+23h], 1 ; [es:di+BDS.flags]
13740 ; fnon_removable
13741 jz short wehaveafloppy
13742 mov si, 0FFFFh ; Indicate media changed
13743 ; 11/08/2023
13744 neg si ; PC DOS 7.1 IBMBIO.COM - BIOSCODE:05E0h
13745 jmp short Media_Done ; Media_Done
13746
13747 ; -----
13748
13749 weAreNotFakingIt:
13750 ; 21/12/2023
13751 test byte [es:di+3Fh], 1
13752 test byte [es:di+BDS.flags], fnon_removable
13753 test byte [es:di+23h], 1
13754 jnz short Media_Done
13755
13756 wehaveafloppy:
13757 xor si, si ; 0 ; Presume "I don't know"
13758 ; 11/08/2023
13759 dec si ; 0 ; PC DOS 7.1 IBMBIO.COM - BIOSCODE:05EBh
13760
13761 ; If we have a floppy with changeline support, we ask the ROM
13762 ; to determine if media has changed. We do not perform the
13763 ; 2 second check for these drives.
13764
13765 cmp byte [fhave96], 0 ; Do we have changeline support?
13766 jz short mChk_NoChangeLine ; Brif not
13767 call mediacheck ; Call into removable routine
13768 jb short err_exitj

```

```

13766 000005EE E89A16          call    haschange
13767 000005F1 7512          jnz     short Media_Done
13768                                mChk_NoChangeLine:
13769                                ; If we come here, we have a floppy with no changeline support
13770
13771 000005F3 BE0100          mov     si, 1          ; Presume no change
13772 000005F6 A01E01          mov     al, [tim_drv] ; Last drive accessed
13773 000005F9 263A4504        cmp     al, [es:di+4] ; [es:di+BDS.drivenum]
13774                                ; Is drive of last access the same?
13775 000005FD 7505          jnz     short Media_Unk ; No, then "i don't know"
13776 000005FF E82800          call   Check_Time_Of_Access
13777 00000602 EB01          jmp     short Media_Done
13778                                ; -----
13779
13780                                Media_Unk:
13781 00000604 4E          dec     si          ; 0 ; Return "I don't know"
13782                                ; SI now contains the correct value for media change.
13783                                ; Clean up the left overs
13784
13785                                Media_Done:
13786                                ; 19/10/2022
13787 00000605 06          push    es
13788 00000606 C41E1200          les     bx, [ptrsav]
13789 0000060A 2689770E        mov     [es:bx+0Eh], si ; [es:bx+trans]
13790 0000060E 07          pop     es
13791 0000060F 09F6          or      si, si
13792 00000611 790F          jns     short ret_carry_clear ; validok
13793 00000613 803E770000        cmp     byte [fhave96], 0
13794 00000618 7403          jz      short mChk1_NoChangeLine ; Brif no changeline support
13795 0000061A E80016          call   media_set_vid
13796                                mChk1_NoChangeLine:
13797 0000061D C6061E01FF        mov     byte [tim_drv], 0FFh ; -1
13798                                ; Make sure we ask rom for media check
13799
13800                                ret_carry_clear:
13801 00000622 F8          clc
13802 00000623 C3          retn
13803                                ; -----
13804
13805                                err_exitj:
13806                                call    maperror ; guaranteed to set carry
13807                                ret81:
13808                                mov     ah, 81h ; return error status
13809                                retn ; return with carry set
13810                                ; ===== S U B R O U T I N E =====
13811
13812                                ; -----
13813                                ; perform a check on the time passed since the last access for this physical
13814                                ; drive.
13815                                ; we are accessing the same drive. if the time of last successful access was
13816                                ; less than 2 seconds ago, then we may presume that the disk was not changed.
13817                                ; returns in si:
13818                                ; 0 - if time of last access was >= 2 seconds
13819                                ; 1 - if time was < 2 seconds (i.e no media change assumed)
13820                                ; registers affected ax,cx,dx, flags.
13821                                ;
13822                                ; assume es:di -> bds, ds->Bios_Data
13823                                ; -----
13824
13825                                ; 21/12/2023 - Retro DOS v5.0 IBMBIO.COM
13826                                ; 19/10/2022
13827                                Check_Time_Of_Access:
13828                                mov     si, 1 ; presume no change.
13829                                call    GetTickCnt ; cx:dx is the elapsed time
13830                                ; 21/12/2023
13831 00000630 268B4579        mov     ax, [es:di+79h]
13832                                ;mov     ax, [es:di+47h] ; [es:di+BDS.tim_lo]
13833                                ; get stored time
13834                                sub     dx, ax
13835                                ; 21/12/2023
13836 00000636 268B457B        mov     ax, [es:di+7Bh]
13837                                ;mov     ax, [es:di+49h] ; [es:di+BDS.tim_hi]
13838 0000063A 19C1          sbb     cx, ax
13839                                ; 11/08/2023
13840                                ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0646h
13841                                ;mov     al, [accesscount]
13842 0000063C 7515          jnz     short timecheck_unk ; cx<0 => >1 hour
13843 0000063E 09D2          or      dx, dx ; time must pass
13844 00000640 750C          jnz     short timepassed ; yes, examine max value
13845                                ; 11/08/2023
13846                                ;inc     al
13847                                ;cmp     al, 5
13848                                ;inc     byte [accesscount]
13849                                ;cmp     byte [accesscount], 5
13850                                ; ; if count is less than threshold, ok
13851                                ;jb      short timecheck_ret
13852                                ;dec     byte [accesscount] ; don't let the count wrap
13853                                ; 11/08/2023
13854                                ;dec     al
13855                                ;jmp     short timecheck_unk ; "i don't know" if media changed
13856                                ; 11/08/2023
13857 00000642 803E1D0104        cmp     byte [accesscount], 4
13858 00000647 730A          jnb     short timecheck_unk
13859 00000649 FE061D01          inc     byte [accesscount]
13860 0000064D C3          retn
13861                                ; -----
13862
13863                                timepassed:
13864                                cmp     dx, 36 ; 18*2 ; 18.2 tics per second.
13865                                ; min elapsed time? (2 seconds)
13866                                jbe     short timecheck_ret ; yes, presume no change
13867                                ; everything indicates that we do not know what has happened.
13868
13869                                timecheck_unk:
13870                                dec     si ; presume i don't know
13871 00000653 4E          dec     si
13872                                timecheck_ret:
13873                                ; 11/08/2023
13874                                ;mov     [accesscount], al
13875                                retn
13876                                ; -----
13877
13878                                ; 15/10/2022
13879                                Err_Exitj2:
13880 00000655 EB0D          jmp     short err_exitj
13881                                ; -----
13882
13883                                ; 15/10/2022
13884
13885                                ; =====
13886                                ; Build a valid bpb for the disk in the drive.
13887                                ; =====
13888
13889

```

```

13890 ; 21/12/2023 - Retro DOS v5.0 IBMBIO.COM
13891 ; 19/10/2022
13892
13893 get_bpb: mov ah, [es:di] ; 2c7h:592h = 70h:2802h
13894 call SetDrive ; get fat id byte read by dos
13895 ; 21/12/2023 ; get the correct bds for the drive
13896 test byte [es:di+3Fh], 1
13897 ;test byte [es:di+23h], 1 ; [es:di+BDS.flags]
13898 ; ; fnon_removable
13899 jnz short already_gotbpb ; no need to build for fixed disks
13900
13901 ; let's set the default value for volid,vol_serial,
13902 ; filesys_id in bds table
13903
13904 call clear_ids
13905 ;mov ds:set_id_flag, 1 ; indicate to set system id in bds
13906 mov byte [set_id_flag], 1
13907 call GetBp ; build a bpb if necessary
13908 jb short ret81
13909 ;cmp ds:set_id_flag, 2 ; already, volume_label set from boot
13910 cmp byte [set_id_flag], 2
13911 ;mov ds:set_id_flag, 0 ; record to bds table?
13912 mov byte [set_id_flag], 0
13913 jz short already_gotbpb ; do not set it again from root dir
13914 ; ; otherwise, conventional boot record
13915 ;cmp ds:fhave96, 0 ; do we have changeline support?
13916 cmp byte [fhave96], 0
13917 jz short already_gotbpb ; brief not
13918 call set_volume_id
13919 already_gotbpb:
13920 add di, 6 ; BDS.BPB
13921 ; return the bpb from the current bds
13922
13923 ; fall into setptrsav, es:di -> result
13924
13925 ; -----
13926
13927 ; 15/10/2022
13928
13929 ; =====
13930 ; SetPtrsav is also jumped to from dsk_init (msbio2.asm). In both cases, the
13931 ; pointer to be returned is in es:di. We were incorrectly returning ds:di.
13932 ; Note that this works in most cases because most pointers are in Bios_Data.
13933 ; It fails, for instance, when we install an external drive using driver.sys
13934 ; because then the BDS segment is no longer Bios_Data.
13935 ; NB: It is fine to corrupt cx because this is not a return value and anyway
13936 ; this returns to Chardev_entry (msbio1.asm) where all registers are
13937 ; restored before returning to the caller.
13938 ; =====
13939
13940 ; 21/12/2023
13941 %if 0
13942 ; 19/10/2022
13943 SetPtrSav: ; return point for dsk_init
13944 mov cx, es ; save es
13945 ;les bx, ds:ptrsav
13946 ;les bx, [ptrsav]
13947 mov [es:bx+0Dh], ah ; [es:bx+media]
13948 mov [es:bx+12h], di ; [es:bx+count]
13949 mov [es:bx+14h], cx ; [es:bx+count+2]
13950 cll
13951 retn
13952 %endif
13953 ; 21/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
13954 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0698h
13955 SetPtrSav: ; return point for dsk_init
13956 push ds
13957 ;lds bx, ds:ptrsav
13958 ;lds bx, [ptrsav]
13959 mov [bx+0Dh], ah ; [bx+media]
13960 mov [bx+12h], di ; [bx+count]
13961 mov [bx+14h], es ; [bx+count+2]
13962 push ds
13963 pop es
13964 pop ds
13965 cll
13966 retn
13967
13968 ; ===== S U B R O U T I N E =====
13969
13970 ; 15/10/2022
13971
13972 ; -----
13973 ; clear ids in bds table. only applied for floppies.
13974 ;input: es:di -> bds table
13975 ; assumes ds: -> Bios_Data
13976 ;output: volid set to "NO NAME "
13977 ; vol_serial set to 0.
13978 ; filesys_id set to "FAT12 " or "FAT16 "
13979 ; depending on the flag fatsize in bds.
13980 ;
13981 ; trashes si, cx
13982 ; -----
13983
13984 ;size_of_EXT_BOOT_VOL_LABEL equ 11
13985 ;size_of_EXT_SYSTEM_ID equ 8
13986
13987 ; 11/09/2023
13988 ; 14/08/2023
13989 ;BDS.fatsiz equ 1Fh
13990 ; 21/12/2023
13991 ;BDS.fatsiz equ 59
13992
13993 ; 22/12/2023
13994 ; 21/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
13995
13996 clear_ids: ;mov al, [es:di+1Fh] ; mov al,[es:di+BDS.fatsiz]
13997 ; 21/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM BugFix)
13998 mov bl, [es:di+3Bh] ; mov bl,[es:di+BDS.fatsiz]; *+
13999
14000 clear_ids_x:
14001 ; 21/12/2023
14002 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:06ABh)
14003 ; 11/09/2023
14004 ; (MSDOS 5.0 IO.SYS - BIOSCODE:05D9h)
14005 push di
14006 xor cx, cx ; no serial number
14007 ; 21/12/2023
14008 mov [es:di+89h], cx ; [es:di+BDS.vol_serial]
14009 mov [es:di+8Bh], cx ; [es:di+BDS.vol_serial+2]
14010 mov [es:di+57h], cx ; [es:di+BDS.vol_serial]
14011 mov [es:di+59h], cx ; [es:di+BDS.vol_serial+2]
14012
14013 ; BUGBUG - there's a lot in common here and with

```

```

14014 ; mov_media_ids.. see if we can save some space by
14015 ; merging them... jgl
14016
14017 ;mov cx, 11 ; size_of_EXT_BOOT_VOL_LABEL
14018 ; 10/12/2022
14019 000006AE B10B mov cl, 11 ; cx = 11
14020
14021 ;;mov si, offset vol_no_name ; "NO NAME "
14022 ;mov si, vol_no_name ; 19/10/2022
14023 ; 22/12/2023
14024 ;mov si, offset nul_vid ; "NO NAME "
14025 000006B0 BE[6305] mov si, nul_vid
14026
14027 ; 21/12/2023
14028 000006B3 83C77D add di, 125
14029 ;add di, 75 ; BDS.volid
14030
14031 ;rep movsb
14032 ; 21/12/2023
14033 ;rep movs byte ptr es:[di], byte ptr cs:[si] ; cs rep movsb
14034 ; 26/12/2023
14035 ;cs ; vol_no_name is in BIOSCODE segment
14036 ;rep movsb
14037 000006B6 F3 rep
14038 000006B7 2E cs
14039 000006B8 A4 movsb
14040
14041 ; 11/09/2023 (BugFix, DI is not start addr of BDS structure here)
14042 ;;test byte [es:di+BDS.fatsiz], fbig
14043 ; (MSDOS 5.0 IO.SYS - BIOSCODE:05EFh)
14044 ;test byte [es:di+1Fh], 40h
14045 ; 21/12/2023 - Retro DOS v5.0
14046 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:06C3h)
14047 ;test byte [es:di+59], 20h
14048 ; (here, es:di points to the BDS offset +136)
14049 ; purpose: test byte [es:di+BDS.fatsiz], fbigbig
14050 ; applied: test byte [es:BDS.fatsiz+136], fbigbig -BUG!-
14051
14052 ; (PCDOS 7.1 BUG note: 26/06/2023 - Erdogan Tan)
14053 ;; ! NOTE - 11/08/2023 - Erdogan Tan (Retro DOS v4.2 IO.SYS bugfix)
14054 ; Microsoft/IBM code has a bug here because the BDS's
14055 ; .volid and .filesys_id fields will be reset
14056 ; (to their default text) according to 'BDS.fatsiz' flags
14057 ; at the BDS offset 59 but current (this) code checks flags
14058 ; at ES:DI+59 while DI points the BDS offset 136!? ; (PCDOS 7.1)
14059 ; at the BDS offset 31 but current (this) code checks flags
14060 ; at ES:DI+31 while DI points the BDS offset 86!? ; (MSDOS 6.22)
14061 ;
14062 ; Correct Code:
14063 ; ;test byte [ES:59], 20h or [ES:BDS.fatsiz], fbigbig ; (PCDOS 7.1)
14064 ; ;test byte [ES:31], 40h or [ES:BDS.fatsiz], fbig ; (MSDOS 6.22)
14065 ; 11/09/2023
14066 ; (before 'rep movsb') 'mov al,[es:di+BDS.fatsiz]' and then
14067 ; (after 'rep movsb') 'test al,fbig' (AL is free/proper to use here)
14068 ;
14069 ; Same BUG is existing in MSDOS 6.22 IO.SYS - BIOSCODE:05EFh
14070 ; and in Windows ME IO.SYS - BIOSCODE:0E1Ah as 'test byte [es:di+59], 20h'
14071
14072 ;
14073 ; (why this bug did not affect MSDOS and PC DOS 7.x applications:
14074 ; 'clear_ids' is used for floppy disks only and the default
14075 ; option of 'clear_ids' is FAT12 volid and filesys_id text
14076 ; when the flag bit has wrong value for FAT16/40h or FAT32/20h.)
14077
14078 ; 21/12/2023 - Retro DOS v5.0
14079 ;mov si, offset fat_32_id ; "FAT32 "
14080 000006B9 BE[5B05] mov si, fat_32_id
14081
14082 ; 21/12/2023
14083 ; BugFix (of the PC DOS 7.1 IBMBIO.COM BUG) ; *+
14084 ;test bl, fbigbig ; FAT32 flag
14085 000006BC F6C320 test bl, 20h ; * ; BL = [es:BDS.fatsiz] = [es:59]
14086 000006BF 750B jnz short ci_bigfat
14087
14088 ;mov si, offset fat_16_id ; "FAT16"
14089 000006C1 BE[5305] mov si, fat_16_id ; 19/10/2022
14090
14091 ; 21/12/2023
14092 ; !BUG! (PCDOS 7.1 IBMBIO.COM BIOSCODE:06CDh)
14093 ;test byte [es:di+59], 40h ; [es:di+BDS.fatsiz], fbig
14094 ; BugFix ; *+
14095 ;test bl, fbig ; FAT16 flag
14096 000006C4 F6C340 test bl, 40h ; * ; Retro DOS v5.0
14097 ;test al, 40h ; * ; Retro DOS v4.2
14098 000006C7 7503 jnz short ci_bigfat
14099
14100 ;mov si, offset fat_12_id ; "FAT12"
14101 000006C9 BE[4B05] mov si, fat_12_id ; 19/10/2022
14102 ci_bigfat:
14103 ;mov cx, 8 ; size_of_EXT_SYSTEM_ID
14104 ; 10/12/2022
14105 000006CC B108 mov cl, 8 ; cx = 8
14106 000006CE 83C705 add di, 5 ; (BDS.filesys_id-BDS.volid)-size_of_EXT_BOOT_VOL_LABEL
14107 ; filesys_id field
14108 ;rep movsb
14109 ; 21/12/2023 - Retro DOS v5.0
14110 ;rep movs byte ptr es:[di], byte ptr cs:[si] ; 0F3h,2Eh,0A4h
14111 ; 26/12/2023
14112 ;cs ; fat32_id, fat16_id and fat12_id are in BIOSCODE segment
14113 ;rep movsb
14114 000006D1 F3 rep
14115 000006D2 2E cs
14116 000006D3 A4 movsb
14117
14118 000006D4 5F pop di ; restore bds pointer
14119 getret_exit: ; 21/12/2023
14120 000006D5 C3 retn
14121
14122 ; ===== S U B R O U T I N E =====
14123
14124 ; 15/10/2022
14125
14126 ; -----
14127 ; getbp - return bpb from the drive specified by the bds.
14128 ; if the return_fake_bpb flag is set, then it does nothing.
14129 ; note that we never come here for fixed disks.
14130 ; for all other cases,
14131 ; - it reads boot sector to pull out the bpb
14132 ; - if no valid bpb is found, it then reads the fat sector,
14133 ; to get the fat id byte to build the bpb from there.
14134 ;
14135 ; inputs: es:di point to correct bds.
14136 ;
14137 ; outputs: fills in bpb in current bds if valid bpb or fat id on disk.

```

```

14138 ; carry set, and al=7 if invalid disk.
14139 ; carry set and error code in al if other error.
14140 ; if failed to recognize the boot record, then will set the
14141 ; set_id_flag to 0.
14142 ; this routine will only work for a floppy diskette.
14143 ; for a fixed disk, it will just return.
14144 ;
14145 ; ***** Note: getbp is a clone of getbp which uses the newer
14146 ; segment definitions. It should be migrated towards.
14147 ; now es:di has the bds, ds: has Bios_Data
14148 ; -----
14149 ;
14150 ; 29/12/2023
14151 ; 21/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
14152
GetBp:
14153 ; if returning fake bpb then return bpb as is.
14154 ; 21/12/2023
14155 000006D6 26F6453F05 test byte [es:di+3Fh], 5 ; PCDOS 7.1
14156 ;test byte [es:di+BDS.flags], return_fake_bpb|fnon_removable
14157 ;test byte [es:di+23h], 5 ; MSDOS 6.22 (& MSDOS 5.0)
14158 ;jz short getbp1 ; getbp1
14159 ;jmp getret_exit
14160 ; 21/12/2023
14161 000006DB 75F8 jnz short getret_exit
14162 ; -----
14163
getbp1:
14164 000006DD 51 push cx
14165 000006DE 52 push dx
14166 000006DF 53 push bx
14167
14168 ; attempt to read in boot sector and determine bpb.
14169 ; we assume that the 2.x and greater dos disks all
14170 ; have a valid boot sector.
14171
14172 000006E0 E8CF00 call readbootsec
14173 000006E3 720A jb short getbp_err_ret_brdg ; carry set if there was error.
14174 000006E5 09DB or bx, bx ; bx is 0 if boot sector is valid.
14175 000006E7 7509 jnz short dofatbpb
14176 000006E9 E81401 call movbpb ; move bpb into registers
14177 ;jmp short Has1
14178 ; 21/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
14179 000006EC E9B500 jmp getret
14180 ; -----
14181
getbp_err_ret_brdg:
14182 jmp getbp_err_ret
14183 000006EF E9B600 ; -----
14184
14185 ; we have a 1.x diskette. In this case read in the fat ID byte
14186 ; and fill in bpb from there.
14187
dofatbpb:
14188
14189 000006F2 E8B401 call readfat ; puts media descriptor byte in ah
14190 000006F5 72F8 jb short getbp_err_ret_brdg
14191 ;cmp ds:fhave96, 0 ; changeline support available?
14192 000006F7 803E[7700]00 cmp byte [fhave96], 0 ; 19/10/2022
14193 000006FC 7403 jz short bpb_nochangeline ; brif not
14194 000006FE E83115 call hidensity ; may not return! May add sp, 2 and
14195 ; jump to has1!!!!!! or has720K
14196
bpb_nochangeline:
14197 ; test for a valid 3.5" medium
14198 00000701 26807D3E02 ; 21/12/2023 - Retro DOS v5.0
14199 cmp byte [es:di+3Eh], 2
14200 ;cmp byte [es:di+22h], 2 ; [es:di+BDS.formfactor]
14201 ; ffSmall
14202 00000706 7512 jnz short is_floppy
14203 00000708 80FCF9 cmp ah, 0F9h ; is it a valid fat id byte for 3.5" ?
14204 0000070B 7512 jnz short got_unknown_medium
14205
Has720K:
14206 ; 21/12/2023
14207 ;mov bx, offset sm92 ; pointer to correct bpb
14208 ;mov bx, sm92 ; 19/10/2022
14209
14210 ; es points to segment of bds. the following should be modified
14211 ; to get spf,csec,spau,spt correctly. it had been wrong if
14212 ; driver.sys is loaded since the bds is inside the driver.sys.
14213
14214 ; 21/12/2023
14215 ; 10/12/2022
14216 ;mov al, [bx+0] ; [bx+bpptype.spf]
14217 ; 21/12/2022
14218 ;mov al, [bx]
14219 ;mov cx, [bx+3] ; [bx+bpptype.csec]
14220 ;mov dx, [bx+5] ; [bx+bpptype.spau]
14221 ;mov bx, [bx+1] ; [bx+bpptype.spt]
14222 ; 19/10/2022 - Temporary !
14223 ;db 8Ah, 87h, 0, 0 ; mov al, [bx+0]
14224 ;db 8Bh, 8Fh, 3, 0 ; mov cx, [bx+3]
14225 ;db 8Bh, 97h, 5, 0 ; mov dx, [bx+5]
14226 ;db 8Bh, 9Fh, 1, 0 ; mov bx, [bx+1]
14227
14228 ; 21/12/2023 - Retro DOS v5.0
14229 mov al, 3 ; bpptype.sbf = 3
14230 mov cx, 1440 ; bpptype.csec = 1440
14231 mov dx, 202h ; dl = bpptype.spau = 2
14232 ; dh = bpptype.chead = 2
14233 mov bx, 7009h ; bl = bpptype.spt = 9
14234 ; bh = bpptype.dire = 112
14235 jmp short Has1
14236 ; -----
14237
is_floppy:
14238 0000071A 80FCF8 cmp ah, 0F8h ; valid media?? (0F8h-0FFh)
14239 ;jb short got_unknown_medium
14240 ; 21/12/2023
14241 0000071D 730A jnb short chk_160K
14242 ; -----
14243
14244 ; 21/12/2023
14245 ; we have a 3.5" diskette for which we cannot build a bpb.
14246 ; we do not assume any type of bpb for this medium.
14247
got_unknown_medium:
14248 0000071F C606[9B04]00 mov ds:set_id_flag, 0
14249 00000724 B007 mov byte [set_id_flag], 0
14250 00000726 F9 stc
14251 00000727 EB7B jmp short getret
14252 ; -----
14253
chk_160K:
14254 00000729 B001 mov al, 1 ; set number of fat sectors
14255 0000072B B80840 mov bx, 16392 ; 64*256+8
14256 ; set dir entries and sector max
14257 0000072E B94001 mov cx, 320 ; 40*8
14258 ; set size of drive
14259 00000731 BA0101 mov dx, 257 ; 01*256+1
14260 ; set head limit and sec/all unit
14261 ; 21/12/2023

```

```

14262             ;mov     al, 1             ; bpbtype.sbf = 1
14263             ;mov     bx, 4008h          ; b1 = bpbtype.spt = 8
14264             ;             ; bh = bpbtype.dire = 64
14265             ;mov     cx, 140h           ; bpbtype.csec = 320
14266             ;mov     dx, 101h          ; d1 = bpbtype.spau = 1
14267             ;             ; dh = bpbtype.thead = 1
14268
14269             test     ah, 2             ; test for 8 or 9 sector
14270             jnz     short has8         ; nz = has 8 sectors
14271
14272             ; 29/12/2023
14273             ;inc     al ; 2             ; inc number of fat sectors
14274             ;inc     bl ; 9             ; inc sector max
14275             inc     ax
14276             inc     bx
14277
14278             ;add     cx, 40             ; increase size (to 360)
14279             ; 18/12/2022
14280             add     cl, 40 ; 28h      ; 180K (360 sectors)
14281
14282             has8: test     ah, 1             ; test for 1 or 2 heads
14283             jz     short Has1          ; jz = 1 head
14284             add     cx, cx             ; double size of disk
14285             mov     bh, 112           ; increase number of directory entries
14286             inc     dh ; 2             ; inc sec/all unit
14287             ; 29/12/2023
14288             ;inc     dl ; 2             ; inc head limit
14289             inc     dx
14290
14291             Has1: ; 02/09/2023 (PCDOS 7.1, IBMBIO.COM - BIOSCODE:0754h)
14292             push    ds
14293             push    es
14294             pop     ds
14295
14296             ;mov     [es:di+8], dh      ; [es:di+BDS.secpersclus]
14297             ;mov     [es:di+0Ch], bh     ; [es:di+BDS.diretries]
14298             ;mov     [es:di+0Eh], cx     ; [es:di+BDS.totalsecs16]
14299             ;mov     [es:di+10h], ah     ; [es:di+BDS.media]
14300             ;mov     [es:di+11h], al     ; [es:di+BDS.fatsecs]
14301             ;mov     [es:di+13h], bl     ; [es:di+BDS.secperstrack]
14302             ;mov     [es:di+15h], dl     ; [es:di+BDS.heads]
14303
14304             mov     [di+8], dh          ; [di+BDS.secpersclus]
14305             xor     dh, dh
14306             mov     [di+15h], dx       ; [di+BDS.heads]
14307             mov     dl, bh
14308             mov     [di+0Ch], dx       ; [di+BDS.diretries]
14309             mov     [di+0Eh], cx       ; [di+BDS.totalsecs16]
14310             mov     [di+1Bh], cx       ; [di+BDS.totalsecs32]
14311             mov     [di+10h], ah       ; [di+BDS.media]
14312             mov     dl, al
14313             mov     [di+11h], dx       ; [di+BDS.fatsecs]
14314             mov     dl, bl
14315             mov     [di+13h], dx       ; [di+BDS.secperstrack]
14316
14317             ; the BDS_BPB.BPB_HIDDESECTORS+2 field and the
14318             ; BDS_BPB.BPB_BIGTOTALSECTORS field need to be set
14319             ; to 0 since this code is for floppies
14320
14321             ; 18/12/2022
14322             ;mov     word [es:di+19h], 0 ; [es:di+BDS.hiddensecs+2]
14323             ;mov     word [es:di+17h], 0 ; [es:di+BDS.hiddensecs]
14324             ;mov     word [es:di+1Dh], 0 ; [es:di+BDS.totalsecs32+2]
14325             ; 18/12/2022
14326             sub     cx, cx ; 0
14327             ;mov     [es:di+19h], cx ; 0 ; [es:di+BDS.hiddensecs+2]
14328             ;mov     [es:di+17h], cx ; 0 ; [es:di+BDS.hiddensecs]
14329             ;mov     [es:di+1Dh], cx ; 0 ; [es:di+BDS.totalsecs32+2]
14330
14331             ; 02/09/2023
14332             mov     [di+19h], cx ; 0 ; [di+BDS.hiddensecs+2]
14333             mov     [di+17h], cx ; 0 ; [di+BDS.hiddensecs]
14334             mov     [di+1Dh], cx ; 0 ; [di+BDS.totalsecs32+2]
14335
14336             ; 21/12/2023 - Retro DOS v5.0
14337             mov     [di+1Fh], cx       ; [di+BDS.fatsecs32] ; BPB_FATSz32
14338             mov     [di+21h], cx       ; [di+BDS.fatsecs32+2]
14339             mov     [di+27h], cx       ; [di+BDS.rootdirclust]
14340             mov     [di+29h], cx       ; [di+BDS.rootdirclust+2]
14341             mov     [di+2Fh], cx       ; [di+BDS.reserved]
14342             ; BPB_Reserved (12 zero bytes)
14343             mov     [di+31h], cx
14344             mov     [di+33h], cx
14345             mov     [di+35h], cx
14346             mov     [di+37h], cx
14347             mov     [di+39h], cx
14348             mov     [di+23h], cx       ; [di+BDS.extflags] ; BPB_ExtFlags
14349             mov     [di+25h], cx       ; [di+BDS.fsver] ; BPB_FSVer
14350
14351             dec     cx ; -1 ; 0FFFFFFFh
14352             mov     [di+2Bh], cx        ; [di+BDS.fsinfo] ; BPB_FSInfo
14353             mov     [di+2Dh], cx        ; [di+BDS.bkbootsec] ; BPB_BkBootSec
14354
14355             pop     ds ; 02/09/2023
14356             getret: pop     bx
14357             pop     dx
14358             pop     cx
14359             ;getret_exit: ; 21/12/2023
14360             retn
14361
14362             ; -----
14363
14364             getbp_err_ret: ; before doing anything else, set set_id_flag to 0.
14365             ;mov     ds:set_id_flag, 0
14366             ; 19/10/2022
14367             mov     byte [set_id_flag], 0
14368             call    maperror
14369             jmp     short getret
14370
14371             ; -----
14372             ; 21/12/2023
14373             ; we have a 3.5" diskette for which we cannot build a bpb.
14374             ; we do not assume any type of bpb for this medium.
14375
14376             got_unknown_medium:
14377             ;mov     ds:set_id_flag, 0
14378             mov     byte [set_id_flag], 0
14379             mov     al, 7
14380             stc
14381             jmp     short getret
14382
14383             ; ===== S U B R O U T I N E =====
14384
14385             ; 15/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS)

```



```

14386 ; -----
14387 ; read in the boot sector. set carry if error in reading sector.
14388 ; bx is set to 1 if the boot sector is invalid, otherwise it is 0.
14389 ;
14390 ; assumes es:di -> bds, ds-> Bios_Data
14391 ; -----
14392
14393 ; 10/03/2019 - Retro DOS v4.0
14394
14395 ; 30/12/2022 - Retro DOS v4.2
14396 ; (MSDOS 6.21 IO.SYS, BIOSCODE:06C3h)
14397 ; ((MSDOS 6.22 IO.SYS, BIOSCODE:06C3h)) ; 22/12/2023
14398
14399 ; 22/12/2023 - Retro DOS v5.0
14400 ; (PCDOS 7.1 IBMBIO.COM, BIOSCODE:07C6h)
14401
14402 readbootsec:
14403     mov     dh, 0          ; head 0
14404     mov     cx, 1          ; cylinder 0, sector 1
14405     call    read_sector
14406     jb      short err_ret
14407     xor     bx, bx         ; assume valid boot sector
14408
14409     ; put a sanity check for the boot sector in here to detect
14410     ; boot sectors that do not have valid bpbs. we examine the
14411     ; first two bytes - they must contain a long jump (69h) or a
14412     ; short jump (EBh) followed by a nop (90h), or a short jump
14413     ; (E9h). if this test is passed, we further check by examining
14414     ; the signature at the end of the boot sector for the word
14415     ; AA55h. if the signature is not present, we examine the media
14416     ; descriptor byte to see if it is valid. for dos 3.3, this
14417     ; logic is modified a little bit. we are not going to check
14418     ; signature. instead we are going to sanity check the media
14419     ; byte in bpb regardless of the validity of signature. this is
14420     ; to save the already developed commercial products that have
14421     ; good jump instruction and signature but with the false bpb
14422     ; informations
14423
14424     ; that will crash the diskette drive operation. (for example, symphony diskette).
14425
14426     ; 02/09/2023
14427     ; 19/10/2022
14428     cmp     byte [disksector], 69h ; is it a direct jump?
14429     jz      short check_bpb_mediabyte ; don't need to find a nop
14430     cmp     byte [disksector], 0E9h ; dos 2.0 jump?
14431     jz      short check_bpb_mediabyte ; no need for nop
14432     cmp     byte [disksector], 0EBh ; how about a short jump?
14433     jnz     short invalidbootsec
14434     cmp     byte [disksector+2], 90h ; is next one a nop?
14435     jnz     short invalidbootsec
14436
14437     ; 02/09/2023 (PCDOS 7.1)
14438     mov     al, [disksector]
14439     cmp     al, 69h         ; is it a direct jump?
14440     je      short check_bpb_mediabyte
14441     ; don't need to find a nop
14442     cmp     al, 0E9h        ; dos 2.0 jump?
14443     je      short check_bpb_mediabyte
14444     ; no need for nop
14445     cmp     al, 0EBh        ; how about a short jump?
14446     jne     short invalidbootsec
14447     cmp     byte [disksector+2], 90h ; is next one a nop?
14448     jne     short invalidbootsec
14449
14450 ; 15/10/5022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
14451 ;
14452 ; 10/03/2019
14453 ; (MSDOS 3.3, MSDISK.ASM, 1988)
14454 ;
14455 ; Don't have to perform the following signature check since
14456 ; we need to check the media byte even with the good signed diskette.
14457 ;
14458 ; check_signature:
14459 ;     cmp     word [cs:disksector+1FEh], 0AA55h ; see if non-ibm
14460 ;     ; disk or 1.x media.
14461 ;     jz      short checksingledsided ; go see if singled sided medium.
14462 ;     ; may need some special handling
14463 ;
14464 ; check for non-ibm disks which do not have the signature AA55h at the
14465 ; end of the boot sector, but still have a valid boot sector. this is done
14466 ; by examining the media descriptor in the boot sector.
14467
14468 ; 19/10/2022
14469 check_bpb_mediabyte:
14470     mov     al, [disksector+15h]
14471     ; [disksector+EXT_BOOT.BPB+EBPB.MEDIADESCRIPTOR]
14472     push    ax ; 02/09/2023
14473     and     al, 0F0h
14474     cmp     al, 0F0h        ; allow for strange media
14475     pop     ax ; 02/09/2023
14476     jnz     short invalidbootsec
14477
14478 ; there were some (apparently a lot of them) diskettes that had been formatted
14479 ; under dos 3.1 and earlier versions which have invalid bpbs in their boot
14480 ; sectors. these are specifically diskettes that were formatted in drives
14481 ; with one head, or whose side 0 was bad. these contain bpbs in the boot
14482 ; sector that have the sec/clus field set to 2 instead of 1, as is standard
14483 ; in dos. in order to support them, we have to introduce a "hack" that will
14484 ; help our build bpb routine to recognise these specific cases, and to
14485 ; set up out copy of the bpb accordingly.
14486 ; we do this by checking to see if the boot sector is off a diskette that
14487 ; is single-sided and is a pre-dos 3.20 diskette. if it is, we set the
14488 ; sec/clus field to 1. if not, we carry on as normal.
14489
14490 checksingledsided:
14491     mov     al, [disksector+15h]
14492     ; 02/09/2023
14493     ; al = [disksector+15h]
14494     cmp     al, 0F0h
14495     jz      short gooddsk
14496     test    al, 1
14497     jnz     short gooddsk
14498     cmp     word [disksector+8], 2E33h ; "3."
14499     jnz     short mustbearlier
14500     cmp     byte [disksector+0Ah], 32h ; "2"
14501     jnb     short gooddsk
14502
14503 ; we must have a pre-3.20 diskette. set the sec/clus field to 1
14504
14505 mustbearlier:
14506     mov     byte [disksector+0Dh], 1
14507     ; [disksector+EXT_BOOT.BPB+EBPB.SECTORSPERCLUSTER]
14508     jmp     short gooddsk
14509 ; -----

```

```

14510
14511
14512 000007FD 43
14513
14514
14515
14516 000007FE F8
14517
14518 000007FF C3
14519
14520
14521
14522
14523
14524
14525
14526
14527
14528
14529
14530
14531
14532
14533
14534
14535
14536
14537
14538
14539
14540
14541
14542
14543
14544
14545
14546
14547
14548
14549
14550
14551
14552
14553
14554
14555
14556
14557
14558
14559
14560
14561
14562
14563
14564
14565
14566
14567
14568
14569
14570
14571
14572
14573 00000800 57
14574 00000801 83C706
14575 00000804 8D36[5D01]
14576 00000808 B93500
14577
14578 0000080B FC
14579 0000080C F3A4
14580 0000080E 8B4CD3
14581
14582 00000811 31C0
14583 00000813 E308
14584 00000815 26894DE0
14585
14586 00000819 268945E2
14587
14588 0000081D 3944D6
14589 00000820 7410
14590
14591 00000822 83EF1C
14592
14593 00000825 B90C00
14594
14595 00000828 F3AA
14596 0000082A 48
14597 0000082B AB
14598
14599 0000082C AB
14600 0000082D 40
14601 0000082E B10C
14602
14603
14604 00000830 F3AA
14605
14606 00000832 5F
14607
14608
14609
14610 00000833 803E[9B04]01
14611 00000838 75C4
14612 0000083A E81200
14613 0000083D 7205
14614 0000083F C606[9B04]02
14615
14616 00000844 803E[7700]01
14617 00000849 75B3
14618 0000084B E83714
14619
14620
14621
14622
14623 0000084E C3
14624
14625
14626
14627
14628
14629
14630
14631
14632
14633

invalidbootsec:
    inc     bx                ; indicate that boot sector invalid
    ; 10/12/2022

movbpb_ret:
gooddisk:
    clc
err_ret:
    retn
; -----
; 10/12/2022
;err_ret:
;    retn

; ===== S U B   R O U T I N E =====
; 15/10/2022
;
; 'movbpb' moves the bpb read from the boot sector into registers for use by
; getbp routine at has1
;
; if the set_id_flag is 1, and if an extended boot record, then set volume
; serial number, volume label, file system id in bds according to
; the boot record. after that, this routine will set the set_id_flag to 2
; to signal that volume label is set already from the extended boot record
; (so, don't set it again by calling "set_volume_id" routine which uses
; the volume label in the root directory.)
; -----
; 10/03/2019 - Retro DOS v4.0
; 22/12/2023
%if 0
    ; 19/10/2022
movbpb:
    mov     dh, [disksector+0Dh]
    ; disksector+EXT_BOOT.BPB+EBPB.SECTORSPERCLUSTER]
    ; sectors per unit
    mov     bh, [disksector+11h]
    ; [disksector+EXT_BOOT.BPB+EBPB.ROOTENTRIES]
    ; number of directory entries
    mov     cx, [disksector+13h]
    ; [disksector+EXT_BOOT.BPB+EBPB.TOTALSECTORS]
    ; size of drive
    mov     ah, [disksector+15h]
    ; [disksector+EXT_BOOT.BPB+EBPB.MEDIADESCRIPTOR]
    ; media descriptor
    mov     al, [disksector+16h]
    ; [disksector+EXT_BOOT.BPB+EBPB.SECTORSPERFAT]
    ; number of fat sectors
    mov     bl, [disksector+18h]
    ; [disksector+EXT_BOOT.BPB+EBPB.SECTORSPERTRACK]
    ; sectors per track
    mov     dl, [disksector+1Ah]
    ; [disksector+EXT_BOOT.BPB+EBPB.HEADS]
    ; number of heads
%else
    ; 29/12/2023
    ; 22/12/2023 - Retro DOS v5.0
    ; (PCDOS 7.1 IBMBIO.COM, BIOSCODE:0814h)
    ;;;
movbpb:
    push    di
    add     di, 6              ; BDS+6 = BDS.BPB
    lea     si, [disksector+0Bh]
    mov     cx, 53             ; copy bios parameters block
    ; from BPB_BytsPerSec to (FAT32) BS_DrvNum (excluded)
    cld
    rep movsb
    mov     cx, [si-45]        ; si = disksector+64 -> 64-45 = 19
    ; disksector+19 = BPB_TotSec16
    xor     ax, ax
    jcxz    movbpb_bigdisk
    mov     [es:di-32], cx      ; write 16 bit total sectors
    ; to 32 bit total sectors field
    mov     [es:di-30], ax      ; BPB_TotalSec32+2 (BDS offset 29, BPB offset 23)
movbpb_bigdisk:
    cmp     [si-42], ax        ; BPB_FATSz16 = disksector+22
    jz      short movbpb_fat32
movbpb_fat:
    sub     di, 28             ; di = BDS offset 31 (BPB offset 25)
    ; 29/12/2023
    mov     cx, 12             ; clear 12 byte extended BDS (FAT32) fields
    ; (which are used only for FAT32 disks)
    rep stosb
    dec     ax                 ; -1 ; 0FFFFh
    stosw                      ; set BDS offset 43 (dword) to -1
    ; dword [BDS.BPB_FSInfo] = 0FFFFFFFFh
    stosw
    inc     ax                 ; ax = 0
    mov     cl, 12
    ;mov     cx, 12            ; clear BDS offset 47 to 59
    ; (BPB offset 41 to 53) (disksector offset 52 to 64)
    rep stosb
movbpb_fat32:
    pop     di
%endif
    ;;;
    cmp     byte [set_id_flag], 1 ; called by get_bpb?
    jnz     short movbpb_ret
    call    mov_media_ids
    jnb     short movbpb_conv ; conventional boot record?
    mov     byte [set_id_flag], 2 ; signals that volume id is set
movbpb_conv:
    cmp     byte [fhav96], 1
    jnz     short movbpb_ret
    call    resetchanged ; reset flags in bds to not fchanged.
    ; 10/12/2022
    ; cf = 0
;movbpb_ret:
;    clc
;    retn

; ===== S U B   R O U T I N E =====
; copy the boot_serial number, volume id, and filesystem id from the
; ***extended boot record*** in ds:disksector to the bds table pointed
; by es:di.
; in.) es:di -> bds
; ds:disksector = valid extended boot record.
; out.) vol_serial, bds_volid and bds_system_id in bds are set according to

```



```

14634 ; the boot record information.
14635 ; carry flag set if not an extended bpb.
14636 ; all registers saved except the flag.
14637
14638 ; 22/12/2023
14639 %if 0
14640 ; 19/10/2022
14641 mov_media_ids:
14642 cmp byte [disksector+26h], 29h
14643 ; [disksector+EXT_BOOT.SIG],
14644 ; EXT_BOOT_SIGNATURE
14645 jnz short mmi_not_ext
14646 push cx
14647 mov cx, [disksector+27h]
14648 ; [disksector+EXT_BOOT.SERIAL]
14649 mov [es:di+57h], cx ; [es:di+BDS.vol_serial]
14650 mov cx, [disksector+29h]
14651 ; [disksector+EXT_BOOT.SERIAL+2]
14652 mov [es:di+59h], cx ; [es:di+BDS.vol_serial+2]
14653 push di
14654 push si
14655 mov cx, 11 ; size_of_EXT_BOOT_VOL_LABEL
14656 mov si, disksector+2Bh
14657 ;mov si, (offset disksector+2Bh) ;
14658 ; disksector+EXT_BOOT.VOL_LABEL
14659 add di, 75 ; BDS.volid
14660 rep movsb
14661 ;mov cx, 8 ; size_of_EXT_SYSTEM_ID
14662 ; 10/12/2022
14663 mov cl, 8 ; cx = 8
14664 mov si, disksector+36h
14665 ;mov si, (offset disksector+36h) ; disksector+EXT_BOOT.SYSTEM_ID
14666 add di, 5 ; (BDS.filesys_id-BDS.volid)-size_of_EXT_BOOT_VOL_LABEL
14667 rep movsb
14668 pop si
14669 pop di
14670 pop cx
14671 ; 10/12/2022
14672 ; cf = 0
14673 ;clic ; this clic is not required (16/06/2019 - Erdogan Tan)
14674 ; (20/09/2022)
14675 retn
14676 %else
14677 ; 22/12/2023 - Retro DOS v5.0
14678 ; (PCDOS 7.1 IBMBIO.COM, BIOSCODE:0865h)
14679 ;;;
14680 mov_media_ids:
14681 cmp word [disksector+16h], 0 ; BPB.FATSz16
14682 jnz short mmi_chk_fat
14683 cmp byte [disksector+42h], 29h
14684 ; [disksector+FAT32_EXT_BOOT.SIG],
14685 ; EXT_BOOT_SIGNATURE
14686 jmp short mmi_chk_fat32
14687 mmi_chk_fat:
14688 cmp byte [disksector+26h], 29h
14689 ; [disksector+EXT_BOOT.SIG],EXT_BOOT_SIGNATURE
14690 mmi_chk_fat32:
14691 jnz short mmi_not_ext
14692 push cx
14693 push ax
14694 push di
14695 push si
14696 push ds
14697 cmp word [disksector+16h], 0 ; BPB.FATSz16
14698 jnz short mmi_fat
14699
14700 mmi_fat32:
14701 ; FAT32 file system
14702 ;lds cx, dword ptr ds:disksector+43h
14703 ;lds cx, [disksector+43h] ; BS_FAT32_VolID
14704 ;mov si, disksector+47h ; BS_FAT32_VolLab
14705 ;mov ax, disksector+52h ; BS_FAT32_FilSysType
14706 jmp short mmi_do
14707
14708 mmi_fat:
14709 ;lds cx, dword ptr ds:disksector+27h
14710 ;lds cx, [disksector+27h] ; BS_VolID
14711 ;mov si, disksector+2Bh ; BS_VolLab
14712 ;mov ax, disksector+36h ; BS_FilSysType
14713 mmi_do:
14714 mov [es:di+89h], cx ; [es:di+BDS.vol_serial]
14715 ; (BDS offset 137)
14716 mov [es:di+8Bh], ds ; [es:di+BDS.vol_serial+2]
14717 pop ds
14718 mov cx, 11
14719 add di, 125 ; di = di+125 = BDS.volid
14720 rep movsb
14721 mov cl, 8 ; di = di+136
14722 mov si, ax ; BS_FilSysType or BS_FAT32_FilSysType
14723 add di, 5 ; di = di+141 = BDS.filesys_id
14724 rep movsb
14725 pop si
14726 pop di
14727 pop ax
14728 pop cx
14729 ;clic ; this clic is not required (16/06/2019 - Erdogan Tan)
14730 ; (20/09/2022 - 27/06/2023) MSDOS 6.21 .. PCDOS 7.1
14731 retn
14732 %endif
14733 ;;;
14734 ; -----
14735
14736 mmi_not_ext:
14737 stc
14738 retn
14739
14740 ; ===== S U B R O U T I N E =====
14741
14742 ; 15/10/2022
14743 ; -----
14744 ; read in the fat sector and get the media byte from it.
14745 ; input : es:di -> bds
14746 ; output:
14747 ; carry set if an error occurs, ax contains error code.
14748 ; otherwise, ah contains media byte on exit
14749 ; -----
14750
14751 readfat:
14752 ;mov dh, 0
14753 ; 10/12/2022
14754 xor dh, dh
14755 mov cx, 2 ; head 0
14756 ; cylinder 0, sector 2
14757 call read_sector

```

```

14758 000008B1 7202          jb      short bad_fat_ret
14759 000008B3 8A27          mov     ah, [bx]      ; mediabyte
14760                                bad_fat_ret:
14761 000008B5 C3              retn
14762
14763 ; ===== S U B   R O U T I N E =====
14764
14765 ; 15/10/2022
14766
14767 ; -----
14768 ; read a single sector into the temp buffer.
14769 ; perform three retries in case of error.
14770 ;   inputs: es:[di].bds_drivenum has physical drive to use
14771 ;           cx has sector and cylinder
14772 ;           dh has head
14773 ;           es:di has bds
14774 ;           ds has Bios_Data
14775 ;
14776 ;   outputs:      carry clear
14777 ;               Bios_Data:bx point to sector
14778 ;               (note: some callers assume location of buffer)
14779 ;
14780 ;               carry set
14781 ;               ax has rom error code
14782 ;
14783 ; register bp is preserved.
14784 ; -----
14785
14786 ; 10/03/2019 - Retro DOS v4.0
14787 ; 22/12/2023 - Retro DOS v5.0
14788
14789 ; 19/10/2022
14790 read_sector:
14791 000008B6 55          push    bp
14792 000008B7 BD0300      mov     bp, 3          ; make 3 attempts
14793 000008BA 268A5504    mov     dl, [es:di+4]    ; [es:di+BDS.drivenum]
14794 000008BE BB[5201]    mov     bx, disksector ; get es:bx to point to      buffer
14795
14796 000008C1 06          rd_ret:  push    es
14797 000008C2 1E          push    ds
14798 000008C3 07          pop     es
14799 000008C4 B80102    mov     ax, 201h
14800 000008C7 CD13      int     13h          ; DISK - READ SECTORS INTO MEMORY
14801                                ; AL = number of sectors to read, CH = track, CL = sector
14802                                ; DH = head, DL      = drive, ES:BX -> buffer to fill
14803                                ; Return: CF set on error, AH =      status, AL = number of sectors read
14804 000008C9 07          pop     es
14805 000008CA 734A      jnb     short okret2
14806
14807 000008CC E81205    call    again          ; resetdisk, decrement      bp, preserve ax
14808 000008CF 7442      jz      short err_rd_ret
14809
14810 ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
14811 000008D1 26F6453F01  test    byte [es:di+3Fh], 1
14812 ; test byte [es:di+23h], 1
14813 ; test byte ptr [es:di+23h], 1 ; [es:di+BDS.flags]
14814 ; fnon_removable
14815 000008D6 75E9      jnz     short rd_ret
14816 000008D8 803E[A905]00  cmp     byte [media_set_for_format], 0
14817 000008DD 7510      jnz     short rd_skip1_dpt
14818 000008DF 50          push    ax
14819 000008E0 1E          push    ds          ; for retry, set the head settle time to 0Fh
14820 000008E1 C536[2D01]  lds     si, [dpt]
14821 ; mov al, [si+9] ; [si+DISK_PARMS.DISK_HEAD_STTL]
14822 ; mov byte [si+9], 15 ; [si+DISK_PARMS.DISK_HEAD_STTL]
14823 ; ; NORMSETTLE
14824 ; 12/12/2022
14825 000008E5 B00F      mov     al, 15
14826 000008E7 864409    xchg    al, [si+9]
14827 ;
14828 000008EA 1F          pop     ds
14829 000008EB A2[2A01]    mov     [save_head_sttl], al
14830 000008EE 58          pop     ax
14831
14832 000008EF 06          rd_skip1_dpt: push    es
14833 000008F0 1E          push    ds
14834 000008F1 07          pop     es
14835 000008F2 B80102    mov     ax, 201h
14836 000008F5 CD13      int     13h          ; DISK - READ SECTORS INTO MEMORY
14837                                ; AL = number of sectors to read, CH = track, CL = sector
14838                                ; DH = head, DL      = drive, ES:BX -> buffer to fill
14839                                ; Return: CF set on error, AH =      status, AL = number of sectors read
14840 000008F7 07          pop     es
14841 000008F8 9C          pushf
14842 000008F9 803E[A905]00  cmp     byte [media_set_for_format], 0
14843 000008FE 750E      jnz     short rd_skip2_dpt
14844 00000900 50          push    ax
14845 00000901 A0[2A01]    mov     al, [save_head_sttl]
14846 00000904 1E          push    ds
14847 00000905 C536[2D01]  lds     si, [dpt]
14848 00000909 884409    mov     [si+9], al    ; [si+DISK_PARMS.DISK_HEAD_STTL]
14849 0000090C 1F          pop     ds
14850 0000090D 58          pop     ax
14851
14852 0000090E 9D          rd_skip2_dpt: popf
14853 0000090F 7305      jnb     short okret2
14854 00000911 EBB9      jmp     short rd_rty
14855 ; -----
14856
14857 err_rd_ret:
14858 00000913 B2FF      mov     dl, 0FFh      ; make sure we ask rom if media      has changed
14859                                ; return error
14860 00000915 F9          stc
14861
14862 ; update information pertaining to last drive accessed, time of access, last
14863 ; track accessed in that drive.
14864
14865 okret2:
14866 00000916 8816[7600]  mov     [step_drv], dl ; set up for head settle logic in disk
14867 0000091A 8816[1E01]  mov     [tim_drv], dl  ; save drive last accessed
14868
14869 ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
14870 0000091E 26886D78    mov     [es:di+78h], ch
14871 ; mov [es:di+46h], ch ; [es:di+BDS.track]
14872 ; save last track accessed on this drive
14873 ; preserve flags in case error occurred
14874 00000922 9C          pushf
14875 00000923 E89B04    call    set_tim
14876 00000926 9D          popf          ; restore flags
14877 00000927 5D          pop     bp
14878 00000928 C3          retn
14879
14880 ; -----
14881 ; disk open/close routines

```

```

14882 ;-----
14883
14884 dsk_open: ; 2C7h:80Ah = 70h:2D7Ah
14885 00000929 803E[7700]00 cmp byte [fhav96], 0
14886 0000092E 7407 jz short dsk_open_exit ; done if no changeline support
14887 00000930 E871FC call SetDrive ; get bds for drive
14888 ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
14889 00000933 26FF453C inc word [es:di+3Ch] ; [es:di+BDS.opcnt] ; BDS offset 60
14890 ;inc word [es:di+20h] ; [es:di+BDS.opcnt]
14891 dsk_open_exit:
14892 ; 10/12/2022
14893 ; cf = 0
14894 ;clc ; CF is already ZERO here (18/09/2022, MSDOS 5.0 IO.SYS)
14895 ; (19/07/2019 - Erdogan Tan - MSDOS 6.0 IO.SYS - retrodos4.s)
14896 00000937 C3 retn
14897 ; -----
14898
14899 dsk_close: ; 2C7h:81Ah = 70h:2D8Ah
14900 00000938 803E[7700]00 cmp byte [fhav96], 0
14901 0000093D 740E jz short exitjx ; done if no changeline support
14902 0000093F E862FC call SetDrive ; get bds for drive
14903 ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
14904 00000942 26837D3C00 cmp word [es:di+3Ch], 0 ; [es:di+BDS.opcnt] ; BDS off 60
14905 ;cmp word [es:di+20h], 0 ; [es:di+BDS.opcnt]
14906 00000947 7404 jz short exitjx ; watch out for wrap
14907 ; 22/12/2023
14908 00000949 26FF4D3C dec word [es:di+3Ch]
14909 ;dec word [es:di+20h]
14910 exitjx:
14911 ; 10/12/2022
14912 ; cf = 0
14913 ;clc ; CF is already ZERO here (18/09/2022, MSDOS 5.0 IO.SYS)
14914 ; (19/07/2019 - Erdogan Tan - MSDOS 6.0 IO.SYS - retrodos4.s)
14915 0000094D C3 retn
14916 ; -----
14917 ;
14918 ; disk removable routine
14919 ; -----
14920
14921 ; al is unit #
14922 dsk_rem: ; 2C7h:831h = 70h:2DA1h
14923 0000094E E853FC call SetDrive ; get bds for this drive
14924 ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
14925 ;test byte [es:di+BDS.flags], fnon_removable
14926 00000951 26F6453F01 test byte [es:di+3Fh], 1 ; [es:di+BDS.flags], fnon_removable
14927 00000956 74F5 jz short exitjx
14928 ;test byte [es:di+23h], 1 ; [es:di+BDS.flags], fnon_removable
14929 ;jnz short x_bus_exit ; non_rem
14930 ;jnz short non_rem ; 15/10/2022
14931 ; 10/12/2022
14932 ; cf = 0
14933 ;clc ; CF is already ZERO here
14934 ; 15/10/2022
14935 ;retn
14936 ; -----
14937
14938 non_rem:
14939 x_bus_exit: mov ah, 3 ; 2C7h:83Dh = 0070h:2DADh
14940 00000958 B403 ; return busy status
14941 stc
14942 0000095A F9
14943 dsk_ret:
14944 0000095B C3 retn
14945 ; -----
14946 ;
14947 ; disk i/o routines
14948 ; -----
14949
14950 dsk_writv: ; 2C7h:841h = 70h:2DB1h
14951 ;mov word [wrtverify], 103h
14952 ; 19/10/2022
14953 0000095C C706[2001]0301 mov word [rflag], 103h
14954 ;mov word ptr ds:rflag, 103h ; write and verify
14955 00000962 EB06 jmp short dsk_cl
14956 ; -----
14957
14958 dsk_writ: ; 2C7h:849h = 70h:2DB9h
14959 ;mov word [wrtverify], 3
14960 ; 19/10/2022
14961 00000964 C706[2001]0300 mov word [rflag], 3
14962 ;mov word ptr ds:rflag, 3 ; romwrite
14963 dsk_cl: call diskio ; romwrite
14964 0000096A E8A400
14965 ; -----
14966
14967 dsk_io:
14968 0000096D 73EC jnb short dsk_ret
14969 0000096F E965F7 jmp bc_err_cnt
14970 ; -----
14971
14972 dsk_read: ; 2C7h:857h = 70h:2DC7h
14973 00000972 E89700 call diskrd
14974 00000975 EBF6 jmp short dsk_io
14975 ; ===== S U B R O U T I N E =====
14976
14977 ; 15/10/2022
14978 ; 10/03/2019 - Retro DOS v4.0
14979 ; 22/12/2023 - Retro DOS v5.0
14980 ; -----
14981 ; miscellaneous odd jump routines.
14982 ; moved out of mainline for speed.
14983
14984 ; if we have a system where we have virtual drives, we need
14985 ; to prompt the user to place the correct disk in the drive.
14986 ;
14987 ; assume es:di -> bds, ds:->Bios_Data
14988 ; -----
14989
14990 ; 19/10/2022
14991 checksingle:
14992 ; 19/10/2022
14993 00000977 50 push ax
14994 00000978 53 push bx
14995 ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
14996 00000979 268B5D3F mov bx, [es:di+3Fh] ; [es:di+BDS.flags]
14997 ;mov bx, [es:di+23h] ; [es:di+BDS.flags]
14998
14999 ; if hard drive, cannot change disk.
15000 ; if current owner of physical drive, no need to change diskette.
15001 test bl, 21h ; fnon_removable|fi_own_physical
15002 jnz short singleret
15003 0000097D F6C321 test bl, 10h ; fi_am_mult
15004 00000980 7573
15005 00000982 F6C310

```

```

15006                                     ; is there a drive sharing this      physical drive?
15007 00000985 746E                     jz      short singleret
15008
15009 ; look for the previous owner of this physical drive
15010 ; and reset its ownership flag.
15011
15012 00000987 268A4504                   mov     al, [es:di+4] ; [es:di+BDS.drivenum]
15013                                     ; get physical drive number
15014 0000098B 06                         push    es          ; preserve pointer to current bds
15015 0000098C 57                         push    di
15016 0000098D C43E[1901]                les     di, [start_bds] ; get first bds
15017
15018 00000991 26384504                   scan_list: cmp     [es:di+4], al
15019 00000995 7553                       jnz     short scan_skip ; Not our drive. Try next bds.
15020 00000997 B320                       mov     bl, 20h ; ' ' ; fi_own_physical ; test ownership flag
15021                                     ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
15022 00000999 26845D3F                   test    [es:di+3Fh], bl ; [es:di+BDS.flags]
15023                                     ;test [es:di+23h], bl
15024 0000099D 744B                       jz      short scan_skip ; he doesn't own it either. continue
15025 0000099F 26305D3F                   xor     [es:di+3Fh], bl
15026                                     ;xor [es:di+23h], bl ; reset ownership flag
15027 000009A3 5F                         pop     di          ; restore pointer to current bds
15028 000009A4 07                         pop     es
15029 000009A5 26085D3F                   or      [es:di+3Fh], bl
15030                                     ;or [es:di+23h], bl ; ; set ownership flag
15031
15032                                     ; we examine the fsetowner flag. if it is set, then we are using the code in
15033 ; checksingle to just set the owner of a drive. we must not issue the prompt
15034 ; in this case.
15035 000009A9 803E[7A00]01                cmp     byte [fsetowner], 1
15036 000009AE 7517                       jnz     short not_fsetowner
15037                                     ;cmp byte ptr es:[di+4], 0 ; are we handling drive number 0 ?
15038 000009B0 26807D0400                 cmp     byte [es:di+4], 0
15039 000009B5 753E                       jnz     short singleret
15040 000009B7 268A4505                   mov     al, [es:di+5]
15041                                     ;mov al, es:[di+5] ; [es:di+BDS.drivelet]
15042                                     ; get the DOS drive letter
15043 000009BB 06                         push    es
15044 000009BC 8E06[1A00]                 mov     es, [zeroseg]
15045 000009C0 26A20405                   mov     [es:LSTDRV], al
15046                                     ;mov es:504h, al ; [es:LSTDRV]
15047                                     ; set up sdsb
15048 000009C4 07                         pop     es          ; restore bds pointer
15049 000009C5 EB2E                       jmp     short singleret
15050
15051 ; -----
15052 ; to support "backward" compatibility with ibm's "single drive status byte"
15053 ; we now check to see if we are in a single drive system and the application
15054 ; has "cleverly" diddled the sdsb
15055
15056 not_fsetowner:
15057 000009C7 803E[7800]02                cmp     byte [single], 2 ; if (single_drive_system)
15058 000009CC 7517                       jnz     short ignore_sdsb
15059 000009CE 50                         push    ax
15060 000009CF 268A4505                   mov     al, [es:di+5] ; if (curr_drv == req_drv)
15061 000009D3 88C4                       mov     ah, al
15062 000009D5 06                         push    es
15063 000009D6 8E06[1A00]                 mov     es, [zeroseg]
15064 000009DA 2686060405                 xchg    al, [es:LSTDRV]
15065                                     ;xchg al, es:504h ; [es:LSTDRV]
15066                                     ; then swap(curr_drv, req_drv)
15067 000009DF 07                         pop     es
15068 000009E0 38C4                       cmp     ah, al ; else
15069 000009E2 58                         pop     ax ; swap(curr_drv, req_drv)
15070 000009E3 7410                       jz      short singleret ; issue swap_dsk_msg
15071
15072 000009E5 E8B310                       ignore_sdsb: call    swpdsk
15073 000009E8 EB0B                       jmp     short singleret
15074
15075 ; -----
15076 scan_skip:
15077 000009EA 26C43D                       les     di, [es:di]
15078                                     ;les di, es:[di] ; [es:di+BDS.link]
15079                                     ; go to next bds
15080 000009ED 83FFFF                       cmp     di, 0FFFFh ; -1 ; end of list?
15081 000009F0 759F                       jnz     short scan_list ; continue until hit end of list
15082 000009F2 F9                         stc
15083 000009F3 5F                         pop     di
15084 000009F4 07                         pop     es ; restore current bds
15085
15086 000009F5 5B                         singleret: pop     bx
15087 000009F6 58                         pop     ax
15088 000009F7 C3                         retn
15089
15090 ; 22/12/2023
15091 %if 0
15092 ; -----
15093 baddrive:
15094                                     mov     al, 8 ; sector not found
15095                                     jmp     short baddrive_ret
15096
15097 %endif
15098
15099 ; -----
15100 unformatteddrive:
15101                                     mov     al, 7 ; unknown media
15102 000009F8 B007                       ;baddrive_ret: stc
15103
15104 000009FA F9                         ; -----
15105
15106 ioret:
15107                                     retn
15108 000009FB C3
15109
15110 ; -----
15111
15112                                     ; 22/12/2023 - Retro DOS v5.0
15113                                     ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0A1Bh
15114
15115 000009FC 10                       LBA_Packet: db 16 ; ...
15116                                     ; DAP buffer
15117 000009FD 00                       db 0
15118 000009FE 0000                       dap_block_cnt: dw 0 ; ...
15119 00000A00 00000000                   dap_trans_buf: dd 0 ; ...
15120 00000A04 00000000                   dap_lba_value: dd 0 ; ...
15121 00000A08 00000000                   dd 0
15122
15123 ; -----
15124
15125 ; 15/10/2022
15126
15127 ; -----
15128 ; disk i/o handler
15129 ;

```

```

15130 ; al = drive number (0-6)
15131 ; ah = media descriptor
15132 ; cx = sector count
15133 ; dx = first sector (low)
15134 ; [start_sec_h] = first sector (high) 32 bit calculation.
15135 ; ds = cs
15136 ; es:di = transfer address
15137 ; [rflag]=operation (2=read, 3=write)
15138 ; [verify]=1 for verify after write
15139 ;
15140 ; if successful carry flag = 0
15141 ; else cf=1 and al contains error code
15142 ; -----
15143 ;
15144 ; 12/12/2023
15145 ; ds = biosdata segment (cs = bioscode segment)
15146 diskrd:
15147 ;mov ds:rflag, 2 ; romread
15148 ; 19/10/2022
15149 mov byte [rflag], 2 ; romread
15150
15151 ; ===== S U B R O U T I N E =====
15152 ;
15153 ; 22/12/2023 - Retro DOS v5.0
15154 ; PC DOS 7.1 IBMBIO.COM - BIOS CODE:0A30h
15155 ; 22/12/2023
15156 %if 0
15157 ; 19/10/2022
15158 diskio:
15159 mov bx, di ; es:bx= transfer address
15160 mov [xfer_seg], es ; save transfer segment
15161 call SetDrive
15162 mov al, [es:di+10h] ; [es:di+BDS.media]
15163 mov [medbyt], al
15164 jcxz short ioret
15165 jcxz ioret
15166
15167 ; see if the media is formatted or not by checking the flags field in
15168 ; in the bds. if it is unformatted we cannot allow i/o, so we should
15169 ; go to the error exit at label unformatteddrive.
15170
15171 test byte [es:di+24h], 2
15172 ;test byte ptr es:[di+24h], 2 ; [es:di+BDS.flags+1]
15173 ;unformatted_media
15174 jnz short unformatteddrive
15175 mov [secnt], cx ; save sector count
15176 mov [spsav], sp ; save sp
15177
15178 ; ensure that we are trying to access valid sectors on the drive
15179
15180 mov ax, dx
15181 xor si, si ; 0
15182 add dx, cx
15183 adc si, 0
15184 ; 02/09/2023 (PC DOS 7.1)
15185 rcl si, 1
15186 cmp word [es:di+0Eh], 0 ; [es:di+BDS.totalsecs16]
15187 ; 32 bit sector ?
15188 jz short sanity32
15189 ;cmp si, 0
15190 ; 02/09/2023
15191 or si, si
15192 jnz short baddrive
15193 cmp dx, [es:di+0Eh] ; [es:di+BDS.totalsecs16]
15194 ja short baddrive
15195 jmp short sanityyok
15196 ; -----
15197
15198 sanity32:
15199 add si, [start_sec_h]
15200 cmp si, [es:di+10h] ; [es:di+BDS.totalsecs32+2]
15201 jb short sanityyok
15202 ja short baddrive
15203 cmp dx, [es:di+18h] ; [es:di+BDS.totalsecs32]
15204 ja short baddrive
15205
15206 sanityyok:
15207 mov dx, [start_sec_h]
15208 add ax, [es:di+17h] ; [es:di+BDS.hiddensecs]
15209 adc dx, [es:di+19h] ; [es:di+BDS.hiddensecs+2]
15210
15211 ; now dx;ax have the physical first sector.
15212 ; since the following procedures is going to destroy ax, let's
15213 ; save it temporarily to saved_word.
15214 mov [saved_word], ax ; save the sector number (low)
15215
15216 ; set up pointer to disk base table in [dpt]. we cannot assume that iosetup
15217 ; will do it because we will skip the set up stuff with hard disks.
15218
15219 push es
15220 ;mov es, [zeroseg]
15221 ; 02/09/2023
15222 xor si, si ; 0
15223 mov es, si
15224 les si, [es:DSKADR]
15225 ;les si, es:78h ; [es:DSKADR]
15226 ; current disk parm table
15227 mov [dpt], si
15228 mov [dpt+2], es
15229 pop es
15230 test byte [es:di+23h], 1 ; [es:di+BDS.flags]
15231 ; fnon_removable
15232 jnz short skip_setup
15233 call checksingle
15234
15235 ; check to see if we have previously noted a change line. the routine
15236 ; returns if everything is ok. otherwise, it pops off the stack and returns
15237 ; the proper error code.
15238
15239 cmp byte [fhave96], 0 ; do we have changeline support?
15240 jz short diskio_nochangeline ; brif not
15241 call checklatchio ; will do a sneaky pop stack return
15242 ; if a disk error occurs
15243 diskio_nochangeline:
15244 call iosetup ; set up tables and variables for i/o
15245
15246 ; now the settle values are correct for the following code
15247
15248 skip_setup:
15249
15250 ; 32 bit sector calculation.
15251 ; dx:[saved_word] = starting sector number.
15252
15253 mov ax, dx

```

```

15254      xor     dx, dx
15255      ;div    word [es:di+13h] ; [es:di+BDS.secptrack]
15256      ; divide by sec per track
15257      ; 02/09/2023
15258      mov     cx, [es:di+13h]
15259      div     cx
15260      mov     [temp_h], ax
15261      mov     ax, [saved_word]
15262      div     cx ; 02/09/2023
15263      ;div    word [es:di+13h] ; [es:di+BDS.secptrack]
15264      ; now, [temp_h]:ax = track #, dx = sector
15265      ; inc     dl ; sector number is 1 based.
15266      ; 18/12/2022
15267      inc     dx
15268      mov     [cursec], dl ; save current sector
15269      mov     cx, [es:di+15h] ; es:di+BDS.heads]
15270      ; get number of heads
15271      push    ax
15272      xor     dx, dx
15273      mov     ax, [temp_h] ; divide tracks by heads per cylinder
15274      div     cx
15275      mov     [temp_h], ax
15276      pop     ax
15277      div     cx ; now, [temp_h]:ax = cylinder #, dx = head
15278      cmp     word [temp_h], 0
15279      ja      short baddrive_brdg
15280      cmp     ax, 1024 ; 2^10 currently maxium for track #.
15281      ja      short baddrive_brdg
15282      mov     [curhd], dl ; save current head
15283      mov     [curtrk], ax ; save current track
15284
15285      ; we are now set up for the i/o. normally, we consider the dma boundary
15286      ; violations here. not true. we perform the operation as if everything is
15287      ; symmetric; let the int 13 handler worry about the dma violations.
15288
15289      mov     ax, [seccnt]
15290      call    block ; (cas - call/ret)
15291      ;call    done
15292      ;retn
15293      ; 18/12/2022
15294      jmp     done
15295
15296      %else
15297      ;;; ; 22/12/2023
15298      diskio:
15299      mov     bx, di ; al = drive number
15300      ; cx = sector count
15301      ; dx = first sector (low)
15302      ; [start_sec_h] = first sector (high)
15303      ;
15304      mov     [xfer_seg], es ; es:bx = transfer address
15305      call    SetDrive ; save transfer segment
15306      mov     al, [es:di+10h] ; [es:di+BDS.media]
15307      mov     [medbyt], al
15308      jcxz    ioret
15309
15310      ; see if the media is formatted or not by checking the flags field in
15311      ; in the bds. if it is unformatted we cannot allow i/o, so we should
15312      ; go to the error exit at label unformatteddrive.
15313
15314      test    byte [es:di+40h], 2 ; [es:di+BDS.flags+1]
15315      ; unformatted_media
15316      jnz     short unformatteddrive
15317      mov     [seccnt], cx ; save sector count
15318      mov     [spsav], sp ; save sp
15319
15320      ; ensure that we are trying to access valid sectors on the drive
15321
15322      mov     ax, dx
15323      xor     si, si ; 0
15324      add     dx, cx
15325      rcl     si, 1
15326      cmp     word [es:di+0Eh], 0 ; [es:di+BDS.totalsecs16]
15327      ; > 32 bit sector ?
15328      jz      short sanity32
15329      or      si, si
15330      jnz     short baddrive
15331      cmp     dx, [es:di+0Eh] ; [es:di+BDS.totalsecs16]
15332      ; ja      short baddrive
15333      ; jmp     short sanityok
15334      ; 22/12/2023
15335      jna     short sanityok
15336      ; 29/12/2023
15337      ; 22/12/2023
15338      ; %if 1
15339      ; -----
15340
15341      baddrive:
15342      mov     al, 8 ; sector not found
15343      jmp     short baddrive_ret
15344      ; -----
15345      ; unformatteddrive:
15346      mov     al, 7 ; unknown media
15347      baddrive_ret:
15348      stc
15349      ; ioret:
15350      retn
15351      ; %endif
15352      ; -----
15353
15354      sanity32:
15355      add     si, [start_sec_h]
15356      cmp     si, [es:di+1Dh] ; [es:di+BDS.totalsecs32+2]
15357      jb      short sanityok
15358      ja      short baddrive
15359      cmp     dx, [es:di+1Bh] ; [es:di+BDS.totalsecs32]
15360      ja      short baddrive
15361      sanityok:
15362      mov     dx, [start_sec_h]
15363      add     ax, [es:di+17h] ; [es:di+BDS.hiddensecs]
15364      adc     dx, [es:di+19h] ; [es:di+BDS.hiddensecs+2]
15365
15366      ; now dx;ax have the physical first sector.
15367      ; since the following procedures is going to destroy ax, let's
15368      ; save it temporarily to saved_word.
15369
15370      mov     [saved_word], ax ; save the sector number (low)
15371
15372      ; set up pointer to disk base table in [dpt]. we cannot assume that iosetup
15373      ; will do it because we will skip the set up stuff with hard disks.
15374
15375      push    es
15376      xor     si, si ; 0
15377

```



```

15378 00000A73 8EC6          mov     es, si
15379                      ;les     si, dword ptr es:78h
15380 00000A75 26C4367800     les     si, [es:78h] ; INT 1Eh vector address
15381                      ; [es:DSKADR] - current disk parm table
15382 00000A7A 8936[2D01]     mov     [dpt], si
15383 00000A7E 8C06[2F01]     mov     [dpt+2], es
15384 00000A82 07              pop     es
15385 00000A83 26F6453F01     test    byte [es:di+3Fh], 1 ; [es:di+BDS.flags], fnon_removable
15386 00000A88 7510          jnz     short chk_13h_ext_flag
15387 00000A8A E8EAFE          call    checksingle
15388
15389                      ; check to see if we have previously noted a change line. the routine
15390                      ; returns if everything is ok. otherwise, it pops off the stack and returns
15391                      ; the proper error code.
15392
15393 00000A8D 803E[7700]00     cmp     byte [fhave96], 0 ; do we have changeline support?
15394 00000A92 7403          jz      short diskio_nochange_line ; brif not
15395 00000A94 E8D210          call    checklatchio ; will do a sneaky pop stack return
15396                      ; if a disk error occurs
15397
diskio_nochange_line:
15398 00000A97 E8E000          call    iosetup ; set up tables and variables for i/o
15399
chk_13h_ext_flag:
15400
15401 00000A9A 26F6454004     test    byte [es:di+40h], 4 ; [es:di+BDS.flags+1], fLBArw
15402                      ; LBA read/write flag
15403 00000A9F 7539          jnz     short set_lbarw_1
15404                      ; jmp skip_setup
15405                      ; 22/12/2023
15406
15407                      ; -----
15408                      ; now the settle values are correct for the following code
15409
15410 skip_setup:
15411
15412                      ; 32 bit sector calculation.
15413                      ; dx:[saved_word] = starting sector number.
15414
15415                      ; push bp ; ! (not necessary) ; 22/12/2023
15416 00000AA1 92          xchg     ax, dx ; mov ax, dx
15417 00000AA2 31D2          xor      dx, dx
15418 00000AA4 268B4D13     mov     cx, [es:di+13h] ; [es:di+BDS.secpertack]
15419                      ; divide by sec per track
15420 00000AA8 F7F1          div     cx
15421 00000AAA 95          xchg     ax, bp ; mov bp, ax
15422 00000AAB A1[9E04]     mov     ax, [saved_word]
15423 00000AAE F7F1          div     cx ; [es:di+BDS.secpertack]
15424                      ; now, bp:ax = track #, dx = sector
15425                      ; sector number is 1 based.
15426 00000AB0 42          inc     dx
15427 00000AB1 8816[3101]     mov     [cursec], dl ; save current sector
15428 00000AB5 268B4D15     mov     cx, [es:di+15h] ; [es:di+BDS.heads]
15429                      ; get number of heads
15430                      ; 22/12/2023
15431                      ; push ax ; *
15432 00000AB9 31D2          xor      dx, dx
15433 00000ABB 95          xchg     ax, bp ; bp = * ; divide tracks by heads per cylinder
15434 00000ABC F7F1          div     cx
15435 00000ABE 95          xchg     ax, bp ; ax = *, bp = **
15436                      ; pop ax ; *
15437 00000ABF F7F1          div     cx ; now, bp:ax = cylinder #, dx = head
15438 00000AC1 09ED          or      bp, bp ; ** = 0 ?
15439                      ; pop bp ; ! ; 22/12/2023
15440                      ; jnz short baddrive_brdg
15441                      ; 22/12/2023
15442 00000AC3 7586          jnz     short baddrive
15443
15444                      ; cmp ax, 1024 ; 2^10 currently maximum for track #.
15445                      ; jnb short baddrive_brdg
15446                      ; 22/12/2023
15447 00000AC5 80FC04     cmp     ah, 4 ; if ax >= 4*256 (1024)
15448 00000AC8 7381          jnb     short baddrive
15449
15450 00000ACA 8816[3201]     mov     [curhd], dl ; save current head
15451 00000ACE A3[3301]     mov     [curtrk], ax ; save current track
15452
15453                      ; we are now set up for the i/o. normally, we consider the dma boundary
15454                      ; violations here. not true. we perform the operation as if everything is
15455                      ; symmetric; let the int 13 handler worry about the dma violations.
15456
15457 00000AD1 A1[2201]     mov     ax, [seccnt]
15458 00000AD4 E81F01          call    block
15459                      ; call done
15460                      ; retn
15461                      ; 22/12/2023
15462 00000AD7 E9E500          jmp     done
15463
15464                      ; -----
15465
set_lbarw_1:
15466
15467 00000ADA A1[9E04]     mov     ax, [saved_word] ; check for mini disk
15468                      ; (logical dos drive/partition)
15469 00000ADD 26837D7901     cmp     word [es:di+79h], 1 ; [di+BDS.bdsminimini]
15470                      ; logical dos partition
15471 00000AE2 750F          jnz     short set_lbarw_2 ; not a logical dos partition/drive
15472 00000AE4 26837D7B00     cmp     word [es:di+7Bh], 0 ; [di+BDS.bdsmin_hidden_trks] (> 0)
15473 00000AE9 7408          jz      short set_lbarw_2
15474 00000AEB 26034517     add     ax, [es:di+17h] ; [es:di+BDS.hiddensecs]
15475 00000AEF 26135519     adc     dx, [es:di+19h] ; [es:di+BDS.hiddensecs+2]
15476
set_lbarw_2:
15477
15478 00000AF3 2EA3[040A]     mov     [cs:dap_lba_value], ax
15479 00000AF7 2E8916[060A]     mov     [cs:dap_lba_value+2], dx
15480 00000AFC 2E891E[000A]     mov     [cs:dap_trans_buf], bx
15481 00000B01 A1[A804]     mov     ax, [xfer_seg]
15482 00000B04 2EA3[020A]     mov     [cs:dap_trans_buf+2], ax
15483 00000B08 A1[2201]     mov     ax, [seccnt]
15484 00000B0B 2EA3[FE09]     mov     [cs:dap_block_cnt], ax
15485 00000B0F BD0500     mov     bp, 5
15486 00000B12 892E[A304]     mov     [vretry_cnt], bp ; verify op. retry cnt for write-verify
15487 00000B16 892E[A504]     mov     [soft_ecc_cnt], bp ; soft ecc error retry count
15488
set_lbarw_3:
15489
15490 00000B1A 268A5504     mov     dl, [es:di+4] ; [es:di+BDS.drivenum]
15491 00000B1E 8A26[2001]     mov     ah, [rflag] ; get read/write indicator
15492 00000B22 80C440     add     ah, 40h
15493 00000B25 30C0     xor     al, al
15494 00000B27 1E          push    ds
15495 00000B28 0E          push    cs
15496 00000B29 1F          pop     ds
15497 00000B2A BE[FC09]     mov     si, LBA_Packet
15498 00000B2D CD13     int     13h ; LBA read/write
15499 00000B2F 1F          pop     ds
15500 00000B30 731A     jnc     short set_lbarw_7
15501 00000B32 E8AC02     call    again

```

```

15502      set_lbarw_9:
15503      jnz      short set_lbarw_4
15504      jmp      harderr
15505      ; -----
15506
15507      set_lbarw_4:
15508      ;set_lbarw_9:      ; 22/12/2023
15509      cmp      ah, 0CCh      ; write fault (hard disk)
15510      jnz      short set_lbarw_5
15511      mov      bp, 1
15512      jmp      short set_lbarw_6
15513      ; 17/04/2024
15514      jmp      short set_lbarw_3
15515      ; -----
15516
15517      set_lbarw_5:
15518      set_lbarw_10:      ; 22/12/2023
15519      mov      word [soft_ecc_cnt], 5 ; soft ecc error retry count
15520
15521      set_lbarw_6:
15522      set_lbarw_11:
15523      jmp      short set_lbarw_3
15524      ; -----
15525
15526      set_lbarw_7:
15527      cmp      word [rflag], 103h
15528      jnz      short set_lbarw_12
15529      mov      ah, 44h
15530      push     ds
15531      push     cs
15532      pop      ds
15533      int      13h      ; DISK - IBM/MS Extension - VERIFY SECTORS
15534      ; (DL - drive, [SI - disk address packet])
15535      pop      ds
15536      jnc      short set_lbarw_12
15537      cmp      ah, 11h      ; ECC corrected data error (soft error - retried OK )
15538      jnz      short set_lbarw_8
15539      dec      word [soft_ecc_cnt]
15540      ;set_lbarw_8:
15541      jz      short set_lbarw_12
15542      set_lbarw_8:
15543      call     ResetDisk
15544      cmp      ah, 11h
15545      jz      short set_lbarw_11
15546      dec      word [vretry_cnt]
15547      jnz      short set_lbarw_9
15548      jmp      harderr
15549      ; 22/12/2023
15550      jmp      short set_lbarw_9
15551      ; -----
15552      ; 22/12/2023
15553      ;set_lbarw_9:
15554      cmp      ah, 0CCh
15555      jnz      short set_lbarw_10
15556      mov      bp, 1
15557      jmp      short set_lbarw_11
15558      ; -----
15559      ; 22/12/2023
15560      ;set_lbarw_10:
15561      mov      word [soft_ecc_cnt], 5 ; soft ecc error retry count
15562      ;set_lbarw_11:
15563      jmp      short set_lbarw_3
15564      ; -----
15565
15566      set_lbarw_12:
15567      xor      ax, ax
15568      skip_dpt_setting:      ; 23/12/2023
15569      retn
15570      ;;;      ; 22/12/2023
15571      %endif
15572      ; -----
15573
15574      ; 22/12/2023
15575      ;baddrive_brdg:
15576      jmp      baddrive
15577
15578      ; ===== S U B   R O U T I N E =====
15579
15580      ; -----
15581      ; set the drive-last-accessed flag for diskette only.
15582      ; we know that the hard disk will not be removed.
15583      ; es:di -> current bds.
15584      ; ds -> Bios_Data
15585      ; ax,cx,si are destroyed.
15586      ; -----
15587
15588      ; 23/12/2023 - Retro DOS v5.0
15589
15590      ; 19/10/2022
15591      iosetup:
15592      mov      al, [es:di+4] ; [es:di+BDS.drivenum]
15593      mov      [tim_drv], al ; save drive letter
15594
15595      ; determine proper head settle values
15596
15597      cmp      byte [media_set_for_format], 0
15598      jnz      short skip_dpt_setting
15599      mov      al, [eot]      ; fetchup eot before changing ds
15600      push     ds
15601      lds      si, [dpt]      ; get pointer to disk base table
15602      mov      [si+4], al
15603
15604      ;; 23/12/2023
15605      mov      ah, al
15606      mov      al, [si+10]      ; [si+DISK_PARMS.DISK_MOTOR_STRT]
15607      mov      ah, [si+4]      ; [si+DISK_PARMS.DISK_EOT]
15608      pop      ds
15609      mov      [motorstartup], al
15610      mov      [save_eot], ah
15611      ; 06/04/2024
15612      mov      ah, [si+10]
15613      pop      ds
15614      mov      [motorstartup], ah
15615      mov      [save_eot], al
15616
15617      ; for 3.5" drives, both external as well as on the k09, we need to set the
15618      ; motor start time to 4. this checking for every i/o is going to affect
15619      ; performance across the board, but is necessary!!
15620
15621      push     ds
15622      lds      si, [dpt]      ; get pointer to disk base table
15623      ; 23/12/2023 - Retro DOS v5.0
15624      cmp      byte [es:di+3Eh], 2 ; (PCDOS 7.1 IBMBIO.COM)
15625

```



```

15626             ;cmp     byte [es:di+22h], 2 ; [es:di+BDS.formfactor]
15627             ; ffsmall
15628 00000BA8 7505     jnz     short motor_start_ok
15629 00000BAA B004     mov     al, 4
15630 00000BAC 86440A   xchg     al, [si+10] ; [si+DISK_PARMS.DISK_MOTOR_STRT]
15631 motor_start_ok:
15632
15633 ; ds:si now points to disk parameter table.
15634 ; get current settle and set fast settle
15635
15636             ;xor     al, al
15637             ;inc     al ; ibm wants fast settle to be 1
15638             ; 18/12/2022
15639 00000BAF 31C0     xor     ax, ax
15640 00000BB1 40         inc     ax
15641 00000BB2 864409   xchg     al, [si+9] ; [si+DISK_PARMS.DISK_HEAD_STTL]
15642             ; get settle and set up for fast
15643 00000BB5 1F         pop     ds
15644 00000BB6 A2[2701]   mov     [settlecurrent], al
15645 00000BB9 B00F     mov     al, 15 ; NORMSETTLE
15646             ; someone has diddled the settle
15647 00000BBB A2[2801]   mov     [settleslow], al
15648             ; 23/12/2023
15649 ;skip_dpt_setting:
15650 00000BBE C3         retn
15651
15652 ; ===== S U B R O U T I N E =====
15653
15654 ;-----
15655 ; set time of last access, and reset default values in the dpt.
15656 ;
15657 ; note: trashes (at least) si
15658 ;-----
15659
15660             ; 23/12/2023 - Retro DOS v5.0
15661
15662             ; 19/10/2022
15663 done:
15664             ;test    byte [es:di+23h], 1 ; [es:di+BDS.flags]
15665             ; fnon_removable
15666             ; 23/12/2023
15667 00000BBF 26F6453F01 test    byte [es:di+3Fh], 1 ; (PCDOS 7.1 IBMBIO.COM)
15668 00000BC4 752F     jnz     short ddbx ; do not set for non-removable media
15669 00000BC6 E8F801   call    set_tim
15670 ;diddleback:
15671 ; 09/12/2022
15672 diddle_back:
15673 00000BC9 9C         pushf
15674 00000BCA 803E[A905]00 cmp     byte [media_set_for_format], 0
15675 00000BCF 7523     jnz     short nodiddleback
15676 00000BD1 50         push    ax
15677 00000BD2 06         push    es
15678 00000BD3 C436[2D01]   les     si, [dpt]
15679 00000BD7 A0[2B01]   mov     al, [save_eot]
15680 00000BDA 26884404   mov     [es:si+4], al ; [es:si+DISK_PARMS.DISK_EOT]
15681 00000BDE A0[2701]   mov     al, [settlecurrent]
15682 00000BE1 8A26[2601]   mov     ah, [motorstartup]
15683 00000BE5 26884409   mov     [es:si+9], al ; [es:si+DISK_PARMS.DISK_HEAD_STTL]
15684 00000BE9 26C6440302   mov     byte [es:si+3], 2 ; [es:si+DISK_PARMS.DISK_SECTOR_SIZ]
15685 00000BEE 2688640A   mov     [es:si+0Ah], ah ; [es:si+DISK_PARMS.DISK_MOTOR_STRT]
15686 00000BF2 07         pop     es
15687 00000BF3 58         pop     ax
15688 nodiddleback:
15689 00000BF4 9D         popf
15690 ddbx:
15691 00000BF5 C3         retn
15692
15693 ; ===== S U B R O U T I N E =====
15694
15695 ;-----
15696 ; read the number of sectors specified in ax,
15697 ; handling track boundaries
15698 ; es:di -> bds for this drive
15699 ;-----
15700
15701             ; 23/12/2023 - Retro DOS v5.0
15702
15703             ; 19/10/2022
15704 block:
15705             or     ax, ax
15706 00000BF8 74FB     jz      short ddbx
15707             ; 23/12/2023
15708 00000BFA 26F6453F01 test    byte [es:di+3Fh], 1 ; (PCDOS 7.1 IBMBIO.COM)
15709             test    byte [es:di+23h], 1 ; [es:di+BDS.flags]
15710             ; fnon_removable
15711             jz      short block_floppy
15712
15713 ; check to see if multi track operation is allowed. if not
15714 ; we have to go to the block_floppy below to break up the operation.
15715
15716             test    byte [multrk_flag], 80h
15717             ;test    byte ptr ds:multrk_flag, 80h ; multrk_on
15718 00000C06 7406     jz      short block_floppy
15719 00000C08 E82800   call    Disk
15720 00000C0B 31C0     xor     ax, ax
15721 00000C0D C3         retn
15722 ; -----
15723
15724 block_floppy:
15725
15726 ; read at most 1 track worth. perform minimization at sector / track
15727
15728 00000C0E 268A4D13   mov     cl, [es:di+19] ; [es:di+BDS.secpertrack]
15729             ;inc     cl
15730             ; 23/12/2023
15731 00000C12 41         inc     cx
15732 00000C13 2A0E[3101]   sub     cl, [cursec]
15733 00000C17 30ED     xor     ch, ch
15734 00000C19 39C8     cmp     ax, cx
15735 00000C1B 7302     jnb     short gotmin
15736 00000C1D 89C1     mov     cx, ax
15737 gotmin:
15738
15739 ; ax is the requested number of sectors to read
15740 ; cx is the number that we can do on this track
15741
15742 00000C1F 50         push    ax
15743 00000C20 51         push    cx
15744 00000C21 89C8     mov     ax, cx
15745 00000C23 E80D00   call    Disk
15746 00000C26 59         pop     cx
15747 00000C27 58         pop     ax
15748
15749 ; cx is the number of sectors just transferred

```

```

15750
15751 00000C28 29C8          sub    ax, cx      ; reduce sectors-remaining by last i/o
15752 00000C2A D0E1          shl    cl, 1
15753 00000C2C 00CF          add    bh, cl      ; adjust transfer address
15754 00000C2E EBC6          jmp     short block
15755 dskerr_brdg:
15756 00000C30 E9F400         jmp     dskerr
15757
15758 ; ===== S U B   R O U T I N E =====
15759
15760 ; 15/10/2022
15761
15762 ;-----
15763 ;perform disk i/o with retries
15764 ; al = number of sectors (1-8, all on one track)
15765 ; es:di point to drive parameters
15766 ; xfer_seg:bx = transfer address
15767 ; (must not cross a 64k physical boundary)
15768 ; [rflag] = 2 if read, 3 if write
15769 ; [verify] = 0 for normal, 1 for verify after write
15770 ;-----
15771
15772 ; 18/04/2024
15773 ; 23/12/2023 - Retro DOS v5.0
15774 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0C74h)
15775
15776 ; 19/10/2022
15777
15778 Disk:
15779 ; Check for hard disk format and
15780 ; if TRUE then set max error count to 2
15781
15782 00000C33 BD0500         mov     bp, 5      ; MAXERR
15783                                     ; set up retry count
15784
15785 ; 18/04/2024
15786 ; 23/12/2023
15787 ;mov     cl, [es:di+3Fh]
15788 00000C36 26F6453F01     ;and     cx, 1
15789 test     byte [es:di+3Fh], 1
15790 ;test    byte [es:di+23h], 1
15791                                     ; [es:di+BDS.flags], fnon_removable
15792 00000C3B 7408          jz      short GetRdWrInd
15793 00000C3D 80FC04         cmp     ah, 4      ; romverify ; Is this a track verify?
15794 00000C40 7403          jz      short GetRdWrInd
15795 00000C42 BD0200         mov     bp, 2      ; This is not verify so only 1 retry
15796 GetRdWrInd:
15797 mov     [vretry_cnt], bp ; verify op. retry cnt for write-verify
15798 mov     [soft_ecc_cnt], bp ; soft ecc error retry count.
15799 mov     ah, [rflag]      ; get read/write indicator
15800 ;retry:
15801 ; 09/12/2022
15802 _retry:
15803 push     ax
15804 mov     dx, [curtrk]
15805 ; 23/12/2023
15806 ;jcxz    disk_not_mini
15807 ; 18/04/2024
15808 test     byte [es:di+3Fh], 1
15809 ;test    byte [es:di+23h], 1
15810 jz      short disk_not_mini
15811
15812 ; 23/12/2023
15813 cmp     word [es:di+79h], 1
15814 ;cmp     word [es:di+47h], 1 ; [es:di+BDS.bdsminimini]
15815 ; is this a mini disk? ((logical dos partition))
15816 jnz     short disk_not_mini ; no. continue to next.
15817 ; 23/12/2023
15818 add     dx, [es:di+7Bh]
15819 ;add     dx, [es:di+49h] ; [es:di+BDS.bdsmin_hidden_trks]
15820                                     ; add hidden trks.
15821 disk_not_mini:
15822 ror     dh, 1
15823 ror     dh, 1
15824 or     dh, [cursec]
15825 mov     cx, dx
15826 xchg    ch, cl      ; cl = sector, ch = cylinder
15827 mov     dh, [curhd] ; load current head number and
15828 mov     dl, [es:di+4] ; physical drive number
15829                                     ; [es:di+BDS.drivenum]
15830 ; 23/12/2023
15831 cmp     byte [es:di+3Eh], 5
15832 ;cmp     byte [es:di+22h], 5 ; [es:di+BDS.formfactor], ffHardFile
15833 jz      short do_fast ; hard files use fast speed
15834
15835 ; if we have [step_drv] set to -1, we use the slow settle time.
15836 ; this helps when we have just done a reset disk operation and the head has
15837 ; been moved to another cylinder - the problem crops up with 3.5" drives.
15838 00000C83 803E[7600]FF   cmp     byte [step_drv], 0FFh ; -1
15839 ;jz      short do_writej
15840 ; 23/12/2023
15841 jz      short do_write
15842 cmp     ah, 2      ; romread
15843 jz      short do_fast
15844 cmp     ah, 4      ; romverify
15845 ;jz      short do_fast
15846 ; 23/12/2023
15847 jnz     short do_write
15848 ;do_writej:
15849
15850 ; reads always fast, unless we have just done a disk reset operation
15851
15852 ;jmp     short do_write ; reads always fast
15853 ; -----
15854
15855 do_fast:
15856 00000C94 E80501         call    fastspeed ; change settle mode
15857 testerr:
15858 00000C97 7297          jb      short dskerr_brdg
15859
15860 ; 23/12/2023 Retro DOS v5.0
15861 ; (PCDOS 7.1 IBMBIO.COM)
15862 cmp     bp, 5      ; is there retry ?
15863 jnz     short testerror ; yes
15864 cmp     ah, 0BBh   ; Undefined error (hard disk)
15865 jz      short dskerr_brdg
15866 testerror:
15867
15868 ; set drive and track of last access
15869
15870 00000CA3 8816[7600]     mov     [step_drv], dl
15871 ; 23/12/2023
15872 00000CA7 26886D78     mov     [es:di+78h], ch
15873                                     ; [es:di+BDS.track]

```

```

15874
15875
15876 00000CAB 813E[2001]0301
15877 00000CB1 7452
15878
15879 00000CB3 58
15880
15881
15882
15883
15884
15885
15886
15887
15888 00000CB4 26F6453F01
15889
15890
15891 00000CB9 7407
15892 00000CBB F606[A004]80
15893 00000CC0 7530
15894
15895 00000CC2 80E13F
15896 00000CC5 30E4
15897 00000CC7 2906[2201]
15898 00000CCB 00C1
15899 00000CCD 880E[3101]
15900 00000CD1 263A4D13
15901
15902 00000CD5 761B
15903 00000CD7 C606[3101]01
15904 00000CDC 8A36[3201]
15905 00000CE0 FEC6
15906 00000CE2 263A7515
15907 00000CE6 7206
15908 00000CE8 30F6
15909 00000CEA F06[3301]
15910
15911 00000CEE 8836[3201]
15912
15913 00000CF2 F8
15914 00000CF3 C3
15915
15916
15917
15918
15919
15920
15921
15922
15923
15924
15925
15926
15927 00000CF4 3A16[7600]
15928 00000CF8 7506
15929
15930 00000CFA 263A6D78
15931
15932 00000CFE 7494
15933
15934 00000D00 E87500
15935 00000D03 EB92
15936
15937
15938
15939
15940
15941
15942
15943 00000D05 58
15944 00000D06 50
15945 00000D07 B404
15946 00000D09 E89000
15947 00000D0C 73A5
15948
15949
15950
15951
15952
15953
15954
15955
15956 00000D0E 80FC11
15957 00000D11 750B
15958 00000D13 FF0E[A504]
15959 00000D17 749A
15960 00000D19 E81F06
15961 00000D1C EB3E
15962
15963
15964
15965 00000D1E E81A06
15966 00000D21 FF0E[A304]
15967 00000D25 EB1C
15968
15969
15970
15971
15972
15973
15974
15975
15976 00000D27 803E[7700]00
15977 00000D2C 7403
15978 00000D2E E8BE0E
15979
15980 00000D31 803E[A704]01
15981 00000D36 7508
15982 00000D38 BD0100
15983 00000D3B C606[A704]00
15984
15985 00000D40 E89E00
15986
15987 00000D43 7420
15988
15989 00000D45 26F6453F01
15990
15991
15992 00000D4A 7505
15993 00000D4C 80FC80
15994 00000D4F 7414
15995
15996 00000D51 80FCCC
15997 00000D54 740A

no_set:
;cmp word [wrtverify], 103h
;cmp word [rflag], 103h ; check for write and verify
;jz short doverify

noverify:
pop ax

; check the flags word in the bds to see if the drive is non removable
; if not we needn't do anything special
; if it is a hard disk then check to see if multi-track operation
; is specified. if specified we don't have to calculate for the next
; track since we are already done. so we can go to the exit of this routine.

; 23/12/2023
test byte [es:di+3Fh], 1
;test byte [es:di+23h], 1 ; [es:di+BDS.flags]
; fnon_removable
;jz short its_removable
test byte [multitrk_flag], 80h ; multitrk_on
;jnz short disk_ret

its_removable:
and cl, 3Fh ; eliminate cylinder bits from sector
xor ah, ah
sub [seccnt], ax ; reduce count of sectors to go next sector
add cl, al
mov [cursec], cl
cmp cl, [es:di+13h] ; [es:di+BDS.secpertrack]
; see if sector/track limit reached
;jbe short disk_ret
mov byte [cursec], 1 ; start with first sector of next track
mov dh, [curhd]
inc dh
cmp dh, [es:di+15h] ; [es:di+BDS.heads]
;jb short noxor
xor dh, dh
inc word [curtrk]

noxor:
mov [curhd], dh

disk_ret:
clic
ret

; -----
; 15/10/2022
; 24/12/2023 - Retro DOS v5.0

; -----
; the request is for write. determine if we are talking about
; the same track and drive. if so, use the fast speed.
; -----

do_write:
cmp dl, [step_drv]
;jnz short do_norm ; we have changed drives
; 24/12/2023
cmp ch, [es:di+78h]
;cmp ch, [es:di+46h] ; [es:di+BDS.track]
;jz short do_fast ; we are still on the same track

do_norm:
call normspeed
jmp short testerr

; -----
; -----
; we have a verify request also. get state info and go verify
; -----

doverify:
pop ax
push ax
mov ah, 4
call fastspeed
jnb short noverify

; check the error returned in ah to see if it is a soft ecc error.
; if it is not we needn't do anything special. if it is a soft
; ecc error then decrement the soft_ecc_cnt error retry count. if
; this retry count becomes 0 then we just ignore the error and go to
; no_verify but if we can still try then we call the routine to reset
; the disk and go to dskerr1 to retry the operation.

cmp ah, 11h ; soft ecc error ?
;jnz short not_softecc_err
dec word [soft_ecc_cnt]
;jz short noverify ; no more retry
call ResetDisk ; reset disk
jmp short dskerr1 ; retry

; -----

not_softecc_err:
; other error.
call ResetDisk
dec word [vretry_cnt]
jmp short dskerr0

; -----
; -----
; need to special case the change-line error ah=06h.
; if we get this, we need to return it.
; -----

dskerr:
cmp byte [fhave96], 0 ; do we have changeline support?
;jz short dskerr_nochangeline ; brief not
call checkio

dskerr_nochangeline:
cmp byte [multitrk_format_flag], 1 ; multi trk format request?
;jnz short dochkagain ; no more retry.
mov bp, 1
mov byte [multitrk_format_flag], 0 ; clear the flag.

dochkagain:
call again

dskerr0:
;jz short harderr
; 24/12/2023
test byte [es:di+3Fh], 1
;test byte [es:di+23h], 1 ; [es:di+BDS.flags]
; fnon_removable
;jnz short skip_timeout_chk
cmp ah, 80h ; timeout?
;jz short harderr

skip_timeout_chk:
cmp ah, 0Cch ; write fault error?
;jz short write_fault_err ; then, don't retry.

```

```

15998 00000D56 C706[A504]0500          mov     word [soft_ecc_cnt], 5 ; MAXERR
15999                                     ; set soft_ecc_cnt back      to maxerr
16000 dskerr1:
16001 00000D5C 58                        pop     ax                ; restore sector count
16002                                     ; jmp     retry
16003                                     ; 09/12/2022
16004 00000D5D E9F1FE                    jmp     _retry
16005                                     ; -----
16006
16007 write_fault_err:
16008 00000D60 BD0100                    mov     bp, 1             ; just retry only once
16009                                     ; for write fault error.
16010 00000D63 EBF7                        jmp     short dskerr1
16011                                     ; fall into harderr
16012                                     ; -----
16013
16014 ; entry point for routines that call maperror themselves
16015
16016 harderr:
16017                                     call    maperror
16018 00000D65 E84100                    harderr2:
16019                                     mov     byte [tim_drv], 0FFh
16020 00000D68 C606[1E01]FF                ; force a media check through rom
16021                                     ; get count of sectors to go
16022 00000D6D 8B0E[2201]                mov     cx, [seccnt]
16023 00000D71 8B26[3501]                mov     sp, [spsav]      ; recover entry      stack pointer
16024                                     ; since we are performing a non-local goto, restore the disk parameters
16025                                     ; jmp     diddleback
16026                                     ; 09/12/2022
16027                                     jmp     diddle_back
16028 00000D75 E951FE                    ; ===== S U B   R O U T I N E =====
16029                                     ; change settle value from settlecurrent to whatever is appropriate
16030                                     ; note that this routine is never called for a fixed disk.
16031                                     ; 19/10/2022
16032
16033 normspeed:
16034                                     cmp     byte [media_set_for_format], 0
16035                                     jnz     short fastspeed
16036                                     push    es
16037                                     push    ax
16038 00000D78 803E[A905]00                mov     al, [settle_slow]
16039 00000D7D 751D                        les     si, [dpt]        ; current disk parm table
16040 00000D7F 06                        mov     [es:si+9], al    ; [es:si+DISK_PARMS.DISK_HEAD_STTL]
16041 00000D80 50                        pop     ax
16042 00000D81 A0[2801]                pop     es
16043 00000D84 C436[2D01]                call   fastspeed
16044 00000D88 26884409                ; 24/12/2023
16045 00000D8C 58                        ; push    es
16046 00000D8D 07                        ; les     si, [dpt]
16047 00000D8E E80B00                ; mov     byte [es:si+9], 1 ; [es:si+DISK_PARMS.DISK_HEAD_STTL]
16048                                     ; 1 is fast settle value
16049                                     ; pop     es
16050                                     push    ds
16051                                     lds     si, [dpt]
16052                                     mov     byte [si+9], 1
16053                                     pop     ds
16054 00000D91 1E                        retn
16055 00000D92 C536[2D01]
16056 00000D96 C6440901
16057 00000D9A 1F
16058
16059 00000D9B C3
16060
16061 ; ===== S U B   R O U T I N E =====
16062
16063 ; if the drive has been marked as too big (i.e. starting sector of the
16064 ; partition is > 16 bits, then always return drive not ready.
16065
16066 ; 24/12/2023 - Retro DOS v5.0
16067 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0DDdh)
16068
16069 fastspeed:
16070                                     ; test byte [es:di+3Bh], 80h ; [es:di+BDS.fatsiz]
16071                                     ; test byte [es:di+1Fh], 80h ; [es:di+BDS.fatsiz]
16072                                     ; jnz     short notready
16073 00000D9C 06                        push    es
16074 00000D9D 8E06[A804]                mov     es, [xfer_seg]
16075 00000DA1 CD13                        int     13h             ; DISK -
16076 00000DA3 8C06[A804]                mov     [xfer_seg], es
16077 00000DA7 07                        pop     es
16078 00000DA8 C3                        retn
16079                                     ; -----
16080                                     ; 24/12/2023
16081 ;notready:
16082                                     ; stc
16083                                     ; mov     ah, 80h
16084                                     ; retn
16085
16086 ; ===== S U B   R O U T I N E =====
16087
16088 ; map error returned by rom in ah into corresponding code to be returned to
16089 ; dos in al. trashes di. guaranteed to set carry.
16090
16091 maperror:
16092 00000DA9 51                        push    cx
16093 00000DAA 06                        push    es
16094 00000DAB 1E                        push    ds              ; set es=Bios_Data
16095 00000DAC 07                        pop     es
16096 00000DAD 88E0                    mov     al, ah          ; put error code in al
16097 00000DAF A2[4601]                mov     [lterr], al    ; terminate list with error code
16098                                     ; 24/12/2023
16099 00000DB2 B90B00                    mov     cx, 11 ; PCDOS 7.1 ; 02/09/2023
16100                                     ; mov     cx, 9          ; numerr (= errout-errin)
16101                                     ; number of possible error conditions
16102 00000DB5 BF[3C01]                mov     di, errin      ; point to error conditions
16103 00000DB8 F2AE                    repne scasb
16104                                     ; 24/12/2023
16105                                     ; 02/09/2023
16106 00000DBA 8A450A                    mov     al, [di+10] ; PCDOS 7.1 IBMBIO.COM
16107                                     ; 10/12/2022
16108                                     ; mov     al, [di+8]          ; [di+numerr-1]
16109                                     ; get translation
16110                                     ; 19/10/2022 - Temporary !
16111                                     ; db     8Ah, 85h, 8, 0 ; mov al, [di+8]
16112 00000DBD 07                        pop     es
16113 00000DBE 59                        pop     cx
16114 00000DBF F9                        stc
16115 00000DC0 C3                        ; flag error condition
16116                                     retn
16117
16118 ; ===== S U B   R O U T I N E =====
16119
16120 ; set the time of last access for this drive.
16121 ; this is done only for removable media. es:di -> bds

```

```

16122
16123
16124 00000DC1 50
16125 00000DC2 E86CF7
16126
16127
16128
16129
16130
16131 00000DC5 263B5579
16132
16133 00000DC9 7506
16134
16135 00000DCB 263B4D7B
16136
16137
16138
16139 00000DCF 740E
16140
16141 00000DD1 C606[1D01]00
16142
16143
16144 00000DD6 26895579
16145 00000DDA 26894D7B
16146
16147
16148
16149 00000DDE F8
16150
16151 00000DDF 58
16152 00000DE0 C3
16153
16154
16155
16156
16157
16158
16159
16160
16161
16162
16163 00000DE1 E85705
16164 00000DE4 80FC06
16165 00000DE7 7402
16166
16167 00000DE9 4D
16168 00000DEA C3
16169
16170
16171
16172 00000DEB 08E4
16173 00000DED C3
16174
16175
16176
16177
16178
16179
16180
16181 00000DEE 00
16182
16183
16184
16185
16186
16187
16188 00000DEF E8B2F7
16189
16190
16191 00000DF2 268A5504
16192 00000DF6 C3
16193
16194
16195
16196
16197
16198 00000DF7 E8F5FF
16199 00000DFA 2E8816[EE0D]
16200 00000DFF B441
16201 00000E01 BBAA55
16202 00000E04 CD13
16203
16204
16205
16206
16207
16208 00000E06 7235
16209
16210 00000E08 C43E[1200]
16211 00000E0C 26C47D0E
16212 00000E10 723E
16213 00000E12 B80046
16214 00000E15 263805
16215 00000E18 7417
16216 00000E1A 26803D01
16217 00000E1E 751B
16218 00000E20 B80145
16219
16220 00000E23 26807D0100
16221 00000E28 7407
16222 00000E2A 26384501
16223 00000E2E 750B
16224 00000E30 48
16225
16226 00000E31 2E8A16[EE0D]
16227 00000E36 CD13
packet)
16228 00000E38 7203
16229
16230
16231
16232
16233 00000E3A C3
16234
16235
16236
16237 00000E3B B401
16238
16239 00000E3D 80FCB0
16240 00000E40 74F8
16241 00000E42 80FCB4
16242 00000E45 74F3
16243 00000E47 E9DAF7
16244

set_tim:
    push    ax
    call    GetTickCnt    ; Does INT 1A ah=0 & updates daycnt

; we have the new time. if we see that the time has passed,
; then we reset the threshold counter...

    ; 24/12/2023 - Retro DOS v5.0
    cmp     dx, [es:di+79h]    ; PCDOS 7.1 IBMBIO.COM
    ;cmp     dx, [es:di+47h]    ; [es:di+BDS.tim_lo]
    jne     short setaccess
    ; 24/12/2023
    cmp     cx, [es:di+7Bh]    ; PCDOS 7.1 IBMBIO.COM
    ;cmp     cx, [es:di+49h]    ; [es:di+BDS.tim_hi]
    ;jz      short done_set
    ; 12/12/2022
    je      short done_set2

setaccess:
    mov     byte [accesscount], 0

    ; 24/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
    mov     [es:di+79h], dx
    mov     [es:di+7Bh], cx
    ;mov     [es:di+47h], dx    ; [es:di+BDS.tim_lo]
    ;mov     [es:di+49h], cx    ; [es:di+BDS.tim_hi]

done_set:
    cllc

done_set2:
    ; 12/12/2022
    pop     ax
    retn

; ===== S U B   R O U T I N E =====

; this routine is called if an error occurs while formatting or verifying.
; it resets the drive, and decrements the retry count.
; on entry - ds:di - points to bds for the drive
;           bp      - contains retry count
; on exit   flags indicate result of decrementing retry count

again:
    call    ResetDisk
    cmp     ah, 6
    jz      short dont_dec_retry_count ; If it is a media change error
    ; do not decrement retry count.
    dec     bp
    ; decrement retry count
    retn

; -----
dont_dec_retry_count:
    or      ah, ah
    retn

; -----
; Retro DOS v5.0 - PCDOS 7.1 IBMBIO.COM - BIOSCODE:0E30h
;
; 24/12/2023 - Retro DOS v5.0
; ; ;
ioctl_drvnum:    db 0

; 24/12/2023

; ===== S U B R O U T I N E =====

get_phy_drv_num:
    call    SetDrive    ; get physical drive number
    ; INPUT: al = logical drive number (BDS.drivelet)
    ; OUTPUT: physical drive number (BDS.drivenum)
    mov     dl, [es:di+4] ; [es:di+BDS.drivenum]
    retn

; ===== S U B R O U T I N E =====

; 24/12/2023
ioctl_output:
    call    get_phy_drv_num
    mov     [cs:ioctl_drvnum], dl
    mov     ah, 41h
    mov     bx, 55AAh
    int     13h
    ; DISK - Check for INT 13h Extensions
    ; BX = 55AAh, DL = drive number
    ; Return: CF set if not supported
    ; AH = extensions version
    ; BX = AA55h
    ; CX = Interface support bit map
    jc      short int13h_exts_err

ioctl_input_1:
    les     di, [ptrsav]
    les     di, [es:di+14] ; [es:di+IOCTL_REQ.MINORFUNCTION]
    jc      short ioctl_input_2
    mov     ax, 4600h    ; Eject removable media
    cmp     [es:di], al    ; al = 0 ; disk ioctl function = 0
    je      short ioctl_output_1
    cmp     byte [es:di], 1 ; al = 1 ; disk ioctl function = 1
    jne     short ioctl_output_2
    mov     ax, 4501h    ; Lock/unlock media
    ; (al, 0 = lock, 1 = unlock)
    cmp     byte [es:di+1], 0 ; unlock (reverse of INT 13h ah=45h)
    jz      short ioctl_output_1
    cmp     [es:di+1], al    ; lock (reverse of INT 13h ah=45h)
    jne     short ioctl_output_2
    dec     ax

ioctl_output_1:
    mov     dl, [cs:ioctl_drvnum]
    int     13h
    ; DISK - IBM/MS Extension - LOCK/UNLOCK DRIVE (DL - drive, [SI - disk address

    jc      short int13h_exts_err

ioctl_lock_err:
    ; cf=0

ioctl_output_ret:
    cllc
    retn

; -----

ioctl_output_2:
    mov     ah, 1

int13h_exts_err:
    cmp     ah, 0B0h    ; volume not locked in drive
    je      short ioctl_lock_err
    cmp     ah, 0B4h    ; lock count exceeded
    je      short ioctl_lock_err
    jmp     err_exitj

```

```

16245 ; ===== S U B R O U T I N E =====
16246 ;
16247 ; 24/12/2023
16248 ioctl_input:
16249     call    get_phy_drv_num
16250     stc
16251     jmp     short ioctl_input_1
16252 ioctl_input_2:
16253     cmp     byte [es:di], 6 ; disk ioctl function = 6
16254     jne     short ioctl_output_2
16255     mov     ax, 4502h ; get lock status
16256     int     13h ; DISK - IBM/MS Extension - LOCK/UNLOCK DRIVE (DL - drive, [SI - disk address
packet)
16257     jc     short int13h_exts_err
16258     mov     bx, 0Ch ; bit 1 lock bit
16259     cmp     al, 0 ; not locked
16260     jz     short ioctl_input_3
16261     mov     bl, 0Eh
16262 ioctl_input_3:
16263     push    bx
16264     mov     ah, 4
16265     mov     cx, 101h
16266     mov     dh, 1
16267     int     13h ; DISK - VERIFY SECTORS
16268     ; AL = number of sectors to verify, CH = track, CL = sector
16269     ; DH = head, DL = drive
16270     ; Return: CF set on error, AH = status
16271     ; AL = number of sectors verified
16272     pop     bx
16273     cmp     ah, 31h ; no media in drive (IBM/MS INT 13 extensions)
16274     je     short ioctl_input_5
16275     cmp     ah, 80h ; timeout (not ready)
16276     je     short ioctl_input_5
16277 ioctl_input_4:
16278     mov     [es:di+1], bx
16279     jmp     short ioctl_lock_err
16280 ioctl_input_5:
16281     or      bx, 801h ; bit 0 error bit (1 = error, 31h or 80h)
16282     ; bit 11 (not ready -removable media error- bit)
16283     ; if bit 11 = 0, another error (except 31h and 80h)
16284     jmp     short ioctl_input_4
16285
16286 ; -----
16287 ;;;;
16288 ; 16/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS)
16289
16290 ; -----
16291 ; MSDIOCTL.ASM - MSDOS 6.0 - 1991
16292 ; -----
16293 ; 11/03/2019 - Retro DOS v4.0
16294
16295 ; 18/03/2019
16296
16297 ; =====
16298 ;
16299 ; NOTE: GetAccessFlag/SetAccessFlag is unpublished function.
16300 ;
16301 ; This function is intended to give the user to control the
16302 ; bds table flags of unformatted_media bit.
16303 ; GetAccessFlag will show the status -
16304 ; a_DiskAccess_Control.dac_access_flag = 0 disk i/o not allowed
16305 ; 1 disk i/o allowed
16306 ; SetAccessFlag will set/reset the unformatted_media bit in flags -
16307 ; a_DiskAccess_Control.dac_access_flag = 0 allow disk i/o
16308 ; 1 disallow disk i/o
16309 ; =====
16310
16311 ; generic ioctl dispatch tables
16312
16313 ; BIOSCODE:0C3Ch (MSDOS 6.21, IO.SYS)
16314
16315 ; 24/12/2023
16316 ; BIOSCODE:0ECAh (PCDOS 7.1, IBMBIO.COM)
16317
16318 ; -----
16319 ; 24/12/2023
16320 ; db 0
16321 ; 09/12/2022
16322 %if 0
16323
16324 IoReadJumpTable: db 8 ; ((IoWriteJumpTable-IoReadJumpTable)-1)/2
16325 ; 60h ; GetDeviceParameters
16326 ; 61h ; ReadTrack
16327 ; 62h ; VerifyTrack
16328 ; Cmd_Error_Proc
16329 ; Cmd_Error_Proc
16330 ; Cmd_Error_Proc
16331 ; Cmd_Error_Proc
16332 ; 66h ; GetMediaId
16333 ; 67h ; GetAccessFlag ; unpublished function
16334 ; 68h ; SenseMediaType
16335
16336 IoWriteJumpTable: db 7 ; ((IOC_DC_Table-IoWriteJumpTable)-1)/2
16337 ; 40h ; SetDeviceParameters
16338 ; 41h ; WriteTrack
16339 ; 42h ; FormatTrack
16340 ; Cmd_Error_Proc
16341 ; Cmd_Error_Proc
16342 ; Cmd_Error_Proc
16343 ; 46h ; SetMediaId
16344 ; 47h ; SetAccessFlag ; unpublished function
16345
16346 %endif
16347 ; 24/12/2023 - Retro DOS v5.0
16348 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0ECAh)
16349
16350 ; 09/12/2022
16351 IoReadJumpTable:
16352 db ((IoWriteJumpTable-IoReadJumpTable)-1)/2 ; 15
16353 dw GetDeviceParameters ; 60h
16354 dw ReadTrack ; 61h
16355 dw VerifyTrack ; 62h
16356 dw Cmd_Error_Proc
16357 dw Cmd_Error_Proc
16358 dw Cmd_Error_Proc
16359 dw Cmd_Error_Proc
16360 dw GetMediaId ; 66h
16361 dw GetAccessFlag ; 67h ; unpublished function
16362 dw SenseMediaType ; 68h
16363 ; 24/12/2023
16364 ; Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
16365 dw Cmd_Error_Proc ; 69h
16366 dw Cmd_Error_Proc ; 6Ah
16367 dw Cmd_Error_Proc
16368 dw Cmd_Error_Proc

```



```

16368 00000EA2 [160F]          dw Cmd_Error_Proc
16369 00000EA4 [160F]          dw Cmd_Error_Proc      ; 6Eh
16370 00000EA6 [C815]          dw GetDrvMapInfo      ; 6Fh
16371
16372
16373 00000EA8 0A                dw ((IOC_DC_Table-IowriteJumpTable)-1)/2 ; 9
16374 00000EA9 [7A0F]          dw SetDeviceParameters; 40h
16375 00000EAB [9A11]          dw WriteTrack          ; 41h
16376 00000EAD [6D10]          dw FormatTrack          ; 42h
16377 00000EAF [160F]          dw Cmd_Error_Proc
16378 00000EB1 [160F]          dw Cmd_Error_Proc
16379 00000EB3 [160F]          dw Cmd_Error_Proc
16380 00000EB5 [5214]          dw SetMediaId          ; 46h
16381 00000EB7 [0415]          dw SetAccessFlag        ; 47h ; unpublished function
16382                                ; 24/12/2023
16383                                ; Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
16384 00000EB9 [8115]          dw SetLockState          ; 48h
16385 00000EBB [9815]          dw EjectMedia            ; 49h
16386
16387
16388
16389
16390
16391
16392
16393
16394
16395                                ; 24/12/2023 - Retro DOS v5.0
16396                                ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0F00h)
16397
16398 00000EBD 60                IOC_DC_Table:          db 60h                ; GET_DEVICE_PARAMETERS
16399 00000EBE 40                db 40h                ; SET_DEVICE_PARAMETERS
16400 00000EBF 61                db 61h                ; READ_TRACK
16401 00000EC0 41                db 41h                ; WRITE_TRACK
16402 00000EC1 62                db 62h                ; VERIFY_TRACK
16403 00000EC2 42                db 42h                ; FORMAT_TRACK
16404 00000EC3 66                db 66h                ; GET_MEDIA_ID
16405 00000EC4 46                db 46h                ; SET_MEDIA_ID
16406 00000EC5 67                db 67h                ; GET_ACCESS_FLAG
16407 00000EC6 47                db 47h                ; SET_ACCESS_FLAG
16408 00000EC7 68                db 68h                ; SENSE_MEDIA_TYPE
16409                                ; 24/12/2023
16410                                ; Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
16411 00000EC8 48                db 48h                ; SET_LOCK_STATE
16412 00000EC9 49                db 49h                ; EJECT_MEDIA
16413 00000ECA 6F                db 6Fh                ; GET_DRV_MAP_INFO
16414
16415
16416
16417                                ; 24/12/2023 - Retro DOS v5.0
16418                                ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0F0Eh)
16419
16420 00000ECB 00                new_genioctl:          db 0
16421
16422
16423
16424
16425
16426
16427
16428
16429
16430
16431
16432
16433
16434
16435
16436
16437
16438
16439
16440
16441
16442
16443 00000ECC E8D5F6          do_generic_ioctl:      ; 2C7h:0C6Bh = 70h:31DBh
16444                                call SetDrive          ; ES:DI Points to bds for drive
16445
16446                                ; 24/12/2023
16447 00000ECF 2EC606[CB0E]00    mov     byte [cs:new_genioctl], 0
16448                                ; 0, old generic ioctl function
16449 00000ED5 06                push     es
16450 00000ED6 C41E[1200]        les     bx, [ptrsav] ; ES:BX Points to request header
16451 00000EDA 26807F0D08        cmp     byte [es:bx+0Dh], 8 ; [es:bx+IOCTL_REQ.MAJORFUNCTION]
16452                                ; RAWIO
16453                                ; 24/12/2023
16454                                ; mov     al, [es:bx+0Eh] ; [es:bx+IOCTL_REQ.MINORFUNCTION]
16455                                ; pop     es
16456                                ; jnz     short IoctlFuncErr
16457 00000EDF 740A                jz      short chk_genioctl_minor
16458 00000EE1 2EFE06[CB0E]        inc     byte [cs:new_genioctl]
16459                                ; 1, new generic ioctl function (FAT32)
16460 00000EE6 26807F0D48        cmp     byte [es:bx+0Dh], 48h ; Generic IOCTL Request support
16461                                ; (called only if bit 6 of attribute is set to 1)
16462
16463 00000EEB 268A470E          chk_genioctl_minor:
16464 00000EEF 07                mov     al, [es:bx+0Eh] ; [es:bx+IOCTL_REQ.MINORFUNCTION]
16465 00000EF0 7525                pop     es
16466                                jnz     short IoctlFuncErr
16467                                ;;;
16468                                ; cas note: Could do the above two blocks in reverse order.
16469                                ; would have to preserve al for SetDrive
16470
16471                                ; 10/12/2022
16472 00000EF2 BE[870E]          mov     si, IoReadJumpTable
16473                                ; mov     si, 0C3Ch ; IoReadJumpTable
16474                                ; at 2C7h:0C3Ch = 70h:31ACh
16475 00000EF5 A820                test    al, 20h ; GEN_IOCTL_FN_TST ; test of req. function
16476 00000EF7 7503                jnz     short NotGenericWrite ; function is a read.
16477                                ; 10/12/2022
16478 00000EF9 BE[A80E]          mov     si, IowriteJumpTable
16479                                ; mov     si, 0C4Fh ; IowriteJumpTable
16480                                ; at 2C7h:0C4Fh = 70h:31BFh
16481
16482 00000EFC 24DF          NotGenericWrite:
16483 00000EFE 2C40                and     al, 0DFh ; ~GEN_IOCTL_FN_TST ; get rid of read/write bit
16484 00000F00 2E3A04                sub     al, 40h ; offset for base function
16485 00000F03 7712                cmp     al, [cs:si]
16486 00000F05 98                ja      short IoctlFuncErr
16487                                cbw
16488                                ; 24/12/2023
16489 00000F06 01C0                shl     ax, 1
16490 00000F08 46                add     ax, ax
16491 00000F09 01C6                inc     si
16491                                add     si, ax

```

```

16492 00000F0B 2EFF14          call    near [cs:si]
16493          ;call    word ptr cs:[si]
16494 00000F0E 2E8E1E[3000]    mov     ds, [cs:BIOSDATAWORD]
16495          ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
16496          ; 2C7h:30h = 70h:25A0h
16497 00000F13 B481          mov     ah, 81h          ; Return this status in case of carry
16498 00000F15 C3          retn          ; Pass carry flag through to exit code
16499          ; -----
16500          ; Cmd_Error_Proc is called as a procedure and also use
16501          ; as a fall through from above
16502          ; 2C7h:0CA3h = 70h:3213h
16503 Cmd_Error_Proc:
16504 00000F16 5A          pop     dx
16505 IoctlFuncErr:
16506 00000F17 E9BBF1          jmp     bc_cmderr
16507          ; -----
16508          ; 16/10/2022
16509
16510          ; =====
16511          ; ** GetDeviceParameters:
16512          ;
16513          ; GetDeviceParameters implements the generic ioctl function:
16514          ; majorcode=RAWIO, minorcode=GetDeviceParameters (60h)
16515          ;
16516          ; ENTRY (ES:di) = BDS for drive
16517          ; PtrSav = long pointer to request header
16518          ;
16519          ; EXIT ??? BUGBUG
16520          ; USES ??? BUGBUG
16521          ; =====
16522          ;
16523          ; 24/12/2023 - Retro DOS v5.0
16524          ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0F5Dh)
16525
16526          ; 19/10/2022
16527 GetDeviceParameters:
16528          ; Copy info from bds to the device parameters packet
16529
16530 00000F1A C51E[1200]    lds     bx, [ptrsav]    ; DS:BX points to request header
16531 00000F1E C55F13      lds     bx, [bx+19]    ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
16532          ; (DS:BX) = return buffer
16533          ; 24/12/2023
16534 00000F21 268A453E    mov     al, [es:di+3Eh]
16535          ;mov     al, [es:di+34] ; [es:di+BDS.formfactor]
16536 00000F25 884701      mov     [bx+1], al    ; [bx+A_DEVICEPARAMETERS.DP_DEVICECTYPE]
16537          ; 24/12/2023
16538 00000F28 268B453F    mov     ax, [es:di+3Fh]
16539          ;mov     ax, [es:di+35] ; [es:di+BDS.flags]
16540 00000F2C 83E003      and     ax, 3          ; fnon_removable+fchangeline
16541          ; Mask off other bits
16542 00000F2F 894702      mov     [bx+2], ax    ; [bx+A_DEVICEPARAMETERS.DP_DEVICEATTRIBUTES]
16543          ; 24/12/2023
16544 00000F32 268B4541    mov     ax, [es:di+41h]
16545          ;mov     ax, [es:di+37] ; [es:di+BDS.cylinders]
16546 00000F36 894704      mov     [bx+4], ax    ; [bx+A_DEVICEPARAMETERS.DP_CYLINDERS]
16547 00000F39 30C0        xor     al, al          ; Set media type to default
16548 00000F3B 884706      mov     [bx+6], al    ; [bx+A_DEVICEPARAMETERS.DP_MEDIATYPE]
16549
16550          ; copy recommended bpb
16551          ; 24/12/2023
16552          ;lea     si, [di+43h]
16553 00000F3E 8D7543      ;lea     si, [di+39]    ; [di+BDS.rbytespersec] = [di+BDS.R_BPB]
16554          ;lea     si, [di+39] ; [di+BDS.rbytespersec] = [di+BDS.R_BPB]
16555 00000F41 F60701      test    byte [bx], 1    ; [bx+A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS]
16556          ; BUILD_DEVICE_BPB
16557 00000F44 7412          jz     short UseBpbPresent
16558 00000F46 1E          push    ds          ; Save request packet segment
16559 00000F47 2E8E1E[3000]    mov     ds, [cs:BIOSDATAWORD]
16560          ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
16561          ; 2C7h:30h = 70h:25A0h
16562          ; Point back to Bios_Data
16563 00000F4C E828FA      call    checksingle
16564 00000F4F E884F7      call    GetBp        ; Build the bpb from scratch
16565 00000F52 1F          pop     ds          ; Restore request packet segment
16566 00000F53 7224          jb     short GetParmRet
16567 00000F55 8D7506      lea     si, [di+6]    ; [di+BDS.bytespersec] = [di+BSD.DP_BPB]
16568          ; Use this subfield of bds instead
16569
16570 00000F58 8D7F07      UseBpbPresent:
16571          lea     di, [bx+7]    ; [bx+A_DEVICEPARAMETERS.DP_BPB]
16572          ; This is where the result goes
16573          ; 24/12/2023
16574          xor     dx, dx ; 0
16575          ; 24/12/2023
16576 00000F5D B91F00      mov     cx, 31        ; A_BPB.size = 31
16577          ;mov     cx, 25        ; A_BPB.size - 6
16578          ; For now use 'small' bpb
16579          ; 24/12/2023
16580          ;;;
16581 00000F60 2E3816[CB0E]    cmp     [cs:new_genioctl], dl ; 0 ? ; *
16582 00000F65 7404          jz     short gdp_1    ; old type (FAT12 & FAT16) structure
16583          ;mov     cx, 53        ; FAT32 BPB size
16584          ;mov     dx, 32        ; 53+32 = 85 bytes (A_BPB_FAT32.size)
16585 00000F67 B135          mov     cl, 53
16586 00000F69 B220          mov     dl, 32
16587          gdp_1:
16588          ;;;
16589 00000F6B 1E          push    ds          ; reverse segments for copy
16590 00000F6C 06          push    es
16591 00000F6D 1F          pop     ds
16592 00000F6E 07          pop     es
16593 00000F6F F3A4          rep movsb
16594
16595          ; 24/12/2023
16596          ;;;
16597 00000F71 89D1          mov     cx, dx        ; 0 or 32
16598 00000F73 E304          jcxz    gdp_2
16599 00000F75 30C0          xor     al, al        ; 32 zeros
16600 00000F77 F3AA          rep stosb
16601          gdp_2:
16602          ;clc    ; cf is already 0 ; * ; 24/12/2023
16603          ;;;
16604          ; 12/12/2022
16605          ; cf=0 (cmp instruction -above- resets cf)
16606          ;clc
16607
16608          GetParmRet:
16609 00000F79 C3          retn
16610          ; -----
16611          ; 17/10/2022
16612          ; 16/10/2022
16613
16614          ; =====
16615

```



```

16616 ; SetDeviceParameters:
16617 ;
16618 ; input: ES:di points to bds for drive
16619 ; =====
16620 ;
16621 ; 24/12/2023 - Retro DOS v5.0
16622 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0FC0h)
16623 ;
16624 ; 19/10/2022
16625 SetDeviceParameters: ; 2C7h:0CF3h = 70h:3263h
16626 lds bx, [ptrsav] ; DS:BX points to request header
16627 lds bx, [bx+19] ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
16628 ; 24/12/2023
16629 or word [es:di+3Fh], 140h
16630 ;or word [es:di+23h], 140h ; [es:di+BDS.flags]
16631 ; fchanged_by_format|fchanged
16632 test byte [bx], 2 ; [bx+A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS]
16633 ; ONLY_SET_TRACKLAYOUT
16634 ;jnz short setTrackTable
16635 ; 24/12/2023
16636 jz short sdp_1
16637 jmp setTrackTable
16638 sdp_1:
16639 mov al, [bx+1] ; [bx+A_DEVICEPARAMETERS.DP_DEVICEYPE]
16640 ; 24/12/2023
16641 mov [es:di+3Eh], al
16642 ;mov [es:di+34], al ; [es:di+BDS.formfactor]
16643 mov ax, [bx+4] ; [bx+A_DEVICEPARAMETERS.DP_CYLINDERS]
16644 ; 24/12/2023
16645 mov [es:di+41h], ax
16646 ;mov [es:di+37], ax ; [es:di+BDS.cylinders]
16647 mov ax, [bx+2] ; [bx+A_DEVICEPARAMETERS.DP_DEVICEATTRIBUTES]
16648 push ds
16649 ; 17/10/2022
16650 mov ds, [cs:BIOSDATAWORD]
16651 ;mov ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
16652 ; 2C7h:30h = 70h:25A0h
16653 ;cmp byte [fhave96], 0
16654 cmp byte [fhave96], 0
16655 pop ds
16656 jnz short HaveChange ; we have changeline support
16657 ; 10/12/2022
16658 and al, 0FDh
16659 ;and ax, 0FFFDh ; ~fchangeline
16660 ; Ignore all bits except non_removable and changeline
16661 HaveChange:
16662 and ax, 3 ; fnon_removable|fchangeline
16663 ; 24/12/2023
16664 mov cx, [es:di+3Fh]
16665 ;mov cx, [es:di+35] ; [es:di+BDS.flags]
16666 and cx, 0FDF4h ; ~(fnon_removable|fchangeline|good_tracklayout|unformatted_media)
16667 or ax, cx
16668 ; 24/12/2023
16669 mov [es:di+3Fh], ax
16670 ;mov [es:di+35], ax ; [es:di+BDS.flags]
16671 mov al, [bx+6] ; [bx+A_DEVICEPARAMETERS.DP_MEDIATYPE]
16672 ; Set media type
16673 push ds
16674 mov ds, [cs:BIOSDATAWORD]
16675 ;mov ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
16676 mov [mediatype], al
16677 ;mov ds:mediatype, al
16678 ; 24/12/2023
16679 ;;;
16680 mov cx, 53 ; FAT32 BPB size
16681 cmp byte [cs:new_genioct1], 0
16682 jnz short sdp_2 ; new type (FAT32) structure
16683 ;mov cx, 31 ; A_BPB.size = 31
16684 mov cl, 31
16685 sdp_2:
16686 ;;;
16687 pop ds
16688 ; The media changed (maybe) so we will have to do a set dasd
16689 ; the next time we format a track
16690 ; 24/12/2023
16691 or byte [es:di+3Fh], 80h
16692 ; 10/12/2022
16693 ;or byte [es:di+35], 80h
16694 ;;or word [es:di+35], 80h ; [es:di+BDS.flags]
16695 ; set_dasd_true
16696 push di ; Save bds pointer
16697 ; Figure out what we are supposed to do with the bpb
16698 ; were we asked to install a fake bpb?
16699 test byte [bx], 1 ; [bx+A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS]
16700 ; INSTALL_FAKE_BPB
16701 jnz short InstallFakeBpb
16702 ; were we returning a fake bpb when asked to build a bpb?
16703 ; 24/12/2023
16704 test byte [es:di+3Fh], 4
16705 ; 10/12/2022
16706 ;test byte [es:di+35], 4 ; [es:di+BDS.flags]
16707 ;;test word [es:di+35], 4 ; [es:di+BDS.flags]
16708 ; return_fake_bpb
16709 jz short InstallRecommendedBpb
16710 ; we were returning a fake bpb but we can stop now
16711 ; 24/12/2023
16712 and byte [es:di+3Fh], 0FBh
16713 ; 10/12/2022
16714 ;and byte [es:di+35], 0FBh
16715 ;;and word [es:di+35], 0FFFBh ; [es:di+BDS.flags]
16716 ; ~return_fake_bpb
16717 InstallRecommendedBpb:
16718 ; 24/12/2023
16719 ;mov cx, 31 ; A_BPB.size
16720 ;lea di, [di+27h] ; [di+BDS.R_BPB] = [di+BDS.rbytespersec]
16721 ; cx = 53 or 31
16722 ;lea di, [di+43h] ; (PCDOS 7.1 IBMBIO.COM)
16723 jmp short CopyTheBpb
16724 ; -----
16725 InstallFakeBpb:
16726 ; 24/12/2023
16727 or byte [es:di+3Fh], 4
16728 ; 10/12/2022

```

```

16740                ;or    byte [es:di+35], 4
16741                ;;or   word [es:di+35], 4 ; byte [es:di+BDS.flags]
16742                ;      ; return_fake_bpb
16743                ; 24/12/2023
16744                ; cx = 53 or 31
16745                ;mov    cx, 25                ; A_BPB.size - 6
16746                ;      ; move 'smaller' bpb
16747 00000FFC 8D7D06    CopyTheBpb: lea    di, [di+6]                ; [es:di+BDS.BPB] = [es:di+BDS.bytespersec]
16748
16749 00000FFF 8D7707    lea    si, [bx+7]                ; [bx+A_DEVICEPARAMETERS.DP_BPB]
16750 00001002 F3A4      rep movsb
16751 00001004 1E        push   ds                ; Save packet segment
16752                ; 17/10/2022
16753 00001005 2E8E1E[3000] mov    ds, [cs:BIOSDATAWORD]
16754                ;mov    ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
16755                ;      ; Setup for ds -> Bios_Data
16756 0000100A E8DD03    call   RestoreOldDpt ; Restore the old Dpt from TempDpt
16757 0000100D 1F        pop     ds                ; Restore packet segment
16758 0000100E 5F        pop     di                ; Restore bds pointer
16759
16760                setTrackTable:
16761                ; 24/12/2023
16762                ;mov    cx, [bx+38]                ; [bx+26h]
16763                ;      ;
16764                ;      ;
16765 0000100F 8B4F5C    mov     cx, [bx+5Ch]                ; [bx+A_DEVICEPARAMETERS.DP_TRACKTABLEENTRIES]
16766                ;      ; offset 85+7 (A_BPB.size+7) (FAT32)
16767 00001012 2E803E[CB0E]00 cmp     byte [cs:new_genioct1], 0
16768                ;      ; new type (FAT32) structure
16769 00001018 7503      jnz     short sdp_3                ; [bx+A_DEVICEPARAMETERS.DP_TRACKTABLEENTRIES]
16770                ;      ; offset 31+7 (A_BPB.size+7)
16771                ;      ;
16772                ;      ;
16773 0000101D 1E        push   ds
16774 0000101E 2E8E1E[3000] mov    ds, [cs:BIOSDATAWORD]
16775 00001023 890E[AA04] mov    [sectorspertrack], cx
16776 00001027 1F        pop     ds
16777                ; 24/12/2023
16778 00001028 2680653FF7 and     byte [es:di+3Fh], 0F7h
16779                ; 10/12/2022
16780                ;and    byte [es:di+35], 0F7h
16781                ;;and   word [es:di+35], 0FFF7h ; [es:di+BDS.flags]
16782                ;      ; ~good_tracklayout
16783 0000102D F60704    test    byte [bx], 4                ; [bx+A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS]
16784                ;      ; TRACKLAYOUT_IS_GOOD
16785 00001030 7405      jz      short UglyTrackLayout
16786                ; 24/12/2023
16787 00001032 26804D3F08 or      byte [es:di+3Fh], 8
16788                ; 10/12/2022
16789                ;or     byte [es:di+35], 8
16790                ;or     word [es:di+35], 8 ; [es:di+BDS.flags]
16791                ;      ; good_tracklayout
16792
16793 00001037 83F93F    UglyTrackLayout: cmp     cx, 63                ; MAX_SECTORS_IN_TRACK
16794 0000103A 772D      ja      short TooManyPerTrack
16795                ;jcxz   short SectorInfoSaved
16796 0000103C E329      jcxz    SectorInfoSaved                ; 19/10/2022
16797
16798 0000103E BF[AC04]    mov     di, tracktable
16799
16800                ; 24/12/2023
16801                ;lea    si, [bx+40]                ; [bx+A_DEVICEPARAMETERS.DP_SECTORTABLE]
16802                ;      ;
16803 00001041 8D775E    lea     si, [bx+5Eh]                ; [bx+A_DEVICEPARAMETERS.DP_SECTORTABLE]
16804                ;      ; offset 85+9 (A_BPB.size+9) (FAT32)
16805 00001044 2E803E[CB0E]00 cmp     byte [cs:new_genioct1], 0
16806 0000104A 7503      jnz     short sdp_4                ; new type (FAT32) structure
16807 0000104C 8D7728    lea     si, [bx+28h]                ; [bx+A_DEVICEPARAMETERS.DP_SECTORTABLE]
16808                ;      ; offset 31+9 (A_BPB.size+9)
16809
16810                sdp_4:
16811                ;      ;
16812                ; 17/10/2022
16813 0000104F 2E8E06[3000] mov     es, [cs:BIOSDATAWORD]
16814                ;mov    es, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
16815                ;      ; Trash our bds pointer
16816
16817 00001054 47      StoreSectorInfo: inc     di
16818 00001055 47      inc     di                ; Skip over cylinder and head
16819 00001056 AD      lodsw                ; Get sector id
16820 00001057 AA      stosb                ; Copy it
16821 00001058 AD      lodsw                ; Get sector size
16822
16823                ; 24/12/2023
16824                ; 02/09/2023 (PCDOS 7.1)
16825                ;call   SectSizeToSectIndex
16826                ; 18/04/2024
16827                ;cmp     ah, 3 ; 02/09/2023
16828 00001059 80FC02    cmp     ah, 2                ; (0=>128,1=>256,2=>512,3=>1024)
16829                ;      ; examine upper byte only
16830 0000105C 7704      ja      short OneK
16831 0000105E 88E0      mov     al, ah                ; value in AH is the index!
16832 00001060 EB02      jmp     short sdp_s
16833
16834 00001062 B003      OneK:    mov     al, 3                ; 1024 bytes per sector
16835
16836 00001064 AA      sdp_s:  stosb                ; Store sector SIZE index
16837 00001065 E2ED      loop   StoreSectorInfo
16838
16839 00001067 F8      SectorInfoSaved: cll
16840 00001068 C3      retn
16841
16842                ; -----
16843
16844 00001069 B00C      TooManyPerTrack: mov     al, 0Ch
16845 0000106B F9      stc
16846 0000106C C3      retn
16847
16848                ; -----
16849
16850                ; 16/10/2022
16851                ; =====
16852                ; FormatTrack:
16853                ; if specialfunction byte is 1, then this is a status call to see if there is
16854                ; rom support for the combination of sec/trk and # of cyl, and if the
16855                ; combination is legal. if specialfunction byte is 0, then format the track.
16856                ;
16857                ; input: ES:di points to bds for drive
16858                ;
16859                ; output:
16860                ; for status call:
16861                ; specialfunction byte set to:
16862                ; 0 - rom support + legal combination
16863                ; 1 - no rom support

```

```

16864 ;          2 - illegal combination
16865 ;          3 - no media present
16866 ;      carry cleared.
16867 ;
16868 ;      for format track:
16869 ;          carry set if error
16870 ;
16871 ; =====
16872 ;
16873 ; 16/03/2019
16874 ;      24/12/2023 - Retro DOS 5.0
16875 ;      (PCDOS 7.1 IBMBIO.COM - BIOSCODE:10B7h)
16876 ;
16877 ; 19/10/2022
16878 FormatTrack:
16879     lds     bx, [ptrsav]
16880     lds     bx, [bx+19] ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET
16881     test    byte [bx], 1 ; [bx+A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS]
16882     ;      STATUS_FOR_FORMAT
16883     jz      short DoFormatTrack
16884     push    ds
16885     ; 17/10/2022
16886     mov     ds, [cs:BIOSDATAWORD]
16887     ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
16888     call    SetMediaForFormat ; Also moves current Dpt to TempDpt
16889     pop     ds
16890     mov     [bx], al ; [bx+A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS]
16891     cll
16892     ret
16893 ; -----
16894 DoFormatTrack:
16895     ; 24/12/2023 - Retro DOS 5.0
16896     cmp     byte [es:di+3Eh], 5
16897     ;cmp     byte [es:di+34], 5 ; [es:di+BDS.formfactor]
16898     ;      DEV_HARDDISK
16899     jnz     short DoFormatDiskette
16900     ; 17/10/2022
16901     mov     ds, [cs:BIOSDATAWORD]
16902     ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
16903     ;      Point to Bios_Data (at 2C7h:30h or 70h:25A0h)
16904     jmp     VerifyTrack
16905 ; -----
16906 DoFormatDiskette:
16907     mov     cx, [bx+1]
16908     mov     dx, [bx+3]
16909     test    byte [bx], 2
16910     ; 17/10/2022
16911     mov     ds, [cs:BIOSDATAWORD]
16912     ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
16913     ;      Setup ds-> Bios_Data for verify
16914     jz      short DoFormatDiskette_1
16915     jmp     VerifyTrack_Err
16916 ; -----
16917 DoFormatDiskette_1:
16918     call    SetMediaForFormat ; Also moves current Dpt to TempDpt
16919     cmp     al, 1 ; ROM support for sec/trk,# trks comb?
16920     jz      short NeedToSetDasd ; Old rom
16921     cmp     al, 3 ; Time out error?
16922     jnz     short NoSetDasd ; No, fine. (at this point, don't care
16923     ;      about the illegal combination)
16924     jmp     short FormatFailed
16925 ; -----
16926 NeedToSetDasd:
16927     push    dx
16928     call    SetDasd ; INT 13h, AH=17h
16929     pop     dx
16930 NoSetDasd:
16931     call    checksingle ; Do any needed diskette swapping
16932     mov     ax, dx ; Get track from packet
16933     mov     [trknum], ax
16934     mov     [hdnum], cl ; Store head from packet
16935     mov     ah, cl
16936     mov     bx, tracktable
16937     mov     cx, [sectorspertrack]
16938     ; 24/12/2023 - Retro DOS 5.0
16939     jcxz    set_fmt_retry_count
16940 StoreCylinderHead:
16941     mov     [bx], ax ; Store into TrackTable
16942     add     bx, 4 ; Skip to next sector field
16943     loop    StoreCylinderHead
16944 set_fmt_retry_count:
16945     ; 24/12/2023
16946     mov     cx, 5 ; MAXERR - Set up retry count
16947     ; 02/09/2023
16948     mov     cl, 5
16949 FormatRetry:
16950     push    cx
16951     mov     bx, tracktable
16952     mov     al, [sectorspertrack]
16953     mov     ah, 5 ; romformat
16954     mov     [xfer_seg], ds
16955     call    ToRom
16956     pop     cx
16957     jnb     short FormatError
16958     push    cx
16959     ; Now verify the sectors just formatted.
16960     ; NOTE: because of bug in some BIOSes we have to
16961     ;      set ES:BX to 00:00
16962     push    bx
16963     xor     bx, bx
16964     mov     [xfer_seg], bx
16965     mov     al, [sectorspertrack]
16966     mov     ah, 4 ; romverify
16967     mov     cl, 1
16968     call    ToRom
16969     pop     bx
16970     pop     cx
16971     jnb     short FormatOk
16972 FormatError:
16973     call    ResetDisk
16974     mov     byte [had_format_error], 1
16975     push    ax
16976     push    cx
16977     push    dx
16978     call    SetMediaForFormat
16979     cmp     al, 1
16980     jnz     short WhileErr
16981     call    SetDasd
16982 WhileErr:
16983     pop     dx
16984     pop     cx
16985     pop     ax
16986
16987

```

```

16988 0000111C E2BD
16989
16990 0000111E C606[AA05]01
16991
16992 00001123 80FC06
16993 00001126 7502
16994 00001128 B480
16995
16996 0000112A E97CFC
16997
16998
16999
17000 0000112D C606[AA05]00
17001 00001132 C3
17002
17003
17004
17005
17006
17007
17008
17009
17010
17011
17012
17013
17014
17015 00001133 1E
17016 00001134 C51E[1200]
17017 00001138 C55F13
17018
17019
17020
17021 0000113B 8B4F03
17022 0000113E 8B4701
17023 00001141 8B5705
17024 00001144 8A1F
17025
17026 00001146 1F
17027 00001147 C606[2001]04
17028 0000114C 890E[3301]
17029 00001150 A2[3201]
17030 00001153 8B0E[AA04]
17031
17032
17033
17034
17035
17036
17037
17038
17039
17040
17041
17042
17043
17044
17045 00001157 F6C302
17046 0000115A 7421
17047 0000115C 89D0
17048 0000115E 08E4
17049 00001160 752C
17050 00001162 F6E1
17051 00001164 08E4
17052 00001166 7526
17053 00001168 89C1
17054
17055 0000116A 26F6453F01
17056
17057
17058
17059
17060 0000116F 740C
17061
17062
17063
17064 00001171 F606[A004]80
17065
17066
17067 00001176 7405
17068 00001178 C606[A704]01
17069
17070 0000117D 31C0
17071 0000117F 31DB
17072 00001181 891E[A804]
17073 00001185 E83F00
17074 00001188 C606[A704]00
17075 0000118D C3
17076
17077
17078
17079 0000118E B401
17080 00001190 E916FC
17081
17082
17083
17084
17085
17086
17087
17088
17089
17090
17091
17092
17093
17094 00001193 C606[2001]02
17095 00001198 EB05
17096
17097
17098
17099
17100
17101
17102
17103
17104
17105
17106
17107
17108 0000119A C606[2001]03
17109
17110
17111

```

```

loop FormatRetry
FormatFailed:
mov byte [had_format_error], 1
; Set the format error flag
cmp ah, 6 ; DSK_CHANGE_LINE_ERR - convert change line
jnz short DoMapIt ; Error to timeout error
mov ah, 80h ; DSK_TIMEOUT_ERR
DoMapIt:
jmp maperror
; -----
FormatOk:
mov byte [had_format_error], 0 ; reset the format error flag
ret
; -----
; 16/10/2022
; =====
;
; VerifyTrack:
;
; input: ES:di points to bds for drive
; =====
; 24/12/2023 - Retro DOS 5.0
VerifyTrack:
push ds
lds bx, [ptrsav] ; DS:BX points to request header.
lds bx, [bx+19] ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
; Come here with DS:[BX] -> packet, ES:[DI] -&; bds
mov cx, [bx+3] ; [bx+A_VERIFYPACKET.VP_CYLINDER]
mov ax, [bx+1] ; [bx+A_VERIFYPACKET.VP_HEAD]
mov dx, [bx+5] ; [bx+A_FORMATPACKET.FP_TRACKCOUNT]
mov bl, [bx] ; [bx+A_FORMATPACKET.FP_SPECIALFUNCTIONS]
; Get option flag word
pop ds
mov byte [rflag], 4 ; romverify
mov [curtrk], cx
mov [curhd], al ; ASSUME heads < 256
mov cx, [sectorspertrack]
; Check specialfunctions to see if DO_FAST_FORMAT has been
; specified if not we should go to the normal track verification
; routine. If fast format has been specified we should get the
; number of tracks to be verified and check it to see if it is
; > 255. If it is then it is an error and we should go to
; VerifyTrack_Err. If not multiply the number of tracks by the
; sectors per track to get the total number of sectors to be
; verified. This should also be less than equal to 255
; otherwise we go to same error exit. If everything is okay
; we initialise cx to the total sectors. use ax as a temporary
; register.
test bl, 2 ; Special function requested?
jz short NormVerifyTrack ; DO_FAST_FORMAT
mov ax, dx ; Get ax = number of trks to verify
or ah, ah
jnz short VerifyTrack_Err ; #tracks > 255
mul cl
or ah, ah
jnz short VerifyTrack_Err ; #sectors > 255
mov cx, ax
; 24/12/2023
test byte [es:di+3Fh], 1 ; PCDOS 7.1 IBMBIO.COM
; 10/12/2022
; test byte [es:di+35], 1
; ;test word [es:di+35], 1 ; [es:di+BDS.flags]
; ;fnon_removable
jz short NormVerifyTrack ; Multitrack operation = on?
; 10/12/2022
; 19/10/2022
test byte [multirk_flag], 80h
; test word [multirk_flag], 80h ; MULTI_TRK_ON
; ;test ds:multirk_flag, 80h ; MULTI_TRK_ON
jz short NormVerifyTrack
mov byte [multitrk_format_flag], 1
NormVerifyTrack:
xor ax, ax ; 1st sector
xor bx, bx
mov [xfer_seg], bx ; Use 0:0 as the transfer address for verify
call TrackIo
mov byte [multitrk_format_flag], 0
ret
; -----
VerifyTrack_Err:
mov ah, 1
jmp maperror
; -----
; 16/10/2022
; =====
;
; ReadTrack:
;
; input: ES:di points to bds for drive
; =====
ReadTrack:
mov byte [rflag], 2 ; romread
jmp short ReadWriteTrack
; -----
WriteTrack:
; =====
;
; WriteTrack:
;
; input: ES:di points to bds for drive
; =====
mov byte [rflag], 3 ; romwrite
; Fall into ReadWriteTrack

```

```

17112 ; =====
17113 ;
17114 ; readWriteTrack:
17115 ;
17116 ; input:
17117 ;   ES:di points to bds for drive
17118 ;   rFlag - 2 for read,3 for write
17119 ;
17120 ; =====
17121 ;
17122 ReadWriteTrack:
17123 ;   save bds pointer segment so we can use it to access
17124 ;   our packet. Notice that this is not the standard register
17125 ;   assignment for accessing packets
17126 ;
17127 ; 19/10/2022
17128 0000119F 06      push     es
17129 000011A0 C41E[1200] les     bx, [ptrsav] ; ES:BX-> to request header
17130 000011A4 26C45F13 les     bx, [es:bx+19] ; [es:bx+IOCTL_REQ.GENERICIOCTL_PACKET]
17131 000011A8 268B4703 mov     ax, [es:bx+3] ; [es:bx+A_TRACKREADWRITEPACKET.TRWP_CYLINDER]
17132 000011AC A3[3301] mov     [curtrk], ax
17133 000011AF 268B4701 mov     ax, [es:bx+1] ; [es:bx+A_TRACKREADWRITEPACKET.TRWP_HEAD]
17134 000011B3 A2[3201] mov     [curhd], al ; Assume heads < 256!!!
17135 000011B6 268B4705 mov     ax, [es:bx+5] ; [es:bx+A_TRACKREADWRITEPACKET.TRWP_FIRSTSECTOR]
17136 000011BA 268B4F07 mov     cx, [es:bx+7] ; [es:bx+A_TRACKREADWRITEPACKET.TRWP_SECTORSTOREADWRITE]
17137 000011BE 26C45F09 les     bx, [es:bx+9] ; [es:bx+A_TRACKREADWRITEPACKET.TRWP_TRANSFERADDRESS]
17138 ; Get transfer address
17139 ;
17140 ; we just trashed our packet address, but we no longer care
17141 ;
17142 000011C2 8C06[A804] mov     [xfer_seg], es ; Pass transfer segment
17143 000011C6 07      pop     es
17144 ;
17145 ; Fall into TrackIo
17146 ;
17147 ; ===== S U B   R O U T I N E =====
17148 ;
17149 ; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
17150 ;
17151 ; =====
17152 ;
17153 ; TrackIo:
17154 ;   performs track read/write/verify
17155 ;
17156 ;   input:
17157 ;     rFlag      - 2 = read
17158 ;                3 = write
17159 ;                4 = verify
17160 ;   AX - Index into track table of first sector to io
17161 ;   CX - Number of sectors to io
17162 ;   Xfer_Seg:BX - Transfer address
17163 ;   ES:DI      - Pointer to bds
17164 ;   CurTrk     - Current cylinder
17165 ;   CurHd      - Current head
17166 ;
17167 ; =====
17168 ;
17169 ; 16/03/2019 - Retro DOS v4.0
17170 ;
17171 ; 24/12/2023 - Retro DOS 5.0
17172 ;
17173 ; 19/10/2022
17174 ;
17175 ; TrackIo:
17176 ;   Procedure 'disk' will pop stack to
17177 ;   mov [spsav], sp ; SpSav and return if error
17178 ;   call checksingle ; Ensure correct disk is in drv
17179 ;   cmp byte [media_set_for_format], 1 ; See if we have already set disk
17180 ;   jz short Dptalreadysset ; base table
17181 ;   push ax ; set up tables and variables for i/o
17182 ;   push cx
17183 ;   call iosetup
17184 ;   pop cx
17185 ;   pop ax
17186 ;
17187 ; Dptalreadysset:
17188 ;   mov si, tracktable ; first sector to be io'd
17189 ;   ; 24/12/2023
17190 ;   add ax, ax ; PC DOS 7.1 IBMBIO.COM
17191 ;   add ax, ax
17192 ;   shl ax, 1
17193 ;   shl ax, 1
17194 ;   add si, ax
17195 ;
17196 ; WE WANT:
17197 ; CX to be the number of times we have to loop
17198 ; DX to be the number of sectors we read on each iteration
17199 ;
17200 ;
17201 ; 24/12/2023
17202 000011E8 26F6453F08 test     byte [es:di+3Fh], 8 ; PC DOS 7.1 IBMBIO.COM
17203 ; 12/12/2022
17204 ; test byte [es:di+23h], 8
17205 ; ;test word [es:di+35], 8 ; [es:di+BDS.flags]
17206 ; ;good_tracklayout
17207 000011ED 7402      jz      short ionextsector
17208 ;
17209 000011EF 87D1      xchg     dx, cx ; HEY! we can read all secs in one blow
17210 ;
17211 000011F1 51      push     cx
17212 000011F2 52      push     dx
17213 000011F3 46      inc     si
17214 000011F4 46      inc     si ; Skip over the cylinder and head in
17215 ; ;the track table
17216 000011F5 AC      lodsb    ; Get sector ID from track table
17217 000011F6 A2[3101] mov     [cursec], al
17218 ;
17219 ; assumptions for a fixed disk multi-track disk i/o
17220 ; 1). In the input CX (# of sectors to go) to TrackIo,
17221 ; only CL is valid.
17222 ; 2). Sector size should be set to 512 bytes.
17223 ; 3). Good track layout
17224 ;
17225 ; 24/12/2023
17226 000011F9 26F6453F01 test     byte [es:di+3Fh], 1 ; PC DOS 7.1 IBMBIO.COM
17227 ; 12/12/2022
17228 ; test byte [es:di+23h], 1
17229 ; ;test word [es:di+35], 1 ; [es:di+BDS.flags]
17230 ; ;fnon_removable ; Fixed disk?
17231 000011FE 7414      jz      short IoRemovable ; No
17232 ;
17233 ; 12/12/2022
17234 00001200 F606[A004]80 test     byte [multrk_flag], 80h
17235 ; test word [multrk_flag], 80h ; MULTI_TRK_ON

```

```

17236                                     ; Allow multi-track operation?
17237 00001205 740D                jz     short IoRemovable ; No,don't do that.
17238 00001207 8916[2201]         mov     [secCnt], dx
17239 0000120B 89D0                mov     ax, dx
17240 0000120D E823FA              call    Disk
17241 00001210 5A                  pop     dx
17242 00001211 59                  pop     cx
17243 00001212 F8                  cld
17244 00001213 C3                  retn
17245                                     ; -----
17246
17247 IoRemovable:
17248 00001214 AC                    lodsb                     ; Get sector size index      from track
17249                                     ; table and save it
17250 00001215 50                    push     ax
17251 00001216 56                    push     si
17252 00001217 1E                    push     ds             ; Save Bios_Data
17253 00001218 50                    push     ax
17254 00001219 8A26[2C01]          mov     ah, [eot]       ; Preserve whatever might be in      ah
17255                                     ; Fetch EOT while ds-> Bios_Data
17256 0000121D C536[2D01]          lds     si, [dpt]
17257 00001221 884403              mov     [si+3], al      ; [si+DISK_PARMS.DISK_SECTOR_SIZ]
17258 00001224 886404              mov     [si+4], ah      ; [si+DISK_PARMS.DISK_EOT]
17259 00001227 58                    pop     ax
17260 00001228 1F                    pop     ds
17261 00001229 88D0              mov     al, dl
17262 0000122B A3[2201]          mov     [secCnt], ax
17263 0000122E E802FA              call    Disk
17264 00001231 5E                    pop     si             ; Advance buffer pointer by adding
17265                                     ; sector size
17266                                     ; pop     ax
17267                                     ; 24/12/2023
17268 00001232 59                    pop     cx
17269
17270                                     ; 02/09/2023 (PCDOS 7.1)
17271                                     ; call    SectorSizeIndexToSectorSize
17272                                     ; mov     cl, al ; 24/12/2023
17273 00001233 888000              mov     ax, 128
17274 00001236 D3E0              shl     ax, cl
17275
17276 00001238 01C3                add     bx, ax
17277 0000123A 5A                  pop     dx
17278 0000123B 59                  pop     cx
17279 0000123C E2B3              loop    ionextsector
17280 0000123E 803E[A905]01      cmp     byte [media_set_for_format], 1
17281                                     ; jz     short NoNeedDone
17282                                     ; 12/12/2022
17283 00001243 7404                je      short NoNeedDone2
17284 00001245 E877F9              call    done           ; set time of last access, and reset
17285                                     ; entries in Dpt.
17286
17287 00001248 F8                  cld      ; not necessary ('done' clears cf) ; 24/12/2023
17288
17289 00001249 C3                  retn
17290
17291                                     ; ===== S U B   R O U T I N E =====
17292
17293                                     ; -----
17294                                     ;
17295                                     ; The sector size in bytes needs to be converted to an index value for the ibm
17296                                     ; rom. (0=>128,1=>256,2=>512,3=>1024). It is assumed that only these values
17297                                     ; are permissible.
17298                                     ;
17299                                     ; On Input  AX contains sector size in bytes
17300                                     ; On Output AL Contains index
17301                                     ; All other registers preserved
17302                                     ; -----
17303
17304                                     ; 02/09/2023
17305 SectSizeToSectIndex:
17306                                     ;
17307                                     ; cmp     ah, 2             ; (0=>128,1=>256,2=>512,3=>1024)
17308                                     ;                                     ; examine upper      byte only
17309                                     ;
17310                                     ; ja      short OneK
17311                                     ; mov     al, ah             ; value in AH is the index!
17312                                     ; retn
17313
17314                                     ; -----
17315
17316 OneK:
17317                                     ; mov     al, 3
17318                                     ; retn
17319
17320                                     ; ===== S U B   R O U T I N E =====
17321
17322                                     ; 02/09/2023
17323 SectorSizeIndexToSectorSize:
17324                                     ; mov     cl, al
17325                                     ; mov     ax, 128
17326                                     ; shl     ax, cl
17327                                     ; retn
17328
17329                                     ; ===== S U B   R O U T I N E =====
17330
17331                                     ; 16/10/2022
17332                                     ; -----
17333                                     ;
17334 SetDASD
17335                                     ;
17336                                     ; Set up the rom for formatting.
17337                                     ; we have to tell the rom bios what type of disk is in the drive.
17338                                     ;
17339                                     ; On Input  - ES:di - Points to bds
17340                                     ; -----
17341
17342                                     ; 24/12/2023 - Retro DOS 5.0
17343                                     ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:129Bh)
17344
17345                                     ; 19/10/2022
17346
17347 SetDasd:
17348 0000124A 803E[AA05]01      cmp     byte [had_format_error], 1 ;
17349                                     ; See if we've previously set dasd type
17350 0000124F 740C                jz      short DoSetDasd
17351                                     ; 24/12/2023
17352 00001251 26F6453F80          test    byte [es:di+3Fh], 80h
17353                                     ; 10/12/2022
17354                                     ; test    byte [es:di+23h], 80h
17355                                     ; ;test    word [es:di+23h], 80h ; [es:di+BDS.flags]
17356                                     ; ; set_dasd_true
17357 00001256 7446                jz      short DasdHasBeenSet
17358                                     ; 24/12/2023
17359 00001258 2680653F7F          and     byte [es:di+3Fh], 7Fh

```



```

17360 ; 10/12/2022
17361 ;and byte [es:di+23h], 7Fh
17362 ;;and word [es:di+23h], 0FF7Fh ; [es:di+BDS.flags]
17363 ; ~set_dasd_true
17364
17365 0000125D C606[AA05]00 DoSetDasd: mov byte [had_format_error], 0 ; Reset it
17366 00001262 C606[3B01]50 mov byte [gap_patch], 50h ; Format gap for 48tpi disks
17367 00001267 B004 mov al, 4
17368 ; 24/12/2023
17369 00001269 268A653E mov ah, [es:di+3Eh]
17370 ; 02/09/2023
17371 ;mov ah, [es:di+22h] ; [es:di+BDS.formfactor]
17372 0000126D 80FC02 cmp ah, 2
17373 ;cmp byte [es:di+22h], 2 ; [es:di+BDS.formfactor]
17374 ; DEV_3INCH720KB
17375 00001270 7414 jz short DoSet
17376 ; 24/12/2023
17377 00001272 B001 mov al, 1
17378 ;cmp ah, 1
17379 00001274 38C4 cmp ah, al ; 1
17380 ;cmp byte [es:di+22h], 1 ; [es:di+BDS.formfactor]
17381 ; DEV_5INCH96TPI
17382 ;jz short GotBig
17383 ; 24/12/2023
17384 ;mov al, 1
17385 ;jmp short DoSet
17386 ; 02/09/2023
17387 00001276 750E jnz short DoSet
17388
17389 GotBig: ;mov al, 2 ; 160/320k in a 1.2 meg drive
17390 ; 02/09/2023
17391 00001278 40 inc ax ; mov al, 2
17392 00001279 803E[A805]00 cmp byte [mediatype], 0
17393 0000127E 7506 jnz short DoSet
17394 ;mov al, 3 ; 1.2meg in a 1.2meg drive
17395 ; 10/12/2022
17396 ;inc al ; al = 3
17397 ; 18/12/2022
17398 00001280 40 inc ax ; al = 3
17399 00001281 C606[3B01]54 mov byte [gap_patch], 54h
17400
17401 00001286 1E DoSet: push ds
17402 00001287 56 push si
17403
17404 ;mov ds, [zeroseg] ; Point to interrupt vectors
17405 ; 02/09/2023
17406 00001288 31F6 xor si, si
17407 0000128A 8EDE mov ds, si ; 0
17408
17409 0000128C C5367800 lds si, [DSKADR]
17410 ;lds si, [78h] ; [DSKADR] (Int 1Eh)
17411 ;lds si, ds:78h
17412
17413 00001290 C644090F mov byte [si+9], 0Fh ;
17414 ; [si+DISK_PARAMS.DISK_HEAD_STTL]
17415 00001294 5E pop si
17416 00001295 1F pop ds
17417 00001296 B417 mov ah, 17h
17418 00001298 268A5504 mov dl, [es:di+4]
17419 0000129C CD13 int 13h ; DISK - DISK - SET TYPE (AT,XT2,XT286,CONV,PS
17420 ; AL = disk type AL = 03h - high-capacity disk in high-capacity drive
17421
17422 0000129E 268A6513 DasdHasBeenSet: mov ah, [es:di+13h] ; [es:di+BDS.secptrack]
17423 000012A2 8826[3701] mov [format_eot], ah
17424 000012A6 C3 retn
17425
17426 ; ===== S U B R O U T I N E =====
17427
17428 ; 16/10/2022
17429
17430 ; -----
17431 ;
17432 ; Set Media Type for Format
17433 ; Performs the int 13 with ah = 18h to see if the medium described in the
17434 ; BPB area in the BDS can be handled by the rom.
17435 ; On Input, ES:DI -> current BDS.
17436 ; The status of the operation is returned in AL
17437 ;
17438 ; - 0 - if the support is available, and the combination is valid.
17439 ; - 1 - no rom support
17440 ; - 2 - illegal combination
17441 ; - 3 - no media present (rom support exists but cannot determine now)
17442 ;
17443 ; Flags also may be altered. All other registers preserved.
17444 ; If the call to rom returns no error, then the current Dpt is "replaced" by
17445 ; the one returned by the rom. This is Done by changing the pointer in [Dpt]
17446 ; to the one returned. the original pointer to the disk base table is stored
17447 ; in Tempdpt, until it is restored.
17448 ;
17449 ; -----
17450
17451 ; 24/12/2023 - Retro DOS 5.0
17452
17453 ; 19/10/2022
17454 SetMediaForFormat:
17455 000012A7 51 push cx
17456 000012A8 52 push dx
17457
17458 ; If we have a format error, then do not change Dpt, TempDpt.
17459 ; but we need to call int 13h, ah=18h again.
17460
17461 000012A9 803E[AA05]01 cmp byte [had_format_error], 1
17462 000012AE 7425 jz short SkipSaveDskAdr
17463 000012B0 30C0 xor al, al ; If already done return 0
17464 000012B2 803E[A905]01 cmp byte [media_set_for_format], 1
17465 000012B7 7502 jnz short DoSetMediaForFormat
17466 000012B9 EB7D jmp SetMediaRet ; Media already set
17467
17468 ; -----
17469 DoSetMediaForFormat:
17470 000012BB 06 push es
17471 000012BC 56 push si
17472
17473 ; 02/09/2023
17474 ;mov es, [zeroseg] ; Point to interrupt vectors
17475 000012BD 31F6 xor si, si ; 0
17476 000012BF 8EC6 mov es, si
17477
17478 000012C1 26C4367800 les si, [es:DSKADR]
17479 ;les si, es:78h ; [es:DSKADR]
17480 ; Get pointer to disk base table
17481 000012C6 8936[2D01] mov [dpt], si
17482 000012CA 8C06[2F01] mov [dpt+2], es ; Save pointer to table
17483

```

```

17484             ; Initialize the head settle time to 0Fh. See the offsets
17485             ; given in dskprm.inc.
17486
17487 000012CE 26C644090F      mov     byte [es:si+9], 0Fh ; [es:si+DISK_PARAMS.DISK_HEAD_STTL]
17488 000012D3 5E             pop      si
17489 000012D4 07             pop      es
17490 skipSaveDskAdr:
17491             ; 24/12/2023
17492 000012D5 268B4D41      mov     cx, [es:di+41h] ; (PCDOS 7.1 IBMBIO.COM)
17493             ;mov    cx, [es:di+25h] ; [es:di+BDS.cylinders]
17494 000012D9 49             dec     cx
17495 000012DA 80E503      and     ch, 3
17496 000012DD D0CD      ror     ch, 1
17497 000012DF D0CD      ror     ch, 1
17498 000012E1 86E9      xchg    ch, cl
17499 000012E3 260A4D13      or      cl, [es:di+13h] ; [es:di+BDS.secperttrack]
17500 000012E7 268A5504      mov     dl, [es:di+4] ; [es:di+BDS.drivenum]
17501 000012EB 06             push    es
17502 000012EC 1E             push    ds
17503 000012ED 56             push    si
17504 000012EE 57             push    di
17505 000012EF B418      mov     ah, 18h
17506 000012F1 CD13      int     13h ; DISK - SET MEDIA TYPE FOR FORMAT (AT model 3x9,XT2,XT286,PS)
17507             ; DL = drive number, CH = lower 8 bits of number of tracks, CL = sectors
per track
17508 000012F3 7231      jc      short FormaStatErr
17509 000012F5 803E[AA05]01    cmp     byte [had_format_error], 1
17510 000012FA 7423      jz      short skip_disk_base_setting
17511 000012FC 06             push    es ; Save segment returned by the rom
17512
17513             ; 02/09/2023
17514             ;mov     es, [zeroseg] ; Point to interrupt vector segment
17515 000012FD 31F6      xor     si, si
17516 000012FF 8EC6      mov     es, si ; 0
17517 00001301 06             push    es ; * ; 02/09/2023
17518
17519 00001302 26C4367800      les     si, [es:DSKADR]
17520             ;les     si, es:78h ; [es:DSKADR] (Int 1Eh)
17521             ; Get current disk base table
17522 00001307 8936[AB05]      mov     [tempdpt], si
17523 0000130B 8C06[AD05]      mov     [tempdpt+2], es ; Save it
17524
17525             ; 02/09/2023
17526             ;mov     es, [zeroseg]
17527             ;xor     si, si ; 0
17528             ;mov     es, si
17529 0000130F 07             pop     es ; * ; 02/09/2023
17530
17531             ;mov     es:78h, di
17532 00001310 26893E7800      mov     [es:DSKADR], di
17533             ;pop     word ptr es:7Ah ; replace with one returned by rom
17534 00001315 268F067A00      pop     word [es:DSKADR+2]
17535 0000131A C606[A905]01    mov     byte [media_set_for_format], 1
17536 skip_disk_base_setting:
17537 0000131F 30C0      xor     al, al ; Legal combination + rom support code
17538             ;mov     ds:had_format_error, al ; Reset the flag
17539 00001321 A2[AA05]      mov     [had_format_error], al
17540 00001324 EB0E      jmp     short PopStatRet
17541
17542             ; -----
17543 FormaStatErr:
17544             ; 10/12/2022
17545 00001326 B003      mov     al, 3
17546
17547             cmp     ah, 0Ch ; DSK_ILLEGAL_COMBINATION
17548             ; Illegal combination = 0Ch
17549 0000132B 7406      jz      short FormatStatIllegalComb
17550 0000132D 80FC80      cmp     ah, 80h ; DSK_TIMEOUT_ERR
17551 00001330 7402      jz      short FormatStatTimeOut
17552             ; 10/12/2022
17553             ;dec     al
17554             ; 18/12/2022
17555 00001332 48             dec     ax
17556             ; al = 2
17557             ;mov     al, 1 ; Function not supported.
17558             ;jmp     short PopStatRet
17559
17560             ; -----
17561 FormatStatIllegalComb:
17562             ; 10/12/2022
17563             ;dec     al ; 3 -> 2 or 2 -> 1
17564             ; 18/12/2022
17565 00001333 48             dec     ax
17566             ; al = 2
17567             ;mov     al, 2 ; Function supported, but
17568             ; Illegal sect/trk, trk combination.
17569             ; 10/12/2022
17570             ;jmp     short PopStatRet
17571
17572             ; -----
17573 FormatStatTimeOut:
17574             ; 10/12/2022
17575             ; al = 3
17576             ;mov     al, 3 ; Function supported, but
17577             ; Media not present.
17578
17579 00001334 5F             pop     di
17580 00001335 5E             pop     si
17581 00001336 1F             pop     ds
17582 00001337 07             pop     es
17583
17584 00001338 5A             pop     dx
17585 00001339 59             pop     cx
17586 0000133A C3             retn
17587
17588             ; ===== S U B R O U T I N E =====
17589
17590             ; 16/10/2022
17591
17592             ; -----
17593             ;
17594             ; RESET THE DRIVE
17595             ;
17596             ; we also set [Step_Drv] to -1 to force the main disk routine to use the
17597             ; slow head settle time for the next operation. this is because the reset
17598             ; operation moves the head to cylinder 0, so we need to do a seek the next
17599             ; time around - there is a problem with 3.5" drives in that the head does
17600             ; not settle down in time, even for read operations!!
17601             ;
17602             ; -----
17603
17604 ResetDisk:
17605 0000133B 50             push    ax
17606

```



```

17607             ; 02/09/2023
17608 0000133C B80100      mov     ax, 1 ; PC DOS 7.1
17609 0000133F 3806[A905]  cmp     [media_set_for_format], al ; 1
17610             ; cmp     byte [media_set_for_format], 1
17611             ; Reset while formatting?
17612 00001343 7503         jnz     short ResetDisk_cont
17613             ; Then verify operation in "fmt & vrfy"
17614             ; mov     byte [had_format_error], 1 ; Might have failed.
17615 00001345 A2[AA05]      mov     [had_format_error], al ; 1
17616             ResetDisk_cont:
17617             ; 02/09/2023 (ah=0)
17618             ; xor     ah, ah ; So signals that we had a format error
17619 00001348 CD13         int     13h ; DISK - RESET DISK SYSTEM
17620             ; DL = drive (if bit 7 is set both hard disks and floppy disks reset)
17621 0000134A C606[7600]FF  mov     byte [step_drv], 0FFh ; -1
17622             ; Zap up the speed
17623 0000134F 58             pop     ax
17624 00001350 C3             retn
17625
17626             ; ===== S U B R O U T I N E =====
17627
17628             ; 16/10/2022
17629
17630             ; -----
17631             ;
17632             ; This routine sets up the drive parameter table with the values needed for
17633             ; format, does an int 13. values in Dpt are restored after a verify is done.
17634             ;
17635             ; on entry - ES:DI - points to bds for the drive
17636             ; Xfer_Seg:BX - points to trkbuf
17637             ; AL - number of sectors
17638             ; AH - int 13 function code
17639             ; CL - sector number for verify
17640             ; DS - Bios_Data
17641             ;
17642             ; ON EXIT - DS,DI,ES,BX remain unchanged.
17643             ; AX and flags are the results of the int 13
17644             ; -----
17645
17646             ; 24/12/2023 - Retro DOS 5.0
17647
17648             ; 19/10/2022
17649
17650             ToRom:
17651 00001351 53             push    bx
17652 00001352 56             push    si
17653
17654             ; Compaq bug fix - check whether we are using new ROM
17655             ; functionality to set up format, not merely if it exists.
17656             ; This was formerly a check against [new_rom]
17657
17658 00001353 F606[A905]01   test     byte [media_set_for_format], 1
17659 00001358 7534         jnz     short GotValidDpt
17660 0000135A 50             push    ax
17661 0000135B 06             push    es ; Save bds segment
17662             ; 24/12/2023
17663 0000135C 26807D3E02     cmp     byte [es:di+3Eh], 2
17664             ; cmp     byte [es:di+22h], 2 ; [es:di+BDS.formfactor]
17665             ; ffSmall ; is it a 3.5" drive?
17666             ; 24/12/2023
17667             ; pushf ; not necessary ; (Save the cmp result)
17668 00001361 8E06[1A00]     mov     es, [zeroseg]
17669             ; les     si, es:78h ; Get pointer to disk base table
17670 00001365 26C4367800     les     si, [es:DSKADR]
17671             ; mov     word ptr ds:dpt, si
17672             ; mov     word ptr ds:dpt+2, es ; Save pointer to table
17673 0000136A 8936[2D01]     mov     [dpt], si
17674 0000136E 8C06[2F01]     mov     [dpt+2], es ; Save pointer to table
17675
17676 00001372 A0[3701]       mov     al, [fmt_eot]
17677 00001375 26884404     mov     [es:si+4], al ; [es:si+DISK_PARAMS.DISK_EOT]
17678 00001379 A0[3B01]       mov     al, [gap_patch]
17679 0000137C 26884407     mov     [es:si+7], al ; [es:si+DISK_PARAMS.DISK_FORMAT_GAP]
17680             ; Important for format
17681 00001380 26C644090F     mov     byte [es:si+9], 0Fh ; [es:si+DISK_PARAMS.DISK_HEAD_STTL]
17682             ; Assume we are doing a seek operation
17683             ; Setup motor start correctly for 3.5" drives
17684             ; 24/12/2023
17685             ; popf ; Get result of earlier cmp
17686 00001385 7505         jnz     short MotorStrtOK
17687 00001387 26C6440A04     mov     byte [es:si+0Ah], 4 ; [es:si+DISK_PARAMS.DISK_MOTOR_STRT]
17688
17689 0000138C 07             pop     es ; Restore bds segment
17690 0000138D 58             pop     ax
17691
17692 0000138E 8B16[3901]     mov     dx, [trknum] ; Set track number
17693 00001392 8B05         mov     ch, dl ; Set low 8 bits in ch
17694 00001394 268A5504     mov     dl, [es:di+4] ; Set drive number
17695 00001398 8A36[3801]     mov     dh, [hdnum] ; Set head number
17696 0000139C 06             push    es ; Save bds segment
17697 0000139D 8E06[A804]     mov     es, [xfer_seg]
17698 000013A1 CD13         int     13h ; DISK -
17699 000013A3 07             pop     es ; Restore bds segment
17700 000013A4 5E             pop     si
17701 000013A5 5B             pop     bx
17702 000013A6 C3             retn
17703
17704             ; -----
17705
17706             ; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
17707             ; 24/12/2023 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMBIO.COM)
17708
17709             ; BIOS CODE:1124h (MSDOS 6.21, IO.SYS)
17710             ; PC DOS 7.1 IBMBIO.COM - BIOS CODE:1404h
17711
17712             ; =====
17713             ;
17714             ; get the owner of the physical drive represented by the logical drive in al.
17715             ; the assumption is that we **always** keep track of the owner of a drive!!
17716             ; if this is not the case, the system may hang, just following the linked list.
17717             ;
17718             ; =====
17719
17720             ; 24/12/2023 - Retro DOS 5.0
17721
17722             ; 19/10/2022
17723
17724 000013A7 E8FAF1       ioctl_getown: call    SetDrive
17725 000013AA 268A4504     mov     al, [es:di+4] ; [es:di+BDS.drivenum]
17726             ; Get physical drive number
17727 000013AE C43E[1901]     les     di, [start_bds] ; Get start of bds chain
17728
17729 000013B2 26384504     ownloop: cmp     [es:di+4], al ; [es:di+BDS.drivenum]
17730 000013B6 7507         jnz     short getnextBDS

```

```

17731             ; 24/12/2023
17732 000013B8 26F6453F20 test byte [es:di+3Fh], 20h ; (PCDOS 7.1 IBMBIO.COM)
17733             ; 10/12/2022
17734             ; test byte [es:di+23h], 20h
17735             ; ; test word [es:di+23h], 20h ; [es:di+BDS.flags]
17736             ; ; fi_own_physical
17737 000013BD 7514 jnz short exitown
17738 getnextBDS:
17739 000013BF 26C43D les di, [es:di] ; [es:di+BDS.link]
17740 000013C2 EBEE jmp short ownloop
17741 ; -----
17742 ;
17743 ; =====
17744 ;
17745 ; set the ownership of the physical drive represented by the logical drive
17746 ; in al to al.
17747 ;
17748 ; =====
17749 ;
17750             ; 24/12/2023 - Retro DOS 5.0
17751             ; 19/10/2022
17752             ;
17753 ioctl_setown:
17754 000013C4 E8DDF1 call SetDrive
17755 000013C7 C606[7A00]01 mov byte [fsetowner], 1
17756             ; set flag for CheckSingle to look at.
17757 000013CC E8A8F5 call checksingle
17758             ; 02/09/2023
17759 000013CF FE0E[7A00] dec byte [fsetowner] ; 0
17760             ; mov byte [fsetowner], 0
17761             ; set ownership of drive reset flag
17762             ; Fall into ExitOwn
17763             ;
17764 ; =====
17765 ;
17766 ; if there is only one logical drive assigned to this physical drive, return
17767 ; 0 to user to indicate this. Enter with ES:di -> the owner's bds.
17768 ;
17769 ; =====
17770 ;
17771             ; 24/12/2023 - Retro DOS 5.0
17772             ;
17773 000013D3 30C9 exitown: xor cl, cl
17774             ; 24/12/2023
17775 000013D5 26F6453F10 test byte [es:di+3Fh], 10h ; (PCDOS 7.1 IBMBIO.COM)
17776             ; 12/12/2022
17777             ; test byte [es:di+23h], 10h
17778             ; ; test word [es:di+23h], 10h ; [es:di+BDS.flags]
17779             ; ; fi_am_mult
17780 000013DA 7406 jz short exitnomult
17781 000013DC 268A4D05 mov cl, [es:di+5] ; [es:di+BDS.drivelet]
17782             ; Get logical drive number
17783             ; Get it 1-based
17784 000013E0 FEC1 inc cl
17785 exitnomult:
17786 000013E2 C51E[1200] lds bx, [ptrsav]
17787 000013E6 884F01 mov [bx+1], cl ; [bx+unit]
17788             ; Exit normal termination
17789             ; 12/12/2022
17790             ; cf=0
17791             ; clc
17792 000013E9 C3 retn
17793
17794 ; ===== S U B R O U T I N E =====
17795
17796 ; 16/10/2022
17797
17798 ; -----
17799 ;
17800 ; moves the old Dpt that had been saved in TempDpt back to Dpt. this is done
17801 ; only if the first byte of TempDpt is not -1.
17802 ; all registers (including flags) are preserved.
17803 ;
17804 ; -----
17805 ;
17806             ; 24/12/2023
17807             ; 19/10/2022
17808 RestoreOldDpt:
17809             ; if we have already restored the disk base table earlier,
17810             ; do not do it again.
17811             ;
17812 000013EA 50 push ax
17813 000013EB 30C0 xor al, al
17814 000013ED A2[AA05] mov [had_format_error], al ; Reset flag and
17815 000013F0 8606[A905] xchg al, [media_set_for_format] ; get current flag setting
17816 000013F4 08C0 or al, al
17817 000013F6 7418 jz short DontRestore
17818 000013F8 56 push si
17819 000013F9 1E push ds
17820 000013FA 06 push es
17821 000013FB C536[AB05] lds si, [tempdpt]
17822             ; 17/10/2022
17823             ; mov es, [cs:BIOSDATAWORD]
17824             ; ; mov es, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
17825             ; ; mov es, [es:zeroseg]
17826             ; ; mov es, es:zeroseg ; CAS -- bleeeech!
17827             ;
17828             ; 24/12/2023
17829             ;
17830 000013FF 31C0 xor ax, ax
17831 00001401 8EC0 mov es, ax ; 0
17832             ;
17833             ; mov es:78h, si ; [es:DSKADR] (Int 1Eh)
17834 00001403 2689367800 mov [es:DSKADR], si
17835             ; word ptr es:7Ah, ds ; [es:DSKADR+2]
17836 00001408 268C1E7A00 mov [es:DSKADR+2], ds
17837 0000140D 07 pop es
17838 0000140E 1F pop ds
17839 0000140F 5E pop si
17840 DontRestore:
17841 00001410 58 pop ax
17842             ; 12/12/2022
17843             ; cf=0
17844             ; clc ; clear carry
17845 00001411 C3 retn
17846 ; -----
17847 ;
17848 ; 16/10/2022
17849 ;
17850 ; =====
17851 ;
17852 ; get media id
17853 ; =====
17854 ;

```

```

17855 ; FUNCTION: get the volume label,the system id and the serial number from
17856 ; the media that has the extended boot record.
17857 ; for the conventional media,this routine will return "unknown
17858 ; media type" error to dos.
17859 ;
17860 ; INPUT : ES:di -> bds table for this drive.
17861 ;
17862 ; OUTPUT: the request packet filled with the information,if not carry.
17863 ; if carry set,then al contains the device driver error number
17864 ; that will be returned to dos.
17865 ; register DS,DX,AX,CX,DI,SI destroyed.
17866 ;
17867 ; SUBROUTINES TO BE CALLED:
17868 ; BootIo:NEAR
17869 ;
17870 ; LOGIC:
17871 ; to recognize the extended boot record,this logic will actually
17872 ; access the boot sector even if it is a hard disk.
17873 ; note:the valid extended bpb is recognized by looking at the mediabyte
17874 ; field of bpb and the extended boot signature.
17875 ;
17876 ; {
17877 ; get logical drive number from bds table;
17878 ; rFlag = read operation;
17879 ; BootIo; /*get the media boot record into the buffer
17880 ; if (no error) then
17881 ; if (extended boot record) then
17882 ; { set volume label,volume serial number and system id
17883 ; of the request packet to those of the boot record;
17884 ; };
17885 ; else /*not an extended bpb */
17886 ; { set register al to "unknown media.." error code;
17887 ; set carry bit;
17888 ; };
17889 ; else
17890 ; ret; /*already error code is set in the register al
17891 ;
17892 ; =====
17893 ;size_of_EXT_BOOT_SERIAL equ 4
17894 ;size_of_EXT_BOOT_VOL_LABEL equ 11
17895 ;size_of_EXT_SYSTEM_ID equ 8
17896 ;
17897 ; 24/12/2023 - Retro DOS 5.0
17898 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:1478h)
17899 ;
17900 ; 19/10/2022
17901 ;
17902 ; GetMediaId:
17903 ; call ChangeLineChk
17904 ; mov al,[es:di+5] ; [es:di+BDS.drivelet]; Logical drive number
17905 ; mov byte[rflag],2 ; Read operation
17906 ; call BootIo ; Read boot sector into DiskSector
17907 ; jb short IOctl_If1
17908 ; ; valid? (0F0h-0FFh?)
17909 ; cmp byte[disksector+15h],0F0h
17910 ; ; [disksector+EXT_BOOT.BPB+EBPB.MEDIADESCRIPTOR]
17911 ; jb short IOctl_If2 ; brif not valid (0F0h - 0FFh)
17912 ; 24/12/2023
17913 ; jb short IOctl_If7
17914 ;
17915 ; 24/12/2023
17916 ; 10/12/2022
17917 ; mov si,disksector+26h
17918 ; ;
17919 ; 24/12/2023
17920 ; mov si,disksector+43h ; BS_FAT32_VolID
17921 ; mov si,disksector+42h ; BS_FAT32_BootSig ; 24/12/2023
17922 ; cmp word[disksector+16h],0 ; BPB.FATSz16
17923 ; jz short IOctl_If3 ; FAT32 fs
17924 ; sub si,1ch ; FAT (12-16) fs ; 43h-1Ch = 27h ; BS_VolID
17925 ; si = disksector+26h = BS_BootSig ; 24/12/2023
17926 ;
17927 ; IOctl_If3:
17928 ; cmp byte[si-1],29h ; BS_BootSig or BS_FAT32_BootSig
17929 ; ;
17930 ; cmp byte[si],29h
17931 ; cmp byte[disksector+26h],29h ; [disksector+EXT_BOOT.SIG]
17932 ; ; EXT_BOOT_SIGNATURE
17933 ; jne short IOctl_If2 ; not extended boot record
17934 ; les di,[ptrsav] ; es:di points to request header
17935 ; les di,[es:bx+19] ; [es:bx+IOCTL_REQ.GENERICIOCTL_PACKET]
17936 ; 10/12/2022
17937 ; mov si,disksector+27h ; disksector+EXT_BOOT.SERIAL
17938 ; inc si
17939 ; 24/12/2023
17940 ; si = disksector+27h (BS_VolID)
17941 ; or disksector+43h (BS_FAT32_VolID)
17942 ;
17943 ; add di,2 ; A_MEDIA_ID_INFO.MI_SERIAL
17944 ; 24/12/2023
17945 ; mov cx,23 ; size_of_EXT_BOOT_SERIAL
17946 ; ; L+size_of_EXT_BOOT_VOL_LABEL
17947 ; ; +size_of_EXT_SYSTEM_ID
17948 ; rep movsb ; Move from Bios_Data into request packet
17949 ;
17950 ; 10/12/2022
17951 ; cf = 0
17952 ; cld
17953 ;
17954 ; retn
17955 ; -----
17956 ; 24/12/2023
17957 ;
17958 ; IOctl_If2: stc
17959 ;
17960 ; IOctl_If7: mov al,7 ; error_unknown_media
17961 ; stc
17962 ;
17963 ; IOctl_If6:
17964 ; IOctl_If1: retn
17965 ; -----
17966 ;
17967 ; 16/10/2022
17968 ;
17969 ; =====
17970 ; set media id
17971 ; =====
17972 ;
17973 ; function: set the volume label, the system id and the serial number of
17974 ; the media that has the extended boot record.
17975 ; for the conventional media, this routine will return "unknown
17976 ; media.." error to dos.
17977 ; this routine will also set the corresponding informations in
17978 ; the bds table.

```

```

17979 ;
17980 ; input : ES:di -> bds table for this drive.
17981 ;
17982 ; output: the extended boot record in the media will be set according to
17983 ; the request packet.
17984 ; if carry set, then al contains the device driver error number
17985 ; that will be returned to dos.
17986 ;
17987 ; subroutines to be called:
17988 ; BootIo:NEAR
17989 ;
17990 ; logic:
17991 ;
17992 ; {
17993 ; get drive_number from bds;
17994 ; rFlag = "read operation";
17995 ; BootIo;
17996 ; if (no error) then
17997 ; if (extended boot record) then
17998 ; { set volume label, volume serial number and system id
17999 ; of the boot record to those of the request packet;
18000 ; rFlag = "write operation";
18001 ; get drive number from bds;
18002 ; BootIo; /*write it back*/
18003 ; };
18004 ; else /*not an extended bpb */
18005 ; { set register al to "unknown media.." error code;
18006 ; set carry bit;
18007 ; ret; /*return back to caller */
18008 ; };
18009 ; else
18010 ; ret; /*already error code is set */
18011 ;
18012 ; =====
18013 ;
18014 ; 24/12/2023 - Retro DOS 5.0
18015 ;
18016 ; 19/10/2022
18017 SetMediaId:
18018 call ChangeLineChk
18019 mov al, [es:di+5] ; [es:di+BDS.drivelet]
18020 ; Logical drive number
18021 mov dl, al
18022 mov byte [rflag], 2 ; romread
18023 push dx
18024 call BootIo ; Read boot sec to Bios_Data:DiskSector
18025 pop dx
18026 jb short IOct1_If6
18027 ; Valid? (0F0h-0FFh?)
18028 cmp byte [disksector+15h], 0F0h
18029 ; [disksector+EXT_BOOT.BPB+EBPB.MEDIADESCRIPTOR]
18030 jb short IOct1_If7 ; Brif not
18031 ;
18032 ; 24/12/2023
18033 ; cmp byte [disksector+26h], 29h ; [disksector+EXT_BOOT.SIG]
18034 ; ; EXT_BOOT_SIGNATURE
18035 ; jnz short IOct1_If7 ; not extended boot record
18036 ;
18037 push es ; Save BDS pointer
18038 push di
18039 push ds ; Point ES To boot record
18040 pop es
18041 ;
18042 ; 24/12/2023
18043 ; ;
18044 ; mov di, disksector+43h ; disksector+EXT_BOOT.SERIAL
18045 ; mov di, disksector+42h ; BS_FAT32_BootSig ; 24/12/2023
18046 ; cmp word [disksector+16h], 0 ; BPB.FATSz16
18047 ; jz short IOct1_If5 ; FAT32 fs
18048 ; sub di, 1Ch ; 67-28 ; offset disksektor+27h
18049 ; di = disksector+26h = BS_BootSig ; 24/12/2023
18050 IOct1_If5:
18051 ; cmp byte [di-1], 29h ; BS_BootSig or BS_FAT32_BootSig
18052 ; cmp byte [di], 29h
18053 ; je short IOct1_If8
18054 ; pop di ; not extended boot record
18055 ; pop es
18056 ; jmp short IOct1_If7
18057 ; 24/12/2023
18058 ; jmp short IOct1_If2
18059 IOct1_If8:
18060 ; ;
18061 ; 24/12/2023
18062 ; mov di, disksector+27h ; disksector+EXT_BOOT.SERIAL
18063 ; inc di
18064 ; di = disksector+27h (BS_VolID)
18065 ; or disksector+43h (BS_FAT32_VolID)
18066 ;
18067 ; lds si, [ptrsav] ; ds:si points to request header.
18068 ; lds si, [si+19] ; [si+IOCTL_REQ.GENERICIOCTL_PACKET]
18069 ; add si, 2 ; A_MEDIA_ID_INFO.MI_SERIAL
18070 ;
18071 ; 24/12/2023
18072 ; mov cx, 23 ; size_of_EXT_BOOT_SERIAL
18073 ; ; +size_of_EXT_BOOT_VOL_LABEL
18074 ; ; +size_of_EXT_SYSTEM_ID
18075 ; rep movsb
18076 ; call IOct1_If4 ; copy volume serial, label and system id
18077 ;
18078 ; push es ; pointds back to Bios_Data
18079 ; pop ds
18080 ; pop di ; restore bds pointer
18081 ; pop es
18082 ; call mov_media_ids ; update the bds media id info.
18083 ; mov al, dl
18084 ; mov byte [rflag], 3 ; romwrite
18085 ; call BootIo ; write it back.
18086 ; mov byte [tim_drv], 0FFh
18087 ; ; make sure chk_media check the driver
18088 ; ; return with error code from BootIo
18089 ; retn
18090 ; -----
18091 ; 24/12/2023
18092 ; IOct1_If7:
18093 ; mov al, 7 ; error_unknown_media
18094 ; stc
18095 ; IOct1_If6:
18096 ; retn
18097 ;
18098 ; ===== S U B R O U T I N E =====
18099 ;
18100 ; 16/10/2022
18101 ;
18102 ;

```

```

18103 ; -----
18104 ;   BootIo
18105 ; -----
18106 ;
18107 ; function: read/write the boot record into boot sector.
18108 ;
18109 ; input :
18110 ;   al=logical drive number
18111 ;   rFlag = operation (read/write)
18112 ;
18113 ; output:  for read operation,the boot record of the drive specified in bds
18114 ;           be read into the DiskSector buffer.
18115 ;           for write operation,the DiskSector buffer image will be written
18116 ;           to the drive specified in bds.
18117 ;           if carry set,then al contains the device driver error number
18118 ;           that will be returned to dos.
18119 ;           AX,CX,DX register destroyed.
18120 ;           if carry set,then al will contain the error code from DiskIO.
18121 ;
18122 ; subroutines to be called:
18123 ;   DiskIO:NEAR
18124 ;
18125 ; logic:
18126 ;
18127 ; {
18128 ;   first_sector = 0;      /*logical sector 0 is the boot sector */
18129 ;   sectorcount = 1;      /*read 1 sector only */
18130 ;   buffer = DiskSector;  /*read it into the DiskSector buffer */
18131 ;   call DiskIO (rFlag,drive_number,first_sector,sectorcount,buffer);
18132 ; }
18133 ; =====
18134 ;
18135 ;   ; 19/10/2022
18136 ;
18137 ; BootIo:
18138 ;   push    es
18139 ;   push    di
18140 ;   push    bx
18141 ;   push    ds
18142 ;   pop     es           ; Point ES: to Bios_Data
18143 ;
18144 ;   ; Call DiskIO to read/write the boot sec. The parameters which
18145 ;   ; need to be initialized for this subroutine out here are
18146 ;   ; - Transfer address to Bios_Data:DiskSector
18147 ;   ; - Low sector needs to be initialized to 0. this is a reg. param
18148 ;   ; - Hi sector in [Start_Sec_H] needs to be initialised to 0.
18149 ;   ; - Number of sectors <-- 1
18150 ;   mov     di,disksector ; es:di -> transfer address
18151 ;   xor     dx,dx         ; Firstsector (h) -> 0
18152 ;   mov     [start_sec_h], dx ; Start sector (h) -> 0
18153 ;   mov     cx,1
18154 ;   call    diskio
18155 ;   pop     bx
18156 ;   pop     di
18157 ;   pop     es
18158 ;   retn
18159 ;
18160 ; ===== S U B   R O U T I N E =====
18161 ;
18162 ;   ; 16/10/2022
18163 ;
18164 ;   -----
18165 ;   ChangelnChk
18166 ;   -----
18167 ;
18168 ;   when the user calls get/set media id call before dos establishes the media
18169 ;   by calling "media_chk",the change line activity of the drive is going to be
18170 ;   lost. this routine will check the change line activity and will save the
18171 ;   history in the flags.
18172 ;
18173 ;   FUNCTION: check the change line error activity
18174 ;
18175 ;   INPUT :  ES:di -> bds table.
18176 ;
18177 ;   OUTPUT:  flag in bds table will be updated if change line occurs.
18178 ;
18179 ;   SUBROUTINES TO BE CALLED:
18180 ;     Set_Changed_DL
18181 ;
18182 ;   -----
18183 ;
18184 ;   ; 24/12/2023 - Retro DOS 5.0
18185 ;   ChangelnChk:
18186 ;     mov     dl,[es:di+4] ; [es:di+BDS.drivenum]
18187 ;     or      dl,dl       ; Fixed disk?
18188 ;     js      short ChangelnChkRet ; Yes, skip it.
18189 ;     ; 24/12/2023
18190 ;     test    byte [es:di+3Fh], 4 ; [es:di+BDS.flags] ; PC DOS 7.1
18191 ;     ; 12/12/2022
18192 ;     test    byte [es:di+23h], 4
18193 ;     ;;test   word [es:di+23h], 4 ; [es:di+BDS.flags]
18194 ;     ; return_fake_bpb
18195 ;     jnz     short ChangelnChkRet
18196 ;     cmp     byte [fhave96], 1 ; This rom support change line?
18197 ;     jnz     short ChangelnChkRet
18198 ;     call    haschange ; This drive support change line?
18199 ;     jz      short ChangelnChkRet ; Do nothing
18200 ;
18201 ;     ; Execute the rom disk interrupt to check changeline activity.
18202 ;
18203 ;     mov     ah, 16h
18204 ;     int     13h ; DISK - FLOPPY DISK - CHANGE OF DISK STATUS (AT,XT2,XT286,CONV,PS)
18205 ;     ; DL = drive to check
18206 ;     ; Return: AH = disk change status
18207 ;     jnb     short ChangelnChkRet
18208 ;     push    bx
18209 ;     mov     bx, 40h ; fchanged
18210 ;     ; update flag in BDS for this
18211 ;     ; physical drive
18212 ;     call    set_changed_dl
18213 ;     pop     bx
18214 ;   ChangelnChkRet:
18215 ;     retn
18216 ;
18217 ;   -----
18218 ;
18219 ;   ; 16/10/2022
18220 ;
18221 ;   =====
18222 ;   GetAccessFlag
18223 ;   =====
18224 ;
18225 ;   FUNCTION: get the status of UNFORMATTED_MEDIA bit of flags in bds table
18226 ;

```

```

18227 ; INPUT :
18228 ; ES:di -> bds table
18229 ;
18230 ; OUTPUT: a_DiskAccess_Control.dac_access_flag = 0 if disk i/o not allowed.
18231 ; = 1 if disk i/o allowed.
18232 ; =====
18233 ;
18234 ; 24/12/2023 - Retro DOS 5.0
18235 ;
18236 ; 19/10/2022
18237 GetAccessFlag:
18238 lds bx, [ptrsav] ; DS:BX points to request header
18239 lds bx, [bx+19] ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
18240 mov al, 0 ; Assume result is unformatted
18241 ; 10/12/2022
18242 sub al, al
18243 ; 24/12/2023
18244 test byte [es:di+40h], 02h ; (PCDOS 7.1 IBMBIO.COM)
18245 ; test word ptr es:[di+3Fh], 200h
18246 ; 10/12/2022
18247 test byte [es:di+36], 02h
18248 ; test word [es:di+35], 200h ; [es:di+BDS.flags]
18249 ; unformatted_media
18250 jnz short GafDone ; Done if unformatted
18251 inc al ; Return true for formatted
18252 ; 24/12/2023
18253 inc ax
18254 GafDone:
18255 mov [bx+1], al ; [bx+A_DISKACCESS_CONTROL.DAC_ACCESS_FLAG]
18256 ret
18257 ; -----
18258 ; 16/10/2022
18259 ;
18260 ; =====
18261 ; SetAccessFlag
18262 ; =====
18263 ;
18264 ; function: set/reset the UNFORMATTED_MEDIA bit of flags in bds table
18265 ;
18266 ; input :
18267 ; ES:di -> bds table
18268 ;
18269 ; output: unformtted_media bit modified according to the user request
18270 ; =====
18271 ;
18272 ; 24/12/2023 - Retro DOS 5.0
18273 ;
18274 ; 19/10/2022
18275 SetAccessFlag:
18276 lds bx, [ptrsav] ; ES:BX points to request header
18277 lds bx, [bx+19] ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
18278 ; 24/12/2023
18279 and byte [es:di+40h], 0FDh ; (PCDOS 7.1 IBMBIO.COM)
18280 and word ptr es:[di+3Fh], 0FDFFh
18281 ; 10/12/2022
18282 and byte [es:di+36], 0FDh
18283 ; and word [es:di+35], 0FDFFh ; [es:di+BDS.flags]
18284 ; ~unformatted_media
18285 cmp byte [bx+1], 0 ; [bx+A_DISKACCESS_CONTROL.DAC_ACCESS_FLAG]
18286 jnz short saf_Done
18287 ; 24/12/2023
18288 or byte [es:di+40h], 02h ; (PCDOS 7.1 IBMBIO.COM)
18289 ; or word ptr es:[di+3Fh], 200h
18290 ; 15/04/2024
18291 ; 10/12/2022
18292 or byte [es:di+36], 02h
18293 ; or word [es:di+35], 200h ; [es:di+BDS.flags]
18294 ; unformatted_media
18295 saf_Done:
18296 ret
18297 ; -----
18298 ; 16/10/2022
18299 ;
18300 ; =====
18301 ; Ioctl_Support_Query
18302 ; =====
18303 ;
18304 ; New device command which was added in DOS 5.00 to allow a query of a
18305 ; specific GENERIC IOCTL to see if it is supported. Bit 7 in the
18306 ; device attributes specifies if this function is supported.
18307 ; =====
18308 ;
18309 ; 24/12/2023 - Retro DOS 5.0
18310 ;
18311 ; 19/10/2022
18312 ioctl_support_query:
18313 push es
18314 les bx, [ptrsav] ; ES:BX Points to request header.
18315 mov ax, [es:bx+13] ; [es:bx+IOCTL_REQ.MAJORFUNCTION]
18316 ; AL == Major, AH == Minor
18317 ; 24/12/2023
18318 ; 02/09/2023 (PCDOS 7.1)
18319 cmp al, 48h ; IOC_NEW_DC (PCDOS 7.1)
18320 ; new generic ioctl function (FAT32)
18321 je short ioctl_support
18322 cmp al, 8 ; IOC_DC
18323 ; See if major code is 8
18324 jne short nosupport
18325 ioctl_support:
18326 push cs
18327 pop es
18328 ; 24/12/2023
18329 ; 02/09/2023
18330 mov cx, 14 ; (PCDOS 7.1) IOC_DC_TABLE_LEN
18331 ; mov cx, 11 ; IOC_DC_TABLE_LEN
18332 ; 10/12/2022
18333 mov di, IOC_DC_Table
18334 ; mov di, 0C60h ; IOC_DC_Table
18335 ; at 2C7h:0C60h = 70h:31D0h
18336 xchg al, ah ; Put minor code in AL
18337 repne scasb ; Scan for minor code in AL
18338 jnz short nosupport ; it was not found
18339 mov ax, 100h
18340 ; 10/12/2022
18341 ; (jump to ioctlsupexit is not required)
18342 jmp short $+2 ; ioctlsupexit
18343 ; Signal ioctl is supported
18344 ; jmp short ioctlsupexit
18345 ; -----
18346 ; ioctlsupexit:
18347 ;
18348 ;
18349 ;
18350 ;

```



```

18351 0000153E 07          pop     es
18352                      ; 10/12/2022
18353                      ; cf = 0
18354                      ; cll
18355 0000153F C3          retl
18356                      ; -----
18357 nosupport:
18358                      pop     es
18359 00001541 E991EB      jmp     bc_cmderr
18360                      ; -----
18361                      ; 16/10/2022
18362
18363                      ; =====
18364                      ; GetMediaSenseStatus
18365                      ; =====
18366
18367                      ; FUNCTION: will return the type of diskette media in the specified DOS
18368                      ; diskette drive and whether the media is the default type
18369                      ; for that drive. (default type means the max size for that
18370                      ; drive)
18371                      ;
18372                      ; INPUT : ES:DI -> BDS table
18373                      ; OUTPUT: If carry clear
18374                      ; DS:BX -> Updated IOCTLPacket
18375                      ;
18376                      ;
18377                      ; Special Function at offset 0:
18378                      ; 0 - Media detected is not default type
18379                      ; 1 - Media detected is default type
18380                      ;
18381                      ; Device Type at offset 1:
18382                      ; 2 - 720K 3.5" 80 tracks
18383                      ; 7 - 1.44M 3.5" 80 tracks
18384                      ; 9 - 2.88M 3.5" 80 tracks
18385                      ;
18386                      ; Error Codes returned in AX if carry set:
18387                      ;
18388                      ; 8102 - Drive not ready - No disk is in the drive.
18389                      ; 8107 - Unknown media type - Drive doesn't support this function or
18390                      ; the media is really unknown, any error
18391                      ; other than "media not present"
18392                      ;
18393                      ; =====
18394                      ; 19/10/2022
18395 SenseMediaType:
18396                      lds     bx, [ptrsav] ; DS:BX points to request header.
18397                      lds     bx, [bx+19] ; bx+IOCTL_REQ.GENERICIOCTL_PACKET]
18398                      ; 10/10/2022
18399                      ; mov     word [bx], 0 ; Initialize the 2 packet bytes
18400                      xor     dx, dx
18401 0000154B 31D2      mov     [bx], dx ; 0
18402 0000154D 8917      ;
18403                      ;
18404 0000154F 268A5504  mov     dl, [es:di+4] ; [es:di+BDS.drivenum]
18405                      ; Get int 13h drive number from BDS
18406                      ; 10/12/2022
18407                      ; xor     dh, dh ; DX = physical drive number
18408 00001553 B420      mov     ah, 20h ; Get Media Type function
18409                      ; If no carry media type in AL
18410                      int     13h ; DISK - QCACHE - DISMOUNT
18411 00001557 7216      jc     short MediaSenseEr ; error code in AH
18412 00001559 FE07      inc     byte [bx] ; Signal media type is default (bit 1)
18413 DetermineMediaType:
18414 0000155B FEC8      dec     al
18415 0000155D 3C02      cmp     al, 2 ; Chk for 720K ie: (3-1) = 2
18416 0000155F 740A      jz     short GotMediaType
18417 00001561 0404      add     al, 4
18418 00001563 3C07      cmp     al, 7 ; Chk for 1.44M ie: (4-1+4) = 7
18419 00001565 7404      jz     short GotMediaType
18420 00001567 3C09      cmp     al, 9 ; Chk for 2.88M ie: (6-1+4) = 9
18421 00001569 7510      jnz     short UnknownMediaType ; Just didn't recognize media type
18422 GotMediaType:
18423 0000156B 884701      mov     [bx+1], al ; Save the return value
18424                      ; 10/12/2022
18425                      ; cf = 0
18426                      ; cll
18427 0000156E C3          retl
18428                      ; -----
18429
18430 MediaSenseEr:
18431 0000156F 80FC32      cmp     ah, 32h ; See if not default media error
18432 00001572 74E7      jz     short DetermineMediaType ; Not really an error
18433 00001574 B002      mov     al, 2 ; Now assume drive not ready
18434 00001576 80FC31      cmp     ah, 31h ; See if media was present
18435 00001579 7402      jz     short SenseErrExit ; Return drive not ready
18436 UnknownMediaType:
18437 0000157B B007      mov     al, 7 ; Just don't know the media type
18438 SenseErrExit:
18439 0000157D B481      mov     ah, 81h ; Signal error return
18440 0000157F F9          stc
18441 00001580 C3          retl
18442                      ; -----
18443                      ; 10/12/2022
18444                      ; db 0
18445                      ; -----
18446
18447                      ; -----
18448                      ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:15F2h
18449                      ; -----
18450                      ; 26/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
18451                      ; -----
18452                      ; ===== S U B R O U T I N E =====
18453
18454 SetLockState:
18455                      lds     bx, [ptrsav] ; set media lock state
18456 00001581 C51E[1200]  lds     bx, [bx+13h] ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
18457 00001585 C55F13      ; mov     dl, [es:di+4] ; [es:di+BDS.drivenum]
18458                      ; call    check_int13h_exts_present
18459                      ; 26/12/2023
18460                      call    check_int13h_exts_p
18461 00001588 E82100      ; mov     al, 3 ; unknown command error
18462                      ; jc     short setlockst_ret
18463 0000158B 721C      mov     al, [bx] ; [bx+A_LOCKSTATE_CONTROL.LOCKSTATE_FUNCTIONS]
18464 0000158D 8A07      mov     ah, 45h
18465 0000158F B445      int     13h ; DISK - IBM/MS Extension - LOCK/UNLOCK DRIVE
18466 00001591 CD13      ; (DL - drive, [SI - disk address packet)
18467                      ; mov     [bx+1], al ; 1 = locked, 0 = not locked
18468 00001593 884701      ; [bx+A_LOCKSTATE_CONTROL.LOCKSTATE_FLAG]
18469                      ; 26/12/2023
18470                      jmp     short s1s_em
18471 00001596 EB0A      ;
18472                      jnc     short setlockst_ret
18473                      ;
18474                      ;

```

```

18475 ; mov al, ah
18476 ; call maperror
18477 ;setlockst_ret:
18478 ; mov ah, 81h ; Return this status in case of carry
18479 ; ret
18480 ;
18481 ; ===== S U B R O U T I N E =====
18482
18483 EjectMedia:
18484 ;mov dl, [es:di+4] ; eject media in drive
18485 ; ; [es:di+BDS.drivenum]
18486 ;call check_int13h_exts_present
18487 ; 26/12/2023
18488 00001598 E81100 call check_int13h_exts_p
18489 ;mov al, 3 ; unknown command error
18490 0000159B 720C jc short ejectm_ret
18491 0000159D B80046 mov ax, 4600h
18492 000015A0 CD13 int 13h ; DISK - IBM/MS Extension - EJECT MEDIA
18493 ; (DL - drive)
18494
18495 000015A2 7305 jnc short ejectm_ret
18496 000015A4 88E0 mov al, ah
18497 000015A6 E800F8 call maperror
18498 ; 26/12/2023
18499 setlockst_ret:
18500 000015A9 B481 ejectm_ret: mov ah, 81h ; Return this status in case of carry
18501 000015AB C3 ret
18502
18503 ; ===== S U B R O U T I N E =====
18504
18505 ; 26/12/2023
18506 check_int13h_exts_p:
18507 000015AC 268A5504 mov dl, [es:di+4]
18508
18509 check_int13h_exts_present:
18510 000015B0 B441 mov ah, 41h
18511 000015B2 53 push bx
18512 000015B3 8BAA55 mov bx, 55AAh
18513 000015B6 CD13 int 13h ; DISK - Check for INT 13h Extensions
18514 ; BX = 55AAh, DL = drive number
18515 ; Return: CF set if not supported
18516 ; AH = extensions version
18517 ; BX = AA55h
18518 ; CX = Interface support bit map
18519 000015B8 81FB55AA cmp bx, 0AA55h
18520 000015BC 5B pop bx
18521 000015BD 7505 jnz short exts_notsupported
18522 000015BF F6C102 test cl, 2 ; bit 1 - drive locking and ejecting subset
18523 000015C2 7503 jnz short exts_supported
18524
18525 exts_notsupported:
18526 000015C4 B003 ; 26/12/2023
18527 mov al, 3
18528 ;
18529 stc
18530 exts_supported:
18531 000015C7 C3 ret
18532
18533 ; ===== S U B R O U T I N E =====
18534
18535 000015C8 8CD9 GetDrvMapInfo:
18536 mov cx, ds ; get drive map information
18537 ;
18538 ; es:di points to BDS which belongs to
18539 ; the requested logical/dos drive number
18540 ;
18541 ; Format of parameter block:
18542 ; Offset Description (Table 01570)
18543 ; 00h (call) length of this buffer (in bytes)
18544 ; 01h (ret) number of bytes in parameter block
18545 ; actually used
18546 ; 02h (ret) drive flags
18547 ; 03h (ret) physical drive number
18548 ; 00h-7Fh floppy
18549 ; 80h-FEh hard
18550 ; FFh no physical drive
18551 ; 04h (ret) bitmap of logical drives associated with
18552 ; physical drive
18553 ; bit 0 = drive A:, etc.
18554 ; 08h (ret) relative block address of partition start
18555 ; qword
18556 ;
18557 ; Ref: Ralf Brown's Interrupt List, INTERRUPT.G
18557 000015CA C51E[1200] lds bx, [ptrsav]
18558 000015CE C5F13 lds bx, [bx+13h] ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
18559 000015D1 B80381 mov ax, 8103h ; ah = generic ioctl error code (81h)
18560 ; al = unknown command error (03h)
18561 000015D4 803F10 cmp byte [bx], 10h ; parameter buffer length = 16 bytes
18562 000015D7 7251 jb short gdmi_4
18563 000015D9 268A5504 mov dl, [es:di+4] ; [es:di+BDS.drivenum]
18564 000015DD 885703 mov [bx+3], dl ; parameter block - offset 3 - physical drive number
18565 000015E0 C6470110 mov byte [bx+1], 10h ; parameter block - actually used length
18566 000015E4 268B4517 mov ax, [es:di+17h] ; [es:di+BDS.hiddensectors]
18567 000015E8 894708 mov [bx+8], ax ; parameter block - offset 8 - partition start LBA
18568 000015EB 268B4519 mov ax, [es:di+19h] ; [es:di+BDS.hiddensectors+2]
18569 000015EF 89470A mov [bx+0Ah], ax ; parameter block - offset 10
18570 000015F2 31C0 xor ax, ax ; 0
18571 000015F4 884702 mov [bx+2], al ; drive flags = 0 (protected mode flags etc.)
18572 000015F7 89470C mov [bx+0Ch], ax ; high dword of partition start address (LBA) is 0
18573 000015FA 89470E mov [bx+0Eh], ax
18574 000015FD 894704 mov [bx+4], ax ; logical drive bitmap of same physical drive
18575 ; initialized as 0
18576 00001600 894706 mov [bx+6], ax ; 0
18577 00001603 8EC1 mov es, cx
18578 ;les di, dword ptr es:start_bds ; 1st BDS
18579 00001605 26C43E[1901] les di, [es:start_bds]
18580 0000160A B90100 mov cx, 1 ; bit 0 (drive A:)
18581
18582 0000160D 83FFFF gdmi_1: cmp di, 0FFFFh ; last BDS ?
18583 00001610 7415 jz short gdmi_3 ; yes
18584 00001612 26385504 cmp [es:di+4], dl ; [es:di+BDS.drivenum], dl
18585 ; is it same physical drive ?
18586 00001616 7506 jnz short gdmi_2 ; no
18587 00001618 094F04 or [bx+4], cx ; set bit for logical drive index of this BDS
18588 ; (previously) shifted bit (which is 1/ON) is in ax:cx
18589 0000161B 094706 or [bx+6], ax
18590
18591 0000161E D1E1 gdmi_2: shl cx, 1 ; shift one left for setting the next drive's bit
18592 00001620 D1D0 rcl ax, 1 ; set high word of the bit select (set) value
18593 00001622 26C43D les di, [es:di] ; next BDS
18594 00001625 EBE6 jmp short gdmi_1 ; loop until di = -1 (last BDS sign)
18595
18596 00001627 B80001 gdmi_3: mov ax, 100h ; success
18597
18598 0000162A C3 gdmi_4: ret

```



```

18599
18600
18601 ;-----
18602 ; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
18603 ; 26/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
18604
18605 ;-----
18606 ; MSINT13.ASM - MSDOS 6.0 - 1991
18607 ;-----
18608 ; 16/03/2019 - Retro DOS v4.0
18609
18610 ; int 2f function 13h allows the user to change the orig13 int_13 vector
18611 ; after booting. this allows testing and implementation of custom int_13
18612 ; handlers, without giving up ms-dos error recovery
18613 ; entry: ds:dx == addr. of new int_13 handler
18614 ;         es:bx == addr. of new int_13 vector used by warm boot (int19)
18615 ; exit:  orig13 == address of new int_13 handler
18616 ;         ds:dx == old orig13 value
18617 ;         es:bx == old old13 value
18618
18619 ; int 2f handler for external block drivers to communicate with the internal
18620 ; block driver in msdisk. the multiplex number chosen is 8. the handler
18621 ; sets up the pointer to the request packet in [ptrsav] and then jumps to
18622 ; dsk_entry, the entry point for all disk requests.
18623
18624 ; on exit from this driver, we will return to the external driver
18625 ; that issued this int 2f, and can then remove the flags from the stack.
18626 ; this scheme allows us to have a small external device driver, and makes
18627 ; the maintainance of the various drivers (driver and msbio) much easier,
18628 ; since we only need to make changes in one place (most of the time).
18629
18630 ; ax=800h - check for installed handler - reserved
18631 ; ax=801h - install the bds into the linked list
18632 ; ax=802h - dos request
18633 ; ax=803h - return bds table starting pointer in ds:di
18634 ;         (ems device driver hooks int 13h to handle 16kb dma overrun
18635 ;         problem. bds table is going to be used to get head/sector
18636 ;         informations without calling generic ioctl get device parm call.)
18637
18638 ;BIOSSEGMENT equ 70h
18639 DOSBIOSSEG equ 0070h ; 17/10/2022
18640
18641 ;;BIOSCODE:1302h (MSDOS 6.21, IO.SYS)
18642 ;;BIOSCODE:16AAh (PCDOS 7.1, IBMBIO.COM) ; 26/12/2023
18643
18644 i2f_handler:
18645         cmp     ah, 13h                ; here is 02C7h:1302h = 0070h:3872h
18646         jz      short int2f_replace_int13
18647         cmp     ah, 8
18648         jz      short mine
18649
18650 ; Check for WIN386 startup and return the BIOS instance data
18651
18652         cmp     ah, 16h                ; Multwin386
18653         jz      short win386call
18654         cmp     ah, 4Ah                ; multMULT
18655         jnz     short i2f_handler_iret
18656         jmp     handle_multmult
18657
18658 ;-----
18659 i2f_handler_iret:
18660         iret
18661 ;-----
18662
18663 int2f_replace_int13:
18664         cli      ; 26/12/2023
18665         push     ax                    ; free up a register for caller's ds
18666         mov     ax, ds                ; then we can use ds: -> Bios_Data
18667         ;mov     ds, word [cs:0030h] ; 15/10/2022
18668         ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:0030h]
18669         ;         ; = [02C7h:0030h] = [0070h:25A0h]
18670         mov     ds, [cs:BIOSDATAWORD] ; 17/10/2022
18671         ; 19/10/2022
18672         ;push     word ptr ds:orig13    ; save old value of old13 and
18673         ;push     word ptr ds:orig13+2 ; orig13 so that we can
18674         ;push     word ptr ds:old13    ; return them to caller
18675         ;push     word ptr ds:old13+2
18676
18677         ; 02/09/2023 (PCDOS 7.1)
18678         ;push     word [orig13]
18679         push     word [orig13+2]
18680         ;push     word [old13]
18681         push     word [old13+2]
18682
18683         ;mov     word ptr ds:orig13, dx; orig13 := addr. of new int_13
18684         ;mov     word ptr ds:orig13+2, ax
18685         ;mov     word ptr ds:old13, bx ; old13 := addr. of new boot_13
18686         ;mov     word ptr ds:old13+2, es
18687
18688         ;mov     [orig13], dx
18689         ; 02/09/2023
18690         xchg     dx, [orig13]
18691         mov     [orig13+2], ax
18692         ;mov     [old13], bx
18693         ; 02/09/2023
18694         xchg     bx, [old13]
18695         mov     [old13+2], es
18696
18697         pop      es                    ; es:bx := old old13 vector
18698         ; 02/09/2023
18699         ;pop     bx
18700         pop      ds                    ; ds:dx := old orig13 vector
18701         ;pop     dx ; 02/09/2023
18702         pop      ax
18703
18704 i2f_iret:
18705         iret
18706 ;-----
18707
18708 mine:
18709         cmp     al, 0F8h                ; iret on reserved functions
18710         jnb     short i2f_iret
18711         or      al, al                  ; a get installed state request?
18712         jnz     short disp_func
18713         mov     al, 0FFh
18714         ;jmp     short i2f_iret
18715         ; 02/09/2023
18716         iret
18717 ;-----
18718
18719 disp_func:
18720         cmp     al, 1                    ; request for installing bds?
18721         jz      short do_subfun_01
18722         cmp     al, 3                    ; get bds vector?
18723         jz      short do_get_bds_vector

```

```

18723
18724
18725 ; set up pointer to request packet
18726 0000167A 1E push ds
18727 0000167B 2E8E1E[3000] mov ds, [cs:BIOSDATAWORD] ; 17/10/2022
18728 ;mov ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
18729 ; = [0070h:25A0h] = [02C7h:0030h]
18730 ; 19/10/2022
18731 ;mov word ptr ds:ptrsav, bx
18732 ;mov word ptr ds:ptrsav+2, es
18733 00001680 891E[1200] mov [ptrsav], bx
18734 00001684 8C06[1400] mov [ptrsav+2], es
18735 00001688 1F pop ds
18736 ;jmp far ptri2f_dskentry
18737 ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
18738 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1708h
18739 00001689 EA[5E06]7000 jmp DOSBIOSSEG:dsk_entry ; BIOSDATA:dsk_entry
18740 ; 17/10/2022
18741 ;jmp far DOSBIOSSEG:dsk_entry
18742 ;jmp DOSBIOSSEG:i2f_dskentry ; 70h:i2f_dskentry
18743 ; NOTE: jump to a FAR function, not an
18744 ; IRET type function. Callers of
18745 ; this int2f subfunction will have
18746 ; to be careful to do a popf
18747
18748 ; -----
18749
18750 do_subfun_01:
18751 0000168E 06 push es
18752 0000168F 1E push ds
18753 00001690 1E push ds
18754 00001691 07 pop es
18755 ; 17/10/2022
18756 00001692 2E8E1E[3000] mov ds, [cs:BIOSDATAWORD]
18757 ;mov ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
18758 ; point ds: -> Bios_Data
18759 00001697 E8BC03 call install_bds
18760 0000169A 1F pop ds
18761 0000169B 07 pop es
18762 ;jmp short i2f_iret
18763 ; 02/09/2023
18764 0000169C CF iret
18765 ; -----
18766
18767 do_get_bds_vector:
18768 ; 17/10/2022
18769 0000169D 2E8E1E[3000] mov ds, [cs:BIOSDATAWORD]
18770 ;mov ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
18771 000016A2 C53E[1901] lds di, [start_bds]
18772 ;lds di, ds:start_bds
18773 ;i2f_iret: ; 10/12/2022
18774 ;jmp short i2f_iret
18775 ; 02/09/2023
18776 000016A6 CF iret
18777 ; -----
18778
18779 ; 17/10/2022
18780 ; 16/10/2022
18781
18782 ; WIN386 startup stuff is done here. If starting up we set our WIN386 present
18783 ; flag and return instance data. If exiting, we reset the WIN386 present flag
18784 ; NOTE: we assume that the BIOS int 2fh is at the bottom of the chain.
18785
18786 win386call:
18787 000016A7 1E push ds
18788 000016A8 2E8E1E[3000] mov ds, [cs:BIOSDATAWORD]
18789 ;mov ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
18790 ; at 2C7h:30h = 70h:25A0h
18791 000016AD 3C05 cmp al, 5 ; win386_Init
18792 ; is itwin386 initializing?
18793 000016AF 7410 jz short win386Init
18794 000016B1 3C06 cmp al, 6 ; win386_Exit
18795 ; is itwin386 exiting?
18796 000016B3 7523 jnz short win_iret ; if not, continue int2f chain
18797 ; 12/12/2022
18798 000016B5 F6C201 test dl, 1
18799 ;test dx, 1 ; is itwin386 or win286 dos extender?
18800 000016B8 751E jnz short win_iret ; if not win386, then continue
18801 ;and ds:Iswin386, 0 ; indicate that win386 is not present
18802 000016BA 8026[1208]00 and byte [Iswin386], 0
18803 000016BF EB17 jmp short win_iret
18804 ; -----
18805
18806 win386Init:
18807 ; 12/12/2022
18808 000016C1 F6C201 test dl, 1
18809 ;test dx, 1 ; is it win386 or win286 dos extender?
18810 000016C4 7512 jnz short win_iret ; if not win386, then continue
18811 ;or ds:Iswin386, 1 ; Indicate WIN386 present
18812 000016C6 800E[1208]01 or byte [Iswin386], 1
18813 ;mov word ptr ds:SI_Next, bx ; Hook our structure into chain
18814 ;mov word ptr ds:SI_Next+2, es
18815 000016CB 891E[E007] mov [SI_Next], bx
18816 000016CF 8C06[E207] mov [SI_Next+2], es
18817 ;mov bx, offset win386_SI ; point ES:BX to win386_SI
18818 000016D3 BB[DE07] mov bx, win386_SI ; 19/10/2022
18819 000016D6 1E push ds
18820 000016D7 07 pop es
18821 win_iret:
18822 000016D8 1F pop ds
18823 ; 10/12/2022
18824 ;jmp short i2f_iret ; return back up the chain
18825 ; 02/09/2023
18826 000016D9 CF iret
18827 ; -----
18828
18829 handle_multmult:
18830 000016DA 3C01 cmp al, 1
18831 000016DC 7514 jnz short try_2
18832 000016DE 1E push ds
18833 000016DF E84500 call HMAPtr ; get offset of free HMA
18834 ; 10/12/2022
18835 ;xor bx, bx
18836 ;dec bx
18837 000016E2 BBFFFF mov bx, 0FFFFh
18838 000016E5 8EC3 mov es, bx ; seg of HMA
18839 000016E7 89FB mov bx, di
18840 000016E9 F7D3 not bx
18841 000016EB 09DB or bx, bx
18842 000016ED 7401 jz short try_1
18843 000016EF 43 inc bx
18844 try_1:
18845 000016F0 1F pop ds
18846 ;jmp short i2f_iret

```

```

18847 ; 02/09/2023
18848 000016F1 CF      ired
18849 ; -----
18850
18851 try_2:
18852         cmp     al, 2      ; multMULTALLOCHMA
18853         jnz     short try_3
18854         push    ds
18855         ; 10/12/2022
18856         ;xor     di, di
18857         ;dec     di
18858         mov     di, 0FFFFh ; assume not enough space
18859         mov     es, di
18860         call    HMAPtr     ; get offset of free HMA
18861         cmp     di, 0FFFFh
18862         jz      short InsuffHMA
18863         neg     di          ; free space in HMA
18864         cmp     bx, di
18865         jbe     short try_4
18866         ; 10/12/2022
18867         ;sub     di, di
18868         ;dec     di
18869         mov     di, 0FFFFh
18870         ;jmp     short InsuffHMA
18871         ; 02/09/2023
18872         pop     ds
18873         ired
18874 ; -----
18875
18876 try_4:
18877         ;mov     di, ds:FreeHMAPtr
18878         mov     di, [FreeHMAPtr]
18879         add     bx, 15
18880         ;and     bx, 0FFF0h
18881         ; 10/12/2022
18882         and     bl, 0F0h
18883         ;add     ds:FreeHMAPtr, bx ; update the free pointer
18884         add     [FreeHMAPtr], bx
18885         jnz     short InsuffHMA
18886         mov     word [FreeHMAPtr], 0FFFFh ; -1
18887         ;mov     ds:FreeHMAPtr, 0FFFFh
18888         ; no more HMA if we have wrapped
18889
18890 InsuffHMA:
18891         pop     ds
18892         ; 10/12/2022
18893
18894 try_3:
18895         ;jmp     short ii2f_ired
18896         ; 02/09/2023
18897         ired
18898 ; -----
18899         ; 10/12/2022
18900
18901 ;try_3:
18902         ;jmp     ii2f_ired
18903
18904 ; ===== S U B   R O U T I N E =====
18905 ; 16/10/2022
18906 ; -----
18907
18908 ; procedure : HMAPtr
18909 ;
18910 ;         Gets the offset of the free HMA area ( with respect to
18911 ;                                     seg ffff )
18912 ;         If DOS has not moved high, tries to move DOS high.
18913 ;         In the course of doing this, it will allocate all the HMA
18914 ;         and set the FreeHMAPtr to past the end of the BIOS and
18915 ;         DOS code. The call to MoveDOSIntoHMA (which is a pointer)
18916 ;         enters the routine in sysinit1 called FTryToMovDOSHi.
18917 ;
18918 ;         RETURNS : offset of free HMA in DI
18919 ;                   BIOS_DATA, seg in DS
18920 ;
18921 ; -----
18922
18923         ; 17/10/2022
18924
18925 HMAPtr:
18926         mov     ds, [cs:BIOSDATAWORD]
18927         ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
18928         mov     di, [FreeHMAPtr]
18929         ;mov     di, ds:FreeHMAPtr
18930         cmp     di, 0FFFFh
18931         jnz     short HMAPtr_retn
18932         cmp     byte [SysinitPresent], 0
18933         ;cmp     ds:SysinitPresent, 0
18934         jz      short HMAPtr_retn
18935         call    far [MoveDOSIntoHMA]
18936         ;call    ds:MoveDOSIntoHMA ; call far [MoveDOSIntoHMA]
18937         mov     di, [FreeHMAPtr]
18938         ;mov     di, ds:FreeHMAPtr
18939
18940 HMAPtr_retn:
18941         retn
18942
18943 ; ===== S U B   R O U T I N E =====
18944 ; 16/10/2022
18945
18946 ; move a 512 byte sector from ds:si to es:di, do not trash cx
18947 ; but go ahead and update direction flag, si, & di
18948
18949 move_sector:
18950 ; The 80386 microprocessor considers an access to WORD 0FFFFh in
18951 ; any segment to be a fault. Theoretically, this could be handled
18952 ; by the fault handler and the behavior of an 8086 could be emulated
18953 ; by wrapping the high byte to offset 0000h. This would be a lot
18954 ; of work and was, indeed, blown off by the win386 guys. COMPAQ
18955 ; also handles the fault incorrectly in their ROM BIOS for real
18956 ; mode. Their fault handler was only designed to deal with one
18957 ; special case which occurred in a magazine benchmark, but didn't
18958 ; handle the general case worth beans.
18959 ;
18960 ; Simply changing this code to do a byte loop would work okay but
18961 ; would involve a general case performance hit. Therefore, we'll
18962 ; check for either source or destination offsets being within one
18963 ; sector of the end of their segments and only in that case fall
18964 ; back to a byte move.
18965
18966         cld
18967         push    cx
18968         mov     cx, 256
18969         cmp     si, 0FE00h
18970         ja      short movsec_bytes

```

```

18971 00001750 81FF00FE      cmp     di, 0FE00h
18972 00001754 7704      ja      short movsec_bytes
18973 00001756 F3A5      rep movsw
18974 00001758 59      pop     cx
18975 00001759 C3      retn
18976
18977
18978
18979 0000175A D1E1      movsec_bytes:
18980 0000175C F3A4      shl     cx, 1
18981 0000175E 59      rep movsb
18982 0000175F C3      pop     cx
18983      retn
18984
18985
18986
18987
18988
18989
18990
18991
18992
18993
18994
18995
18996
18997
18998
18999
19000
19001
19002
19003
19004
19005
19006
19007
19008 00001760 50
19009 00001761 53
19010 00001762 06
19011 00001763 57
19012 00001764 E86C00      call    find_bds      ; get pointer to bds for drive in dl
19013 00001767 725E      jb      short no_wrap ; finished if DOS doesn't use it
19014
19015 00001769 26F6453F01      ; 26/12/2023
19016      test    byte [es:di+3Fh], 1
19017      ; 12/12/2022
19018      ; test    byte [es:di+23h], 1
19019      ; test    word [es:di+23h], 1 ; [es:di+BDS.flags], fnon_removable
19020 00001770 268B5D13      jz      short no_wrap ; no wrapping for removable media
19021 00001774 89C8      mov     bx, [es:di+13h] ; [es:di+BDS.sectortrack]
19022 00001776 83E03F      mov     ax, cx
19023 00001779 39D8      and     ax, 3Fh      ; extract sector number
19024 0000177B 764A      cmp     ax, bx      ; are we going to wrap?
19025 0000177D F6F3      jbe     short no_wrap
19026      div     bl      ; ah=new sector      #, al=# of headwraps
19027
19028
19029
19030 0000177F 08E4
19031 00001781 7503
19032
19033 00001783 48
19034 00001784 88DC
19035
19036
19037
19038 00001786 80E1C0
19039 00001789 08E1
19040 0000178B 30E4
19041 0000178D 40
19042 0000178E 00F0
19043 00001790 80D400
19044
19045 00001793 268B5D15
19046 00001797 39D8
19047
19048
19049 00001799 7632
19050 0000179B 52
19051 0000179C 31D2
19052
19053 0000179E F7F3
19054
19055
19056
19057 000017A0 09D2
19058 000017A2 7507
19059 000017A4 89DA
19060
19061
19062
19063 000017A6 09C0
19064 000017A8 7401
19065 000017AA 48
19066
19067 000017AB 88D7
19068 000017AD 5A
19069 000017AE FECF
19070 000017B0 88FE
19071 000017B2 88CF
19072 000017B4 80E73F
19073 000017B7 B306
19074 000017B9 86CB
19075 000017BB D2EB
19076 000017BD 00C5
19077 000017BF 10E3
19078 000017C1 D2E3
19079 000017C3 86D9
19080 000017C5 08F9
19081
19082 000017C7 F8
19083 000017C8 5F
19084 000017C9 07
19085 000017CA 5B
19086 000017CB 58
19087 000017CC C3
19088
19089
19090
19091 000017CD 88C6
19092 000017CF FECE
19093 000017D1 EBF4
19094

```

```

movsec_bytes:
    shl     cx, 1
    rep movsb
    pop     cx
    retn

; ===== S U B   R O U T I N E =====

; 16/10/2022

; check_wrap is a routine that adjusts the starting sector, starting head
; and starting cylinder for an int 13 request that requests i/o of a lot
; of sectors. it only does this for fixed disks. it is used in the sections
; of code that handle ecc errors and dma errors. it is necessary, because
; ordinarily the rom would take care of wraps around heads and cylinders,
; but we break down a request when we get an ecc or dma error into several
; i/o of one or more sectors. in this case, we may already be beyond the
; number of sectors on a track on the medium, and the request would fail.
;
; input conditions:
;     all registers set up for an int 13 request.
;
; output:
;     dh - contains starting head number for request
;     cx - contains starting sector and cylinder numbers
;     (the above may or may not have been changed, and are 0-based)
;     all other registers preserved.

; 26/12/2023 - Retro DOS 5.0
check_wrap:
    push    ax
    push    bx
    push    es
    push    di
    call    find_bds      ; get pointer to bds for drive in dl
    jb      short no_wrap ; finished if DOS doesn't use it
    ; 26/12/2023
    test    byte [es:di+3Fh], 1
    ; 12/12/2022
    ; test    byte [es:di+23h], 1
    ; test    word [es:di+23h], 1 ; [es:di+BDS.flags], fnon_removable
    jz      short no_wrap ; no wrapping for removable media
    mov     bx, [es:di+13h] ; [es:di+BDS.sectortrack]
    mov     ax, cx
    and     ax, 3Fh      ; extract sector number
    cmp     ax, bx      ; are we going to wrap?
    jbe     short no_wrap
    div     bl      ; ah=new sector      #, al=# of headwraps

; we need to be careful here. if the new sector # is 0, then we are on the
; last sector on that track.

    or      ah, ah
    jnz     short not_on_bound
    ; 18/12/2022
    dec     ax ; *
    mov     ah, bl      ; set sector=BDS_BPB.BPB_SECTORS PER TRACK
    ; if on boundary
    ; also decrement # of head wraps
    dec     al ; *
not_on_bound:
    and     cl, 0C0h      ; zero out sector #
    or      cl, ah      ; or in new sector #
    xor     ah, ah      ; ax = # of head wraps
    inc     ax
    add     al, dh      ; add in starting head #
    adc     ah, 0      ; catch any carry
    ; 02/09/2023
    mov     bx, [es:di+15h] ; [es:di+BDS.heads]
    cmp     ax, bx
    ; cmp     ax, [es:di+15h] ; [es:di+BDS.heads]
    ; are we going to wrap around a head?
    jbe     short no_wrap_head ; do not lose new head number!!
    push    dx      ; preserve drive number and head number
    xor     dx, dx
    ; mov     bx, [es:di+15h] ; [es:di+BDS.heads]
    div     bx      ; dx=new head #, ax=# of cylinder wraps

; careful here! if new head # is 0, then we are on the last head.

    or      dx, dx
    jnz     short no_head_bound
    mov     dx, bx      ; on boundary. set to BDS_BPB.BPB_HEADS

; if we had some cylinder wraps, we need to reduce them by one!!

    or      ax, ax
    jz      short no_head_bound
    dec     ax      ; reduce number      of cylinder wraps
no_head_bound:
    mov     bh, dl      ; bh has new head number
    pop     dx      ; restore drive number and head number
    dec     bh      ; get it 0-based
    mov     dh, bh      ; set up new head number in dh
    mov     bh, cl
    and     bh, 3Fh      ; preserve sector number
    mov     bl, 6
    xchg    cl, bl
    shr     bl, cl      ; get ms cylinder bits to ls end
    add     ch, al      ; add in cylinder wrap
    adc     bl, ah      ; add in high byte
    shl     bl, cl      ; move up to ms      end
    xchg    bl, cl      ; restore cylinder bits      into cl
    or      cl, bh      ; or in sector number
no_wrap:
    cld
    pop     di
    pop     es
    pop     bx
    pop     ax
    retn

; -----

no_wrap_head:
    mov     dh, al      ; do not lose new head number
    dec     dh      ; get it 0-based
    jmp     short no_wrap

```

```

19095 ; ===== S U B   R O U T I N E =====
19096
19097 ; 16/10/2022
19098
19099 ; this is a special version of the bds lookup code which is
19100 ; based on physical drives rather than the usual logical drives
19101 ; carry is set if the physical drive in dl is found, es:di -> its bds
19102 ; otherwise carry is clear
19103 ;
19104 ; guaranteed to trash no registers except es:di
19105
19106 ; 19/10/2022
19107
19108 find_bds:    les     di, [start_bds]      ; point es:di to first bds
19109 fbds_1:      cmp     [es:di+4], dl      ; [es:di+BDS.drivenum]
19110             jz      short fbds_2      ;
19111             les     di, [es:di]        ; [es:di+BDS.link]
19112             ; go to next bds
19113             cmp     di, 0FFFFh
19114             jnz     short fbds_1
19115             stc
19116 fbds_2:      retn
19117
19118 ; ===== S U B   R O U T I N E =====
19119
19120 ; 16/10/2022
19121 ; 17/10/2022
19122 doint:
19123 ; 10/12/2022
19124 mov     dl, [bp+8]      ; [bp+INT13FRAME.olddx]
19125             ; get physical drive number
19126             ; 19/10/2022 - Temporary !
19127             ; db 8Ah, 96h, 8, 0 ; mov dl, [bp+8]
19128
19129             xor     ah, ah
19130             or      al, al
19131             jz      short dointdone    ; if zero sectors, return ax=0
19132             ; 10/12/2022
19133             mov     ah, [bp+3]        ; [bp+INT13FRAME.olddx+1]
19134             ; get request code
19135             ; db 8Ah, 0A6h, 3, 0 ; mov ah, [bp+3]
19136             push    word [bp+10h]    ; [bp+INT13FRAME.olddx]
19137             ; db 0FFh, 0B6h, 10h, 0 ; push word [bp+10h]
19138             popf
19139             ; call far 70h:797h ; MSDOS 6.21 IO.SYS BIOSCODE:14EAh
19140             ; 17/10/2022
19141             call    DOSBIOSEG:call_orig13
19142             ; call far 70h:797h
19143             ; call far KERNEL_SEGMENT:call_orig13
19144             pushf
19145             ; 10/12/2022
19146             pop     word [bp+10h]    ; [bp+INT13FRAME.olddx]
19147             ; db 8Fh, 86h, 10h, 0 ; pop word [bp+10h]
19148 dointdone:   retn
19149
19150 ; -----
19151 ; 16/10/2022
19152
19153 ; this is the true int 13 handler. we parse the request to see if there is
19154 ; a dma violation. if so, depending on the function, we:
19155 ;   read/write break the request into three pieces and move the middle one
19156 ;   into our internal buffer.
19157 ;
19158 ;   format    copy the format table into the buffer
19159 ;   verify    point the transfer address into the buffer
19160 ;
19161 ; this is the biggest bogosity of all. the ibm controller does not handle
19162 ; operations that cross physical 64k boundaries. in these cases, we copy
19163 ; the offending sector into the buffer below and do the i/o from there.
19164
19165 ; struc INT13FRAME
19166 ; .oldbp: resw
19167 ; .oldax: resw
19168 ; .oldbx: resw
19169 ; .oldcx: resw
19170 ; .olddx: resw
19171 ; .oldds: resw      ; now we save caller's ds, too
19172 ; .oldd: resd
19173 ; .oldf: resw
19174 ; end struc
19175
19176 ; -----
19177 ; entry conditions:
19178 ; ah = function
19179 ; al = number of sectors
19180 ; es:bx = dma address
19181 ; cx = packed track and sector
19182 ; dx = head and drive
19183 ; output conditions:
19184 ; no dma violation.
19185
19186 ; use extreme caution when working with this code. In general,
19187 ; all registers are hot at all times.
19188
19189 ; question: does this code handle cases where dma errors
19190 ; occur during ecc retries, and where ecc errors occur during
19191 ; dma breakdowns???? Hmmmmmm.
19192
19193 ; -----
19194 ; -----
19195 ; 26/12/2023 - Retro DOS v5.0
19196 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1889h
19197 dtype_array: dd 400090h      ; 40h:90h is drive type array addr
19198
19199 ; 17/10/2022
19200 ; DTYPEARRAY equ dtype_array - DOSBIOSEG_2C7h ; (14F5h for MSDOS 5.0 IO.SYS)
19201 ; 09/12/2022
19202 DTYPEARRAY equ dtype_array
19203
19204 ; -----
19205 ; stick some special stuff out of mainline
19206
19207 ; we know we're doing a format command. if we have changeline
19208 ; support, then flag some special changed stuff and set changed
19209 ; by format bit for all logical drives using this physical drive
19210
19211
19212
19213
19214
19215
19216
19217
19218

```

```

19219
19220
19221 00001805 803E[7700]00
19222 0000180A 7459
19223 0000180C 53
19224 0000180D B84001
19225 00001810 E85104
19226 00001813 5B
19227 00001814 EB4F
19228
19229
19230
19231
19232
19233
19234
19235
19236
19237
19238 00001816 84D2
19239 00001818 7852
19240 0000181A 50
19241 0000181B 51
19242 0000181C 88D1
19243 0000181E B001
19244 00001820 D2E0
19245 00001822 8406[A204]
19246 00001826 59
19247 00001827 58
19248 00001828 7442
19249
19250 0000182A 53
19251 0000182B 06
19252
19253 0000182C 2EC41E[0118]
19254
19255
19256 00001831 00D3
19257 00001833 80D700
19258 00001836 26C60793
19259
19260
19261 0000183A 07
19262 0000183B 5B
19263 0000183C EB2E
19264
19265
19266
19267
19268
19269
19270
19271
19272
19273
19274 0000183E 803E[1E00]08
19275
19276 00001843 7407
19277 00001845 803E[1E00]15
19278
19279 0000184A 752D
19280
19281 0000184C 50
19282 0000184D B401
19283
19284
19285
19286
19287
19288 0000184F 9A[0B07]7000
19289
19290
19291 00001854 58
19292 00001855 EB22
19293
19294
19295
19296
19297
19298
19299
19300
19301
19302
19303
19304
19305
19306 00001857 1E
19307
19308
19309 00001858 2E8E1E[3000]
19310
19311
19312
19313
19314
19315 0000185D A3[1E00]
19316 00001860 80FC05
19317 00001863 74A0
19318
19319
19320 00001865 803E[A204]00
19321 0000186A 75AA
19322
19323
19324
19325
19326
19327 0000186C 9A[0B07]7000
19328
19329
19330 00001871 9C
19331
19332 00001872 803E[AF05]FA
19333
19334 00001877 74C5
19335
19336
19337 00001879 9D
19338 0000187A 7221
19339
19340 0000187C 1F
19341 0000187D CA0200
19342

format_special_stuff:
    cmp     byte [fhave96], 0      ; do we have changeline support?
    jz      short format_special_stuff_done ; brif not
    push    bx
    mov     bx, 140h              ; fchanged_by_format+fchanged
    call    set_changed_d1 ; indicate that media changed by format
    pop     bx
    jmp     short format_special_stuff_done
; -----
; 16/10/2022
; we know we've got ec35's on the system. Now see if we're doing
; a floppy. If so, create a mask and see if this particular
; drive is an ec35. If so, set dtype_array[drive]=93h
; 19/10/2022
ec35_special_stuff:
    test    dl, dl                ; floppy or hard disk?
    js      short ec35_special_stuff_done ; if harddrive, we're done
    push    ax                    ; see if this PARTICULAR drive is ec35
    push    cx
    mov     cl, dl                ; turn drive number into bit map
    mov     al, 1                 ; assume drive 0
    shl     al, cl                ; shiftover correct number of times
    test    [ec35flag], al ; electrically compatible 3.5 incher?
    pop     cx
    pop     ax
    jz      short ec35_special_stuff_done
    push    bx                    ; done if this floppy is not an ec35
    push    es                    ; free up a far pointer (es:bx)
    ; 17/10/2022
    les     bx, [cs:DTYPEARRAY]
    ;les     bx, dword ptr cs:DTYPEARRAY ; [cs:dtype_array]
    ; 0070h:3A65h = 2C7h:14F5h
    add     bl, dl
    adc     bh, 0                 ; find entry for this drive
    mov     byte [es:bx], 93h ; establish drive type as:
    ; (360kdisk in360k drive,
    ; no double-stepping, 250 kbs transfer rate)
    pop     es
    pop     bx
    jmp     short ec35_special_stuff_done
; -----
; 16/10/2022
; ps2_30 machine has some problem with ah=8h (read drive parm), int 13h.
; this function does not reset the common buses after the execution.
; to solve this problem, when we detect ah=8h, then we will save the result and
; will issue ah=1 (read status) call to reset the buses.
ps2_special_stuff:
    cmp     byte [prevoper], 8 ; (ps2_30)
    ; read driver parm ?
    jz      short ps2_30_problem
    cmp     byte [prevoper], 15h
    ; apparently function 15h fails, too
    jnz     short ps2_special_stuff_done
ps2_30_problem:
    push    ax
    mov     ah, 1
    ; 26/12/2023
    ; call 70h:70Bh ; PC DOS 7.1 IBMBIO.COM BIOS CODE:18D7h
    ; call BIOS DATA:call_orig13
    ; call far 70:797h ; MSDOS 6.21 IO.SYS BIOS CODE:1543h
    ; 17/10/2022
    call    DOSBIOSSEG:call_orig13
    ; call call_orig13 ; call far 70:797h
    ; call far KERNEL_SEGMENT:call_orig13
    pop     ax
    jmp     short ps2_special_stuff_done
; -----
; 17/10/2022
; 16/10/2022
; here is the actual int13 handler
i13z:
    ; 0070h:3ABh = 02C7h:154Bh
    ; cas -- inefficient! could push ds and load ds-> Bios_Data before
    ; vectoring up here from Bios_Data
    ; 19/10/2022
    push    ds                    ; save caller's ds register first thing
    ; mov     ds, word [cs:0030h]
    ; and set up our own ds -> Bios_Data
    mov     ds, [cs:BIOSDATAWORD]
    ; mov     ds, word ptr cs:BIOSDATAWORD ; [cs:0030h]
    ; = [02C7h:0030h] = [0070h:25A0h]
; let the operation proceed. if there is a dma violation, then we do things
    mov     [prevoper], ax ; save request
    cmp     ah, 5              ; romformat
    jz      short format_special_stuff
    ; go do special stuff for format
format_special_stuff_done:
    cmp     byte [ec35flag], 0 ; any electrically compat 3.5 inchers?
    jnz     short ec35_special_stuff
    ; go handle it out of line if so
ec35_special_stuff_done:
    ; 26/12/2023
    call    70h:70Bh ; PC DOS 7.1 IBMBIO.COM BIOS CODE:18EDh
    ; call far 70:797h ; MSDOS 6.21 IO.SYS BIOS CODE:1560h
    call    DOSBIOSSEG:call_orig13
    ; call call_orig13 ; call far KERNEL_SEGMENT:call_orig13
    pushf
    ; save result flags
    cmp     byte [model_byte], 0FAh ; is this a ps2/30?
    ; mdl_ps2_30
    jz      short ps2_special_stuff
    ; exit mainline to address special
ps2_special_stuff_done:
    ; ps2/30 problem if so
    popf
    jnb     short goterr13 ; error on original orig13 call-thru?
ret_from_i13:
    pop     ds
    retf    2                    ; restore ds & iret w/flags
; -----

```



```

19343
19344 ; most of our code exits through here. If carry isn't set, then
19345 ; just do a simple exit. Else doublecheck that we aren't getting
19346 ; a changeline error.
19347
19348 i13ret_ck_chglinerr:
19349 00001880 73FA jnb short ret_from_i13 ; done if not an error termination
19350
19351 i13_ret_error:
19352 00001882 80FC06 cmp ah, 6 ; did i see a change event?
19353 00001885 7513 jnz short int13b ; skip if wrong error
19354 00001887 08D2 or dl, dl ; is this for the hard disk?
19355 00001889 780F js short int13b ; yes, ignore
19356 0000188B 803E[7700]00 cmp byte [fhave96], 0
19357 00001890 7408 jz short int13b ; just in case ROM returned this
19358 ; error even though it told us it
19359 ; never would
19360 00001892 53 push bx
19361 00001893 BB4000 mov bx, 40h ; fchanged
19362 00001896 E8CB03 call set_changed_dl
19363 00001899 5B pop bx
19364 0000189A F9 int13b: stc ; now return the error
19365 0000189B E8DF jmp short ret_from_i13
19366 ; -----
19367 ; some kind of error occurred. see if it is dma violation
19368
19369 goterr13:
19370 0000189D 80FC09 cmp ah, 9 ; dma error?
19371 000018A0 747C jz short gotdmaerr
19372
19373 goterr13_xxxx:
19374 000018A2 80FC11 cmp ah, 11h ; ecc error?
19375 000018A5 75DB jnz short i13_ret_error ; other error. just return back.
19376 000018A7 803E[A905]01 cmp byte [media_set_for_format], 1 ; formatting?
19377 000018AC 74D4 jz short i13_ret_error
19378
19379 000018AE 803E[1F00]02 cmp byte [prevoper+1], 2
19380 ; cmp byte ptr ds:prevoper+1, 2 ; ecc-corrected error
19381 ; (2 = romread)
19382 ; ECC correction only applies to reads
19383 000018B3 75CD jnz short i13_ret_error
19384
19385 000018B5 30E4 xor ah, ah
19386 ; call far 70:797h ; MSDOS 6.21 IO.SYS BIOSCODE:15ABh
19387 ; 17/10/2022
19388 000018B7 9A[0B07]7000 call DOSBIOSSEG:call_orig13
19389 ; call far KERNEL_SEGMENT:call_orig13
19390 ; call far 70:797h
19391 000018BC A1[1E00] mov ax, [prevoper]
19392 000018BF 30E4 xor ah, ah ; return code = no error
19393 000018C1 3C01 cmp al, 1 ; if request for one sector, assume ok
19394 000018C3 74B7 jz short ret_from_i13 ; return with carry clear
19395 000018C5 53 push bx
19396 000018C6 51 push cx
19397 000018C7 52 push dx
19398 000018C8 A2[2000] mov [number_of_sec], al
19399
19400 000018CB B80102 loop_ecc: mov ax, 201h ; read one sector
19401
19402 ; we do reads one sector at a time. this ensures that we will eventually
19403 ; finish the request since ecc errors on one sector do read in that sector.
19404 ;
19405 ; we need to put in some "intelligence" into the ecc handler to handle reads
19406 ; that attempt to read more sectors than are available on a particular
19407 ; track.
19408 ;
19409 ; we call check_wrap to set up the sector #, head # and cylinder # for
19410 ; this request.
19411 ;
19412 ; at this point, all registers are set up for the call to orig13, except
19413 ; that there may be a starting sector number that is bigger than the number
19414 ; of sectors on a track.
19415 ;
19416 000018CE E8FFFE call check_wrap ; get correct parameters for int 13
19417 ; call far 70:797h ; MSDOS 6.21 IO.SYS BIOSCODE:15C5h
19418 ; 17/10/2022
19419 000018D1 9A[0B07]7000 call DOSBIOSSEG:call_orig13
19420 ; call far KERNEL_SEGMENT:call_orig13
19421 000018D6 730C jnb short ok11_op
19422 000018D8 80FC09 cmp ah, 9 ; DMA error during ECC read?
19423 000018DB 741B jz short handle_dma_during_ecc
19424 000018DD 80FC11 cmp ah, 11h ; only allow ecc errors
19425 000018E0 7510 jnz short ok11_exit_err
19426 ; 10/12/2022
19427 ; xor ax ax -> ah = 0
19428 ; mov ah, 0 ; ecc error. reset the system again.
19429 000018E2 31C0 xor ax, ax ; clear the error code so that if this
19430 ; was the last sector, no error code
19431 ; will be returned for the corrected
19432 ; read. (clear carry too.)
19433
19434 000018E4 FE0E[2000] ok11_op: dec byte [number_of_sec]
19435 000018E8 7409 jz short ok11_exit ; all done?
19436 000018EA FEC1 inc cl ; advance sector number
19437 ; add 200h to address
19438 000018EC FEC7 inc bh
19439 000018EE FEC7 inc bh
19440 000018F0 E8D9 jmp short loop_ecc
19441 ; -----
19442 ; locate error returns centrally
19443
19444 ok11_exit_err:
19445 000018F2 F9 stc ; set carry bit again.
19446
19447 ok11_exit:
19448 000018F3 5A pop dx
19449 000018F4 59 pop cx
19450 000018F5 5B pop bx
19451 000018F6 E888 jmp short i13ret_ck_chglinerr
19452 ; -----
19453 ; do the single sector read again, this time into our temporary
19454 ; buffer, which is guaranteed not to have a DMA error, then
19455 ; move the data to its proper location and proceed
19456
19457 handle_dma_during_ecc:
19458 000018F8 06 push es
19459 000018F9 53 push bx
19460 000018FA BB[5201] mov bx, disksector
19461 000018FD 1E push ds
19462 000018FE 07 pop es ; point es:bx to buffer
19463 000018FF B80102 mov ax, 201h ; read one sector
19464 ; call far 70:797h ; MSDOS 6.21 IO.SYS BIOSCODE:15F8h
19465 ; 17/10/2022
19466

```

```

19467 00001902 9A[0B07]7000      call    DOSBIOSSEG:call_orig13
19468                          ;call    call_orig13      ; call far KERNEL_SEGMENT:call_orig13
19469 00001907 5B                pop     bx
19470 00001908 07                pop     es
19471 00001909 7305             jnb     short handle_dma_during_ecc_noerr
19472 0000190B 80FC11          cmp     ah, 11h
19473 0000190E 75E2             jnz     short ok11_exit_err ; if anything but ecc error, bomb out
19474
19475                          ; now we're kosher. Copy the data to where it belongs and resume
19476                          ; the ECC looping code.
19477
19478                          handle_dma_during_ecc_noerr:
19479 00001910 56                push    si
19480 00001911 57                push    di
19481 00001912 89DF          mov     di, bx
19482 00001914 BE[5201]       mov     si, disksector
19483 00001917 E82BFE          call   move_sector
19484 0000191A 5F                pop     di
19485 0000191B 5E                pop     si
19486 0000191C EBC6             jmp     short ok11_op
19487
19488                          ; -----
19489                          ; we truly have a dma violation. restore register ax and retry the
19490                          ; operation as best we can.
19491
19492                          gotdmaerr:
19493 0000191E A1[1E00]       mov     ax, [prevoper] ; 19/10/2022
19494 00001921 FB                sti
19495 00001922 80FC02          cmp     ah, 2          ; romread
19496 00001925 723B             jb      short i13_done_dmaerr
19497                          ; just pass dma error thru for
19498                          ; functions we don't handle
19499 00001927 80FC04          cmp     ah, 4          ; romverify
19500 0000192A 743C             jz      short intverify
19501 0000192C 80FC05          cmp     ah, 5          ; romformat
19502 0000192F 7448             jz      short intformat
19503 00001931 772F             ja      short i13_done_dmaerr
19504
19505                          ; we are doing a read/write call. check for dma problems
19506
19507                          ; ***** set up stack frame here!!! *****
19508
19509 00001933 52                push    dx
19510 00001934 51                push    cx
19511 00001935 53                push    bx
19512 00001936 50                push    ax
19513 00001937 55                push    bp
19514 00001938 89E5             mov     bp, sp
19515 0000193A 8CC2             mov     dx, es          ; check for 64k boundary error
19516                          ; 26/12/2023
19517                          ;add     dx, dx
19518                          ;add     dx, dx
19519                          ;add     dx, dx
19520                          ;add     dx, dx          ; dx = dx*16
19521 0000193C D1E2             shl     dx, 1
19522 0000193E D1E2             shl     dx, 1
19523 00001940 D1E2             shl     dx, 1
19524 00001942 D1E2             shl     dx, 1          ; segment converted to absolute address
19525 00001944 01DA             add     dx, bx          ; combine with offset
19526 00001946 81C2FF01        add     dx, 511         ; simulate a transfer
19527
19528                          ; if carry is set, then we are within 512 bytes of the end of the segment.
19529                          ; we skip the first transfer and perform the remaining buffering and transfer
19530
19531 0000194A 7303             jnb     short no_skip_first
19532 0000194C E98300          jmp     bufferx          ; restore dh=head & do buffer
19533
19534                          ; -----
19535                          no_skip_first:
19536 0000194F D0EE             shr     dh, 1          ; dh = number of sectors before address
19537 00001951 B480             mov     ah, 128        ; ah = max number of sectors in segment
19538 00001953 28F4             sub     ah, dh
19539
19540                          ; ah is now the number of sectors that we can successfully write in this
19541                          ; segment. if this number is above or equal to the requested number, then we
19542                          ; continue the operation as normal. otherwise, we break it into pieces.
19543                          ;
19544                          ; wait a sec. this is goofy. the whole reason we got here in the
19545                          ; first place is because we got a dma error. so it's impossible
19546                          ; for the whole block to fit, unless the dma error was returned
19547                          ; in error.
19548
19549 00001955 38C4             cmp     ah, al          ; can we fit it in?
19550 00001957 7236             jb      short doblock ; no, perform blocking.
19551
19552                          ; yes, the request fits. let it happen.
19553
19554 00001959 8A7609          mov     dh, [bp+9]      ; [bp+INT13FRAME.olddx+1]
19555                          ; set up head number
19556 0000195C E888FE          call   doint
19557 0000195F E9D900          jmp     bad13          ; and return from this place
19558
19559                          ; -----
19560                          i13_done_dmaerr:
19561 00001962 B409             mov     ah, 9          ; pass dma error thru to caller
19562 00001964 F9                stc
19563 00001965 E914FF          jmp     ret_from_i13   ; return with error,
19564                          ; we know it's not a changeline error
19565
19566                          ; -----
19567                          ; verify the given sectors. place the buffer pointer into our space.
19568
19569                          intverify:
19570 00001968 06                push    es          ; save caller's dma address
19571 00001969 53                push    bx
19572 0000196A 1E                push    ds          ; es:bx-> Bios_Data:disksector
19573 0000196B 07                pop     es
19574
19575                          dosimple:
19576 0000196C BB[5201]       mov     bx, disksector
19577                          ; do the i/o from Bios_Data:disksector
19578                          ; ;call far 70:797h ; MSDOS 6.21 IO.SYS BIOSCODE:1665h
19579                          ; 17/10/2022
19580 0000196F 9A[0B07]7000      call   DOSBIOSSEG:call_orig13
19581 00001974 5B                call   call_orig13      ; call far KERNEL_SEGMENT:call_orig13
19582 00001975 07                pop     bx
19583 00001976 E907FF          pop     es
19584                          jmp     i13ret_ck_chglinerr
19585
19586                          ; -----
19587                          ; format operation. copy the parameter table into Bios_Data:disksector
19588
19589                          intformat:
19590 00001979 06                push    es
19591 0000197A 53                push    bx

```



```

19591 0000197B 56          push    si
19592 0000197C 57          push    di
19593 0000197D 1E          push    ds
19594
19595          ; point ds to the caller's dma buffer, es to Bios_Data
19596          ; in other words, swap (ds, es)
19597
19598 0000197E 06          push    es
19599 0000197F 1E          push    ds
19600 00001980 07          pop     es
19601 00001981 1F          pop     ds
19602 00001982 89DE        mov     si, bx
19603 00001984 BF[5201]    mov     di, disksector
19604 00001987 E8B8FD    call    move_sector ; user's data into Bios_Data:disksector
19605 0000198A 1F          pop     ds
19606 0000198B 5F          pop     di
19607 0000198C 5E          pop     si
19608 0000198D EBDD        jmp     short dosimple ; Bios_Data:disksector
19609
19610          ; -----
19611          ; we can't fit the request into the entire block. perform the operation on
19612          ; the first block.
19613          ;
19614          ; doblock is modified to correctly handle multi-sector disk i/o.
19615          ; old doblock had added the number of sectors i/oed (ah in old doblock) after
19616          ; the doint call to cl. observing only the lower 6 bits of cl(=max. 64) can
19617          ; represent a starting sector, if ah was big, then cl would be clobbered.
19618          ; by the way, we still are going to use cl for this purpose since checkwrap
19619          ; routine will use it as an input. to prevent cl from being clobbered, a
19620          ; safe number of sectors should be calculated like "63 - # of sectors/track".
19621          ; doblock will handle the first block of requested sectors within the
19622          ; boundary of this safe value.
19623
19624          ; 26/12/2023 - Retro DOS v5.0
19625
19626          doblock:
19627          ; try to get the # of sectors/track from bds via rom drive number.
19628          ; for any mini disks installed, here we have to pray that they have the
19629          ; same # of sector/track as the main dos partition disk drive.
19630
19631 0000198F 8B5608    mov     dx, [bp+8] ; [bp+INT13FRAME.olddx]
19632                                ; get head #, drive #
19633          push    cx
19634 00001993 06          push    es
19635 00001994 57          push    di
19636                                ; ah - # of sectors before dma boundary
19637                                ; al - requested # of sectors for i/o.
19638          call    find_bds
19639          mov     cx, [es:di+13h] ; [es:di+BDS.sectpertrack]
19640          ; 26/12/2023
19641          test    byte [es:di+3Fh], 1
19642          ; 12/12/2022
19643          ; test byte [es:di+23h], 1
19644          ; test word [es:di+23h], 1 ; [es:di+BDS.flags], fnon_removable
19645          pop     di
19646          pop     es
19647          mov     al, ah ; set al=ah for floppies
19648          jz      short doblockflop ; they are track by track operation
19649          mov     ah, 63 ; ah = 63-secpt (# safe sectors??)
19650          sub     ah, cl ; al - # of sectors before dma boundary
19651          doblockflop:
19652          pop     cx
19653          doblockcontinue:
19654          cmp     ah, al ; if safe_# >= #_of_sectors_to_go_before dma,
19655          jnb     short doblocklast ; then #_of_sectors_to_go as it is for doint.
19656          push    ax
19657          mov     al, ah ; otherwise, set al to ah to operate.
19658          jmp     short doblockdoint
19659
19660          ; -----
19661          doblocklast:
19662          mov     ah, al
19663          push    ax
19664          doblockdoint:
19665                                ; let ah = al = # of sectors for this shot
19666          call    doint
19667          jb      short bad13 ; something happened, bye!
19668          pop     ax
19669          sub     [bp+2], ah ; sub [bp+INT13FRAME.olddx], ah
19670                                ; decrement by the successful operation
19671          add     cl, ah ; advance sector #. safety gauranteed.
19672          add     bh, ah ; advance dma address
19673          add     bh, ah ; twice for 512 byte sectors
19674          cmp     ah, al ; check the previous value
19675          jz      short buffer ; if #_of_sectors_to_go < safe_#,
19676                                ; then we are done already.
19677          sub     al, ah ; otherwise,
19678                                ; #_sector_to_go = #_of_sector_to_go - safe_#
19679          call    check_wrap ; get new cx, dh for the next operation.
19680          jmp     short doblockcontinue ; handles next sectors left.
19681
19682          ; -----
19683          bufferx:
19684          mov     dh, [bp+9] ; [bp+INT13FRAME.olddx+1]
19685                                ; set up head number
19686          buffer:
19687          push    bx
19688          mov     ah, [bp+3] ; [bp+INT13FRAME.olddx+1]
19689          cmp     ah, 3 ; romwrite
19690          jnz     short doread ;
19691
19692          ; copy the offending sector into local buffer
19693
19694          push    es
19695          push    ds
19696          push    si
19697          push    di
19698          push    ds ; exchange segment registers
19699          pop     es
19700          pop     ds
19701          mov     di, disksector ; where to move
19702          push    di ; save it
19703          mov     si, bx ; source
19704          call    move_sector ; move sector into local buffer
19705          pop     bx ; new transfer address
19706                                ; (es:bx = Bios_Data:diskbuffer)
19707          pop     di ; restore caller's di & si
19708          pop     si
19709          pop     ds ; restore Bios_Data
19710
19711          ; see if we are wrapping around a track or head
19712          mov     al, 1 ; [bp+INT13FRAME.olddx]
19713                                ; get drive number
19714          mov     dl, [bp+8]

```

```

19715 000019F8 E865FD          call    check_wrap      ; sets up registers if wrap-around
19716                                ;
19717                                ; ah is function
19718                                ; al is 1 for single sector transfer
19719                                ; es:bx is local transfer address
19720                                ; cx is track/sector number
19721                                ; dx is head/driver number
19722                                ; si, di unchanged
19723 000019FB E8E9FD          call    doint
19724 000019FE 07              pop     es                ; restore caller's dma segment
19725 000019FF 723A          jb     short bad13      ; go clean up
19726 00001A01 EB22          jmp     short dotail
19727                                ; -----
19728                                ;
19729                                ; reading a sector. do int first, then move things around
19730
19731 doread:
19732 00001A03 06              push    es
19733 00001A04 53              push    bx
19734 00001A05 1E              push    ds                ; es = Bios_Code
19735 00001A06 07              pop     es
19736 00001A07 8B[5201]      mov     bx, disksector
19737 00001A0A B001          mov     al, 1
19738 00001A0C 8A5608          mov     dl, [bp+8]        ; [bp+INT13FRAME.olddx]
19739                                ; get drive number
19740 00001A0F E84EFD          call    check_wrap      ;
19741                                ; ah = function
19742                                ; al = 1 for single sector
19743                                ; es:bx points to local buffer
19744                                ; cx, dx are track/sector, head/driver
19745 00001A12 E8D2FD          call    doint
19746 00001A15 5B              pop     bx
19747 00001A16 07              pop     es
19748 00001A17 7222          jb     short bad13
19749 00001A19 56              push    si
19750 00001A1A 57              push    di
19751 00001A1B 89DF          mov     di, bx
19752 00001A1D BE[5201]      mov     si, disksector
19753 00001A20 E822FD          call    move_sector
19754 00001A23 5F              pop     di
19755 00001A24 5E              pop     si
19756                                ;
19757                                ; note the fact that we've done 1 more sector
19758
19759 dotail:
19760 00001A25 5B              pop     bx                ; retrieve new dma area
19761 00001A26 80C702          add     bh, 2            ; advance over sector
19762 00001A29 41              inc     cx
19763 00001A2A 8A4602          mov     al, [bp+2]      ; [bp+INT13FRAME.olddx]
19764 00001A2D F8              clc
19765 00001A2E FEC8          dec     al
19766 00001A30 7409          jz     short bad13      ; no more i/o
19767                                ;
19768                                ; see if we wrap around a track or head boundary with starting sector
19769                                ; we already have the correct head number to pass to check_wrap
19770
19771 00001A32 8A5608          mov     dl, [bp+8]      ; [bp+INT13FRAME.olddx]
19772 00001A35 E828FD          call    check_wrap
19773 00001A38 E8ACFD          call    doint
19774                                ;
19775                                ; we are done. ax has the final code; we throw away what we got before
19776
19777 ; M046 -- okay gang. Now we've either terminated our DMA loop,
19778 ; or we've finished. If carry is set now, our only
19779 ; hope for salvation is that it was a read operation
19780 ; and the error code is ECC error. In that case, we'll
19781 ; just pop the registers and go do the old ECC thing.
19782 ; When the DMA error that got us here in the first
19783 ; place occurs, it'll handle it.
19784
19785 bad13:
19786 00001A3B 89EC          mov     sp, bp
19787 00001A3D 5D              pop     bp
19788 00001A3E 5B              pop     bx
19789 00001A3F 5B              pop     bx
19790 00001A40 59              pop     cx
19791 00001A41 5A              pop     dx
19792 00001A42 7203          jb     short xgoterr13_xxxx ; go handle ECC errors
19793 00001A44 E935FE          jmp     ret_from_i13    ; non-error exit
19794                                ; -----
19795
19796 xgoterr13_xxxx:
19797 00001A47 E958FE          jmp     goterr13_xxxx
19798                                ; -----
19799                                ;
19800                                ; 10/12/2022
19801                                ; db 0
19802                                ; -----
19803
19804 ;Bios_Code ends
19805
19806 ; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
19807
19808 ;-----
19809 ; MSBIO2.ASM - MSDOS 6.0 - 1991
19810 ;-----
19811 ; 17/03/2019 - Retro DOS v4.0
19812
19813 ; 19/10/2022
19814
19815 00001A4A 8A26[7500]      mov     ah, [drvmax]
19816 00001A4E BF[3C05]      mov     di, diskdrvs
19817 00001A51 1E              push    ds                ; pass result in es:di
19818 00001A52 07              pop     es
19819 00001A53 E934EC          jmp     SetPtrSav
19820
19821 ; ===== S U B R O U T I N E =====
19822
19823 ;-----
19824 ; install_bds installs a bds at location es:di into the current linked list of
19825 ; bds maintained by this device driver. it places the bds at the end of the
19826 ; list. Trashes (at least) ax, bx, di, si
19827 ;-----
19828
19829 ; 26/12/2023 - Retro DOS v5.0
19830 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1AE0h
19831
19832 00001A56 1E              push    ds                ; save Bios_Data segment
19833 00001A57 BE[1901]      mov     si, start_bds    ; beginning of chain
19834
19835 ; ds:si now points to link to first bds
19836 ; assume bds list is non-empty
19837
19838 00001A5A C534          loop_next_bds:
19839                                lds     si, [si]                ; [si+BDS.link]

```

```

19839                                ; fetch next bds
19840 00001A5C 268A4504          mov     al, [es:di+4] ; [es:di+BDS.drivenum]
19841 00001A60 384404          cmp     [si+4], al ; does this one share a physical
19842                                ; drivewith new one?
19843 00001A63 7518             jnz     short next_bds
19844 00001A65 B310             mov     bl, 10h ; fi_am_mult
19845                                ; 26/12/2023
19846 00001A67 26085D3F          or      [es:di+3Fh], bl
19847                                ; [es:di+BDS.flags]
19848                                ; set both of them to i_am_mult if so
19849 00001A6B 085C3F          or      [si+3Fh], bl
19850                                ; [si+BDS.flags]
19851 00001A6E 2680653FDF          and     byte [es:di+3Fh], 0DFh
19852                                ; and byte [es:di+23h], 0DFh ; [es:di+BDS.flags], ~fi_own_physical
19853                                ; we don't own it
19854 00001A73 8A5C3F          mov     bl, [si+3Fh]
19855                                ; [si+BDS.flags]
19856                                ; determine if changeline available
19857 00001A76 80E302          and     bl, 2 ; fchangeline
19858 00001A79 26085D3F          or      [es:di+3Fh], bl
19859                                ; [es:di+BDS.flags]
19860 next_bds:                    ; 02/09/2023 (PCDOS 7.1)
19861                                ; mov ax, 0FFFFh ; -1
19862 00001A7D B8FFFF          cmp     [si], ax ; [si+BDS.link], -1
19863 00001A80 3904             ; cmp word [si], 0FFFFh ; [si+BDS.link], -1
19864                                ; are we at end of list?
19865                                ; jnz short loop_next_bds
19866 00001A82 75D6             mov     [si+2], es ; [si+BDS.link+2], es
19867 00001A84 8C4402          ; install bds
19868                                ; mov [si], di
19869 00001A87 893C             mov     [es:di], ax ; [es:di+BDS.link], -1
19870 00001A89 268905          mov     word [es:di], 0FFFFh ; [es:di+BDS.link], -1
19871                                ; set next pointer to null
19872                                ; pop ds
19873 00001A8C 1F              pop     ds
19874
19875 ; 01/07/2023 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS - BIOSCODE:1785h)
19876 ; 16/10/2022 (MSDOS 6.0 Code)
19877
19878 ; **** If the new drive has a higher EOT value, we must alter the
19879 ; 'eot' variable appropriately.
19880
19881                                ; 26/12/2023
19882 00001A8D 268A4550          mov     al, [es:di+50h] ; [es:di+BDS.rsecpertrack]
19883                                ; 01/06/2019
19884                                ; mov al, [es:di+52]
19885                                ; 22/07/2023
19886                                ; mov al, [es:di+BDS.rsecpertrack]
19887 00001A91 3A06[2C01]          cmp     al, [eot]
19888 00001A95 7603             jbe     short _eot_ok
19889 00001A97 A2[2C01]          mov     [eot], al
19890 _eot_ok:
19891 00001A9A C3              retn
19892
19893 ; -----
19894
19895 ; 17/10/2022
19896 ; DRVLET equ drvlet - DOSBIOSEG_2C7h
19897 ; SNGMSG equ sngmsg - DOSBIOSEG_2C7h
19898 ; 09/12/2022
19899 DRVLET equ drvlet
19900 SNGMSG equ sngmsg
19901
19902 ; 16/10/2022
19903
19904 ; -----
19905 ; ask to swap the disk in drive a:
19906 ; es:di -> bds
19907 ; ds -> Bios_Data
19908 ; -----
19909
19910 ; 26/12/2023 - Retro DOS v5.0
19911
19912 ; 19/10/2022
19913 00001A9B F606[1208]01      swpdsk: test     byte [IsWin386], 1
19914                                ; test ds:IsWin386, 1 ; Is win386 present?
19915 00001AA0 7405             jz      short no_win386 ; no, skip SetFocus
19916
19917                                ; set focus to the correct VM
19918                                ; call far ptr 70h:813h ; PCDOS 7.1 IBMBIO.COM BIOSCODE:1B2Ch
19919                                ; call far 70h:8D1h ; MSDOS 6.21 IO.SYS BIOSCODE:179Ah
19920                                ; 17/10/2022
19921 00001AA2 9A[1308]7000      call     DOSBIOSEG:V86_Crit_SetFocus ; BIOSDATA:V86_Crit_SetFocus
19922                                ; call far ptr V86_Crit_SetFocus ; call far 70h:8D1h
19923                                ; call far KERNEL_SEGMENT:V86_Crit_SetFocus
19924 no_win386:
19925 00001AA7 51              push     cx
19926 00001AA8 52              push     dx
19927 00001AA9 268A5505          mov     dl, [es:di+5] ; [es:di+BDS.drivelet]
19928                                ; get the drive letter
19929
19930 ; WARNING : next two instructions assume that if the new disk is for drive B
19931 ; then existing dsk is drive A & vice versa
19932
19933 00001AAD 88D6             mov     dh, dl
19934 00001AAF 80F601          xor     dh, 1
19935 00001AB2 29C9             sub     cx, cx ; nobody has handled swap disk
19936 00001AB4 B8004A          mov     ax, 4A00h ; multMULT<<8)|multMULTSWPDSK
19937                                ; broadcast code for swap disk
19938                                ; Broadcast it
19939 00001AB7 CD2F             int     2Fh
19940 00001AB9 41              inc     cx ; cx == -1 ?
19941 00001ABA 741E             jz      short swpdsk9 ; somebody has handled it
19942
19943 ; using a different drive in a one drive system so request the user change disks
19944
19945 00001ABC 80C241          add     dl, 'A'
19946                                ; 17/10/2022
19947 00001ABF 2E8816[F91A]      mov     [cs:DRVLET], dl ; "A: and press any key when ready\r\n\n"
19948                                ; 16/10/2022
19949                                ; mov byte [cs:drvlet], dl
19950                                ; mov byte ptr cs:17E4h, dl ; [cs:drvlet]
19951                                ; 0070h:3D54h = 2C7h:17E4h
19952 00001AC4 BE[DD1A]          mov     si, SNGMSG ; "\r\nInsert diskette for drive "
19953                                ; mov si, 17C8h ; sngmsg
19954                                ; 0070h:3D38h = 2C7h:17C8h
19955 00001AC7 53              push     bx
19956 00001AC8 2E              cs
19957 00001AC9 AC              lodsb ; get the next character of the message
19958                                ; lods byte ptr cs:[si]
19959 wrmsg_loop:
19960 00001ACA CD29             int     29h ; DOS 2+ internal - FAST PUTCHAR
19961                                ; AL = character to display
19962 00001ACC 2E              cs

```

```

19963 00001ACD AC          lodsb
19964                      ;lods byte ptr cs:[si] ; cs lodsb
19965                      ; get the next character of the message
19966 00001ACE 08C0          or al, al
19967 00001AD0 75F8          jnz short wrmsg_loop
19968 00001AD2 E833E7        call con_flush ; flush out keyboard queue
19969                      ; call rom-bios
19970 00001AD5 30E4          xor ah, ah
19971 00001AD7 CD16          int 16h ; KEYBOARD - READ CHAR FROM BUFFER, WAIT IF EMPTY
19972                      ; Return: AH = scan code, AL = character
19973 00001AD9 5B            pop bx
19974 swpdsk9:
19975 00001ADA 5A            pop dx
19976 00001ADB 59            pop cx
19977 00001ADC C3            retn
19978
19979                      ; -----
19980                      ; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
19981
19982                      ; -----
19983                      ; include msbio.c12 (MSDOS 6.0, 1991)
19984                      ; -----
19985                      ; (MSDOS 6.21 IO.SYS BIOSCODE:17D5h)
19986                      ; -----
19987                      ; 17/03/2019 - Retro DOS v4.0
19988                      ; 26/12/2023 - Retro DOS v5.0
19989
19990                      ; MSDOS 5.0 IO.SYS offset 0070h:3D38h or 02C7h:17C8h
19991                      ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1B67h
19992 sngmsg:
19993 00001ADD 0D0A          db 0Dh,0Ah
19994 00001ADF 496E73657274206469- db 'Insert diskette for drive '
19995 00001AE8 736B6574746520666F-
19996 00001AF1 7220647269766520
19997
19998                      ; MSDOS 5.0 IO.SYS offset 0070h:3D54h or 02C7h:17E4h
19999                      ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1B83h
20000 drvlet:
20001                      db 'A: and press any key when ready',0Dh,0Ah
20002
20003                      db 0Ah,0
20004
20005                      ; ===== S U B R O U T I N E =====
20006                      ; -----
20007                      ; input : es:di points to current bds for drive.
20008                      ; return : zero set if no open files
20009                      ; zero reset if open files
20010                      ; -----
20011                      ; 26/12/2023 - Retro DOS v5.0
20012 chkopcnt:
20013 cmp word [es:di+3Ch], 0
20014 ;cmp word [es:di+20h], 0 ; [es:di+BDS.opcnt]
20015 retn
20016
20017                      ; ===== S U B R O U T I N E =====
20018                      ; -----
20019                      ; at media check time, we need to really get down and check what the change is.
20020                      ; this is guaranteed to be expensive.
20021                      ;
20022                      ; es:di -> bds, ds -> Bios_Data
20023                      ; -----
20024                      ; 26/12/2023 - Retro DOS v5.0
20025                      ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1BA6h
20026 mediacheck:
20027 call checksingle ; make sure correct disk is in place
20028 xor si, si
20029 call haschange
20030 jz short mediaret
20031 ; 26/12/2023
20032 ;test byte [es:di+3Fh], 40h ; [es:di+BDS.flags], fchanged ; 40h
20033 call checkromchange
20034 jnz short mediadovolid
20035 push ax
20036 push dx
20037 mov dl, [es:di+4] ; [es:di+BDS.drivenum]
20038 ; set logical drive number
20039 mov ah, 16h
20040 int 13h ; DISK - FLOPPY DISK - CHANGE OF DISK STATUS (AT,XT2,XT286,CONV,PS)
20041 ; DL = drive to check
20042 ; Return: AH = disk change status
20043 pop dx
20044 pop ax
20045 jb short mediadovolid
20046 mov si, 1 ; signal no change
20047
20048 ; there are some drives with changeline that "lose" the changeline indication
20049 ; if a different drive is accessed after the current one. in order to avoid
20050 ; missing a media change, we return an "i don't know" to dos if the changeline
20051 ; is not active and we are accessing a different drive from the last one.
20052 ; if we are accessing the same drive, then we can safely rely on the changeline
20053 ; status.
20054 ; 19/10/2022
20055 mov bl, [tim_drv] ; get last drive accessed
20056 cmp [es:di+4], bl ; [es:di+BDS.drivenum]
20057 ; (If the last drive accessed is not current drive
20058 ; media change status may be incorrect. So,
20059 ; "I don't now" will be returned even if it is indicated
20060 ; as media is not changed.)
20061 jz short mediaret ; (same drive,
20062 ; mediachangeline indication is reliable)
20063
20064 ; do the 2 second twiddle. if time >= 2 seconds, do a valid check.
20065 ; otherwise return "i don't know" (strictly speaking, we should return a
20066 ; "not changed" here since the 2 second test said no change.)
20067
20068 push ax
20069 push cx
20070 push dx
20071 call Check_Time_Of_Access
20072 pop dx
20073 pop cx
20074 pop ax
20075 or si, si
20076 jz short mediadovolid ; check_time says ">= 2 secs passed"
20077 ; (volume id will be checked)
20078 xor si, si ; return "i don't know"
20079 mediaret:
20080 retn
20081                      ; -----

```

```

20082
20083 ; somehow the media was changed. look at vid to see. we do not look at fat
20084 ; because this may be different since we only set medbyt when doing a read
20085 ; or write.
20086
20087 mediadovolid:
20088     call    GetBp          ; builda new bpb in current bds
20089     jb      short mediaret
20090     call    check_vid
20091     jnb     short mediaret
20092     jmp     maperror       ; fix up al for      return to dos
20093 ; -----
20094
20095 ; simple, quick check of latched change. if no indication, then return
20096 ; otherwise do expensive check. if the expensive test fails, pop off the
20097 ; return and set al = 15 (for invalid media change) which will be returned to
20098 ; dos.
20099 ;
20100 ; for dos 3.3, this will work only for the drive that has changeline.
20101 ;
20102 ; call with es:di -> bds, ds -> Bios_Data
20103 ; ***** warning: this routine will return one level up on the stack
20104 ; if an error occurs!
20105
20106 checklatchio:
20107
20108 ; if returning fake bpb then assume the disk has not changed
20109
20110     ; 26/12/2023
20111     ; cmp word [es:di+3Ch], 0 ; [es:di+BDS.opcnt]
20112     call    chkopcmt
20113     jz      short checkret ; done if zero
20114
20115 ; check for past rom indications. if no rom change indicated, then return ok.
20116
20117     ; 26/12/2023
20118     ; test word [es:di+3Fh], 40h
20119     ; test [es:di+BDS.flags], fchanged ; 40h
20120     call    checkromchange
20121     jz      short checkret
20122
20123 ; we now see that a change line has been seen in the past. let's do the
20124 ; expensive verification.
20125
20126     call    GetBp          ; buildbpb in current bds
20127     jb      short ret_no_error_map ; getbp has already called maperror
20128     call    check_vid
20129     jb      short checklatchret ; disk error trying      to read in.
20130     or      si, si         ; is changed for sure?
20131     jns     short checkret
20132     call    returnvid
20133
20134 checklatchret:
20135     call    maperror       ; fix up al for      return to dos
20136 ret_no_error_map:
20137     stc
20138     pop     si             ; pop off return address
20139
20140 checkret:
20141     ret
20142 ; -----
20143
20144 ; check the fat and the vid. return in di -1 or 0. return with carry set
20145 ; only if there was a disk error. return that error code in ax.
20146 ;
20147 ; called with es:di -> bds, ds -> Bios_Data
20148
20149 checkfatvid:
20150     call    fat_check      ; check the fat and the vid
20151     or      si, si
20152     js      short changed_drv
20153
20154 ; the fat was the same. fall into check_vid and check volume id.
20155
20156     ; fall into check_vid
20157
20158 ; ===== S U B   R O U T I N E =====
20159
20160 ; now with the extended boot record, the logic should be enhanced.
20161 ;
20162 ; if it is the extended boot record, then we check the volume serial
20163 ; number instead of volume id. if it is different, then set si to -1.
20164 ;
20165 ; if it is same, then si= 1 (no change).
20166 ;
20167 ; if it is not the extended boot record, then just follows the old
20168 ; logic. dos 4.00 will check if the # of fat in the boot record bpb
20169 ; is not 0. if it is 0 then it must be non_fat based system and
20170 ; should have already covered by extended boot structure checking.
20171 ; so, we will return "i don't know" by setting si to 0.
20172 ;
20173 ; this routine assume the newest valid boot record is in cs:[disksector].
20174 ; (this will be gauranteed by a successful getbp call right before this
20175 ; routine.)
20176 ;
20177 ; called with es:di -> bds, ds -> bds
20178
20179     ; 26/12/2023 - Retro DOS v5.0
20180     ; 19/10/2022
20181
20182 check_vid:
20183
20184 ; check the disksector.EXT_BOOT_SIG variable for the extended
20185 ; boot signature. if it is set then go to do the extended
20186 ; id check otherwise continue with code below
20187
20188     ; 26/12/2023
20189     ;;;
20190     cmp     word [disksector+16h], 0 ; BPB_FATSz16
20191     jnz     short chk_vid_1
20192     cmp     byte [disksector+42h], 29h ; BS_FAT32_BootSig
20193     ; [disksector+EXT_BOOT.SIG], EXT_BOOT_SIGNATURE
20194     jmp     short chk_vid_2
20195
20196 chk_vid_1:
20197     ;;;
20198     cmp     byte [disksector+26h], 29h
20199     ; [disksector+EXT_BOOT.SIG],
20200     ; EXT_BOOT_SIGNATURE
20201
20202 chk_vid_2:
20203     ; 26/12/2023
20204     jz      short do_ext_check_id
20205     call    haschange
20206     jz      short checkret
20207     xor     si, si
20208     cmp     byte [disksector+10h], 0 ; BPB_NumFATS
20209     ; [disksector+EXT_BOOT.BPB+EBPB.NUMBEROFFATS]
20210     jz      short checkfatret ; don't read vol id
20211     ; if not fat system

```

```

20206 00001BB4 E8F400      call    read_volume_id
20207 00001BB7 720C      jb     short checkfatret
20208 00001BB9 E89901      call    check_volume_id
20209 00001BBC BEFFFF      mov     si, 0FFFFh ; -1
20210                                ; definitely changed
20211 00001BBF 7505      jnz     short changed_drv
20212
20213 00001BC1 46      inc     si ; not changed
20214 vid_no_changed:
20215 00001BC2 E8C000      call    resetchanged
20216                                ; 12/12/2022
20217                                ; cf=0 ('and' instruction in 'resetchanged' clears cf)
20218                                clc
20219 checkfatret:
20220 00001BC5 C3      retn
20221 ; -----
20222
20223                                ; 12/12/2022
20224 changed_drv:
20225 00001BC6 F8      clc
20226 00001BC7 C606[1E01]FF      mov     byte [tim_drv], 0FFh ; cas -- return no error
20227                                ; ensure that we ask rom for media
20228 00001BCC C3      retn ; checknext time round
20229 ; -----
20230
20231 ; extended id check
20232
20233 ; 16/10/2022
20234
20235 ; the code to check extended id is basically a check to see if the
20236 ; volume serial number is still the same. the volume serial number
20237 ; previously read is in cs:disksector.EXT_BOOT_SERIAL
20238 ; ds:di points to the bds of the drive under consideration.
20239 ; the bds has fields containing the high and low words
20240 ; of the volume serial number of the media in the drive.
20241 ; compare these fields to the fields mentioned above. if these fields
20242 ; do not match the media has changed and so we should jump to the code
20243 ; starting at ext_changed else return "i don't know" status
20244 ; in the register used for the changeline status and continue executing
20245 ; the code given below. for temporary storage use the register which
20246 ; has been saved and restored around this block.
20247 ;
20248 ; bds fields in inc\msbds.inc
20249
20250                                ; 26/12/2023 - Retro DOS v5.0
20251                                ; 19/10/2022
20252 do_ext_check_id:
20253                                ; 26/12/2023
20254                                ; push ax
20255                                ; mov ax, word ptr ds:disksector+27h
20256                                ; mov ax, [DiskSector+EXT_BOOT.SERIAL]
20257                                ; mov ax, [disksector+27h]
20258 ; 26/12/2023
20259 %if 1
20260                                ;;;
20261 00001BCD 57      push    di
20262 00001BCE BE[9501]      mov     si, disksector+43h ; BS_FAT32_VolID
20263                                ; [DiskSector+FAT32_EXT_BOOT.SERIAL]
20264 00001BD1 833E[6801]00      cmp     word [disksector+16h], 0 ; BPB_FATSz16
20265 00001BD6 7403      jz      short chk_vid_3
20266 00001BD8 83EE1C      sub     si, 28 ; BS_VolID
20267                                ; si = disksector+27h ; [DiskSector+EXT_BOOT.SERIAL]
20268 chk_vid_3:
20269                                ; [es:di+89h] = [es:di+BDS.vol_serial]
20270 00001BDB 81C78900      add     di, 137 ; BDS.vol_serial
20271 00001BDF A7      cmpsw    ; [DiskSector+EXT_BOOT.SERIAL] ; (or FAT32_EXT_BOOT)
20272                                ; = [di+BDS.vol_serial] ?
20273 00001BE0 7501      jnz     short chk_vid_4
20274 00001BE2 A7      cmpsw    ; [DiskSector+EXT_BOOT.SERIAL+2] ; (or FAT32_EXT_BOOT)
20275                                ; = [di+BDS.vol_serial+2] ?
20276 chk_vid_4:
20277 00001BE3 5F      pop     di
20278                                ; pop ax
20279 00001BE4 7504      jnz     short ext_changed ; not equal/same
20280 00001BE6 31F6      xor     si, si ; 0 ; don't know
20281 00001BE8 EBD8      jmp     short vid_no_changed ; reset the flag
20282                                ;;;
20283 %else
20284                                ; 02/09/2023
20285                                xor     si, si ; 0
20286                                cmp     ax, [es:di+57h] ; [di+BDS.vol_serial]
20287                                jnz     short ext_changed
20288                                mov     ax, [disksector+29h] ; [DiskSector+EXT_BOOT.SERIAL+2]
20289                                cmp     ax, [es:di+59h] ; [di+BDS.vol_serial+2]
20290                                jnz     short ext_changed
20291                                xor     si, si ; 0
20292                                ; don't know
20293                                pop     ax
20294                                jmp     short vid_no_changed
20295                                ; reset the flag
20296 %endif
20297 ; -----
20298
20299 ext_changed:
20300                                ; 26/12/2023
20301                                ; pop ax
20302                                ; 02/09/2023
20303                                ; dec si ; mov si, 0FFFFh ; -1
20304                                ; mov si, 0FFFFh ; -1
20305 00001BEA BEFFFF      mov     si, 0FFFFh ; disk changed!
20306                                ; 12/12/2022
20307                                ; ('changed_drv' clears cf)
20308                                clc
20309 00001BED EBD7      jmp     short changed_drv
20310 ; -----
20311
20312 ; at i/o time, we detected the error. now we need to determine whether the
20313 ; media was truly changed or not. we return normally if media change unknown.
20314 ; and we pop off the call and jmp to harderr if we see an error.
20315 ;
20316 ; es:di -> bds
20317
20318 checkio:
20319                                cmp     ah, 6
20320                                jnz     short checkfatret
20321 00001BEF 80FC06      call    chkopcmt
20322 00001BF2 75D1      jz      short checkfatret
20323 00001BF4 E825FF      call    GetBp
20324 00001BF7 74CC      jb     short no_error_map
20325 00001BF9 E8DAEA      call    checkfatvid
20326 00001BFC 7212      jb     short checkioret ; disk error trying to read in.
20327 00001BFE E889FF      call    short checkioret ; disk error trying to read in.
20328 00001C01 7209      or      si, si ; is changed for sure?
20329 00001C03 09F6

```



```

20454             ;cmp     byte [es:di+22h], 9 ; [es:di+BDS.formfactor]
20455             ;         ; ff288
20456             jz      short Is720K
20457             mov     al, 7 ; seven sectors / fat
20458             mov     bx, 57359 ; 224*256+0Fh
20459             ;         ; 224 root dir entries
20460             ;         ; & 0Fh sector max
20461             mov     cx, 2400 ; 80*15*2
20462             ;         ; 80 tracks, 15 sectors/track,
20463             ;         ; 2 sides
20464             ; 02/09/2023
20465             pop     dx ; pop off return address
20466             mov     dx, 258 ; 1*256+2
20467             ;         ; sectors/allocation unit
20468             ;         ; & head max
20469             ;         ; pop off return address
20470             add     sp, 2 ; return to tail of getbp
20471             jmp     Has1
20472             ; -----
20473             Is720K:
20474             ; 02/09/2023
20475             pop     bx ; pop off return address
20476             add     sp, 2 ; pop off return address
20477             jmp     Has720K ; return to 720K code
20478             ; -----
20479             ; 18/12/2022
20480             ;dofloppy:
20481             ;retn
20482
20483             ; ===== S U B R O U T I N E =====
20484             ; 16/10/2022
20485
20486             ; -----
20487             ; set_changed_d1 - sets flag bits according to bits set in bx.
20488             ;         essentially used to indicate changeline, or format.
20489             ;
20490             ; inputs: d1 contains physical drive number
20491             ;         bx contains bits to set in the flag field in the bdss
20492             ; outputs: none
20493             ; registers modified: flags
20494             ;
20495             ; called from int13 hooker. Must preserve ALL registers!!!
20496             ;
20497             ; in the virtual drive system we *must* flag the other drives as being changed
20498             ; -----
20499
20500             ; 26/12/2023 - Retro DOS v5.0
20501             set_changed_d1:
20502             push     es
20503             push     di
20504             les     di, ds:start_bds
20505             ; 19/10/2022
20506             les     di, [start_bds]
20507
20508             ; note: we assume that the list is non-empty
20509
20510             scan_bds:
20511             cmp     [es:di+4], d1 ; [es:di+BDS.drivenum]
20512             jnz     short get_next_bds
20513
20514             ; someone may complain, but this *always* must be done when a disk change is
20515             ; noted. there are *no* other compromising circumstances.
20516
20517             ; 26/12/2023
20518             or      [es:di+3Fh], bx ; [es:di+BDS.flags]
20519             or      [es:di+23h], bx ; [es:di+BDS.flags]
20520             ; signal change on other drive
20521
20522             get_next_bds:
20523             les     di, [es:di] ; [es:di+BDS.link]
20524             ; go to next bds
20525             cmp     di, 0FFFFh
20526             jnz     short scan_bds ; loop unless we hit end of chain
20527             pop     di
20528             pop     es
20529             retn
20530
20531             ; ===== S U B R O U T I N E =====
20532             ; -----
20533             ; checkromchange - see if external program has diddled rom change line.
20534             ;
20535             ; inputs: es:di points to current bds.
20536             ; outputs: zero set - no change
20537             ;         zero reset - change
20538             ; registers modified: none
20539             ; -----
20540
20541             ; 26/12/2023 - Retro DOS v5.0
20542             checkromchange:
20543             test     word [es:di+BDS.flags], fchanged ; 40h
20544             ; 26/12/2023
20545             test     byte [es:di+3Fh], 40h
20546             ; 10/12/2022
20547             test     byte [es:di+23h], 40h
20548             ;test     word [es:di+23h], 40h ; [es:di+BDS.flags]
20549             ;fchanged
20550             retn
20551
20552             ; ===== S U B R O U T I N E =====
20553             ; -----
20554             ; resetchanged - restore value of change line
20555             ;
20556             ; inputs: es:di points to current bds
20557             ; outputs: none
20558             ; registers modified: none
20559             ; -----
20560
20561             ; 26/12/2023 - Retro DOS v5.0
20562             resetchanged:
20563             and     word [es:di+BDS.flags], ~fchanged ; 0FFBFh
20564             ; 26/12/2023
20565             and     byte [es:di+3Fh], 0BFh
20566             ; 10/12/2022
20567             and     byte [es:di+23h], 0BFh
20568             ;and     word [es:di+23h], 0FFBFh ; [es:di+BDS.flags]
20569             ;~fchanged
20570             retn
20571
20572             ; ===== S U B R O U T I N E =====
20573             ; -----
20574
20575             ; -----
20576
20577             ; -----

```



```

20578 ; haschange - see if drive can supply change line
20579 ;
20580 ; inputs: es:di points to current bds
20581 ; outputs: zero set - no change line available
20582 ; zero reset - change line available
20583 ; registers modified: none
20584 ;-----
20585 ; 26/12/2023 - Retro DOS v5.0
20586
20587 haschange:
20588 ;test word [es:di+BDS.flags], fchangeline ; 2
20589 ; 26/12/2023
20590 00001C8B 26F6453F02 test byte [es:di+3Fh], 2
20591 ; 10/12/2022
20592 ;test byte [es:di+23h], 2
20593 ;;test word [es:di+23h], 2 ; [es:di+BDS.flags]
20594 ; fchangeline
20595 00001C90 C3 retn
20596
20597 ; -----
20598
20599 ; 16/10/2022
20600
20601 ;-----
20602 ; set_volume_id - main routine, calls other routines.
20603 ; read_volume_id - read the volume id and tells if it has been changed.
20604 ; transfer_volume_id - copy the volume id from tmp to special drive.
20605 ; check_volume_id - compare volume id in tmp area with one expected for drive.
20606 ; fat_check - see of the fatid has changed in the specified drive.
20607 ;-----
20608
20609 ; set_volume_id
20610 ; if drive has changeline support, read in and set the volume_id
20611 ; and the last fat_id byte. if no change line support then do nothing.
20612 ;
20613 ; on entry:
20614 ; es:di points to the bds for this disk.
20615 ; ah contains media byte
20616 ;
20617 ; on exit:
20618 ; carry clear:
20619 ; successful call
20620 ; carry set
20621 ; error and ax has error code
20622
20623 set_volume_id:
20624 00001C91 52 push dx ; save registers
20625 00001C92 50 push ax
20626 00001C93 E8F5FF call haschange ; does drive have changeline support?
20627 00001C96 740B jz short setvret ; no, get out
20628 00001C98 E81000 call read_volume_id
20629 00001C9B 7209 jb short seterr
20630 00001C9D E8A900 call transfer_volume_id ; copy the volume id to special drive
20631 00001CA0 E8E2FF call resetchanged ; restore value of change line
20632
20633 setvret:
20634 ; 10/12/2022
20635 ; cf = 0
20636 ;clc ; no error, clear carry flag
20637 00001CA3 58 pop ax ; restore registers
20638 00001CA4 5A pop dx
20639 00001CA5 C3 retn
20640
20641 ; -----
20642
20643 seterr:
20644 00001CA6 5A pop dx ; pop stack but don't overwrite ax
20645 00001CA7 5A pop dx ; restore dx
20646 00001CA8 C3 retn
20647
20648 ; -----
20649
20650 root_sec: dw 0 ; root sector #
20651
20652 ; 16/10/2022
20653 ;ROOTSEC equ root_sec - DOSBIOSEG_2C7h
20654 ; 09/12/2022
20655 ROOTSEC equ root_sec
20656
20657 ; ===== S U B R O U T I N E =====
20658
20659 ; 16/10/2022
20660
20661 ; read_volume_id read the volume id and tells if it has been changed.
20662 ;
20663 ; on entry:
20664 ; es:di points to current bds for drive.
20665 ;
20666 ; on exit:
20667 ; carry clear
20668 ; si = 1 no change
20669 ; si = 0 ?
20670 ; si = -1 change
20671 ;
20672 ; carry set:
20673 ; error and ax has error code.
20674
20675 read_volume_id:
20676 00001CAB 52 push dx ; preserve registers
20677 00001CAC 51 push cx
20678 00001CAD 53 push bx
20679 00001CAE 50 push ax
20680 00001CAF 06 push es ; stack the bds last
20681 00001CB0 57 push di
20682 00001CB1 1E push ds ; point es to Bios_Data
20683 00001CB2 07 pop es
20684 00001CB3 BF[4008] mov di, tmp_vid ; "NO NAME "
20685 00001CB6 BE[6305] mov si, nul_vid ; "NO NAME "
20686 ; 26/12/2023
20687 00001CB9 B90B00 mov cx, 11 ; PCDOS 7.1 - 02/09/2023
20688 ;mov cx, 12 ; initialize tmp_vid to nul vi_id
20689
20690 ;rep movsb
20691 ; 26/12/2023
20692 ;rep movs byte ptr es:[di], byte ptr cs:[si]
20693 ;db 0FBh,2Eh,0A4h
20694 ;cs ; nul_vid is in BIOSCODE segment
20695 ;rep movsb
20696 rep cs
20697 movsb
20698
20699 pop di
20700 pop es
20701 mov al, [es:di+11] ; [es:di+BDS.fats]
; # of fats
; [es:di+BDS.fatsecs]
; sectors / fat

```

```

20702 00001CC9 F6E1          mul    cx, [es:di+9] ; size taken by fats
20703 00001CCB 26034509      add    ax, [es:di+9] ; [es:di+BDS.resectors]
20704                                     ; add on reserved sectors
20705                                     ;
20706                                     ; ax is now sector # (0 based)
20707                                     ; 17/10/2022
20708 00001CCF 2EA3[A91C]    mov     [cs:ROOTSEC], ax
20709                                     ;mov     word ptr cs:198Fh, ax ; [cs:root_sec]
20710                                     ; 0070h:3EFFh = 2C7h:198Fh
20711 00001CD3 268B450C      mov     ax, [es:di+12] ; [es:di+BDS.direntries]
20712                                     ; # root dir entries
20713 00001CD7 B104          mov     cx, 4 ; 16 entries/sector
20714 00001CD9 D3E8          shr     ax, cx ; divide by 16
20715                                     ;mov     cx, ax ; cx is # of sectors to scan
20716                                     ; 02/09/2023 (PCDOS 7.1, one byte opcode)
20717 00001CDB 91            xchg     ax, cx ; cx is # of sectors to scan
20718                                     next_sec:
20719 00001CDC 51            push    cx ; save outer loop counter
20720 00001CDD 2EA1[A91C]    mov     ax, [cs:ROOTSEC]
20721                                     ;mov     ax, word ptr cs:198Fh ; [cs:root_sec]
20722                                     ; get sector #
20723 00001CE1 268B4D13      mov     cx, [es:di+19] ; [es:di+BDS.sectortrack]
20724                                     ; sectors / track
20725 00001CE5 31D2          xor     dx, dx
20726 00001CE7 F7F1          div     cx
20727                                     ; set up registers for call to read_sector
20728
20729                                     inc     dx ; dx= sectors into track
20730 00001CE9 42            ; ax= track count from 0
20731                                     ; sector to read
20732 00001CEA 88D1          mov     cx, dx
20733 00001CEC 31D2          xor     dx, dx
20734 00001CEE 26F77515      div     word [es:di+21] ; [es:di+BDS.heads]
20735                                     ; # heads on this disc
20736 00001CF2 88D6          mov     dh, dx ; head number
20737 00001CF4 88C5          mov     ch, al ; track #
20738 00001CF6 E8BDEB      call    read_sector ; get first sector of the root directory,
20739                                     ; ds:bx-> directory sector
20740 00001CF9 723F          jnb     short readviderr
20741 00001CFB B91000      mov     cx, 16 ; # of dir entries in a block of root
20742 00001CFE B008          mov     al, 8 ; volume label bit
20743                                     fvid_loop:
20744                                     ; 02/09/2023 (PCDOS 7.1)
20745 00001D00 382F          cmp     [bx], ch ; 0
20746                                     ;cmp     byte [bx], 0 ; end of dir?
20747 00001D02 7433          jz      short no_vid ; yes, no vol id
20748 00001D04 803FE5      cmp     byte [bx], 0E5h ; empty entry?
20749 00001D07 7405          jz      short ent_loop ; yes, skip
20750 00001D09 84470B      test    [bx+11], al ; is volume label bit set in fcb?
20751 00001D0C 750F          jnz     short found_vid ; jmp yes
20752                                     ent_loop:
20753 00001D0E 83C320      add     bx, 32 ; add length of directory entry
20754 00001D11 E2ED          loop    fvid_loop
20755 00001D13 59            pop     cx ; outer loop
20756 00001D14 2EFF06[A91C]  inc     word [cs:ROOTSEC]
20757                                     ;inc     word ptr cs:198Fh ; inc word [root_sec]
20758                                     ; next sector
20759 00001D19 E2C1          loop    next_sec ; continue
20760                                     notfound:
20761                                     ; 02/09/2023
20762                                     ;xor     si, si
20763 00001D1B EB13          jmp     short fvid_ret
20764                                     ; -----
20765                                     found_vid:
20766                                     ; 02/09/2023
20767                                     ; cf = 0 ('test' instruction clears cf)
20768 00001D1D 59            pop     cx ; clean stack of outer loop counter
20769 00001D1E 89DE          mov     si, bx ; point to volume_id
20770 00001D20 06            push    es ; preserve current bds
20771 00001D21 57            push    di
20772 00001D22 1E            push    ds
20773 00001D23 07            pop     es ; point es to Bios_Data
20774 00001D24 BF[4008]    mov     di, tmp_vid ; "NO NAME "
20775 00001D27 B90B00      mov     cx, 11 ; VALID_SIZ-1
20776                                     ; length of string minus nul
20777                                     ; mov volume label to tmp_vid
20778 00001D2A F3A4          rep movsb
20779                                     ;xor     al, al
20780                                     ; 02/09/2023
20781 00001D2C 91            xchg     ax, cx ; ax = 0
20782 00001D2D AA          stosb ; null terminate
20783                                     ;xor     si, si
20784                                     ; 02/09/2023
20785 00001D2E 5F            xchg     ax, si ; si = 0
20786 00001D2F 07            pop     di ; restore current bds
20787 00001D2F 07            pop     es
20788                                     fvid_ret:
20789                                     ; 02/09/2023
20790 00001D30 31F6          xor     si, si ; 0
20791                                     ;
20792 00001D32 58            pop     ax
20793                                     ; 10/12/2022
20794                                     ; cf = 0
20795                                     ;clc
20796                                     rvidret:
20797 00001D33 5B            pop     bx ; restore registers
20798 00001D34 59            pop     cx
20799 00001D35 5A            pop     dx
20800 00001D36 C3          retn
20801                                     ; -----
20802                                     no_vid:
20803                                     pop     cx ; clean stack of outer loop counter
20804 00001D37 59            ;jmp     short notfound ; not found
20805                                     ; 02/09/2023
20806 00001D38 EBF6          jmp     short fvid_ret
20807                                     ; -----
20808                                     readviderr:
20809                                     pop     si ; trash the outer loop counter
20810 00001D3A 5E            pop     si ; caller's ax, return error code instead
20811 00001D3B 5E            jmp     short rvidret
20812 00001D3C EBF5
20813                                     ; -----
20814                                     preset_valid_addr:
20815                                     ; 26/12/2023 - Retro DOS v5.0
20816                                     ; 02/09/2023 - Retro DOS v4.2 (IO.SYS optimization)
20817                                     ; PCDOS 7.1 - IBMBIO.COM - BIOSCODE:1BCFh
20818 00001D3E BE[4008]    mov     si, tmp_vid ; "NO NAME "
20819                                     ; 26/12/2023
20820 00001D41 83C77D      add     di, 125 ; BDS.valid
20821 00001D44 B90B00      mov     cx, 11 ; VALID_SIZ (12 for MSDOS 5.0-6.22 versions)
20822                                     ; MSDOS 6.21 (MSDOS 5.0 & 6.?)

```

```

20826             ;add    di, 75          ; BDS.volid
20827             ;mov    cx, 12          ; VALID_SIZ
20828             ;
20829 00001D47 FC   ;cld
20830 00001D48 C3   ;retn
20831
20832             ; ===== S U B   R O U T I N E =====
20833
20834             ; transfer_volume_id - copy the volume id from tmp to special drive
20835             ;
20836             ; inputs:  es:di has current bds
20837             ; outputs: bds for drive has volume id from tmp
20838
20839             ; 27/12/2023 - Retro DOS v5.0
20840 transfer_volume_id:
20841 00001D49 57     ; push    di          ; copy the volume id from tmp to special drive
20842             ; push    si
20843 00001D4A 51     ; push    cx
20844             ; 27/12/2023
20845 00001D4B 56     ; push    si
20846
20847             ; mov     si, tmp_vid      ; "NO NAME      "
20848             ; add     di, BDS.volid
20849             ; add     di, 75           ; BDS.volid
20850             ; mov     cx, VALID_SIZ
20851             ; mov     cx, 12          ; VALID_SIZ
20852             ; cld
20853             ; 02/09/2023 (PCDOS 7.1)
20854 00001D4C E8FFFF ; call    preset_volid_addr
20855
20856 00001D4F F3A4    ; rep movsb
20857
20858             ; 27/12/2023
20859 00001D51 5E     ; pop     si
20860 chk_volid_ok:
20861 00001D52 59     ; pop     cx
20862             ; pop     si
20863 00001D53 5F     ; pop     di
20864 00001D54 C3     ; retn
20865
20866             ; ===== S U B   R O U T I N E =====
20867
20868             ; check_volume_id - compare volume id in tmp area with
20869             ; one expected for drive
20870             ;
20871             ; inputs: es:di has current bds for drive
20872             ; outputs: zero true means it matched
20873
20874             ; 27/12/2023 - Retro DOS v5.0
20875 check_volume_id:
20876 00001D55 57     ; push    di
20877 00001D56 51     ; push    cx
20878
20879             ; mov     si, tmp_vid      ; "NO NAME      "
20880             ; add     di, BDS.volid
20881             ; add     di, 75           ; BDS.volid
20882             ; mov     cx, VALID_SIZ
20883             ; mov     cx, 12          ; VALID_SIZ
20884             ; cld
20885             ; 02/09/2023 (PCDOS 7.1)
20886 00001D57 E8E4FF ; call    preset_volid_addr
20887
20888 00001D5A F3A6    ; repe cmpsb          ; are the 2 volume_ids the same?
20889
20890             ; 27/12/2023
20891             ; pop     cx
20892             ; pop     di
20893             ; retn
20894 00001D5C EBF4    ; jmp     short chk_volid_ok
20895
20896             ; ===== S U B   R O U T I N E =====
20897
20898             ; fat_check - see of the fatid has changed in the specified drive.
20899             ; - uses the fat id obtained from the boot sector.
20900             ;
20901             ; inputs: medbyt is expected fat id
20902             ; es:di points to current bds
20903             ;
20904             ; output: si = -1 if fat id different,
20905             ; si = 0 otherwise
20906             ;
20907             ; no other registers changed.
20908
20909 fat_check:
20910 00001D5E 50     ; push    ax
20911 00001D5F 31F6   ; xor     si, si          ; say fat id's are same.
20912 00001D61 A0[1F01] ; mov     al, [medbyt]      ; 19/10/2022
20913 00001D64 263A4510 ; cmp     al, [es:di+10h]   ; [es:di+BDS.media]
20914             ; jz      short okret1    ; compare it with the bds medbyte
20915 00001D68 7401   ; jz      short okret1      ; carryclear
20916 00001D6A 4E     ; dec     si
20917 okret1:
20918 00001D6B 58     ; pop     ax
20919 00001D6C C3     ; retn
20920
20921             ; -----
20922
20923             ; BIOSCODE:1DFEh (PCDOS 7.1 IBMBIO.COM) ; 27/12/2023
20924             ; times 2 db 0
20925
20926             ; BIOSCODE:1A69h (MSDOS 6.21 IO.SYS) ((& MSDOS 6.22 IO.SYS))
20927             ; times 7 db 0
20928
20929             ; BIOSCODE:180Bh (MSDOS 5.0 IO.SYS)
20930
20931             ; 09/12/2022
20932             ; times 4 db 0 ; 17/10/2022
20933             ; db 4 dup(0) ; times 4 db 0
20934
20935             ; -----
20936
20937             ; 09/12/2022
20938             ; db 0
20939
20940 number2div equ ($-BCode_start)
20941 number2mod equ (number2div % 16)
20942
20943 %if number2mod>0 & number2mod<16
20944 00001D6D 00<rep 3h> times (16-number2mod) db 0
20945 %endif
20946
20947 ;align 16
20948
20949 ; 09/12/2022

```

```

20950 BCODE_END equ $ - BCode_start
20951 ; 31/03/2024
20952 BCODEEND:
20953 ;SYSINITSEG equ IOSYSCODESEG+(BCODE_END>>4)
20954 ; 13/12/2022
20955 SYSINITOFFSET equ BCODE_END
20956 SYSINITSEG equ IOSYSCODESEG+(SYSINITOFFSET>>4)
20957
20958 ; 30/12/2022 - Retro DOS v4.2
20959 ; (SYSINITSEG is 473h for MSDOS 6.21 IO.SYS)
20960
20961 ;--- End of DOSBIOS code segment -----
20962
20963 ; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
20964 ; 01/05/2019 - Retro DOS v4.0
20965 =====
20966 ; end of BIOSCODE
20967
20968 ; -----
20969 ; %include sysinit5.s ; 09/12/2022
20970 ; -----
20971
20972 ; =====
20973 ; (IO.SYS) SYSINIT SEGMENT
20974 ; =====
20975 ; 09/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
20976 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
20977 ;
20978 ; 27/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
20979
20980 section .SYSINIT vstart=0
20981 ; *****
20982 ; SYSINIT.BIN (MSDOS 6.21 IO.SYS) - RETRO DOS v4.0 by ERDOGAN TAN - 21/10/2022
20983 ; -----
20984 ; Last Update: 04/01/2023 (Modified IO.SYS) ((Previous: 30/12/2022))
20985 ; -----
20986 ; Beginning: 03/06/2018 (Retro DOS 3.0), 21/03/2019 (Retro DOS 4.0)
20987 ; -----
20988 ; Assembler: NASM version 2.15
20989 ; -----
20990 ; ((nasm sysinit6.s -l sysinit6.lst -o SYSINIT6.BIN -Z error.txt))
20991 ; -----
20992 ; Modified from 'sysinit2.s' (SYSINIT2.BIN) file of Retro DOS v3.0 (6/7/2018)
20993 ; -----
20994 ; Derived from 'SYSINIT1.ASM' and 'SYSINIT2.ASM' files of MSDOS 6.0
20995 ; source code by Microsoft, 1991
20996 ; -----
20997 ; Derived from 'SYSINIT.ASM' file of MSDOS 2.0 (IBM PCDOS v2.0) source code
20998 ; by Microsoft, 12/10/1983
20999 ; *****
21000 ; main file: 'retrodos4.s'
21001 ; incbin 'SYSINIT3.BIN' ; (SYINITSEG)
21002
21003 ; 30/12/2022 - Retro DOS v4.2
21004 ; Retro DOS v4.0 - 2019
21005 ; SYSINIT (MSDOS 6.21 IO.SYS) draft: 'sysinit3.s' (01/07/2019)
21006
21007 ; 21/10/2022
21008 ; -----
21009 ; This source code (version) is based on SYSINIT source code of disassembled
21010 ; MSDOS 5.0 IO.SYS file (SYSINIT.BIN)
21011 ; Disassembler: Hex-Rays Interactive Disassembler (IDA)
21012 ; -----
21013 ; Binary file splitter & joiner: FFSJ v3.3
21014
21015 ; -----
21016 ; SYSINIT.TXT (27/01/1983)
21017 ; -----
21018 ; SYSINIT is a module linked behind the OEM bios. It takes
21019 ; over the system initialization after the OEM bios has
21020 ; performed any initialization it needs to do. Control is
21021 ; transfered with a long jump to the external variable SYSINIT
21022 ;
21023 ;
21024 ; The OEM has the following variables declared external:
21025 ;
21026 ; CURRENT_DOS_LOCATION WORD
21027 ;
21028 ; This word contains the segment number of the DOS before it
21029 ; is relocated. The OEM bios must set this value.
21030 ;
21031 ; FINAL_DOS_LOCATION WORD
21032 ;
21033 ; This word contains the segment number of the DOS after SYSINIT
21034 ; moves it. The OEM bios must set this value.
21035 ;
21036 ; DEVICE_LIST DWORD
21037 ;
21038 ; This double word pointer points to the linked list of
21039 ; character and block device drivers. The OEM must set this
21040 ; value.
21041 ;
21042 ; MEMORY_SIZE WORD
21043 ;
21044 ; This word contains the number of RAM paragraphs. If the
21045 ; bios doesn't set this variable SYSINIT will automatically
21046 ; calculate it. NOTE: systems with PARITY checked memory must
21047 ; size memory in the BIOS. SYSINITs method is to write memory
21048 ; and read it back until it gets a mismatch.
21049 ;
21050 ; DEFAULT_DRIVE BYTE
21051 ;
21052 ; This is the initial default drive when the system first comes
21053 ; up. drive a=0, drive b=1, etc. If the bios doesn't set
21054 ; it then drive a is assumed.
21055 ;
21056 ; BUFFERS BYTE
21057 ;
21058 ; This is the default number of buffers for the system. This
21059 ; value may be overridden by the user in the CONFIG.SYS file.
21060 ; It is DBed to 2 in SYSINIT it should be greater than 1.
21061 ;
21062 ; FILES BYTE
21063 ;
21064 ; This is the default number of files for the system. This
21065 ; value may be overridden by the user in the CONFIG.SYS file.
21066 ; It is DBed to 8 in SYSINIT, values less than 5 are ignored.
21067 ;
21068 ; SYSINIT FAR
21069 ;
21070 ; The entry point of the SYSINIT module. OEM BIOS jumps to
21071 ; this label at the end of its INIT code.
21072 ;
21073 ;

```

```

21074 ; The OEM has the following variables declared public:
21075 ;
21076 ; RE_INIT FAR
21077 ;
21078 ;This is an entry point which allows the BIOS to do some INIT
21079 ;work after the DOS is initialized. ALL REGISTERS MUST BE
21080 ;PRESERVED. On entry DS points to the first available memory
21081 ;(after the DOS). DS:0 points to a 100H byte program header
21082 ;prefix which represents the "program" currently running.
21083 ;This program should be thought of as the OEM BIOS and
21084 ;SYSINIT taken together. This is not a normal program in
21085 ;that no memory is allocated to it, it is running in free
21086 ;memory.
21087 ;NOTES:
21088 ; At the time this routine is called SYSINIT occupies the
21089 ;highest 10K of memory ("highest" is determined by the value
21090 ;of the MEMORY_SIZE variable), DO NOT DO WRITES THERE.
21091 ; Since this is called AFTER DOS is initialized, you can
21092 ;make system calls. This also implies that the code for this
21093 ;routine CANNOT be thrown away by use of the
21094 ;FINAL_DOS_LOCATION since the DOS has already been moved.
21095 ; If you don't want anything done just set this to point
21096 ;at a FAR RET instruction.
21097
21098 ; -----
21099 ; TITLE BIOS SYSTEM INITIALIZATION
21100 ; -----
21101
21102 ;include version.inc
21103 ; -----
21104
21105 ;FALSE EQU 0
21106 ;TRUE EQU 0FFFFh
21107
21108 ;IBMVER EQU TRUE
21109 ;IBMCPYRIGHT EQU FALSE
21110 ;STACKSW EQU TRUE ;Include Switchable Hardware Stacks
21111 ;IBMJAPVER EQU FALSE ; If TRUE set KANJI true also
21112 ;MSVER EQU FALSE
21113 ;ALTVECT EQU FALSE ; Switch to build ALTVECT version
21114 ;KANJI EQU FALSE
21115
21116 ;(MSDOS 6.0, versiona.inc, 1991)
21117 ; -----
21118 ;MAJOR_VERSION EQU 6
21119 ;MINOR_VERSION EQU 0 ;6.00
21120 ;MINOR_VERSION EQU 21 ;6.21 ; 21/03/2019 - Retro DOS v4.0
21121
21122 ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0)
21123 ; -----
21124 ;MAJOR_VERSION EQU 5
21125 ;MINOR_VERSION EQU 0
21126
21127 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21)
21128 ;MAJOR_VERSION EQU 6
21129 ;MINOR_VERSION EQU 22
21130
21131 ; 21/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1)
21132 MAJOR_VERSION EQU 7
21133 MINOR_VERSION EQU 10
21134
21135 expected_version equ (MINOR_VERSION<<8)+MAJOR_VERSION
21136
21137 ;DOSREVN equ 00000000b ; m037 - bits 0-2 = revision number of DOS
21138 ; currently 0.
21139 DOSREVN equ 00000111b ; [[[ 7 for Retro DOS v4.0 ]]] (21/03/2019)
21140 DOSINROM equ 00001000B ; bit 3 of ver flags returned in BH
21141 DOSINHMA equ 00010000B ; bit 4 of ver flags
21142
21143 ; ifl
21144 ; %OUT ... for DOS Version 5.00 ...
21145 ; endif
21146
21147 ;*****
21148 ;Each assembler program should:
21149 ; mov ah,030h ;DOS Get Version function
21150 ; int 021h ;Version ret. in AX,minor version first
21151 ; cmp ax,expected_version ;ALL utilities should check for an
21152 ; jne error_handler ; EXACT version match.
21153 ;*****
21154
21155 ; -----
21156 ; device definitions
21157
21158 ;Attribute bit masks
21159 DEVTYP EQU 8000h ;Bit 15 - 1 if Char, 0 if block
21160 DEVIOTCL EQU 4000h ;Bit 14 - CONTROL mode bit
21161 ISFATBYDEV EQU 2000h ;Bit 13 - Device uses FAT ID bytes, comp media.
21162 ISGIN EQU 0001h ;Bit 0 - This device is the console input.
21163 ISGOUT EQU 0002h ;Bit 1 - This device is the console output.
21164 ISNULL EQU 0004h ;Bit 2 - This device is the null device.
21165 ISCLOCK EQU 0008h ;Bit 3 - This device is the clock device.
21166 ISIBM EQU 0010h ;Bit 4 - This device is special
21167
21168 ; The device table list has the form:
21169 struc SYSDEV
21170 .NEXT: resd 1 ;Pointer to next device header
21171 .ATT: resw 1 ;Attributes of the device
21172 .STRAT: resw 1 ;Strategy entry point
21173 .INT: resw 1 ;Interrupt entry point
21174 .NAME: resb 8 ;Name of device (only first byte used for block)
21175 .size:
21176 endstruc
21177
21178 ;Static Request Header
21179 struc SRHEAD
21180 .REQLEN: resb 1 ;Length in bytes of request block
21181 .REQUNIT: resb 1 ;Device unit number
21182 .REQFUNC: resb 1 ;Type of request
21183 .REQSTAT: resw 1 ;Status word
21184 .size: resb 8 ;Reserved for queue links
21185 endstruc
21186
21187 ;Status word masks
21188 STERR EQU 8000H ;Bit 15 - Error
21189 STBUI EQU 0200H ;Bit 9 - Buisy
21190 STDON EQU 0100H ;Bit 8 - Done
21191 STECODE EQU 00FFH ;Error code
21192 WRECODE EQU 0
21193
21194 ;Function codes
21195 DEVINIT EQU 0 ;Initialization
21196 DINITHL EQU 26 ;Size of init header
21197

```

```

21198 DEVMDCH EQU 1 ;Media check
21199 DMEDHL EQU 15 ;Size of media check header
21200 DEVBPB EQU 2 ;Get BPB
21201 DEVRDIOCTL EQU 3 ;IOCTL read
21202 DBPBHL EQU 22 ;Size of Get BPB header
21203 DEVRD EQU 4 ;Read
21204 DRDWRHL EQU 22 ;Size of RD/WR header
21205 DEVRDND EQU 5 ;Non destructive read no wait (character devs)
21206 DRDNDHL EQU 14 ;Size of non destructive read header
21207 DEVIST EQU 6 ;Input status
21208 DSTATHL EQU 13 ;Size of status header
21209 DEVIFL EQU 7 ;Input flush
21210 ; 21/02/2024
21211 ;DFLSHL EQU 15 ;Size of flush header
21212 DFLSHL EQU 13 ; PCDOS 7.1 IBMDOS.COM ; 21/02/2024
21213 DEVRT EQU 8 ;write
21214 DEVRTV EQU 9 ;write with verify
21215 DEVOST EQU 10 ;Output status
21216 DEVOTFL EQU 11 ;Output flush
21217 DEVRIOCTL EQU 12 ;IOCTL write
21218
21219 ; -----
21220 struc SYS_FCB
21221 00000000 ?? .fcb_drive: resb 1
21222 00000001 ?????????????????? .fcb_name: resb 8
21223 00000009 ?????? .fcb_ext: resb 3
21224 0000000C ????.fcb_EXTENT: resw 1
21225 0000000E ????.fcb_RECSIZ: resw 1 ; Size of record (user settable)
21226 00000010 ????.fcb_FLSIZ: resw 1 ; Size of file in bytes; used with the following
21227 ; word
21228 00000012 ????.fcb_DRVBP: resw 1 ; BP for SEARCH FIRST and SEARCH NEXT
21229 00000014 ????.fcb_FDATE: resw 1 ; Date of last writing
21230 00000016 ????.fcb_FTIME: resw 1 ; Time of last writing
21231 00000018 ?? .fcb_DEVID: resb 1 ; Device ID number, bits 0-5 if file.
21232 ; bit 7=0 for file, bit 7=1 for I/O device
21233 ; If file, bit 6=0 if dirty
21234 ; If I/O device, bit 6=0 if EOF (input)
21235 ; Bit 5=1 if Raw mode
21236 ; Bit 0=1 if console input device
21237 ; Bit 1=1 if console output device
21238 ; Bit 2=1 if null device
21239 ; Bit 3=1 if clock device
21240 00000019 ????.fcb_FIRCLUS: resw 1 ; First cluster of file
21241 0000001B ????.fcb_CLUSPOS: resw 1 ; Position of last cluster accessed
21242 0000001D ????.fcb_LSTCLUS: resw 1 ; Last cluster accessed and directory
21243 0000001F ?? resb 1 ; pack 2 12 bit numbers into 24 bits...
21244 00000020 ?? .fcb_NR: resb 1 ; Next record
21245 00000021 ???????? .fcb_RR: resb 4 ; Random record
21246 .size:
21247 endstruc
21248
21249 ; -----
21250 ; Field definition for I/O buffer information
21251
21252 ; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0, BUFFER.INC, 1991)
21253
21254 ; 03/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
21255
21256 struc BUFFINFO
21257 00000000 ????.buf_next: resw 1 ; Pointer to next buffer in list
21258 00000002 ????.buf_prev: resw 1 ; Pointer to prev buffer in list
21259 00000004 ?? .buf_ID: resb 1 ; Drive of buffer (bit 7 = 0)
21260 ; SFT table index (bit 7 = 1)
21261 ; = FFh if buffer free
21262 00000005 ?? .buf_flags: resb 1 ; Bit 7 = 1 if Remote file buffer
21263 ; = 0 if Local device buffer
21264 ; Bit 6 = 1 if buffer dirty
21265 ; Bit 5 = Reserved
21266 ; Bit 4 = Search bit (bit 7 = 1)
21267 ; Bit 3 = 1 if buffer is DATA
21268 ; Bit 2 = 1 if buffer is DIR
21269 ; Bit 1 = 1 if buffer is FAT
21270 ; Bit 0 = Reserved
21271 00000006 ???????? .buf_sector: resd 1 ; Sector number of buffer (flags bit 7 = 0)
21272 ; The next two items are often refed as a word (flags bit 7 = 0)
21273 0000000A ?? .buf_wrtcnt: resb 1 ; For FAT sectors, # times sector written out
21274 0000000B ????.buf_wrtcntinc: resw 1 ; " " " " , # sectors between each write
21275 0000000D ????.resw 1 ; * ; 03/01/2024 ; PCDOS 7.1
21276 ; hw of sectors per FAT
21277 0000000F ???????? .buf_DPB: resd 1 ; Pointer to drive parameters
21278 00000013 ????.buf_fill: resw 1 ; How full buffer is (flags bit 7 = 1)
21279 00000015 ?? .buf_reserved: resb 1 ; make DWORD boundary for 386
21280 00000016 ????.resw 1 ; * ; 03/01/2024 ; PCDOS 7.1
21281 ; reserved word for dword boundary
21282 .size: ; 20 bytes ; MSDOS 5.0 to 6.22
21283 ; 24 bytes ; PCDOS 7.1 ; 03/01/2024
21284 endstruc
21285
21286 %define buf_offset BUFFINFO.buf_sector ; 22/07/2019
21287 ; For buf_flags bit 7 = 1, this is the byte
21288 ; offset of the start of the buffer in
21289 ; the file pointed to by buf_ID. Thus
21290 ; the buffer starts at location
21291 ; buf_offset in the file and contains
21292 ; buf_fill bytes.
21293
21294 bufinsiz equ BUFFINFO.size ; Size of structure in bytes
21295
21296
21297 buf_Free equ 0FFh ; buf_id of free buffer
21298
21299 ; Flag byte masks
21300 buf_isnet EQU 10000000B
21301 buf_dirty EQU 01000000B
21302 ;***
21303 buf_visit EQU 00100000B
21304 ;***
21305 buf_snbuf EQU 00010000B
21306
21307 buf_isDATA EQU 00001000B
21308 buf_isDIR EQU 00000100B
21309 buf_isFAT EQU 00000010B
21310 buf_type_0 EQU 11110001B ; AND sets type to "none"
21311
21312 buf_NetID EQU bufinsiz
21313
21314 ; -----
21315
21316 ; -----
21317 ;** DPB - Drive Parameter Block
21318
21319 ; 25/03/2019 - Retro DOS v4.0 (MSDOS 6.0, DPB.INC, 1991)
21320
21321 ; BUGBUG - this isn't authoritative - it's my probably incomplete and

```



```

21322 ; possibly inaccurate deductions from code study... - jgl
21323 ;
21324 ; The DPB is DOS's main structure for describing block devices.
21325 ; It contains info about the "Drive" intermingled with info about
21326 ; the FAT file system which is presumably on the drive. I don't know
21327 ; how those fields are used if it's not the FAT file system - BUGBUG
21328 ;
21329 ; The DPBs are statically allocated and chained off of DPBHead.
21330 ; Users scan this chain looking for a match on DPB_DRIVE.
21331 ; The DPBs are built at init time from info in the SYSDEV structure.
21332
21333 ; 08/06/2018 - Retro DOS v3.0 (MSDOS 3.3, DPB.INC, 24/07/1987)
21334
21335 ; 12/05/2019 - Retro DOS v4.0
21336
21337 ; 01/01/2024
21338 %if 0
21339
21340 struct DPB
21341 .DRIVE: resb 1 ; Logical drive # assoc with DPB (A=0,B=1,...)
21342 .UNIT: resb 1 ; Driver unit number of DPB
21343 .SECTOR_SIZE: resw 1 ; Size of physical sector in bytes
21344 .CLUSTER_MASK: resb 1 ; Sectors/cluster - 1
21345 .CLUSTER_SHIFT: resb 1 ; Log2 of sectors/cluster
21346 .FIRST_FAT: resw 1 ; Starting record of FATs
21347 .FAT_COUNT: resb 1 ; Number of FATs for this drive
21348 .ROOT_ENTRIES: resw 1 ; Number of directory entries
21349 .FIRST_SECTOR: resw 1 ; First sector of first cluster
21350 .MAX_CLUSTER: resw 1 ; Number of clusters on drive + 1
21351 ; MSDOS 3.3
21352 ; .FAT_SIZE: resb 1 ; Number of records occupied by FAT
21353 ; MSDOS 6.0
21354 .FAT_SIZE: resw 1 ; Number of records occupied by FAT
21355 .DIR_SECTOR: resw 1 ; Starting record of directory
21356 .DRIVER_ADDR: resd 1 ; Pointer to driver
21357 .MEDIA: resb 1 ; Media byte
21358 .FIRST_ACCESS: resb 1 ; This is initialized to -1 to force a media
21359 ; check the first time this DPB is used
21360 .NEXT_DPB: resd 1 ; Pointer to next Drive parameter block
21361 .NEXT_FREE: resw 1 ; Cluster # of last allocated cluster
21362 .FREE_CNT: resw 1 ; Count of free clusters, -1 if unknown
21363 .size:
21364 endstruc
21365
21366 %else
21367
21368 ; 01/01/2024 - Retro DOS v5.0 (PCDOS 7.1)
21369
21370 struct DPB
21371 .DRIVE: resb 1 ; 0 ; Logical drive # assoc with DPB (A=0,B=1,...)
21372 .UNIT: resb 1 ; 1 ; Driver unit number of DPB
21373 .SECTOR_SIZE: resw 1 ; 2 ; Size of physical sector in bytes
21374 .CLUSTER_MASK: resb 1 ; 4 ; Sectors/cluster - 1
21375 .CLUSTER_SHIFT: resb 1 ; 5 ; Log2 of sectors/cluster
21376 .FIRST_FAT: resw 1 ; 6 ; Starting record of FATs
21377 .FAT_COUNT: resb 1 ; 8 ; Number of FATs for this drive
21378 .ROOT_ENTRIES: resw 1 ; 9 ; Number of directory entries
21379 .FIRST_SECTOR: resw 1 ; 11 ; First sector of first cluster
21380 .MAX_CLUSTER: resw 1 ; 13 ; Number of clusters on drive + 1
21381 .FAT_SIZE: resw 1 ; 15 ; Number of records occupied by FAT
21382 .DIR_SECTOR: resw 1 ; 17 ; Starting record of directory
21383 .DRIVER_ADDR: resd 1 ; 19 ; Pointer to driver
21384 .MEDIA: resb 1 ; 23 ; Media byte
21385 .FIRST_ACCESS: resb 1 ; 24 ; This is initialized to -1 to force a media
21386 ; check the first time this DPB is used
21387 .NEXT_DPB: resd 1 ; 25 ; Pointer to next Drive parameter block
21388 .NEXT_FREE: resw 1 ; 29 ; Cluster # of last allocated cluster
21389 .FREE_CNT: resw 1 ; 31 ; Count of free clusters, -1 if unknown
21390 ; FAT32 fs ; 01/01/2024
21391 ; ref: https://en.wikibooks.org/wiki/
21392 ; First_steps_towards_system_programming_under_MS-DOS_7/Appendix
21393 ; -- A.03-1. Structure of Drive Parameters Blocks (DPB) ---
21394 .FREE_CNT_HW: resw 1 ; 33 ; High word of free cluster count
21395 .EXT_FLAGS: resw 1 ; 35 ; FAT32 extended flags (active FAT number)
21396 .FSINFO_SECTOR: resw 1 ; 37 ; (FAT32 fs) FSINFO structure sector address
21397 .BKBOOT_SECTOR: resw 1 ; 39 ; (FAT32 fs) Backup Boot Sector address
21398 .FCLUS_FSECTOR: resd 1 ; 41 ; The first cluster's first sector address
21399 .LAST_CLUSTER: resd 1 ; 45 ; The last cluster number
21400 .FAT32_SIZE: resd 1 ; 49 ; Number of FAT sectors (for FAT32 fs)
21401 .ROOT_CLUSTER: resd 1 ; 53 ; Root directory's cluster number (FAT32 fs)
21402 ; 01/01/2024 - Retro DOS v5.0
21403 .FAT32_NXTFREE: resd 1 ; 57 ; The next free cluster (for FAT32 fs)
21404 .size: ; 61 bytes ; 01/01/2024 (PCDOS 7.1)
21405 endstruc
21406
21407 %endif
21408
21409 DPBSIZ EQU DPB.size ; Size of the structure in bytes
21410
21411 DSKSIZ EQU DPB.MAX_CLUSTER ; Size of disk (temp used during init only)
21412
21413 ; -----
21414 ; 26/03/2018
21415
21416 ; IOCTL SUB-FUNCTIONS
21417 IOCTL_GET_DEVICE_INFO EQU 0
21418 IOCTL_SET_DEVICE_INFO EQU 1
21419 IOCTL_READ_HANDLE EQU 2
21420 IOCTL_WRITE_HANDLE EQU 3
21421 IOCTL_READ_DRIVE EQU 4
21422 IOCTL_WRITE_DRIVE EQU 5
21423 IOCTL_GET_INPUT_STATUS EQU 6
21424 IOCTL_GET_OUTPUT_STATUS EQU 7
21425 IOCTL_CHANGEABLE? EQU 8
21426 IOCTL_SHARING_RETRY EQU 11
21427 GENERIC_IOCTL_HANDLE EQU 12
21428 GENERIC_IOCTL EQU 13
21429
21430 ; GENERIC IOCTL SUB-FUNCTIONS
21431 RAWIO EQU 8
21432
21433 ; RAWIO SUB-FUNCTIONS
21434 GET_DEVICE_PARAMETERS EQU 60H
21435 SET_DEVICE_PARAMETERS EQU 40H
21436 READ_TRACK EQU 61H
21437 WRITE_TRACK EQU 41H
21438 VERIFY_TRACK EQU 62H
21439 FORMAT_TRACK EQU 42H
21440
21441 ; DEVICETYPE VALUES
21442 MAX_SECTORS_IN_TRACK EQU 63
21443 DEV_5INCH EQU 0
21444 DEV_5INCH96TPI EQU 1
21445 DEV_3INCH720KB EQU 2

```

```

21446 DEV_8INCHSS EQU 3
21447 DEV_8INCHDS EQU 4
21448 DEV_HARDDISK EQU 5
21449 DEV_OTHER EQU 7
21450 ;DEV_3INCH1440KB EQU 7
21451 DEV_3INCH2880KB EQU 9
21452 ; Retro DOS v2.0 - 26/03/2018
21453 ;;DEV_TAPE EQU 6
21454 ;;DEV_ERIMO EQU 8
21455 ;DEV_3INCH2880KB EQU 9
21456 DEV_3INCH1440KB EQU 10
21457
21458 ;MAX_DEV_TYPE EQU 9 ; MAXIMUM DEVICE TYPE THAT WE
21459 ; CURRENTLY SUPPORT.
21460 MAX_DEV_TYPE EQU 10
21461
21462 struc A_SECTORTABLE
21463 00000000 ???? .ST_SECTORNUMBER: resw 1
21464 00000002 ???? .ST_SECTORSIZE: resw 1
21465 .size:
21466 endstruc
21467
21468 ; 25/03/2019 - Retro DOS v4.0 (MSDOS 6.0, BPB.INC, IOCTL.INC)
21469
21470 ;** BIOS PARAMETER BLOCK DEFINITION
21471 ;
21472 ; The BPB contains information about the disk structure. It dates
21473 ; back to the earliest FAT systems and so FAT information is
21474 ; intermingled with physical driver information.
21475 ;
21476 ; A boot sector contains a BPB for its device; for other disks
21477 ; the driver creates a BPB. DOS keeps copies of some of this
21478 ; information in the DPB.
21479 ;
21480 ; The BDS structure contains a BPB within it.
21481
21482 ; 01/01/2024
21483 %if 0
21484
21485 struc A_BPB
21486 .BPB_BYTESPERSECTOR: resw 1
21487 .BPB_SECTORSERCLUSTER: resb 1
21488 .BPB_RESERVEDSECTORS: resw 1
21489 .BPB_NUMBEROFFATS: resb 1
21490 .BPB_ROOTENTRIES: resw 1
21491 .BPB_TOTALSECTORS: resw 1
21492 .BPB_MEDIADSCRIPTOR: resb 1
21493 .BPB_SECTORSERFAT: resw 1
21494 .BPB_SECTORSERPTRACK: resw 1
21495 .BPB_HEADS: resw 1
21496 .BPB_HIDDENSECTORS: resw 1
21497 .resw 1
21498 .BPB_BIGTOTALSECTORS: resw 1
21499 .resw 1
21500 .resb 6 ; NOTE: many times these
21501 ; ; 6 bytes are omitted
21502 ; ; when BPB manipulations
21503 ; ; are performed!
21504 .size:
21505 endstruc
21506
21507 %else
21508
21509 ; 14/04/2024
21510 ; 01/01/2024 - Retro DOS v5.0
21511
21512 struc A_BPB
21513 00000000 ???? .BYTESPERSECTOR: resw 1
21514 00000002 ?? .SECTORSERCLUSTER: resb 1
21515 00000003 ???? .RESERVEDSECTORS: resw 1
21516 00000005 ?? .NUMBEROFFATS: resb 1
21517 00000006 ???? .ROOTENTRIES: resw 1
21518 00000008 ???? .TOTALSECTORS: resw 1
21519 0000000A ?? .MEDIADSCRIPTOR: resb 1
21520 0000000B ???? .SECTORSERFAT: resw 1
21521 0000000D ???? .SECTORSERPTRACK: resw 1
21522 0000000F ???? .HEADS: resw 1
21523 00000011 ???? .HIDDENSECTORS: resd 1
21524 00000015 ???? .BIGTOTALSECTORS: resd 1
21525 ;..... FAT32 ..... + 28
21526 00000019 ???? .FATSIZE32: resd 1
21527 0000001D ???? .EXTFLGAS: resw 1
21528 0000001F ???? .FSVER: resw 1
21529 00000021 ???? .ROOTDIRCLUSTER: resd 1
21530 00000025 ???? .FSINFOSECTOR: resw 1 ; (offset from FAT32 bs)
21531 00000027 ???? .BACKUPBOOTSECTOR: resw 1 ; (offset from FAT32 bs)
21532 00000029 <res Ch> .RESERVEDBYTES: resb 12 ; (zero bytes)
21533 ; 14/04/2024
21534 00000035 ???? .resb 6 ; A_BPB.size must be 59
21535 .size:
21536 endstruc
21537
21538 %endif
21539
21540 struc A_DEVICEPARAMETERS
21541 00000000 ?? .DP_SPECIALFUNCTIONS: resb 1
21542 00000001 ?? .DP_DEVICE TYPE: resb 1
21543 00000002 ???? .DP_DEVICEATTRIBUTES: resw 1
21544 00000004 ???? .DP_CYLINDERS: resw 1
21545 00000006 ?? .DP_MEDIATYPE: resb 1
21546 00000007 <res 3Bh> .DP_BPB: resb A_BPB.size
21547 00000042 ???? .DP_TRACKTABLEENTRIES: resw 1
21548 00000044 <res FCh> .DP_SECTORTABLE: resb MAX_SECTORS_IN_TRACK * A_SECTORTABLE.size
21549 endstruc
21550
21551 ; -----
21552 ; structure, equates for devmark for mem command.
21553
21554 ; 25/03/2019 - Retro DOS v4.0 (MSDOS 6.0, DEVMARK.INC, 1991)
21555
21556 struc devmark
21557 00000000 ?? .id: resb 1
21558 00000001 ???? .seg: resw 1
21559 00000003 ???? .size: resw 1
21560 00000005 ???? .dum: resb 3
21561 00000008 ???? .filename: resb 8
21562 endstruc
21563
21564 devmark_stk equ 'S'
21565 devmark_device equ 'D'
21566 devmark_ifs equ 'I'
21567 devmark_buf equ 'B'
21568 devmark_cds equ 'L' ; lastdrive
21569 devmark_files equ 'F'

```



```

21570 devmark_fcbs equ 'X'
21571 devmark_inst equ 'T' ; used for sysinit base for install= command.
21572 devmark_ems_stub equ 'E'
21573
21574 setbrkdone equ 00000001b
21575 for_devmark equ 00000010b
21576 not_for_devmark equ 11111101b
21577
21578 ; -----
21579 ; Memory arena structure
21580
21581 ; 24/03/2019 - Retro DOS v4.0
21582 ; (MSDOS 6.0, ARENA.INC)
21583
21584 ;** Arena Header
21585
21586 struc ARENA
21587 00000000 ?? .SIGNATURE: resb 1 ; 4D for valid item, 5A for last item
21588 00000001 ????.OWNER: resw 1 ; owner of arena item
21589 00000003 ????.SIZE: resw 1 ; size in paragraphs of item
21590 00000005 ???????.RESERVED resb 3 ; reserved
21591 00000008 ???????????????.NAME: resb 8 ; owner file name
21592 endstruc
21593
21594 ; 12/04/2019
21595
21596 arena_owner_system EQU 0 ; free block indication
21597
21598 arena_signature_normal EQU 4dh ; valid signature, not end of arena
21599 arena_signature_end EQU 5Ah ; valid signature, last block in arena
21600
21601 ; -----
21602 ; Process data block (otherwise known as program header)
21603
21604 ; 23/03/2019 - Retro DOS v4.0
21605 ; (MSDOS 6.0 - PDB.INC, 1991)
21606
21607 FILPERPROC EQU 20
21608
21609 struc PDB ; Process_data_block
21610 00000000 ????.EXIT_CALL: resw 1 ; INT int_abort system terminate
21611 00000002 ????.BLOCK_LEN: resw 1 ; size of execution block
21612 00000004 ?? resb 1
21613 00000005 ???????????.CPM_CALL: resb 5 ; ancient call to system
21614 0000000A ??????????.EXIT: resd 1 ; pointer to exit routine
21615 0000000E ??????????.CTRL_C: resd 1 ; pointer to ^C routine
21616 00000012 ??????????.FATAL_ABORT: resd 1 ; pointer to fatal error
21617 00000016 ????.PARENT_PID: resw 1 ; PID of parent (terminate PID)
21618 00000018 <res 14h> .JFN_TABLE: resb FILPERPROC ; indices into system table
21619 0000002C ????.ENVIRON: resw 1 ; seg addr of environment
21620 0000002E ??????????.USER_STACK: resd 1 ; stack of self during system calls
21621 00000032 ????.JFN_LENGTH: resw 1 ; number of handles allowed
21622 00000034 ??????????.JFN_POINTER: resd 1 ; pointer to JFN table
21623 00000038 ??????????.NEXT_PDB: resd 1 ; pointer to nested PDB's
21624 0000003C ?? .INTERCON: resb 1 ; *** jh-3/28/90 ***
21625 0000003D ?? .APPEND: resb 1 ; *** Not sure if still used ***
21626 0000003E ????.NOVELL_USED: resb 2 ; Novell shell (redir) uses these
21627 00000040 ????.VERSION: resw 1 ; DOS version reported to this app
21628 00000042 <res Eh> .PAD1: resb 14 ;
21629 00000044 <res Eh> .CALL_SYSTEM: resb 5 ; portable method of system call
21630 00000050 ???????????.PAD2: resb 7 ; reserved so FCB 1 can be used as an extended FCB
21631 00000055 ???????????????.FCB1: resb 16 ; default FCB 1
21632 0000005C <res 10h> .FCB2: resb 16 ; default FCB 2
21633 0000006C <res 10h> .PAD3: resb 4 ; not sure if this is used by PDB_FCB2
21634 0000007C ??????????.TAIL: resb 128 ; command tail and default DTA
21635 00000080 <res 80h> ;.size:
21636 endstruc
21637
21638 ; -----
21639 ; <system call definitions>
21640
21641 ; 23/03/2019 - Retro DOS v4.0
21642 ; (MSDOS 6.0 - SYSCALL.INC, 1991)
21643
21644 ABORT EQU 0 ; 0 0
21645 STD_CON_INPUT EQU 1 ; 1 1
21646 STD_CON_OUTPUT EQU 2 ; 2 2
21647 STD_AUX_INPUT EQU 3 ; 3 3
21648 STD_AUX_OUTPUT EQU 4 ; 4 4
21649 STD_PRINTER_OUTPUT EQU 5 ; 5 5
21650 RAW_CON_IO EQU 6 ; 6 6
21651 RAW_CON_INPUT EQU 7 ; 7 7
21652 STD_CON_INPUT_NO_ECHO EQU 8 ; 8 8
21653 STD_CON_STRING_OUTPUT EQU 9 ; 9 9
21654 STD_CON_STRING_INPUT EQU 10 ; 10 A
21655 STD_CON_INPUT_STATUS EQU 11 ; 11 B
21656 STD_CON_INPUT_FLUSH EQU 12 ; 12 C
21657 DISK_RESET EQU 13 ; 13 D
21658 SET_DEFAULT_DRIVE EQU 14 ; 14 E
21659 FCB_OPEN EQU 15 ; 15 F
21660 FCB_CLOSE EQU 16 ; 16 10
21661 DIR_SEARCH_FIRST EQU 17 ; 17 11
21662 DIR_SEARCH_NEXT EQU 18 ; 18 12
21663 FCB_DELETE EQU 19 ; 19 13
21664 FCB_SEQ_READ EQU 20 ; 20 14
21665 FCB_SEQ_WRITE EQU 21 ; 21 15
21666 FCB_CREATE EQU 22 ; 22 16
21667 FCB_RENAME EQU 23 ; 23 17
21668 GET_DEFAULT_DRIVE EQU 25 ; 25 19
21669 SET_DMA EQU 26 ; 26 1A
21670 GET_DEFAULT_DPB EQU 31 ; 31 1F
21671 FCB_RANDOM_READ EQU 33 ; 33 21
21672 FCB_RANDOM_WRITE EQU 34 ; 34 22
21673 GET_FCB_FILE_LENGTH EQU 35 ; 35 23
21674 GET_FCB_POSITION EQU 36 ; 36 24
21675 SET_INTERRUPT_VECTOR EQU 37 ; 37 25
21676 CREATE_PROCESS_DATA_BLOCK EQU 38 ; 38 26
21677 FCB_RANDOM_READ_BLOCK EQU 39 ; 39 27
21678 FCB_RANDOM_WRITE_BLOCK EQU 40 ; 40 28
21679 PARSE_FILE_DESCRIPTOR EQU 41 ; 41 29
21680 GET_DATE EQU 42 ; 42 2A
21681 SET_DATE EQU 43 ; 43 2B
21682 GET_TIME EQU 44 ; 44 2C
21683 SET_TIME EQU 45 ; 45 2D
21684 SET_VERIFY_ON_WRITE EQU 46 ; 46 2E
21685 ; Extended functionality group
21686 GET_DMA EQU 47 ; 47 2F
21687 GET_VERSION EQU 48 ; 48 30
21688 KEEP_PROCESS EQU 49 ; 49 31
21689 GET_DPB EQU 50 ; 50 32
21690 SET_CTRL_C_TRAPPING EQU 51 ; 51 33
21691 GET_INDOS_FLAG EQU 52 ; 52 34

```

```

21694 GET_INTERRUPT_VECTOR EQU 53 ; 53 35
21695 GET_DRIVE_FREESPACE EQU 54 ; 54 36
21696 CHAR_OPER EQU 55 ; 55 37
21697 INTERNATIONAL EQU 56 ; 56 38
21698 ; Directory Group
21699 MKDIR EQU 57 ; 57 39
21700 RMDIR EQU 58 ; 58 3A
21701 CHDIR EQU 59 ; 59 3B
21702 ; File Group
21703 CREAT EQU 60 ; 60 3C
21704 OPEN EQU 61 ; 61 3D
21705 CLOSE EQU 62 ; 62 3E
21706 READ EQU 63 ; 63 3F
21707 WRITE EQU 64 ; 64 40
21708 UNLINK EQU 65 ; 65 41
21709 LSEEK EQU 66 ; 66 42
21710 CHMOD EQU 67 ; 67 43
21711 IOCTL EQU 68 ; 68 44
21712 XDUP EQU 69 ; 69 45
21713 XDUP2 EQU 70 ; 70 46
21714 CURRENT_DIR EQU 71 ; 71 47
21715 ; Memory Group
21716 ALLOC EQU 72 ; 72 48
21717 DEALLOC EQU 73 ; 73 49
21718 SETBLOCK EQU 74 ; 74 4A
21719 ; Process Group
21720 EXEC EQU 75 ; 75 4B
21721 EXIT EQU 76 ; 76 4C
21722 WAITPROCESS EQU 77 ; 77 4D
21723 FIND_FIRST EQU 78 ; 78 4E
21724 ; Special Group
21725 FIND_NEXT EQU 79 ; 79 4F
21726 ; SPECIAL SYSTEM GROUP
21727 SET_CURRENT_PDB EQU 80 ; 80 50
21728 GET_CURRENT_PDB EQU 81 ; 81 51
21729 GET_IN_VARS EQU 82 ; 82 52
21730 SETDPB EQU 83 ; 83 53
21731 GET_VERIFY_ON_WRITE EQU 84 ; 84 54
21732 DUP_PDB EQU 85 ; 85 55
21733 RENAME EQU 86 ; 86 56
21734 FILE_TIMES EQU 87 ; 87 57
21735 ;
21736 ALLOCOPER EQU 88 ; 88 58
21737 ; Network extention system calls
21738 GetExtendedError EQU 89 ; 89 59
21739 CreateTempFile EQU 90 ; 90 5A
21740 CreateNewFile EQU 91 ; 91 5B
21741 LockOper EQU 92 ; 92 5C Lock and Unlock
21742 ServerCall EQU 93 ; 93 5D CommitAll, ServerDOSCall,
21743 ; CloseByName, CloseUser,
21744 ; CloseUserProcess,
21745 ; GetOpenFileList
21746 UserOper EQU 94 ; 94 5E Get and Set
21747 AssignOper EQU 95 ; 95 5F On, Off, Get, Set, Cancel
21748 xNameTrans EQU 96 ; 96 60
21749 PathParse EQU 97 ; 97 61
21750 GetCurrentPSP EQU 98 ; 98 62
21751 Hongeu1 EQU 99 ; 99 63
21752 ECS_CALL EQU 99 ; 99 63 ;; DBCS support
21753 Set_Printer_Flag EQU 100 ; 100 64
21754 GetExtCntry EQU 101 ; 101 65
21755 GetSetCdPg EQU 102 ; 102 66
21756 ExtHandle EQU 103 ; 103 67
21757 Commit EQU 104 ; 104 68
21758 GetSetMediaID EQU 105 ; 105 69
21759 IFS_IOCTL EQU 107 ; 107 6B
21760 ExtOpen EQU 108 ; 108 6C
21761 ;
21762 ;ifdef ROMEXEC
21763 ;ROM_FIND_FIRST EQU 109 ; 109 6D
21764 ;ROM_FIND_NEXT EQU 110 ; 110 6E
21765 ;ROM_EXCLUDE EQU 111 ; 111 6F
21766 ;endif
21767 ;
21768 Set_Oem_Handler EQU 248 ; 248 F8
21769 OEM_C1 EQU 249 ; 249 F9
21770 OEM_C2 EQU 250 ; 250 FA
21771 OEM_C3 EQU 251 ; 251 FB
21772 OEM_C4 EQU 252 ; 252 FC
21773 OEM_C5 EQU 253 ; 253 FD
21774 OEM_C6 EQU 254 ; 254 FE
21775 OEM_C7 EQU 255 ; 255 FF
21776 ;
21777 ; -----
21778 ; SYSCONF.ASM (MSDOS 3.3 - 24/07/1987)
21779 ; -----
21780 ;
21781 ;; IF STACKSW
21782 ;
21783 ;
21784 ; Internal Stack Parameters
21785 ;EntrySize equ 8
21786 ;
21787 ;MinCount equ 8
21788 ;DefaultCount equ 9
21789 ;MaxCount equ 64
21790 ;
21791 ;MinSize equ 32
21792 ;DefaultSize equ 128
21793 ;MaxSize equ 512
21794 ;
21795 ;; ENDIF
21796 ;
21797 ; -----
21798 ; BIOSTRUC.INC (MSDOS 3.3 - 24/07/1987)
21799 ; -----
21800 ;
21801 ; ROM BIOS CALL PACKET STRUCTURES
21802 ;
21803 ;*****
21804 ;System Service call ( Int 15h )
21805 ;*****
21806 ;Function AH = 0C0h, Return system configuration
21807 ;For PC and PCJR on return:
21808 ; (AH) = 80h
21809 ; (CY) = 1
21810 ;For PCXT, PC PORTABLE and PCAT on return:
21811 ; (AH) = 86h
21812 ; (CY) = 1
21813 ;For all others:
21814 ; (AH) = 0
21815 ; (CY) = 0
21816 ; (ES:BX) = pointer to system descriptor vector in ROS
21817 ; System descriptor :

```

```

21818 ; DW      xxxx      length of descriptor in bytes,
21819 ;                      minimum length = 8
21820 ; DB      xx        model byte
21821 ;                      0FFh  = PC
21822 ;                      0FEh  = PC/XT, Portable
21823 ;                      0FDh  = PC/JR
21824 ;                      0FCh  = PC/AT
21825 ;                      0F9h  = Convertable
21826 ;                      0F8h  = Model 80
21827 ;                      0E0 thru 0EFh = reserved
21828 ;
21829 ; DB      xx        secondary model byte
21830 ;                      000h  = PC1
21831 ;                      000h  = PC/XT, Portable
21832 ;                      000h  = PC/JR
21833 ;                      000h  = PC/AT
21834 ;                      001h  = PC/AT Model 339
21835 ;                      003h  = PC/RT
21836 ;                      000h  = Convertable
21837 ;
21838 ; DB      xx        bios revision level
21839 ;                      00 for first release, subsequent release
21840 ;                      of code with same model byte and
21841 ;                      secondary model byte require revision level
21842 ;                      to increase by one.
21843 ;
21844 ; DB      xx        feature information byte 1
21845 ;                      X0000000 = 1, bios use DMA channel 3
21846 ;                      = 0, DMA channel 3 not used
21847 ;
21848 ;                      0X000000 = 1, 2nd Interrupt chip present
21849 ;                      = 0, 2nd Interrupt chip not present
21850 ;
21851 ;                      00X00000 = 1, Real Time Clock present
21852 ;                      = 0, Real Time Clock not present
21853 ;
21854 ;                      000X0000 = 1, Keyboard escape sequence(INT 15h)
21855 ;                      called in keyboard interrupt
21856 ;                      (Int 09h).
21857 ;                      = 0, Keyboard escape sequence not
21858 ;                      called.
21859 ;                      0000XXXX reserved
21860 ;
21861 ; DB      xx        feature information byte 2 - reserved
21862 ;
21863 ; DB      xx        feature information byte 2 - reserved
21864 ;
21865 ; DB      xx        feature information byte 2 - reserved
21866 ;
21867 ; DB      xx        feature information byte 2 - reserved
21868 ;
21869 ;
21870 ; 22/03/2019
21871 struct ROMBIOS_DESC ; BIOS_SYSTEM_DESCRIPTOR
21872 .bios_sd_leng:      resw 1
21873 .bios_sd_modelbyte: resb 1
21874 .bios_sd_scnd_modelbyte: resb 1
21875 .bios_sd_scnd_modelbyte: resb 1
21876 .bios_sd_featurebyte1: resb 1
21877 .bios_sd_featurebyte1: resb 1
21878 .bios_sd_featurebyte1: resb 4
21879 endstruct
21880
21881 ;FeatureByte1      bit map equates
21882 DMAChannel3        equ 10000000b
21883 ScndIntController  equ 01000000b
21884 RealTimeClock      equ 00100000b
21885 KeyEscapeSeq       equ 00010000b
21886 ;;End of Modification
21887
21888 ; -----
21889 ; SYSVAR.INC (MSDOS 6.0 - 1991)
21890 ; -----
21891 ; 22/03/2019 - Retro DOS v4.0
21892
21893 ; SCCSID = @(#)sysvar.asm 1.1 85/04/10
21894
21895 struct SysInitVars
21896 ; MSDOS 3.3
21897 .SYSI_DPB:      resd 1 ; DPB chain
21898 .SYSI_SFT:      resd 1 ; SFT chain
21899 .SYSI_CLOCK:    resd 1 ; CLOCK device
21900 .SYSI_CON:      resd 1 ; CON device
21901 .SYSI_MAXSEC:   resw 1 ; maximum sector size
21902 .SYSI_BUF:      resd 1 ; buffer chain
21903 .SYSI_CDS:      resd 1 ; CDS list
21904 .SYSI_FCB:      resd 1 ; FCB chain
21905 .SYSI_KEEP:     resw 1 ; keep count
21906 .SYSI_NUMIO:   resb 1 ; number of block devices
21907 .SYSI_NCDS:    resb 1 ; number of CDS's
21908 .SYSI_DEV:      resd 1 ; device list
21909 ; MSDOS 6.0
21910 .SYSI_ATTR:     resw 1 ; null device attribute word
21911 .SYSI_STRAT:    resw 1 ; null device strategy entry point
21912 .SYSI_INTER:    resw 1 ; null device interrupt entry point
21913 .SYSI_NAME:     resb 8 ; null device name
21914 .SYSI_SPLICE:   resb 0 ; TRUE -> splices being done
21915 .SYSI_IBMDOS_SIZE: resw 1 ; DOS size in paragraphs
21916 .SYSI_IFS_DOSCALL@: resd 1 ; IFS DOS service routine entry
21917 .SYSI_IFS:      resd 1 ; IFS header chain
21918 .SYSI_BUFFERS:  resw 2 ; BUFFERS= values (m,n)
21919 .SYSI_BOOT_DRIVE: resb 1 ; boot drive A=1 B=2,...
21920 .SYSI_DWMOVE:   resb 1 ; 1 if 386 machine
21921 .SYSI_EXT_MEM:  resw 1 ; Extended memory size in KB.
21922 .size:
21923 endstruct
21924
21925 ;This is added for more information exchange between DOS, BIOS.
21926 ;DOS will give the pointer to SysInitTable in ES:DI. - J.K. 5/29/86
21927
21928 ; 22/03/2019
21929 struct SysInitVars_Ext
21930 .SYSI_InitVars:  resd 1 ; Points to the above structure.
21931 .SYSI_Country_Tab: resd 1 ; DOS_Country_cdpdg_info
21932 endstruct
21933
21934 ; 09/06/2018
21935 ; 08/06/2018 - Retro DOS v3.0 (MSDOS 3.3)
21936 SYSI_DPB      equ 0
21937 SYSI_SFT      equ 4
21938 SYSI_CLOCK    equ 8
21939 SYSI_CON      equ 12
21940 SYSI_MAXSEC   equ 16
21941 SYSI_BUF      equ 18

```

```

21942 SYSI_CDS      equ 22
21943 SYSI_FCB     equ 26
21944 SYSI_KEE     equ 30
21945 SYSI_NUMIO   equ 32
21946 SYSI_NCDS   equ 33
21947 SYSI_DEV     equ 34
21948 ; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0)
21949 SYSI_ATTR     equ 38
21950 SYSI_STRAT    equ 40
21951 SYSI_INTER    equ 42
21952 SYSI_NAME     equ 44
21953 SYSI_SPLICE   equ 52
21954 SYSI_IBMDOS_SIZE equ 53
21955 SYSI_IFS_DOSCALL@ equ 55
21956 SYSI_IFS      equ 59
21957 SYSI_BUFFERS  equ 63
21958 SYSI_BOOT_DRIVE equ 67
21959 SYSI_DWMOVE   equ 68
21960 SYSI_EXT_MEM   equ 69
21961
21962 ;The SYSI_BUF of SysInitVars points to the following structure
21963
21964 EMS_MAP_BUFF_SIZE EQU 12 ; EMS map buffer size
21965
21966 struc BUFFINF ; BUFFINFO
21967 00000000 ???????? .Buff_Queue: resd 1 ; Head of list of buffers
21968 00000004 ????.Dirty_Buff_Count: resw 1 ; number of dirty buffers in list
21969 00000006 ???????? .Cache_ptr: resd 1 ; pointer to secondary cache
21970 0000000A ????.Cache_count: resw 1 ; number of secondary cache entries
21971
21972 0000000C ?? .Buff_In_HMA: resb 1 ; flag to indicate that buffers
21973 ; are in HMA
21974 0000000D ???????? .Lo_Mem_Buff: resd 1 ; Ptr to scratch buff in Low Mem
21975 ; used to read/write on disks
21976 00000011 ???????? .UU_EMS_FIRST_PAGE: resw 2
21977 00000015 ????.UU_EMS_NPA640: resw 1
21978 00000017 ?? .UU_EMS_mode: resb 1 ; no EMS = -1
21979 00000018 ????.UU_EMS_handle: resw 1 ; EMS handle for buffers
21980 0000001A ????.UU_EMS_PageFrame_Number: resw 1 ; EMS page frame number
21981 0000001C ????.UU_EMS_Seg_Cnt: resw 1 ; EMS segment count
21982 0000001E ????.UU_EMS_Page_Frame: resw 1 ; EMS page frame segment address
21983 00000020 ????.UU_EMS_reserved: resw 1 ; EMS segment count
21984 00000022 ?? .UU_EMS_Map_Buff: resb 1 ; map buffer
21985 .size:
21986 endstruc
21987
21988 ; -----
21989 ; CURDIR.INC (MSDOS 6.0 - 1991)
21990 ; -----
21991 ; 22/03/2019 - Retro DOS v4.0
21992
21993 ;** CDS - Current Directory Structure
21994 ;
21995 ; CDS items are used by the internal routines to store cluster numbers and
21996 ; network identifiers for each logical name. The ID field is used dually,
21997 ; both as net ID and for a cluster number for local devices. In the case
21998 ; of local devices, the cluster number will be -1 if there is a potential
21999 ; of the disk being changed or if the path must be rechecked.
22000 ;
22001 ; Some pathnames have special preambles, such as
22002 ;
22003 ; \\machine\sharename\...
22004 ; For these pathnames we can't allow "." processing to back us
22005 ; up into the special front part of the name. The CURDIR_END field
22006 ; holds the address of the separator character which marks
22007 ; the split between the special preamble and the regular
22008 ; path list; "." processing isn't allowed to back us up past
22009 ; (i.e., before) CURDIR_END
22010 ; For the root, it points at the leading /. For net
22011 ; assignments it points at the end (nul) of the initial assignment:
22012 ; A:/ \\foo\bar \\foo\bar\blech\bozo
22013 ; ^ ^ ^
22014
22015 DIRSTRLEN EQU 64+3 ; Max length in bytes of directory strings
22016 TEMPLLEN EQU DIRSTRLEN*2
22017
22018 struc curdir_list
22019 ; MSDOS 3.3
22020 00000000 <res 43h> .cdir_text resb DIRSTRLEN ; text of assignment and curdir
22021 00000043 ????.cdir_flags resw 1 ; various flags
22022 00000045 ???????? .cdir_devptr resd 1 ; local pointer to DPB or net device
22023 00000049 ???????? .cdir_ID resw 2 ; cluster of current dir (net ID)
22024 0000004D ????.cdir_usr_word resw 1
22025 0000004F ????.cdir_end resw 1 ; end of assignment
22026 ; MSDOS 6.0
22027 00000051 ?? .cdir_type: resb 1 ; IFS drive (2=ifs, 4=netuse)
22028 00000052 ???????? .cdir_ifd_hdr: resd 1 ; Ptr to File System Header
22029 00000056 ????.cdir_fsda: resb 2 ; File System Dependent Data Area
22030 .size:
22031 endstruc
22032
22033 curdirlen EQU curdir_list.size ; Needed for screwed up
22034 ; ASM87 which doesn't allow
22035 ; size directive as a macro
22036 ; argument
22037 %define curdir_netID dword [curdir_list.cdir_ID]
22038
22039 ;** Flag values for CURDIR_FLAGS
22040
22041 ;Flag word masks
22042 curdir_isnet EQU 1000000000000000B
22043 curdir_isifs EQU 1000000000000000B
22044 curdir_inuse EQU 0100000000000000B
22045 curdir_splice EQU 0010000000000000B
22046 curdir_local EQU 0001000000000000B
22047
22048 ; -----
22049 ; SF.INC (MSDOS 6.0 - 1991)
22050 ; -----
22051 ; 25/03/2019 - Retro DOS v4.0
22052
22053 ; 09/04/2024 - Retro DOS v4.2 (BugFix)
22054 ; 09/04/2024 - Retro DOS v5.0
22055
22056 ; system file table
22057
22058 ;** System File Table SuperStructure
22059 ;
22060 ; The system file table entries are allocated in contiguous groups.
22061 ; There may be more than one such groups; the SF "superstructure"
22062 ; tracks the groups.
22063
22064 struc SF
22065 00000000 ???????? .SFLink: resd 1

```

```

22066 00000004 ???? .SFCount: resw 1 ; number of entries
22067 00000006 ???? .SFTable: resw 1 ; beginning of array of the following
22068 .size:
22069 endstruc
22070
22071 ;** System file table entry
22072 ;
22073 ; These are the structures which are at SFTABLE in the SF structure.
22074
22075 struc SF_ENTRY
22076 00000000 ???? .sf_ref_count: resw 1 ; number of processes sharing entry
22077 ; if FCB then ref count
22078 00000002 ???? .sf_mode: resw 1 ; mode of access or high bit on if FCB
22079 00000004 ?? .sf_attr: resb 1 ; attribute of file
22080 00000005 ???? .sf_flags: resw 1 ; Bits 8-15
22081 ; Bit 15 = 1 if remote file
22082 ; = 0 if local file or device
22083 ; Bit 14 = 1 if date/time is not to be
22084 ; set from clock at CLOSE. Set by
22085 ; FILETIMES and FCB_CLOSE. Reset by
22086 ; other reseters of the dirty bit
22087 ; (WRITE)
22088 ; Bit 13 = Pipe bit (reserved)
22089 ;
22090 ; Bits 0-7 (old FCB_devid bits)
22091 ; If remote file or local file, bit
22092 ; 6=0 if dirty Device ID number, bits
22093 ; 0-5 if local file.
22094 ; bit 7=0 for local file, bit 7
22095 ; =1 for local I/O device
22096 ; If local I/O device, bit 6=0 if EOF (input)
22097 ;
22098 ; Bit 5=1 if Raw mode
22099 ; Bit 0=1 if console input device
22100 ; Bit 1=1 if console output device
22101 ; Bit 2=1 if null device
22102 00000007 ???????? .sf_devptr: resd 1 ; Points to DPB if local file, points
22103 ; to device header if local device,
22104 ; points to net device header if
22105 ; remote
22106 0000000B ???? .sf_firclus: resw 1 ; First cluster of file (bit 15 = 0)
22107 ;.sf_lstclus: resw 1 ; *
22108 0000000D ???? .sf_time: resw 1 ; Time associated with file
22109 0000000F ???? .sf_date: resw 1 ; Date associated with file
22110 00000011 ???????? .sf_size: resd 1 ; Size associated with file
22111 00000015 ???????? .sf_position: resd 1 ; Read/Write pointer or LRU count for FCBs
22112 ;
22113 ; Starting here, the next 7 bytes may be used by the file system to store an
22114 ; ID
22115 ;
22116 00000019 ???? .sf_cluspos: resw 1 ; Position of last cluster accessed
22117 0000001B ???????? .sf_dirsec: resd 1 ; 09/04/2024 ; sector number of directory sector for this file
22118 0000001F ?? .sf_dirpos: resb 1 ; Offset of this entry in the above
22119 ;
22120 ; End of 7 bytes of file-system specific info.
22121 ;
22122 00000020 <res Bh> .sf_name: resb 11 ; 11 character name that is in the
22123 ; directory entry. This is used by
22124 ; close to detect file deleted and
22125 ; disk changed errors.
22126 ; SHARING INFO
22127 0000002B ???????? .sf_chain: resd 1 ; link to next SF
22128 0000002F ???? .sf_UID: resw 1
22129 00000031 ???? .sf_PID: resw 1
22130 00000033 ???? .sf_MFT: resw 1
22131 00000035 ???? .sf_lstclus: resw 1 ; * ; Last cluster accessed
22132 00000037 ???????? .sf_IFS_HDR: resd 1 ; **
22133 .size:
22134 endstruc
22135
22136 ; -----
22137 ; DOSCNTRY.INC (MSDOS 3.3 - 24/07/1987)
22138 ; -----
22139 ; 11/06/2018 - Retro DOS v3.0
22140
22141 ; Equates for COUNTRY INFORMATION.
22142 SetCountryInfo EQU 1 ; country info
22143 SetUcase EQU 2 ; uppercase table
22144 SetLcase EQU 3 ; lowercase table (Reserved)
22145 SetUcaseFile EQU 4 ; uppercase file spec table
22146 SetFileList EQU 5 ; valid file character list
22147 SetCollate EQU 6 ; collating sequence
22148 SetDBCS EQU 7 ; double byte character set
22149 SetALL EQU -1 ; all the entries
22150
22151 ; DOS country and code page information table structure.
22152 ; Internally, IBMDOS gives a pointer to this table.
22153 ; IBMBIO, MODE and NLSFUNC modules communicate with IBMDOS through
22154 ; this structure.
22155
22156 struc country_cdpkg_info ; DOS_country_cdpkg_info
22157 00000000 ?????????????? .ccInfo_reserved: resb 8 ; reserved for internal use
22158 00000008 <res 40h> .ccPath_CountrySys: resb 64 ; path and filename for country info
22159 00000048 ???? .ccSysCodePage: resw 1 ; system code page id
22160 0000004A ???? .ccNumber_of_entries: resw 1 ; dw 5
22161 0000004C ?? .ccSetUcase: resb 1 ; db SetUcase ; = 2
22162 0000004D ???????? .ccUcase_ptr: resd 1 ; pointer to Ucase table
22163
22164 00000051 ?? .ccSetUcaseFile: resb 1 ; db SetUcaseFile ; = 4
22165 00000052 ???????? .ccFileUcase_ptr: resd 1 ; pointer to File Ucase table
22166
22167 00000056 ?? .ccSetFileList: resb 1 ; db SetFileList ; = 5
22168 00000057 ???????? .ccFilechar_ptr: resd 1 ; pointer to File char list table
22169
22170 0000005B ?? .ccSetCollate: resb 1 ; db SetCollate ; = 6
22171 0000005C ???????? .ccCollate_ptr: resd 1 ; pointer to collate table
22172
22173 00000060 ?? .ccSetCountryInfo: resb 1 ; db SetCountryInfo ; = 1
22174 00000061 ???? .ccCountryInfolen: resw 1 ; length of country info
22175 00000063 ???? .ccDosCountry: resw 1 ; system country code id
22176 00000065 ???? .ccDosCodePage: resw 1 ; system code page id
22177 00000067 ???? .ccDFormat: resw 1 ; date format
22178 00000069 ???????? .ccCurSymbol: resb 5 ; db " ",0
22179 ; 5 byte of (currency symbol+0)
22180 0000006E ???? .cc1000Sep: resb 2 ; db " ",0 ; 2 byte of (1000 sep. + 0)
22181 00000070 ???? .ccDecSep: resb 2 ; db " ",0 ; 2 byte of (Decimal sep. + 0)
22182 00000072 ???? .ccDateSep: resb 2 ; db " ",0 ; 2 byte of (date sep. + 0)
22183 00000074 ???? .ccTimeSep: resb 2 ; db " ",0 ; 2 byte of (time sep. + 0)
22184 00000076 ?? .ccCFormat: resb 1 ; currency format flags
22185 00000077 ?? .ccCSigDigits: resb 1 ; # of digits in currency
22186 00000078 ?? .ccTFormat: resb 1 ; time format
22187 00000079 ???????? .ccMono_Ptr: resd 1 ; monospace routine entry point
22188 0000007D ???? .ccListSep: resb 2 ; db " ",0 ; data list separator
22189 0000007F <res Ah> .ccReserved_area: resw 5 ; dw 5 dup(?) ; reserved

```

```

22190 .size:
22191 endstruc
22192
22193 NEW_COUNTRY_SIZE equ country_cdpd_info.size - country_cdpd_info.ccDosCountry
22194
22195 ; =====
22196 ; retrodos4.s (offset addresses in MSDOS.SYS or RETRODOS.SYS)
22197 ; =====
22198 ; 21/03/2019 - Retro DOS v4.0
22199 ; 21/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
22200
22201 ;KERNEL_SEGMENT equ 0070h ; (IO.SYS loading segment, BIOS_DATA segment)
22202 ; 21/10/2022
22203 DOSBIODATASEG equ 0070h ; (IO.SYS loading segment, BIOS_DATA segment)
22204 ; 22/10/2022
22205 ;DOSBIOCODESEG equ 02C7h ; (MSDOS 5.0 IO.SYS, BIOS_CODE segment)
22206 ; 09/12/2022
22207 DOSBIOCODESEG equ IOSYSCODESEG
22208
22209 ; Note: These offset addresses must be changed when the code
22210 ; in retrodos4.s (MSDOS.SYS) file will be changed.
22211
22212 ; (following addresses can be verified by searching them in retrodos4.lst)
22213
22214 ; 09/12/2022
22215 %if 0
22216
22217 ; 13/05/2019
22218
22219 ;Iswin386 equ 08CFh
22220 ;V86_Crit_SetFocus equ 08D0h
22221 ; 21/10/2022
22222 Iswin386 equ 08D0h
22223 V86_Crit_SetFocus equ 08D1h
22224
22225 ;seg_reinit equ 0772h ; not used in Retro DOS v4.0
22226 ; 21/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
22227 seg_reinit equ 0032h ; DOSBIOCODESEG:0032h
22228
22229 ;SysinitPresent equ 08FCh
22230 ; 21/10/2022
22231 SysinitPresent equ 08FDh
22232
22233 inHMA equ 000Dh
22234 xms equ 000Eh
22235 ;FreeHMAPtr equ 08F6h
22236 ;multrk_flag equ 0533h
22237 ;ec35_flag equ 0535h
22238 ;EOT equ 012Eh
22239 ; 21/10/2022
22240 FreeHMAPtr equ 08F7h
22241 multrk_flag equ 052Fh
22242 ec35_flag equ 0531h
22243 EOT equ 012Ch
22244
22245 ;NextStack equ 08BFh
22246 ;IT_StackLoc equ 08C5h
22247 ;IT_StackSize equ 08C9h
22248 ; 21/10/2022
22249 NextStack equ 08C0h
22250 IT_StackLoc equ 08C6h
22251 IT_StackSize equ 08CAh
22252
22253 ;MoveDOSIntoHMA equ 08F8h
22254 ; 21/10/2022
22255 MoveDOSIntoHMA equ 08F9h
22256
22257 ;INT19SEM equ 0644h ; 01/05/2019 - retrodos4.lst
22258 ;I19_LST equ 0645h ; 27/03/2019 - retrodos4.lst
22259 ; 21/10/2022
22260 INT19SEM equ 0640h ; (iosys5.txt)
22261 I19_LST equ 0641h ; (iosys5.txt)
22262
22263 %endif
22264
22265 ; 09/12/2022
22266 seg_reinit equ _seg_reinit
22267 ec35_flag equ ec35flag
22268 INT19SEM equ int19sem
22269 I19_LST equ i19_lst
22270
22271 INT19OLD02 equ I19_LST+1 ; 0642h ; 21/10/2022
22272 INT19OLD08 equ I19_LST+6
22273 INT19OLD09 equ I19_LST+11
22274 INT19OLD0A equ I19_LST+16
22275 INT19OLD0B equ I19_LST+21
22276 INT19OLD0C equ I19_LST+26
22277 INT19OLD0D equ I19_LST+31
22278 INT19OLD0E equ I19_LST+36
22279 INT19OLD70 equ I19_LST+41
22280 INT19OLD72 equ I19_LST+46
22281 INT19OLD73 equ I19_LST+51
22282 INT19OLD74 equ I19_LST+56
22283 INT19OLD76 equ I19_LST+61
22284 INT19OLD77 equ I19_LST+66 ; 0683h ; 21/10/2022
22285
22286 ; 09/12/2022
22287 %if 0
22288
22289 ;keyrd_func equ 04E9h
22290 ;keysts_func equ 04EAh
22291 ;t_switch equ 04F6h
22292 ; 21/10/2022
22293 keyrd_func equ 04E5h
22294 keysts_func equ 04E6h
22295 t_switch equ 04F2h
22296
22297 ; 22/10/2022
22298 SYSINITSEG equ 046Dh ; SYSINIT segment
22299 BCODE_END equ (SYSINITSEG-DOSBIOCODESEG)*16 ; = 1A60h
22300 BCODE_START equ 30h ; (offset BiosDataWord in DOSBIOCODESEG)
22301 RE_INIT equ 089Bh ; (re_init offset in DOSBIODATASEG)
22302
22303 %endif
22304
22305 ; 09/12/2022
22306 BCODESTART equ BIOSDATAWORD
22307 RE_INIT equ re_init
22308
22309 ; -----
22310 ; CONFIG.INC (MSDOS 6.0 - 1991)
22311 ; -----
22312 ; 15/04/2019 - Retro DOS v4.0
22313

```



```

22314 CONFIG_BEGIN equ '['
22315 CONFIG_BREAK equ 'C'
22316 CONFIG_BUFFERS equ 'B'
22317 CONFIG_COMMENT equ 'Y'
22318 CONFIG_COUNTRY equ 'Q'
22319 CONFIG_DEVICE equ 'D'
22320 CONFIG_DEVICEHIGH equ 'U'
22321 CONFIG_DOS equ 'H'
22322 CONFIG_DRIVPARM equ 'P'
22323 CONFIG_FCBS equ 'X'
22324 CONFIG_FILES equ 'F'
22325 CONFIG_INCLUDE equ 'J'
22326 CONFIG_INSTALL equ 'I'
22327 CONFIG_INSTALLHIGH equ 'W'
22328 CONFIG_LASTDRIVE equ 'L'
22329 CONFIG_MENUCOLOR equ 'R'
22330 CONFIG_MENUEFAULT equ 'A'
22331 CONFIG_MENUITEM equ 'E'
22332 CONFIG_MULTITRACK equ 'M'
22333 CONFIG_NUMLOCK equ 'N'
22334 CONFIG_REM equ 'O'
22335 CONFIG_SEMICOLON equ ';'
22336 CONFIG_SET equ 'V'
22337 CONFIG_SHELL equ 'S'
22338 CONFIG_STACKS equ 'K'
22339 CONFIG_SUBMENU equ 'O'
22340 CONFIG_SWITCHES equ 'I'
22341
22342 CONFIG_UNKNOWN equ 'Z'
22343
22344 ; 13/05/2024 - Retro DOS v5.0 (PCDOS 71 IBMBIO.COM)
22345 CONFIG_DOSDATA equ 'T'
22346
22347 CONFIG_OPTION_QUERY equ 80h
22348
22349 ; -----
22350 ; SYSINIT1.ASM (MSDOS 6.0 - 1991)
22351 ; -----
22352 ; 21/03/2019 - Retro DOS v4.0
22353
22354 true equ 0FFFFh
22355 false equ 0
22356 cr equ 13
22357 lf equ 10
22358 tab equ 9
22359
22360 multMULT equ 4Ah
22361 multMULTGETHMAPTR equ 1
22362 multMULTALLOCHMA equ 2
22363
22364 ;NOEXEC equ FALSE
22365
22366 stacksw equ true ;include switchable hardware stacks
22367 mycds_size equ 88 ;size of curdir_list. if it is not
22368 ;the same, then will generate compile error.
22369
22370 entrysize equ 8
22371
22372 mincount equ 8
22373 defaultcount equ 9
22374 maxcount equ 64
22375
22376 minsize equ 32
22377 defaultsize equ 128
22378 maxsize equ 512
22379
22380 ;%define allocbyte byte [es:bp+0]
22381 ;%define intlevel byte [es:bp+1]
22382 ;%define savedsp word [es:bp+2]
22383 ;%define savedss word [es:bp+4]
22384 ;%define newsp word [es:bp+6]
22385
22386 allocbyte equ 0
22387 intlevel equ 1
22388 savedsp equ 2
22389 savedss equ 4
22390 newsp equ 6
22391
22392 free equ 0
22393 allocated equ 1
22394 overflowed equ 2
22395 clobbered equ 3
22396
22397 ;-----
22398 ; external variable defined in ibmbio module for multi-track
22399
22400 multrk_on equ 10000000b ;user specified mutitrack=on,or system turns
22401 ; it on after handling config.sys file as a
22402 ; default value,if multrk_flag = multrk_off1.
22403 multrk_off1 equ 00000000b ;initial value. no "multitrack=" command entered.
22404 multrk_off2 equ 00000001b ;user specified multitrack=off.
22405
22406 ; SYSINITSEG SEGMENT PUBLIC 'SYSTEM_INIT'
22407
22408 SYSINIT$:
22409 ;IF STACKSW
22410 ; include MSSTACK.INC ;Main stack program and data definitions
22411 ; include STKMES.INC ;Fatal stack error message
22412 ; public Endstackcode
22413 ;Endstackcode label byte
22414 ;ENDIF
22415
22416 ; 05/07/2018
22417 ; -----
22418 ; 04/06/2018 - Retro DOS v3.0
22419
22420 ; -----
22421 ; 21/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS - SYSINIT)
22422 ; -----
22423
22424 ; MSStack.inc
22425 ;
22426 ; Interrupt level 2, 3, 4, 5, 6, 7,(10, 11, 12, 14, 15 - AT level)
22427 ; should follow the standard Interrupt Sharing Scheme which has
22428 ; a standard header structure.
22429 ; Fyi, the following shows the relations between
22430 ; the interrupt vector and interrupt level.
22431 ; VEC(Hex) 2 8 9 A B C D E 70 72 73 74 76 77
22432 ; LVL(Deci) 9 0 1 2 3 4 5 6 8 10 11 12 14 15
22433 ; MSSTACK module modifies the following interrupt vectors
22434 ; to meet the standard Interrupt Sharing standard;
22435 ; A, B, C, D, E, 72, 73, 74, 76, 77.
22436 ; Also, for interrupt level 7 and 15, the FirstFlag in a standard header
22437 ; should be initialized to indicat whether this interrupt handler is

```

```

22438 ; the first (= 80h) or not. The FirstFlag entry of INT77h's
22439 ; program header is initialized in STKINIT.INC module.
22440 ; FirstFlag is only meaningful for interrupt level 7 and 15.
22441 ;
22442 ;
22443 ; User specifies the number of stack elements - default = 9
22444 ; minimum = 8
22445 ; maximum = 64
22446 ;
22447 ; Intercepts Asynchronous Hardware Interrupts only
22448 ;
22449 ; Picks a stack from pool of stacks and switches to it
22450 ;
22451 ; Calls the previously saved interrupt vector after pushing flags
22452 ;
22453 ; on return, returns the stack to the stack pool
22454 ;
22455 ;
22456 ; This is a modification of STACKS:
22457 ; 1. To fix a bug which was causing the program to take up too much space.
22458 ; 2. To dispense stack space from hi-mem first rather than low-mem first.
22459 ; . Clobbers the stack that got too big instead of innocent stack
22460 ; . Allows system to work if the only stack that got too big was the most
22461 ; . deeply nested one
22462 ; 3. Disables NMI interrupts while setting the NMI vector.
22463 ; 4. Does not intercept any interrupts on a PCjr.
22464 ; 5. Double checks that a nested interrupt didn't get the same stack.
22465 ; 6. Intercepts Ints 70, 72-77 for PC-ATs and other future products
22466 ;
22467 ;EVEN
22468 ;align 2
22469 ; 21/10/2022
22470 ;
22471 00000000 0000 dw 0 ; spare field but leave these in order
22472 00000002 0000 stackcount: dw 0
22473 00000004 0000 stackat: dw 0
22474 00000006 0000 stacksize: dw 0
22475 00000008 0000 stacks: dw 0 0
22476 0000000A 0000 dw 0
22477 ;
22478 0000000C [0800] firstentry: dw stacks
22479 0000000E [4800] lastentry: dw stacks+(defaultcount*entrysize)-entrysize
22480 00000010 [4800] nextentry: dw stacks+(defaultcount*entrysize)-entrysize
22481 ;
22482 ;*****
22483 ; THESE ARE THE INDIVIDUAL INTERRUPT HANDLERS
22484 ;
22485 ; -----
22486 ;
22487 00000012 00000000 old02: dd 0
22488 ;
22489 int02:
22490 ;
22491 ; *****
22492 ;
22493 ; this is special support for the pc convertible / nmi handler
22494 ;
22495 ; on the pc convertible, there is a situation where an nmi can be
22496 ; caused by using the "out" instructions to certain ports. when this
22497 ; occurs, the pc convertible hardware *guarantees* that *nothing*
22498 ; can stop the nmi or interfere with getting to the nmi handler. this
22499 ; includes other type of interrupts (hardware and software), and
22500 ; also includes other type of nmi's. when any nmi has occurred,
22501 ; no other interrupt (hardware, software or nmi) can occur until
22502 ; the software takes specific steps to allow further interrupting.
22503 ;
22504 ; for pc convertible, the situation where the nmi is generated by the
22505 ; "out" to a control port requires "fixing-up" and re-attempting. in
22506 ; otherwords, it is actually a "restartable exception". in this
22507 ; case, the software handler must be able to get to the stack in
22508 ; order to figure out what instruction caused the problem, where
22509 ; it was "out"ing to and what value it was "out"ing. therefore,
22510 ; we will not switch stacks in this situation. this situation is
22511 ; detected by interrogating port 62h, and checking for a bit value
22512 ; of 80h. if set, *****do not switch stacks*****.
22513 ;
22514 ; *****
22515 ;
22516 00000016 50 push ax
22517 00000017 06 push es
22518 00000018 B800F0 mov ax,0F000h
22519 0000001B 8EC0 mov es,ax
22520 ; 02/11/2022
22521 0000001D 26803EFEFF9 cmp byte [es:0FFFEh],0F9h ; mdl_convert ; check if convertible
22522 00000023 07 pop es
22523 00000024 750C jne short normal02
22524 ;
22525 00000026 E462 in al,62h ; PC/XT PPI port C. Bits:
22526 ; 0-3: values of DIP switches
22527 ; 5: 1=Timer 2 channel out
22528 ; 6: 1=I/O channel check
22529 ; 7: 1=RAM parity check error occurred.
22530 00000028 A880 test al,80h
22531 0000002A 7406 jz short normal02
22532 ;
22533 0000002C 58 special02: pop ax
22534 0000002D 2EFF2E[1200] jmp far [cs:old02]
22535 ;
22536 00000032 58 normal02: pop ax
22537 00000033 E81101 call do_int_stacks
22538 00000036 [1200] dw old02
22539 ;
22540 ; -----
22541 ;
22542 00000038 00000000 old08: dd 0
22543 ;
22544 int08:
22545 0000003C E80801 call do_int_stacks
22546 0000003F [3800] dw old08
22547 ;
22548 ; -----
22549 ;
22550 00000041 00000000 old09: dd 0
22551 ;
22552 int09:
22553 ;
22554 ; keyboard interrupt must have a three byte jump, a nop and a zero byte
22555 ; as its first instruction for compatibility reasons
22556 ;
22557 00000045 EB02 jmp short keyboard_1b1
22558 00000047 90 nop
22559 00000048 00 db 0
22560 ;
22561 keyboard_1b1:

```



```

22562 00000049 E8FB00      call    do_int_stacks
22563 0000004C [4100]      dw      old09
22564
22565      ; -----
22566
22567 0000004E 00000000      old70:    dd      0
22568
22569      int70:
22570 00000052 E8F200      call    do_int_stacks
22571 00000055 [4E00]      dw      old70
22572
22573      ; -----
22574
22575      ; irp      a,<0a,0b,0c,0d,0e,72,73,74,76,77>
22576      ;public   int&a
22577      ;public   old&a
22578      ;public   firstflag&a
22579      ;int&a    proc    far
22580      ;        jmp    short entry_int&a&_stk
22581      ;old&a    dd      0          ;forward pointer
22582      ;        dw      424bh       ;compatible signature for int. sharing
22583      ;firstflag&a db 0          ;the firstly hooked.
22584      ;        jmp    short intret_&a ;reset routine. we don't care this.
22585      ;        db      7 dup (0)    ;reserved for future.
22586      ;entry_int&a&_stk:
22587      ;        call    do_int_stacks
22588      ;        dw      old&a
22589      ;intret_&a:
22590      ;        iret
22591      ;int&a    endp
22592      ;        endm
22593
22594      ; -----
22595
22596      int0A:
22597 00000057 EB10      jmp     short entry_int0A_stk
22598 00000059 00000000      old0A:    dd      0
22599 0000005D 4B42      dw      424Bh
22600
22601 0000005F 00      firstflag0A: db 0
22602 00000060 EB0C      jmp     short intret_0A
22603 00000062 00<rep 7h>    times 7 db 0
22604
22605      entry_int0A_stk:
22606 00000069 E8DB00      call    do_int_stacks
22607 0000006C [5900]      dw      old0A
22608
22609 0000006E CF      intret_0A:  iret
22610
22611      ; -----
22612
22613      int0B:
22614 0000006F EB10      jmp     short entry_int0B_stk
22615 00000071 00000000      old0B:    dd      0
22616 00000075 4B42      dw      424Bh
22617
22618 00000077 00      firstflag0B: db 0
22619 00000078 EB0C      jmp     short intret_0B
22620 0000007A 00<rep 7h>    times 7 db 0
22621
22622      entry_int0B_stk:
22623 00000081 E8C300      call    do_int_stacks
22624 00000084 [7100]      dw      old0B
22625
22626 00000086 CF      intret_0B:  iret
22627
22628      ; -----
22629
22630      int0C:
22631 00000087 EB10      jmp     short entry_int0C_stk
22632 00000089 00000000      old0C:    dd      0
22633 0000008D 4B42      dw      424Bh
22634
22635 0000008F 00      firstflag0C: db 0
22636 00000090 EB0C      jmp     short intret_0C
22637 00000092 00<rep 7h>    times 7 db 0
22638
22639      entry_int0C_stk:
22640 00000099 E8AB00      call    do_int_stacks
22641 0000009C [8900]      dw      old0C
22642
22643 0000009E CF      intret_0C:  iret
22644
22645      ; -----
22646
22647      int0D:
22648 0000009F EB10      jmp     short entry_int0D_stk
22649 000000A1 00000000      old0D:    dd      0
22650 000000A5 4B42      dw      424Bh
22651
22652 000000A7 00      firstflag0D: db 0
22653 000000A8 EB0C      jmp     short intret_0D
22654 000000AA 00<rep 7h>    times 7 db 0
22655
22656      entry_int0D_stk:
22657 000000B1 E89300      call    do_int_stacks
22658 000000B4 [A100]      dw      old0D
22659
22660 000000B6 CF      intret_0D:  iret
22661
22662      ; -----
22663
22664      int0E:
22665 000000B7 EB10      jmp     short entry_int0E_stk
22666 000000B9 00000000      old0E:    dd      0
22667 000000BD 4B42      dw      424Bh
22668
22669 000000BF 00      firstflag0E: db 0
22670 000000C0 EB0C      jmp     short intret_0E
22671 000000C2 00<rep 7h>    times 7 db 0
22672
22673      entry_int0E_stk:
22674 000000C9 E87B00      call    do_int_stacks
22675 000000CC [B900]      dw      old0E
22676
22677 000000CE CF      intret_0E:  iret
22678
22679      ; -----
22680
22681      int72:
22682 000000CF EB10      jmp     short entry_int72_stk
22683 000000D1 00000000      old72:    dd      0
22684 000000D5 4B42      dw      424Bh
22685
22686      firstflag72:

```

```

22686 000000D7 00          db      0
22687 000000D8 EB0C          jmp     short intret_72
22688 000000DA 00<rep 7h>    times  7 db 0
22689
22690 entry_int72_stk:
22691 000000E1 E86300        call    do_int_stacks
22692 000000E4 [D100]         dw      old72
22693 intret_72:
22694 000000E6 CF             ired
22695
22696 ; -----
22697
22698 int73:
22699 000000E7 EB10          jmp     short entry_int73_stk
22700 000000E9 00000000      old73:   dd      0
22701 000000ED 4B42          dw      424Bh
22702 firstflag73:
22703 000000EF 00           db      0
22704 000000F0 EB0C          jmp     short intret_73
22705 000000F2 00<rep 7h>    times  7 db 0
22706
22707 entry_int73_stk:
22708 000000F9 E84B00        call    do_int_stacks
22709 000000FC [E900]         dw      old73
22710 intret_73:
22711 000000FE CF             ired
22712
22713 ; -----
22714
22715 int74:
22716 000000FF EB10          jmp     short entry_int74_stk
22717 00000101 00000000      old74:   dd      0
22718 00000105 4B42          dw      424Bh
22719 firstflag74:
22720 00000107 00           db      0
22721 00000108 EB0C          jmp     short intret_74
22722 0000010A 00<rep 7h>    times  7 db 0
22723
22724 entry_int74_stk:
22725 00000111 E83300        call    do_int_stacks
22726 00000114 [0101]         dw      old74
22727 intret_74:
22728 00000116 CF             ired
22729
22730 ; -----
22731
22732 int76:
22733 00000117 EB10          jmp     short entry_int76_stk
22734 00000119 00000000      old76:   dd      0
22735 0000011D 4B42          dw      424Bh
22736 firstflag76:
22737 0000011F 00           db      0
22738 00000120 EB0C          jmp     short intret_76
22739 00000122 00<rep 7h>    times  7 db 0
22740
22741 entry_int76_stk:
22742 00000129 E81B00        call    do_int_stacks
22743 0000012C [1901]         dw      old76
22744 intret_76:
22745 0000012E CF             ired
22746
22747 ; -----
22748
22749 int77:
22750 0000012F EB10          jmp     short entry_int77_stk
22751 00000131 00000000      old77:   dd      0
22752 00000135 4B42          dw      424Bh
22753 firstflag77:
22754 00000137 00           db      0
22755 00000138 EB0C          jmp     short intret_77
22756 0000013A 00<rep 7h>    times  7 db 0
22757
22758 entry_int77_stk:
22759 00000141 E80300        call    do_int_stacks
22760 00000144 [3101]         dw      old77
22761 intret_77:
22762 00000146 CF             ired
22763
22764 ; -----
22765
22766 ;*****
22767 ;common routines
22768 ;*****
22769
22770 ; do interrupt stack switching. the fake return address holds
22771 ; a pointer to the far-pointer of the actual interrupt
22772 ; service routine
22773
22774 ; 21/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 SYSINIT)
22775 ; 21/03/2019 - Retro DOS v4.0
22776
22777 ;allocbyte equ 0
22778 ;intlevel equ 1
22779 ;savedsp equ 2
22780 ;savedss equ 4
22781 ;newsp equ 6
22782
22783 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 SYSINIT)
22784 ; (MSDOS 6.21 IO.SYS, SYSINIT:0147h)
22785
22786 do_int_stacks:
22787 00000147 50          push    ax
22788 00000148 55          push    bp
22789 00000149 06          push    es
22790 0000014A 2E8E06[0A00] mov     es,[cs:stacks+2] ; Get segment of stacks
22791 0000014F 2E8B2E[1000] mov     bp,[cs:nextentry] ; get most likely candidate
22792 00000154 B001          mov     al,allocated ; 1
22793 ; 21/10/2022
22794 ;xchg [es:bp+allocbyte],al
22795 ; 11/12/2022
22796 00000156 26864600    xchg     [es:bp],al ; grab the entry
22797 0000015A 3C00          cmp     al,free ; 0 ; still avail?
22798 0000015C 7551          jne     short notfree02
22799
22800 0000015E 2E832E[1000]08 sub     word [cs:nextentry],entrysize ; set for next interrupt
22801
22802 found02:
22803 00000164 26896602    mov     [es:bp+savedsp],sp ; save sp value
22804 00000168 268C5604    mov     [es:bp+savedss],ss ; save ss also
22805
22806 0000016C 89E8          mov     ax,bp ; temp save of table offset
22807
22808 0000016E 268B6E06    mov     bp,[es:bp+newsp] ; get new SP value
22809 ; 21/10/2022

```

```

22810      ;mov     bp,[es:bp+6]
22811      ; 11/12/2022
22812      ;cmp     [es:bp+0],ax
22813 00000172 26394600      cmp     [es:bp],ax      ; check for offset into table
22814 00000176 7544         jne     short foundbad02
22815
22816      ; 02/07/2023 (MSDOS 6.21 SYSINIT code)
22817 00000178 8CC0         mov     ax,es      ; point ss,sp to the new stack
22818 0000017A 8EC5         mov     es,bp
22819 0000017C 89E5         mov     bp,sp
22820 0000017E 8B6E06         mov     bp,[bp+6]
22821 00000181 8ED0         mov     ss,ax
22822 00000183 8CC4         mov     sp,es
22823 00000185 8EC0         mov     es,ax
22824 00000187 2E8B6E00      mov     bp,[cs:bp]
22825
22826      ; 21/10/2022 (MSDOS 5.0 SYSINIT code)
22827      ;push     bp
22828      ;mov     bp,sp
22829      ;mov     ax,[bp+8]
22830      ;pop     bp
22831      ;push     es
22832      ;pop     ss
22833      ;mov     sp,bp
22834      ;mov     bp,ax
22835      ; 11/12/2022
22836      ;mov     bp,[cs:bp+0]
22837      ;mov     bp,[cs:bp]
22838
22839 0000018B 9C             pushf      ; go execute the real interrupt handler
22840      ; 11/12/2022
22841 0000018C 2EFF5E00      call     far [cs:bp]      ; which will iret back to here
22842      ; 21/10/2022
22843      ;call     far [cs:bp+0]
22844
22845 00000190 89E5         mov     bp,sp      ; retrieve the table offset for us
22846      ; 11/12/2022
22847 00000192 268B6E00      mov     bp,[es:bp]      ; but leave it on the stack
22848      ; 21/10/2022
22849      ;mov     bp,[es:bp+0]
22850 00000196 268E5604      mov     ss,[es:bp+savesdss] ; get old stack back
22851 0000019A 268B6602      mov     sp,[es:bp+savesdss]
22852
22853      ; 11/12/2022
22854      ;mov     byte [es:bp+allocbyte],free ; free the entry
22855      ; 21/10/2022
22856 0000019E 26C6460000      mov     byte [es:bp],free ; 0
22857 000001A3 2E892E[1000]      mov     [cs:nextentry],bp ; setup to use next time
22858
22859 000001A8 07             pop     es
22860 000001A9 5D             pop     bp      ; saved on entry
22861 000001AA 58             pop     ax      ; saved on entry
22862 000001AB 83C402         add     sp,2
22863 000001AE CF             iret      ; done with this interrupt
22864
22865      notfree02:
22866 000001AF 3C01         cmp     al,allocated ; error flag
22867 000001B1 7404         je     short findnext02 ; no, continue
22868      ; 11/12/2022
22869      ;xchg     [es:bp+allocbyte],al ; yes, restore error value
22870      ; 21/10/2022
22871 000001B3 26864600      xchg     [es:bp],al
22872
22873      findnext02:
22874 000001B7 E81200         call     longpath
22875 000001BA EBA8         jmp     short found02
22876
22877      foundbad02:
22878 000001BC 2E3B2E[0C00]      cmp     bp,[cs:firstentry]
22879 000001C1 72F4         jc     short findnext02
22880 000001C3 89C5         mov     bp,ax      ; flag this entry
22881      ; 11/12/2022
22882      ;mov     byte [es:bp+allocbyte],clobbered
22883      ; 21/10/2022
22884 000001C5 26C6460003      mov     byte [es:bp],clobbered ; 3
22885 000001CA EBEB         jmp     short findnext02 ; keep looking
22886
22887      ; -----
22888
22889      ; Common routines
22890
22891      longpath:
22892      ; 21/03/2019
22893 000001CC 2E8B2E[0E00]      mov     bp,[cs:lastentry] ; start with last entry in table
22894
22895      lploopp:
22896      ; 11/12/2022
22897      ;cmp     byte [es:bp+allocbyte],free ; is entry free?
22898      ; 21/10/2022
22899 000001D1 26807E0000      cmp     byte [es:bp],free
22900 000001D6 7512         jne     short inuse ; no, try next one
22901
22902      mov     al,allocated
22903      ; 11/12/2022
22904      ;xchg     [es:bp+allocbyte],al ; allocate entry
22905      ; 21/10/2022
22906 000001DA 26864600      xchg     [es:bp],al
22907 000001DE 3C00         cmp     al,free ; is it still free?
22908 000001E0 7414         je     short found ; yes, go use it
22909
22910      cmp     al,allocated ; is it other than Allocated or Free?
22911 000001E2 3C01         je     short inuse ; no, check the next one
22912
22913      ; 11/12/2022
22914      ;mov     [es:bp+allocbyte],al ; yes, put back the error state
22915      ; 21/10/2022
22916      mov     [es:bp],al
22917      inuse:
22918      cmp     bp,[cs:firstentry]
22919      je     short fatal
22920      sub     bp,entrysize
22921      jmp     short lploopp
22922      found:
22923      retn
22924      fatal:
22925      push     ds
22926      mov     ax,0F000h ;look at the model byte
22927      mov     ds,ax
22928      cmp     byte [0FFFEh],0F9h ; mdl_convert ; convertible?
22929      pop     ds
22930      jne     short skip_nmis
22931
22932      mov     al,07h ; disable pc convertible nmis
22933      out     72h,al

```

```

22934
22935 00000209 FA
22936 0000020A B0FF
22937 0000020C E621
22938 0000020E E6A1
22939
22940 00000210 8CCE
22941 00000212 8EDE
22942 00000214 BE[3B02]
22943
22944
22945
22946 00000217 50
22947 00000218 1E
22948
22949
22950
22951 00000219 B87000
22952 0000021C 8ED8
22953
22954
22955 0000021E F606[1208]01
22956 00000223 1F
22957 00000224 58
22958 00000225 7405
22959
22960
22961
22962
22963 00000227 9A[1308]7000
22964
22965
22966
22967
22968 0000022C AC
22969 0000022D 3C24
22970 0000022F 7408
22971
22972 00000231 B307
22973 00000233 B40E
22974 00000235 CD10
22975 00000237 EBF3
22976
22977
22978 00000239 EBFE
22979
22980
22981
22982
22983
22984
22985
22986
22987
22988
22989
22990
22991
22992 0000023B 0D0A
22993 0000023D 070D0A
22994 00000240 496E7465726E616C20-
22994 00000249 737461636B206F7665-
22994 00000252 72666C6F770D0A
22995 00000259 53797374656D206861-
22995 00000262 6C7465640D0A24
22996
22997
22998
22999
23000
23001
23002
23003
23004
23005
23006
23007
23008
23009
23010
23011
23012
23013
23014
23015
23016
23017
23018
23019
23020
23021
23022 00000269 E9AD01
23023
23024
23025
23026
23027
23028 00000000 ????????
23029 00000004 ???
23030 00000006 ????????
23031
23032
23033
23034
23035 0000026C 00
23036
23037
23038
23039
23040
23041 0000026D 00000000
23042
23043
23044
23045 00000271 0000
23046
23047
23048
23049
23050
23051
23052
23053 00000273 0000
23054

skip_nmis:
cli                    ; disable and mask
mov     al,0FFh        ; all other ints
out     021h,al
out     0A1h,al

mov     si,cs
mov     ds,si
mov     si,fatal_msg

;SR;
; we set all foci to this VM to issue the stack failure message
;
push     ax
push     ds
;;mov     ax,Bios_Data ; 0070h
;mov     ax,KERNEL_SEGMENT ; 0070h
; 21/10/2022
mov     ax,DOSBIODATASEG
mov     ds,ax

;test     byte [08D0h],1 ; (MSDOS 6.21, IO.SYS - SYSINIT:021Eh)
test     byte [IsWin386],1 ; (retrodos4.sys, offset: ****h)
pop     ds
pop     ax
jz       short fatal_loop ; win386 not present, continue

;;call    far ptr 0070h:08D1h ; (MSDOS 621, IO.SYS - SYSINIT:0227h)
;call     KERNEL_SEGMENT:V86_Crit_SetFocus ; set focus to this VM
; 21/10/2022
call     DOSBIODATASEG:V86_Crit_SetFocus ; 0070h:08D1h
;
;SR; we do not bother about the returned status of this call.
;
fatal_loop:
lods     al,'$'
cmp     al,'$'
je       short fatal_done

mov     bl,7
mov     ah,14
int     10h ; whoops, this enables ints
jmp     short fatal_loop

fatal_done:
jmp     short fatal_done

; 21/03/2019 - Retro DOS v4.0 (MSDOS 6.0, SYSINIT1.ASM, 1991)
; -----
; include msbio.c15 ; fatal stack error message

; MSDOS 6.21, IO.SYS, SYSINIT:023Bh

; STKMES.INC - MSDOS 3.3 (24/07/1987)
; -----
; 04/06/2018 - Retro DOS v3.0

fatal_msg:
db       0Dh,0Ah
db       7,0Dh,0Ah
db       "Internal stack overflow",0Dh,0Ah

db       "System halted",0Dh,0Ah,"$"

endstackcode:
; -----
; SYINIT1.ASM (MSDOS 6.0, 1991) 'SYSINIT' jump addr from 'MSINIT.ASM'
; -----
; 04/06/2018 - Retro DOS v3.0 (MSDOS 3.3, SYSINIT1.ASM, 24/07/1987)
; 22/03/2019 - Retro DOS v4.0
; SYSINIT:0269h (MSDOS 6.21 IO.SYS, SYSINIT segment, offset: 0269h)
; ('SYSINIT:' location/address is used in 'retrodos4.s'. If following
; address will be changed, it must also be changed in 'retrodos4.s'.)
; 21/10/2022- Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
; -----
; SYSINITSEG:0267h (MSDOS 5.0 IO.SYS, SYSINIT segment, offset: 0267h)
; SYSINIT:0269h (MSDOS 6.22 IO.SYS, SYSINIT segment, offset: 0269h)
; 29/12/2023- Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
; -----
; SYSINITSEG:0269h (PCDOS 7.1 IBMBIO.COM, SYSINIT segment, offset: 0269h)

SYSINIT:
JMP     GOINIT
;JMP     SYSIN ; 25/02/2018 - Retro DOS 2.0 modification
; -----

struc    DDHighInfo
.ddhigh_CSegPtr resd 1 ; pointer to code segment to be relocated
.ddhigh_CSegLen resw 1 ; length of code segment to be relocated
.ddhigh_CallBak resd 1 ; pointer to the call back routine
endstruc

; 22/03/2019 - Retro DOS v4.0

runhigh: db 0

; 02/11/2022
;align 4

DOSINFO:
dd       0 ; address of the DOS Sysini variables
;MSDOS:
dos_temp_location: ; dword ; MSDOS 6.0
dosinit: ; MSDOS 6.0
dw       0

; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
;FINAL_DOS_LOCATION: ; 20/04/2019 - Retro DOS v4.0
; dw 0
;MSDOS 5.0 IO.SYS - SYSINIT:0271h

CURRENT_DOS_LOCATION:
dw       0

```

```

23055 ;DOSSIZE: ; Retro DOS 2.0 feature - 25/02/2018
23056 ; dw 0 ; 'MSDOS.BIN' kernel size in words
23057
23058 ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
23059 ; (MSDOS 5.0 MSDOS.SYS size is 37394 bytes)
23060 ;DOSSIZE equ 0A000h ; (MSDOS 6.0 - SYSINIT1.ASM - 1991)
23061 ; 30/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
23062 ; 03/09/2023 (PCDOS 7.1 IBMDOS.COM size is 42566 bytes, 04/12/2003)
23063 DOSSIZE equ 0B000h ; (PCDOS 7.1 - SYSINIT)
23064
23065 DEVICE_LIST:
23066 dd 0
23067
23068 ; 04/06/2018 - Retro DOS v3.0
23069 ; 28/03/2018
23070 ;; MSDOS 3.3 - SYSINIT1.ASM - 24/07/1987
23071 ;
23072 sysi_country:
23073 dd 0 ; 5/29/86 Pointer to country table in DOS
23074
23075 ; MSDOS 6.0
23076 dos_segrenit: dw 0,0 ; room for dword
23077
23078 lo_doscod_size: dw 0 ; dos code size when in low mem
23079 hi_doscod_size: dw 0 ; dos code size when in HMA
23080
23081 def_php: dw 0
23082
23083 ; M022--
23084 ; pointer for calling into Bios_Code for re-initializing segment values.
23085 ; call with ax = new segment for Bios_Code. Notice that we'll
23086 ; call it in its temporary home, cuz seg_reinit won't get moved to
23087 ; the new home.
23088
23089 ;Bios_Code equ KERNEL_SEGMENT ; 0070h
23090 ; 21/10/2022
23091 ;DOSBIOCODESEG equ 02C7h ; (MSDOS 5.0 IO.SYS)
23092
23093 ; 22/10/2022
23094 seg_reinit_ptr: ; label dword
23095 dw seg_reinit ; Bios_Code:0032h for MSDOS 6.21 IO.SYS
23096 temp_bcode_seg:
23097 ;dw Bios_Code ; 02CCh for MSDOS 6.21 IO.SYS
23098 ; 22/10/2022
23099 dw DOSBIOCODESEG ; 02C7h for MSDOS 5.0 IO.SYS
23100 ; 364h for PCDOS 7.1 IBMBIO.COM - 29/12/2023
23101 fake_floppy_drv:
23102 db 0 ; set to 1 if this machine
23103 ; does not have any floppies!!!
23104
23105 ; Internal Stack Parameters
23106
23107 stack_count: dw defaultcount ; 9
23108 stack_size: dw defaultsize ; 128
23109 stack_addr: dd 0
23110
23111 ; 05/06/2018 - Retro DOS v3.0
23112
23113 ; various default values
23114
23115 MEMORY_SIZE: dw 1
23116
23117 ; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0 source, MSDOS 6.21 disassembled src.)
23118
23119 RPLMemTop: dw 0 ; 22/10/2022 (MSDOS 5.0 IO.SYS SYSINIT:0294h)
23120 DEFAULT_DRIVE: db 0 ; initialized by ibminit.
23121 buffers: dw 0FFFFh ; initialized during buffer allocation
23122 h_buffers: dw 0 ; # of the heuristic buffers. initially 0.
23123 singlebuffersize: dw 0 ; maximum sector size + buffer head
23124
23125 FILES: db 8 ; enough files for pipe
23126 FCBS: db 4 ; performance for recycling
23127 KEEP: db 0 ; keep original set
23128 NUM_CDS: db 5 ; 5 net drives
23129
23130 ; 22/10/2022 (MSDOS 5.0 SYSINIT)
23131 ;;CONFBOT: dw 0
23132 ;;ALLOCLIM: dw 0
23133 ;CONFBOT: ; 02/11/2022
23134 ;top_of_cdss: dw 0
23135
23136 ; 30/12/2022 - RetroDOS v4.2 (MSDOS 6.21 SYSINIT)
23137 ; (SYSINIT:02A3h)
23138 CONFBOT: dw 0
23139 ALLOCLIM: dw 0
23140 top_of_cdss: dw 0
23141
23142 ; 02/11/2022 (MSDOS 5.0 SYSINIT)
23143 ; 30/12/2022 (MSDOS 6.21 SYSINIT)
23144 ;ALLOCLIM: dw 0 ; (SYSINIT:02A3h)
23145
23146 DirStrng: db "A:\",0 ; string for the root directory of a drive
23147
23148 ; 30/12/2022 - Retro DOS v4.2 (MSDOS 6.21 SYSINIT)
23149 %if 0
23150 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
23151 ; (SYSINIT:02A9h)
23152
23153 command_line:
23154 db 2,0
23155 db 'p'
23156 db 0
23157 times 124 db 0 ; db 124 dup(0)
23158
23159 %endif
23160
23161 ; (SYSINIT:0329h)
23162 ZERO: db 0
23163 sepchr: db 0
23164 linecount: dw 0 ; line count in config.sys
23165 showcount: db ' ' ; used to convert linecount to ascii.
23166 buffer_linenum: dw 0 ; line count for "buffers=" command if entered.
23167
23168 sys_model_byte: db 0FFh ; model byte used in sysinit
23169 sys_scnd_model_byte: db 0 ; secondary model byte used in sysinit
23170
23171 buf_prev_off: dw 0
23172
23173 ;IF NOT NOEXEC
23174 ;COMEXE EXEC0 <0,COMMAND_LINE,DEFAULT_DRIVE,ZERO>
23175 ;ENDIF
23176
23177 ; 29/12/2023
23178 ; 01/05/2018

```

```

23179 COMEXE:
23180 EXEC0.ENVIRON: dw 0 ; seg addr of environment
23181 EXEC0.COM_LINE: dw command_line ; pointer to asciz command line
23182 dw 0 ; SYSINIT segment (0473h for MSDOS 6.21 IO.SYS)
23183 ; SYSINIT segment (0544h for PC DOS 7.1 IBMBIO.COM)
23184 EXEC0.5C_FCB: dw DEFAULT_DRIVE ; default fcb at 5C
23185 dw 0 ; SYSINIT segment (0473h for MSDOS 6.21 IO.SYS)
23186 EXEC0.6C_FCB: dw ZERO ; default fcb at 6C
23187 dw 0
23188
23189 ; variables for install= command.
23190
23191 multi_pass_id: db 0 ; parameter passed to multi_pass
23192 ; indicating the pass number
23193 ; 0 - do scan for DOS=HIGH/LOW
23194 ; 1 - load device drivers
23195 ; 2 - was to load IFS
23196 ; now it is unused
23197 ; 3 - do install=
23198 ; >3 - nop
23199 install_flag: dw 0
23200
23201 have_install_cmd equ 00000001b ; config.sys has install= commands
23202 has_installed equ 00000010b ; sysinit_base installed.
23203
23204 config_size: dw 0 ; size of config.sys file. set by sysconf.asm
23205 sysinit_base_ptr: dd 0 ; pointer to sysinit_base
23206 sysinit_ptr: dd 0 ; returning addr. from sysinit_base
23207 checksum: dw 0 ; used by sum_up
23208
23209 ldexec_fcb: times 20 db 20h ; db 20 dup (' ') ; big enough
23210 ldexec_line: db 0 ; # of parm characters
23211 ldexec_start: db ' '
23212 ldexec_parm: times 80 db 0 ; db 80 dup (0)
23213
23214 ; instexe exec0 <0,ldexec_line,ldexec_fcb,ldexec_fcb>
23215
23216 instexe:
23217 iexec.environ: dw 0 ; seg addr of environment
23218 iexec.ldexec_line: dw ldexec_line ; pointer to asciz command line
23219 dw 0 ; SYSINIT segment (0473h for MSDOS 6.21 IO.SYS)
23220 ; SYSINIT segment (0544h for PC DOS 7.1 IBMBIO.COM)
23221 iexec.ldexec_5c_fcb: dw ldexec_fcb ; default fcb at 5C
23222 dw 0 ; SYSINIT segment (0473h for MSDOS 6.22 IO.SYS)
23223 iexec.ldexec_6c_fcb: dw ldexec_fcb ; default fcb at 6C
23224 dw 0
23225
23226 ; variables for comment=
23227
23228 com_level: db 0 ; level of " " in command line
23229 cmmt: db 0 ; length of comment string token
23230 cmmt1: db 0 ; token
23231 cmmt2: db 0 ; token
23232 cmd_indicator: db 0
23233 donotshownum: db 0
23234
23235 count: dw 0
23236 org_count: dw 0
23237 chrptr: dw 0
23238 cntryfilehandle: dw 0
23239 old_area: dw 0
23240 impossible_owner_size: dw 0 ; paragraph
23241
23242 bucketptr: ; label dword
23243 bufptr: ; label dword ; leave this stuff in order!
23244 memlo: dw 0
23245 prmbk: ; label word
23246 memhi: dw 0
23247 ldoff: dw 0
23248 area: dw 0
23249
23250 ; 29/12/2023 - PC DOS 7.1 IBMBIO.COM - SYSINIT:036Ah
23251 prev_memhi: dw 0
23252 prev_alloclim: dw 0
23253 dosdata_umb: db 0
23254
23255 ; Following is the request packet used to call INIT routines for
23256 ; all device drivers. Some fields may be accessed individually in
23257 ; the code, and hence have individual labels, but they should not
23258 ; be separated.
23259
23260 packet: db 25 ; PC DOS 7.1 IBMBIO.COM
23261 ; db 24 ; was 22
23262 db 0
23263 db 0 ; initialize code
23264 dw 0
23265 times 8 db 0 ; db 8 dup (?)
23266
23267 unitcount: db 0
23268 break_addr: dd 0
23269 bpb_addr: dd 0
23270 drivenumber: ; 22/10/2022
23271 devdrivenum: db 0
23272 configmsgflag: dw 0 ; used to control "error in config.sys line #" message
23273
23274 ; end of request packet
23275
23276 ; drivenumber: db 0 ; 22/03/2019
23277
23278 toomanydrivesflag:
23279 db 0 ; >24 fixed disk partitions flag ; M029
23280 align 2
23281
23282 BCodeSeg: ; 21/10/2022
23283 dw DOSBIOCODESEG ; (02C7h for MSDOS 5.0 IO.SYS)
23284 ; 0364h for PC DOS 7.1 IBMBIO.COM - 29/12/2023
23285 ; dw Bios_Code ; = KERNEL_SEGMENT = 0070h (for Retro DOS v4.0)
23286 ; BCodeSeg = 2CCh (for MSDOS 6.21 IO.SYS)
23287
23288 ; 30/12/2022
23289 ; MSDOS 6.21 IO.SYS, SYSINIT:0387h
23290 ;
23291 ; Magicbackdoor: dd 0
23292 ; NullBackdoor:
23293 ; retf
23294
23295 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
23296 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
23297 ; 19/04/2019
23298 _timer_lw:
23299 dw 0 ; MSDOS 6.21 IO.SYS - SYSINIT:038Ch
23300
23301 ; 29/12/2023 - Retro DOS v5.0
23302 ; PC DOS 7.1 IBMBIO.COM - SYSINIT:038Eh

```



```

23303
23304 0000038E 00      F5_key:      db 0
23305 0000038F 00      F8_key:      db 0
23306 00000390 00000000 MagicBackdoor:      dd 0
23307 NullBackdoor:
23308 00000394 CB      retf
23309
23310 ;SR;
23311 ; This is the communication block between the DOS and the BIOS. It starts at
23312 ; the SysinitPresent flag. Any other data that needs to be communicated
23313 ; to the DOS should be added after SysinitPresent. The pointer to this block
23314 ; is passed to DOS as part of the DOSINIT call.
23315 ;
23316
23317 BiosComBlock:
23318 ;dd      Bios_Data:SysinitPresent
23319 ;      ; 0070h:08FDh for MSDOS 6.21 IO.SYS
23320 00000395 [DD07]      dw      SysinitPresent ; (retrodos4.sys, offset: ****h)
23321 ;dw      KERNEL_SEGMENT ; 0070h
23322 ;      ; 21/10/2022
23323 00000397 7000      dw      DOSBIODATASEG ; 0070h
23324
23325 ;align 2
23326
23327 ;      ; 22/10/2022 - (MSDOS 5.0 IO.SYS, SYSINIT:0406h)
23328 ;      ; 30/12/2022 - (MSDOS 6.21 IO.SYS, SYSINIT:0392h)
23329
23330 00000399 00<rep 80h> tempstack:
23331 times 128 db 0 ; db 80h dup (?)
23332
23333 ; -----
23334 ;      ; 29/12/2023 - Retro DOS v5.0
23335 ;      ; 22/10/2022 - Retro DOS v4.0
23336 ;      ; (MSDOS 5.0 IO.SYS, SYSINIT:0486h)
23337 GOINIT:      ; (MSDOS 6.22 IO.SYS, SYSINIT:0412h)
23338 ;      ; (PCDOS 7.1 IBMBIO.COM, SYSINIT:0419h)
23339 ;      ; 12/12/2023
23340 00000419 0E      push     cs
23341 0000041A 1F      pop      ds
23342
23343 ;      ; 12/12/2022
23344 ;      ; 22/03/2019 - Retro DOS v4.0
23345 ;      ; 06/07/2018
23346 ;      ; 04/06/2018 - Retro DOS v3.0
23347 ; before doing anything else, let's set the model byte
23348 0000041B B4C0      mov      ah,0C0h ; get system configuration
23349 0000041D CD15      int      15h ;
23350 0000041F 7214      jc      short no_rom_config
23351
23352 ;cmp      ah,0 ; double check
23353 ;jne      short no_rom_config
23354 ;      ; 03/09/2023
23355 00000421 08E4      or       ah,ah
23356 00000423 7510      jnz     short no_rom_config
23357
23358 ;      ; 12/12/2023 ; *
23359 ;      ; ds = cs
23360
23361 00000425 268A4702    mov      al,[es:bx+ROMBIOS_DESC.bios_sd_modelbyte]
23362 ;mov      [cs:sys_model_byte],al
23363 00000429 A2[BB02]    mov      [sys_model_byte],al ; *
23364 0000042C 268A4703    mov      al,[es:bx+ROMBIOS_DESC.bios_sd_scnd_modelbyte]
23365 ;mov      [cs:sys_scnd_model_byte],al
23366 00000430 A2[BC02]    mov      [sys_scnd_model_byte],al ; *
23367 ;jmp      short SYSIN
23368 ;      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
23369 00000433 EB29      jmp      short move_myself
23370
23371 no_rom_config:      ; old ROM
23372 ;      ; 12/12/2023
23373 ;mov      ax,0F000h
23374 ;mov      ds,ax
23375 ;mov      al,[0FFFEh]
23376 ;mov      [cs:sys_model_byte],al; set the model byte.
23377 ;      ; 12/12/2023
23378 ;      ; ds = cs
23379 00000435 B800F0    mov      ax,0F000h
23380 00000438 8EC0      mov      es,ax
23381 0000043A 26A0FEFF    mov      al,[es:0FFFEh]
23382 0000043E A2[BB02]    mov      [sys_model_byte],al ; set the model byte.
23383
23384 ; set fake_floppy_drv if there is no diskette drives in this machine.
23385 ; execute the equipment determination interrupt and then
23386 ; check the returned value to see if we have any floppy drives
23387 ; if we have no floppy drive we set cs:fake_floppy_drv to 1
23388 ; see the at tech ref bios listings for help on the equipment
23389 ; flag interrupt (11h)
23390
23391 ;      ; 22/10/2022
23392 ;check_for_fake_floppy:      ; entry point for rom_config above
23393 00000441 CD11      int      11h ; check equipment flag
23394
23395 ;      ; 29/12/2023 - Retro DOS v5.0
23396 ;jmp      short check_for_fake_floppy
23397 ;      ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0446h
23398 ;db      52h ; 'RPS' sign
23399 ;db      50h
23400 ;db      53h
23401
23402 check_for_fake_floppy:
23403 ;      ; 29/12/2023
23404 ;      ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0449h
23405 ;or       ax,1 ; (nonsense! this may be overwritten/disabled
23406 ;      ; ; by using 'RPS' sign position)
23407 ;      ; ; 03/07/2023 - Erdogan Tan
23408 ;test     ax,1 ; have any floppies?
23409
23410 ;      ; 12/12/2022
23411 00000443 A801      test     al,1
23412 ;test     ax,1 ; have any floppies?
23413 00000445 7517      jnz     short move_myself ; yes,normal system
23414
23415 ; Some ROM BIOSs lie that there are no floppy drives. Lets find out
23416 ; whether it is an old ROM BIOS or a new one
23417 ;
23418 ; WARNING !!!
23419 ;
23420 ; This sequence of code is present in MSINIT.ASM also. Any modification
23421 ; here will require an equivalent modification in MSINIT.ASM also
23422
23423 ;      ; 12/12/2023
23424 ;push     es ; not necessary
23425
23426 00000447 30C9      xor      cl,cl

```

```

23427 00000449 B408      mov     ah,8           ; get disk parameters
23428 0000044B B200      mov     dl,0           ; of drive 0
23429 0000044D CD13      int      13h
23430
23431                    ;pop     es ; 12/12/2023
23432
23433 0000044F 720D      jc      short move_myself ; if error lets assume that the
23434                    ; ROM BIOS lied
23435                    ; double check (max sec no cannot be 0)
23436                    ;cmp     cl,0
23437                    ;je      short move_myself
23438 00000451 08C9      ; 03/09/2023
23439 00000453 7409      or      cl,cl
23440                    jz      short move_myself
23441 00000455 08D2      or      dl,dl ; number of flp drvs == 0?
23442 00000457 7505      jnz     short move_myself ; no
23443
23444                    ;mov     byte [cs:fake_floppy_drv],1 ; set fake flag.
23445                    ; 12/12/2023
23446                    ; ds = cs
23447 00000459 C606[8B02]01 mov     byte [fake_floppy_drv],1 ; set fake flag.
23448
23449 move_myself:
23450                    ; 12/12/2023
23451                    ;cld      ; not necessary ; set up move
23452                    ;xor     si,si
23453                    ;mov     di,si
23454
23455                    ; 12/12/2023
23456                    ; ds = cs
23457                    ; 12/12/2022
23458                    ;push    cs
23459                    ;pop     ds
23460
23461                    ;mov     cx,[cs:MEMORY_SIZE]
23462 0000045E 8B0E[9402] mov     cx,[MEMORY_SIZE] ; 12/12/2022
23463
23464                    ; (MSDOS 6.0 - SYSINIT1.ASM - 1991)
23465                    ;;; if msver
23466                    ; cmp     cx,1 ; 1 means do scan
23467                    ; jnz     short noscan
23468                    ; mov     cx,2048 ; start scanning at 32k boundary
23469                    ; xor     bx,bx
23470
23471 memscan:inc     cx
23472                    ; jz      short setend
23473                    ; mov     ds,cx
23474                    ; mov     al,[bx]
23475                    ; not     al
23476                    ; mov     [bx],al
23477                    ; cmp     al,[bx]
23478                    ; not     al
23479                    ; mov     [bx],al
23480                    ; jz      short memscan
23481 ;setend:
23482                    ; mov     cs:[memory_size],cx
23483                    ;;; endif
23484
23485 ;noscan:
23486                    ; cx is mem size in para
23487                    ; cas -- a) if we got our memory size from the ROM, we should test it
23488                    ; before we try to run.
23489                    ; b) in any case, we should check for sufficient memory and give
23490                    ; an appropriate error diagnostic if there isn't enough
23491
23492                    ; push    cs
23493                    ; pop     ds
23494
23495                    ; cas note: It would be better to put dos + bios_code BELOW sysinit
23496                    ; that way it would be easier to slide them down home in a minimal
23497                    ; memory system after sysinit. As it is, you need room to keep
23498                    ; two full non-overlapping copies, since sysinit sits between the
23499                    ; temporary home and the final one. the problem with doing that
23500                    ; is that sys*.asm are filled with "mov ax,cs, sub ax,11h" type stuff.
23501
23502                    ; dec     cx ; one para for an arena at end of mem
23503                    ; ; in case of UMBS
23504
23505                    ; 22/10/2022
23506                    ; (MSDOS 5.0 IO.SYS SYSINIT:04DBh)
23507
23508                    ; 12/12/2022
23509                    ;push    cs
23510                    ;pop     ds
23511
23512 00000462 49      dec     cx
23513
23514 ;----- Check if an RPL program is present at TOM and do not tromp over it
23515
23516 00000463 31DB      xor     bx,bx
23517 00000465 8EC3      mov     es,bx
23518                    ;mov     bx,[es:(2Fh*4)] ; INT 2Fh address (0:0BCh)
23519                    ;mov     es,[es:((2Fh*4)+2)] ; INT 2Fh segment (0:0BEh)
23520                    ; 29/09/2023
23521 00000467 26C41EBC00 les     bx,[es:(2Fh*4)]
23522 0000046C 26817F035250 cmp     word [es:bx+3], 'RP'
23523 00000472 751B      jne     short NoRPL
23524 00000474 26807F054C cmp     byte [es:bx+5], 'L'
23525 00000479 7514      jne     short NoRPL
23526
23527 0000047B 89CA      mov     dx,cx ; get TOM into DX
23528 0000047D 52      push    dx
23529 0000047E B8064A      mov     ax,4A06h
23530                    ;mov     ax,(multMULT<<8)+multMULTRPLTOM
23531 00000481 CD2F      int      2Fh ; Get new TOM from any RPL
23532 00000483 58      pop     ax
23533 00000484 89D1      mov     cx,dx
23534 00000486 39C2      cmp     dx,ax
23535 00000488 7405      je      short NoRPL
23536
23537                    ; 11/12/2022
23538                    ; ds = cs
23539 0000048A 8916[9602] mov     [RPLMemTop],dx
23540                    ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
23541                    ;mov     [cs:RPLMemTop],dx
23542
23543 0000048E 49      dec     cx
23544 NoRPL:
23545 0000048F B8[1054] mov     ax,SI_end ; need this much room for sysinit
23546                    ; (SI_end == sysinit code size)
23547                    ; 03/09/2023
23548                    ; (58A0h for MSDOS 6.21 IO.SYS)
23549                    ; (5B40h for PCDOS 7.1 IBMBIO.COM)
23550 00000492 E80509      call    off_to_para

```



```

23551 00000495 29C1      sub     cx,ax
23552
23553 ; we need to leave room for the DOS and (if not ROMDOS) for the BIOS
23554 ; code above sysinit in memory
23555 ;
23556 00000497 81E9000B    sub     cx,DOSSIZE/16 ; (0A00h) ; leave this much room for DOS
23557 ; (0B00h) ; (PCDOS 7.1 IBMBIO.COM) -03/09/2023-
23558
23559 0000049B B8701D      mov     ax,BCODE_END      ; (1A60h for MSDOS 5.0 IO.SYS)
23560 ; (1A70h for MSDOS 6.21 IO.SYS)
23561 ; 03/09/2023
23562 ; (1E00h for PCDOS 7.1 IBMBIO.COM)
23563 0000049E E8F908      call    off_to_para      ; leave this much room for BIOS code
23564 000004A1 29C1      sub     cx,ax
23565 000004A3 8EC1      mov     es,cx            ; segment where sysinit will be located
23566
23567 ; 12/12/2023
23568 000004A5 FC          cld     ; not necessary ; set up move
23569 000004A6 31F6      xor     si,si
23570 000004A8 89F7      mov     di,si
23571
23572 000004AA B9[1054]    mov     cx,SI_end        ; (sysinit code size)
23573 000004AD D1E9      shr     cx,1            ; divide by 2 to get words
23574 000004AF F3A5      rep     movsw           ; relocate sysinit
23575
23576 000004B1 06          push    es              ; push relocated segment
23577 000004B2 B8[B704]    mov     ax,SYSIN
23578 000004B5 50          push    ax              ; push relocated entry point
23579
23580 000004B6 CB          retf     ; far jump to relocated sysinit
23581
23582 ; ===== S U B R O U T I N E =====
23583
23584 ; 30/12/2023
23585 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:04CEh
23586 %if 0
23587 get_cpu_type:
23588     pushf
23589     push    bx
23590     xor     bx,bx
23591     xor     ax,ax
23592     push    ax
23593     popf
23594     pushf
23595     pop     ax
23596     and     ax,0F000h
23597     cmp     ax,0F000h
23598     jz      short cpu_8086
23599     mov     ax,0F000h
23600     push    ax
23601     popf
23602     pushf
23603     pop     ax
23604     and     ax,0F000h
23605     jz      short cpu_286
23606 cpu_386:
23607     inc     bx
23608 cpu_286:
23609     inc     bx
23610 cpu_8086:
23611     mov     ax,bx
23612     pop     bx
23613     popf
23614     retn
23615 %endif
23616
23617 ; -----
23618
23619 ; MOVE THE DOS TO ITS PROPER LOCATION
23620
23621 ; 22/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS)
23622 ; (SYSINIT:0533h)
23623 ; 30/12/2022 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS)
23624 ; (SYSINIT:04BFh)
23625 ; 03/09/2023 - Retro DOS 4.2 (5.0 - Modified PCDOS 7.1 IBMBIO.COM)
23626 ; (SYSINIT:04F3h)
23627 SYSIN:
23628 ; Retro DOS 5.0 - 30/12/2023
23629 ; Retro DOS 4.0 - 22/03/2019
23630 ; Retro DOS 2.0 - 25/02/2018
23631
23632 ; 23/04/2019
23633 ; mov ax,Bios_Data
23634 ; mov ax,KERNEL_SEGMENT ; 0070h
23635 ; 21/10/2022
23636 000004B7 B87000    mov     ax,DOSBIODATASEG ; 0070h
23637 000004BA 8ED8      mov     ds,ax
23638
23639 ; 30/12/2023 - Retro DOS v5.0
23640 ;;;
23641 ; push es
23642 ; push ax ; not needed (*) E.TAN - 03/07/2023
23643 ; push di
23644
23645 ; call get_cpu_type ; determine if 386 system
23646 ;
23647 get_cpu_type:
23648 000004BC 9C          pushf
23649 000004BD 31C0      xor     ax,ax
23650 000004BF 50          push    ax
23651 000004C0 9D          popf
23652 000004C1 9C          pushf
23653 000004C2 58          pop     ax
23654 000004C3 2500F0      and     ax,0F000h
23655 000004C6 3D00F0      cmp     ax,0F000h
23656 000004C9 740F      jz      short cpu_8086
23657 000004CB B800F0      mov     ax,0F000h
23658 000004CE 50          push    ax
23659 000004CF 9D          popf
23660 000004D0 9C          pushf
23661 000004D1 58          pop     ax
23662 000004D2 2500F0      and     ax,0F000h
23663 000004D5 7402      jz      short cpu_286
23664 cpu_386:
23665 000004D7 29C0      sub     ax,ax
23666 cpu_286:
23667 000004D9 40          inc     ax
23668 cpu_8086:
23669 ; ax = 0
23670 000004DA 2EA2[B606]    mov     [cs:cpu_type],al ; 07/04/2024
23671 000004DE 9D          popf
23672 ;
23673 ; cmp ax,2 ; 0 = 8086, 1 = 286, 2 = 386
23674 000004DF 3C02      cmp     al,2

```

```

23675 000004E1 7512      jnz      short not_386_system
23676 000004E3 FC        cld                ; 80386
23677 000004E4 1E        push     ds
23678 000004E5 07        pop      es          ; change A20 line on/off check code
23679 000004E6 BF[4D07]  mov     di,cpu386_cmpsd
23680 000004E9 B8B904  mov     ax,04B9h      ; mov cx,4 ; B90400
23681 000004EC AB        stosw
23682 000004ED B800F3  mov     ax,0F300h     ; repz ; F3
23683 000004F0 AB        stosw
23684 000004F1 B866A7  mov     ax,0A766h     ; cmpsd ; 66A7
23685 000004F4 AB        stosw
23686 not_386_system:
23687 ;pop     di
23688 ;pop     ax
23689 ;pop     es
23690 ;;;
23691
23692 000004F5 8C0E[DB07]  mov     [MoveDOSIntoHMA+2],cs ; set seg of routine to move DOS
23693 000004F9 C606[DD07]01  mov     byte [SysinitPresent],1 ; flag that MoveDOSIntoHMA can be called
23694
23695 ; first move the MSDOS.SYS image up to a harmless place
23696 ; on top of our new sysinitseg
23697
23698 ; 22/10/2022
23699 000004FE B8[1054]  mov     ax,SI_end      ; how big is sysinitseg?
23700 00000501 E89608  call    off_to_para
23701 00000504 8CC9      mov     cx,cs          ; pick a buffer for msdos above us
23702 00000506 01C8      add     ax,cx
23703 00000508 8EC0      mov     es,ax
23704
23705 0000050A 31F6      xor     si,si
23706 0000050C 89F7      mov     di,si
23707
23708 0000050E 2E8E1E[7302]  mov     ds,[cs:CURRENT_DOS_LOCATION] ; where it is (set by msinit)
23709
23710 ;mov     ax,cs
23711 ;mov     ds,ax
23712
23713 ;;;mov   cx,20480 ; MSDOS 6.21 IO.SYS - SYSINIT:04E2h
23714 ;;;mov   cx,dossize/2 ; MSDOS 6.0
23715 ;mov     cx,[DOSSIZE] ; words (not bytes!) ; Retro DOS v4.0 (3.0, 2.0)
23716 ;mov     es,[FINAL_DOS_LOCATION] ; on top of SYSINIT code
23717 ;mov     ds,[CURRENT_DOS_LOCATION]
23718
23719 ; 22/10/2022
23720 00000513 B90058  mov     cx,DOSSIZE/2 ; 5000h
23721 ; 03/09/2023
23722 ; 5800h (PCDOS 7.1)
23723 00000516 F3A5      rep     movsw
23724 00000518 2E8C06[7302]  mov     [cs:CURRENT_DOS_LOCATION],es
23725
23726 ; The DOS code is ORGed at a non-zero value to allow it to be located in
23727 ; HIMEM. Thus, the DOS segment location must be adjusted accordingly.
23728 ; If this is ROMDOS, however, only the init code is loaded into RAM, so
23729 ; this ORG is not done. The entry point is at offset zero in the segment.
23730
23731 ; 22/04/2019 (MSDOS 6.0 & MSDOS 6.21 kernel address modification)
23732 ;mov     ax,cs
23733 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
23734 ;mov     ds,ax
23735
23736 ; 29/04/2019 - Retro DOS v4.0 ! important MODIFICATION !
23737
23738 ; ; 24/04/2019
23739 ;;ifndef ROMDOS
23740 ; mov     ax,[es:3] ; get offset of dos
23741 ; ; ax = 3DE0h for MSDOS 6.21 kernel (MSDOS.SYS, offset 3)
23742 ; mov     [dosinit],ax ; that's the entry point offset
23743 ; call    off_to_para ; subtract this much from segment
23744 ; ; 23/04/2019
23745 ; sub     [CURRENT_DOS_LOCATION],ax
23746 ; sub     [FINAL_DOS_LOCATION],ax
23747 ;;else
23748 ; mov     word [dosinit],0 ; entry to init is at zero
23749 ;;
23750 ;;endif ; ROMDOS
23751
23752 ; 29/04/2019 - Retro DOS v4.0 ! important MODIFICATION !
23753 ; (! MSDOS6.BIN starts with DOSDATA ! - Retro DOS v4.0 modification)
23754
23755 ;mov     ax,[es:0] ; DOSCODE start address = DOSDATA size (= 136Ah)
23756 ; ; (Valid for Retro DOS v4.0 only!)
23757
23758 ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
23759 ; (SYSINIT:0563h for MSDOS 5.0 IO.SYS SYSINIT)
23760 ; 03/09/2023 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS)
23761 ; (SYSINIT:04ECh for MSDOS 6.21 IO.SYS SYSINIT)
23762 ; (SYSINIT:0540h for PCDOS 7.1 IBMBIO.COM SYSINIT)
23763 0000051D A10300  mov     ax,[3] ; mov ax, word ptr ds:3
23764 ; 30/12/2023
23765 ; ax = 3F10h for IBMDOS 7.1 kernel
23766 ; (IBMDOS.SYS, offset 3)
23767
23768 00000520 2EA3[7102]  mov     [cs:dosinit],ax ; (SYSINIT:0563h for MSDOS 5.0 IO.SYS SYSINIT)
23769 ; 02/11/2022
23770 00000524 E87308  call    off_to_para ; subtract this much from segment
23771 00000527 2E2906[7302]  sub     [cs:CURRENT_DOS_LOCATION],ax
23772
23773 ; Current DOSCODE start address = dword [dosinit]
23774
23775 ;; If this is not ROMDOS, then the BIOS code is moved to the top of memory
23776 ;; until it is determined whether it will be running in HIMEM or not.
23777
23778 ;ifndef ROMDOS
23779
23780 ; now put Bios_Code up on top of that. Assume Bios_Code + dossize < 64k
23781
23782 ; 22/10/2022
23783 0000052C 8CC0      mov     ax,es
23784 0000052E 05000B  add     ax,DOSSIZE/16 ; get paragraph of end of dos
23785 00000531 8EC0      mov     es,ax
23786 00000533 2E8706[8902]  xchg    ax,[cs:temp_bcode_seg] ; swap with original home of Bios_Code
23787 00000538 8ED8      ds,ax ; point to loaded image of Bios_Code
23788
23789 ;mov     si,BCODE_START ; mov si,30h
23790 ; 09/12/2022
23791 0000053A BE[3000]  mov     si,BCODESTART
23792 ; 02/11/2022
23793 0000053D 89F7      mov     di,si
23794 ; 30/12/2023
23795 ;mov     cx,1E00h ; BCODE_END = (SYSINITSEG-DOSBIOCODESEG)*16
23796 ; ; (544h-364h)*10h = 1E00h (for PCDOS 7.1 IBMBIO.COM)
23797 ;mov     cx,BCODE_END ; mov cx,1A60h ; mov cx,1A70h ; 30/12/2022
23798 ;sub     cx,si

```

```

23799          ; 31/03/2024
23800          BCODESIZE equ BCODEEND-BCODESTART
23801 0000053F B9401D      mov     cx,BCODESIZE
23802 00000542 D1E9        shr     cx,1
23803 00000544 F3A5        rep     movsw          ; move Bios_Code into place
23804
23805 00000546 8CC0        mov     ax,es          ; tell it what segment it's in
23806 00000548 2EFF1E[8702] call    far [cs:seg_reinit_ptr] ; far call to seg_reinit in Bios_Code (M022)
23807
23808          ;endif          ; not ROMDOS
23809
23810          ; now call dosinit while it's in its temporary home
23811
23812          ;mov     ax,cs
23813          ;mov     ds,ax
23814
23815          ;mov     dx,[MEMORY_SIZE]          ; set for call to dosinit
23816
23817          ; 22/10/2022
23818
23819 0000054D 2EC43E[9503]    les     di,[cs:BiosComBlock] ; ptr to BIOS communication block
23820          ; es = KERNEL_SEGMENT (70h), di = 'SysInitPresent' address
23821 00000552 2EC536[7502]    lds     si,[cs:DEVICE_LIST] ; set for call to dosinit
23822          ; ds = KERNEL_SEGMENT (70h), si = 'res_dev_list' address
23823
23824 00000557 2E8B16[9402]    mov     dx,[cs:MEMORY_SIZE] ; set for call to dosinit
23825
23826 0000055C FA             cli
23827 0000055D 8CC8        mov     ax,cs
23828 0000055F 8ED0        mov     ss,ax
23829
23830          ; 30/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
23831          ; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0, SYSINIT1.ASM)
23832          %define locstack ($ - SYSINIT$) & 0FFFEh ; 532h in MSDOS 6.21 IO.SYS
23833          ; 5A6h in MSDOS 5.0 IO.SYS SYSINIT
23834          ; 586h in PCDOS 7.1 IBMBIO.COM SYSINIT
23835
23836          ;SYSINIT:0532h:
23837
23838          ; 22/10/2022
23839          ;-----
23840          ;SYSINIT:05A6h:
23841          ;locstack: ; (at SYSINIT:05A6h for MSDOS 5.0 IO.SYS)
23842
23843          ; 03/09/2023
23844          ; (locstack at SYSINIT:0586h in PCDOS 7.1 IBMBIO.COM SYSINIT)
23845
23846 00000561 BC6005        ;mov     sp,05A6h
23847          mov     sp,locstack          ; set stack
23848
23849 00000564 FB             sti
23850
23851          ;align 2
23852          ; 30/03/2018
23853          ;LOCSTACK:
23854          ;CALL     FAR [CS:MSDOS] ; FINAL_DOS_LOCATION:0
23855          ;('jmp DOSINIT' in 'MSHEAD.ASM')
23856          ;('DOSINIT:' is in 'MSINIT.ASM')
23857
23858          ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
23859          ; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0, 6.21)
23860
23861          ; This call to DOSINIT will relocate the DOS data from its present location
23862          ; at the top of memory, to its final location in low memory just above the
23863          ; BIOS data. It will then build important DOS data structures in low
23864          ; memory following the DOS data. It returns (among many other things) the
23865          ; new starting address of free memory.
23866 00000565 2EFF1E[7102]    call    far [cs:dosinit]          ; call dosinit
23867          ; es:di -> sysinitvars_ext
23868
23869 0000056A 2E8C1E[8502]    mov     [cs:def_php],ds          ; save pointer to PSP
23870
23871          ; 11/12/2022
23872          ; 22/03/2019
23873 0000056F 0E             push    cs
23874 00000570 1F             pop     ds
23875          ; 22/10/2022
23876 00000571 A3[8302]        mov     [hi_doscod_size],ax
23877 00000574 890E[8102]        mov     [lo_doscod_size],cx
23878 00000578 8916[7D02]        mov     [dos_segrenit],dx
23879
23880          ; 11/12/2022
23881          ; ds = cs
23882          ;mov     [cs:hi_doscod_size],ax; size of doscode (including exepatch)
23883          ;mov     [cs:lo_doscod_size],cx; (not including exepatch)
23884          ;mov     [cs:dos_segrenit],dx ; save offset of segrenit
23885
23886          ; 05/06/2018 - Retro DOS v3.0
23887          ; ES:DI = Address of pointer to SYSINITVARS structure (MSDOS 3.3)
23888
23889          ; 11/12/2022
23890          ; ds = cs
23891          ; 22/10/2022
23892          ;mov     ax,[es:di+SysInitVars.Ext.SYSI_InitVars] ; 5/29/86
23893 0000057C 268B05        mov     ax,[es:di] ; 22/03/2019
23894          ;mov     [cs:DOSINFO],ax
23895 0000057F A3[6D02]        mov     [DOSINFO],ax
23896          ;mov     ax,[es:di+SysInitVars.Ext.SYSI_InitVars+2]
23897 00000582 268B4502        mov     ax,[es:di+2]
23898          ;mov     [cs:DOSINFO+2],ax
23899 00000586 A3[6F02]        mov     [DOSINFO+2],ax ; set the sysvar pointer
23900
23901          ;mov     ax,[es:di+SysInitVars.Ext.SYSI_Country_Tab]
23902 00000589 268B4504        mov     ax,[es:di+4]
23903          ;mov     [cs:sysi_country],ax
23904 0000058D A3[7902]        mov     [sysi_country],ax
23905          ;mov     ax,[es:di+SysInitVars.Ext.SYSI_Country_Tab+2]
23906 00000590 268B4506        mov     ax,[es:di+6]
23907          ;mov     [cs:sysi_country+2],ax
23908 00000594 A3[7B02]        mov     [sysi_country+2],ax ; set the SYSI_Country pointer
23909
23910          ; 20/04/2019
23911          ;mov     ax,[CURRENT_DOS_LOCATION]
23912          ;;mov     es,[CURRENT_DOS_LOCATION]
23913          ;mov     ax,[FINAL_DOS_LOCATION] ; give dos its temporary location
23914          ; 22/10/2022
23915          ;mov     ax,[cs:CURRENT_DOS_LOCATION]
23916          ;;mov     [dos_segrenit+2],es
23917          ;;mov     [dos_segrenit+2],ax
23918          ;mov     [cs:dos_segrenit+2],ax
23919          ; 11/12/2022
23920          ; ds = cs
23921 00000597 8E06[7302]        mov     es,[CURRENT_DOS_LOCATION]
23922 0000059B 8C06[7F02]        mov     [dos_segrenit+2],es

```

```

23923             ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
23924             ;mov     es,[cs:CURRENT_DOS_LOCATION]
23925             ;mov     [cs:dosegreinit+2],es
23926
23927             ; -----
23928
23929             ;SYSINIT:0577h:
23930             ; ... RPLArena ... MSDOS 6.21 IO.SYS (SYSINIT:0577h to SYSINIT:05D1h)
23931             ;SYSINIT:05D1h:      ; NoRPLArena
23932
23933             ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS SYSINIT)
23934             ;----- Cover up RPL code with an arena
23935             ;SYSINIT:05EBh:
23936             ; 11/12/2022
23937             ; ds = cs
23938             xor     bx,bx
23939             cmp     [RPLMemTop],bx ; 0
23940             ;cmp     word [RPLMemTop],0
23941             ;;cmp     word [cs:RPLMemTop],0
23942             je      short NoRPLArena
23943
23944             ;----- alloc all memory
23945
23946             ; 11/12/2022
23947             ;mov     bx,0FFFFh
23948             dec     bx
23949             ; bx = 0FFFFh
23950             mov     ah,48h
23951             int     21h
23952             ; DOS - 2+ - ALLOCATE MEMORY
23953             ; BX = number of 16-byte paragraphs desired
23954             mov     ah,48h
23955             int     21h
23956
23957             mov     es,ax
23958             push    es
23959             ; get it into ES and save it
23960
23961             ;----- resize upto RPL mem
23962
23963             ; 11/12/2022
23964             ; ds = cs
23965             sub     ax,[cs:RPLMemTop]
23966             sub     ax,[RPLMemTop]
23967             neg     ax
23968             dec     ax
23969             mov     bx,ax
23970             mov     ah,4Ah
23971             int     21h
23972             ; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
23973             ; ES = segment address of block to change
23974             ; BX = new size in paragraphs
23975
23976             ;----- allocate the free (RPL MEM)
23977             mov     bx,0FFFFh
23978             mov     ah,48h
23979             int     21h
23980             mov     ah,48h
23981             int     21h
23982
23983             ;----- mark that it belongs to RPL
23984
23985             dec     ax
23986             mov     es,ax
23987             ;mov     word [es:arena_owner],8
23988             mov     word [es:1],8
23989             ;mov     word [es:arena_name],'RP'
23990             mov     word [es:8],'RP'
23991             ;mov     word [es:arena_name+2],'L'
23992             mov     word [es:10],'L'
23993             ;mov     word [es:arena_name+4],0
23994             mov     word [es:12],0
23995             ;mov     word [es:arena_name+6],0
23996             mov     word [es:14],0
23997
23998             pop     es
23999             mov     ah,49h
24000             int     21h
24001             ; get back ptr to first block
24002             ; Dealloc
24003             ; and free it
24004             ; DOS - 2+ - FREE MEMORY
24005             ; ES = segment address of area to be freed
24006
24007             ; 11/12/2022
24008             cld
24009
24010             ; -----
24011
24012             ;-----
24013
24014             ;-----
24015
24016             ;-----
24017
24018             ;-----
24019
24020             ;-----
24021
24022             ;-----
24023
24024             ;-----
24025             ; 11/12/2022
24026             ; ds = cs
24027             ; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0, 6.21, IO.SYS)
24028             ; les     di,[DOSINFO] ; es:di -> dosinfo
24029             ; 22/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS SYSINIT)
24030             ; les     di,[cs:DOSINFO] ; es:di -> dosinfo
24031
24032             ; 11/12/2022
24033             ; cld
24034             ; get the extended memory size
24035
24036             ; execute the get extended memory size subfunction in the bios int 15h
24037             ; if the function reports an error do nothing else store the extended
24038             ; memory size reported at the appropriate location in the dosinfo buffer
24039             ; currently pointed to by es:di. use the offsets specified in the
24040             ; definition of the sysinitvars struct in inc\sysvar.inc
24041
24042             mov     ah,88h
24043             int     15h
24044             ; check extended memory size
24045             jc      short no_ext_memory
24046             ; Get Extended Memory Size
24047             ; Return: CF clear on success
24048             ; AX = size of memory above 1M in K
24049             ;mov     [es:di+SYSI_EXT_MEM],ax ; save extended memory size
24050             ; 22/10/2022
24051             mov     [es:di+45h],ax ; save extended memory size
24052             or      ax,ax
24053             jz      short no_ext_memory
24054             call    C1rVDISKHeader
24055             no_ext_memory:
24056             ;mov     ax,[es:di+SYSI_MAXSEC]; get the sector size
24057             mov     ax,[es:di+10h]
24058             ;add     ax,bufinsiz
24059             ; 30/12/2023 - Retro DOS v5.0
24060             ;add     ax,20
24061             ; size of buffer header
24062             add     ax,24
24063             ; bufinsiz
24064             ; size of buffer header = 24 (PCDOS v7.1 IBMBIO.COM)
24065             ; (it was 20 in MSDOS 6.22 IO.SYS)
24066             ; 11/12/2022

```

```

24047             ; ds = cs
24048 00000613 A3[9D02] mov     [singlebuffersize],ax ; total size for a buffer
24049             ;mov     [cs:singlebuffersize],ax
24050             ; 11/12/2022
24051 00000616 A0[9802] mov     al,[DEFAULT_DRIVE] ; get the 1 based boot drive number set by msinit
24052             ;mov     al,[cs:DEFAULT_DRIVE]
24053             ;mov     [es:di+SYSI_BOOT_DRIVE],al ; set sysi_boot_drive
24054 00000619 26884543 mov     [es:di+43h],al
24055
24056             ; determine if 386 system...
24057
24058             ; 30/12/2023
24059 %if 0
24060             ;get_cpu_type                ; macro to determine cpu type
24061
24062 get_cpu_type:
24063             ; 11/12/2022
24064             pushf
24065             ;push     bx
24066             ;xor      bx,bx
24067             ; 11/12/2022
24068             ;xor      cx,cx
24069             ;
24070             xor      ax,ax
24071             ; ax = 0
24072             push     ax
24073             popf
24074             pushf
24075             pop      ax
24076             and     ax,0F000h
24077             ;cmp     ax,0F000h
24078             cmp     ah,0F0h
24079             je      short cpu_8086
24080             ;mov     ax,0F000h
24081             mov     ah,0F0h
24082             ; ax = 0F000h
24083             push     ax
24084             popf
24085             pushf
24086             pop      ax
24087             ;and     ax,0F000h
24088             and     ah,0F0h
24089             jz      short cpu_286
24090 cpu_386:
24091             ; 11/12/2022
24092             ;inc     bx
24093             ;inc     cx
24094             ; 11/12/2022
24095             ;mov     byte [es:di+SYSI_DWMOVE],1
24096             mov     byte [es:di+44h],1
24097
24098             ; 03/09/2023 - Retro DOS v5.0 (PCDOS 7.1 Modified SYSINIT)
24099             ; change A20 line on/off check code to the faster (for 32 bit cpu)
24100             ;push     es
24101             ;push     di
24102             ;mov     ax,DOSBIODATASEG ; 0070h
24103             mov     es,ax
24104             ;cld
24105             ;mov     di,cpu386_cmpsd ; (IsA20Off)
24106             mov     ax,4B9h ; mov cx,4 ; B90400
24107             ;stosw
24108             mov     ax,0F300h ; repz ; F3
24109             ;stosw
24110             mov     ax,0A766h ; cmpsd ; 66A7
24111             ;stosw
24112             ;pop     di
24113             ;pop     es
24114
24115 cpu_286:
24116             ;inc     bx
24117             ;inc     cx
24118 cpu_8086:
24119             ; 11/12/2022
24120             ;mov     ax,bx
24121             pop      bx
24122             popf
24123 %endif
24124             ;...
24125
24126             ; 11/12/2022
24127             ;or      cl,cl
24128             ;jz      short not_386_system
24129             ; 11/12/202
24130             ;cmp     cl,2
24131             ;cmp     ax,2 ; is it a 386?
24132             ;jne     short not_386_system ; no: don't mess with flag
24133
24134             ; 30/12/2023 - Retro DOS v5.0
24135 0000061D 803E[B606]02 cmp     byte [cpu_type], 2 ; is it a 386?
24136 00000622 7505 jne     short _not_386_cpu ; no: don't mess with flag
24137
24138             ;mov     byte [es:di+SYSI_DWMOVE],1
24139             ; 11/12/2022
24140             ; 22/10/2022
24141 00000624 26C6454401 mov     byte [es:di+44h],1
24142 _not_386_cpu:
24143             ;mov     al,[es:di+SYSI_NUMIO]
24144 00000629 268A4520 mov     al,[es:di+20h]
24145             ; 11/12/2022
24146             ; ds = cs
24147 0000062D A2[8503] mov     [drivenumber],al ; save start of installable block drvs
24148             ;mov     [cs:drivenumber],al
24149
24150             mov     ax,cs
24151 00000632 83E811 sub     ax,11h ; room for PSP we will copy shortly
24152             ; 11/12/2022
24153             ;mov     cx,[singlebuffersize] ; temporary single buffer area
24154             ;mov     cx,[cs:singlebuffersize]
24155             shr     cx,1
24156             shr     cx,1 ; divide size by 16...
24157             shr     cx,1
24158             shr     cx,1 ; ...to get paragraphs...
24159             inc     cx ; ... and round up
24160             ; 11/12/2022
24161 00000635 8B1E[9D02] mov     bx,[singlebuffersize]
24162 00000639 B104 mov     cl,4
24163 0000063B D3EB shr     bx,cl
24164 0000063D 43 inc     bx
24165
24166             ; cas note: this unorthodox paragraph rounding scheme wastes a byte
24167             ; if [singlebuffersize] ever happens to be zero mod 16. Could this
24168             ; ever happen? Only if the buffer overhead was zero mod 16, since
24169             ; it is probably safe to assume that the sector size always will be.
24170

```

```

24171 ; mohans also found a bug in CONFIG.SYS processing where it replaces
24172 ; EOF's with cr,lf's, without checking for collision with [confbot].
24173 ; perhaps the extra byte this code guarantees is what has kept that
24174 ; other code from ever causing a problem???
24175 ;
24176 ; 11/12/2022
24177 0000063E 29D8 sub ax,bx
24178 ;sub ax,cx
24179 00000640 A3[A702] mov [top_of_cdss],ax ; temp "unsafe" location
24180 ; 22/10/2022
24181 ;mov [cs:top_of_cdss],ax
24182 ;
24183 ; chuckst -- 25 Jul 92 -- added code here to pre-allocate space
24184 ; for 26 temporary CDSSs, which makes it easier to use allocim
24185 ; for allocating memory for MagicDrv.
24186 ;
24187 ; 30/12/2023
24188 ;push es ; not necessary (!*) ; preserve pointer to dosinfo
24189 ;push di
24190 ;
24191 ; 22/10/2022
24192 ; mov cx,ax ; save pointer for buffer
24193 ;
24194 ;
24195 ; now allocate space for 26 CDSS
24196 ;
24197 ; sub ax,((26*(curdirlen))+15)/16
24198 ; mov [ALLOCLIM],ax ; init top of free memory pointer
24199 ; mov [CONFBOT],ax ; init this in case no CONFIG.SYS
24200 ;
24201 ; 30/12/2022 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS)
24202 ; (SYSINIT:064Ch)
24203 00000643 89C1 mov cx,ax ; (*)
24204 00000645 2B8F00 sub ax,((26*(curdirlen))+15)/16 ; sub ax,143
24205 00000648 A3[A502] mov [ALLOCLIM],ax ; init top of free memory pointer
24206 0000064B A3[A302] mov [CONFBOT],ax ; init this in case no CONFIG.SYS
24207 ;
24208 ; setup and initialize the temporary buffer at cx
24209 ;
24210 0000064E 26C47D12 ;les di,[es:di+SYSI_BUF] ; get the buffer chain entry pointer
24211 ;les di,[es:di+12h]
24212 ; 11/12/2022
24213 00000652 31DB xor bx,bx
24214 ;xor ax,ax
24215 ;mov [es:di+BUFFINF.Dirty_Buff_Count],ax ; 0
24216 00000654 26895D04 ;mov word [es:di+4],0
24217 ;mov [es:di+4],bx ; 0
24218 ;mov [es:di+BUFFINF.Buff_Queue],ax ; 0
24219 00000658 26891D ;mov word [es:di],0
24220 ;mov [es:di],bx ; 0
24221 ;mov [es:di+BUFFINF.Buff_Queue+2],cx ; cx = [top_of_cdss] ; 6.21
24222 ;mov [es:di+BUFFINF.Buff_Queue+2],ax ; ax = [top_of_cdss] ; 5.0
24223 ;mov [es:di+2],ax
24224 ;mov es,ax ; [top_of_cdss] = [CONFBOT]
24225 0000065B 26894D02 ; 30/12/2022 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS, SYSINIT)
24226 0000065F 8EC1 mov [es:di+2],cx ; [top_of_cdss] ; (*)
24227 ;mov es,cx
24228 ;
24229 ; 11/12/2022
24230 ;xor ax,ax
24231 00000661 89DF ;mov di,ax ; es:di -> single buffer
24232 ;mov di,bx
24233 ; di = 0
24234 ;mov [es:di+buffinfo.buf_next],ax ; points to itself
24235 ; 11/12/2022
24236 ;mov [es:di],ax ; 0
24237 00000663 26891D ;mov [es:di],bx ; 0
24238 ;mov [es:di+buffinfo.buf_prev],ax ; points to itself
24239 ; 11/12/2022
24240 ;mov [es:di+2],ax ; 0
24241 00000666 26895D02 ;mov [es:di+2],bx ; 0
24242 ;
24243 ; 22/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS SYINIT)
24244 ; MSDOS 5.0 IO.SYS - SYSINIT:06E0h
24245 ;
24246 ;mov word [es:di+buffinfo.buf_ID],00FFh ; free buffer,clear flag
24247 0000066A 26C74504FF00 ;mov word [es:di+4],00FFh
24248 ;SYSINIT:06E6h
24249 ;mov [es:di+buffinfo.buf_sector],ax ; 0
24250 ;mov word [es:di+6],0
24251 ; 11/12/2022
24252 ;mov [es:di+buffinfo.buf_sector],bx ; 0
24253 00000670 26895D06 ;mov [es:di+6],bx ; 0
24254 ;mov [es:di+buffinfo.buf_sector+2],ax ; 0
24255 ;mov word [es:di+8],0
24256 ; 11/12/2022
24257 ;mov [es:di+buffinfo.buf_sector+2],bx ; 0
24258 00000674 26895D08 ;mov [es:di+8],bx ; 0
24259 ;
24260 ; 30/12/2023 (!*)
24261 ;pop di ; restore pointer to DOSINFO data
24262 ;pop es
24263 ;
24264 ; 11/12/2022
24265 ; ds = cs
24266 ; 22/10/2022
24267 ;push cs
24268 ;pop ds
24269 ;
24270 00000678 E82807 call TempCDS ; set up cdss so re_init and sysinit
24271 ; can make disk system calls
24272 ; tempcds trashes ds
24273 ;
24274 0000067B 2E8E1E[8502] ; 10/05/2019
24275 ;mov ds,[cs:def_php] ; retrieve pointer to PSP returned by DOSINIT
24276 ;
24277 ;if not ibmjapver
24278 ;call far KERNEL_SEGMENT:re_init ; re-call the bios
24279 ;endif
24280 ;
24281 ; 22/10/2022
24282 ;SYSINIT:06FEh: ; (MSDOS 5.0 IO.SYS, SYSINIT)
24283 ; 30/12/2022
24284 ;SYSINIT:0697h: ; (MSDOS 6.21 IO.SYS, SYSINIT)
24285 00000680 9A[2807]7000 ;call far ptr 70h:89Bh
24286 ;call DOSBIODATASEG:RE_INIT
24287 00000685 FB ; sti ; ints ok
24288 00000686 FC ; cld ; make sure
24289 ;
24290 ; 23/03/2019
24291 ;
24292 ;SYSINIT:069Eh ; 30/12/2022
24293 ;
24294 ; dosinit has set up a default "process" (php) at ds:0. we will move it out

```



```

24295 ; of the way by putting it just below sysinit at end of memory.
24296
24297 00000687 8CCB      mov     bx,cs
24298 00000689 83EB10    sub     bx,10h
24299 0000068C 8EC3      mov     es,bx
24300 0000068E 31F6      xor     si,si
24301 00000690 89F7      mov     di,si
24302 00000692 B98000    mov     cx,128
24303 00000695 F3A5      rep     movsw
24304
24305 ;mov     [es:PDB.JFN_POINTER+2],es ; Relocate
24306 ; 22/10/2022
24307 00000697 268C063600 mov     [es:36h],es
24308
24309 ; Set Process Data Block - Program Segment Prefix address
24310 ; BX = PDB/PSP segment
24311 0000069C B450      mov     ah,50h ; SET_CURRENT_PDB
24312 0000069E CD21      int     21h      ; tell DOS we moved it
24313 ; DOS - 2+ internal - SET PSP SEGMENT
24314 ; BX = segment address of new PSP
24315 ; 22/10/2022
24316 ; 27/03/2019
24317 ; 30/12/2023
24318 ;push    ds ; */
24319 ; preserve DS returned by DOSINIT
24320 000006A0 0E      push    cs
24321 000006A1 1F      pop     ds
24322
24323 ; set up temp. critical error handler
24324 000006A2 BA[774A]  mov     dx,int24 ; set up int 24 handler
24325 ;mov     ax,(SET_INTERRUPT_VECTOR*256)+24h
24326 ;mov     ax,(SET_INTERRUPT_VECTOR<<8)|24h
24327 000006A5 B82425  mov     ax,2524h
24328 000006A8 CD21      int     21h
24329
24330 000006AA 803E[8803]00    cmp     byte [toomanydrivesflag],0 ; Q: >24 partitions? M029
24331 000006AF 7406      je      short no_err ; N: continue M029
24332 000006B1 BA[9C53]  mov     dx,TooManyDrivesMsg ; Y: print error message M029
24333 ; 22/10/2022
24334 ;call    print ; M029
24335 ; 12/12/2022
24336 000006B4 EB04      jmp     short p_dosinit_msg ; 23/03/2019 - Retro DOS v4.0
24337
24338 ; 30/12/2023 - Retro DOS v5.0
24339 cpu_type:
24340 000006B6 FF      db 0FFh ; db 0
24341
24342 no_err:
24343 ; 12/05/2019
24344 ;-----
24345 ; 27/06/2018 - Retro DOS v3.0 ; 23/03/2019 - Retro DOS v4.0
24346 ; 22/10/2022 - Retro DOS v4.0
24347 ; 12/12/2022
24348 ; 30/12/2023 - Retro DOS v5.0
24349 000006B7 BA[7B4A]  mov     dx,BOOTMES ; Display (fake) MSDOS version message
24350
24351 000006BA E89543  p_dosinit_msg:
24352 call     print ; Print message
24353 ;-----
24354 ; 11/12/2022
24355 ; 22/10/2022
24356 ; 23/03/2019 - Retro DOS v4.0
24357 ;pop     ds
24358 ;mov     dl,[cs:DEFAULT_DRIVE] ; start of free memory
24359
24360 ; 11/12/2022
24361 ; 27/03/2019
24362 000006BD 8A16[9802]  mov     dl,[DEFAULT_DRIVE]
24363 ; 30/12/2023
24364 ;pop     ds ; */
24365
24366 000006C1 08D2      or      dl,dl
24367 ; 30/12/2023
24368 000006C3 7405      jz      short nodrvset ; bios didn't say
24369 ;jz      short ProcessConfig ; (Retro DOS v4.0 does not contain DBLSPACE code)
24370 ;dec     dl ; A = 0
24371 ; 18/12/2022
24372 000006C5 4A      dec     dx
24373 000006C6 B40E      mov     ah,0Eh ; SET_DEFAULT_DRIVE
24374 000006C8 CD21      int     21h ; select the disk
24375 ; DOS - SELECT DISK
24376 ; DL = new default drive number (0 = A, 1 = B, etc.)
24377 ; Return: AL = number of logical drives
24378
24379 nodrvset:
24380 ; 04/01/2023 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS SYINIT)
24381 ; (SYSINIT:06DFh)
24382 ; 30/12/2023 - Retro DOS 5.0
24383 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:0733h)
24384 000006CA 1E      push    ds
24385 000006CB 29C0      sub     ax,ax
24386 000006CD 8ED8      mov     ds,ax ; 0 ; ROM BIOS Data Area
24387 000006CF A16C04    mov     ax,[46Ch] ; timer tick count (18.2 ticks per second)
24388 ;mov     [cs:_timer_lw_],ax
24389 000006D2 1F      pop     ds
24390 ; ds = cs
24391 000006D3 A3[8C03]  mov     [_timer_lw_],ax
24392
24393 ; -----
24394 ;ifdef dbldspace_hooks
24395 ;
24396 ;     ....
24397 ;     ....
24398 ;endif
24399
24400 ; -----
24401
24402 ; 30/12/2023 - Retro DOS 5.0 (Modified MSDOS 7.1 IBMBIO.COM SYS SYINIT)
24403 ; -----
24404 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0740h
24405
24406 000006D6 0E      push    cs
24407 000006D7 07      pop     es
24408
24409 ; 07/04/2024
24410 ;mov     word [cs:MagicBackdoor+2],cs
24411 ;mov     word [cs:MagicBackdoor], NullBackdoor
24412 000006D8 8C0E[9203]  mov     word [MagicBackdoor+2],cs
24413 000006DC C706[9003][9403]  mov     word [MagicBackdoor], NullBackdoor
24414
24415 ; ds = es = cs = SYSINIT segment
24416 set_drvspc_size:
24417 000006E2 BE[AB16]  mov     si,MagicDDName ; "\DBLSPACE.BIN"
24418 set_dblspc_size:

```

```

24419 000006E5 E8772F      call    SizeDevice
24420 000006E8 732B      jnc     short wait_for_key_2s
24421                ;cmp    byte [cs:si], 'C'
24422 000006EA 803C43      cmp     byte [si], 'C' ; "C:\STACKER.BIN"
24423 000006ED 740C      je      short set_drvspc_name
24424                ;cmp    byte [cs:DEFAULT_DRIVE], 3
24425 000006EF 803E[9802]03    cmp     byte [DEFAULT_DRIVE], 3
24426 000006F4 7405      je      short set_drvspc_name
24427 000006F6 83EE02      sub     si, 2 ; "C:\DBLSPACE.BIN"
24428 000006F9 EBEA      jmp     short set_dblspc_size
24429
24430                set_drvspc_name:
24431                ;cmp     byte [cs:MagicDDName+2], 'R' ; "BLSPACE.BIN"
24432 000006FB 803E[AD16]52    cmp     byte [MagicDDName+2], 'R'
24433 00000700 7408      je      short set_stack_name
24434                ;mov     word [cs:MagicDDName+2], 'RV' ; "DRVSPACE.BIN"
24435 00000702 C706[AD16]5256    mov     word [MagicDDName+2], 'RV'
24436 00000708 EBD8      jmp     short set_drvspc_size
24437
24438                set_stack_name:
24439 0000070A 81FE[B916]      cmp     si, StackerName ; "C:\STACKER.BIN"
24440 0000070E 734B      jnb     short wfk2s_4
24441 00000710 BE[BB16]      mov     si, StackerName+2 ; "\STACKER.BIN"
24442 00000713 EBD0      jmp     short set_dblspc_size
24443
24444                wait_for_key_2s:
24445                ;mov     [cs:MagicDDNamePtr], si
24446 00000715 8936[A716]      mov     [MagicDDNamePtr], si
24447 00000719 1E      push    ds
24448 0000071A 29C0      sub     ax, ax
24449 0000071C 8ED8      mov     ds, ax ; 0 ; ROMBIOS data area
24450 0000071E 8B166C04      mov     dx, [46Ch] ; Counter for Interrupt 1Ah
24451                wfk2s_1:
24452 00000722 B401      mov     ah, 1
24453 00000724 CD16      int     16h ; KEYBOARD - CHECK BUFFER, DO NOT CLEAR
24454                ; Return: ZF clear if character in buffer
24455                ; AH = scan code, AL = character
24456                ; ZF set if no character in buffer
24457 00000726 7511      jnz     short wfk2s_2
24458 00000728 B402      mov     ah, 2
24459 0000072A CD16      int     16h ; KEYBOARD - GET SHIFT STATUS
24460                ; AL = shift status bits
24461 0000072C A803      test    al, 3
24462 0000072E 7509      jnz     short wfk2s_2
24463 00000730 A16C04      mov     ax, [46Ch] ; tick count
24464 00000733 29D0      sub     ax, dx
24465 00000735 3C25      cmp     al, 37 ; 2 seconds
24466 00000737 72E9      jb      short wfk2s_1 ; wait for user's key press
24467                wfk2s_2:
24468 00000739 1F      pop     ds ; read/check the pressed key
24469 0000073A 29DB      sub     bx, bx ; bx = 0
24470 0000073C B402      mov     ah, 2
24471 0000073E CD16      int     16h ; KEYBOARD - GET SHIFT STATUS
24472                ; AL = shift status bits
24473 00000740 A803      test    al, 3 ; Left or Right SHIFT key pressed ?
24474 00000742 7402      jz      short wfk2s_3 ; no
24475 00000744 43      inc     bx
24476 00000745 43      inc     bx ; bx = 2
24477                wfk2s_3:
24478 00000746 B401      mov     ah, 1
24479 00000748 CD16      int     16h ; KEYBOARD - CHECK BUFFER, DO NOT CLEAR
24480                ; Return: ZF clear if character in buffer
24481                ; AH = scan code, AL = character
24482                ; ZF set if no character in buffer
24483 0000074A 7418      jz      short wfk2s_6
24484 0000074C 80FC65      cmp     ah, 65h ; F8 key pressed ?
24485 0000074F 740C      jz      short wfk2s_5
24486 00000751 80FC62      cmp     ah, 62h ; F5 key pressed ?
24487 00000754 750E      jnz     short wfk2s_6
24488                ;mov     byte [cs:F5_key], 1
24489 00000756 C606[8E03]01    mov     byte [F5_key], 1
24490                wfk2s_4:
24491 0000075B EB49      jmp     short ProcessConfig ; continue (as normal/default state)
24492
24493                wfk2s_5:
24494                ;mov     byte [cs:F8_key], 1
24495 0000075D C606[8F03]01    mov     byte [F8_key], 1
24496 00000762 EB42      jmp     short ProcessConfig
24497
24498                wfk2s_6:
24499 00000764 E8AA02      call    AllocFreeMem ; get the largest free block from DOS
24500 00000767 E8700F      call    MagicPreload ; **** PRE-LOAD MAGICDRV!!! ****
24501
24502                ; 07/04/2024 - Retro DOS v5.0
24503                ; (DS may not be same with CS here!)
24504 0000076A 0E      push    cs
24505 0000076B 1F      pop     ds ; *
24506 0000076C 8E06[6803]      mov     es, [area]
24507
24508 00000770 09C0      or      ax, ax ; error?
24509 00000772 7406      jz      short wfk2s_7
24510                preloadFailed:
24511 00000774 B449      mov     ah, 49h ; Dealloc ; free the block if no load
24512                ;mov     es, [cs:area]
24513                ;mov     es, [area]
24514 00000776 CD21      int     21h ; DOS - 2+ - FREE MEMORY
24515                ; ES = segment address of area to be freed
24516 00000778 EB2C      jmp     short ProcessConfig
24517
24518                wfk2s_7:
24519                ;mov     bx, [cs:memhi]
24520                ;mov     es, [cs:area]
24521                ;sub     bx, [cs:area] ; get desired block size in paras
24522                ; 07/04/2024 - Retro DOS v5.0
24523                ; ds = cs ; *
24524 0000077A 8CC3      mov     bx, es
24525 0000077C F7DB      neg     bx ; bx = - [cs:area]
24526 0000077E 031E[6403]      add     bx, [memhi] ; bx = [cs:memhi] - [cs:area]
24527
24528 00000782 B44A      mov     ah, 4Ah
24529 00000784 CD21      int     21h ; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
24530                ; ES = segment address of block to change
24531                ; BX = new size in paragraphs
24532 00000786 8CC0      mov     ax, es
24533 00000788 48      dec     ax
24534 00000789 8EC0      mov     es, ax ; get Magicdrv arena
24535
24536 0000078B 26C70601000800    mov     word [es:1], 8 ; [es:arena_owner]
24537                ;mov     word [es:ARENA.OWNER], 8 ; set impossible owner
24538 00000792 26C70608005344    mov     word [es:8], 4453h ; [es:arena_name], 'SD' ; System Data
24539                ;mov     word [es:ARENA.NAME], 'SD' ; 4453h
24540 00000799 2603060300      add     ax, [es:3] ; get MCB length
24541                ;add     ax, [es:ARENA.SIZE]
24542

```



```

24543      ;lds    si,[cs:DOSINFO]          ; get to arena header
24544 0000079E C536[6D02]      lds    si,[DOSINFO]
24545 000007A2 40              inc     ax          ; get addr of next MCB
24546 000007A3 8944FE          mov     [si-2], ax      ; store that
24547
24548 ; -----
24549
24550 ; MSDOS 6.21 IO.SYS, SYSINIT:0744h
24551
24552 ; 23/03/2019 - Retro DOS v4.0 (MSDOS 6.0, SYSINIT1.ASM, 1991)
24553 ; -----
24554 ; 22/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS SYSINIT)
24555 ; -----
24556 ; 30/12/2022 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS SYSINIT)
24557 ; -----
24558 ; 07/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM SYSINIT)
24559 ; -----
24560 ; (MSDOS 6.22 IO.SYS - SYSINIT:0744h)
24561
24562 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0820h
24563
24564 ProcessConfig:
24565     ;; ds = cs ; 27/03/2019
24566     ; 11/12/2022
24567     ; ds <> cs
24568
24569 ; (MSDOS 5.0 IO.SYS - SYSINIT:0746h)
24570
24571 000007A6 E8BD1C          call    doconf          ; do pre-scan for dos=high/low
24572
24573     ; 11/12/2022
24574     ; 27/03/2019
24575     ; ds = cs (at return from doconf)
24576
24577 ; Now, if this is not romdos, we decide what to do with the DOS code.
24578 ; It will either be relocated to low memory, above the DOS data structures,
24579 ; or else it will be located in HiMem, in which case a stub with the DOS
24580 ; code entry points will be located in low memory. Dos_segrenit is used
24581 ; to tell the DOS data where the code has been placed, and to install the
24582 ; low memory stub if necessary. If the DOS is going to go into HiMem, we
24583 ; must first initialize it in its present location and load the installable
24584 ; device drivers. Then, if a HiMem driver has been located, we can actually
24585 ; relocate the DOS code into HiMem.
24586 ;
24587 ; For ROMDOS, if DOS=HIGH is indicated, then we need to call dos_segrenit
24588 ; to install the low memory stub (this must be done before allowing any
24589 ; device drivers to hook interrupt vectors). Otherwise, we don't need to
24590 ; call dos_segrenit at all, since the interrupt vector table has already
24591 ; been patched.
24592
24593     ; 22/10/2022 - Retro DOS v4.0
24594     ; (MSDOS 5.0 IO.SYS - SYSINIT:0749h)
24595     ; cmp     byte [cs:runhigh],0      ; Did user choose to run low ?
24596     ; 11/12/2022
24597 000007A9 803E[6C02]00      cmp     byte [runhigh],0
24598 000007AE 7404             je      short dont_install_stub      ; yes, don't install dos low mem stub
24599
24600 ;----- user chose to load high
24601
24602     ; 22/10/2022
24603     ; mov     es,[cs:CURRENT_DOS_LOCATION] ; MSDOS 6.21 (& MSDOS 6.0)
24604     ; 11/12/2022
24605     ; ds = cs
24606 ; 13/04/2024
24607 %if 0
24608     mov     es,[CURRENT_DOS_LOCATION]
24609 %endif
24610     ; mov     es,[cs:FINAL_DOS_LOCATION] ; Retro DOS v4.0
24611     ; 27/03/2019
24612     ; mov     es,[FINAL_DOS_LOCATION]
24613
24614 000007B0 31C0             xor     ax,ax          ; ax = 0 ---> install stub
24615
24616 ; 13/04/2024
24617 %if 0
24618     ; 11/12/2022
24619     ; ds = cs
24620     ; call    far [cs:dos_segrenit]; call dos segrenit
24621     call    far [dos_segrenit]
24622 %endif
24623 000007B2 EB08             jmp     short do_multi_pass
24624
24625 ;----- User chose to load dos low
24626
24627 dont_install_stub:
24628     ; 22/10/2022
24629 000007B4 31DB             xor     bx,bx          ; M012
24630     ; don't use int 21 call to alloc mem
24631 000007B6 E80E03          call    MovDOSLo      ; move it !
24632
24633 000007B9 B80100          mov     ax,1          ; dont install stub
24634
24635 ; 13/04/2024
24636 %if 1
24637 do_multi_pass:
24638 %endif
24639     ; 11/12/2022
24640     ; ds = cs
24641 000007BC 8E06[7302]      mov     es,[CURRENT_DOS_LOCATION]
24642     ; mov     es,[cs:CURRENT_DOS_LOCATION] ; set_dos_final_position set it up
24643     ; mov     es,[cs:FINAL_DOS_LOCATION] ; Retro DOS v4.0
24644     ; 27/03/2019
24645 ;do_multi_pass:
24646     ; mov     es,[FINAL_DOS_LOCATION]
24647
24648     ; 11/12/2022
24649     ; ds =cs
24650     ; call    far [cs:dos_segrenit]; inform dos about new seg
24651 000007C0 FF1E[7D02]      call    far [dos_segrenit]
24652
24653 ; 13/04/2024
24654 %if 0
24655 do_multi_pass:
24656 %endif
24657
24658 000007C4 E84A02          call    AllocFreeMem    ; allocate all the free mem
24659     ; & update [memhi] & [area]
24660     ; start of free memory.
24661
24662 ;ifdef dblspace_hooks
24663 ;mov     bx,0          ; magic backdoor to place int hooks
24664 ;call    cs:MagicBackdoor
24665 ;endif
24666
24667 ; 07/04/2024 - Retro DOS v5.0

```

```

24667 ; (PCDOS 7.1 IBMBIO.COM)
24668 ; cmp byte [cs:F5_key],1
24669 000007C7 803E[8E03]01 cmp byte [F5_key],1
24670 000007CC 740D je short skip_magicbackdoor
24671 ; cmp byte [cs:F8_key],1
24672 000007CE 803E[8F03]01 cmp byte [F8_key],1
24673 000007D3 7406 je short skip_magicbackdoor
24674 000007D5 31DB xor bx,bx ; bx = 0 ; magic backdoor to place int hooks
24675 ; call far [cs:MagicBackdoor]
24676 000007D7 FF1E[9003] call far [MagicBackdoor]
24677
24678 skip_magicbackdoor:
24679
24680 ; Now, process config.sys some more.
24681 ; Load the device drivers and install programs
24682
24683 ; 22/10/2022
24684 ; inc byte [cs:multi_pass_id] ; multi_pass_id = 1
24685 ; 11/12/2022
24686 ; ds = cs
24687 000007DB FE06[CD02] inc byte [multi_pass_id]
24688 000007DF E8201D call multi_pass ; load device drivers
24689 000007E2 E8E831 call ShrinkUMB
24690 000007E5 E80C32 call unLinkUMB ; unlink all UMBs ; M002
24691 ; 02/11/2022
24692 ; inc byte [cs:multi_pass_id] ; multi_pass_id = 2
24693 ; 11/12/2022
24694 ; ds = cs
24695 000007E8 FE06[CD02] inc byte [multi_pass_id]
24696 000007EC E8131D call multi_pass ; was load ifs (now does nothing)
24697
24698 ; ifdef dblspace_hooks
24699 ; call MagicPostload ; make sure Magicdrv is final placed
24700 ; endif
24701
24702 ; ds = cs
24703
24704 ; 07/04/2024
24705 ; call endfile ; setup fcbs, files, buffers etc
24706
24707 ; ifdef dblspace_hooks
24708 ; call MagicSetCdss ; disable CDSs of reserved drives
24709 ; endif
24710
24711 ; 07/04/2024 - Retro DOS v5.0
24712 ; (PCDOS 7.1 IBMBIO.COM)
24713 ; cmp byte [cs:F5_key],1
24714 000007EF 803E[8E03]01 cmp byte [F5_key],1
24715 000007F4 7412 je short skip_magicpostload
24716 ; cmp byte [cs:F8_key],1
24717 000007F6 803E[8F03]01 cmp byte [F8_key],1
24718 000007FB 740B je short skip_magicpostload
24719 000007FD E8B710 call MagicPostload ; make sure Magicdrv is final placed
24720 ; 13/04/2024
24721 ; ds = cs
24722 00000800 E83E06 call endfile ; setup fcbs, files, buffers etc
24723 00000803 E81011 call MagicSetCdss ; disable CDSs of reserved drives
24724 ; ds = cs
24725 00000806 EB03 jmp short _@_
24726
24727 skip_magicpostload:
24728 ; 13/04/2024
24729 ; ds = cs
24730 00000808 E83606 call endfile ; setup fcbs, files, buffers etc
24731 _@_:
24732
24733 ; Reset SysinitPresent flag here. This is needed for the special fix for lying
24734 ; to device drivers. This has been moved up to this point to avoid problems
24735 ; with overlays called from installed programs
24736
24737 ; 11/12/2022
24738 ; ds = cs
24739
24740 ; mov ax,Bios_Data ; 0070h
24741 ; mov ax,KERNEL_SEGMENT
24742 ; 21/10/2022
24743 0000080B B87000 mov ax,DOSBIODATASEG ; 0070h
24744 0000080E 8EC0 mov es,ax ; point ES to bios data
24745
24746 00000810 26C606[DD07]00 mov byte [es:SysinitPresent],0 ; clear SysinitPresent flag
24747
24748 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
24749 ; test word [cs:install_flag],have_install_cmd ; 1
24750 ; test byte [cs:install_flag],1
24751 ; 11/12/2022
24752 ; ds = cs
24753 00000816 F606[CE02]01 test byte [install_flag],1
24754 ; test byte [cs:install_flag],have_install_cmd
24755 ; are there install commands?
24756 0000081B 7407 jz short dolast ; no, no need for further processing
24757 ; inc byte [cs:multi_pass_id] ; multi_pass_id = 3
24758 ; 11/12/2022
24759 ; ds = cs
24760 0000081D FE06[CD02] inc byte [multi_pass_id]
24761 00000821 E8DE1C call multi_pass ; execute install= commands
24762
24763 dolast:
24764
24765 ; [area] has the segment address for the allocated memory of sysinit, confbot.
24766 ; free the confbot area used for config.sys and sysinit itself.
24767
24768 ; Now if DOS is supposed to run high, we actually move it into high memory
24769 ; (if HiMem manager is available). For ROMDOS, we don't actually move
24770 ; anything, but just set up the ROM area for suballocation (or print
24771 ; a message if HiMem is not available).
24772 ;
24773 ; There is also this little hack for CPM style DOS calls that needs to
24774 ; be done when A20 is set...
24775
24776 ; 11/12/2022
24777 ; ds = cs
24778
24779 ; 22/10/2022
24780 ; cmp byte [cs:runhigh],0FFh; are we still waiting to be moved?
24781 ; 11/12/2022
24782 00000824 803E[6C02]FF cmp byte [runhigh],0FFh
24783 00000829 7503 jne short _@@_ ; 09/12/2022 ; no, our job is over
24784 0000082B E84802 call LoadDOSHiOrLo
24785 _@@_:
24786 ; cmp byte [cs:runhigh],0 ; are we running low
24787 ; 11/12/2022
24788 ; ds = cs
24789 0000082E 803E[6C02]00 cmp byte [runhigh],0
24790 ; je short _@@@

```

```

24791 00000833 7403          je      short ConfigDone      ; yes, no CPM hack needed
24792 00000835 E84C05        call    CPMHack              ; make ffff:d0 same as 0:c0
24793 _@@@:
24794
24795 ; We are now done with CONFIG.SYS processing
24796
24797 ConfigDone:
24798 ; 12/12/2022
24799 ; 22/10/2022
24800 ;mov     byte [cs:donotshownum],1
24801 ; done with config.sys.
24802 ; do not show line number message.
24803
24804 ;mov     es,[cs:area]
24805 ; 12/12/2022
24806 ; ds = cs
24807 ; 27/03/2019
24807 00000838 C606[5503]01   mov     byte [donotshownum],1
24808 0000083D 8E06[6803]     mov     es,[area]
24809
24810 00000841 B449           mov     ah,49h ; DEALLOC ; free allocated memory for command.com
24811 00000843 CD21           int     21h
24812 ; DOS - 2+ - FREE MEMORY
24813 ; ES = segment address of area to be freed
24814
24815 ; 22/10/2022
24816 ;test    word [cs:install_flag],2
24817 ;test    word [cs:install_flag],has_installed ; sysinit_base installed?
24818 ;test    byte [cs:install_flag],has_installed
24819 ; 11/12/2022
24820 ; ds = cs
24821 00000845 F606[CE02]02   test    byte [install_flag],2 ; has_installed
24822 ;test    byte [install_flag],has_installed
24823 0000084A 741F           jz      short skip_free_sysinitbase ; no.
24824
24825 ; set block from the old_area with impossible_owner_size.
24826 ; this will free the unnecessary sysinit_base that had been put in memory to
24827 ; handle install= command.
24828
24829 ; 12/12/2022
24830 ;push     es
24831 ;push     bx
24832
24833 ; 22/10/2022
24834 ;mov     es,[cs:old_area]
24835 ;mov     bx,[cs:impossible_owner_size]
24836 ; 12/12/2022
24837 ; ds = cs
24838 0000084C 8E06[5E03]     mov     es,[old_area]
24839 00000850 8B1E[6003]     mov     bx,[impossible_owner_size]
24840
24841 00000854 B44A           mov     ah,4Ah ; SETBLOCK
24842 00000856 CD21           int     21h
24843 ; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
24844 ; ES = segment address of block to change
24845 ; BX = new size in paragraphs
24846 00000858 8CC0           mov     ax,es
24847 0000085A 48             dec     ax
24848 0000085B 8EC0           mov     es,ax
24849 ; point to arena
24850 0000085D 26C70601000800        ;mov     word [es:ARENA.OWNER],8 ; set impossible owner
24851 ;mov     word [es:1],8
24852 00000864 26C70608005344        ;mov     word [es:ARENA.NAME],'SD' ; 4453h ; System Data
24853 ;mov     word [es:8],'SD'
24854
24855 ; 12/12/2022
24856 ;pop      bx
24857 ;pop      es
24858 ; BUGBUG 3-30-92 JeffPar: no reason to save ES
24859
24860 skip_free_sysinitbase:
24861 ; 22/10/2022
24862 ;cmp      byte [cs:runhigh],0
24863 ; 12/12/2022
24864 ; ds = cs
24865 0000086B 803E[6C02]00   cmp     byte [runhigh],0
24866 00000870 7403           je      short _@@@_ ; 04/07/2023
24867
24868 call     InstVDiskHeader ; Install VDISK header (allocates some mem from DOS)
24869
24870 ; -----
24871 _@@@_:
24872 ; 12/12/2022
24873 ; ds = cs
24874 ; 22/10/2022
24875 ; 27/03/2019
24876 ;push     cs
24877 ;pop      ds
24878 ; point DS to sysinitseg
24879
24880 ; set up the parameters for command
24881 ;
24882 ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS SYSINIT)
24883 ;;ifdef    MULTI_CONFIG
24884 ;mov     byte [config_cmd],0 ; set special code for query_user
24885 ; call    query_user ; to issue the AUTOEXEC prompt
24886 ; jnc     short process_autoexec; we should process autoexec normally
24887 ; ; !!!
24888 ; or      byte [bQueryOpt],4 ; MSDOS 6.21 IO.SYS - SYSINIT:081Fh
24889 ; ; !!!
24890 ; call    disable_autoexec ; no, we should disable it
24891 ;process_autoexec:
24892 ;;endif    ; !!!
24893 ; call    CheckQueryOpt ; MSDOS 6.21 IO.SYS - SYSINIT:0827h
24894 ; ; !!!
24895 ; 22/10/2022
24896 ;mov     cl,[command_line]
24897 ;mov     ch,0
24898 ;inc     cx
24899 ;mov     si,command_line
24900 ;add     si,cx
24901 ;mov     byte [si],cr ; cr-terminate command line
24902
24903 ; 22/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS)
24904 ; (SYSINIT:0809h)
24905 ;;;;
24906
24907 ; 30/12/2022 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS)
24908 ; (SYSINIT:0813h)
24909 ; ds = cs
24910 ; push   cs
24911 ; pop    ds
24912
24913 00000875 C606[6419]00   mov     byte [config_cmd],0 ; set special code for query_user
24914 0000087A E89D3D        call    query_user ; to issue the AUTOEXEC prompt

```

```

24915 ; 07/04/2024
24916 ;jnc short process_autoexec; we should process autoexec normally
24917
24918 ; 07/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
24919 ;;;
24920 0000087D 9C pushf
24921 0000087E F606[7F4C]01 test byte [bDisableUI],1
24922 00000883 7507 jnz short _@@@_ ; F5 clean/interactive boot option (has been) disabled
24923 00000885 803E[8E03]01 cmp byte [F5_key],1
24924 0000088A 7405 je short _@@@@_ ; F5 key pressed, bypass AUTOEXEC.BAT (clean boot)
24925 _@@@@_:
24926 0000088C 9D popf
24927 0000088D 730B jnc short process_autoexec; we should process autoexec normally
24928 0000088F EB01 jmp short bypass_autoexec
24929 _@@@@_:
24930 00000891 9D popf ; cf status at the return from 'query_user' call
24931 bypass_autoexec:
24932 ;;;
24933
24934 ; !!!
24935 00000892 800E[834C]04 or byte [bQueryOpt],4 ; MSDOS 6.21 IO.SYS - SYSINIT:081Fh
24936 ; !!!
24937 00000897 E87B3E call disable_autoexec ; no, we should disable it
24938 process_autoexec:
24939 ; !!!
24940 0000089A E8C33E call CheckQueryOpt ; MSDOS 6.21 IO.SYS - SYSINIT:0827h
24941
24942 ;mov cl,[command_line]
24943 ; 30/12/2022
24944 0000089D BE[B94B] mov si,command_line
24945 000008A0 8A0C mov cl,[si]
24946 000008A2 B500 mov ch,0
24947 000008A4 41 inc cx
24948 ;mov si,command_line
24949 000008A5 01CE add si,cx
24950 000008A7 C6040D mov byte [si],cr ; 0Dh ; cr-terminate command line
24951
24952 ;;;;
24953
24954 ; 30/12/2022 - Retro DOS v4.2
24955 %if 0
24956 ;mov si,(offset command_line+1)
24957 mov si,command_line+1
24958 push ds
24959 pop es
24960 mov di,si
24961 mov cl,0FFh ; -1
24962 _@_loop:
24963 inc cl ; +1
24964 lodsb
24965 stosb
24966 or al,al
24967 jnz short _@_loop
24968 dec di
24969 mov al,0Dh
24970 stosb ; cr-terminate command line
24971 mov [command_line],cl ; command line length (except CR)
24972 %endif
24973
24974 ; -----
24975
24976 ; Once we get to this point, the above code, which is below "retry"
24977 ; in memory, can be trashed (and in fact is -- see references to retry
24978 ; which follow....)
24979
24980 retry: ; PCDOS 7.1 IBMBIO.COM - SYSINIT:094Ch ; 07/04/2024
24981 000008AA BA[2B4B] mov dx,commnd ; now pointing to file description
24982
24983 ; we are going to open the command interpreter and size it as is done in
24984 ; ldfil. the reason we must do this is that sysinit is in free memory. if
24985 ; there is not enough room for the command interpreter,exec will probably
24986 ; overlay our stack and code so when it returns with an error sysinit won't be
24987 ; here to catch it. this code is not perfect (for instance .exe command
24988 ; interpreters are possible) because it does its sizing based on the
24989 ; assumption that the file being loaded is a .com file. it is close enough to
24990 ; correctness to be usable.
24991
24992 ; first, find out where the command interpreter is going to go.
24993
24994 000008AD 52 push dx ; save pointer to name
24995 000008AE BBFFFF mov bx,0FFFFh
24996 000008B1 B448 mov ah,48h ; ALLOC
24997 000008B3 CD21 int 21h ; get biggest piece
24998 000008B5 B448 mov ah,48h ; ALLOC
24999 000008B7 CD21 int 21h ; second time gets it
25000 000008B9 726B jc short memerrjx ; oooops
25001
25002 000008BB 8EC0 mov es,ax
25003 000008BD B449 mov ah,49h ; DEALLOC
25004 000008BF CD21 int 21h ; give it right back
25005 000008C1 89DD mov bp,bx
25006
25007 ; es:0 points to block,and bp is the size of the block in para.
25008
25009 ; we will now adjust the size in bp down by the size of sysinit.
25010 ; we need to do this because exec might get upset if some of the exec
25011 ; data in sysinit is overlaid during the exec.
25012
25013 ; 22/10/2022
25014 ; (MSDOS 5.0 IO.SYS SYSINIT:083Bh)
25015 000008C3 8B1E[9402] mov bx,[MEMORY_SIZE] ; get location of end of memory
25016 000008C7 8CC8 mov ax,cs ; get location of beginning of sysinit
25017
25018 ; Note that the "config_wrkseg" environment data is a segment in
25019 ; unallocated memory (as of the Dealloc of [area], above). This is ideal
25020 ; in one sense, because Exec is going to make a copy of it for COMMAND.COM
25021 ; anyway, and no one has responsibility for freeing "config_wrkseg". But
25022 ; we need to make sure that there's no way Exec will stomp on that data
25023 ; before it can copy it, and one way to do that is to make the available
25024 ; memory calculation even more "paranoid", by subtracting "config_wrkseg"
25025 ; from the "memory_size" segment value (which is typically A000h) instead
25026 ; of the current sysinit CS....
25027 ;
25028 ; The reason I use the term "paranoid" is because this code should have
25029 ; slid the data required by Exec up to the very top of memory, because as
25030 ; it stands, you have to have sizeof(COMMAND.COM) PLUS 64K to load just
25031 ; COMMAND.COM (64k is about what sysinit, and all the goop above sysinit,
25032 ; consumes). Now it's just a little worse (65K or more, depending on
25033 ; the size of your CONFIG.SYS, since the size of the environment workspace
25034 ; is determined by the size of CONFIG.SYS.... -JTP
25035
25036 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21, IO.SYS)
25037 ; (SYSINIT:0858h)
25038 000008C9 8B0E[6019] mov cx,[config_envlen]

```

```

25039 000008CD E303          jcxz     no_env      ; use config_wrkseg only if there's env data
25040 000008CF A1[6219]      mov ax,[config_wrkseg]
25041
25042          ; 22/10/2022
25043          ;mov     cx,[config_envlen]
25044          ;jcxz     no_env      ; use config_wrkseg only if there's env data
25045          ;mov     ax,[config_wrkseg]
25046
25047          ;no_env:
25048          ; 22/10/2022
25049          ; (MSDOS 5.0 IO.SYS SYSINIT:0841h)
25050          no_env:
25051          ; 30/12/2022
25052          ; (MSDOS 6.21 IO.SYS SYSINIT:0861h)
25053          sub     bx,ax      ; bx is size of sysinit in para
25054          add     bx,11h     ; add the sysinit php
25055          sub     bp,bx      ; sub sysinit size from amount of free memory
25056          jc      short memerrjx ; if there isn't even this much memory, give up
25057
25058          ;mov     ax,(OPEN<<8) ; open the file being execed
25059          mov ax,3D00h
25060          stc             ; in case of int 24
25061          int     21h
25062          jc      short comerr ; ooops
25063          ; DOS - 2+ - OPEN DISK FILE WITH HANDLE
25064          ; DS:DX -> ASCIZ filename
25065          ; AL = access mode
25066          ; 0 - read
25067          ; 22/10/2022
25068          ; (MSDOS 5.0 IO.SYS SYSINIT:0852h)
25069          mov     bx,ax      ; handle in bx
25070
25071          ; If the standard command interpreter is being used, verify it is correct
25072
25073          ; 30/12/2022 - Retro DOS v4.2
25074          ; (MSDOS 6.21 IO.SYS, SYSINIT:0874h)
25075          cmp     byte [newcmd],0 ; was a new shell selected?
25076          jne     short skip_validation ; yes
25077          ; 07/04/2024 - Retro DOS v5.0
25078          ; (PCDOS 7.1 IBMBIO.COM, SYSINIT:098Eh)
25079          mov     dx,retry-4 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0948h
25080          mov     cx,4
25081          mov     ah,READ
25082          int     21h
25083          cmp     byte [retry-4],0E9h
25084          jne     short comerr
25085          ; 20/04/2019 - Retro DOS v4.0
25086          ; 30/12/2022
25087          ;cmp     byte [retry-1],64h ; MSDOS 6.21 IO.SYS - SYSINIT:088Ch
25088          ; .. COMMAND.COM Version 6.20 (14h&0Fh)
25089          ; 07/04/2024 - Retro DOS v5.0
25090          ;cmp     byte [retry-1],66h ; .. COMMAND.COM Version 6.22 (16h&0Fh)
25091          ;cmp     byte [retry-1],7Ah ; PCDOS 7.1 IBMBIO.COM - SYSINIT:099Fh
25092          ; .. COMMAND.COM Version 7.10 (0Ah&0Fh)
25093          cmp     byte [retry-1],((MAJOR_VERSION&0Fh)<<4)|(MINOR_VERSION&0Fh)
25094          jne     short comerr
25095
25096          ; 22/10/2022
25097          ;cmp     byte [newcmd],0 ; was a new shell selected?
25098          ;jne     short skip_validation ; yes
25099          ;mov     dx,retry-4
25100          ;mov     cx,4
25101          ;mov     ah,READ
25102          ;int     21h
25103          ;cmp     byte [retry-4],0E9h
25104          ;jne     short comerr
25105          ; 20/04/2019 - Retro DOS v4.0
25106          ;cmp     byte [retry-1],64h ; MSDOS 6.21 IO.SYS - SYSINIT:088Ch
25107          ;cmp     byte [retry-1],((MAJOR_VERSION&0Fh)<<4)|(MINOR_VERSION&0Fh)
25108          ;jne     short comerr
25109
25110          ;skip_validation:
25111          ; 22/10/2022
25112          ; (MSDOS 5.0 IO.SYS SYSINIT:0854h)
25113          skip_validation:
25114          ; 30/12/2022
25115          ; (MSDOS 6.21 IO.SYS SYSINIT:0893h)
25116          xor     cx,cx
25117          xor     dx,dx
25118          mov     ax,(LSEEK<<8)|2
25119          mov     ax,4202h
25120          stc             ; in case of int 24
25121          int     21h      ; get file size in dx:ax
25122          jc      short comerr
25123          add     ax,15     ; convert size in dx:ax to para in ax
25124          adc     dx,0      ; round up size for conversion to para
25125          call    off_to_para
25126          mov     cx,12
25127          shl     dx,cx     ; low nibble of dx to high nibble
25128          or      ax,dx     ; ax is now # of para for file
25129          add     ax,10h    ; 100h byte php
25130          cmp     ax,bp     ; will command fit in available mem?
25131          jnb     short okld ; jump if yes.
25132
25133          ; 30/12/2022
25134          %if 0
25135          ; 22/10/2022
25136          memerrjx: ; (MSDOS 5.0 IO.SYS SYSINIT:0876h)
25137          ;jmp     memerr ; (MSDOS 5.0 IO.SYS SYSINIT:34D5h)
25138          ; 02/11/2022
25139          ;jmp     mem_err
25140          ; 11/12/2022
25141          ; ds = cs
25142          jmp     mem_err2
25143          %endif
25144          ; 30/12/2022
25145          ; (MSDOS 6.21, IO.SYS, SYSINIT:08B5h)
25146          memerrjx:
25147          mov     dx,badmem ; "Configuration too large for memory"
25148          call    print
25149          jmp     short continue
25150
25151          okld:
25152          mov     ah,3Eh ; CLOSE
25153          int     21h      ; close file
25154
25155          ; 22/10/2022
25156          pop     dx      ; (MSDOS 5.0 IO.SYS SYSINIT:087Dh)
25157
25158          ; 24/03/2019
25159
25160          push    cs      ; point es to sysinitseg
25161          pop     es
25162          mov     bx,COMEXE ; point to exec block

```

```

25163             ; 22/10/2022
25164             ;pop     dx             ; recover pointer to name
25165
25166             ;;ifdef     MULTI_CONFIG
25167
25168             ;   If there's any environment data in "config_wrkseg", pass it to shell;
25169             ;   there will be data if there were any valid SET commands and/or if a menu
25170             ;   selection was made (in which case the CONFIG environment variable will be
25171             ;   set to that selection).
25172
25173             ; 23/10/2022
25174             ;mov     cx,[config_envlen]
25175             ;jcxz    no_envdata
25176             ;mov     cx,[config_wrkseg]
25177
25178             no_envdata:
25179             ;mov     [bx+EXEC0.ENVIRON],cx
25180             ;mov     [bx],cx
25181
25182             ;;endif     ;MULTI_CONFIG
25183
25184             ; 30/12/2022 - Retro DOS v4.2
25185             ; (MSDOS 6.21 IO.SYS SYSINIT:08c7h)
25186             mov     cx,[config_envlen]
25187             jcxz    no_envdata
25188             mov     cx,[config_wrkseg]
25189             no_envdata:
25190             ;mov     [bx+EXEC0.ENVIRON],cx
25191             ;mov     [bx],cx
25192
25193             ; 23/10/2022
25194             ; (MSDOS 5.0 IO.SYS SYSINIT:0883h)
25195
25196             ;mov     [bx+EXEC0.COM_LINE+2],cs ; set segments
25197             mov     [bx+4],cs
25198             ;mov     [bx+EXEC0.5C_FCB+2],cs
25199             mov     [bx+8],cs
25200             ;mov     [bx+EXEC0.6C_FCB+2],cs
25201             mov     [bx+12],cs
25202
25203             ;mov     ax,(EXEC<<8) + 0
25204             ; 23/10/2022
25205             ;xor     ax,ax
25206             ;mov     ah,4Bh
25207             ; 04/07/2023
25208             ;mov     ax,4B00h
25209             mov     ax,(EXEC<<8)
25210
25211             stc             ; in case of int 24
25212             int     21h             ; go start up command
25213             ; DOS - 2+ - LOAD OR EXECUTE (EXEC)
25214             ; DS:DX -> ASCIZ filename
25215             ; ES:BX -> parameter block
25216             ; AL = subfunc: load & execute program
25217             ;push    cs
25218             ;pop     ds
25219
25220             ; 13/04/2024
25221             ; 23/10/2022
25222             push    dx             ; push to balance fall-through pop
25223
25224             ; note fall through if exec returns (an error)
25225             comerr:
25226             ; 23/10/2022
25227             ;;ifdef     MULTI_CONFIG
25228             ;cmp     byte [commnd4],0
25229             ;je      short comerr2 ; all defaults exhausted, print err msg
25230             ;cmp     byte [newcmd],0
25231             ;je      short continue ; don't print err msg for defaults just yet
25232             ;comerr2:
25233             ;;endif
25234
25235             ; 30/12/2022 - Retro DOS v4.2
25236             ;push    cs
25237             ;pop     ds
25238             ; 07/04/2024
25239             ; ds = cs
25240             cmp     byte [commnd4],0
25241             je      short comerr2 ; all defaults exhausted, print err msg
25242             cmp     byte [newcmd],0
25243             je      short continue ; don't print err msg for defaults just yet
25244             comerr2:
25245             ; 07/04/2024
25246             ;push    dx ; 30/12/2022
25247
25248             ; 23/10/2022
25249             mov     dx,badcom ; want to print command error
25250             call    badfil
25251
25252             ; 07/04/2024
25253             ;pop     dx ; 30/12/2022
25254             continue:
25255             ; 13/04/2024
25256             ; 23/10/2022
25257             pop     dx
25258
25259             ; 30/12/2022
25260             %if 0
25261
25262             ;;ifndef MULTI_CONFIG
25263             ;jmp     stall
25264             ; 24/10/2022
25265             stall:             ; (MSDOS 5.0 IO.SYS, SYSINIT:0899h)
25266             jmp     short stall
25267             ;else
25268
25269             %endif
25270
25271             ; 30/12/2022 (MSDOS 6.21 SYSINIT, Retro DOS v4.2)
25272             ;%if 1
25273             ; 23/10/2022 (MSDOS 5.0 SYSINIT, Retrodos v4.0)
25274             ;%if 0
25275             mov     ah,GET_DEFAULT_DRIVE ; 19h
25276             int     21h             ;
25277             add     al,'A'             ;
25278             mov     dl,al             ; DL == default drive letter
25279             mov     si,commnd2
25280             cmp     byte [newcmd],0 ; if a SHELL= was given
25281             jne     short do_def2 ; then try the 2nd alternate;
25282             mov     byte [si],0 ; otherwise, the default SHELL= was tried,
25283             jmp     short do_def3 ; which is the same as our 2nd alt, so skip it
25284             do_def2:
25285             cmp     byte [si],0 ; has 2nd alternate been tried?
25286             jne     short do_alt ; no

```



```

25287
25288 00000985 BE[7C4B]
25289 00000988 803C00
25290 0000098B 754C
25291 0000098D BE[9C4B]
25292 00000990 803C00
25293 00000993 7544
25294 00000995 52
25295 00000996 BA[3653]
25296 00000999 E8B640
25297 0000099C 5A
25298
25299 0000099D B402
25300 0000099F CD21
25301 000009A1 52
25302 000009A2 B23E
25303 000009A4 CD21
25304 000009A6 8A1E[2A4B]
25305 000009AA B700
25306 000009AC C687[2B4B]0D
25307 000009B1 BA[294B]
25308 000009B4 B40A
25309 000009B6 CD21
25310 000009B8 BA[6E50]
25311 000009BB E89440
25312 000009BE 5A
25313 000009BF 8A1E[2A4B]
25314 000009C3 08DB
25315 000009C5 74D6
25316 000009C7 C606[284B]01
25317 000009CC C687[2B4B]00
25318 000009D1 C706[B94B]000D
25319 000009D7 EB35
25320
25321 000009D9 1E
25322 000009DA 07
25323 000009DB C606[284B]00
25324 000009E0 BF[2B4B]
25325
25326 000009E3 AC
25327 000009E4 C644FF00
25328 000009E8 AA
25329 000009E9 08C0
25330 000009EB 75F6
25331 000009ED BF[B94B]
25332 000009F0 807C023A
25333 000009F4 7503
25334 000009F6 885401
25335
25336 000009F9 AC
25337 000009FA AA
25338 000009FB 08C0
25339 000009FD 75FA
25340 000009FF C645FF0D
25341
25342
25343
25344
25345
25346
25347
25348
25349
25350
25351 00000A03 C606[794C]00
25352 00000A08 E80A3D
25353 00000A0B E8523D
25354
25355 00000A0E E999FE
25356
25357
25358
25359
25360
25361
25362
25363
25364
25365
25366
25367
25368
25369
25370
25371
25372
25373
25374
25375
25376
25377
25378
25379
25380
25381
25382
25383
25384
25385
25386
25387 00000A11 BBFFFF
25388 00000A14 B448
25389 00000A16 CD21
25390 00000A18 B448
25391 00000A1A CD21
25392
25393
25394
25395
25396 00000A1C A3[6803]
25397 00000A1F A3[6403]
25398 00000A22 C3
25399
25400
25401
25402
25403 00000A23 484D41206E6F742061-
25403 00000A2C 7661696C61626C653A-
25403 00000A35 204C6F6164696E6720-
25403 00000A3E 444F53206C6F770D0A-
25403 00000A47 24
25404
25405 00000A48 466174616C20457272-
25405 00000A51 6F723A2043616E6E6F-

do_def3:
    mov     si,commnd3
    cmp     byte [si],0 ; has 3rd alternate been tried?
    jne     short do_alt ; no
    mov     si,commnd4
    cmp     byte [si],0 ; has 4th alternate been tried?
    jne     short do_alt ; no
    push    dx
    mov     dx,badcomprmt
    call    print
    pop     dx ; recover default drive letter in DL
request_input:
    mov     ah,STD_CON_OUTPUT
    int     21h
    push    dx
    mov     dl,'>'
    int     21h
    mov     bl,[tmplate+1] ; [tmplate+1] = 12
    mov     bh,0
    mov     byte [commnd+bx],0Dh
    mov     dx,tmplate
    mov     ah,STD_CON_STRING_INPUT
    int     21h ; read a line of input
    mov     dx,crlfm
    call    print
    pop     dx
    mov     bl,[tmplate+1]
    or      bl,bl ; was anything typed?
    jz      short request_input
    mov     byte [newcmd],1 ; disable validation for user-specified binaries
    mov     byte [commnd+bx],0 ; NULL-terminate it before execing it
    mov     word [command_line],0D00h
    jmp     short do_exec
do_alt:
    push    ds
    pop     es
    mov     byte [newcmd],0 ; force validation for alternate binaries
    mov     di,commnd
do_alt1:
    lodsb
    mov     byte [si-1],0 ; copy the alternate, zapping it as we go,
    ; so that we know it's been tried
    stosb
    or      al,al
    jnz     short do_alt1
    mov     di,command_line
    cmp     byte [si+2],':'
    jne     short do_alt2
    mov     [si+1],dl ; stuff default drive into alt. command line
do_alt2:
    lodsb
    stosb
    or      al,al
    jnz     short do_alt2
    mov     byte [di-1],cr
;; Last but not least, see if we need to call disable_autoexec
; MSDOS 6.0 (SYSINIT1.ASM)
;cmp     [command_line-1],0
;jne     short do_exec
;mov     [command_line-1], '/'
;call    disable_autoexec
; MSDOS 6.21 IO.SYS (SYSINIT:0994h)
;mov     byte [dae_flag],0 ; 24/03/2019 - Retro DOS v4.0
;call    disable_autoexec
;call    CheckQueryOpt ; 24/03/2019 - Retro DOS v4.0
do_exec:
    jmp     retry
;endif ;MULTI_CONFIG
;endif ; 23/10/2022 (MSDOS 5.0 SYSINIT)
;endif ; 30/12/2022 (MSDOS 6.21 SYSINIT)
; 24/03/2019 - Retro DOS v4.0
; -----
; procedure : AllocFreeMem
;
; Allocate Max memory from DOS to find out where to load DOS.
; DOS is at temporary location when this call is being made
;
; Inputs : None
; Outputs: The biggest chunk of memory is allocated (all mem at init time)
; [area] & [memhi] set to the para value of the start of the
; free memory.
;
; Uses : AX, BX
; -----
; 30/12/2022 - Retro DOS v4.2
; (MSDOS 6.21 IO.SYS, SYSINIT:09A2h)
; 08/04/2024 - Retro DOS v5.0
; (PCDOS 7.1 IBMBIO.COM, SYSINIT:0AB5h)
; 23/10/2022
AllocFreeMem:
    mov     bx,0FFFFh
    mov     ah,48h ; ALLOC
    int     21h ; first time fails
    mov     ah,48h ; ALLOC
    int     21h ; second time gets it
    ; 11/12/2022
    ; ds = cs
    mov     [cs:area],ax
    mov     [cs:memhi],ax ; memhi:memlo now points to
    mov     [area],ax
    mov     [memhi],ax ; memhi:memlo now points to
    retn ; start of free memory
; include msbio.c16
; -----
DOSLOMSG:
    db      'HMA not available: Loading DOS low',0Dh,0Ah,'$'
FEmsg:
    db      'Fatal Error: Cannot allocate Memory for DOS',0Dh,0Ah,'$'

```

```

25405 00000A5A 7420616C6C6F636174-
25405 00000A63 65204D656D6F727920-
25405 00000A6C 666F7220444F530D0A-
25405 00000A75 24
25406
25407
25408
25409
25410
25411
25412
25413
25414
25415
25416
25417
25418
25419
25420
25421
25422 00000A76 E81F00
25423
25424
25425
25426 00000A79 731C
25427
25428
25429
25430
25431
25432
25433 00000A7B B409
25434 00000A7D BA[230A]
25435 00000A80 CD21
25436
25437
25438
25439
25440 00000A82 BB0100
25441
25442 00000A85 E83F00
25443
25444
25445
25446
25447 00000A88 8E06[7302]
25448
25449
25450 00000A8C 31C0
25451
25452
25453
25454 00000A8E FF1E[7D02]
25455
25456
25457
25458
25459
25460
25461 00000A92 C606[6C02]00
25462
25463 00000A97 C3
25464
25465
25466
25467
25468
25469
25470
25471
25472
25473
25474
25475
25476
25477
25478
25479
25480
25481
25482
25483
25484 00000A98 E81300
25485 00000A9B 7210
25486
25487
25488
25489
25490
25491
25492
25493 00000A9D 8E06[7302]
25494
25495
25496
25497
25498
25499
25500 00000AA1 31C0
25501
25502 00000AA3 FF1E[7D02]
25503
25504 00000AA7 C606[6C02]01
25505 00000AAC F8
25506
25507 00000AAD C3
25508
25509
25510
25511
25512
25513
25514
25515
25516
25517
25518
25519
25520
25521
25522
25523
25524
25525

```

```

; -----
;
; procedure : LoadDOSHiOrLo
;
;     Tries to move DOS into HMA. If it fails then loads
;     DOS into Low memory. For ROMDOS, nothing is actually
;     moved; this just tries to allocate the HMA, and prints
;     a message if this is not possible.
; -----
;
; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
LoadDOSHiOrLo:
; 27/03/2019 - Retro DOS v4.0
; ds = cs
call TryToMovDOSHi ; Try moving it into HMA (M024)
; jc short LdngLo ; If that don't work...
; ret
; 18/12/2022
jnc short LoadDoshi_ok
LdngLo:
; 23/10/2022
; push cs
; pop ds
; 11/12/2022
; ds = cs
mov ah,9
mov dx,DOSLOMSG ; inform user that we are
int 21h ; loading low

;ifndef ROMDOS
; actually move the dos, and reinitialize it.
mov bx,1 ; M012
; use int 21 alloc for mem
call MovDOSLo
; 11/12/2022
; ds = cs
; mov es,[cs:CURRENT_DOS_LOCATION] ; give dos its temporary loc.
; 23/10/2022
mov es,[CURRENT_DOS_LOCATION]
; mov es,[cs:FINAL_DOS_LOCATION] ; 24/03/2019 - Retro DOS v4.0
; mov es,[FINAL_DOS_LOCATION] ; 27/03/2019
xor ax,ax ; ax = 00 ---> install stub
; 11/12/2022
; ds = cs
call far [cs:dosegreinit] ; call dos segreinit
call far [dos_segreinit] ; 27/03/2019
;endif ; ROMDOS
; 23/10/2022
; mov byte [cs:runhigh],0 ; mark that we are running lo
; 11/12/2022
; ds = cs
mov byte [runhigh],0 ; 27/03/2019
LoadDoshi_ok: ; 18/12/2022
ret
; -----
;
; procedure : TryToMovDOSHi
;
;     This tries to move DOS into HMA.
;     Returns CY if it failed.
;     If it succeeds returns with carry cleared.
;
;     For ROMDOS, dosegreinit must be called again to allow
;     the A20 switching code in the low mem stub to be installed.
; -----
;
; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
; (MSDOS 5.0 IO.SYS - SYSINIT:092Ah)
TryToMovDOSHi:
; 11/12/2022
; 27/03/2019 - Retro DOS v4.0
; ds = cs
call MovDOSHi
jc short ttldhx

;ifndef ROMDOS
; 23/10/2022
; mov es,[cs:CURRENT_DOS_LOCATION] ; give dos its temporary loc.
; mov es,[cs:FINAL_DOS_LOCATION] ; 24/03/2019 - Retro DOS v4.0
; 11/12/2022
; ds = cs
mov es,[CURRENT_DOS_LOCATION]
;else
; ..
;endif ; ROMDOS

; 11/12/2022
; ds = cs
xor ax,ax ; ax = 00 ---> install stub
call far [cs:dosegreinit] ; call dos segreinit
call far [dos_segreinit]
; mov byte [cs:runhigh],1
mov byte [runhigh],1
clc
ttldhx:
ret
; -----
;
; procedure : MovDOSHi
;
;     Tries to allocate HMA and Move DOS/BIOS code into HMA
;     For ROMDOS, the code is not actually moved, but the
;     HMA is allocated and prepared for sub-allocation.
;
;     Returns : CY if it failed
; -----
;
; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
MovDOSHi:
; 14/05/2019
; 27/03/2019 - Retro DOS v4.0
; ds = cs

```



```

25526 00000AAE E8D600      call    AllocHMA
25527 00000AB1 7213        jc      short mdhx      ; did we get HMA?
25528 00000AB3 B8FFFF      mov     ax,0FFFFh      ; yes, HMA seg = 0ffffh
25529 00000AB6 8EC0        mov     es,ax
25530
25531 ;ifndef ROMDOS
25532 ; actually move the BIOS and DOS
25533
25534 ; NOTE: Retro DOS v4.0 does not move BIOS (IO.SYS) to HMA
25535 ; 24/03/2019
25536
25537 ; 23/10/2022
25538 00000AB8 E83200      call    MovBIOS          ; First move BIOS into HMA
25539
25540 ; ES:DI points to free HMA after BIOS
25541
25542 ; 14/05/2019
25543 ; 24/03/2019 - Retro DOS v4.0
25544 ;xor     di,di
25545
25546 ; 23/10/2022
25547 ;mov     cx,[cs:hi_doscod_size]      ; pass the code size of DOS
25548 ; 11/12/2022
25549 ; ds = cs
25550 00000ABB 8B0E[8302]    mov     cx,[hi_doscod_size]      ; when it is in HMA
25551 00000ABF E81100      call    MovDOS          ; and move it
25552
25553 ; ES:DI points to free HMA after DOS
25554 ;else
25555 ; allocate space at beginning of HMA to allow for CPMHack
25556
25557 ; mov     di,0E0h          ; room for 5 bytes at ffff:d0
25558 ;
25559 ;endif ; ROMDOS
25560
25561 00000AC2 E87602      call    SaveFreeHMAPtr      ; Save the Free HMA ptr
25562 00000AC5 F8          cld
25563 mdhx:
25564 00000AC6 C3          retn
25565
25566 ; -----
25567 ;
25568 ; procedure : MovDOSLo
25569 ;
25570 ; Allocates memory from DOS and moves BIOS/bos code into it
25571 ;
25572 ; -----
25573
25574 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
25575
25576 ;ifndef ROMDOS
25577
25578 MovDOSLo:
25579 ; 14/05/2019
25580 ; 27/03/2019 - Retro DOS v4.0
25581 ; ds = cs
25582 00000AC7 E84500      call    AllocMemForDOS      ; incestuosly!!!
25583
25584 ; 23/10/2022
25585 ; 14/05/2019
25586 ;inc     ax ; skip MCB
25587
25588 00000ACA 8EC0        mov     es,ax          ; pass the segment to MovBIOS
25589 ; NOTE: Retro DOS v4.0 does not move BIOS (IO.SYS) to HMA
25590 ; 24/03/2019
25591
25592 ; 23/10/2022
25593 00000ACC E81E00      call    MovBIOS
25594
25595 ;----- ES:DI points memory immediately after BIOS
25596
25597 ; 14/05/2019
25598 ; NOTE:
25599 ; Order of (RETRO) DOS kernel sections at memory:
25600 ; BIOSDATA+BIOSCODE+BIOSDATAINIT+DOSDATA+DOSCODE(LOW)
25601
25602 ; 24/03/2019 - Retro DOS v4.0
25603 ;xor     di,di
25604
25605 ; 23/10/2022
25606 ;mov     cx,[cs:lo_doscod_size]      ; DOS code size when loaded
25607 ; 11/12/2022
25608 ; ds = cs
25609 00000ACF 8B0E[8102]    mov     cx,[lo_doscod_size]      ; low
25610 ;call    MovDOS
25611 ;retn
25612 ; 11/12/2022
25613 ;jmp     short MovDOS
25614
25615 ;endif ; ROMDOS
25616
25617 ; 11/12/2022
25618
25619 ; -----
25620 ;
25621 ; procedure : MovDOS
25622 ;
25623 ; Moves DOS code into requested area
25624 ;
25625 ; In : ES:DI - pointer to memory where DOS is to be moved
25626 ; CX - size of DOS code to be moved
25627 ;
25628 ; Out : ES:DI - pointer to memory immediately after DOS
25629 ;
25630 ; -----
25631
25632 ; 11/12/2022
25633 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
25634
25635 ;ifndef ROMDOS
25636
25637 MovDOS:
25638 ; 14/05/2019
25639 ; 27/03/2019 - Retro DOS v4.0
25640
25641 ; 11/12/2022
25642 ; ds = cs
25643
25644 ; 23/10/2022
25645 ;push    ds ; *//
25646
25647 00000AD3 06          push    es
25648 00000AD4 57          push    di
25649

```

```

25650          ; 11/12/2022
25651 00000AD5 1E      push    ds ; *// ; 11/12/202
25652
25653          ; 29/04/2019
25654 00000AD6 C536[7102]  lds     si,[dosinit] ; 11/12/2022
25655          ; 23/10/2022
25656          ; lds     si,[cs:dosinit]
25657          ; 03/09/2023
25658 00000ADA 89F0      mov     ax,si
25659
25660 00000ADC F3A4      rep     movsb
25661
25662 00000ADE 1F          pop     ds ; *// ; 11/12/2022
25663
25664 00000ADF 5B          pop     bx                ; get back offset into which
25665                                ; DOS was moved
25666          ; 03/09/2023
25667 ; mov     ax,[cs:dosinit]                ; get the offset at which DOS
25668                                ; wants to run
25669          ; 03/09/2023
25670 ; mov     ax,[dosinit]
25671 ; ax = [dosinit]
25672
25673 00000AE0 29D8      sub     ax,bx
25674 00000AE2 E8B502   call    off_to_para
25675 00000AE5 5B          pop     bx                ; get the segment at which
25676                                ; we moved DOS into
25677 00000AE6 29C3      sub     bx,ax                ; Adjust segment
25678
25679          ; 11/12/2022
25680          ; 23/10/2022
25681 ; mov     [cs:CURRENT_DOS_LOCATION],bx ; and save it
25682 ; mov     [cs:FINAL_DOS_LOCATION],bx
25683          ; 11/12/2022
25684 00000AE8 891E[7302]  mov     [CURRENT_DOS_LOCATION],bx
25685
25686          ; 27/03/2019
25687 ; pop     ds ; *//
25688 ; ds = cs
25689 ; mov     [FINAL_DOS_LOCATION],bx
25690
25691 00000AEC C3          retn
25692
25693 ;endif ;ROMDOS
25694
25695 ; NOTE: Retro DOS v4.0 does not move BIOS (IO.SYS) to HMA
25696 ; 24/03/2019
25697 ; -----
25698 ;
25699 ; procedure : MovBIOS
25700 ;
25701 ;           Moves BIOS code into requested segment
25702 ;
25703 ;   In : ES - segment to which BIOS is to be moved
25704 ;         ( it moves always into offset BCode_Start)
25705 ;
25706 ;   Out : ES:DI - pointer to memory immediately after BIOS
25707 ;
25708 ; -----
25709
25710          ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
25711          ; 08/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
25712
25713 ;ifndef ROMDOS
25714
25715 MovBIOS: ; proc     near
25716          ; 11/12/2022
25717 00000AED 1E          push    ds ; ds = cs
25718
25719          ; 23/10/2022
25720 ; mov     ds,[cs:temp_bcode_seg]          ; current BIOS code seg
25721 ; 17/09/2023 ; 08/04/2024
25722 00000AEE 8E1E[8902]  mov     ds,[temp_bcode_seg]
25723 ; mov     si,BCODE_START ; mov si,30h
25724          ; 09/12/2022
25725 00000AF2 BE[3000]    mov     si,BCODESTART ; 30h
25726 00000AF5 89F7      mov     di,si
25727 ; mov     cx,1E00h ; PCDOS 7.1 IBMBIO.COM ; 08/04/2024
25728 00000AF7 B9701D     mov     cx,BCODE_END ; mov cx,1A60h
25729 00000AFA 29F1      sub     cx,si                ; size of BIOS
25730 00000AFC D1E9      shr     cx,1                ; Both the labels are para
25731                                ; aligned
25732 00000AFE F3A5      rep     movsw
25733
25734          ; 11/12/2022
25735 00000B00 1F          pop     ds ; ds = cs
25736
25737 00000B01 06          push    es
25738 00000B02 57          push    di                ; save end of BIOS
25739 00000B03 8CC0      mov     ax,es
25740
25741          ; 11/12/2022
25742 ; mov     [cs:BCodeSeg],ax                ; save it for later use
25743 ; call    dword ptr cs:_seg_reinit_ptr
25744 ; call    far [cs:seg_reinit_ptr]          ; far call to seg_reinit (M022)
25745 ; ds = cs
25746 00000B05 A3[8A03]    mov     [BCodeSeg],ax
25747 00000B08 FF1E[8702]  call    far [seg_reinit_ptr]
25748
25749 00000B0C 5F          pop     di
25750 00000B0D 07          pop     es                ; get back end of BIOS
25751 00000B0E C3          retn
25752
25753 ;MovBIOS endp
25754
25755 ;endif ; ROMDOS
25756
25757 ; 11/12/2022
25758 %if 0
25759
25760 ; 24/03/2019
25761
25762 ; -----
25763 ;
25764 ; procedure : MovDOS
25765 ;
25766 ;           Moves DOS code into requested area
25767 ;
25768 ;   In : ES:DI - pointer to memory where DOS is to be moved
25769 ;         CX - size of DOS code to be moved
25770 ;
25771 ;   Out : ES:DI - pointer to memory immediately after DOS
25772 ;
25773 ; -----

```

```

25774
25775 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
25776
25777 ;ifndef ROMDOS
25778
25779 MovDOS:
25780 ; 14/05/2019
25781 ; 27/03/2019 - Retro DOS v4.0
25782
25783 ; 11/12/2022
25784 ; ds = cs
25785
25786 ; 23/10/2022
25787 ;push ds ; *//
25788
25789 push es
25790 push di
25791
25792 ; 11/12/2022
25793 push ds ; *// ; 11/12/202
25794
25795 ; 29/04/2019
25796 lds si,[dosinit] ; 11/12/2022
25797 ; 23/10/2022
25798 ;lds si,[cs:dosinit]
25799 ; 03/09/2023
25800 mov ax,si
25801
25802 rep movsb
25803
25804 pop ds ; *// ; 11/12/2022
25805
25806 pop bx ; get back offset into which
25807 ; DOS was moved
25808 ;mov ax,[dosinit] ; 03/09/2023
25809 ;;mov ax,[cs:dosinit] ; get the offset at which DOS
25810 ; wants to run
25811
25812 sub ax,bx
25813 call off_to_para
25814 pop bx ; get the segment at which
25815 ; we moved DOS into
25816 sub bx,ax ; Adjust segment
25817
25818 ; 11/12/2022
25819 ; 23/10/2022
25820 ;mov [cs:CURRENT_DOS_LOCATION],bx ; and save it
25821 ;;mov [cs:FINAL_DOS_LOCATION],bx
25822 ; 11/12/2022
25823 mov [CURRENT_DOS_LOCATION],bx
25824
25825 ; 27/03/2019
25826 ;pop ds ; *//
25827 ; ds = cs
25828 ;mov [FINAL_DOS_LOCATION],bx
25829
25830 retn
25831
25832 ;endif ;ROMDOS
25833
25834 %endif
25835
25836 ; -----
25837 ;
25838 ; procedure : AllocMemForDOS
25839 ;
25840 ; Allocate memory for DOS/BIOS code from DOS !!!
25841 ;
25842 ; Out : AX - seg of allocated memoryblock
25843 ; -----
25844
25845 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
25846 ; 08/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
25847
25848 ;ifndef ROMDOS
25849
25850 AllocMemForDOS:
25851 ; 11/12/2022
25852 ; 14/05/2019
25853 ; 27/03/2019 - Retro DOS v4.0
25854 ; ds = cs
25855 ;mov ax,BCode_end
25856 ;sub ax,BCode_start ; BIOS code size
25857 ; 23/10/2022
25858 00000B0F B8701D mov ax,BCODE_END ; 1A60h ; 1A70h for MSDOS 6.21
25859 ; 30/12/2022
25860 ;mov ax,1E00h ; PCDOS 7.1 IBMBIO.COM ; 08/04/2024
25861 ;sub ax,BCODE_START ; 30h
25862 ; 09/12/2022
25863 00000B12 2D[3000] sub ax,BCODESTART ; sub ax,30h ; 08/04/2024
25864 ; 24/03/2019 - Retro DOS v4.0
25865 ; 02/11/2022
25866 ;add ax,[cs:lo_doscod_size]; DOS code size
25867 ; 11/12/2022
25868 ; ds = cs
25869 00000B15 0306[8102] add ax,[lo_doscod_size]
25870 00000B19 83C00F add ax,15
25871 00000B1C E87B02 call off_to_para ; convert to para
25872 ; 23/10/2022
25873 ; 14/05/2019
25874 ;inc ax ; + 1 paragraph for MCB
25875 00000B1F 09DB or bx,bx ; M012
25876 00000B21 89C3 mov bx,ax ; can we use int 21 for alloc
25877 00000B23 741A jz short update_arena ; M012
25878 00000B25 B448 mov ah,48h ; request DOS
25879 00000B27 CD21 int 21h
25880 00000B29 7250 jc short FatalErr ; IF ERR WE ARE HOSED
25881 ; 23/10/2022
25882 ; 24/03/2019 - Retro DOS v4.0 (ORG 0)
25883 00000B2B 83E803 sub ax,3 ; Take care ORG 30h of
25884 ; BIOS code
25885 00000B2E 8EC0 mov es,ax
25886 ;mov word [es:20h+ARENA.OWNER],08h ; mark it as system
25887 ;mov word [es:20h+ARENA.NAME],'SC' ; code area
25888 ; 14/05/2019
25889 ;mov word [es:ARENA.OWNER],08h ; mark it as system
25890 ;mov word [es:ARENA.NAME],'SC' ; code area
25891 ; 08/04/2024 (PCDOS 7.1 IBMBIO.COM)
25892 ; 23/10/2022
25893 00000B30 26C70621000800 mov word [es:20h+1],08h ; mark it as system
25894 00000B37 26C70628005343 mov word [es:20h+8],'SC' ; 4353h ; code area
25895
25896 00000B3E C3 retn
25897

```

```

25898 ; BUGBUG -- 5 Aug 92 -- chuckst -- Allocating space for DOS
25899 ; using DOS itself causes an arena to be generated.
25900 ; Unfortunately, certain programs (like PROTMAN$)
25901 ; assume that the device drivers are loaded into
25902 ; the first arena. For this reason, MagicDrv's
25903 ; main device driver header arena is manually
25904 ; truncated from the arena chain, and the space
25905 ; for DOS is allocated using the following
25906 ; simple code, which also assumes that the
25907 ; first arena is the free one where DOS's low
25908 ; stub will go.
25909 ;
25910 ; M012 : BEGIN
25911 ;
25912 ; 23/10/2022
25913 update_arena:
25914     push    ds ; ds = cs
25915     push    di
25916     push    cx
25917     push    dx
25918     ; 23/10/2022
25919     ;lds    di,[cs:DOSINFO] ; get ptr to DOS var
25920     ; 11/12/2022
25921     ; ds = cs
25922     lds     di,[DOSINFO] ; 27/03/2019
25923     dec     di
25924     dec     di ; Arena head is immediately
25925     ; before sysvar
25926     mov     es,[di] ; es = arena head
25927     ;mov    cx,[es:ARENA.SIZE] ; cx = total low mem size
25928     mov     cx,[es:3]
25929     cmp     cx,bx ; is it sufficient ?
25930     jnb     short FatalErr ; no, fatal error
25931
25932     ;mov     dl,[es:ARENA.SIGNATURE]
25933     mov     dl,[es:0]
25934     mov     ax,es
25935     add     ax,bx ; ax = new arena head
25936     mov     [di],ax ; store it in DOS data area
25937     mov     ds,ax
25938     ;mov     [ARENA.SIGNATURE],dl ; type of arena
25939     mov     [0],dl
25940     ;mov     word [ARENA.OWNER],0 ; free
25941     mov     word [1],0
25942     sub     cx,bx ; size of the new block
25943     ;mov     [ARENA.SIZE],cx ; store it in the arena
25944     mov     [3],cx
25945     mov     ax,es ; return seg to the caller
25946     ; 23/10/2022
25947     ; 24/03/2019 - Retro DOS v4.0 (ORG 0) ; Take care ORG 30h of
25948     sub     ax,3 ; BIOS code
25949     pop     dx
25950     pop     cx
25951     pop     di
25952     pop     ds ; ds = cs
25953     ret     0
25954 ;
25955 ; M012 : END
25956 ;
25957 FatalErr:
25958     push    cs
25959     pop     ds
25960     mov     dx,FErrorMsg
25961     mov     ah,9
25962     int     21h ; DOS - PRINT STRING
25963     ; 30/12/2022 (MSDOS 6.21 SYSINIT) ; DS:DX -> string terminated by "$"
25964     jmp     stall
25965     ; 23/10/2022 (MSDOS 5.0 SYSINIT)
25966     ;cli
25967     ;hlt
25968
25969 ;endif ;ROMDOS
25970
25971 ; 25/03/2019 - Retro DOS v4.0
25972 ;
25973 ; -----
25974 ;
25975 ; procedure : AllocHMA
25976 ;
25977 ; grab_the_hma tries to enable a20 and make sure there is memory
25978 ; up there. If it gets any sort of error, it will return with
25979 ; carry set so that we can resort to running low.
25980 ;
25981 ; It also returns ES: -> 0ffffh if it returns success
25982 ;
25983 ; -----
25984 ;
25985 AllocHMA:
25986 ; cas note: The pre-286 check is no longer needed here since the
25987 ; presence of XMS is sufficient. However, this code hasn't
25988 ; been deleted because it can be recycled for skipping the
25989 ; extra pass of CONFIG.SYS and assuming we're running low
25990 ; in the case of a pre-286.
25991 ;
25992 ; see if we're running on a pre-286. If not, force low.
25993 ;
25994 ;
25995     xor     ax,ax
25996     pushf   ; save flags (like int)
25997     push    ax
25998     popf
25999     pushf
26000     pop     ax
26001     popf   ; restore original flags (like int)
26002     and     ax,0F000h
26003     cmp     ax,0F000h ; 8088/8086?
26004     jz      short grab_hma_error
26005
26006     ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
26007     ; (SYSINIT:0A26h)
26008
26009     ; 13/04/2024 - Retro DOS v5.0
26010     ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:0C41h)
26011
26012     push    ds
26013     ;mov     ax,Bios_Data
26014     ;mov     ax,KERNEL_SEGMENT
26015     ; 21/10/2022
26016     mov     ax,DOSBIODATASEG ; 70h
26017     mov     ds,ax
26018
26019     call    IsXMSLoaded
26020     jnz     short grabhma_error
26021

```

```

26022 00000B92 B81043      mov     ax,4310h
26023 00000B95 CD2F        int     2Fh          ; get the vector into es:bx
26024                                ; - Multiplex - XMS - GET DRIVER ADDRESS
26025                                ; Return: ES:BX -> driver entry point
26026
26027 00000B97 891E[0E00]    mov     [xms],bx
26028                                ;mov     [0Eh], bx
26029 00000B9B 8C06[1000]    mov     [xms+2],es
26030                                ;mov     [10h],es
26031
26032 00000B9F B401          mov     ah,1          ; request HMA
26033 00000BA1 BAFFFF        mov     dx,0FFFFh
26034                                ;call    dword ptr ds:0Eh
26035 00000BA4 FF1E[0E00]    call    far [xms]
26036 00000BA8 48            dec     ax
26037 00000BA9 7409          jz      short allocHMA_1 ; error if not able to allocate HMA
26038
26039                                ;----- Himem may be lying because it has allocated mem for int 15
26040
26041 00000BAB B488          mov     ah,88h
26042 00000BAD CD15        int     15h
26043                                ; Get Extended Memory Size
26044                                ; Return: CF clear on success
26045                                ; AX = size of memory above 1M in K
26046 00000BAF 83F840        cmp     ax,64          ; less than 64 K of hma ?
26047                                ;jb      short grabhma_error
26048                                ; 11/12/2022
26049 00000BB2 7224          jnb     short grabhma_err ; cf=1
26050
26051 00000BB4 B405          mov     ah,5          ; localenableA20
26052                                ;call    dword ptr ds:0Eh
26053 00000BB6 FF1E[0E00]    call    far [xms]
26054 00000BBA 48            dec     ax
26055 00000BBB 751A          jnz     short grabhma_error ; error if couldn't enable A20
26056
26057 00000BBD E89D01        call    IsVDiskInstalled
26058 00000BC0 7415          jz      short grabhma_error ; yes, we cant use HMA
26059
26060 00000BC2 B8FFFF        mov     ax,0FFFFh
26061 00000BC5 8EC0          mov     es,ax
26062 00000BC7 26C70610003412 mov     word [es:10h],1234h ; see if we can really read/write there
26063 00000BCE 26813E10003412 cmp     word [es:10h],1234h
26064                                ;jne     short grabhma_error ; don't try to load there if XMS lied
26065                                ; 11/12/2022
26066 00000BD5 7401          je      short allocHMA_ok
26067
26068                                ; 11/12/2022
26069                                ; 11/12/2022
26070                                ; cf=0
26071                                ; c1c
26072                                ; pop     ds
26073                                ; retn
26074
26075 grabhma_error:
26076 00000BD7 F9            stc
26077                                ; 11/12/022
26078 grabhma_err:          ; cf=1
26079 allocHMA_ok:          ; cf=0
26080                                pop     ds
26081                                retn
26082
26083                                ; -----
26084                                ;
26085                                ; procedure : IsXMSLoaded
26086                                ;
26087                                ; Checks whether a XMS driver is loaded
26088                                ;
26089                                ; Returns : Z flag set if XMS driver loaded
26090                                ; Z flag reset if no XMS drivers are present
26091                                ;
26092                                ; -----
26093
26094 IsXMSLoaded:
26095 00000BDA B80043        mov     ax,4300h
26096 00000BDD CD2F        int     2Fh          ; - Multiplex - XMS - INSTALLATION CHECK
26097                                ; Return: AL = 80h XMS driver installed
26098                                ; AL <> 80h no driver
26099 00000BDF 3C80          cmp     al,80h        ; XMS installed?
26100 00000BE1 C3            retn
26101
26102                                ; -----
26103                                ; procedure : FTryToMovDOSHi
26104                                ;
26105                                ; Called from HMA suballoc calls
26106                                ;
26107                                ; -----
26108
26109                                ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
26110                                ; (MSDOS 5.0 IO.SYS - SYSINIT:0A84h)
26111
26112                                ; 08/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
26113                                ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:0C9Fh)
26114
26115                                ; ((MSDOS 6.22 IO.SYS - SYSINIT:0B8Ch))
26116
26117 FTryToMovDOSHi:        ; proc far
26118
26119 00000BE2 50            push    ax
26120 00000BE3 53            push    bx
26121 00000BE4 51            push    cx
26122 00000BE5 52            push    dx
26123 00000BE6 56            push    si
26124 00000BE7 57            push    di
26125 00000BE8 1E            push    ds
26126 00000BE9 06            push    es
26127
26128                                ; 23/10/2022
26129                                ; 27/03/2019 - Retro DOS v4.0
26130                                ; 11/12/2022
26131 00000BEA 0E            push    cs
26132 00000BEB 1F            pop     ds
26133
26134                                ;cmp     byte [cs:runhigh],0FFh
26135                                ; 11/12/2022
26136 00000BEC 803E[6C02]FF    cmp     byte [runhigh],0FFh
26137 00000BF1 7503          jne     short _ftymdh_1
26138
26139                                ; ds = cs
26140 00000BF3 E8A2FE        call    TryToMovDOSHi
26141
26142 _ftymdh_1:            pop     es
26143                                pop     ds
26144                                pop     di
26145                                pop     si

```

```

26146 00000BFA 5A      pop     dx
26147 00000BFB 59      pop     cx
26148 00000BFC 5B      pop     bx
26149 00000BFD 58      pop     ax
26150
26151 00000BFE CB      retf
26152
26153 ; -----
26154 ;
26155 ; following piece of code will be moved into a para boundary. And the para
26156 ; address posted in seg of int 19h vector. Offset of int 19h will point to
26157 ; VdInt19. This is to protect HMA from apps which use VDISK header method
26158 ; to determine free extended memory.
26159 ;
26160 ; For more details read "power programming" column by Ray Duncan in the
26161 ; May 30 1989 issue of PC Magazine (pp 377-388) [USING EXTENDED MEMORY,PART 1]
26162 ;
26163 ; -----
26164
26165 ; 30/12/2023 - Retro DOS 5.0
26166 00000BFF 00      db     0
26167
26168 ; 13/04/2024
26169 ;align 2
26170
26171 ; 30/12/2023
26172 ; PCDOS v7.1 IBMBIO.COM, SYSYINIT:0CBCh
26173
26174 StartVDHead:
26175 ;----- what follows is a dummy device driver header (not used by DOS)
26176
26177 00000C00 00000000      dd     0          ; link to next device driver
26178 00000C04 0080      dw     8000h        ; device attribute
26179 00000C06 0000      dw     0          ; strategy routine offset
26180 00000C08 0000      dw     0          ; interrupt routine offset
26181 00000C0A 01      db     1          ; number of units
26182 ;db     7 dup(0)
26183 00000C0B 00<rep 7h>      times 7 db 0      ; reserved area
26184
26185 00000C12 564449534B      db     'VDISK'
26186
26187 VLEN1      equ     ($-VDiskSig1)
26188
26189 00000C17 202056332E33      db     ' v3.3'      ; vdisk label
26190 ;db     15 dup (0)      ; pad
26191 00000C1D 00<rep Fh>      times 15 db 0
26192 00000C2C 0000      dw     0          ; bits 0-15 of free HMA
26193 00000C2E 11      db     11h        ; bits 16-23 of free HMA (1M + 64K)
26194
26195 00000C2F EA      db     0EAh        ; jmp to old vector
26196
26197 00000C30 00000000      oldvDInt19: dd 0          ; Saved int 19 vector
26198
26199 EndVDHead: ; label byte
26200
26201 VDiskHMAHead:
26202 00000C34 000000      db     0,0,0          ; non-bootable disk
26203
26204 00000C37 564449534B      db     'VDISK'
26205
26206 VLEN2      equ     ($-VDiskSig2)
26207
26208 00000C3C 332E33      db     '3.3'          ; OEM - signature
26209 00000C3F 8000      dw     128         ; number of bytes/sector
26210 00000C41 01      db     1          ; sectors/cluster
26211 00000C42 0100      dw     1          ; reserved sectors
26212 00000C44 01      db     1          ; number of FAT copies
26213 00000C45 4000      dw     64         ; number of root dir entries
26214 00000C47 0002      dw     512        ; number of sectors
26215 00000C49 FE      db     0FEh        ; media descriptor
26216 00000C4A 0600      dw     6          ; number of sectors/FAT
26217 00000C4C 0800      dw     8          ; sectors per track
26218 00000C4E 0100      dw     1          ; number of heads
26219 00000C50 0000      dw     0          ; number of hidden sectors
26220 00000C52 4004      dw     440h        ; Start of free HMA in K (1M+64K)
26221
26222 EndVDiskHMAHead: ; label byte
26223
26224 ; -----
26225 ;
26226 ; procedure : InstVDiskHeader
26227 ;
26228 ; Installs the VDISK header to reserve the 64k of HMA
26229 ; It puts a 32 byte header at 10000:0 and
26230 ; another header at (seg of int19):0
26231 ;
26232 ; Inputs : None
26233 ;
26234 ; Outputs : None
26235 ;
26236 ; USES : DS,SI,AX,CX,DX
26237 ;
26238 ; -----
26239
26240 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
26241
26242 InstVDiskHeader:
26243 00000C54 31C0      xor     ax,ax
26244 00000C56 8ED8      mov     ds,ax          ; seg of int vect table
26245
26246 ;----- save old int 19 vector
26247
26248 ; 23/10/2022
26249 00000C58 A16400      mov     ax,[19h*4]
26250 ;mov     [oldvDInt19],ax
26251 00000C5B 2EA3[300C]      mov     [cs:oldvDInt19],ax
26252 00000C5F A16600      mov     ax,[19h*4+2]
26253 ;mov     [oldvDInt19+2],ax
26254 00000C62 2EA3[320C]      mov     [cs:oldvDInt19+2],ax
26255
26256 ;----- calculate seg of new int 19 handler
26257
26258 00000C66 B448      mov     ah,48h        ; allocate memory
26259 ;mov     bx,(EndVDHead-StartVDHead+15)>>4
26260 ; 23/10/2022
26261 00000C68 BB0400      mov     bx,4
26262 00000C6B CD21      int     21h
26263
26264 ; if carry, fatal hanging error!!!!
26265
26266 00000C6D 48      dec     ax          ; point to arena
26267 00000C6E 8EC0      mov     es,ax
26268 ;mov     word [es:ARENA.OWNER],8          ; owner = System
26269 00000C70 26C70601000800      mov     word [es:1],8

```



```

26270          ;mov     word [es:ARENA.NAME],'SC' ; System Code
26271 00000C77 26C70608005343      mov     word [es:8],'SC' ; 4353h
26272 00000C7E 40                  inc     ax
26273 00000C7F 8EC0                mov     es,ax          ; get back to allocated memory
26274
26275          ;----- install new int 19 vector
26276
26277 00000C81 FA                    cli          ; no reboots at this time
26278          ;mov     word [19h*4],(VDInt19-StartVDHead)
26279 00000C82 C70664002F00      mov     word [19h*4],47
26280 00000C88 A36600            mov     [19h*4+2],ax
26281
26282          ;----- move the code into proper place
26283
26284          ;mov     cx,(EndVDHead-StartVDHead)
26285 00000C8B B93400            mov     cx,52
26286 00000C8E BE[000C]        mov     si,StartVDHead
26287 00000C91 31FF            xor     di,di
26288 00000C93 0E              push    cs
26289 00000C94 1F              pop     ds
26290 00000C95 FC              cld
26291 00000C96 F3A4            rep     movsb
26292 00000C98 FB                    sti          ; BUGBUG is sti OK now?
26293
26294          ;----- mov the HMA VDisk head into HMA
26295
26296          ; 23/10/2022
26297 00000C99 57              push    di
26298 00000C9A 06              push    es
26299
26300          ;mov     ax,0FFFFh
26301          ;mov     es,ax
26302          ; 03/09/2023
26303 00000C9B 49              dec     cx
26304          ; cx = 0FFFFh
26305 00000C9C 8EC1            mov     es,cx
26306
26307          mov     di,10h
26308          ;mov     cx,(EndVDiskHMAHead-VDiskHMAHead)
26309 00000CA1 B92000            mov     cx,32
26310 00000CA4 BE[340C]        mov     si,VDiskHMAHead
26311 00000CA7 F3A4            rep     movsb          ; ds already set to cs
26312
26313          pop     di
26314 00000CAA 07              pop     es
26315
26316 00000CAB C3              retn
26317
26318          ; -----
26319          ; procedure : ClrVDISKHeader
26320          ;
26321          ; clears the first 32 bytes at 1MB boundary
26322          ; so that DOS/HIMEM is not confused about the VDISK header
26323          ; left by previous DOS=HIGH session
26324          ; -----
26325
26326          struc desc
26327          .seg_lim: resw 1          ; seg limit 64K
26328 00000000 ????            .lo_word: resw 1          ; 24 bit seg physical
26329 00000002 ????            .hi_byte: resb 1          ; address
26330 00000004 ??             .acc_rights: resb 1          ; access rights ( CPL0 - R/W )
26331 00000005 ??             .reserved: resw 1          ;
26332 00000006 ????            .size:
26333          endstruc
26334
26335          ; 23/10/2022
26336          bmove:          ;label byte
26337
26338          dummy:          ;times desc.size db 0 ; desc <>
26339          times 8 db 0
26340 00000CAC 00<rep 8h>      gdt:          ;times desc.size db 0 ; desc <>
26341          times 8 db 0
26342 00000CB4 00<rep 8h>      src_desc: dw 0FFFFh          ; desc <0ffffh,0,0,93h,0>
26343 00000CBC FFFF          dw 0
26344 00000CBE 0000          db 0
26345 00000CC0 00            db 93h
26346 00000CC1 93            dw 0
26347 00000CC2 0000          tgt_desc: dw 0FFFFh          ; desc <0ffffh,0,10h,93h,0> ; 1MB
26348 00000CC4 FFFF          dw 0
26349 00000CC6 0000          db 10h
26350 00000CC8 10            db 93h
26351 00000CC9 93            dw 0
26352 00000CCA 0000
26353
26354          rombios_code:    ;times desc.size db 0 ; desc <>
26355 00000CCC 00<rep 8h>      times 8 db 0
26356          temp_stack:    ;times desc.size db 0 ; desc <>
26357 00000CD4 00<rep 8h>      times 8 db 0
26358
26359 00000CDC 00<rep 20h>      CldrVDISKHead: times 32 db 0          ; db 32 dup (0)
26360
26361          ; 25/03/2019 - Retro DOS v4.0 (MSDOS 6.21 IO.SYS, MSDOS 6.0 SYSINIT1.ASM)
26362
26363          ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
26364          ; (SYSINIT:0CA6h)
26365
26366          CldrVDISKHeader: ; proc near
26367
26368          ; 04/07/2023 - Retro DOS v4.2 IO.SYS (optimized)
26369          ;-----
26370          ; The following workaround get around a problem with the
26371          ; Tortugas and PS/2 30-286 BIOS when password server mode
26372          ; is set. On those machines the INT 15h block move code
26373          ; goes through the 8042 to twiddle A20 instead of port 92h.
26374          ; In password server mode the 8042 is disabled so the block
26375          ; move crashes the system. We can do this because these
26376          ; systems clear all of memory on a cold boot.
26377          ;-----
26378          ;
26379          in     al,64h          ; Test for password servr mode
26380          test   al,10h          ; Is keyboard inhibited?
26381          jnz    short CldrVDISKok ; No, go do block move.
26382          ;
26383          cmp     word [cs:sys_model_byte],19F8h
26384          je      short CldrVDISKno ; Check for Tortugas...
26385          ;
26386          cmp     word [cs:sys_model_byte],09Fch
26387          jne     short CldrVDISKok ; Check for mod 30-286
26388          CldrVDISKno: retn          ; Return w/o block move.
26389          CldrVDISKok:
26390          ;-----
26391          ; 30/12/2023 - Retro DOS v5.0

```

```

26394 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0DBAh
26395 ClrVDISKHeader:
26396 00000CFC E464 in al,64h ; 8042 keyboard controller status register
26397 ; 7: PERR 1=parity error in data received from keyboard
26398 ; +----- AT Mode -----+----- PS/2 Mode -----+
26399 ; 6: |RxTO receive (Rx) timeout | TO general timeout (Rx or Tx) |
26400 ; 5: |TxTO transmit (Tx) timeout | MOBF mouse output buffer full |
26401 ; +-----+-----+
26402 ; 4: INH 0=keyboard communications inhibited
26403 ; 3: A2 0=60h was the port last written to, 1=64h was last
26404 ; 2: SYS distinguishes reset types: 0=cold reboot, 1=warm reboot
26405 ; 1: IBF 1=input buffer full (keyboard can't accept data)
26406 ; 0: OBF 1=output buffer full (data from keyboard is available)
26407 00000CFE A810 test al,10h ; test bit 4 - Is keyboard inhibited?
26408 00000D00 7511 jnz short ClrVDISKok ; No, go do block move
26409 ; 30/12/2023
26410 ; ds = cs
26411 00000D02 813E[BB02]F819 cmp word [sys_model_byte],19F8h ; check for TORTUGA models
26412 00000D08 7408 jz short ClrVDISKno ; do not use INT 15h block move code
26413 ; (while 8042 is disabled)
26414 00000D0A 813E[BB02]FC09 cmp word [sys_model_byte],9FCh ; check for PS/2 30-286 model
26415 00000D10 7501 jnz short ClrVDISKok
26416 ClrVDISKno:
26417 00000D12 C3 retn
26418 ; -----
26419 ; 30/12/2023
26420 ClrVDISKok:
26421 ; 12/12/2022
26422 ; ds = cs
26423 ;
26424 ; 30/12/2022 - Retro DOS v4.2
26425 ; (MSDOS 6.21 IO.SYS SYSINIT:0CBFh)
26426
26427 00000D13 06 push es
26428 00000D14 8CC8 mov ax,cs
26429 00000D16 89C2 mov dx,ax
26430 00000D18 B10C mov cl,12
26431 00000D1A D3EA shr dx,cl
26432 00000D1C B104 mov cl,4
26433 00000D1E D3E0 shl ax,cl
26434 00000D20 05[DC0C] add ax,ClrVDISKHead
26435 00000D23 80D200 adc dl,0
26436
26437 ; 23/10/2022
26438 ; mov [cs:src_desc+desc.lo_word],ax
26439 ; mov [cs:src_desc+2],ax
26440 ; mov [cs:src_desc+desc.hi_byte],dl
26441 ; mov [cs:src_desc+4],dl
26442 ; 12/12/2022
26443 ; mov [src_desc+desc.lo_word],ax
26444 00000D26 A3[BE0C] mov [src_desc+2],ax
26445 ; mov [src_desc+desc.hi_byte],dl
26446 00000D29 8816[C00C] mov [src_desc+4],dl
26447
26448 00000D2D B91000 mov cx,16 ; 16 words
26449 00000D30 0E push cs
26450 00000D31 07 pop es
26451 00000D32 BE[AC0C] mov si,bmove
26452 00000D35 B487 mov ah,87h
26453 00000D37 CD15 int 15h ; EXTENDED MEMORY - BLOCK MOVE (AT,XT286,PS)
26454 ; CX = number of words to move
26455 ; ES:SI -> global descriptor table
26456 ; Return: CF set on error, AH = status
26457 00000D39 07 pop es
26458 00000D3A C3 retn
26459
26460 ; -----
26461 ;
26462 ; procedure : SaveFreeHMAPtr
26463 ;
26464 ; Save the Free HMA pointer in BIOS variable for later use.
26465 ; (INT 2f ax==4a01 call returns pointer to free HMA)
26466 ; Normalizes the pointer to ffff:xxxx format and stores only
26467 ; the offset.
26468 ;
26469 ; Inputs : ES:DI - pointer to free HMA
26470 ; Output : FreeHMAPtr in BIOS data segment updated
26471 ; -----
26472 ;
26473 SaveFreeHMAPtr:
26474 ; 03/09/2023
26475 push ds
26476 00000D3B 1E mov ax,DOSBIODATASEG ; 0070h
26477 00000D3C B87000 mov ds,ax
26478 00000D3F 8ED8 ;
26479 ;
26480 00000D41 8CC3 mov bx,es
26481 00000D43 B8FFFF mov ax,0FFFFh ; HMA segment
26482 ; 03/09/2023
26483 00000D46 A2[0D00] mov [inHMA],al ; 0FFh ; (BIOSDATA:000Dh) ; 08/04/2024
26484 ;
26485 00000D49 29D8 sub ax,bx
26486 00000D4B 83C70F add di,15 ; para round
26487 00000D4E 83E7F0 and di,0FFF0h
26488 00000D51 B104 mov cl,4
26489 00000D53 D3E0 shl ax,cl
26490 00000D55 29C7 sub di,ax
26491 ;
26492 ; 03/09/2023
26493 ; push ds
26494 ; mov ax,Bios_Data ; 0070h
26495 ; mov ax,KERNEL_SEGMENT ; 0070h
26496 ; 21/10/2022
26497 ; 03/09/2023
26498 ; mov ax,DOSBIODATASEG ; 0070h
26499 ; mov ds,ax
26500 ; (BIOSDATA:07D7h for PCDOS 7.1 IBMBIO.COM) ; 08/04/2024
26501 00000D57 893E[D707] mov [FreeHMAPtr],di ; (ds:8F7h for MSDOS 6.21 IO.SYS)
26502 ; mov byte [inHMA],0FFh ; (ds:0Dh)
26503 00000D5B 1F pop ds
26504 00000D5C C3 retn
26505
26506 ; -----
26507 ;
26508 ; procedure : IsVDiskInstalled
26509 ;
26510 ; Checks for the presence of VDISK header at 1MB boundary
26511 ; & INT 19 vector
26512 ;
26513 ; Inputs : A20 flag should be ON
26514 ; Outputs : Zero set if VDISK header found else Zero cleared
26515 ; -----
26516 ;
26517

```



```

26518
26519 00000D5D 31C0
26520 00000D5F 8ED8
26521 00000D61 8E1E4E00
26522
26523
26524 00000D65 BE1200
26525
26526 00000D68 B90500
26527 00000D68 0E
26528 00000D6C 07
26529 00000D6D BF[120C]
26530 00000D70 F3A6
26531 00000D72 740F
26532 00000D74 B8FFFF
26533 00000D77 8ED8
26534
26535 00000D79 BE1300
26536 00000D7C BF[370C]
26537
26538
26539
26540 00000D7F B105
26541 00000D81 F3A6
26542
26543 00000D83 C3
26544
26545
26546
26547
26548
26549
26550
26551
26552
26553
26554
26555
26556 00000D84 1E
26557 00000D85 B9FFFF
26558 00000D88 8EC1
26559
26560
26561 00000D8A 41
26562 00000D8B 8ED9
26563 00000D8D BEC000
26564 00000D90 BFD000
26565
26566 00000D93 B105
26567 00000D95 FC
26568 00000D96 F3A4
26569 00000D98 1F
26570 00000D99 C3
26571
26572
26573
26574
26575
26576
26577
26578 00000D9A D1E8
26579 00000D9C D1E8
26580 00000D9E D1E8
26581 00000DA0 D1E8
26582 00000DA2 C3
26583
26584
26585
26586
26587
26588
26589
26590
26591
26592
26593
26594
26595
26596
26597 00000DA3 C43E[6D02]
26598 00000DA7 268A4D20
26599
26600
26601 00000DAB 30ED
26602
26603 00000DAD 26884D21
26604
26605
26606
26607
26608
26609
26610
26611 00000DB1 B058
26612
26613 00000DB3 F6E1
26614
26615 00000DB5 E85D05
26616 00000DB8 8B36[A702]
26617
26618
26619
26620
26621
26622
26623
26624 00000DBC 29C6
26625
26626
26627
26628
26629
26630
26631
26632
26633
26634
26635
26636
26637
26638
26639
26640 00000DBE 26897518
26641

```

```

IsVDiskInstalled:
    xor     ax,ax
    mov     ds,ax
    mov     ds,[19*4+2]
    ;mov     si,VDiskSig1-StartVDHead ; 12h
    ; 23/10/2022
    mov     si,12h ; 18
    ;mov     cx,VLEN1 ; 5
    mov     cx,5
    push    cs
    pop     es
    mov     di,vdiskSig1
    rep     cmpsb
    je      short ivdins_retn
    mov     ax,0FFFFh
    mov     ds,ax
    ;mov     si,10h+(VDiskSig2-VDiskHMAHead) ; 13h
    mov     si,13h
    mov     di,VDiskSig2
    ;;mov     cx,VLEN2 ; 5
    ;mov     cx,5
    ; 03/09/2023
    mov     cl,5
    rep     cmpsb
ivdins_retn:
    retn                                ; returns the zero flag
;
; -----
;
; procedure : CPMHack
;
;         Copies the code from 0:c0 into ffff:0d0h
;         for CPM compatibility
;
; -----
;
; 11/12/2022
CPMHack:
    push    ds
    mov     cx,0FFFFh
    mov     es,cx                ; ES = FFFF
    ;xor     cx,cx
    ; 11/12/2022
    inc     cx ; cx = 0
    mov     ds,cx                ; DS = 0
    mov     si,0C0h
    mov     di,0D0h
    ;mov     cx,5
    mov     cl,5
    rep     movsb                ; move 5 bytes from 0:C0 to FFFF:D0
    pop     ds
    retn
;
; -----
;
; procedure : off_to_para
;
; -----
off_to_para:
    shr     ax,1
    shr     ax,1
    shr     ax,1
    shr     ax,1
    retn
;
; -----
;
; ** TempCDS - Create (Temporary?) CDS
;
; ENTRY    ?? BUGBUG
;          (DS) = SysInitSeg
; EXIT     ?? BUGBUG
; USES     ?? BUGBUG
;
; -----
;
; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; 30/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
TempCDS:
    les     di,[DOSINFO]
    mov     cl,[es:di+SYSI_NUMIO]
;
;mov     cl,[es:di+20h]
xor     ch,ch                ; (cx) = # of block devices
;
mov     [es:di+SYSI_NCDS],cl ; one CDS per device
;mov     [es:di+21h],cl
;
;mov     al,cl
;mov     ah,curdirilen ; curdir_list.size ; 88
;mov     ah,88
;mul     ah                ; (ax) = byte size for those CDSs
; 30/12/2023
mov     al,curdirilen ; curdir_list.size ; 88
;mov     al,88
;mul     cl                ; (ax) = byte size for those CDSs
;
call     ParaRound           ; (ax) = paragraph size for CDSs
mov     si,[top_of_cdss] ; 31/12/2022
;
; BUGBUG - we don't update confbot - won't someone else use it?
; chunkst -- answer: no. Confbot is used to access the CDSs,
; 25 Jul 92 which are stored BELOW it. Alloclim is the
; variable which has the top of free memory for
; device driver loads, etc.
;
sub     si,ax
;
; chunkst, 25 Jul 92 -- note: I'm removing the code here
; that automatically updates alloclim every time we
; set up some new CDSs. Instead, I've added code
; which pre-allocates space for 26 CDSs. This
; way we've got room for worst case CDSs before
; we place MagicDrv.sys
;
mov     [ALLOCIM],si        ; can't alloc past here!
;
; 30/12/2022
; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; (SYSINIT:0C52h)
;mov     [ALLOCIM],si ; (MSDOS 5.0 SYSINIT)
;
mov     [es:di+SYSI_CDS+2],si
;mov     [es:di+18h],si

```

```

26642 00000DC2 89F0      mov     ax,si
26643 00000DC4 26C745160000      mov     word [es:di+SYSI_CDS],0      ; set address of CDS list
26644      ;mov     [word es:di+16h],0
26645      ;lds     si,[es:di+SYSI_DPB]      ; (ds:si) = address of first DPB
26646 00000DCA 26C535      lds     si,[es:di]
26647 00000DCD 8EC0      mov     es,ax
26648 00000DCF 31FF      xor     di,di      ; (es:di) = address of 1st CDS
26649
26650      ;* Initialize our temporary CDSs. We'll init each CDS with the
26651      ; info from the corresponding DPB.
26652      ;
26653      ; (cx) = count of CDSs left to process
26654      ; (es:di) = address of next CDS
26655
26656 fooset:
26657      ; 23/10/2022
26658 00000DD1 2EA1[A902]      mov     ax,[cs:DirStrng] ; "A:"
26659 00000DD5 AB      stosw      ; setup the root as the curdir
26660
26661      ; 23/10/2022 (MSDOS 5.0 SYSINIT)
26662      ; call get_dpb_for_drive_a1 ; get dpb for drive in dpb
26663
26664      ; 30/12/2022
26665      ; (MSDOS 6.21 SYSINIT:0D8Bh)
26666 00000DD6 E85200      call    get_dpb_for_drive_a1 ; get dpb for drive in dpb
26667
26668      ; (ds:si) = address of DPB
26669      ; (si) = -1 if no drive
26670
26671 00000DD9 2EA1[AB02]      mov     ax,[cs:DirStrng+2] ; "\",0
26672 00000DDD AB      stosw
26673 00000DDE 2EFE06[A902]      inc     byte [cs:DirStrng]
26674 00000DE3 31C0      xor     ax,ax ; 0
26675 00000DE5 51      push    cx
26676      ;mov     cx,curdir_list.cdir_flags - 4 ; 63
26677 00000DE6 B93F00      mov     cx,63 ; 23/10/2022
26678 00000DE9 F3AA      rep     stosb      ; zero out rest of CURDIR_TEXTs
26679
26680      ; should handle the system that does not have any floppies.
26681      ; in this case, we are going to pretend there are two dummy floppies
26682      ; in the system. still they have dpb and cds, but we are going to
26683      ; 0 out curdir_flags, curdir_devptr of cds so ibmdos can issue
26684      ; "invalid drive specification" message when the user try to
26685      ; access them.
26686      ;
26687      ; (ax) = 0
26688      ; (es:di) = CURDIR_FLAGS in the CDS records
26689      ; (ds:si) = Next DPB (-1 if none)
26690
26691 00000DEB 83FEFF      cmp     si,-1 ; cmp si,0FFFFh
26692 00000DEE 740C      je      short fooset_zero ; don't have any physical drive.
26693
26694      ; check to see if we are faking floppy drives. if not go to normcds.
26695      ; if we are faking floppy drives then see if this cds being initialised
26696      ; is for drive a: or b: by checking the appropriate field in the dpb
26697      ; pointed to by ds:si. if not for a: or b: then go to normcds. if
26698      ; for a: or b: then execute the code given below starting at fooset_zero.
26699      ; for dpb offsets look at inc\dpb.inc.
26700
26701      ; 03/09/2023
26702 00000DF0 41      inc     cx ; cx = 1
26703
26704 00000DF1 2E380E[8B02]      cmp     [cs:fake_floppy_drv],cl ; 1 ; 03/09/2023
26705      ; cmp     byte [cs:fake_floppy_drv],1
26706 00000DF6 750A      jne     short normcds ; machine has floppy drives
26707      ; cmp     byte [si+DPB.drive],1 ; if dpb_drive = 0 (a) or 1 (b).
26708      ; cmp     byte [si],1
26709 00000DF8 380C      cmp     [si],cl ; 1 ; 03/09/2023
26710 00000DFA 7706      ja      short normcds
26711
26712      ; 30/12/2023
26713      ; ax = 0
26714 fooset_zero:
26715 00000DFC B103      mov     cl,3 ; the next dpb pointer
26716      ; AX should be zero here
26717 00000DFE F3AB      rep     stosw
26718      ; 30/12/2023
26719      ; pop     cx
26720 00000E00 EB0F      jmp     short get_next_dpb ; findcds
26721
26722      ; (ax) = 0
26723
26724      ; 30/12/2023
26725 ;fooset_zero:
26726      ; mov     cl,3
26727      ; rep     stosw
26728      ; pop     cx
26729      ; jmp     short findcds
26730
26731      ;* We have a "normal" DPB and thus a normal CDS.
26732      ;
26733      ; (ax) = 0
26734      ; (es:di) = CURDIR_FLAGS in the CDS records
26735      ; (ds:si) = Next DPB (-1 if none)
26736
26737 normcds:
26738      ; 30/12/2023
26739      ; pop     cx
26740
26741      ; if a non-fat based media is detected (by dpb.numberoffat == 0), then
26742      ; set curdir_flags to 0. this is for signaling ibmdos and ifsfunc that
26743      ; this media is a non-fat based one.
26744
26745      ; cmp     byte [si+DPB.FAT_COUNT],0 ; non fat system?
26746      ; 23/10/2022
26747      ; cmp     byte [si+8],0
26748      ; 03/09/2023 (ax=0)
26749 00000E02 384408      cmp     [si+8],al ; 0
26750 00000E05 7403      je      short setnormcds ; yes. set curdir_flags to 0. ax = 0 now.
26751 00000E07 B80040      mov     ax,curdir_inuse ; 4000h ; else, fat system. set the flag to curdir_inuse.
26752      ; mov     ax,4000h
26753 setnormcds:
26754 00000E0A AB      stosw      ; curdir_flags
26755 00000E0B 89F0      mov     ax,si
26756 00000E0D AB      stosw      ; curdir_devptr
26757 00000E0E 8CD8      mov     ax,ds
26758 00000E10 AB      stosw
26759
26760 get_next_dpb: ; entry point for fake_fooset_zero
26761      ; 30/12/2022
26762      ; (MSDOS 6.21 SYSINIT:0DD1h)
26763      ; 23/10/2022
26764      ; lds     si,[si+19h] ; (MSDOS 5.0 SYSINIT)
26765      ; ; lds     si,[si+DPB.NEXT_DPB] ; [si+19h]

```

```

26766 fincds: ; get_next_dpb
26767 ; 30/12/2023
26768 00000E11 59 pop cx
26769 ; 30/12/2022
26770 ; (MSDOS 6.21 SYSINIT:0DD1h)
26771 00000E12 B8FFFF mov ax,-1 ; mov ax,0FFFFh
26772 00000E15 AB stosw ; curdir_id
26773 00000E16 AB stosw ; curdir_id
26774 00000E17 AB stosw ; curdir_user_word
26775 00000E18 B80200 mov ax,2
26776 00000E1B AB stosw ; curdir_end
26777 00000E1C B000 mov al,0 ; clear out 7 bytes (curdir_type,
26778 00000E1E AA stosb ; curdir_ifs_hdr,curdir_fsda)
26779 00000E1F AB stosw
26780 00000E20 AB stosw
26781 00000E21 AB stosw
26782
26783 00000E22 E2AD loop fooset
26784
26785 00000E24 2EC606[A902]41 mov byte [cs:DirStrng],"A"; "A:\"
26786
26787 00000E2A C3 retn
26788
26789 ;-----
26790 ;**get_dpb_for_drive_al -- lookup the DPB for drive in al
26791 ;
26792 ; entry:
26793 ; al == ASCII CAPS drive letter
26794 ;
26795 ; exit:
26796 ; ds:si -> DPB, or si = -1 if not found
26797 ;-----
26798
26799 ; 30/12/2023
26800 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0EFEh
26801
26802 ; 30/12/2022
26803 ; (MSDOS 6.21 SYSINIT:0DEAh)
26804 ; 23/10/2022
26805 get_dpb_for_drive_al:
26806 00000E2B 2EC536[6D02] lds si,[cs:DOSINFO] ; point to first DPB
26807 ; lds si,[si+SYSI_DPB] ; (ds:si) = address of first DPB
26808 00000E30 C534 lds si,[si]
26809 00000E32 2C41 sub al,'A'
26810
26811 get_dpb_for_drive_1:
26812 ; cmp al,[si+DPB.DRIVE] ; match?
26813 00000E34 3A04 cmp al,[si]
26814 00000E36 7408 je short got_dpb_for_drive ; done if so
26815
26816 lds si,[si+DPB.NEXT_DPB] ; [si+19h]
26817 00000E3B 83FEFF cmp si,-1
26818 00000E3E 75F4 jne short get_dpb_for_drive_1 ; loop until hit end of DPBs
26819
26820 got_dpb_for_drive:
26821 00000E40 C3 retn
26822
26823 ;=====
26824 ;** EndFile - Build DOS structures
26825 ;
26826 ; This procedure is called after the config.sys has been processed and
26827 ; installable device drivers have been loaded (but before "install="
26828 ; programs are loaded) to create the dos structures such as SFTS, buffers,
26829 ; FCBS, CDSS, etc. It also loads the sysinit_base module in low memory
26830 ; to allow for the safe EXECing of "install=" programs. All memory
26831 ; above these structures is deallocated back to DOS.
26832 ;
26833 ; ENTRY ?? BUGBUG
26834 ; EXIT ?? BUGBUG
26835 ; USES ?? BUGBUG
26836
26837 ;=====
26838 ; allocate files
26839 ;-----
26840
26841 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
26842 ; (SYSINIT:0CCDh)
26843
26844 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
26845 ; (SYSINIT:0E00h)
26846
26847 ; 09/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
26848 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:0F14h)
26849
26850 ; ((MSDOS 6.22 IO.SYS - SYSINIT:0E00h))
26851
26852 endfile:
26853 ; we are now setting up final cdss,buffers,files,fcss strings etc. we no
26854 ; longer need the space taken by the temp stuff below confbot,so set allocim
26855 ; to confbot.
26856
26857 ; if this procedure has been called to take care of install= command,
26858 ; then we have to save es,si registers.
26859
26860 ; 11/12/2022
26861 ; ds = cs
26862
26863 ; 23/10/2022
26864 ; 31/03/2019
26865 00000E41 1E push ds
26866
26867 ; mov ax,Bios_Data ; 0070h
26868 ; mov ax,KERNEL_SEGMENT ; 0070h
26869 ; 21/10/2022
26870 00000E42 B87000 mov ax,DOSBIODATASEG ; 0070h
26871 00000E45 8ED8 mov ds,ax
26872
26873 ; cmp word [052Fh],0
26874 00000E47 833E[A004]00 cmp word [multrk_flag],multrk_off1 ;=0,multrack= command entered?
26875 00000E4C 7505 jne short multrk_flag_done
26876 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
26877 ; or word [multrk_flag],multrk_on ; 80h ; default will be on.
26878 ; 12/12/2022
26879 or byte [multrk_flag],multrk_on ; 80h
26880 multrk_flag_done:
26881 ; 23/10/2022
26882 ; 31/03/2019
26883 00000E53 1F pop ds
26884
26885 ; 11/12/2022
26886 ; ds = cs
26887 ; mov ax,[top_of_cdss] ; mov ax,[CONFBOT]
26888 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
26889

```

```

26890 ; (SYSINIT:0E14h)
26891 00000E54 A1[A302] mov ax,[CONFBOT]
26892 00000E57 A3[A502] mov [ALLOCLIM],ax
26893 ; 23/10/2022
26894 ;mov ax,[cs:top_of_cdss]
26895 ;mov [cs:ALLOCLIM],ax
26896
26897 ; 11/12/2022
26898 ; ds = cs
26899 ;push cs
26900 ;pop ds
26901
26902 ;mov ax,[CONFBOT]
26903 ;mov [ALLOCLIM],ax
26904
26905 ; 09/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
26906 ;;;
26907 ;mov ax,[cs:ALLOCLIM]
26908 ;mov ax,[ALLOCLIM]
26909 ;mov [cs:prev_alloclim],ax
26910 00000E5A A3[6C03] mov [prev_alloclim],ax
26911 ;mov ax,[cs:memhi]
26912 00000E5D A1[6403] mov ax,[memhi]
26913 ;mov [cs:prev_memhi],ax
26914 00000E60 A3[6A03] mov [prev_memhi],ax
26915 dosfts:
26916 ;;;
26917
26918 00000E63 E88A39 call round
26919
26920 ; 11/12/2022
26921 ; ds = cs
26922 00000E66 A0[9F02] mov al,[FILES]
26923 ; 23/10/2022
26924 ;mov al,[cs:FILES]
26925 00000E69 2C05 sub al,5
26926 00000E6B 764B jbe short dofcbcs
26927
26928 00000E6D 50 push ax
26929 ;mov al,devmark_files ; 'F'
26930 00000E6E B046 mov al,'F'
26931 00000E70 E81808 call setdevmark ; set devmark for sfts (files)
26932 00000E73 58 pop ax
26933 00000E74 30E4 xor ah,ah ; do not use cbw instruction!!!!
26934 ; it does sign extend.
26935
26936 ; 11/12/2022
26937 ; ds = cs
26938 00000E76 8B1E[6203] mov bx,[memlo]
26939 00000E7A 8B16[6403] mov dx,[memhi]
26940 00000E7E C53E[6D02] lds di,[DOSINFO] ;get pointer to dos data
26941 ; 23/10/2022
26942 ;mov bx,[cs:memlo]
26943 ;mov dx,[cs:memhi]
26944 ;lds di,[cs:DOSINFO]
26945
26946 00000E82 C57D04 ;lds di,[di+SYSI_SFT] ;ds:bp points to sft
26947 ;lds di,[di+4]
26948
26949 ;mov [di+SF.SFLink],bx
26950 00000E85 891D mov [di],bx
26951 00000E87 895502 mov [di+SF.SFLink+2],dx ;set pointer to new sft
26952
26953 00000E8A 0E push cs
26954 00000E8B 1F pop ds
26955
26956 ; 11/12/2022
26957 ; ds = cs
26958 00000E8C C43E[6203] les di,[memlo] ;point to new sft
26959 ; 23/10/2022
26960 ;les di,[cs:memlo]
26961
26962 00000E90 26C705FFFF ;mov word [es:di+SF.SFLink],-1
26963 ;mov word [es:di],-1 ; 0FFFFh
26964 00000E95 26894504 ;mov [es:di+SF.SFCount],ax
26965 ;mov [es:di+4],ax
26966 ; 09/04/2024
26967 00000E99 B33B mov bl,SF_ENTRY.size ; 59
26968 00000E9B F6E3 mov bl,59
26969 00000E9D 89C1 mul bl ;ax = number of bytes to clear
26970
26971 ; 11/12/2022
26972 ; ds = cs
26973 00000E9F 0106[6203] add [memlo],ax ;allocate memory
26974 ; 23/10/2022
26975 00000EA3 B80600 ;add [cs:memlo],ax
26976 ;mov ax,6
26977 ; 11/12/2022
26978 00000EA6 0106[6203] add [memlo],ax ;remember the header too
26979 ;add [cs:memlo],ax
26980 00000EAA 800E[6919]02 ; 11/12/2022
26981 ;or byte [setdevmarkflag],for_devmark ; 2
26982 ; 23/10/2022
26983 00000EAF E83E39 ;or byte [cs:setdevmarkflag],2
26984 00000EB2 01C7 call round ; check for mem error before the stosb
26985 00000EB4 31C0 add di,ax
26986 00000EB6 F3AA xor ax,ax
26987 rep stosb ;clean out the stuff
26988
26989 ; allocate fcbs
26990 ; -----
26991 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
26992 ; (SYSINIT:0D48h)
26993 dofcbcs:
26994 ; 11/12/2022
26995 ; ds = cs
26996 ;push cs
26997 ;pop ds
26998 00000EB8 E83539 call round
26999 ;mov al,devmark_fcbs ; 'X' ;='x'
27000 00000EBB B058 mov al,'X'
27001 00000EBD E8CB07 call setdevmark
27002 ; 11/12/2022
27003 ; ds = cs
27004 00000EC0 A0[A002] mov al,[FCBS]
27005 ;mov al,[cs:FCBS]
27006 00000EC3 30E4 xor ah,ah ; do not use cbw instruction!!!!
27007 ; it does sign extend.
27008
27009 ; 11/12/2022
27010 00000EC5 8B1E[6203] mov bx,[memlo]
27011 00000EC9 8B16[6403] mov dx,[memhi]
27012 00000ECD C53E[6D02] lds di,[DOSINFO] ;get pointer to dos data
27013 ; 23/10/2022
27014 ;mov bx,[cs:memlo]

```

```

27014      ;mov     dx,[cs:memhi]
27015      ;lds     di,[cs:DOSINFO]
27016
27017      ;mov     [di+SYSI_FCB],bx
27018      ;mov     [di+SYSI_FCB+2],dx ;set pointer to new table
27019      ; 23/10/2022
27020 00000ED1 895D1A      mov     [di+1Ah],bx      ; [di+SYSI_FCB]
27021 00000ED4 89551C      mov     [di+1Ch],dx      ; [di+SYSI_FCB+2]
27022
27023 00000ED7 2E8A1E[A102]      mov     bl,[cs:KEEP]
27024 00000EDC 30FF          xor     bh,bh
27025      ;mov     [di+SYSI_KEE],bx
27026 00000EDE 895D1E      mov     [di+1Eh],bx      ; [di+SYSI_KEE]
27027
27028 00000EE1 0E          push    cs
27029 00000EE2 1F          pop     ds
27030
27031 00000EE3 C43E[6203]      les     di,[memlo]      ;point to new table
27032      ;mov     word [es:di+SF.SFLink],-1
27033 00000EE7 26C705FFFF      mov     word [es:di],-1
27034      ;mov     [es:di+SF.SFCount],ax
27035      ; 02/11/2022
27036 00000EEC 26894504      mov     [es:di+4],ax
27037 00000EF0 B33B          mov     bl,SF_ENTRY.size ; 59
27038 00000EF2 89C1          mov     cx,ax
27039 00000EF4 F6E3          mul     bl      ;ax = number of bytes to clear
27040 00000EF6 0106[6203]      add     [memlo],ax      ;allocate memory
27041      ;mov     ax,6
27042 00000EFA B80600      mov     ax,SF.size-2 ; 6
27043 00000EFD 0106[6203]      add     [memlo],ax      ;remember the header too
27044      ;or     byte [setdevmarkflag],for_devmark ; 2
27045 00000F01 800E[6919]02      or     byte [setdevmarkflag],2
27046 00000F06 E8E738      call    round      ; check for mem error before the stosb
27047 00000F09 01C7          add     di,ax      ;skip over header
27048 00000F0B B041          mov     al,'A'
27049      fillloop:
27050 00000F0D 51          push    cx      ; save count
27051 00000F0E B93B00      mov     cx,SF_ENTRY.size ; 59 ; number of bytes to fill
27052 00000F11 FC          cld
27053 00000F12 F3AA          rep     stosb      ; filled
27054
27055      ;mov     word [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_ref_count],0 ; [es:di-59]
27056      ;mov     word [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_position],0 ; [es:di-38]
27057      ;mov     word [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_position+2],0 ; [es:di-36]
27058
27059      ; 18/12/2022
27060      ;cx = 0
27061 00000F14 26894DC5      mov     [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_ref_count],cx ;0 ; [es:di-59]
27062 00000F18 26894DDA      mov     [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_position],cx ;0 ; [es:di-38]
27063 00000F1C 26894DDC      mov     [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_position+2],cx ;0 ; [es:di-36]
27064
27065      ; 23/10/2022
27066      ;mov     word [es:di-38h],0
27067      ;mov     word [es:di-26h],0
27068      ;mov     word [es:di-24h],0
27069
27070 00000F20 59          pop     cx
27071 00000F21 E2EA          loop    fillloop
27072
27073      ; allocate buffers
27074      ; -----
27075
27076      ; search through the list of media supported and allocate 3 buffers if the
27077      ; capacity of the drive is > 360kb
27078
27079      ; 18/12/2022
27080      ; cx = 0
27081 00000F23 833E[9902]FF      cmp     word [buffers],-1      ; has buffers been already set?
27082 00000F28 7403          je      short dodefaultbuff
27083 00000F2A E98000      jmp     dobuff      ; the user entered the buffers=.
27084
27085      dodefaultbuff:
27086      ; 18/12/2022
27087 00000F2D 890E[9B02]      mov     [h_buffers],cx ; 0
27088      ;inc     cx
27089      ;inc     cx
27090      ;mov     [buffers],cx ; 2
27091      ; 10/04/2024
27092 00000F31 C706[9902]0200      mov     word [buffers],2
27093
27094      ;mov     word [h_buffers],0      ; default is no heuristic buffers.
27095      ;mov     word [buffers],2      ; default to 2 buffers
27096
27097      ; 23/10/2022
27098      ; 04/09/2023
27099      ;push    ax
27100      ;push    ds ; 26/03/2019
27101
27102      ; 04/09/2023
27103      ; ds = cs
27104 00000F37 C42E[6D02]      les     bp,[DOSINFO]      ; search through the dpb's
27105      ;les     bp,[cs:DOSINFO]
27106      ;les     bp,[es:bp+SYSI_DPB]      ; get first dpb
27107      ; 11/12/2022
27108 00000F3B 26C46E00      les     bp,[es:bp]
27109      ; 23/10/2022
27110      ;les     bp,[es:bp+0]      ; ! (MSDOS 5.0 IO.SYS address compability) !
27111
27112      ; 04/09/2023
27113      ; ds = cs
27114      ;push    cs
27115      ;pop     ds
27116      ;SYSINIT:0DE2h:
27117      nextdpb:      ; test if the drive supports removeable media
27118      ;mov     bl,[es:bp+DPB.drive]
27119      ; 11/12/2022
27120 00000F3F 268A5E00      mov     bl,[es:bp]
27121      ; 23/10/2022
27122      ;mov     bl,[es:bp+0]      ; ! (MSDOS 5.0 IO.SYS address compability) !
27123
27124      ;inc     bl
27125      ; 18/12/2022
27126 00000F43 43          inc     bx
27127
27128      ;mov     ax,(IOCTL<<8)|8
27129 00000F44 B80844      mov     ax,4408h
27130 00000F47 CD21          int     21h      ; DOS - 2+ - IOCTL -
27131
27132      ; ignore fixed disks
27133
27134 00000F49 09C0      or     ax,ax      ; ax is nonzero if disk is nonremoveable
27135 00000F4B 7534      jnz     short nosetbuf
27136
27137      ; get parameters of drive

```

```

27138
27139 0000F4D 31DB          xor     bx,bx
27140                      ;;mov  b1,[es:bp+DPB.drive]
27141                      ; 11/12/2022
27142 0000F4F 268A5E00      mov     b1,[es:bp]
27143                      ; 23/10/2022
27144                      ;mov   b1,[es:bp+0] ; ! (MSDOS 5.0 IO.SYS address compability) !
27145
27146                      ;inc   b1
27147                      ; 18/12/2022
27148 0000F53 43           inc     bx
27149
27150 0000F54 BA[BC4D]       mov     dx,deviceparameters
27151                      ;mov   ax,(IOCTL<<8)|GENERIC_IOCTL
27152 0000F57 B80D44        mov     ax,440Dh
27153                      ;mov   cx,(RAWIO<<8)|GET_DEVICE_PARAMETERS
27154 0000F5A B96008        mov     cx,860h
27155 0000F5D CD21          int     21h ; DOS - 2+ - IOCTL -
27156 0000F5F 7220          jc      short nosetbuf ; get next dpb if driver doesn't support
27157                      ; generic ioctl
27158                      ; determine capacity of drive
27159                      ; media capacity = #sectors * bytes/sector
27160
27161                      ;mov   bx,[deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.TOTALSECTORS]
27162                      ; 23/10/2022
27163 0000F61 8B1E[CB4D]     mov     bx,[deviceparameters+15] ; total sectors (16 bit)
27164
27165                      ; to keep the magnitude of the media capacity within a word,
27166                      ; scale the sector size
27167                      ; (ie. 1 -> 512 bytes, 2 -> 1024 bytes,...)
27168
27169                      ;mov   ax,[deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.BYTESPERSECTOR]
27170                      ; 23/10/2022
27171 0000F65 A1[C34D]       mov     ax,[deviceparameters+7] ; bytes per sector
27172 0000F68 31D2          xor     dx,dx
27173 0000F6A B90002        mov     cx,512
27174 0000F6D F7F1          div     cx ; scale sector size in factor of
27175                      ; 512 bytes
27176 0000F6F F7E3          mul     bx ; ax = #sectors * size factor
27177 0000F71 09D2          or      dx,dx ; just in case of large floppies
27178 0000F73 7505          jnz     short setbuf
27179 0000F75 3D0002        cmp     ax,720 ; 720 sectors * size factor of 1
27180 0000F78 7607          jbe     short nosetbuf
27181 setbuf:
27182                      ; 18/12/2022
27183                      ; word [buffers] = 2
27184 0000F7A C606[9902]03   mov     byte [buffers],3
27185                      ;mov   word [buffers],3
27186 0000F7F EB0D          jmp     short chk_memsize_for_buffers ; now check the memory size
27187                      ; for default buffer count
27188 nosetbuf:
27189                      ; 23/10/2022
27190                      ;cmp   word [es:bp+DPB.NEXT_DPB],-1
27191 0000F81 26837E19FF     cmp     word [es:bp+19h], -1 ; 0FFFFh
27192 0000F86 7406          je      short chk_memsize_for_buffers
27193                      ;les   bp,[es:bp+DPB.NEXT_DPB] ; [es:bp+19h]
27194 0000F88 26C46E19     les     bp,[es:bp+19h]
27195 0000F8C EBB1          jmp     short nextdpb
27196
27197                      ;from dos 3.3,the default number of buffers will be changed according to the
27198                      ;memory size too.
27199                      ; default buffers = 2
27200                      ; if diskette media > 360 kb,then default buffers = 3
27201                      ; if memory size > 128 kb (2000h para),then default buffers = 5
27202                      ; if memory size > 256 kb (4000h para),then default buffers = 10
27203                      ; if memory size > 512 kb (8000h para),then default buffers = 15.
27204
27205 chk_memsize_for_buffers:
27206                      ; 18/12/2022
27207                      ;cmp   word [MEMORY_SIZE],2000h
27208                      ;jbe   short bufset
27209                      ;mov   word [buffers],5
27210                      ;cmp   word [MEMORY_SIZE],4000h
27211                      ;jbe   short bufset
27212                      ;mov   word [buffers],10
27213                      ;cmp   word [MEMORY_SIZE],8000h
27214                      ;jbe   short bufset
27215                      ;mov   word [buffers],15
27216
27217                      ; 18/12/2022
27218                      ; word [buffers] = 3 or 2
27219 0000F8E BB[9902]       mov     bx,buffers
27220 0000F91 A1[9402]       mov     ax,[MEMORY_SIZE]
27221 0000F94 48           dec     ax ; [MEMORY_SIZE] - 1
27222
27223                      cmp     ah,20h ; ax >= 2000h ([MEMORY_SIZE] > 2000h) ; *
27224 0000F98 7213          jb      short bufset
27225 0000F9A C6070F       mov     byte [bx],15 ; [buffers] = 15 ; ***
27226 0000F9D 80FC80       cmp     ah,80h ; ax >= 8000h ([MEMORY_SIZE] > 8000h) ; ***
27227 0000FA0 730B          jnb     short bufset
27228 0000FA2 C6070A       mov     byte [bx],10 ; [buffers] = 10 ; **
27229 0000FA5 80FC40       cmp     ah,40h ; ax >= 4000h ([MEMORY_SIZE] > 4000h) ; **
27230 0000FA8 7303          jnb     short bufset
27231 0000FAA C60705       mov     byte [bx],5 ; [buffers] = 5 ; *
27232 bufset:
27233                      ; 23/10/2022
27234                      ; 26/03/2019
27235                      ; 04/09/2023
27236                      ;pop   ds
27237                      ;pop   ax
27238
27239                      ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
27240                      ;j.k. here we should put extended stuff and new allocation scheme!!!
27241                      ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
27242
27243                      ; 26/03/2019
27244
27245                      ;*****
27246                      ;
27247                      ; function: actually allocate buffers in the memory and initialize it. *
27248                      ; input: *
27249                      ; memhi:memlo - start of the next available memory *
27250                      ; buffers = number of buffers *
27251                      ; h_buffers = number of secondary buffers *
27252                      ;
27253                      ; output: *
27254                      ; buffinfo.cache_count - # of caches to be installed. *
27255                      ; buffinfo.set. *
27256                      ; bufferqueue.set. *
27257                      ;
27258                      ; subroutines to be called: *
27259                      ;
27260                      ;*****
27261

```



```

27262             ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
27263             ; (SYSINIT:0E60h)
27264 dobuff:
27265             ; ds = cs ; 31/03/2019
27266             ; 23/10/2022
27267             ; lds bx,[cs:DOSINFO] ; ds:bx -> sysinitvar
27268             ; 04/09/2023
27269 00000FAD A1[9902] mov ax,[buffers] ; 31/03/2019
27270 00000FB0 8B0E[9B02] mov cx,[h_buffers] ; *
27271 00000FB4 C51E[6D02] lds bx,[DOSINFO]
27272             ;mov ax,[cs:buffers] ; set sysi_buffers
27273             ;mov [bx+SYSI_BUFFERS],ax ; [bx+3Fh]
27274 00000FB8 89473F mov [bx+3Fh],ax
27275             ; 04/09/2023
27276             ;mov ax,[cs:h_buffers]
27277             ;mov [bx+SYSI_BUFFERS+2],ax ; [bx+41h]
27278             ;mov [bx+41h],ax
27279             ; 04/09/2023
27280 00000FBB 894F41 mov [bx+41h],cx ; *
27281 00000FBE C55F12 lds bx,[bx+12h]
27282             ;lds bx,[bx+SYSI_BUF] ; now,ds:bx -> buffinfo
27283 00000FC1 E82C38 call round ; get [memhi]:[memlo]
27284             ;mov al,devmark_buf ; ='B'
27285 00000FC4 B042 mov al,'B'
27286 00000FC6 E8C206 call setdevmark
27287
27288 ;allocate buffers
27289
27290 00000FC9 1E push ds ; save buffer info. ptr.
27291 00000FCA 53 push bx
27292
27293 00000FCB E8D403 call set_buffer
27294
27295 00000FCE 5B pop bx
27296 00000FCF 1F pop ds
27297
27298 ;now set the secondary buffer if specified.
27299
27300 00000FD0 2E833E[9B02]00 cmp word [cs:h_buffers],0
27301 00000FD6 742D je short xif16
27302 00000FD8 E81538 call round
27303             ; 23/10/2022
27304 00000FDB 2E8B0E[6203] mov cx,[cs:memlo]
27305             ;mov [bx+BUFFINF.Cache_ptr],cx ; [bx+6]
27306 00000FE0 894F06 mov [bx+6],cx
27307 00000FE3 2E8B0E[6403] mov cx,[cs:memhi]
27308             ;mov [bx+BUFFINF.Cache_ptr+2],cx ; [bx+8]
27309 00000FE8 894F08 mov [bx+8],cx
27310 00000FEB 2E8B0E[9B02] mov cx,[cs:h_buffers]
27311             ;mov [bx+BUFFINF.Cache_count],cx ; [bx+10]
27312 00000FF0 894F0A mov [bx+10],cx
27313 00000FF3 B80002 mov ax,512 ; 512 byte
27314 00000FF6 F7E1 mul cx
27315 00000FF8 2EA3[6203] mov [cs:memlo],ax
27316             ;or byte [cs:setdevmarkflag],for_devmark ; 2
27317 00000FFC 2E800E[6919]02 or byte [cs:setdevmarkflag],2
27318 00001002 E8EB37 call round
27319 xif16:
27320
27321 ; -----
27322 ; allocate cdss
27323 ; -----
27324
27325 buf1:
27326 00001005 E8E837 call round
27327
27328 00001008 50 push ax
27329             ; 23/10/2022
27330             ;mov ax,devmark_cds ;='L'
27331 00001009 B84C00 mov ax,'L'
27332 0000100C E87C06 call setdevmark
27333 0000100F 58 pop ax
27334
27335 00001010 2EC43E[6D02] les di,[cs:DOSINFO]
27336             ;mov c1,[es:di+SYSI_NUMIO]
27337 00001015 268A4D20 mov c1,[es:di+20h]
27338 00001019 2E3A0E[A202] cmp c1,[cs:NUM_CDS]
27339 0000101E 7305 jae short gotncds ; user setting must be at least numio
27340 00001020 2E8A0E[A202] mov c1,[cs:NUM_CDS]
27341 gotncds:
27342 00001025 30ED xor ch,ch
27343             ;mov [es:di+SYSI_NCDS],c1 ; [es:di+33]
27344 00001027 26884D21 mov [es:di+21h],c1
27345 0000102B 2EA1[6403] mov ax,[cs:memhi]
27346             ;mov [es:di+SYSI_CDS+2],ax
27347 0000102F 26894518 mov [es:di+18h],ax
27348 00001033 2EA1[6203] mov ax,[cs:memlo]
27349             ;mov [es:di+SYSI_CDS],ax
27350 00001037 26894516 mov [es:di+16h],ax
27351 0000103B 88C8 mov al,c1
27352             ;mov ah,curdirlen ; curdir_list.size
27353 0000103D B458 mov ah,88
27354 0000103F F6E4 mul ah
27355 00001041 E8D102 call ParaRound
27356 00001044 2E0106[6403] add [cs:memhi],ax
27357
27358             ;or byte [cs:setdevmarkflag],for_devmark ; 2
27359 00001049 2E800E[6919]02 or byte [cs:setdevmarkflag],2
27360 0000104F E89E37 call round ; check for mem error before initializing
27361             ;lds si,[es:di+SYSI_DPB] ; [es:di+0]
27362 00001052 26C535 lds si,[es:di]
27363             ;les di,[es:di+SYSI_CDS] ; [es:di+22]
27364 00001055 26C47D16 les di,[es:di+16h]
27365 00001059 E875FD call fooset
27366
27367 ; -----
27368 ; allocate space for internal stack
27369 ; -----
27370
27371 0000105C 0E push cs
27372 0000105D 1F pop ds
27373
27374 ; if the user did not entered stacks= command, as a default, do not install
27375 ; sytem stacks for pc1,pc xt,pc portable cases.
27376 ; otherwise,install it to the user specified value or to the default
27377 ; value of 9,128 for other systems.
27378
27379 0000105E 833E[9002]FF cmp word [stack_addr],-1 ; has the user entered "stacks=" command?
27380 00001063 740E je short doinstallstack ; then install as specified by the user
27381 00001065 803E[BC02]00 cmp byte [sys_scnd_model_byte],0 ; pc1,xt has the secondary model byte = 0
27382 0000106A 7507 jne short doinstallstack ; other model should have default stack of 9,128
27383 0000106C 803E[BB02]FE cmp byte [sys_model_byte],0FEh ; pc1, pc/xt or pc portable ?
27384 00001071 736D jae short skipstack
27385 doinstallstack:

```

```

27386 00001073 A1[8C02]      mov     ax,[stack_count]      ; stack_count = 0?
27387 00001076 09C0        or      ax,ax                ; then, stack size must be 0 too.
27388 00001078 7466        jz      short skipstack        ; don't install stack.
27389
27390
27391
27392 0000107A E87337      call    round                ;[memhi] = seg. for stack code
27393                                ;[memlo] = 0
27394
27395
27396
27397
27398
27399
27400 0000107D B053        mov     al,devmark_stk ;='s'
27401 0000107F E80906      ; 23/10/2022
27402                                mov     al,'s'
27403 00001082 A1[6403]      mov     ax,[memhi]
27404 00001085 8EC0        mov     es,ax                ;es -> seg. the stack code is going to move.
27405                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
27406                                ; 11/12/2022
27407                                ; ds = cs
27408                                ;push    cs
27409                                ;pop     ds
27410 00001087 31F6        xor     si,si                ;!!we know that stack code is at the beginning of sysinit.
27411 00001089 31FF        xor     di,di
27412 0000108B B9[6902]      mov     cx,endstackcode
27413 0000108E 890E[6203]     mov     [memlo],cx
27414 00001092 E85B37      call    round                ;have enough space for relocation?
27415 00001095 F3A4        rep     movsb
27416
27417 00001097 1E          push    ds                ; stick the location of the NextStack entry
27418                                ;;mov    ax,Bios_Data    ; into the win386 Instance Data tables
27419                                ;mov    ax,KERNEL_SEGMENT ; 70h
27420                                ; 21/10/2022
27421 00001098 B87000      mov     ax,DOSBIODATASEG ; 0070h
27422 0000109B 8ED8        mov     ds,ax
27423 0000109D C706[0208][1000] mov     word [NextStack],nextentry ; (8C0h for MSDOS 6.21 IO.SYS)
27424 000010A3 8C06[0408]     mov     [NextStack+2],es      ; (8C2h for MSDOS 6.21 IO.SYS)
27425
27426 000010A7 2EA1[6203]     mov     ax,[cs:memlo]
27427 000010AB 2EA3[9002]     mov     [cs:stack_addr],ax ;set for stack area initialization
27428 000010AF A3[0808]        mov     [IT_StackLoc],ax ; pass it as Instance Data, too
27429 000010B2 2EA1[6403]     mov     ax,[cs:memhi] ;this will be used by stack_init routine.
27430 000010B6 2EA3[9202]     mov     [cs:stack_addr+2],ax
27431 000010BA A3[0A08]        mov     [IT_StackLoc+2],ax
27432
27433
27434
27435
27436
27437 000010BD B80800      ;mov    ax,entrysize ; mov ax,8
27438 000010C0 2E0306[8E02]     ; 23/10/2022
27439 000010C5 2EF726[8C02]     mov     ax,8
27440                                add     ax,[cs:stack_size]
27441 000010CA A3[0C08]        mul     word [cs:stack_count]
27442
27443 000010CD 1F          mov     [IT_StackSize],ax ; pass through to Instance Tables
27444
27445 000010CE E84402      pop     ds                ; no more need to access Instance Table
27446
27447
27448
27449
27450 000010D1 0106[6403]     call    ParaRound          ; convert size to paragraphs
27451                                ; 11/12/2022
27452                                ; ds = cs
27453                                ;add     [cs:memhi],ax
27454                                ;add     [memhi],ax
27455                                ;or      byte [cs:setdevmarkflag],for_devmark ; 2
27456 000010D5 800E[6919]02     ;or      byte [cs:setdevmarkflag],2
27457                                ;or      byte [setdevmarkflag],2
27458                                ;or      byte [setdevmarkflag],for_devmark ; 2
27459                                ;to set the devmark_size for stack by round routine.
27460 000010DA E81337      call    round                ; check for memory error before
27461                                ; continuing
27462 000010DD E87D03      call    stackinit           ; initialize hardware stack.
27463                                ; cs=ds=sysinitseg,es=relocated stack code & data
27464
27465
27466
27467
27468 000010E0 803E[6E03]01     skipstack:
27469                                ; 10/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
27470                                ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:11F0h)
27471                                ;;;
27472                                ;push    cs
27473                                ;pop     ds
27474                                ; ds = cs
27475 000010E9 803E[8B16]EA     cmp     byte [dosdata_umb],1 ; PCDOS 7 feature - DOSDATA=UMB/NOUMB configuration
27476 000010EE 7476        ja      short dosdata_umb_done ; 1 = DOSDATA=UMB, 2 = (UMB) done, 0 = NOUMB
27477                                ; 2 - done
27478                                ;jb      short dosdata_noumb ; 0 - DOSDATA=NOUMB
27479
27480 000010F0 B80258      mov     ax,5802h
27481 000010F3 CD21        int     21h                ; DOS - 3+ - GET/SET MEMORY ALLOCATION STRATEGY
27482                                ; AL = function code: (DOS 5beta) get UMB link state
27483
27484 000010F5 98          cbw
27485 000010F6 89C7        mov     di,ax                ; al = 01h -> UMBs in DOS memory chain
27486 000010F8 BB0100      mov     bx,1                ; save current (previous) UMB link state
27487                                ; bx = 01h -> add UMBs to DOS memory chain
27488
27489 000010FB B80358      mov     ax,5803h
27490 000010FE CD21        int     21h
27491 00001100 7264        jc      short dosdata_noumb
27492
27493 00001102 B80058      mov     ax,5800h
27494 00001105 CD21        int     21h                ; DOS - 3+ - GET/SET MEMORY ALLOCATION STRATEGY
27495                                ; AL = function code: get allocation strategy
27496
27497 00001107 89C6        mov     si,ax                ; ax = current strategy
27498                                ; save current (previous) allocation strategy
27499 00001109 BB4000      mov     bx,40h              ; bl = new strategy = 40h - high memory first fit
27500
27501 0000110C B80158      mov     ax,5801h
27502 0000110F CD21        int     21h
27503
27504 00001111 8B1E[6403]     mov     bx,[memhi]
27505 00001115 2B1E[6A03]     sub     bx,[prev_memhi]
27506
27507 00001119 B448        mov     ah,48h
27508 0000111B CD21        int     21h                ; DOS - 2+ - ALLOCATE MEMORY
27509                                ; BX = number of 16-byte paragraphs desired
27510 0000111D 89C1        mov     cx,ax                ; ax = segment of allocated block
27511 0000111F 89FB        mov     bx,di                ; restore previous UMB link state
27512
27513 00001121 B80358      mov     ax,5803h
27514 00001124 CD21        int     21h                ; DOS - 3+ - GET/SET MEMORY ALLOCATION STRATEGY

```



```

27510                                     ; AL = function code: (DOS 5beta) set UMB link state
27511 00001126 89F3      mov     bx,si      ; restore previous allocation strategy
27512
27513 00001128 B80158    mov     ax,5801h
27514 0000112B CD21     int     21h      ; DOS - 3+ - GET/SET MEMORY ALLOCATION STRATEGY
27515                                     ; AL = function code: set allocation strategy
27516 0000112D 81F900A0  cmp     cx,0A000h      ; Is the allocated memory block (segment) a UMB?
27517 00001131 7233      jnb     short dosdata_numb ; no
27518
27519                                     ;mov     word [ALLOCLIM],0FFFFh
27520                                     ;mov     word [memlo],0
27521 00001133 890E[6403] mov     [memhi],cx
27522 00001137 49        dec     cx
27523 00001138 8EC1      mov     es,cx      ; point to arena/mcb
27524                                     ; 10/04/2024
27525 0000113A 31C9      xor     cx,cx ; 0
27526 0000113C 890E[6203] mov     [memlo],cx ; 0
27527 00001140 49        dec     cx
27528 00001141 890E[A502] mov     [ALLOCLIM],cx ; 0FFFFh
27529
27530 00001145 26C70601000800 mov     word [es:1],8      ; [es:arena_owner], 8 ; set impossible owner
27531 0000114C 26C70608005344 mov     word [es:8],4453h  ; [es:arena_name],'SD' ; System Data
27532 00001153 FE06[6E03] inc     byte [dosdata_umb] ; 1 -> 2 ; DOSDATA=UMB done.
27533 00001157 E909FD      jmp     dosfts
27534
27535 dosdata_umb_done:
27536 0000115A A1[6A03]    mov     ax,[prev_memhi]      ; (recent memory block/segment before UMBs)
27537 0000115D A3[6403]    mov     [memhi],ax
27538 00001160 A1[6C03]    mov     ax,[prev_alloclim]
27539 00001163 A3[A502]    mov     [ALLOCLIM],ax
27540
27541 dosdata_numb:
27542     ;;
27543
27544 ;skipstack:
27545 ; 24/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
27546 ; (SYSINIT:0F99h)
27547
27548 ; 11/12/2022
27549 ; ds = cs
27550 ;push    cs
27551 ;pop     ds
27552 00001166 A0[9F02]    mov     al,[FILES]
27553 00001169 30E4      xor     ah,ah      ; do not use cbw instruction!!!!
27554                                     ; it does sign extend.
27555 0000116B 89C1      mov     cx,ax
27556 0000116D 31DB      xor     bx,bx      ;close standard input
27557 0000116F B43E      mov     ah,3Eh ; CLOSE
27558 00001171 CD21     int     21h
27559 00001173 B80200    mov     bx,2
27560
27561 00001176 B43E      mov     ah,3Eh ; CLOSE ;close everybody but standard output
27562 00001178 CD21     int     21h      ; need output so we can print message
27563 0000117A 43        inc     bx      ; in case we can't get new one open.
27564 0000117B E2F9      loop    rcclloop
27565
27566 0000117D BA[C34A]    mov     dx,condev
27567 00001180 B002      mov     al,2
27568 00001182 B43D      mov     ah,3Dh ; OPEN ;open con for read/write
27569 00001184 F9        stc         ; set for possible int 24
27570 00001185 CD21     int     21h
27571 00001187 7305      jnc     short goaux
27572 00001189 E89A38    call    badfil
27573 0000118C EB13      jmp     short goaux2
27574
27575 0000118E 50        push    ax
27576 0000118F B80100    mov     bx,1      ;close standard output
27577 00001192 B43E      mov     ah,3Eh ; CLOSE
27578 00001194 CD21     int     21h
27579 00001196 58        pop     ax
27580
27581 00001197 89C3      mov     bx,ax      ;new device handle
27582 00001199 B445      mov     ah,45h ; XDUP
27583 0000119B CD21     int     21h      ;dup to 1,stdout
27584 0000119D B445      mov     ah,45h ; XDUP
27585 0000119F CD21     int     21h      ;dup to 2,stderr
27586
27587 000011A1 BA[C74A]    mov     dx,auxdev
27588 000011A4 B002      mov     al,2      ;read/write access
27589 000011A6 E8AE38    call    open_dev
27590
27591 000011A9 BA[CB4A]    mov     dx,prndev
27592 000011AC B001      mov     al,1      ;write only
27593 000011AE E8A638    call    open_dev
27594
27595 ;global rearm command for shared interrupt devices attached in the system;
27596 ;shared interrupt attachment has some problem when it issues interrupt
27597 ;during a warm reboot. once the interrupt is presented by the attachment,
27598 ;no further interrupts on that level will be presented until a global rearm
27599 ;is issued. by the request of the system architecture group, msbio will
27600 ;issue a global rearm after every device driver is loaded.
27601 ;to issue a global rearm: ;for pc1,xt,palace
27602 ;
27603 ; out 02f2h,xx ; interrupt level 2
27604 ; out 02f3h,xx ; interrupt level 3
27605 ; out 02f4h,xx ; interrupt level 4
27606 ; out 02f5h,xx ; interrupt level 5
27607 ; out 02f6h,xx ; interrupt level 6
27608 ; out 02f7h,xx ; interrupt level 7
27609 ;
27610 ; for pc at,in addition to the above commands,
27611 ; need to handle the secondary interrupt handler
27612 ;
27613 ; out 06f2h,xx ; interrupt level 10
27614 ; out 06f3h,xx ; interrupt level 11
27615 ; out 06f4h,xx ; interrupt level 12
27616 ; out 06f6h,xx ; interrupt level 14
27617 ; out 06f7h,xx ; interrupt level 15
27618 ;
27619 ; for round-up machine
27620 ;
27621 ; none.
27622 ;
27623 ; where xx stands for any value.
27624 ;
27625 ; for your information,after naples level machine,the system service bios
27626 ; call (int 15h),function ah=0c0h returns the system configuration parameters
27627
27628 ; 24/10/2022
27629
27630 000011B1 50        push    ax
27631 000011B2 53        push    bx
27632 000011B3 52        push    dx
27633 000011B4 06        push    es

```

```

27634
27635 000011B5 B0FF      mov     al,0FFh          ;reset h/w by writing to port
27636 000011B7 BAF202    mov     dx,2F2h        ;get starting address
27637 000011BA EE        out     dx,al          ; out 02f2h,0ffh
27638 000011BB 42        inc     dx
27639 000011BC EE        out     dx,al          ; out 02f3h,0ffh
27640 000011BD 42        inc     dx
27641 000011BE EE        out     dx,al          ; out 02f4h,0ffh
27642 000011BF 42        inc     dx
27643 000011C0 EE        out     dx,al          ; out 02f5h,0ffh
27644 000011C1 42        inc     dx
27645 000011C2 EE        out     dx,al          ; out 02f6h,0ffh
27646 000011C3 42        inc     dx
27647 000011C4 EE        out     dx,al          ; out 02f7h,0ffh
27648
27649
27650
27651 000011C5 B800F0      mov     ax,0F000h        ;get machine type
27652 000011C8 8EC0      mov     es,ax
27653 000011CA 26803EFEFFFC cmp     byte [es:0FFFEh],0FCh ;q:is it a at type machine
27654 000011D0 740D      je      short startrearm ; *if at no need to check
27655
27656 000011D2 B4C0      mov     ah,0C0h        ;get system configuration
27657 000011D4 CD15      int     15h           ; *
27658 000011D6 7216      jc      short finishrearm ; *jmp if old rom
27659
27660
27661
27662 000011D8 26F6470540    test    byte [es:bx+5],40h
27663
27664
27665 000011DD 740F      je      short finishrearm ;jmp if it is there
27666
27667
27668 000011DF B0FF      mov     al,0FFh        ;write any pattern to port
27669 000011E1 BAF206    mov     dx,6F2h        ;get starting address
27670 000011E4 EE        out     dx,al          ;out 06f2h,0ffh
27671 000011E5 42        inc     dx              ;bump address
27672 000011E6 EE        out     dx,al          ;out 06f3h,0ffh
27673 000011E7 42        inc     dx              ;bump address
27674 000011E8 EE        out     dx,al          ;out 06f4h,0ffh
27675 000011E9 42        inc     dx              ;bump address
27676 000011EA 42        inc     dx              ;bump address
27677 000011EB EE        out     dx,al          ;out 06f6h,0ffh
27678 000011EC 42        inc     dx              ;bump address
27679 000011ED EE        out     dx,al          ;out 06f7h,0ffh
27680
27681
27682 000011EE 07        pop     es
27683 000011EF 5A        pop     dx
27684 000011F0 5B        pop     bx
27685 000011F1 58        pop     ax
27686
27687
27688
27689
27690
27691
27692
27693
27694
27695
27696
27697
27698
27699
27700
27701
27702
27703
27704
27705
27706
27707
27708
27709
27710
27711 000011F2 50        ; ds = cs
27712 000011F3 A1[6403]    push    ax              ; set devmark for mem command
27713 000011F6 2B06[6803]    mov     ax,[memhi]
27714 000011FA A3[6003]    sub     ax,[area]
27715
27716 000011FD B054      mov     [impossible_owner_size],ax ;remember the size in case.
27717 000011FF E88904      ;mov     al,devmark_inst ; 'T'
27718 00001202 58        mov     al,'T'
27719
27720 00001203 8B3E[6403]    mov     di,[memhi]
27721 00001207 8EC7      mov     es,di
27722 00001209 893E[D402]    mov     [sysinit_base_ptr+2],di ; save this entry for the next use.
27723 0000120D 31FF      xor     di,di
27724 0000120F 893E[D202]    mov     [sysinit_base_ptr],di ; es:di -> destination.
27725 00001213 BE[2113]    mov     si,sysinit_base ;ds:si -> source code to be relocated.
27726 00001216 B98100      mov     cx,end_sysinit_base-sysinit_base ; 129
27727
27728
27729 00001219 010E[6203]    ; 24/10/2022
27730
27731
27732
27733
27734 0000121D 800E[6919]02    ;mov     cx,128 ; 11DCh-115Ch ; (MSDOS 5.0 IO.SYS, SYSINIT)
27735
27736 00001222 E8CB35      add     [memlo],cx
27737 00001225 F3A4      ;or     byte cs:[setdevmarkflag],for_devmark ; 2
27738
27739 00001227 C706[D602][0813]    ; 11/12/2022
27740 0000122D 8C0E[D802]    ; ds = cs
27741
27742
27743
27744 00001231 800E[CE02]02    ;or     byte [cs:setdevmarkflag],2
27745
27746
27747
27748
27749
27750
27751
27752
27753
27754 00001236 E8B735      ;or     byte [setdevmarkflag],2
27755 00001239 8B1E[6403]    ;or     byte [setdevmarkflag],for_devmark
27756 0000123D A1[6803]    call    round           ; check mem error. also,readjust memhi for the next use.
27757 00001240 A3[5E03]    rep     movsb          ; reallocate it.

```

```

27758 00001243 8EC0      mov     es,ax                ;calc what we needed
27759 00001245 29C3      sub      bx,ax
27760                ; 24/10/2022
27761 00001247 B44A      mov     ah,4Ah ; SETBLOCK
27762 00001249 CD21      int      21h                ;give the rest back
27763                ; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
27764                ; ES = segment address of block to change
27765                ; BX = new size in paragraphs
27766 0000124B 06        push     es
27767 0000124C 8CC0      mov     ax,es
27768 0000124E 48        dec      ax
27769 0000124F 8EC0      mov     es,ax                ;point to arena
27770                ;mov word [es:ARENA.OWNER],8 ;set impossible owner
27771                ;;mov word [es:ARENA.NAME],4453h ; System Data
27772                ;mov word [es:ARENA.NAME],'SD' ; System Data
27773                ; 24/10/2022
27774 00001251 26C70601000800 mov     word [es:1],8 ;set impossible owner
27775 00001258 26C70608005344 mov     word [es:8],'SD' ; System Data
27776 0000125F 07        pop      es
27777
27778 00001260 BBFFFF      mov     bx,0FFFFh
27779 00001263 B448      mov     ah,48h ; ALLOC
27780 00001265 CD21      int      21h
27781 00001267 B448      mov     ah,48h ; ALLOC
27782 00001269 CD21      int      21h                ; allocate the rest of the memory
27783                ; DOS - 2+ - ALLOCATE MEMORY
27784                ; BX = number of 16-byte paragraphs desired
27785 0000126B A3[6403]    mov     [memhi],ax        ; start of the allocated memory
27786 0000126E C706[6203]0000 mov     word [memlo],0    ; to be used next.
27787
27788                ;;;; at this moment,memory from [memhi]:0 to top-of-the memory is
27789                ;;;; allocated.
27790                ;;;; to protect sysinit,confbot module (from confbot (or =allocim at
27791                ;;;; this time) to the top-of-the memory),here we are going to
27792                ;;;; 1). "setblock" from memhi to confbot.
27793                ;;;; 2). "alloc" from confbot to the top of the memory.
27794                ;;;; 3). "free alloc memory" from memhi to confbot.
27795
27796                ;memory allocation for sysinit,confbot module.
27797
27798 00001274 8EC0      mov     es,ax
27799                ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
27800                ; (SYSINIT:11DFh)
27801 00001276 8B1E[A302]  mov     bx,[CONFBOT]
27802                ; 24/10/2022
27803                ;mov bx,[top_of_cdss] ; mov bx,[confbot]
27804 0000127A 29C3      sub      bx,ax                ; confbot - memhi
27805 0000127C 4B        dec      bx                ; make a room for the memory block id.
27806 0000127D 4B        dec      bx                ; make sure!!!.
27807 0000127E B44A      mov     ah,4Ah ; SETBLOCK
27808 00001280 CD21      int      21h                ; this will free (confbot to top of memory)
27809                ; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
27810                ; ES = segment address of block to change
27811                ; BX = new size in paragraphs
27812 00001282 BBFFFF      mov     bx,0FFFFh
27813 00001285 B448      mov     ah,48h ; ALLOC
27814 00001287 CD21      int      21h
27815 00001289 B448      mov     ah,48h ; ALLOC
27816 0000128B CD21      int      21h                ; allocate (confbot to top of memory)
27817                ; DOS - 2+ - ALLOCATE MEMORY
27818                ; BX = number of 16-byte paragraphs desired
27819 0000128D A3[6803]    mov     [area],ax        ; save allocated memory segment.
27820                ; need this to free this area for command.com.
27821 00001290 8E06[6403]  mov     es,[memhi]
27822 00001294 B449      mov     ah,49h                ; free allocated memory.
27823 00001296 CD21      int      21h                ; free (memhi to confbot(=area))
27824                ; DOS - 2+ - FREE MEMORY
27825                ; ES = segment address of area to be freed
27826
27827 00001298 C3        endfile_ret:
27828                retn
27829
27830                ; End of "EndFile" DOS structure configuration.
27831
27832                ; -----
27833                ; 26/03/2019 - Retro DOS v4.0
27834                ; 24/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
27835                ; -----
27836                ; Do_Install_Exec
27837                ;
27838                ; This procedure is used to EXEC a program being loaded via the
27839                ; "install=" mechanism in config.sys. It does this by setting up
27840                ; the parameters, and then jumping to sysinit_base, which has been
27841                ; setup in low memory. When complete, sysinit_base will jump back
27842                ; up to this procedure (if sysinit remains uncorrupted by the installed
27843                ; program).
27844
27845                ;SYSINIT:10CFh:
27846
27847 do_install_exec:                ; now,handles install= command.
27848                ;
27849                push     si                ; save si for config.sys again.
27850
27851                ; we are going to call load/exec function.
27852                ; set es:bx to the parameter block here;;;;;
27853                ; set ds:dx to the asciiz string. remember that we already has 0
27854                ; after the filename. so parameter starts after that. if next
27855                ; character is a line feed (i.e. 10),then assume that the 0
27856                ; we already encountered used to be a carriage return. in this
27857                ; case,let's set the length to 0 which will be followed by
27858                ; carriage return.
27859
27860                ; es:si -> command line in config.sys. points to the first non blank
27861                ; character after =.
27862                push     es
27863                push     ds
27864                pop      es
27865                pop      ds                ; es->sysinitseg,ds->confbot seg
27866                mov     dx,si            ; ds:dx->file name,0 in config.sys image.
27867
27868 000012A0 31C9      xor      cx,cx
27869 000012A2 FC        cld
27870 000012A3 2EC606[F102]20 mov     byte [cs:ldexec_start],' ' ; clear out the parm area
27871 000012A9 BF[F202]    mov     di,ldexec_parm
27872
27873 000012AC AC        installfilename:
27874                lodsb                ; skip the file name
27875                ; 05/09/2023
27876                or      al,al
27877                cmp     al,0
27878                jne     short got_installparm
27879                jmp     short installfilename
27880                ; 10/04/2024
27881                jnz     short installfilename
27882                got_installparm:                ; copy the parameters to ldexec_parm

```

```

27882 000012B1 AC          lodsb
27883 000012B2 268805      mov     [es:di],al
27884 000012B5 3C0A        cmp     al,1f ; cmp al,0Ah ; line feed?
27885 000012B7 7405        je      short done_installparm
27886 000012B9 FEC1        inc     cl ; # of char. in the parm.
27887 000012BB 47         inc     di
27888 000012BC EBF3        jmp     short got_installparm
27889
27890 000012BE 2E880E[F002] done_installparm:
27891                                mov     byte [cs:ldexec_line],cl ; length of the parm.
27892                                ; 05/09/2023
27893                                or      cl,cl
27894                                ;cmp    cl,0 ; if no parm,then
27895                                jne     short install_seg_set ; let the parm area
27896                                mov     byte [cs:ldexec_start],cr ; 0Dh
27897                                ; starts with cr.
27898
27899 000012CD 31DB          install_seg_set:
27900                                ; 05/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
27901                                xor     bx,bx
27902                                ;mov    word [cs:0],0 ; make a null environment segment
27903                                mov     [cs:bx],bx ; 05/09/2023
27904                                mov     ax,cs ; by overlap jmp instruction of sysinitseg.
27905
27906                                ;-----M067-----
27907                                ;
27908                                ; the environment pointer is made 0. so the current environment ptr.
27909                                ; will be the same as pdb_envron which after dosinit is 0.
27910                                ;
27911                                ; mov     cs:[instexe.exec0_envron],0 ; set the environment seg.
27912                                ;
27913                                ; instexe.exec0_envron need not be initialized to 0 above. It was
27914                                ; done as a fix for bug #529. The actual bug was in NLSFUNC and
27915                                ; was fixed.
27916                                ;-----
27917
27918                                ;;ifdef MULTI_CONFIG
27919                                ; If there's any environment data in "config_wrkseg", pass to app
27920
27921                                ; 30/12/2022 - Retro DOS v4.0 (Modified MSDOS 6.21 IO.SYS SYSINIT)
27922                                ; 24/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS SYSINIT)
27923                                ;%if 0
27924                                mov     cx,ax ; *
27925                                ; 05/09/2023
27926                                cmp     [cs:config_envlen],bx ; 0
27927                                ;cmp    word [cs:config_envlen],0
27928                                je      short no_envdata2
27929                                mov     cx,[cs:config_wrkseg] ; *
27930                                no_envdata2:
27931                                ;;endif ;MULTI_CONFIG
27932
27933                                ;%endif ; 24/10/2022
27934
27935                                ;mov     [cs:instexe.exec0_envron],cx ; set the environment seg.
27936                                ; 05/09/2023 (BugFix)
27937                                ; 24/10/2022
27938                                mov     [cs:iexec.envron],cx ; *
27939                                ; 02/11/2022
27940                                ;mov     [cs:iexec.envron],ax ; 05/09/2023
27941
27942                                ;mov     [cs:instexe.exec0_com_line+2],ax ; set the seg.
27943                                mov     [cs:iexec.ldexec_line+2],ax
27944                                ;mov     [cs:instexe.exec0_5c_fcb+2],ax
27945                                mov     [cs:iexec.ldexec_5c_fcb+2],ax
27946                                ;mov     [cs:instexe.exec0_6c_fcb+2],ax
27947                                mov     [cs:iexec.ldexec_6c_fcb+2],ax
27948                                call    sum_up
27949                                mov     [es:checksum],ax ; save the value of the sum
27950                                xor     ax,ax
27951                                mov     ah,4Bh ; EXEC ; load/exec
27952                                mov     bx,instexe ; es:bx -> parm block.
27953                                push    es ; save es,ds for load/exec
27954                                push    ds ; these registers will be restored in sysinit_base.
27955                                jmp     far [cs:sysinit_base_ptr] ; jmp to sysinit_base to execute
27956                                ; load/exec function and check sum.
27957
27958                                ;-----
27959
27960                                ;j.k. this is the returning address from sysinit_base.
27961
27962                                ; 24/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS SYSINIT)
27963
27964                                sysinitptr:
27965                                si ; returning far address from sysinit_base
27966                                push    es ; restore si for config.sys file.
27967                                push    ds
27968                                pop     es
27969                                pop     ds ; now ds - sysinitseg, es - confbot
27970                                jnc     short install_exit_ret
27971
27972                                push    si ; error in loading the file for install=.
27973                                call    badload ; es:si-> path,filename,0.
27974                                pop     si
27975
27976                                ; 24/10/2022
27977                                ;jmp     short sysinitptr_retn ; (MSDOS 5.0 IO.SYS, SYSINIT:1140h)
27978                                ; 11/12/2022
27979                                ; ds = cs
27980
27981                                ; 30/12/2022 - Retro DOS v4.2
27982                                ; (MSDOS 6.21 IO.SYS, SYSINIT:1283h)
27983
27984                                install_exit_ret:
27985                                retn
27986
27987                                ; 30/12/2022 - Retro DOS v4.2
27988                                %if 0
27989                                install_exit_ret:
27990                                ;retn ; retn (MSDOS 6.21 IO.SYS, SYSINIT:1283h) ; 18/12/2022
27991
27992                                ; 24/10/2022 (MSDOS 5.0 IO.SYS SYSINIT)
27993                                ;SYSINIT:1142h:
27994                                mov     ah,4Dh
27995                                int     21h ; DOS - 2+ - GET EXIT CODE OF SUBPROGRAM (WAIT)
27996                                cmp     ah,3
27997                                jz      short sysinitptr_retn
27998                                call    error_line
27999                                stc
28000                                sysinitptr_retn: ; (SYSINIT:114Fh)
28001                                retn
28002
28003                                %endif ; 24/10/2022
28004
28005                                ; -----

```

```

28006
28007
28008
28009
28010
28011
28012
28013
28014
28015
28016
28017 00001315 83C00F
28018 00001318 D1D8
28019 0000131A D1E8
28020 0000131C D1E8
28021 0000131E D1E8
28022 00001320 C3
28023
28024
28025
28026
28027
28028
28029
28030
28031
28032
28033
28034
28035
28036
28037
28038
28039
28040
28041
28042
28043
28044
28045
28046
28047
28048
28049
28050
28051
28052 00001321 2E8C166200
28053 00001326 2E89266400
28054 0000132B CD21
28055 0000132D 2E8E166200
28056 00001332 2E8B266400
28057 00001337 1F
28058 00001338 07
28059 00001339 7216
28060
28061
28062 0000133B E81800
28063 0000133E 263906[DA02]
28064 00001343 740C
28065
28066
28067
28068 00001345 B409
28069 00001347 0E
28070 00001348 1F
28071
28072
28073
28074 00001349 BA6600
28075
28076 0000134C CD21
28077
28078
28079
28080
28081
28082
28083
28084
28085 0000134E F4
28086
28087
28088 0000134F EBFD
28089
28090
28091 00001351 26FF2E[D602]
28092
28093
28094
28095
28096
28097
28098
28099
28100
28101
28102
28103
28104
28105
28106
28107 00001356 1E
28108
28109
28110 00001357 26A1[A302]
28111
28112
28113 0000135B 8ED8
28114 0000135D 31F6
28115 0000135F 31C0
28116 00001361 268B0E[D002]
28117
28118 00001366 D1E9
28119 00001368 7406
28120
28121 0000136A 0304
28122 0000136C 46
28123 0000136D 46
28124 0000136E E2FA
28125
28126
28127
28128 00001370 BE7013
28129

; ** ParaRound - Round Up length to paragraph multiple
;
; ParaRound rounds a byte count up to a multiple of 16, then divides
; by 16 yielding a "length in paragraphs" value.
;
; ENTRY (ax) = byte length
; EXIT (ax) = rounded up length in paragraphs
; USES ax, flags
;
ParaRound:
    add ax,15
    rcr ax,1
    shr ax,1
    shr ax,1
    shr ax,1
    shr ax,1
    retn

; -----
; sysinit_base module.
;
; This module is relocated by the routine EndFile to a location in low
; memory. It is then called by SYSINIT to perform the EXEC of programs
; that are being loaded by the "install=" command. After the EXEC call
; completes, this module performs a checksum on the SYSINIT code (at the
; top of memory) to be sure that the EXECed program did not damage it.
; If it did, then this module will print an error message and stop the
; system. Otherwise, it returns control to SYSINIT.
;
; in: after relocation,
; ax = 4b00h - load and execute the program dos function.
; ds = confbot. segment of config.sys file image
; es = sysinitseg. segment of sysinit module itself.
; ds:dx = pointer to asciiz string of the path,filename to be executed.
; es:bx = pointer to a parameter block for load.
; SI_end (byte) - offset vaule of end of sysinit module label
; bigsize (word) - # of word from confbot to SI_end.
; chksum (word) - sum of every byte from confbot to SI_end in a
; word boundary modular form.
; sysinit_ptr (dword ptr) - return address to sysinit module.
;
; note: sysinit should save necessary registers and when the control is back
;
; 24/10/2022
; (SYSINIT:115Ch for MSDOS 5.0 SYSINIT)
sysinit_base:
    mov [cs:sysinit_base_ss],ss ; save stack
    mov [cs:sysinit_base_sp],sp
    int 21h ; load/exec dos call.
    mov ss,[cs:sysinit_base_ss] ; restore stack
    mov sp,[cs:sysinit_base_sp]
    pop ds ; restore confbot seg
    pop es ; restore sysinitseg
    jc short sysinit_base_end; load/exec function failed.
    ; at this time,i don't have to worry about
    ; that sysinit module has been broken or not.
    ; otherwise,check if it is good.
    call sum_up
    cmp [es:checksum],ax
    je short sysinit_base_end

; memory broken. show "memory allocation error" message and stall.

    mov ah,9
    push cs
    pop ds
    ; 30/12/2022
    ; (MSDOS 6.21 IO.SYS, SYSINIT:12B8h)
    mov dx,102
    mov dx,mem_alloc_err_msgx-sysinit_base ; 65h (for MSDOS 5.0 SYSINIT)
    ; 66h (for MSDOS 6.21 SYSINIT)
    int 21h
    ; DOS - PRINT STRING
    ; DS:DX -> string terminated by "$"

; 30/12/2022 - Retro DOS v4.2
stall:
; 24/10/2022
_stall:
; 11/12/2022
    hlt
    ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
    ; use HLT to minimize energy consumption
    jmp short _stall

sysinit_base_end:
    jmp far [es:sysinit_ptr] ;return back to sysinit module

; -----
sum_up:
; in: es - sysinitseg.
; out: ax - result
;
; remark: since this routine will only check starting from "locstack" to the end of
; sysinit segment,the data area, and the current stack area are not
; covered. in this sense,this check sum routine only gives a minimal
; gaurantee to be safe.
;
; first sum up confbot seg.

    push ds
    ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
    ; (SYSINIT:12C6h)
    mov ax,[es:CONFBOT]
    ; 24/10/2022
    mov ax,[es:top_of_cdss]
    mov ds,ax
    xor si,si
    xor ax,ax
    mov cx,[es:config_size] ; if config_size has been broken,then this
    ; whole test better fail.
    shr cx,1 ; make it a word count
    jz short sum_sys_code ; when config.sys file not exist.

sum1:
    add ax,[si]
    inc si
    inc si
    loop sum1
; now,sum up sysinit module.
sum_sys_code:
; 24/10/2022
    mov si,locstack ; 5A6h (MSDOS 5.0 IO.SYS, SYSINIT)
    ; 532h (MSDOS 6.21 IO.SYS, SYSINIT)

```

```

28130                                ; 10/04/2024
28131                                ; 586h (PCDOS 7.1 IBMBIO.COM, SYSINIT)
28132                                ; starting after the stack. M069
28133                                ; this does not cover the possible stack code!!!
28134                                ;;mov cx,22688 ; for MSDOS 6.21 IO.SYS
28135                                ; 02/11/2022
28136                                ;;mov cx,3D20h ; (15648) for MSDOS 5.0 IO.SYS (SYSINIT)
28137                                ; 10/04/2024
28138                                ;mov cx,5B40h ; (23360) for PCDOS 7.1 IBMBIO.COM (SYSINIT)
28139                                ; 30/12/2022
28140 00001373 B9[1054] mov cx,SI_end ; (22688) ; SI_end is the label at the end of sysinit
28141 00001376 29F1 sub cx,si ; from after_checksum to SI_end
28142 00001378 D1E9 shr cx,1
28143 sum2:
28144 0000137A 260304 add ax,[es:si]
28145 0000137D 46 inc si
28146 0000137E 46 inc si
28147 0000137F E2F9 loop sum2
28148 00001381 1F pop ds
28149 00001382 C3 retn
28150
28151 ; 24/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
28152 ; 30/12/2022 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS, SYSINIT)
28153 ; (SYSINIT:12F2h)
28154 ; 10/04/2024 - Retro DOS 5.0 (Modified PCDOS 7.1 IBMBIO.COM, SYSINIT)
28155 ; (SYSINIT:149Dh)
28156
28157 sysinit_base_ss equ $-sysinit_base ; = 61 (MSDOS 5.0 IO.SYS, SYSINIT:115Ch)
28158 ;SYSINIT:11BDh: ; = 62 (MSDOS 6.21 IO.SYS, SYSINIT:1290h)
28159 ; = 62 (PCDOS 7.1 IBMBIO.COM, SYSINIT:143Bh)
28160 sysinit_base_sxx:
28161 00001383 0000 dw 0
28162 sysinit_base_sp equ $-sysinit_base ; = 63 (MSDOS 5.0 IO.SYS, SYSINIT:1161h)
28163 ;SYSINIT:11BFh: ; = 64 (MSDOS 6.21 IO.SYS, SYSINIT:1295h)
28164 ; = 64 (PCDOS 7.1 IBMBIO.COM, SYSINIT:1440h)
28165 sysinit_base_spx:
28166 00001385 0000 dw 0
28167
28168 mem_alloc_err_msgx:
28169
28170 ;include msbio.c14 ; memory allocation error message
28171
28172 ;(SYSINIT:12F6h: ; MSDOS 6.21 IO.SYS)
28173 ;SYSINIT:14A1h: ; PCDOS 7.1 IBMBIO.COM
28174 00001387 0D0A db 0Dh,0Ah
28175 00001389 4D656D6F727920616C- db 'Memory allocation error $'
28175 00001392 6C6F63617469666E20-
28175 0000139B 6572726F722024
28176
28177 end_sysinit_base: ; label byte
28178 ; 24/10/2022
28179 ; (SYSINIT:11DCh for MSDOS 5.0 SYSINIT)
28180
28181 ; -----
28182 ; Set_Buffer
28183 ;
28184 ;function: set buffers in the real memory.
28185 ; lastly set the memhi,memlo for the next available free address.
28186 ;
28187 ;input: ds:bx -> buffinfo.
28188 ; [memhi]:[memlo = 0] = available space for the hash bucket.
28189 ; singlebuffersize = buffer header size + sector size
28190 ;
28191 ;output: buffers queue established.
28192 ; [memhi]:[memlo] = address of the next available free space.
28193 ; -----
28194
28195 ; 25/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
28196 ; (SYSINIT:11DCh)
28197
28198 ; 11/04/2024 - Retro DOS 5.0 (Modified PCDOS 7.1 IBMBIO.COM)
28199 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:14BCh)
28200
28201 set_buffer:
28202 000013A2 30D2 xor dl,dl ; assume buffers not in HMA
28203 000013A4 E85500 call GetBufferAddr
28204 000013A7 7402 jz short set_buff_1
28205 000013A9 B201 mov dl,1 ; buffers in HMA
28206
28207 set_buff_1:
28208 ; 25/10/2022
28209 000013AB 893F mov [bx+BUFFINF.Buff_Queue],di ; head of Buff Q
28210 mov [bx],di
28211 000013AD 8C4702 mov [bx+BUFFINF.Buff_Queue+2],es
28212 mov [bx+2],es
28213 000013B0 C747040000 mov word [bx+BUFFINF.Dirty_Buff_Count],0 ;set dirty_count to 0.
28214 mov word [bx+4],0
28215 000013B5 89F8 mov ax,di
28216 000013B7 2E8B0E[9902] mov cx,[cs:buffers]
28217 000013BC 57 push di ; remember first buffer
28218
28219 ; for each buffer
28220
28221 nxt_buff:
28222 000013BD E87500 call set_buffer_info ; set buf_link,buf_id...
28223 000013C0 89C7 mov di,ax
28224 000013C2 E2F9 loop nxt_buff
28225
28226 000013C4 2E2B3E[9D02] sub di,[cs:singlebuffersize] ; point to last buffer
28227
28228 000013C9 59 pop cx ; get first buffer
28229 mov [es:di+buffinfo.buf_next],cx ; last->next = first
28230 000013CA 26890D mov [es:di],cx
28231 000013CD 87CF xchg cx,di
28232 mov [es:di+buffinfo.buf_prev],cx ; first->prev = last
28233 ; 25/10/2022
28234 000013CF 26894D02 mov [es:di+2],cx
28235
28236 000013D3 08D2 or dl,dl ; In Hma ?
28237 000013D5 7417 jz short set_buff_2 ; no
28238 ;mov byte [bx+BUFFINF.Buff_In_HMA],1
28239 000013D7 C6470C01 mov byte [bx+12],1
28240 000013DB 2EA1[6403] mov ax,[cs:memhi] ; seg of scratch buff
28241 ;mov word [bx+BUFFINF.Lo_Mem_Buff],0 ; offset of sctarch buff is 0
28242 000013DF C7470D0000 mov word [bx+13],0
28243 ;mov [bx+BUFFINF.Lo_Mem_Buff+2],ax
28244 000013E4 89470F mov word [bx+15],ax
28245 000013E7 2EA1[9D02] mov ax,[cs:singlebuffersize] ; size of scratch buff
28246 ; 11/04/2024 - Retro DOS v5.0
28247 ; 05/09/2023
28248 ;sub ax,bufinsiz ; 20 ; buffer head not required
28249 ;sub ax,20
28250 000013EB 83E818 sub ax,24 ; bufinsiz ; (bufinsiz is 24 in PCDOS 7.1)
28251

```



```

28252 set_buff_2:
28253     add     [cs:memlo],ax
28254     ;or     byte [cs:setdevmarkflag],for_devmark ; 2
28255     or      byte [cs:setdevmarkflag],2
28256     ;call   round
28257     ;retn
28258     ; 12/12/2022
28259     jmp     round
28260
28261 ; -----
28262 ; procedure : GetBufferAddr
28263 ;
28264 ;         Gets the buffer address either in HMA or in Lo Mem
28265 ;
28266 ; returns in es:di the buffer address
28267 ; returns NZ if allocated in HMA
28268 ; -----
28269
28270     ; 25/10/2022
28271 GetBufferAddr:
28272     push    bx
28273     push    dx
28274
28275     ; 11/04/2024 - Retro DOS v5.0
28276     ; PCDOS 7.1 IBMBIO.COM
28277     ;;
28278     cmp     byte [cs:dosdata_umb],2
28279     ;         ; is dosdata moved to UMB ? (DOSDATA=UMB done)
28280     jne     short gba_1 ; no
28281     cmp     word [bx+2],0FFFFh ; is the buffer (already) in HMA ?
28282     je      short gba_2 ; yes
28283 gba_1:
28284     ;;
28285
28286     mov     ax, [cs:singlebuffersize]
28287     mul     word [cs:buffers]
28288     ;add     ax,0Fh
28289     add     ax,15
28290     ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
28291     ;and     ax,~15 ; 0FFF0h ; para round
28292     ; 12/12/2022
28293     and     al,~15 ; 0F0h
28294     mov     bx,ax
28295     mov     ax,4A02h
28296     ;mov     ax,((multMULT<<8)+multMULTALLOCHMA)
28297     int     2Fh ; DOS 5+ - ALLOCATE HMA SPACE
28298     ;         ; AX = 4A02h
28299     ;         ; BX = number of bytes
28300     ; Return:
28301     ;         ; ES:DI -> start of allocated HMA block or FFFFh:FFFFh
28302     ;         ; BX = number of bytes actually allocated
28303     ;         ; (rounded up to next paragraph)
28304     ; Notes:
28305     ;         ; this call is not valid unless DOS is loaded in the HMA
28306     ;         ; (DOS=HIGH)
28307
28308     cmp     di,0FFFFh
28309     jne     short got_hma
28310
28311     ;mov     di,0 ; dont xor di,di Z flag needed
28312     ; 05/09/2023
28313     ; zf=1
28314     inc     di ; 0FFFFh -> 0
28315     ; zf=1
28316
28317     ;zf=1
28318     ;xor     di,di ; 25/10/2022
28319     ;zf=1
28320     mov     es,[cs:memhi]
28321 got_hma:
28322     pop     dx
28323     pop     bx
28324     retn
28325
28326     ; 11/04/2024 - Retro DOS v5.0
28327     ; PCDOS 7.1 IBMBIO.COM
28328     ;;
28329 gba_2:
28330     les     di,[bx]
28331     or      di,di
28332     ;pop     dx
28333     ;pop     bx
28334     ;retn
28335     ; 11/04/2024 - Retro DOS v5.0
28336     jmp     short got_hma
28337     ;;
28338
28339 ; -----
28340
28341 set_buffer_info:
28342 ;function: set buf_link,buf_id,buf_sector
28343 ;
28344 ;in: es:di -> buffer header to be set.
28345 ;     ax = di
28346 ;
28347 ;out:
28348 ;     above entries set.
28349 ;
28350     ; 25/10/2022
28351     push    word [cs:buf_prev_off]
28352     pop     word [es:di+buffinfo.buf_prev]
28353     pop     word [es:di+2]
28354     mov     [cs:buf_prev_off],ax
28355     add     ax,[cs:singlebuffersize] ;adjust ax
28356     mov     [es:di+buffinfo.buf_next],ax
28357     mov     [es:di],ax
28358     mov     word [es:di+buffinfo.buf_ID],00FFh ; new buffer free
28359     mov     word [es:di+4],00FFh
28360     mov     word [es:di+buffinfo.buf_sector],0 ; to compensate the masm 3 bug
28361     mov     word [es:di+6],0
28362     mov     word [es:di+buffinfo.buf_sector+2],0 ; to compensate the masm 3 bug
28363     mov     word [es:di+8],0
28364     retn
28365
28366 ; =====
28367 ; MSSTACK initialization routine - MSDOS 6.0 - SYSDINIT1.ASM - 1991
28368 ; -----
28369 ; 27/03/2019 - Retro DOS v4.0
28370
28371 ; -----
28372 ; ibmstack initialization routine.
28373 ;
28374 ; to follow the standard interrupt sharing scheme, msstack.asm
28375

```

```

28376 ; has been modified. this initialization routine also has to
28377 ; be modified because for the interrupt level 7 and 15, firstflag
28378 ; should be set to signal that this interrupt handler is the
28379 ; first handler hooked to this interrupt vector.
28380 ; we determine this by looking at the instruction pointed by
28381 ; this vector. if it is iret, then this handler should be the
28382 ; first one. in our case, only the interrupt vector 77h is the
28383 ; interrupt level 15. (we don't hook interrupt level 7.)
28384 ;
28385 ; the followings are mainly due to m.r.t; ptm fix of p886 12/3/86
28386 ; some design changes are needed to the above interrupt sharing
28387 ; method. the above sharing scheme assumes that 1). interrupt
28388 ; sharing is never done on levels that have bios support. 2). "phantom"
28389 ; interrupts would only be generated on levels 7 and 15.
28390 ; these assumptions are not true any more. we have to use the firstflag
28391 ; for every level of interrupt. we will set the firstflag on the following
28392 ; conditions:
28393 ;
28394 ; a. if the cs portion of the vector is 0000, then "first"
28395 ; b. else if cs:ip points to valid shared header, then not "first"
28396 ; c. else if cs:ip points to an iret, then "first"
28397 ; d. else if cs:ip points to dummy, then "first"
28398 ;
28399 ; where dummy is - the cs portion must be f000, and the ip portion must
28400 ; be equal to the value at f000:ff01. this location is the initial value
28401 ; from vector_table for interrupt 7, one of the preserved addresses in all
28402 ; the bioses for all of the machines.
28403 ;
28404 ; system design group requests bios to handle the phantom interrupts.
28405 ;
28406 ; the "phantom" interrupt is an illegal interrupt such as an interrupt
28407 ; produced by the bogus adapter card even without interrupt request is
28408 ; set. more specifically, 1). the 8259 has a feature when running in
28409 ; edge triggered mode to latch a pulse and present the interrupt when
28410 ; the processor indicates interrupt acknowledge (inta). the interrupt
28411 ; pulse was exist at the time of inta to get a "phantom" interrupt.
28412 ; 2). or, this is caused by adapter cards placing a glitch on the
28413 ; interrupt line.
28414 ;
28415 ; to handle those "phantom" interrupts, the main stack code will check
28416 ; the own firstflag, and if it is not "first" (which means the forward
28417 ; pointer points to the legal shared interrupt handler), then pass the
28418 ; control. if it is the first, then the following action should be
28419 ; taken. we don't have to implement skack logic in this case.
28420 ;
28421 ; to implement this logic, we rather choose a simple method.
28422 ; if out of the above "firstflag" conditions is met, we are not
28423 ; going to hook this interrupt vector. the reason is if the original
28424 ; vector points to "iret" and do nothing, we don't need
28425 ; to implement the stack logic for it. this will simplify implementation
28426 ; while maintaining compatibility with the old version of dos.
28427 ; this implies that in the main stack code, there might be a stack code
28428 ; that will never be used, a dead code.
28429 ;
28430 ;in - cs, ds -> sysinitseg, es -> relocated stack code & data.
28431 ;
28432 ; 25/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
28433 ; (SYSINIT:1287h)
28434 ;
28435 ; 11/04/2024 - Retro DOS 5.0 (Modified PCDOS 7.1 IBMBIO.COM)
28436 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:157Ch)
28437 ;
28438 ; 14/12/2022
28439 stackinit:
28440 push ax
28441 push ds
28442 push es
28443 push bx
28444 push cx
28445 push dx
28446 push di
28447 push si
28448 push bp
28449 ;
28450 ;currently es -> stack code area
28451 ;
28452 ; 12/12/2022
28453 ; ds = cs
28454 mov ax,[stack_count]
28455 mov cx,ax ; *!*
28456 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
28457 ; (MSDOS 5.0 IO.SYS - SYSINIT:1290h)
28458 ;mov ax,[cs:stack_count] ; !! ;defined in cs
28459 mov [es:stackcount],ax ;defined in stack code area
28460 ; (MSDOS 5.0 IO.SYS - SYSINIT:1298h)
28461 mov ax,[stack_size] ; !! ;in cs
28462 mov [es:stacksize],ax
28463 ; 12/12/2022
28464 mov ax,[stack_addr] ; offset
28465 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
28466 ; (MSDOS 5.0 IO.SYS - SYSINIT:129Fh)
28467 ;mov ax,[cs:stack_addr] ; !!
28468 mov [es:stacks],ax
28469 ; 12/12/2022
28470 mov bp,ax ; *!*
28471 mov ax,[stack_addr+2]
28472 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
28473 ; (MSDOS 5.0 IO.SYS - SYSINIT:129Fh)
28474 ;mov ax,[cs:stack_addr+2] ; !! ; segment
28475 mov [es:stacks+2],ax
28476 ;
28477 ; initialize the data fields with the parameters
28478 ;
28479 ; "firstentry" will always be at stacks
28480 ;
28481 ;mov bp,[es:stacks] ; get offset of stack
28482 ; 12/12/2022
28483 ; bp = [es:stacks] ; *!*
28484 mov [es:firstentry],bp
28485 ;
28486 ; the stacks will always immediately follow the table entries
28487 ;
28488 mov ax,entrysize ; 8
28489 ;mov cx,[es:stackcount]
28490 ; 12/12/2022
28491 ; cx = [es:stackcount] ; *!*
28492 mul cx
28493 add ax,bp
28494 mov [es:stackat],ax
28495 mov bx,ax
28496 sub bx,2
28497 ;
28498 ; zero the entire stack area to start with
28499 ;

```



```

28500 0000149B 268B3E[0400]      mov     di,[es:stackat]
28501 000014A0 26A1[0600]      mov     ax,[es:stacksize]
28502 000014A4 F7E1      mul     cx
28503 000014A6 89C1      mov     cx,ax
28504 000014A8 31C0      xor     ax,ax
28505 000014AA 06      push    es
28506 000014AB 1F      pop     ds                ;ds = relocated stack code seg.
28507
28508      ;now, ds -> stack code area
28509
28510 000014AC 8E06[0A00]      mov     es,[stacks+2]      ; get segment of stack area.
28511 000014B0 FC      cld
28512 000014B1 F3AA      rep     stosb
28513
28514 000014B3 8B0E[0200]      mov     cx,[stackcount]
28515
28516      ; loop for "count" times, building a table entry
28517      ; cs = sysinitseg, ds = relocated stack code seg, es = segment of stack space
28518      ; cx = number of entries
28519      ; es:bp => base of stacks - 2
28520      ; es:bx => first table entry
28521
28522      buildloop:
28523      ; 11/12/2022
28524      ;mov     byte [es:bp+allocbyte],free ; mov [es:bp+0],0
28525      ; 25/10/2022
28526      ;mov     byte [es:bp],free
28527      ; 06/07/2023
28528 000014B7 26884600      mov     [es:bp],al ; 0 ; free
28529 000014B8 26884601      mov     [es:bp+intlevel],al ; ax = 0
28530      ;mov     [es:bp+1],al
28531 000014BF 26894602      mov     [es:bp+savesp],ax
28532      ;mov     [es:bp+2],ax
28533 000014C3 26894604      mov     [es:bp+savesd],ax
28534      ;mov     [es:bp+4],ax
28535 000014C7 031E[0600]      add     bx,[stacksize]
28536 000014CB 26895E06      mov     [es:bp+newsp],bx      ; mov [es:bp+6],bx
28537      ;mov     [es:bp+6],bx
28538 000014CF 26892F      mov     [es:bx],bp
28539 000014D2 83C508      add     bp,entrysize ; 8
28540
28541 000014D5 E2E0      loop    buildloop
28542
28543 000014D7 83ED08      sub     bp,entrysize ; 8
28544 000014DA 892E[0E00]      mov     [lastentry],bp
28545 000014DE 892E[1000]      mov     [nextentry],bp
28546
28547 000014E2 1E      push    ds
28548      ;mov     ax,0F000h      ;look at the model byte
28549      ; 05/09/2023
28550 000014E3 B4F0      mov     ah,0F0h ; ax = 0F000h
28551 000014E5 8ED8      mov     ds,ax
28552 000014E7 803EFEFF9      cmp     byte [0FFFFh],0F9h ; mdl_convert ; convertible?
28553 000014EC 1F      pop     ds
28554 000014ED 7504      jne     short skip_disablenmis
28555
28556 000014EF B007      mov     al,07h      ; disable convertible nmis
28557 000014F1 E672      out     72h,al
28558
28559      skip_disablenmis:
28560 000014F3 31C0      xor     ax,ax
28561 000014F5 8EC0      mov     es,ax      ;es - segid of vector table at 0
28562      ;ds - relocated stack code segment
28563 000014F7 FA      cli
28564
28565      ;irp     aa,<02,08,09,70>
28566      ;
28567      ;mov     si,aa&h*4      ;pass where vector is to be adjusted
28568      ;mov     di,offset int19old&aa ;we have to set old&aa for int19 handler too.
28569      ;mov     bx,offset old&aa      ;pass where to save original owner pointer
28570      ;mov     dx,offset int&aa      ;pass where new handler is
28571      ;call    new_init_loop      ;adjust the vector to new handler,
28572      ;      ;saving pointer to original owner
28573      ;endm
28574
28575      stkinit_02:
28576 000014F8 BE0800      mov     si,02h*4 ; 8
28577 000014FB BF[B305]      mov     di,INT19OLD02
28578 000014FE BB[1200]      mov     bx,old02
28579 00001501 BA[1600]      mov     dx,int02
28580 00001504 E84801      call    new_init_loop
28581
28582 00001507 BE2000      mov     si,08h*4 ; 32
28583 0000150A BF[B805]      mov     di,INT19OLD08
28584 0000150D BB[3800]      mov     bx,old08
28585 00001510 BA[3C00]      mov     dx,int08
28586 00001513 E83901      call    new_init_loop
28587
28588 00001516 BE2400      mov     si,09h*4 ; 36
28589 00001519 BF[BD05]      mov     di,INT19OLD09
28590 0000151C BB[4100]      mov     bx,old09
28591 0000151F BA[4500]      mov     dx,int09
28592 00001522 E82A01      call    new_init_loop
28593
28594 00001525 BEC001      mov     si,70h*4 ; 448
28595 00001528 BF[DB05]      mov     di,INT19OLD70
28596 0000152B BB[4E00]      mov     bx,old70
28597 0000152E BA[5200]      mov     dx,int70
28598 00001531 E81B01      call    new_init_loop
28599
28600      ;irp     aa,<0a,0b,0c,0d,0e,72,73,74,76,77> ;shared interrupts
28601      ;
28602      ;mov     si,aa&h*4      ;pass where vector is to be adjusted
28603      ;push    ds      ;save relocated stack code segment
28604      ;lds     bx, es:[si]      ;ds:bx -> original interrupt handler
28605      ;push    ds
28606      ;pop     dx      ;dx = segment value
28607      ;
28608      ;cmp     dx,0
28609      ;jz      int&aa&_first
28610      ;
28611      ;cmp     byte ptr ds:[bx],0cfh ;does vector point to an iret?
28612      ;jz      int&aa&_first
28613      ;
28614      ;cmp     word ptr ds:[bx.6],424bh ;magic offset (see int&aa, msstack.inc)
28615      ;jz      int&aa&_not_first
28616      ;
28617      ;cmp     dx,0f000h      ;rom bios segment
28618      ;jnz     int&aa&_not_first
28619      ;
28620      ;push    es
28621      ;push    dx
28622      ;mov     dx,0f000h
28623      ;mov     es,dx

```

```

28624         ;cmp     bx,word ptr es:0ff01h
28625         ;pop     dx
28626         ;pop     es
28627         ;jz      int&aa&_first
28628         ;
28629         ;int&aa&_not_first:                ;not the first. we are going to hook vector.
28630         ;pop     ds
28631         ;mov     di, offset int19old&aa;we have to set old&aa for int19 handler too.
28632         ;mov     bx, offset old&aa        ;pass where to save original owner pointer
28633         ;mov     dx, offset int&aa        ;pass where new handler is
28634         ;call    new_init_loop            ;adjust the vector to new handler, saving
28635         ;        ;pointer to original owner.
28636         ;
28637         ;jmp     short int&aa&_end
28638         ;int&aa&_first:                    ;the first. don't have to hook stack code.
28639         ;pop     ds
28640         ;int&aa&_end:
28641         ;
28642         ;endm
28643         stkinit_0A:
28644         00001534 BE2800
28645         mov     si,0Ah*4 ; 40
28646         ; 14/12/2022
28647         %if 0
28648         ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
28649         push    ds
28650
28651         lds     bx,[es:si]
28652         push    ds
28653         pop     dx
28654
28655         cmp     dx,0
28656         je      short int_0A_first
28657
28658         cmp     byte [bx],0CFh
28659         je      short int_0A_first
28660
28661         cmp     word [bx+6],424Bh
28662         je      short int_0A_not_first
28663
28664         cmp     dx,0F000h
28665         jne     short int_0A_not_first
28666
28667         push    es
28668         push    dx
28669         mov     dx,0F000h
28670         mov     es,dx
28671         cmp     bx,[es:0FF01h]
28672         pop     dx
28673         pop     es
28674         je      short int_0A_first
28675         %endif
28676
28677         ; 14/12/2022
28678         ; 25/10/2022
28679         00001537 E8EB00
28680         0000153A 730C
28681         call    int_xx_first_check ; 27/03/2019 - Retro DOS v4.0
28682         jnc     short int_0A_first
28683
28684         int_0A_not_first:
28685         ; 14/12/2022
28686         ; 25/10/2022
28687         ;pop     ds
28688         0000153C BFC205]
28689         0000153F BB5900]
28690         00001542 BA5700]
28691         00001545 E80701
28692         mov     di,INT19OLD0A
28693         mov     bx,old0A
28694         mov     dx,int0A
28695         call    new_init_loop
28696
28697         ; 14/12/2022
28698         ;jmp     short int_0A_end
28699         ;int_0A_first:
28700         ; 25/10/2022
28701         ;pop     ds
28702
28703         ; 14/12/2022
28704         ; 25/10/2022
28705         0000154B E8D700
28706         0000154E 730C
28707         call    int_xx_first_check ; 27/03/2019 - Retro DOS v4.0
28708         jnc     short int_0B_end ; int_0B_first
28709
28710         ; 14/12/2022
28711         %if 0
28712         ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
28713         push    ds
28714         lds     bx,[es:si]
28715         push    ds
28716         pop     dx
28717
28718         cmp     dx,0
28719         je      short int_0B_first
28720
28721         cmp     byte [bx],0CFh
28722         je      short int_0B_first
28723
28724         cmp     word [bx+6],424Bh
28725         je      short int_0B_not_first
28726
28727         cmp     dx,0F000h
28728         jne     short int_0B_not_first
28729
28730         push    es
28731         push    dx
28732         mov     dx,0F000h
28733         mov     es,dx
28734         cmp     bx,[es:0FF01h]
28735         pop     dx
28736         pop     es
28737         je      short int_0B_first
28738         %endif
28739
28740         int_0B_not_first:
28741         ; 14/12/2022
28742         ; 25/10/2022
28743         ;pop     ds
28744         00001550 BFC705]
28745         00001553 BB7100]
28746         00001556 BA6F00]
28747         00001559 E8F300
28748         mov     di,INT19OLD0B
28749         mov     bx,old0B
28750         mov     dx,int0B
28751         call    new_init_loop

```

```

28748             ; 14/12/2022
28749             ; jmp     short int_0B_end
28750 ;int_0B_first:
28751             ; 25/10/2022
28752             ; pop     ds
28753
28754 int_0B_end:
28755
28756 stkinit_0C:
28757     mov     si,0Ch*4 ; 48
28758
28759             ; 14/12/2022
28760             ; 25/10/2022
28761     call    int_xx_first_check
28762     jnc     short int_0C_end ; int_0C_first
28763
28764 ; 14/12/2022
28765 %if 0
28766             ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
28767     push    ds
28768     lds     bx,[es:si]
28769     push    ds
28770     pop     dx
28771
28772     cmp     dx,0
28773     je      short int_0C_first
28774
28775     cmp     byte [bx],0CFh
28776     je      short int_0C_first
28777
28778     cmp     word [bx+6],424Bh
28779     je      short int_0C_not_first
28780
28781     cmp     dx,0F000h
28782     jne     short int_0C_not_first
28783
28784     push    es
28785     push    dx
28786     mov     dx,0F000h
28787     mov     es,dx
28788     cmp     bx,[es:0FF01h]
28789     pop     dx
28790     pop     es
28791     je      short int_0C_first
28792 %endif
28793
28794 int_0C_not_first:
28795             ; 14/12/2022
28796             ; 25/10/2022
28797             ; pop     ds
28798     mov     di,INT19OLD0C
28799     mov     bx,old0C
28800     mov     dx,int0C
28801     call    new_init_loop
28802
28803             ; 14/12/2022
28804             ; jmp     short int_0C_end
28805 ;int_0C_first:
28806             ; 25/10/2022
28807             ; pop     ds
28808
28809 int_0C_end:
28810
28811 stkinit_0D:
28812     mov     si,0Dh*4 ; 52
28813
28814             ; 14/12/2022
28815             ; 25/10/2022
28816     call    int_xx_first_check
28817     jnc     short int_0D_end ; int_0D_first
28818
28819 ; 14/12/2022
28820 %if 0
28821             ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
28822     push    ds
28823     lds     bx,[es:si]
28824     push    ds
28825     pop     dx
28826
28827     cmp     dx,0
28828     je      short int_0D_first
28829
28830     cmp     byte [bx],0CFh
28831     je      short int_0D_first
28832
28833     cmp     word [bx+6],424Bh
28834     je      short int_0D_not_first
28835
28836     cmp     dx,0F000h
28837     jne     short int_0D_not_first
28838
28839     push    es
28840     push    dx
28841     mov     dx,0F000h
28842     mov     es,dx
28843     cmp     bx,[es:0FF01h]
28844     pop     dx
28845     pop     es
28846     je      short int_0D_first
28847 %endif
28848
28849 int_0D_not_first:
28850             ; 14/12/2022
28851             ; 25/10/2022
28852             ; pop     ds
28853     mov     di,INT19OLD0D
28854     mov     bx,old0D
28855     mov     dx,int0D
28856     call    new_init_loop
28857
28858             ; 14/12/2022
28859             ; jmp     short int_0D_end
28860             ; 02/11/2022
28861 ;int_0D_first:
28862             ; pop     ds
28863
28864 int_0D_end:
28865
28866 stkinit_0E:
28867     mov     si,0Eh*4 ; 56
28868
28869             ; 14/12/2022
28870             ; 25/10/2022
28871     call    int_xx_first_check

```

```

28872 0000158A 730C      jnc     short int_0E_end ; int_0E_first
28873
28874 ; 14/12/2022
28875 %if 0
28876 ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
28877 push    ds
28878 lds     bx,[es:si]
28879 push    ds
28880 pop     dx
28881
28882 cmp     dx,0
28883 je      short int_0E_first
28884
28885 cmp     byte [bx],0CFh
28886 je      short int_0E_first
28887
28888 cmp     word [bx+6],424Bh
28889 je      short int_0E_not_first
28890
28891 cmp     dx,0F000h
28892 jne     short int_0E_not_first
28893
28894 push    es
28895 push    dx
28896 mov     dx,0F000h
28897 mov     es,dx
28898 cmp     bx,[es:0FF01h]
28899 pop     dx
28900 pop     es
28901 je      short int_0E_first
28902 %endif
28903
28904 int_0E_not_first:
28905 ; 14/12/2022
28906 ; 25/10/2022
28907 ;pop     ds
28908 mov     di,INT19OLD0E
28909 0000158F BB[B900]   mov     bx,old0E
28910 00001592 BA[B700]   mov     dx,int0E
28911 00001595 E8B700     call    new_init_loop
28912
28913 ; 14/12/2022
28914 ;jmp     short int_0E_end
28915 ;int_0E_first:
28916 ; 25/10/2022
28917 ;pop     ds
28918
28919 int_0E_end:
28920
28921 stkinit_72:
28922 00001598 BEC801     mov     si,72h*4 ; 456
28923
28924 ; 14/12/2022
28925 ; 25/10/2022
28926 0000159B E88700     call    int_xx_first_check
28927 0000159E 730C      jnc     short int_72_end ; int_72_first
28928
28929 ; 14/12/2022
28930 %if 0
28931 ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
28932 push    ds
28933 lds     bx,[es:si]
28934 push    ds
28935 pop     dx
28936
28937 cmp     dx,0
28938 je      short int_72_first
28939
28940 cmp     byte [bx],0CFh
28941 je      short int_72_first
28942
28943 cmp     word [bx+6],424Bh
28944 je      short int_72_not_first
28945
28946 cmp     dx,0F000h
28947 jne     short int_72_not_first
28948
28949 push    es
28950 push    dx
28951 mov     dx,0F000h
28952 mov     es,dx
28953 cmp     bx,[es:0FF01h]
28954 pop     dx
28955 pop     es
28956 je      short int_72_first
28957 %endif
28958
28959 int_72_not_first:
28960 ; 14/12/2022
28961 ; 25/10/2022
28962 ;pop     ds
28963 000015A0 BF[E005]   mov     di,INT19OLD72
28964 000015A3 BB[D100]   mov     bx,old72
28965 000015A6 BA[CF00]   mov     dx,int72
28966 000015A9 E8A300     call    new_init_loop
28967
28968 ; 14/12/2022
28969 ;jmp     short int_72_end
28970 ;int_72_first:
28971 ; 25/10/2022
28972 ;pop     ds
28973
28974 int_72_end:
28975
28976 stkinit_73:
28977 000015AC BECC01     mov     si,73h*4 ; 460
28978
28979 ; 14/12/2022
28980 ; 25/10/2022
28981 000015AF E87300     call    int_xx_first_check
28982 000015B2 730C      jnc     short int_73_end ; int_73_first
28983
28984 ; 14/12/2022
28985 %if 0
28986 ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
28987 push    ds
28988 lds     bx,[es:si]
28989 push    ds
28990 pop     dx
28991
28992 cmp     dx,0
28993 je      short int_73_first
28994
28995 cmp     byte [bx],0CFh

```

```

28996         je      short int_73_first
28997
28998         cmp     word [bx+6],424Bh
28999         je      short int_73_not_first
29000
29001         cmp     dx,0F000h
29002         jne     short int_73_not_first
29003
29004         push    es
29005         push    dx
29006         mov     dx,0F000h
29007         mov     es,dx
29008         cmp     bx,[es:0FF01h]
29009         pop     dx
29010         pop     es
29011         je      short int_73_first
29012 %endif
29013
29014 int_73_not_first:
29015     ; 14/12/2022
29016     ; 25/10/2022
29017     ;pop     ds
29018     mov     di,INT19OLD73
29019     mov     bx,old73
29020     mov     dx,int73
29021     call    new_init_loop
29022
29023     ; 14/12/2022
29024     ;jmp     short int_73_end
29025 ;int_73_first:
29026     ; 25/10/2022
29027     ;pop     ds
29028
29029 int_73_end:
29030
29031 stkinit_74:
29032     mov     si,74h*4 ; 464
29033
29034     ; 14/12/2022
29035     ; 25/10/2022
29036     call    int_xx_first_check
29037     jnc     short int_74_end ; int_74_first
29038
29039     ; 14/12/2022
29040 %if 0
29041     ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
29042     push    ds
29043     lds     bx,[es:si]
29044     push    ds
29045     pop     dx
29046
29047     cmp     dx,0
29048     je      short int_74_first
29049
29050     cmp     byte [bx],0CFh
29051     je      short int_74_first
29052
29053     cmp     word [bx+6],424Bh
29054     je      short int_74_not_first
29055
29056     cmp     dx,0F000h
29057     jne     short int_74_not_first
29058
29059     push    es
29060     push    dx
29061     mov     dx,0F000h
29062     mov     es,dx
29063     cmp     bx,[es:0FF01h]
29064     pop     dx
29065     pop     es
29066     je      short int_74_first
29067 %endif
29068
29069 int_74_not_first:
29070     ; 14/12/2022
29071     ; 25/10/2022
29072     ;pop     ds
29073     mov     di,INT19OLD74
29074     mov     bx,old74
29075     mov     dx,int74
29076     call    new_init_loop
29077
29078     ; 14/12/2022
29079     ;jmp     short int_74_end
29080 ;int_74_first:
29081     ; 25/10/2022
29082     ;pop     ds
29083
29084 int_74_end:
29085
29086 stkinit_76:
29087     mov     si,76h*4 ; 472
29088
29089     ; 14/12/2022
29090     ; 25/10/2022
29091     call    int_xx_first_check
29092     jnc     short int_76_end ; int_76_first
29093
29094     ; 14/12/2022
29095 %if 0
29096     ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
29097     push    ds
29098     lds     bx,[es:si]
29099     push    ds
29100     pop     dx
29101
29102     cmp     dx,0
29103     je      short int_76_first
29104
29105     cmp     byte [bx],0CFh
29106     je      short int_76_first
29107
29108     cmp     word [bx+6],424Bh
29109     je      short int_76_not_first
29110
29111     cmp     dx,0F000h
29112     jne     short int_76_not_first
29113
29114     push    es
29115     push    dx
29116     mov     dx,0F000h
29117     mov     es,dx
29118     cmp     bx,[es:0FF01h]
29119     pop     dx

```

```

29120      pop      es
29121      je       short int_76_first
29122  %endif
29123
29124      int_76_not_first:
29125      ; 14/12/2022
29126      ; 25/10/2022
29127      ;pop      ds
29128      mov     di,INT19OLD76
29129      mov     bx,old76
29130      mov     dx,int76
29131      call    new_init_loop
29132
29133      ; 14/12/2022
29134      jmp     short int_76_end
29135  ;int_76_first:
29136      ; 25/10/2022
29137      ;pop      ds
29138
29139      int_76_end:
29140
29141      stkinit_77:
29142      mov     si,77h*4 ; 476
29143
29144      ; 14/12/2022
29145      ; 25/10/2022
29146      call    int_xx_first_check
29147      jnc     short int_77_end ; int_77_first
29148
29149      ; 14/12/2022
29150  %if 0
29151      ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
29152      push    ds
29153      lds     bx,[es:si]
29154      push    ds
29155      pop     dx
29156
29157      cmp     dx,0
29158      je      short int_77_first
29159
29160      cmp     byte [bx],0CFh
29161      je      short int_77_first
29162
29163      cmp     word [bx+6],424Bh
29164      je      short int_77_not_first
29165
29166      cmp     dx,0F000h
29167      jne     short int_77_not_first
29168
29169      push    es
29170      push    dx
29171      mov     dx,0F000h
29172      mov     es,dx
29173      cmp     bx,[es:0FF01h]
29174      pop     dx
29175      pop     es
29176      je      short int_77_first
29177  %endif
29178
29179      int_77_not_first:
29180      ; 14/12/2022
29181      ; 25/10/2022
29182      ;pop      ds
29183      mov     di,INT19OLD77
29184      mov     bx,old77
29185      mov     dx,int77
29186      call    new_init_loop
29187
29188      ; 14/12/2022
29189      ;jmp     short int_77_end
29190  ;int_77_first:
29191      ; 25/10/2022
29192      ;pop      ds
29193
29194      int_77_end:
29195      push    ds
29196      mov     ax,0F000h ; look at the model byte
29197      mov     ds,ax
29198      cmp     byte [0FFFEh],0F9h ; mdl_convert ; pc convertible?
29199      pop     ds
29200      jne     short skip_enablenmis
29201
29202      mov     al,27h ; enable convertible nmis
29203      out     72h,al
29204
29205      ; 25/10/2022
29206      ; (MSDOS 5.0 SYSINIT:15FBh)
29207
29208      skip_enablenmis:
29209      sti
29210      ;mov     ax,Bios_Data ; 70h
29211      ;mov     ax,KERNEL_SEGMENT ; 70h
29212      ; 21/10/2022
29213      mov     ax,DOSBIODATASEG ; 0070h
29214      mov     ds,ax
29215
29216      ;mov     [640h],1 ; SYSINIT:1736h for MSDOS 6.21 IO.SYS
29217
29218      mov     byte [INT19SEM],1 ; indicate that int 19
29219      ; initialization is complete
29220
29221      pop     bp ; restore all
29222      pop     si
29223      pop     di
29224      pop     dx
29225      pop     cx
29226      pop     bx
29227      pop     es
29228      pop     ds
29229      pop     ax
29230      retn
29231
29232      ; 14/12/2022
29233      ; -----
29234
29235      ; 14/12/2022
29236      ; 25/10/2022
29237  %if 0
29238      ; 27/03/2019 - Retro DOS v4.0
29239      int_xx_first_check:
29240      push    ds
29241      lds     bx,[es:si]
29242      push    ds
29243      pop     dx

```



```

29244
29245 ;cmp dx,0
29246 ;je short int_xx_first
29247 ; 05/09/2023
29248 0000162B 21D2 and dx,dx
29249 0000162D 741E jz short int_xx_first
29250
29251 0000162F 803FCF cmp byte [bx],0CFh
29252 00001632 7419 je short int_xx_first
29253
29254 00001634 817F064B42 cmp word [bx+6],424Bh
29255 00001639 7411 je short int_xx_not_first
29256
29257 0000163B 81FA00F0 cmp dx,0F000h
29258 0000163F 750B jne short int_xx_not_first
29259
29260 00001641 06 push es
29261 ;push dx
29262 ;mov dx,0F000h
29263 00001642 8EC2 mov es,dx
29264 00001644 263B1E01FF cmp bx,[es:0FF01h]
29265 ;pop dx
29266 00001649 07 pop es
29267 0000164A 7401 je short int_xx_first
29268
29269 int_xx_not_first:
29270 stc
29271 int_xx_first:
29272 0000164D 1F pop ds
29273 0000164E C3 retn
29274
29275 ;%endif
29276
29277 ; -----
29278 ; 27/03/2019 - Retro DOS v4.0
29279
29280 ; 25/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
29281 ; (SYSINIT:1610h)
29282
29283 ; 13/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM, SYSINIT)
29284 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1905h)
29285
29286 new_init_loop:
29287 ;;; 13/04/2024 (PCDOS 7.1 IBMBIO.COM)
29288 0000164F 2E803E[6E03]02 cmp byte [cs:dosedata_umb],2
29289 ; is DOSDATA=UMB done ? (DOSDATA is in UMB)
29290 00001655 7510 jne short new_init_loop_1st
29291 00001657 1E push ds
29292 ; restore original/previous interrupt handler
29293 ; (from int19old?? field in BIOSDATA)
29294 ;mov ax,70h
29295 mov ax,DOSBIODATASEG
29296 mov ds,ax
29297 lds ax,[di] ; restore original Int ?? handler addr from int19old?? field
29298 0000165F 268904 mov [es:si],ax ; copy the original int handler addr to its int vector addr
29299 00001662 268C5C02 mov [es:si+2],ds
29300 pop ds
29301 new_init_loop_1st:
29302 ;;;
29303 ;input: si=offset into vector table of the particular int vector being adjusted
29304 ; bx=ds:offset of oldxx, where will be saved the pointer to original owner
29305 ; dx=ds:offset of intxx, the new interrupt handler
29306 ; di=offset value of int19old&aa variable in bios.
29307 ; es=zero, segid of vector table
29308 ; ds=relocated stack code segment
29309
29310 ; 13/04/2024
29311 %if 0
29312 mov ax,[es:si] ;remember offset in vector
29313 [bx],ax ; to original owner in ds
29314 mov ax,[es:si+2] ;remember segid in vector
29315 [bx+2],ax ; to original owner in ds
29316
29317 push ds
29318 ;mov ax,Bios_Data ; 70h
29319 ;mov ax,KERNEL_SEGMENT ; 70h
29320 ; 21/10/2022
29321 mov ax,DOSBIODATASEG ; 0070h
29322 mov ds,ax ;set int19oldxx value in bios for
29323 ax,[es:si] ;int 19 handler
29324 [di],ax
29325 mov ax,[es:si+2]
29326 [di+2],ax
29327 pop ds
29328 %else
29329 ; 13/04/2024 - Retro DOS v5.0
29330 00001667 1E push ds
29331 00001668 268B4402 mov ax,[es:si+2] ;remember segid in vector
29332 0000166C 894702 [bx+2],ax ; to original owner in ds
29333 0000166F 50 push ax
29334 00001670 268B04 mov ax,[es:si] ;remember offset in vector
29335 00001673 8907 [bx],ax ; to original owner in ds
29336 00001675 50 push ax
29337 00001676 887000 mov ax,DOSBIODATASEG ; 0070h
29338 00001679 8ED8 mov ds,ax ;set int19oldxx value in bios for
29339 0000167B 58 pop ax ;int 19 handler
29340 0000167C 8905 [di],ax
29341 0000167E 58 pop ax
29342 0000167F 894502 mov [di+2],ax
29343 00001682 1F pop ds
29344 %endif
29345 00001683 268914 mov [es:si],dx ;set vector to point to new int handler
29346 00001686 268C5C02 mov [es:si+2],ds
29347 0000168A C3 retn
29348
29349 ; End of STACK initialization routine
29350 ; -----
29351
29352 ; -----
29353 ;set the devmark for mem command.
29354 ;in: [memhi] - the address to place devmark
29355 ; [memlo] = 0
29356 ; al = id for devmark_id
29357 ;out: devmark established.
29358 ; the address saved in cs:[devmark_addr]
29359 ; [memhi] increase by 1.
29360 ; -----
29361
29362 ; 25/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
29363 ; (SYSINIT:1637h)
29364 ; 04/09/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS, SYSINIT)
29365 ; (SYSINIT:176Ch)
29366
29367 ; 04/09/2023 - PCDOS 7.1 - IBMBIO.COM (SYSINIT:1944h)

```

```

29368
29369
29370
29371
29372
29373
29374
29375 0000168B 2E8B0E[6403]
29376 00001690 2E890E[6719]
29377 00001695 8EC1
29378
29379
29380 00001697 26A20000
29381 0000169B 41
29382
29383 0000169C 26890E0100
29384
29385
29386
29387
29388
29389 000016A1 2EFF06[6403]
29390 000016A6 C3
29391
29392
29393
29394
29395
29396
29397
29398
29399
29400
29401
29402 000016A7 [AB16]
29403 000016A9 433A
29404 000016AB 5C44424C5350414345-
29404 000016B4 2E42494E00
29405 000016B9 433A5C535441434B45-
29405 000016C2 522E42494E00
29406
29407 000016C8 FFFF
29408 000016CA FFFF
29409 000016CC 0080
29410 000016CE 00000000
29411 000016D2 44424C5342494E24
29412
29413
29414
29415
29416
29417
29418
29419
29420
29421
29422
29423
29424
29425
29426
29427
29428 000016DA C606[6919]00
29429 000016DF E80E31
29430 000016E2 0E
29431 000016E3 07
29432
29433
29434 000016E4 C606[4F24]00
29435 000016E9 E8771E
29436
29437
29438
29439
29440
29441
29442
29443
29444 000016EC A1[3324]
29445
29446 000016EF 0306[3124]
29447 000016F3 725E
29448
29449 000016F5 3B06[3524]
29450 000016F9 7758
29451
29452
29453
29454
29455
29456
29457
29458 000016FB 8B16[A716]
29459 000016FF E8A21F
29460 00001702 724F
29461
29462
29463
29464
29465 00001704 C41E[3724]
29466 00001708 26817F122C2E
29467 0000170E 7543
29468
29469
29470
29471 00001710 C706[9003]1400
29472 00001716 8C06[9203]
29473
29474 0000171A 0E
29475 0000171B 07
29476 0000171C BB[6F03]
29477
29478
29479
29480
29481
29482
29483 0000171F C706[7D03]0000
29484 00001725 A1[3524]
29485 00001728 A3[7F03]
29486 0000172B A0[8503]
29487 0000172E A2[8503]
29488
29489 00001731 B80A00

setdevmark:
; 04/09/2023
;push es
;push cx
mov cx,[cs:memhi]
mov [cs:devmark_addr],cx
mov es,cx
; 25/10/2022
;mov [es:devmark.id],al
mov [es:0],al
inc cx
;mov [es:devmark.seg],cx
mov [es:1],cx
; 04/09/2023
;pop cx
;pop es
inc word [cs:memhi]
retn

; -----
; SYSPRE.ASM - MSDOS 6.0 - 1992
; -----
; ; pre-load and final placement of dblspace.bin
;
; 08/04/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMBIO.COM)
; =====
; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1964h)
; -----
; MagicDDNamePtr: dw MagicDDName ; "\DBLSPACE.BIN"
; db 'C:'
; MagicDDName: db '\DBLSPACE.BIN',0
; StackerName: db 'C:\STACKER.BIN',0
;
; tiny_stub_start:
; dw 0FFFFh ; phony device driver link
; dw 0FFFFh ; dw -1, -1
; dw 8000h ; mark as character device for MEM display
; dw 2 dup(0) ; strategy and interrupt
; db 'DBLSBIN$' ; magic default load
; tiny_stub_end: ; (tiny_stub_end-tiny_stub_start = 18)
;
; ===== S U B R O U T I N E =====
;
; 08/04/2024 - Retro DOS v5.0
; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1997h)
;
; ***MagicPreload - pre-load dblspace.bin
;
; EXIT ax = error code, 00 means none.
; ZF = true if ax == 0
;
; MagicPreload:
; ; 13/04/2024 - Retro DOS v5.0
; ; ds = cs
; ;mov byte [cs:setdevmarkflag],0 ; not for devmark
; mov byte [setdevmarkflag],0
; call round
; push cs
; pop es
; ;mov byte [cs:DeviceHi],0 ; not to be loaded in UMB
; ; 13/04/2024
; mov byte [DeviceHi],0
; call InitDevLoad ; set up sub-arena, DevLoadAddr,
; ; DevLoadEnd, and DevEntry
; ; gets arena name from bpb_addr
;
; ; 13/04/2024
; ; ds = cs
;
; ; check to make sure device driver fits our available space.
;
; ;mov ax,[cs:DevLoadAddr]
; mov ax,[DevLoadAddr]
; ;add ax,[cs:DevSize] ; calculate seg after DD load
; add ax,[DevSize]
; jc short pre_exit_err ; choke if overflows address space
; ;cmp ax,[cs:DevLoadEnd] ; does it overflow available space?
; cmp ax,[DevLoadEnd]
; ja short pre_exit_err
;
; _LoadDev: ; we're golden if not
; ; 13/04/2024
; ; ds = cs
; ;push cs
; ;pop ds
; ;mov dx,[cs:MagicDDNamePtr]
; mov dx,[MagicDDNamePtr]
; call ExecDev ; load device driver using exec call
; jb short pre_exit_err
;
; ; 13/04/2024
; ; ds = cs
; ;les bx,[cs:DevEntry] ; point to the Magic DD header
; les bx,[DevEntry]
; cmp word [es:bx+12h],2E2Ch; is it our stamp? ; ','
; jnz short pre_exit_err
; ;mov word [cs:MagicBackdoor],14h ; save the backdoor entry.
; ; ; (initial IP -EXE header offset 20-)
; ;mov [cs:MagicBackdoor+2],es
; mov word [MagicBackdoor],14h
; mov [MagicBackdoor+2],es
;
; push cs
; pop es
; mov bx,packet
;
; ;mov word [cs:break_addr],0
; ;mov ax,[cs:DevLoadEnd]
; ;mov [cs:break_addr+2],ax
; ;mov al,[cs:drivenumber] ; pass drive number to DBLSPACE as if
; ;mov [cs:devdrivenum],al ; it is a normal block device driver
; mov word [break_addr],0
; mov ax,[DevLoadEnd]
; mov [break_addr+2],ax
; mov al,[drivenumber] ; pass drive number to DBLSPACE as if
; mov [devdrivenum],al ; it is a normal block device driver
;
; mov ax,10 ; DS_INTERNAL_REVISION

```



```

29490                                     ; tell it what revision we expect
29491                                     ; call far [cs:MagicBackdoor]; first time call is init entry point
29492 00001734 FF1E[9003] call far [MagicBackdoor]
29493                                     ; with a standard device driver
29494                                     ; init packet at es:bx
29495 00001738 731D jnb short no_driver_version_fail ; skip if not a version failure
29496 0000173A B80600 mov ax,6 ; DS_INTERNAL_REVISION_6 ; (Stacker ?)
29497                                     ; tell it what revision we expect
29498                                     ; call far [cs:MagicBackdoor]
29499 0000173D FF1E[9003] call far [MagicBackdoor]
29500 00001741 7314 jnb short no_driver_version_fail
29501
29502 ; In this case, we're going to display a message
29503
29504 ;push cs
29505 ;pop ds
29506 ; 13/04/2024
29507 ; ds = cs
29508 00001743 BA[D853] mov dx,badblspace ; "Required system component is not instal"...
29509 00001746 E80933 call print ; display the message
29510
29511 ; point backdoor call back to safe far return
29512
29513 fail_driver_load:
29514 ;mov [cs:MagicBackdoor+2],cs
29515 ;mov word [cs:MagicBackdoor],NullBackdoor
29516 00001749 8C0E[9203] mov [MagicBackdoor+2],cs
29517 0000174D C706[9003][9403] mov word [MagicBackdoor],NullBackdoor
29518
29519 pre_exit_err:
29520 mov ax,40h ; SYSPRE_BADFILE_ERROR
29521 ; (problem loading dblspace.bin)
29522 retn
29523
29524 no_driver_version_fail:
29525 or ax,ax ; error code returned?
29526 jnz short fail_driver_load
29527
29528 magic_is_resident:
29529 ; 13/04/2024
29530 ; ds = cs
29531 0000175B A1[7D03] mov ax,[cs:break_addr]
29532 0000175E E8B4FB call ParaRound ; convert to paragraphs
29533 ;add ax,[cs:break_addr+2]
29534 ;mov [cs:DevBrkAddr+2],ax
29535 ;mov word [cs:DevBrkAddr],0; store normalized end here
29536 00001761 0306[7F03] add ax,[break_addr+2]
29537 00001765 A3[3D24] mov [DevBrkAddr+2],ax
29538 00001768 C706[3B24]0000 mov word [DevBrkAddr],0
29539 0000176E B80400 mov bx,4 ; inquire how many paragraphs it wants
29540
29541 00001771 FF1E[9003] ;call far [cs:MagicBackdoor]
29542 call far [MagicBackdoor]
29543 00001775 8B1E[A502] ;mov bx,[cs:ALLOCLIM] ; get top of free memory
29544 00001779 29C3 mov bx,[ALLOCLIM]
29545 sub bx,ax ; see how much we'll lower it
29546 0000177B 3B1E[3D24] ;cmp bx,[cs:DevBrkAddr+2] ; is there that much room free?
29547 0000177F 7212 cmp bx,[DevBrkAddr+2]
29548 jnb short cant_move_driver
29549 ;sub [cs:ALLOCLIM],ax ; (mov [cs:ALLOCLIM],bx)
29550 ;mov [cs:ALLOCLIM],bx ; Retro DOS v5.0 ; 08/04/2024
29551 ; 13/04/2024
29552 mov [ALLOCLIM],bx
29553 ;mov es,[cs:ALLOCLIM]
29554 00001781 891E[A502] mov es,[ALLOCLIM]
29555 00001785 8E06[A502] mov es,[ALLOCLIM]
29556 00001789 B80600 mov bx,6 ; tell the driver to move itself
29557 ;call far [cs:MagicBackdoor]
29558 0000178C FF1E[9003] call far [MagicBackdoor]
29559 ;mov [cs:DevBrkAddr+2],ax ; save end of low stub
29560 mov [DevBrkAddr+2],ax ; save end of low stub
29561
29562 cant_move_driver:
29563 ;mov ax,[cs:DevBrkAddr+2] ; get terminate segment
29564 00001793 A1[3D24] mov ax,[DevBrkAddr+2]
29565 ;cmp ax,[cs:DevLoadEnd] ; terminate size TOO big?
29566 00001796 3B06[3524] cmp ax,[DevLoadEnd]
29567 0000179A 77B7 ja short pre_exit_err ; error out if so
29568
29569 ;----- deal with block device drivers
29570
29571 _isblock: ; if no units found,erase the device
29572 ; 13/04/2024
29573 ; ds = cs
29574 0000179C A0[7C03] mov al,[cs:unitcount]
29575 0000179F 08C0 mov al,[unitcount]
29576 000017A1 74B0 or al,al
29577 000017A3 30E4 jz short pre_exit_err
29578 000017A5 C536[3724] xor ah,ah
29579 000017A9 88440A ;lds si,[cs:DevEntry] ; set ds:si to header
29580 ;lds si,[DevEntry]
29581 mov [si+10],al ; mov [si+SYSDEV.NAME],al
29582 ; number of units in name field
29583 ; device drivers are *supposed*
29584 ; to do this for themselves.
29585 000017AC 89C1 mov cx,ax
29586 000017AE 2EC43E[6D02] les di,[cs:DOSINFO] ; es:di point to dos info
29587 000017B3 268A6520 mov ah,[es:di+20h] ; [es:di+SYSI_NUMIO]
29588 ; get number of devices
29589 000017B7 88E2 mov dl,ah
29590 000017B9 00C4 add ah,al
29591 000017BB 80FC1A cmp ah,26 ; check for too many devices
29592 000017BE 7793 ja short pre_exit_err ; 'A' - 'Z' is 26 devices
29593 000017C0 2E800E[6919]02 or byte [cs:setdevmarkflag],2
29594 000017C6 E83B1F call DevSetBreak
29595 ;jnc short _ok_block
29596 ;jmp pre_exit_err
29597 ; 13/04/2024
29598 000017C9 7288 jc short pre_exit_err ; ds <> cs
29599
29600 _ok_block:
29601 000017CB 26886520 mov [es:di+20h],ah ; [es:di+SYSI_NUMIO] ; update the amount
29602 000017CF 2EC51E[8103] lds bx,[cs:bpb_addr] ; point to bpb array (*)
29603 000017D4 30F6 xor dh,dh
29604
29605 _perunit:
29606 000017D6 2EC42E[6D02] les bp,[cs:DOSINFO]
29607 000017DB 26C46E00 les bp,[es:bp+0] ; [es:bp.sysi_dpb]
29608 ; get first dpb
29609 ; [es:bp+SysInitvars.SYSI_DPB] ; [es:bp+0]
29610
29611 _scandpb:
29612 000017DF 26837E19FF cmp word [es:bp+19h],0FFFFh ; -1 ; [es:bp.dpb_next_dpb]
29613 000017E4 7406 jz short _foundpb
29614 000017E6 26C46E19 les bp,[es:bp+19h] ; les bp,[es:bp.dpb_next_dpb]
29615 ; [es:bp+DPB.NEXT_DPB]

```

```

29614 000017EA EBF3      jmp     short _scandpb
29615
29616      ; we've found the end of the DPB chain. Now extend it.
29617
29618      _foundpb:
29619      mov     ax,[cs:DevBrkAddr]
29620      mov     [es:bp+19h],ax      ; [es:bp.dpb_next_dpb] ; DPB.NEXT_DPB
29621      mov     ax,[cs:DevBrkAddr+2]
29622      mov     [es:bp+18h],ax      ; [es:bp.dpb_next_dpb+2] ; DPB.NEXT_DPB+2
29623      les     bp,[cs:DevBrkAddr]
29624      mov     word [es:bp+19h],0FFFFh ; -1
29625      mov     byte [es:bp+18h],0FFh ; [es:bp.dpb_first_access],-1
29626      ; DPB.FIRST_ACCESS
29627      add     word [cs:DevBrkAddr],61 ; DPBSIZ ; 3Dh
29628      call    RoundBreakAddr
29629      mov     si,[bx]
29630      ; ds:si points to bpb (*)
29631      ; (mov si,[bx] ..and then.. add bx,2)
29632      ; Note: If unit count > 1,bx points to a BPB in the BPB array,
29633      ; the array address is in [bpb_addr] (*)
29634      ; Erdogan Tan - 07/07/2023
29635      mov     [es:bp+0],dl      ; mov word [es:bp.dpb_drive],dx
29636      ; [es:bp+DPB.DRIVE],dl
29637      mov     [es:bp+1],dh      ; [es:bp+DPB.UNIT],dh
29638      push    dx
29639      push    cx
29640      mov     dx,4152h          ; DX = signature 4152h ('AR') for FAT32 extended BPB/DPB
29641      xor     cx,cx             ; 0
29642      mov     [es:bp+1Dh],cx      ; DPB.NEXT_FREE ; last allocated cluster #
29643      cmp     [si+0Bh],cx      ; BPB.fatsecs16 ; [si+A_BPB.BPB_SECTORSPERFAT]
29644      jnz     short _setdpb      ; FAT DPB (33 bytes)
29645      mov     [es:bp+39h],cx      ; FAT32 DPB (61 bytes)
29646      mov     [es:bp+38h],cx      ; DPB.FAT32_NXTFREE = 0
29647      dec     cx               ; DPB.FAT32_NXTFREE+2 = 0
29648      mov     [es:bp+1Fh],cx      ; 0FFFFh ; -1
29649      mov     [es:bp+21h],cx      ; DPB.FREE_CNT (-1 = unknown)
29650      mov     cx,4558h          ; DPB.FREE_CNT+2 (-1 = unknown)
29651      ; CX = signature 4558h ('EX') for FAT32 extended BPB/DPB
29652
29653      _setdpb:
29654      mov     ah,53h            ; SETDPB ; hidden system call
29655      int     21h               ; DOS - 2+ internal - TRANSLATE BIOS PARAMETER BLOCK
29656      ; DS:SI -> BPB (BIOS Parameter Block)
29657      ; ES:BP -> buffer for DOS Drive Parameter Block
29658      ; (if CX=4558h & DX=4152h,FAT32 Extended DPB will be set)
29659      pop     cx
29660      pop     dx
29661      mov     ax,[es:bp+2]      ; [es:bp.dpb_sector_size] ; [es:bp+DPB.SECTOR_SIZE]
29662      push    es
29663      mov     di,[cs:DOSINFO]
29664      cmp     ax,[es:di+10h]    ; [es:di.sysi_maxsec] ; [es:di+SysInitvars.SYSI_MAXSEC]
29665      pop     es
29666      jbe     short _iblk_1
29667      ; 13/04/2024
29668      ; ds <= cs
29669      mov     ax,40h           ; SYSPRE_BADFILE_ERROR ; (pre_exit_err)
29670      ; (problem loading dblspace.bin)
29671      retn
29672
29673      _iblk_1:
29674      push    ds
29675      lds     ax,[cs:DevEntry]
29676      mov     [es:bp+13h],ax      ; [es:bp+DPB.DRIVER_ADDR]
29677      mov     [es:bp+15h],ds      ; [es:bp+DPB.DRIVER_ADDR+2]
29678      pop     ds
29679      inc     dl                 ; increment drive number
29680      inc     dh                 ; increment unit number
29681      inc     bx
29682      inc     bx
29683      ; point to next BPB
29684      ; (in the BPB array) (*) -add bx,2-
29685      ; loop _foundpb
29686      dec     cx
29687      jz     short _linkit
29688      jmp     _foundpb
29689
29690      _linkit:
29691      push    cs
29692      pop     ds
29693      call    TempCDS           ; set cds for new drives
29694      ; 13/04/2024
29695      ; (DS may not be same with CS here)
29696      les     di,[cs:DOSINFO]      ; es:di = dos table (SysInitvars)
29697      mov     ax,[es:di+22h]      ; [es:di+SYSI_DEV] ; dx:cx = head of list
29698      mov     bx,[es:di+24h]      ; [es:di+SYSI_DEV+2]
29699      lds     si,[cs:DevEntry]
29700      mov     [si],ax             ; ds:si = device location
29701      mov     [si+2],bx          ; link in the driver
29702      mov     [es:di+22h],si      ; [es:di+SYSI_DEV] ; set head of list in dos
29703      mov     [es:di+24h],ds      ; [es:di+SYSI_DEV+2]
29704      call    DevBreak           ; mark successful install
29705      ; 13/04/2024
29706      ; ds = cs
29707      mov     cx,[cs:DevBrkAddr+2] ; pass it a work buffer
29708      mov     dx,[cs:ALLOCLIM]     ; address in cx (segment)
29709      mov     cx,[DevBrkAddr+2]
29710      mov     dx,[ALLOCLIM]
29711      sub     dx,cx               ; for len dx (paragraphs)
29712      mov     ax,5500h            ; we're shuffle aware,but don't move
29713      ; any drives at this point.
29714      mov     bx,2                ; switch what we can now
29715      ; call far [cs:MagicBackdoor]
29716      call    far [MagicBackdoor]
29717      pre_exit:
29718      xor     ax,ax               ; no errors!
29719      ; zf=1
29720      no_magic:
29721      ; 13/04/2024
29722      retn
29723
29724      ; ===== S U B R O U T I N E =====
29725
29726      ; 08/04/2024 - Retro DOS v5.0
29727      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1B9Fh)
29728
29729      ;***MagicPostload -- called to clean up and make sure Magic is final placed
29730
29731      MagicPostload:
29732      ; 13/04/3024
29733      ; ds = cs
29734      call    get_dblspace_version ; is it there?
29735      jnz     short no_magic        ; done if not
29736      test    dx,8000h             ; is it already permanent?
29737      jz     short no_magic        ; no,done if so (not in final position)
29738      mov     bx,0FFFFh ; -1      ; how much space does it want?
29739      mov     ax,4A11h             ; multMagicdrv
29740      ; DBLSPACE.BIN - GET RELOCATION SIZE
29741      int     2Fh                  ; get paragraphs into ax
29742      inc     ax                   ; extra 2 paragraphs for the stub

```

```

29738 000018CB 40          inc     ax                ; ((tiny_stub_end-tiny_stub_start)+15)/16
29739                                ; (18+15)/16 = 2
29740          ;mov     [cs:DevSize],ax      ; store that (**)
29741          ;mov     byte [cs:DeviceHi],0 ; not to be loaded in UMB
29742          ;mov     [cs:bpb_addr+2],cs    ; pass name so that
29743          ;                                ; arena header can be set
29744          ;mov     word [cs:bpb_addr],MagicDDName ; "\DBLSPACE.BIN"
29745          ; 13/04/2024
29746          ; ds = cs
29747 000018CC A3[3124]      mov     [DevSize],ax
29748 000018CF C606[4F24]00 mov     byte [DeviceHi],0
29749 000018D4 8C0E[8303]   mov     [bpb_addr+2],cs
29750 000018D8 C706[8103][AB16] mov     word [bpb_addr],MagicDDName
29751
29752 000018DE E80F2F        call    round                ; normalize memhi:memlo
29753 000018E1 E87F1C        call    InitDevLoad          ; set up sub-arena,DevLoadAddr,
29754                                ; DevLoadEnd,and DevEntry
29755                                ; gets arena name from bpb_addr
29756          ; 13/04/2024
29757          ; ds = cs
29758          ;mov     es,[cs:DevLoadAddr]    ; (**) (InitDevload sets this)
29759 000018E4 8E06[3324]     mov     es,[DevLoadAddr]
29760
29761          ; First, move a little header in place so that this looks
29762          ; to the mem command like a legitimate driver load. Otherwise,
29763          ; it will display garbage for the device name
29764
29765 000018E8 31FF          xor     di,di                ; move a little header in place
29766                                ; so that this looks to the mem command
29767                                ; like a legitimate driver load
29768 000018EA BE[C816]      mov     si,tiny_stub_start
29769          ;mov     cx,18                ; (tiny_stub_end-tiny_stub_start)
29770          mov     cx,tiny_stub_end-tiny_stub_start
29771 000018F0 F3A4          rep movsb                ; move it!
29772 000018F2 8CC0          mov     ax,es                ; advance es appropriately
29773 000018F4 40           inc     ax                ; add ax,((tiny_stub_end-tiny_stub_start)+15)/16
29774 000018F5 40           inc     ax
29775 000018F6 8EC0          mov     es,ax
29776 000018F8 BBFEFF        mov     bx,0FFFEh ; -2                ; final placement!
29777 000018FB B8114A        mov     ax,4A11h                ; multMagicdrv
29778 000018FE CD2F          int     2Fh                    ; DBLSPACE.BIN - RELOCATE
29779                                ; es = segment to which to relocate DBLSPACE.BIN
29780          ;mov     ax,[cs:DevLoadAddr] ; (**)
29781          ;add     ax,[cs:DevSize]        ; calculate seg after DD load
29782          ;mov     [cs:DevBrkAddr+2],ax  ; save as ending address!
29783          ;mov     word [cs:DevBrkAddr],0
29784          ; 13/04/2024
29785          ; ds = cs
29786 00001900 A1[3324]      mov     ax,[DevLoadAddr]
29787 00001903 0306[3124]   add     ax,[DevSize]
29788 00001907 A3[3D24]      mov     [DevBrkAddr+2],ax
29789 0000190A C706[3B24]0000 mov     word [DevBrkAddr],0
29790
29791 00001910 E8F11D        call    DevSetBreak          ; go ahead and alloc mem for device
29792          ;call    DevBreak
29793          ;no_magic:
29794          ;retn
29795          ; 13/04/2024 - Retro DOS v5.0
29796 00001913 E9101E        jmp     DevBreak
29797
29798          ; ===== S U B R O U T I N E =====
29799
29800          ; 08/04/2024 - Retro DOS v5.0
29801          ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1C08h)
29802
29803          ;***MagicSetCdss -- disable CDSS for still unmounted DbLspace drives
29804          ;
29805          ; entry:
29806          ; CDSS are now persistent and in their final place
29807
29808          MagicSetCdss:
29809          ; 13/04/2024 - Retro DOS v5.0
29810          ; ds = cs
29811 00001916 E83D00        call    get_dbLspace_version ; is it there?
29812 00001919 753A          jnz     short magic_set_exit ; done if not
29813          ; c1 = first DbLspace drive in ASCII
29814          ; ch = number of DbLspace drive letters
29815          ; point to DOS data area (SysInitVars)
29816 0000191B 2EC536[6D02]   lds     si,[cs:DOSINFO]
29817 00001920 C57416        lds     si,[si+16h]
29818 00001923 B458          mov     ah,88                ; curdirLen
29819 00001925 80E941        sub     cl,'A'                ; make it zero based.
29820 00001928 88C8          mov     al,cl                ; get first DbLspace drive letter
29821 0000192A F6E4          mul     ah                    ; find first DbLspace CDS
29822 0000192C 01C6          add     si,ax                ; cds pointer
29823 0000192E 88CA          mov     dl,cl                ; save for drive testing loop
29824 00001930 88E9          mov     cl,ch                ; get DbLspace drive count into cx
29825 00001932 30ED          xor     ch,ch
29826
29827          ; We know cx > 0, or else the driver wouldn't have stayed resident
29828
29829 00001934 51           push    cx
29830 00001935 52           push    dx
29831 00001936 1E           push    ds
29832 00001937 56           push    si
29833 00001938 B8114A        mov     ax,4A11h                ; multMagicdrv
29834 0000193B BB0100        mov     bx,1                    ; MD_DRIVE_MAP ; inquire drive map
29835 0000193E CD2F          int     2Fh                    ; DBLSPACE.BIN - "GetDriveMapping"
29836                                ; see if this is an unused DbLspace drive
29837 00001940 5E           pop     si
29838 00001941 1F           pop     ds
29839 00001942 5A           pop     dx
29840 00001943 59           pop     cx
29841 00001944 38DA        cmp     dl,b1                ; if mapped to itself,it is vacant
29842 00001946 7504          jnz     short magic_set_cdss_2 ; skip if used
29843 00001948 806444BF        and     byte [si+44h],0BFh    ; Retro DOS v5.0 ; 08/04/2024
29844          ;and     word [si+43h],0BFFFh
29845          ; reset the bit in flags (curdir_inuse bit)
29846          ; [si+curdir_list.cdir_flags],~curdir_inuse ; word
29847          ;(.. [si+1+curdir_list.cdir_flags],0BFh ; byte)
29848
29849 0000194C 83C658        add     si,88                ; curdirLen
29850 0000194F FEC2          inc     dl                    ; next drive
29851 00001951 E2E1          loop   magic_set_cdss_1
29852          ; 13/04/2024
29853 00001953 0E           push    cs
29854 00001954 1F           pop     ds
29855          magic_set_exit:
29856 00001955 C3           retn
29857
29858          ; ===== S U B R O U T I N E =====
29859
29860          ; 08/04/2024 - Retro DOS v5.0
29861          ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1C47h)

```

```

29862
29863
29864 00001956 B8114A
29865
29866
29867 00001959 31DB
29868 0000195B CD2F
29869
29870
29871
29872
29873
29874
29875
29876 0000195D 09C0
29877 0000195F C3
29878
29879
29880
29881
29882
29883
29884
29885
29886
29887
29888
29889
29890
29891
29892
29893
29894
29895
29896
29897
29898
29899
29900
29901
29902
29903
29904
29905
29906
29907
29908
29909
29910
29911
29912
29913
29914
29915
29916
29917
29918
29919
29920
29921
29922
29923
29924
29925
29926
29927 00001960 0000
29928
29929 00001962 0000
29930
29931
29932 00001964 00
29933
29934 00001965 00
29935
29936
29937
29938 00001966 00
29939
29940 00001967 0000
29941
29942 00001969 00
29943
29944
29945
29946 0000196A 00
29947
29948
29949
29950
29951
29952
29953
29954
29955
29956 0000196B 0000
29957 0000196D 0000
29958
29959
29960
29961
29962
29963
29964
29965
29966
29967
29968
29969
29970
29971
29972
29973
29974
29975
29976
29977
29978
29979
29980
29981
29982
29983
29984
29985

get_dblspace_version:
    mov     ax,4A11h
    xor     bx,bx
    int     2Fh
    or      ax,ax
    retn

; -----
; SYSCONF.ASM - MSDOS 6.0 - 1991
; -----
; 27/03/2019 - Retro DOS v4.0

;MULTI_CONFIG      equ 1
HIGH_FIRST equ 080h          ; from ARENA.INC - modifier for
                             ; allocation strategy call

;have_install_cmd equ 00000001b ; config.sys has install= commands
;has_installed     equ 00000010b ; sysinit_base installed.

default_filenum equ 8

;stacksw      equ true          ; include switchable hardware stacks

; external variable defined in ibmbio module for multi-track

;multrk_on equ 10000000b        ;user spcified mutitrack=on,or system turns
                                ; it on after handling config.sys file as a
                                ; default value,if multrk_flag = multrk_off1.
;multrk_off1 equ 00000000b      ;initial value. no "multitrack=" command entered.
;multrk_off2 equ 00000001b      ;user specified multitrack=off.

; if stacksw

; internal stack parameters

;entrysize equ 8

;mincount equ 8
;defaultcount equ 9
;maxcount equ 64

;minsize equ 32
;defaultsize equ 128
;maxsize equ 512

DOS_FLAG_OFFSET equ 86h

;ifdef MULTI_CONFIG
;
; config_envlen must immediately precede config_wrkseg, because they
; may be loaded as a dword ptr
;
; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; 25/10/2022
config_envlen: dw 0          ; when config_wrkseg is being used as
                             ; a scratch env, this is its length
config_wrkseg: dw 0          ; config work area (above confbot)
                             ; segment of work area

config_cmd: db 0             ; current config cmd
                             ; (with CONFIG_OPTION_QUERY bit intact)
config_multi: db 0           ; non-zero if multi-config config.sys

;endif ; MULTI_CONFIG

multdeviceflag: db 0

devmark_addr: dw 0           ;segment address for devmark.

setdevmarkflag: db 0         ;flag used for devmark

; 30/12/2022
; 12/12/2022
driver_units: db 0           ;total unitcount for driver

; 12/12/2022
;ems_stub_installed:
;    db 0

; 12/12/2022
;align 2

badparm_ptr: ; label dword
badparm_off: dw 0
badparm_seg: dw 0

;*****
;take care of config.sys file.
;system parser data and code.
;*****

;*****
; parser options set for msbio sysconf module
;*****
;
;**** default assemble swiches definition ****

;farsw equ 0 ; near call expected
;datesw equ 0 ; check date format
;timesw equ 0 ; check time format
;filesw equ 1 ; check file specification
;capsw equ 0 ; perform caps if specified
;cmpxsw equ 0 ; check complex list
;numsw equ 1 ; check numeric value
;keysw equ 0 ; support keywords
;swsw equ 1 ; support switches
;vallsw equ 1 ; support value definition 1
;val2sw equ 0 ; support value definition 2
;val3sw equ 1 ; support value definition 3
;drvsw equ 1 ; support drive only format
;qussw equ 0 ; support quoted string format

; psdata_seg equ cs

```

```

29986
29987
29988
29989
29990
29991
29992
29993
29994
29995
29996
29997
29998
29999
30000
30001
30002
30003
30004
30005
30006
30007
30008
30009
30010
30011
30012
30013
30014
30015
30016
30017
30018
30019
30020
30021
30022
30023
30024
30025
30026
30027
30028
30029
30030
30031
30032
30033
30034
30035
30036
30037
30038
30039
30040
30041
30042
30043
30044
30045
30046
30047
30048
30049
30050
30051
30052
30053
30054
30055
30056
30057
30058
30059
30060
30061
30062
30063
30064
30065
30066
30067
30068
30069
30070
30071
30072
30073
30074
30075
30076
30077
30078
30079
30080
30081
30082
30083
30084
30085
30086
30087
30088
30089
30090
30091
30092
30093
30094
30095
30096
30097
30098
30099
30100
30101
30102
30103
30104
30105
30106
30107
30108
30109

; .xlist
; include parse.asm ; together with psdata.inc
; .list

; PSDATA.INC - MSDOS 6.0 - 1991
; =====
; 27/03/2019 - Retro DOS v4.0
;
; 30/03/2019
; VERSION.INC (MSDOS 6.0)
; Set DBCS Blank constant
;
; ifndef DBCS
DB_SPACE EQU 2020h
DB_SP_HI EQU 20h
DB_SP_LO EQU 20h
; else

; *****
; Parser include file
; *****

; *** Equation field
; ----- Character code definition

_P_DBS_P1 equ DB_SP_HI ; AN000; 1st byte of DBCS blank
_P_DBS_P2 equ DB_SP_LO ; AN000; 2nd byte of DBCS blank
_P_Period equ "." ; AN020;
_P_Slash equ "/" ; AN020;
_P_Space equ " " ; AN000; SBCS blank
_P_Comma equ "," ; AN000;
_P_Switch equ "/" ; AN000;
_P_Keyword equ "=" ; AN000;
_P_Colon equ ":" ; AN000;
_P_Plus equ "+" ; AN000;
_P_Minus equ "-" ; AN000;
_P_Rparen equ ")" ; AN000;
_P_Lparen equ "(" ; AN000;
; _P_SQuote equ '"' ; AN025; deleted
_P_DQuote equ '"' ; AN000;
_P_NULL equ 0 ; AN000;
_P_TAB equ 9 ; AN000;
_P_CR equ 0Dh ; AN000;
_P_LF equ 0Ah ; AN000;
_P_ASCII80 equ 80h ; AN000; ASCII 80h character code

; ----- Masks
_P_Make_Lower equ 20h ; AN000; make lower case character
_P_Make_Upper equ 0FFh-_P_Make_Lower ; AN000; make upper case character

; ----- DOS function call related equs

_P_DOS_Get_CDI equ 3800h ; AN000; get country dependent information
; by this call, following information

struc _P_CDI
.DateF: resw 1
.Money: resb 5
.1000: resb 2
.Dec: resb 2
.DateS: resb 2
.TimeS: resb 2
resb 1
resb 1
.TimeF: resb 1
resw 2
resb 2
resw 5
.size:
endstruc

_P_Date_MDY equ 0 ; AN000;
_P_Date_DMY equ 1 ; AN000;
_P_Date_YMD equ 2 ; AN000;
; -----
_P_DOS_Get_EV equ 6300h ; AN000; get DBCS EV call
; AN000; DS:SI will points to DBCS EV
; -----
_P_DOS_Get_TBL equ 65h ; AN000; get uppercase table call
; AN000; following parameters are set
; AN000; to get casemap table.
_P_DOSTBL_Def equ -1 ; AN000; get default
_P_DOSTBL_BL equ 5 ; AN000; buffer length for Tbl pointer
_P_DOSTBL_File equ 4 ; AN000; get file uppercase table
_P_DOSTBL_Char equ 2 ; AN000; get character uppercase table
; By this call following information
; is returned.

struc _P_DOS_TBL
.InfoID: resb 1 ; AN000; information id for the table
.Off: resw 1 ; AN000; offset address of the table
.Seg: resw 1 ; AN000; segment address of the table
endstruc

; -----
; PARMS LABEL BYTE
; DW PARMSX
; DB 2 ; NUMBER OF STRINGS (0, 1, 2)
; DB length ; LENGTH OF THE NEXT LIST, 0 IF NONE
; DB " ." ; EXTRA DELIMITER LIST,
; ; TYPICAL ARE ";", "=",
; ; " " & WHITESPACE ALWAYS
; DB length ; LENGTH OF THE NEXT LIST, 0 IF NONE
; DB " ." ; EXTRA END OF LINE LIST, CR, LF OR 0 ALWAYS
; -----

; ----- PARMS block structure
struc _P_PARMS_Blk
.PARMSX_Address: resw 1 ; AN000; Address of PARMSX
.Num_Extra: resb 1 ; AN000; Number of extra stuff
.Len_Extra_Delim: resb 1 ; AN000; Length of extra delimiter
endstruc

_P_Len_PARMS equ 4 ; AN000;
_P_I_Use_Default equ 0 ; AN000; no extra stuff specified
_P_I_Have_Delim equ 1 ; AN000; extra delimiter specified
_P_I_Have_EOL equ 2 ; AN000; extra EOL specified

; -----
; PARMSX LABEL BYTE
; DB minp,maxp ; MIN, MAX POSITIONAL OPERANDS ALLOWED
; DW CONTROL ; DESCRIPTION OF POSITIONAL 1
; : ; REPEATS maxp-1 TIMES
; DB maxs ; # OF SWITCHES
; DW CONTROL ; DESCRIPTION OF SWITCH 1

```

```

30110 ;
30111 ;           :           ; REPEATS maxs-1 TIMES
30112 ;           DB      maxk ; # OF KEYWORD
30113 ;           DW      CONTROL ; DESCRIPTION OF KEYWORD 1
30114 ;           :           ; REPEATS maxk-1 TIMES
30115 ;-----
30116 ;----- PARMsX block structure
30117 struc _$P_PARMsX_Blk ;AN000;
30118 .MinP: resb 1 ;AN000; Minimum positional number
30119 .MaxP: resb 1 ;AN000; Maximum positional number
30120 .1st_Control: resw 1 ;AN000; Address of the 1st CONTROL block
30121 endstruc
30122
30123 ;-----
30124 ; << Control field definition >>
30125 ;
30126 ;
30127 ; CONTROL LABEL BYTE
30128 ; DW MATCH_FLAGS ; CONTROLS TYPE MATCHED
30129 ; ; 8000H=NUMERIC VALUE, (VALUE LIST WILL BE CHECKED)
30130 ; ; 4000H=SIGNED NUMERIC VALUE (VALUE LIST WILL BE CHECKED)
30131 ; ; 2000H=SIMPLE STRING(VALUE LIST WILL BE CHECKED)
30132 ; ; 1000H=DATE STRING (VALUE LIST WON'T BE CHECKED)
30133 ; ; 0800H=TIME STRING (VALUE LIST WON'T BE CHECKED)
30134 ; ; 0400H=COMPLEX LIST (VALUE LIST WON'T BE CHECKED)
30135 ; ; 0200H=FILE SPEC (VALUE LIST WON'T BE CHECKED)
30136 ; ; 0100H=DRIVE ONLY (VALUE LIST WON'T BE CHECKED)
30137 ; ; 0080H=QUOTED STRING (VALUE LIST WON'T BE CHECKED)
30138 ; ; 0010H=IGNORE ":" AT END IN MATCH
30139 ; ; 0002H=REPEATS ALLOWED
30140 ; ; 0001H=OPTIONAL
30141 ;
30142 ; DW FUNCTION_FLAGS ; 0001H=CAP RESULT BY FILE TABLE
30143 ; ; 0002H=CAP RESULT BY CHAR TABLE
30144 ; ; 0010H=REMOVE ":" AT END
30145 ; ; 0020H=colon is not necessary for switch
30146 ; (tm10) ; RESULT BUFFER
30147 ; DW VALUES ; VALUE LISTS
30148 ; DB nid ; NUMBER OF KEYWORD/SWITCH SYNONYMS IN FOLLOWING LIST
30149 ; DB "...",0 ; IF n >0, KEYWORD 1
30150 ;
30151 ;
30152 ; Note:
30153 ; - The MATCH_FLAG is bit significant. You can set, for example, TIME bit and
30154 ; DATE bit simalteniously.
30155 ;
30156 ; The parser examines each bit along with the following priority.
30157 ;
30158 ; COMPLEX -> DATE -> TIME -> NUMERIC VAL -> SIGNED NUMERIC VAL -> DRIVE ->
30159 ; FILE SPEC -> SIMPLE STRING.
30160 ;
30161 ; - When the FUNCTION_FLAG is 0001 or 0002, the STRING pointed to by a pointer
30162 ; in the result buffer is capitalized.
30163 ;
30164 ; - Match_Flags 0001H and 0002H have meaning only for the positional.
30165 ;
30166 ; - The "...",0 (bottom most line) does require '=' or '/'. When you need a
30167 ; switch, for example, '/A', then STRING points to;
30168 ;
30169 ; DB 1 ; number of following synonyms
30170 ; DB '/A',0
30171 ;
30172 ; when you need a keyword, for example, 'CODEPAGE=', then "...",0 will be;
30173 ;
30174 ; DB 1 ; number of following synonyms
30175 ; DB 'CODEPAGE=',0
30176 ;
30177 ; - "...", must consist of upper case characters only because the parser
30178 ; performs pattern matching after converting input to upper case (by
30179 ; using the current country upper case table)
30180 ;
30181 ; - One "...", can contain only one switch or keyword. If you need, for
30182 ; example /A and /B, the format will be;
30183 ;
30184 ; DB 2 ; number of following synonyms
30185 ; DB '/A',0
30186 ; DB '/B',0
30187 ;-----
30188 ;
30189 ;**** Match_Flags
30190 ;
30191 ;_P_Num_Val equ 8000h ;AN000; Numeric value
30192 ;_P_SNum_Val equ 4000h ;AN000; Signed numeric value
30193 ;_P_Simple_S equ 2000h ;AN000; Simple string
30194 ;_P_Date_S equ 1000h ;AN000; Date string
30195 ;_P_Time_S equ 0800h ;AN000; Time string
30196 ;_P_Cmpx_S equ 0400h ;AN000; Complex string
30197 ;_P_File_Spc equ 0200h ;AN000; File Spec
30198 ;_P_Drv_Only equ 0100h ;AN000; Drive Only
30199 ;_P_Qu_String equ 0080h ;AN000; Quoted string
30200 ;_P_Ig_Colon equ 0010h ;AN000; Ignore colon at end in match
30201 ;_P_Repeat equ 0002h ;AN000; Repeat allowed
30202 ;_P_Optional equ 0001h ;AN000; Optional
30203 ;
30204 ;**** Function flags
30205 ;
30206 ;_P_CAP_File equ 0001h ;AN000; CAP result by file table
30207 ;_P_CAP_Char equ 0002h ;AN000; CAP result by character table
30208 ;_P_Rm_Colon equ 0010h ;AN000; Remove ":" at the end
30209 ;_P_colon_is_not_necessary equ 0020h ;AN000;(tm10) /+10 and /+:10
30210 ;
30211 ;----- Control block structure
30212 ;
30213 struc _$P_Control_Blk
30214 .Match_Flag: resw 1 ;AN000; Controls type matched
30215 .Function_Flag: resw 1 ;AN000; Function should be taken
30216 .Result_Buf: resw 1 ; Result buffer address
30217 .Value_List: resw 1 ;AN000; value list address
30218 .nid: resb 1 ;AN000; # of keyword/sw synonyms
30219 .KEYorSW: resb 1 ;AN000; keyword or sw
30220 endstruc
30221 ;-----
30222 ; << Value List Definition >>
30223 ;
30224 ;
30225 ;VALUES LABEL BYTE
30226 ; DB nval ; NUMBER OF VALUE DEFINITIONS (0 - 3)
30227 ; +-
30228 ; DB nrng ; NUMBER OF RANGES
30229 ; +DB ITEM_TAG ; RETURN VALUE IF RANGE MATCHED
30230 ; +DD X,Y ; RANGE OF VALUES
30231 ; :
30232 ; DB nnval ; NUMBER OF CHOICES
30233 ; +DB ITEM_TAG ; RETURN VALUE IF NUMBER CHOICE MATCHED
30234 ; +DD VALUE ; SPECIFIC CHOICE IF NUMBER

```



```

30234 ; | :
30235 ; | DB nstrval ; NUMBER OF CHOICES
30236 ; | +DB ITEM_TAG ; RETURN VALUE IF STRING CHOICE MATCHED
30237 ; | +DW STRING ; SPECIFIC CHOICE IF STING
30238 ; +- :
30239 ;
30240 ; STRING DB "...",0 ; ASCII STRING IMAGE
30241 ;
30242 ; Note:
30243 ; - ITEM_TAG must not be 0FFH, which will be used in the result buffer
30244 ; when no choice lists are provided.
30245 ;
30246 ; - STRING must consist of upper case characters only because the parser
30247 ; performs pattern matching after converting input to upper case (by
30248 ; using the current country upper case table)
30249 ; -----
30250 ;
30251 ; _$P_nval_None equ 0 ; AN000; no value list ID
30252 ; _$P_nval_Range equ 1 ; AN000; range list ID
30253 ; _$P_nval_Value equ 2 ; AN000; value list ID
30254 ; _$P_nval_String equ 3 ; AN000; string list ID
30255 ; _$P_Len_Range equ 9 ; AN000; Length of a range choice(two DD plus one DB)
30256 ; _$P_Len_Value equ 5 ; AN000; Length of a value choice(one DD plus one DB)
30257 ; _$P_Len_String equ 3 ; AN000; Length of a string choice(one DW plus one DB)
30258 ; _$P_No_nrng equ 0 ; AN000; (tm07) no nrng. nval must not be 0.
30259 ;
30260 ;
30261 ; struc _$P_Val_List
30262 ; .NumofList: resb 1 ; AN000; number of following choice
30263 ; .Val_XL: resw 1 ; AN000; lower word of value
30264 ; .Val_XH: resw 1 ; AN000; higher word of value
30265 ; .Val_YL: resw 1 ; AN000; lower word of another value
30266 ; .Val_YH: resw 1 ; AN000; higher word of another value
30267 ; endstruc
30268 ;
30269 ; << Result Buffer Definition >>
30270 ;
30271 ; RESULT LABEL BYTE ; BELOW FILLED IN FOR DEFAULTS
30272 ; DB type ; TYPE RETURNED: 0=RESERVED,
30273 ; ; 1=NUMBER, 2=LIST INDEX,
30274 ; ; 3=STRING, 4=COMPLEX,
30275 ; ; 5=FILESPEC, 6=DRIVE
30276 ; ; 7=DATE, 8=TIME
30277 ; ; 9=QUOTED STRING
30278 ; DB ITEM_TAG ; MATCHED ITEM TAG
30279 ;
30280 ; dw synonym@ ; es:@ points to found SYNONYM if provided.
30281 ;
30282 ; +-
30283 ; | DD n ; VALUE IF NUMBER
30284 ; | or
30285 ; | DW i ; INDEX (OFFSET) INTO VALUE LIST
30286 ; | ; (ES presents Segment address)
30287 ; | or
30288 ; | DD STRING ; OFFSET OF STRING VALUE
30289 ; | or
30290 ; | DB drv ; DRIVE NUMBER (1-A, 2-B,..., 26-Z)
30291 ; | or
30292 ; | DW YEAR ;(1980-2099) IN CASE OF DATE
30293 ; | DB MONTH ;(1-12) Note: Range check is not performed.
30294 ; | DB DATE ;(1-31) 0 is filled when the corresponding field was not specified.
30295 ; | or
30296 ; | DB HOUR ;(0-23) IN CASE OF TIME
30297 ; | DB MINUTES ;(0-59) Note: Range check is not performed .
30298 ; | DB SECONDS ;(0-59) 0 is filled when the corresponding field was not specified .
30299 ; | DB HUNDREDTHS ;(0-99)
30300 ; +-
30301 ;
30302 ;
30303 ; Note: ITEM_TAG is 0FFH when the caller does not specify the choice
30304 ; list.
30305 ;
30306 ; YEAR: If the input value for the year is less than 100, parser
30307 ; adds 1900 to it. For example, when 87 is input to parser for
30308 ; the year value, he returns 1987.
30309 ; -----
30310 ;
30311 ; ----- Result block structure
30312 ;
30313 ; struc _$P_Result_Blk
30314 ; .Type: resb 1 ; AN000; Type returned
30315 ; .Item_Tag: resb 1 ; AN000; Matched item tag
30316 ; .SYNONYM_Ptr: resw 1 ; AN000; pointer to Synonym list returned
30317 ; .Picked_Val: resb 4 ; AN000; value
30318 ; endstruc
30319 ;
30320 ; -----
30321 ; **** values for the type field in the result block
30322 ;
30323 ; _$P_EOL equ 0 ; AN000; End of line
30324 ; _$P_Number equ 1 ; AN000; Number
30325 ; _$P_List_Idx equ 2 ; AN000; List Index
30326 ; _$P_String equ 3 ; AN000; String
30327 ; _$P_Complex equ 4 ; AN000; Complex
30328 ; _$P_File_Spec equ 5 ; AN000; File Spec
30329 ; _$P_Drive equ 6 ; AN000; Drive
30330 ; _$P_Date_F equ 7 ; AN000; Date
30331 ; _$P_Time_F equ 8 ; AN000; Time
30332 ; _$P_Quoted_String equ 9 ; AN000; Quoted String
30333 ; _$P_No_Tag equ 0FFh ; AN000; No ITEM_TAG found
30334 ;
30335 ; **** Return code
30336 ;
30337 ; following return code will be returned in the AX register.
30338 ;
30339 ; _$P_No_Error equ 0 ; AN000; No error
30340 ; _$P_Too_Many equ 1 ; AN000; Too many operands
30341 ; _$P_Op_Missing equ 2 ; AN000; Required operand missing
30342 ; _$P_Not_In_SW equ 3 ; AN000; Not in switch list provided
30343 ; _$P_Not_In_Key equ 4 ; AN000; Not in keyword list provided
30344 ; _$P_Out_Of_Range equ 6 ; AN000; Out of range specified
30345 ; _$P_Not_In_Val equ 7 ; AN000; Not in value list provided
30346 ; _$P_Not_In_Str equ 8 ; AN000; Not in string list provided
30347 ; _$P_Syntax equ 9 ; AN000; Syntax error
30348 ; _$P_RC_EOL equ -1 ; AN000; End of command line
30349 ;
30350 ; DATA - Retro DOS v4.0 - 27/03/2019
30351 ;
30352 ; MSDOS 6.2 IO.SYS SYSINIT:179Ch
30353 ;
30354 ; 14/04/2024 - Retro DOS v5.0
30355 ; PC DOS 7.1 IBMBIO.COM - SYSINIT:1C62h
30356 ;
30357 ; ***** Local Data *****

```

```

30358 0000196F 0000  _$P_ORDINAL:      dw  0          ;AN000; Operand ordinal save area
30359 00001971 0000  _$P_RC:           dw  0          ;AN000; Return code from parser
30360 00001973 0000  _$P_SI_Save:      dw  0          ;AN000; Pointer of command buffer
30361 00001975 0000  _$P_DX:           dw  0          ;AN000; Return result buffer address
30362 00001977 00    _$P_Terminator:    db  0          ;AN000; Terminator code (ASCII)
30363 00001978 0000  _$P_DBCSEV_OFF:   dw  0          ;AN000; Offset of DBCS EV
30364 0000197A 0000  _$P_DBCSEV_SEG:   dw  0          ;AN000; Segment of DBCS EV
30365 0000197C 0000  _$P_Flags:        dw  0          ;AN000; Parser internal flags
30366                                     %define _$P_Flags1 _$P_Flags      ;AN038; to reference first byte flags
30367                                     %define _$P_Flags2 _$P_Flags+1    ;AN038; to reference second byte flags only
30368
30369 ;in second byte of _$P_Flags, referenced as _$P_Flags2:
30370 _$P_equ           equ 01h        ;AN000; "=" packed in string buffet
30371 _$P_Neg          equ 02h        ;AN000; Negative value
30372 _$P_Time12       equ 04h        ;AN000; set when PM is specified
30373 _$P_Key_Cmp       equ 08h        ;AN000; set when keyword compare
30374 _$P_Sw_Cmp        equ 10h        ;AN000; set when switch compare
30375 _$P_Extra         equ 20h        ;AN000; set when extra delimiter found
30376 _$P_Sw            equ 40h        ;AN000; set when switch found (tm08)
30377 _$P_Signed        equ 80h        ;AN000; signed numeric specified
30378
30379 ;in first byte of _$P_Flags, referenced as _$P_Flags1:
30380 _$P_time12am      equ 01h        ;AN038; set when AM is specified on time
30381 _$P_TIME_AGAIN    equ 02h        ;AN039; SET WHEN READY TO RE-PARSE TIME
30382
30383 0000197E 0000  _$P_SaveSI_Cmpx:   dw  0          ;AN000; save si for later use by complex
30384 00001980 0000  _$P_KEYorSW_Ptr:   dw  0          ;AN000; points next to "=" or ":" code
30385 00001982 0000  _$P_Save_EOB:      dw  0          ;AN000; save pointer to EOB
30386 00001984 0000  _$P_Found_SYNONYM: dw  0          ;AN000; es:@ points to found synonym
30387
30388 00001986 00<rep 80h>  _$P_STRING_BUF:    times 128 db 0  ;AN000; Pick a operand from command line
30389  _$P_STRING_BUF_END equ  $          ;AN000;
30390
30391 ; 25/10/2022
30392 ; (MSDOS 5.0 IO.SYS, SYSINIT:16F8h)
30393
30394 00001A06 FF      _$P_Char_CAP_Ptr: db  0FFh      ;AN000; info id
30395 00001A07 0000                                     dw  0          ;AN000; offset of char case map table
30396 00001A09 0000                                     dw  0          ;AN000; segment of char case map table
30397
30398 ; 25/10/2022
30399 ; IF CAPSW
30400 _$P_File_CAP_Ptr: db  0FFh      ;AN000; info id
30401                                     dw  0          ;AN000; offset of file case map table
30402                                     dw  0          ;AN000; segment of file case map table
30403
30404 ; (tm06) IF FileSW          ;AN000;(Check if file spec is supported)
30405 ;
30406
30407 ;M029
30408 ;!!!WARNING!!!
30409 ; In routine SYSPARSE (parse.asm), _$P_FileSp_Char is reinitialized using
30410 ;hardcoded strings. If the chars in the string are changed here, corresponding
30411 ;changes need to be made in SYSPARSE
30412
30413 ;IF FileSW+DrvSW          ;AN000;(Check if file spec is supported)
30414
30415 ; 25/10/2022
30416 ; (MSDOS 5.0 IO.SYS, SYSINIT:16FDh)
30417
30418 00001A0B 5B5D7C3C3E2B3D3B22  _$P_FileSp_Char     db  '[]|<+=;'"' ;AN000; delimiter of file spec
30419  _$P_FileSp_Len      equ  $-_$P_FileSp_Char ;AN000;
30420
30421 ;ENDIF          ;AN000;(of FileSW)
30422
30423 ; delimiter parsing
30424 _$P_colon_period     equ  01h        ;AN032; check for colon & period
30425 _$P_period_only      equ  02h        ;AN032; check only for period
30426
30427 ;filespec error flag
30428 00001A14 00      _$P_err_flag:      db  0          ;AN033; flag set if filespec parsing error
30429                                     ;AN033; was detected.
30430  _$P_error_filespec  equ  01h        ;AN033; mask to set flag
30431
30432
30433 ; PARSE.ASM - MSDOS 6.0 - 1991
30434 ; =====
30435 ; 27/03/2019 - Retro DOS v4.0
30436
30437 ;*****
30438 ; SysParse;
30439 ;
30440 ; Function : Parser Entry
30441 ;
30442 ; Input: DS:SI -> command line
30443 ;        ES:DI -> parameter block
30444 ;        CS -> psdata.inc
30445 ;        CX = operand ordinal
30446 ;
30447 ; Note: ES is the segment containing all the control blocks defined
30448 ;       by the caller, except for the DOS COMMAND line parms, which
30449 ;       is in DS.
30450 ;
30451 ; Output: CY = 1 error of caller, means invalid parameter block or
30452 ;          invalid value list. But this parser does NOT implement
30453 ;          this feature. Therefore CY always zero.
30454 ;
30455 ;          CY = 0 AX = return code
30456 ;                  BL = terminated delimiter code
30457 ;                  CX = new operand ordinal
30458 ;                  SI = set past scanned operand
30459 ;                  DX = selected result buffer
30460 ;
30461 ; Use:      _$P_Skip_Delim, _$P_Chk_EOL, _$P_Chk_Delim, _$P_Chk_DBCS
30462 ;           _$P_Chk_Swtch, _$P_Chk_Pos_Control, _$P_Chk_Key_Control
30463 ;           _$P_Chk_Sw_Control, _$P_Fill_Result
30464 ;
30465 ; Vars:     _$P_Ordinal(RW), _$P_RC(RW), _$P_SI_Save(RW), _$P_DX(R), _$P_Terminator(R)
30466 ;           _$P_SaveSI_Cmpx(W), _$P_Flags(RW), _$P_Found_SYNONYM(R), _$P_Save_EOB(W)
30467 ;
30468 ;----- Modification History -----
30469 ;
30470 ; 4/04/87 : Created by K. K,
30471 ; 4/28/87 : _$P_Val_YH assemble error (tm01)
30472 ;          : JMP SHORT assemble error (tm02)
30473 ; 5/14/87 : Someone doesn't want to include psdata (tm03)
30474 ; 6/12/87 : _$P_Bridge is missing when Timesw equ 0 and (CmpxSw equ 1 or
30475 ;          : DateSw equ 1) (tm04)
30476 ; 6/12/87 : _$P_SorD_Quote is missing when QusSw equ 0 and CmpxSw equ 1
30477 ;          : (tm05) in PSDATA.INC
30478 ; 6/12/87 : _$P_FileSp_Char and _$P_FileSp_Len are missing
30479 ;          : when FileSW equ 0 and DrvSW equ 1 (tm06) in PSDATA.INC
30480 ; 6/18/87 : $VAL1 and $VAL3, $VAL2 and $VAL3 can be used in the same
30481 ;          : value-list block (tm07)

```



```

30482 ; 6/20/87 : Add _$P_SW to check if there's an omitting parameter after
30483 ; switch (keyword) or not. If there is, backup si for next call
30484 ; (tm08)
30485 ; 6/24/87 : Complex Item checking does not work correctly when CmpSw equ 1
30486 ; and DateSw equ 0 and TimeSw equ 0 (tm09)
30487 ; 6/24/87 : New function flag _$P_colon_is_not_necessary for switch
30488 ; /+15 and /+:15 are allowed for user (tm10)
30489 ; 6/29/87 : ECS call changes DS register but it causes the address problem
30490 ; in user's routines. _$P_Chk_DBCS (tm11)
30491 ; 7/10/87 : Switch with no_match flag (0x0000H) does not work correctly
30492 ; (tm12)
30493 ; 7/10/87 : Invalid switch/keyword does not work correctly
30494 ; (tm13)
30495 ; 7/10/87 : Drive_only breaks 3 bytes after the result buffer
30496 ; (tm14)
30497 ; 7/12/87 : Too_Many_Operands sets DX=0 as the PARSE result
30498 ; (tm15)
30499 ; 7/24/87 : Negative lower bound on numeric ranges cause trouble
30500 ;
30501 ; 7/24/87 : Quoted strings being returned with quotes.
30502 ;
30503 ; 7/28/87 : Kerry S (;AN018;)
30504 ; Non optional value on switch (match flags<>0 and <>1) not flagged
30505 ; as an error when missing. Solution: return error 2. Modules
30506 ; affected: _$P_Chk_SW_Control.
30507 ;
30508 ; 7/29/87 : Kerry S (;AN019;)
30509 ; Now allow the optional bit in match flags for switches. This
30510 ; allows the switch to be encountered with a value or without a
30511 ; value and no error is returned.
30512 ;
30513 ;
30514 ; 8/28/87 : Ed K, Kerry S (;AN020;)
30515 ; 9/14/87 : In PROC _$P_Get_DecNum, when checking for field separators
30516 ; within a date response, instead of checking just for the one
30517 ; character defined by the COUNTRY DEPENDENT INFO, check for
30518 ; all three chars, "-", "/", and ".". Change _$P_Chk_Switch to allow
30519 ; slashes in date strings when DateSw (assembler switch) is set.
30520 ;
30521 ; 9/1/87 : Kerry S (;AN021;)
30522 ; In PROC _$P_String_Comp, when comparing the switch or keyword on
30523 ; the command line with the string in the control block the
30524 ; comparing was stopping at a colon (switch) or equal (keyword)
30525 ; on the command line and assuming a match. This allowed a shorter
30526 ; string on the command line than in the synonym list in the control
30527 ; block. I put in a test for a null in the control block so the
30528 ; string in the control block must be the same length as the string
30529 ; preceeding the colon or equal on the command line.
30530 ;
30531 ; 8/28/87 : Kerry S (;AN022;)
30532 ; All references to data in PSDATA.INC had CS overrides. This caused
30533 ; problems for people who included it themselves in a segment other
30534 ; than CS. Added switch to allow including PSDATA.INC in any
30535 ; segment.
30536 ;
30537 ; 9/16/87 : Ed K (;AN023;) PTM1040
30538 ; in _$P_set_cdi PROC, it assumes CS points to psdata. Change Push CS
30539 ; into PUSH cs. In _$P_Get_DecNum PROC, fix AN020
30540 ; forced both TIME and DATE to use the delims, "-", "/", ".".
30541 ; Created FLag, in _$P_time_Format PROC, to request the delim in
30542 ; BL be used if TIME is being parsed.
30543 ;
30544 ; 9/24/87 : Ed K
30545 ; Removed the include to STRUC.INC. Replaced the STRUC macro
30546 ; invocations with their normally expanded code; made comments
30547 ; out of the STRUC macro invocation statements to maintain readability.
30548 ;
30549 ; 9/24/87 : Ed K (;AN024;) PTM1222
30550 ; When no CONTROL for a keyword found, tried to fill in RESULT
30551 ; pointed to by non-existent CONTROL.
30552 ;
30553 ; 10/15/87 : Ed K (;AN025;) PTM1672
30554 ; A quoted text string can be framed only by double quote. Remove
30555 ; support to frame quoted text string with single quote.
30556 ; (apostrophe) _$P_Sord_Quote is removed from PSDATA.INC.
30557 ; _$P_SQuote EQU also removed from PSDATA.INC. Any references to
30558 ; single quote in PROC prologues are left as is for history reasons.
30559 ;
30560 ; This fixes another bug, not mentioned in p1672, in that two
30561 ; quote chars within a quoted string is supposed to be reported as
30562 ; one quote character, but is reported as two quotes. This changed
30563 ; two instructions in PROC _$P_Quoted_Str.
30564 ;
30565 ; Also fixed are several JMP that caused a NOP, these changed to
30566 ; have the SHORT operator to avoid the unneeded NOP.
30567 ;
30568 ; The code and PSDATA.INC have been aligned for ease of reading.
30569 ;
30570 ; 10/26/87 : Ed K (;AN026;) PTM2041, DATE within SWITCH, BX reference to
30571 ; psdata buffer should have cs.
30572 ;
30573 ; 10/27/87 : Ed K (;AN027;) PTM2042 comma between keywords implies
30574 ; positional missing.
30575 ;
30576 ; 11/06/87 : Ed K (;AN028;) PTM 2315 Parser should not use line feed
30577 ; as a line delimiter, should use carriage return.
30578 ; Define switch: LFEOLSW, if on, accept LF as end of line char.
30579 ;
30580 ; 11/11/87 : Ed K (;AN029;) PTM 1651 GET RID OF WHITESPACE AROUND "=".
30581 ;
30582 ; 11/18/87 : Ed K (;AN030;) PTM 2551 If filename is just "", then
30583 ; endless loop since SI is returned still pointing to start
30584 ; of that parm.
30585 ;
30586 ; 11/19/87 : Ed K (;AN031;) PTM 2585 date & time getting bad values.
30587 ; Vector to returned string has CS instead of cs, but
30588 ; when tried to fix it on previous version, changed similar
30589 ; but wrong place.
30590 ;
30591 ; 12/09/87 : Bill L (;AN032;) PTM 2772 colon and period are now valid
30592 ; delimiters between hours, minutes, seconds for time. And period
30593 ; and comma are valid delimiters between seconds and 100th second.
30594 ;
30595 ; 12/14/87 : Bill L (;AN033;) PTM 2722 if illegal delimiter characters
30596 ; in a filespec, then flag an error.
30597 ;
30598 ; 12/22/87 : Bill L (;AN034;) All local data to parser is now
30599 ; indexed off of the cs equate instead of the DS register.
30600 ; Using this method, DS can point to the segment of PSP or to psdata
30601 ; --> local parser data. Why were some references to local data changed
30602 ; to do this before, but not all ?????
30603 ;
30604 ; 02/02/88 : Ed K (;AC035;) INSPECT utility, suggests optimizations.
30605 ;

```

```

30606 ; 02/05/88 : Ed K (;AN036;) P3372-UPPERCASE TRANSLATION, cs HOSED.
30607 ;
30608 ; 02/08/88 : Ed K (;AN037;) P3410-AVOID POP OF CS, CHECK BASESW FIRST.
30609 ;
30610 ; 02/19/88 : Ed K (;AN038;) p3524 above noon and "am" should be error
30611 ;
30612 ; 02/23/88 : Ed K (;AN039;) p3518 accept "comma" and "period" as decimal
30613 ; separator in TIME before hundredths field.
30614 ;
30615 ; 08/09/90 : SA M005 Prevented parser from recognizing '=' signs within
30616 ; strings as keywords.
30617 ;
30618 ;*****
30619 ;
30620 ;IF FarSw ;AN000;(Check if need far return)
30621 ;SysParse proc far ;AN000;
30622 ;ELSE ;AN000;
30623 ;SysParse proc near ;AN000;
30624 ;ENDIF ;AN000;(of FarSw)
30625 ;
30626 ; 27/03/2019 - Retro DOS v4.0
30627 ; (MSDOS 6.21 IO.SYS - SYSINIT:1842h)
30628 ;
30629 ; 25/10/2022 - Retro DOS v4.0
30630 ; (MSDOS 5.0 IO.SYS - SYSINIT:1707h)
30631 ;
30632 ; 06/09/2023 - Retro DOS v4.2 IO.SYS Optimization (& Retro DOS v5.0 pre-work)
30633 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1D08h)
30634 ;
30635 SysParse:
30636 ; 06/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
30637 ; dx = 0
30638 00001A15 1E push ds ; !*
30639 00001A16 0E push cs
30640 00001A17 1F pop ds
30641 ;
30642 ;mov word [cs:$_P_Flags],0 ;AC034; Clear all internal flags
30643 ;cld ;AN000; confirm forward direction
30644 ;mov word [cs:$_P_ORDINAL],cx ;AC034; save operand ordinal
30645 ;mov word [cs:$_P_RC],$_P_No_Error ;AC034; Assume no error
30646 ;mov word [cs:$_P_Found_SYNONYM],0 ;AC034; initialize synonym pointer
30647 ;
30648 ;mov word [cs:$_P_DX],0 ;AC034; (tm15)
30649 ;
30650 ; 06/09/2023
30651 00001A18 8916[7C19] mov [_P_Flags],dx ; 0 ;AC034; Clear all internal flags
30652 00001A1C FC cld ;AN000; confirm forward direction
30653 00001A1D 890E[6F19] mov [_P_ORDINAL],cx ;AC034; save operand ordinal
30654 00001A21 8916[7119] mov [_P_RC],dx ; $P_No_Error ;AC034; Assume no error
30655 00001A25 8916[8419] mov [_P_Found_SYNONYM],dx ; 0 ;AC034; initialize synonym pointer
30656 00001A29 8916[7519] mov [_P_DX],dx ; 0 ;AC034; (tm15)
30657 ;
30658 ;M029 -- Begin changes
30659 ; The table of special chars $_P_FileSp_Char should be initialized on every
30660 ;entry to SysParse. This is in the non-checksum region and any program that
30661 ;corrupts this table but does not corrupt the checksum region will leave
30662 ;command.com parsing in an inconsistent state.
30663 ; NB: The special characters string has been hardcoded here. If any change
30664 ;is made to it in psdata.inc, a corresponding change needs to be made here.
30665 ;
30666 ;IF FileSw + DrvSw
30667 ; 14/04/2024 (NASM syntax BugFix) .. '[' (MASM) -> '[' (NASM)
30668 ;
30669 ;mov word [cs:$_P_FileSp_Char], '['
30670 ;mov word [cs:$_P_FileSp_Char+2], '|<'
30671 ;mov word [cs:$_P_FileSp_Char+4], '>+'
30672 ;mov word [cs:$_P_FileSp_Char+6], '='
30673 ;
30674 ; 14/04/2024
30675 ; 06/09/2023
30676 00001A2D C706[0B1A]5B5D mov word [_P_FileSp_Char], '[' ; mov word [_P_FileSp_Char],5D5Bh
30677 00001A33 C706[0D1A]7C3C mov word [_P_FileSp_Char+2], '|<' ; 3C7Ch
30678 00001A39 C706[0F1A]3E2B mov word [_P_FileSp_Char+4], '>+' ; 2B3Eh
30679 00001A3F C706[111A]3D3B mov word [_P_FileSp_Char+6], '=' ; 3B3Dh
30680 ;
30681 ;ENDIF
30682 00001A45 1F ; 06/09/2023
30683 pop ds ; !*
30684 ;
30685 ;M029 -- End of changes
30686 ;
30687 00001A46 E87806 call $_P_Skip_Delim ;AN000; Move si to 1st non white space
30688 00001A49 7313 jnc short $_P_Start ;AN000; If EOL is not encountered, do parse
30689 ;----- End of Line
30690 00001A4B B8FFFF mov ax,$_P_RC_EOL ;AN000; set exit code to -1
30691 00001A4E 53 push bx ;AN000;
30692 ;mov bx,[es:di+$_P_PARAMS_Blk.PARMSX_Address] ;AN000; Get the PARMSX address to
30693 00001A4F 268B1D mov bx,[es:di]
30694 00001A52 263A0F cmp cl,[es:bx+$_P_PARAMSX_Blk.MinP] ;AN000; check ORDINAL to see if the minimum
30695 00001A55 7303 jae short $_P_Fin ;AN000; positional found.
30696 ;
30697 00001A57 B80200 mov ax,$_P_Op_Missing ;AN000; If no, set exit code to missing operand
30698 00001A5A 5B ;AN000;
30699 00001A5B E90A01 pop bx ;AN000;
30700 jmp $_P_Single_Exit ;AN000; return to the caller
30701 ;-----
30702 00001A5E 2E8936[7E19] $_P_Start: ;AN000;
30703 00001A63 53 mov [cs:$_P_SaveSI_Cmpx],si ;AN000;AC034; save ptr to command line for later use by complex,
30704 00001A64 57 push bx ;AN000; quoted string or file spec.
30705 00001A65 55 push di ;AN000;
30706 ;lea bx,[cs:$_P_STRING_BUF] ;AC034; set buffer to copy from command string
30707 ; 02/11/2022
30708 ;lea bx,$_P_STRING_BUF
30709 ; 07/09/2023
30710 00001A66 BB[8619] mov bx,$_P_STRING_BUF
30711 00001A69 2EF606[7D19]20 test byte [cs:$_P_Flags2],$_P_Extra ;AC034; 3/9 extra delimiter encountered ?
30712 00001A6F 7543 jnz short $_P_Pack_End ;AN000; 3/9 if yes, no need to copy
30713 ;
30714 $_P_Pack_Loop: ;AN000;
30715 lodsb ;AN000; Pick a operand from buffer
30716 call $_P_Chk_Switch ;AN000; Check switch character
30717 jc short $_P_Pack_End_BY_EOL ;AN020; if carry set found delimiter type slash, need backup si, else
30718 ;
30719 00001A77 E86906 call $_P_Chk_EOL ;AN000; Check EOL character
30720 00001A7A 7437 je short $_P_Pack_End_BY_EOL ;AN000; need backup si
30721 ;
30722 00001A7C E89906 call $_P_Chk_Delim ;AN000; Check delimiter
30723 00001A7F 7518 jne short $_P_PL01 ;AN000; If no, process next byte
30724 ;
30725 00001A81 2EF606[7D19]20 test byte [cs:$_P_Flags2],$_P_Extra ;AC034; 3/9 If yes and white spec,

```

```

30729 ; (tm08)jne short _$P_Pack_End ;AN000; 3/9 then
30730 00001A87 7505 jnz short _$P_Pack_End_backup_si ;AN000; (tm08)
30731
30732 call _$P_Skip_Delim ;AN000; skip subsequent white space,too
30733 00001A8C EB26 jmp short _$P_Pack_End ;AN000; finish copy by placing NUL at end
30734
30735 _$P_Pack_End_backup_si: ;AN000; (tm08)
30736 00001A8E 2EF606[7D19]41 test byte [cs:$_P_Flags2],_$P_SW+_$P_equ ;AN000;AC034; (tm08)
30737 00001A94 741E jz short _$P_Pack_End ;AN000; (tm08)
30738
30739 00001A96 4E dec si ;AN000; (tm08)
30740 00001A97 EB1B jmp short _$P_Pack_End ;AN025; (tm08)
30741
30742 _$P_PL01: ;AN000;
30743 00001A99 2E8807 mov [cs:bx],al ;AN000; move byte to STRING_BUF
30744 00001A9C 3C3D cmp al,_$P_Keyword ;'= ' ;AN000; if it is equal character,
30745 00001A9E 7506 jne short _$P_PL00 ;AN000; then
30746
30747 00001AA0 2E800E[7D19]01 or byte [cs:$_P_Flags2],_$P_equ ;AC034; remember it in flag
30748 _$P_PL00: ;AN000;
30749 00001AA6 43 inc bx ;AN000; ready to see next byte
30750 00001AA7 E8D506 call _$P_Chk_DBCS ;AN000; was it 1st byte of DBCS ?
30751 00001AAA 73C5 jnc short _$P_Pack_Loop ;AN000; if no, process to next byte
30752
30753 00001AAC AC lodsb ;AN000; if yes, store
30754 00001AAD 2E8807 mov [cs:bx],al ;AN000; 2nd byte of DBCS
30755 00001AB0 43 inc bx ;AN000; update pointer
30756 00001AB1 EBBE jmp short _$P_Pack_Loop ;AN000; process to next byte
30757
30758 _$P_Pack_End_BY_EOL: ;AN000;
30759 00001AB3 4E dec si ;AN000; backup si pointer
30760 _$P_Pack_End: ;AN000;
30761 00001AB4 2E8936[7319] mov [cs:$_P_SI_Save],si ;AC034; save next pointer, SI
30762 ; 07/09/2023
30763 ;mov byte [cs:bx],_$P_NULL ;AN000; put NULL at the end
30764 00001AB9 30E4 xor ah,ah ; 0 ; *
30765 00001ABB 2E8827 mov [cs:bx],ah ; _$P_NULL ;AN000; put NULL at the end
30766 ;
30767 00001ABE 2E891E[8219] mov [cs:$_P_Save_EOB],bx ;AC034; 3/17/87 keep the address for later use of complex
30768 ;mov bx,[es:di+$_P_PARMSX_Blk.PARMSX_Address] ;AN000; get PARMSX address
30769 00001AC3 268B1D mov bx,[es:di]
30770 ;lea si,[cs:$_P_STRING_BUF] ;AC034;
30771 ; 02/11/2022
30772 ;lea si,[$_P_STRING_BUF]
30773 ; 07/09/2023
30774 00001AC6 BE[8619] mov si,_$P_STRING_BUF
30775 00001AC9 2E803C2F cmp byte [cs:si],_$P_Switch ;AN000; the operand begins w/ switch char ?
30776 00001ACD 7440 je short _$P_SW_Manager ;AN000; if yes, process as switch
30777
30778 00001ACF 2E803C22 cmp byte [cs:si],_$P_DQuote ;M005;is it a string?
30779 00001AD3 7408 je short _$P_Positional_Manager ;M005;if so, process as one!
30780
30781 00001AD5 2EF606[7D19]01 test byte [cs:$_P_Flags2],_$P_equ ;AC034; the operand includes equal char ?
30782 00001ADB 7552 jnz short _$P_Key_Manager ;AN000; if yes, process as keyword
30783
30784 _$P_Positional_Manager: ;AN000; else process as positional
30785 00001ADD 268A4701 mov al,[es:bx+$_P_PARMSX_Blk.MaxP] ;AN000; get maxp
30786 ; 07/09/2023
30787 ;xor ah,ah ;AN000; ax = maxp
30788 00001AE1 2E3906[6F19] cmp [cs:$_P_ORDINAL],ax ;AC034; too many positional ?
30789 00001AE6 7312 jae short _$P_Too_Many_Error ;AN000; if yes, set exit code to too many
30790
30791 00001AE8 2EA1[6F19] mov ax,[cs:$_P_ORDINAL] ;AC034; see what the current ordinal
30792 00001AEC D1E0 shl ax,1 ;AN000; ax = ax*2
30793 00001AEE 43 inc bx ;AC035; add '2' to
30794 00001AEF 43 inc bx ;AC035; BX reg
30795 ;AN000; now bx points to 1st CONTROL
30796 00001AF0 01C3 add bx,ax ;AN000; now bx points to specified CONTROL address
30797 00001AF2 268B1F mov bx,[es:bx] ;AN000; now bx points to specified CONTROL itself
30798 00001AF5 E87200 call _$P_Chk_Pos_Control ;AN000; Do process for positional
30799 00001AF8 EB53 jmp short _$P_Return_to_Caller ;AN000; and return to the caller
30800
30801 _$P_Too_Many_Error: ;AN000;
30802 00001AFA 2EC706[7119]0100 mov word [cs:$_P_RC],_$P_Too_Many ;AC034; set exit code
30803 00001B01 EB4A jmp short _$P_Return_to_Caller ;AN000; and return to the caller
30804
30805 ; 07/09/2023 - Retro DOSD v4.2 IO.SYS (Optimization)
30806 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1E06h)
30807 get_maxp:
30808 ;mov al,[es:bx+1]
30809 00001B03 268A4701 mov al,[es:bx+$_P_PARMSX_Blk.MaxP] ;AN000; get maxp
30810 ; 07/09/2023
30811 ; ah=0 ; *
30812 ;xor ah,ah ; 0 ;AN000; ax = maxp
30813 00001B07 30ED xor ch,ch ; **
30814 00001B09 40 inc ax ;AN000;
30815 00001B0A D1E0 shl ax,1 ;AN000; ax = (ax+1)*2
30816 00001B0C 01C3 add bx,ax ;AN000; now bx points to maxs
30817 00001B0E C3 retn
30818
30819 _$P_SW_Manager: ;AN000;
30820 ; 07/09/2023
30821 ;mov al,[es:bx+$_P_PARMSX_Blk.MaxP] ;AN000; get maxp
30822 ;xor ah,ah ;AN000; ax = maxp
30823 ;inc ax ;AN000;
30824 ;shl ax,1 ;AN000; ax = (ax+1)*2
30825 ;add bx,ax ;AN000; now bx points to maxs
30826 00001B0F E8F1FF call get_maxp ; 07/09/2023
30827
30828 00001B12 268A0F mov cl,[es:bx] ;AN000;
30829 ; 07/09/2023
30830 ;xor ch,ch ; ** (ch=0) ;AN000; cx = maxs
30831 ;or cx,cx ;AN000; at least one switch ?
30832 ;jz short _$P_SW_Not_Found ;AN000;
30833 ; 07/07/2023
30834 00001B15 E30F jcxz _$P_SW_Not_Found ; no
30835
30836 00001B17 43 inc bx ;AN000; now bx points to 1st CONTROL address
30837
30838 _$P_SW_Mgr_Loop: ;AN000;
30839 00001B18 53 push bx ;AN000;
30840 00001B19 268B1F mov bx,[es:bx] ;AN000; bx points to Switch CONTROL itself
30841 00001B1C E8A900 call _$P_Chk_SW_Control ;AN000; do process for switch
30842 00001B1F 5B pop bx ;AN000;
30843 00001B20 732B jnc short _$P_Return_to_Caller ;AN000; if the CONTROL is for the switch, exit
30844
30845 00001B22 43 inc bx ;AC035; add '2' to
30846 00001B23 43 inc bx ;AC035; BX reg
30847 ;AN000; else bx points to the next CONTROL
30848 00001B24 E2F2 loop _$P_SW_Mgr_Loop ;AN000; and loop
30849
30850 _$P_SW_Not_Found: ;AN000;
30851 00001B26 2EC706[7119]0300 mov word [cs:$_P_RC],_$P_Not_In_SW ;AC034; here no CONTROL for the switch has
30852 00001B2D EB1E jmp short _$P_Return_to_Caller ;AN000; not been found, means error.

```

```

30853
30854
30855
30856
30857
30858
30859
30860
30861 00001B2F E8D1FF
30862
30863 00001B32 268A07
30864 00001B35 30E4
30865 00001B37 D1E0
30866 00001B39 40
30867 00001B3A 01C3
30868 00001B3C 268A0F
30869
30870
30871
30872
30873
30874 00001B3F E305
30875
30876 00001B41 43
30877
30878
30879
30880
30881
30882
30883
30884
30885
30886
30887
30888
30889
30890 00001B42 43
30891 00001B43 43
30892
30893 00001B44 E2FC
30894
30895
30896 00001B46 2EC706[7119]0400
30897
30898 00001B4D 5D
30899 00001B4E 5F
30900 00001B4F 5B
30901 00001B50 2E8B0E[6F19]
30902 00001B55 2EA1[7119]
30903 00001B59 2E8B36[7319]
30904 00001B5E 2E8B16[7519]
30905 00001B63 2E8A1E[7719]
30906
30907 00001B68 F8
30908 00001B69 C3
30909
30910
30911
30912
30913
30914
30915
30916
30917
30918
30919
30920
30921
30922
30923
30924
30925
30926 00001B6A 50
30927
30928 00001B6B 268B07
30929
30930 00001B6E A802
30931
30932 00001B70 7505
30933
30934 00001B72 2EFF06[6F19]
30935
30936 00001B77 2E803C00
30937 00001B7B 7517
30938
30939
30940 00001B7D A801
30941
30942 00001B7F 7509
30943
30944 00001B81 2EC706[7119]0200
30945 00001B88 E80D
30946
30947
30948 00001B8A 50
30949
30950
30951
30952 00001B8B B803FF
30953 00001B8E E89600
30954 00001B91 58
30955 00001B92 EB03
30956
30957
30958 00001B94 E81101
30959
30960 00001B97 58
30961 00001B98 C3
30962
30963
30964
30965
30966
30967
30968
30969
30970
30971
30972
30973
30974
30975
30976

_P$Key_Manager:
; 07/09/2023
;mov al,[es:bx+_P_PARMSX_Blk.MaxP] ;AN000; get maxp
;xor ah,ah ;AN000; ax = maxp
;inc ax ;AN000;
;shl ax,1 ;AN000; ax = (ax+1)*2
;add bx,ax ;AN000; now bx points to maxs
call get_maxp ; 07/09/2023

mov al,[es:bx] ;AN000;
xor ah,ah ; 0 ;AN000; ax = maxs
shl ax,1 ;AN000;
inc ax ;AN000; ax = ax*2+1
add bx,ax ;AN000; now bx points to maxk
mov cl,[es:bx] ;AN000;
; 07/09/2023
;xor ch,ch ; ** (ch=0) ;AN000; cx = maxk
;or cx,cx ;AN000; at least one keyword ?
;jz short _P_Key_Not_Found ;AN000;
; 07/07/2023
jcxz _P_Key_Not_Found ; no

inc bx ;AN000; now bx points to 1st CONTROL

_P$Key_Mgr_Loop:
; 07/09/2023
; ('_P_Chk_Key_Control' contains only 'stc' instruction)
; (always returns with cf=1)
;push bx ;AN000;
;mov bx,[es:bx] ;AN000; bx points to keyword CONTROL itself
;call _P_Chk_Key_Control ;AN000; do process for keyword
;pop bx ;AN000;
;jnc short _P_Return_to_Caller ;AN000; if the CONTROL is for the keyword, exit
; 07/09/2023
; cf=1 (after 'call _P_Chk_Key_Control')

inc bx ;AC035; add '2' to
inc bx ;AC035; BX reg
;AN000; else bx points to the next CONTROL
loop _P_Key_Mgr_Loop ;AN000; and loop

_P$Key_Not_Found:
;AN000;
mov word [cs:_P_RC],_P_Not_In_Key ;AC034; here no CONTROL for the keyword has
_P_Return_to_Caller:
;AN000;
pop bp ;AN000;
pop di ;AN000;
pop bx ;AN000;
mov cx,[cs:_P_ORDINAL] ;AC034; return next ordinal
mov ax,[cs:_P_RC] ;AC034; return exit code
mov si,[cs:_P_SI_Save] ;AC034; return next operand pointer
mov dx,[cs:_P_DX] ;AC034; return result buffer address
mov bl,[cs:_P_Terminator] ;AC034; return delimiter code found
_P_Single_Exit:
;AN000;
clic ;AN000;
retn ;AN000;

;*****
;
; _P_Chk_Pos_Control
;
; Function: Parse CONTROL block for a positional
;
; Input: ES:BX -> CONTROL block
; cs:SI -> _P_STRING_BUF
;
; Output: None
;
; Use: _P_Fill_Result, _P_Check_Match_Flags
;
; Vars: _P_Ordinal(w), _P_RC(w)
;*****

_P$Chk_Pos_Control:
push ax ;AN000;
;mov ax,[es:bx+_P_Control_Blk.Match_Flag] ;AN000;
mov ax,[es:bx]
; 12/12/2022
test al,_P_Repeat
;test ax,_P_Repeat ;AN000; repeat allowed ?
;jnz short _P_CPC00 ;AN000; then do not increment ORDINAL

inc word [cs:_P_ORDINAL] ;AC034; update the ordinal
_P_CPC00:
;AN000;
cmp byte [cs:si],_P_NULL ;AN000; no data ?
jne short _P_CPC01 ;AN000;

; 12/12/2022
test al,_P_Optional
;test ax,_P_Optional ;AN000; yes, then is it optional ?
;jnz short _P_CPC02 ;AN000;

mov word [cs:_P_RC],_P_Op_Missing ;AC034; no, then error 3/17/87
jmp short _P_CPC_Exit ;AN000;

_P_CPC02:
;AN000;
push ax ;AN000;
;mov al,_P_String ;AN000; if it is optional return NULL
;mov ah,_P_No_Tag ;AN000; no item tag indication
; 07/07/2023
mov ax,(_P_No_Tag<<8)|_P_String
call _P_Fill_Result ;AN000;
pop ax ;AN000;
jmp short _P_CPC_Exit ;AN000;

_P_CPC01:
;AN000;
call _P_Check_Match_Flags ;AN000;
_P_CPC_Exit:
;AN000;
pop ax ;AN000;
retn ;AN000;

;*****
;
; _P_Chk_Key_Control
;
; Function: Parse CONTROL block for a keyword
;
; Input: ES:BX -> CONTROL block
; cs:SI -> _P_STRING_BUF
;
; Output: CY = 1 : not match
;
; Use: _P_Fill_Result, _P_Search_KEYorSW, _P_Check_Match_Flags
;
; Vars: _P_RC(w), _P_SaveSI_Cmpx(w), _P_KEYorSW_Ptr(R), _P_Flags(w)
;*****

```

```

30977
30978 ; 07/09/2023
30979 ;_SP_Chk_Key_Control:
30980 ; stc ;AN000; this logic works when the KeySW
30981 ; retn ;AN000; is reset.
30982
30983 ;*****
30984 ;_SP_Search_KEYorSW:
30985 ;
30986 ; Function: Search specified keyword or switch from CONTROL
30987 ;
30988 ; Input: ES:BX -> CONTROL block
30989 ; CS:SI -> _SP_STRING_BUF
30990 ;
30991 ; Output: CY = 1 : not match
30992 ;
30993 ; Use: _SP_String_Comp, _SP_MoveBP_NUL, _SP_Found_SYNONYM
30994 ;*****
30995
30996 ; 25/10/2022 - Retro DOS v4.0
30997 ; (MSDOS 5.0 IO.SYS - SYSINIT:18B6h)
30998
30999 ;_SP_Search_KEYorSW: ;AN000;
31000 push bp ;AN000;
31001 push cx ;AN000;
31002 mov cl,[es:bx+_SP_Control_Blk.nid] ;AN000; Get synonym count
31003 xor ch,ch ;AN000; and set it to cx
31004 ;or cx,cx ;AN000; No synonyms specified ?
31005 ;jz short _SP_KEYorSW_Not_Found ;AN000; then indicate not found by CY
31006 ; 07/07/2023
31007 jcxz _SP_KEYorSW_Not_Found
31008
31009 ;lea bp,[es:bx+_SP_Control_Blk.KEYorSW] ;AN000; BP points to the 1st synonym
31010 ; 25/10/2022
31011 lea bp,[bx+_SP_Control_Blk.KEYorSW]
31012 ;lea bp,[bx+9]
31013 ;_SP_KEYorSW_Loop: ;AN000;
31014 call _SP_String_Comp ;AN000; compare string in buffer w/ the synonym
31015 jnc short _SP_KEYorSW_Found ;AN000; If match, set it to synonym pointer
31016
31017 call _SP_MoveBP_NUL ;AN000; else, bp points to the next string
31018 loop _SP_KEYorSW_Loop ;AN000; loop nid times
31019 ;_SP_KEYorSW_Not_Found: ;AN000;
31020 stc ;AN000; indicate not found in synonym list
31021 jmp short _SP_KEYorSW_Exit ;AN000; and exit
31022
31023 ;_SP_KEYorSW_Found: ;AN000;
31024 mov [cs:_SP_Found_SYNONYM],bp ;AC034; set synonym pointer
31025 cld ;AN000; indicate found
31026 ;_SP_KEYorSW_Exit: ;AN000;
31027 pop cx ;AN000;
31028 pop bp ;AN000;
31029 retm ;AN000;
31030
31031 ;*****
31032 ;_SP_MoveBP_NUL
31033 ;*****
31034
31035 ;_SP_MoveBP_NUL:
31036 ;_SP_MBP_Loop: ;AN000;
31037 ; 11/12/2022
31038 cmp byte [es:bp],_SP_NULL ;AN000; Increment BP that points
31039 ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
31040 ; (SYSINIT:18DBh)
31041 ;cmp byte [es:bp+0],0
31042 je short _SP_MBP_Exit ;AN000; to the synonym list
31043
31044 inc bp ;AN000; until
31045 jmp short _SP_MBP_Loop ;AN000; NULL encountered.
31046
31047 ;_SP_MBP_Exit: ;AN000;
31048 inc bp ;AN000; bp points to next to NULL
31049 retm ;AN000;
31050
31051 ;*****
31052 ;_SP_Chk_SW_Control
31053 ;
31054 ; Function: Parse CONTROL block for a switch
31055 ;
31056 ; Input: ES:BX -> CONTROL block
31057 ; CS:SI -> _SP_STRING_BUF
31058 ;
31059 ; Output: CY = 1 : not match
31060 ;
31061 ; Use: _SP_Fill_Result, _SP_Search_KEYorSW, _SP_Check_Match_Flags
31062 ;
31063 ; Vars: _SP_SaveSI_Cmpx(W), _SP_KEYorSW_Ptr(R), _SP_Flags(W)
31064 ;*****
31065
31066 ;_SP_Chk_SW_Control:
31067
31068 ;IF SwSW ;AN000;(Check if switch is supported)
31069 ;or byte [cs:_SP_Flags+1],10h
31070 ;or byte [cs:_SP_Flags2],_SP_SW_Cmp ;AC034; Indicate switch for later string comparison
31071 call _SP_Search_KEYorSW ;AN000; Search the switch in the CONTROL block
31072 jc short _SP_Chk_SW_Err0 ;AN000; not found, then try next CONTROL
31073
31074 ;and [cs:_SP_Flags+],0EFh
31075 and byte [cs:_SP_Flags2],0FFh-_SP_SW_Cmp
31076 ;AC034; reset the indicator previously set
31077 push ax ;AN000; /switch:
31078 mov ax,[cs:_SP_KEYorSW_Ptr] ;AC034; ^
31079 sub ax,si ;AN000; SI KEYorSW
31080 add [cs:_SP_SaveSI_Cmpx],ax ;AC034; update for complex list
31081 pop ax ;AN000;
31082
31083 mov si,[cs:_SP_KEYorSW_Ptr] ;AC034; set si at the end or colon
31084 cmp byte [cs:si],_SP_NULL ;AN000; any data after colon
31085 jne short _SP_CS_W00 ;AN000; if yes, process match flags
31086
31087 cmp byte [cs:si-1],_SP_Colon ;AN000; if no, the switch terminated by colon ?
31088 jne short _SP_Chk_if_data_required ;AN000; if yes,
31089
31090 mov word [cs:_SP_RC],_SP_Syntax ;AC034; return syntax error
31091 jmp short _SP_Chk_SW_Exit ;AN000;
31092
31093 ;_SP_Chk_if_data_required: ;AN018; no data, no colon
31094 ;cmp word [es:bx+_SP_Control_Blk.Match_Flag],0
31095 cmp word [es:bx],0 ;AN018; should have data? zero match flag means switch followed by nothing is
31096 je short _SP_Chk_SW_Exit ;AN018; match flags not zero so should have something if optional bit is not
31097
31098 ;test word [es:bx+_SP_Control_Blk.Match_Flag],_SP_Optional

```

```

31099          ; 02/11/2022 (MSDOS 5.0 IO.SYS SYINIT compatibility)
31100          ; test word [es:bx],1
31101          ; 12/12/2022
31102          ; test word [es:bx],_$_Optional ;AN019; see if no value is valid
31103 00001C07 26F60701 test byte [es:bx],_$_Optional
31104 00001C0B 7510     jnz short _$_Chk_SW_Exit ;AN019; if so, then leave, else yell
31105
31106 00001C0D 2EC706[7119]0200 mov word [cs:_$_P_RC],_$_P_Op_Missing ;AC034; return required operand missing
31107 00001C14 EB07     jmp short _$_Chk_SW_Exit ;AN018;
31108
31109 _$_P_CSW00:          ;AN000;
31110          call _$_Check_Match_Flags ;AN000; process match flag
31111 00001C16 E88F00     cld ;AN000; indicate match
31112          ; jmp short _$_P_Chk_SW_Single_Exit ;AN000;
31113          ; 12/12/2022
31114 00001C1A C3       retn
31115
31116 _$_P_Chk_SW_Err0:    ;AN000;
31117 00001C1B F9       stc ;AN000; not found in switch synonym list
31118          ; jmp short _$_P_Chk_SW_Single_Exit ;AN000;
31119          ; 12/12/2022
31120 00001C1C C3       retn
31121
31122 _$_P_Chk_SW_Exit:    ;AN000;
31123 00001C1D 50       push ax ;AN000;
31124          ; mov al,_$_P_String ;AN000;
31125          ; mov ah,_$_P_No_Tag ;AN000;
31126          ; 07/07/2023
31127 00001C1E B803FF     mov ax,(_$_P_No_Tag<<8)|_$_P_String
31128 00001C21 E80300     call _$_P_Fill_Result ;AN000; set result buffer
31129 00001C24 58       pop ax ;AN000;
31130 00001C25 F8       cld ;AN000;
31131
31132 00001C26 C3       _$_P_Chk_SW_Single_Exit: ;AN000;
31133          ; retn ;AN000;
31134          ; ELSE ;AN000;(of IF SwSw)
31135          ; stc ;AN000; this logic works when the SwSw
31136          ; retn ;AN000; is reset.
31137
31138          ; *****
31139          ; _$_P_Fill_Result
31140          ; *****
31141          ; Function: Fill the result buffer
31142          ;
31143          ; Input: AH = Item tag
31144          ; AL = type
31145          ; AL = 1: CX,DX has 32bit number (CX = high)
31146          ; AL = 2: DX has index(offset) into value list
31147          ; AL = 6: DL has driver # (1-A, 2-B, ..., 26 - Z)
31148          ; AL = 7: DX has year, CL has month and CH has date
31149          ; AL = 8: DL has hours, DH has minutes, CL has seconds,
31150          ; and CH has hundredths
31151          ; AL = else: cs:SI points to returned string buffer
31152          ; ES:BX -> CONTROL block
31153          ;
31154          ; Output: None
31155          ;
31156          ; Use: _$_Do_CAPS_String, _$_Remove_Colon, _$_Found_SYNONYM
31157          ;
31158          ; Vars: _$_P_DX(W)
31159          ; *****
31160
31161 00001C27 57       _$_P_Fill_Result:
31162 00001C28 268B7F04     push di ;AN000;
31163          mov di,[es:bx+_$_Control_Blk.Result_Buf] ;AN000; di points to result buffer
31164 00001C2C 2E893E[7519] mov [cs:_$_P_DX],di ;AC034; set returned result address
31165          ; mov [es:di+_$_Result_Blk.Type],al ;AN000; store type
31166          ; mov [es:di+_$_Result_Blk.Item_Tag],ah ;AN000; store item tag
31167          ; 07/09/2023
31168          ; mov [es:di+_$_Result_Blk.Type], ax
31169 00001C31 268905     mov [es:di],ax ; store type (al) and item tag (ah)
31170
31171 00001C34 50       push ax ;AN000;
31172 00001C35 2EA1[8419]     mov ax,[cs:_$_Found_SYNONYM] ;AC034; if yes,
31173 00001C39 26894502     mov [es:di+_$_Result_Blk.SYNONYM_Ptr],ax ;AN000; then set it to the result
31174          ;
31175 00001C3D 58       pop ax ;AN000;
31176          _$_P_RLT04: ;AN000;
31177 00001C3E 3C01     cmp al,_$_P_Number ;AN000; if number
31178 00001C40 750A     jne short _$_P_RLT00 ;AN000;
31179
31180          _$_P_RLT02: ;AN000;
31181 00001C42 26895504     mov [es:di+_$_Result_Blk.Picked_Val],dx ;AN000; then store 32bit
31182 00001C46 26894D06     mov [es:di+_$_Result_Blk.Picked_Val+2],cx ;AN000; number
31183 00001C4A EB5A     jmp short _$_P_RLT_Exit ;AN000;
31184
31185          _$_P_RLT00: ;AN000;
31186 00001C4C 3C02     cmp al,_$_P_List_Idx ;AN000; if list index
31187 00001C4E 7506     jne short _$_P_RLT01 ;AN000;
31188
31189          mov [es:di+_$_Result_Blk.Picked_Val],dx ;AN000; then store list index
31190          jmp short _$_P_RLT_Exit ;AN000;
31191 00001C54 EB50
31192
31193          _$_P_RLT01: ;AN000;
31194 00001C56 3C07     cmp al,_$_P_Date_F ;AN000; Date format ?
31195 00001C58 74E8     je short _$_P_RLT02 ;AN000;
31196
31197          cmp al,_$_P_Time_F ;AN000; Time format ?
31198 00001C5C 74E4     je short _$_P_RLT02 ;AN000;
31199
31200          cmp al,_$_P_Drive ;AN000; drive format ?
31201 00001C60 7506     jne short _$_P_RLT03 ;AN000;
31202
31203          mov [es:di+_$_Result_Blk.Picked_Val],dl ;AN000; store drive number
31204 00001C66 EB3E     jmp short _$_P_RLT_Exit ;AN000;
31205
31206          _$_P_RLT03: ;AN000;
31207 00001C68 3C04     cmp al,_$_P_Complex ;AN000; complex format ?
31208 00001C6A 750F     jne short _$_P_RLT05 ;AN000;
31209
31210          mov ax,[cs:_$_P_SaveSI_Cmpx] ;AC034; then get pointer in command buffer
31211 00001C70 40       inc ax ;AN000; skip left Parentheses
31212 00001C71 26894504     mov [es:di+_$_Result_Blk.Picked_Val],ax ;AN000; store offset
31213 00001C75 268C5D06     mov [es:di+_$_Result_Blk.Picked_Val+2],ds ;AN000; store segment
31214 00001C79 EB2B     jmp short _$_P_RLT_Exit ;AN000;
31215
31216          _$_P_RLT05: ;AN000;
31217          ;----- AL = 3, 5, or 9
31218 00001C7B 26897504     mov [es:di+_$_Result_Blk.Picked_Val],si ;AN000; store offset of STRING_BUF
31219          ;
31220          mov [es:di+_$_Result_Blk.Picked_Val+2],cs ;AN031; store segment of STRING_BUF
31221          ;
31222 00001C83 50       push ax ;AN000;

```



```

31223 00001C84 26F6470201      test    byte [es:bx+$_P_Control_Blk.Function_Flag],$_P_CAP_File
31224                                ;AN000; need CAPS by file table?
31225 00001C89 7404          jz      short $_P_RLT_CAP00      ;AN000;
31226
31227 00001C8B B004          mov     al,$_P_DOSTBL_File      ;AN000; use file upper case table
31228 00001C8D EB09          jmp     short $_P_RLT_CAP02      ;AN000;
31229
31230
31231 00001C8F 26F6470202      _$P_RLT_CAP00:
31232                                test    byte [es:bx+$_P_Control_Blk.Function_Flag],$_P_CAP_Char
31233                                ;AN000; need CAPS by char table ?
31234                                jz      short $_P_RLT_CAP01      ;AN000;
31235
31236 00001C96 B002          mov     al,$_P_DOSTBL_Char      ;AN000; use character upper case table
31237 00001C98 E8DF00      _$P_RLT_CAP02:
31238                                call    $_P_Do_CAPS_String      ;AN000; process CAPS along the table
31239                                _$P_RLT_CAP01:
31240                                pop     ax                      ;AN000;
31241                                test    byte [es:bx+$_P_Control_Blk.Function_Flag],$_P_Rm_Colon
31242                                ;AN000; removing colon at end ?
31243                                jz      short $_P_RLT_Exit      ;AN000;
31244 00001CA1 7403          call    $_P_Remove_Colon      ;AN000; then process it.
31245 _$P_RLT_Exit:
31246 00001CA6 5F          pop     di                      ;AN000;
31247 00001CA7 C3          retn                     ;AN000;
31248
31249 *****
31250 ; _$P_Check_Match_Flags
31251 ;
31252 ; Function: Check the mutch_flags and make the exit code and set the
31253 ; result buffer
31254 ;
31255 ; Check for types in this order:
31256 ; Complex
31257 ; Date
31258 ; Time
31259 ; Drive
31260 ; Filespec
31261 ; Quoted String
31262 ; Simple String
31263 ;
31264 ; Input:  cs:SI -> $_P_STRING_BUF
31265 ;         ES:BX -> CONTROL block
31266 ;
31267 ; Output:  None
31268 ;
31269 ; Use:    _$P_Value, P$_SValue, $_P_Simple_String, $_P_Date_Format
31270 ;         _$P_Time_Format, _$P_Complex_Format, $_P_File_Foemat
31271 ;         _$P_Drive_Format
31272 ; *****
31273
31274 ; 25/10/2022 - Retro DOS v4.0
31275 ; (MSDOS 5.0 IO.SYS - SYSINIT:19CFh)
31276
31277 ; 14/04/2024 - Retro DOS v5.0
31278 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1FC3h)
31279
31280 ; 12/12/2022
31281 _$P_Check_Match_Flags:
31282 00001CA8 2EC606[141A]00      mov     byte [cs:$_P_err_flag],$_P_NULL
31283                                ;AN033;AC034;; clear filespec error flag.
31284 00001CAE 50          push    ax                      ;AN000;
31285                                ;mov     ax,[es:bx+$_P_Control_Blk.Match_Flag]
31286 00001CAF 268B07      mov     ax,[es:bx]              ;AN000; load match flag(16bit) to ax
31287 00001CB2 09C0          or      ax,ax                  ;AC035; test ax for zero
31288 00001CB4 7517          jnz     short $_P_Mat          ;AN000; (tm12)
31289 00001CB6 50          push    ax                      ;AN000; (tm12)
31290 00001CB7 53          push    bx                      ;AN000; (tm12)
31291 00001CB8 52          push    dx                      ;AN000; (tm12)
31292 00001CB9 57          push    di                      ;AN000; (tm12)
31293 00001CBA 2EC706[7119]0900      mov     word [cs:$_P_RC],$_P_Syntax ;AC034; (tm12)
31294                                ;mov     ah,$_P_No_Tag          ;AN000; (tm12)
31295                                ;mov     al,$_P_String          ;AN000; (tm12)
31296                                ; 07/07/2023
31297 00001CC1 B803FF      mov     ax,($_P_No_Tag<<8)|$_P_String
31298 00001CC4 E860FF      call    $_P_Fill_Result      ;AN000; (tm12)
31299 00001CC7 5F          pop     di                      ;AN000; (tm12)
31300 00001CC8 5A          pop     dx                      ;AN000; (tm12)
31301 00001CC9 5B          pop     bx                      ;AN000; (tm12)
31302 00001CCA 58          pop     ax                      ;AN000; (tm12)
31303                                ; 12/12/2022
31304                                ;jmp     short $_P_Bridge      ;AC035; (tm12)
31305                                ; 12/12/2022
31306 ;_$_P_Mat:
31307                                ;jmp     short $_P_Match03      ;AN000; (tm12)
31308                                ;AN025; (tm09)
31309 00001CCB EB6E          _$P_Bridge:
31310                                jmp     short $_P_Match_Exit ;AN000; (tm02)
31311                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
31312                                ; (SYSINIT:19F9h)
31313                                ; 12/12/2022
31314                                ;nop      ; db 90h
31315
31316 ; 12/12/2022
31317 _$P_Mat:
31318 _$P_Match03:
31319                                ;test    ax,$_P_Num_Val ; 8000h;AN000; Numeric value
31320                                ; 07/07/2023
31321 00001CCD F6C480      test    ah,($_P_Num_Val>>8) ; 80h
31322 00001CD0 7412          jz      short $_P_Match04      ;AN000;
31323
31324 00001CD2 2EC706[7119]0000      mov     word [cs:$_P_RC],$_P_No_Error ;AC034; assume no error
31325 00001CD9 E81E01      call    $_P_Value            ;AN000; do process
31326 00001CDC 2E833E[7119]09      cmp     word [cs:$_P_RC],$_P_Syntax ;AC034; if error, examine the next type
31327 00001CE2 7557          jne     short $_P_Match_Exit ;AN000;
31328 _$P_Match04:
31329                                ;test    ax,$_P_SNum_Val ; 4000h      ;AN000; Signed numeric value
31330                                ; 07/07/2023
31331 00001CE4 F6C440      test    ah,($_P_SNum_Val>>8) ; 40h
31332 00001CE7 7412          jz      short $_P_Match05      ;AN000;
31333
31334 00001CE9 2EC706[7119]0000      mov     word [cs:$_P_RC],$_P_No_Error ;AC034; assume no error
31335 00001CF0 E8E300      call    $_P_SValue           ;AN000; do process
31336 00001CF3 2E833E[7119]09      cmp     word [cs:$_P_RC],$_P_Syntax ;AC034; if error, examine the next type
31337 00001CF9 7540          jne     short $_P_Match_Exit ;AN000;
31338 _$P_Match05:
31339                                ;test    ax,$_P_Drv_Only ; 100h;AN000; Drive only
31340                                ; 07/07/2023
31341 00001CFB F6C401      test    ah,($_P_Drv_Only>>8) ; 1
31342 00001CFE 7415          jz      short $_P_Match06      ;AN000;
31343
31344 00001D00 2EC706[7119]0000      mov     word [cs:$_P_RC],$_P_No_Error ;AC034; assume no error
31345 00001D07 E8F202      call    $_P_File_Format      ;AN000; 1st, call file format
31346 00001D0A E87203      call    $_P_Drive_Format      ;AN000; check drive format, next

```

```

31347 00001D0D 2E833E[7119]09      cmp     word [cs:$_P_RC],$_P_Syntax ;AC034; if error, examine the next type
31348 00001D13 7526                jne     short $_P_Match_Exit ;AN000;
31349                                _P_Match06:
31350                                ;test     ax,$_P_File_Spc ; 200h;AN000; File spec
31351                                ; 07/07/2023
31352 00001D15 F6C402            test     ah,($_P_File_Spc>>8) ; 2
31353 00001D18 7412                jz      short $_P_Match07 ;AN000;
31354
31355 00001D1A 2EC706[7119]0000        mov     word [cs:$_P_RC],$_P_No_Error ;AC034; assume no error
31356 00001D21 E8D802            call    $_P_File_Format ;AN000; do process
31357 00001D24 2E833E[7119]09      cmp     word [cs:$_P_RC],$_P_Syntax ;AC034; if error, examine the next type
31358 00001D2A 750F                jne     short $_P_Match_Exit ;AN000;
31359                                _P_Match07:
31360                                ;test     ax,$_P_Simple_S ; 2000h;AN000; simple string
31361                                ; 07/07/2023
31362 00001D2C F6C420            test     ah,($_P_Simple_S>>8) ; 20h
31363 00001D2F 740A                jz      short $_P_Match09 ;AN000;
31364
31365 00001D31 2EC706[7119]0000        mov     word [cs:$_P_RC],$_P_No_Error ;AC034; assume no error
31366 00001D38 E8BA01            call    $_P_Simple_String ;AN000; do process
31367                                _P_Match09:
31368                                ;test     ax,$_P_Simple_S ; 2000h;AN000; simple string
31369 00001D3B 2E833E[141A]01      cmp     word [cs:$_P_err_flag],$_P_error_filespec ;AC034; bad filespec ?
31370 00001D41 750F                jne     short $_P_Match2_Exit ;AN033; no, continue
31371 00001D43 2E833E[7119]00      cmp     word [cs:$_P_RC],$_P_No_Error ;AN033;AC034;; check for other errors ?
31372 00001D49 7507                jne     short $_P_Match2_Exit ;AN033; no, continue
31373 00001D4B 2EC706[7119]0900        mov     word [cs:$_P_RC],$_P_Syntax ;AN033;AC034;; set error flag
31374                                _P_Match2_Exit:
31375 00001D52 58                    pop     ax ;AN000;
31376 00001D53 C3                    retn    ;AN000;
31377
31378                                ;*****
31379                                ; _P_Remove_Colon;
31380                                ;
31381                                ; Function: Remove colon at end
31382                                ;
31383                                ; Input: cs:SI points to string buffer to be examined
31384                                ;
31385                                ; Output: None
31386                                ;
31387                                ; Use: _P_Chk_DBCS
31388                                ;*****
31389
31390                                _P_Remove_Colon:
31391 00001D54 50                    push    ax ;AN000;
31392 00001D55 56                    push    si ;AN000;
31393                                _P_RCOL_Loop:
31394 00001D56 2E8A04            mov     al,[cs:si] ;AN000; get character
31395 00001D59 08C0            or      al,al ;AN000; end of string ?
31396 00001D5B 741A            jz      short $_P_RCOL_Exit ;AN000; if yes, just exit
31397
31398 00001D5D 3C3A            cmp     al,$_P_Colon ;AN000; is it colon ?
31399 00001D5F 750D            jne     short $_P_RCOL00 ;AN000;
31400
31401 00001D61 2E807C0100        cmp     byte [cs:si+1],$_P_NULL ;AN000; if so, next is NULL ?
31402 00001D66 7506            jne     short $_P_RCOL00 ;AN000; no, then next char
31403
31404 00001D68 2EC60400        mov     byte [cs:si],$_P_NULL ;AN000; yes, remove colon
31405 00001D6C EB09            jmp     short $_P_RCOL_Exit ;AN000; and exit.
31406
31407                                _P_RCOL00:
31408 00001D6E E80E04            call    $_P_Chk_DBCS ;AN000;
31409 00001D71 7301            jnc     short $_P_RCOL01 ;AN000; if not colon, then check if
31410                                ; DBCS leading byte.
31411                                inc     si ;AN000; if yes, skip trailing byte
31412                                _P_RCOL01:
31413 00001D74 46                    inc     si ;AN000;
31414 00001D75 EBD F            jmp     short $_P_RCOL_Loop ;AN000; si points to next byte
31415                                ; loop until NULL encountered
31416                                _P_RCOL_Exit:
31417 00001D77 5E                    pop     si ;AN000;
31418 00001D78 58                    pop     ax ;AN000;
31419 00001D79 C3                    retn    ;AN000;
31420
31421                                ;*****
31422                                ; _P_Do_CAPS_String;
31423                                ;
31424                                ; Function: Perform capitalization along with the file case map table
31425                                ; or character case map table.
31426                                ;
31427                                ; Input: AL = 2 : Use character table
31428                                ; AL = 4 : Use file table
31429                                ; cs:SI points to string buffer to be capitalized
31430                                ;
31431                                ; Output: None
31432                                ;
31433                                ; Use: _P_Do_CAPS_Char, _P_Chk_DBCS
31434                                ;*****
31435
31436                                _P_Do_CAPS_String:
31437 00001D7A 56                    push    si ;AN000;
31438 00001D7B 52                    push    dx ;AN000;
31439 00001D7C 88C2            mov     di,al ;AN000; save info id
31440
31441                                _P_DCS_Loop:
31442 00001D7E 2E8A04            mov     al,[cs:si] ;AN000; load charater and
31443 00001D81 E8FB03            call    $_P_Chk_DBCS ;AN000; check if DBCS leading byte
31444 00001D84 720C            jc      short $_P_DCS00 ;AN000; if yes, do not need CAPS
31445
31446 00001D86 08C0            or      al,al ;AN000; end of string ?
31447 00001D88 740C            jz      short $_P_DCS_Exit ;AN000; then exit.
31448
31449 00001D8A E80C00            call    $_P_Do_CAPS_Char ;AN000; Here a SBCS char need to be CAPS
31450 00001D8D 2E8804            mov     [cs:si],al ;AN000; stored upper case char to buffer
31451 00001D90 EB01            jmp     short $_P_DCS01 ;AN000; process next
31452                                _P_DCS00:
31453 00001D92 46                    inc     si ;AN000; skip DBCS leading and trailing byte
31454                                _P_DCS01:
31455 00001D93 46                    inc     si ;AN000;
31456 00001D94 EBE8            jmp     short $_P_DCS_Loop ;AN000; si point to next byte
31457                                ; loop until NULL encountered
31458                                _P_DCS_Exit:
31459 00001D96 5A                    pop     dx ;AN000;
31460 00001D97 5E                    pop     si ;AN000;
31461                                retn
31462
31463                                ;*****
31464                                ; _P_Do_CAPS_Char;
31465                                ;
31466                                ; Function: Perform capitalization along with the file case map table
31467                                ; or character case map table.
31468                                ;
31469                                ; Input: DL = 2 : Use character table
31470                                ; DL = 4 : Use file table
31471                                ; AL = character to be capitalized

```



```

31471 ;
31472 ; Output:  None
31473 ;
31474 ; Use:      INT 21h /w AH=65h
31475 ;*****
31476
31477 _$P_Do_CAPS_Char:
31478 cmp     al, _$P_ASCII80 ;80h ;AN000; need upper case table ?
31479 jae     short _$P_DCC_Go ;AN000;
31480
31481 cmp     al,"a" ;AN000; if no,
31482 jnb     short _$P_CAPS_Ret ;AN000; check if "a" <= AL <= "z"
31483
31484 cmp     al,"z" ;AN000;
31485 ja      short _$P_CAPS_Ret ;AN000; if yes, make CAPS
31486
31487 and     al, _$P_Make_Upper ;0DFh ;AN000; else do nothing.
31488 jmp     short _$P_CAPS_Ret ;AN000;
31489 ; 07/07/2023
31490 retn
31491
31492 _$P_DCC_Go: ;AN000;
31493 push    bx ;AN000;
31494 push    es ;AN000;
31495 push    di ;AN000;
31496
31497 ;lea     di,[cs:_$P_Char_CAP_Ptr] ;AC034; or use char CAPS table ?
31498 ;lea     di,[_$P_Char_CAP_Ptr]
31499 ; 07/09/2023
31500 mov     di, _$P_Char_CAP_Ptr
31501 _$P_DCC00: ;AN000;
31502 cmp     [cs:di],dl ;AN000; already got table address ?
31503 je      short _$P_DCC01 ;AN000; if no,
31504
31505 ;In this next section, ES will be used to pass a 5 byte workarea to INT 21h,
31506 ; the GET COUNTRY INFO call. This usage of ES is required by the function
31507 ; call, regardless of what base register is currently be defined as cs.
31508
31509 push    ax ;AN000; get CAPS table thru DOS call
31510 push    cx ;AN000;
31511 push    dx ;AN000;
31512
31513 push    cs ;AC036; pass current base seg into
31514 ;(Note: this used to push CS. BUG...
31515 pop     es ;AN000; ES reg, required for
31516 ;get extended country information
31517 ;mov     al,dl ; function ;AN000; upper case table
31518 ; 07/07/2023
31519 xchg     ax,dx
31520 mov     ah, _$P_DOS_Get_TBL ; 65h ;AN000; get extended CDI
31521 mov     bx, _$P_DOSTBL_Def ; -1;AN000; get active CON
31522 mov     cx, _$P_DOSTBL_BL ; 5 ;AN000; buffer length
31523 ;mov     dx, _$P_DOSTBL_Def ;AN000; get for default code page
31524 ; 07/07/2023
31525 mov     dx,bx ; 0FFFFh
31526
31527 int     21h ;DI already set to point to buffer
31528 ;AN000; es:di point to buffer that
31529 ;now has been filled in with info
31530 pop     dx ;AN000;
31531 pop     cx ;AN000;
31532 pop     ax ;AN000;
31533
31534 _$P_DCC01: ;AN000;
31535
31536 ;In this next section, ES will be used as the base of the XLAT table, provided
31537 ; by the previous GET COUNTRY INFO DOS call. This usage of ES is made
31538 ; regardless of which base reg is currently the cs reg.
31539
31540 ; 14/04/2024
31541 mov     bx,[cs:di+_$P_DOS_TBL.Off] ;AN000; get offset of table
31542 mov     es,[cs:di+_$P_DOS_TBL.Seg] ;AN000; get segment of table
31543 ; 07/07/2023
31544 les     bx,[cs:di+_$P_DOS_TBL.Off]
31545 inc     bx ;AC035; add '2' to
31546 inc     bx ;AC035; BX reg
31547 sub     al, _$P_ASCII80 ; 80h ;AN000; skip length field
31548 xlat     es:[bx] ;AN000; make char to index
31549 ;AN000; perform case map
31550 xlat     es
31551 pop     di ;AN000;
31552 pop     es ;AN000;
31553 pop     bx ;AN000;
31554 _$P_CAPS_Ret: ;AN000;
31555 retn ;AN000;
31556
31557 ;*****
31558 ; _$P_Value / _$P_SValue
31559 ;
31560 ; Function: Make 32bit value from cs:SI and see value list
31561 ; and make result buffer.
31562 ; _$P_SValue is an entry point for the signed value
31563 ; and this will simply call _$P_Value after the handling
31564 ; of the sign character, "+" or "-"
31565 ;
31566 ; Input: cs:SI -> _$P_STRING_BUF
31567 ; ES:BX -> CONTROL block
31568 ;
31569 ; Output:  None
31570 ;
31571 ; Use:      _$P_Fill_Result, _$P_Check_OVF
31572 ;
31573 ; Vars:  _$P_RC(W), _$P_Flags(RW)
31574 ;*****
31575
31576 ; 26/10/2022 - Retro DOS v4.0
31577 ; (MSDOS 5.0 IO.SYS - SYSINIT:1B0Bh)
31578
31579 ; 07/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
31580 ; (MSDOS 6.21 IO.SYS - SYSINIT:1C46h)
31581 _$P_SValue: ;AN000; when signed value here
31582 push    ax ;AN000;
31583 or      byte [cs:_$P_Flags2], _$P_Signed ;AC034; indicate a signed numeric
31584 and     byte [cs:_$P_Flags2], 0FFh-_$P_Neg ;AC034; assume positive value
31585 ;and     byte [cs:_$P_Flags2], ~_$P_Neg ; 07/07/2023
31586 mov     al,[cs:si] ;AN000; get sign
31587 cmp     al, _$P_Plus ;AN000; "+" ?
31588 je      short _$P_Sva100 ;AN000;
31589
31590 cmp     al, _$P_Minus ;AN000; "-" ?
31591 jne     short _$P_Sva101 ;AN000; else
31592
31593 or      byte [cs:_$P_Flags2], _$P_Neg ;AC034; set this is negative value
31594 _$P_Sva100: ;AN000;

```

```

31595 00001DF4 46      inc     si                ;AN000; skip sign char
31596 _$P_Sval01:      ;AN000;
31597 00001DF5 E80200  call    _$P_Value          ;AN000; and process value
31598 00001DF8 58      pop     ax                ;AN000;
31599 00001DF9 C3      retn
31600
31601 ;*****
31602
31603 ; 07/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
31604 ; (MSDOS 6.21 IO.SYS - SYSINIT:1C6Ah)
31605
31606 ; 26/10/2022
31607 _$P_Value:          ;AN000;
31608 00001DFA 50      push    ax                ;AN000;
31609 00001DFB 51      push    cx                ;AN000;
31610 00001DFC 52      push    dx                ;AN000;
31611 00001DFD 56      push    si                ;AN000;
31612 00001DFE 31C9    xor     cx,cx          ;AN000; cx = higher 16 bits
31613 00001E00 31D2    xor     dx,dx          ;AN000; dx = lower 16 bits
31614 00001E02 53      push    bx                ;AN000; save control pointer
31615 _$P_Value_Loop:      ;AN000;
31616 00001E03 2E8A04  mov     al,[cs:si]        ;AN000; get character
31617 00001E06 08C0    or      al,al          ;AN000; end of line ?
31618 00001E08 7438    jz      short _$P_Value00    ;AN000;
31619
31620 00001E0A E8DC00  call    _$P_0099          ;AN000; make asc(0..9) to bin(0..9)
31621 00001E0D 722F    jc      short _$P_Value_Err0 ;AN000;
31622
31623 00001E0F 30E4    xor     ah,ah            ;AN000;
31624 00001E11 89C5    mov     bp,ax           ;AN000; save binary number
31625
31626 ; 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
31627 ; Ref: Disassembled PC DOS 7.1 IBMBIO.COM SYSINIT code
31628 ; Erdogan Tan - July 2023
31629 %if 0
31630 shl     dx,1          ;AN000; to have 2*x
31631 rcl     cx,1          ;AN000; shift left w/ carry
31632 call    _$P_Check_OVF ;AN000; Overflow occurred ?
31633 jc      short _$P_Value_Err0 ;AN000; then error, exit
31634
31635 mov     bx,dx          ;AN000; save low(2*x)
31636 mov     ax,cx          ;AN000; save high(2*x)
31637 shl     dx,1          ;AN000; to have 4*x
31638 rcl     cx,1          ;AN000; shift left w/ carry
31639 call    _$P_Check_OVF ;AN000; Overflow occurred ?
31640 jc      short _$P_Value_Err0 ;AN000; then error, exit
31641
31642 shl     dx,1          ;AN000; to have 8*x
31643 rcl     cx,1          ;AN000; shift left w/ carry
31644 call    _$P_Check_OVF ;AN000; Overflow occurred ?
31645 jc      short _$P_Value_Err0 ;AN000; then error, exit
31646
31647 add     dx,bx          ;AN000; now have 10*x
31648 adc     cx,ax          ;AN000; 32bit ADD
31649 call    _$P_Check_OVF ;AN000; Overflow occurred ?
31650 jc      short _$P_Value_Err0 ;AN000; then error, exit
31651
31652 add     dx,bp          ;AN000; Add the current one degree decimal
31653 adc     cx,0           ;AN000; if carry, add 1 to high 16bit
31654 call    _$P_Check_OVF ;AN000; Overflow occurred ?
31655 jc      short _$P_Value_Err0 ;AN000; then error, exit
31656
31657 inc     si             ;AN000; update pointer
31658 jmp     short _$P_Value_Loop ;AN000; loop until NULL encountered
31659 ;_ $P_Value_Err0:
31660 %endif
31661 ;****
31662 %if 1
31663 ; 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
31664 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:2130h)
31665
31666 ; 14/04/2024 - Retro DOS v5.0
31667 xor     ah,ah
31668 mov     bp,ax          ; save binary number
31669
31670 00001E13 E81C00  call    _$P_Value_2x_OVF ; multiply cx:dx by 2 and then check overflow
31671 00001E16 89D3    mov     bx,dx          ; ax:bx = 2*(cx:dx)
31672 00001E18 89C8    mov     ax,cx
31673 00001E1A E81500  call    _$P_Value_2x_OVF ; multiply cx:dx by 2 and then check overflow
31674 00001E1D E81200  call    _$P_Value_2x_OVF ; multiply cx:dx by 2 and then check overflow
31675 00001E20 01DA    add     dx,bx          ; 8*(cx:dx)+2*(cx:dx) = 10*(cx:dx)
31676 00001E22 11C1    adc     cx,ax
31677 00001E24 E80F00  call    _$P_Value_Chk_Add_OVF
31678 00001E27 01EA    add     dx,bp          ; Add the current one degree decimal
31679                                ; if carry, add 1 to high 16bit
31680 adc     cx,0           ;
31681 00001E2C E80700  call    _$P_Value_Chk_Add_OVF ; Overflow occurred ?
31682                                ; then error, exit (without return here)
31683 00001E2F 46      inc     si             ; update pointer
31684 00001E30 EBD1    jmp     short _$P_Value_Loop
31685
31686 _$P_Value_2x_OVF:
31687 00001E32 D1E2    shl     dx,1          ; to have 2*x
31688 00001E34 D1D1    rcl     cx,1          ; shift left w/ carry
31689
31690 00001E36 E89E00  call    _$P_Check_OVF ; check overflow (for the last shift or add)
31691 00001E39 7201    jc      short _$P_Value_OVF
31692 00001E3B C3      retn
31693
31694 00001E3C 44      inc     sp             ; skip "call" return address to the caller
31695 00001E3D 44      inc     sp
31696
31697 ;_ $P_Value_Err0:
31698 %endif
31699 ;****
31700
31701 _$P_Value_Err0:
31702 00001E3E 5B      pop     bx            ;AN000;
31703 00001E3F E98300  jmp     _$P_Value_Err  ;AN000; Bridge
31704
31705 _$P_Value00:
31706 00001E42 5B      pop     bx            ;AN000; restore control pointer
31707 00001E43 2EF606[7D19]02 test    byte [cs:$_P_Flags2],-$P_Neg ;AC034; here cx,dx = 32bit value
31708 00001E49 740A    jz      short _$P_Value01 ;AN000; was it negative ?
31709
31710 00001E4B F7D1    not     cx            ;AN000; +
31711 00001E4D F7D2    not     dx            ;AN000; |- Make 2's complement
31712 00001E4F 83C201  add     dx,1          ;AN000; |
31713 00001E52 83D100  adc     cx,0           ;AN000; +
31714
31715 _$P_Value01:
31716 00001E55 268B7706 mov     si,[es:bx+$_P_Control_Blk.Value_List] ;AN000; / nval = 0
31717 00001E59 268A04  mov     al,[es:si]    ;AN000; get nval
31718 ; 07/09/2023

```

```

31719      ;cmp     al,_$P_nval_None ; 0 ;AN000; no value list ?
31720      ;;*jne    short _$P_Value02 ;AN000;
31721      ;;* 07/07/2023
31722      ;je      short _$P_Value05
31723      ; 07/09/2023
31724      or      al,al
31725      jz      short _$P_Value05 ; _$P_nval_None
31726
31727      ;mov     al,_$P_Number ;AN000; Set type
31728      ;mov     ah,_$P_No_Tag ;AN000; No ITEM_TAG set
31729      ; 07/07/2023
31730      ;*mov     ax,(_$P_No_Tag<<8)|_$P_Number
31731      ;*jmp     short _$P_Value_Exit ;AN000;
31732
31733      ; 26/10/2022 (MSDOS 5.0 IO.SYS, SYSINIT compatibility)
31734      ; (SYSINIT:1BA5h)
31735      ; 12/12/2022
31736      ;nop     ; db 90h
31737
31738      _$P_value02: ;AN000; / nval = 1
31739      ;IF val1sw ;AN000; (Check if value list id #1 is supported)
31740      ;(tm07) cmp al,_$P_nval_Range ;AN000; have range list ?
31741      ;(tm07) jne short _$P_Value03 ;AN000;
31742
31743      inc      si ;AN000;
31744      mov     al,[es:si] ;AN000; al = number of range
31745
31746      ; 07/09/2023
31747      ;cmp     al,_$P_No_nrng ;AN000; (tm07)
31748      ;je      short _$P_Value03 ;AN000; (tm07)
31749      or      al,al
31750      jz      short _$P_Value03 ; _$P_No_nrng
31751
31752      inc      si ;AN000; si points to 1st item_tag
31753      _$P_val02_Loop: ;AN000;
31754      test     byte [cs:_$P_Flags2],_$P_Signed ;AC034;
31755      jnz      short _$P_Val02_Sign ;AN000;
31756
31757      cmp     cx,[es:si+_$P_Val_List.val_XH] ;AN000; comp cx with XH
31758      jnb     short _$P_Val02_Next ;AN000;
31759      ja      short _$P_Val_In ;AN000;
31760
31761      cmp     dx,[es:si+_$P_Val_List.val_XL] ;AN000; comp dx with XL
31762      jnb     short _$P_Val02_Next ;AN000;
31763
31764      _$P_Val_In: ;AN000;
31765      cmp     cx,[es:si+_$P_Val_List.val_YH] ;AN000; comp cx with YH (tm01)
31766      ja      short _$P_Val02_Next ;AN000;
31767      jb      short _$P_Val_Found ;AN000;
31768
31769      cmp     dx,[es:si+_$P_Val_List.val_YL] ;AN000; comp dx with YL
31770      ja      short _$P_Val02_Next ;AN000;
31771
31772      jmp     short _$P_Val_Found ;AN000;
31773
31774      _$P_val02_Sign: ;AN000;
31775      cmp     cx,[es:si+_$P_Val_List.val_XH] ;AN000; comp cx with XH
31776      jl      short _$P_Val02_Next ;AN000;
31777      jg      short _$P_Sval_In ;AN000;
31778
31779      cmp     dx,[es:si+_$P_Val_List.val_XL] ;AN000; comp dx with XL
31780      jl      short _$P_Val02_Next ;AN000;
31781
31782      _$P_Sval_In: ;AN000;
31783      cmp     cx,[es:si+_$P_Val_List.val_YH] ;AN000; comp cx with YH
31784      jg      short _$P_Val02_Next ;AN000;
31785
31786      jl      short _$P_Val_Found ;AN000;
31787
31788      cmp     dx,[es:si+_$P_Val_List.val_YL] ;AN000; comp dx with YL
31789      ;jg      short _$P_Val02_Next ;AN000;
31790      ;jmp     short _$P_Val_Found ;AN000;
31791      ; 07/07/2023
31792      jng     short _$P_Val_Found
31793
31794      _$P_val02_Next: ;AN000;
31795      add     si,_$P_Len_Range ;AN000;
31796      dec     al ;AN000; loop nrng times in AL
31797      jne     short _$P_Val02_Loop ;AN000;
31798      ; / Not found
31799      mov     word [cs:_$P_RC],_$P_Out_Of_Range ;AC034;
31800      ;mov     al,_$P_Number ;AN000;
31801      ;mov     ah,_$P_No_Tag ;AN000; No ITEM_TAG set
31802      _$P_Value05: ;* 07/07/2023
31803      ; 07/07/2023
31804      mov     ax,(_$P_No_Tag<<8)|_$P_Number
31805      jmp     short _$P_Value_Exit ;AN000;
31806
31807      _$P_Val_Found: ;AN000;
31808      mov     al,_$P_Number ;AN000;
31809      mov     ah,[es:si] ;AN000; found ITEM_TAG set
31810      jmp     short _$P_Value_Exit ;AN000;
31811
31812      _$P_value03: ;AN000; / nval = 2
31813
31814      ;IF val2sw ;AN000; (Check if value list id #2 is supported)
31815      ;;;cmp     al,$P_nval_Value ;AN000; have match list ? ASSUME nval=2,
31816      ;;;jne     $P_Value04 ;AN000; even if it is 3 or more.
31817      ;(tm07) inc si ;AN000;
31818      ;(tm07) mov al,es:[si] ;AN000; al = nrng
31819      ; mov     ah,$P_Len_Range ;AN000;
31820      ; mul     ah ;AN000; skip nrng field
31821      ; inc     ax ;AN000;
31822      ; add     si,ax ;AN000; si points to nnval
31823      ; mov     al,es:[si] ;AN000; get nnval
31824      ; inc     si ;AN000; si points to 1st item_tag
31825      _$P_Val03_Loop: ;AN000;
31826      cmp     cx,es:[si+$P_Val_XH] ;AN000; comp cx with XH
31827      jne     $P_Val03_Next ;AN000;
31828
31829      cmp     dx,es:[si+$P_Val_XL] ;AN000; comp dx with XL
31830      je      $P_Val_Found ;AN000;
31831
31832      _$P_Val03_Next: ;AN000;
31833      add     si,$P_Len_Value ;AN000; points to next value choice
31834      dec     al ;AN000; loop nval times in AL
31835      jne     $P_Val03_Loop ;AN000;
31836      ; / Not found
31837      mov     psdata_seg:$P_RC,$P_Not_in_Val ;AC034;
31838      mov     al,$P_Number ;AN000;
31839      mov     ah,$P_No_Tag ;AN000; No ITEM_TAG set
31840      jmp     short $P_Value_Exit ;AN000;
31841
31842      ;ENDIF ;AN000; (of val2sw)

```

```

31843 ;$P_Value04:
31844
31845
31846 00001EC5 2EC706[7119]0900
31847
31848
31849
31850
31851 00001ECC B803FF
31852
31853 00001ECF E855FD
31854 00001ED2 5E
31855 00001ED3 5A
31856 00001ED4 59
31857 00001ED5 58
31858 00001ED6 C3
31859
31860 ; 28/03/2019 - Retro DOS v4.0
31861
31862
31863
31864
31865 ; Function: Check if overflow is occurred with consideration of
31866 ; signed or un-signed numeric value
31867
31868 ; Input: Flag register
31869
31870 ; Output: CY = 1 : overflow
31871
31872 ; Vars: _$P_Flags(R)
31873
31874
31875
31876
31877 00001ED7 9C
31878 00001ED8 2EF606[7D19]02
31879 00001EDE 7502
31880
31881 00001EE0 9D
31882 00001EE1 C3
31883
31884
31885 00001EE2 9D
31886 00001EE3 7002
31887
31888 00001EE5 F8
31889 00001EE6 C3
31890
31891
31892 00001EE7 F9
31893 00001EE8 C3
31894
31895
31896
31897
31898
31899
31900
31901
31902
31903
31904
31905
31906
31907
31908
31909
31910
31911
31912
31913
31914
31915
31916
31917
31918
31919
31920
31921
31922
31923
31924
31925
31926
31927
31928
31929
31930
31931
31932 00001EE9 3C30
31933 00001EEB 7207
31934 00001EED 3C3A
31935 00001EEF F5
31936 00001EF0 7202
31937 00001EF2 2C30
31938
31939
31940 00001EF4 C3
31941
31942
31943
31944
31945
31946
31947
31948
31949
31950
31951
31952
31953
31954
31955
31956
31957
31958
31959
31960 00001EF5 50
31961 00001EF6 53
31962 00001EF7 52
31963 00001EF8 57
31964 00001EF9 268B7F06
31965 00001EFD 268A05
31966 00001F00 08C0

;$P_Value04:
_$P_Value_Err:
    mov word [cs:_$P_RC],_$P_Syntax ;AN000;
    ;mov al,_$P_String ;AN000; Set type
    ;mov ah,_$P_No_Tag ;AN000; No ITEM_TAG set
    ; 07/09/2023
    ; 07/07/2023
    mov ax,(_$P_No_Tag<<8)|_$P_String
_$P_Value_Exit:
    call _$P_Fill_Result ;AN000;
    ;AN000;
    pop si ;AN000;
    pop dx ;AN000;
    pop cx ;AN000;
    pop ax ;AN000;
    retn ;AN000;

; 28/03/2019 - Retro DOS v4.0
; *****
; _$P_Check_OVF
;
; Function: Check if overflow is occurred with consideration of
; signed or un-signed numeric value
;
; Input: Flag register
;
; Output: CY = 1 : overflow
;
; Vars: _$P_Flags(R)
; *****
; 26/10/2022
_$P_Check_OVF:
    pushf ;AN000;
    test byte [cs:_$P_Flags2],_$P_Neg ;AC034; is it negative value ?
    jnz short _$P_COVF ;AN000; if no, check overflow
    popf ;AN000; by the CY bit
    retn ;AN000;
_$P_COVF:
    popf ;AN000;
    jo short _$P_COVF00 ;AN000; else,
    ;AN000; check overflow by the OF
    cllc ;AN000; indicate it with CY bit
    retn ;AN000; CY=0 means no overflow
_$P_COVF00:
    stc ;AN000;
    retn ;AN000; and CY=1 means overflow
; *****
; _$P_0099;
;
; Function: Make ASCII 0-9 to Binary 0-9
;
; Input: AL = character code
;
; Output: CY = 1 : AL is not number
;
; CY = 0 : AL contains binary value
; *****
; 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
%if 0
_$P_0099:
    cmp al,"0" ;AN000;
    jnb short _$P_0099Err ;AN000; must be 0 =< al =< 9
    ; 12/12/2022
    jnb short _$P_0099Err2 ; cf=1
    cmp al,"9" ;AN000;
    ja short _$P_0099Err ;AN000; must be 0 =< al =< 9
    sub al,"0" ;AN000; make char -> bin
    ; 12/12/2022
    ; cf=0
    ; cllc ;AN000; indicate no error
    retn ;AN000;
_$P_0099Err:
    stc ;AN000;
    _$P_0099Err2: ; 12/12/2022
    retn ;AN000;
%endif
; 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
%if 1
_$P_0099:
    cmp al,"0" ; cmp al,30h
    jnb short _$P_0099Err ; must be 0 =< al =< 9
    cmp al,"9"+1 ; cmp al,3Ah
    cmc ; cf=0 -> cf=1
    jnb short _$P_0099Err
    sub al,"0" ; sub al,30h ; make char -> bin
    ; cf=0
    _$P_0099Err: ; cf=1
    retn
%endif
; *****
; _$P_Simple_String
;
; Function: See value list for the simple string
; and make result buffer.
;
; Input: cs:SI -> _$P_STRING_BUF
; ES:BX -> CONTROL block
;
; Output: None
;
; Use: _$P_Fill_Result, _$P_String_Comp
;
; Vars: _$P_RC(W)
; *****
_$P_Simple_String:
    push ax ;AN000;
    push bx ;AN000;
    push dx ;AN000;
    push di ;AN000;
    mov di,[es:bx+_$P_Control_Blk.Value_List] ;AN000; di points to value list
    mov al,[es:di] ;AN000; get nval
    or al,al ;AN000; no value list ?

```

```

31967 00001F02 7504      jnz      short _$P_Sim00      ;AN000; then
31968
31969 00001F04 B4FF      mov      ah,_$P_No_Tag      ;AN000; No ITEM_TAG set
31970 00001F06 EB4C      jmp      short _$P_Sim_Exit  ;AN000; and set result buffer
31971
31972      _$P_Sim00:              ;AN000;
31973      ;IF Val3SW+KeySW      ;AN000;(Check if keyword or value list id #3 is supported)
31974 00001F08 3C03      cmp      al,_$P_nval_String  ;AN000; String choice list provided ?
31975 00001F0A 753F      jne      short _$P_Sim01     ;AN000; if no, syntax error
31976
31977 00001F0C 47          inc      di      ;AN000;
31978 00001F0D 268A05     mov      al,[es:di]      ;AN000; al = nrng
31979 00001F10 B409     mov      ah,_$P_Len_Range ;AN000;
31980 00001F12 F6E4     mul      ah      ;AN000; Skip nrng field
31981 00001F14 40          inc      ax      ;AN000; ax = (nrng*9)+1
31982 00001F15 01C7     add      di,ax      ;AN000; di points to nval
31983 00001F17 268A05     mov      al,[es:di]      ;AN000; get nval
31984 00001F1A B405     mov      ah,_$P_Len_Value  ;AN000;
31985 00001F1C F6E4     mul      ah      ;AN000; Skip nval field
31986 00001F1E 40          inc      ax      ;AN000; ax = (nval*5)+1
31987 00001F1F 01C7     add      di,ax      ;AN000; di points to nstrval
31988 00001F21 268A05     mov      al,[es:di]      ;AN000; get nstrval c
31989 00001F24 47          inc      di      ;AC035; add '2' to
31990 00001F25 47          inc      di      ;AC035; DI reg
31991
31992      _$P_Sim_Loop:          ;AN000;
31993 00001F26 268B2D     mov      bp,[es:di]      ;AN000; get string pointer
31994 00001F29 E83200     call     _$P_String_Comp    ;AN000; compare it with operand
31995 00001F2C 7312     jnc      short _$P_Sim_Found ;AN000; found on list ?
31996
31997 00001F2E 83C703     add      di,_$P_Len_String ; 3 ;AN000; if no, point to next choice
31998 00001F31 FEC8     dec      al      ;AN000; loop nstrval times in AL
31999 00001F33 75F1     jne      short _$P_Sim_Loop  ;AN000;
32000      ;AN000; / Not found
32001 00001F35 2EC706[7119]0800     mov      word [cs:_$P_RC],_$P_Not_In_Str ;AC034;
32002 00001F3C B4FF      mov      ah,_$P_No_Tag      ;AN000; No ITEM_TAG set
32003 00001F3E EB14      jmp      short _$P_Sim_Exit  ;AN000;
32004
32005      _$P_Sim_Found:          ;AN000;
32006 00001F40 268A65FF     mov      ah,[es:di-1]      ;AN000; set item_tag
32007 00001F44 B002     mov      al,_$P_List_Idx    ;AN000;
32008 00001F46 268B15     mov      dx,[es:di]      ;AN000; get address of STRING
32009 00001F49 EB0B     jmp      short _$P_Sim_Exit0 ;AN000;
32010      ;ENDIF
32011      _$P_Sim01:              ;AN000;
32012 00001F4B 2EC706[7119]0900     mov      word [cs:_$P_RC],_$P_Syntax ;AC034;
32013 00001F52 B4FF      mov      ah,_$P_No_Tag      ;AN000; No ITEM_TAG set
32014
32015 00001F54 B003     mov      al,_$P_String      ;AN000; Set type
32016      _$P_Sim_Exit0:          ;AN000;
32017 00001F56 E8CEFC     call     _$P_Fill_Result    ;AN000;
32018 00001F59 5F          pop      di      ;AN000;
32019 00001F5A 5A          pop      dx      ;AN000;
32020 00001F5B 5B          pop      bx      ;AN000;
32021 00001F5C 58          pop      ax      ;AN000;
32022 00001F5D C3          retn      ;AN000;
32023
32024      ;*****
32025      ; _$P_String_Comp:
32026      ;
32027      ; Function: Compare two string
32028      ;
32029      ; Input:      cs:SI -> 1st string
32030      ;             ES:BP -> 2nd string (Must be upper case)
32031      ;             ES:BX -> CONTROL block
32032      ;
32033      ; Output:     CY = 1 if not match
32034      ;
32035      ; Use:        _$P_Chk_DBCS, _$P_Do_CAPS_Char
32036      ;
32037      ; Vars:       _$P_KeyOr_SW_Ptr(W), _$P_Flags(R), _$P_KeyOrSW_Ptr
32038      ;*****
32039
32040      _$P_String_Comp:
32041 00001F5E 50          push     ax      ;AN000;
32042 00001F5F 55          push     bp      ;AN000;
32043 00001F60 52          push     dx      ;AN000;
32044 00001F61 56          push     si      ;AN000;
32045 00001F62 B202     mov      di,_$P_DOSTBL_Char ;AN000; use character case map table
32046
32047 00001F64 2E8A04     mov      al,[cs:si]      ;AN000; get command character
32048 00001F67 E81502     call     _$P_Chk_DBCS      ;AN000; DBCS ?
32049 00001F6A 723A     jc       short _$P_SCOM00    ;AN000; yes, DBCS
32050
32051 00001F6C E82AFE     call     _$P_Do_CAPS_Char   ;AN000; else, upper case map before comparison
32052      ;IF KeySW+SwSW      ;AN000;(Check if keyword or switch is supported)
32053 00001F6F 2EF606[7D19]08     test     byte [cs:_$P_Flags2],_$P_Key_Cmp ;AC034; keyword search ?
32054 00001F75 740D     jz       short _$P_SCOM04    ;AN000;
32055
32056 00001F77 3C3D     cmp      al,_$P_Keyword     ;AN000; "=" is delimiter
32057 00001F79 751F     jne      short _$P_SCOM03     ;AN000; IF "=" on command line AND (bp+1=> char after the "=" in synonym
32058      list)
32059 00001F7B 26807E0100     cmp      byte [es:bp+1],_$P_NULL ;AN021; at end of keyword string in the control block THEN
32060 00001F80 756D     jne      short _$P_SCOM_Differ ;AN021;
32061
32062 00001F82 EB13     jmp      short _$P_SCOM05     ;AN000; keyword found in synonym list
32063
32064      _$P_SCOM04:              ;AN000;
32065 00001F84 2EF606[7D19]10     test     byte [cs:_$P_Flags2],_$P_SW_Cmp ;AC034; switch search ?
32066 00001F8A 740E     jz       short _$P_SCOM03     ;AN000;
32067
32068 00001F8C 3C3A     cmp      al,_$P_Colon        ;AN000; ":" is delimiter, at end of switch on command line
32069 00001F8E 750A     jne      short _$P_SCOM03     ;AN000; continue compares
32070
32071      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32072      ; cmp      byte [es:bp+0],_$P_NULL
32073      ; 11/12/2022
32074 00001F90 26807E0000     cmp      byte [es:bp],_$P_NULL ;AN021; IF at end of switch on command AND
32075 00001F95 7558     jne      short _$P_SCOM_Differ ;AN021; at end of switch string in the control block THEN
32076
32077      _$P_SCOM05:              ;AN000; found a match
32078 00001F97 46          inc      si      ;AN000; si points to just after "=" or ":"
32079 00001F98 EB58     jmp      short _$P_SCOM_Same  ;AN000; exit
32080
32081      _$P_SCOM03:              ;AN000;
32082      ;ENDIF
32083      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32084      ; cmp      al,[es:bp+0]
32085      ; 11/12/2022
32086 00001F9A 263A4600     cmp      al,[es:bp]      ;AN000; compare operand w/ a synonym
32087 00001F9E 751B     jne      short _$P_SCOM_Differ0 ;AN000; if different, check ignore colon option
32088
32089 00001FA0 08C0     or       al,al      ;AN000; end of line

```

```

32090 00001FA2 744E      jz      short _P_SCOM_Same      ;AN000; if so, exit
32091
32092      ; 12/12/2022
32093      ;inc      si                      ;AN000; update operand pointer
32094      ;inc      bp                      ;AN000; and synonym pointer
32095      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32096 00001FA4 EB11      jmp      short _P_SCOM01      ;AN000; loop until NULL or "=" or ":" found in case
32097
32098      _P_SCOM00:                      ;AN000; Here al is DBCS leading byte
32099      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32100      ;cmp      al,[es:bp+0]
32101      ; 11/12/2022
32102 00001FA6 263A4600    cmp      al,[es:bp]      ;AN000; compare leading byte
32103 00001FAA 7543      jne      short _P_SCOM_Differ ;AN000; if not match, say different
32104
32105 00001FAC 46          inc      si                      ;AN000; else, load next byte
32106 00001FAD 2E8A04    mov      al,[cs:si]      ;AN000; and
32107 00001FB0 45          inc      bp                      ;AN000;
32108      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32109      ;cmp      al,[es:bp+0]
32110      ; 11/12/2022
32111 00001FB1 263A4600    cmp      al,[es:bp]      ;AN000; compare 2nd byte
32112 00001FB5 7538      jne      short _P_SCOM_Differ ;AN000; if not match, say different, too
32113
32114      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32115      ; 12/12/2022
32116      _P_SCOM01:                      ;AN000;
32117 00001FB7 46          inc      si                      ;AN000; else update operand pointer
32118 00001FB8 45          inc      bp                      ;AN000; and synonym pointer
32119      ;_P_SCOM01:                      ;AN000;
32120 00001FB9 EBA9      jmp      short _P_SCOM_Loop    ;AN000; loop until NULL or "=" or "/" found in case
32121
32122      _P_SCOM_Differ0:                  ;AN000;
32123      ;IF SwSW                          ;AN000;(tm10)
32124 00001FBB 2EF606[7D19]40 test     byte [cs:_P_Flags2],_P_Sw ;AC034;(tm10)
32125 00001FC1 740E      jz      short _P_not_applicable ;AN000;(tm10)
32126
32127      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32128      ;test     word [es:bx+_P_Control_Blk.Function_Flag],_P_colon_is_not_necessary ;AN000;(tm10)
32129      ; 12/12/2022
32130 00001FC3 26F6470220 test     byte [es:bx+_P_Control_Blk.Function_Flag],_P_colon_is_not_necessary
32131 00001FC8 7407      jz      short _P_not_applicable ;AN000;(tm10)
32132
32133      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32134      ;cmp      byte [es:bp+0],_P_NULL
32135      ; 11/12/2022
32136 00001FCA 26807E0000 cmp      byte [es:bp],_P_NULL ;AN000;(tm10)
32137      ;(deleted ;AN025;) jne short _P_not_applicable ;AN000;(tm10)
32138 00001FCF 7421      je      short _P_SCOM_Same    ;AN025;(tm10)
32139
32140      _P_not_applicable:                  ;AN000;(tm10)
32141      ;ENDIF                          ;AN000;(tm10)
32142
32143      ;test     word [es:bx+_P_Control_Blk.Match_Flag],_P_Ig_Colon
32144      ;AN000; ignore colon option specified ?
32145      ;test     byte [es:bx+_P_Control_Blk.Match_Flag],_P_Ig_Colon
32146      ; 12/12/2022
32147 00001FD1 26F60710 test     byte [es:bx],_P_Ig_Colon
32148      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32149      ;test     word [es:bx],_P_Ig_Colon ; 10h
32150 00001FD5 7418      jz      short _P_SCOM_Differ ;AN000; if no, say different.
32151
32152 00001FD7 3C3A      cmp      al,_P_Colon          ;AN000; End up with ":" and
32153 00001FD9 7509      jne      short _P_SCOM02      ;AN000; subsequently
32154
32155      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32156      ;cmp      byte [es:bp+0],_P_NULL
32157      ; 11/12/2022
32158 00001FDB 26807E0000 cmp      byte [es:bp],_P_NULL ;AN000; NULL ?
32159 00001FE0 750D      jne      short _P_SCOM_Differ ;AN000; if no, say different
32160
32161 00001FE2 EB0E      jmp      short _P_SCOM_Same    ;AN000; else, say same
32162
32163      _P_SCOM02:                      ;AN000;
32164 00001FE4 3C00      cmp      al,_P_NULL          ;AN000; end up NULL and :
32165 00001FE6 7507      jne      short _P_SCOM_Differ ;AN000;
32166
32167      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32168      ;cmp      byte [es:bp+0],_P_Colon
32169      ; 11/12/2022
32170 00001FE8 26807E003A cmp      byte [es:bp],_P_Colon;AN000; if no, say different
32171 00001FED 7403      je      short _P_SCOM_Same    ;AN000; else, say same
32172
32173      _P_SCOM_Differ:                  ;AN000;
32174 00001FEF F9          stc                          ;AN000; indicate not found
32175 00001FF0 EB05      jmp      short _P_SCOM_Exit    ;AN000;
32176
32177      _P_SCOM_Same:                      ;AN000;
32178      ; 12/12/2022
32179      ; cf=0
32180 00001FF2 2E8936[8019] mov      [cs:_P_KEYorSW_Ptr],si ;AC034; for later use by keyword or switch
32181      ; 12/12/2022
32182      ;clc                          ;AN000; indicate found
32183      _P_SCOM_Exit:                      ;AN000;
32184 00001FF7 5E          pop      si                      ;AN000;
32185 00001FF8 5A          pop      dx                      ;AN000;
32186 00001FF9 5D          pop      bp                      ;AN000;
32187 00001FFA 58          pop      ax                      ;AN000;
32188 00001FFB C3          retn
32189
32190      ; 30/03/2019
32191
32192      ;IF FileSW+DrvSW                  ;AN000;(Check if file spec or drive only is supported)
32193
32194      ;*****
32195      ; _P_File_Format;
32196      ;
32197      ; Function: Check if the input string is valid file spec format.
32198      ; And set the result buffer.
32199      ;
32200      ; Input:      cs:SI -> _P_STRING_BUF
32201      ;             ES:BX -> CONTROL block
32202      ;
32203      ; Output:     None
32204      ;
32205      ; Use:        _P_Fill_Result, _P_Chk_DBCS, _P_FileSp_Chk
32206      ;
32207      ; Vars: _P_RC(W), _P_SI_Save(W), _P_Terminator(W), _P_SaveSI_CmpX(R)
32208      ;          _P_SaveSI_CmpX(R)
32209      ;*****
32210
32211      _P_File_Format:
32212 00001FFC 50          push     ax                      ;AN000;
32213 00001FFD 57          push     di                      ;AN000;

```



```

32214 00001FFE 56          push    si          ;AN000;
32215 00001FFF 2E8B3E[7E19] mov     di,[cs:$_P_SaveSI_Cmpx] ;AC034; get user buffer address
32216          _$_P_FileF_Loop0: ;AN000; / skip special characters
32217 00002004 2E8A04      mov     al,[cs:si]          ;AN000; load character
32218 00002007 08C0        or      al,al             ;AN000; end of line ?
32219 00002009 7413        jz       short _$_P_FileF_Err ;AN000; if yes, error exit
32220
32221 0000200B E85D00      call    _$_P_FileSp_Chk      ;AN000; else, check if file special character
32222 0000200E 7523        jne     short _$_P_FileF03   ;AN000; if yes,
32223
32224 00002010 2EC606[141A]01 mov     byte [cs:$_P_err_flag],_$_P_error_filespec
32225          ;AN033;AC034;; set error flag- bad char.
32226 00002016 5E          pop     si          ;AN033;
32227 00002017 2EC60400     mov     byte [cs:si],_$_P_NULL ;AN033;
32228 0000201B 5F          pop     di          ;AN033;
32229 0000201C EB3E        jmp     short _$_P_FileF02    ;AN033;
32230
32231          _$_P_FileF_Err: ;AN000;
32232 0000201E 5E          pop     si          ;AN000;
32233 0000201F 2EC60400     mov     byte [cs:si],_$_P_NULL ;AN000;
32234 00002023 5F          pop     di          ;AN000;
32235
32236          ;test word [es:bx+$_P_Control_Blk.Match_Flag],_$_P_Optional ;AN000; is it optional ?
32237          ;test byte [es:bx+$_P_Control_Blk.Match_Flag],_$_P_Optional
32238          ; 12/12/2022
32239 00002024 26F60701     test    byte [es:bx],_$_P_Optional
32240          ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32241          ;test word [es:bx],_$_P_Optional
32242 00002028 7532        jnz     short _$_P_FileF02    ;AN000;
32243
32244 0000202A 2EC706[7119]0200 mov     word [cs:$_P_RC],_$_P_Op_Missing ;AC034; 3/17/87
32245 00002031 EB29        jmp     short _$_P_FileF02    ;AN000;
32246
32247          _$_P_FileF03: ;AN000;
32248 00002033 58          pop     ax          ;AN000; discard save si
32249 00002034 56          push    si          ;AN000; save new si
32250          _$_P_FileF_Loop1: ;AN000;
32251 00002035 2E8A04      mov     al,[cs:si]          ;AN000; load character (not special char)
32252 00002038 08C0        or      al,al             ;AN000; end of line ?
32253 0000203A 741E        jz       short _$_P_FileF_RLT ;AN000;
32254
32255 0000203C E82C00      call    _$_P_FileSp_Chk      ;AN000; File special character ?
32256 0000203F 740B        je       short _$_P_FileF00   ;AN000;
32257
32258 00002041 E83B01      call    _$_P_Chk_DBCS        ;AN000; no, then DBCS ?
32259 00002044 7302        jnc     short _$_P_FileF01    ;AN000;
32260 00002046 47          inc     di             ;AN000; if yes, skip next byte
32261 00002047 46          inc     si             ;AN000;
32262          _$_P_FileF01: ;AN000;
32263 00002048 47          inc     di             ;AN000;
32264 00002049 46          inc     si             ;AN000;
32265 0000204A EBE9        jmp     short _$_P_FileF_Loop1 ;AN000;
32266
32267          ;
32268 0000204C 2EA2[7719]   mov     [cs:$_P_Terminator],al ;AC034;
32269 00002050 2EC60400     mov     byte [cs:si],_$_P_NULL ;AN000; update end of string
32270 00002054 47          inc     di             ;AN000;
32271 00002055 2E893E[7319] mov     [cs:$_P_SI_Save],di    ;AC034; update next pointer in command line
32272          _$_P_FileF_RLT: ;AN000;
32273 0000205A 5E          pop     si             ;AN000;
32274 0000205B 5F          pop     di             ;AN000;
32275          _$_P_FileF02: ;AN000;
32276 0000205C 58          pop     ax             ;AN000; (tm14)
32277          ;test ax,_$_P_File_Spc ; 200h ;AN000; (tm14)
32278          ; 08/07/2023
32279 0000205D F6C402     test    ah,(_$_P_File_Spc>>8) ; 2
32280 00002060 7408        jz       short _$_P_Drv_Only_Exit ;AN000; (tm14)
32281
32282          push    ax          ;AN000; (tm14)
32283          mov     ah,_$_P_No_Tag ;AN000; set
32284          mov     al,_$_P_File_Spec ;AN000; result
32285          ; 08/07/2023
32286 00002063 B805FF     mov     ax,(_$_P_No_Tag<<8)|_$_P_File_Spec ; 0FF05h
32287          ; set result
32288          call    _$_P_Fill_Result ;AN000; buffer to file spec
32289 00002069 58          pop     ax             ;AN000;
32290
32291          _$_P_Drv_Only_Exit: ;AN000; (tm14)
32292 0000206A C3          retn                ;AN000;
32293
32294          ;*****
32295          ; _$_P_FileSp_Chk
32296          ;
32297          ; Function: Check if the input byte is one of file special characters
32298          ;
32299          ; Input: cs:SI -> _$_P_STRING_BUF
32300          ; AL = character code to be examined
32301          ;
32302          ; Output: ZF = 1 , AL is one of special characters
32303          ;*****
32304
32305          _$_P_FileSp_Chk:
32306 0000206B 53          push    bx             ;AN000;
32307 0000206C 51          push    cx             ;AN000;
32308          ;lea bx,[cs:$_P_FileSp_Char] ;AC034; special character table
32309          ;lea bx,[_$_P_FileSp_Char] ; "[ ] | <> + = ; \ " " at
32310          ; 07/09/2023 ; MSDOS 6.21 IO.SYS - SYSINIT:1838h
32311          mov     bx,_$_P_FileSp_Char
32312 0000206D BB[0B1A]   mov     cx,_$_P_FileSp_Len ; 9 ;AN000; load length of it
32313 00002070 B90900     mov     cx,_$_P_FileSp_Len ; 9 ;AN000; load length of it
32314          _$_P_FileSp_Loop: ;AN000;
32315 00002073 2E3A07     cmp     al,[cs:bx]        ;AN000; is it one of special character ?
32316 00002076 7404        je       short _$_P_FileSp_Exit ;AN000;
32317
32318          inc     bx          ;AN000;
32319 00002079 E2F8        loop    _$_P_FileSp_Loop    ;AN000;
32320
32321          inc     cx          ;AN000; reset ZF
32322          _$_P_FileSp_Exit: ;AN000;
32323 0000207C 59          pop     cx             ;AN000;
32324 0000207D 5B          pop     bx             ;AN000;
32325 0000207E C3          retn
32326
32327          ;ENDIF ;AN000;(of FileSw+DrvSw)
32328
32329          ;IF DrvSw ;AN000;(check if drive only is supported)
32330
32331          ;*****
32332          ; _$_P_Drive_Format;
32333          ;
32334          ; Function: Check if the input string is valid drive only format.
32335          ; And set the result buffer.
32336          ;
32337          ; Input: cs:SI -> _$_P_STRING_BUF

```

```

32338      ; ES:BX -> CONTROL block
32339      ;
32340      ; Output:      None
32341      ;
32342      ; Use:         _$P_Fill_Result, _$P_Chk_DBCS
32343      ;
32344      ; Vars:        _$P_RC(W)
32345      ; *****
32346
32347      _$P_Drv_Format:
32348      push    ax                ;AN000;
32349      push    dx                ;AN000;
32350      mov     al,[cs:si]         ;AN000;
32351      or      al,al             ;AN000; if null string
32352      je      short _$P_Drv_Exit ;AN000; do nothing
32353
32354      call    _$P_Chk_DBCS      ;AN000; is it leading byte ?
32355      jc      short _$P_Drv_Err ;AN000;
32356
32357      cmp     word [cs:si+1],_$P_Colon ;AN000; "d", ":", 0 ?
32358      je      short _$P_DrvF00  ;AN000;
32359
32360      ;test    word [es:bx+_$P_Control_Blk.Match_Flag],_$P_Ig_Colon
32361      ;test    byte [es:bx+_$P_Control_Blk.Match_Flag],_$P_Ig_Colon ;AN000; colon can be ignored?
32362      ; 12/12/2022
32363      test    byte [es:bx],_$P_Ig_Colon
32364      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32365      ;test    word [es:bx],_$P_Ig_Colon
32366      jz      short _$P_Drv_Err  ;AN000;
32367
32368      cmp     byte [cs:si+1],_$P_NULL ;AN000; "d", 0 ?
32369      jne     short _$P_Drv_Err  ;AN000;
32370
32371      _$P_DrvF00:
32372      or      al,_$P_Make_Lower ;AN000; lower case
32373      cmp     al,"a"            ;AN000; drive letter must
32374      jb      short _$P_Drv_Err  ;AN000; in range of
32375
32376      cmp     al,"z"            ;AN000; "a"-"z"
32377      ja      short _$P_Drv_Err  ;AN000; if no, error
32378
32379      sub     al,"a"-1          ;AN000; make text drive to binary drive
32380      mov     dl,al             ;AN000; set
32381      mov     ah,_$P_No_Tag      ;AN000; result
32382      mov     al,_$P_Drive       ;AN000; buffer
32383      ; 08/07/2023
32384      mov     ax,(_$P_No_Tag<<8)|_$P_Drive ; 0FF06h
32385      ; set result buffer
32386      call    _$P_Fill_Result    ;AN000; to drive
32387      jmp     short _$P_Drv_Exit ;AN000;
32388
32389      _$P_Drv_Err:
32390      mov     word [cs:_$P_RC],_$P_Syntax ;AC034;
32391      _$P_Drv_Exit:
32392      pop     dx                ;AN000;
32393      pop     ax                ;AN000;
32394      retn                     ;AN000;
32395
32396      ;ENDIF
32397      ;AN000;(of DrvSw)
32398
32399      ; *****
32400      ; _$P_Skip_Delim;
32401      ;
32402      ; Function: Skip delimiters specified in the PARMS list, white space
32403      ; and comma.
32404      ;
32405      ; Input:      DS:SI -> Command String
32406      ; ES:DI -> Parameter List
32407      ;
32408      ; Output:     CY = 1 if the end of line encountered
32409      ; CY = 0 then SI move to 1st non-delimiter character
32410      ; AL = Last examined character
32411      ;
32412      ; Use:        _$P_Chk_EOL, _$P_Chk_Delim,
32413      ;
32414      ; Vars:        _$P_Flags(R)
32415      ; *****
32416
32417      _$P_Skip_Delim:
32418      _$P_Skip_Delim_Loop:
32419      lodsb                     ;AN000;
32420      call    _$P_Chk_EOL        ;AN000; is it EOL character ?
32421      jz      short _$P_Skip_Delim_CY ;AN000; if yes, exit w/ CY on
32422
32423      call    _$P_Chk_Delim      ;AN000; is it one of delimiters ?
32424      jnz     short _$P_Skip_Delim_NCY ;AN000; if no, exit w/ CY off
32425
32426      test    byte [cs:_$P_Flags2],_$P_Extra ;AC034; extra delim or comma found ?
32427      jz      short _$P_Skip_Delim_Loop ;AN000; if no, loop
32428
32429      test    byte [cs:_$P_Flags2],_$P_SW+_$P_equ ;AC034; /x , or xxx=zzz , (tm08)
32430      jz      short _$P_Exit_At_Extra ;AN000; no switch, no keyword (tm08)
32431      ; 08/07/2023
32432      ; cf=0
32433      jnz     short _$P_Skip_Delim_Exit
32434      retn
32435
32436      ;dec     si                ;AN000; backup si for next call (tm08)
32437      ;jmp     short _$P_Exit_At_Extra ;AN000; else exit w/ CY off
32438      ; 12/12/2022
32439      ; cf=0
32440      ; 08/07/2023
32441      ;jmp     short _$P_Skip_Delim_Exit
32442
32443      _$P_Skip_Delim_CY:
32444      stc                     ;AN000; indicate EOL
32445      jmp     short _$P_Skip_Delim_Exit ;AN000;
32446
32447      _$P_Skip_Delim_NCY:
32448      clc                     ;AN000; indicate non delim
32449      _$P_Skip_Delim_Exit:
32450      dec     si                ;AN000; backup index pointer
32451      ; 08/07/2023
32452      ; 12/12/2022
32453      ;_$P_Exit_At_Extra:
32454      retn                     ; cf=0 ;AN000;
32455
32456      ; 12/12/2022
32457      ;_$P_Exit_At_Extra:
32458      clc                     ;AN000; indicate extra delim
32459      retn                     ;AN000;
32460
32461      ; *****
32462      ;
32463      ; _$P_Chk_EOL;

```



```

32462 ;
32463 ; Function: Check if AL is one of End of Line characters.
32464 ;
32465 ; Input:  AL = character code
32466 ;        ES:DI -> Parameter List
32467 ;
32468 ; Output: ZF = 1 if one of End of Line characters
32469 ; *****
32470
32471 _$P_Chk_EOL:
32472     push    bx                ;AN000;
32473     push    cx                ;AN000;
32474     cmp     al,_$P_CR         ;AN000; Carriage return ?
32475     je      short _$P_Chk_EOL_Exit ;AN000;
32476     cmp     al,_$P_NULL      ;AN000; zero ?
32477     je      short _$P_Chk_EOL_Exit ;AN000;
32478 ;IF LFEOLSW                 ;AN028; IF LF TO BE ACCEPTED AS EOL
32479     cmp     al,_$P_LF        ;AN000; Line feed ?
32480     je      short _$P_Chk_EOL_Exit ;AN000;
32481 ;ENDIF                       ;AN028;
32482     cmp     byte [es:di+_$P_PARAMS_Blk.Num_Extra],_$P_I_Have_EOL
32483             ;AN000; EOL character specified ?
32484     jnb     short _$P_Chk_EOL_Exit ;AN000;
32485     xor     bx,bx             ;AN000;
32486     mov     bl,[es:di+_$P_PARAMS_Blk.Len_Extra_Delim]
32487             ;AN000; get length of delimiter list
32488     add     bx,_$P_Len_PARAMS ;AN000; skip it
32489     ; 08/07/2023
32490     xor     cx,cx ; *
32491     cmp     byte [es:bx+di],_$P_I_Use_Default ;AN000; No extra EOL character ?
32492     je      short _$P_Chk_EOL_NZ ;AN000;
32493     ; 08/07/2023
32494     ;xor     cx,cx             ;AN000; Get number of extra character
32495     ;xor     ch,ch ; *
32496     mov     cl,[es:bx+di]     ;AN000;
32497     _$P_Chk_EOL_Loop:         ;AN000;
32498     inc     bx                ;AN000;
32499     cmp     al,[es:bx+di]     ;AN000; Check extra EOL character
32500     je      short _$P_Chk_EOL_Exit ;AN000;
32501     loop    _$P_Chk_EOL_Loop ;AN000;
32502     ; 08/07/2023
32503     ; cx=0
32504     _$P_Chk_EOL_NZ:          ;AN000;
32505     ;cmp     al,_$P_CR         ;AN000; reset ZF
32506     ; 08/07/2023
32507     inc     cx ; zf=0 (cx=1) ; *
32508     _$P_Chk_EOL_Exit:        ;AN000;
32509     pop     cx                ;AN000;
32510     pop     bx                ;AN000;
32511     retn
32512 ; *****
32513 ; _$P_Chk_Delim;
32514 ;
32515 ; Function: Check if AL is one of delimiter characters.
32516 ; if AL+[si] is DBCS blank, it is replaced with two SBCS
32517 ; blanks.
32518 ;
32519 ; Input:  AL = character code
32520 ;        DS:SI -> Next Character
32521 ;        ES:DI -> Parameter List
32522 ;
32523 ; Output: ZF = 1 if one of delimiter characters
32524 ;        SI points to the next character
32525 ; Vars:  _$P_Terminator(W), _$P_Flags(W)
32526 ; *****
32527 ; 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
32528 ; MSDOS 6.21 IO.SYS - SYSINIT:1FAEH
32529 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:2451h) ; (Retro DOS v5.0)
32530
32531 _$P_Chk_Delim:
32532     push    bx                ;AN000;
32533     push    cx                ;AN000;
32534     mov     byte [cs:_$P_Terminator],_$P_Space
32535             ;AC034; Assume terminated by space
32536     ;and     byte [cs:_$P_Flags20,0DFh
32537     ;and     byte [cs:_$P_Flags2],0FFh-_$P_Extra ;AC034;
32538     cmp     al,_$P_Space ; 20h ;AN000; Space ?
32539     je      short _$P_Chk_Delim_Exit ;AN000;
32540     cmp     al,_$P_TAB        ;AN000; TAB ?
32541     je      short _$P_Chk_Delim_Exit ;AN000;
32542     cmp     al,_$P_Comma      ;AN000; Comma ?
32543     je      short _$P_Chk_Delim_Exit ;AN000;
32544 ; Note: _$P_Chk_Delim00 part of code is nonsense here
32545 ; because _$P_Space = _$P_DBSP1 = 20h
32546 ; Erdogan Tan - 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
32547 _$P_Chk_Delim00:
32548     ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:246Bh)
32549     ; (MSDOS 6.21 IO.SYS - SYSINIT:1FC8h)
32550 %if 0
32551     ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32552 _$P_Chk_Delim00:
32553     cmp     al,_$P_DBSP1 ; 20h ;AN000; 1st byte of DBCS Space ?
32554     jne     short _$P_Chk_Delim01 ;AN000;
32555     cmp     byte [si],_$P_DBSP2 ; 20h ;AN000; 2nd byte of DBCS Space ?
32556     jne     short _$P_Chk_Delim01 ;AN000;
32557     mov     al,_$P_Space      ;AN000;
32558     inc     si                ;AN000; make si point to next character
32559     cmp     al,al             ;AN000; Set ZF
32560     jmp     short _$P_Chk_Delim_Exit ;AN000;
32561 %endif
32562 _$P_Chk_Delim01:
32563     cmp     byte [es:di-_$P_PARAMS_Blk.Num_Extra],_$P_I_Have_Delim
32564             ;AN000; delimiter character specified ?
32565     jnb     short _$P_Chk_Delim_Exit ;AN000;
32566     ;xor     cx,cx             ;AN000;
32567     ;xor     ch,ch
32568     ;mov     cl,[es:di+3]
32569     ;mov     cl,[es:di+_$P_PARAMS_Blk.Len_Extra_Delim]
32570             ;AN000; get length of delimiter list
32571     ;or     cx,cx             ;AN000; No extra Delim character ?
32572     ;jz      short _$P_Chk_Delim_NZ ;AN000;
32573     ; 08/07/2023
32574     jcxz    _$P_Chk_Delim_NZ
32575     mov     bx,_$P_Len_PARAMS-1 ; 3;AN000; set bx to 1st extra delimiter

```

```

32586      _$P_Chk_Delim_Loop:                                ;AN000;
32587      inc     bx                                         ;AN000;
32588      cmp     al,[es:bx+di]                               ;AN000; Check extra Delim character
32589      je      short _$P_Chk_Delim_Exit0 ;AN000;
32590
32591      loop     _$P_Chk_Delim_Loop      ;AN000; examine all extra delimiter
32592
32593      _$P_Chk_Delim_NZ:                                     ;AN000;
32594      ;cmp     al,_$P_Space                                ;AN000; reset ZF
32595      ; 08/07/2023
32596      ; cx=0 here
32597      inc     cx ; cx=1, zf=0
32598      _$P_Chk_Delim_Exit:                                   ;AN000;
32599      _$P_ChkDfin:                                         ;AN000;
32600      pop     cx                                         ;AN000;
32601      pop     bx                                         ;AN000;
32602      retn                                           ;AN000;
32603
32604      _$P_Chk_Delim_Exit0:                                   ;AN000;
32605      mov     [cs:_$P_Terminator],al ;AC034; keep terminated delimiter
32606      test    byte [cs:_$P_Flags2],_$P_equ ;AN027;AC034;; if terminating a key=
32607      jnz     short _$P_No_Set_Extra ;AN027; then do not set the EXTRA bit
32608
32609      or      byte [cs:_$P_Flags2],_$P_Extra
32610      ;AC034; flag terminated extra delimiter or comma
32611
32612      _$P_No_Set_Extra:                                     ;AN027;
32613      cmp     al,al                                       ;AN000; set ZF
32614      jmp     short _$P_Chk_Delim_Exit ;AN000;
32615
32616      ;*****
32617      ; _$P_Chk_Switch;
32618      ;
32619      ; Function: Check if AL is the switch character not in first position of
32620      ; _$P_STRING_BUF
32621      ;
32622      ; Input:  AL = character code
32623      ;         BX = current pointer within _$P_String_Buf
32624      ;         SI =>next char on command line (following the one in AL)
32625      ;
32626      ; Output: CF = 1 (set)if AL is switch character, and not in first
32627      ;         position, and has no chance of being part of a date string,
32628      ;         i.e. should be treated as a delimiter.
32629      ;
32630      ;         CF = 0 (reset, cleared) if AL is not a switch char, is in the first
32631      ;         position, or is a slash but may be part of a date string, i.e.
32632      ;         should not be treated as a delimiter.
32633      ;
32634      ; Vars:  _$P_Terminator(W)
32635      ;
32636      ; Use:    _$P_0099
32637      ;*****
32638
32639      _$P_Chk_Switch:
32640      ;;lea     bp,[cs:_$P_STRING_BUF];AN020;AC034
32641      ;;lea     bp,[_$P_STRING_BUF] ;BP=OFFSET of _$P_String_Buf even in group addressing
32642      ; 08/07/2023
32643      mov     bp,_$P_STRING_BUF
32644
32645      ; .IF <BX NE BP> THEN ;AN020;IF not first char THEN
32646      cmp     bx,bp                                       ;AN000;
32647      je      short _$P_STRUC_L2 ;AN000;
32648
32649      ; .IF <AL EQ _$P_Switch> THEN ;AN020;otherwise see if a slash
32650      cmp     al,_$P_Switch ;AN000;
32651      jne     short _$P_STRUC_L5 ;AN000;
32652
32653      stc                                           ;AN020;not in first position and is slash
32654      jmp     short _$P_STRUC_L1 ;AN000;
32655      ; 12/12/2022
32656      retn
32657
32658      ; 12/12/2022
32659      _$P_STRUC_L5:                                       ;AN000;
32660      ; CLC                                           ;AN020;not a slash
32661      ; .ENDIF                                         ;AN020;
32662      ; .ELSE                                           ;AN020;is first char in the buffer, ZF=0
32663      jmp     short _$P_STRUC_L1 ;AN000;
32664
32665      _$P_STRUC_L2:                                       ;AN000;
32666      ; .IF <AL EQ _$P_Switch> THEN ;AN020;
32667      cmp     al,_$P_Switch ;AN000;
32668      jne     short _$P_STRUC_L12 ;AN000;
32669
32670      or      byte [cs:_$P_Flags2],_$P_SW ;AN020 ;AC034;;could be valid switch, first char and is slash
32671      ; .ENDIF                                         ;AN020;
32672
32673      ; 12/12/2022
32674      ; cf=0
32675      ;retn
32676
32677      _$P_STRUC_L5:
32678      ; 12/12/2022
32679      _$P_STRUC_L12:                                       ;AN000;
32680      cll                                           ;AN020;CF=0 indicating first char
32681      ; .ENDIF                                         ;AN020;
32682      _$P_STRUC_L1:                                       ;AN000;
32683      retn                                           ;AN000;
32684
32685      ;*****
32686      ; _$P_Chk_DBCS:
32687      ;
32688      ; Function: Check if a specified byte is in ranges of the DBCS lead bytes
32689      ;
32690      ; Input:
32691      ;     AL = Code to be examined
32692      ;
32693      ; Output:
32694      ;     If CF is on then a lead byte of DBCS
32695      ;
32696      ; Use: INT 21h w/AH=63
32697      ;
32698      ; Vars:  _$P_DBCSEV_Seg(RW), _$P_DBCSEV_Off(RW)
32699      ;*****
32700
32701      _$P_Chk_DBCS:
32702      push    ds                                         ;AN000;
32703      push    si                                         ;AN000;
32704      push    bx                                         ;AN000; (tm11)
32705      ;cmp     word [cs:_$P_DBCSEV_SEG],0 ;AC034; ALREADY SET ?
32706      ;jne     short _$P_DBCS00 ;AN000;
32707      ; 08/07/2023
32708      mov     si,[cs:_$P_DBCSEV_SEG]
32709      and     si,si ; 0 ?
32710      jnz     short _$P_DBCS00 ; already set

```

```

32710 0000218B 50      push    ax                ;AN000;
32711 0000218C 1E      push    ds                ;AN000; (tm11)
32712 0000218D 51      push    cx                ;AN000;
32713 0000218E 52      push    dx                ;AN000;
32714 0000218F 57      push    di                ;AN000;
32715 00002190 55      push    bp                ;AN000;
32716 00002191 06      push    es                ;AN000;
32717                ; si = 0 ; 08/07/2023
32718                ;xor    si,si                ;AN000;
32719 00002192 8EDE      mov     ds,si ; 0                ;AN000;
32720 00002194 B80063    mov     ax,_$P_DOS_GetEV ; 6300h ;AN000; GET DBCS EV CALL
32721 00002197 CD21      int     21h                ;AN000;
32722                ; DOS - 3.2+ only - GET DOUBLE BYTE CHARACTER SET LEAD TABLE
32723 00002199 8CDB      mov     bx,ds                ;AN000; (tm11)
32724 0000219B 09DB      or      bx,bx                ;AN000; (tm11)
32725 0000219D 07      pop     es                ;AN000;
32726 0000219E 5D      pop     bp                ;AN000;
32727 0000219F 5F      pop     di                ;AN000;
32728 000021A0 5A      pop     dx                ;AN000;
32729 000021A1 59      pop     cx                ;AN000;
32730 000021A2 1F      pop     ds                ;AN000; (tm11)
32731 000021A3 58      pop     ax                ;AN000;
32732 000021A4 741E      jz      short _$P_NON_DBCS ;AN000;
32733                _$P_DBCS02:                ;AN000;
32734 000021A6 2E8936[7819]    mov     [cs:_$P_DBCSEV_OFF],si ;AC034; save EV offset
32735 000021AB 2E891E[7A19]    mov     [cs:_$P_DBCSEV_SEG],bx ;AC034; save EV segment (tm11)
32736                _$P_DBCS00:                ;AN000;
32737                ;mov     si,[cs:_$P_DBCSEV_OFF] ;AC034; load EV offset
32738                ;mov     ds,[cs:_$P_DBCSEV_SEG] ;AC034; and segment
32739                ; 08/07/2023
32740 000021B0 2EC536[7819]    lds     si,[cs:_$P_DBCSEV_OFF]
32741                _$P_DBCS_LOOP:                ;AN000;
32742 000021B5 833C00      cmp     word [si],0                ;AN000; zero vector ?
32743 000021B8 740A      je      short _$P_NON_DBCS ;AN000; then exit
32744 000021BA 3A04      cmp     al,[si]                ;AN000;
32745 000021BC 720A      jb      short _$P_DBCS01 ;AN000; Check if AL is in
32746 000021BE 3A4401      cmp     al,[si+1]                ;AN000; range of
32747 000021C1 7705      ja      short _$P_DBCS01 ;AN000; the vector
32748 000021C3 F9      stc                        ;AN000; if yes, indicate DBCS and exit
32749                ; 04/02/2026
32750                ;jmp     short _$P_DBCS_EXIT ;AN000;
32751
32752                _$P_NON_DBCS:                ;AN000;
32753                ; 12/12/2022
32754                ; cf=0
32755                ;clc                        ;AN000; indicate SBCS
32756                _$P_DBCS_EXIT:                ;AN000;
32757 000021C4 5B      pop     bx                ;AN000; (tm11)
32758 000021C5 5E      pop     si                ;AN000;
32759 000021C6 1F      pop     ds                ;AN000;
32760 000021C7 C3      retn                     ;AN000;
32761
32762                _$P_DBCS01:                ;AN000;
32763 000021C8 46      inc     si                ;AC035; add '2' to
32764 000021C9 46      inc     si                ;AC035; SI reg
32765                ;AN000; get next vector
32766 000021CA EBE9      jmp     short _$P_DBCS_LOOP ;AN000; loop until zero vector found
32767
32768                ; SYSCONF.ASM - MSDOS 6.0 - 1991
32769                ; =====
32770                ; 27/03/2019 - Retro DOS v4.0
32771
32772                ;control block definitions for parser.
32773                ;-----
32774                ; buffer = [n | n,m] {/e}
32775
32776                ; 30/03/2019
32777
32778                struc p_parms
32779 00000000 ????      resw    1                ; dw ?
32780 00000002 ??      resb    1                ; db 1 ; an extra delimiter list
32781 00000003 ??      resb    1                ; db 1 ; length is 1
32782 00000004 ??      resb    1                ; db ',' ; delimiter
32783                .size:
32784                endstruc
32785
32786                struc p_pos
32787 00000000 ????      resw    1                ; dw ? ; numeric value??
32788 00000002 ????      resw    1                ; dw ? ; function
32789 00000004 ????      resw    1                ; dw ? ; result value buffer
32790
32791                ; note: by defining result_val before this structure, we could remove
32792                ; the "result_val" from every structure invocation
32793
32794 00000006 ????      resw    1                ; dw ? ; value list
32795 00000008 ??      resb    1                ; db 0 ; no switches/keywords
32796                .size:
32797                endstruc
32798
32799                struc p_range
32800 00000000 ??      resb    1                ; db 1 ; range definition
32801 00000001 ??      resb    1                ; db 1 ; 1 definition of range
32802 00000002 ??      resb    1                ; db 1 ; item tag for this range
32803 00000003 ???????? resd    1                ; dd ? ; numeric min
32804 00000007 ???????? resd    1                ; dd ? ; numeric max
32805                .size:
32806                endstruc
32807
32808                ;-----
32809
32810                ; 26/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
32811                ; (SYSINIT:1F48h)
32812
32813                ; 08/07/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32814                ; MSDOS 6.21 IO.SYS - SYSINIT:2083h
32815                ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:251Dh) ; (Retro DOS v5.0)
32816
32817                ; buffer = [n | n,m] {/e}
32818
32819                ;buf_parms p_parms <buf_parmsx>
32820                buf_parms:
32821 000021CC [D121]    dw      buf_parmsx
32822 000021CE 01      db      1                ; an extra delimiter list
32823 000021CF 01      db      1                ; length is 1
32824 000021D0 3B      db      ','                ; delimiter
32825
32826                buf_parmsx:
32827 000021D1 0102[DB21][EF21]    dw      201h,buf_pos1,buf_pos2; min 1, max 2 positionals
32828 000021D7 01      db      1                ; one switch
32829 000021D8 [0322]    dw      sw_x_ctr1
32830 000021DA 00      db      0                ; no keywords
32831
32832                ;buf_pos1 p_pos <8000h,0,result_val,buf_range_1> ; numeric
32833                buf_pos1:

```

```

32834 000021DB 0080          dw      8000h ; numeric value??
32835 000021DD 0000          dw      0 ; function
32836 000021DF [1522]        dw      result_val ; result value buffer
32837 000021E1 [E421]        dw      buf_range_1 ; value list
32838 000021E3 00            db      0 ; no switches/keywords
32839
32840          ;buf_range_1 p_range <,,,1,99> ; M050
32841 buf_range_1:
32842 000021E4 01            db      1 ; range definition
32843 000021E5 01            db      1 ; 1 definition of range
32844 000021E6 01            db      1 ; item tag for this range
32845 000021E7 01000000      dd      1 ; numeric min
32846 000021EB 63000000      dd      99 ; numeric max
32847
32848          ;buf_pos2 p_pos <8001h,0,result_val,buf_range_2> ; optional num.
32849 buf_pos2:
32850 000021EF 0180          dw      8001h
32851 000021F1 0000          dw      0
32852 000021F3 [1522]        dw      result_val
32853 000021F5 [F821]        dw      buf_range_2
32854 000021F7 00            db      0
32855
32856          ;buf_range_2 p_range <,,,0,8>
32857 buf_range_2:
32858 000021F8 01            db      1
32859 000021F9 01            db      1
32860 000021FA 01            db      1
32861 000021FB 00000000      dd      0
32862 000021FF 08000000      dd      8
32863
32864          ;sw_x_ctrl p_pos <0,0,result_val,noval,1> ; followed by one switch
32865 sw_x_ctrl:
32866 00002203 0000          dw      0
32867 00002205 0000          dw      0
32868 00002207 [1522]        dw      result_val
32869 00002209 [1422]        dw      noval
32870 0000220B 01            db      1 ; 1 switch
32871
32872          switch_x:
32873 0000220C 2F5800          db      '/x',0 ; M016
32874
32875          p_buffers:
32876 0000220F 0000          dw      0 ; local variables
32877          p_h_buffers:
32878 00002211 0000          dw      0
32879          ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32880          p_buffer_slash_x:
32881 00002213 00            db      0 ; 31/03/2019
32882
32883          ;-- common definitions -----
32884          noval: db 0
32885 00002214 00
32886          result_val: ;label byte
32887 00002215 00            db      0 ; type returned
32888          result_val_itag:
32889 00002216 00            db      0 ; item tag returned
32890          result_val_swoff:
32891 00002217 0000          dw      0 ; es:offset of the switch defined
32892          rv_byte: ;label byte
32893 00002219 00000000      dd      0 ; value if number,or seg:offset to string.
32894          rv_dword:
32895          ;-----
32896          ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
32897          ; (SYSINIT:1F99h)
32898
32899          ; break = [ on | off ]
32900
32901          ;brk_parms p_parms <brk_parmsx>
32902          brk_parms:
32903          dw      brk_parmsx
32904 0000221D [2222]          db      1 ; an extra delimiter list
32905 0000221F 01            db      1 ; length is 1
32906 00002220 01            db      ',' ; delimiter
32907 00002221 3B
32908
32909          brk_parmsx:
32910          dw      101h,brk_pos ; min,max = 1 positional
32911 00002222 0101[2822]      db      0 ; no switches
32912 00002226 00            db      0 ; no keywords
32913 00002227 00
32914
32915          ;brk_pos p_pos <2000h,0,result_val,on_off_string> ; simple string
32916          brk_pos:
32917 00002228 0020          dw      2000h
32918 0000222A 0000          dw      0
32919 0000222C [1522]        dw      result_val
32920 0000222E [3122]        dw      on_off_string
32921 00002230 00            db      0
32922
32923          on_off_string: ;label byte
32924 00002231 03            db      3 ; signals that there is a string choice
32925 00002232 00            db      0 ; no range definition
32926 00002233 00            db      0 ; no numeric values choice
32927 00002234 02            db      2 ; 2 strings for choice
32928 00002235 01            db      1 ; the 1st string tag
32929 00002236 [3B22]        dw      on_string
32930 00002238 02            db      2 ; the 2nd string tag
32931 00002239 [3E22]        dw      off_string
32932
32933          on_string:
32934 0000223B 4F4E00          db      "ON",0
32935          off_string:
32936 0000223E 4F464600        db      "OFF",0
32937
32938          p_ctrl_break:
32939 00002242 00            db      0 ; local variable
32940
32941          ;-----
32942          ; 27/10/2022
32943
32944          ; country = n {m {path}}
32945          ; or
32946          ; country = n,,path
32947
32948          ;cntry_parms p_parms <cntry_parmsx>
32949          cntry_parms:
32950          dw      cntry_parmsx
32951 00002243 [4822]          db      1
32952 00002245 01            db      1
32953 00002246 01            db      1
32954 00002247 3B            db      ','
32955
32956          cntry_parmsx:
32957 00002248 0103[5222][6622]- dw      301h,cntry_pos1,cntry_pos2,cntry_pos3 ; min 1, max 3 pos.

```

```

32957 0000224E [6F22]
32958 00002250 00          db      0          ; no switches
32959 00002251 00          db      0          ; no keywords
32960
32961 ;cntry_pos1 p_pos <8000h,0,result_val,cc_range> ; numeric value
32962 cntry_pos1:
32963 00002252 0080          dw      8000h
32964 00002254 0000          dw      0
32965 00002256 [1522]      dw      result_val
32966 00002258 [5B22]      dw      cc_range
32967 0000225A 00          db      0
32968
32969 ;cc_range p_range <,,,1,999>
32970 cc_range:
32971 0000225B 01          db      1
32972 0000225C 01          db      1
32973 0000225D 01          db      1
32974 0000225E 01000000      dd      1
32975 00002262 E7030000      dd      999
32976
32977 ;cntry_pos2 p_pos <8001h,0,result_val,cc_range> ; optional num.
32978 cntry_pos2:
32979 00002266 0180          dw      8001h
32980 00002268 0000          dw      0
32981 0000226A [1522]      dw      result_val
32982 0000226C [5B22]      dw      cc_range
32983 0000226E 00          db      0
32984
32985 ;cntry_pos3 p_pos <201h,0,result_val,noval> ; optional filespec
32986 cntry_pos3:
32987 0000226F 0102          dw      201h
32988 00002271 0000          dw      0
32989 00002273 [1522]      dw      result_val
32990 00002275 [1422]      dw      noval
32991 00002277 00          db      0
32992
32993 p_cntry_code:
32994 00002278 0000          dw      0          ; local variable
32995
32996 p_code_page:
32997 0000227A 0000          dw      0          ; local variable
32998
32999 ;-----
33000 ; 27/10/2022
33001
33002 ; files = n
33003
33004 ;files_parms p_parms <files_parmsx>
33005 files_parms:
33006 0000227C [8122]      dw      files_parmsx
33007 0000227E 01          db      1
33008 0000227F 01          db      1
33009 00002280 3B          db      ','
33010
33011 files_parmsx:
33012 00002281 0101[8722]      dw      101h,files_pos ; min,max 1 positional
33013 00002285 00          db      0          ; no switches
33014 00002286 00          db      0          ; no keywords
33015
33016 ;files_pos p_pos <8000h,0,result_val,files_range,0> ; numeric value
33017 files_pos:
33018 00002287 0080          dw      8000h
33019 00002289 0000          dw      0
33020 0000228B [1522]      dw      result_val
33021 0000228D [9022]      dw      files_range
33022 0000228F 00          db      0
33023
33024 ;files_range p_range <,,,8,255>
33025 files_range:
33026 00002290 01          db      1
33027 00002291 01          db      1
33028 00002292 01          db      1
33029 00002293 08000000      dd      8
33030 00002297 FF000000      dd      255
33031
33032 p_files:
33033 0000229B 00          db      0          ; local variable
33034
33035 ;-----
33036 ; 27/10/2022
33037
33038 ; fcbs = n,m
33039
33040 ;fcbs_parms p_parms <fcbs_parmsx>
33041 fcbs_parms:
33042 0000229C [A122]      dw      fcbs_parmsx
33043 0000229E 01          db      1
33044 0000229F 01          db      1
33045 000022A0 3B          db      ','
33046
33047 fcbs_parmsx:
33048 000022A1 0102[A922][BD22] dw      201h,fcbs_pos_1,fcbs_pos_2 ; min,max = 2 positional
33049 000022A7 00          db      0          ; no switches
33050 000022A8 00          db      0          ; no keywords
33051
33052 ;fcbs_pos_1 p_pos <8000h,0,result_val,fcbs_range> ; numeric value
33053 fcbs_pos_1:
33054 000022A9 0080          dw      8000h
33055 000022AB 0000          dw      0
33056 000022AD [1522]      dw      result_val
33057 000022AF [B222]      dw      fcbs_range
33058 000022B1 00          db      0
33059
33060 ;fcbs_range p_range <,,,1,255>
33061 fcbs_range:
33062 000022B2 01          db      1
33063 000022B3 01          db      1
33064 000022B4 01          db      1
33065 000022B5 01000000      dd      1
33066 000022B9 FF000000      dd      255
33067
33068 ;fcbs_pos_2 p_pos <8000h,0,result_val,fcbs_keep_range> ; numeric value
33069 fcbs_pos_2:
33070 000022BD 0080          dw      8000h
33071 000022BF 0000          dw      0
33072 000022C1 [1522]      dw      result_val
33073 000022C3 [C622]      dw      fcbs_keep_range
33074 000022C5 00          db      0
33075
33076 ;fcbs_keep_range p_range <,,,0,255>
33077 fcbs_keep_range:
33078 000022C6 01          db      1
33079 000022C7 01          db      1

```

```

33081 000022C8 01          db      1
33082 000022C9 00000000    dd      0
33083 000022CD FF000000    dd      255
33084
33085 000022D1 00          p_fcbs:   db      0          ; local variable
33086 000022D2 00          p_keep:   db      0          ; local variable
33087
33088 ;-----
33089
33090          ; 27/10/2022
33091
33092          ; lastdrive = x
33093
33094          ; ldrv_parms p_parms <ldrv_parmsx>
33095          ldrv_parms:
33096 000022D3 [D822]        dw      ldrv_parmsx
33097 000022D5 01          db      1
33098 000022D6 01          db      1
33099 000022D7 3B          db      ','
33100
33101          ldrv_parmsx:
33102 000022D8 0101[DE22]    dw      101h,ldrv_pos ; min,max = 1 positional
33103 000022DC 00          db      0          ; no switches
33104 000022DD 00          db      0          ; no keywords
33105
33106          ; ldrv_pos p_pos <110h,10h,result_val,noval> ; drive only, ignore colon
33107          ldrv_pos:
33108 000022DE 1001        dw      110h
33109 000022E0 1000        dw      10h
33110 000022E2 [1522]    dw      result_val
33111 000022E4 [1422]    dw      noval
33112 000022E6 00          db      0
33113
33114 000022E7 00          p_ldrv:   db      0          ; local variable
33115
33116 ;-----
33117
33118          ; 27/10/2022
33119
33120          ; stacks = n,m
33121
33122          ; stks_parms p_parms <stks_parmsx>
33123          stks_parms:
33124 000022E8 [ED22]        dw      stks_parmsx
33125 000022EA 01          db      1
33126 000022EB 01          db      1
33127 000022EC 3B          db      ','
33128
33129          stks_parmsx:
33130 000022ED 0202[F522][0923] dw      202h,stks_pos_1,stks_pos_2 ; min,max = 2 positionals
33131 000022F3 00          db      0          ; no switches
33132 000022F4 00          db      0          ; no keywords
33133
33134          ; stks_pos_1 p_pos <8000h,0,result_val,stks_range> ; numeric value
33135          stks_pos_1:
33136 000022F5 0080        dw      8000h
33137 000022F7 0000        dw      0
33138 000022F9 [1522]    dw      result_val
33139 000022FB [FE22]    dw      stks_range
33140 000022FD 00          db      0
33141
33142          ; stks_range p_range <,,,0,64>
33143          stks_range:
33144 000022FE 01          db      1
33145 000022FF 01          db      1
33146 00002300 01          db      1
33147 00002301 00000000    dd      0
33148 00002305 40000000    dd      64
33149
33150          ; stks_pos_2 p_pos <8000h,0,result_val,stk_size_range> ; numeric value
33151          stks_pos_2:
33152 00002309 0080        dw      8000h
33153 0000230B 0000        dw      0
33154 0000230D [1522]    dw      result_val
33155 0000230F [1223]    dw      stk_size_range
33156 00002311 00          db      0
33157
33158          ; stk_size_range p_range <,,,0,512>
33159          stk_size_range:
33160 00002312 01          db      1
33161 00002313 01          db      1
33162 00002314 01          db      1
33163 00002315 00000000    dd      0
33164 00002319 00020000    dd      512
33165
33166          p_stack_count:
33167 0000231D 0000        dw      0          ; local variable
33168
33169          p_stack_size:
33170 0000231F 0000        dw      0          ; local variable
33171
33172 ;-----
33173
33174          ; 27/10/2022
33175
33176          ; multitrack = [ on | off ]
33177
33178          ; mtrk_parms p_parms <mtrk_parmsx>
33179          mtrk_parms:
33180 00002321 [2623]        dw      mtrk_parmsx
33181 00002323 01          db      1
33182 00002324 01          db      1
33183 00002325 3B          db      ','
33184
33185          mtrk_parmsx:
33186 00002326 0101[2C23]    dw      101h,mtrk_pos ; min,max = 1 positional
33187 0000232A 00          db      0          ; no switches
33188 0000232B 00          db      0          ; no keywords
33189
33190          ; mtrk_pos p_pos <2000h,0,result_val,on_off_string> ; simple string
33191          mtrk_pos:
33192 0000232C 0020        dw      2000h
33193 0000232E 0000        dw      0
33194 00002330 [1522]    dw      result_val
33195 00002332 [3122]    dw      on_off_string
33196 00002334 00          db      0
33197
33198          p_mtrk:   db      0          ; local variable
33199
33200 ;-----
33201          ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
33202          ; (SYSINIT:20B2h)
33203
33204          ; switches=/k

```



```

33205
33206
33207 ;swit_parms p_parms <swit_parmsx>
33208 00002336 [3B23] swit_parms:
33209 00002338 01 dw swit_parmsx
33210 00002339 01 db 1
33211 0000233A 3B db 1
33212
33213
33214 0000233B 0000 swit_parmsx:
33215 dw 0 ; no positionals
33216 ; 08/07/2023 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS, SYSINIT)
33217 ;db 5 ; # of switches
33218 0000233D 06 dw 6
33219 ; 27/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS, SYSINIT)
33220 ;db 3
33221 0000233E [4B23] dw swit_k_ctrl ; /k control
33222 ; 01/01/2023 - Retro DOS v4.2 ; *
33223 00002340 [5723] dw swit_n_ctrl ; * ; /n control (for MULTI_CONFIG only)
33224 00002342 [6323] dw swit_f_ctrl ; * ; /f control (for MULTI_CONFIG only)
33225 00002344 [6F23] dw swit_t_ctrl ; /t control
33226 00002346 [7B23] dw swit_w_ctrl ; /w control
33227 ; 14/04/2024 - Retro DOS v5.0 ; **
33228 00002348 [8723] dw swit_i_ctrl ; /i control
33229 0000234A 00 db 0 ; no keywords
33230
33231 ;swit_k_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows
33232 swit_k_ctrl:
33233 0000234B 00000000[1522]- dw 0,0,result_val,noval
33234 00002351 [1422]
33235 00002353 01 db 1
33236 00002354 2F4B00 swit_k: db '/K',0
33237
33238 ; 01/01/2023 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS)
33239 ; (SYSINIT:220Ch) ; *
33240
33241 ; 27/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS, SYSINIT)
33242 ;
33243 ;swit_n_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows
33244 00002357 00000000[1522]- swit_n_ctrl: ; *
33245 0000235D [1422] dw 0,0,result_val,noval
33246 0000235F 01 db 1
33247 00002360 2F4E00 swit_n: db '/N',0
33248
33249 ;swit_f_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows
33250 00002363 00000000[1522]- swit_f_ctrl: ; *
33251 00002369 [1422] dw 0,0,result_val,noval
33252 0000236B 01 db 1
33253 0000236C 2F4600 swit_f: db '/F',0
33254
33255 ; 27/10/2022
33256
33257 ;swit_t_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows M059
33258 0000236F 00000000[1522]- swit_t_ctrl:
33259 00002375 [1422] dw 0,0,result_val,noval
33260 00002377 01 db 1
33261 00002378 2F5400 swit_t: db '/T',0
33262 ;swit_w_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows M059
33263 0000237B 00000000[1522]- swit_w_ctrl: M063
33264 00002381 [1422] dw 0,0,result_val,noval
33265 00002383 01 db 1
33266 00002384 2F5700 swit_w: db '/W',0 ; M063
33267
33268 ; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM, SYSINIT)
33269 ;;;
33270 ;swit_i_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows
33271 00002387 0000 swit_i_ctrl:
33272 00002389 0000 dw 0
33273 0000238B [1522] dw result_val
33274 0000238D [1422] dw noval
33275 0000238F 01 db 1
33276 00002390 2F4900 swit_i: db '/I',0
33277 ;;;
33278
33279 ; There doesn't need to be p_swit_n or p_swit_f because /N and /F are
33280 ; acted upon during MULTI_CONFIG processing; we only needed entries
33281 ; in the above table to prevent the parsing code from complaining about them
33282
33283 00002393 00 p_swit_k: db 0 ; local variable
33284 00002394 00 p_swit_t: db 0 ; local variable M059
33285 00002395 00 p_swit_w: db 0 ; local variable M063
33286 ; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
33287 00002396 00 p_swit_i: db 0
33288
33289 ;-----
33290
33291 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
33292 ; (SYSINIT:20E8h)
33293
33294 ; DOS = [ high | low ]
33295
33296 ;dos_parms p_parms <dos_parmsx>
33297 dos_parms:
33298 00002397 [9C23] dw dos_parmsx
33299 00002399 01 db 1
33300 0000239A 01 db 1
33301 0000239B 3B db 1
33302
33303 0000239C 01 dos_parmsx:
33304 0000239D 02 db 1 ; min parameters
33305 0000239E [A423] db 2 ; max parameters
33306 000023A0 [A423] dw dos_pos ;
33307 000023A2 00 dw dos_pos ;
33308 000023A3 00 db 0 ; no switches
33309 db 0 ; no keywords
33310
33311 ;dos_pos p_pos <2000h,0,result_val,dos_strings> ; simple string
33312 ; p_pos <2000h,0,result_val,dos_strings> ; simple string
33313 000023A4 00200000[1522]- dos_pos:
33314 000023AA [B623] dw 2000h,0,result_val,dos_strings
33315 000023AC 00 db 0
33316 000023AD 00200000[1522]- dw 2000h,0,result_val,dos_strings
33317 000023B3 [B623] db 0
33318 000023B5 00
33319 dos_strings: ;label byte
33320 000023B6 03 db 3 ; signals that there is a string choice
33321 000023B7 00 db 0 ; no range definition
33322 000023B8 00 db 0 ; no numeric values choice

```

```

33322 000023B9 04          db      4          ; 4 strings for choice
33323 000023BA 01          db      1          ; the 1st string tag
33324 000023BB [E423]     dw      hi_string
33325 000023BD 02          db      2          ; the 2nd string tag
33326 000023BE [E923]     dw      lo_string
33327 000023C0 03          db      3
33328 000023C1 [ED23]     dw      umb_string
33329 000023C3 04          db      4
33330 000023C4 [F123]     dw      noumb_string
33331
33332 ; 14/04/2024 - Retro DOS v5.0
33333 ; (PCDOS 7.1 IBMDOS.COM - SYSINIT:273Eh)
33334 ;;;
33335 dosdata_parms:
33336 000023C6 [CB23]     dw      dosdata_parmsx ; DOSDATA = UMB|NOUMB
33337 000023C8 01          db      1
33338 000023C9 01          db      1
33339 000023CA 3B          db      ','
33340 dosdata_parmsx:
33341 000023CB 01          db      1
33342 000023CC 01          db      1          ; min,max = 1 positional
33343 000023CD [D123]     dw      dosdata_pos
33344 000023CF 00          db      0          ; no switches
33345 000023D0 00          db      0          ; no keywords
33346
33347 ; dosdata_pos p_pos <2000h,0,result_val,dosdata_strings>
33348 dosdata_pos:
33349 000023D1 0020         dw      2000h          ; simple string
33350 000023D3 0000         dw      0
33351 000023D5 [1522]     dw      result_val
33352 000023D7 [DA23]     dw      dosdata_strings
33353 000023D9 00          db      0
33354 dosdata_strings:
33355 000023DA 03          db      3          ; signals that there is a string choice
33356 000023DB 00          db      0          ; no range definition
33357 000023DC 00          db      0          ; no numeric values choice
33358 000023DD 02          db      2          ; 2 strings for choice
33359 000023DE 01          db      1          ; the 1st string tag
33360 000023DF [ED23]     dw      umb_string          ; "UMB"
33361 000023E1 02          db      2          ; the 2nd string tag
33362 000023E2 [F123]     dw      noumb_string          ; "NOUMB"
33363 ;;;
33364
33365 000023E4 4849474800     hi_string: db      "HIGH",0
33366 000023E9 4C4F5700     lo_string: db      "LOW",0
33367 000023ED 554D4200     umb_string: db      "UMB",0
33368 000023F1 4E4F554D4200 noumb_string: db      "NOUMB",0
33369
33370 p_dos_hi:
33371 000023F7 00          db      0          ; local variable
33372 ; BUGBUG : I dont know whether PARSER uses
33373 ; this variable or not
33374 ; 14/04/2024 (PCDOS 7.1 IBMBIO.COM)
33375 000023F8 00          db      0
33376
33377 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33378 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
33379 ;%if 0
33380
33381 ;***** RICHID ****
33382
33383 ;include highvar.inc ; devicehigh variables (used by loadhigh also)
33384
33385 ; 30/03/2019 - Retro DOS v4.0
33386 -----
33387
33388 ; Module: HIGHVAR.INC - Data common to LOADHIGH and DEVICEHIGH, res seg
33389 ;
33390 ; Date: May 14, 1992
33391 ;
33392 ;*****
33393 ;
33394 ; Modification log:
33395 ;
33396 ; DATE WHO DESCRIPTION
33397 ; -----
33398 ; 05/14/92 t-richj Original
33399 ; 06/21/92 t-richj Final revisions before check-in
33400 ;
33401 ;*****
33402 ;
33403 ; There are two primary definitions which need to be made, selectively, before
33404 ; this include file should be used. These are:
33405 ; HV_Extern - If this has been defined, variables for this module will be
33406 ; declared as external. Otherwise, variables will be declared
33407 ; public, as well as defined, here. LoadHigh declares HV_Extern
33408 ; in stub.asm and loadhi.asm, and does not declare it in
33409 ; rdata.asm... DeviceHigh does not declare HV_Extern anywhere
33410 ; (as only one module, sysconf.asm, includes this file).
33411 ; HV_LoadHigh - This should be defined when this module is going into
33412 ; command.com, for LoadHigh. All of loadhi.asm, stub.asm and
33413 ; rdata.asm define this, while io.sys' sysconf.asm does not.
33414 ;
33415 ;*****
33416 ;
33417 ; To keep track of which UMBs were specified on the DH/LH command lines, and
33418 ; to keep track of the minimum sizes given for each, there're two arrays kept
33419 ; in { IO.SYS: sysinitseg / COMMAND.COM: DATARES }... each is MAXUMB elements
33420 ; big. 16 should be around 14 too many for most users, so there's no expected
33421 ; space problem (it's just such a nice round number, eh?).
33422
33423 MAXUMB equ 16
33424
33425 ; Memory elements owned by the system are marked as PSP address 8 in both the
33426 ; USA and Japan; Japanese systems also use 9 under more bizarre conditions.
33427
33428 FreePSPOwner equ 0 ; Free MCBs all have an owner PSP address of 0
33429 SystemPSPOwner equ 8
33430 JapanPSPOwner equ 9
33431
33432 ; for LoadHigh and DeviceHigh:
33433 ;
33434 ; fInHigh - Is set to 1 during HideUMBs(), and back to zero in
33435 ; UnHideUMBs().
33436 ; fUmbTiny - Is set to 1 iff the user has specified /S on the command-
33437 ; line.
33438 ; SegLoad - Segment address for first UMB specified; set automatically.
33439 ; UmbLoad - The load UMB number; for example, this is 3 if the user has
33440 ; given a command-line like "/L:3,500;4"
33441 ; Umbused - An array of characters, each of which is 1 iff the UMB
33442 ; matching its index number was specified on the command-line;
33443 ; for example, after "/L:3,500;4;7", Umbused[3], [4] and [7]
33444 ; will be set to 1. All others will be set to 0.
33445 ; UmbSize - An array of words, each of which is interpreted as a size

```



```

33446 ; specified by the user for a UMB (in the above example, all
33447 ; elements would be zero save UmbSize[3], which would be 500.
33448 ; fm_umb - Set to the old UMB link-state (0x80 or 0x00)
33449 ; fm_strat - Set to the old memory-allocation strategy (0$00000???)
33450 ; fm_argc - Number of arguments received by ParseVar() (see ParseVar()
33451 ; for details).
33452
33453 000023F9 00 fInHigh: db 0
33454 000023FA 00 fUmbTiny: db 0
33455 000023FB 0000 SegLoad: dw 0
33456 000023FD 00 UmbLoad: db 0
33457 000023FE 00<rep 10h> UmbUsed: times MAXUMB db 0 ; times 16 db 0 ; db 16 dup(?)
33458 0000240E 0000<rep 10h> UmbSize: times MAXUMB dw 0 ; times 16 dw 0 ; dw 16 dup(?)
33459 0000242E 00 fm_umb: db 0
33460 0000242F 00 fm_strat: db 0
33461 00002430 00 fm_argc: db 0
33462
33463 ; UmbLoad is set to UNSPECIFIED, below, until /L:umb is read; at which point
33464 ; UmbLoad is set to the UMB number given.
33465
33466 UNSPECIFIED equ -1
33467
33468 ;%endif ; 27/10/2022
33469
33470 ;***** RICHID ****
33471
33472 ; 30/03/2019 - Retro DOS v4.0 (MSDOS 6.0, SYSCONF.ASM)
33473 ; ((MSDOS 6.21 IO.SYS -> SYNINIT:22BAh))
33474
33475 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
33476 ; (SYSCONF:212Bh)
33477
33478 ;public DevEntry
33479
33480 00002431 0000 DevSize: dw 0 ; size of the device driver being loaded (paras)
33481 00002433 0000 DevLoadAddr: dw 0 ; Mem addr where the device driver is 2 b loaded
33482 00002435 0000 DevLoadEnd: dw 0 ; MaxAddr to which device can be loaded
33483 00002437 00000000 DevEntry: dd 0 ; Entry point to the device driver
33484 0000243B 00000000 DevBrkAddr: dd 0 ; Break address of the device driver
33485 ; 30/12/2022
33486 ; 27/10/2022
33487 0000243F 00 ConvLoad: db 0 ; Use conventional (dos 5 -style) InitDevLoad?
33488
33489 00002440 00 DevUMB: db 0 ; byte indicating whether to load DDs in UMBS
33490 00002441 0000 DevUMBAddr: dw 0 ; current UMB used for loading devices (paras)
33491 00002443 0000 DevUMBSize: dw 0 ; Size of the current UMB being used (paras)
33492 00002445 0000 DevUMBFree: dw 0 ; Start of free area in the current UMB (paras)
33493
33494 00002447 00000000 DevXMSAddr: dd 0
33495
33496 0000244B 0000 DevExecAddr: dw 0 ; Device load address parameter to Exec call
33497 0000244D 0000 DevExecReloc: dw 0 ; Device load relocation factor
33498
33499 0000244F 00 DeviceHi: db 0 ; Flag indicating whether the current device
33500 ; is being loaded into UMB
33501 00002450 0000 DevSizeOption: dw 0 ; SIZE= option
33502
33503 00002452 00 Int12Lied: db 0 ; did we trap int 12 ?
33504 00002453 0000 OldInt12Mem: dw 0 ; value in 40:13h (int 12 ram)
33505 00002455 50524F544D414E24 ThreeComName: db 'PROTMAN$' ; 3Com Device name
33506
33507 0000245D 00 FirstUMBLinked: db 0
33508 0000245E 0000 DevDOSData: dw 0 ; segment of DOS Data
33509 00002460 00000000 DevCmdLine: dd 0 ; Current Command line
33510 00002464 00 DevSavedDelim: db 0 ; The delimiter which was replaced with null
33511 ; to use the file name in the command line
33512 ; 13/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
33513 ; ifndef dbldspace_hooks
33514 00002465 00 MagicHomeFlag: db 0 ; set non-zero when MagicDrv is finally placed
33515 ; endif
33516
33517 ; =====
33518
33519 ; 31/03/2019 - Retro DOS v4.0
33520
33521 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
33522 ; (SYSCONF:215Eh)
33523
33524 ; -----
33525 ;
33526 ; procedure : doconf
33527 ;
33528 ; Config file is parsed initially with this routine. For the
33529 ; Subsequent passes 'multi_pass' entry is used .
33530 ;
33531 ; -----
33532
33533 ; 27/10/2022
33534 doconf:
33535 00002466 0E push cs
33536 00002467 1F pop ds
33537
33538 00002468 B80037 mov ax,3700h
33539 ;mov ax,(CHAR_OPER<<8) ; get switch character
33540 0000246B CD21 int 21h
33541 0000246D 8816[B44B] mov [command_line+1],dl ; set in default command line
33542
33543 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33544 ; 27/10/2022
33545 ;ifndef MULTI_CONFIG
33546 ; mov [command_line-1],dl ; save default switchchar
33547 00002471 8816[B44B] mov [def_swchr],dl ; 31/03/2019
33548 ;endif ;MULTI_CONFIG
33549
33550 00002475 BA[CF4A] mov dx,config ;'\CONFIG.SYS' ;now pointing to file description
33551 00002478 B8003D mov ax,3D00h
33552 ;mov ax,OPEN<<8 ;open file "config.sys"
33553 0000247B F9 stc ;in case of int 24
33554 0000247C CD21 int 21h ;function request
33555 0000247E 7309 jnc short noprob ;brief opened okay
33556
33557 ; 31/12/2022
33558 ; 27/10/2022
33559 ;ifndef MULTI_CONFIG
33560 00002480 E8A019 call kbd_read ; we still want to give the guy
33561 ; a chance to select clean boot!
33562 ;endif ; (ie, no autoexec.bat processing)
33563 00002483 C606[CD02]0B mov byte [multi_pass_id],11 ; set it to unreasonable number
33564 00002488 C3 retn
33565 noprob:
33566 00002489 89C3 mov bx,ax ; File handle
33567 0000248B 31C9 xor cx,cx ; 0
33568 0000248D 31D2 xor dx,dx ; 0
33569 ;mov ax,4202h

```

```

33570 0000248F B80242      mov     ax,(LSEEK<<8)|2
33571 00002492 CD21        int     21h
33572 00002494 A3[5603]    mov     [count],ax          ; dx:ax = file size ; 08/09/2023
33573                                     ; 08/09/2023 - Erdogan Tan - Note:
33574 00002497 31D2      xor     dx,dx                ; dx already must be 0 here ; 08/09/2023
33575                                     ; I am not removing 'xor dx,dx' here
33576                                     ; for MSDOS compatibility.
33577                                     ; ((Also PCDOS 7.1 has 'xor dx,dx' here))
33578                                     ; (Error will be same if CONGIG.SYS file
33579                                     ; size > 64KB)
33580                                     ;mov     ax,4200h
33581 00002499 B80042      mov     ax,LSEEK<<8          ;reset pointer to beginning of file
33582 0000249C CD21        int     21h
33583
33584                                     ; 31/12/2022 - Retro DOS v4.2
33585 0000249E 8B16[A502]   mov     dx,[ALLOCLIM]        ;use current alloclim value
33586                                     ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33587                                     ;mov     dx,[top_of_cdss]
33588
33589 000024A2 A1[5603]    mov     ax,[count]
33590 000024A5 A3[D002]    mov     [config_size],ax      ;save the size of config.sys file.
33591 000024A8 E86AEE      call    ParaRound
33592 000024AB 29C2      sub     dx,ax
33593
33594                                     ; 31/12/2022
33595                                     ; 27/10/2022
33596                                     ;ifdef     MULTI_CONFIG
33597                                     ;
33598                                     ; The size of the CONFIG.SYS workspace (for recreating the in-memory
33599                                     ; CONFIG.SYS image, and later for building the initial environment) need
33600                                     ; not be any larger than CONFIG.SYS itself, EXCEPT for the fact that
33601                                     ; we (may) add a variable to the environment that does not explicitly appear
33602                                     ; in CONFIG.SYS, and that variable is CONFIG (as in CONFIG=COMMON).
33603                                     ; The default setting for CONFIG cannot result in more than 1 paragraph
33604                                     ; of extra space, so here we account for it (the worst case of course is
33605                                     ; when CONFIG.SYS is some very small size, like 0 -JTP)
33606                                     ;
33607 000024AD 4A          dec     dx                      ;reserve 1 additional paragraph
33608 000024AE 8916[6219]   mov     [config_wrkseg],dx      ;this is the segment to be used for
33609 000024B2 29C2      sub     dx,ax                      ;rebuilding the config.sys memory image
33610                                     ;;endif     MULTI_CONFIG
33611
33612 000024B4 83EA11      sub     dx,11h                ;room for header
33613
33614                                     ; 31/12/2022
33615 000024B7 8916[A502]   mov     [ALLOCLIM],dx          ;config starts here. new alloclim value.
33616 000024BB 8916[A302]   mov     [CONFBOT],dx
33617
33618                                     ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33619                                     ;mov     [top_of_cdss],dx
33620                                     ;call    TempCDS
33621                                     ; 31/12/2022
33622                                     ; 11/12/2022
33623                                     ; ds <> cs
33624                                     ;mov     dx,[cs:top_of_cdss]
33625
33626                                     ; 08/09/2023
33627                                     ; ds = cs
33628 000024BF 8B0E[5603]   mov     cx,[count]
33629
33630 000024C3 8EDA        mov     ds,dx
33631 000024C5 8EC2        mov     es,dx
33632
33633 000024C7 31D2      xor     dx,dx
33634                                     ; 08/09/2023
33635                                     ;mov     cx,[cs:count]
33636 000024C9 B43F        mov     ah,3Fh
33637                                     ;mov     ah,READ    ; 3Fh
33638 000024CB F9          stc                      ;in case of int 24
33639 000024CC CD21        int     21h                ;function request
33640 000024CE 9C          pushf
33641
33642                                     ; find the eof mark in the file. if present,then trim length.
33643
33644 000024CF 50          push     ax
33645 000024D0 57          push     di
33646 000024D1 51          push     cx
33647 000024D2 B01A        mov     al,1Ah                ; eof mark
33648 000024D4 89D7        mov     di,dx                ; point to buffer
33649 000024D6 E305        jcxz    puteo1                ; no chars
33650 000024D8 F2AE        repnz   scasb                ; find end
33651 000024DA 7501        jnz     short puteo1          ; none found and count exhausted
33652
33653                                     ; we found a 1a. back up
33654
33655 000024DC 4F          dec     di                      ; backup past 1Ah
33656
33657                                     ; just for the halibut, stick in an extra eol
33658
33659 puteo1:
33660 000024DD B00D        mov     al,cr ; 0Dh
33661 000024DF AA          stosb
33662 000024E0 B00A        mov     al,lf ;0Ah
33663 000024E2 AA          stosb
33664 000024E3 29D7        sub     di,dx                ; difference moved
33665                                     ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33666                                     ;mov     [cs:count],di          ; new count
33667
33668                                     ; 11/12/2022
33669                                     ; 31/03/2019 - Retro DOS v4.0
33670 000024E5 0E          push     cs
33671 000024E6 1F          pop      ds
33672
33673 000024E7 893E[5603]   mov     [count],di          ; new count
33674
33675 000024EB 59          pop     cx
33676 000024EC 5F          pop     di
33677 000024ED 58          pop     ax
33678
33679                                     ; 11/12/2022
33680                                     ; 27/10/2022
33681                                     ;push    cs
33682                                     ;pop     ds
33683
33684 000024EE 50          push     ax
33685                                     ;mov     ah,CLOSE
33686 000024EF B43E        mov     ah,3Eh
33687 000024F1 CD21        int     21h
33688 000024F3 58          pop     ax
33689 000024F4 9D          popf
33690 000024F5 7204        jc      short conferr        ;if not we've got a problem
33691 000024F7 39C1        cmp     cx,ax
33692 000024F9 742C        jz      short getcom        ;couldn't read the file
33693 conferr:

```

```

33694 000024FB BA[CF4A]      mov     dx,config      ;want to print config error
33695 000024FE E82525      call    badfil
33696                      ; 14/04/2024
33697                      endconv: ; 01/01/2023
33698 00002501 C3          retn
33699
33700                      ;-----
33701                      ; entry : multi_pass
33702                      ;
33703                      ;
33704                      ;         called to execute device=,install= commands
33705                      ;
33706                      ;-----
33707
33708                      ; 27/10/2022
33709 multi_pass:
33710 00002502 0E          push    cs
33711 00002503 1F          pop     ds
33712
33713 00002504 803E[CD02]0A   cmp     byte [multi_pass_id],10
33714                      ;jae_endconv:
33715 00002509 73F6          jae     short endconv      ; do nothing. just return.
33716
33717                      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33718 0000250B FF36[A302]   push    word [CONFBOT]
33719                      ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33720                      ;push word [top_of_cdss]
33721 0000250F 07          pop     es      ; es -> confbot
33722
33723 00002510 8836[5803]   mov     si,[org_count]
33724 00002514 8936[5603]   mov     [count],si      ; set count
33725 00002518 31F6          xor     si,si ; 0
33726 0000251A 8936[5A03]   mov     [chrptr],si      ; reset chrptr
33727 0000251E 8936[AF02]   mov     [linecount],si   ; reset linecount
33728
33729 00002522 E88822      call    getchr
33730 00002525 EB06          jmp     short conflp
33731
33732                      ; 14/04/2024
33733                      ; 01/01/2023
33734                      ;endconv:
33735                      ;retn
33736
33737 getcom:
33738                      ; 03/01/2023
33739                      ; ds = cs
33740 00002527 E8C016      call    organize      ; organize the file
33741 0000252A E88022      call    getchr
33742                      ;
33743 0000252D 72D2          jc      short endconv
33744
33745 0000252F FF06[AF02]   inc     word [linecount] ; increase linecount
33746
33747                      ; 08/09/2023
33748 00002533 30E4          xor     ah,ah ; 0
33749                      ;mov byte [multideviceflag],0      ; reset multideviceflag.
33750                      ;mov byte [setdevmarkflag],0      ; reset setdevmarkflag.
33751 00002535 8826[6619]   mov     [multideviceflag],ah ; 0
33752 00002539 8826[6919]   mov     [setdevmarkflag],ah ; 0
33753
33754 0000253D 3C0A          cmp     al,1f      ; linefeed?
33755 0000253F 7448          je      short blank_line ; then ignore this line.
33756
33757                      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33758                      ; (SYSINIT:23CCh)
33759                      ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33760                      ;%if 0
33761
33762                      ;ifdef MULTI_CONFIG
33763
33764                      ; If this is a genuine CONFIG.SYS command, then there should be a line
33765                      ; number immediately following it....
33766
33767 00002541 A2[6419]      mov     [config_cmd],al      ; save original command code
33768                      ;and al,NOT CONFIG_OPTION_QUERY
33769 00002544 247F          and     al,~CONFIG_OPTION_QUERY ; and al,7Fh
33770
33771                      ; 08/09/2023
33772 00002546 3826[6519]   cmp     [config_multi],ah ; 0
33773                      ;cmp byte [config_multi],0 ; is this a multi-config config.sys?
33774 0000254A 7427          je      short not_final ; no, line number is not embedded
33775
33776                      push    ax
33777 0000254C E85D22      call    getchr      ; ignore end-of-image errors,
33778 00002550 88C4          mov     ah,al      ; because if there's an error
33779 00002552 E85822      call    getchr      ; fetching the line number that's
33780 00002555 86C4          xchg    al,ah      ; supposed to be there, the next
33781 00002557 A3[AF02]   mov     [linecount],ax ; getchr call will get the same error
33782 0000255A 58          pop     ax
33783
33784                      ;
33785                      ; HACK: when 4DOS.COM is the shell and it doesn't have an environment from
33786                      ; which to obtain its original program name, it grovels through all of
33787                      ; memory to find the filename that was used to exec it; it wants to find
33788                      ; the SHELL= line in the in-memory copy of CONFIG.SYS, and it knows that
33789                      ; sysinit converts the SHELL= keyword to an 'S', so it expects to find an 'S'
33790                      ; immediately before the filename, but since we are now storing line # info
33791                      ; in the config.sys memory image, 4DOS fails to find the 'S' in the right
33792                      ; spot.
33793
33794                      ;
33795                      ; So, on the final pass of CONFIG.SYS, copy the command code (eg, 'S')
33796                      ; over the line number info, since we no longer need that info anyway. This
33797                      ; relies on the fact that getchr leaves ES:SI pointing to the last byte
33798                      ; retrieved.
33799                      ;
33800                      ;
33801 0000255B 803E[CD02]02   cmp     byte [multi_pass_id],2; final pass?
33802 00002560 7211          jb     short not_final ; no
33803                      ;test word [install_flag],have_install_cmd
33804 00002562 F606[CE02]01   test    byte [install_flag],have_install_cmd ; 1
33805 00002567 7407          jz     short final ; no install cmds, so yes it is
33806 00002569 803E[CD02]03   cmp     byte [multi_pass_id],3; final pass?
33807 0000256E 7203          jb     short not_final ; no
33808                      ;
33809                      ;
33810                      ;
33811                      ;
33812                      ;
33813 00002573 88C4          mov     ah,al
33814 00002575 E83522      call    getchr
33815 00002578 7314          jnc     short tryi
33816
33817 0000257A 803E[CD02]02   cmp     byte [multi_pass_id],2

```

```

33818             ;jae short jae_endconv      ; do not show badop again for multi_pass.
33819             ; 27/10/2022
33820 0000257F 7380             jnb short endconv
33821 00002581 E90009             jmp badop
33822
33823 coff:
33824             ; 11/12/2022
33825             ; ds = cs
33826             ;push cs
33827             ;pop ds
33828 00002584 E81D22             call newline
33829 00002587 EBA4             jmp short conflp ; 13/05/2019
33830
33831 blank_line:
33832 00002589 E82122             call getch
33833 0000258C EB9F             jmp short conflp
33834
33835             ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33836             ; 11/12/2022
33837             ; (there is not a jump or call to here from anywhere!)
33838             ;coff_p:
33839             ;push cs
33840             ;pop ds
33841
33842 ;to handle install= commands,we are going to use multi-pass.
33843 ;the first pass handles the other commands and only set install_flag when
33844 ;it finds any install command. the second pass will only handle the
33845 ;install= command.
33846
33847 -----
33848 ;install command
33849 -----
33850
33851             ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
33852             ; (SYSINIT:2250h)
33853 tryi:
33854 0000258E 803E[CD02]00         cmp byte [multi_pass_id],0; the initial pass for DOS=HI
33855 00002593 7503             jne short not_init_pass
33856 00002595 E97F01             jmp multi_try_doshi
33857 not_init_pass:
33858 00002598 803E[CD02]02         cmp byte [multi_pass_id],2; the second pass was for ifs=
33859             ; 11/12/2022
33860             ;je short multi_pass_coff2; now it is NOPS
33861 0000259D 74E5             je short coff
33862             ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33863             ;je short multi_pass_coff
33864
33865             ; This pass can be made use of if
33866             ; we want do some config.sys process
33867             ; after device drivers are loaded
33868             ; and before install= commands
33869             ; are processed
33870 0000259F 803E[CD02]03         cmp byte [multi_pass_id],3; the third pass for install= ?
33871 000025A4 741D             je short multi_try_i
33872 000025A6 80FC48             cmp ah, CONFIG_DOS ; 'H'
33873             ; 11/12/2022
33874             ;je short multi_pass_coff2
33875 000025A9 74D9             je short coff
33876             ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33877             ;je short multi_pass_coff
33878
33879 ; make note of any INSTALL= or INSTALLHIGH= commands we find,
33880 ; but don't process them now.
33881
33882 000025AB 80FC49             cmp ah,CONFIG_INSTALL ; 'I' ; install= command?
33883             ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33884 000025AE 7507             jne short precheck_installhigh ; the first pass is for normal operation.
33885             ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33886             ;jne short tryb
33887
33888             ;or word [install_flag],have_install_cmd ; set the flag
33889 000025B0 800E[CE02]01         or byte [install_flag],have_install_cmd ; 1
33890 multi_pass_coff2:
33891 000025B5 EBCD             jmp short coff ; 13/05/2019 ; and handles the next command
33892
33893             ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33894             ; (SYSINIT:2448h)
33895             ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33896             ;%if 0
33897 precheck_installhigh:
33898 000025B7 80FC57             cmp ah,CONFIG_INSTALLHIGH ; 'w' ; signifier for INSTALLHIGH
33899 000025BA 756B             jne short tryb ; carry on with normal processing
33900
33901             ;or word [install_flag],have_install_cmd
33902 000025BC 800E[CE02]01         or byte [install_flag],have_install_cmd ; 1
33903             jmp short coff
33904             ;%endif ; 27/10/2022
33905
33906 multi_try_i:
33907             cmp ah,CONFIG_INSTALL ; 'I' ; install= command?
33908             ; 31/12/2022 - Retro DOS v4.2
33909 000025C6 750A             jne short multi_try_n ; no, check for installhigh
33910             ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33911             ;jne short multi_pass_filter
33912
33913             ; 31/12/2022
33914             ;%if 1
33915             ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33916             ;%if 0
33917 000025C8 E84F20             ;ifdef MULTI_CONFIG
33918 000025CB 7241             call query_user ; query the user if config_cmd
33919             jc short multi_pass_filter ; has the CONFIG_OPTION_QUERY bit set
33920             ;endif
33921             ;%endif ; 27/10/2022
33922             call do_install_exec ;install it.
33923 000025D0 EBB2             jmp short coff ;to handle next install= command.
33924
33925             ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33926             ; (SYSINIT:2463h)
33927             ;%if 1
33928             ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33929             ;%if 0
33930
33931 multi_try_n:
33932 000025D2 80FC57             cmp ah,CONFIG_INSTALLHIGH ; installhigh= command?
33933 000025D5 7537             jne short multi_pass_filter ; no. ignore this.
33934             ;ifdef MULTI_CONFIG
33935 000025D7 E84020             call query_user ; query the user if config_cmd
33936 000025DA 7232             jc short multi_pass_filter ; has the CONFIG_OPTION_QUERY bit set
33937             ;endif
33938
33939 ; The memory environment is in its normal DOS state, so do
33940 ; the standard calls to set the alloc strategy for loading high
33941

```

```

33942 000025DC B80058      mov     ax,(ALLOCOPE<<8)|0 ; 5800h
33943 000025DF CD21        int     21h           ;get alloc strategy
33944 000025E1 89C3        mov     bx,ax
33945 000025E3 53          push     bx           ; save for the return
33946
33947 000025E4 81CB8000     or      bx,HIGH_FIRST ; 80h ;set alloc to HighFirst
33948 000025E8 B80158      mov     ax,(ALLOCOPE<<8)|1 ; 5801h
33949 000025EB CD21        int     21h           ;set alloc strategy
33950
33951 000025ED B80258      mov     ax,(ALLOCOPE<<8)|2 ; 5802h
33952 000025F0 CD21        int     21h           ; get link state
33953 000025F2 30E4        xor     ah,ah           ; clear top byte
33954 000025F4 50          push     ax           ; save for return
33955
33956 000025F5 B80358      mov     ax,(ALLOCOPE<<8)|3 ; 5803h
33957 000025F8 B80100      mov     bx,1
33958 000025FB CD21        int     21h           ;link in UMBS
33959
33960 000025FD E899EC      call    do_install_exec ;install it.
33961
33962 00002600 B80358      mov     ax,(ALLOCOPE<<8)|3
33963 00002603 5B          pop     bx           ; recover original link state
33964 00002604 CD21        int     21h
33965 00002606 5B          pop     bx           ; recover original alloc strategy
33966 00002607 B80158      mov     ax,(ALLOCOPE<<8)|1
33967 0000260A CD21        int     21h
33968
33969      ;jmp     short coff           ;to handle next install= command.
33970      ; 01/01/2023
33971 0000260C EBA7        jmp     short multi_pass_coff2
33972
33973 ;%endif ; 27/10/2022
33974
33975 multi_pass_filter:
33976 0000260E 80FC59      cmp     ah,CONFIG_COMMENT ; 'Y' ; comment?
33977 00002611 740A        je      short multi_pass_adjust
33978 00002613 80FC5A      cmp     ah,CONFIG_UNKNOWN ; 'Z' ; bad command?
33979 00002616 7405        je      short multi_pass_adjust
33980 00002618 80FC30      cmp     ah,CONFIG_REM ; 'O' ; rem?
33981 0000261B 7508        jne     short multi_pass_coff ; ignore the rest of the commands.
33982
33983 multi_pass_adjust:
33984 0000261D FF0E[5A03]    dec     word [chrptr] ; these commands need to
33985 00002621 FF06[5603]    inc     word [count] ; adjust chrptr,count
33986                                     ; for newline proc.
33987
33988 multi_pass_coff:
33989      ; 11/12/2022
33990      ; jmp     short coff           ; to handle next install= commands.
33991      ; 01/01/2023
33992      jmp     short multi_pass_coff2
33993
33994 ;-----
33995 ; buffer command
33996 ;-----
33997
33998 ;*****
33999 ; function: parse the parameters of buffers= command. *
34000 ; *
34001 ; input : *
34002 ; es:si -> parameters in command line. *
34003 ; output: *
34004 ; buffers set *
34005 ; buffer_slash_x flag set if /x option chosen. *
34006 ; h_buffers set if secondary buffer cache specified. *
34007 ; *
34008 ; subroutines to be called: *
34009 ; sysinit_parse *
34010 ; logic: *
34011 ; { *
34012 ; set di points to buf_parms; /*parse control definition*/ *
34013 ; set dx,cx to 0; *
34014 ; reset buffer_slash_x; *
34015 ; while (end of command line) *
34016 ; { sysinit_parse; *
34017 ; if (no error) then *
34018 ; if (result_val._$P_synonym_ptr == slash_e) then /*not a switch *
34019 ; buffer_slash_x = 1 *
34020 ; else if (cx == 1) then /* first positional */ *
34021 ; buffers = result_val._$P_picked_val; *
34022 ; else h_buffers = result_val._$P_picked_val; *
34023 ; else {show error message;error exit} *
34024 ; } *
34025 ; if (buffer_slash_x is off & buffers > 99) then show_error; *
34026 ; } *
34027 ; *
34028 ;*****
34029
34030 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
34031 ; (SYSINIT:229Ch)
34032
34033 00002627 80FC42      cmp     ah,CONFIG_BUFFERS ; 'B'
34034 0000262A 755C        jne     short tryc
34035
34036 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34037 ; (SYSINIT:24BFh)
34038 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34039 ;%if 0
34040 ;%ifdef MULTI_CONFIG
34041 0000262C E8EB1F      call    query_user ; query the user if config_cmd
34042 0000262F 7257        jc      short tryc ; has the CONFIG_OPTION_QUERY bit set
34043 ;%endif
34044 ;%endif ; 27/10/2022
34045
34046 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34047 ; 18/12/2022
34048 00002631 31C9        xor     cx,cx
34049 ;mov     byte [p_buffer_slash_x],0 ; 31/03/2019
34050 00002633 880E[1322]    mov     [p_buffer_slash_x],cl ; 0
34051
34052 00002637 BF[CC21]      mov     di,buf_parms
34053 ;xor     cx,cx ; 18/12/2022
34054 ; 03/01/2023
34055 ;mov     dx,cx
34056
34057 0000263A E82808      call    sysinit_parse
34058 0000263D 7303        jnc     short if7 ; parse error,
34059 ;call    badparm_p ; and show messages and end the search loop.
34060 ; jmp     short sr7
34061 ; 31/12/2022
34062 ; sr7:
34063 ; jmp     coff
34064 ; 03/01/2023
34065 0000263F E91207      jmp     badparm_p_coff

```

```

34066      cmp     ax, _$P_RC_EOL ; 0FFFFh; end of line?
34067 00002642 83F8FF      je     short en7 ; then jmp to $endloop for semantic check
34068 00002645 741A      ; cmp     word [result_val_swoff], switch_x
34069      cmp     word [result_val+_ $P_Result_Blk.SYNONYM_Ptr], switch_x
34070 00002647 813E[1722][0C22]      jne     short if11
34071      ; 31/12/2022
34072      je     short do7 ; je short en11
34073 0000264D 74EB      ;
34074      ; mov     byte [p_buffer_slash_x], 1 ; set the flag M016
34075      ; jmp     short en11 ; 31/12/2022
34076
34077 if11:
34078      ; mov     ax, [rv_dword]
34079 0000264F A1[1922]      mov     ax, [result_val+_ $P_Result_Blk.Picked_Val]
34080 00002652 83F901      cmp     cx, 1
34081 00002655 7505      jne     short if13
34082
34083 00002657 A3[0F22]      mov     [p_buffers], ax
34084      ; jmp     short en11
34085      ; 31/12/2022
34086 0000265A EBDE      jmp     short do7
34087
34088 0000265C A3[1122]      mov     [p_h_buffers], ax
34089
34090 0000265F EBD9      en11: jmp     short do7
34091
34092 00002661 833E[0F22]63      en7:  cmp     word [p_buffers], 99
34093 00002666 760B      jbe     short if18
34094
34095      ; cmp     byte [p_buffer_slash_x], 0 ; M016
34096      ; jne     short if18
34097
34098 00002668 E82508      call    badparm_p
34099 0000266B C706[1122]0000      mov     word [p_h_buffers], 0
34100 00002671 EB12      jmp     short sr7
34101
34102 00002673 A1[0F22]      if18: mov     ax, [p_buffers] ; we don't have any problem.
34103 00002676 A3[9902]      mov     [buffers], ax ; now, let's set it really.
34104
34105 00002679 A1[1122]      mov     ax, [p_h_buffers]
34106 0000267C A3[9B02]      mov     [h_buffers], ax
34107
34108      ; mov     al, [p_buffer_slash_x] ; M016
34109      ; mov     [buffer_slash_x], al
34110
34111 0000267F A1[AF02]      mov     ax, [linecount]
34112 00002682 A3[B902]      mov     [buffer_linenum], ax ; save the line number for the future use.
34113      ; 31/12/2022
34114      ; jmp     short sr7
34115      ; 03/01/2023
34116
34117 00002685 E9FCFE      sr7:  jmp     coff
34118
34119      ; -----
34120      ; break command
34121      ; -----
34122
34123      ; *****
34124      ;
34125      ; function: parse the parameters of break = command.
34126      ;
34127      ; input :
34128      ; es:si -> parameters in command line.
34129      ; output:
34130      ; turn the control-c check on or off.
34131      ;
34132      ; subroutines to be called:
34133      ; sysinit_parse
34134      ; logic:
34135      ; {
34136      ; set di to brk_parms;
34137      ; set dx, cx to 0;
34138      ; while (end of command line)
34139      ; { sysinit_parse;
34140      ; if (no error) then
34141      ; if (result_val._ $P_item_tag == 1) then /*on */ *
34142      ; set p_ctrl_break, on;
34143      ; else /*off */ *
34144      ; set p_ctrl_break, off;
34145      ; else {show message; error_exit};
34146      ; }
34147      ; if (no error) then
34148      ; dos function call to set ctrl_break check according to
34149      ; }
34150      ;
34151      ; *****
34152
34153      ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
34154      ; (SYSINIT:22FFh)
34155
34156 00002688 80FC43      tryc:  cmp     ah, CONFIG_BREAK ; 'C'
34157 0000268B 7539      jne     short trym
34158
34159      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34160      ; (SYSINIT:2527h)
34161      ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34162      ; %if 0
34163      ; ifndef MULTI_CONFIG
34164 0000268D E88A1F      call    query_user ; query the user if config_cmd
34165 00002690 7234      jc     short trym ; has the CONFIG_OPTION_QUERY bit set
34166      ; endif
34167      ; %endif ; 27/10/2022
34168
34169 00002692 BF[1D22]      mov     di, brk_parms
34170 00002695 31C9      xor     cx, cx
34171      ; 03/01/2023
34172      ; mov     dx, cx
34173
34174 00002697 E8CB07      do22: call    sysinit_parse
34175 0000269A 7303      jnc     short if22 ; parse error
34176      ; call    badparm_p ; show message and end the search loop.
34177      ; jmp     short sr22
34178      ; 31/12/2022
34179
34180      ; sr22:
34181      ; jmp     coff
34182      ; 03/01/2023
34183      ; jmp     badparm_p_coff
34184
34185 0000269F 83F8FF      if22: cmp     ax, _$P_RC_EOL ; end of line?
34186 000026A2 7415      je     short en22 ; then end the $endloop
34187
34188      ; cmp     byte [result_val_itag], 1
34189 000026A4 803E[1622]01      cmp     byte [result_val+_ $P_Result_Blk.Item_Tag], 1
34190 000026A9 7507      jne     short if26

```



```

34190
34191 000026AB C606[4222]01      mov     byte [p_ctrl_break],1 ; turn it on
34192      ; jmp     short en26
34193      ; 31/12/2022
34194 000026B0 EBE5      jmp     short do22
34195
34196 000026B2 C606[4222]00      if26:  mov     byte [p_ctrl_break],0 ; turn it off
34197      en26:  jmp     short do22          ; we actually set the ctrl break
34198 000026B7 EBDE      en22:  mov     ah,SET_CTRL_C_TRAPPING ; if we don't have any parse error.
34199      mov     al,1
34200 000026B9 B433      mov     dl,[p_ctrl_break]
34201 000026BB B001      int     21h
34202 000026BD 8A16[4222]      ; 31/12/2022
34203 000026C1 CD21      ; jmp     short sr22
34204      ; 03/01/2023
34205
34206      sr22:  jmp     coff
34207 000026C3 E9BEFE
34208      jmp     coff
34209
34210      ;-----
34211      ; multitrack command
34212      ;-----
34213
34214      ;*****
34215      ;
34216      ; function: parse the parameters of multitrack= command.
34217      ;
34218      ; input :
34219      ; es:si -> parameters in command line.
34220      ; output:
34221      ; turn multrk_flag on or off.
34222      ;
34223      ; subroutines to be called:
34224      ; sysinit_parse
34225      ; logic:
34226      ; {
34227      ;     set di to brk_parms;
34228      ;     set dx,cx to 0;
34229      ;     while (end of command line)
34230      ;     { sysinit_parse;
34231      ;       if (no error) then
34232      ;       {
34233      ;         if (result_val._$P_item_tag == 1) then          /*on      */
34234      ;         {
34235      ;           set p_mtrk,on;
34236      ;         }
34237      ;       }
34238      ;       else {show message;error_exit};
34239      ;       if (no error) then
34240      ;       {
34241      ;         dos function call to set multrk_flag according to p_mtrk.
34242      ;       }
34243      ;     }
34244      ; }
34245      ;*****
34246      ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
34247      trym:  cmp     ah,CONFIG_MULTITRACK ; 'M'
34248      jne     short tryu
34249
34250      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34251      ; (SYSINIT:2569h)
34252      ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34253      ;%if 0
34254      ;ifdef MULTI_CONFIG
34255      call     query_user      ; query the user if config_cmd
34256      jc      short tryu      ; has the CONFIG_OPTION_QUERY bit set
34257      ;endif
34258      ;%endif      ; 27/10/2022
34259
34260 000026D0 BF[2123]      mov     di,mtrk_parms
34261 000026D3 31C9      xor     cx,cx
34262      ; 03/01/2023
34263      ;mov     dx,cx
34264
34265 000026D5 E88D07      do31:  call     sysinit_parse
34266 000026D8 7303      jnc     short if31      ; parse error
34267      ;call     badparm_p      ; show message and end the search loop.
34268      ; jmp     short sr31
34269      ; 31/12/2022
34270
34271      ;sr31:  jmp     coff
34272      ; 03/01/2023
34273 000026DA E97706      jmp     badparm_p_coff
34274
34275 000026DD 83F8FF      if31:  cmp     ax,_$P_RC_EOL ; end of line?
34276 000026E0 7415      je      short en31      ; then end the $endloop
34277
34278      ;cmp     byte [result_val_itag],1
34279 000026E2 803E[1622]01  cmp     byte [result_val+_$P_Result_Blk.Item_Tag],1
34280 000026E7 7507      jne     short if35
34281
34282 000026E9 C606[3523]01      mov     byte [p_mtrk],1      ; turn it on temporarily.
34283      ; jmp     short en35
34284      ; 31/12/2022
34285 000026EE EBE5      jmp     short do31
34286
34287 000026F0 C606[3523]00      if35:  mov     byte [p_mtrk],0      ; turn it off temporarily.
34288      en35:  jmp     short do31      ; we actually set the multrk_flag here
34289 000026F5 EBDE
34290
34291 000026F7 1E      en31:  push     ds
34292      ; mov     ax,Bios_Data ; 70h
34293      ; mov     ax,KERNEL_SEGMENT ; 70h
34294      ; 21/10/2022
34295 000026F8 B87000      mov     ax,DOSBIODATASEG ; 0070h
34296 000026FB 8ED8      mov     ds,ax
34297
34298 000026FD 2E803E[3523]00      cmp     byte [cs:p_mtrk],0
34299 00002703 7508      jne     short if39
34300
34301 00002705 C706[A004]0100      mov     word [multrk_flag],multrk_off2 ; 0001h
34302 0000270B EB06      jmp     short en39
34303
34304 0000270D C706[A004]8000      if39:  mov     word [multrk_flag],multrk_on ; 0080h
34305      en39:  pop     ds
34306      ; 31/12/2022
34307      ; jmp     short sr31
34308      ; 03/01/2023
34309
34310      sr31:  jmp     coff
34311 00002714 E96DFE
34312
34313      ;-----

```

```

34314 ; DOS=HIGH/LOW command
34315 ;-----
34316 ;
34317 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
34318 multi_try_doshi:
34319     cmp     ah,CONFIG_DOS ; 'H'
34320     je      short it_is_h
34321 skip_it:
34322     jmp     multi_pass_filter
34323 it_is_h:
34324     ; M003 - removed initing DevUMB
34325     ; & runhigh
34326 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34327 ; (SYSINIT:25C1h)
34328 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34329 ;%if 0
34330 ;ifdef MULTI_CONFIG
34331     call    query_user ; query the user if config_cmd
34332     jc      short skip_it ; has the CONFIG_OPTION_QUERY bit set
34333 ;endif
34334 ;%endif ; 27/10/2022
34335     mov     di,dos_parms
34336     xor     cx,cx
34337     ; 03/01/2023
34338     ;mov     dx,cx
34339 h_do_parse:
34340     call    sysinit_parse
34341     jnc     short h_parse_ok ; parse error
34342 h_badparm:
34343     ; 03/01/2023
34344     ;call    badparm_p ; show message and end the search loop.
34345     ;jmp     short h_end
34346     ; 11/12/2022
34347 ;h_end:
34348     ;jmp     coff
34349     ; 03/01/2023
34350     jmp     badparm_p_coff
34351 h_parse_ok:
34352     cmp     ax,_$P_RC_EOL ; end of line?
34353     je      short h_end ; then end the $endloop
34354     call    ProcDOS
34355     jmp     short h_do_parse
34356     ; 11/12/2022
34357     ; 03/01/2023
34358 h_end:
34359     jmp     coff
34360 ;-----
34361 ; devicehigh command
34362 ;-----
34363 ;
34364 ; 28/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
34365 tryu:
34366     cmp     ah,CONFIG_DEVICEHIGH ; 'U'
34367     jne     short tryd
34368 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34369 ; (SYSINIT:25E9h)
34370 ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34371 ;%if 0
34372 ;ifdef MULTI_CONFIG
34373     call    query_user ; query the user if config_cmd
34374     jc      short tryd ; has the CONFIG_OPTION_QUERY bit set
34375 ;endif
34376 ;%endif ; 28/10/2022
34377 ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34378 ;%if 0
34379     ; 01/01/2023
34380     ; ds = cs
34381     call    InitVar
34382     call    ParseSize ; process the size= option
34383     ;jnc     short tryu_0
34384     ; 31/12/2022
34385     jc      short tryu_1 ; 31/03/2019 - Retro DOS v4.0
34386 ;%endif ; 28/10/2022
34387 ; 31/12/2022
34388 ;%if 0
34389     ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34390     ;mov     [cs:badparm_off], si ; stash it there in case of an error
34391     ;mov     [cs:badparm_seg], es
34392     ; 11/12/2022
34393     ; ds = cs
34394     mov     [badparm_off], si
34395     mov     [badparm_seg], es
34396     ; 31/12/2022
34397     ;call    ParseSize
34398     ;jnc     short tryu_2 ; 28/10/2022
34399     ;call    badparm_p
34400     ;jmp     coff
34401     ; 03/01/2023
34402     jmp     badparm_p_coff
34403 ;endif
34404 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34405 ; (SYSINIT:2606h)
34406 ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34407 ;%if 0
34408 tryu_0:
34409     ;mov     ax,[cs:DevSizeOption]
34410     ; 31/12/2022
34411     mov     ax,[DevSizeOption] ; ds = cs
34412     or      ax,ax
34413     jnz     short tryu_2
34414     call    ParseVar
34415     jnc     short tryu_2
34416 tryu_1:
34417     ; 31/12/2022
34418     ; ds = cs
34419     mov     [badparm_off], si
34420     mov     [badparm_seg], es
34421     ;mov     [cs:badparm_off], si ; If ParseVar up there failed, then
34422     ;mov     [cs:badparm_seg], es ; ES:SI points to its problem area...
34423     ;call    badparm_p ; so all we have to do is choke and
34424     ;jmp     coff ; die, rather verbosely.
34425     ; 03/01/2023
34426     jmp     badparm_p_coff
34427 ; 00002750 A1[5024]
34428 ; 00002753 09C0
34429 ; 00002755 7510
34430 ; 00002757 E8B408
34431 ; 0000275A 730B
34432 ; 0000275C 8936[6B19]
34433 ; 00002760 8C06[6D19]
34434 ; 00002764 E9ED05

```



```

34438
34439 ;%endif ; 28/10/2022
34440
34441 tryu_2:
34442     push    si
34443     push    es
34444
34445     ; 08/09/2023 - Retro DOS 4.2 IO.SYS (Optimization)
34446     ; MSDOS 6.21 IO.SYS - SYSINIT:2623h
34447     ; PC DOS 7.1 IBMBIO.COM - SYSINIT:2B17h
34448
34449 tryu_3:
34450     mov     al,[es:si]
34451     cmp     al,cr
34452     ; 14/04/2024
34453     ;je     short tryu_4
34454     ; 08/09/2023 - Retro DOS 5.0 (PCDOS 7.1 IBMBIO.COM)
34455     je     short tryu_5
34456     cmp     al,lf
34457     je     short tryu_4
34458     call    delim
34459     jz     short tryu_4
34460     inc     si
34461     jmp     short tryu_3
34462
34463     ; 14/04/2024
34464     ; 08/09/2023 - Retro DOS 5.0 (PCDOS 7.1 IBMBIO.COM)
34465 tryu_5:
34466     mov     al,20h ; ' ' ; blank instead of cr
34467
34468 tryu_4:
34469     ; 11/12/2022
34470     ; ds = cs
34471     mov     [DevSavedDelim],al
34472     ;mov     [cs:DevSavedDelim],al ; Save the delimiter before replacing
34473     ; it with null
34474     ; 18/12/2022
34475     sub     bx,bx
34476     mov     [es:si],bl ; 0
34477     ;mov     byte [es:si],0
34478     pop     es
34479     pop     si ; 14/04/2024
34480
34481 ;-----
34482 ; BEGIN PATCH TO CHECK FOR NON-EXISTANT UMBS -- t-richj 7-21-92
34483 ;-----
34484
34485 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34486 ; (SYSINIT:2642h)
34487 ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34488 ;%if 0
34489 ; 10/07/2023 - Retro DOS v4.2 IO.SYS (optimized)
34490 ; MSDOS 6.21 IO.SYS - SYSINIT:2642h
34491 ;%if 1
34492     ; 01/01/2023
34493     ; ds = cs
34494     call    UmbTst ; See if UMBS are around...
34495     ; 01/01/2023
34496     ;jnc     short NrmTst ; ...yep. So do that normal thang.
34497
34498     ;mov     byte [cs:DeviceHi],0 ; ...nope... so load low.
34499     ; 31/12/2022
34500     ; ds = cs, bx = 0
34501     ;mov     byte [DeviceHi],bl ; 0
34502     ;jmp     short LoadDevice
34503     ; 01/01/2023
34504     ;jc     short LoadDevice ; bl = 0
34505 ;%endif
34506 ;%endif
34507 ;-----
34508 ; END PATCH TO CHECK FOR NON-EXISTANT UMBS -- t-richj 7-21-92
34509 ;-----
34510
34511 NrmTst:
34512     ; 11/12/2022
34513     ; ds = cs
34514     ;mov     byte [cs:DeviceHi],0
34515     ;mov     byte [DeviceHi],0
34516     ; 18/12/2022
34517     ; bx = 0
34518     cmp     [DevUMB],bl ; 0
34519     ;cmp     byte [DevUMB],0
34520     ;cmp     byte [cs:DevUMB],0 ; do we support UMBS
34521     ;je     short LoadDevice ; no, we don't
34522     ;mov     byte [cs:DeviceHi],1
34523     ; 11/12/2022
34524     ;mov     byte [DeviceHi],1
34525     ; 18/12/2022
34526     inc     bl ; mov bl,1 ; (*)
34527     ; 11/12/2022
34528     ;jmp     short LoadDevice2 ; 11/12/2022
34529     jmp     short LoadDevice
34530
34531 ;-----
34532 ; device command
34533 ;-----
34534
34535 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34536 ; (SYSINIT:2665h)
34537
34538 ; 28/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
34539 ; (SYSINIT:2401h)
34540 tryd:
34541     ; 11/12/2022
34542     ;xor     bx,bx ; 31/12/2022
34543     ;
34544     cmp     ah,CONFIG_DEVICE ; 'D'
34545     je     short gotd
34546 skip_it2:
34547     jmp     tryq
34548 gotd:
34549
34550 ; 31/12/2022 - Retro DOS v4.2
34551 ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34552 ;%if 0
34553 ;ifdef     MULTI_CONFIG
34554     call    query_user ; query the user if config_cmd
34555     jc     short skip_it2 ; has the CONFIG_OPTION_QUERY bit set
34556 ;endif
34557 ;%endif ; 28/10/2022
34558
34559 ; 31/12/2022
34560     sub     bx,bx
34561     ; bx = 0

```

```

34562             ; 11/12/2022
34563             ; ds = cs
34564             ;mov     byte [DeviceHi],0
34565             ;mov     word [DevSizeOption],0
34566 000027A6 891E[5024]     mov     [DevSizeOption],bx ; 0
34567 000027AA C606[6424]20   mov     byte [DevSavedDelim],' '
34568             ;mov     byte [cs:DeviceHi],0 ; not to be loaded in UMB ;M007
34569             ;mov     word [cs:DevSizeOption],0
34570             ;mov     byte [cs:DevSavedDelim],' ' ; In case of DEVICE= the null has to
34571             ;                     ; be replaced with a ' '
34572 LoadDevice:
34573             ; 11/12/2022
34574             ;mov     byte [DeviceHi],0
34575 000027AF 881E[4F24]     mov     byte [DeviceHi],bl ; 0 or 1 (*)
34576 LoadDevice2:
34577             ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34578             ;
34579             ;push     cs
34580             ;pop      ds
34581             ;
34582             ;mov     [bpb_addr],si ; pass the command line to the dvice
34583             ;mov     [bpb_addr+2],es
34584             ;
34585             ;mov     [DevCmdLine],si ; save it for ourself
34586             ;mov     [DevCmdLine+2],es
34587             ;
34588             ;mov     byte [driver_units],0 ; clear total block units for driver
34589
34590             ; 11/12/2022
34591             ; ds = cs
34592             ;mov     bx,cs
34593             ;mov     ds,bx
34594
34595             ;mov     [cs:bpb_addr],si ; pass the command line to the dvice
34596 000027B3 8936[8103]     mov     [bpb_addr],si
34597             ;mov     [cs:bpb_addr+2],es
34598 000027B7 8C06[8303]     mov     [bpb_addr+2],es
34599
34600             ;mov     [cs:DevCmdLine],si ; save it for ourself
34601 000027BB 8936[6024]     mov     [DevCmdLine],si
34602             ;mov     [cs:DevCmdLine+2],es
34603 000027BF 8C06[6224]     mov     [DevCmdLine+2],es
34604
34605             ; 31/12/2022 - Retro DOS v4.2
34606 000027C3 C606[6A19]00   mov     byte [driver_units],0 ; clear total block units for driver
34607
34608 000027C8 E82520         call     round
34609
34610 000027CB E8910E         call     SizeDevice
34611 000027CE 723F         jc      short BadFile
34612
34613             ; 11/12/2022
34614             ; ds = cs
34615
34616 ; - Begin DeviceHigh primary logic changes -----
34617
34618 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34619 ; (SYSINIT:26A4h)
34620
34621 ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34622 ;%if 0
34623 000027D0 C606[3F24]01   mov     byte [ConvLoad],1 ; Doesn't matter if DeviceHi==0
34624
34625             ; 22/07/2023
34626             ;mov     al,[DeviceHi] ; If not using upper memory,
34627 000027D5 800E[4F24]00   or      byte [DeviceHi],0 ; Skip all this and go on to
34628             ; 10/07/2023
34629             ;or      al,al
34630 000027DA 741E         jz      short DevConvLoad ; the actual load.
34631
34632             ;call    GetLoadUMB ; Returns first UMB spec'ed in AX
34633 000027DC A0[FD23]     mov     al,[UmbLoad] ; 19/04/2019 - Retro DOS v4.0
34634
34635             cmp     al,-1 ; If umb0 not specified, it's old style
34636 000027E1 7417         jz      short DevConvLoad ; so load high even if SIZE= is smaller
34637
34638             dec     byte [ConvLoad] ; 0 ; They specified /L, so use new loader
34639
34640             call    GetLoadSize ; Returns size of first UMB specified
34641             or      ax,ax
34642 000027EC 7406         jz      short tryd_1 ; If size1 not specified, nada to do:
34643
34644             cmp     ax,[DevSize] ; /L:...,Size < DevSize?
34645 000027F2 7D06         jge     short DevConvLoad
34646 tryd_1:
34647 000027F4 A1[3124]     mov     ax,[DevSize] ; Size < DevSize, so write DevSize as
34648 000027F7 E8550A         call    StoLoadSize ; minsize for load UMB.
34649
34650 ;%endif ; 28/10/2022
34651
34652 ; - End DeviceHigh primary logic changes -----
34653
34654 DevConvLoad:
34655             ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34656 000027FA E8660D         call    InitDevLoad
34657
34658             ; 11/12/2022
34659             ; ds = cs
34660 000027FD A1[3324]     mov     ax,[DevLoadAddr]
34661 00002800 0306[3124]     add     ax,[DevSize]
34662 00002804 7206         jc      short NoMem
34663 00002806 3906[3524]     cmp     [DevLoadEnd],ax
34664 0000280A 7315         jae     short LoadDev
34665
34666             ; 11/12/2022
34667             ;mov     ax,[cs:DevLoadAddr]
34668             ;add     ax,[cs:DevSize]
34669             ;jc      short NoMem
34670             ;cmp     [cs:DevLoadEnd],ax
34671             ;jae     short LoadDev
34672 NoMem:
34673             ; 11/12/2022
34674             ; ds = cs
34675             ;jmp     mem_err
34676 0000280C E92020         jmp     mem_err2
34677
34678 BadFile:
34679             ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34680             ;;call    RetFromUM ; Does nothing if didn't call HideUMBS
34681             ;;cmp     byte [es:si],' '
34682             ;;jae     short tryd_2
34683             ; 31/12/2022
34684             ;cmp     byte [es:si],0dh ; cr
34685             ;jne     short tryd_2

```

```

34686         ;jmp     badop
34687         ; 31/12/2022
34688         ; ds = cs
34689         ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34690         ; (SYSINIT:26E6h)
34691 0000280F E8AE0E      call     RetFromUM          ; Does nothing if didn't call HideUMBS
34692 00002812 26803C20    cmp     byte [es:si], ' '
34693         ;cmp     byte [es:si], 20h ; space
34694 00002816 7303        jnb     short tryd_2
34695 00002818 E96906      jmp     badop
34696
34697 0000281B E80C22      tryd_2: call     badload
34698 0000281E E963FD      jmp     coff
34699
34700
34701 00002821 06          LoadDev: push    es
34702 00002822 1F          pop     ds
34703
34704 00002823 89F2        mov     dx, si          ;ds:dx points to file name
34705 00002825 E87C0E      call    ExecDev         ; load device driver using exec call
34706
34707 00002828 1E          badldreset: push    ds
34708 00002829 07          pop     es          ;es:si back to config.sys
34709 0000282A 0E          push    cs
34710 0000282B 1F          pop     ds          ;ds back to sysinit
34711 0000282C 72E1        jc      short BadFile
34712
34713         goodld:      ; 11/12/2022
34714         ; ds = cs
34715
34716 0000282E 06          push    es ; + ; 31/12/2022
34717 0000282F 56          push    si ; ++
34718 00002830 E89E0E      call    RemoveNull
34719 00002833 06          push    es
34720 00002834 56          push    si
34721
34722 00002835 0E          push    cs
34723 00002836 07          pop     es
34724
34725 00002837 1E          push    ds ; ** ; ds = cs
34726 00002838 56          push    si
34727
34728         ;lds     si,[cs:DevEntry]      ; peeks the header attribute
34729         ; 31/12/2022
34730         ; ds = cs
34731 00002839 C536[3724]   lds     si,[DevEntry]
34732
34733         ;test    word [si+4],8000h
34734         ; 11/12/2022
34735 0000283D F6440580    test    byte [si+SYSDEV.ATT+1],DEVTP>>8
34736         ;test    word [si+SYSDEV.ATT],DEVTP ; block device driver?
34737 00002841 7514        jnz     short got_device_com_cont ; no.
34738
34739 00002843 2EC536[6D02]   lds     si,[cs:DOSINFO]          ; ds:si -> sys_var
34740         ;cmp     byte [si+32],26
34741 00002848 807C201A    cmp     byte [si+SYSI_NUMIO],26 ; no more than 26 drive number
34742 0000284C 7209        jb      short got_device_com_cont
34743
34744 0000284E 5E          pop     si
34745 0000284F 1F          pop     ds ; **
34746
34747 00002850 5E          pop     si          ; clear the stack
34748 00002851 07          pop     es
34749
34750         ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34751         ;call    RetFromUM
34752         ; 31/12/2022
34753         ; ds = cs ; **
34754 00002852 E86B0E      call    RetFromUM          ; Do this before we leave
34755
34756         ;jmp     short badnumblock
34757         ; 31/12/2022
34758 00002855 EB73        jmp     short badnumblock2 ; ds = cs
34759
34760         got_device_com_cont:
34761 00002857 5E          pop     si
34762 00002858 1F          pop     ds
34763
34764         ; 11/12/2022
34765         ; ds = cs
34766
34767 00002859 E8AE06      call    LieInt12Mem
34768 0000285C E80B07      call    UpdatePDB          ; update the PSP:2 value M020
34769
34770         ; 11/12/2022
34771         ; ds = cs
34772         ; 08/09/2023
34773 0000285F 31C0        xor     ax, ax ; 0
34774 00002861 3806[6619]   cmp     byte [multdeviceflag],al ; 0
34775         ;cmp     byte [multdeviceflag],0
34776         ;cmp     byte [cs:multdeviceflag],0 ; Pass limit only for the 1st device
34777         ; driver in the file ; M027
34778 00002865 750B        jne     short skip_pass_limit ; M027
34779
34780         ; 11/12/2022
34781         ; ds = cs
34782         ;mov     word [cs:break_addr],0; pass the limit to the DD
34783         ;mov     bx,[cs:DevLoadEnd]
34784         ;mov     [cs:break_addr+2],bx
34785
34786         ;mov     word [break_addr],0
34787         ; 08/09/2023
34788 00002867 A3[7D03]      mov     [break_addr],ax ; 0
34789 0000286A 8B1E[3524]   mov     bx,[DevLoadEnd]
34790 0000286E 891E[7F03]   mov     [break_addr+2],bx
34791
34792         skip_pass_limit:
34793         ; Note: sysi_numio (in DOS DATA) currently reflects the REAL
34794         ; number of installed devices (including DblSpace drives) where
34795         ; "drivenumber" is the number that the next block device will
34796         ; be assigned to. Because some naughty device drivers (like
34797         ; interlnk) look at the internal DOS variable instead of the
34798         ; value we pass it, we'll temporarily stick our value into
34799         ; DOS DATA while we're initializing the device drivers.
34800         ;
34801         ; Note that this will make it impossible for this device
34802         ; driver to access the DblSpace drive letters, whether
34803         ; they are swapped-hosts or unswapped compressed drives,
34804         ; during its initialization phase.
34805
34806         ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34807         ; (SYSINIT:2752h)
34808         ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34809         ;%if 0

```

```

34810 ; 31/12/2022
34811 ;push ds
34812
34813 ;lds bx,[cs:DOSINFO] ; ds:bx -> sys_var
34814 ; 31/12/2022
34815 ; ds = cs
34816 ; 08/09/2023
34817 ;lds bx,[DOSINFO] ; ds:bx -> sys_var
34818
34819 ;mov al,[cs:drivenumber] ; temporarily use this next drv value
34820 ;mov [cs:devdrivenum],al ; pass drive number in packet to driver
34821 ;mov ah,al
34822
34823 ; 08/09/2023
34824 ; ds = cs
34825 00002872 A0[8503] mov al,[drivenumber] ; temporarily use this next drv value
34826 00002875 A2[8503] mov [devdrivenum],al ; pass drive number in packet to driver
34827 00002878 88C4 mov ah,al
34828 0000287A C51E[6D02] lds bx,[DOSINFO] ; ds:bx -> sys_var
34829
34830 0000287E 874720 xchg ax,[bx+SYSI_NUMIO] ; swap with existing values
34831 ; 31/12/2022
34832 ;pop ds
34833
34834 00002881 50 push ax ; save real sysi_numio/ncds in ax
34835
34836 ;%endif ; 29/10/2022
34837
34838 ; 29/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
34839 ; (SYSINIT:24B9h)
34840
34841 00002882 BB0600 mov bx,SYSDEV.STRAT ; 6
34842 00002885 E8B01F call calldev ; calldev (sdevstrat);
34843 00002888 BB0800 mov bx,SYSDEV.INT ; 8
34844 0000288B E8AA1F call calldev ; calldev (sdevint);
34845
34846 ; 11/12/2022
34847 ; ds <> cs (from calldev) ; 31/12/2022
34848
34849 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34850 ; (SYSINIT:2773h)
34851 ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34852 ;%if 0
34853 0000288E 58 pop ax ; get real sysi_numio value
34854 ; 31/12/2022
34855 ;push ds
34856 0000288F 2EC51E[6D02] lds bx,[cs:DOSINFO] ; ds:bx -> sys_var
34857 00002894 894720 mov [bx+SYSI_NUMIO],ax ; swap with existing values
34858 ; 31/12/2022
34859 ;pop ds
34860
34861 ;%endif ; 29/10/2022
34862
34863 ; 11/12/2022
34864 00002897 0E push cs
34865 00002898 1F pop ds
34866
34867 00002899 E89C06 call TrueInt12Mem
34868
34869 ; 11/12/2022
34870 ; ds = cs
34871 ;mov ax,[cs:break_addr] ; move break addr from the req packet
34872 ;mov [cs:DevBrkAddr],ax
34873 ;mov ax,[cs:break_addr+2]
34874 ;mov [cs:DevBrkAddr+2],ax
34875 0000289C A1[7D03] mov ax,[break_addr]
34876 0000289F A3[3B24] mov [DevBrkAddr],ax
34877 000028A2 A1[7F03] mov ax,[break_addr+2]
34878 000028A5 A3[3D24] mov [DevBrkAddr+2],ax
34879
34880 ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34881 ;call RetFromUM ; There we go... all done.
34882 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34883 ; (SYSINIT:2791h)
34884 000028A8 E8150E call RetFromUM ; There we go... all done.
34885
34886 ; 31/12/2022
34887 ; ds = cs
34888
34889 ; 11/12/2022
34890 000028AB 803E[4024]00 cmp byte [DevUMB],0
34891 ;cmp byte [cs:DevUMB],0
34892 000028B0 7403 je short tryd_3
34893 000028B2 E80010 call AllocUMB
34894 ; 31/12/2022
34895 ; ds = cs
34896 tryd_3:
34897
34898 ;ifndef ROMDOS
34899 ;----- If we are waiting to be moved into hma lets try it now !!!
34900
34901 ; 11/12/2022
34902 ; ds = cs
34903
34904 ;cmp byte [cs:runhigh],0FFh
34905 000028B5 803E[6C02]FF cmp byte [runhigh],0FFh ; 11/12/2022
34906 000028BA 7503 jne short tryd_4
34907
34908 ; 11/12/2022
34909 ; ds = cs
34910 000028BC E8D9E1 call TryToMovDOSHi ; move DOS into HMA if reqd
34911 tryd_4:
34912 ;endif ; ROMDOS
34913
34914 000028BF 5E pop si
34915 000028C0 1F pop ds
34916 000028C1 C60400 mov byte [si],0 ; *p = 0;
34917
34918 000028C4 0E push cs
34919 000028C5 1F pop ds
34920
34921 000028C6 EB1F jmp short was_device_com
34922
34923 ;-----
34924
34925 ; 02/04/2019 - Retro DOS v4.0
34926
34927 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34928 ; (SYSINIT:27B3h)
34929
34930 badnumblock:
34931 000028C8 0E push cs
34932 000028C9 1F pop ds
34933 badnumblock2: ; 31/12/2022 (ds=cs)

```

```

34934 000028CA BA[6E51]      mov     dx,badblock
34935 000028CD E88221      call    print
34936
34937 ;----- fall thru -----
34938
34939 ; 31/12/2022 - Retro DOS v4.2
34940
34941 erase_dev_do:              ; modified to show message "error in config.sys..."
34942
34943 ;call  CheckDoubleSpace ; MSDOS 6.21 IO.SYS SYSINIT:27BBh
34944 ; (Note: 'call CheckDoubleSpace'
34945 ; has been removed at 'erase_dev_do:' pos
34946 ; in PC DOS 7.1 IBMBIO.COM - SYSINIT:2CBAh)
34947 ; Erdogan Tan - 10/07/2023
34948 000028D0 5E          pop     si ; ++
34949 000028D1 07          pop     es ; + ; 31/12/2022
34950
34951 000028D2 0E          push    cs
34952 000028D3 1F          pop     ds
34953
34954 skip1_resetmemhi:
34955 ; 11/12/2022
34956 ; ds = cs
34957 000028D4 833E[8603]00  cmp     word [configmsgflag],0
34958 ;cmp   word [cs:configmsgflag],0
34959 000028D9 7409      je      short no_error_line_msg
34960
34961 000028DB E8DA05      call    error_line ; no "error in config.sys" msg for device driver. dcr d493
34962 ; 11/12/2022
34963 ; ds = cs
34964 ;mov   word [cs:configmsgflag],0
34965 000028DE C706[8603]0000  mov     word [configmsgflag],0; set the default value again.
34966
34967 no_error_line_msg:
34968 000028E4 E99DFC      jmp     coff
34969
34970 ;-----
34971
34972 was_device_com:
34973 ; 14/12/2022
34974 ; ds = cs
34975 000028E7 A1[3D24]      mov     ax,[DevBrkAddr+2]
34976 ;mov   ax,[cs:DevBrkAddr+2] ; 13/05/2019
34977 000028EA 3B06[3524]  cmp     ax,[DevLoadEnd]
34978 ;cmp   ax,[cs:DevLoadEnd]
34979 000028EE 7605      jbe     short breakok
34980
34981 000028F0 5E          pop     si
34982 000028F1 07          pop     es
34983 000028F2 E91AFF      jmp     BadFile
34984
34985 breakok:
34986 ; 14/12/2022
34987 ; ds = cs
34988 000028F5 C43E[6D02]  les     di,[DOSINFO]
34989 000028F9 C516[3724]  lds     dx,[DevEntry]
34990 ;lds   dx,[cs:DevEntry] ;set ds:dx to header
34991 000028FD 89D6      mov     si,dx
34992
34993 ; 14/11/2022
34994 ;les   di,[cs:DOSINFO] ;es:di point to dos info
34995
34996 ; 14/12/2022
34997 ; ds <> cs
34998
34999 ;mov   ax,[si+4]
35000 000028FF 8B4404      mov     ax,[si+SYSDEV.ATT] ;get attributes
35001 ; 12/12/2022
35002 00002902 F6C480      test    ah,DEVTYPE>>8 ; 80h
35003 ;test  ax,DEVTYPE ; 8000h ;test if block dev
35004 00002905 7426      jz      short isblock
35005
35006 ;----- lets deal with character devices
35007
35008 00002907 2E800E[6919]02  or      byte [cs:setdevmarkflag],for_devmark ; 2
35009 0000290D E8F40D      call    DevSetBreak ;go ahead and alloc mem for device
35010
35011 jc_edd:
35012 jc      short erase_dev_do ;device driver's init routine failed.
35013
35014 ; 12/12/2022
35015 test    al,ISCIN
35016 ;test  ax,ISCIN ; 1 ;is it a console in?
35017 jz      short tryclk
35018
35019 mov     [es:di+SYSI_CON],dx ; es:di+12
35020 mov     [es:di+SYSI_CON+2],ds ; es:di+14
35021
35022 tryclk:
35023 ; 12/12/2022
35024 test    al,ISCLOCK
35025 ;test  ax,ISCLOCK ; 8 ;is it a clock device?
35026 jz      short golink
35027
35028 mov     [es:di+SYSI_CLOCK],dx ; es:di+8
35029 mov     [es:di+SYSI_CLOCK+2],ds ; es:di+10
35030
35031 golink:
35032 jmp     linkit
35033
35034 ;----- deal with block device drivers
35035
35036 isblock:
35037 mov     al,[cs:unitcount] ;if no units found,erase the device
35038 or      al,al
35039 jz      short erase_dev_do
35040 ;mov   [si+10],al
35041 mov     [si+SYSDEV.NAME],al ; number of units in name field
35042 ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35043 ;add   [cs:driver_units],al
35044 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35045 add     [cs:driver_units],al ; keep total for all drivers in file
35046
35047 perdrv:
35048 cbw     ; warning no device > 127 units
35049 mov     cx,ax
35050 mov     dh,ah
35051 ;mov   dl,[es:di+32]
35052 mov     dl,[es:di+SYSI_NUMIO] ;get number of devices
35053 mov     ah,dl
35054 add     ah,al ; check for too many devices
35055 cmp     ah,26 ; 'A' - 'Z' is 26 devices
35056 jbe     short ok_block
35057 jmp     badnumblock
35058
35059 ok_block:
35060 or      byte [cs:setdevmarkflag],for_devmark ; 2
35061 call    DevSetBreak ; alloc the device

```

```

35058 0000295B 72B3          jc      short jc_edd
35059 0000295D 26004520        add     [es:di+SYSI_NUMIO],al ; update the amount
35060
35061 00002961 2E0006[8503]      add     [cs:drivenumber],al ; remember amount for next device
35062 00002966 2EC51E[8103]      lds     bx,[cs:bpb_addr] ; point to bpb array
35063
35064 0000296B 2EC42E[6D02]      perunit: les     bp,[cs:DOSINFO]
35065                                     ;les bp,[es:bp+SYSI_DPB] ; get first dpb
35066                                     ; 11/12/2022
35067 00002970 26C46E00          les     bp,[es:bp]
35068                                     ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35069                                     ;les bp,[es:bp+0] ; [es:bp+SYSI_DPB]
35070
35071                                     scandpb: ;cmp word [es:bp+25],-1
35072 00002974 26837E19FF        cmp     word [es:bp+DPB.NEXT_DPB],-1
35073 00002979 7406             je      short foundpb
35074                                     ;les bp,[es:bp+25]
35075 0000297B 26C46E19          les     bp,[es:bp+DPB.NEXT_DPB]
35076 0000297F EBF3             jmp     short scandpb
35077
35078 00002981 2EA1[3B24]        foundpb: mov     ax,[cs:DevBrkAddr]
35079 00002985 26894619        mov     [es:bp+DPB.NEXT_DPB],ax
35080 00002989 2EA1[3D24]        mov     ax,[cs:DevBrkAddr+2]
35081 0000298D 2689461B        mov     [es:bp+DPB.NEXT_DPB+2],ax
35082
35083 00002991 2EC42E[3B24]      les     bp,[cs:DevBrkAddr]
35084 00002996 2E8306[3B24]3D    add     word [cs:DevBrkAddr],DPBSIZ ; 33
35085                                     ; 08/09/2023
35086                                     ; (61 in PC DOS 7.1 IBMBIO.COM)
35087 0000299C E8440D          call    RoundBreakAddr
35088
35089 0000299F 26C74619FFFF    mov     word [es:bp+DPB.NEXT_DPB],-1
35090 000029A5 26C64618FF    mov     byte [es:bp+DPB.FIRST_ACCESS],-1
35091
35092 000029AA 8B37          mov     si,[bx] ;ds:si points to bpb
35093 000029AC 43             inc     bx
35094 000029AD 43             inc     bx ;point to next guy
35095                                     ;mov [es:bp+DPB.DRIVE],dx
35096                                     ; 11/12/2022
35097 000029AE 26895600    mov     [es:bp],dx ; 13/05/2019
35098                                     ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35099                                     ;mov [es:bp+0],dx ; [es:bp+DPB.DRIVE]
35100
35101                                     ; 13/04/2024 - Retro DOS v5.0
35102                                     ; PC DOS 7.1 IBMBIO.COM
35103                                     ;;;
35104 000029B2 52             push    dx
35105 000029B3 51             push    cx
35106 000029B4 BA5241    mov     dx,4152h ; initialize FAT32 extended DPB parameters/fields
35107 000029B7 31C9          xor     cx,cx ; 0
35108                                     ;mov [es:bp+10h],cx
35109 000029B9 26894E1D    mov     [es:bp+DPB.NEXT_FREE],cx ; last allocated cluster #
35110                                     ;cmp [si+0Bh],cx ; BPB.fatsecs16
35111 000029BD 394C0B          cmp     [si+A_BPB.SECTORSPERFAT],cx ; 0
35112 000029C0 7514             jnz     short set_dpb ; FAT DPB (33 bytes) -jnz-
35113                                     ; FAT32 DPB (61 bytes) -jz-
35114
35115 000029C2 26894E39    ;mov [es:bp+39h],cx
35116                                     mov     [es:bp+DPB.FAT32_NXTFREE],cx ; 0
35117                                     ;mov [es:bp+3Bh],cx
35118 000029C6 26894E3B    mov     [es:bp+DPB.FAT32_NXTFREE+2],cx ; 0
35119 000029CA 49             dec     cx ; 0FFFFh ; -1
35120 000029CB 26894E1F    ;mov [es:bp+1Fh],cx
35121                                     mov     [es:bp+DPB.FREE_CNT],cx ; -1 = unknown
35122                                     ;mov [es:bp+21h],cx
35123 000029CF 26894E21    mov     [es:bp+DPB.FREE_CNT+2],cx ; -1 = unknown
35124 000029D3 B95845          mov     cx,4558h ; 'EX' signature for FAT32 extended DPB
35125
35126 set_dpb:
35127                                     ;;;
35127 000029D6 B453          mov     ah,SETDPB ; 53h ;hidden system call
35128 000029D8 CD21          int     21h
35129                                     ; DOS - 2+ internal - TRANSLATE BIOS PARAMETER BLOCK
35130                                     ; DS:SI -> BPB (BIOS Parameter Block)
35131                                     ; ES:BP -> buffer for DOS Drive Parameter Block
35132
35133                                     ; 13/04/2024
35134 000029DA 59             pop     cx
35135 000029DB 5A             pop     dx
35136                                     ;;;
35137
35138                                     ;mov ax,[es:bp+2]
35139 000029DC 268B4602    mov     ax,[es:bp+DPB.SECTOR_SIZE]
35140 000029E0 06             push    es
35141 000029E1 2EC43E[6D02]      les     di,[cs:DOSINFO] ;es:di point to dos info
35142                                     ;cmp ax,[es:di+10h]
35143 000029E6 263B4510    cmp     ax,[es:di+SYSI_MAXSEC]
35144 000029EA 07             pop     es
35145                                     ; 13/04/2024
35146                                     ;jna short iblk_1
35147                                     ;jmp bad_bpb_size_sector
35148                                     ; 29/10/2022
35149 000029EB 777F          ja      short bad_bpb_size_sector
35150
35151 000029ED 1E             iblk_1: push    ds
35152 000029EE 52             push    dx
35153
35154 000029EF 2EC516[3724]      lds     dx,[cs:DevEntry]
35155                                     ;mov [es:bp+13h],dx
35156 000029F4 26895613        mov     [es:bp+DPB.DRIVER_ADDR],dx
35157                                     ;mov [es:bp+15h],ds
35158 000029F8 268C5E15        mov     [es:bp+DPB.DRIVER_ADDR+2],ds
35159
35160 000029FC 5A             pop     dx
35161 000029FD 1F             pop     ds
35162
35163 000029FE 42             inc     dx
35164 000029FF FEC6          inc     dh
35165                                     ;loop perunit
35166                                     ; 13/04/2024
35167                                     ;;;
35168 00002A01 49             dec     cx ; cx = cx - 1
35169                                     ; cx = remain count from [cs:unitcount]
35170 00002A02 7403             jz      short iblk_2 ; cx = 0 -> done
35171 00002A04 E964FF          jmp     perunit ; loop until cx is 0
35172
35173 iblk_2:
35174                                     ;;;
35175 00002A07 0E             push    cs
35176 00002A08 1F             pop     ds
35177
35178 00002A09 E897E3          call    TempCDS ; set cds for new drives
35179                                     ; 31/12/2022
35180                                     ; ds <= cs
35181 linkit:

```



```

35182 00002A0C 2EC43E[6D02]    les     di,[cs:DOSINFO]      ;es:di = dos table
35183 00002A11 268B4D22    mov     cx,[es:di+SYSI_DEV]  ;dx:cx = head of list
35184 00002A15 268B5524    mov     dx,[es:di+SYSI_DEV+2]
35185
35186 00002A19 2EC536[3724]    lds     si,[cs:DevEntry]     ;ds:si = device location
35187 00002A1E 26897522    mov     [es:di+SYSI_DEV],si  ;set head of list in dos
35188 00002A22 268C5D24    mov     [es:di+SYSI_DEV+2],ds
35189 00002A26 8B04        mov     ax,[si]              ;get pointer to next device
35190 00002A28 2EA3[3724]    mov     [cs:DevEntry],ax     ;and save it
35191
35192 00002A2C 890C        mov     [si],cx              ;link in the driver
35193 00002A2E 895402    mov     [si+2],dx
35194
35195 00002A31 5E          pop     si
35196 00002A32 07          pop     es
35197 00002A33 40          inc     ax                   ;ax = ffff (no more devs if yes)?
35198 00002A34 740B        jz      short coffj3
35199
35200 00002A36 2EFE06[6619]    inc     byte [cs:multdeviceflag] ; possibly multiple device driver.
35201 00002A3B E8E80C    call    DevBreak             ; M009
35202    ; 11/12/2022
35203    ; ds = cs (DevBreak)
35204
35205    ; 03/04/2019 - Retro DOS v4.0
35206    ; MSDOS 6.21 IO.SYS - SYSINIT:290Dh
35207 00002A3E E9EDFD    jmp     goodld               ; otherwise pretend we loaded it in
35208
35209    coffj3:
35210    ; 18/12/2022
35211    ; ax = 0
35212    mov     [cs:multdeviceflag],al ; 0
35213    ;mov     byte [cs:multdeviceflag],0 ; reset the flag
35214    call    DevBreak
35215    ; 11/12/2022
35216    ; ds = cs (DevBreak)
35217
35218    ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35219    ; (SYSINIT:2919h)
35220 00002A48 E80204    call    CheckProtmanArena
35221
35222    ; 02/11/2022 (MSDOS 5.0 IO.SYS compatibility)
35223    ;;call CheckProtmanArena ; adjust allocim if Protman$ just
35224    ;
35225    ;
35226    ;
35227    ; 13/04/2024 - Retro DOS v5.0
35228    ; PC DOS 7.1 IBMBIO.COM
35229    ;;call CheckDoubleSpace
35230    ;jmp     coff
35231
35232    ;-----
35233
35234    ; 13/04/2024 - Retro DOS v5.0
35235    ; PC DOS 7.1 IBMBIO.COM
35236    ;;
35237
35238    CheckDoubleSpace:
35239
35240    ;; ifdef  dblspace_hooks
35241
35242    ; Now check for two special MagicDrv cases:
35243    ;
35244    ; a) the last driver load was MagicDrv final placement:
35245    ; -> add number of MagicDrv reserved drives to drivenumber
35246    ;
35247    ; b) MagicDrv is currently in temporary home:
35248    ; -> call it to give it a chance to mount and shuffle drives
35249
35250    ;cmp     byte [cs:MagicHomeFlag],0 ; already home?
35251    ; 13/04/2024 (PCDOS 7.1 IBMBIO.COM)
35252 00002A4B 2EF606[6524]01    test    byte [cs:MagicHomeFlag],1 ; already home?
35253 00002A51 7545        jnz     short no_more_magic_calls ; nothing more to do if so
35254
35255    ; Now inquire of driver whether it is present, and final located
35256
35257    ;mov     ax,multMagicdrv ; 4A11h
35258    ;mov     bx,MD_VERSION ; 0
35259    ;int     2fh              ; ch = number of MagicDrv drive letters
35260    ;or      ax,ax            ; is it there?
35261    ; 13/04/2024 (PCDOS 7.1 IBMBIO.COM)
35262    ;;
35263 00002A53 E800EF    call    get_dblspace_version ; is it there?
35264    ;jnz     short no_more_magic_calls ; done if not
35265 00002A56 750B        jnz     short set_magichomeflag
35266    ;;
35267
35268 00002A58 F7C20080    test    dx,8000h            ; is it final placed?
35269 00002A5C 751C        jnz     short magic_not_yet_home ; skip if not
35270
35271    ; Okay, now the driver is final placed! Set the flag so we
35272    ; don't keep checking it, and add its number of drive letters
35273    ; to drivenumber.
35274
35275    ; 13/04/2024 (PCDOS 7.1 IBMBIO.COM)
35276    ;mov     byte [cs:MagicHomeFlag],0ffh ; set the flag!
35277 00002A5E 2E002E[8503]    add     [cs:drivenumber],ch  ; add number of MagicDrv volumes to
35278    ; the drive number we'll pass to the
35279    ; next loadable block device.
35280    ;jmp     short no_more_magic_calls ; and finished.
35281
35282    ;;
35283    set_magichomeflag:
35284 00002A63 2EC606[6524]01    mov     byte [cs:MagicHomeFlag],1 ; set the flag!
35285 00002A69 E918FB    jmp     coff
35286    ;;
35287
35288    ; 03/04/2019 - Retro DOS v4.0
35289
35290    bad_bpb_size_sector:
35291    pop     si
35292    pop     es
35293 00002A6E BA[9050]    mov     dx,badsiz_pre
35294 00002A71 BB[6E50]    mov     bx,crlfm
35295 00002A74 E8B91F    call    prnerr
35296
35297 00002A77 E90AFB    jmp     coff
35298
35299    magic_not_yet_home:
35300    push    es
35301 00002A7B 56        push    si
35302
35303 00002A7C 2E8B0E[6403]    mov     cx,[cs:memhi]        ; pass it a work buffer
35304 00002A81 2E8B16[A502]    mov     dx,[cs:ALLOCLIM]     ; address in cx (segment)
35305 00002A86 29CA        sub     dx,cx                ; for len dx (paragraphs)

```

```

35306
35307 00002A88 BB0200      mov     bx,2
35308 00002A8B 2EA0[6A19]  mov     al,[cs:driver_units] ; shuffle magicdrives and new drives
35309                                     ; by this many units
35310
35311 ;BUGBUG 29-Oct-1992 bens Take this 55h out after Beta 4
35312 00002A8F B455      mov     ah,55h ; backdoor won't shuffle unless it
35313                                     ; sees this, to prevent bad things
35314                                     ; from happening if people run the
35315                                     ; new driver with an old BIOS
35316 00002A91 2EFF1E[9003] call     far [cs:MagicBackdoor]
35317
35318 00002A96 5E      pop     si
35319 00002A97 07      pop     es
35320
35321 ;no_more_magic_calls:
35322 ;
35323 ;; endif
35324 ; retn
35325
35326 ; 13/04/2024
35327 ;;;
35328 no_more_magic_calls:
35329 00002A98 E9E9FA      jmp     coff
35330 ;;;
35331
35332 -----
35333 ; country command
35334 ; the syntax is:
35335 ; country=country id {,codepage {,path}}
35336 ; country=country id {,,path} :default codepage id in dos
35337 -----
35338
35339 ; 30/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
35340 ; (SYSINIT:2663h)
35341 tryq:
35342 00002A9B 80FC51      cmp     ah,CONFIG_COUNTRY ; 'Q'
35343 00002A9E 7403      je      short tryq_cont
35344 skip_it3:
35345 00002AA0 E90D01      jmp     tryf
35346 tryq_cont:
35347
35348 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35349 ; (SYSINIT:297Eh)
35350 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35351 ;%if 0
35352 ;ifdef MULTI_CONFIG
35353 00002AA3 E8741B      call    query_user ; query the user if config_cmd
35354 00002AA6 72F8      jc      short skip_it3 ; has the CONFIG_OPTION_QUERY bit set
35355 ;endif
35356 ;%endif ; 02/11/2022
35357
35358 ; 31/12/2022
35359 ;xor bx,bx
35360 00002AA8 31C9      xor     cx,cx
35361 ; 14/12/2022
35362 ; ds = cs
35363 ; bx = 0
35364 ;mov byte [cs:centry_drv],0 ; reset the drive,path to default value.
35365 ;mov word [cs:p_code_page],0
35366 ; 31/12/2022
35367 ; cx = 0
35368 ;mov [centry_drv],b1 ; 0
35369 ;mov [p_code_page],bx ; 0
35370 00002AAA 880E[DB4A]  mov     [centry_drv],cl ; 0
35371 00002AAE 890E[7A22]  mov     [p_code_page],cx ; 0
35372
35373 00002AB2 BF[4322]    mov     di,centry_parms
35374 ;xor cx,cx ; 31/12/2022
35375 ; 03/01/2023
35376 ;mov dx,cx
35377 do52:
35378 00002AB5 E8AD03      call    sysinit_parse
35379 00002AB8 730B      jnc     short if52 ; parse error,check error code and
35380
35381 00002ABA E8E000      call    centry_error ; show message and end the search loop.
35382 ; 14/12/2022
35383 ; ds = cs
35384 00002ABD C706[7822]FFFF mov     word [p_centry_code],-1
35385 ;mov word [cs:p_centry_code],-1 ; signals that parse error.
35386 00002AC3 EB34      jmp     short sr52
35387 if52:
35388 00002AC5 83F8FF      cmp     ax,_$P_RC_EOL ; 0FFFFh; end of line?
35389 00002AC8 742F      jz      short sr52 ; then end the search loop
35390
35391 ;cmp byte [cs:result_val+$_P_Result_Blk.Type],$_P_number ; numeric?
35392 ; 14/12/2022
35393 ; ds = cs
35394 00002ACA 803E[1522]01 cmp     byte [result_val],$_P_Number
35395 ;cmp byte [cs:result_val],$_P_Number
35396 00002ACF 7512      jnz     short if56
35397
35398 ;;mov ax,[cs:rw_dword]
35399 ;mov ax,[cs:result_val+$_P_Result_Blk.Picked_Val]
35400 ; 14/12/2022
35401 00002AD1 A1[1922]    mov     ax,[result_val+$_P_Result_Blk.Picked_Val]
35402 00002AD4 83F901      cmp     cx,1
35403 00002AD7 7505      jne     short if57
35404
35405 ;mov [cs:p_centry_code],ax
35406 ; 14/12/2022
35407 00002AD9 A3[7822]    mov     [p_centry_code],ax
35408
35409 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35410 ;jmp short en57
35411 ; 12/12/2022
35412 ;jmp short en56
35413 00002ADC EBD7      jmp     short do52
35414 if57:
35415 ;mov [cs:p_code_page],ax
35416 ; 14/12/2022
35417 ; ds = cs
35418 00002ADE A3[7A22]    mov     [p_code_page],ax
35419 en57:
35420 ;jmp short en56 ; path entered
35421 ; 12/12/2022
35422 00002AE1 EBD2      jmp     short do52
35423 if56:
35424 00002AE3 1E      push    ds
35425 00002AE4 06      push    es
35426 00002AE5 56      push    si
35427 00002AE6 57      push    di
35428
35429 00002AE7 0E      push    cs

```



```

35430 00002AE8 07      pop     es
35431
35432      ;lds    si,[cs:rv_dword]      ; move the path to known place.
35433      ; 14/12/2022
35434 00002AE9 C536[1922] lds     si,[rv_dword]
35435 00002AED BF[DB4A]   mov     di,cntry_drv
35436 00002AF0 E82C1F     call    move_asciiz
35437
35438 00002AF3 5F      pop     di
35439 00002AF4 5E      pop     si
35440 00002AF5 07      pop     es
35441 00002AF6 1F      pop     ds
35442
35443 00002AF7 EBBC      jmp     short do52
35444
35445 sr52:
35446      ; 14/12/2022
35447 00002AF9 833E[7822]FF cmp     word [p_cntry_code],-1
35448      ;cmp    word [cs:p_cntry_code],-1      ; had a parse error?
35449 00002AFE 7509      jne     short tryq_open
35450 00002B00 E981FA     jmp     coff
35451
35452 tryqbad:              ;"invalid country code or code page"
35453 00002B03 F9      stc
35454 00002B04 BA[D750]   mov     dx,badcountry
35455 00002B07 EB79      jmp     tryqchkerr
35456
35457 tryq_open:
35458      ; 14/12/2022
35459      ; ds = cs
35460 00002B09 803E[DB4A]00 cmp     byte [cntry_drv],0
35461      ;cmp    byte [cs:cntry_drv],0
35462 00002B0E 7405      je      short tryq_def
35463 00002B10 BA[DB4A]   mov     dx,cntry_drv
35464 00002B13 EB03      jmp     short tryq_openit
35465
35466 tryq_def:
35467 00002B15 BA[DD4A]   mov     dx,cntry_root
35468
35469 tryq_openit:
35470 00002B18 B8003D     mov     ax,3D00h      ;open a file
35471 00002B1B F9      stc
35472 00002B1C CD21      int     21h
35473      jc      short tryqfilebad      ;open failure
35474
35475      ; 14/12/2022
35476 00002B20 A3[5C03]   mov     [cntryfilehandle],ax
35477      ;mov    [cs:cntryfilehandle],ax      ;save file handle
35478 00002B23 89C3      mov     bx,ax
35479 00002B25 A1[7822]   mov     ax,[p_cntry_code]
35480 00002B28 8B16[7A22] mov     dx,[p_code_page]
35481      ;mov    ax,[cs:p_cntry_code]
35482      ;mov    dx,[cs:p_code_page]      ;now,ax=country id,bx=filehandle
35483      ;mov    cx,[cs:memhi]
35484 00002B2C 8B0E[6403] mov     cx,[memhi]
35485 00002B30 81C18001   add     cx,384      ;need 6k buffer to handle country.sys
35486      ;M023
35487
35488      ; 14/12/2022
35489 00002B34 3B0E[A502] cmp     cx,[ALLOCLIM]
35490      ;cmp    cx,[cs:ALLOCLIM]
35491 00002B38 7745      ja      short tryqmemory      ;cannot allocate the buffer for country.sys
35492
35493 00002B3A BE[DB4A]   mov     si,cntry_drv      ;ds:si -> cntry_drv
35494 00002B3D 803C00   cmp     byte [si],0      ;default path?
35495 00002B40 7502      jne     short tryq_set_for_dos
35496
35497 00002B42 46      inc     si
35498 00002B43 46      inc     si      ;ds:si -> cntry_root
35499
35500 tryq_set_for_dos:
35501      ; 14/12/2022
35502      ; ds = cs
35503 00002B44 C43E[7902] les     di,[sysi_country]
35504      ;les    di,[cs:sysi_country]      ;es:di -> country info tab in dos
35505 00002B48 57      push    di      ;save di
35506      ;add    di,8
35507 00002B49 83C708     add     di,country_cdpdpg_info.ccPath_CountrySys ; 8
35508 00002B4C E8D01E     call    move_asciiz      ;set the path to country.sys in dos.
35509 00002B4F 5F      pop     di      ;es:di -> country info tab again.
35510
35511      ; 14/12/2022
35512 00002B50 8B0E[6403] mov     cx,[memhi]
35513      ;mov    cx,[cs:memhi]
35514 00002B54 8ED9      mov     ds,cx
35515 00002B56 31F6      xor     si,si      ;ds:si -> 2k buffer to be used.
35516 00002B58 E8601D     call    setdoscountryinfo ;now do the job!!!
35517      ; ds <= cs ; 14/12/2022
35518 00002B5B 7325      jnc     short tryqchkerr      ;read error or could not find country,code page combination
35519
35520 00002B5D 83F9FF     cmp     cx,-1      ;could not find matching country_id,code page?
35521 00002B60 74A1      je      short tryqbad      ;then "invalid country code or code page"
35522
35523 tryqfilebad:
35524 00002B62 0E      push    cs
35525 00002B63 07      pop     es
35526 00002B64 2E803E[DB4A]00 cmp     byte [cs:cntry_drv],0 ;is the default file used?
35527 00002B6A 7405      je      short tryqdefbad
35528
35529 00002B6C BE[DB4A]   mov     si,cntry_drv
35530 00002B6F EB03      jmp     short tryqbadload
35531
35532 tryqdefbad:          ;default file has been used.
35533 00002B71 BE[DD4A]   mov     si,cntry_root      ;es:si -> \country.sys in sysinit_seg
35534
35535 tryqbadload:
35536 00002B74 E8B31E     call    badload      ;ds will be restored to sysinit_seg
35537      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35538      ; (SYSINIT:2A69h)
35539 00002B77 8B0E[A302] mov     cx,[CONFBOT] ; ds = cs (from badload)
35540      ;mov    cx,[cs:CONFBOT]
35541      ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35542      ;mov    cx,[cs:top_of_cdss]
35543      ; 11/12/2022
35544      ; ds = cs
35545 00002B7B 8EC1      mov     cx,[top_of_cdss] ; mov cx,[CONFBOT]
35546 00002B7D EB13      mov     es,cx      ;restore es -> confbot.
35547      jmp     short coffj4
35548
35549 tryqmemory:
35550 00002B7F BA[1A51]   mov     dx,insufmemory
35551
35552 tryqchkerr:
35553      ;mov    cx,[cs:CONFBOT]
35554      ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35555      ;mov    cx,[cs:top_of_cdss]

```

```

35554             ; 12/12/2022
35555 00002B82 0E    push    cs
35556 00002B83 1F    pop     ds
35557             ; 31/12/2022 - Retro DOS v4.2
35558 00002B84 8B0E[A302] mov    cx,[CONFBOT] ; (MSDOS 6.21 IO.SYS, SYSINIT)
35559             ;mov    cx,[top_of_cdss] ; mov cx,[CONFBOT]
35560 00002B88 8EC1    mov     es,cx          ;restore es -> confbot seg
35561             ;push    cs
35562             ;pop     ds          ;restore ds to sysinit_seg
35563 00002B8A 7306    jnc     short coffj4      ;if no error,then exit
35564             ;
35565 00002B8C E8C31E    call    print            ;else show error message
35566 00002B8F E82603    call    error_line
35567             ;
35568             ;mov     bx,[cs:centryfilehandle]
35569             ; 11/12/2022
35570             ; ds = cs
35571 00002B92 8B1E[5C03] mov    bx,[centryfilehandle]
35572 00002B96 B43E    mov     ah,3Eh
35573 00002B98 CD21    int     21h              ;close a file. don't care even if it fails.
35574 00002B9A E9E7F9    jmp     coff
35575             ;
35576             ;-----
35577             ;
35578             ;centry_error:
35579             ;
35580             ;function: show "invalid country code or code page" messages,or
35581             ;          "error in country command" depending on the error code
35582             ;          in ax returned by sysparse;
35583             ;in: ax - error code
35584             ;     ds - sysinitseg
35585             ;     es - confbot
35586             ;out:      show message.  dx destroyed.
35587             ;
35588 00002B9D 83F806    cmp     ax,_$P_Out_Of_Range ; 6
35589 00002BA0 7505    jne     short if64
35590 00002BA2 BA[D750]    mov     dx,badcountry      ;"invalid country code or code page"
35591 00002BA5 EB03    jmp     short en64
35592             ;
35593 00002BA7 BA[FD50]    mov     dx,badcountrycom    ;"error in contry command"
35594             ;
35595 00002BAA E8A51E    call    print
35596             ;call    error_line
35597             ;retn
35598             ; 11/12/2022
35599 00002BAD E90803    jmp     error_line
35600             ;
35601             ;-----
35602             ; files command
35603             ;-----
35604             ;
35605             ;*****
35606             ; function: parse the parameters of files= command.
35607             ;
35608             ; input :
35609             ;     es:si -> parameters in command line.
35610             ; output:
35611             ;     variable files set.
35612             ;
35613             ; subroutines to be called:
35614             ;     sysinit_parse
35615             ; logic:
35616             ; {
35617             ;     set di points to files_parms;
35618             ;     set dx,cx to 0;
35619             ;     while (end of command line)
35620             ;     { sysinit_parse;
35621             ;       if (no error) then
35622             ;         files = result_val._$P_picked_val
35623             ;       else
35624             ;         error exit;
35625             ;     };
35626             ; }
35627             ;
35628             ;*****
35629             ;
35630             ;tryf:
35631 00002BB0 80FC46    cmp     ah,CONFIG_FILES ; 'F'
35632 00002BB3 7528    jne     short tryl
35633             ;
35634             ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35635             ; (SYSINIT:2AABh)
35636             ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35637             ;%if 0
35638             ;ifdef    MULTI_CONFIG
35639 00002BB5 E8621A    call    query_user          ; query the user if config_cmd
35640 00002BB8 7223    jc      short tryl          ; has the CONFIG_OPTION_QUERY bit set
35641             ;endif
35642             ;%endif ; 30/10/2022
35643             ;
35644             ; 14/12/2022
35645             ; ds = cs
35646             ;
35647 00002BBA BF[7C22]    mov     di,files_parms
35648 00002BBD 31C9    xor     cx,cx
35649             ; 03/01/2023
35650             ;mov     dx,cx
35651             ;
35652 00002BBF E8A302    call    sysinit_parse
35653 00002BC2 7303    jnc     short if67          ; parse error
35654             ;call    badparm_p          ; and show messages and end the search loop.
35655             ;jmp     short sr67
35656             ; 03/01/2023
35657 00002BC4 E98D01    jmp     badparm_p_coff
35658             ;
35659 00002BC7 83F8FF    cmp     ax,_$P_RC_EOL      ; end of line?
35660 00002BCA 7408    je      short en67          ; then end the $endloop
35661             ;
35662             ; 14/12/2022
35663             ; ds = cs
35664             ;mov     al,[cs:rv_dword]
35665             ;mov     al,[cs:result_val+_$P_Result_Blk.Picked_val]
35666             ;mov     [cs:p_files],al          ; save it temporarily
35667             ;mov     al,[rv_dword]
35668 00002BCC A0[1922]    mov     al,[result_val+_$P_Result_Blk.Picked_val]
35669 00002BCF A2[9B22]    mov     [p_files],al
35670             ;
35671 00002BD2 EBEB     jmp     short do67
35672             ;
35673             ;en67:
35674             ; 14/12/2022
35675             ; ds = cs
35676 00002BD4 A0[9B22]    mov     al,[p_files]
35677 00002BD7 A2[9F02]    mov     [FILES],al
35678             ;mov     al,[cs:p_files]

```

```

35678             ;mov     [cs:FILES],al           ; no error. really set the value now.
35679 sr67:
35680 00002BDA E9A7F9     jmp     coff
35681
35682 ; 04/04/2019 - Retro DOS v4.0
35683
35684 ;-----
35685 ; lastdrive command
35686 ;-----
35687
35688 *****
35689 ; function: parse the parameters of lastdrive= command.
35690 ;
35691 ; input :
35692 ; es:si -> parameters in command line.
35693 ; output:
35694 ; set the variable num_cds.
35695 ;
35696 ; subroutines to be called:
35697 ; sysinit_parse
35698 ; logic:
35699 ; {
35700 ;     set di points to ldrv_parms;
35701 ;     set dx,cx to 0;
35702 ;     while (end of command line)
35703 ;     { sysinit_parse;
35704 ;       if (no error) then
35705 ;         set num_cds to the returned value;
35706 ;       else /*error exit*/
35707 ;         error exit;
35708 ;     };
35709 ; };
35710 ;
35711 *****
35712
35713 tryl:
35714 00002BDD 80FC4C     cmp     ah,CONFIG_LASTDRIVE ; 'L'
35715 00002BE0 7528     jne     short tryp
35716
35717 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35718 ; (SYSINIT:2AE0h)
35719 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35720 ;%if 0
35721 00002BE2 E8351A     call    query_user      ; query the user if config_cmd
35722 00002BE5 7223     jc      short tryp      ; has the CONFIG_OPTION_QUERY bit set
35723 ;endif
35724 ;%endif ; 30/10/2022
35725
35726 ; 14/12/2022
35727 ; ds = cs
35728
35729 00002BE7 BF[D322]    mov     di,ldrv_parms
35730 00002BEA 31C9     xor     cx,cx
35731 ; 03/01/2023
35732 ;mov     dx,cx
35733
35734 00002BEC E87602     call    sysinit_parse
35735 00002BEF 7303     jnc     short if73      ; parse error
35736 ;call    badparm_p      ; and show messages and end the search loop.
35737 ;jmp     short sr73
35738 ; 03/01/2023
35739 00002BF1 E96001     jmp     badparm_p_coff
35740
35741 00002BF4 83F8FF     if73:  cmp     ax,_$_RC_EOL ; end of line?
35742 00002BF7 7408     je      short en73      ; then end the $endloop
35743
35744 ; 14/12/2022
35745 ; ds = cs
35746 ;mov     al,[cs:rv_dword]
35747 ;mov     al,[cs:rv_byte] ; pick up the drive number
35748 ;mov     [cs:p_ldrv],al ; save it temporarily
35749
35750 ;mov     al,[rv_dword]
35751 00002BF9 A0[1922]    mov     al,[rv_byte]
35752 00002BFC A2[E722]    mov     [p_ldrv],al
35753
35754 00002BFF EBEB     jmp     short do73
35755
35756 en73:
35757 ; 14/12/2022
35758 ; ds = cs
35759 00002C01 A0[E722]    mov     al,[p_ldrv]
35760 00002C04 A2[A202]    mov     [NUM_CDS],al
35761 ;mov     al,[cs:p_ldrv]
35762 ;mov     [cs:NUM_CDS],al ; no error. really set the value now.
35763 00002C07 E97AF9     sr73:  jmp     coff
35764
35765 ;-----
35766 ; setting drive parameters
35767 ;-----
35768
35769
35770 00002C0A 80FC50     tryp:  cmp     ah,CONFIG_DRIVPARM ; 'P'
35771 00002C0D 7516     jne     short tryk
35772
35773 ; 31/12/2022 - Retro DOS v4.2
35774 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35775 ;%if 0
35776 ;ifdef MULTI_CONFIG
35777 00002C0F E8081A     call    query_user      ; query the user if config_cmd
35778 00002C12 7211     jc      short tryk      ; has the CONFIG_OPTION_QUERY bit set
35779 ;endif
35780 ;%endif ; 30/10/2022
35781
35782 00002C14 E8EC0E     call    parseline
35783 00002C17 7209     jc      short trypbad
35784 00002C19 E8050E     call    setparms
35785 00002C1C E8470E     call    diddleback
35786
35787 ; No error check here, because setparms and diddleback have no error
35788 ; returns, and setparms as coded now can return with carry set.
35789 ; jc short trypbad
35790
35791 ; 12/12/2022
35792 ; cf = 0
35793 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35794 ;jc      short trypbad
35795
35796 00002C1F E962F9     jmp     coff
35797
35798 00002C22 E95F02     trypbad: jmp     badop
35799
35800 ;-----
35801 ; setting internal stack parameters

```

```

35802 ; stacks=m,n where
35803 ; m is the number of stacks (range 8 to 64,default 9)
35804 ; n is the stack size (range 32 to 512 bytes,default 128)
35805 ; j.k. 5/5/86: stacks=0,0 implies no stack installation.
35806 ; any combinations that are not within the specified limits will
35807 ; result in "unrecognized command" error.
35808 ;-----
35809 ;*****
35810 ;
35811 ;
35812 ; function: parse the parameters of stacks= command.
35813 ; the minimum value for "number of stacks" and "stack size" is
35814 ; 8 and 32 each. in the definition of sysparse value list,they
35815 ; are set to 0. this is for accepting the exceptional case of
35816 ; stacks=0,0 case (,which means do not install the stack.)
35817 ; so,after sysparse is done,we have to check if the entered
35818 ; values (stack_count,stack_size) are within the actual range,
35819 ; (or if "0,0" pair has been entered.)
35820 ;
35821 ; input :
35822 ; es:si -> parameters in command line.
35823 ; output:
35824 ; set the variables stack_count,stack_size.
35825 ;
35826 ; subroutines to be called:
35827 ; sysinit_parse
35828 ; logic:
35829 ; {
35830 ; set di points to stks_parms;
35831 ; set dx,cx to 0;
35832 ; while (end of command line)
35833 ; { sysinit_parse;
35834 ; if (no error) then
35835 ; { if (cx == 1) then /* first positional = stack count */
35836 ; p_stack_count = result_val._$P_picked_val;
35837 ; if (cx == 2) then /* second positional = stack size */
35838 ; p_stack_size = result_val._$P_picked_val;
35839 ; }
35840 ; else /*error exit*/
35841 ; error exit;
35842 ; }
35843 ; here check p_stack_count,p_stack_size if it meets the condition;
35844 ; if o.k.,then set stack_count,stack_size;
35845 ; else error_exit;
35846 ; }
35847 ;*****
35848
35849 tryk:
35850 ;if stacksw
35851 ;cmp ah,CONFIG_STACKS ; 'K'
35852 ;je short do_tryk
35853 skip_it4:
35854 ;jmp short trys ; 15/12/2022
35855 do_tryk:
35856 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35857 ; (SYSINIT:2B33h)
35858 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35859 ;%if 0
35860 ;ifdef MULTI_CONFIG
35861 ;call query_user ; query the user if config_cmd
35862 ;jc short skip_it4 ; has the CONFIG_OPTION_QUERY bit set
35863 ;endif
35864 ;%endif ; 30/10/2022
35865 ; 14/12/2022
35866 ; ds = cs
35867
35868 mov di,stks_parms
35869 xor cx,cx
35870 ; 03/01/2023
35871 ;mov dx,cx
35872 do79:
35873 call sysinit_parse
35874 jnc short if79 ; parse error
35875 mov dx,badstack ; "invalid stack parameter"
35876 call print ; and show messages and end the search loop.
35877 call error_line
35878 ;jmp sr79
35879 ; 11/12/2022
35880 jmp short sr79
35881 if79:
35882 cmp ax,_$P_RC_EOL ; end of line?
35883 je short en79 ; then end the $endloop
35884 ; 14/12/2022
35885 ; ds = cs
35886 ;mov ax,[cs:rv_dword]
35887 ;mov ax,[cs:result_val+_$P_Result_Blk.Picked_Val]
35888 ;mov ax,[rv_dword]
35889 mov ax,[result_val+_$P_Result_Blk.Picked_Val]
35890 cmp cx,1
35891 jne short if83
35892 ; 14/12/2022
35893 ;mov [cs:p_stack_count],ax
35894 ;jmp short en83
35895 mov [p_stack_count],ax
35896 jmp short do79
35897 if83:
35898 ; 14/12/2022
35899 ;mov [cs:p_stack_size],ax
35900 mov [p_stack_size],ax
35901 en83:
35902 jmp short do79
35903 en79:
35904 ; 14/12/2022
35905 ; ds = cs
35906 mov ax,[p_stack_count]
35907 or ax,ax
35908 jz short if87
35909 ; 14/12/2022
35910 ;cmp word [p_stack_count],0
35911 ;cmp word [cs:p_stack_count],0
35912 ;je short if87
35913 ; 14/12/2022
35914 cmp ax,mincount ; 8
35915 cmp word [cs:p_stack_count],mincount ; 8
35916 ; 15/12/2022

```

```

35926 00002C67 721F          jb      short en87
35927 00002C69 833E[1F23]20    cmp     word [p_stack_size],minsize ; 32
35928                      ; cmp     word [cs:p_stack_size],minsize ; 32
35929                      ; 15/12/2022
35930 00002C6E 7218          jb      short en87
35931 if94:
35932                      ; 14/12/2022
35933                      ; ds = cs
35934                      ; ax = [p_stack_count]
35935                      ; mov     ax,[p_stack_count]
35936                      ; ; mov     ax,[cs:p_stack_count]
35937 00002C70 A3[8C02]        mov     [stack_count],ax
35938                      ; mov     [cs:stack_count],ax
35939                      ; mov     ax,[cs:p_stack_size]
35940 00002C73 A1[1F23]        mov     ax,[p_stack_size]
35941                      ; mov     [cs:stack_size],ax
35942 00002C76 A3[8E02]        mov     [stack_size],ax
35943                      ; mov     word [cs:stack_addr],-1      ; stacks= been accepted.
35944 00002C79 C706[9002]FFFF  mov     word [stack_addr],-1
35945 sr79:
35946 00002C7F E902F9          jmp     coff
35947
35948 if87:
35949                      ; 14/12/2022
35950 00002C82 3906[1F23]      cmp     [p_stack_size],ax ; 0
35951 00002C86 74E8            je      short if94 ; ax = [p_stack_count] = 0
35952                      ; cmp     word [cs:p_stack_size],0
35953                      ; je      short if94
35954 en87:
35955                      ; 15/12/2022
35956                      ; ([p_stack_count] is invalid, use default values)
35957                      ; 14/12/2022
35958                      ; ds = cs
35959 00002C88 C706[8C02]0900  mov     word [stack_count],defaultcount ; 9
35960 00002C8E C706[8E02]8000  mov     word [stack_size],defaultsize ; 128
35961 00002C94 C706[9002]0000  mov     word [stack_addr],0
35962                      ; mov     word [cs:stack_count],defaultcount ; 9
35963                      ; ; reset to default value.
35964                      ; mov     word [cs:stack_size],defaultsize ; 128
35965                      ; mov     word [cs:stack_addr],0
35966
35967 00002C9A BA[8951]         mov     dx,badstack
35968 00002C9D E8B21D         call    print
35969 00002CA0 E81502         call    error_line
35970 00002CA3 EBDA          jmp     short sr79
35971
35972                      ; 15/12/2022
35973 %if 0
35974                      mov     di,stks_parms
35975                      xor     cx,cx
35976                      ; 03/01/2023
35977                      ; mov     dx,cx
35978 do79:
35979                      call    sysinit_parse
35980                      jnc     short if79      ; parse error
35981
35982                      mov     dx,badstack      ; "invalid stack parameter"
35983                      call    print              ; and show messages and end the search loop.
35984                      call    error_line
35985                      ; jmp     sr79
35986                      ; 11/12/2022
35987                      jmp     short sr79
35988 if79:
35989                      cmp     ax,_P_RC_EOL      ; end of line?
35990                      je      short en79        ; then end the $endloop
35991
35992                      ; mov     ax,[cs:rv_dword]
35993                      mov     ax,[cs:result_val+_P_Result_Blk.Picked_val]
35994                      cmp     cx,1
35995                      jne     short if83
35996
35997                      mov     [cs:p_stack_count],ax
35998                      jmp     short en83
35999 if83:
36000                      mov     [cs:p_stack_size],ax
36001 en83:
36002                      jmp     short do79
36003 en79:
36004                      cmp     word [cs:p_stack_count],0
36005                      je      short if87
36006
36007                      cmp     word [cs:p_stack_count],mincount ; 8
36008                      jb      short 1188
36009                      cmp     word [cs:p_stack_size],minsize ; 32
36010                      jnb     short if88
36011 1188:
36012                      mov     word [cs:p_stack_count],-1 ; invalid
36013 if88:
36014                      jmp     short en87
36015
36016                      ; 11/12/2022
36017 if94:
36018                      mov     ax,[cs:p_stack_count]
36019                      mov     [cs:stack_count],ax
36020                      mov     ax,[cs:p_stack_size]
36021                      mov     [cs:stack_size],ax
36022                      mov     word [cs:stack_addr],-1      ; stacks= been accepted.
36023 sr79:
36024                      jmp     coff
36025
36026 if87:
36027                      cmp     word [cs:p_stack_size],0
36028                      je      short en87
36029                      mov     word [cs:p_stack_count],-1 ; invalid
36030 en87:
36031                      cmp     word [cs:p_stack_count],-1 ; invalid?
36032                      jne     short if94
36033
36034                      mov     word [cs:stack_count],defaultcount ; 9
36035                      ; reset to default value.
36036                      mov     word [cs:stack_size],defaultsize ; 128
36037                      mov     word [cs:stack_addr],0
36038
36039                      mov     dx,badstack
36040                      call    print
36041                      call    error_line
36042                      jmp     short sr79
36043
36044 %endif
36045
36046                      ; 11/12/2022
36047 %if 0
36048 if94:
36049                      mov     ax,[cs:p_stack_count]

```

```

36050         mov     [cs:stack_count],ax
36051         mov     ax,[cs:p_stack_size]
36052         mov     [cs:stack_size],ax
36053         mov     word [cs:stack_addr],-1      ; stacks= been accepted.
36054 sr79:
36055         jmp      coff
36056 %endif
36057         ;endif
36058
36059 ;-----
36060 ; shell command
36061 ;-----
36062
36063 trys:
36064 00002CA5 80FC53      cmp     ah,CONFIG_SHELL ; 'S'
36065 00002CA8 755A      jne     short tryx
36066
36067 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
36068 ; (SYSINIT:2BE1h)
36069 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36070 %if 0
36071 ;ifdef MULTI_CONFIG
36072 00002CAA E86D19      call    query_user      ; query the user if config_cmd
36073 00002CAD 7255      jc     short tryx      ; has the CONFIG_OPTION_QUERY bit set
36074 ; 14/04/2024
36075 ; ds = cs
36076 ;mov     byte [cs:newcmd],1
36077 00002CAF C606[284B]01 mov     byte [newcmd],1
36078 ;endif
36079 %endif ; 30/10/2022
36080
36081 ;mov     word [cs:command_line],0 ; zap length,first byte of command-line
36082 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36083 ;mov     byte [cs:command_line+1],0
36084 ; 15/12/2022
36085 ; ds = cs
36086 ; 08/09/2023
36087 ;mov     byte [command_line+1],0
36088 00002CB4 C706[B94B]0000 mov     word [command_line],0 ; zap length,first byte of command-line
36089
36090 00002CBA BF[2C4B]      mov     di,commnd+1      ; we already have the first char
36091 00002CBD 8845FF      mov     [di-1],al      ; of the new shell in AL, save it now
36092
36093 00002CC0 E8EA1A      call    getchr
36094 00002CC3 08C0      or     al,al
36095 00002CC5 741C      jz     short getshparms ; put a ZERO right after the filename
36096
36097 00002CC7 3C20      cmp     al," "
36098 00002CC9 7209      jb     short endofshell ; this may happen if there are no args
36099 00002CCB 8805      mov     [di],al      ; I suppose...
36100 00002CCD 47      inc     di
36101 ;cmp     di,commnd+63 ; this makes sure we don't overflow
36102 ;jb     short storeshell ; commnd (the filename)
36103 ;jmp     short endofshell
36104 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36105 ;jmp     short storeshell
36106 ; 03/01/2023
36107 00002CCE 81FF[6A4B]      cmp     di,commnd+63 ; this makes sure we don't overflow
36108 00002CD2 72EC      jb     short storeshell ; commnd (the filename)
36109 ;jmp     short endofshell
36110
36111 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36112 ;getshparms:
36113 ; mov     byte [di],0 ; zero-terminate the filename
36114 ; mov     di,command_line+1 ; prepare to process the command-line
36115 ;
36116 ;parmloop:
36117 ; call    getchr
36118 ; cmp     al," "
36119 ; jb     short endofparms
36120 ; mov     [di],al
36121 ; inc     di
36122 ; cmp     di,command_line+126
36123 ; jb     short parmloop
36124 ;endofparms:
36125 ; mov     cx,di
36126 ; sub     cx,command_line+1
36127 ; mov     [cs:command_line],cx
36128 ;
36129 ;endofshell:
36130 ; mov     byte [di],0 ; zero-terminate the filename (or
36131 ; ; the command-line as the case may be)
36132 ;
36133 ;skipline:
36134 ; cmp     al,1f ; 0Ah ; the safest way to eat the rest of
36135 ; je     short endofline ; the line: watch for ever-present LF
36136 ;call    getchr
36137 ; jnc     short skipline ; keep it up as long as there are chars
36138 ;
36139 ;endofline:
36140 ; jmp     conflp
36141
36142 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36143 00002CD4 C60500      endofshell: mov     byte [di],0 ; zero-terminate the filename (or
36144 ; ; the command-line as the case may be)
36145 ; 11/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
36146 ; MSDOS 6.21 IO.SYS - SYSINIT:2C33h
36147 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:314Eh
36148 ;call    getchr
36149 ; skipline: ; MSDOS 6.21 IO.SYS - SYSINIT:2C33h
36150 00002CD7 3C0A      cmp     al,1f ; 0Ah ; the safest way to eat the rest of
36151 00002CD9 7405      je     short endofline ; the line: watch for ever-present LF
36152 00002CDB E8CF1A      call    getchr
36153 ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.1 IO.SYS)
36154 ; (SYSINIT:2C3Ah)
36155 00002CDE 73F7      jnb     short skipline
36156
36157 ;endofline:
36158 00002CE0 E94AF8      jmp     conflp
36159
36160 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36161 ;getshparms:
36162 ; 18/12/2022
36163 ; al = 0
36164 00002CE3 8805      mov     [di],al ; 0
36165 ;mov     byte [di],0 ; zero-terminate the filename
36166 00002CE5 BF[BA4B]      mov     di,command_line+1 ; prepare to process the command-line
36167 ;
36168 00002CE8 E8C21A      parmloop: call    getchr
36169 00002CEB 3C20      cmp     al," " ; 20h
36170 ;jb     short endofshell
36171 ; 03/01/2023
36172 00002CED 7209      jb     short endofparms
36173

```



```

36174 00002CEF 8805      mov     [di],al
36175 00002CF1 47        inc     di
36176                      ;jmp     short parmloop
36177                      ; 03/01/2023 - Retro DOS v4.2
36178 00002CF2 81FF[374C] cmp     di,command_line+126
36179 00002CF6 72F0      jnb     short parmloop
36180
36181                      ; 03/01/2023 - Retro DOS v4.2
36182 endofparms:
36183 00002CF8 89F9      mov     cx,di
36184 00002CFA 81E9[BA4B] sub     cx,command_line+1
36185                      ;mov     [cs:command_line],c1
36186                      ; 03/01/2023
36187 00002CFE 880E[B94B] mov     [command_line],c1
36188 00002D02 EBD0      jmp     short endofshell
36189
36190                      ;-----
36191                      ; fcbs command
36192                      ;-----
36193
36194                      ;*****
36195                      ; function: parse the parameters of fcbs= command.
36196                      ;
36197                      ; input :
36198                      ; es:si -> parameters in command line.
36199                      ; output:
36200                      ; set the variables fcbs,keep.
36201                      ;
36202                      ; subroutines to be called:
36203                      ; sysinit_parse
36204                      ; logic:
36205                      ; {
36206                      ;   set di points to fcbs_parms;
36207                      ;   set dx,cx to 0;
36208                      ;   while (end of command line)
36209                      ;   { sysparse;
36210                      ;     if (no error) then
36211                      ;     { if (cx == 1) then /* first positional = fcbs */
36212                      ;       fcbs = result_val._$P_picked_val;
36213                      ;       if (cx == 2) then /* second positional = keep */
36214                      ;         keep = result_val._$P_picked_val;
36215                      ;     }
36216                      ;     else /*error exit*/
36217                      ;       error exit;
36218                      ;   };
36219                      ; };
36220                      ;*****
36221
36222 tryx:
36223 00002D04 80FC58      cmp     ah,CONFIG_FCBS ; 'X'
36224                      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
36225 00002D07 7534      jne     short try1
36226                      ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36227                      ;jne     short tryx ; comment command
36228
36229                      ; 31/12/2022 - Retro DOS v4.2
36230                      ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36231                      ;%if 0
36232                      ;ifdef MULTI_CONFIG
36233 00002D09 E80E19      call    query_user ; query the user if config_cmd
36234 00002D0C 722F      jc     short try1 ; has the CONFIG_OPTION_QUERY bit set
36235                      ;endif
36236                      ;%endif ; 30/10/2022
36237
36238 00002D0E BF[9C22]      mov     di,fcbs_parms
36239 00002D11 31C9      xor     cx,cx
36240                      ; 03/01/2023
36241                      ;mov     dx,cx
36242 do98:
36243      call    sysinit_parse
36244                      ; 03/01/2023
36245                      ;jnc     short if98 ; parse error
36246                      ;call    badparm_p ; and show messages and end the search loop.
36247                      ;jmp     short sr98
36248                      ;-----
36249                      ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
36250 00002D16 723C      jc     short badparm_p_coff
36251 if98:
36252 00002D18 83F8FF      cmp     ax,_$P_RC_EOL ; end of line?
36253 00002D1B 7412      je     short en98 ; then end the $endloop
36254
36255                      ;mov     al,[cs:rv_dword]
36256                      ;mov     al,[cs:result_val+_$P_Result_Blk.Picked_Val]
36257                      ; 15/12/2022
36258                      ; ds = cs
36259 00002D1D A0[1922]      mov     al,[result_val+_$P_Result_Blk.Picked_Val]
36260 00002D20 83F901      cmp     cx,1 ; the first positional?
36261 00002D23 7505      jne     short if102
36262                      ;mov     [cs:p_fcbs],al
36263                      ; 15/12/2022
36264 00002D25 A2[D122]      mov     [p_fcbs],al
36265                      ;jmp     short en102
36266 00002D28 EBE9      jmp     short do98
36267 if102:
36268                      ;mov     [cs:p_keep],al
36269                      ; 15/12/2022
36270 00002D2A A2[D222]      mov     [p_keep],al
36271 en102:
36272 00002D2D EBE4      jmp     short do98
36273 en98:
36274                      ; 15/12/2022
36275                      ; ds = cs
36276 00002D2F A0[D122]      mov     al,[p_fcbs]
36277 00002D32 A2[A002]      mov     [FCBS],al
36278 00002D35 C606[A102]00 mov     byte [KEEP],0
36279                      ;mov     al,[cs:p_fcbs] ; M017
36280                      ;mov     [cs:FCBS],al ; M017
36281                      ;mov     byte [cs:KEEP],0 ; M017
36282 sr98:
36283 00002D3A E947F8      jmp     coff
36284
36285                      ; 31/12/2022 - Retro DOS v4.2
36286                      ;%if 0
36287
36288                      ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36289                      ;
36290                      ; comment= do nothing. just decrease chrptr,and increase count for correct
36291                      ; line number
36292                      ;-----
36293
36294 tryy:
36295      cmp     ah,CONFIG_COMMENT ; 'Y'
36296      jne     short try0
36297

```

```

36298 donothing:
36299 ; 15/12/2022
36300 ; ds = cs
36301 dec word [chrptr]
36302 inc word [count]
36303 ; 02/11/2022
36304 ;dec word [cs:chrptr]
36305 ;inc word [cs:count]
36306
36307 jmp coff
36308
36309 ;-----
36310 ; rem command
36311 ;-----
36312
36313 try0:
36314 cmp ah,CONFIG_REM ; '0' ; do nothing with this line.
36315 je short donothing
36316
36317 %endif
36318
36319 ; 07/04/2019 - Retro DOS v4.0
36320
36321 ;-----
36322 ; switches command
36323 ;-----
36324
36325 ;*****
36326 ; function: parse the option switches specified. *
36327 ; note - this command is intended for the future use also. *
36328 ; when we need to set system data flag,use this command. *
36329 ;
36330 ; input : *
36331 ; es:si -> parameters in command line. *
36332 ; output: *
36333 ; p_swit_k set if /k option chosen. *
36334 ;
36335 ; subroutines to be called: *
36336 ; sysinit_parse *
36337 ; logic: *
36338 ; { *
36339 ; set di points to swit_parms; /*parse control definition*/ *
36340 ; set dx,cx to 0; *
36341 ; while (end of command line) *
36342 ; { sysinit_parse; *
36343 ; if (no error) then *
36344 ; if (result_val._$P_synonym_ptr == swit_k) then *
36345 ; p_swit_k = 1 *
36346 ; endif *
36347 ; else {show error message;error exit} *
36348 ; } *
36349 ; } *
36350 ; *
36351 ; *
36352 ;*****
36353
36354 SUPPRESS_WINA20 EQU 00000010b ; M025 ; (DOSSYM.INC, MSDOS 6.0)
36355
36356 try1:
36357 cmp ah,CONFIG_SWITCHES ; '1'
36358 je short do_try1 ; switches= command entered?
36359
36360 skip_it5:
36361 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
36362 ; (SYSINIT:2C8Ah)
36363 jmp tryv
36364 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36365 jmp tryz
36366
36367 do_try1:
36368 ; 31/12/2022 - Retro DOS v4.2
36369 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36370 ;%if 0
36371 ;ifdef MULTI_CONFIG
36372 call query_user ; query the user if config_cmd
36373 jc short skip_it5 ; has the CONFIG_OPTION_QUERY bit set
36374 ;endif
36375 ;%endif ; 30/10/2022
36376
36377 mov di,swit_parms
36378 xor cx,cx
36379 ; 03/01/2023
36380 mov dx,cx
36381
36382 do110:
36383 call sysinit_parse
36384 jnc short if110 ; parse error
36385 call badparm_p ; and show messages and end the search loop.
36386 jmp short sr110
36387 ;-----
36388 ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
36389 badparm_p_coff:
36390 call badparm_p
36391 jmp coff
36392 ;-----
36393 if110:
36394 cmp ax,_$P_RC_EOL ; end of line?
36395 je short en110 ; then jmp to $endloop for semantic check
36396
36397 ; 15/12/2022
36398 ; ds = cs
36399 ;cmp word [cs:result_val_swoff],swit_k
36400 ;cmp word [cs:result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_k
36401 cmp word [result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_k
36402 jne short if115 ; M059
36403 ; 15/12/2022
36404 mov byte [p_swit_k],1
36405 mov byte [cs:p_swit_k],1 ; set the flag
36406 jmp short do110
36407
36408 if115:
36409 ; 15/12/2022 ;M059
36410 ;cmp word [cs:result_val_swoff],swit_t
36411 ;cmp word [cs:result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_t ;M059
36412 cmp word [result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_t
36413 jne short if116 ;M059 M063
36414 ; 14/04/2024
36415 ;;;
36416 jne short if118 ; (PCDOS 7.1 IBMBIO.COM)
36417 ;;;
36418 ; 15/12/2022
36419 mov byte [p_swit_t],1
36420 mov byte [cs:p_swit_t],1 ;M059
36421 jmp short do110 ;M059
36422
36423 ; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)

```



```

36422
36423
36424
36425
36426 00002D7D 813E[1722][9023]
36427 00002D83 7507
36428
36429 00002D85 C606[9623]01
36430 00002D8A EBC3
36431
36432
36433
36434
36435
36436 00002D8C 813E[1722][8423]
36437 00002D92 75BB
36438
36439 00002D94 C606[9523]01
36440
36441 00002D99 EBB4
36442
36443
36444
36445 00002D9B 803E[9323]01
36446
36447 00002DA0 1E
36448
36449
36450
36451 00002DA1 B87000
36452 00002DA4 8ED8
36453 00002DA6 750A
36454
36455 00002DA8 C606[7E04]00
36456
36457 00002DAD C606[7F04]01
36458
36459
36460
36461
36462 00002DB2 2EA0[9423]
36463 00002DB6 A2[8B04]
36464
36465 00002DB9 2E803E[9523]00
36466 00002DBF 740E
36467 00002DC1 06
36468 00002DC2 53
36469 00002DC3 B452
36470 00002DC5 CD21
36471
36472
36473
36474 00002DC7 26800E860002
36475 00002DCD 5B
36476 00002DCE 07
36477
36478 00002DCF 1F
36479
36480 00002DD0 E9B1F7
36481
36482
36483
36484
36485
36486
36487
36488
36489
36490
36491
36492
36493
36494 00002DD3 80FC56
36495 00002DD6 750F
36496 00002DD8 E83F18
36497 00002DDB 720A
36498 00002DDD E83614
36499 00002DE0 73EE
36500
36501 00002DE2 E8D300
36502 00002DE5 EBE9
36503
36504
36505
36506
36507
36508 00002DE7 80FC4E
36509
36510
36511 00002DEA 750C
36512
36513 00002DEC E82B18
36514 00002DEF 7238
36515 00002DF1 E8B710
36516 00002DF4 72EC
36517 00002DF6 EBD8
36518
36519
36520
36521
36522
36523
36524
36525
36526
36527 00002DF8 80FC54
36528 00002DFB 752C
36529
36530 00002DFD E81A18
36531 00002E00 7227
36532
36533 00002E02 BF[C623]
36534 00002E05 31C9
36535
36536
36537
36538 00002E07 E85B00
36539 00002E0A 7303
36540
36541
36542
36543
36544 00002E0C E945FF
36545

;;;
if118:
;;cmp word [cs:result_val_swoff],swit_i ; offset "/"
;;cmp word [cs:result_val+$_P_Result_Blk.SYNONYM_Ptr],swit_i
cmp word [result_val+$_P_Result_Blk.SYNONYM_Ptr],swit_i
jne short if116
mov byte [cs:p_swit_i],1 ; set the flag
mov byte [p_swit_i],1
jmp short do110
;;;
if116:
; 15/12/2022
;;cmp word [cs:result_val_swoff],swit_w
;;cmp word [cs:result_val+$_P_Result_Blk.SYNONYM_Ptr],swit_w ;M063
cmp word [result_val+$_P_Result_Blk.SYNONYM_Ptr],swit_w
jne short do110 ;M063
; 15/12/2022
mov byte [p_swit_w],1
mov byte [cs:p_swit_w],1 ;M063
jmp short do110 ;M063
en110:
; 15/12/2022
; ds = cs
cmp byte [p_swit_k],1
cmp byte [cs:p_swit_k],1 ; if /k entered,
push ds
;;mov ax,Bios_Data
mov ax,KERNEL_SEGMENT ; 0070h
; 21/10/2022
mov ax,DOSBIODATASEG ; 0070h
mov ds,ax
jne short if117
; 14/04/2024
mov byte [keyrd_func],0 ; 4E5h ; use the conventional keyboard functions
; BIOSDATA:047Eh for PC DOS 7.1 IBMBIO.COM
mov byte [keysts_func],1 ; 4E6h (for MSDOS 6.21 IO.SYS)
; BIOSDATA:047Fh for PC DOS 7.1 IBMBIO.COM
if117:
; 15/12/2022
; ds <= cs
mov al,[cs:p_swit_t] ;M059
mov [t_switch],al ; 4F2h (for MSDOS 6.21 IO.SYS) ;M059
; 14/04/2024 ; BIOSDATA:048Bh for PC DOS 7.1 IBMBIO.COM
cmp byte [cs:p_swit_w],0 ;M063
je short skip_dos_flag ;M063
push es
push bx
mov ah,GET_IN_VARS ; 52h ;M063
int 21h ;M063
; DOS - 2+ internal - GET LIST OF LISTS
; Return: ES:BX -> DOS list of lists
or bytes [es:86h],2
or byte [es:DOS_FLAG_OFFSET],SUPPRESS_WINA20 ; 2 ;M063
pop bx
pop es
skip_dos_flag:
pop ds ;M063
sr110:
jmp coff

; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:2D14h)
; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
;%if 0

tryv:

;ifdef MULTI_CONFIG
;-----
; set command (as in "set var=value<cr/lf>")
;-----
cmp ah,CONFIG_SET ; 'v'
jne short tryn
call query_user ; query the user if config_cmd
jc short tryn ; has the CONFIG_OPTION_QUERY bit set
call copy_envvar ; copy var at ES:SI to "config_wrkseg"
jnc short sr110 ; no error
err:
call error_line ; whoops, display error in line xxx
jmp short sr110 ; jump to coff (to skip to next line)

;-----
; numlock command (as in "numlock=on|off")
;-----
tryn:
cmp ah,CONFIG_NUMLOCK ; 'N'
jne short tryy
; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
jne short tryt

call query_user ; query the user if config_cmd
jc short tryy ; has the CONFIG_OPTION_QUERY bit set
call set_numlock
jc short err
jmp short sr110 ; all done

endif ;MULTI_CONFIG

; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
;-----
; dosdata command
;-----
tryt:
cmp ah,54h ; 'T'
cmp ah,CONFIG_DOSDATA ; 'T' ; PC DOS 7 new config cmd
jne short tryy

call query_user
jc short tryy

mov di,dosdata_parms
xor cx,cx
; 14/04/2024 - Retro DOS v5.0
mov dx,cx ; 0
do120:
call sysinit_parse
jnc short if120

call badparm_p
jmp short en120
; 14/04/2024 - Retro DOS v5.0
jmp badparm_p_coff
if120:

```

```

36546             ;cmp     ax,0FFFFh
36547 00002E0F 83F8FF      cmp     ax,_$P_RC_EOL ; -1 ; end of line?
36548 00002E12 7422        jz      short en120
36549 00002E14 803E[1622]01 cmp     byte [result_val_itag],1 ; tag 1 (UMB)
36550                                     ; [result_val+_$P_Result_Blk.Item_Tag]
36551 00002E19 7507        jnz     short if121
36552 00002E1B C606[6E03]01 mov     byte [dosdata_umb],1 ; DOSDATA=UMB (1) NOUMB (0)
36553                                     ; jmp     short sr120
36554                                     ; 14/04/2024
36555 00002E20 EBE5        jmp     short do120
36556 if121:
36557 00002E22 C606[6E03]00 mov     byte [dosdata_umb],0 ; DOSDATA=UMB (1) NOUMB (0)
36558 sr120:
36559 00002E27 EBDE        jmp     short do120
36560                                     ; 14/04/2024
36561 ;en120:
36562                                     ; jmp     coff
36563
36564 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36565 -----
36566 ; comment= do nothing. just decrease chrptr,and increase count for correct
36567 ; line number
36568 -----
36569
36570             ; 31/12/2022
36571 try:
36572 00002E29 80FC59      cmp     ah,CONFIG_COMMENT ; 'Y'
36573 00002E2C 750B        jne     short try0
36574
36575 donothing:
36576             ; 15/12/2022
36577             ; ds = cs
36578 00002E2E FF0E[5A03]   dec     word [chrptr]
36579 00002E32 FF06[5603]   inc     word [count]
36580             ; 02/11/2022
36581             ; dec     word [cs:chrptr]
36582             ; inc     word [cs:count]
36583 en120:
36584 00002E36 E94BF7      jmp     coff
36585                                     ; 14/04/2024
36586 -----
36587 ; rem command
36588 -----
36589
36590 try0:
36591 00002E39 80FC30      cmp     ah,CONFIG_REM ; '0'
36592 00002E3C 74F0        je      short donothing
36593
36594 ;%endif             ; 30/10/2022
36595
36596 ; 30/10/2022
36597 ; (MSSOS 5.0 IO.SYS - SYSINIT:29D7h)
36598 -----
36599 ; bogus command
36600 -----
36601
36602 tryz:
36603             cmp     ah,0FFh             ;null command? (BUGBUG - who sets FFh anyway?)
36604 00002E3E 80FCFF      ; 31/12/2022
36605                                     ; je      short donothing
36606 00002E41 74EB        ; 02/11/2022
36607                                     ; je      short tryz_donothing
36608
36609             dec     word [chrptr]
36610 00002E43 FF0E[5A03]   inc     word [count]
36611 00002E47 FF06[5603]   jmp     short badop
36612 00002E4B EB37
36613
36614 ; 31/12/2022
36615 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
36616 ; tryz_donothing:
36617 ; jmp     donothing
36618
36619 ; 07/04/2019 - Retro DOS v4.0
36620 -----
36621
36622 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
36623 ; (SYSINIT:2D5Dh)
36624
36625 ; 11/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
36626
36627 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
36628 ;
36629 ; ***CheckProtmanArena -- special hack for adjusting alloclim with Protman$
36630 ;
36631 ; adjusts alloclim if Protman$ reduced our arena through a manual hack.
36632 ;
36633 ;
36634 CheckProtmanArena:
36635             ; 08/09/2023
36636             ; ds = cs
36637 00002E4D 06          push    es
36638             ; mov     ax,[cs:area] ; get our arena header
36639 00002E4E A1[6803]     mov     ax,[area] ; 08/09/2023
36640 00002E51 48          dec     ax
36641 00002E52 8EC0        mov     es,ax
36642             ; add     ax,[es:ARENA.SIZE]
36643 00002E54 2603060300  add     ax,[es:3] ; find end of arena
36644 00002E59 40          inc     ax
36645             ; 08/09/2023
36646 00002E5A 3B06[A502]   cmp     ax,[ALLOCLIM]
36647             ; cmp     ax,[cs:ALLOCLIM] ; is it less than alloclim?
36648 00002E5E 7703        ja      short CheckProtmanDone
36649
36650             ; mov     [cs:ALLOCLIM],ax ; reduce alloclim then
36651             ; 08/09/2023
36652 00002E60 A3[A502]     mov     [ALLOCLIM],ax
36653 CheckProtmanDone:
36654 00002E63 07          pop     es
36655 00002E64 C3          retn
36656 -----
36657
36658 sysinit_parse:
36659 -----
36660
36661 ; set up registers for sysparse
36662 ; in es:si -> command line in confbot
36663 ; di -> offset of the parse control definition.
36664 ;
36665 ; out) calls sysparse.
36666 ; carry will set if parse error.
36667 ; *** the caller should check the eol condition by looking at ax
36668 ; *** after each call.
36669

```

```

36670 ; *** if no parameters are found,then ax will contain a error code.
36671 ; *** if the caller needs to look at the synonym@ of the result,
36672 ; *** the caller should use cs:@ instead of es:@.
36673 ; cx register should be set to 0 at the first time the caller calls this
36674 ; procedure.
36675 ; ax - exit code
36676 ; bl - terminated delimiter code
36677 ; cx - new positional ordinal
36678 ; si - set to pase scanned operand
36679 ; dx - selected result buffer
36680 ;-----
36681 ;
36682 ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
36683 ; (SYSINIT:2D78h)
36684 ;
36685 ; 14/04/2024 - Retro DOS v5.0
36686 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:32F3h)
36687 ;
36688 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
36689 ; ds = cs
36690 00002E65 8C06[6D19] mov [badparm_seg],es ;save the pointer to the parm
36691 00002E69 8936[6B19] mov [badparm_off],si ;we are about to parse for badparm msg.
36692 ;
36693 ; 24/10/2022
36694 00002E6D 06 push es ;save es,ds
36695 00002E6E 1E push ds
36696 ;
36697 00002E6F 06 push es
36698 00002E70 1F pop ds ;now ds:si -> command line
36699 ;
36700 00002E71 0E push cs
36701 00002E72 07 pop es ;now es:di -> control definition
36702 ;
36703 ; 09/09/2023
36704 ;mov [cs:badparm_seg],ds ;save the pointer to the parm
36705 ;mov [cs:badparm_off],si ;we are about to parse for badparm msg.
36706 ;
36707 ;mov dx,0
36708 ; 04/01/2023
36709 00002E73 29D2 sub dx,dx ; 0
36710 00002E75 E89DEB call SysParse
36711 ;cmp ax,_$P_No_Error ; 0 ;no error
36712 ; 06/09/2023
36713 00002E78 21C0 and ax,ax
36714 ;
36715 ;**cas note: when zero true after cmp, carry clear
36716 ;
36717 ;je short 114
36718 ; 24/10/2022 (MSDOS 5.0 IO.SYS compatibility, SYSINIT:2A02h)
36719 ; 12/12/2022
36720 00002E7A 7405 je short en4 ; cf=0
36721 00002E7C 83F8FF cmp ax,_$P_RC_EOL ; 0FFFFh;or the end of line?
36722 ;jne short if4
36723 ; 12/12/2022
36724 00002E7F 7400 je short en4 ; cf=0
36725 ; 06/09/2023
36726 ; cf=1
36727 ;
36728 ; 12/12/2022
36729 ;114:
36730 ; ; 12/12/2022
36731 ; ; cf=0
36732 ; ;clc
36733 ; jmp short en4
36734 ;
36735 if4:
36736 ; 24/10/2022
36737 ; 06/09/2023 (cf=1)
36738 ;stc
36739 en4:
36740 00002E81 1F pop ds
36741 00002E82 07 pop es
36742 00002E83 C3 retn
36743 ;
36744 ; 11/12/2022
36745 %if 0
36746 ;
36747 ;-----
36748 ;
36749 ; procedure : badop_p
36750 ;
36751 ; same thing as badop,but will make sure to set ds register back
36752 ; to sysinitseg and return back to the caller.
36753 ;
36754 ;-----
36755 ;
36756 badop_p:
36757 push cs
36758 pop ds ;set ds to configsys seg.
36759 mov dx,badopm
36760 call print
36761 ;call error_line
36762 ;retn
36763 ; 11/12/2022
36764 jmp error_line
36765 ;
36766 %endif
36767 ;
36768 ;-----
36769 ;
36770 ; label : badop
36771 ;
36772 ;-----
36773 ;
36774 badop:
36775 00002E84 BA[4A50] mov dx,badopm ;want to print command error "unrecognized command..."
36776 00002E87 E8C81B call print
36777 00002E8A E82B00 call error_line ;show "error in config.sys ..." .
36778 00002E8D E9F4F6 jmp coff
36779 ;
36780 ;-----
36781 ;
36782 ; procedure : badparm_p
36783 ;
36784 ; show "bad command or parameters - xxxxxx"
36785 ; in badparm_seg,badparm_off -> xxxxx
36786 ;
36787 ;-----
36788 ;
36789 ; 24/10/2022
36790 badparm_p:
36791 ; 11/12/2022
36792 ; ds = cs
36793 ; 11/12/2022

```

```

36794          ;push ds ; *
36795 00002E90 52      push dx
36796 00002E91 56      push si
36797
36798          ; 11/12/2022
36799          ; ds = cs
36800          ;push cs
36801          ;pop ds
36802
36803 00002E92 BA[7150]  mov dx,badparm
36804 00002E95 E8BA1B   call print          ; "bad command or parameters - "
36805 00002E98 C536[6B19] lds si,[badparm_ptr]
36806
36807          ; print "xxxx" until cr.
36808
36809
36810 00002E9C 8A14      mov dl,[si]          ; get next character
36811 00002E9E 80FA0D   cmp dl,cr ; 0dh      ; is a carriage return?
36812 00002EA1 7407     je short en1         ; exit loop if so
36813
36814 00002EA3 B402      mov ah,2 ; STD_CON_OUTPUT ; function 2
36815 00002EA5 CD21     int 21h          ; display character
36816 00002EA7 46       inc si              ; next character
36817 00002EA8 EBF2     jmp short do1
36818
36819 00002EAA 0E        push cs
36820 00002EAB 1F        pop ds
36821
36822 00002EAC BA[6E50]   mov dx,crlfm
36823 00002EAF E8A01B   call print
36824 00002EB2 E80300   call error_line
36825
36826 00002EB5 5E        pop si
36827 00002EB6 5A        pop dx
36828          ; 11/12/2022
36829          ;pop ds ; *
36830 badparm_ret:
36831 00002EB7 C3        retn
36832
36833          ; 10/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
36834 %if 0
36835
36836          ;-----
36837          ;
36838          ; procedure : getchr
36839          ;
36840          ;-----
36841
36842          ; 24/10/2022
36843 getchr:
36844          ; 12/12/2022
36845          ;push cx
36846          ;mov cx,[count]
36847          ;jcxz nochar
36848          ; 12/12/2022
36849          cmp word [count],1
36850          jb short nochar ; cf=1 ([count] = 0)
36851
36852          mov si,[chrptr]
36853          mov al,[es:si]
36854          dec word [count]
36855          inc word [chrptr]
36856          ; 12/12/202
36857          ; cf=0
36858          ;clc
36859 %get_ret:
36860          ;pop cx
36861          ;retn
36862 nochar:
36863          ; 12/12/2022
36864          ; cf=1
36865          ;stc
36866          ;jmp short get_ret
36867
36868          retn
36869 %endif
36870
36871          ; 11/12/2022
36872 %if 0
36873
36874          ;-----
36875          ;
36876          ; procedure : incorrect_order
36877          ;
36878          ; show "incorrect order in config.sys ..." message.
36879          ;
36880          ;-----
36881
36882 incorrect_order:
36883          mov dx,badorder
36884          call print
36885          call showlinenum
36886          retn
36887
36888 %endif
36889
36890          ;-----
36891          ;
36892          ; procedure : error_line
36893          ;
36894          ; show "error in config.sys ..." message.
36895          ;
36896          ;-----
36897
36898          ; 11/12/2022
36899          ; 24/10/2022
36900 error_line:
36901          ; 11/12/2022
36902          ; ds = cs
36903          ;push cs
36904          ;pop ds
36905
36906 00002EB8 BA[A651]   mov dx,errorcmd
36907 00002EBB E8941B   call print
36908          ;call showlinenum
36909          ;retn
36910          ; 11/12/2022
36911          ;jmp short shortlinenum
36912
36913          ;-----
36914          ;
36915          ; procedure : showlinenum
36916          ;
36917          ; convert the binary linecount to decimal ascii string in showcount

```

```

36918 ; and display showcount at the current curser position.
36919 ; in.) linecount
36920 ;
36921 ; out) the number is printed.
36922 ;
36923 ; -----
36924 ;
36925 ; 11/12/2022
36926 ; ds = cs
36927 ; 24/10/2022
36928 showlinenum:
36929 00002EBE 06 push es
36930 ; 11/12/2022
36931 ;push ds
36932 00002EBF 57 push di
36933
36934 00002EC0 0E push cs
36935 00002EC1 07 pop es ; es=cs
36936
36937 ; 11/12/2022
36938 ;push cs
36939 ;pop ds
36940
36941 00002EC2 BF[B502] mov di,showcount+4 ; di -> the least significant decimal field.
36942 00002EC5 B90A00 mov cx,10 ; decimal divide factor
36943 ;mov ax,[cs:linecount]
36944 ; 11/12/2022
36945 00002EC8 A1[AF02] mov ax,[linecount]
36946 s1n_loop:
36947 ; 11/12/2022
36948 00002ECB 39C8 cmp ax,cx ; < 10 ?
36949 ;cmp ax,10 ; < 10?
36950 00002ECD 720C jnb short s1n_last
36951
36952 00002ECF 31D2 xor dx,dx
36953 00002ED1 F7F1 div cx ; cx = 10
36954 00002ED3 80CA30 or dl,30h ; add "0" (= 30h) to make it an ascii.
36955 00002ED6 8815 mov [di],dl
36956 00002ED8 4F dec di
36957 00002ED9 EBF0 jmp short s1n_loop
36958
36959 s1n_last:
36960 00002EDB 0C30 or al,30h ; "0"
36961 00002EDD 8805 mov [di],al
36962 00002EDF 89FA mov dx,di
36963 00002EE1 E86E1B call print ; show it.
36964 00002EE4 5F pop di
36965 ; 11/12/2022
36966 ;pop ds
36967 00002EE5 07 pop es
36968 00002EE6 C3 retn
36969
36970 ; 07/04/2019 - Retro DOS v4.0
36971 ; (MSDOS 6.21 IO.SYS, SYSINIT:2E44h)
36972 ; -----
36973 ;
36974 ;
36975 ; procedure : ProcDOS
36976 ;
36977 ; Process the result of DOS= parsing
36978 ;
36979 ; result_val._$P_item_tag = 1 for DOS=HIGH
36980 ; = 2 for DOS=LOW
36981 ; = 3 for DOS=UMB
36982 ; = 4 for DOS=NOUMB
36983 ; -----
36984 ;
36985 ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
36986 ; (SYTSINIT:2AB5h)
36987 ProcDOS:
36988 ; 01/01/2023
36989 ; ds = cs
36990 00002EE7 30E4 xor ah,ah
36991 ;mov al,[cs:result_val_itag]
36992 ;mov al,[cs:result_val+_$P_Result_Blk.Item_Tag]
36993 ; 01/01/2023
36994 00002EE9 A0[1622] mov al,[result_val+_$P_Result_Blk.Item_Tag]
36995 00002EEC 48 dec ax
36996 00002EED 7415 jz short pd_hi
36997 00002EEF 48 dec ax
36998 00002EF0 740E jz short pd_lo
36999 00002EF2 48 dec ax
37000 00002EF3 7405 jz short pd_umb
37001 ;mov byte [cs:DevUMB],0
37002 ; 18/12/2022
37003 ;mov byte [cs:DevUMB],ah ; 0
37004 ; 01/01/2023
37005 00002EF5 8826[4024] mov byte [DevUMB],ah ; 0
37006 00002EF9 C3 retn
37007 pd_umb:
37008 ; 01/01/2023
37009 00002EFA C606[4024]FF mov byte [DevUMB],0FFh
37010 ;mov byte [cs:DevUMB],0FFh
37011 00002EFF C3 retn
37012 pd_lo:
37013 ; 01/01/2023
37014 00002F00 A2[6C02] mov [runhigh],al ; 0
37015 ; 18/12/2022
37016 ;mov [cs:runhigh],al ; 0
37017 ;mov byte [cs:runhigh],0
37018 00002F03 C3 retn
37019 pd_hi:
37020 ; 01/01/2023
37021 00002F04 C606[6C02]FF mov byte [runhigh],0FFh
37022 ;mov byte [cs:runhigh],0FFh
37023 limx: ; 11/12/2022
37024 00002F09 C3 retn
37025 ; -----
37026 ;
37027 ;
37028 ; procedure : LieInt12Mem
37029 ;
37030 ; Input : DevEntry points to Device Start address (offset == 0)
37031 ; alloclim set to the limit of low memory.
37032 ;
37033 ; Output : none
37034 ;
37035 ; Changes the ROM BIOS variable which stores the total low memory
37036 ; If a 3com device driver (any character device with name 'PROTMAN$')
37037 ; is being loaded alloclim is converted into Ks and stored in 40:13h
37038 ; Else if a device driver being loaded into UMB the DevLoadEnd is
37039 ; converted into Ks and stored in 40:13h
37040 ;
37041 ; -----

```

```

37042
37043
37044
37045
37046 00002F0A A1[A502]
37047
37048
37049 00002F0D E84200
37050 00002F10 740A
37051
37052
37053
37054
37055
37056
37057 00002F12 803E[4F24]00
37058 00002F17 74F0
37059 00002F19 A1[3524]
37060
37061
37062
37063
37064
37065
37066
37067
37068
37069
37070
37071
37072
37073
37074
37075
37076
37077
37078
37079
37080
37081
37082
37083
37084
37085 00002F1C 1E
37086 00002F1D BB4000
37087 00002F20 8EDB
37088 00002F22 8B1E1300
37089
37090 00002F26 B106
37091 00002F28 D3E8
37092 00002F2A A31300
37093
37094 00002F2D 1F
37095
37096
37097 00002F2E 891E[5324]
37098 00002F32 C606[5224]FF
37099
37100 00002F37 C3
37101
37102
37103
37104
37105
37106
37107
37108
37109
37110
37111
37112
37113
37114
37115
37116
37117
37118
37119
37120 00002F38 803E[5224]00
37121
37122
37123
37124 00002F3D 7412
37125
37126 00002F3F B84000
37127 00002F42 8826[5224]
37128
37129
37130 00002F46 1E
37131
37132 00002F47 8ED8
37133 00002F49 2EA1[5324]
37134 00002F4D A31300
37135 00002F50 1F
37136
37137 00002F51 C3
37138
37139
37140
37141
37142
37143
37144
37145
37146
37147
37148
37149
37150
37151
37152 00002F52 1E
37153 00002F53 06
37154 00002F54 56
37155
37156 00002F55 C536[3724]
37157
37158 00002F59 83C60A
37159 00002F5C 0E
37160 00002F5D 07
37161 00002F5E BF[5524]
37162 00002F61 B90800
37163 00002F64 F3A6
37164 00002F66 5E
37165 00002F67 07

```

```

LieInt12Mem:
; 11/12/2022
; ds = cs
mov ax,[ALLOCLIM]
;mov ax,[cs:ALLOCLIM] ; lie INT 12 as alloclim
; assuming that it is 3Com
call IsIt3Com ; Is it 3Com driver?
jz short lim_set ; yes, lie to him differently
; 13/05/2019
;cmp byte [cs:DeviceHi],0 ; Is the DD being loaded in UMB
;je short limx ; no, don't lie
;mov ax,[cs:DevLoadEnd] ; lie INT 12 as end of UMB
; 11/12/2022
; ds = cs
cmp byte [DeviceHi],0
je short limx
mov ax,[DevLoadEnd]
lim_set:
; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; 11/12/2022
;call SetInt12Mem
;limx:
;retn
;jmp short SetInt12Mem
;-----
;
; procedure : SetInt12Mem
;
; Input : AX = Memory size to be set (in paras)
; Output : none
;
; Sets the variable 40:13 to the memory size passed in AX
; It saves the old value in 40:13 in OldInt12Mem,
; It also sets a flag Int12Lied to 0FFh, which is checked before
; restoring the value of 40:13
;-----
; 01/11/2022
SetInt12Mem:
push ds
mov bx,40h
mov ds,bx
; ROM BIOS Data Segment
mov bx,[13h] ; INT 12 memory variable
;mov [cs:OldInt12Mem],bx ; save it
mov cl,6
shr ax,cl ; convert paras into Ks
mov [13h],ax ; Lie
;mov byte [cs:Int12Lied],0FFh ; mark that we are lying
pop ds
; 14/04/2024
; ds = cs
mov [OldInt12Mem],bx
mov byte [Int12Lied],0FFh
;limx:
retn
;-----
;
; procedure : TrueInt12Mem
;
; Input : Int12Lied = 0 if we are not lying currently
; = 0FFh if we are lying
; OldInt12Mem = Saved value of 40:13h
;
; Output : none
;
; Resets the INT 12 Memory variable if we were lying about int 12
; and resets the flag which indicates that we were lying
;-----
TrueInt12Mem:
; 11/12/2022
; ds = cs
cmp byte [Int12Lied],0
;cmp byte [cs:Int12Lied],0 ; were we lying so far?
; 01/11/2022 (MSDOS 5.0 IO.SYS, SYS.INIT:2B1Dh)
;mov byte [cs:Int12Lied],0 ; reset it anyway
je short timx ; no, we weren't
; 18/12/2022
mov ax,40h
mov [Int12Lied],ah ; 0
;mov byte [Int12Lied],0
;mov byte [cs:Int12Lied],0
push ds
;mov ax,40h
mov ds,ax
mov ax,[cs:OldInt12Mem]
mov [13h],ax ; restore INT 12 memory
pop ds
timx:
retn
;-----
;
; procedure : IsIt3Com?
;
; Input : DevEntry = Seg:0 of device driver
; Output : Zero flag set if device name is 'PROTMAN$'
; else zero flag is reset
;-----
IsIt3Com:
; 11/12/2022
; ds = cs
push ds
push es
push si
; 11/12/2022
lds si,[DevEntry]
;lds si,[cs:DevEntry] ; ptr to device header
add si,SYSDEV.NAME ; 10 ; ptr device name
push cs
pop es
mov di,ThreeComName
mov cx,8 ; name length
rep cmpsb
pop si
pop es

```



```

37166 00002F68 1F      pop     ds
37167 00002F69 C3      retn
37168
37169
37170 ;M020 : BEGIN
37171 ;-----
37172
37173 UpdatePDB:
37174     push    ds
37175     mov     ah,62h
37176     int     21h      ; DOS - 3+ - GET PSP ADDRESS
37177     mov     ds,bx
37178     mov     bx,[CS:ALLOCLIM]
37179     ;mov     [2],bx
37180     mov     [PDB.BLOCK_LEN],bx
37181     pop     ds
37182     retn
37183
37184 ; M020 : END
37185
37186 ;-----
37187 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
37188 ;%if 0
37189
37190 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37191 ; (SYSINIT:2EEeh)
37192
37193 ;include highload.inc      ; Routines for devicehigh parsing, control of HIDDEN
37194 ;include highexit.inc     ; umb's, etc
37195
37196 ; -----
37197 ; HIGHLOAD.INC (MSDOS 6.0 - 1991)
37198 ; -----
37199 ; 07/04/2019 - Retro DOS v4.0
37200
37201 ;*****
37202 ;
37203 ; This file contains routines needed to parse and implement user-given
37204 ; command-line options of the form "/S/L:3,0x500;2;7,127;0x0BE4". InitVar()
37205 ; and Parsevar() are used to parse this data and place it in encoded form into
37206 ; the variables in highvar.inc, for use by the rest of the routines.
37207 ;
37208 ; DeviceHigh accepts this command-line (handled in sysconf.asm, not here):
37209 ;     DEVICEHIGH SIZE=hhhhh module opts
37210 ; Or, DeviceHigh and LoadHigh accept any of the following:
37211 ;     DH/LH module opts
37212 ;     DH/LH [/S][L:umb[,size][;umb[,size]]*] module opts
37213 ;     DH/LH [/L:umb[,size][;umb[,size]]*][/S] module opts
37214 ; The initial UMB,SIZE pair designates the module's load address; the remainder
37215 ; of the UMB and SIZE pairs are used to indicate specific UMBs to be left
37216 ; available during the load.
37217 ;
37218 ; When an actual load is ready to be performed, a call to HideUMBs() will
37219 ; temporarily allocate (as owner 8+"HIDDEN ") all free elements in any
37220 ; upper-memory block which was not specified by the user... in addition, if
37221 ; UMBs were marked to shrink (/S option) to a certain size ("umb,size"), any
37222 ; elements in that umb SAVE the lower-half of the newly-shrunk one are also
37223 ; allocated. After the load, the function UnHideUMBs() (in highexit.inc) will
37224 ; free any UMBs so allocated.
37225 ;
37226 ; When a device driver loads, there is the additional problem of allocating its
37227 ; initial load site; this should be restricted to the first UMB specified on
37228 ; the command-line. The function FreezeUM temporarily allocates all remaining
37229 ; free upper-memory elements (as owner 8+"FROZEN "), except those in the load
37230 ; UMB. Then the initial allocation may be made, and a call to UnFreeze will
37231 ; return any so-allocated memory elements to FREE, for the true load. Note
37232 ; that UnFreeze leaves HIDDEN elements allocated; it only frees FROZEN ones.
37233 ;
37234 ;*****
37235
37236 SWTCH      equ      '/'      ; Switch character
37237
37238 DOS_CHECK_STRATEGY equ 5800h ; Int 21h, Func 58h, Svc 0 = check alloc strat
37239 DOS_SET_STRATEGY   equ 5801h ; Int 21h, Func 58h, Svc 1 = set alloc strategy
37240 DOS_CHECK_UMBLINK  equ 5802h ; Int 21h, Func 58h, Svc 2 = check link state
37241 DOS_GET_UMBLINK    equ 5802h ; 20/04/2019
37242 DOS_SET_UMBLINK    equ 5803h ; Int 21h, Func 58h, Svc 3 = set link state
37243 DOS_GET_DOS_LISTS  equ 52h   ; Int 21h, Func 52h = return list of lists
37244 DOS_UMB_HEAD       equ 8Ch   ; Offset from ES (after func52h) to get UMBHead
37245
37246 CR equ 0Dh          ; Carriage Return
37247 LF equ 0Ah          ; Line Feed
37248 TAB equ 09h         ; Tab character (^I)
37249
37250 ; -----
37251 ;** InitVar - initializes all the variables used in ParseVar and HideUMBs
37252 ; -----
37253 ; ENTRY:      None
37254 ; EXIT:       Variables listed in highvar.inc are initialized
37255 ; ERROR EXIT: None
37256 ; USES:       Flags, variables in highvar.inc
37257 ; -----
37258 ; Note that element 0 references UMB 0 (conventional), not UMB 1. Its contents
37259 ; are largely ignored, but it is initialized nonetheless.
37260 ; -----
37261
37262 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37263 ; (SYSINIT:2EEeh)
37264
37265 InitVar:
37266     ; 01/01/2023
37267     ; ds = cs
37268
37269     ;pushreg <ax, cx, di, es>
37270     ; 03/01/2023
37271     ;push ax
37272     ;push cx
37273     ;push di
37274     ;push es
37275
37276     ;dataseg es                      ;Point ES into appropriate data segment
37277     push    cs
37278     pop     es
37279
37280     xor     ax,ax
37281     ;mov     [es:fUmbTiny],al        ;Shrink UMBs? (made 1 if /S given)
37282     ;mov     [es:fInHigh],al         ;Set to 1 when DH/LH has been called
37283     ;mov     [es:SegLoad],ax         ;Load Address (seg), used for DH only
37284     ;mov     byte [es:UmbLoad],UNSPECIFIED ; 0FFh
37285     ;
37286     ;mov     [es:fm_argc], al        ;Later is the # of the 1st spec'd UMB
37287     ;
37288     ; 01/01/2023
37289     ; ds = cs

```

```

37290 00002F81 A2[FA23]      mov     [fumbtiny],al      ;Shrink UMBS? (made 1 if /S given)
37291 00002F84 A2[F923]      mov     [finhigh],al     ;Set to 1 when DH/LH has been called
37292 00002F87 A3[FB23]      mov     [segload],ax     ;Load Address (seg), used for DH only
37293 00002F8A C606[FD23]FF  mov     byte [umbload],UNSPECIFIED ; 0FFh
37294                                     ;Later is the # of the 1st spec'd UMB
37295 00002F8F A2[3024]      mov     [fm_argc], al      ;Start with zero args having been read
37296                                     ;
37297 00002F92 FC             cld
37298                                     ;
37299 00002F93 B91000          mov     cx,MAXUMB ; 16      ;For each entry
37300 00002F96 BF[FE23]      mov     di,umbused      ;on the Umbused array,
37301 00002F99 F3AA          rep     stosb                ;      Store 0
37302                                     ;
37303                                     ;mov     cx,MAXUMB ; 16      ;Okay... for each entry
37304                                     ; 01/01/2033
37305 00002F9B B110          mov     cl,MAXUMB ; 16
37306 00002F9D BF[0E24]      mov     di,umbsize      ;on the UmbSize array,
37307 00002FA0 F3AB          rep     stosw                ;      Store 0
37308                                     ;
37309                                     ;normseg es      ; Return ES
37310                                     ;
37311                                     ;popreg<es, di, cx, ax>
37312 00002FA2 07             pop     es
37313                                     ; 03/01/2023
37314                                     ;pop     di
37315                                     ;pop     cx
37316                                     ;pop     ax
37317                                     ;
37318 00002FA3 C3             retn
37319                                     ;
37320                                     ;-----
37321                                     ;*** FixMem - scans the upper memory chain and concatenates adjacent free MCBs
37322                                     ;-----
37323                                     ; ENTRY : None
37324                                     ; EXIT : None
37325                                     ; ERROR : None
37326                                     ; USES : Flags, fm_umb, fm_strat
37327                                     ;-----
37328                                     ;
37329                                     ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37330                                     ; (SYSINIT:2F22h)
37331 FixMem:
37332                                     ; 01/01/2023
37333                                     ;push    ax
37334                                     ;push    bx
37335                                     ;push    cx
37336                                     ;push    dx
37337 00002FA4 06             push    es
37338                                     ;
37339 00002FA5 E84900          call    fm_link      ; Link in UMBS
37340                                     ;
37341 00002FA8 E80002          call    umbhead      ; Get first upper-memory MCB address (0x9FFF)
37342 00002FAB 723F          jc      short fmX      ; (if couldn't get it, leave now).
37343                                     ;
37344 00002FAD 8EC0          mov     es,ax      ; It returns in AX, so move it to ES.
37345                                     ;
37346                                     ; - walk MCB Chain -----
37347                                     ;
37348 00002FAF 31D2          xor     dx,dx      ; we're keeping the address of the last MCB
37349 00002FB1 89D1          mov     cx,dx      ; in CX... and the last owner
37350 00002FB3 42             inc     dx      ; in dx as we go through the loop:
37351                                     ;
37352                                     ; -----
37353                                     ; FM10--DX = last MCB's owner's PSP address
37354                                     ; CX = last MCB's address (segment)
37355                                     ; -----
37356                                     ;
37357 00002FB4 26A00000        fm10:    mov     al,[es:ARENA.SIGNATURE] ; if 'Z', don't repeat loop
37358 00002FB8 268B1E0100      mov     bx,[es:ARENA.OWNER] ; if not zero, do nothing
37359 00002FBD 09D3          or      bx,dx      ; dx was owner of previous MCB
37360 00002FBF 7516          jnz     short fm30      ; If not both zero, don't cat.
37361                                     ;
37362                                     ; - Coalesce memory blocks at ES:00 and CX:00 -----
37363                                     ;
37364 00002FC1 268B1E0300      fm20:    mov     bx,[es:ARENA.SIZE] ; Grab this block's Size,
37365 00002FC6 8EC1          mov     es,cx      ; Go back to prev MCB's address
37366 00002FC8 26A20000      mov     [es:ARENA.SIGNATURE],al ; & move the SECOND sig here
37367                                     ;
37368 00002FCC 26031E0300      add     bx,[es:ARENA.SIZE] ; Size += first MCB's size
37369                                     ;add     bx,1      ; And add one for the header
37370                                     ; 11/07/2023
37371 00002FD1 43             inc     bx
37372 00002FD2 26891E0300      mov     [es:ARENA.SIZE],bx ; write the size
37373                                     ;
37374                                     ; -----
37375                                     ;
37376 00002FD7 8CC1          fm30:    mov     cx,es      ; Put this address on the stack
37377 00002FD9 268B160100      mov     dx,[es:ARENA.OWNER] ; And remember its owner
37378                                     ;
37379 00002FDE 8CC3          mov     bx,es      ; Move to the next MCB
37380 00002FE0 26031E0300      add     bx,[es:ARENA.SIZE]
37381 00002FE5 43             inc     bx
37382 00002FE6 8EC3          mov     es,bx
37383                                     ;
37384                                     ;cmp     al,'Z'
37385 00002FE8 3C5A          cmp     al,arena_signature_end
37386 00002FEA 75C8          jne     short fm10      ; If signature != 'Z', there are more.
37387                                     ;
37388 00002FEC E81300          fmX:    call    fm_unlink      ; Unlink UMBS
37389                                     ;
37390 00002FEF 07             pop     es
37391                                     ; 01/01/2023
37392                                     ;pop     dx
37393                                     ;pop     cx
37394                                     ;pop     bx
37395                                     ;pop     ax
37396                                     ;
37397 00002FF0 C3             retn
37398                                     ;
37399                                     ;-----
37400                                     ;*** fm_link - links UMBS not already linked in
37401                                     ;-----
37402                                     ; ENTRY: None
37403                                     ; EXIT: fm_umb == 0 if not linked in previously, 1 if already linked in
37404                                     ; ERROR: None
37405                                     ; USES: AX, BX, fm_umb
37406                                     ;-----
37407                                     ;
37408                                     ; 01/01/2023 - Retro DOS v4.2
37409 fm_link:
37410 00002FF1 B80258          mov     ax,DOS_CHECK_UMBLINK ; 5802h
37411 00002FF4 CD21          int     21h      ; Current link-state is now in al
37412                                     ;
37413                                     ;putdata fm_umb,al      ; So store it in fm_umb for later

```



```

37414 ;
37415 ;push es
37416 ;push cs
37417 ;pop es
37418 ;mov [es:fm_umb],al
37419 ;pop es
37420
37421 ; 01/01/2023
37422 ; ds = cs
37423 ;mov [cs:fm_umb],al
37424 00002FF6 A2[2E24] mov [fm_umb],al
37425
37426 00002FF9 B80358 mov ax,DOS_SET_UMBLINK ; 5803h
37427 00002FFC BB0100 mov bx,1
37428 00002FFF CD21 int 21h
37429 00003001 C3 ret
37430
37431 ;
37432 ;*** fm_unlink - unlinks UMBs if fm_umb is set to 0
37433 ;
37434 ; ENTRY: fm_umb == 1 : leave linked, else unlink
37435 ; EXIT: None
37436 ; ERROR: None
37437 ; USES: AX, BX
37438 ;
37439 ;
37440 ; 01/01/2023 - Retro DOS v4.2
37441 fm_unlink:
37442 00003002 31DB xor bx,bx
37443
37444 ;getdata bl,fm_umb ; fm_umb already has the old link-state
37445 ;
37446 ;push ds
37447 ;push cs
37448 ;pop ds
37449 ;mov bl,[fm_umb]
37450 ;pop ds
37451
37452 ; 01/01/2023
37453 ; ds = cs
37454 ;mov bl,[cs:fm_umb]
37455 00003004 8A1E[2E24] mov bl,[fm_umb]
37456
37457 00003008 B80358 mov ax,DOS_SET_UMBLINK ; 5803h
37458 0000300B CD21 int 21h ; so just use that, and call int 21h
37459 0000300D C3 ret
37460
37461 ; 08/04/2019 - Retro DOS v4.0
37462 ;
37463 ;
37464 ;*** ParseVar - parses [/S][/L:umb[,size][;umb[,size]]*] and builds the table
37465 ; laid out in highvar.inc
37466 ;
37467 ; ENTRY: ES:SI points to command tail of LoadHigh/DeviceHigh (whitespace ok)
37468 ; EXIT: ES:SI points to first character in child program name
37469 ; ERROR: ES:SI points to character which caused error, carry set, AX == code
37470 ; USES: ES:SI, AX, flags, variables in highvar.inc
37471 ;
37472 ; Error codes (in AX if carry set on return):
37473 ;
37474 PV_InvArg equ 1 ; Invalid argument passed
37475 PV_BadUMB equ 2 ; Bad UMB number passed (duplicate?)
37476 PV_InvSwt equ 3 ; Unrecognized switch passed
37477 ;
37478 ; This routine expects ES:SI to point to a string much like the following:
37479 ; "/S/L:1,200;2 module options"
37480 ; optionally, the string can begin with whitespace; neither /S nor /L is
37481 ; required, though that's what this routine is supposed to parse.
37482 ;
37483 optS equ 'S' ; /S
37484 optL equ 'L' ; /L:...
37485 ;
37486 ;
37487 ; LoadHigh has a list of arguments, returned by cparse, which is used to create
37488 ; a command-line for spawning a child process. For a typical LH command, say,
37489 ; lh /l:1,1000;2 print/d:lpt2
37490 ; the arguments would look like (one per line):
37491 ; lh
37492 ; /l
37493 ; 1
37494 ; 1000
37495 ; 2
37496 ; print
37497 ; /d
37498 ; :lpt2
37499 ; In short, if "print" were, say, "43", there'd be no way to determine which
37500 ; arg was the filename. So, inside this routine, we keep a running counter
37501 ; of the number of arguments LH will need to skip in order to get to the
37502 ; program name. The "lh" is implicit--it'll always have to skip that. So if
37503 ; there's no "/l" or "/s", fm_argc will be 0 ... other than that, 1 is added
37504 ; for:
37505 ; Each /L
37506 ; Each /S (there should be only one)
37507 ; Each UMB number (they follow ":" or ";")
37508 ; Each UMB size (they follow ",")
37509 ; So, in the above example, fm_argc would be 4-- and LH would skip right to
37510 ; "print". Note that InitVar initializes fm_argc to zero.
37511 ;
37512 ;
37513 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37514 ; (SYSINIT:2F9Fh)
37515
37516 ParseVar:
37517 ;pushreg <di, ds, es>
37518 ; 01/01/2023
37519 ;push di ; * ; (not required) ; 01/01/2023
37520 0000300E 1E push ds
37521 0000300F 06 push es
37522
37523 00003010 06 push es ; Make DS:SI point to it, as well as ES:SI
37524 00003011 1F pop ds ; (regardless if we're in devhigh or loadhigh)
37525 00003012 FC cld
37526
37527 ;
37528 ; PV10--ES:SI = any whitespace on the command-line
37529 ;
37530
37531 pv10: lodsb ; here, ES:SI==" /L..."--must eat whitespace
37532 00003014 E8A200 call iswhite
37533 00003017 74FA jz short pv10 ; ES:SI==" /L..."--keep eating.
37534 ;cmp al,'/'
37535 00003019 3C2F cmp al,SWTCH
37536 0000301B 7404 je short pv20 ; ES:SI==" /L..."--go process a switch
37537

```

```

37538 0000301D 4E      dec     si           ; Backup--it's now "odule options", and we need
37539 0000301E F8      clc             ; that "m" we just read (or whatever it is).
37540 0000301F EB2B    jmp      short pvX   ; Then return with carry clear == we're done.
37541
37542 00003021 AC      pv20: lodsb           ; Just read 's' or 'L', hopefully
37543                      ;toUpper al          ; So we make it upper-case, and...
37544 00003022 24DF    and     al,0DFh
37545                      ;cmp     al,'S'
37546 00003024 3C53    cmp     al,opts      ; just read 'S'?
37547 00003026 750D    jne     short pv30
37548
37549                      ;call    incArgc      ; If it's /S, it's another arg for LH to skip.
37550 00003028 2EFE06[3024] inc     byte [cs:fm_argc] ; 19/04/2019
37551
37552                      ;putdata fUmbTiny,1    ; /S, so ES:SI==" /L..." or " module opts", or
37553                      ;
37554                      ;push     es
37555                      ;push     cs
37556                      ;pop      es
37557                      ;mov      [es:fUmbTiny],1
37558                      ;pop      es
37559
37560 0000302D 2EC606[FA23]01 mov     byte [cs:fUmbTiny],1
37561
37562 00003033 EBDE      jmp      short pv10    ; possibly even "/L...".
37563
37564                      pv30: ;cmp     al,'L'
37565 00003035 3C4C    cmp     al,optL      ; If it's not 'L' either, then 'tis a bad
37566 00003037 750D    jne     short pVE1    ; switch!
37567
37568                      ;call    incArgc      ; If it's /L, it's another arg for LH to skip.
37569 00003039 2EFE06[3024] inc     byte [cs:fm_argc] ; 19/04/2019
37570
37571 0000303E E80E00    call    parseL
37572 00003041 73D0    jnc     short pv10    ; If no carry, go back and look for more
37573
37574 00003043 4E      dec     si           ; Else, back up and exit.
37575 00003044 EB03    jmp     short pVErr   ; AX has already been set by parseL
37576
37577                      pVE1: ;mov     ax,3
37578 00003046 B80300    mov     ax,PV_InvSwT ; Unrecognized switch passed
37579 00003049 4E      dec     si
37580 0000304A 4E      dec     si
37581 0000304B F9      stc
37582
37583                      pvX: ;popreg <es, ds, di>
37584 0000304D 1F      pop     es
37585                      pop     ds
37586                      ; 01/01/2023
37587 0000304E C3      ;pop     di ; * ; (not required) ; 01/01/2023
37588                      retn
37589
37590                      ; -----
37591                      ; *** parseL - parses ":nnnn[,nnnn][;nnnn[,nnnn]]*" for ParseVar
37592                      ; -----
37593                      ENTRY: ES:SI points to colon
37594                      EXIT: ES:SI points to first character not parsed
37595                      ERROR: Carry set; rewind three characters and return (see ParseVar)
37596                      USES: ES:SI, flags, AX, CX, DX, variables in highvar.inc
37597                      ; -----
37598                      ; If the string here is terminated with anything other than whitespace or a
37599                      ; switchchar (perhaps it's /S or another /L:... ), then we return with carry
37600                      ; set, indicating that they've screwed up the syntax. The 3-character rewind
37601                      ; makes sure the app /L: is reported as being the culprit.
37602                      ; -----
37603
37604 0000304F AC      parseL: lodsb
37605 00003050 3C3A    cmp     al,':'      ; Make sure they did /L:
37606 00003052 754E    jne     short pLE1   ; If they didn't, return with carry set.
37607
37608                      ; -----
37609                      ; PL10--ES:SI = a UMB number, after /L: or ;
37610                      ; -----
37611
37612 00003054 E8DB00    pL10: call    GetXNum      ; After this, 'tis "size" or ";umb" or " mod"
37613 00003057 724F    jc     short pLE2    ; And error if it's a bad number.
37614 00003059 E89D01    call    convUMB      ; Convert any address to a UMB number
37615
37616 0000305C 88C1    mov     cl,al        ; Remember the UMB number
37617 0000305E E87600    call    stowUMB      ; Mark this UMB # as used;
37618 00003061 7245    jc     short pLE2    ; If it was already marked, it'll error
37619
37620                      ;call    incArgc      ; Each UMB number is another arg for LH to skip
37621 00003063 2EFE06[3024] inc     byte [cs:fm_argc] ; 08/04/2019 - Retro DOS v4.0
37622
37623 00003068 AC      lodsb
37624 00003069 3C3B    cmp     al,','      ; Did "umb;" ?
37625 0000306B 74E7    je     short pL10    ; Yep: go back and get another UMB.
37626
37627 0000306D E84900    call    iswhite      ; Did "umb " ?
37628 00003070 743B    jz     short pLX      ; Yep: return (it'll go back to whitespace)
37629
37630 00003072 E83900    call    isEOL        ; Did "umb" ?
37631 00003075 7435    jz     short pLSwX    ; If so, backup and exit like everything's ok
37632
37633                      ;cmp     al, '/'
37634 00003077 3C2F    cmp     al,SWTCH     ; Did "umb/" ? (as in, "/L:1,100;2/S")
37635 00003079 7431    je     short pLSwX    ; If so, back up ES:SI one character and return
37636
37637 0000307B 3C2C    cmp     al,','      ; Did "umb," ?
37638 0000307D 7523    jne     short pLE1    ; Just what the heck DID they do? Return error.
37639
37640                      ; --- Read a size -----
37641
37642 0000307F E8B000    call    GetXNum      ; Stop on "size;" or "size " or anything else
37643 00003082 721E    jc     short pLE1    ; And error if it's a bad size.
37644
37645 00003084 E81601    call    toPara       ; Convert from bytes to paragraphs
37646
37647 00003087 E8CE01    call    stowSiz      ; CL still has the UMB number for this routine
37648
37649                      ;call    incArgc      ; Each UMB size is another arg for LH to skip
37650 0000308A 2EFE06[3024] inc     byte [cs:fm_argc] ; 08/04/2019 - Retro DOS v4.0
37651
37652 0000308F AC      lodsb
37653 00003090 3C3B    cmp     al,','      ; They did "umb,size;", so get another UMB.
37654 00003092 74C0    je     short pL10    ;
37655
37656 00003094 E82200    call    iswhite      ; Did it end with whitespace?
37657 00003097 7414    jz     short pLX      ; If so, we're done here--go back.
37658
37659 00003099 E81200    call    isEOL        ; Did they do "umb,size" and end??? (stupid)
37660 0000309C 740E    jz     short pLSwX    ; If so, backup and exit like everything's ok
37661

```

```

37662             ;cmp     al,'/'
37663 0000309E 3C2F    cmp     al,SWTCH      ; Did they do "umb,size/" ?
37664 000030A0 740A    je      short plSwX    ; If so, again, we're done here.
37665
37666 plE1:
37667             ;mov     ax,1
37668 000030A2 B80100    mov     ax,PV_InvArg    ; If not, we don't know WHAT they did...
37669 000030A5 4E       dec     si
37670 000030A7 C3       stc
37671             retn
37672
37673 000030A8 B80200    plE2:             ;mov     ax,2
37674             mov     ax,PV_BadUMB    ; In this case, they've specified a UMB twice
37675             ; 12/12/2022
37676             ; cf=1
37677 000030AB C3       ;stc
37678             retn
37679 000030AC 4E       plSwX:
37680             dec     si                ; If we hit a '/' character, back up one char
37681             ; so the whitespace checker will see it too.
37682 plX: 12/12/2022
37683             ; cf=0
37684 000030AD C3       ;clc
37685             retn                ; Then just return with carry clear, so
37686                                     ; Parsevar will go about its business.
37687
37688 ;-----
37689 *** incArgc - increments fm_argc, for use with LoadHigh command-line parsing
37690 ;-----
37691 ; ENTRY:      None
37692 ; EXIT:       None
37693 ; ERROR:      None
37694 ; USES:       fm_argc, flags
37695 ;-----
37696
37697 incArgc:
37698             ;push     ax
37699
37700             ;;getdata al, fm_argc ; Obtain previous value of fm_argc,
37701             ;mov     al,[cs:fm_argc]
37702             ;inc     al                ; Increment it,
37703             ;;putdata fm_argc, al ; And store it right back.
37704             ;mov     [cs:fm_argc],al
37705             ;pop     ax
37706             ;retn
37707
37708 ;-----
37709 *** isEOL - returns with ZF set if AL contains CR or LF, or 0
37710 ;-----
37711 ; ENTRY:      AL contains character to test
37712 ; EXIT:       ZF set iff AL contains CR or LF, or 0
37713 ; ERROR:      None
37714 ; USES:       ZF
37715 ;-----
37716
37717 isEOL:
37718             cmp     al,0                ; Null-terminator
37719             je      short iex
37720             cmp     al,CR ; 0Dh        ; Carriage Return
37721             je      short iex
37722             cmp     al,LF ; 0Ah        ; LineFeed
37723             retn
37724
37725 ;-----
37726 *** iswhite - returns with ZF set if AL contains whitespace (or "=")
37727 ;-----
37728 ; ENTRY:      AL contains character to test
37729 ; EXIT:       ZF set iff AL contains space, tab, or equals
37730 ; ERROR:      None
37731 ; USES:       ZF
37732 ;-----
37733
37734 iswhite:
37735             cmp     al,' '                ; Space
37736             je      short iwX
37737             cmp     al,'='                ; Equals (treat as whitespace)
37738             je      short iwX
37739             cmp     al,tab ; 9           ; Tab
37740             retn
37741
37742 ;-----
37743 *** unMarkUMB - marks a given UMB as unused, even if previously marked used
37744 ;-----
37745 ; ENTRY:      AL contains UMB number
37746 ; EXIT:       None
37747 ; ERROR:      None
37748 ; USES:       Flags, variables in highvar.inc
37749 ;-----
37750
37751 ; 02/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37752
37753 unMarkUMB:
37754             ; 02/01/2023
37755             ;push     ax
37756             ;push     bx
37757             ;push     di
37758             ;push     es
37759             ;
37760             ;push     cs
37761             ;pop      es
37762
37763             xor     ah,ah
37764             mov     bx,ax
37765
37766             ; 19/04/2019
37767
37768             ;mov     byte [es:bx+UmbUsed],0
37769             ;mov     [es:bx+UmbUsed],ah ; 0
37770             ; 02/01/2023
37771             ; ds= cs
37772             ;mov     [cs:bx+UmbUsed],ah ; 0
37773             mov     [bx+UmbUsed],ah ; 0
37774
37775             cmp     [UmbLoad],al
37776             ;cmp     [cs:UmbLoad],al
37777             ;cmp     [es:UmbLoad],al
37778             jne     short umu10
37779
37780             ;mov     [es:UmbLoad],0 ; If unmarked the load UMB, load into convent.
37781
37782
37783
37784
37785

```

```

37786         ;mov     [es:UmbLoad],ah ; 0
37787         ; 02/01/2023
37788         ; ds = cs
37789         ;mov     [cs:UmbLoad],ah ; 0
37790 000030D2 8826[FD23]    mov     [UmbLoad],ah ; 0
37791 umu10:
37792         ;pop      es
37793         ;pop      di
37794         ;pop      bx
37795         ;pop      ax
37796 000030D6 C3           retn
37797
37798         ;-----
37799         ;*** stowUMB - marks a given UMB as used, if it hasn't been so marked before
37800         ;-- accepts a UMB # in AL, and makes sure it hasn't yet been
37801         ;listed in the /L:... chain. If it's the first one specified, it sets UmbLoad
37802         ;to that UMB #... and in any case, it marks the UMB as specified.
37803         ;-----
37804         ; ENTRY:    AL contains UMB number, as specified by the user
37805         ; EXIT:     None
37806         ; ERROR:    Carry set if UMB # is less than 0 or >= MAXUMB (see highvar.inc)
37807         ; USES:     AX, Flags, variables in highvar.inc
37808         ;-----
37809
37810         ; 01/01/2023 - Retro DOS v4.2
37811 stowUMB:
37812         cmp     al,MAXUMB ; 16
37813         jb      short su10
37814         stc
37815         retn                ; Ooops-- UMB>=MAXUMB
37816 su10:
37817         ; 01/01/2023
37818         ;push     bx
37819         ;push     di
37820         ;push     si
37821         ;push     ds
37822         ;push     es
37823         ;push     cs
37824         ;pop      es
37825         ;push     cs
37826         ;pop      ds
37827
37828         ; 01/01/2023
37829         ; ds <> cs
37830         ;cmp     byte [cs:UmbLoad],0FFh
37831 000030DD 2E803E[FD23]FF    cmp     byte [cs:UmbLoad],UNSPECIFIED
37832                                     ; If this, we haven't been here before
37833         jne     short su20
37834         mov     [cs:UmbLoad],al        ; So remember this UMB as the load UMB slot.
37835
37836         ;;cmp     byte [UmbLoad],0FFh
37837         ;;cmp     byte [UmbLoad],UNSPECIFIED ; If this, we haven't been here before
37838         ;jne     short su20
37839         ;mov     [UmbLoad],al        ; So remember this UMB as the load UMB slot.
37840 su20:
37841         or      al,al                ; If they gave UMB 0, there's really nothing
37842         jz      short su30          ; that we should do here.
37843
37844         ;mov     bl,al
37845         ;xor     bh,bh
37846         ;mov     ax,1                ; Now, AX = 1, and BX = UMB Number
37847         ; 01/01/2023
37848         xor     ah,ah
37849         mov     bx,ax
37850         mov     al,1
37851
37852         ;xchg     [es:bx+UmbUsed],al
37853         ; 01/01/2023
37854 000030F3 2E8687[FE23]    xchg     [cs:bx+UmbUsed],al
37855
37856         ;or      al,al                ; If it was already 1, then al==1... and that
37857         ;jz      short su30          ; means an error.
37858         ;
37859         ;stc                        ; OOOOPS! This one's been used before. :(
37860
37861         ; 01/01/2023
37862 000030F8 3C01           cmp     al,1
37863 000030FA F5           cmc     ; if al > 0 -> cf = 1
37864 su30:
37865         ; 01/01/2023
37866         ;pop      es
37867         ;pop      ds
37868         ;pop      si
37869         ;pop      di
37870         ;pop      bx
37871 000030FB C3           retn
37872
37873         ; 08/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
37874 %if 0
37875         ;-----
37876         ;*** stowSiz - marks a given UMB as having a given minimum size
37877         ;-----
37878         ; ENTRY:    CL contains UMB number, AX contains size
37879         ; EXIT:     None
37880         ; ERROR:    None
37881         ; USES:     AX, DX, Flags, variables in highvar.inc
37882         ;-----
37883
37884         ; 13/05/2019
37885
37886         ; 01/01/2023 - Retro DOS v4.2
37887 stowSiz:
37888         ; 01/01/2023
37889         ;push     bx
37890         ;;push     di ; ?
37891         ;push     es
37892
37893         ;push     cs
37894         ;pop      es
37895
37896         mov     bl,cl                ; Now bl==UMB number, AX==size
37897         mov     bh,0                ; bx==UMB number, AX==size
37898         shl     bl,1                ; bx==offset into array, AX=size
37899         ;mov     [es:bx+UmbSize],ax ; Store the size
37900         ; 01/01/2023
37901         mov     [cs:bx+UmbSize],ax ; Store the size
37902
37903         ; 01/01/2023
37904         ;pop      es
37905         ;;pop     di ; ?
37906         ;pop      bx
37907
37908         retn
37909 %endif

```

```

37910
37911
37912
37913
37914
37915
37916
37917
37918
37919
37920
37921
37922
37923
37924
37925
37926
37927
37928 000030FC 0000
37929
37930
37931 000030FE 2E833E[FC30]10
37932 00003104 751C
37933
37934
37935 00003106 80F961
37936 00003109 7209
37937 0000310B 80F966
37938 0000310E 7720
37939 00003110 80E957
37940
37941 00003113 C3
37942
37943 00003114 80F941
37944 00003117 7209
37945 00003119 80F946
37946 0000311C 7712
37947 0000311E 80E937
37948
37949 00003121 C3
37950
37951
37952 00003122 80F930
37953
37954 00003125 720A
37955 00003127 80F939
37956 0000312A 7704
37957 0000312C 80E930
37958
37959 0000312F C3
37960
37961 00003130 F9
37962
37963 00003131 C3
37964
37965
37966
37967
37968
37969
37970
37971
37972
37973
37974
37975
37976
37977
37978
37979
37980
37981
37982
37983
37984
37985
37986
37987 00003132 51
37988
37989
37990 00003133 FC
37991 00003134 31C0
37992 00003136 31DB
37993 00003138 31C9
37994 0000313A 31D2
37995
37996 0000313C 2EC706[FC30]0A00
37997
37998 00003143 268A0C
37999
38000 00003146 E8D9FF
38001
38002
38003 00003149 7233
38004
38005 0000314B 08C9
38006 0000314D 7517
38007 0000314F 268A4C01
38008 00003153 80F978
38009 00003156 7405
38010 00003158 80F958
38011 0000315B 7509
38012
38013
38014 0000315D 2EC706[FC30]1000
38015 00003164 46
38016 00003165 46
38017
38018
38019
38020
38021
38022
38023
38024
38025
38026 00003166 268A0C
38027 00003169 46
38028
38029 0000316A E891FF
38030 0000316D 720D
38031
38032 0000316F E80E00
38033 00003172 720A

```

```

;-----
;*** toDigit - converts a character-digit to its binary counterpart
;-- verifies that CL contains a valid character-digit; if so, it
;changes CL to its counterpart binary digit ((CL-'0') or (CL-'A'+10)).
;A-F are considered valid iff gnradox is 16.
;-----
; ENTRY: CL contains a digit ('0' to '9' or, if gnradox==16, 'A' to 'F')
; EXIT: CL contains digit in binary (0 to 9 or, if gnradox==16, 0 to 15)
; ERROR: Carry set indicates invalid digit; carry clear indicates good digit
; USES: CL, Flags
;-----
; If the string is preceeded with "0x", the value is read as hexadecimal; else,
; as decimal. After a read, you may check the radix by examining gnradox--it
; will be 10 or 16.
;-----
gnradox:
dw 0 ; Must be a word--16x16 multiplication

toDigit:
cmp word [cs:gnradox],16
jne short td20 ; Don't check hex digits if radix isn't 16

toDigit_hex:
cmp cl,'a' ; 61h
jb short td10
cmp cl,'f' ; 66h
ja short tdE ; Nothing valid above 'z' at all...
sub cl,'a'-10 ; 57h ; Make 'a'==10 and return.
;clc ; <- CLC is implicit from last SUB
ret

td10:
cmp cl,'A' ; 41h
jb short td20 ; Below 'A'? Not a letter...
cmp cl,'F' ; 46h
ja short tdE ; Above 'F'? Not a digit.
sub cl,'A'-10 ; 37h ; Make 'A'==10 and return.
;clc ; <- CLC is implicit from last SUB
ret

toDigit_dec:
td20:
cmp cl,'0' ; If less than zero,
jb short tdE ; Done.
jb short tdEr ; 08/04/2019
cmp cl,'9' ; Or, if greater than nine,
ja short tdE ; Done.
sub cl,'0' ; 30h ; Okay--make '0'==0 and return.
;clc ; <- CLC is implicit from last SUB
ret

tdE:
stc

tdEr: ; 08/04/2019 - Retro DOS v4.0
ret

;-----
;*** GetXNum - reads a 32-bit ASCII number at ES:SI and returns it in DX:AX
;-----
; ENTRY: ES:SI points to an ascii string to scan
; EXIT: ES:SI moved to first invalid digit, DX:AX contains value read
; ERROR: Carry set if # is too big, or has no digits (EOL possibly)
; USES: ES:SI, DX, AX, Flags, gnradox
;-----
; If the string is preceeded with "0x", the value is read as hexadecimal; else,
; as decimal. After a read, you may check the radix by examining gnradox--it
; will be 10 or 16.
;-----
; 08/04/2019 - Retro DOS v4.0
; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:3109h)

GetXNum:
;pushreg <bx, cx, ds>
; 01/01/2023
;push bx
;push cx ; *
;push ds

cld
xor ax,ax
xor bx,bx
xor cx,cx
xor dx,dx ; Start with 0 (makes sense)

mov word [cs:gnradox],10 ; And default to a radix of 10 (dec)

mov cl,[es:si] ; Now AX=0, BX=0, CH=0/CL=char, DX=0
;call toDigit
call toDigit_dec
jc short gxnE ; If it's not a digit, leave now.
; 01/01/2023
jc short gxnX

or cl,cl
jnz short gxn20 ; Doesn't have '0x'
mov cl,[es:si+1]
cmp cl,'x' ; Either 'x'...
je short gxn10
je short gxn10 ; ...or 'x' means it's hexadecimal
jne short gxn20

gxn10:
mov word [cs:gnradox], 16
inc si ; Since we read "0x", march over it.
inc si

;-----
; GxN20--ES:SI = a digit in a number; if not, we're done
; DX:AX = current total
; BX = 0
; CH = 0
;-----
gxn20:
mov cl,[es:si] ; Now DX:AX=current total, CH=0/CL=char
inc si

call toDigit ; Accepts only valid digits, A-F -> 10-16
jc short gxnQ ; <- Ah... wasn't a digit. Stop.

call mul32 ; Multiply DX:AX by gnradox
jc short gxnX ; (if it's too big, error out)

```

```

38034
38035 00003174 01C8      add     ax,cx          ; Add the digit
38036 00003176 11DA      adc     dx,bx          ; (BX is 0!)--Adds 1 iff last add wrapped
38037                      jnc     short gxnX      ; If _that_ wrapped, it's too big.
38038                      jmp     short gxn20
38039 00003178 73EC      jnc     short gxn20
38040
38041                      gxnE:
38042 0000317A EB02      ; stc             ; In this case, we need to set the carry
38043                      jmp     short gxnX      ; and leave--there were no digits given.
38044 0000317C 4E          gxnQ:
38045 0000317D F8          dec     si             ; Don't read in the offensive character.
38046                      clc                     ; And clear carry, so they know it's okay.
38047                      gxnX:
38048                      ; 01/01/2023
38049 0000317E 59          ; pop     ds
38050                      pop     cx ; *
38051 0000317F C3          ; pop     bx
38052                      retn
38053
38054                      ; -----
38055                      *** mul32 - multiplies the number in DX:AX by gnradox
38056                      ; -----
38057                      ENTRY:  DX:AX = the number to be multiplied, BX = 0, gnradox = multiplier
38058                      EXIT:   DX:AX has been multiplied by gnradox if carry clear; BX still 0
38059                      ERROR:  Carry set if number was too large
38060                      USES:   Flags, AX, DX
38061                      ; -----
38062
38063 00003180 50          mul32:
38064 00003181 89D0      push    ax             ; DX=old:hi, AX=old:lo, TOS=old:lo, BX=0
38065 00003183 2EF726[FC30] mov     ax,dx          ; DX=old:hi, AX=old:hi, TOS=old:lo, BX=0
38066 00003188 7211      mul     word [cs:gnradix] ; DX=?, AX=new:hi, TOS=old:lo, BX=0
38067                      jc      short m32E      ; Too big?
38068
38069 0000318A 89C2      mov     dx,ax          ; DX=new:hi, AX=new:hi, TOS=old:lo, BX=0
38070 0000318C 58          pop     ax             ; DX=new:hi, AX=old:lo, TOS=orig, BX=0
38071
38072 0000318D 87D3      xchg    dx,bx          ; DX=0, AX=old:lo, TOS=orig, BX=new:hi
38073 0000318F 2EF726[FC30] mul     word [cs:gnradix] ; DX=carry, AX=new:lo, TOS=orig, BX=new:hi
38074 00003194 87D3      xchg    dx,bx          ; DX=new:hi, AX=new:lo, TOS=orig, BX=carry
38075 00003196 01DA      add     dx,bx          ; DX=new:hi, AX=new:lo, TOS=orig, BX=carry
38076 00003198 31DB      xor     bx,bx          ; DX=new:hi, AX=new:lo, TOS=orig, BX=0
38077                      retn
38078 0000319B 58          m32E:
38079 0000319C C3          pop     ax
38080                      retn
38081
38082                      ; -----
38083                      *** toPara - divides DX:AX by 16; result in AX only (discards extra DX data)
38084                      ; -----
38085                      ENTRY:  DX:AX = the number to be divided
38086                      EXIT:   Interpereting DX:AX as bytes, AX=paragraph equivalent, 0xFFFF max
38087                      ERROR:  None
38088                      USES:   Flags, AX, DX
38089                      ; -----
38090                      Note: The 386 has a 32-bit SHR, which would work perfectly for this... but we
38091                      ; can't ensure a 386 host machine. Sorry.
38092                      ; -----
38093
38094                      ; 01/01/2023 - Retro DOS v4.2
38095 0000319D 51          toPara:
38096                      push    cx             ; DX:AX=HHHH hhhh hhhh hhhh:LLLL 1111 1111 1111
38097
38098 0000319E B104      mov     cl,4           ;
38099 000031A0 D3E8      shr     ax,cl          ; DX:AX=HHHH hhhh hhhh hhhh:0000 LLLL 1111 1111
38100 000031A2 92          xchg    ax,dx          ; DX:AX=0000 LLLL 1111 1111:HHHH hhhh hhhh hhhh
38101 000031A3 B10C      mov     cl,12          ;
38102 000031A5 D3E0      shl     ax,cl          ; DX:AX=0000 LLLL 1111 1111:hhhh 0000 0000 0000
38103 000031A7 09D0      or      ax,dx          ; AX=hhhh LLLL 1111 1111
38104
38105 000031A9 59          pop     cx
38106 000031AA C3          retn
38107
38108                      ; -----
38109                      *** UmbHead - returns in AX the address of the first UMB block (0x9FFF)
38110                      ; -----
38111                      ENTRY:  Nothing
38112                      EXIT:   AX contains 0x9FFF for most systems
38113                      ERROR:  Carry set if pointer is 0xFFFF (if not set up yet--DH runs into this)
38114                      USES:   Flags, AX
38115                      ; -----
38116                      ; Early in the boot-cycle, the pointer used to obtain this value isn't set up;
38117                      ; to be precise, before a UMB provider is around. In this event, the pointer
38118                      ; is always set to 0xFFFF; it changes once a provider is around. On most
38119                      ; machines (all of 'em I've seen), it changes to 0x9FFF at that point.
38120                      ; -----
38121
38122                      ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
38123                      UmbHead:
38124                      ; 13/05/2019 (because of callers, pushes & pops are not needed here)
38125
38126                      ; push    si ; ?
38127                      ; push    ds ; ?
38128                      ; push    es
38129                      ; push    bx ; *
38130
38131                      ; 09/04/2019
38132                      ; !!! No need to save es,bx,ds,si above !!! (es,bx are changed here)
38133 000031AB B452      mov     ah,GET_IN_VARS ; Call int 21h, function 52h...
38134 000031AD CD21      int     21h
38135
38136 000031AF 26A18C00 mov     ax,[es:DOS_UMB_HEAD] ; And read what's in ES:[008C]
38137
38138                      ; 01/01/2023
38139 000031B3 83F8FF      cmp     ax,0FFFFh
38140 000031B6 F5          cmc
38141                      ; if AX=0FFFFh -> CF=1
38142 000031B7 C3          retn
38143
38144                      ; 01/01/2023
38145                      ; %if 0
38146                      ; cmp     ax,0FFFFh
38147                      ; je      short uhE          ; If it's 0xFFFF, it's an error...
38148
38149                      ;
38150                      ; clc                     ; Else, it isn't (CLC done by prev cmp)
38151                      ; jmp     short uhX
38152                      ; 12/12/2022
38153                      ; retn
38154                      ; uhE:
38155                      ; stc
38156                      ; uhX:
38157                      ; pop     bx ; *
38158                      ; pop     es

```



```

38158 ; ;pop ds ; ?
38159 ; ;pop si ; ?
38160 ; ;retn
38161 ;,%endif
38162
38163 ; -----
38164 ; *** isSysMCB - sets ZF if ES points to an MCB owned by "SC" + (8 or 9)
38165 ; -----
38166 ; ENTRY: ES:0 should point to a valid MCB
38167 ; EXIT: ZF set if owned by SC+8 or SC+9 (for japan)
38168 ; USES: Flags
38169 ; -----
38170
38171 isSysMCB:
38172 ;push ax
38173
38174 ;mov ax,[es:ARENA.OWNER] ; Check the owner...
38175 ;cmp ax,SystemPSPOwner ; 8 (for US OR Japan) is valid
38176 ;je short ism10
38177 ;cmp ax,JapanPSPOwner ; 9 (for Japan) is valid
38178 ;je short ism10
38179 ;jmp short ismX ; Anything else isn't.
38180 ;jne short ismX
38181 000031B8 26833E010008 cmp word [es:ARENA.OWNER],SystemPSPOwner ; 8 ; 09/04/2019
38182 000031BE 7507 jne short ismX
38183
38184 ism10:
38185 ;mov ax,[es:ARENA.NAME] ; Check the name...
38186 000031C0 26813E08005343 ;cmp ax,'SC' ; 4353h
38187 ;cmp word [es:ARENA.NAME],'SC'
38188 ismX:
38189 000031C7 C3 ;pop ax
38190 ;retn
38191
38192 ; 09/04/2019 - Retro DOS v4.0
38193
38194 ; -----
38195 ; *** AddrToUmb - converts a segment address in AX to its appropriate UMB number
38196 ; -----
38197 ; ENTRY: AX contains a segment address
38198 ; EXIT: AX will contain the UMB number which contains the address (0==conv)
38199 ; ERROR: If the address is above UM Range, AX will return as FFFF.
38200 ; USES: Flags, AX
38201 ; -----
38202 ; An address in the following areas is treated as:
38203 ; 0 <-> umbhead (0x9FFF) = Conventional memory
38204 ; 0x9FFF <-> addr of first UM sys MCB = UMB #1
38205 ;
38206 ; addr of last UM sys MCB <-> TOM = invalid; returns #0xFFFF
38207 ; -----
38208 ; 01/01/2023 - Retro DOS v4.2
38209 AddrToUmb:
38210 ; 01/01/2023
38211 ;push cx
38212 ;push dx
38213 000031C8 06 push es
38214
38215 000031C9 89C2 mov dx,ax ; DX = address to search for
38216
38217 000031CB E8DDFF call UmbHead ; AX = first segment
38218 000031CE 7222 jc short atuE ; If it couldn't get it, error out.
38219
38220 ; 22/07/2023
38221 ;mov es,ax ; * ; ES = first UMB segment
38222 000031D0 31C9 xor cx,cx ; 0 ; Pretend we're on UMB 0 for now... (cx = UMB#)
38223
38224 ; 22/07/2023
38225 atu10:
38226 000031D2 8EC0 mov es,ax ; * ; ** ; 22/07/2023
38227 ; -----
38228 ; ATU10--ES - Current MCB address
38229 ; DX - Address given for conversion
38230 ; CX - Current UMB #
38231 ; -----
38232
38233 ;atu10:
38234 ;mov ax,es ; * ; 18/07/2023
38235 000031D4 39D0 cmp ax,dx ; Present segment >= given segment?
38236 000031D6 731D jae short atuX ; Yep--done.
38237
38238 000031D8 E8DDFF call isSysMCB ; Returns with ZF set if this is a system MCB
38239 000031DB 7501 jnz short atu20
38240
38241 000031DD 41 inc cx ; If it _was_ a system MCB, we're in a new UMB.
38242 atu20:
38243 ;mov al,[es:ARENA.SIGNATURE]
38244 ;cmp al,arena_signature_end ; 'z'
38245 ; 22/07/2023
38246 ; ax = es
38247 ;mov ax,es ; **
38248 000031DE 2603060300 add ax,[es:ARENA.SIZE]
38249 000031E3 26803E00005A cmp byte [es:ARENA.SIGNATURE],arena_signature_end
38250 000031E9 7403 je short atu30 ; 'z' means this was the last MCB... that's it.
38251
38252 ;NextMCB es,ax
38253
38254 ;mov ax,es ; **
38255 ;add ax,[es:3]
38256 ;add ax,[es:ARENA.SIZE]
38257 000031EB 40 inc ax
38258 ; 22/07/2023
38259 ;mov es,ax ; *
38260 000031EC EBE4 jmp short atu10
38261
38262 ; -----
38263 ; if we get to atu30, they specified a number that was past the last MCB.
38264 ; make sure it's not _inside_ that MCB before we return an error condition.
38265 ; -----
38266
38267 atu30:
38268 ; 22/07/2023
38269 ; ax = es + [es:ARENA.SIZE]
38270 ;mov ax,es ; **
38271 ;add ax,[es:ARENA.SIZE] ; **
38272 000031EE 39D0 cmp ax,dx ; Present >= given?
38273 000031F0 7303 jae short atuX ; Yep! It _was_ inside.
38274
38275 atuE:
38276 000031F2 31C9 xor cx,cx ; 0 ; Else, fall through with UMB # == -1
38277 000031F4 49 dec cx ; (that makes it return 0xFFFF and sets CF)
38278 atuX:
38279 000031F5 89C8 mov ax,cx ; Return the UMB number in AX
38280
38281 pop es
; 01/01/2023

```

```

38282         ;pop     dx
38283         ;pop     cx
38284 000031F8 C3         retn
38285
38286 ;-----
38287 *** convUMB - checks after GetXNum to convert an address to a UMB number
38288 -- if GetXNum read a hex number, we interpret it as a segment
38289 address rather than a UMB number... and use that address to look up a UMB.
38290 This routine checks for that condition and calls AddrToUmb if necessary.
38291 ;-----
38292 ENTRY:  AX contains a UMB number or segment, gnradox has been set by GetXNum
38293 EXIT:   AX will contain a UMB number
38294 ERROR:  None
38295 USES:   Flags, AX
38296 ;-----
38297
38298         ; 01/01/2023 - Retro DOS v4.2
38299 convUMB:
38300         cmp     word [cs:gnradix],16
38301         jne     short cu10      ; If it didn't read in hex, it's not an address
38302         call    AddrToUmb      ; Else, convert the address to a UMB number
38303         ;cmp     ax,0FFFFh
38304         ;jne     short cu10
38305         ;inc     ax              ; If too high, ignore it (make it conventional)
38306         ; 01/01/2023
38307         inc     ax
38308         jz      short cu10      ; If too high, ignore it (make it conventional)
38309         dec     ax
38310 cu10:
38311         retn
38312
38313 ; 01/01/2023 - Retro DOS v4.2
38314 %if 0
38315 ;-----
38316 *** setUMBs - links umbs and sets allocation strategy for a load
38317 -- if LoadHigh, the allocation strategy MAY be LOW_FIRST instead
38318 of the usual HIGH_FIRST. See the code.
38319 ;-----
38320 ENTRY:  None
38321 EXIT:   None
38322 ERROR:  None
38323 USES:   Flags, fm_umb, fm_strat
38324 ;-----
38325 setUMBs:
38326         push    ax
38327         push    bx
38328         call    fm_link
38329         pop     bx
38330         pop     ax
38331         retn
38332 %endif
38333
38334 ; 18/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
38335 ; loadLow subroutine is not used anywhere of IO.SYS 6.22 (& 5.0)
38336 %if 0
38337 ;-----
38338 *** loadLow - returns AL==0 if UMB0 == 0, else AL==1
38339 ;-----
38340 ENTRY:  None
38341 EXIT:   AL==0 if mem strategy should be set to LOW_FIRST, else AL==1
38342         Carry set if UMB0 not specified (_NOT_ an error)
38343 ERROR:  None
38344 USES:   Flags, fm_strat, fm_umb
38345 ;-----
38346 ; We want to set the memory strategy to LOW_FIRST if the user specified a
38347 ; load UMB, and it is 0. That 0 can be either from the user having _specified_
38348 ; zero (/L:0;...), or from having specified a too-big min size (/L:1,99999999)
38349 ; such that the load UMB is too small, and shouldn't be used.
38350 ;-----
38351 loadLow:
38352         push    ds
38353         push    cs              ; Point DS into appropriate data segment
38354         pop     ds
38355
38356         mov     al,[UmbLoad]
38357         mov     al,[cs:UmbLoad]
38358         cmp     al,UNSPECIFIED ; 0FFh, -1
38359         jne     short l110
38360
38361         stc
38362 l115:    mov     al,1            ; Return with AL==1 && STC if no UMBS specified
38363         ;stc
38364         jmp     short l1x
38365         retn
38366 l110:
38367         or      al,al           ; AL=the load UMB: Is it == 0?
38368         jz      short l1x      ; Yep... CF==0 (from OR) && AL=0, so just exit
38369         jnz     short l115     ; 09/04/2019 - Retro DOS v4.0
38370         retn
38371
38372         mov     al,1
38373         cll
38374 ;l1x:
38375         pop     ds              ; Return DS to where it was
38376         retn
38377 %endif
38378
38379 ;-----
38380 *** HideUMBs - links UMBS and hides upper-memory as appropriate
38381 ;-----
38382 ENTRY:  None
38383 EXIT:   None
38384 ERROR:  None
38385 USES:   Flags, fm_strat, fm_umb
38386 ;-----
38387
38388         ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
38389         ; (SYSINIT:322Fh)
38390 HideUMBs:
38391         ; 01/01/2023
38392         push    ax
38393         push    cx
38394         push    ds
38395         push    es
38396
38397         ; 01/01/2023

```



```

38406             ; ds = cs
38407
38408 0000320A E86E02 call    UmbTest      ; See if we REALLY linked in anything...
38409 0000320D 7232   jc      short husX    ; ...if not, there's nothing for us to do.
38410
38411 0000320F E892FD call    FixMem        ; Concatenate adjacent free MCBs in upper mem
38412
38413             ; call setUMBs      ; Link UMBS and set memory-allocation strategy
38414             ; 01/01/2023
38415 00003212 E8DCFD call    fm_link
38416
38417             ; putdata fInHigh,1 ; Remember that we're now running high
38418             ; mov byte [cs:fInHigh],1
38419             ; 01/01/2023
38420 00003215 C606[F923]01 mov    byte [fInHigh],1
38421
38422             ; call GetLoadUMB    ; See if they gave us a list to leave free
38423             ; mov al,[cs:UmbLoad] ; 09/04/2019 - Retro DOS v4.0
38424             ; 01/01/2023
38425 0000321A A0[FD23] mov    al,[UmbLoad]
38426
38427 0000321D 3CFF     cmp    al,UNSPECIFIED ; If they didn't,
38428 0000321F 7420     je      short husX    ; then we shouldn't do this loop:
38429
38430 00003221 31C9     xor     cx,cx
38431
38432             ; -----
38433             ; HUS10-CX - UMB number (after inc, 1==first UMB)
38434             ; -----
38435
38436 00003223 41       hus10:   inc     cx            ; For each UMB:
38437             ; 01/01/2023
38438 00003224 80F910   cmp     cl,MAXUMB
38439             ; cmp cx,MAXUMB ; 16
38440 00003227 730E     jae     short hus20
38441
38442 00003229 88C8     mov     al,cl          ; (stopping as soon as we're outside of the
38443 0000322B 06       push    es
38444 0000322C E8A200   call    findUMB        ; valid range of UMBS)
38445 0000322F 07       pop     es            ; push/pop: trash what findumb finds. :-)
38446 00003230 7205     jc      short hus20
38447
38448             ; 02/01/2023
38449             ; push cx ; *
38450 00003232 E84F01   call    _hideUMB_      ; hide what we need to hide.
38451             ; pop cx ; *
38452
38453 00003235 EBEC     jmp     short hus10
38454
38455             hus20:
38456             ; call GetLoadUMB    ; Now check if they offered /L:0
38457             ; 01/01/2023
38458             ; ds = cs
38459             ; mov al,[UmbLoad]
38460             ; mov al,[cs:UmbLoad] ; 09/04/2019 - Retro DOS v4.0
38461             ; or byte [UmbLoad],0
38462 00003237 800E[FD23]00 or     al,al
38463             ; or al,al          ; --Is the load UMB 0? (-1==unspecified)
38464 0000323E E86802   jnz     short husX     ; If not, we're done.
38465
38466             call    h1_unlink    ; If so, however, fix UMBS and strategy.
38467             husX:
38468             pop     es
38469             ; 01/01/2023
38470             ; pop ds
38471 00003242 C3       ; pop cx
38472             ; pop ax
38473             ; retn
38474
38475             ; -----
38476             ; *** GetLoadUMB - Returns the load UMB number in AL (-1 if not specified)
38477             ; -----
38478             ; ENTRY:  None
38479             ; EXIT:   AL == load UMB
38480             ; ERROR:  None
38481             ; USES:   Flags, AX
38482             ; -----
38483             ; GetLoadUMB:
38484             ; ;getdata al, UmbLoad
38485             ; push ds
38486             ; push cs
38487             ; pop ds
38488             ; mov al,[UmbLoad]
38489             ; pop ds
38490             ; retn
38491
38492             ; -----
38493             ; *** GetLoadSize - Returns the load UMB minimum size (0 if not specified)
38494             ; -----
38495             ; ENTRY:  None
38496             ; EXIT:   AX == load UMB minimum size
38497             ; ERROR:  None
38498             ; USES:   Flags, AX
38499             ; -----
38500             ; 08/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
38501             %if 0
38502             ; 01/01/2023 - Retro DOS v4.2
38503             GetLoadSize:
38504             ; 09/04/2019 - Retro DOS v4.0
38505             ; mov al,[cs:UmbLoad]
38506             ; 01/01/2023
38507             ; ds = cs
38508             mov     al,[UmbLoad]
38509             jmp     short GetSize
38510
38511             ; push bx
38512             ; ;push si
38513             ; push ds
38514             ; push cs
38515             ; pop ds
38516
38517             ; mov al,[UmbLoad]
38518
38519             ; xor ah,ah          ; ax==UMB
38520             ; mov bx,UmbSize     ; bx==array
38521             ; shl al,1           ; ax==offset
38522             ; ;add ax,bx         ; ax==element index
38523             ; ;mov si,ax        ; ds:si==element index
38524
38525             ; ;lodsw             ; hh
38526
38527             ; add bx,ax
38528             ; mov ax,[bx]
38529

```

```

38530         ;pop     ds
38531         ;;pop    si
38532         ;pop     bx
38533         ;retn
38534     %endif
38535
38536     ; -----
38537     ; *** GetSize - Returns the UMB in AL's minimum size (0 if not specified)
38538     ; -----
38539     ; ENTRY:  AL == a UMB number
38540     ; EXIT:   AX == UMB minimum size, as specified by the user
38541     ; ERROR:  None
38542     ; USES:   Flags, AX
38543     ; -----
38544
38545     ; 08/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
38546     GetLoadSize:
38547         ; ds = cs
38548         ;mov     al,[UmbLoad]
38549         ; al = [UmbLoad]
38550         ; ....
38551
38552         ; 01/01/2023 - Retro DOS v4.2
38553     GetSize:
38554         ; 09/04/2019 - Retro DOS v4.0
38555
38556         ;push    bx ; 01/01/2023
38557         ;push    si
38558         ;push    ds
38559         ;push    cs
38560         ;pop     ds
38561
38562         xor     ah,ah                ; ax==UMB
38563         mov     bx,UmbSize           ; bx==array
38564         shl     al,1                ; ax==offset
38565         add     ax,bx                ; ax==element index
38566         mov     si,ax                ; ds:si==element index
38567
38568         ;lodsw                        ; ax==size
38569
38570         add     bx,ax
38571         ; 01/01/2023
38572         ; ds = cs
38573         mov     ax,[bx]
38574         ;mov     ax,[cs:bx]
38575
38576         ;pop     ds
38577         ;pop     si
38578         ;pop     bx ; 01/01/2023
38579     s1s10:    ; 08/09/2023
38580         retn
38581
38582     ; -----
38583     ; *** StoLoadUMB - Overrides the load UMB number with what's in AL
38584     ; -----
38585     ; ENTRY:  AL == new load UMB
38586     ; EXIT:   None
38587     ; ERROR:  None
38588     ; USES:   Flags, AX
38589     ; -----
38590     ; CAUTION: Should only be used if /L:... was used. Logically, that is the only
38591     ;           time you would ever need this, so that's okay.
38592     ; -----
38593
38594     ; StoLoadUMB subroutine is not used anywhere
38595     ; of PC DOS 7.1 IBMBIO.COM (& MSDOS 6.21 IO.SYS)
38596     ; Erdogan Tan - 18/07/2023
38597
38598     ;StoLoadUMB:
38599     ; ;putdata UmbLoad, al
38600     ; ;push    es
38601     ; ;push    cs
38602     ; ;pop     es ; mov [cs:UmbLoad], al !!!! ; 08/09/2023
38603     ; ;mov     [es:UmbLoad],al
38604     ; ;pop     es
38605     ; ;retn
38606
38607     ; -----
38608     ; *** StoLoadSize - Overrides the load UMB minimum size with what's in AX
38609     ; -----
38610     ; ENTRY:  AL == new load size
38611     ; EXIT:   None
38612     ; ERROR:  None
38613     ; USES:   Flags, AX
38614     ; -----
38615
38616     ; 08/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
38617     ; 01/01/2023 - Retro DOS v4.2
38618     StoLoadSize:
38619         ; 01/01/2023
38620         ;push    dx
38621
38622         ;getdata dl, UmbLoad          ; Put UMB# in DL and size in AX
38623         ;
38624         ;push    ds
38625         ;push    cs
38626         ;pop     ds
38627         ;mov     dl,[UmbLoad]
38628         ;pop     ds
38629
38630         ; 08/09/2023
38631         ; MSDOS 6.21 IO.SYS - SYSINIT:32B6h
38632         ; PC DOS 7.1 IBMBIO.COM - SYSINIT:3831h
38633
38634         ; ;mov     dl,[UmbLoad]          ; BUG ! CL would/must be used here
38635         ; ; instead of DL (*) ; 18/07/2023
38636         ;mov     dl,[cs:UmbLoad] ; Retro DOS v4.0, v4.1, v4.2
38637         ;cmp     dl,UNSPECIFIED ; 0FFh
38638         ;je      short s1s10
38639
38640         ; ; BUG ! stowSiz uses CL instead of DL !
38641         ; ; (CL is set in ParseL which calls stowSiz)
38642         ; ; (This BUG existing in PC DOS 7.1 IBMBIO.COM also)
38643         ; ; Erdogan Tan - 18/07/2023
38644
38645         ; 08/09/2023 (BugFix)
38646         ;mov     cl,[cs:UmbLoad]
38647         ; 08/09/2023
38648         ; ds = cs
38649         mov     cl,[UmbLoad]
38650         cmp     cl,UNSPECIFIED ; 0FFh
38651         je      short s1s10
38652
38653         ; 08/09/2023
38654         ; call     stowSiz                ; we've got a function to do just this

```

```

38654 ;sls10:
38655 ; ; 01/01/2023
38656 ; ;pop dx
38657 ; ;retn
38658
38659 ; 08/09/2023
38660 ; ;jmp stowSiz
38661 ; ;jmp short stowSiz
38662
38663 ; 08/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
38664 %if 1
38665 ;
38666 ;*** stowSiz - marks a given UMB as having a given minimum size
38667 ;
38668 ; ENTRY: CL contains UMB number, AX contains size
38669 ; EXIT: None
38670 ; ERROR: None
38671 ; USES: AX, DX, Flags, variables in highvar.inc
38672 ;
38673
38674 ; 13/05/2019
38675
38676 ; 01/01/2023 - Retro DOS v4.2
38677 stowSiz:
38678 ; 01/01/2023
38679 ;push bx
38680 ; ;push di ; ?
38681 ;push es
38682
38683 ;push cs
38684 ;pop es
38685
38686 00003258 88CB mov bl,cl ; Now bl==UMB number, AX==size
38687 0000325A B700 mov bh,0 ; bx==UMB number, AX==size
38688 0000325C D0E3 shl bl,1 ; bx==offset into array, AX=size
38689 ;mov [es:bx+UmbSize],ax ; Store the size
38690 ; 01/01/2023
38691 0000325E 2E8987[0E24] mov [cs:bx+UmbSize],ax ; Store the size
38692
38693 ; 01/01/2023
38694 ;pop es
38695 ; ;pop di ; ?
38696 ;pop bx
38697
38698 00003263 C3 retn
38699 %endif
38700
38701 ;
38702 ;*** hideUMB - marks as HIDDEN all FREE elements in UMB passed as AL
38703 ;
38704 ; ENTRY: AL must indicate a valid UMB; 0==conv && is invalid.
38705 ; EXIT: None; free elements in UMB marked as hidden
38706 ; ERROR: None
38707 ; USES: Flags
38708 ;
38709
38710 ; 01/01/2023 - Retro DOS v4.2
38711 hideUMB:
38712 ; 02/01/2023
38713 00003264 52 push dx ; (*)
38714 ; 01/01/2023
38715 ;push ax
38716 00003265 06 push es
38717
38718 00003266 E86800 call findUMB ; (*) ; Returns with carry if err, else ES == MCB
38719 00003269 7224 jc short hux
38720
38721 ;
38722 ; HU10--ES - MCB inside UMB; if it's a system MCB,
38723 ; we're not in the same UMB, so exit.
38724 ;
38725 ;
38726 0000326B E84AFF hu10: call isSysMCB ; Returns with ZF set if owner is SYSTEM
38727 0000326E 741F jz short hux ; If it is, we've finished the UMB.
38728 ;call isFreeMCB ; Returns with ZF set if owner is 0
38729 00003270 26830E010000 or word [es:ARENA.OWNER],0
38730 00003276 7503 jnz short hu20
38731
38732 00003278 E81700 call hideMCB
38733 hu20:
38734 ;mov al,[es:ARENA.SIGNATURE]
38735 ;cmp al,arena_signature_end ;'z'
38736 ; 19/07/2023
38737 0000327B 26803E00005A cmp byte [es:ARENA.SIGNATURE],'z'
38738 00003281 740C jz short hux ; 'z' means this was the last MCB... that's it.
38739
38740 ;NextMCB es,ax ; Go on forward.
38741 00003283 8CC0 mov ax,es
38742 ;add ax,[es:3]
38743 00003285 2603060300 add ax,[es:ARENA.SIZE]
38744 0000328A 40 inc ax
38745 0000328B 8EC0 mov es,ax
38746
38747 0000328D EBDC jmp short hu10
38748
38749 0000328F 07 hux:
38750 pop es
38751 ; 01/01/2023
38752 ;pop ax
38753 00003290 5A pop dx ; (*)
38754 00003291 C3 retn
38755
38756 ; 02/01/2023
38757 %if 0
38758 ;
38759 ;*** isTiny - returns with ZF set if user didn't specify /S
38760 ;
38761 ; ENTRY: None
38762 ; EXIT: ZF set if user DIDN'T specify /S
38763 ; ERROR: None
38764 ; USES: Flags
38765 ;
38766 ;
38767 ;
38768 ; 02/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
38769 isTiny:
38770 ; 02/01/2023
38771 ;push ax
38772
38773 ;getdata al,fUmbTiny
38774 ;
38775 ;push ds
38776 ;push cs
38777 ;pop ds

```

```

38778         ;mov     al,[fumbTiny]
38779         ;pop     ds
38780
38781         ; 09/09/2023
38782         ;mov     al,[cs:fumbTiny]
38783         ; 02/01/2023
38784         ; ds = cs
38785         mov     al,[fumbTiny]
38786
38787         or      al,al
38788         ; 02/01/2023
38789         ;pop     ax
38790         retn
38791
38792 %endif
38793
38794 ; -----
38795 *** isFreeMCB - returns with ZF set if current MCB (ES:0) is FREE
38796 ; -----
38797 ; ENTRY:      ES:0 should point to an MCB
38798 ; EXIT:       ZF set if MCB is free, else !ZF
38799 ; ERROR:      None
38800 ; USES:       Flags
38801 ; -----
38802
38803 ;isFreeMCB:
38804 ;     or      word [es:ARENA.OWNER],0
38805 ;     retn
38806
38807 ; -----
38808 *** hideMCB - marks as HIDDEN the MCB at ES:0
38809 ; -----
38810 ; ENTRY:      ES:0 should point to an MCB
38811 ; EXIT:       None; MCB marked as HIDDEN
38812 ; ERROR:      None
38813 ; USES:       None
38814 ; -----
38815
38816 hideMCB:
38817     mov     word [es:ARENA.OWNER],SystemPSPOwner ; 8
38818     mov     word [es:ARENA.NAME+0], 'HI' ; 4948h
38819     mov     word [es:ARENA.NAME+2], 'DD' ; 4444h
38820     mov     word [es:ARENA.NAME+4], 'EN' ; 4E45h
38821     mov     word [es:ARENA.NAME+6], ' ' ; 2020h
38822     retn
38823
38824 ; -----
38825 *** unHideMCB - marks as FREE the MCB at ES:0
38826 ; -----
38827 ; ENTRY:      ES:0 should point to an MCB
38828 ; EXIT:       None; MCB marked as FREE
38829 ; ERROR:      None
38830 ; USES:       None
38831 ; -----
38832
38833         ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
38834
38835 unHideMCB:
38836     ; 03/01/2023
38837     ;push     ax
38838     mov     word [es:ARENA.OWNER],FreePSPOwner ; 0
38839     mov     ax,' ' ; 2020h
38840     mov     [es:ARENA.NAME+0],ax
38841     mov     [es:ARENA.NAME+2],ax
38842     mov     [es:ARENA.NAME+4],ax
38843     mov     [es:ARENA.NAME+6],ax
38844     ; 03/01/2023
38845     ;pop     ax
38846     retn
38847
38848 ; -----
38849 *** findUMB - makes ES:0 point to the first MCB in UMB given as AL
38850 -- returns UmbHEAD pointer (0x9FFF) if passed AL==0
38851 ; -----
38852 ; ENTRY:      AL should be to a valid UMB number
38853 ; EXIT:       ES:0 points to first MCB in UMB (_not_ the 8+SC MCB that heads it)
38854 ; ERROR:      Carry set if couldn't reach UMB (too high)
38855 ; USES:       Flags, ES
38856 ; -----
38857
38858         ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
38859         ; (SYSINIT:3344h)
38860 findUMB:
38861     ; 01/01/2023
38862     ;push     ax
38863     ; 02/01/2023
38864     push     cx ; *
38865     push     dx
38866
38867     xor     ah,ah ; Zap ah, so al==ax
38868
38869     mov     dx,ax ; Store the to-be-found UMB number in DX
38870
38871     call    UmbHead ; Returns first UMB segment in AX
38872     ; 22/07/2023
38873     ;mov     es,ax ; *
38874     xor     cx,cx ; Pretend we're on UMB 0 for now...
38875
38876     ; 22/07/2023
38877 fu10:
38878     mov     es,ax ; * ; **
38879
38880 ; FU10--CX - This UMB number; 0 == conventional
38881 ; DX - The UMB number they're looking for
38882 ; ES - The current MCB address
38883 ; -----
38884
38885 ;fu10:
38886     cmp     cx,dx ; If CX==DX, we've found the UMB we're
38887     je      short fuX ; searching for--so exit.
38888
38889     call    issysMCB ; Returns with ZF set if owner is SYSTEM
38890     jnz     short fu20
38891
38892     inc     cx ; If it _was_ SYSTEM, we're in a new UMB.
38893 fu20:
38894     mov     al,[es:ARENA.SIGNATURE]
38895     cmp     al,arena_signature_end ; 'z'
38896     ; 19/07/2023
38897     cmp     byte [es:ARENA.SIGNATURE],arena_signature_end
38898     je      short fuE ; 'z' means this was the last MCB... that's it.
38899
38900     ;NextMCB es,ax ; Go on forward.
38901     ; 22/07/2023

```

```

38902             ; ax = es
38903             ;mov ax,es ; * ; 22/07/2023
38904             ;add ax,[es:3]
38905 000032EF 2603060300 add ax,[es:ARENA.SIZE]
38906 000032F4 40 inc ax
38907             ; 22/07/2023
38908             ;mov es,ax ; **
38909 000032F5 EBE4 jmp short fu10
38910 fuE:
38911 000032F7 F9 stc
38912 fuX:
38913             ; 01/01/2023
38914             ;pop dx
38915             ; 02/01/2023
38916 000032F8 59 pop cx ; *
38917             ;pop ax ; The address is already in ES.
38918 000032F9 C3 retn
38919
38920             ; -----
38921             ; *** BigFree - makes ES:0 point to the largest free MCB in UMB given as AL
38922             ; -----
38923             ; ENTRY: AL should be to a valid UMB number
38924             ; EXIT: ES:0 points to largest free MCB in UMB, AX returns its size
38925             ; ERROR: Carry set if couldn't reach UMB (0 or too high)
38926             ; USES: Flags, ES
38927             ; -----
38928
38929             ; 01/01/2023 - Retro DOS v4.2
38930 BigFree:
38931             ; 01/01/2023
38932             ;push bx
38933 000032FA 51 push cx
38934
38935 000032FB E8D3FF call findUMB ; Returns with CF if err, else ES==MCB
38936 000032FE 723A jc short bfx ; (would be "jc bfe"; it just does stc)
38937
38938 00003300 31DB xor bx,bx ; Segment address of largest free MCB
38939 00003302 31C9 xor cx,cx ; Size of largest free MCB
38940
38941             ; -----
38942             ; BF10--ES - Current MCB address
38943             ; BX - Address of largest free MCB so far
38944             ; CX - Size of largest free MCB so far
38945             ; -----
38946
38947 bf10:
38948 00003304 E8B1FE call isSysMCB ; If we've left the MCB, we're done.
38949 00003307 7428 jz short bf30
38950
38951             ;call isFreeMCB ; Returns with ZF set if owner is 0
38952 00003309 26830E010000 or word [es:ARENA.OWNER],0
38953 0000330F 750C jnz short bf20
38954
38955 00003311 26A10300 mov ax,[es:ARENA.SIZE]
38956             ;cmp cx,[es:ARENA.SIZE] ; Compare sizes...
38957 00003315 39C1 cmp cx,ax
38958             ;jg short bf20 ; Unless we're bigger,
38959             ; 19/07/2023
38960 00003317 7D04 jge short bf20
38961
38962 00003319 8CC3 mov bx,es ; Store this new element's address,
38963             ;mov cx,[es:ARENA.SIZE] ; and its size.
38964 0000331B 89C1 mov cx,ax
38965
38966 bf20:
38967             ;mov al,[es:ARENA.SIGNATURE]
38968             ;cmp al,arena_signature_end; 'z'
38969             ; 19/07/2023
38970             ;cmp byte [es:0],'z'
38971 0000331D 26803E00005A cmp byte [es:ARENA.SIGNATURE],arena_signature_end
38972 00003323 740C jz short bf30 ; 'z' means this was the last MCB.
38973
38974             ;NextMCB es,ax ; Go on forward.
38975 00003325 8CC0 mov ax,es
38976             ;add ax,[es:3]
38977 00003327 2603060300 add ax,[es:ARENA.SIZE]
38978 0000332C 40 inc ax
38979 0000332D 8EC0 mov es,ax
38980
38981 0000332F EBD3 jmp short bf10
38982
38983 bf30: mov es,bx ; Return the address
38984             mov ax,cx ; Return the size
38985             or bx,bx
38986 00003337 7501 jnz short bfx ; (if size==0, there's nothing free)
38987
38988 bfE: stc
38989
38990 bfX:
38991             pop cx
38992             ; 01/01/2023
38993 0000333B C3 pop bx
38994             retn
38995
38996             ; -----
38997             ; *** isSpecified - sets ZF if UMB in AL wasn't specified in DH/LH line.
38998             ; -----
38999             ; ENTRY: AL should be to a valid UMB number
39000             ; EXIT: ZF set if UMB wasn't specified, ZF clear if it was
39001             ; ERROR: None
39002             ; USES: Flags
39003             ; -----
39004
39005             ; 02/01/2023 - Retro DOS v4.2
39006 isSpecified:
39007             ; 02/01/2023
39008             ;push ax
39009
39010 0000333C 30FF xor bh,bh
39011 0000333E 88C3 mov bl,al
39012
39013             ;getdata al,DS:UmbUsed[bx]
39014             ;
39015             ;push ds
39016             ;push cs
39017             ;pop ds
39018             ;mov al,[bx+UmbUsed]
39019             ;pop ds
39020
39021             ;mov al,[cs:bx+UmbUsed]
39022             ; 02/01/2023
39023             ; ds = cs
39024 00003340 8A87[FE23] mov al,[bx+UmbUsed]
39025

```

```

39026 00003344 08C0      or      al,al          ; Sets ZF if al==0 (ie, if unspecified)
39027
39028      ; 09/09/2023
39029      ; 02/01/2023
39030      ;pop      ax
39031
39032 00003346 C3        retn
39033
39034      ; -----
39035      ; *** shrinkMCB - breaks an MCB into two pieces, the lowest one's size==AX
39036      ; -----
39037      ; ENTRY:      AX == new size, ES:0 == current MCB
39038      ; EXIT:       None; MCB broken if carry clear
39039      ; ERROR:      Carry set if MCB isn't as large as AX+0x20 (not a useful split)
39040      ; USES:       Flags
39041      ; -----
39042      ; If the size of the to-be-split MCB isn't at least 0x20 bytes greater than
39043      ; the specified new size, the split is useless; if it's only 0x10 bytes, that
39044      ; 0x10 will be used to make a header that mentions a 0-byte free space, and
39045      ; that just sucks up 0x10 bytes for nothing. So we make 0x20 bytes the
39046      ; minimum for performing a split.
39047      ; -----
39048
39049      MIN_SPLIT_SIZE      equ      20h
39050
39051      ; 02/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
39052
39053      shrinkMCB:
39054      ;pushreg <bx,cx,es>
39055      ; 02/01/2023
39056      ;push      bx
39057      ;push      cx
39058      ;push      es
39059
39060      mov      bx,ax          ; Move things around... and
39061      ; 02/01/2023
39062      ;mov      ax,es          ; save this one for later.
39063
39064      mov      cx,[es:ARENA.SIZE]
39065      ; 02/01/2023
39066      mov      ax,cx
39067
39068      sub      ax,MIN_SPLIT_SIZE ; 32
39069      ;sub      cx,MIN_SPLIT_SIZE ; 32
39070      ;cmp      bx,cx          ; {New size} vs {Current size-20h}
39071      ;ja      short smE      ; if wanted_size > cur-20h, abort.
39072      ; 18/12/2022
39073      ;cmp      cx,bx
39074      ; 02/01/2023
39075      cmp      ax,bx
39076      jnb     short smE ; (*)
39077
39078      mov      dl,[es:ARENA.SIGNATURE]
39079
39080      ;mov      cx,[es:ARENA.SIZE]
39081      ; 02/01/2023
39082      mov      ax,es
39083
39084      mov      [es:ARENA.SIZE],bx
39085      mov      byte [es:ARENA.SIGNATURE],'M'
39086
39087      add      ax,bx
39088      inc      ax
39089      mov      es,ax          ; Move to new arena area
39090
39091      mov      ax,cx
39092      sub      ax,bx
39093      ; 12/12/2022
39094      ; ax > 0
39095      dec      ax          ; And prepare the new size
39096
39097      ; 18/12/2022
39098      mov      [es:ARENA.SIGNATURE],dl
39099      ;mov      word [es:ARENA.OWNER],0 ; (**)
39100      mov      [es:ARENA.SIZE],ax
39101      ;mov      ax, ; 2020h
39102      ;mov      [es:ARENA.NAME+0],ax ; (**)
39103      ;mov      [es:ARENA.NAME+2],ax ; (**)
39104      ;mov      [es:ARENA.NAME+4],ax ; (**)
39105      ;mov      [es:ARENA.NAME+6],ax ; (**)
39106
39107      ; 18/12/2022
39108      call     freeMCB ; (**)
39109
39110      ; 12/12/2022
39111      ; cf=0
39112      ;clc
39113      ; 18/12/2022
39114      ;jmp      short smX
39115      smE:
39116      ; 18/12/2022
39117      ; cf=1 (*)
39118      ;stc
39119      smX:
39120      ;popreg <es,cx,bx>
39121      pop      es
39122      pop      cx
39123      ; 02/01/2023
39124      ;pop      bx
39125      retn
39126
39127      ; -----
39128      ; *** hideUMB? - hides as appropriate the UMB in CL
39129      ; -----
39130      ; ENTRY:      CL should be to a valid UMB number, and AX to its address (findUMB)
39131      ; EXIT:       None; UMB is hidden as necessary
39132      ; ERROR:      None
39133      ; USES:       Flags, AX, CX
39134      ; -----
39135      ; PRIMARY LOGIC:
39136      ;
39137      ; If the UMB is specified in the DH/LH statement, then:
39138      ; If the largest free segment is too small (check specified size), then:
39139      ; Pretend it wasn't ever specified, and fall out of this IF.
39140      ; Else, if largest free segment is LARGER than specified size, then:
39141      ; If /S was given on the command-line, then:
39142      ; Break that element into two pieces
39143      ; Set a flag that we're shrinking
39144      ; Endif
39145      ; Endif
39146      ; Endif
39147      ; If the UMB is NOT specified (or was removed by the above):
39148      ; Hide all free elements in the UMB
39149      ; If the flag that we're shrinking was set, then:

```



```

39150 ; UN-hide the lower portion of the shrunken UMB
39151 ; ENDF
39152 ; ENDF
39153 ; -----
39154 ;
39155 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
39156 ; (SYSINIT:3426h)
39157 _hideUMB:
39158 ; 02/01/2023
39159 ; ds = cs
39160 ;
39161 ; 01/01/2023
39162 ;push bx
39163 ;push dx
39164 push es
39165
39166 mov al,cl
39167 call isSpecified ; Returns ZF set if al's umb was NOT specified
39168 jz short hu_20
39169
39170 mov al,cl ; Retrieve the size of the largest
39171 call BigFree ; free element in AX; put its address in ES
39172 jc short hu_20 ; Oops. Errors mean skip this part.
39173
39174 push ax ; TOS==size of BigFree in UMB (popped as BX)
39175 mov al,cl ; Retrieve the user's specified
39176 call GetSize ; minimum size for this umb (into AX)
39177 pop bx ; Now BX==BigFree, AX==Specified Size
39178
39179 or ax,ax ; If they didn't specify one,
39180 jz short hu_20 ; Skip over all this.
39181
39182 cmp ax,bx ; Ah... if (specified > max free)
39183 jbe short hu_10
39184
39185 mov al,cl ; Then mark that UMB as unused. Nya nya.
39186 call unMarkUMB
39187 jmp short hu_20
39188 hu_10:
39189 ;call isTiny ; Returns ZF clear if user specified /S
39190 ;jz short hu_20
39191 ; 02/01/2023
39192 ;isTiny:
39193 ;mov al,[fumbTiny] ; ds = cs
39194 ;or al,al
39195 or byte [fumbTiny],0
39196 jz short hu_20
39197
39198 call shrinkMCB ; They specified /S, so shrink the MCB to AX
39199 jc short hu_20 ; Ah... if didn't shrink after all, skip this:
39200
39201 mov dx,es
39202 jmp short hu_30 ; skip the spec check.. we wanna hide this one.
39203
39204 hu_20: mov ax,cx
39205 call isSpecified ; If they specified this UMB, we're done...
39206 jnz short hu_X ; so leave.
39207
39208 xor dx,dx
39209 hu_30: mov al,cl
39210
39211 call hideUMB ; Hides everything in UMB #al
39212
39213 or dx,dx ; Did we shrink a UMB? If not, DX==0,
39214 jz short hu_X ; So we should leave.
39215
39216 mov es,dx ; Ah, but if it isn't, DX==the MCB's address;
39217 call unHideMCB ; Un-hides the lower portion of that MCB.
39218 hu_X:
39219 pop es
39220 ; 01/01/2023
39221 ;pop dx
39222 ;pop bx
39223 retn
39224
39225 ; -----
39226 ; *** UnFreeze - Marks FROZEN elements as FREE
39227 ; -----
39228 ;
39229 ; Entry: None
39230 ; Exit: None; all 8+FROZEN elements are marked as FREE, from any UMB.
39231 ; Error: None
39232 ; Uses: Flags
39233 ; -----
39234 ;
39235 ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
39236 UnFreeze:
39237 ; 03/01/2023
39238 ;push ax
39239 push es
39240
39241 call UmbHead ; Returns with carry if err, else ES == MCB
39242 jc short ufx
39243
39244 ; 22/07/2023
39245 uf10: mov es,ax ; *
39246
39247 ; -----
39248 ; UF10--ES - Current MCB address
39249 ; -----
39250
39251 ;uf10:
39252 call isFrozMCB ; Returns with ZF set if MCB is FROZEN
39253 jnz short uf20
39254 call unHideMCB
39255 ; 09/09/2023
39256 ; ax <> es
39257 mov ax,es ; *
39258 uf20:
39259 ;mov al,[es:ARENA.SIGNATURE]
39260 ;cmp al,arena_signature_end ; 'z'
39261 ; 22/07/2023
39262 cmp byte [es:ARENA.SIGNATURE],arena_signature_end ; 'z'
39263 jz short ufx ; 'z' means this was the last MCB.. that's it.
39264
39265 ;NextMCB es,ax ; Go on forward.
39266 ; 22/07/2023
39267 ; ax = es
39268 ;mov ax,es ; *
39269 ;add ax,[es:3]
39270 add ax,[es:ARENA.SIZE]
39271 inc ax
39272 ; 22/07/2023
39273

```

```

39274             ;mov     es,ax
39275 000033F2 EBE4 jmp     short uf10
39276
39277 000033F4 07   ufx:    pop     es
39278             ; 03/01/2023
39279             ;pop     ax
39280 000033F5 C3   retn
39281
39282
39283 ;-----
39284 *** isFrozMCB - returns with ZF set if current MCB (ES:0) is FROZEN
39285 ;-----
39286 ; ENTRY:     ES:0 should point to an MCB
39287 ; EXIT:      ZF set if MCB is frozen, else !ZF
39288 ; ERROR:     None
39289 ; USES:      Flags
39290 ;-----
39291
39292 isFrozMCB:
39293     ;push     ax
39294
39295     ;mov     ax,[es:ARENA.OWNER] ; Check the owner...
39296     ;cmp     ax,SystemPSPOwner ; 8 (for US OR Japan) is valid
39297 000033F6 26833E010008 cmp     word [es:ARENA.OWNER],SystemPSPOwner
39298 000033FC 7522     jne     short ifmX
39299
39300     ;mov     ax,[es:ARENA.NAME+0]
39301 000033FE 26813E08004652 cmp     ax,'FR' ; 5246h
39302 00003405 7519     jne     short ifmX
39303     ;mov     ax,[es:ARENA.NAME+2]
39304     ;cmp     ax,'OZ' ; 5A4Fh
39305 00003407 26813E0A004F5A cmp     word [es:ARENA.NAME+2],'OZ'
39306 0000340E 7510     jne     short ifmX
39307     ;mov     ax,[es:ARENA.NAME+4]
39308     ;cmp     ax,'EN' ; 4E45h
39309 00003410 26813E0C00454E cmp     word [es:ARENA.NAME+4],'EN'
39310 00003417 7507     jne     short ifmX
39311     ;mov     ax,[es:ARENA.NAME+6]
39312     ;cmp     ax,' ' ; 2020h
39313 00003419 26813E0E002020 cmp     word [es:ARENA.NAME+6],' '
39314
39315 ifmX:
39316     ;pop     ax
39317     retn
39318
39319 ;-----
39320 *** frezMCB - marks as 8+FROZEN the MCB at ES:0
39321 ;-----
39322 ; ENTRY:     ES:0 should point to an MCB
39323 ; EXIT:      None; MCB frozen
39324 ; ERROR:     None
39325 ; USES:      None
39326 ;-----
39327
39328 frezMCB:
39329     mov     word [es:ARENA.OWNER],SystemPSPOwner ; 8
39330     mov     word [es:ARENA.NAME+0],'FR'
39331     mov     word [es:ARENA.NAME+2],'OZ'
39332     mov     word [es:ARENA.NAME+4],'EN'
39333     mov     word [es:ARENA.NAME+6],' '
39334     retn
39335
39336 ;-----
39337 *** FreezeUMB - Marks FROZEN all UMB elements now FREE, save those in load UMB
39338 ;-----
39339 ; Entry:     None
39340 ; Exit:      None; all free elements not in load UMB marked as 8+FROZEN
39341 ; Error:     None
39342 ; Uses:      Flags
39343 ;-----
39344
39345     ; 01/01/2023 - Retro DOS v4.2
39346 FreezeUMB:
39347     ; 01/01/2023
39348     ;push     ax
39349     ;push     cx
39350     ;push     dx
39351     push     es
39352
39353     ;;call GetLoadUMB
39354     ; 01/01/2023
39355     ; ds = cs
39356 00003446 A0[FD23] mov     al,[cs:UmbLoad] ; 19/04/2019 - Retro DOS v4.0
39357     mov     al,[UmbLoad]
39358 00003449 30E4     xor     ah,ah ; Zap ah, so al==ax
39359 0000344B 89C2     mov     dx,ax ; Store the load UMB in DX, so we can skip it
39360
39361 0000344D E85BFD     call    UmbHead ; Returns first UMB segment in AX
39362     ; 22/07/2023
39363     ;mov     es,ax ; *
39364 00003450 31C9     xor     cx,cx ; Pretend we're on UMB 0 for now...
39365
39366     ; 22/07/2023
39367 fum10:
39368 00003452 8EC0     mov     es,ax ; *
39369
39370 ;-----
39371 ; FUM10--ES - Current MCB address
39372 ; CX - Current UMB number
39373 ; DX - UMB number to skip (load UMB)
39374 ;-----
39375
39376 ;fum10:
39377 00003454 E861FD     call    issysMCB ; Returns with ZF set if owner is SYSTEM
39378 00003457 7501     jnz     short fum20
39379
39380 00003459 41         inc     cx ; If it _was_ SYSTEM, we're in a new UMB.
39381 fum20:
39382 0000345A 39D1     cmp     cx,dx ; If this is the load UMB, we don't want to
39383 0000345C 740B     je      short fum30 ; freeze anything... so skip that section.
39384
39385     ;call    isFreeMCB ; Oh. If it's not free, we can't freeze it
39386 0000345E 26830E010000 or      word [es:ARENA.OWNER],0
39387 00003464 7503     jnz     short fum30 ; either.
39388
39389 00003466 E8B8FF     call    frezMCB
39390 fum30:
39391     ;mov     al,[es:ARENA.SIGNATURE]
39392     ;cmp     al,arena_signature_end ; 'Z'
39393     ; 22/07/2023
39394 00003469 26803E00005A cmp     byte [es:ARENA.SIGNATURE],arena_signature_end ; 'Z'
39395 0000346F 7408     je      short fumX ; 'Z' means this was the last MCB.. that's it.
39396
39397     ;NextMCB es, ax ; Go on forward.

```



```

39398             ; 22/07/2023
39399             ; ax = es
39400             ;mov ax,es
39401             ;add ax,[es:3]
39402 00003471 2603060300 add ax,[es:ARENA.SIZE]
39403 00003476 40 inc ax
39404             ; 22/07/2023
39405             ;mov es,ax ; *
39406 00003477 EBD9 jmp short fum10
39407
39408 00003479 07 fumX: pop es
39409             ; 01/01/2023
39410             ;pop dx
39411             ;pop cx
39412             ;pop ax
39413 0000347A C3 retn
39414
39415 ; -----
39416 ; *** UmbTest - returns with carry set if UMBS are not available, else CF==false
39417 ; -----
39418 ; ENTRY: None
39419 ; EXIT: Carry is clear if UMBS are available, or set if they are not
39420 ; ERROR: None
39421 ; USES: CF (AX,BX,DS,ES pushed 'cause they're used by others)
39422 ; -----
39423
39424             ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
39425 UmbTest:
39426             ; 01/01/2023
39427             ;push ax
39428 0000347B 53 push bx ; *
39429             ;push ds
39430 0000347C 06 push es ; **
39431
39432             ; 01/01/2023
39433             ; ds = cs
39434
39435 0000347D E871FB call fm_link ; Link in UMBS (if not already linked)
39436 00003480 E80800 call walkMem ; Check to see if they're really linked
39437 00003483 9C pushf ; And remember what we found out
39438 00003484 E87BFB call fm_unlink ; Unlink UMBS (if WE have linked 'em)
39439 00003487 9D popf ; And restore what we found out.
39440
39441 00003488 07 pop es ; **
39442             ; 01/01/2023
39443             ;pop ds
39444 00003489 5B pop bx ; *
39445             ;pop ax
39446 0000348A C3 retn
39447
39448 ; -----
39449 ; *** walkMem - travels memory chain and returns carry clear iff UMBS are linked
39450 ; -----
39451 ; ENTRY: None
39452 ; EXIT: Carry SET if MCB chain stops before 9FFF, CLEAR if stops >= 9FFF.
39453 ; ERROR: None
39454 ; USES: Flags
39455 ; -----
39456
39457             ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
39458             ; (SYSINIT:3541h)
39459
39460 walkMem:
39461             ;push ax ; ?
39462             ;push bx ; ?
39463             ;push ds ; ? ; 01/01/2023 (MSDOS 6.21 IO.SYS, SYSINIT:352Fh)
39464             ;push es ; ? no need to save contents of these registers ?
39465
39466 0000348B B452 mov ah,GET_IN_VARS ; Call int 21h, function 52h...
39467 0000348D CD21 int 21h
39468
39469 0000348F 268B47FE mov ax,[es:bx-2]
39470             ; 22/07/2023
39471 um10:
39472 00003493 8EC0 mov es,ax ; * ; **
39473
39474 ; -----
39475 ; UM10: ES = Current MCB pointer
39476 ; -----
39477
39478 ;um10:
39479             ;mov al,[es:ARENA.SIGNATURE]
39480             ;cmp al,arena_signature_end ; 'z'
39481             ; 22/07/2023
39482 00003495 26803E00005A cmp byte [es:ARENA.SIGNATURE],arena_signature_end ; 'z'
39483 0000349B 7408 je short um20 ; If signature == 'Z', hay no more.
39484
39485             ;NextMCB es,bx ; Move to the next MCB
39486
39487             ;mov bx,es
39488             ;add bx,[es:3]
39489             ;add bx,[es:ARENA.SIZE]
39490             ;inc bx
39491             ;mov es,bx
39492             ; 22/07/2023
39493             ; ax = es
39494             ;mov ax,es ; *
39495 0000349D 2603060300 add ax,[es:ARENA.SIZE]
39496 000034A2 40 inc ax
39497             ;mov es,ax ; **
39498
39499 000034A3 EBEE jmp short um10 ; And restart the loop.
39500 um20:
39501             ; 22/07/2023
39502             ; ax = es
39503             ;mov ax,es
39504
39505 000034A5 3DFF9F cmp ax,9FFFh ; This sets CF if ax < 9FFF.
39506
39507             ;pop es ; ?
39508             ;pop ds ; ? ; 01/01/2023 (MSDOS 6.21 IO.SYS, SYSINIT:353Dh)
39509             ;pop bx ; ?
39510             ;pop ax ; ?
39511
39512 000034A8 C3 retn
39513
39514 ; -----
39515 ; *** hl_unlink - unlinks UMBS if fm_umb is set to 0; restores strategy too
39516 ; -----
39517 ; ENTRY: fm_umb == 1 : leave linked, else unlink
39518 ; EXIT: None
39519 ; ERROR: None
39520 ; USES: AX, BX
39521 ; -----

```

```

39522
39523
39524
39525 000034A9 30FF
39526
39527
39528
39529
39530
39531
39532
39533
39534
39535
39536
39537
39538 000034AB 8A1E[2E24]
39539
39540 000034AF B80358
39541 000034B2 CD21
39542 000034B4 C3
39543
39544
39545
39546
39547
39548
39549
39550
39551
39552
39553
39554
39555
39556
39557
39558
39559
39560
39561
39562
39563
39564
39565
39566
39567
39568
39569
39570
39571
39572
39573
39574
39575 000034B5 50
39576
39577
39578
39579
39580
39581
39582
39583
39584
39585
39586
39587
39588
39589
39590
39591
39592
39593
39594
39595
39596 000034B6 A0[F923]
39597
39598 000034B9 08C0
39599 000034BB 7503
39600
39601 000034BD 58
39602 000034BE F9
39603 000034BF C3
39604
39605 000034C0 E88E00
39606 000034C3 E81200
39607
39608
39609
39610
39611
39612
39613
39614
39615
39616
39617
39618
39619 000034C6 C606[F923]00
39620
39621
39622
39623
39624 000034CB 30FF
39625
39626
39627
39628 000034CD 8A1E[2E24]
39629
39630 000034D1 B80358
39631 000034D4 CD21
39632
39633
39634 000034D6 58
39635
39636
39637 000034D7 C3
39638
39639
39640
39641
39642
39643
39644
39645

; 01/01/2023 - Retro DOS v4.2
hl_unlink:
xor     bh,bh

;getdata bl,fm_umb          ; Restore original link-state
;
;push    ds
;push    cs
;pop     ds
;mov     bl,[fm_umb]
;pop     ds

; 01/01/2023
; ds = cs
;mov     bl,[cs:fm_umb]
;mov     bl,[fm_umb]

mov     ax,DOS_SET_UMBLINK ; 5803h
int     21h
retn

; -----
; HIGHEXIT.INC (MSDOS 6.0 - 1991)
; -----
; 09/04/2019 - Retro DOS v4.0

; Module:  HIGHEXIT.INC - Code executed after LoadHigh or DeviceHigh
; Date:    May 14, 1992

; Modification log:
;
; DATE      WHO      DESCRIPTION
; -----
; 05/14/92  t-richj  Original
; 06/21/92  t-richj  Final revisions before check-in

UMB_HeadIdx equ     8Ch      ; Offset from ES (after func52h) to get UMBHead

; -----
; *** UnHideUMBs - Marks HIDDEN elements as FREE
; -----
; ENTRY:  None; perhaps, earlier, HideUMBs was called... if not, we have
;         very little to do, as no elements will be marked as HIDDEN.
; EXIT:   Sets InHigh to zero; carry clear if HideUMBs was called earlier.
; ERROR:  None
; USES:   fInHigh (from highvar.inc), carry flag
; -----

; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:357Bh)

UnHideUMBs:
push    ax          ; Save ax for what we're about to do

; -----
; BUGBUG t-richj 11-8-92: The following six lines were commented out for a good
; length of time. Those six constitute a check of whether or not we should
; indeed clean up the upper-memory chain; without such a check, COMMAND.COM
; will destroy the current link-state and memory-allocation strategy after
; every command execution.
; -----

;getdata al,fInHigh   ; Get InHigh from data segment
;
;push    ds
;push    cs
;pop     ds
;mov     al,[fInHigh]
;pop     ds

;mov     al,[cs:fInHigh]
; 31/12/2022
; ds = cs
mov     al,[fInHigh]

or     al,al
jnz     short uhu10    ; If didn't call loadhigh/devicehigh earlier,

pop     ax              ; then there's nothing to do here... so
stc                     ; restore everything and return. Just like
retn                    ; that.

uhu10:
call    linkumb         ; Make sure UMBs are linked in.
call    FreeUMBs

;putdata fInHigh,0      ; we're leaving, so update fInHigh.
;
;push    es
;push    cs
;pop     es
;mov     byte [es:fInHigh],0
;pop     ds

; 31/12/2022
; ds = cs
;mov     byte [cs:fInHigh],0
mov     byte [fInHigh],0

;call    he_unlink      ; Unlink UMBs
; 31/12/2022
;he_unlink:
xor     bh,bh

;getdata bl,fm_umb      ; Restore original link-state
;mov     bl,[cs:fm_umb]
;mov     bl,[fm_umb]

mov     ax,DOS_SET_UMBLINK ; 5803h
int     21h
;retn

pop     ax
; 12/12/2022
;clc      ; 12/12/2022 (this clc may not be necessary!?)
retn

; 31/12/2022
;if 0
;
; -----
; *** he_unlink - unlinks UMBs if fm_umb is set to 0
; -----
; ENTRY:  fm_umb == 1 : leave linked, else unlink

```

```

39646 ; EXIT:      None
39647 ; ERROR:     None
39648 ; USES:      AX, BX
39649 ; -----
39650 ;
39651 ; he_unlink:
39652 ;     xor     bh, bh
39653 ;
39654 ;     ;getdata b1, fm_umb ; Restore original link-state
39655 ;     mov     b1,[cs:fm_umb]
39656 ;
39657 ;     mov     ax,DOS_SET_UMBLINK ; 5803h
39658 ;     int     21h
39659 ;     retn
39660 ;
39661 ; %endif
39662 ;
39663 ; -----
39664 ; *** freeUMBs - frees all HIDDEN memory elements in upper-memory.
39665 ; -----
39666 ; ENTRY:      None
39667 ; EXIT:       None; HIDDEN memory elements returned to FREE
39668 ; ERROR:      None (ignore CF)
39669 ; USES:       Flags
39670 ; -----
39671 ;
39672 ; FreeUMBs:
39673 ;     push    ax
39674 ;     push    es
39675 ;
39676 ;     call    HeadUmb ; Returns with carry if err, else ES == MCB
39677 ;     jc     short fusX
39678 ; fus10:
39679 ;     mov     es,ax ; Prepare for the loop; ES = current MCB addr.
39680 ; ;fus10:
39681 ;     call    isHideMCB ; Returns with ZF set if owner is 0
39682 ;     jnz     short fus20
39683 ;     call    freeMCB
39684 ;     ; 09/09/2023
39685 ;     ; ax <> es
39686 ;     mov     ax,es
39687 ; fus20:
39688 ;     ;mov     al,[es:ARENA.SIGNATURE]
39689 ;     ;cmp     al,arena_signature_end ; 'Z'
39690 ;     ; 22/07/2023
39691 ;     cmp     byte [es:ARENA.SIGNATURE],arena_signature_end ; 'Z'
39692 ;     jz      short fusX ; That means this was the last MCB--that's it.
39693 ;
39694 ;     ; 22/07/2023
39695 ;     ; ax = es
39696 ;     ;mov     ax,es
39697 ;     add     ax,[es:ARENA.SIZE]
39698 ;     inc     ax
39699 ;     ; 22/07/2023
39700 ;     ;mov     es,ax
39701 ;     jmp     short fus10 ; Go on forward.
39702 ; fusX:
39703 ;     pop     es
39704 ;     pop     ax
39705 ;     retn
39706 ;
39707 ; -----
39708 ; *** isHideMCB - returns with ZF set if current MCB (ES:0) is HIDDEN
39709 ; -----
39710 ; ENTRY:      ES:0 should point to an MCB
39711 ; EXIT:       ZF set if MCB is hidden, else !ZF
39712 ; ERROR:      None
39713 ; USES:       Flags
39714 ; -----
39715 ;
39716 ; isHideMCB:
39717 ;     push    ax
39718 ;
39719 ;     cmp     word [es:ARENA.OWNER],SystemPSPowner ; If the owner's SYSTEM
39720 ;     jne     short ihm_x ; then check for HIDDEN
39721 ;
39722 ;     ;mov     ax,[es:ARENA.NAME]
39723 ;     ;cmp     ax,'HI' ; 4948h
39724 ;     cmp     word [es:ARENA.NAME+0],'HI'
39725 ;     jne     short ihm_x
39726 ;     ;mov     ax,[es:ARENA.NAME+2]
39727 ;     ;cmp     ax,'DD' ; 4444h
39728 ;     cmp     word [es:ARENA.NAME+2],'DD'
39729 ;     jne     short ihm_x
39730 ;     ;mov     ax,[es:ARENA.NAME+4]
39731 ;     ;cmp     ax,'EN' ; 4E45h
39732 ;     cmp     word [es:ARENA.NAME+4],'EN'
39733 ;     jne     short ihm_x
39734 ;     ;mov     ax,[es:ARENA.NAME+6]
39735 ;     ;cmp     ax,' ' ; 2020h
39736 ;     cmp     word [es:ARENA.NAME+6],' '
39737 ; ihm_x:
39738 ;     pop     ax
39739 ;     retn
39740 ;
39741 ; -----
39742 ; *** freeMCB - marks as free the MCB at ES:0
39743 ; -----
39744 ; ENTRY:      ES:0 should point to an MCB
39745 ; EXIT:       None; MCB free'd
39746 ; ERROR:      None
39747 ; USES:       AX
39748 ; -----
39749 ;
39750 ; freeMCB:
39751 ;     mov     word [es:ARENA.OWNER],0
39752 ;     mov     ax,' ' ; mov ax,2020h ; 31/12/2022
39753 ;     mov     [es:ARENA.NAME+0],ax
39754 ;     mov     [es:ARENA.NAME+2],ax
39755 ;     mov     [es:ARENA.NAME+4],ax
39756 ;     mov     [es:ARENA.NAME+6],ax
39757 ;     retn
39758 ;
39759 ; -----
39760 ; *** HeadUmb - returns in AX the address of the first UMB block (0x9FFF)
39761 ; -----
39762 ; ENTRY:      Nothing
39763 ; EXIT:       AX contains 0x9FFF for most systems
39764 ; ERROR:      Carry set if pointer is 0xFFFF (if not set up yet--DH runs into this)
39765 ; USES:       Flags, AX
39766 ; -----
39767 ;
39768 ; HeadUmb:
39769 ;     ; 13/05/2019

```

```

39770
39771 ;push si ; ?
39772 ;push ds ; ?
39773 ;push es
39774 ;push bx ; *
39775
39776 ; 09/04/2019
39777 ; !!! No need to save es,bx,ds,si above !!! (es,bx are changed here)
39778
39779 00003544 B452 mov ah,GET_IN_VARS ; Call int 21h, function 52h...
39780 00003546 CD21 int 21h
39781 ; DOS - 2+ internal - GET LIST OF LISTS
39782 ; Return: ES:BX -> DOS list of lists
39783
39784 00003548 26A18C00 mov ax,[es:8Ch]
39785 0000354C 83F8FF mov ax,[es:UMB_HeadIdx] ; And read what's in ES:008C
39786 cmp ax,0FFFFh
39787 ;je short xhu_e ; If it's 0xFFFF, it's an error...
39788 ;clc ; Else, it isn't.
39789 ;jmp short xhu_x
39790 xhu_e:
39791 ;stc
39792 0000354F F5 cmc ; 09/04/2019 - Retro DOS v4.0 ; *
39793 xhu_x:
39794 ;pop bx ; *
39795 ;pop es
39796 ;pop ds ; ?
39797 ;pop si ; ?
39798 00003550 C3 retn
39799
39800 ;-----
39801 ;*** linkumb - links UMBs not already linked in; updates fm_umb as needed
39802 ;-----
39803 ; ENTRY: None
39804 ; EXIT: fm_umb == 0 if not linked in previously, 1 if already linked in
39805 ; ERROR: None
39806 ; USES: AX, BX, fm_umb
39807 ;-----
39808
39809 linkumb:
39810 00003551 B80258 mov ax,DOS_GET_UMBLINK ; 5802h
39811 00003554 CD21 int 21h ; Current link-state is now in al
39812
39813 00003556 08C0 or al,al ; BUGBUG: proper check?
39814 00003558 7508 jnz short lumbx ; Jumps if UMBs already linked in
39815
39816 0000355A B80358 mov ax,DOS_SET_UMBLINK ; 5803h
39817 0000355D BB0100 mov bx,1
39818 00003560 CD21 int 21h
39819 lumbx:
39820 00003562 C3 retn
39821
39822 ;%endif
39823
39824 ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
39825 ; (SYSINIT:2B5Fh)
39826
39827 ;-----
39828 ; SYSCONF.ASM (MSDOS 6.0 - 1991)
39829 ;-----
39830 ; 09/04/2019 - Retro DOS v4.0
39831
39832 ;-----
39833 ;
39834 ; procedure : InitDevLoad
39835 ;
39836 ; Input : DeviceHi = 0 indicates load DD in low memory
39837 ; = 1 indicates load in UMB:
39838 ; ConvLoad = 0 indicates a new-style load (see below)
39839 ; = 1 indicates a DOS 5-style load
39840 ; DevSize = Size of the device driver file in paras
39841 ;
39842 ; Output : none
39843 ;
39844 ; Initializes DevLoadAddr, DevLoadEnd & DevEntry.
39845 ; Also sets up a header for the Device driver entry for mem utility
39846 ;
39847 ;-----
39848 ; For a "new-style load", we break off the current DevEntry and link the umbs
39849 ; as we see fit, using HideUMBs (and UnHideUMBs at exit, though _it_ decides
39850 ; whether it's entitled to do anything). HideUMBs uses the chart built by
39851 ; ParseVar to determine which UMBs to leave FREE, and which not.
39852 ;-----
39853
39854 ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
39855 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
39856 ; (SYSINIT:364Ah)
39857 InitDevLoad:
39858 ; 01/01/2023
39859 ;push es ; *
39860
39861 ; 11/12/2022
39862 ; ds = cs
39863 00003563 803E[4F24]00 cmp byte [DeviceHi],0
39864 ;cmp byte [cs:DeviceHi],0 ; Are we loading in UMB ?
39865 ;je short InitForLo ; no, init for lo mem
39866 00003568 7439 je short initforlo_x ; 09/04/2019
39867
39868 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
39869 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
39870 ;%if 0
39871 ; 01/01/2023
39872 0000356A 803E[3F24]01 cmp byte [ConvLoad],1 ; Are we loading as per DOS 5?
39873 ;cmp byte [cs:ConvLoad],1 ; Are we loading as per DOS 5?
39874 0000356F 7413 je short InitForConv
39875
39876 ; There are two stages to preparing upper-memory; first, we mark as 8+HIDDEN
39877 ; any areas not specified on the /L:... chain. Second, we mark as 8+FROZEN
39878 ; any areas left in upper-memory, except for elements in the load UMB...
39879 ; we then malloc space as per Dos-5 style, and mark as free any spaces which
39880 ; are 8+FROZEN (but leave 8+HIDDEN still hidden). The load is performed,
39881 ; and UnHideUMBs later on marks all 8+HIDDEN as free.
39882
39883 00003571 E85904 call ShrinkUMB ; Stop using the old device arena
39884
39885 00003574 E892FC call HideUMBs ; Mark up the UM area as we see fit
39886 00003577 E8CBFE call FreezeUM ; Hide everything BUT the load area
39887 0000357A E85700 call GetUMBForDev ; And grab that load area as needed
39888 0000357D 9C pushf
39889 0000357E E851FE call UnFreeze ; Then unhide everything frozen
39890 00003581 9D popf
39891 ;jc short InitForLo ; (if carry, it's loading low)
39892 ;jmp short InitForHi
39893 ; 06/07/2023 - Retro DOS v4.2 IO.SYS (optimization)

```

```

39894 00003582 EB0B      jmp     short idl0
39895
39896 ;%endif ; 01/11/2022
39897
39898 ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
39899 ; (SYSINIT:2B67h)
39900 InitForConv:
39901 ; 11/12/2022
39902 ; ds = cs
39903 00003584 E83700      call    SpaceInUMB      ; Do we have space left in the
39904 ; current UMB ?
39905 00003587 7308        jnc     short InitForHi      ; yes, we have
39906 00003589 E84104      call    ShrinkUMB          ; shrink the current UMB in use
39907 0000358C E84500      call    GetUMBForDev         ; else try to allocate new UMB
39908 idl0: ; 06/07/2023
39909 0000358F 720D        jc      short InitForLo      ; we didn't succeed, so load
39910 ; in low memory
39911 InitForHi:
39912 ; 11/12/2022
39913 ; ds = cs
39914 ;mov     ax,[cs:DevUMBFree] ; get Para addr of free mem
39915 ;mov     dx,[cs:DevUMBAddr] ; UMB start addr
39916 ;add     dx,[cs:DevUMBSize] ; DX = UMB End addr
39917 00003591 A1[4524]    mov     ax,[DevUMBFree]
39918 00003594 8B16[4124]  mov     dx,[DevUMBAddr]
39919 00003598 0316[4324]  add     dx,[DevUMBSize]
39920 0000359C EB0C        jmp     short idl1
39921
39922 InitForLo:
39923 ; 11/12/2022
39924 ; ds = cs
39925 ;mov     byte [cs:DeviceHi],0 ; in case we failed to load
39926 0000359E C606[4F24]00 mov     byte [DeviceHi],0
39927 initforlo_x:
39928 ; 11/12/2022
39929 ; ds = cs
39930 ; into UMB indicate that
39931 ; we are loading low
39932 ; AX = start of Low memory
39933 ; DX = End of Low memory
39934 000035A3 A1[6403]    mov     ax,[memhi]
39935 000035A6 8B16[A502]  mov     dx,[ALLOCLIM]
39936 idl1:
39937 000035AA E86600      call    DevSetMark          ; setup a sub-arena for DD
39938 ; 11/12/2022
39939 ; ds = cs
39940 ;mov     [cs:DevLoadAddr],ax ; init the Device load address
39941 ;mov     [cs:DevLoadEnd],dx  ; init the limit of the block
39942 ;mov     word [cs:DevEntry],0 ; init Entry point to DD
39943 ;mov     [cs:DevEntry+2],ax
39944 000035AD A3[3324]    mov     [DevLoadAddr],ax
39945 000035B0 8916[3524]  mov     [DevLoadEnd],dx
39946 000035B4 C706[3724]0000 mov     word [DevEntry],0
39947 000035BA A3[3924]    mov     [DevEntry+2],ax
39948 ; 01/01/2023
39949 ;pop     es ; *
39950 000035BD C3          retn
39951
39952 ;-----
39953 ;
39954 ; procedure : SpaceInUMB?
39955 ;
39956 ; Input : DevUMBAddr, DevUMBSize, DevUMBFree & DevSize
39957 ; Output : Carry set if no space in UMB
39958 ; Carry clear if Space is available for the device in
39959 ; current UMB
39960 ;-----
39961
39962 SpaceInUMB:
39963 ; 11/12/2022
39964 ; ds = cs
39965 ;mov     ax,[cs:DevUMBSize]
39966 ;add     ax,[cs:DevUMBAddr] ; End of UMB
39967 ;sub     ax,[cs:DevUMBFree] ; - Free = Remaining space
39968 mov     ax,[DevUMBSize]
39969 000035BE A1[4324]    add     ax,[DevUMBAddr] ; End of UMB
39970 000035C1 0306[4124]  sub     ax,[DevUMBFree] ; - Free = Remaining space
39971 000035C5 2B06[4524]
39972 ; 11/12/2022
39973 ;or      ax,ax ; Nospace ?
39974 ;jnz     short spcinumb1
39975 ;stc
39976 ;retn
39977 ; 11/12/2022
39978 000035C9 83F801    cmp     ax,1
39979 000035CC 7205        jb      short spcinumb2 ; cf=1
39980 spcinumb1:
39981 000035CE 48          dec     ax ; space for sub-arena
39982 ; 11/12/2022
39983 ; ds = cs
39984 000035CF 3B06[3124]  cmp     ax,[DevSize]
39985 ;cmp     ax,[cs:DevSize] ; do we have space ?
39986 spcinumb2:
39987 000035D3 C3          retn
39988
39989 ;-----
39990 ;
39991 ; procedure : PrepareMark
39992 ;
39993 ; Input : AX==Address of MCB (not addr of free space), BX==Size
39994 ; Output : None; MCB marked appropriately and DevUMB* set as needed.
39995 ;-----
39996
39997 ;
39998 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
39999 ;
40000 ;PrepareMark:
40001 ; push     ds
40002 ; mov      ds,ax
40003 ; mov      word [ARENA.OWNER],8
40004 ; mov      word [ARENA.NAME],'SD' ; 4453h
40005 ; pop      ds
40006 ;
40007 ; inc      ax
40008 ; mov      [cs:DevUMBAddr],ax
40009 ; mov      [cs:DevUMBFree],ax
40010 ; mov      [cs:DevUMBSize],bx ; update the UMB variables
40011 ; retn
40012
40013 ;-----
40014 ;
40015 ; procedure : GetUMBForDev
40016 ;
40017 ; Input : DevSize

```

```

40018 ; Output : Carry set if couldn't allocate a UMB to fit the
40019 ; the device.
40020 ; If success carry clear
40021 ;
40022 ; Allocates the biggest UMB for loading devices and updates
40023 ; DevUMBSize, DevUMBAddr & DevUMBFree if it succeeded in allocating
40024 ; UMB.
40025 ;
40026 ; This routine relies on the fact that all of the low memory
40027 ; is allocated, and any DOS alloc calls should return memory
40028 ; from the UMB pool.
40029 ;
40030 ;-----
40031 ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
40032 ; (SYSINIT:2BC6h)
40033
40034 GetUMBForDev:
40035 ; 11/12/2022
40036 ; ds = cs
40037 000035D4 BBFFFF mov     bx,0FFFFh
40038 000035D7 B80048 mov     ax,4800h
40039 000035DA CD21 int     21h
40040 ; DOS - 2+ - ALLOCATE MEMORY
40041 ; BX = number of 16-byte paragraphs desired
40042
40043 000035DC 09DB or      bx,bx
40044 ;jz     short gufd_err
40045 ; 09/09/2023
40046 000035DE 742E jz      short gufd_error ; bx = 0
40047
40048 000035E0 4B dec     bx
40049 ; 11/12/2022
40050 ; ds = cs
40051 000035E1 391E[3124] cmp     [DevSize],bx
40052 ;cmp    [cs:DevSize],bx
40053 000035E5 7725 ja      short gufd_err
40054
40055 000035E7 43 inc     bx
40056
40057 000035E8 B80048 mov     ax,4800h
40058 000035EB CD21 int     21h
40059 000035ED 721D jc      short gufd_err
40060
40061 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40062 ;dec     ax
40063 ;call    PrepareMark
40064 ;
40065 PrepareMark:
40066 000035EF 1E push    ds
40067 000035F0 48 dec     ax
40068 000035F1 8ED8 mov     ds,ax
40069 000035F3 C70601000800 mov     word [ARENA.OWNER],8
40070 000035F9 C70608005344 mov     word [ARENA.NAME],'SD' ; 4453h
40071 000035FF 40 inc     ax
40072 00003600 1F pop     ds
40073 ; 11/12/2022
40074 ; ds = cs
40075 ;mov     [cs:DevUMBSize],bx ; update the UMB variables
40076 ;mov     [cs:DevUMBAddr],ax
40077 ;mov     [cs:DevUMBFree],ax
40078 gufd_x: ; 09/09/2023
40079 00003601 891E[4324] mov     [DevUMBSize],bx ; update the UMB variables
40080 00003605 A3[4124] mov     [DevUMBAddr],ax
40081 00003608 A3[4524] mov     [DevUMBFree],ax
40082 ;
40083 ; 11/12/2022
40084 ; cf=0
40085 ;clc ; mark no error
40086 0000360B C3 retn
40087
40088 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
40089 %if 1
40090 gufd_err:
40091 0000360C 31DB xor     bx,bx ; 0
40092 gufd_error:
40093 0000360E 31C0 xor     ax,ax ; 0
40094 00003610 F9 stc ; cf=1
40095 00003611 EBEE jmp     short gufd_x
40096 %endif
40097
40098 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
40099 %if 0
40100 gufd_err:
40101 xor     ax,ax ; 0
40102 ; 11/12/2022
40103 ; ds = cs
40104 ;mov     [cs:DevUMBSize],ax ; erase the previous values
40105 ;mov     [cs:DevUMBAddr],ax
40106 ;mov     [cs:DevUMBFree],ax
40107 mov     [DevUMBSize],ax ; erase the previous values
40108 mov     [DevUMBAddr],ax
40109 mov     [DevUMBFree],ax
40110 stc
40111 retn
40112 %endif
40113
40114 ;-----
40115 ;
40116 ; procedure : DevSetMark
40117 ;
40118 ; Input : AX - Free segment where device is going to be loaded
40119 ; Output : AX - Segment at which device can be loaded (AX=AX+1)
40120 ;
40121 ; Creates a sub-arena for the device driver
40122 ; puts 'D' marker in the sub-arena
40123 ; Put the owner of the sub-arena as (AX+1)
40124 ; Copies the file name into sub-arena name field
40125 ;
40126 ; Size field of the sub-arena will be set only at successful
40127 ; completion of Device load.
40128 ;
40129 ;-----
40130
40131 ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
40132 ; (SYSINIT:2C13h)
40133
40134 DevSetMark:
40135 00003613 06 push    es
40136 ; 03/01/2023
40137 ;push    di
40138 00003614 1E push    ds
40139 00003615 56 push    si
40140 00003616 8EC0 mov     es,ax
40141 00003618 26C606000044 mov     byte [es:devmark.id],devmark_device ; 'D'

```



```

40142 0000361E 40          inc     ax
40143 0000361F 26A30100      mov     [es:devmark.seg],ax
40144
40145 ;----- Copy file name
40146
40147 00003623 50          push    ax                ; save load addr
40148
40149 ; 09/09/2023
40150 ; ds = cs
40151 ;lds     si,[cs:bpb_addr]    ; command line is still there
40152 00003624 C536[8103]   lds     si,[bpb_addr]
40153
40154 00003628 89F7          mov     di,si
40155 0000362A FC          cld
40156 dsm_again:
40157 0000362B AC          lodsb
40158 0000362C 3C3A          cmp     al,':'
40159 0000362E 7504          jne     short isit_slash
40160 00003630 89F7          mov     di,si
40161 00003632 EBF7          jmp     short dsm_again
40162 isit_slash:
40163 00003634 3C5C          cmp     al,'\ '
40164 00003636 7504          jne     short isit_null
40165 00003638 89F7          mov     di,si
40166 0000363A EBEF          jmp     short dsm_again
40167 isit_null:
40168 0000363C 08C0          or      al,al
40169 0000363E 75EB          jnz     short dsm_again
40170 00003640 89FE          mov     si,di
40171
40172 00003642 BF0800      mov     di,devmark.filename ; 8
40173 00003645 B90800      mov     cx,8                ; maximum 8 characters
40174 dsm_next_char:
40175 00003648 AC          lodsb
40176 00003649 08C0          or      al,al
40177 0000364B 7407          jz      short blankout
40178 0000364D 3C2E          cmp     al,','
40179 0000364F 7403          je      short blankout
40180 00003651 AA          stosb
40181 00003652 E2F4          loop   dsm_next_char
40182 blankout:
40183 00003654 E304          jcxz    dsm_exit
40184 00003656 B020          mov     al,','
40185 00003658 F3AA          rep     stosb                ; blank out the rest
40186 dsm_exit:
40187 0000365A 58          pop     ax                ; restore load addr
40188 0000365B 5E          pop     si
40189 0000365C 1F          pop     ds
40190 ; 03/01/2023
40191 ;pop     di
40192 0000365D 07          pop     es
40193 0000365E C3          retn
40194
40195 ;-----
40196
40197 ; procedure : SizeDevice
40198
40199 ; Input : ES:SI - points to device file to be sized
40200
40201 ; Output : Carry set if file cannot be opened or if it is an OS2EXE file
40202
40203 ; Calculates the size of the device file in paras and stores it
40204 ; in DevSize
40205 ;-----
40206
40207 ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
40208 SizeDevice:
40209 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40210 ; 11/12/2022 ; *
40211 ; ds ; *
40212 0000365F 1E          push    ds ; *
40213 00003660 06          push    es
40214 00003661 1F          pop     ds
40215 00003662 89F2          mov     dx,si                ; ds:dx -> file name
40216 00003664 B8003D      mov     ax,3D00h              ; open
40217 00003667 CD21          int     21h
40218 00003669 7237          jc      short sd_err         ; open failed
40219
40220 0000366B 89C3          mov     bx,ax                ; BX - file handle
40221 0000366D B80242      mov     ax,4202h              ; seek
40222 00003670 31C9          xor     cx,cx
40223 00003672 89CA          mov     dx,cx                ; to end of file
40224 00003674 CD21          int     21h
40225 00003676 7223          jc      short sd_close       ; did seek fail (impossible)
40226 00003678 83C00F      add     ax,15                 ; para convert
40227 0000367B 83D200      adc     dx,0
40228 0000367E F7C2F0FF    test    dx,0FFFFh            ; size > 0ffffh paras ?
40229 ;jz      short szdev1         ; no
40230 ; 22/07/2023
40231 00003682 7409          jz      short sd_ctp
40232 00003684 2EC706[3124]FFFF mov     word [cs:DevSize],0FFFFh ; invalid device size
40233 ; assuming that we fail later
40234 0000368B EB0E          jmp     short sd_close
40235 sd_ctp:
40236 ; 22/07/2023
40237 ;szdev1:
40238 0000368D B104          mov     cl,4                  ; convert it to paras
40239 0000368F D3E8          shr     ax,cl
40240 00003691 B10C          mov     cl,12
40241 00003693 D3E2          shl     dx,cl
40242 00003695 09D0          or      ax,dx ; * ; cf=0
40243 ;
40244 ; 22/07/2023 - Retro DOS v4.2 IO.SYS (optimized)
40245 ; MSDOS 6.21 IO.SYS - SYSINIT:37A6h
40246 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40247 ;cmp     ax,[cs:DevSizeOption]
40248 ;ja      short szdev2
40249 ;mov     ax,[cs:DevSizeOption]
40250 ; 12/12/2022
40251 ;clc
40252 ;szdev2:
40253 00003697 2EA3[3124]   mov     [cs:DevSize],ax        ; save file size (in paragraphs)
40254 ; 22/07/2023
40255 ;clc ; cf=0 ; *                ; CLC is not needed here
40256 ; (OR instruction clears CF) - E.TAN 22/07/2023
40257 ;
40258 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40259 ; 12/12/2022
40260 ; cf=0
40261 ;clc
40262 sd_close:
40263 0000369B 9C          pushf                          ; let close not spoil our
40264 ; carry flag
40265 0000369C B8003E      mov     ax,3E00h              ; close

```

```

40266 0000369F CD21      int     21h                ; we are not checking for err
40267 000036A1 9D        popf
40268                                sd_err:
40269                        ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40270                        ; 11/12/2022 ; *
40271 000036A2 1F        pop     ds ; *
40272 000036A3 C3        retn
40273
40274
40275
40276
40277
40278
40279
40280
40281
40282
40283
40284
40285
40286
40287
40288
40289
40290 000036A4 2E8B1E[3324]
40291 000036A9 2E891E[4B24]
40292
40293
40294 000036AE 2E891E[4D24]
40295 000036B3 8CCB
40296 000036B5 8EC3
40297 000036B7 BB[4B24]
40298
40299
40300
40301 000036BA B8034B
40302 000036BD CD21
40303
40304
40305
40306
40307 000036BF C3
40308
40309
40310
40311
40312
40313
40314
40315
40316
40317
40318
40319
40320
40321
40322
40323
40324
40325
40326
40327
40328
40329
40330 000036C0 9C
40331
40332 000036C1 C606[3F24]01
40333 000036C6 E8ECFD
40334 000036C9 7204
40335
40336
40337
40338
40339
40340 000036CB FE0E[3F24]
40341
40342 000036CF 9D
40343 000036D0 C3
40344
40345
40346
40347
40348
40349
40350
40351
40352
40353
40354
40355
40356
40357
40358
40359
40360
40361
40362
40363
40364
40365 000036D1 268A1C
40366 000036D4 08DB
40367 000036D6 7403
40368 000036D8 46
40369 000036D9 EBF6
40370
40371
40372 000036DB 8A1E[6424]
40373
40374 000036DF 26881C
40375
40376
40377
40378 000036E2 C3
40379
40380
40381
40382
40383
40384
40385
40386
40387
40388
40389

;-----
;
; procedure : ExecDev
;
; Input : ds:dx -> device to be executed
;         DevLoadAddr - contains where device has to be loaded
;
; Output : Carry if error
;         Carry clear if no error
;
; Loads a device driver using the 4b03h function call
;-----
; 01/11/2022
ExecDev:
    mov     bx,[cs:DevLoadAddr]
    mov     [cs:DevExecAddr],bx    ; Load the parameter block
                                        ; block for exec with
                                        ; load address
    mov     [cs:DevExecReloc],bx
    mov     bx,cs
    mov     es,bx
    mov     bx,DevExecAddr    ; es:bx points to parameters
    ;mov     al,3    ; (load program only)
    ;mov     ah,EXEC ; 4Bh
    ; 04/07/2023
    mov     ax,(EXEC<<8)|03h
    int     21h                ; load in the device driver
                                ; DOS - 2+ - LOAD OR EXECUTE (EXEC)
                                ; DS:DX -> ASCIZ filename
                                ; ES:BX -> parameter block
                                ; AL = subfunction
    retn

;-----
;
; procedure : RetFromUM
;
; Input : None
; Output : ConvLoad set if didn't previously call HideUMBs
;         ConvLoad clear if did.
;
; Prepares memory for more devices after returning from loading one
; using the DOS 6 options (/L:... etc).
;-----
; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:37D1h)
; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; %if 0
RetFromUM:
    ; 31/12/2022
    ; ds = cs
    pushf
    mov     byte [cs:ConvLoad],1
    mov     byte [ConvLoad],1
    call    unHideUMBs
    jc      short rfUM1    ; skip this if didn't HideUMBs
    ; 31/12/2022
    ; ds = cs
    ; mov     byte [cs:ConvLoad],0
    ; mov     byte [ConvLoad],0
    ; 09/09/2023
    dec     byte [ConvLoad] ; -> 0
rfUM1:
    popf
    retn
; %endif ; 01/11/2022
;-----
;
; procedure : RemoveNull
;
; Input : ES:SI points to a null terminated string
;
; Output : none
;
; Replaces the null at the end of a string with blank
;-----
; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
; (SYSINIT:2CCEh)
RemoveNull:
    ; 11/12/2022
    ; ds = cs
rn_next:
    mov     bl,[es:si]
    or      bl,bl    ; null ?
    jz      short rn_gotnull
    inc     si    ; advance the pointer
    jmp     short rn_next
rn_gotnull:
    ; 11/12/2022
    mov     bl,[DevSavedDelim]
    ;mov     bl,[cs:DevSavedDelim]
    mov     [es:si],bl    ; replace null with blank
    ; 02/11/2022
; 11/12/2022
rba_ok:
    ; 10/04/2019
    retn
;-----
;
; procedure : RoundBreakAddr
;
; Input : DevBrkAddr
; Output : DevBrkAddr
;
; Rounds DevBrkAddr to a para address so that it is of the form xxxx:0
;-----

```



```

40390
40391
40392 000036E3 2EA1[3B24]
40393 000036E7 E82BDC
40394 000036EA 2E0106[3D24]
40395 000036EF 2EC706[3B24]0000
40396 000036F6 2EA1[3524]
40397 000036FA 2E3906[3D24]
40398 000036FF 76E1
40399 00003701 E92911
40400
40401
40402
40403
40404
40405
40406
40407
40408
40409
40410
40411
40412
40413
40414
40415
40416 00003704 50
40417
40418 00003705 2EA1[3D24]
40419 00003709 2E803E[6619]00
40420 0000370F 750F
40421 00003711 2E3B06[3324]
40422 00003716 7508
40423
40424
40425
40426
40427 00003718 2E833E[3B24]01
40428 0000371E 7204
40429
40430
40431 00003720 E8C0FF
40432
40433 00003723 F8
40434
40435 00003724 58
40436
40437 00003725 C3
40438
40439
40440
40441
40442
40443
40444
40445
40446
40447
40448
40449
40450
40451
40452
40453
40454
40455
40456
40457
40458
40459
40460
40461
40462
40463 00003726 0E
40464 00003727 1F
40465
40466
40467 00003728 A1[3324]
40468 0000372B 8B1E[3D24]
40469
40470 0000372F 1E
40471
40472 00003730 48
40473 00003731 8ED8
40474 00003733 40
40475 00003734 29D8
40476 00003736 F7D8
40477 00003738 A30300
40478
40479
40480 0000373B 1F
40481
40482
40483 0000373C 803E[4F24]00
40484
40485 00003741 7405
40486
40487
40488
40489 00003743 891E[4524]
40490 00003747 C3
40491
40492
40493
40494
40495
40496 00003748 891E[6403]
40497 0000374C C706[6203]0000
40498
40499
40500
40501 00003752 C3
40502
40503
40504
40505
40506
40507
40508
40509
40510
40511
40512
40513

RoundBreakAddr:
    mov     ax,[cs:DevBrkAddr]
    call    ParaRound
    add     [cs:DevBrkAddr+2],ax
    mov     word [cs:DevBrkAddr],0
    mov     ax,[cs:DevLoadEnd]
    cmp     [cs:DevBrkAddr+2],ax
    jbe     short rba_ok
    jmp     mem_err
    ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
    ; 11/12/2022
; rba_ok:
;     retn

;-----
;
; procedure : DevSetBreak
;
; Input : DevBrkAddr
; Output : Carry set if Device returned Init failed
;         Else carry clear
;-----

DevSetBreak:
    push    ax
    mov     ax,[cs:DevBrkAddr+2] ;remove the init code
    cmp     byte [cs:multdeviceflag],0
    jne     short set_break_continue ;do not check it.
    cmp     ax,[cs:DevLoadAddr]
    jne     short set_break_continue ;if not same, then o.k.

    ;cmp     word [cs:DevBrkAddr],0
    ;je      short break_failed ;[DevBrkAddr+2]=[memhi] & [DevBrkAddr]=0
    ; 12/12/2022
    cmp     word [cs:DevBrkAddr],1
    jb      short break_failed

set_break_continue:
    call    RoundBreakAddr
    ; 12/12/2022
    cld
break_failed:
    pop     ax
    ;cld
    retn

    ; 12/12/2022
; break_failed:
; pop     ax
; stc
; retn

;-----
;
; procedure : DevBreak
;
; Input : DevLoadAddr & DevBrkAddr
; Output : none
;
; Marks a succesful install of a device driver
; Sets device size field in sub-arena &
; Updates Free ptr in UMB or adjusts memhi
;-----

; 11/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
DevBreak:
; push    ds ; 11/12/2022

    ; 11/12/2022
    push    cs
    pop     ds
    ;mov     ax,[cs:DevLoadAddr]
    ;mov     bx,[cs:DevBrkAddr+2]
    mov     ax,[DevLoadAddr]
    mov     bx,[DevBrkAddr+2]
    ; 11/12/2022
    push    ds

    dec     ax ; seg of sub-arena
    mov     ds,ax
    inc     ax ; Back to Device segment
    sub     ax,bx
    neg     ax ; size of device in paras
    mov     [devmark.size],ax ; store it in sub-arena

    ; 11/12/2022
    pop     ds
    ; ds = cs

    cmp     byte [DeviceHi],0
    ;cmp     byte [cs:DeviceHi],0
    je      short db_lo
    ;mov     [cs:DevUMBFree],bx ; update Free ptr in UMB
    ;jmp     short db_exit
    ; 11/12/2022
    mov     [DevUMBFree],bx
    retn
db_lo:
    ; 11/12/2022
    ; ds = cs
    ;mov     [cs:memhi],bx
    ;mov     word [cs:memlo],0
    mov     [memhi],bx
    mov     word [memlo],0 ; 18/12/2022
db_exit:
    ;pop     ds ; 11/12/2022
sd_ret:
    ; 09/09/2023
    retn

; 10/04/2019 - Retro DOS v4.0

;-----
;
; procedure : ParseSize
;
; Parses the command line for SIZE= command
;
; ES:SI = command line to parsed
;
; returns ptr to command line after SIZE= option in ES:SI
;-----

```

```

40514 ; updates the DevSizeOption variable with value supplied
40515 ; in SIZE=option
40516 ; Returns carry if the SIZE option was invalid
40517 ;
40518 ;-----
40519
40520 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
40521 ; (SYSINIT:2D5Ah)
40522
40523 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization) ((&BugFix))
40524 ; (MSDOS 6.21 IO.SYS - SYSINIT:3871h) - Retro DOS v4.2 -
40525 ; (PCDOS 7.1 IO.SYS - SYSINIT:3D6Eh) - Retro DOS v5.0 -
40526 ParseSize:
40527 ;push bx
40528 ;mov bx,si
40529
40530 ; 09/09/2023
40531 00003753 56 push si ; * ; mov bx,si
40532
40533 ; 11/12/2022
40534 ; ds = cs
40535 ;mov word [cs:DevSizeOption],0 ; init the value
40536 ;mov [cs:DevCmdLine],si
40537 ;mov [cs:DevCmdLine+2],es
40538 00003754 C706[5024]0000 mov word [DevSizeOption],0 ; init the value
40539 0000375A 8936[6024] mov [DevCmdLine],si
40540 0000375E 8C06[6224] mov [DevCmdLine+2],es
40541 00003762 E82400 call SkipDelim
40542 00003765 26813C5349 cmp word [es:si],'SI' ; 4953h
40543 0000376A 7528 jne short ps_no_size
40544 0000376C 26817C025A45 cmp word [es:si+2],'ZE' ; 455Ah
40545 00003772 7520 jne short ps_no_size
40546 00003774 268A4404 mov al,[es:si+4]
40547 00003778 E80D10 call delim
40548 ;jne short ps_no_size
40549 ; 22/07/2023
40550 0000377B 7518 jne short ps_no_size_2 ; cf=0 here
40551 0000377D 83C605 add si,5
40552 00003780 E81400 call GetHexNum
40553 00003783 7210 jc short ps_err
40554 ; 11/12/2022
40555 ; ds = cs
40556 ;mov [cs:DevSizeOption],ax
40557 00003785 A3[5024] mov [DevSizeOption],ax
40558
40559 ; 09/09/2023
40560 00003788 58 pop ax ; * (discard previous si value on top of stack)
40561
40562 ; call SkipDelim ; **
40563 ;
40564 ; 22/07/2023
40565 ;ps_no_size_2:
40566 ; cf = 0
40567 ; retn
40568
40569 ; 09/09/2023
40570 ;jmp short SkipDelim
40571
40572 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
40573 %if 1
40574 ; 01/11/2022
40575 SkipDelim:
40576 sd_next_char:
40577 00003789 268A04 mov al,[es:si]
40578 0000378C E8F90F call delim
40579 0000378F 75C1 jnz short sd_ret ; cf=0 ; 09/09/2023
40580 00003791 46 inc si
40581 00003792 EBF5 jmp short sd_next_char ; 01/11/2022
40582 ; 11/12/2022
40583 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40584 ;sd_ret:
40585 ;retn
40586 %endif
40587
40588 ;;;call SkipDelim ; **
40589 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40590 ;mov bx,si
40591 ps_no_size:
40592 ;mov si,bx
40593 ;pop bx
40594 00003794 F8 clc ; cf=0
40595 ;retn
40596 ; 11/12/2022
40597 ps_err: ; cf=1
40598 ps_no_size_2: ; 09/09/2023 (cf=0)
40599 ; 09/09/2023
40600 00003795 5E pop si ; * ; mov si,bx
40601 ;sd_ret: ; cf=?
40602 00003796 C3 retn
40603
40604 ;ps_err:
40605 ; 02/11/2022
40606 ;pop bx
40607 ;stc
40608 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40609 ; 11/12/2022
40610 ; cf=1
40611 ;stc
40612 ; 11/12/2022
40613 ;sd_ret:
40614 ; 22/07/2023
40615 ; 12/04/2019
40616 ;retn
40617
40618 ; 12/04/2019 - Retro DOS v4.0
40619
40620 ;-----
40621 ;
40622 ; procedure : SkipDelim
40623 ;
40624 ; Skips delimiters in the string pointed to by ES:SI
40625 ; Returns ptr to first non-delimiter character in ES:SI
40626 ;
40627 ;-----
40628
40629 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
40630 %if 0
40631 ; 01/11/2022
40632 SkipDelim:
40633 sd_next_char:
40634 mov al,[es:si]
40635 call delim
40636 jnz short sd_ret
40637 inc si

```

```

40638         jmp     short sd_next_char ; 01/11/2022
40639         ; 11/12/2022
40640         ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40641 ;sd_ret:
40642         ;retn
40643 %endif
40644
40645 ;-----
40646 ;
40647 ; procedure : GetHexNum
40648 ;
40649 ; Converts an ascii string terminated by a delimiter into binary.
40650 ; Assumes that the ES:SI points to a Hexadecimal string
40651 ;
40652 ; Returns in AX the number number of paras equivalent to the
40653 ; hex number of bytes specified by the hexadecimal string.
40654 ;
40655 ; Returns carry in case it encountered a non-hex character or
40656 ; if it encountered crlf
40657 ;
40658 ;-----
40659
40660 ; 13/05/2019
40661
40662         ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
40663         ; (SYSINIT:38C5h)
40664
40665         ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
40666         ; (SYSINIT:2DA5h)
40667 GetHexNum:
40668     xor     ax,ax
40669     xor     dx,dx
40670 ghn_next:
40671     mov     bl,[es:si]
40672     cmp     bl,cr ; 0Dh
40673     je      short ghn_err
40674     cmp     bl,lf ; 0Ah
40675     je      short ghn_err
40676     push    ax
40677     mov     al,bl
40678     call    delim
40679     pop     ax
40680     ; 03/01/2023
40681     mov     cx,4
40682     jz      short ghn_into_paras
40683     call    GetNibble
40684     ;jc     short ghn_err
40685     ; 11/12/2022
40686     jc      short ghn_ret ; cf=1
40687     ; 03/01/2023
40688     ;mov    cx,4
40689 ghn_shift1:
40690     shl     ax,1
40691     rcl     dx,1
40692     loop    ghn_shift1
40693     or      al,bl
40694     inc     si
40695     jmp     short ghn_next
40696 ghn_into_paras:
40697     add     ax,15
40698     adc     dx,0
40699     test    dx,0FFF0h
40700     jnz     short ghn_err
40701     ; 03/01/2023
40702     ;mov    cx,4
40703 ghn_shift2:
40704     clc
40705     rcr     dx,1
40706     rcr     ax,1
40707     loop    ghn_shift2
40708     clc
40709     retn
40710     ; 11/12/2022
40711 ghn_err:
40712 gnib_err:
40713     stc
40714 ghn_ret:
40715 gnib_ret:
40716     retn
40717
40718 ;-----
40719 ;
40720 ; procedure : GetNibble
40721 ;
40722 ; Convert one nibble (hex digit) in BL into binary
40723 ;
40724 ; Returns binary value in BL
40725 ;
40726 ; Returns carry if BL contains non-hex digit
40727 ;
40728 ;-----
40729
40730 GetNibble:
40731     cmp     bl,'0'
40732     ;jb     short gnib_err
40733     ; 11/12/2022
40734     jb      short gnib_ret ; cf=1
40735     cmp     bl,'9'
40736     ja      short is_it_hex
40737     sub     bl,'0' ; clc
40738     retn
40739 is_it_hex:
40740     cmp     bl,'A'
40741     ;jb     short gnib_err
40742     ; 11/12/2022
40743     jb      short gnib_ret ; cf=1
40744     cmp     bl,'F'
40745     ja      short gnib_err ; 11/12/2022
40746     sub     bl,'A'-10 ; clc
40747     retn
40748
40749     ; 11/12/2022
40750 ;gnib_err:
40751 ;     stc
40752 ;gnib_ret:
40753 ;     retn
40754 ;
40755 ;=====
40756
40757 ; 12/04/2019 - Retro DOS v4.0
40758
40759 ; umb.inc (MSDOS 6.0, 1991)
40760 DOS_ARENA equ 24h ; offset of arena_head var in DOS data segm.
40761 UMB_ARENA equ 8Ch ; offset of umb_head in DOS data

```

```

40762
40763 XMM_REQUEST_UMB equ 10h
40764 XMM_RELEASE_UMB equ 11h
40765
40766 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
40767
40768 ;-----
40769 ;
40770 ; Procedure Name : umb_insert
40771 ;
40772 ; Inputs : DOSDATA:UMB_HEAD = start of umb chain
40773 ; : BX = seg address of UMB to be linked in
40774 ; : DX = size of UMB to be linked in paras
40775 ; : DS = data
40776 ;
40777 ; Outputs : links the UMB into the arena chain
40778 ;
40779 ; Uses : AX, CX, ES, DX, BX
40780 ;-----
40781
40782 umb_insert:
40783 push ds
40784 000037F7 1E ; 31/12/2022
40785 ; ds = cs
40786
40787 ;mov ds,[cs:DevDOSData]
40788 mov ds,[DevDOSData] ; 31/12/2022
40789 000037F8 8E1E[5E24] ;mov ds,[8Ch]
40790 mov ds,[UMB_ARENA] ; es = UMB_HEAD
40791 000037FC 8E1E8C00 mov ax,ds
40792 00003800 8CD8 mov es,ax
40793 00003802 8EC0 ui_next:
40794 00003804 39D8 cmp ax,bx ; Q: is current block above
40795 00003806 770F ja short ui_insert ; new block
40796 ; Y: insert it
40797 ; Q: is current block the
40798 ; last
40799 00003808 26803E00005A cmp byte [es:ARENA.SIGNATURE],arena_signature_end ; 'z'
40800 0000380E 745C je short ui_append ; Y: append new block to chain
40801 ; N: get next block
40802 00003810 8ED8 mov ds,ax ; M005
40803 ;call get_next ; ax = es = next block
40804 00003812 E83B01 call _get_next_ ; 13/04/2019 - Retro DOS v4.0
40805 00003815 EBED jmp short ui_next
40806
40807 ui_insert:
40808 mov cx,ds ; ds = previous arena
40809 inc cx ; top of previous block
40810
40811 sub cx,bx
40812 neg cx ; cx = size of used block
40813 ;mov byte [0],'M'
40814 0000381A 29D9 mov byte [ARENA.SIGNATURE],arena_signature_normal ; 'M'
40815 0000381C F7D9 ;mov word [1],8
40816 0000381E C60600004D mov word [ARENA.OWNER],8 ; mark as system owned
40817 00003823 C70601000800 ;mov [3],cx
40818 00003825 890E0300 mov [ARENA.SIZE],cx
40819 00003827 C70608005343 ;mov word [8],4353h ; 'SC'
40820 00003829 890E0300 mov word [ARENA.NAME],'SC' ; 4353h
40821 0000382B C70608005343
40822
40823 ; prepare the arena at start of new block
40824
40825 mov es,bx
40826 00003833 8EC3 mov byte [es:ARENA.SIGNATURE],arena_signature_normal ; 'M'
40827 00003835 26C60600004D mov word [es:ARENA.OWNER],arena_owner_system ; 0
40828 0000383B 26C70601000000 ; mark as free
40829 0000383D 83EA02 sub dx,2 ; make room for arena at
40830 0000383F 2689160300 mov [es:ARENA.SIZE],dx ; start & end of new block
40831
40832 ; prepare arena at end of new block
40833
40834 add bx,dx
40835 inc bx
40836 0000384A 01D3 mov es,bx ; es=arena at top of new block
40837 0000384C 43 inc bx ; bx=top of new block
40838 0000384E 8EC3
40839 0000384F 43
40840
40841 ; ax contains arena just above
40842 ; this block
40843 00003850 29D8 sub ax,bx ; ax = size of used block
40844
40845 mov byte [es:ARENA.SIGNATURE],arena_signature_normal
40846 00003852 26C60600004D mov word [es:ARENA.OWNER],8 ; mark as system owned
40847 00003854 26C70601000800 mov [es:ARENA.SIZE],ax
40848 00003856 26A30300 mov word [es:ARENA.NAME],'SC' ; 4353h
40849 00003858 26C70608005343
40850 0000385A EB47 jmp short ui_done
40851
40852 ui_append:
40853 add ax,[es:ARENA.SIZE] ; es = arena of last block
40854 0000386C 2603060300 ; ax=top of last block-1 para
40855 00003871 26832E030001 sub word [es:ARENA.SIZE],1 ; reflect the space we are
40856 ; going to rsrv on top of this
40857 ; block for the next arena.
40858 ; 13/05/2019
40859 00003877 26C60600004D mov byte [es:ARENA.SIGNATURE],arena_signature_normal
40860
40861 0000387D 89C1 mov cx,ax ; cx=top of prev block-1
40862 0000387F 40 inc ax
40863 00003880 29D8 sub ax,bx ; ax=top of prev block -
40864 ; seg. address of new block
40865 00003882 F7D8 neg ax
40866
40867 00003884 8EC1 mov es,cx ; ds = arena of unused block
40868
40869 mov byte [es:ARENA.SIGNATURE],arena_signature_normal
40870 0000388C 26C70601000800 mov word [es:ARENA.OWNER],8 ; mark as system owned
40871 00003893 26A30300 mov [es:ARENA.SIZE],ax
40872 00003897 26C70608005343 mov word [es:ARENA.NAME],'SC'
40873
40874 ; prepare the arena at start of new block
40875 0000389E 8EC3 mov es,bx
40876 000038A0 26C60600005A mov byte [es:ARENA.SIGNATURE],arena_signature_end
40877 000038A6 26C70601000000 mov word [es:ARENA.OWNER],arena_owner_system
40878 ; mark as free
40879 000038AD 4A dec dx ; make room for arena
40880 000038AE 2689160300 mov [es:ARENA.SIZE],dx
40881 ui_done:
40882 uc_done: ; 31/12/2022 ; *!
40883 000038B3 1F pop ds
40884 ; ds = cs ; 31/12/2022
40885 ;uc_done: ; 18/12/2022

```

```

40886 au_exit:      ; 09/09/2023
40887 000038B4 C3      retn
40888
40889
40890
40891
40892
40893
40894
40895
40896
40897
40898
40899
40900 000038B5 E84700      call    InitAllocUMB      ; link in the first UMB
40901 000038B8 72FA      jc      short au_exit      ; quit on error
40902
40903 000038BA E87000      call    umb_allocate      ; allocate
40904 000038BD 7205      jc      short au_coalesce
40905 000038BF E835FF      call    umb_insert      ; & insert till no UMBs
40906 000038C2 EBF6      jmp     short au_next
40907
40908 au_coalesce:
40909      ; 09/09/2023
40910      ; call    umb_coalesce      ; coalesce all UMBs
40911      ; au_exit:
40912      ;      ; 31/12/2022
40913      ;      ; ds = cs
40914      ;      retn
40915
40916      ; 09/09/2023
40917      ; jmp     short umb_coalesce
40918
40919 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
40920
40921 ; 13/04/2019 - Retro DOS v4.0
40922
40923
40924
40925
40926
40927
40928
40929
40930
40931
40932
40933
40934
40935
40936
40937
40938
40939
40940
40941
40942 000038C4 1E      push    ds ; *!
40943
40944 000038C5 31FF      xor     di, di
40945
40946
40947
40948 000038C7 8E06[5E24]      mov     es,[cs:DevDOSData]
40949 000038CB 268E068C00      ; 31/12/2022
40950      mov     es,[DevDOSData]
40951      mov     es,[es:UMB_arena]      ; es = UMB_HEAD
40952
40953 uc_nextfree:
40954      mov     ax,es
40955      mov     ds,ax
40956      ; cmp     [es:1],di
40957      ; cmp     [es:arena.owner],di      ; Q: is current arena free
40958      ; je      short uc_again      ; Y: try to coalesce with next block
40959      ;      ; N: get next arena
40960      ;      ; es, ax = next arena
40961      call    get_next
40962      jc      short uc_done ; *!
40963      jmp     short uc_nextfree
40964
40965 uc_again:
40966      call    get_next
40967      jc      short uc_done ; *!
40968      ; es, ax = next arena
40969
40970 uc_check:
40971      cmp     [es:arena.owner],di      ; Q: is arena free
40972      ; jne     short uc_nextfree      ; N: get next free arena
40973      ;      ; Y: coalesce
40974      mov     cx,[es:arena.size]      ; cx <- next block size
40975      ; inc     cx      ; cx <- cx + 1 (for header size)
40976      ; add     [3],cx
40977      ; add     [arena.size],cx      ; current size <- current size + cx
40978      mov     cl,[es:di]      ; move up signature
40979      mov     [di],cl
40980      jmp     short uc_again      ; try again
40981
40982      ; 18/12/2022
40983 ;uc_done:
40984 ;retn
40985
40986
40987
40988
40989
40990
40991
40992
40993
40994
40995
40996
40997
40998
40999
41000
41001
41002
41003
41004
41005
41006
41007
41008
41009

```

```

41010          ; 12/12/2022
41011 00003923 7207      jc      short iau_err2 ; cf=1
41012          ; 31/12/2022
41013          ; ds = cs
41014 00003925 C606[5D24]FF      mov     byte [FirstUMBLinked],0FFh ; mark that 1st UMB linked
41015          ;mov     byte [cs:FirstUMBLinked],0FFh ; mark that 1st UMB linked
41016 ia_1:
41017          ; 12/12/2022
41018          ; cf=0
41019          ;clc
41020 0000392A C3          retn
41021 iau_err:
41022 0000392B F9          stc
41023 iau_err2:
41024 0000392C C3          retn
41025
41026 ;-----
41027 ;
41028 ; Procedure Name      : umb_allocate
41029 ;
41030 ; Inputs              : DS = data
41031 ;
41032 ; Outputs             : if UMB available
41033 ;                      Allocates the largest available UMB and
41034 ;                      BX = segment of allocated block
41035 ;                      DX = size of allocated block
41036 ;                      NC
41037 ;                      else
41038 ;                      CY
41039 ;
41040 ; Uses                : BX, DX
41041 ;
41042 ;-----
41043
41044 umb_allocate:
41045          ; 31/12/2022
41046          ; ds = cs
41047 0000392D 50          push     ax
41048 0000392E B410      mov     ah,XMM_REQUEST_UMB ; 16
41049 00003930 BAFfff    mov     dx,0FFFFh ; try to allocate largest
41050          ;                      ; possible
41051          ; 31/12/2022
41052 00003933 FF1E[4724] call     far [DevXMSAddr]
41053          ;call     far [cs:DevXMSAddr]
41054          ; dx now contains the size of
41055          ; the largest UMB
41056          or     dx,dx
41057 00003939 740B      jz      short ua_err
41058
41059 0000393B B410      mov     ah,XMM_REQUEST_UMB ; 16
41060
41061          ; 31/12/2022
41062 0000393D FF1E[4724] call     far [DevXMSAddr]
41063          ;call     far [cs:DevXMSAddr]
41064
41065 00003941 83F801     cmp     ax,1 ; Q: was the reqst successful
41066          ;jne     short ua_err ; N: error
41067          ; 27/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
41068 00003944 7601     jna     short ua_done ; if ax=1 then cf=0, else cf=1 (ax=0)
41069 ua_err:
41070 00003946 F9          stc
41071
41072          ;clc
41073          ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41074          ; 12/12/2022
41075          ; cf=0
41076          ;clc
41077 ua_done:
41078 00003947 58          pop     ax
41079 00003948 C3          retn
41080          ; 27/07/2023
41081 ;ua_err:
41082          ;stc
41083          ;jmp     short ua_done
41084
41085 ;-----
41086 ;
41087 ;** get_next - Find Next item in Arena
41088 ;
41089 ; ENTRY  ds - pointer to block head
41090 ; EXIT   AX,ES - pointers to next head
41091 ; 'C' set if arena damaged
41092 ;
41093 ;-----
41094
41095          ; 01/11/2022
41096 get_next:
41097 00003949 803E00005A cmp     byte [0],arena_signature_end ; 'z'
41098 0000394E 740A      je      short gn_err
41099 _get_next_:
41100          mov     ax,ds ; ax=current block
41101          add     ax,[ARENA.SIZE] ; ax=ax + current block length
41102          inc     ax ; remember that header!
41103          mov     es,ax
41104          ;clc
41105          ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41106          ; 11/12/2022
41107          ; cf=0
41108          ;clc
41109 00003959 C3          retn
41110 gn_err:
41111          stc
41112          ; 11/12/2022
41113 0000395B C3          lfu_err: ; cf=1
41114          retn
41115
41116 ;-----
41117 ;
41118 ; procedure : LinkFirstUMB
41119 ;
41120 ;-----
41121
41122          ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
41123          ; (SYSINIT:2F81h)
41124 LinkFirstUMB:
41125          ; 31/12/2022
41126          ; ds = cs
41127 0000395C E8CEFF    call     umb_allocate
41128 0000395F 72FA      jc      short lfu_err ; ds = cs ; 31/12/2022
41129
41130          ; bx = segment of allocated UMB
41131          ; dx = size of UMB
41132
41133          ; 31/12/2022

```



```

41134             ; ds = cs
41135
41136             int     12h                ; ax = size of memory
41137             mov     cx,6
41138             shl     ax,cx              ; ax = size in paragraphs
41139
41140             mov     cx,ax
41141             sub     ax,bx              ; ax = - size of unused block
41142
41143             neg     ax
41144
41145             ;sub     cx,1                ; cx = first umb_arena
41146             ; 09/09/2023
41147             dec     cx
41148             mov     es,cx              ; es = first umb_arena
41149
41150             mov     byte [es:ARENA.SIGNATURE],arena_signature_normal ; 'M'
41151             mov     word [es:ARENA.OWNER],8          ; mark as system owned
41152
41153             mov     [es:ARENA.SIZE],ax
41154             mov     word [es:ARENA.NAME],'sc' ; 4353h
41155
41156             ; put in the arena for the first UMB
41157
41158             mov     es,bx                ; es has first free umb seg
41159             mov     byte [es:ARENA.SIGNATURE],arena_signature_end ; 'Z'
41160             mov     word [es:ARENA.OWNER],arena_owner_system ; 0
41161             ; mark as free
41162             dec     dx                    ; make room for arena
41163             mov     [es:ARENA.SIZE],dx
41164
41165             ;mov     es,[cs:DevDOSData]
41166             ; 31/12/2022
41167             mov     es,[DevDOSData] ; ds = cs
41168             mov     di,UMB_ARENA ; 8Ch
41169             mov     [es:di],cx          ; initialize umb_head in DOS
41170             ; data segment with the arena
41171             ; just below Top of Mem
41172
41173             ; we must now scan the arena chain and update the size of the last arena
41174
41175             mov     di,DOS_ARENA ; 24h
41176             mov     es,[es:di]          ; es = start arena
41177             xor     di,di
41178             ;scan_next
41179             ; 09/12/2022
41180             scannext:
41181             cmp     byte [es:di],arena_signature_end ; 'Z'
41182             je      short got_last
41183
41184             mov     ax,es
41185             add     ax,[es:ARENA.SIZE]
41186             inc     ax
41187             mov     es,ax
41188             jmp     short scan_next
41189             ; 09/12/2022
41190             jmp     short scannext
41191             got_last:
41192             sub     word [es:ARENA.SIZE],1
41193             ; 09/09/2023
41194             dec     word [es:ARENA.SIZE]
41195
41196             mov     byte [es:ARENA.SIGNATURE],arena_signature_normal ; 'M'
41197             ;clc
41198             ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41199             ; 11/12/2022
41200             ; cf=0
41201             ;clc
41202             retn
41203
41204             ; 11/12/2022
41205             ; ;lfu_err:
41206             ; ;stc
41207             ; ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41208             ; ; 11/12/2022
41209             ; ; cf=1
41210             ; ;stc
41211             ; retn
41212
41213             ;-----
41214             ;
41215             ; procedure : ShrinkUMB
41216             ;
41217             ; Shrinks the current UMB in use, so that the unused portions
41218             ; of the UMB is given back to the DOS free mem pool
41219             ;
41220             ;-----
41221
41222             ShrinkUMB:
41223             ; 12/12/2022
41224             ; ds = cs
41225             cmp     word [DevUMBAddr],0
41226             cmp     word [cs:DevUMBAddr],0
41227             je      short su_exit
41228             push     es
41229             ; 01/01/2023
41230             push     bx
41231             ; 12/12/2022
41232             mov     bx,[cs:DevUMBFree]
41233             sub     bx,[cs:DevUMBAddr]
41234             mov     es,[cs:DevUMBAddr]
41235             mov     bx,[DevUMBFree]
41236             sub     bx,[DevUMBAddr]
41237             mov     es,[DevUMBAddr]
41238
41239             mov     ax,4A00h
41240             int     21h
41241             ; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
41242             ; ES = segment address of block to change
41243             ; BX = new size in paragraphs
41244             mov     ax,es
41245             dec     ax
41246             mov     es,ax
41247             mov     word [es:ARENA.OWNER],8
41248             ; 01/01/2023
41249             pop     bx
41250             pop     es
41251             su_exit:
41252             retn
41253
41254             ;-----
41255             ;
41256             ; procedure : UnlinkUMB
41257             ;

```

```

41258 ; Unlinks the UMBs from the DOS arena chain
41259 ;
41260 ;-----
41261
41262 UnlinkUMB:
41263 ; 12/12/2022
41264 ; ds = cs
41265 000039F4 1E push ds
41266 000039F5 06 push es
41267 ; 12/12/2022
41268 000039F6 803E[5D24]00 cmp byte [FirstUMBLinked],0
41269 ;cmp byte [cs:FirstUMBLinked],0
41270 000039FB 7420 je short ulu_x ; nothing to unlink
41271 ; 12/12/2022
41272 000039FD 8E06[5E24] mov es,[DevDOSData]
41273 ;mov es,[cs:DevDOSData] ; get DOS data seg
41274 00003A01 268E1E2400 mov ds,[es:DOS_ARENA]
41275 00003A06 268B3E8C00 mov di,[es:UMB_ARENA]
41276
41277 00003A0B E83BFF ulu_next: call get_next
41278 00003A0E 720D jc short ulu_x
41279 00003A10 39C7 cmp di,ax ; is the next one UMB ?
41280 00003A12 7404 je short ulu_found
41281 00003A14 8ED8 mov ds,ax
41282 00003A16 EBF3 jmp short ulu_next
41283
41284 ulu_found:
41285 00003A18 C60600005A ;mov byte [0],'z'
41286 ;mov byte [ARENA.SIGNATURE],arena_signature_end ; 'z'
41287 00003A1D 07 ulu_x: pop es
41288 00003A1E 1F pop ds
41289 00003A1F C3 retn
41290
41291 ;-----
41292 ; SYSINIT2.ASM - MSDOS 6.0 - 1991
41293 ;-----
41294 ; 14/04/2019 - Retro DOS v4.0
41295
41296 ; Multiple configuration block support Created 16-Mar-1992 by JeffPar
41297 ;
41298 ; Summary:
41299 ;
41300 ; The procedure "organize" crunches the in-memory copy of config.sys
41301 ; into lines delimited by CR/LF (sometimes no CR, but *always* an LF)
41302 ; with the leading "keyword=" replaced by single character codes (eg, B
41303 ; for BUFFERS, D for DEVICE, Z for any unrecognized keyword); see comtab
41304 ; and/or config.inc for the full list.
41305 ;
41306 ; [blockname] and INCLUDE are the major syntactical additions for multi-
41307 ; configuration support. blockname is either MENU, which contains one
41308 ; or more MENUITEM lines, an optional MENUDEFAULT (which includes optional
41309 ; time-out), or any user-defined keyword, such as NETWORK, CD-ROM, etc.
41310 ; INCLUDE allows the current block to name another block for inclusion
41311 ; during the processing phase of CONFIG.SYS. An INCLUDE is only honored
41312 ; once, precluding nasty infinite-loop scenarios. If blocks are present
41313 ; without a MENU block, then only lines inside COMMON blocks are processed.
41314 ;
41315 ; Example:
41316 ;
41317 ; [menu]
41318 ; menuitem=misc,Miscellaneous
41319 ; menuitem=network,Network Configuration
41320 ; menudefault=network,15
41321 ;
41322 ; [network]
41323 ; include misc
41324 ; device=foo
41325 ;
41326 ; [misc]
41327 ; device=bar
41328 ; include alternate
41329 ;
41330 ; [alternate]
41331 ; device=tar
41332 ;
41333 ;
41334 ; when the menu is displayed
41335 ;
41336 ; 1. Miscellaneous
41337 ; 2. Network Configuration
41338 ;
41339 ; #2 is highlighted as the default option, and will be automatically
41340 ; selected after 15 seconds. It will invoke the following lines in the
41341 ; following order:
41342 ;
41343 ; DEVICE=BAR
41344 ; DEVICE=TAR
41345 ; DEVICE=FOO
41346 ;
41347 ;
41348 ;MULTI_CONFIG equ 1
41349
41350 ; the following depend on the positions of the various letters in switchlist
41351
41352 switchnum equ 11111000b ; 0F8h ; which switches require number
41353
41354 flagec35 equ 00000100b ; 4 ; electrically compatible 3.5 inch disk drive
41355 flagdrive equ 00001000b ; 8
41356 flagcylh equ 00010000b ; 16
41357 flagseclim equ 00100000b ; 32
41358 flagheads equ 01000000b ; 64
41359 flagff equ 10000000b ; 128
41360
41361 ;-----
41362 ; 19/04/2019 - Retro DOS v4.0
41363
41364 ; MSDOS 6.21 IO.SYS - SYSINIT:3E78h
41365
41366 ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
41367 ; MSDOS 5.0 IO.SYS - SYSINIT:3054h
41368
41369 00003A20 00 insert_blank: db 0
41370
41371 ;-----
41372 ;
41373 ; procedure : setparms
41374 ;
41375 ; the following set of routines is used to parse the drivparm = command in
41376 ; the config.sys file to change the default drive parameters.
41377 ;-----
41378
41379
41380 setparms:
41381 00003A21 1E push ds

```



```

41382 00003A22 50      push    ax
41383 00003A23 53      push    bx
41384 00003A24 51      push    cx
41385 00003A25 52      push    dx
41386
41387 00003A26 0E      push    cs
41388 00003A27 1F      pop     ds
41389
41390 00003A28 31DB     xor     bx,bx
41391 00003A2A 8A1E[1A4F]   mov     bl,[drive]
41392                ; 18/12/2022
41393 00003A2E 43      inc     bx
41394                ; inc     bl ; get it correct for ioctl call
41395                ; (1=a,2=b...)
41396 00003A2F BA[BC4D]   mov     dx,deviceparameters
41397                ;mov     ah,IOCTL ; 44h
41398                ;mov     al,GENERIC_IOCTL ; 0Dh
41399                ; 04/07/2023
41400 00003A32 B80D44     mov     ax,(IOCTL<<8)|GENERIC_IOCTL
41401                ;mov     ch,RAWIO ; 8
41402                ;mov     cl,SET_DEVICE_PARAMETERS ; 40h
41403                ; 04/07/2023
41404 00003A35 B94008     mov     cx,(RAWIO<<8)|SET_DEVICE_PARAMETERS
41405 00003A38 CD21      int     21h
41406
41407                ; 27/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
41408 00003A3A 8A26[1B4F]   mov     ah,[switches]
41409                ;mov     al,[deviceparameters+20]
41410 00003A3E A0[D04D]   mov     al,[deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK]
41411 00003A41 8A0E[1A4F]   mov     cl,[drive]
41412                ;
41413                ;; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41414                ;;mov     ax,Bios_Data ; get Bios_Data segment
41415                ;;mov     ax,KERNEL_SEGMENT ; 70h
41416                ;; 21/10/2022
41417                ;;mov     ax,DOSBIODATASEG ; 0070h
41418                ;;mov     ds,ax ; set Bios_Data segment
41419                ;
41420                ; 27/07/2023
41421                ;;test word [cs:switches],flagec35 ; 4
41422                ;;test byte [cs:switches],flagec35
41423                ;;jz     short not_ec35
41424                ;
41425                ; 27/07/2023
41426                ;; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41427                ;;test word [switches],flagec35 ; 4
41428                ;; 12/12/2022
41429                ;;test byte [switches],flagec35 ; 4
41430                ;;jz     short eot_ok
41431                ;
41432                ;mov     cl,[cs:drive] ; which drive was this for?
41433                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41434                ;mov     cl,[drive]
41435                ; 27/07/2023
41436                ;mov     ax,DOSBIODATASEG ; 0070h
41437                ;mov     ds,ax
41438
41439 00003A45 BA7000     mov     dx,DOSBIODATASEG
41440 00003A48 8EDA      mov     ds,dx
41441
41442 00003A4A F6C404     test     ah,flagec35 ; test byte [cs:switches],flagec35
41443 00003A4D 7408      jz      short not_ec35
41444
41445                ;mov     al,1 ; assume drive 0
41446                ;shl     al,cl ; set proper bit depending on drive
41447                ;;or     [531h],al ; (MSDOS 6.21 IO.SYS Offset SYINIT:3E4Ch)
41448                ;;or     [ec35_flag],al ; set the bit in the permanent flags
41449                ; 27/07/2023
41450 00003A4F B401      mov     ah,1
41451 00003A51 D2E4      shl     ah,cl
41452 00003A53 0826[A204] or      [ec35_flag],ah
41453
41454                ; 07/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
41455                ; MSDOS 6.21 IO.SYS - SYINIT:3EB0h
41456                ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41457 not_ec35:
41458                ; Now adjust the BIOS's EOT variable if our new drive has more
41459                ; sectors per track than any old ones.
41460
41461                ; 27/07/2023
41462                ;;mov     al,[cs:deviceparameters+20]
41463                ;;mov     al,[cs:deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK]
41464
41465                ;cmp     al,[12Ch] ; (MSDOS 6.21 IO.SYS Offset SYINIT:3EB4h)
41466 00003A57 3A06[2C01]   cmp     al,[eot]
41467 00003A58 7603      jbe     short eot_ok
41468 00003A5D A2[2C01]   mov     [eot],al
41469 eot_ok:
41470 00003A60 5A      pop     dx ; fix up all the registers
41471 00003A61 59      pop     cx
41472 00003A62 5B      pop     bx
41473 00003A63 58      pop     ax
41474 00003A64 1F      pop     ds ; 13/05/2019
41475 00003A65 C3      retn
41476
41477                ;
41478                ;
41479                ; procedure : diddleback
41480                ;
41481                ; replace default values for further drivparm commands
41482                ;
41483                ;
41484                ;
41485 diddleback:
41486 00003A66 1E      push    ds
41487 00003A67 0E      push    cs
41488 00003A68 1F      pop     ds
41489                ;mov     word [deviceparameters+4],80
41490 00003A69 C706[C04D]5000   mov     word [deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS],80
41491                ;mov     byte [deviceparameters+1],2
41492 00003A6F C606[BD4D]02     mov     byte [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICECTYPE],DEV_3INCH720KB ; 2
41493                ;mov     word [deviceparameters+2],0
41494 00003A74 C706[BE4D]0000   mov     word [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICEATTRIBUTES],0
41495 00003A7A C706[1B4F]0000   mov     word [switches],0 ; zero all switches
41496 00003A80 1F      pop     ds
41497 00003A81 C3      retn
41498
41499                ; 03/01/2023
41500                %if 0
41501
41502                ; 15/04/2019 - Retro DOS v4.0
41503
41504                ;
41505                ;

```

```

41506 ; procedure : parseline
41507 ;
41508 ; entry point is parseline. al contains the first character in command line.
41509 ;
41510 ;-----
41511 ;
41512 ; 03/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
41513 ; (SYSINIT:3EDFh)
41514 ;
41515 ; 01/11/2022 - RetroDOS v4.0 (Modified MSDOS 5.0 IO.SYS)
41516 ; (SYSINIT:30ACh)
41517 parseline:
41518 ; 03/01/2023
41519 ; ds = cs ; *
41520 ;push ds ; *
41521 ;push cs ; *
41522 ;pop ds ; *
41523
41524 nextswtch:
41525 cmp al,cr ; carriage return?
41526 je short done_line
41527 cmp al,lf ; linefeed?
41528 je short put_back ; put it back and done
41529
41530 ; anything less or equal to a space is ignored.
41531
41532 cmp al,' ' ; space?
41533 jbe short getnext ; skip over space
41534 cmp al,'/'
41535 je short getparm ; mark error invalid-character-in-input
41536 stc
41537 jmp short exitp1
41538 ; 03/01/2023
41539 swterr:
41540 retn
41541
41542 getparm:
41543 call check_switch
41544 mov [switches],bx ; save switches read so far
41545 jc short swterr
41546
41547 getnext:
41548 call getchr
41549 ;jc short done_line
41550 ;jmp short nextswtch
41551 ; 03/01/2023
41552 jnc short nextswtch
41553 ;swterr:
41554 ;jmp short exitp1 ; exit if error
41555
41556 done_line:
41557 ; 12/12/2022
41558 test byte [switches],flagdrive ; 8
41559 ;test word [switches],flagdrive ; 8 ; see if drive specified
41560 jnz short okay
41561 stc
41562 jmp short exitp1 ; mark error no-drive-specified
41563 ; 03/01/2023
41564 retn
41565
41566 okay:
41567 mov ax,[switches]
41568 and ax,0003h ; get flag bits for changeline and non-rem
41569 mov [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICEATTRIBUTES],ax
41570 mov word [deviceparameters+A_DEVICEPARAMETERS.DP_TRACKTABLEENTRIES],0
41571 ;clc ; everything is fine
41572 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41573 ; 12/12/2022
41574 ; cf=0
41575 ;clc
41576 ;call setdeviceparameters
41577 ; 03/01/2023
41578 jmp setdeviceparameters
41579
41580 ;exitp1:
41581 ; 03/01/2023
41582 ; ds = cs
41583 ;pop ds ; *
41584 retn
41585
41586 put_back:
41587 inc word [count] ; one more char to scan
41588 dec word [chrptr] ; back up over linefeed
41589 jmp short done_line
41590
41591 %endif
41592 ;-----
41593 ;
41594 ; procedure : check_switch
41595 ;
41596 ; processes a switch in the input. it ensures that the switch is valid, and
41597 ; gets the number, if any required, following the switch. the switch and the
41598 ; number *must* be separated by a colon. carry is set if there is any kind of
41599 ; error.
41600 ;-----
41601 ;
41602 ; 09/09/2023
41603
41604 err_swth:
41605 xor bx,cx ; remove this switch from the records
41606
41607 err_check:
41608 stc
41609 err_chk:
41610 done_swth: ; 09/09/2023 (cf=0)
41611 retn
41612
41613 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
41614
41615 check_switch:
41616 call getchr
41617 ;jc short err_check
41618 jc short err_chk
41619 and al,0DFh ; convert it to upper case
41620 cmp al,'A'
41621 ;jb short err_check
41622 jb short err_chk ; 15/04/2019 - Retro DOS v4.0
41623 cmp al,'Z'
41624 ja short err_check
41625
41626 push es
41627
41628 push cs
41629 pop es

```

```

41630
41631 ;mov cl,[switchlist] ; get number of valid switches
41632 ;mov ch,0
41633 ;mov di,1+switchlist ; point to string of valid switches
41634 ; 09/09/2023
41635 00003A98 BF[4050] mov di,switchlist
41636 00003A9B 8A0D mov cl,[di]
41637 00003A9D B500 mov ch,0
41638 00003A9F 47 inc di ; 1+switchlist
41639
41640 00003AA0 F2AE repne scasb
41641
41642 00003AA2 07 pop es
41643 00003AA3 75DF jnz short err_check
41644
41645 00003AA5 B80100 mov ax,1
41646 00003AA8 D3E0 shl ax,cl ; set bit to indicate switch
41647 00003AAA 8B1E[1B4F] mov bx,[switches] ; get switches so far
41648 00003AAE 09C3 or bx,ax ; save this with other switches
41649 00003AB0 89C1 mov cx,ax
41650 ; 12/12/2022
41651 00003AB2 A8F8 test al,switchnum ; 0F8h
41652 ;test ax,switchnum ; 0F8h ; test against switches that require number to follow
41653 00003AB4 74CF jz short done_swch
41654
41655 00003AB6 E8F40C call getchr
41656 00003AB9 72C7 jc short err_swch
41657
41658 00003ABB 3C3A cmp al,':'
41659 00003ABD 75C3 jne short err_swch
41660
41661 00003ABF E8EB0C call getchr
41662 00003AC2 53 push bx ; preserve switches
41663 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41664 ;mov byte [cs:sepchr], ' ' ; allow space separators
41665 ; 12/12/2022
41666 ; ds = cs
41667 00003AC3 C606[AE02]20 mov byte [sepchr], ' '
41668 00003AC8 E8980D call getnum
41669 ;mov byte [cs:sepchr],0
41670 ; 12/12/2022
41671 00003ACB C606[AE02]00 mov byte [sepchr],0
41672 00003AD0 5B pop bx ; restore switches
41673
41674 ; because getnum does not consider carriage-return or line-feed as ok, we do
41675 ; not check for carry set here. if there is an error, it will be detected
41676 ; further on (hopefully).
41677
41678 ; 09/09/2023
41679 ;call process_num
41680 ;jmp short process_num
41681
41682 ;done_swch:
41683 ; clc
41684 ; ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41685 ; ; 12/12/2022
41686 ; ; cf=0
41687 ; clc
41688 ; retn
41689
41690 ;-----
41691 ;
41692 ; procedure : process_num
41693 ;
41694 ; this routine takes the switch just input, and the number following (if any),
41695 ; and sets the value in the appropriate variable. if the number input is zero
41696 ; then it does nothing - it assumes the default value that is present in the
41697 ; variable at the beginning. zero is ok for form factor and drive, however.
41698 ;
41699 ;-----
41700
41701 ; 02/11/2022 - Retrodos v4.0 (Modified MSDOS 5.0 IO.SYS)
41702 ; (SYSINIT:3156h)
41703 process_num:
41704 00003AD1 850E[1B4F] test [switches],cx ; if this switch has been done before,
41705 00003AD5 752B jnz short done_ret ; ignore this one.
41706 ; 12/12/2022
41707 00003AD7 F6C108 test cl,flagdrive ; 8
41708 ;test cx,flagdrive ; 8
41709 00003ADA 7404 jz short try_f
41710 00003ADC A2[1A4F] mov byte [drive],al
41711 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41712 ;jmp short done_ret
41713 ; 12/12/2022
41714 ; cf=0
41715 00003ADF C3 retn ; 13/05/2019
41716
41717 try_f:
41718 ; 12/12/2022
41719 00003AE0 F6C180 test cl,flagff ; 80h
41720 00003AE3 7404 ;test cx,flagff ; 80h
41721 jz short try_t
41722
41723 ; ensure that we do not get bogus form factors that are not supported
41724
41725 00003AE5 A2[BD4D] ;mov [deviceparameters+1],al
41726 mov [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICETYPE],al
41727 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41728 ;jmp short done_ret
41729 ; 12/12/2022
41730 ; cf=0
41731 00003AE8 C3 retn ; 13/05/2019
41732
41733 try_t:
41734 or ax,ax
41735 00003AE9 09C0 jz short done_ret ; if number entered was 0, assume default value
41736 ; 12/12/2022
41737 00003AED F6C110 test cl,flagcyl ; 10h
41738 ;test cx,flagcyl ; 10h
41739 jz short try_s
41740
41741 ;mov [deviceparameters+4],ax
41742 mov [deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS],ax
41743 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41744 ;jmp short done_ret
41745 ; 12/12/2022
41746 ; cf=0
41747 00003AF5 C3 retn ; 13/05/2019
41748
41749 try_s:
41750 ; 12/12/2022
41751 00003AF6 F6C120 test cl,flagseclim ; 20h
41752 ;test cx,flagseclim ; 20h
41753 jz short try_h
41754 mov [slim],ax
41755 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41756 ;jmp short done_ret

```

```

41754             ; 12/12/2022
41755             ; cf=0
41756 00003AFE C3      retn             ; 13/05/2019
41757
41758             ; must be for number of heads
41759
41760 try_h:
41761 00003AFF A3[164F]  mov     [hlim],ax
41762 done_ret:          ;
41763                 ;clc
41764                 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41765                 ; 12/12/2022
41766                 ; cf=0 (test instruction resets cf)
41767                 ;clc
41768 00003B02 C3      retn
41769
41770
41771             ; 16/04/2024 - Retro DOS v5.0
41772             ; 03/01/2023 - Retro DOS v4.2
41773 %if 1
41774
41775             ; 15/04/2019 - Retro DOS v4.0
41776
41777             ;-----
41778             ;
41779             ; procedure : parseline
41780
41781             ; entry point is parseline. al contains the first character in command line.
41782             ;
41783             ;-----
41784
41785             ; 16/04/2024 - RetroDOS v5.0 (Modified PC DOS 7.1 IBMBIO.COM)
41786             ; (PC DOS 7.1 IBMBIO.COM - SYSINIT:4151h)
41787
41788             ; 03/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
41789             ; (SYSINIT:3EDFh)
41790
41791             ; 01/11/2022 - RetroDOS v4.0 (Modified MSDOS 5.0 IO.SYS)
41792             ; (SYSINIT:30ACH)
41793
41794 parseline:
41795             ; 03/01/2023
41796             ; ds = cs ; *
41797
41798             ;push ds ; *
41799
41800             ;push cs ; *
41801             ;pop ds ; *
41802
41803 nextswtch:
41804 cmp     al,cr          ; carriage return?
41805 je      short done_line
41806 cmp     al,lf          ; linefeed?
41807 je      short put_back ; put it back and done
41808
41809             ; anything less or equal to a space is ignored.
41810
41811 cmp     al,' '          ; space?
41812 jbe     short getnext  ; skip over space
41813 cmp     al,'/'
41814 je      short getparm
41815 stc     ; mark error invalid-character-in-input
41816 jmp     short exitpl
41817             ; 03/01/2023
41818 swterr:
41819 retn
41820
41821 getparm:
41822 call    check_switch
41823 mov     [switches],bx ; save switches read so far
41824 jc      short swterr
41825
41826 getnext:
41827 call    getchr
41828 jc      short done_line
41829 jmp     short nextswtch
41830             ; 03/01/2023
41831 jnc     short nextswtch
41832 ;swterr:
41833 jmp     short exitpl ; exit if error
41834
41835 done_line:
41836             ; 12/12/2022
41837 test    byte [switches],flagdrive ; 8
41838 ;test   word [switches],flagdrive ; 8 ; see if drive specified
41839 jnz     short okay
41840 stc     ; mark error no-drive-specified
41841 jmp     short exitpl
41842             ; 03/01/2023
41843 retn
41844
41845 ;exitpl:
41846             ; 03/01/2023
41847             ; ds = cs
41848             ;pop ds ; *
41849             ;retn
41850
41851 put_back:
41852 inc     word [count] ; one more char to scan
41853 dec     word [chrptr] ; back up over linefeed
41854 jmp     short done_line
41855
41856 okay:
41857 mov     ax,[switches]
41858 and     ax,0003h ; get flag bits for changeline and non-rem
41859 mov     [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICEATTRIBUTES],ax
41860             ; 16/04/2024
41861 ;mov     word [deviceparameters+A_DEVICEPARAMETERS.DP_TRACKTABLEENTRIES],0
41862 ;;;
41863 mov     word [deviceparameters+92],0 ; PC DOS 7.1 IBMBIO.COM
41864 ;;;
41865 ;clc     ; everything is fine
41866             ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41867             ; 12/12/2022
41868             ; cf=0
41869             ;clc
41870 ;call    setdeviceparameters
41871             ; 03/01/2023
41872 jmp     short setdeviceparameters
41873
41874 %endif
41875
41876             ; M047 -- Begin modifications (too numerous to mark specifically)
41877             ;-----

```

```

41878 ;
41879 ; procedure : setdeviceparameters
41880 ;
41881 ; setdeviceparameters sets up the recommended bpb in each bds in the
41882 ; system based on the form factor. it is assumed that the bpbs for the
41883 ; various form factors are present in the bpbtable. for hard files,
41884 ; the recommended bpb is the same as the bpb on the drive.
41885 ; no attempt is made to preserve registers since we are going to jump to
41886 ; sysinit straight after this routine.
41887 ;
41888 ; if we return carry, the DRIVPARM will be aborted, but presently
41889 ; we always return no carry
41890 ;
41891 ; note: there is a routine by the same name in msdioc1.asm
41892 ;
41893 ;-----
41894 ;
41895 ; 15/04/2019 - Retro DOS v4.0
41896 ;
41897 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
41898 ;
41899 ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
41900 ; (SYSINIT:3FC4h)
41901 ;
41902 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
41903 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:4236h)
41904 ;
41905 setdeviceparameters:
41906 ; 03/01/2023
41907 ; ds = cs
41908 ;
41909 00003B45 06          push     es
41910 ;
41911 00003B46 0E          push     cs
41912 00003B47 07          pop      es
41913 ;
41914 00003B48 31DB        xor      bx,bx
41915 00003B4A 8A1E[BD4D]   mov      bl,[deviceparameters+A_DEVICEPARAMETERS.DP_DEVICETYPE]
41916 00003B4E 80FB00        cmp      bl,DEV_5INCH ; 0
41917 00003B51 7506          jne      short got_80
41918 ;
41919 00003B53 C706[C04D]2800      mov      word [deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS],40
41920 ; 48 tpi=40 cyl
41921 got_80:
41922 00003B59 D1E3        shl      bx,1          ; get index into bpb table
41923 00003B5B 8BB7[2C50]      mov      si,[bpbtable+bx] ; get address of bpb
41924 ;
41925 ;mov di,deviceparameters+7
41926 ; 02/11/2022
41927 00003B5F BF[C34D]      mov      di,deviceparameters+A_DEVICEPARAMETERS.DP_BPB ; es:di -> bpb
41928 00003B62 B93B00      mov      cx,A_BPB.size ; 31
41929 ; 09/09/2023
41930 ;mov cx,59 ; PCDOS 7.1 IBMBIO.COM A_BPB.size
41931 00003B65 FC          cld
41932 ;repe movsb
41933 ; 02/11/2022
41934 00003B66 F3A4        rep      movsb
41935 ;
41936 00003B68 07          pop      es
41937 ;
41938 ; 12/12/2022
41939 00003B69 F606[1B4F]20 test     byte [switches],flagseclim ; 20h
41940 ;test word [switches],flagseclim ; 20h
41941 00003B6E 7406          jz       short see_heads
41942 ;
41943 00003B70 A1[184F]      mov      ax,[slim]
41944 ;mov [deviceparameters+20],ax
41945 00003B73 A3[D04D]      mov      [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK],ax
41946 ;
41947 see_heads:
41948 ; 12/12/2022
41949 00003B76 F606[1B4F]40 test     byte [switches],flagheads ; 40h
41950 ;test word [switches],flagheads ; 40h
41951 00003B7B 7406          jz       short heads_not_altered
41952 ;
41953 00003B7D A1[164F]      mov      ax,[hlim]
41954 ;mov [deviceparameters+22],ax
41955 00003B80 A3[D24D]      mov      [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.HEADS],ax
41956 ;
41957 heads_not_altered:
41958 ;
41959 ; set up correct media descriptor byte and sectors/cluster
41960 ; sectors/cluster is always 2 except for any one sided disk or 1.44M
41961 ;
41962 ;mov byte [deviceparameters+9],2
41963 ; 02/11/2022
41964 ;mov byte [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERCLUSTER],2
41965 ; 03/01/2023
41966 00003B83 B80200      mov      ax,2
41967 00003B86 A2[C54D]      mov      [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERCLUSTER],ax ; 2
41968 ;
41969 00003B89 B3F0        mov      bl,0F0h          ; get default mediabyte
41970 ;
41971 ; preload the mediadescriptor from the bpb into bh for convenient access
41972 ;
41973 ;mov bh,[deviceparameters+17]
41974 ; 02/11/2022
41975 00003B8B 8A3E[CD4D]   mov      bh,[deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.MEDIADESCRIPTOR]
41976 ;
41977 ; 03/01/2023
41978 ; ax = 2
41979 00003B8F 3906[D24D]   cmp      [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.HEADS],ax ; >2 heads?
41980 ;cmp word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.HEADS],2 ; >2 heads?
41981 00003B93 773C          ja       short got_correct_mediad ; just use default if heads>2
41982 ;
41983 00003B95 7524          jne      short only_one_head ; one head, do one head stuff
41984 ;
41985 ; two head drives will use the mediadescriptor from the bpb
41986 ;
41987 00003B97 88FB        mov      bl,bh          ; get mediadescriptor from bpb
41988 ;
41989 ; two sided drives have two special cases to look for. One is
41990 ; a 320K diskette (40 tracks, 8 secs per track). It uses
41991 ; a mediaid of 0fch. The other is 1.44M, which uses only
41992 ; one sector/cluster.
41993 ;
41994 ; any drive with 18secs/trk, 2 heads, 80 tracks, will be assumed
41995 ; to be a 1.44M and use only 1 sector per cluster. Any other
41996 ; type of 2 headed drive is all set.
41997 ;
41998 00003B99 833E[D04D]12      cmp      word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK],18
41999 00003B9E 7509          jne      short not_144m
42000 00003BA0 833E[C04D]50      cmp      word [deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS],80
42001 00003BA5 7502          jne      short not_144m

```

```

42002
42003
42004
42005 00003BA7 EB24
42006
42007
42008
42009
42010 00003BA9 833E[C04D]28
42011 00003BAE 7521
42012 00003BB0 833E[D04D]08
42013 00003BB5 751A
42014
42015 00003BB7 B3FC
42016 00003BB9 EB16
42017
42018
42019
42020
42021
42022 00003BBB 803E[BD4D]00
42023 00003BC0 740B
42024
42025
42026
42027
42028
42029 00003BC2 B3FC
42030 00003BC4 833E[D04D]08
42031
42032 00003BC9 7502
42033
42034 00003BCB B3FE
42035
42036
42037
42038
42039
42040
42041
42042 00003BCD 48
42043 00003BCE A2[C54D]
42044
42045
42046
42047 00003BD1 881E[CD4D]
42048
42049
42050
42051 00003BD5 A1[C04D]
42052 00003BD8 F726[D24D]
42053 00003BDC F726[D04D]
42054 00003BE0 A3[CB4D]
42055 00003BE3 F8
42056
42057 00003BE4 C3
42058
42059
42060
42061
42062
42063
42064
42065
42066
42067
42068
42069
42070 00003BE5 AA
42071
42072
42073
42074
42075
42076
42077 00003BE6 EB0E
42078
42079 00003BE8 F9
42080 00003BE9 C3
42081
42082
42083
42084
42085
42086
42087
42088
42089
42090
42091
42092
42093
42094 00003BEA 8B0E[5603]
42095
42096 00003BEE E3F8
42097
42098
42099
42100
42101
42102
42103
42104
42105
42106
42107
42108
42109 00003BF0 31F6
42110 00003BF2 89F7
42111 00003BF4 31C0
42112
42113
42114
42115
42116
42117
42118
42119
42120
42121 00003BF6 C606[5003]00
42122
42123 00003BFB E8EF01
42124 00003BFE 74E5
42125 00003C00 E8D001

; we've got cyl=80, heads=2, secpertrack=18. Set cluster size to 1.
jmp short got_one_secperclus_drive

; check for 320K
not_144m:
cmp word [deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS],40
jne short got_correct_mediad
cmp word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK],8
jne short got_correct_mediad

mov bl,0FCh
jmp short got_correct_mediad

only_one_head:
; if we don't have a 360K drive, then just go use 0f0h as media descr.
cmp byte [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICETYPE],DEV_5INCH ; 0
je short got_one_secperclus_drive

; single sided 360K drive uses either 0fch or 0feh, depending on
; whether sectorspertrack is 8 or 9. For our purposes, anything
; besides 8 will be considered 0fch
mov bl,0FCh ; single sided 9 sector media id
cmp word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK],8
; 12/12/2022
jne short got_one_secperclus_drive ; okay if anything besides 8

mov bl,0FEh ; 160K mediaid

; we've either got a one sided drive, or a 1.44M drive
; either case we'll use 1 sector per cluster instead of 2

got_one_secperclus_drive:
; 03/01/2023
; ax = 2
dec ax ; ax = 1
mov [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERCLUSTER],al ; 1
;mov byte [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERCLUSTER],1

got_correct_mediad:
mov [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.MEDIADESCRIPTOR],bl

; Calculate the correct number of Total Sectors on medium
mov ax,[deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS]
mul word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.HEADS]
mul word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK]
mov word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.TOTALSECTORS],ax
clc ; we currently return no errors

retn

; M047 -- end rewritten routine

;-----
;
; procedure : organize
;
;-----

; 09/09/2023 - Retrodos v4.2 IO.SYS (Optimization)
%if 1
end_commd_line:
stosb ; store line feed char in buffer for the linecount.
;mov byte [cs:com_level],0 ; reset the command level.
; 03/01/2023
; ds = cs
;mov byte [com_level],0
;jmp short org1
; 09/09/2023
;jmp short org0
nochar1:
stc
retn
%endif
; 02/11/2022 - Retrodos v4.0 (Modified MSDOS 5.0 IO.SYS)
; (SYSINIT:3234h)

; 03/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:4067h)

; 09/09/2023 - Retrodos v4.2 IO.SYS (Optimization)
; (PCDOS 7.1 IBMBIO.COM - SYSINIT:42D9h)

organize:
; 03/01/2023
; ds = cs
mov cx,[count]
;mov cx,[cs:count]
jcxz nochar1

;ifndef MULTI_CONFIG
;
; In MULTI_CONFIG, we map to upper case on a line-by-line basis,
; because we the case of values in SET commands preserved
;
; call mapcase
;endif
; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; 03/01/2023 - Retro DOS v4.2
;call mapcase

xor si,si
mov di,si
xor ax,ax
; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
;mov byte [cs:com_level],0
; 12/12/2022
;mov [cs:com_level],al ; 0
; 03/01/2023
; ds = cs
; 09/09/2023
;mov [com_level],al ; 0

org0:
mov byte [com_level],0 ; 09/09/2023
org1:
call skip_comment
jz short end_commd_line ; found a comment string and skipped.
call get2 ; not a comment string. then get a char.

```



```

42126 00003C03 3C0A      cmp     al,lf ; 0Ah
42127 00003C05 74DE      je      short end_commd_line ; starts with a blank line.
42128 00003C07 3C20      cmp     al,' ' ; 20h
42129 00003C09 76F0      jbe     short org1 ; skip leading control characters
42130                      ; 09/09/2023
42131                      ; jmp     short findit
42132
42133 ; 09/09/2023
42134 %if 0
42135 end_commd_line:
42136      stosb ; store line feed char in buffer for the linecount.
42137      mov     byte [cs:com_level],0 ; reset the command level.
42138      ; 03/01/2023
42139      ; ds = cs
42140      mov     byte [com_level],0
42141      jmp     short org1
42142
42143 nochar1:
42144      stc
42145      retn
42146 %endif
42147
42148 findit:
42149      push    cx
42150      push    si
42151      push    di
42152      mov     bp,si
42153      dec     bp
42154      mov     si,comtab ; prepare to search command table
42155      mov     ch,0
42156 findcom:
42157      mov     di,bp
42158      mov     cl,[si]
42159      inc     si
42160      jcxz     nocom
42161
42162 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
42163
42164 ; 03/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
42165
42166 ;ifdef     MULTI_CONFIG
42167
42168 ; Simplify future parsing by collapsing ";" onto "REM", and at the same
42169 ; time skip the upcoming delimiter test (since ";" need not be followed by
42170 ; anything in particular)
42171
42172      cmp     byte [es:di],CONFIG_SEMICOLON ; ';'
42173      je      short semicolon
42174 loopcom:
42175      mov     al,[es:di]
42176      inc     di
42177      and     al,~20h ; 0DFh ; force upper case
42178      inc     si ; compare to byte @es:di
42179      cmp     al,[si-1]
42180      ; 28/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
42181      mov     ah,[es:di]
42182      inc     di
42183      and     ah,~20h ; 0DFh
42184      lodsb ; mov al,[si]
42185      ; inc si
42186      cmp     al,ah
42187      loope   loopcom
42188      ; 28/07/2023
42189      xor     ah,al ; result: ah = 0 (*) if ah = al
42190      loopz   loopcom
42191 ;else
42192 ; repe     cmpsb
42193 %endif
42194 ; 02/11/2022
42195 ; 03/01/2023 - Retro DOS v4.2
42196 ; repe     cmpsb
42197
42198 ; 28/07/2023
42199 ;lahf
42200      add     si,cx ; bump to next position without affecting flags
42201      sahlf
42202      lodsb ; get indicator letter
42203      jnz     short findcom
42204      ; 28/07/2023
42205      or      ah,ah ; (*)
42206      jnz     short findcom
42207
42208      cmp     byte [es:di],cr ; the next char might be cr,lf
42209      je      short gotcom0 ; such as in "rem",cr,lf case.
42210      cmp     byte [es:di],lf
42211      je      short gotcom0
42212
42213 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
42214
42215 ; 03/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
42216
42217 ;ifdef     MULTI_CONFIG
42218
42219 ; Skip the delimiter test for the BEGIN identifier (it doesn't have one).
42220
42221      cmp     al,CONFIG_BEGIN ; '['
42222      je      short gotcom0
42223 ;endif
42224      push    ax
42225      mov     al,[es:di] ; now the next char. should be a delim.
42226
42227 ;ifdef     MULTI_CONFIG
42228
42229 ; If keyword is *immediately* followed by a question mark (?), then
42230 ; set the high bit of the ASCII command code (CONFIG_OPTION_QUERY) that is
42231 ; stored in the CONFIG.SYS memory image.
42232
42233      cmp     al,'?' ; explicit interactive command?
42234      jne     short no_query ; no
42235      pop     ax ; yes, so retrieve the original code
42236      ;or     al,80h ; 03/01/2023
42237      or      al,CONFIG_OPTION_QUERY ; and set the QUERY bit
42238      jmp     short gotcom0 ;
42239 semicolon:
42240      mov     al,CONFIG_REM ; '0'
42241      jmp     short gotcom0
42242 no_query:
42243 %endif ;MULTI_CONFIG
42244
42245 ; 02/11/2022
42246 ; 03/01/2023 - Retro DOS v4.2
42247 ;push     ax
42248 ;mov     al,[es:di] ; now the next char. should be a delim.
42249

```

```

42250 00003C57 E82E0B      call    delim
42251 no_delim:
42252      pop     ax
42253      jnz     short findcom
42254 gotcom0:
42255      pop     di
42256      pop     si
42257      pop     cx
42258      jmp     short gotcom
42259 nocom:
42260      pop     di
42261      pop     si
42262      pop     cx
42263      mov     al,CONFIG_UNKNOWN ; 'Z'
42264      stosb    ; save indicator char.
42265 _skipline:
42266      call    get2
42267      cmp     al,lf ; 0Ah      ; skip this bad command line
42268      jne     short _skipline
42269      jmp     short end_commd_line ; handle next command line
42270      ; 09/09/2023
42271      jmp     end_commd_line
42272 gotcom:
42273      stosb    ; save indicator char in buffer
42274
42275      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
42276
42277      ; 03/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
42278
42279      ;ifdef    MULTI_CONFIG
42280
42281      ; Don't pollute "cmd_indicator" with the CONFIG_OPTION_QUERY bit though;
42282      ; it screws up the direct comparisons below.
42283
42284      and     al,~CONFIG_OPTION_QUERY ; 7Fh
42285      ;endif
42286      mov     [cs:cmd_indicator],al ; save it for the future use.
42287      ; 03/01/2023
42288      ; ds = cs
42289      mov     [cmd_indicator],al ; save it for the future use.
42290
42291      ;ifdef    MULTI_CONFIG
42292
42293      ; There is no whitespace/delimiter between the "begin block" character
42294      ; ([]) and the name of block (eg, [menu]), therefore skip this delimiter
42295      ; skipping code
42296
42297      cmp     al,CONFIG_BEGIN
42298      je      short org31
42299      cmp     al,CONFIG_SUBMENU ; 'O'
42300      je      short no_mapcase
42301      cmp     al,CONFIG_MENUITEM ; 'E'
42302      je      short no_mapcase
42303      cmp     al,CONFIG_MENUEFAULT ; 'A'
42304      je      short no_mapcase
42305      cmp     al,CONFIG_INCLUDE ; 'J'
42306      je      short no_mapcase
42307      call    mapcase ; map case of rest of line to UPPER
42308 no_mapcase:
42309      ;endif
42310      ; 02/11/2022
42311      ; mov     [cs:cmd_indicator],al ; save it for the future use.
42312      ; 03/01/2023
42313      ; ds = cs
42314      ; mov     [cmd_indicator],al
42315 org2:
42316      call    get2 ; skip the command name until delimiter
42317      cmp     al,lf ; 0Ah
42318      je      short org21
42319      cmp     al,cr ; 0Dh
42320      je      short org21
42321      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
42322      ; 03/01/2023 - Retro DOS v4.2
42323      cmp     al,'/' ; T-RICHJ: Added to allow DEVHIGH/L:...
42324      je      short org21 ; T-RICHJ: to be parsed properly.
42325
42326      call    delim
42327      jnz     short org2
42328      jmp     short org3
42329 org21:
42330      dec     si ; undo si, cx register
42331      inc     cx ; and continue
42332 org3:
42333      cmp     byte [cs:cmd_indicator],CONFIG_COMMENT ; 'Y'
42334      je      short get_cmt_token
42335      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
42336      ; 03/01/2023 - Retro DOS v4.2
42337      cmp     byte [cs:cmd_indicator],CONFIG_DEVICE ; 'D'
42338      je      short org_file
42339      cmp     byte [cs:cmd_indicator],CONFIG_INSTALL ; 'I'
42340      je      short org_file
42341      cmp     byte [cs:cmd_indicator],CONFIG_INSTALLHIGH ; 'W'
42342      je      short org_file
42343      ; 02/11/2022
42344      ; 03/01/2023 - Retro DOS v4.2
42345      ; cmp     byte [cs:cmd_indicator],CONFIG_DEVICE ; 'D'
42346      ; je      short org_file
42347      cmp     byte [cs:cmd_indicator],CONFIG_SHELL ; 'S'
42348      je      short org_file
42349      ; cmp     byte [cs:cmd_indicator],CONFIG_SWITCHES ; '1'
42350      je      short org_switch
42351
42352      ; 03/01/2023
42353      ; ds = cs
42354
42355      cmp     byte [cmd_indicator],CONFIG_COMMENT ; 'Y'
42356      je      short get_cmt_token
42357      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
42358      ; 03/01/2023 - Retro DOS v4.2
42359      cmp     byte [cmd_indicator],CONFIG_DEVICE ; 'D'
42360      je      short org_file
42361      cmp     byte [cmd_indicator],CONFIG_INSTALL ; 'I'
42362      je      short org_file
42363      cmp     byte [cmd_indicator],CONFIG_INSTALLHIGH ; 'W'
42364      je      short org_file
42365      ; 02/11/2022
42366      ; 03/01/2023 - Retro DOS v4.2
42367      cmp     byte [cmd_indicator],CONFIG_DEVICE ; 'D'
42368      je      short org_file
42369      cmp     byte [cmd_indicator],CONFIG_SHELL ; 'S'
42370      je      short org_file
42371      cmp     byte [cmd_indicator],CONFIG_SWITCHES ; '1'
42372      je      short org_switch
42373

```



```

42374
42375 00003CD1 E99500
42376
42377
42378 00003CD4 E81601
42379 00003CD7 7472
42380
42381 00003CD9 E8F700
42382 00003CDC E8B10A
42383 00003CDF 74F3
42384
42385 00003CE1 AA
42386 00003CE2 E99300
42387
42388
42389 00003CE5 E80501
42390 00003CE8 7464
42391
42392 00003CEA E8E600
42393 00003CED E8980A
42394 00003CF0 74F3
42395
42396 00003CF2 AA
42397
42398
42399 00003CF3 E8F700
42400 00003CF6 7456
42401
42402 00003CF8 E8D800
42403 00003CFB 3C2F
42404 00003CFD 7457
42405
42406 00003CFF AA
42407 00003D00 E8850A
42408 00003D03 7459
42409
42410 00003D05 3C20
42411 00003D07 77EA
42412 00003D09 EB53
42413
42414
42415 00003D0B E8C500
42416 00003D0E 3C20
42417 00003D10 74F9
42418 00003D12 3C09
42419 00003D14 74F5
42420 00003D16 3C3D
42421 00003D18 74F1
42422 00003D1A 3C0D
42423 00003D1C 7426
42424 00003D1E 3C0A
42425 00003D20 7422
42426
42427
42428
42429
42430
42431 00003D22 A2[5203]
42432 00003D25 C606[5103]01
42433 00003D2A E8A600
42434 00003D2D 3C20
42435 00003D2F 7413
42436 00003D31 3C09
42437 00003D33 740F
42438 00003D35 3C0D
42439 00003D37 740B
42440 00003D39 3C0A
42441 00003D3B 740E
42442
42443
42444
42445
42446 00003D3D A2[5303]
42447 00003D40 FE06[5103]
42448
42449
42450 00003D44 E88C00
42451 00003D47 3C0A
42452 00003D49 75F9
42453
42454 00003D4B E997FE
42455
42456
42457 00003D4E 26C60500
42458 00003D52 47
42459 00003D53 E98FFE
42460
42461
42462 00003D56 26C60500
42463 00003D5A 47
42464 00003D5B AA
42465 00003D5C EB1A
42466
42467
42468 00003D5E 26C645FF00
42469 00003D63 3C0A
42470 00003D65 74E4
42471 00003D67 EB0F
42472
42473
42474 00003D69 E88100
42475 00003D6C 74DD
42476
42477 00003D6E E86200
42478 00003D71 E81C0A
42479 00003D74 74F3
42480 00003D76 EB08
42481
42482
42483 00003D78 E87200
42484 00003D7B 74CE
42485 00003D7D E85300
42486
42487
42488 00003D80 AA
42489 00003D81 3C22
42490 00003D83 743A
42491 00003D85 3C20
42492 00003D87 77EF
42493
42494
42495
42496
42497

org31:
    jmp     org4

org_switch:
    call    skip_comment
    jz      short end_commd_line_brdg

    call    get2
    call    org_delim
    jz      short org_switch

    stosb
    jmp     org5

org_file:
    ; get the filename and put 0 at end
    call    skip_comment
    jz      short org_put_zero

    call    get2
    call    delim
    jz      short org_file ; skip the possible delimiters

    stosb
    ; copy the first non delim char found in buffer

org_copy_file:
    call    skip_comment ; comment char in the filename?
    jz      short org_put_zero ; then stop copying filename at that point

    call    get2
    cmp     al,'/'
    je      short end_file_slash ; a switch char? (device=filename/xxx)
    ; this will be the special case.

    stosb
    ; save the char. in buffer
    call    delim
    jz      short end_copy_file

    cmp     al, ' '
    ja      short org_copy_file ; keep copying
    jmp     short end_copy_file ; otherwise, assume end of the filename.

get_cmt_token:
    ; get the token. just max. 2 char.
    call    get2
    cmp     al,' '
    je      short get_cmt_token ; skip white spaces or "=" char.
    ; (we are allowing the other special
    cmp     al,tab ; 9 ; characters can used for comment id.
    je      short get_cmt_token ; character.)
    cmp     al,'='
    je      short get_cmt_token ; = is special in this case.
    cmp     al,cr
    je      short get_cmt_end ; cannot accept the carriage return
    cmp     al,lf
    je      short get_cmt_end

    ; 03/01/2023
    ; ds = cs
    mov     [cs:cmmt1],al ; store it
    mov     byte [cs:cmmt1],1 ; 1 char. so far.
    mov     [cmmt1],al ; store it
    mov     byte [cmmt1],1 ; 1 char. so far.
    call    get2
    cmp     al,' ' ; 20h
    je      short get_cmt_end
    cmp     al,tab ; 9
    je      short get_cmt_end
    cmp     al,cr ; 0Dh
    je      short get_cmt_end
    cmp     al,lf ; 0Ah
    je      short end_commd_line_brdg

    ;mov     [cs:cmmt2],al
    ;inc     byte [cs:cmmt2]
    ; 03/01/2023
    mov     [cmmt2],al
    inc     byte [cmmt2]

get_cmt_end:
    call    get2
    cmp     al,lf
    jne     short get_cmt_end ; skip it.

end_commd_line_brdg:
    jmp     end_commd_line ; else jmp to end_commd_line

org_put_zero:
    ; make the filename in front of
    mov     byte [es:di],0 ; the comment string to be an asciiz.
    inc     di
    jmp     end_commd_line ; (maybe null if device=/*)

end_file_slash:
    ; al = "/" option char.
    mov     byte [es:di],0 ; make a filename an asciiz
    inc     di ; and
    stosb ; store "/" after that.
    jmp     short org5 ; continue with the rest of the line

end_copy_file:
    mov     byte [es:di-1],0 ; make it an asciiz and handle the next char.
    cmp     al,lf
    je      short end_commd_line_brdg
    jmp     short org5

org4:
    ; org4 skips all delimiters after the command name except for '/'
    call    skip_comment
    jz      short end_commd_line_brdg

    call    get2
    call    org_delim
    jz      short org4 ; skip delimiters except '/' (mrw 4/88)
    jmp     short org51

org5:
    ; rest of the line
    call    skip_comment ; comment?
    jz      short end_commd_line_brdg
    call    get2 ; not a comment.

org51:
    stosb
    ; copy the character
    cmp     al,'"'; 22h ; a quote ?
    je      short at_quote
    cmp     al,' ' ; 20h
    ja      short org5

    ; 09/09/2023
    ; (Note: PCDOS 7.1 IBMBIO.COM does not contain M051 modification)

    ; M051 - Start

```

```

42498 ; 03/01/2023
42499 ; ds = cs
42500 00003D89 803E[5403]55 cmp byte [cmd_indicator],CONFIG_DEVICEHIGH
42501 ;cmp byte [cs:cmd_indicator],CONFIG_DEVICEHIGH ; Q: is this devicehigh
42502 00003D8E 7514 jne short not_dh ; N:
42503 00003D90 3C0A cmp al,lf ; Q: is this line feed
42504 00003D92 7416 je short org_dhlf ; Y: stuff a blank before the lf
42505 00003D94 3C0D cmp al,cr ; Q: is this a cr
42506 00003D96 75E0 jne short org5 ; N:
42507 00003D98 26C645FF20 mov byte [es:di-1], ' ' ; overwrite cr with blank
42508 00003D9D AA stosb ; put cr after blank
42509 00003D9E FE06[203A] inc byte [insert_blank]
42510 ;inc byte [cs:insert_blank]; indicate that blank has been
42511 ; inserted
42512 00003DA2 EBD4 jmp short org5
42513 not_dh: ; M051 - End
42514
42515 00003DA4 3C0A cmp al,lf ; line feed?
42516 00003DA6 740F je short org1_brdg ; ; handles the next command line.
42517 00003DA8 EBCE jmp short org5 ; handles next char in this line.
42518
42519 org_dhlf: ; M051 - Start
42520 ; 03/01/2023
42521 ; ds = cs
42522 00003DAA 803E[203A]01 cmp byte [insert_blank],1
42523 ;cmp byte [cs:insert_blank],1 ; Q:has a blank already been inserted
42524 00003DAF 7406 je short org1_brdg ; Y:
42525 00003DB1 26C645FF20 mov byte [es:di-1], ' ' ; overwrite lf with blank
42526 00003DB6 AA stosb ; put lf after blank
42527 ; M051 - End
42528
42529 00003DB7 C606[203A]00 org1_brdg:
42530 mov byte [insert_blank],0
42531 ;mov byte [cs:insert_blank],0 ; M051: clear blank indicator for
42532 ; M051: devicehigh
42533 jmp org1
42534
42535 00003DBF 803E[5003]00 at_quote:
42536 cmp byte [com_level],0
42537 00003DC4 7407 ;cmp byte [cs:com_level],0
42538 je short up_level
42539 00003DC6 C606[5003]00 ;mov byte [cs:com_level],0 ; reset it.
42540 00003DCB EBAB mov byte [com_level],0
42541 jmp short org5
42542
42543 up_level:
42544 00003DCD FE06[5003] inc byte [cs:com_level] ; set it.
42545 00003DD1 EBA5 inc byte [com_level]
42546 jmp short org5
42547
42548 ;-----
42549 ; procedure : get2
42550 ;
42551 ;-----
42552
42553 ; 02/11/2022 - Retrodos v4.0 (Modified MSDOS 5.0 IO.SYS)
42554 ; (SYSINIT:33FAh)
42555
42556 ; 03/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
42557 ; (SYSINIT:4270h)
42558 get2:
42559 00003DD3 E304 jcxz noget
42560
42561 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
42562 ; ;lods byte ptr es:[si]
42563 ; 12/12/2022
42564 00003DD5 26 es
42565 00003DD6 AC lodsb
42566 ;mov al, [es:si]
42567 ;inc si
42568
42569 00003DD7 49 dec cx
42570 00003DD8 C3 retn
42571
42572 00003DD9 59 noget:
42573 pop cx
42574 ; 03/01/2023
42575 ; ds = cs
42576 ;mov [cs:count],di ; 13/05/2019
42577 00003DDA 893E[5603] mov [cs:org_count],di
42578 00003DDE 893E[5803] mov [count],di
42579 00003DE2 31F6 mov [org_count],di
42580 xor si,si
42581 00003DE4 8936[5A03] mov [cs:chrptr],si
42582 mov [chrptr],si
42583
42584 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
42585 ; 03/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
42586
42587 ;ifndef MULTI_CONFIG
42588 ; retn
42589 ;else
42590
42591 ; This was the rather kludgy way out of procedure "organize", but instead
42592 ; of returning to doconf, we now want to check config.sys BEGIN/END blocks
42593 ; and the new boot menu stuff...
42594
42595 00003DE8 89F9 mov cx,di
42596 00003DEA E9E300 jmp menu_check
42597
42598 ;endif
42599 ; 02/11/2022
42600 ; 03/01/2023 - Retro DOS v4.2
42601 ;retn
42602
42603 ;-----
42604 ; procedure : skip_comment
42605 ;
42606 ;skip the commented string until lf, if current es:si-> a comment string.
42607 ;in) es:si-> string
42608 ; cx -> length.
42609 ;out) zero flag not set if not found a comment string.
42610 ; zero flag set if found a comment string and skipped it. al will contain
42611 ; the line feed character at this moment when return.
42612 ; ax register destroyed.
42613 ; if found, si, cx register adjusted accordingly.
42614
42615 ;-----
42616
42617 ; 03/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
42618 ; (SYSINIT:428Dh)
42619
42620 skip_comment:
42621

```

```

42622 00003DED E3EA      jcxz    noget          ; get out of the organize routine.
42623
42624      ; 03/01/2023
42625      ; ds = cs
42626
42627 00003DEF 803E[5003]00      cmp     byte [com_level],0
42628      ;cmp    byte [cs:com_level],0 ; only check it if parameter level is 0.
42629 00003DF4 752C      jne     short no_commt ; (not inside quotations)
42630
42631 00003DF6 803E[5103]01      cmp     byte [cmmt],1
42632      ;cmp    byte [cs:cmmt],1
42633 00003DFB 7225      jnb     short no_commt
42634
42635 00003DFD 268A04      mov     al,[es:si]
42636
42637 00003E00 3806[5203]      cmp     [cmmt1],al
42638      ;cmp    [cs:cmmt1],al
42639 00003E04 751C      jne     short no_commt
42640
42641 00003E06 803E[5103]02      cmp     byte [cmmt],2
42642      ;cmp    byte [cs:cmmt],2
42643 00003E0B 750A      jne     short skip_cmmt
42644
42645 00003E0D 268A4401      mov     al,[es:si+1]
42646
42647 00003E11 3806[5303]      cmp     [cmmt2],al
42648      ;cmp    [cs:cmmt2],al
42649 00003E15 750B      jne     short no_commt
42650      skip_cmmt:
42651 00003E17 E3C0      jcxz    noget          ; get out of organize routine.
42652 00003E19 268A04      mov     al,[es:si]
42653 00003E1C 46      inc     si
42654 00003E1D 49      dec     cx
42655 00003E1E 3C0A      cmp     al,1f          ; line feed?
42656 00003E20 75F5      jne     short skip_cmmt
42657      no_commt:
42658 00003E22 C3      retn
42659
42660      ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
42661      ; (SYSINIT:42C8h)
42662
42663      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
42664      ;%if 0
42665
42666      ;ifdef    MULTI_CONFIG
42667
42668      ;-----
42669      ;
42670      ; kbd_read: wait for keystroke
42671      ;
42672      ; INPUT
42673      ; DS == CS == sysinitseg
42674      ;
42675      ; OUTPUT
42676      ; Carry SET to clean boot, CLEAR otherwise
42677      ;
42678      ; OTHER REGS USED
42679      ; All
42680      ;
42681      ; HISTORY
42682      ; Created 16-Nov-1992 by JeffPar
42683      ;
42684      ;-----
42685
42686      kbd_read:
42687 00003E23 F606[7F4C]02      test    byte [bDisableUI],2
42688 00003E28 7520      jnz     short kbd_nodelay
42689
42690 00003E2A 1E      push    ds          ; the bios timer tick count is incremented
42691 00003E2B 29C0      sub     ax,ax        ; 18.2 times per second;
42692 00003E2D 8ED8      mov     ds,ax        ; watch the timer tick count for 37 transitions
42693      ;mov    dx,[046Ch]      ; get initial value
42694      kbd_loop:
42695 00003E2F B401      mov     ah,1          ;
42696 00003E31 CD16      int     16h          ; peek the keyboard
42697 00003E33 7514      jnz     short kbd_loopdone ; something's there, get out
42698 00003E35 B402      mov     ah,2          ; peek the shift states
42699 00003E37 CD16      int     16h          ;
42700 00003E39 A803      test    al,03h        ; either right or left shift key bits set?
42701 00003E3B 750C      jnz     short kbd_loopdone ; yes
42702 00003E3D A16C04      mov     ax,[046Ch] ;
42703      ;sub    ax,dx          ; get difference
42704      ;15/04/2019 - Retro DOS v4.0
42705 00003E40 2E2B06[8C03]      sub     ax,[cs:_timer_lw_] ; MSDOS 6.21 IO.SYS - SYSINIT:42E5h
42706
42707 00003E45 3C25      cmp     al,37          ; reached limit? ; (2 seconds)
42708 00003E47 72E6      jnb     short kbd_loop ; not yet
42709      kbd_loopdone:
42710 00003E49 1F      pop     ds          ; delay complete!
42711      kbd_nodelay:
42712 00003E4A 29DB      sub     bx,bx          ; assume clean boot
42713 00003E4C B402      mov     ah,2          ; peek the shift states
42714 00003E4E CD16      int     16h          ;
42715 00003E50 A803      test    al,03h        ; either right or left shift key bits set?
42716 00003E52 7407      jz      short kbd_notshift ; no
42717 00003E54 43      inc     bx          ; yes
42718 00003E55 43      inc     bx
42719      ; MSDOS 6.21 IO.SYS - SYSINIT:4301h
42720 00003E56 800E[834C]04      or      byte [bQueryOpt],4
42721      kbd_notshift:
42722 00003E5B B401      mov     ah,1          ; peek the keyboard
42723 00003E5D CD16      int     16h          ;
42724 00003E5F 743E      jz      short kbd_test ; no key present
42725 00003E61 08C0      or      al,al          ; is it a function key?
42726 00003E63 753A      jnz     short kbd_test ; no
42727
42728      ; MSDOS 6.21 IO.SYS - SYSINIT:430Bh
42729 00003E65 80FC62      cmp     ah,62h        ; CTRL F5
42730 00003E68 7405      je      short kbd_cfg_bypass
42731
42732 00003E6A 80FC3F      cmp     ah,3Fh        ; F5 function key?
42733 00003E6D 750D      jne     short kbd_notf5 ; no
42734      kbd_cfg_bypass:
42735 00003E6F BA[FA51]      mov     dx,_CleanMsg
42736 00003E72 E8DD0B      call    print
42737      ; MSDOS 6.21 IO.SYS - SYSINIT:431Bh
42738 00003E75 800E[834C]04      or      byte [bQueryOpt],4
42739 00003E7A EB16      jmp     short kbd_eat ; yes, clean boot selected
42740      kbd_notf5:
42741      ; MSDOS 6.21 IO.SYS - SYSINIT:4322h
42742 00003E7C 80FC65      cmp     ah,65h        ; CTRL F8
42743 00003E7F 7405      je      short kbd_cfg_confirm
42744
42745 00003E81 80FC42      cmp     ah,42h        ; F8 function key?

```

```

42746 00003E84 7523          jne short kbd_exit ; no
42747                                kbd_cfg_confirm:
42748                                mov     dx, _$InterMsg
42749                                call    print
42750                                mov     bl, 1
42751                                mov     [bQueryOpt], bl ; yes, interactive-boot option enabled
42752                                ; change default setting
42753 00003E92 B400          kbd_eat:
42754 00003E94 CD16          mov     ah, 0
42755 00003E96 C606[894C]FF  int     16h ; eat the key we assumed was a signal
42756 00003E9B 09DB          mov     byte [secElapsed], -1
42757 00003E9D 7405          or     bx, bx
42758                                jz     short kbd_clean ;
42759 00003E9F 80FB02        kbd_test:
42760 00003EA2 7205          cmp     bl, 2
42761                                jnb    short kbd_exit ;
42762 00003EA4 E86E08        kbd_clean:
42763 00003EA7 F9          call    disable_autoexec ; yes, tell COMMAND to skip autoexec.bat
42764 00003EA8 C3          stc     ; set carry to indicate abort
42765                                retn
42766 00003EA9 F8          kbd_exit:
42767 00003EAA C3          clc     ; clear carry to indicate success
42768                                retn
42769                                ;
42770                                ; -----
42771                                ; set_numlock: set numlock LED
42772                                ;
42773                                ; INPUT
42774                                ; ES:SI -> numlock setting (ie, "ON" or "OFF")
42775                                ;
42776                                ; OUTPUT
42777                                ; None
42778                                ;
42779                                ; OTHER REGS USED
42780                                ; None
42781                                ;
42782                                ; HISTORY
42783                                ; Created 16-Nov-1992 by JeffPar
42784                                ; -----
42785                                ;
42786                                ; 04/01/2023 - Retro DOS v4.2
42787                                ;
42788                                ;
42789                                ; set_numlock:
42790                                ; 04/01/2023
42791                                ; push ax
42792 00003EAB 1E          push    ds
42793 00003EAC 29C0          sub     ax, ax
42794 00003EAE 8ED8          mov     ds, ax
42795 00003EB0 268B04        mov     ax, [es:si] ; get 1st 2 bytes of value (ON or OF)
42796 00003EB3 2E3B06[C251]    cmp     ax, [cs:OnOff+2] ; should we turn it off?
42797 00003EB8 7507          jne     short not_off ; no
42798 00003EBA 80261704DF    and     byte [0417h], ~20h ; 0DFh
42799 00003EBF EB0D          jmp     short set_done
42800                                ;
42801 00003EC1 2E3B06[C051]    not_off:
42802 00003EC6 F9          cmp     ax, [cs:OnOff] ; should we turn it on?
42803 00003EC7 7505          stc
42804 00003EC9 800E170420    jne     short set_done ; no
42805                                or     byte [0417h], 20h
42806 00003ECE 1F          set_done:
42807                                pop     ds
42808                                ; 04/01/2023
42809 00003ECF C3          ; pop ax
42810                                retn
42811                                ; 16/04/2019 - Retro DOS v4.0
42812                                ;
42813                                ; -----
42814                                ; menu_check: check for presence of menu (and other) configuration blocks
42815                                ;
42816                                ; INPUT
42817                                ; CX == "organized" config.sys memory image length
42818                                ; ES:SI -> "organized" config.sys memory image
42819                                ; DS == CS == sysinitseg
42820                                ;
42821                                ; OUTPUT
42822                                ; Same as above; the idea is that menu_check simply transforms
42823                                ; a block-structured config.sys image into a conventional image,
42824                                ; based on the user's block selection and any other boot-time options
42825                                ; the user may have employed...
42826                                ;
42827                                ;
42828                                ; OTHER REGS USED
42829                                ; All
42830                                ;
42831                                ; NOTES
42832                                ; [count] and [org_count] are set to the new config.sys image length
42833                                ;
42834                                ; HISTORY
42835                                ; Created 16-Mar-1992 by JeffPar
42836                                ; -----
42837                                ;
42838                                ; 04/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
42839                                ; (SYSINIT:4378h)
42840                                ;
42841                                ;
42842                                ; menu_check:
42843                                ;
42844                                ; Search for SWITCHES, determine if /N or /F are present; if so, then
42845                                ; disable clean/interactive boot options
42846                                ;
42847 00003ED0 51          push    cx
42848 00003ED1 56          push    si
42849 00003ED2 29DB          sub     bx, bx ; remains ZERO until first block
42850                                ;
42851 00003ED4 E83507        swchk_loop:
42852 00003ED7 724C          call    get_char ; get first char of current line
42853 00003ED9 3C5B          jc     short swchk_end ; hit eof
42854 00003EDB 7503          cmp     al, CONFIG_BEGIN ; '['
42855 00003EDD 43          jne     short swchk_next1 ;
42856 00003EDE EB40          inc     bx ; remember that we've seen a block
42857                                jmp     short swchk_nextline
42858                                ;
42859 00003EE0 3C4E          swchk_next1:
42860 00003EE2 750E          cmp     al, CONFIG_NUMLOCK
42861 00003EE4 09DB          jne     short swchk_next2 ;
42862 00003EE6 7538          or     bx, bx ; only do NUMLOCK commands that exist
42863 00003EE8 E8C0FF          jnz     short swchk_nextline ; before the first block
42864 00003EEB 26C644FF30    call    set_numlock ; REM it out so we don't act on it later, too
42865 00003EF0 EB2E          mov     byte [es:si-1], CONFIG_REM
42866                                jmp     short swchk_nextline
42867                                ;
42868 00003EF2 3C31          swchk_next2:
42869 00003EF4 752A          cmp     al, CONFIG_SWITCHES
42870                                jne     short swchk_nextline ; this line ain't it
42871                                ;
42872 00003EF6 E81307        swchk_scan:
42873                                call    get_char ; look for /N or /F

```

```

42870
42871 00003EF9 3C0A
42872 00003EFB 7423
42873 00003EFD 3C2F
42874 00003EFF 75F5
42875 00003F01 E80807
42876 00003F04 24DF
42877 00003F06 3A06[6123]
42878 00003F0A 7507
42879 00003F0C 800E[7F4C]01
42880 00003F11 EBE3
42881
42882 00003F13 3A06[6D23]
42883 00003F17 75E0
42884 00003F19 800E[7F4C]02
42885 00003F1E EBD6
42886
42887 00003F20 E8C306
42888 00003F23 EBAF
42889
42890 00003F25 5E
42891 00003F26 59
42892
42893
42894
42895
42896 00003F27 F606[7F4C]01
42897 00003F2C 7508
42898
42899
42900
42901
42902 00003F2E E8F2FE
42903 00003F31 7303
42904 00003F33 E9EE01
42905
42906
42907
42908
42909 00003F36 29DB
42910 00003F38 BF[C44C]
42911 00003F3B E80304
42912 00003F3E 7337
42913 00003F40 C606[BC4C]00
42914 00003F45 E90C02
42915
42916
42917
42918
42919 00003F48 51
42920 00003F49 52
42921
42922
42923
42924 00003F4A E89E06
42925 00003F4D 80E30F
42926 00003F50 88DD
42927
42928
42929
42930
42931 00003F52 88DA
42932 00003F54 E83108
42933 00003F57 750E
42934 00003F59 E88F06
42935 00003F5C 80E30F
42936 00003F5F 88DE
42937
42938
42939 00003F61 B104
42940 00003F63 D2E3
42941 00003F65 08DA
42942
42943 00003F67 38F5
42944 00003F69 7503
42945 00003F6B 80F208
42946
42947 00003F6E 8816[7A4C]
42948 00003F72 5A
42949 00003F73 59
42950 00003F74 E9A900
42951
42952
42953
42954
42955
42956 00003F77 C606[844C]01
42957
42958 00003F7C C606[884C]FF
42959 00003F81 8026[834C]FD
42960
42961 00003F86 29D2
42962 00003F88 8916[864C]
42963
42964 00003F8C E85706
42965
42966
42967
42968
42969
42970
42971 00003F8F E87A06
42972 00003F92 722E
42973 00003F94 247F
42974 00003F96 3C5B
42975 00003F98 7428
42976 00003F9A 3C4F
42977 00003F9C 744D
42978 00003F9E 3C45
42979 00003FA0 7449
42980 00003FA2 3C41
42981 00003FA4 741E
42982 00003FA6 3C52
42983 00003FA8 749E
42984 00003FAA 3C4E
42985 00003FAC 740F
42986 00003FAE 3C30
42987 00003FB0 746E
42988 00003FB2 E8C307
42989 00003FB5 7469
42990 00003FB7 F9
42991 00003FB8 E82607
42992 00003FBB EB63
42993

swchk_scan1:
    cmp     al,LF          ; end of line?
    je      short swchk_nextline ; yes
    cmp     al,'/'         ; switch-char?
    jne     short swchk_scan ; no
    call    get_char       ;
    and     al,~20h ; 0DFh ; convert to upper case
    cmp     al,[swit_n+1]   ; 'N'
    jne     short swchk_scan2 ; no
    or      byte [bDisableUI],1
    jmp     short swchk_scan ; continue looking for switches of interest
swchk_scan2:
    cmp     al,[swit_f+1]   ; 'F'
    jne     short swchk_scan1 ; no
    or      byte [bDisableUI],2
    jmp     short swchk_scan ; continue looking for switches of interest
swchk_nextline:
    call    skip_opt_line   ;
    jmp     short swchk_loop ;
swchk_end:
    pop     si              ;
    pop     cx              ;

; Do the keyboard tests for clean/interactive boot now, but only if
; the DisableUI flag is still clear
    test    byte [bDisableUI],1
    jnz     short menu_search

; wait for 2 seconds first, UNLESS the /F bit was set in bDisableUI, or
; there is anything at all in the keyboard buffer
    call    kbd_read
    jnc     short menu_search
    jmp     menu_abort

; Search for MENU block; it is allowed to be anywhere in config.sys
menu_search:
    sub     bx,bx           ; if no MENU, default to zero for no_selection
    mov     di,szMenu       ;
    call    find_block      ; find the MENU block
    jnc     short menu_found ;
    mov     byte [szBoot],0
    jmp     no_selection ; not found

; Process the requested menu color(s)
menu_color:
    push    cx              ;
    push    dx              ;
    ;mov     dx,0007h       ; default color setting
    ; 10/09/2023
    ;mov     dl,7 ; !*!
    call    get_number       ; get first number
    and     bl,0Fh ; !*! ; first # is foreground color (for low nibble)
    mov     ch,bl           ; save it in CH
    ; 01/08/2023 - Retro DOS v4.2 IO.SYS (optimization) by Erdogan Tan
    ; (high nibble of dl is 0)
    ;and     dl,0F0h ; !*! ; (low nibble of dl would be zero)
    ;or      dl,bl         ; (low nibble of dl is 7) ! 14/08/2023
    mov     dl,bl           ; 14/08/2023
    call    delim           ; did we hit a delimiter
    jne     short check_color ; no, all done
    call    get_number       ; get next number
    and     bl,0Fh         ; second # is background color (for high nibble)
    mov     dh,bl          ; save it in DH
    ; 10/09/2023
    ;and     dl,0Fh ; !*! ;
    mov     cl,4            ;
    shl     bl,cl           ;
    or      dl,bl          ;
check_color:
    cmp     ch,dh           ; are foreground/background the same?
    jne     short set_color ; no
    xor     dl,08h         ; yes, so modify the fgnd intensity
set_color:
    mov     [bMenuColor],dl ;
    pop     dx              ;
    pop     cx              ;
    jmp     menu_nextitem

; Back to our regularly scheduled program (the COLOR and other goop
; above is there simply to alleviate short jump problems)
menu_found:
    mov     byte [bDefBlock],1
    ;mov     word [offDefBlock],0
    mov     byte [sectimeOut],-1
    and     byte [bQueryOpt],~2 ; 0FDh
    ; 10/09/2023
    sub     dx,dx
    mov     [offDefBlock],dx ; 0
    call    skip_opt_line   ; skip to next line
    ; 10/09/2023
    sub     dx,dx          ; initialize total block count (0 => none yet)

; Process the menu block now
menu_process:
    call    get_char       ; get first char of current line
    jc      short to_menu_getdefault ; could happen if menu block at end (rare)
    and     al,~CONFIG_OPTION_QUERY ; 7Fh
    cmp     al,CONFIG_BEGIN ; BEGIN implies END
    je      short to_menu_getdefault
    cmp     al,CONFIG_SUBMENU
    je      short menu_item ; go process sub-menu
    cmp     al,CONFIG_MENUITEM
    je      short menu_item ; go process menu item
    cmp     al,CONFIG_MENUDEFAULT
    je      short menu_default ; go process menu default
    cmp     al,CONFIG_MENUCOLOR
    je      short menu_color ; go process menu color
    cmp     al,CONFIG_NUMLOCK
    je      short menu_numlock ;
    cmp     al,CONFIG_REM ; allow remarks in menu block
    je      short menu_nextitem ;
    call    any_delim       ; allow blank lines and such
    je      short menu_nextitem ;
    stc
    call    print_error     ; non-MENU command!
    jmp     short menu_nextitem
menu_numlock:

```

```

42994 00003FBD E8EBFE      call    set_numlock
42995 00003FC0 EB5E      jmp     short menu_nextitem
42996 to_menu_getdefault:
42997 00003FC2 EB62      jmp     short menu_getdefault
42998
42999 ; Save the offset of the default block name, we'll need it later
43000
43001 menu_default:
43002 00003FC4 8936[864C]    mov     [offDefBlock],si ; save address of default block name
43003 00003FC8 803E[894C]00    cmp     byte [secElapsed],0
43004 00003FCD 751A      jne short timeout_skip ; secElapsed is only zero for the FIRST menu,
43005 00003FCF E8EA05    call    skip_token ; and for subsequent menus IF nothing was typed;
43006 00003FD2 724C      jc short menu_nextitem ; secElapsed becomes -1 forever as soon as
43007 00003FD4 E8FB05    call    skip_delim ; something is typed
43008 00003FD7 7247      jc short menu_nextitem ;
43009 00003FD9 89DE      mov     si,bx ;
43010 00003FDB E80D06    call    get_number ; get number (of seconds for timeout)
43011 00003FDE 80FB5A    cmp     bl,90 ; limit it to a reasonable number
43012 ;jb short timeout_ok ; (besides, 99 is the largest # my simple
43013 00003FE1 7602      jna short timeout_ok ; 01/08/2023
43014 00003FE3 B35A      mov     bl,90 ; display function can handle)
43015 timeout_ok:
43016 00003FE5 881E[884C]    mov     [secTimeout],bl ;
43017 timeout_skip:
43018 00003FE9 EB35      jmp     short menu_nextitem
43019
43020 ; Verify that this is a valid menu item by searching for the named block
43021
43022 menu_item:
43023 ;cmp dl,9 ; 04/01/2023
43024 00003FEB 80FA09    cmp     dl,MAX_MULTI_CONFIG ; have we reached the max # of items yet?
43025 00003FEE 7330      jae short menu_nextitem ;
43026 00003FF0 89F7      mov     di,si ; DS:DI -> block name to search for
43027 00003FF2 E83303    call    srch_block ;
43028 00003FF5 7406      je short menu_itemfound ;
43029 00003FF7 F9          stc ;
43030 00003FF8 E8E606    call    print_error ; print error and pause
43031 00003FFB EB23      jmp     short menu_nextitem ; if not found, ignore this menu item
43032
43033 ; srch_block, having succeeded, returns DI -> past the token that it
43034 ; just matched, which in this case should be a descriptive string; ES:SI
43035 ; and CX are unmodified
43036
43037 menu_itemfound:
43038 00003FFD 42          inc     dx ; otherwise, increment total block count
43039 00003FFE 89D3      mov     bx,dx ; and use it to index the arrays of offsets
43040 00004000 8887[8A4C]    mov     [abBlockType+bx],al
43041 00004004 01DB      add     bx,bx ; of recorded block names and descriptions
43042
43043 ; There should be a description immediately following the block name on
43044 ; MENUITEM line; failing that, we'll just use the block name as the
43045 ; description...
43046
43047 00004006 89B7[944C]    mov     [aoffBlockName+bx],si
43048 0000400A 89B7[A84C]    mov     [aoffBlockDesc+bx],si
43049 0000400E 89DF      mov     di,bx ; skip_delim modifies BX, so stash it in DI
43050 00004010 E8A905    call    skip_token ;
43051 00004013 720B      jc short menu_nextitem ; hit eol/eof
43052 00004015 E8BA05    call    skip_delim ;
43053 00004018 7206      jc short menu_nextitem ; hit eol/eof
43054 0000401A 87DF      xchg    bx,di ;
43055 0000401C 89BF[A84C]    mov     [aoffBlockDesc+bx],di
43056
43057 menu_nextitem:
43058 00004020 E8C305    call    skip_opt_line ;
43059 00004023 E969FF      jmp     menu_process ; go back for more lines
43060
43061 ; Display menu items now, after determining which one is default
43062
43063 menu_getdefault:
43064 00004026 08D2      or      dl,dl ; where there any valid blocks at all?
43065 00004028 7505      jnz short menu_valid ; yes
43066 0000402A 29DB      sub     bx,bx ; no, so force autoselect of 0
43067 0000402C E9ED00      jmp     menu_autoselect ; (meaning: process common blocks only)
43068
43069 menu_valid:
43070 0000402F 29DB      sub     bx,bx ;
43071 00004031 8816[854C]    mov     [bMaxBlock],dl ; first, record how many blocks we found
43072 00004033 8B3E[864C]    mov     di,[offDefBlock] ;
43073 00004035 09FF      or      di,di ; does a default block exist?
43074 00004037 741C      jz short menu_noddefault ; no
43075 00004039 43          inc     bx ; yes, walk name table, looking for default
43076 menu_chkdefault:
43077 0000403E 53          push    bx ;
43078 0000403F 01DB      add     bx,bx ;
43079 00004041 8BB7[944C]    mov     si,[aoffBlockName+bx]
43080 00004043 B98000      mov     cx,128 ; arbitrary maximum length of a name
43081 00004045 1E          push    ds ;
43082 00004047 06          push    es ;
43083 00004049 1F          pop     ds ;
43084 0000404B E81A03      call    comp_names ; is this block the same as the default?
43085 0000404D 1F          pop     ds ;
43086 0000404F 5B          pop     bx ;
43087 00004051 7409      je short menu_setdefault ; yes
43088 00004053 43          inc     bx ;
43089 00004055 3A1E[854C]    cmp     bl,[bMaxBlock] ; all done searching?
43090 00004057 76E5      jbe short menu_chkdefault ; not yet
43091 menu_noddefault:
43092 00004059 B301      mov     bl,1 ; if no default, force default to #1
43093 menu_setdefault:
43094 0000405B 881E[844C]    mov     [bDefBlock],bl ; yes, this will be the initial current block
43095
43096 ; If the timeout was explicitly set to 0 (or technically, anything that
43097 ; failed to resolve to a number, like "NONE" or "EAT POTATOES"), then we're
43098 ; supposed to skip menu display and run with the specified default block;
43099 ; however, if the user hit Enter prior to boot, thereby requesting fully
43100 ; INTERACTIVE boot, then we shall display the menu block anyway (though still
43101 ; with no timeout)
43102 0000405F 803E[884C]00    cmp     byte [secTimeout],0 ; is timeout zero? (ie, assume default)
43103 00004064 750A      jne short menu_display ; no
43104 00004066 F606[834C]01    test    byte [bQueryOpt],1 ; yes, but was INTERACTIVE requested?
43105 00004068 7503      jnz short menu_display ; yes, so *don't* assume default after all
43106 0000406D E9C700      jmp     not_topmenu ;
43107
43108 ; Reset the mode, so that we know screen is clean and cursor is home
43109
43110 menu_display:
43111 00004070 B40F      mov     ah,0Fh ; get current video mode
43112 00004072 CD10      int     10h ;
43113 00004074 B400      mov     ah,00h ; just re-select that mode
43114 00004076 CD10      int     10h ;
43115 00004078 06          push    es ;
43116 00004079 B84000      mov     ax,40h ; reach down into the ROM BIOS data area
43117 0000407C 8EC0      mov     es,ax ; and save the current (default) video page

```



```

43118 0000407E 26A14E00      mov     ax,[es:004Eh] ; start address and page #, in case the
43119 00004082 A3[814C]      mov     [wCRTStart],ax ; undocumented QUIET option was enabled
43120 00004085 26A06200      mov     al,[es:0062h] ;
43121 00004089 A2[804C]      mov     [bCRTPage],al ;
43122 0000408C A1[7B4C]      mov     ax,[bMenuPage] ; select new page for menu
43123 0000408F CD10      int     10h ;
43124 00004091 B80006      mov     ax,0600h ; clear entire screen
43125 00004094 8A3E[7A4C]      mov     bh,[bMenuColor] ; using this color
43126 00004098 29C9      sub     cx,cx ; upper left row/col
43127 ;mov     dl,[es:CRT_Cols]
43128 0000409A 268A164A00      mov     dl,[es:4Ah]
43129 0000409F FECA      dec     dl ;
43130 ;mov     dh,[es:CRT_Rows];
43131 000040A1 268A368400      mov     dh,[es:84h]
43132 000040A6 08F6      or      dh,dh ; # of rows valid?
43133 000040A8 7504      jnz     short menu_clear ; hopefully
43134 000040AA 8A36[7E4C]      mov     dh,[bLastRow] ; no, use a default
43135 menu_clear:
43136 000040AE CD10      int     10h ; clear the screen using the req. attribute
43137 000040B0 07      pop     es ;
43138 000040B1 8836[7E4C]      mov     [bLastRow],dh ; save DH
43139 000040B5 BA[7552]      mov     dx,$MenuHeader
43140 000040B8 E89709      call    print ; cursor now on row 3 (numbered from 0)
43141
43142 000040BB F606[7F4C]01      test     byte [bDisableUI],1
43143 000040C0 751F      jnz     short menu_nostatus
43144 000040C2 8A3E[7B4C]      mov     bh,[bMenuPage] ;
43145 000040C6 8A36[7E4C]      mov     dh,[bLastRow] ; restore DH
43146 000040CA B200      mov     dl,0 ; print the status line on row DH, col 0,
43147 000040CC B402      mov     ah,02h ; now that we can trash the cursor position
43148 000040CE CD10      int     10h ;
43149 000040D0 BA[C152]      mov     dx,$StatusLine
43150 000040D3 E87C09      call    print
43151 000040D6 B403      mov     ah,3 ; get cursor position
43152 000040D8 CD10      int     10h ;
43153 000040DA 80EA02      sub     dl,2 ;
43154 000040DD 8816[7D4C]      mov     [bLastCol],dl ; save column where status char will go
43155
43156 menu_nostatus:
43157 000040E1 BB0100      mov     bx,1 ; now prepare to display all the menu items
43158 menu_displloop:
43159 000040E4 E8B002      call    print_item; print item #BL
43160 000040E7 43      inc     bx ; why "inc bx"? because it's a 1-byte opcode
43161 000040E8 3A1E[854C]      cmp     bl,[bMaxBlock] ; all done?
43162 000040EC 76F6      jbe     short menu_displloop ; not yet
43163
43164 ; Set cursor position to just below the menu items
43165
43166 000040EE B200      mov     dl,0 ; select column
43167 000040F0 88DE      mov     dh,b1 ;
43168 000040F2 80C604      add     dh,4 ; select row below menu
43169 000040F5 8A3E[7B4C]      mov     bh,[bMenuPage] ;
43170 000040F9 B402      mov     ah,02h ; set cursor position beneath the block list
43171 000040FB CD10      int     10h ;
43172
43173 000040FD BA[AE52]      mov     dx,$MenuPrmpt
43174 00004100 E84F09      call    print ;
43175 00004103 E82903      call    select_item ; make a selection, return # in BX
43176 00004106 BA[6E50]      mov     dx,crlfm ;
43177 00004109 E84609      call    print ;
43178 0000410C FF36[7F4C]      push     word [bDisableUI]
43179 00004110 800E[7F4C]01      or      byte [bDisableUI],1
43180 00004115 E86704      call    show_status ; clear the status line now
43181 00004118 8F06[7F4C]      pop     word [bDisableUI]
43182
43183 ; Now begins the "re-organization" process...
43184
43185 menu_autoselect:
43186 0000411C 83FBFF      cmp     bx,-1 ; 0FFFFh ; clean boot requested?
43187 0000411F 7508      jne     short normal_boot ; no
43188 00004121 E8F105      call    disable_autoexec; basically, add a /D to the command.com line
43189
43190 00004124 29C9      sub     cx,cx ; then immediately exit with 0 config.sys image
43191 00004126 E9E400      jmp     menu_exit ;
43192
43193 normal_boot:
43194 00004129 83FBFE      cmp     bx,-2 ; 0FFFEh ; back to top-level menu?
43195 0000412C 7509      jne     short not_topmenu ; no
43196 0000412E 8B0E[5603]      mov     cx,[count] ; yes, start all over
43197 00004132 29F6      sub     si,si ;
43198 00004134 E9FFFD      jmp     menu_search
43199
43200 not_topmenu:
43201 00004137 80BF[8A4C]4F      cmp     byte [abBlockType+bx],CONFIG_SUBMENU
43202 0000413C 7510      jne     short not_submenu
43203 0000413E 01DB      add     bx,bx ;
43204 00004140 8BBF[944C]      mov     di,[aoffBlockName+bx]
43205 00004144 E8E101      call    srch_block ; THIS CANNOT FAIL!
43206 00004147 89FE      mov     si,di ;
43207 00004149 89D9      mov     cx,bx ; ES:SI and CX are ready for another round
43208 0000414B E929FE      jmp     menu_found
43209
43210 not_submenu:
43211 0000414E 01DB      add     bx,bx ; get BX -> name of selected block
43212 00004150 8B9F[944C]      mov     bx,[aoffBlockName+bx]
43213
43214 ; BX should now either be ZERO (meaning no block has been selected) or
43215 ; the offset relative to ES of the block name to be processed (along with
43216 ; all the "common" lines of course)
43217
43218 no_selection:
43219 00004154 891E[864C]      mov     [offDefBlock],bx; save selection
43220 00004158 8B0E[5603]      mov     cx,[count] ; reset ES:SI and CX for reprocessing
43221 0000415C 29F6      sub     si,si ;
43222 0000415E 1E      push     ds ;
43223 0000415F 8E1E[6219]      mov     ds,[config_wrkseg]; this is where we'll store new config.sys image
43224 00004163 29FF      sub     di,di ;
43225
43226 ; ES:SI-> config.sys, DS:DI -> new config.sys workspace
43227 ;
43228 ; work our way through the config.sys image again, this time copying
43229 ; all lines that are (A) "common" lines outside any block or (B) lines
43230 ; within the requested block. Lines inside INCLUDED blocks are transparently
43231 ; copied by copy_block in a recursive fashion; the amount of recursion is
43232 ; limited by the fact INCLUDE statements are REMED by copy_block as they are
43233 ; processed and by the number of unique INCLUDE stmts in config.sys...
43234 ;
43235 ; BUGBUG 20-Mar-1992 JeffPar: If we can figure out the lower bound of the
43236 ; stack we're running on, then we should check it inside copy_block
43237
43238 copyblock_loop:
43239 00004165 53      push     bx ; save selected block name
43240 00004166 E82F01      call    copy_block ; process (named or common) block
43241 00004169 5B      pop     bx ;

```

```

43242 0000416A 7232          jc short move_config ; hit eof
43243
43244          ; copy_block can only return for two reasons: it hit eof or a new block
43245
43246 copyblock_begin:
43247
43248          ; 10/09/2023
43249          %if 0
43250          push    ax          ;
43251          push    cx          ;
43252          push    si          ;
43253          push    di          ; always do "common" blocks
43254          mov     di,szCommon
43255          push    ds          ;
43256          push    cs          ;
43257          pop     ds          ;
43258          call    comp_names  ;
43259          pop     ds          ;
43260          pop     di          ;
43261          pop     si          ;
43262          pop     cx          ;
43263          pop     ax          ;
43264          je     short copyblock_check
43265          %endif
43266          ; 10/09/2023
43267 0000416C 57          push    di
43268 0000416D BF[C94C]    mov     di,szCommon ; always do "common" blocks
43269 00004170 E81602    call    comp_names_x ; (comp_names_safe)
43270 00004173 5F          pop     di
43271 00004174 740F          je     short copyblock_check
43272
43273 00004176 09DB          or      bx,bx          ; is there a block name to check?
43274 00004178 7414          jz     short copyblock_skip ; no
43275 0000417A 57          push    di
43276 0000417B 89DF          mov     di,bx          ; check block against given block name
43277 0000417D 1E          push    ds
43278 0000417E 06          push    es
43279 0000417F 1F          pop     ds
43280 00004180 E8E501    call    comp_names      ; is this the block we really want to do?
43281 00004183 1F          pop     ds
43282 00004184 5F          pop     di
43283
43284 00004185 7217          copyblock_check:
43285 00004187 7505          jc     short move_config ; hit eof
43286 00004189 E85A04    jne     short copyblock_skip ;
43287 0000418C EBD7          call    skip_opt_line ;
43288          jmp     short copyblock_loop
43289
43290 0000418E E85504          copyblock_skip:
43291 00004191 E87804          call    skip_opt_line ; this ain't the block we wanted, so skip it
43292 00004194 7208          call    get_char ;
43293 00004196 247F          jc     short move_config ; hit eof
43294 00004198 3C5B          and     al,~CONFIG_OPTION_QUERY ; 7Fh
43295 0000419A 74D0          cmp     al,CONFIG_BEGIN ;
43296 0000419C EBF0          je     short copyblock_begin
43297          jmp     short copyblock_skip ; anything else is just skipped
43298
43299          ; To create as little risk to the rest of SysInit as little as possible,
43300          ; and to free the workspace at "config_wrkseg" for creating an environment,
43301          ; copy the new config.sys image to "confbot"
43302
43303 0000419E 89F9          move_config:
43304 000041A0 51          mov     cx,di          ; now copy workspace at DS:DI to "confbot"
43305          push    cx          ;
43306
43307          ; But first, copy the CONFIG=<configuration><0> string to the workspace,
43308          ; since the configuration name only currently exists in the "confbot" area
43309
43310 000041A1 B90700          ;mov     cx,7
43311 000041A4 BE[BC4C]    mov     cx,szMenu-szBoot-1
43312 000041A7 47          mov     si,szBoot ; first copy the CONFIG= part
43313          inc     di          ; skip a byte, in case absolutely nothing
43314          ; was copied to the workspace, because we always
43315          ; zero the first byte of the workspace (below)
43316
43317 000041A8 2E          copy_boot:
43318 000041A9 AC          ;lods     byte ptr cs:[si];
43319 000041AA 8805          lodsb
43320 000041AC 47          mov     [di],al
43321 000041AD E2F9          inc     di
43322          loop    copy_boot
43323
43324 000041AF 06          push    es
43325          ;mov     cx,128-7 ; then copy the configuration name
43326          ; 04/01/2023 ; put an upper limit on the name, to be safe
43327 000041B2 2E8B36[864C]    mov     cl,128-7
43328 000041B7 09F6          mov     si,[cs:offDefBlock]; ES:SI -> default block name
43329 000041B9 7505          or      si,si          ; valid?
43330 000041BB 0E          jnz     short l1 ; yes
43331 000041BC 07          push    cs
43332 000041BD BE[C94C]    pop     es
43333 000041C0 268A04    mov     si,szCommon
43334 000041C3 E8B205    mov     al,[es:si]
43335 000041C6 7406          call    any_delim
43336 000041C8 8805          je     short l2
43337 000041CA 46          mov     [di],al
43338 000041CB 47          inc     si
43339 000041CC E2F2          inc     di
43340 000041CE C6050A    loop    l1
43341 000041D1 07          mov     byte [di],1f ; terminate the configuration string
43342          pop     es
43343
43344          ; Now we can copy "config_wrkseg" (DS) to "confbot" (ES)
43345 000041D2 29FF          sub     di,di
43346 000041D4 2E893E[6019]    mov     [cs:config_envlen],di
43347 000041D9 29F6          sub     si,si
43348 000041DB 59          pop     cx ; recover the size of "config_wrkseg"
43349
43350 000041DC 51          push    cx
43351 000041DD F3A4          rep     movsb ; moved!
43352 000041DF 59          pop     cx
43353 000041E0 8CD8          mov     ax,ds
43354 000041E2 1F          pop     ds
43355
43356          ; Now that the config_wrkseg is available once again, we shall
43357          ; use it to create an environment. The first thing to go in will be
43358          ; the "CONFIG=configuration" thing. It is also important to zero
43359          ; the first byte of the workspace, so that copy_envvar knows the buffer
43360          ; is empty.
43361
43362 000041E3 06          push    es
43363 000041E4 8EC0          mov     es,ax
43364 000041E6 46          inc     si
43365 000041E7 26C606000000    mov     byte [es:0],0 ; ES:SI -> "CONFIG=configuration"
43366          ; empty the environment block

```



```

43366 000041ED E82600      call    copy_envvar      ; copy envvar at ES:SI to "config_wrkseg"
43367 000041F0 07          pop     es
43368
43369 ; Before returning, restore the default video page setting but do NOT
43370 ; do it using INT 10h's Set Active Page function, because if the menu was
43371 ; displayed on a different page, then it's because we don't want to see
43372 ; all the device driver/TSR goop (which goes to the default page)
43373
43374 menu_done:
43375 000041F1 803E[7B4C]00    cmp     byte [bMenuPage],0
43376 000041F6 7415          je      short menu_exit
43377 000041F8 06          push    es
43378 000041F9 B84000          mov     ax,40h
43379 000041FC 8EC0          mov     es,ax
43380 000041FE A1[814C]        mov     ax,[wCRTStart]
43381 00004201 26A34E00       mov     [es:004Eh],ax
43382 00004205 A0[804C]        mov     al,[bCRTPage]
43383 00004208 26A26200       mov     [es:0062h],al
43384 0000420C 07          pop     es
43385
43386 0000420D 890E[5603]       mov     [count],cx      ; set new counts
43387 00004211 890E[5803]       mov     [org_count],cx
43388 ; 10/09/2023 (*) - Erdogan Tan
43389 ; MSDOS 6.21 IO.SYS - SYSINIT:46D3h
43390 ; PC DOS 7.1 IBMBIO.COM - SYSINIT:491Ah
43391 ; sub si,si      ; always return ES:SI pointing to config.sys
43392 00004215 C3          retn
43393
43394 ; (*) NOTE: MSDOS 6.0 source code (SYSINIT2.ASM) contains 'sub si,si' at this
43395 ; position (then 'retn' just after it)
43396 ; but MSDOS 6.21 and PC DOS 7.1 SYSINITs contain only 'retn' here.
43397
43398 -----
43399 ;
43400 ; copy_envvar: copy the envvar at ES:SI to "config_wrkseg"
43401 ;
43402 ; INPUT
43403 ; ES:SI -> environment variable (in the form "var=string<cr/lf>")
43404 ;
43405 ; OUTPUT
43406 ; config_envlen (ie, where to put next envvar) updated appropriately
43407 ; carry set if error (eg, missing =); clear otherwise
43408 ;
43409 ; OTHER REGS USED
43410 ; None
43411 ;
43412 ; NOTES
43413 ; None
43414 ;
43415 ; HISTORY
43416 ; Created 29-Mar-1992 by JeffPar
43417 ;
43418 -----
43419
43420 ; 04/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
43421 ; (SYSINIT:46D4h)
43422
43423 copy_envvar:
43424 00004216 51          push    cx
43425 00004217 56          push    si
43426 00004218 1E          push    ds
43427 00004219 06          push    es
43428 0000421A 06          push    es
43429 0000421B 8E06[6219]     mov     es,[config_wrkseg] ; ES:DI to point to next available byte
43430 0000421F 1F          pop     ds      ; DS:SI to point to envvar
43431
43432 ; Have to calculate the length of the variable name (and if we hit
43433 ; the end of the line before we hit '=', then it's curtains for this
43434 ; config.sys line)
43435 ;
43436 ; The check for NULL is important because copy_envvar is also used to copy
43437 ; the initial CONFIG= setting, which will have been zapped by a NULL if no
43438 ; menu block existed (in order to prevent the creation of an environment)
43439
43440 00004220 29C9          sub     cx,cx
43441
43442 00004222 AC          copy_varlen:
43443 00004223 08C0          lodsb
43444 ; or al,al      ; NULL?
43445 ; stc          ; 10/09/2023 (x)
43446 00004225 746B          jz      short copy_envexit ; yes, abort
43447 00004227 3C0D          cmp     al,cr
43448 ; stc          ; 10/09/2023 (x)
43449 00004229 7467          je      short copy_envexit
43450 0000422B 3C0A          cmp     al,lf
43451 ; stc          ; 10/09/2023 (x)
43452 0000422D 7463          je      short copy_envexit
43453 0000422F 41          inc     cx
43454 00004230 3C3D          cmp     al,'='
43455 00004232 75EE          jne     short copy_varlen
43456 00004234 B000          mov     al,0
43457 00004236 8A24          mov     ah,[si]      ; save char after '='
43458 00004238 29CE          sub     si,cx        ; back up to given varname
43459 0000423A 49          dec     cx          ; CX == # of bytes in varname
43460 0000423B 29FF          sub     di,di        ; start looking for DS:SI at ES:0
43461
43462 0000423D 263805       copy_varsrch:
43463 00004240 7425          cmp     byte [es:di],al
43464 00004242 89FB          je      short copy_envprep ; search failed, just copy var
43465 00004244 51          mov     bx,di      ; ES:BX -> start of this varname
43466 00004245 56          push    cx
43467 00004246 F3A6          push    si
43468 00004248 5E          repe    cmpsb
43469 0000424A 59          pop     si
43470 0000424C 7531          pop     cx
43471 0000424E 26803D3D     jne     short copy_varnext ; no match, skip to next varname
43472 00004250 752B          cmp     byte [es:di],'='
43473 ; jne short copy_varnext ; no match, there's more characters
43474
43475 ; Previous occurrence of variable has been found; determine the
43476 ; entire length and then destroy it
43477
43478 00004252 B9FFFF          mov     cx,-1
43479 00004255 F2AE          repne   scasb        ; guaranteed to get null (since we put it there)
43480 00004257 56          push    si
43481 00004258 89FE          mov     si,di
43482 0000425A 89DF          mov     di,bx
43483 0000425C 2E8B0E[6019]  mov     cx,[cs:config_envlen]
43484 00004261 29F1          sub     cx,si        ; destroy variable now
43485 ; rep movs byte ptr es:[di],byte ptr es:[si]
43486 ; ;db 0F3h,26h,0A4h ; MSDOS 6.21 IO.SYS - SYSINIT:4724h
43487
43488 00004263 F3          rep     ; 0F3h
43489 00004264 26          es     ; 26h
43490 00004265 A4          movsb   ; 0A4h

```

```

43490 00004266 5E      pop     si
43491 copy_envprep:
43492      cmp     ah,cr      ; if there is nothing after the '='
43493      je      short copy_envdel ; then just exit with variable deleted
43494      cmp     ah,lf      ;
43495      je      short copy_envdel
43496      jmp     short copy_envloop
43497      ; 04/01/2023
43498 copy_envloop:
43499      lodsb      ;
43500      cmp     al,cr      ;
43501      je      short copy_envdone
43502      cmp     al,lf      ;
43503      je      short copy_envdone
43504      stosb      ;
43505      jmp     short copy_envloop
43506
43507 copy_varnext:
43508      push     cx      ;
43509      mov     cx,-1      ;
43510      repne    scasb     ;
43511      pop     cx      ;
43512      jmp     short copy_varsrch
43513
43514      ; 04/01/2023
43515 ;copy_envloop:
43516      lodsb      ;
43517      cmp     al,cr      ;
43518      je      short copy_envdone
43519      cmp     al,lf      ;
43520      je      short copy_envdone
43521      stosb      ;
43522      jmp     short copy_envloop
43523
43524 copy_envdone:
43525      sub     al,al      ; do SUB to clear carry as well
43526      stosb      ; always null-terminate these puppies
43527 copy_envdel:
43528      mov     [es:di],al ; and stick another null to terminate the env.
43529      mov     [cs:config_envlen],di
43530      ; 10/09/2023 (x) - Erdogan Tan
43531      stc      ; in order to clear carry flag via cmc (compact code trick!)
43532 copy_envexit:
43533      cmc     ; (X) ; reverse carry flag status (je -> cf=1)
43534      pop     es      ;
43535      pop     ds      ;
43536      pop     si      ;
43537      pop     cx      ;
43538
43539 copy_done: ; 18/12/2022
43540      retn
43541
43542 ;-----
43543 ;
43544 ; copy_block: copy the current block to the new config.sys workspace
43545 ;
43546 ; INPUT
43547 ; CX == remaining bytes in "organized" config.sys memory image
43548 ; ES:SI -> remaining bytes in "organized" config.sys memory image
43549 ; DS:DI -> new config.sys workspace (equal in size to the original
43550 ;         config.sys image) where the current block is to be copied
43551 ;
43552 ; OUTPUT
43553 ; Same as above
43554 ; AL also equals the last character read from the organized image
43555 ;
43556 ; OTHER REGS USED
43557 ; All
43558 ;
43559 ; NOTES
43560 ; None
43561 ;
43562 ; HISTORY
43563 ; Created 16-Mar-1992 by JeffPar
43564 ;-----
43565
43566      ; 04/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
43567      ; (SYSINIT:4759h)
43568
43569 copy_block:
43570      call     get_char      ; check for include
43571      jc      short copy_done
43572      and     al,~CONFIG_OPTION_QUERY ; 7Fh
43573      cmp     al,CONFIG_BEGIN ; another BEGIN implies END as well
43574      je      short copy_done
43575
43576      cmp     al,CONFIG_INCLUDE ; 'J'
43577      mov     al,ah      ; AL == the original line code
43578      jne     short copy_line ; not an "include" line
43579
43580 ; We have hit an "INCLUDE" line; first, REM out the line so that we
43581 ; never try to include the block again (no infinite include loops please),
43582 ; then search for the named block and call copy_block again.
43583
43584      mov     byte [es:si-1],CONFIG_REM ; '0'
43585      push     di      ;
43586
43587      mov     di,szMenu
43588      call    comp_names_safe ; don't allow INCLUDE MENU
43589      je      short copy_skip
43590
43591      mov     di,szCommon
43592      call    comp_names_safe ; don't allow INCLUDE COMMON
43593      je      short copy_skip
43594
43595      mov     di,si      ; try to find the block
43596      call    srch_block
43597      mov     dx,di      ;
43598
43599      ; 10/09/2023
43600      pop     di
43601      jne     short copy_error ; no such block
43602      pop     di ; 10/09/2023
43603      push     cx      ;
43604      mov     cx,bx      ;
43605      push     si      ;
43606      dec     dx      ;
43607      mov     si,dx      ;
43608      call    skip_line ; skip the rest of the "block name" line
43609      call    copy_block ; and copy in the rest of that block
43610      pop     si      ;
43611      pop     cx      ;
43612      sub     al,al      ; force skip_opt_line to skip...
43613      jmp     short copy_nextline

```

```

43614
43615
43616
43617 000042DC F8
43618
43619 000042DD 5F
43620
43621
43622
43623 000042DE E80004
43624 000042E1 EB24
43625
43626
43627
43628
43629 000042E3 8805
43630 000042E5 47
43631 000042E6 3C20
43632 000042E8 721D
43633 000042EA 2E803E[6519]00
43634 000042F0 7409
43635 000042F2 E81700
43636 000042F5 891D
43637 000042F7 47
43638 000042F8 47
43639 000042F9 EB08
43640
43641 000042FB E80E03
43642 000042FE 7297
43643 00004300 8805
43644 00004302 47
43645
43646 00004303 3C0A
43647 00004305 75F4
43648
43649
43650 00004307 E8DC02
43651 0000430A EB8C
43652
43653
43654
43655
43656
43657
43658
43659
43660
43661
43662
43663
43664
43665
43666
43667
43668
43669
43670
43671
43672
43673
43674
43675
43676
43677
43678
43679 0000430C 50
43680 0000430D 29DB
43681 0000430F 51
43682 00004310 89F2
43683 00004312 56
43684 00004313 2E8B0E[5603]
43685 00004318 29F6
43686
43687 0000431A E8C402
43688 0000431D 7205
43689 0000431F 43
43690 00004320 39D6
43691 00004322 72F6
43692
43693 00004324 5E
43694 00004325 59
43695 00004326 58
43696 00004327 C3
43697
43698
43699
43700
43701
43702
43703
43704
43705
43706
43707
43708
43709
43710
43711
43712
43713
43714
43715
43716
43717
43718
43719
43720
43721
43722
43723
43724
43725
43726 00004328 50
43727 00004329 51
43728 0000432A 2E8B0E[5603]
43729 0000432F 56
43730 00004330 29F6
43731 00004332 1E
43732 00004333 06
43733 00004334 1F
43734 00004335 E80900
43735 00004338 89F7
43736 0000433A 89CB
43737 0000433C 1F

copy_error:
; 10/09/2023
clc
copy_skip:
pop di
;copy_error:
; 10/09/2023 (cf=0)
;clc
call print_error ; note that carry is clear, no pause
jmp short copy_nextline

; Copy the line at ES:SI to the current location at DS:DI

copy_line:
mov [di],al ;
inc di ;
cmp al,' ' ; is this is a "real" line with a "real" code?
jb short copy_nextline ; no
cmp byte [cs:config_multi],0 ;
je short copy_loop ; not a multi-config config.sys, don't embed #s
call get_linenum ; BX == line # of line @ES:SI
mov [di],bx ; stash it immediately following the line code
inc di ;
inc di ;
jmp short copy_next ;

copy_loop:
call get_char ;
jc short copy_done ; end of file
mov [di],al ;
inc di ;

copy_next:
cmp al,lf ; 0Ah ; done with line?
jne short copy_loop ; nope

copy_nextline:
call skip_opt_line ;
jmp short copy_block

; 18/12/2022
;copy_done:
;retn

;-----
;
; get_linenum: return line # (in BX) of current line (@ES:SI)
;
; INPUT
; ES:SI -> some line in the config.sys memory image
;
; OUTPUT
; BX == line # (relative to 1)
;
; OTHER REGS USED
; DX
;
; NOTES
; None
;
; HISTORY
; Created 16-Mar-1992 by JeffPar
;-----

get_linenum:
push ax ;
sub bx,bx ; BX == line # (to be returned)
push cx ;
mov dx,si ; DX == the offset we're looking for
push si ;
mov cx,[cs:count] ;
sub si,si ; prepare to scan entire file

get_linenum_loop:
call skip_line ;
jc short get_linenum_done
inc bx ;
cmp si,dx ; have we exceeded the desired offset yet?
jb short get_linenum_loop ; no

get_linenum_done:
pop si ;
pop cx ;
pop ax ;
retn ;

;-----
;
; srch_block: searches entire config.sys for block name @ES:DI
;
; INPUT
; ES -> config.sys image
; ES:DI -> block name to find
;
; OUTPUT
; ZF flag set, if found
; ES:DI -> just past the name in the block heading, if found
; BX == # bytes remaining from that point, if found
;
; OTHER REGS USED
; None
;
; NOTES
; This differs from "find_block" in that it searches the ENTIRE
; config.sys image, not merely the remaining portion, and that it
; takes a pointer to block name that is *elsewhere* in the image
; (ie, ES) as opposed to some string constant in our own segment (DS).
;
; HISTORY
; Created 16-Mar-1992 by JeffPar
;-----

srch_block:
; returns BX -> named block in CONFIG.SYS
push ax ;
push cx ;
mov cx,[cs:count] ;
push si ;
sub si,si ;
push ds ;
push es ;
pop ds ;
call find_block ;
mov di,si ;
mov bx,cx ;
pop ds ;

```

```

43738 0000433D 5E      pop     si      ;
43739 0000433E 59      pop     cx      ;
43740 0000433F 58      pop     ax      ;
43741 find_exit: ; 16/04/2019
43742 00004340 C3      ret     ;
43743
43744 ;-----
43745 ;
43746 ;   find_block:  searches rest of config.sys for block name @DS:DI
43747 ;
43748 ; INPUT
43749 ;   DS:DI -> block name to find
43750 ;   ES:SI -> remainder of config.sys image
43751 ;   CX == remaining size of config.sys image
43752 ;
43753 ; OUTPUT
43754 ;   ZF flag set, if found (also, CF set if EOF)
43755 ;   ES:SI -> where the search stopped (at end of block name or EOF)
43756 ;   CX == # bytes remaining from that point
43757 ;
43758 ; OTHER REGS USED
43759 ;   AX
43760 ;
43761 ; NOTES
43762 ;   This differs from "srch_block" in that it searches only the
43763 ;   remaining portion of the config.sys image and leaves SI and CX
43764 ;   pointing to where the search left off, and that it takes a pointer
43765 ;   to search string in our own segment (DS:DI instead of ES:DI).
43766 ;
43767 ; HISTORY
43768 ;   Created 16-Mar-1992 by JeffPar
43769 ;
43770 ;-----
43771
43772 find_block:
43773 00004341 E8C802    call    get_char      ; get line code
43774 00004344 72FA      jc     short find_exit ; end of file
43775 00004346 247F      and     al,~CONFIG_OPTION_QUERY
43776 00004348 3C5B      cmp     al,CONFIG_BEGIN ; beginning of a block?
43777 0000434A 740C      je     short check_line ; no
43778 0000434C 3C4A      cmp     al,CONFIG_INCLUDE
43779 0000434E 7513      jne    short next_line ;
43780 00004350 2E800E[6519]01 or     byte [cs:config_multi],1
43781 00004356 EB0B      jmp     short next_line ;
43782
43783 00004358 2E800E[6519]01 or     byte [cs:config_multi],1
43784 0000435E E80700    call    comp_names      ; compare block names
43785 00004361 76DD      jbe    short find_exit ; end of file, or names matched
43786
43787 00004363 E88002    call    skip_opt_line   ; no, so skip to next line
43788 00004366 EBD9      jmp     short find_block ;
43789 find_exit:
43790 ;   ret     ;
43791 ;
43792 ;-----
43793 ;
43794 ;   comp_names:  compares keyword @DS:DI to position in config.sys @ES:SI
43795 ;
43796 ; INPUT
43797 ;   DS:DI -> keyword to compare
43798 ;   ES:SI -> position in config.sys
43799 ;   CX == remaining bytes in config.sys
43800 ;
43801 ; OUTPUT
43802 ;   ZF flag set, if match (also, CF set if EOF)
43803 ;   ES:SI -> where the comparison stopped (at end of block name or EOF)
43804 ;   CX == # bytes remaining from that point
43805 ;
43806 ; OTHER REGS USED
43807 ;   AX
43808 ;
43809 ; NOTES
43810 ;   None
43811 ;
43812 ; HISTORY
43813 ;   Created 16-Mar-1992 by JeffPar
43814 ;
43815 ;-----
43816
43817 comp_names:
43818 00004368 57      push    di      ;
43819 comp_loop:
43820 00004369 E8A002    call    get_char      ;
43821 0000436C 7210      jc     short comp_exit ;
43822 0000436E E80704    call    any_delim     ; is next character a delimiter?
43823 00004371 8A25      mov     ah,[di]      ; (get next character we're supposed to match)
43824 00004373 740B      je     short comp_almost ; yes, it *could* be a match
43825 00004375 47      inc     di      ;
43826 00004376 25DFDF    and     ax,~2020h ; 0DFDFh
43827 00004378 740B      je     short comp_almost ; BUGBUG -- assumes both names are alphanumeric -JTP
43828 00004379 38E0      cmp     al,ah      ; match?
43829 0000437B 74EC      je     short comp_loop ; yes, keep looking at the characters
43830 0000437D F8      clc      ; prevent erroneous eof indication: clear carry
43831 comp_exit:
43832 0000437E 5F      pop     di      ;
43833 0000437F C3      ret     ;
43834
43835 00004380 86C4      xchg    al,ah      ; we don't know for sure if it's a match
43836 00004382 E8F303    call    any_delim     ; until we verify that the second string has
43837 00004385 86C4      xchg    al,ah      ; been exhausted also...
43838 00004387 EBF5      jmp     short comp_exit ; if we are, this call to any_delim will tell...
43839
43840 ;-----
43841 ;
43842 ;   ; 10/09/2023 - RetroDOS v4.2 IO.SYS (Optimization)
43843 ;   comp_names_x:
43844 ;
43845 ;   comp_names_safe:
43846 00004389 50      push    ax
43847 0000438A 51      push    cx
43848 0000438B 56      push    si
43849 0000438C 1E      push    ds
43850 0000438D 0E      push    cs
43851 0000438E 1F      pop     ds
43852 0000438F E8D6FF    call    comp_names
43853 00004392 1F      pop     ds
43854 00004393 5E      pop     si
43855 00004394 59      pop     cx
43856 00004395 58      pop     ax
43857 00004396 C3      ret     ;
43858 ;
43859 ;-----
43860 ;
43861 ;   print_item:  display menu item #BL

```

```

43862 ;
43863 ; INPUT
43864 ; BL == menu item # to display
43865 ;
43866 ; OUTPUT
43867 ; Menu item displayed, with appropriate highlighting if BL == bDefBlock
43868 ;
43869 ; OTHER REGS USED
43870 ; None
43871 ;
43872 ; NOTES
43873 ; This function saves/restores the current cursor position, so you
43874 ; needn't worry about it.
43875 ;
43876 ; HISTORY
43877 ; Created 16-Mar-1992 by JeffPar
43878 ;
43879 ;-----
43880 ;
43881 ; 04/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
43882 ; (SYSINIT:485Ah)
43883 ;
43884 ; print_item: ; prints menu item #BL (1 to N)
43885 ; push ax ;
43886 ; push bx ;
43887 ; push cx ;
43888 ; push dx ;
43889 ; push si ;
43890 ; mov ah,03h ; get cursor position
43891 ; mov bh,[bMenuPage] ; always page zero
43892 ; int 10h ; DH/DL = row/column
43893 ; push dx ; save it
43894 ; mov ah,02h ; set cursor position
43895 ; mov dh,bl ;
43896 ; add dh,3 ;
43897 ; mov dl,5 ;
43898 ; int 10h ; set cursor position for correct row/col
43899 ; mov al,bl ;
43900 ; add al,'0' ; convert menu item # to ASCII digit
43901 ; mov ah,[bMenuColor] ; normal attribute
43902 ; cmp bl,[bDefBlock] ; are we printing the current block?
43903 ; jne short print_other ; no
43904 ; or ah,70h ; yes, set bgnd color to white
43905 ; mov ch,ah ;
43906 ; mov cl,4 ;
43907 ; rol ch,cl ;
43908 ; cmp ch,ah ; are fgnd/bgnd the same?
43909 ; jne short print_other ; no
43910 ; xor ah,08h ; yes, so modify the fgnd intensity
43911 ;
43912 ; print_other:
43913 ; mov bh,0 ;
43914 ; add bx,bx ;
43915 ; mov di,[aoffBlockDesc+bx] ;
43916 ; mov bl,ah ; put the attribute in the correct register now
43917 ; mov bh,[bMenuPage] ; get correct video page #
43918 ; mov ah,09h ; write char/attr
43919 ; mov cx,1 ;
43920 ; int 10h ;
43921 ; inc dl ; increment column
43922 ; mov ah,02h ;
43923 ; int 10h ;
43924 ; mov ax,0900h+'.' ;
43925 ; mov ax,092Eh ;
43926 ; int 10h ; display '.'
43927 ; inc dl ; increment column
43928 ; mov ah,02h ;
43929 ; int 10h ;
43930 ; mov ax,0900h+' ' ;
43931 ; mov ax,0920h ;
43932 ; int 10h ; display ' '
43933 ; inc dl ; increment column
43934 ; mov ah,02h ;
43935 ; int 10h ;
43936 ; push es ;
43937 ; print_loop:
43938 ; mov al,[es:di] ; get a character of the description
43939 ; inc di ;
43940 ; cmp al,TAB ; 9; substitute spaces for tabs
43941 ; jne short print_nontab ;
43942 ; mov al,' ' ;
43943 ; print_nontab:
43944 ; cmp al,' ' ;
43945 ; jb short print_done ; stop at the 1st character < space
43946 ; cmp al','$' ;
43947 ; je short print_done ; also stop on $
43948 ; mov ah,09h ; display function #
43949 ; int 10h ;
43950 ; inc dl ; increment column
43951 ; cmp dl,78 ; far enough?
43952 ; jae short print_done ; yes
43953 ; mov ah,02h ;
43954 ; int 10h ;
43955 ; jmp short print_loop
43956 ; print_done:
43957 ; pop es ;
43958 ; pop dx ;
43959 ; mov ah,02h ;
43960 ; int 10h ; restore previous row/col
43961 ; pop si ;
43962 ; pop dx ;
43963 ; pop cx ;
43964 ; pop bx ;
43965 ; pop ax ;
43966 ; retn ;
43967 ;
43968 ;-----
43969 ; select_item: wait for user to select menu item, with time-out
43970 ;
43971 ; INPUT
43972 ; None
43973 ;
43974 ; OUTPUT
43975 ; BX == menu item # (1-N), or -1 for clean boot
43976 ; Selected menu item highlighted
43977 ; Cursor positioned beneath menu, ready for tty-style output now
43978 ;
43979 ; OTHER REGS USED
43980 ; None
43981 ;
43982 ; NOTES
43983 ; None
43984 ;
43985 ; HISTORY

```

```

43986 ; Created 16-Mar-1992 by JeffPar
43987 ;
43988 ;-----
43989
43990 select_item: ; returns digit value in BX (trashes AX/CX/DX)
43991 mov bl,[bDefBlock] ; BL will be the default block #
43992 mov al,bl ;
43993 call disp_num ;
43994 call show_status ; display current interactive status
43995 cmp byte [secTimeOut],-1
43996 je short input_key ; no time-out, just go to input
43997 mov ah,GET_TIME ; 2ch
43998 int 21h ;
43999 mov bh,dh ; BH = initial # of seconds
44000
44001 check_time:
44002 mov al,[secTimeOut] ;
44003 sub al,[secElapsed] ;
44004 jae short show_time ;
44005 or byte [bQueryOpt],2 ; disable all further prompting
44006 mov byte [secElapsed],0
44007 jmp select_done ; time's up!
44008
44009 show_time:
44010 push bx ;
44011 mov bl,al ; save # in BL
44012 mov bh,[bMenuPage] ;
44013 mov ah,03h ; get cursor position
44014 int 10h ;
44015 push dx ;
44016 add dl,8 ; move cursor to the right
44017 mov ah,02h ; set cursor position
44018 int 10h ;
44019 mov dx,$TimeOut ;
44020 call print ; print the "Time remaining: " prompt
44021 mov al,bl ; recover # from BL
44022 cbw ; this works because AL is always <= 90
44023 mov cl,10 ; AL = tens digit, AH = ones digit
44024 div cl ;
44025 mov cl,ah ;
44026 add al,'0' ;
44027 mov ah,0Eh ;
44028 int 10h ; write TTY tens digit
44029 mov al,cl ;
44030 add al,'0' ;
44031 mov ah,0Eh ;
44032 int 10h ; write TTY ones digit
44033 pop dx ;
44034 mov ah,02h ; set cursor position back to where it was
44035 int 10h ;
44036 pop bx ;
44037
44038 input_key:
44039 mov ah,RAW_CON_IO ; 6
44040 mov dl,0FFh ; input request
44041 int 21h ;
44042 jnz short got_key ;
44043 cmp byte [secTimeOut],-1 ; is there a time-out?
44044 je short input_key ; no, just go back to input
44045 mov ah,GET_TIME ;
44046 int 21h ; DH = seconds
44047 mov ah,dh ;
44048 sub dh,bh ; should generally be zero or one
44049 mov bh,ah ;
44050 jnc short got_time ;
44051 mov dh,1 ; it wrapped back to zero, so assume one
44052
44053 got_time:
44054 or dh,dh ; any change?
44055 jz short input_key ; no
44056 add [secElapsed],dh ;
44057 jmp short check_time ;
44058
44059 got_key:
44060 push ax ;
44061 mov ax,-1 ; zap both secTimeOut and secElapsed
44062 xchg [secTimeOut],ax ;
44063 cmp al,-1 ; was time-out already disabled?
44064 je short timeout_disabled ; yes
44065 push bx ; let's disable # seconds display
44066 mov ax,0A20h ; write multiple spaces
44067 mov bx,[bMenuColor] ;
44068 mov cx,80 ; 80 of them, to be safe
44069 int 10h ; to completely obliterate # seconds display
44070 pop bx ;
44071
44072 timeout_disabled:
44073 pop ax ;
44074 or al,al ; extended key pressed?
44075 jnz short normal_key ; no
44076 int 21h ; get the next part of the key then
44077 jz short input_key ; hmmm, what happened to the second part?
44078
44079 cmp al,48h ; up arrow?
44080 jne short not_up ; no
44081 cmp bl,1 ; are we as up as up can get?
44082 jbe short input_key ; yes, ignore it
44083 dec byte [bDefBlock] ;
44084 call print_item ; re-print the current item
44085 dec bl ; and then print the new current item
44086 jmp short print1
44087
44088 not_up:
44089 cmp al,50h ; down arrow?
44090 jne short not_down ; no
44091 cmp bl,[bMaxBlock] ; are we as down as down can get?
44092 jae short to_input_key ; yes, ignore it
44093 inc byte [bDefBlock] ;
44094 call print_item ; re-print the current item
44095 inc bx ; and then print the new current item
44096
44097 print1:
44098 mov al,bl ;
44099
44100 print2:
44101 call print_item ;
44102 call disp_num ;
44103
44104 to_input_key:
44105 jmp short input_key ; 10/09/2023
44106
44107 not_down:
44108 test byte [bDisableUI],1
44109 jnz short to_input_key ; don't allow F8 or F5
44110 cmp al,42h ; F8 function key?
44111 jne short not_f8 ; no
44112 xor byte [bQueryOpt],1
44113 call show_status ;
44114 jmp input_key ;
44115
44116 not_f8:
44117 cmp al,3Fh ; F5 function key?
44118 jne short to_input_key ; no
44119 ; 02/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
44120 ; MSDOS 6.21 IO.SYS - SYSINIT:49EBh

```



```

44110 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:4C32h)
44111 00004528 800E[834C]04 or byte [bQueryOpt],4 ; no more queries
44112 0000452D BBFFFF mov bx,-1 ; special return code (-1) indicating clean boot
44113 00004530 B020 mov al,-1 ; don't want to display anything really;
44114 00004532 EB26 jmp short disp_input ; just want to display the cr/lf sequence...
44115
44116 normal_key:
44117 00004534 3C0D cmp al,0Dh ; Enter?
44118 00004536 741C je short select_done ; yes
44119 00004538 3C08 cmp al,08h ; backspace?
44120 0000453A 7504 jne short not_backspace ; no
44121 0000453C BBFEFF mov bx,-2 ; 0FFFEh ; yes, special return code
44122 0000453F C3 retn ;
44123
44124 00004540 2C30 sub al,'0' ; is greater than '0'?
44125 00004542 76C8 jbe short to_input_key ; no
44126 00004544 3A06[854C] cmp al,[bMaxBlock] ; is less than or equal to the maximum digit?
44127 00004548 77C2 ja short to_input_key ; no
44128 0000454A A2[844C] mov [bDefBlock],al ;
44129 0000454D E847FE call print_item ; redisplay the current selection
44130 00004550 88C3 mov bl,al ; set new selection
44131 00004552 EBB2 jmp short print2
44132
44133 select_done:
44134 00004554 B700 mov bh,0 ; return a full 16-bit value (for indexing)
44135 00004556 88D8 mov al,bl ;
44136 00004558 0430 add al,'0' ; convert it into a digit, then display it
44137
44138 ; fall into disp_input
44139
44140 ; 16/04/2019 - Retro DOS v4.0
44141
44142 -----
44143 ;
44144 ; disp_input: display a single character + cr/lf
44145 ;
44146 ; INPUT
44147 ; AL == character to display
44148 ;
44149 ; OUTPUT
44150 ; None
44151 ;
44152 ; OTHER REGS USED
44153 ; None
44154 ;
44155 ; NOTES
44156 ; This function is used not only for the menu input selection but
44157 ; also for the interactive line prompting (the y/n/a thing).
44158 ;
44159 ; HISTORY
44160 ; Created 16-Mar-1992 by JeffPar
44161 ;
44162 -----
44163
44164 disp_input:
44165 push ax
44166 0000455A 50 ;cmp al,' '
44167 ;jae short disp_ok
44168 ;mov al,' '
44169 ; 10/09/2023 - Retro DOS v4.2 IO:SYS (Optimization)
44170 mov dl,' ' ; 20h
44171 0000455B B220 cmp al,dl
44172 0000455D 38D0 jna short disp_input_ok
44173 0000455F 7602
44174 disp_ok:
44175 mov dl,al
44176 disp_input_ok:
44177 mov ah,STD_CON_OUTPUT ; 2
44178 00004563 B402 int 21h
44179 00004565 CD21 mov dx,crlfm
44180 00004567 BA[6E50] call print
44181 0000456A E8E504 pop ax
44182 0000456E C3 retn
44183
44184 -----
44185
44186 disp_num:
44187 push bx
44188 add al,'0'
44189 mov ah,0Ah
44190 mov bx,[bMenuColor]
44191 mov cx,1
44192 int 10h
44193 pop bx
44194 retn
44195
44196 -----
44197 ;
44198 ; show_status: display current interactive mode setting (on/off/none)
44199 ;
44200 ; INPUT
44201 ; None
44202 ;
44203 ; OUTPUT
44204 ; None
44205 ;
44206 ; OTHER REGS USED
44207 ; None
44208 ;
44209 ; NOTES
44210 ; None
44211 ;
44212 ; HISTORY
44213 ; Created 16-Mar-1992 by JeffPar
44214 ;
44215 -----
44216
44217 show_status:
44218 0000457F 53 push bx ; BL = video page #
44219 00004580 8B1E[7A4C] mov bx,[bMenuColor]
44220 00004584 B403 mov ah,03h ; get cursor position
44221 00004586 CD10 int 10h ;
44222 00004588 52 push dx ; save it
44223 00004589 B402 mov ah,02h ; set cursor position
44224 0000458B 8B16[7D4C] mov dx,[bLastCol] ; set correct row/col
44225 0000458F F606[7F4C]01 test byte [bDisableUI],1
44226 00004594 740C jz short show_onoff ; just show on/off
44227 00004596 B200 mov dl,0 ;
44228 00004598 CD10 int 10h ;
44229 0000459A B8200A mov ax,0A20h ; write multiple spaces
44230 0000459D B95000 mov cx,80 ; 80 of them, to be exact
44231 ; 10/09/2023
44232 ;int 10h ; to obliterate the status line
44233 000045A0 EB11 jmp short show_done ;

```

```

44234 show_onoff:
44235 int 10h
44236 ; - VIDEO - WRITE CHARACTERS ONLY AT CURSOR POSITION
44237 ; AL = character, BH = display page - alpha mode
44238 ; BL = color of character (graphics mode, PCjr only)
44239 ; CX = number of times to write character
44240
44241 mov al,[_$NO] ; assume OFF
44242 cmp byte [bQueryOpt],1 ; is interactive mode on?
44243 jne short show_noton ; no
44244 mov al,[_$YES] ; yes
44245
44246 show_noton:
44247 mov ah,0Eh ; write TTY
44248 show_done: ; 10/09/2023
44249 int 10h ;
44250 ;show_done: ;
44251 pop dx ;
44252 mov ah,02h ;
44253 int 10h ; restore original cursor position
44254 pop bx ;
44255 retn ;
44256 ; 16/04/2019 - Retro DOS v4.0
44257
44258 -----
44259 ;
44260 skip_token: advances ES:SI/CX past the current token
44261 ;
44262 INPUT
44263 ES:SI -> position in config.sys
44264 CX == remaining bytes in config.sys
44265 ;
44266 OUTPUT
44267 CF set if EOL/EOF hit
44268 AL == 1st char of delimiter
44269 ES:SI -> just past the delimiter
44270 CX == # bytes remaining from that point
44271 ;
44272 OTHER REGS USED
44273 AX
44274 ;
44275 NOTES
44276 None
44277 ;
44278 HISTORY
44279 Created 16-Mar-1992 by JeffPar
44280 ;
44281 -----
44282
44283 skip_token:
44284 call get_char
44285 jc short skip_token_done
44286 call any_delim
44287 jne short skip_token
44288
44289 skip_check_eol:
44290 cmp al,cr ; 0Dh
44291 je short skip_token_eol
44292 cmp al,lf ; 0Ah
44293 je short skip_token_eol
44294 cld
44295 jmp short skip_token_done
44296
44297 retn
44298 skip_token_eol:
44299 stc
44300 skip_token_done:
44301 retn
44302
44303 -----
44304 ;
44305 skip_delim: advances ES:SI/CX past the current delimiter
44306 ;
44307 INPUT
44308 ES:SI -> position in config.sys
44309 CX == remaining bytes in config.sys
44310 ;
44311 OUTPUT
44312 CF set if EOF hit
44313 AL == 1st char of token
44314 ES:SI -> just past the token
44315 CX == # bytes remaining from that point
44316 ES:BX -> new token (since ES:SI is already pointing 1 byte past token)
44317 ;
44318 OTHER REGS USED
44319 AX
44320 ;
44321 NOTES
44322 None
44323 ;
44324 HISTORY
44325 Created 16-Mar-1992 by JeffPar
44326 ;
44327 -----
44328 skip_delim: ; returns carry set if eol/eof
44329 call get_char
44330 lea bx,[si-1] ; also returns BX -> next token
44331 jc short skip_token_done ;
44332 call delim ;
44333 je short skip_delim ;
44334 jmp short skip_check_eol ; 13/05/2019
44335
44336 -----
44337 ;
44338 skip_opt_line: same as skip_line provided AL != LF
44339 ;
44340 INPUT
44341 AL == last character read
44342 ES:SI -> position in config.sys
44343 CX == remaining bytes in config.sys
44344 ;
44345 OUTPUT
44346 CF set if EOF hit
44347 AL == 1st char of new line
44348 ES:SI -> just past 1st char of new line
44349 CX == # bytes remaining from that point
44350 ;
44351 OTHER REGS USED
44352 AX
44353 ;
44354 NOTES
44355 In other words, the purpose here is to skip to the next line,
44356 unless ES:SI is already sitting at the front of the next line (which
44357 it would be if the last character fetched -- AL -- was a linefeed)

```



```

44358 ; HISTORY
44359 ; Created 16-Mar-1992 by JeffPar
44360 ;
44361 ;-----
44362 ; 03/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
44363 ; skip_opt_line:
44364 ; cmp al,lf ; 0Ah
44365 ; je short skip_line_done
44366 ;
44367 ; fall into skip_line
44368 ;
44369 ;-----
44370 ;
44371 ; skip_line: skip to the next line
44372 ;
44373 ; INPUT
44374 ; ES:SI -> position in config.sys
44375 ; CX == remaining bytes in config.sys
44376 ;
44377 ; OUTPUT
44378 ; CF set if EOF hit
44379 ; ES:SI -> just past 1st char of new line
44380 ; CX == # bytes remaining from that point
44381 ;
44382 ; OTHER REGS USED
44383 ; AX
44384 ;
44385 ; NOTES
44386 ; None
44387 ;
44388 ; HISTORY
44389 ; Created 16-Mar-1992 by JeffPar
44390 ;
44391 ;-----
44392 ;
44393 ; skip_line:
44394 ; call get_char
44395 000045E1 E82800 jc short skip_line_done
44396 000045E4 7204 skip_opt_line: ; 03/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
44397 ; cmp al,lf ; 0Ah
44398 000045E6 3C0A jne short skip_line
44399 000045E8 75F7 skip_line_done:
44400 num_done: ; 18/12/2022
44401 ; retn
44402 000045EA C3
44403 ;
44404 ;-----
44405 ;
44406 ; get_number: return binary equivalent of numeric string
44407 ;
44408 ; INPUT
44409 ; ES:SI -> position in config.sys
44410 ; CX == remaining bytes in config.sys
44411 ;
44412 ; OUTPUT
44413 ; AL == non-digit encountered
44414 ; BX == binary #
44415 ; ES:SI -> just past 1st non-digit
44416 ; CX == # bytes remaining from that point
44417 ;
44418 ; OTHER REGS USED
44419 ; AX
44420 ;
44421 ; NOTES
44422 ; None
44423 ;
44424 ; HISTORY
44425 ; Created 16-Mar-1992 by JeffPar
44426 ;
44427 ;-----
44428 ;
44429 ; 13/05/2019
44430 ;
44431 ; get_number:
44432 000045EB 29DB sub bx,bx ; BX = result
44433 ;
44434 000045ED E81C00 num_loop: call get_char ;
44435 000045F0 72F8 jc short num_done ;
44436 000045F2 3C30 cmp al,'0' ; convert to value
44437 000045F4 72F4 jb short num_done ; no more number
44438 000045F6 3C39 cmp al,'9' ;
44439 000045F8 77F0 ja short num_done ;
44440 000045FA 50 push ax ;
44441 000045FB B80A00 mov ax,10 ;
44442 000045FE 52 push dx ;
44443 000045FF F7E3 mul bx ;
44444 00004601 5A pop dx ;
44445 00004602 89C3 mov bx,ax ;
44446 00004604 58 pop ax ;
44447 00004605 2C30 sub al,'0' ;
44448 00004607 98 cbw ;
44449 00004608 01C3 add bx,ax ;
44450 0000460A EBE1 jmp short num_loop ;
44451 ;
44452 ; 18/12/2022
44453 ; num_done:
44454 ; ;retn
44455 ;
44456 ;-----
44457 ;
44458 ; get_char: return next character, advance ES:SI, and decrement CX
44459 ;
44460 ; INPUT
44461 ; ES:SI -> position in config.sys
44462 ; CX == remaining bytes in config.sys
44463 ;
44464 ; OUTPUT
44465 ; AL == next character
44466 ; ES:SI -> just past next character
44467 ; CX == # bytes remaining from that point
44468 ;
44469 ; OTHER REGS USED
44470 ; AX
44471 ;
44472 ; NOTES
44473 ; None
44474 ;
44475 ; HISTORY
44476 ; Created 16-Mar-1992 by JeffPar
44477 ;
44478 ;-----
44479 ;
44480 ; get_char:
44481 0000460C 83E901 sub cx,1 ; use SUB to set carry,zero

```

```

44482 0000460F 7205          jb short get_fail ; out of data
44483                      ;lods      byte ptr es:[si] ;
44484 00004611 26          es
44485 00004612 AC          lodsb
44486 00004613 88C4        mov     ah,al          ;
44487 00004615 C3          retn          ;
44488                      ; restore CX to zero
44489 00004616 B90000      get_fail: mov     cx,0          ; leave carry set, zero not set
44490                      nearby_ret:
44491 00004619 C3          retn
44492
44493
44494
44495
44496
44497
44498
44499
44500
44501
44502
44503
44504
44505
44506
44507
44508
44509
44510
44511
44512
44513
44514
44515
44516
44517
44518
44519
44520
44521
44522 0000461A F606[834C]04 query_user: test     byte [bQueryOpt],4          ; answer no to everything?
44523                      ; 01/01/2023
44524 0000461F 7403          jz      short qu_1          ;
44525 00004621 E9B900      jmp     skip_all          ;
44526                      ; 12/12/2022
44527                      ;;jmp short skip_all          ;
44528                      ;jnz short skip_all
44529
44530 00004624 F606[834C]02 qu_1: test     byte [bQueryOpt],2          ; answer yes to everything?
44531 00004629 75EE          jnz short nearby_ret          ; yes (and return carry clear!)
44532 0000462B 50          push     ax          ;
44533 0000462C A0[6419]      mov     al,[config_cmd]          ;
44534 0000462F F606[834C]01 test     byte [bQueryOpt],1          ; query every command?
44535 00004634 7506          jnz short query_all          ; yes
44536 00004636 A880          test     al,CONFIG_OPTION_QUERY ;
44537                      ; 01/01/2023
44538 00004638 7502          jnz short query_all          ;
44539                      ; 12/12/2022
44540                      ;;jmp short do_cmd          ;
44541                      ;jz short do_cmd ; cf=0
44542
44543                      ; 01/01/2023
44544 0000463A 58          pop      ax
44545 0000463B C3          retn
44546
44547
44548
44549
44550
44551
44552
44553 0000463C 56          push     si          ; save pointer to rest of CONFIG.SYS line
44554 0000463D BA[8053]      mov     dx,_$AutoPrmpt          ;
44555 00004640 247F          and     al,~CONFIG_OPTION_QUERY ; 7Fh
44556 00004642 7450          jz      short generic_prompt ; config_cmd must have been 0
44557
44558 00004644 88C6          mov     dh,al          ; save config_cmd in DH
44559 00004646 29DB          sub     bx,bx          ;
44560 00004648 BF[D04C]      mov     di,comtab          ;
44561
44562 0000464B 8A1D          find_match: mov     bl,[di]          ; get size of current keyword
44563 0000464D 08DB          or      bl,bl          ;
44564 0000464F 7425          jz      short line_print          ; end of table
44565 00004651 47          inc     di          ;
44566 00004652 3A01          cmp     al,[di+bx]          ; match?
44567 00004654 7405          je      short cmd_match          ; yes
44568 00004656 8D7901      lea     di,[di+bx+1]          ; otherwise, skip this command code
44569                      ; 13/05/2019
44570 00004659 EBF0          jmp short find_match          ; loop
44571
44572 0000465B 8A4DFF      cmd_match: mov     cl,[di-1]          ;
44573 0000465E B500          mov     ch,0          ;
44574 00004660 B402          mov     ah,STD_CON_OUTPUT ; 2
44575
44576 00004662 8A05          cmd_print: mov     al,[di]          ;
44577 00004664 47          inc     di          ;
44578 00004665 88C2          mov     dl,al          ;
44579 00004667 C021          int     21h          ;
44580 00004669 E2F7          loop   cmd_print          ;
44581 0000466B B23D          mov     dl,'='          ;
44582 0000466D 80FE56      cmp     dh,CONFIG_SET ; 'v'          ; for SET commands, don't display a '='
44583 00004670 7502          jne short cmd_notset          ;
44584 00004672 B220          mov     dl,' '          ;
44585
44586 00004674 CD21          cmd_notset: int     21h          ; '=' looks funny on SET commands
44587                      line_print:
44588                      ;lods      byte ptr es:[si]          ;
44589 00004676 26          es
44590 00004677 AC          lodsb
44591 00004678 08C0          or      al,al          ;
44592 0000467A 7502          jnz short non_null          ;
44593 0000467C B020          mov     al,' '          ;
44594
44595 0000467E 3C20          non_null: cmp     al,' '          ; control code?
44596 00004680 720F          jb short prompt_user          ; yes, assume end of line
44597 00004682 7505          jne short non_space          ;
44598                      ; 10/09/2023
44599 00004684 263804      cmp     [es:si],al ; 20h          ;
44600                      ;cmp     byte [es:si],' '          ;
44601 00004687 7208          jb short prompt_user          ;
44602
44603 00004689 88C2          non_space: mov     dl,al          ;
44604 0000468B B402          mov     ah,STD_CON_OUTPUT ; 2          ;
44605 0000468D CD21          int     21h          ;

```

```

44606 0000468F EBE5          jmp short line_print      ;
44607
44608
44609 00004691 BA[1153]      prompt_user:          ;
                                mov     dx, _$InterPrmpt      ;
44610
44611
44612 00004694 E8BB03      generic_prompt:
                                call    print                ;
44613
44614 00004697 B400      input_loop:
                                mov     ah, 0                ; read a key
44615 00004699 CD16      int     16h                      ;
44616 0000469B 08C0      or     al, al                ; is it a function key?
44617 0000469D 750F      jnz short not_func          ; no
44618 0000469F 80FC3F      cmp     ah, 3Fh          ; F5 function key?
44619 000046A2 75F3      jne short input_loop      ; no
44620 000046A4 A0[2153]      mov     al, [_$NO]          ;
44621 000046A7 800E[834C]04 or     byte [bQueryOpt], 4 ; no more queries
44622 000046AC EB21      jmp     short legal_char  ;
44623
44624 000046AE 24DF      not_func:
                                and     al, ~20h ; 0DFh      ; converting to upper case
44625 000046B0 3A06[2153] cmp     al, [_$NO]          ; verify character is legal
44626 000046B4 7419      je     short legal_char  ;
44627 000046B6 3A06[1D53] cmp     al, [_$YES]          ;
44628 000046BA 7413      je     short legal_char  ;
44629 000046BC 803E[6419]00 cmp     byte [config_cmd], 0 ;
44630 000046C1 74D4      je     short input_loop      ; don't allow Esc on this query
44631 000046C3 3C1B      cmp     al, 1Bh          ; Esc?
44632 000046C5 75D0      jne short input_loop      ;
44633 000046C7 800E[834C]02 or     byte [bQueryOpt], 2 ; no more interactive boot prompts
44634 000046CC A0[1D53]      mov     al, [_$YES]          ;
44635
44636 000046CF E888FE      legal_char:
                                call    disp_input          ;
44637 000046D2 5E      pop     si                ; restore pointer to rest of CONFIG.SYS line
44638
44639 000046D3 3A06[2153]      cmp     al, [_$NO]          ; process line?
44640 000046D7 7403      je     short skip_cmd      ; no
44641
44642 000046D9 F8      ; 12/12/2022
44643      cld
44644 000046DA 58      do_cmd:
                                pop     ax                    ;
44645      ; 12/12/2022
44646      ; cf=0
44647      ; cld
44648      ; just do the command
44649      retn
44650
44651 000046DC 58      skip_cmd:
                                pop     ax                    ;
44652
44653 000046DD B430      skip_all:
                                mov     ah, CONFIG_REM ; '0' ; fake out the rest of sysinit's processing
44654 000046DF F9      stc
44655 000046E0 C3      retn
44656
44657
44658
44659
44660
44661
44662
44663
44664
44665
44666
44667
44668
44669
44670
44671
44672
44673
44674
44675
44676
44677
44678
44679
-----
;
; print_error: displays multi-config error conditions
;
; INPUT
; Carry set to pause, clear to not
; ES:SI -> current command line in config.sys
;
; OUTPUT
; None
;
; OTHER REGS USED
; None
;
; NOTES
; None
;
; HISTORY
; Created 16-Mar-1992 by JeffPar
;
-----
44680 000046E1 50      print_error:
                                push    ax
44681 000046E2 53      push    bx
44682 000046E3 51      push    cx
44683 000046E4 52      push    dx
44684 000046E5 1E      push    ds
44685 000046E6 0E      push    cs
44686 000046E7 1F      pop     ds
44687 000046E8 9C      pushf
44688 000046E9 E820FC      call    get_linenum
44689 000046EC 891E[AF02]      mov     [linecount], bx
44690 000046F0 E8C5E7      call    error_line
44691 000046F3 9D      popf
44692 000046F4 7319      jnc short pe_ret
44693 000046F6 BA[DB51]      mov     dx, _$PauseMsg
44694 000046F9 E85603      call    print
44695 000046FC B8070C      mov     ax, 0C07h          ; flush input buffer, then wait for key
44696 000046FF CD21      int     21h                ; wait for a key
44697 00004701 08C0      or     al, al                ; extended key?
44698 00004703 7504      jnz short pe_1              ; no
44699 00004705 B407      mov     ah, 07h            ; yes
44700 00004707 CD21      int     21h                ; eat it too
44701
44702 00004709 BA[6E50]      pe_1:
                                mov     dx, crlfm
44703 0000470C E84303      call    print
44704
44705 0000470F 1F      pe_ret:
                                pop     ds
44706 00004710 5A      pop     dx
44707 00004711 59      pop     cx
44708 00004712 5B      pop     bx
44709 00004713 58      pop     ax
44710 00004714 C3      retn
44711
44712
44713
44714
44715
44716
44717
44718
44719
44720
44721
44722
44723 00004715 F606[834C]04 test    byte [bQueryOpt], 4
44724 0000471A 7443      jz     short disable_exit
44725 0000471C F606[794C]01 test    byte [dae_flag], 1
44726 00004721 753C      jnz short disable_exit
44727 00004723 800E[794C]01 or     byte [dae_flag], 1
44728
44729 00004728 810E[834C]0201 or     ;or byte [bQueryOpt], 2 ; MSDOS 6.0
                                word [bQueryOpt], 102h ; [bDefBlock] = 1

```

```

44730 0000472E BA4420      mov     dx,'D ' ; 2044h
44731
44732      dae_1:
44733 00004731 A0[B84B]      ; 03/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
44734      mov     al,[def_swchr]
44735 00004734 08C0      ;mov     al,[command_line-1] ; get default switchchar
44736 00004736 7427      ;or      al,al ; anything there?
44737 00004738 8A1E[B94B]    jz      short disable_exit ; no, disable_autoexec already called
44738 0000473C B700      mov     bl,[command_line] ;
44739 0000473E 89D9      mov     bh,0 ; BX == command-line length
44740 00004740 80C303    mov     cx,bx ;
44741 00004743 80FB7E    add     bl,3 ;
44742 00004746 7717      cmp     bl,126 ;
44743 00004748 881E[B94B]    ja      short disable_exit ;
44744 0000474C 81C3[BA4B]    mov     [command_line],bl ; update length
44745 00004750 41        add     bx,command_line+1 ; make sure we move the NULL too
44746      inc     cx ; (just for consistency sake)
44747 00004751 8A67FD      disable_loop:
44748 00004754 8827      mov     ah,[bx-3] ;
44749 00004756 4B        mov     [bx],ah ;
44750 00004757 E2F8      dec     bx ;
44751 00004759 8847FE      loop    disable_loop ;
44752      mov     word [bx-1],'D ' ; 2044h ; /D is stuffed into place now
44753 0000475C 8957FF      mov     [bx-1],dx ; MSDOS 6.21 IO.SYS - SYSINIT:4C29h
44754      ;mov     byte [command_line-1],0 ;
44755      disable_exit:
44756 0000475F C3        retn
44757
44758      CheckQueryOpt: ; MSDOS 6.21 IO.YSYS - SYSINIT:4C2Dh
44759 00004760 803E[834C]01  cmp     byte [bQueryOpt],1
44760 00004765 75F8      jnz     short disable_exit
44761 00004767 F606[794C]02  test    byte [dae_flag],2
44762 0000476C 75F1      jnz     short disable_exit
44763 0000476E 800E[794C]02  or      byte [dae_flag],2
44764      ;mov     dx,'Y' ; (MASM syntax) ; 2059h
44765      ; 10/09/2023 (BugFix)
44766 00004773 BA5920    mov     dx,'Y ' ; (NASM syntax) ; 2059h
44767 00004776 EBB9      jmp     short dae_1
44768
44769      ;endif ;MULTI_CONFIG
44770
44771      ;%endif ; 02/11/2022
44772
44773
44774      ; 19/04/2019 - Retro DOS v4.0
44775
44776      ;-----
44777      ;
44778      ; procedure : delim
44779      ;
44780      ;-----
44781
44782      ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
44783      ; (SYSINIT:4C45h)
44784
44785      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
44786      ;%if 0
44787      ;;ifdef MULTI_CONFIG
44788      ;
44789      any_delim:
44790 00004778 3C0D      cmp     al,cr
44791 0000477A 7427      je      short delim_ret
44792 0000477C 3C0A      cmp     al,lf
44793 0000477E 7423      je      short delim_ret
44794 00004780 3C5B      cmp     al,[' '
44795 00004782 741F      je      short delim_ret
44796 00004784 3C5D      cmp     al,']'
44797 00004786 741B      je      short delim_ret
44798      ;
44799      ;;endif ;MULTI_CONFIG
44800      ;%endif ; 02/11/2022
44801
44802      ; 02/11/2022 - RetroDOS v4.0 (Modified MSDOS 5.0 IO.SYS)
44803      ; (SYSINIT:3450h)
44804      delim:
44805 00004788 3C2F      cmp     al,'/' ; ibm will assume "/" as an delimiter.
44806 0000478A 7417      je      short delim_ret
44807
44808 0000478C 3C00      cmp     al,0 ; special case for sysinit!!!
44809 0000478E 7413      je      short delim_ret
44810
44811      org_delim: ; used by organize routine except for getting
44812 00004790 3C20      cmp     al,' ' ; the filename.
44813 00004792 740F      je      short delim_ret
44814 00004794 3C09      cmp     al,tab ; 9
44815 00004796 740B      je      short delim_ret
44816 00004798 3C3D      cmp     al,'='
44817 0000479A 7407      je      short delim_ret
44818 0000479C 3C2C      cmp     al,', '
44819 0000479E 7403      je      short delim_ret
44820 000047A0 3C3B      cmp     al,','
44821
44822      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
44823
44824      ; 04/01/2023 - Retro DOS v4.2
44825      ;ifdef MULTI_CONFIG
44826      ; Make sure there's no chance of a false EOF indication
44827 000047A2 F8        clc
44828
44829      ;endif ; 02/11/2022
44830      delim_ret:
44831      ; 04/01/2023
44832      ; cf = 0
44833      nl_ret: ; 10/09/2023
44834 000047A3 C3        retn
44835
44836      ;-----
44837      ;
44838      ; procedure : newline
44839      ;
44840      ; newline returns with first character of next line
44841      ;
44842      ;-----
44843
44844      newline:
44845 000047A4 E80600    call    getchr ;skip non-control characters
44846 000047A7 72FA      jc      short nl_ret
44847 000047A9 3C0A      cmp     al,lf ;look for line feed
44848 000047AB 75F7      jne     short newline
44849
44850      ; 10/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
44851      ;call    getchr
44852      ;nl_ret:
44853      ;retn

```

```

44854         ; 10/09/2023
44855         ;jmp     short getchr
44856
44857     ; 10/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
44858     %if 1
44859
44860     ;-----
44861     ;
44862     ; procedure : getchr
44863     ;
44864     ;-----
44865
44866         ; 24/10/2022
44867     getchr:
44868         ; 12/12/2022
44869         ;push     cx
44870         ;mov      cx,[count]
44871         ;jcxz     nochar
44872         ; 12/12/2022
44873     000047AD 833E[5603]01    cmp     word [count],1
44874     000047B2 720F          jnb     short nochar ; cf=1 ([count] = 0)
44875
44876     000047B4 8B36[5A03]      mov     si,[chrptr]
44877     000047B8 268A04          mov     al,[es:si]
44878     000047BB FF0E[5603]      dec     word [count]
44879     000047BF FF06[5A03]      inc     word [chrptr]
44880         ; 12/12/202
44881         ; cf=0
44882         ;clc
44883     ;get_ret:
44884         ;pop      cx
44885         ;retn
44886     nochar:
44887         ; 12/12/2022
44888         ; cf=1
44889         ;stc
44890         ;jmp      short get_ret
44891
44892     000047C3 C3              retn
44893     %endif
44894
44895     ;-----
44896     ;
44897     ; procedure : mapcase
44898     ;
44899     ;-----
44900
44901         ; 02/11/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS)
44902
44903         ; 04/01/2023 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS)
44904         ; (SYSINIT:4C7Eh)
44905     mapcase:
44906     000047C4 51              push     cx
44907     000047C5 56              push     si
44908     000047C6 1E              push     ds
44909
44910         push     es
44911     000047C8 1F              pop      ds
44912
44913         ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
44914
44915     ; 04/01/2023 - Retro DOS 4.2
44916
44917     %ifdef     MULTI_CONFIG
44918     000047C9 88C3          mov     bl,al          ; same cmd code this line
44919     %else
44920         ; xor     si,si
44921     %endif
44922         ; 02/11/2022
44923         ; 04/01/2023 - Retro DOS 4.2
44924         ;xor      si, si
44925
44926     convloop:
44927     000047CB AC              lodsb
44928     000047CC 3C61          cmp     al,'a'
44929     000047CE 7209          jnb     short noconv
44930     000047D0 3C7A          cmp     al,'z'
44931     000047D2 7705          ja      short noconv
44932     000047D4 2C20          sub     al,20h
44933     000047D6 8844FF          mov     [si-1],al
44934     noconv:
44935
44936         ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
44937
44938     ; 04/01/2023 - Retro DOS 4.2
44939     %ifdef     MULTI_CONFIG
44940
44941     ; when MULTI_CONFIG enabled, "mapcase" is used to map everything to
44942     ; upper-case a line at a time, after we've been able to figure out whether
44943     ; the line is a SET command or not (since we don't want to upper-case
44944     ; anything after the "=" in a SET)
44945
44946     000047D9 80FB56          cmp     bl,CONFIG_SET ; 'v' ; preserve case for part of the line?
44947     000047DC 7504          jne     short check_eol ; no, just check for end-of-line
44948     000047DE 3C3D          cmp     al,'=' ; separator between SET var and value?
44949     000047E0 740A          je      short convdone ; yes
44950
44951     check_eol:
44952         cmp     al,cr
44953         je      short convdone
44954         cmp     al,lf
44955         je      short convdone
44956     %endif
44957     000047EA E2DF          loop    convloop
44958
44959     convdone:
44960         pop     ds
44961         pop     si
44962         pop     cx
44963         retn
44964
44965     ;-----
44966     ;
44967     ; procedure : round
44968     ;
44969     ; round the values in memlo and memhi to paragraph boundary.
44970     ; perform bounds check.
44971     ;
44972     ;-----
44973
44974     round:
44975         ; 10/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
44976     000047F0 1E          push     ds
44977     000047F1 0E          push     cs
44978     000047F2 1F          pop      ds

```

```

44978
44979 000047F3 50      push    ax
44980      ;mov     ax,[cs:memlo]
44981 000047F4 A1[6203]  mov     ax,[memlo]
44982
44983 000047F7 E81BCB    call    ParaRound      ; para round up
44984
44985      ;add     [cs:memhi],ax
44986 000047FA 0106[6403]  add     [memhi],ax
44987      ;mov     word [cs:memlo],0
44988 000047FE C706[6203]0000  mov     word [memlo],0
44989      ;mov     ax,[cs:memhi]
44990 00004804 A1[6403]      mov     ax,[memhi]      ; ax = new memhi
44991      ;cmp     ax,[cs:ALLOCLIM]
44992 00004807 3B06[A502]    cmp     ax,[ALLOCLIM]   ; if new memhi >= alloclim, error
44993      ;jae     short mem_err
44994      ; 13/04/2024
44995 0000480B 7322      jae     short mem_err2 ; ds = cs
44996      ;test    byte [cs:setdevmarkflag],for_devmark ; 2
44997 0000480D F606[6919]02  test    byte [setdevmarkflag],for_devmark ; 2
44998 00004812 7416      jz      short skip_set_devmarksize
44999 00004814 06      push    es
45000 00004815 56      push    si
45001      ;mov     si,[cs:devmark_addr]
45002 00004816 8B36[6719]  mov     si,[devmark_addr]
45003 0000481A 8EC6      mov     es,si
45004 0000481C 29F0      sub     ax,si
45005 0000481E 48      dec     ax
45006      ;mov     [es:3],ax
45007 0000481F 26A30300    mov     [es:devmark.size],ax ; paragraph
45008      ;and     byte [cs:setdevmarkflag],not_for_devmark ; 0FDh
45009 00004823 8026[6919]FD  and     byte [setdevmarkflag],not_for_devmark ; 0FDh
45010 00004828 5E      pop     si
45011 00004829 07      pop     es
45012      skip_set_devmarksize:
45013 0000482A 58      pop     ax
45014
45015      ; 10/09/2023
45016 0000482B 1F      pop     ds
45017
45018      ; 11/12/2022
45019      ; cf = 0
45020      ; 02/11/2022
45021      ;clc     ; ? (not needed here) ; clear carry
45022 0000482C C3      retn
45023
45024      ;-----
45025
45026      mem_err:
45027      ; 11/12/2022
45028 0000482D 0E      push    cs
45029 0000482E 1F      pop     ds
45030
45031 0000482F BA[4751]    mem_err2:
45032      mov     dx,badmem
45033      ;push    cs
45034 00004832 E81D02    ;pop     ds
45035 00004835 E916CB    call    print
45036      jmp     stall
45037
45038      ;-----
45039      ;
45040      ; procedure : calldev
45041      ;
45042      ;-----
45043      ; 02/11/2022 - Retrodos v4.0 (Modified MSDOS 5.0 IO.SYS)
45044      ; (SYSINIT:34E0h)
45045
45046      ; 13/04/2024 - Retrodos v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
45047      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:4F3Eh)
45048
45049      calldev:
45050 00004838 2E8E1E[3924]  mov     ds,[cs:DevEntry+2]
45051 0000483D 2E031E[3724]  add     bx,[cs:DevEntry]   ; do a little relocation
45052 00004842 8B07      mov     ax,[bx]
45053
45054 00004844 2EFF36[3724]  push    word [cs:DevEntry]
45055 00004849 2EA3[3724]    mov     [cs:DevEntry],ax
45056 0000484D BB[6F03]      mov     bx,packet
45057 00004850 2EFF1E[3724]  call    far [cs:DevEntry]
45058 00004855 2E8F06[3724]  pop     word [cs:DevEntry]
45059 0000485A C3      retn
45060
45061      ;-----
45062      ;
45063      ; procedure : todigit
45064      ;
45065      ;-----
45066
45067      todigit:
45068 0000485B 2C30      sub     al,'0'
45069      ;jb      short notdig ; 02/11/2022
45070      ; 12/12/2022
45071 0000485D 7203      jb      short notdig2
45072      ;cmp     al,9
45073      ;ja      short notdig
45074      ;clc
45075      ;retn
45076      ; 12/12/2022
45077 0000485F 3C0A      cmp     al,10
45078 00004861 F5      cmc
45079      notdig:
45080      ;stc
45081      notdig2:
45082 00004862 C3      retn
45083
45084      ;-----
45085      ;
45086      ; procedure : getnum
45087      ;
45088      ; getnum parses a decimal number.
45089      ; returns it in ax, sets zero flag if ax = 0 (may be considered an
45090      ; error), if number is bad carry is set, zero is set, ax=0.
45091      ;
45092      ;-----
45093
45094      getnum:
45095 00004863 53      push    bx
45096 00004864 31DB      xor     bx,bx      ; running count is zero
45097
45098 00004866 E8F2FF    b2:      call    todigit      ; do we have a digit ?
45099 00004869 7247      jc      short badnum ; no, bomb
45100
45101 0000486B 93      xchg    ax,bx      ; put total in ax

```



```

45102 0000486C 53      push    bx                ; save digit (0 to 9)
45103                ;mov    bx,10          ; base of arithmetic
45104                ; 12/12/2022
45105 0000486D B30A    mov     b1,10
45106 0000486F F7E3    mul     bx                ; shift by one decimal digit
45107 00004871 5B      pop     bx                ; get back digit (0 to 9)
45108 00004872 00D8    add     al,b1            ; get total
45109 00004874 80D400  adc     ah,0             ; make that 16 bits
45110 00004877 7239    jc      short badnum     ; too big a number
45111
45112 00004879 93      xchg    ax,bx            ; stash total
45113
45114 0000487A E830FF    call   getchr            ;get next digit
45115 0000487D 722D    jc      short b1         ; no more characters
45116 0000487F 3C20    cmp     al,' '           ; space?
45117 00004881 741F    je      short b15        ; then end of digits
45118 00004883 3C2C    cmp     al,', '          ; ', ' is a seperator!!!
45119 00004885 741B    je      short b15        ; then end of digits.
45120 00004887 3C09    cmp     al,tab ; 9       ; tab
45121 00004889 7417    je      short b15
45122 0000488B 2E3A06[AE02] cmp     al,[cs:sepchr]    ; allow 0 or special separators
45123 00004890 7410    je      short b15
45124 00004892 3C2F    cmp     al,'/'          ; see if another switch follows
45125                ; 12/12/2022
45126                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
45127                ;nop
45128                ;nop
45129 00004894 740C    je      short b15
45130 00004896 3C0A    cmp     al,lf            ; line-feed?
45131 00004898 7408    je      short b15
45132 0000489A 3C0D    cmp     al,cr            ; carriage return?
45133 0000489C 7404    je      short b15
45134 0000489E 08C0    or      al,al            ; end of line separator?
45135 000048A0 75C4    jnz     short b2         ; no, try as a valid char...
45136
45137 000048A2 2EFF06[5603] b15:  inc     word [cs:count]    ; one more character to s...
45138 000048A7 2EFF0E[5A03] dec     word [cs:chrptr]  ; back up over separator
45139
45140 000048AC 89D8    mov     ax,bx            ; get proper count
45141 000048AE 09C0    or      ax,ax            ; clears carry, sets zero accordingly
45142 000048B0 5B      pop     bx
45143 000048B1 C3      retn
45144
45145                badnum:
45146                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
45147 000048B2 31C0    ;mov    byte [cs:sepchr],0
45148                xor     ax,ax            ; set zero flag, and ax = 0
45149 000048B4 2EA2[AE02] ; 12 /12/2022
45150 000048B8 5B      mov     [cs:sepchr],al ; 0
45151 000048B9 F9      pop     bx
45152 000048BA C3      stc
45153                ; and carry set
45154                retn
45155
45156                ;*****
45157
45158                setdoscountryinfo:
45159
45160                ;-----
45161                ;input: es:di -> pointer to dos_country_cdpq_info
45162                ; ds:0 -> buffer.
45163                ; si = 0
45164                ; ax = country id
45165                ; dx = code page id. (if 0, then use ccscodespage as a default.)
45166                ; bx = file handle
45167                ; this routine can handle maximum 438 country_data entries.
45168                ;
45169                ;output: dos_country_cdpq_info set.
45170                ; carry set if any file read failure or wrong information in the file.
45171                ; carry set and cx = -1 if cannot find the matching country_id,
45172                ; codepage_id in the file.
45173                ;-----
45174
45175                ; 04/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
45176                ; (SYSINIT:4D83h)
45177
45178                ; 10/09/2023 - Retrodos v4.2 IO.SYS (Optimization)
45179                ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:4FCAh)
45180
45181 000048BB 57      push    di
45182 000048BC 50      push    ax
45183 000048BD 52      push    dx
45184
45185 000048BE 31C9    xor     cx,cx
45186 000048C0 31D2    xor     dx,dx
45187 000048C2 B80002    mov     ax,512           ;read 512 bytes
45188 000048C5 E84301    call   readincontrolbuffer ;read the file header
45189 000048C8 724A    jc      short setdosdata_fail
45190
45191 000048CA 06      push    es
45192 000048CB 56      push    si
45193
45194 000048CC 0E      push    cs
45195 000048CD 07      pop     es
45196
45197 000048CE BF1E4B[BF] mov     di,country_file_signature ; db 0FFh,'COUNTRY'
45198 000048D1 B90800    mov     cx,8             ;length of the signature
45199 000048D4 F3A6    repz    cmpsb
45200
45201 000048D6 5E      pop     si
45202 000048D7 07      pop     es
45203 000048D8 753A    jnz     short setdosdata_fail ;signature mismatch
45204
45205 000048DA 83C612  add     si,18            ;si -> county info type
45206 000048DD 803C01  cmp     byte [si],1       ;only accept type 1 (currently only 1 header type)
45207 000048E0 7532    jne     short setdosdata_fail ;cannot proceed. error return
45208
45209 000048E2 46      inc     si                ;si -> file offset
45210 000048E3 8B14    mov     dx,[si]          ;get the info file offset.
45211 000048E5 8B4C02  mov     cx,[si+2]
45212 000048E8 B80018    mov     ax,6144          ;read 6144 bytes.
45213 000048EB E81D01    call   readincontrolbuffer ;read info
45214 000048EE 7224    jc      short setdosdata_fail
45215
45216 000048F0 8B0C    mov     cx,[si]          ;get the # of country, codepage combination entries
45217 000048F2 81F9B601 cmp     cx,438           ;cannot handle more than 438 entries.
45218 000048F6 771C    ja      short setdosdata_fail
45219
45220 000048F8 46      inc     si
45221 000048F9 46      inc     si                ;si -> entry information packet
45222 000048FA 5A      pop     dx                ;restore code page id
45223 000048FB 58      pop     ax                ;restore country id
45224 000048FC 5F      pop     di
45225
45226                setdoscntry_find:
45227                ;search for desired country_id,codepage_id.
45228                cmp     ax,[si+2]
45229                ;compare country_id

```

```

45226 00004900 7509      jne      short setdoscntry_next
45227
45228      ;cmp      dx,0      ;no user specified code page ?
45229      ;je       short setdoscntry_any_codepage ;then no need to match code page id.
45230      ; 10/09/2023
45231 00004902 09D2      or       dx,dx ; cmp dx,0
45232 00004904 7413      jz       short setdoscntry_any_codepage
45233 00004906 3B5404     cmp      dx,[si+4] ;compare code page id
45234 00004909 7411      je       short setdoscntry_got_it
45235
45236      setdoscntry_next:
45237 0000490B 0334      add      si,[si]      ;next entry
45238 0000490D 46          inc      si
45239 0000490E 46          inc      si      ;take a word for size of entry itself
45240 0000490F E2EC      loop    setdoscntry_find
45241
45242      ;mov      cx,-1      ;signals that bad country id entered.
45243      ; 10/09/2023
45244 00004911 49          dec      cx ; 0 -> -1
45245      setdoscntry_fail:
45246 00004912 F9          stc
45247 00004913 C3          retn
45248
45249      setdosdata_fail:
45250 00004914 5E          pop      si
45251 00004915 59          pop      cx
45252 00004916 5F          pop      di
45253 00004917 EBF9     jmp      short setdoscntry_fail
45254
45255      setdoscntry_any_codepage:      ;use the code_page_id of the country_id found.
45256 00004919 8B5404     mov      dx,[si+4]
45257
45258      setdoscntry_got_it:      ;found the matching entry
45259 0000491C 2E8916[264B]  mov      [cs:cntrycodepage_id],dx ;save code page id for this country.
45260 00004921 8B540A     mov      dx,[si+10] ;get the file offset of country data
45261 00004924 8B4C0C     mov      cx,[si+12]
45262 00004927 B80002     mov      ax,512 ;read 512 bytes
45263 0000492A E8DE00     call     readincontrolbuffer
45264 0000492D 72E3      jc       short setdoscntry_fail
45265
45266 0000492F 8B0C     mov      cx,[si] ;get the number of entries to handle.
45267 00004931 46          inc      si
45268 00004932 46          inc      si ;si -> first entry
45269
45270      setdoscntry_data:
45271 00004933 57          push     di ;es:di -> dos_country_cdpd_info
45272 00004934 51          push     cx ;save # of entry left
45273 00004935 56          push     si ;si -> current entry in control buffer
45274
45275 00004936 8A4402     mov      al,[si+2] ;get data entry id
45276 00004939 E8A400     call     getcountrydestination ;get the address of destination in es:di
45277 0000493C 727C      jc       short setdoscntry_data_next ;no matching data entry id in dos
45278
45279 0000493E 8B5404     mov      dx,[si+4] ;get offset of data
45280 00004941 8B4C06     mov      cx,[si+6]
45281 00004944 B80042     mov      ax,4200h
45282 00004947 F9          stc
45283 00004948 CD21     int      21h ;move pointer
45284 0000494A 72C8      jc       short setdosdata_fail
45285
45286 0000494C BA0002     mov      dx,512 ;start of data buffer
45287 0000494F B91400     mov      cx,20 ;read 20 bytes only. we only need to
45288 00004952 B43F     mov      ah,3Fh ;look at the length of the data in the file.
45289 00004954 F9          stc
45290 00004955 CD21     int      21h ;read the country.sys data
45291 00004957 72BB     jc       short setdosdata_fail ;read failure
45292
45293 00004959 39C8     cmp      ax,cx
45294 0000495B 75B7     jne      short setdosdata_fail ; 13/05/2019
45295
45296 0000495D 8B5404     mov      dx,[si+4] ;get offset of data again.
45297 00004960 8B4C06     mov      cx,[si+6]
45298 00004963 B80042     mov      ax,4200h
45299 00004966 F9          stc
45300 00004967 CD21     int      21h ;move pointer back again
45301 00004969 72A9     jc       short setdosdata_fail
45302
45303 0000496B 56          push     si
45304 0000496C BE0802     mov      si,(512+8) ;get length of the data from the file
45305 0000496F 8B0C     mov      cx,[si]
45306 00004971 5E          pop      si
45307 00004972 BA0002     mov      dx,512 ;start of data buffer
45308 00004975 83C10A     add      cx,10 ;signature + a word for the length itself
45309 00004978 B43F     mov      ah,3Fh ;read the data from the file.
45310 0000497A F9          stc
45311 0000497B CD21     int      21h
45312 0000497D 7295     jc       short setdosdata_fail
45313
45314 0000497F 39C8     cmp      ax,cx
45315 00004981 7591     jne      short setdosdata_fail
45316
45317 00004983 8A4402     mov      al,[si+2] ;save data id for future use.
45318 00004986 BE0802     mov      si,(512+8) ;si-> data buffer + id tag field
45319 00004989 8B0C     mov      cx,[si] ;get the length of the file
45320 0000498B 41          inc      cx ;take care of a word for lenght of tab
45321 0000498C 41          inc      cx ;itself.
45322 0000498D 81F9F805  cmp      cx,(2048-512-8) ; 1528 ;fit into the buffer?
45323 00004991 7781     ja       short setdosdata_fail
45324
45325      ;if      bugfix
45326 00004993 E83100     call     setdbcs_before_copy
45327      ;endif
45328
45329 00004996 3C01     cmp      al,SetCountryInfo ; 1 ;is the data for setcountryinfo table?
45330 00004998 7511     jne      short setdoscntry_mov ;no, don't worry
45331
45332 0000499A 26FF7518  push     word [es:di+country_cdpd_info.ccMono_Ptr-country_cdpd_info.ccCountryInfoLen]
45333      ;push     word [es:di+24] ;cannot destroy ccmono_ptr address. save them.
45334 0000499E 26FF751A  push     word [es:di+country_cdpd_info.ccMono_Ptr-country_cdpd_info.ccCountryInfoLen+2]
45335      ;push     word [es:di+26] ;at this time di -> cccountryinfoLen
45336
45337 000049A2 57          push     di ;save di
45338
45339      ;push     ax
45340      ;mov      ax,[cs:cntrycodepage_id] ;do not use the code page info in country_info
45341      ;mov      [si+4],ax ;use the saved one for this !!!!
45342      ;pop      ax
45343      ; 10/09/2023
45344 000049A3 2EFF36[264B]  push     word [cs:cntrycodepage_id]
45345 000049A8 8F4404     pop      word [si+4]
45346
45347      setdoscntry_mov:
45348 000049AB F3A4     rep      movsb ;copy the table into dos
45349 000049AD 3C01     cmp      al,SetCountryInfo ;was the ccmono_ptr saved?

```



```

45350 000049AF 7509          jne     short setdoscntry_data_next
45351
45352 000049B1 5F            pop     di                ;restore di
45353 000049B2 268F451A        pop     word [es:di+country_cdpdpg_info.ccMono_Ptr-country_cdpdpg_info.ccCountryInfoLen+2]
45354                      ;pop     word [es:di+26]                ;restore
45355 000049B6 268F4518        pop     word [es:di+country_cdpdpg_info.ccMono_Ptr-country_cdpdpg_info.ccCountryInfoLen]
45356                      ;pop     word [es:di+24]
45357
45358 setdoscntry_data_next:
45359 000049BA 5E            pop     si                ;restore control buffer pointer
45360 000049BB 59            pop     cx                ;restore # of entries left
45361 000049BC 5F            pop     di                ;restore pointer to dso_country_cdpdpg
45362 000049BD 0334        add     si,[si]           ;try to get the next entry
45363 000049BF 46            inc     si
45364 000049C0 46            inc     si                ;take a word of entry length itself
45365 000049C1 49            dec     cx
45366                      ; 10/09/2023
45367 000049C2 741B        jz      short setdoscntry_ok
45368                      ;cmp     cx,0
45369                      ;je      short setdoscntry_ok
45370 000049C4 E96CFF        jmp     setdoscntry_data
45371
45372                      ; 18/12/2022
45373 ;setdoscntry_ok:
45374 ;retn
45375
45376 ;-----
45377
45378 ;if      bugfix
45379
45380 ; 10/09/2023 - Retrodos v4.2 IO.SYS (Optimization)
45381
45382 setdbcs_before_copy:
45383 000049C7 3C07        cmp     al,SetDBCS ; 7      ; dbcs vector set?
45384 000049C9 7514        jne     short sdbcsbc      ; jump if not
45385
45386 ; 10/09/2023
45387 000049CB 50            push    ax
45388 000049CC 31C0        xor     ax,ax
45389 000049CE 263905      cmp     [es:di],ax ; 0
45390 000049D1 740B        je      short sdbcsbc_pop
45391
45392 ;cmp     word [es:di],0      ; zero byte data block?
45393 ;je      short sdbcsbc      ; jump if so
45394
45395 000049D3 57            push    di
45396                      ; 10/09/2023
45397 ;push    ax
45398 000049D4 51            push    cx
45399 000049D5 268B0D      mov     cx,[es:di]         ; load block length
45400                      ;add     di,2             ; points actual data
45401 000049D8 47            inc     di
45402 000049D9 47            inc     di
45403                      ;xor     al,al         ; fill bytes
45404 000049DA F3AA        rep     stosb             ; clear data block
45405 000049DC 59            pop     cx
45406                      ;pop     ax
45407 000049DD 5F            pop     di
45408
45409 sdbcsbc_pop:      ; 10/09/2023
45410 000049DE 58            pop     ax
45411 sdbcsbc:
45412 setdoscntry_ok:  ; 18/12/2022
45413 000049DF C3            retn
45414
45415 ;endif
45416
45417 ;-----
45418
45419 getcountrydestination:
45420
45421 ;-----
45422 ;get the destination address in the dos country info table.
45423 ;
45424 ;input: al - data id
45425 ; es:di -> dos_country_cdpdpg_info
45426 ;on return:
45427 ; es:di -> destination address of the matching data id
45428 ; carry set if no matching data id found in dos.
45429 ;-----
45430
45431 ; 04/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
45432 ; (SYSINIT:4EB2h)
45433
45434 ; 10/09/2023 - Retrodos v4.2 IO.SYS (Optimization)
45435 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:50F9h)
45436
45437 000049E0 51            push    cx
45438                      ;add     di,74
45439 000049E1 83C74A      add     di,country_cdpdpg_info.ccNumber_of_entries
45440                      ;skip the reserved area, syscodepage etc.
45441 000049E4 268B0D      mov     cx,[es:di]         ;get the number of entries
45442 000049E7 47            inc     di
45443 000049E8 47            inc     di                ;si -> the first start entry id
45444
45445 getcntrydest:
45446 000049E9 263805      cmp     byte [es:di],al
45447 000049EC 7413        je      short getcntrydest_ok
45448
45449 000049EE 26803D01    cmp     byte [es:di],SetCountryInfo ;was it setcountryinfo entry?
45450 000049F2 7405        je      short getcntrydest_1
45451
45452 000049F4 83C705      add     di,5                ;next data id
45453 000049F7 EB03        jmp     short getcntrydest_loop
45454
45455 getcntrydest_1:
45456                      ;add     di,41
45457 000049F9 83C729      add     di,NEW_COUNTRY_SIZE+3 ;next data id
45458
45459 000049FC E2EB        loop   getcntrydest
45460 000049FE F9            stc
45461                      ;jmp     short getcntrydest_exit
45462 getcntrydest_exit:
45463                      ; 10/09/2023
45464 000049FF 59            pop     cx
45465 00004A00 C3            retn
45466
45467 getcntrydest_ok:
45468                      ; 10/09/2023
45469 00004A01 47            inc     di
45470
45471 ; cmp     al,SetCountryInfo ; 1 ;select country info?
45472 ; jne     short getcntrydest_ok1
45473 ;

```

```

45474 ; ;inc di ;now di -> cccountryinfo len
45475 ; jmp short getcntrydest_exit
45476
45477 ; 10/09/2023
45478 00004A02 3C01 cmp al,SetCountryInfo ; 1 ;select country info?
45479 00004A04 74F9 je short getcntrydest_exit
45480
45481 getcntrydest_ok1:
45482 ;les di,[es:di+1] ;get the destination in es:di
45483 ; 10/09/2023
45484 00004A06 26C43D les di,[es:di]
45485 ;getcntrydest_exit:
45486 00004A09 59 pop cx
45487 00004A0A C3 retn
45488
45489 ;-----
45490
45491 readincontrolbuffer:
45492
45493 ;-----
45494 ;move file pointer to cx:dx
45495 ;read ax bytes into the control buffer. (should be less than 2 kb)
45496 ;si will be set to 0 hence ds:si points to the control buffer.
45497 ;
45498 ;entry: cx,dx offset from the start of the file where the read/write pointer
45499 ; be moved.
45500 ; ax - # of bytes to read
45501 ; bx - file handle
45502 ; ds - buffer seg.
45503 ;return: the control data information is read into ds:0 - ds:0200.
45504 ; cx,dx value destroyed.
45505 ; carry set if error in reading file.
45506 ;-----
45507
45508 00004A0B 50 push ax ;# of bytes to read
45509 00004A0C B80042 mov ax,4200h
45510 00004A0F F9 stc
45511 00004A10 CD21 int 21h ;move pointer
45512 00004A12 59 pop cx ;# of bytes to read
45513 00004A13 7209 jc short ricb_exit
45514
45515 00004A15 31D2 xor dx,dx ;ds:dx -> control buffer
45516 00004A17 31F6 xor si,si
45517 00004A19 B43F mov ah,3Fh ;read into the buffer
45518 00004A1B F9 stc
45519 00004A1C CD21 int 21h ;should be less than 1024 bytes.
45520 ricb_exit:
45521 00004A1E C3 retn
45522
45523 ;-----
45524
45525 ;! set_country_path procedure is not called from anywhere !
45526 ; Erdogan Tan - 04/08/2023
45527 %if 0
45528
45529 set_country_path:
45530
45531 ;-----
45532 ;in: ds - sysinitseg, es - confbot, si -> start of the asciiz path string
45533 ; dosinfo_ext, cntry_drv, cntry_root, cntry_path
45534 ; assumes current directory is the root directory.
45535 ;out: ds:di -> full path (cntry_drv).
45536 ; set the cntry_drv string from the country=,path command.
45537 ; ds, es, si value saved.
45538 ;-----
45539
45540 ; 04/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
45541 ; (SYSINIT:4EF4h)
45542
45543 ; 10/09/2023 - Retrodos v4.2 IO.SYS (Optimization)
45544 ; (Retrodos v5.0 Pre-Works)
45545 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:513Bh)
45546
45547 push si
45548
45549 push ds ;switch ds, es
45550 push es
45551 pop ds
45552 pop es ;now ds -> confbot, es -> sysinitseg
45553
45554 call chk_drive_letter ;current ds:[si] is a drive letter?
45555 jc short scp_default_drv ;no, use current default drive.
45556
45557 mov al,[si]
45558 inc si
45559 inc si ;si -> next char after ":"
45560 jmp short scp_setdrv
45561
45562 scp_default_drv:
45563 mov ah,19h
45564 int 21h
45565 add al,"A" ;convert it to a character.
45566
45567 scp_setdrv:
45568 mov [cs:cntry_drv],al ;set the drive letter.
45569 mov di,cntry_path
45570 mov al,[si]
45571 cmp al,"\"
45572 je short scp_root_dir
45573
45574 cmp al,"/" ;let's accept "/" as an directory delim
45575 ;je short scp_root_dir
45576 ;jmp short scp_path
45577 ; 04/01/2023
45578 ;jne short scp_path
45579
45580 scp_root_dir:
45581 dec di ;di -> cntry_root
45582 scp_path:
45583 call move_asciiz ;copy it
45584
45585 mov di,cntry_drv
45586 scpath_exit:
45587
45588 push ds ;switch ds, es
45589 push es
45590 pop ds
45591 pop es ;ds, es value restored
45592
45593 pop si
45594 retn
45595
45596 ;-----
45597

```

```

45598      chk_drive_letter:
45599
45600      ;check if ds:[si] is a drive letter followed by ":".
45601      ;assume that every alpha character is already converted to upper case.
45602      ;carry set if not.
45603
45604      ; 04/01/2023 - RetroDOS v4.2
45605
45606      push    ax
45607      cmp     byte [si], "A"
45608      ;jb     short cdletter_no
45609      jb      short cdletter_exit
45610      cmp     byte [si], "Z"
45611      ja      short cdletter_no
45612      cmp     byte [si+1], ":"
45613      ;jne     short cdletter_no
45614      ;jmp     short cdletter_exit
45615      ; 04/01/2023
45616      je      short cdletter_exit
45617
45618      cdletter_no:
45619      stc
45620      cdletter_exit:
45621      pop     ax
45622      retn
45623
45624      %endif
45625
45626      ;-----
45627
45628      move_asciiz:
45629
45630      ;in: ds:si -> source es:di -> target
45631      ;out: copy the string until 0.
45632      ;assumes there exists a 0.
45633
45634      ; 10/09/2023 - RetroDOS v4.2 IO.SYS (Optimization)
45635      ; (MSDOS 6.21 IO.SYS - SYSINIT:4F40h)
45636      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:5187h)
45637
45638      masciiz_loop:
45639      ; 10/09/2023
45640      test    byte [si], 0FFh
45641      movsb
45642      ;cmp     byte [si-1], 0 ; was it 0?
45643      ;jne     short masciiz_loop
45644      jnz     short masciiz_loop ; 10/09/2023
45645      retn
45646
45647      ;-----
45648
45649      ; ds:dx points to string to output (asciz)
45650      ;
45651      ; prints <badld_pre> <string> <badld_post>
45652
45653      badfil:
45654      push    cs
45655      pop     es
45656
45657      mov     si, dx
45658
45659      badload:
45660      mov     dx, badld_pre ; want to print config error
45661      mov     bx, crlfm
45662
45663      prnerr:
45664      push    cs
45665      pop     ds ; *
45666      call    print
45667
45668      prn1:
45669      mov     dl, [es:si]
45670      or      dl, dl
45671      jz      short prn2
45672      mov     ah, STD_CON_OUTPUT ; 2
45673      int     21h
45674      inc     si
45675      jmp     short prn1
45676
45677      prn2:
45678      mov     dx, bx
45679      call    print
45680      ; 11/12/2022
45681      ; ds = cs ; *
45682      cmp     byte [donotshownum], 1
45683      ; suppress line number when handling command.com
45684      ;cmp     byte [cs:donotshownum], 1
45685      je      short prnexit
45686
45687      ; 18/12/2022
45688      ;call    error_line
45689      jmp     error_line
45690
45691      ;prnexit:
45692      ;retn
45693
45694      ;-----
45695
45696      print:
45697      mov     ah, STD_CON_STRING_OUTPUT ; 9
45698      int     21h
45699
45700      prnexit:
45701      ; 18/12/2022
45702      retn
45703
45704      ;-----
45705
45706      ; open device pointed to by dx, al has access code
45707      ; if unable to open do a device open null device instead
45708
45709      ; 02/11/2022 - RetroDOS v4.0 (Modified MSDOS 5.0 IO.SYS)
45710      ; (SYSINIT:3764h)
45711
45712      open_dev:
45713      call    open_file
45714      jnc     short open_dev3
45715
45716      open_dev1:
45717      mov     dx, nuldev
45718      ; 18/12/2022
45719      ;call    open_file
45720
45721      ;of_retn:
45722      ;retn
45723      ; 18/12/2022
45724      ;jmp     short open_file
45725
45726      open_file:
45727      mov     ah, OPEN ; 3Dh
45728      stc
45729      int     21h
45730
45731      of_retn:
45732      ; 18/12/2022
45733      retn

```

```

45722
45723
45724 00004A65 89C3
45725
45726
45727
45728
45729
45730
45731
45732 00004A67 B80044
45733
45734 00004A6A CD21
45735
45736 00004A6C F6C280
45737 00004A6F 75F3
45738
45739 00004A71 B43E
45740 00004A73 CD21
45741 00004A75 EBE5
45742
45743
45744
45745
45746
45747
45748
45749
45750
45751
45752
45753
45754
45755
45756
45757
45758
45759 00004A77 B003
45760 00004A79 CF
45761
45762
45763
45764
45765
45766
45767
45768
45769
45770
45771
45772
45773
45774
45775
45776
45777
45778
45779
45780
45781
45782
45783
45784
45785
45786
45787
45788
45789
45790
45791
45792
45793
45794
45795
45796
45797
45798
45799
45800
45801
45802
45803
45804
45805
45806
45807
45808
45809
45810
45811
45812
45813
45814
45815
45816
45817
45818
45819
45820
45821 00004A7A 00
45822
45823
45824 00004A7B 0D0A
45825
45826
45827
45828
45829
45830
45831 00004A7D 526574726F20444F53-
45831 00004A86 2076352E3020284D6F-
45831 00004A8F 6469666696564205043-
45831 00004A98 444F5320372E312920
45832
45833 00004AA1 0D0A
45834
45835 00004AA3 6279204572646F6761-
45835 00004AAC 6E2054616E205B3230-
45835 00004AB5 32365D20
45836 00004AB9 0D0A
45837 00004ABB 0D0A2400
45838
45839 00004ABF 4E554C00
45840 00004AC3 434F4E00

open_dev3:
    mov     bx,ax                ; handle from open to bx
    ;xor     ax,ax                ; get device info
    ;mov     ah,IOCTL ; 44h
    ;mov     ax,(IOCTL<<8) ; 13/05/2019
    ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
    ;xor     ax,ax
    ;mov     ah,44h ; IOCTL
    ; 11/12/2022
    mov     ax,4400h ; IOCTL<<8

    int     21h

    test    dl,10000000b ; 80h
    jnz     short of_retn

    mov     ah,CLOSE ; 3Eh
    int     21h
    jmp     short open_dev1

;-----
; 18/12/2022
%if 0
open_file:
    mov     ah,OPEN ; 3Dh
    stc
    int     21h
    retn
%endif

;-----
; test int24. return back to dos with the fake user response of "fail"

int24:
    mov     al,3                ; fail the system call
    iret                     ; return back to dos.

; 19/04/2019 - Retro DOS v4.0

;-----
; DATA
;-----

;include copyrigh.inc                ; copyright statement

; MSDOS 6.21 IO.SYS - SYSINIT:4FA3h

;MSDosVersion6Copyr:
; db      'MS DOS Version 6 (C)Copyright 1981-1993 Microsoft Corp '
; db      'Licensed Material - Property of Microsoft All rights reserved '

; 22/10/2022
; MSDOS 5.0 IO.SYS - SYSINIT:378Ch

; 28/12/2022
%if 0
; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
MSDosVersion5Copyr:
    db      'MS DOS Version 5.00 (C)Copyright 1981-1991 Microsoft Corp '
    db      'Licensed Material - Property of Microsoft All rights reserved '
%endif

; 13/04/2024 - Retro DOS v5.0
; PCDOS 7.1 IBMBIO.COM - SYSINIT:51EAh (IBMBIO.COM offset 42266)
%if 0
IBMDOSV71COPYR:
    db      'IBM DOS Version 7.1 (C)Copyright 1981-2002 IBM Corporation '
    db      'Licensed Material - Property of IBM All rights reserved '
%endif

; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
; 20/04/2019 - Retro DOS v4.0
;BOOTMES:
; db      13
; db      10
; db      "MS-DOS version "
; db      MAJOR_VERSION + "0"
; db      "."
; db      (MINOR_VERSION / 10) + "0"
; db      (MINOR_VERSION % 10) + "0"
; db      13,10
; db      "Copyright 1981-1993 Microsoft Corp.",13,10,"$"
; ; 22/10/2022
; db      "Copyright 1981-1991 Microsoft Corp.",13,10,"$"
; ;
; db      0

; 01/01/2023 - Retro DOS v4.2

; 28/12/2022 - Retro DOS v4.1
;MSDosVersion5Copyr:
; db      13,10,"MS DOS Version 5.0"
; db      13,10,"Copyright 1981-1991 Microsoft Corp.",13,10,"$",0

; 12/12/2022
db      0
; 12/12/2022
;BOOTMES:
db      13,10
;;db      "Retro DOS v4.0 (Modified MSDOS 5.0) "
; 28/12/2022
;;db      "Retro DOS v4.1 (Modified MSDOS 5.0) "
; 01/01/2023
;db      "Retro DOS v4.2 (Modified MSDOS 6.22) "
; 30/12/2023
db      "Retro DOS v5.0 (Modified PCDOS 7.1) "

db      13,10
;db      "by Erdogan Tan [2024] " ; 01/01/2024
db      "by Erdogan Tan [2026] " ; 19/01/2026

db      13,10
db      13,10,"$",0

nuldev:    db      "NUL",0
condev:    db      "CON",0

```

```

45841 00004AC7 41555800      auxdev:    db      "AUX",0
45842 00004ACB 50524E00      prndev:    db      "PRN",0
45843
45844
45845 00004ACF 5C434F4E4649472E53-  ;IFDEF      CONFIGPROC
45845 00004AD8 595300      config:    db      "\CONFIG.SYS",0
45846
45847 00004ADB 413A      cntry_drv: db      "A:"
45848 00004ADD 5C      cntry_root: db      "\"
45849 00004ADE 434F554E5452592E53-  cntry_path: db      "COUNTRY.SYS",0
45849 00004AE7 595300
45850      ;db 52 dup (0)
45851 00004AEA 00<rep 34h>      times 52 db 0
45852
45853      country_file_signature:
45854 00004B1E FF434F554E545259      db      0FFh,"COUNTRY"
45855
45856      cntrycodepage_id:
45857 00004B26 0000      dw      0
45858
45859      ;ENDIF ; CONFIGPROC
45860
45861      ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
45862      ; (SYSINIT:5081h)
45863
45864      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
45865      ;ifdef      MULTI_CONFIG
45866 00004B28 00      newcmd:    db 0      ; non-zero if non-std shell specified
45867 00004B29 40      tmplate:   db 64      ; must precede commnd
45868      ;endif
45869
45870      ;ifdef ROMEXEC
45871      ; db 7      ; size of commnd line (excl. null)
45872      ;commnd: db "COMMAND",0
45873      ; db 56 dup (0)
45874      ;else
45875      ; 02/11/2022
45876 00004B2A 0C      db 12      ; size of commnd line (excl. null)
45877 00004B2B 5C434F4D4D414E442E-  commnd:    db      "\COMMAND.COM",0
45877 00004B34 434F4D00
45878      ;db 51 dup (0)
45879 00004B38 00<rep 33h>      times 51 db 0
45880      ;endif
45881
45882      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
45883      ;ifdef      MULTI_CONFIG
45884 00004B6B 5C434F4D4D414E442E-  commnd2:   db      "\COMMAND.COM",0      ; alternate commands to exec,
45884 00004B74 434F4D00      db 2,"/P",0      ; followed by their respective alternate
45885 00004B78 022F5000      commnd3:   db "\MSDOS\COMMAND.COM",0; command lines
45886 00004B7C 5C4D53444F535C434F-
45886 00004B85 4D4D414E442E434F4D-
45886 00004B8E 00
45887 00004B8F 0B413A5C4D53444F53-
45887 00004B98 202F5000
45888 00004B9C 5C444F535C434F4D4D-
45888 00004BA5 414E442E434F4D00
45889 00004BAD 09413A5C444F53202F-
45889 00004BB6 5000
45890
45891 00004BB8 00      def_swchr: db 0      ; default switchchar (referenced as command_line-1)
45892      ;endif
45893      ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
45894      command_line:
45895 00004BB9 022F50      db 2,"/P"      ; default command.com args
45896      ;db 125 dup (0)
45897 00004BBC 00<rep 7Dh>      times 125 db 0
45898
45899      pathstring:
45900      ;db 64 dup (0)
45901 00004C39 00<rep 40h>      times 64 db 0
45902
45903      ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
45904      ; (SYSINIT:51D3h)
45905      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
45906      ;%if 0
45907
45908      dae_flag:
45909 00004C79 00      db 0 ; MSDOS 6.21 IO.SYS - SYSINIT:51D2h
45910
45911      ;ifdef      MULTI_CONFIG
45912
45913      ; 04/03/2022- Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS, SYSINIT)
45914      MAX_MULTI_CONFIG equ 9      ; max # of multi-config menu items supported
45915
45916      ; Beware of byte pairs accessed as words (see all "KEEP AFTER" notes below)
45917
45918 00004C7A 07      bMenuColor: db 07h ; 1Fh      ; default fgnd/bgnd color
45919 00004C7B 00      bMenuPage:  db 0      ; menu video page (KEEP AFTER bMenuColor)
45920 00004C7C 05      db 5      ; video page function # (KEEP AFTER bMenuPage)
45921 00004C7D 00      bLastCol:   db 0      ; ending column on status line
45922 00004C7E 18      bLastRow:   db 24     ; row # of status line (KEEP AFTER bLastCol)
45923 00004C7F 00      bDisableUI: db 0      ; 1=disable clean/interactive
45924      ; 2=disable default 2-second delay
45925 00004C80 00      bCRTPage:   db 0      ; value saved from BIOS data area
45926 00004C81 0000      wCRTStart:  dw 0      ; value saved from BIOS data area
45927 00004C83 00      bQueryOpt:  db 0      ; 0=off, 1=prompt all, 2=prompt none, 4=skip all
45928 00004C84 01      bDefBlock:  db 1      ; default block #
45929 00004C85 00      bMaxBlock:  db 0      ; maximum block #
45930 00004C86 0000      offDefBlock: dw 0      ; offset of name of default block (if any)
45931 00004C88 FF      secTimeout: db -1 ; 0FFh      ; # of seconds for timeout (-1 == indefinite)
45932 00004C89 00      secElapsed: db 0      ; # of seconds elapsed so far (KEEP AFTER secTimeout)
45933 00004C8A 00<rep Ah>      abBlockType: times MAX_MULTI_CONFIG+1 db 0 ; array of block types
45934 00004C94 0000<rep Ah>      aoffBlockName: times MAX_MULTI_CONFIG+1 dw 0 ; array of offsets of block names
45935 00004CA8 0000<rep Ah>      aoffBlockDesc: times MAX_MULTI_CONFIG+1 dw 0 ; array of offsets of block descriptions
45936
45937 00004CBC 434F4E4649473D00      szBoot:     db      "CONFIG=",0
45938 00004CC4 4D454E5500      szMenu:     db      "MENU",0
45939 00004CC9 434F4D4D4F4E00      szCommon:   db      "COMMON",0
45940
45941      ;endif      ;MULTI_CONFIG
45942
45943      ; 10/09/2023
45944      ; MSDOS 6.21 IO.SYS - SYSINIT:5229h
45945      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:546Eh)
45946
45947      comtab:      ; label byte
45948
45949      ;      cmd len      command      cmd code
45950      ;      -----      -----      -----
45951
45952      ;ifdef MULTI_CONFIG
45953 00004CD0 015B5B      db 1,      "[",      CONFIG_BEGIN
45954      ;endif
45955 00004CD3 05425245414B43      db 5,      "BREAK",  CONFIG_BREAK

```

```

45956 00004CDA 074255464645525342          db      7,      "BUFFERS",      CONFIG_BUFFERS
45957 00004CE3 07434F4D4D454E5459          db      7,      "COMMENT",      CONFIG_COMMENT
45958 00004CEC 07434F554E54525951          db      7,      "COUNTRY",      CONFIG_COUNTRY
45959 00004CF5 0644455649434544          db      6,      "DEVICE",      CONFIG_DEVICE
45960 00004CFD 0A4445564943454849-          db      10,     "DEVICEHIGH",    CONFIG_DEVICEHIGH
45961 00004D06 474855
45962 00004D09 03444F5348          db      3,      "DOS",          CONFIG_DOS
45963 00004D0E 08445249565041524D-          db      8,      "DRIVPARM",    CONFIG_DRIVPARM
45964 00004D17 50
45965 00004D18 044643425358          db      4,      "FCBS",          CONFIG_FCBS
45966 00004D1E 0546494C455346          db      5,      "FILES",         CONFIG_FILES
45967 ;ifdef MULTI_CONFIG
45968 db      7,      "INCLUDE",      CONFIG_INCLUDE
45969 ;endif
45970 db      7,      "INSTALL",      CONFIG_INSTALL
45971 db      11,     "INSTALLHIGH",   CONFIG_INSTALLHIGH
45972
45973 db      9,      "LASTDRIVE",     CONFIG_LASTDRIVE
45974 ;ifdef MULTI_CONFIG
45975 db      7,      "SUBMENU",      CONFIG_SUBMENU
45976 db      9,      "MENUMCOLOR",    CONFIG_MENUMCOLOR
45977
45978 db      11,     "MENUDEFAULT",   CONFIG_MENUDEFAULT
45979
45980 db      8,      "MENUITEM",      CONFIG_MENUITEM
45981 ;endif
45982 db      10,     "MULTITRACK",     CONFIG_MULTITRACK
45983 ;ifdef MULTI_CONFIG
45984 db      7,      "NUMLOCK",      CONFIG_NUMLOCK
45985 ;endif
45986 db      3,      "REM",          CONFIG_REM
45987 ;ifdef MULTI_CONFIG
45988 db      3,      "SET",          CONFIG_SET
45989 ;endif
45990 db      5,      "SHELL",        CONFIG_SHELL
45991 ;if STACKSW
45992 db      6,      "STACKS",       CONFIG_STACKS
45993 ;endif
45994 db      8,      "SWITCHES",     CONFIG_SWITCHES
45995
45996 ; 18/03/2025 (BugFix)
45997 ;db      0
45998
45999 ; 10/09/2023
46000 ;aDosdata: ; PCDOS 7.1 IBMBIO.COM - SYSINIT:5550h
46001 ; 13/04/2024 - Retro DOS v5.0
46002 db      7,      "DOSDATA",      CONFIG_DOSDATA ; 'T'
46003 db      0
46004
46005 ;%endif ; 02/11/2022
46006
46007 ; 01/01/2023 - Retro DOS v4.2
46008 %if 0
46009 comtab:
46010 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
46011 ; (SYSINIT:38Edh)
46012 db      7,      "BUFFERS",      CONFIG_BUFFERS
46013 db      5,      "BREAK",        CONFIG_BREAK
46014 db      6,      "DEVICE",        CONFIG_DEVICE
46015 db      10,     "DEVICEHIGH",    CONFIG_DEVICEHIGH
46016 db      5,      "FILES",         CONFIG_FILES
46017 db      4,      "FCBS",          CONFIG_FCBS
46018 db      9,      "LASTDRIVE",     CONFIG_LASTDRIVE
46019 db      10,     "MULTITRACK",     CONFIG_MULTITRACK
46020 db      8,      "DRIVPARM",      CONFIG_DRIVPARM
46021 db      6,      "STACKS",        CONFIG_STACKS
46022 db      7,      "COUNTRY",       CONFIG_COUNTRY
46023 db      5,      "SHELL",         CONFIG_SHELL
46024 db      7,      "INSTALL",       CONFIG_INSTALL
46025 db      7,      "COMMENT",       CONFIG_COMMENT
46026 db      3,      "REM",           CONFIG_REM
46027 db      8,      "SWITCHES",      CONFIG_SWITCHES
46028 db      3,      "DOS",          CONFIG_DOS
46029 db      0
46030
46031 %endif
46032
46033 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
46034 ; (SYSINIT:530Ch)
46035
46036 ; 13/04/2024 - Retro DOS v5.0
46037 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:555Ah)
46038
46039 deviceparameters:
46040 ; A_DEVICEPARAMETERS <0,dev_3inch720kb,0,80>
46041 devp.specialfunc: ; deviceparameters +
46042 db      0          ; A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS
46043 devp.devtype:
46044 db      2          ; A_DEVICEPARAMETERS.DP_DEVICECTYPE
46045 devp.devattr:
46046 dw      0          ; A_DEVICEPARAMETERS.DP_DEVICEATTRIBUTES
46047 devp.cylinders:
46048 dw      80         ; A_DEVICEPARAMETERS.DP_CYLINDERS
46049
46050 ; 04/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
46051
46052 ;times 286 db 0
46053 devp.mediatype:
46054 db      0          ; A_DEVICEPARAMETERS.DP_MEDIATYPE
46055 devp.bpb:
46056 ; A_DEVICEPARAMETERS.DP_BPB
46057 devp.bps:
46058 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.BYTESPERSECTOR
46059 dw      0
46060 devp.secpersclus:
46061 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERCLUSTER
46062 db      0
46063 dw      0
46064 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.RESERVEDSECTORS
46065 db      0
46066 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.NUMBEROFFATS
46067 dw      0
46068 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.ROOTENTRIES
46069 devp.totalsecs:
46070 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.TOTALSECTORS
46071 dw      0
46072 devp.mediaid:
46073 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.MEDIADESCRIPTOR
46074 db      0
46075 dw      0
46076 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERFAT
46077 devp.spt:
46078 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK
46079 dw      0
46080 devp.heads:
46081 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.HEADS
46082 dw      0
46083
46084 ; 13/04/2024 - Retro DOS v5.0
46085 ; (PCDOS 7.1 IBMBIO.COM)
46086 times 68 db 0 ; PCDOS 7.1 (FAT32 BPB)
46087
46088 00004DD4 00<rep 44h>

```



```

46071      ;;times 14 db 0 ; MSDOS 6.21
46072      ;dw      0      ; A_DEVICEPARAMETERS.DP_BPBA+A_BPBA.HIDDENSECTORS
46073      ;dw      0
46074      ;dw      0      ; A_DEVICEPARAMETERS.DP_BPBA+A_BPBA.BIGTOTALSECTORS
46075      ;dw      0
46076      ;times 6 db 0
46077
46078      devp.trkbtblents:
46079      dw      0      ; A_DEVICEPARAMETERS.DP_TRACKTABLEENTRIES
46080      devp.secttbl:      ; A_DEVICEPARAMETERS.DP_SECTORTABLE
46081      times 252 db 0 ; MAX_SECTORS_IN_TRACK * A_SECTORTABLE.size
46082      ; 63*4 bytes
46083
46084      ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
46085      ; (SYSINIT:5430h)
46086
46087      ; 13/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
46088      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:56B4h)
46089
46090      hlim:      dw      2
46091      slim:      dw      9
46092
46093      drive:      db      0
46094
46095      switches:
46096      dw      0
46097
46098      ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
46099      ; (SYSINIT:5437h)
46100
46101      ; the following are the recommended bpbs for the media that
46102      ; we know of so far.
46103
46104      ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
46105      ; MSDOS 5.0 IO.SYS - SYSINIT:3AA9h
46106
46107      ; 27/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
46108      ; PCDOS 7.1 IBMBIO.COM - SYSINIT:56BBh
46109
46110      ; 48 tpi diskettes
46111
46112      bpb48t:      dw      512
46113      db      2
46114      dw      1
46115      db      2
46116      dw      112
46117      dw      2*9*40 ; 720
46118      db      0FDh
46119      dw      2
46120      dw      9
46121      dw      2
46122      dd      0
46123      ; 27/12/2023
46124      times 28 db 0 ; FAT32 extensions (to BDS)
46125      db      90h
46126
46127      ; 96tpi diskettes
46128
46129      bpb96t:      dw      512
46130      db      1
46131      dw      1
46132      db      2
46133      dw      224
46134      dw      2*15*80 ; 2400
46135      db      0F9h
46136      dw      7
46137      dw      15
46138      dw      2
46139      dd      0
46140      ; 27/12/2023
46141      times 28 db 0 ; FAT32 extensions (to BDS)
46142      db      90h
46143
46144      ; 3 1/2 inch diskette bpb
46145
46146      bpb35:      dw      512
46147      db      2
46148      dw      1
46149      db      2
46150      dw      112
46151      dw      2*9*80 ; 1440
46152      db      0F9h
46153      dw      3
46154      dw      9
46155      dw      2
46156      dd      0
46157      ; 27/12/2023
46158      times 28 db 0 ; FAT32 extensions (to BDS)
46159      db      90h
46160
46161      bpb35h:      dw      512
46162      db      1
46163      dw      1
46164      db      2
46165      dw      224
46166      dw      2*18*80 ; 2880
46167      db      0F0h
46168      dw      9
46169      dw      18
46170      dw      2
46171      dd      0
46172      ; 27/12/2023
46173      times 28 db 0 ; FAT32 extensions (to BDS)
46174      db      90h
46175
46176      ; m037 - BEGIN
46177
46178      bpb288:      dw      512
46179      db      2
46180      dw      1
46181      db      2
46182      dw      240
46183      dw      2*36*80 ; 5760
46184      db      0F0h
46185      dw      9
46186      dw      36
46187      dw      2
46188      dd      0
46189      ; 27/12/2023
46190
46191      ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
46192      ; (SYSINIT:5430h)
46193
46194      ; 13/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
46195      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:56B4h)
46196
46197      hlim:      dw      2
46198      slim:      dw      9
46199
46200      drive:      db      0
46201
46202      switches:
46203      dw      0
46204
46205      ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
46206      ; (SYSINIT:5437h)
46207
46208      ; the following are the recommended bpbs for the media that
46209      ; we know of so far.
46210
46211      ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
46212      ; MSDOS 5.0 IO.SYS - SYSINIT:3AA9h
46213
46214      ; 27/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
46215      ; PCDOS 7.1 IBMBIO.COM - SYSINIT:56BBh
46216
46217      ; 48 tpi diskettes
46218
46219      bpb48t:      dw      512
46220      db      2
46221      dw      1
46222      db      2
46223      dw      112
46224      dw      2*9*40 ; 720
46225      db      0FDh
46226      dw      2
46227      dw      9
46228      dw      2
46229      dd      0
46230      ; 27/12/2023
46231      times 28 db 0 ; FAT32 extensions (to BDS)
46232      db      90h
46233
46234      ; 96tpi diskettes
46235
46236      bpb96t:      dw      512
46237      db      1
46238      dw      1
46239      db      2
46240      dw      224
46241      dw      2*15*80 ; 2400
46242      db      0F9h
46243      dw      7
46244      dw      15
46245      dw      2
46246      dd      0
46247      ; 27/12/2023
46248      times 28 db 0 ; FAT32 extensions (to BDS)
46249      db      90h
46250
46251      ; 3 1/2 inch diskette bpb
46252
46253      bpb35:      dw      512
46254      db      2
46255      dw      1
46256      db      2
46257      dw      112
46258      dw      2*9*80 ; 1440
46259      db      0F9h
46260      dw      3
46261      dw      9
46262      dw      2
46263      dd      0
46264      ; 27/12/2023
46265      times 28 db 0 ; FAT32 extensions (to BDS)
46266      db      90h
46267
46268      bpb35h:      dw      512
46269      db      1
46270      dw      1
46271      db      2
46272      dw      224
46273      dw      2*18*80 ; 2880
46274      db      0F0h
46275      dw      9
46276      dw      18
46277      dw      2
46278      dd      0
46279      ; 27/12/2023
46280      times 28 db 0 ; FAT32 extensions (to BDS)
46281      db      90h
46282
46283      ; m037 - BEGIN
46284
46285      bpb288:      dw      512
46286      db      2
46287      dw      1
46288      db      2
46289      dw      240
46290      dw      2*36*80 ; 5760
46291      db      0F0h
46292      dw      9
46293      dw      36
46294      dw      2
46295      dd      0
46296      ; 27/12/2023

```

```

46195 0000500E 00<rep 1Ch>          times 28 db 0          ; FAT32 extensions (to BDS)
46196 0000502A 90                   db      90h
46197
46198 ; m037 - END
46199
46200 ; 12/05/2019
46201
46202 0000502B 90                   align 2
46203
46204 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
46205 ; MSDOS 5.0 IO.SYS - SYSINIT:3B26h
46206
46207 ; 13/04/2024 - Retro DOS v5.0
46208 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:5738h)
46209
46210 0000502C [1D4F]          bpbtable: dw      bpb48t          ; 48tpi drives
46211 0000502E [534F]          dw      bpb96t          ; 96tpi drives
46212 00005030 [894F]          dw      bpb35          ; 3.5" drives
46213 ; the following are not supported, so default to 3.5" media layout
46214 00005032 [894F]          dw      bpb35          ; not used - 8" drives
46215 00005034 [894F]          dw      bpb35          ; not used - 8" drives
46216 00005036 [894F]          dw      bpb35          ; not used - hard files
46217 00005038 [894F]          dw      bpb35          ; not used - tape drives
46218 0000503A [BF4F]          dw      bpb35h         ; 3-1/2" 1.44mb drive
46219 0000503C [894F]          dw      bpb35          ; ERIMO
46220 0000503E [F54F]          dw      bpb288         ; 2.88 MB diskette drives      m037
46221
46222 switchlist:
46223 00005040 08464853544449434E    db      8,"FHSTDICN"          ; preserve the positions of n and c.
46224
46225 ;-----
46226 ; Messages
46227 ;-----
46228
46229 ; 19/04/2019 - Retro DOS v4.0
46230
46231 ; MSDOS 6.21 IO.SYS - SYSINIT:54D1h
46232
46233 00005049 00                   db      0
46234
46235 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
46236 ; MSDOS 5.0 IO.SYS - SYSINIT:3B44h
46237
46238 ; 13/04/2024
46239 ; MSDOS 6.22 IO.SYS - SYSINIT:559Eh
46240
46241 ; 13/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
46242 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:5756h
46243
46244 badopm:
46245 0000504A 0D0A                db      0Dh,0Ah
46246 0000504C 556E7265636F676E69-    db      'Unrecognized command in CONFIG.SYS'
46247
46248 0000506E 0D0A24              db
46249
46250 00005071 0D0A                db      0Dh,0Ah
46251 00005073 42616420636F6D6D61-    db      'Bad command or parameters - $'
46252 0000507C 6E64206F7220706172-
46253 00005085 616D6574657273202D-
46254 0000508E 2024
46255
46256 00005090 0D0A                db      0Dh,0Ah
46257 00005092 536563746F72207369-    db      'Sector size too large in file $'
46258 0000509B 7A6520746F6F206C61-
46259 000050A4 72676520696E206669-
46260 000050AD 6C652024
46261
46262 000050B1 0D0A                db      0Dh,0Ah
46263 000050B3 426164206F72206D69-    db      'Bad or missing $'
46264 000050BC 7373696E672024
46265
46266 000050C3 436F6D6D616E642049-    db
46267 000050CC 6E7465727072657465-    db      'Command Interpreter',0
46268 000050D5 7200
46269
46270 000050D7 0D0A                db      0Dh,0Ah
46271 000050D9 496E76616C69642063-    db      'Invalid country code or code page',0Dh,0Ah,'$'
46272 000050E2 6F756E74727920636F-
46273 000050EB 6465206F7220636F64-
46274 000050F4 6520706167650D0A24
46275
46276 000050FD 0D0A                db      0Dh,0Ah
46277 000050FF 4572726F7220696E20-    db      'Error in COUNTRY command',0Dh,0Ah,'$'
46278 00005108 434F554E5452592063-
46279 00005111 6F6D6D616E640D0A24
46280
46281 0000511A 0D0A                db      0Dh,0Ah
46282 0000511C 496E73756666696369-    db      'Insufficient memory for COUNTRY.SYS file',0Dh,0Ah,'$'
46283 00005125 656E74206D656D6F72-
46284 0000512E 7920666F7220434F55-
46285 00005137 4E5452592E53595320-
46286 00005140 66696C650D0A24
46287
46288 00005147 0D0A                db      0Dh,0Ah
46289 00005149 436F6E666967757261-    db      'Configuration too large for memory',0Dh,0Ah,'$'
46290 00005152 74696F6E20746F6F20-
46291 0000515B 6C6172676520666F72-
46292 00005164 206D656D6F72790D0A-
46293 0000516D 24
46294
46295 0000516E 0D0A                db      0Dh,0Ah
46296 00005170 546F6F206D616E7920-    db      'Too many block devices',0Dh,0Ah,'$'
46297 00005179 626C6F636B20646576-
46298 00005182 696365730D0A24
46299
46300 00005189 0D0A                db      0Dh,0Ah
46301 0000518B 496E76616C69642053-    db      'Invalid STACK parameters',0Dh,0Ah,'$'
46302 00005194 5441434B2070617261-
46303 0000519D 6D65746572730D0A24
46304
46305 ; 18/12/2022
46306 ;badorder:
46307 ;db      0Dh,0Ah
46308 ;db      'Incorrect order in CONFIG.SYS line $'
46309
46310 errorcmd:
46311 db      'Error in CONFIG.SYS line $'
46312
46313 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
46314 ; (SYSINIT:566Eh)

```



```

46288 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
46289 ;%if 0
46290
46291 000051C0 4F4E OnOff: db 'ON'
46292 000051C2 4F4646 OnOff2: db 'OFF'
46293
46294 ; 04/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
46295 ; (SYSINIT:5673h)
46296 ;StartMsg:
46297 ; db 'Starting MS-DOS...',0Dh,0Ah
46298 ; db 0Ah,0
46299
46300 ; 17/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
46301 ; (SYSINIT:58F7h)
46302 StartMsg:
46303 000051C5 5374617274696E6720- db 'Starting PC DOS...',0Dh,0Ah
46303 000051CE 504320444F532E2E2E-
46303 000051D7 0D0A
46304 000051D9 0A00 db 0Ah,0
46305
46306 _$PauseMsg:
46307 ; 17/12/2023
46308 ;db 'Press any key to continue . . .',0Dh,0Ah,'$'
46309 ; 04/08/2023 (PCDOS 7.10 - IBMBIO.COM SYSINIT:590Dh)
46310 000051DB 507265737320616E79- db 'Press any key to continue...',0Dh,0Ah,'$'
46310 000051E4 206B657920746F2063-
46310 000051ED 6F6E74696E75652E2E-
46310 000051F6 2E0D0A24
46311
46312 _$CleanMsg:
46313 ;db 'MS-DOS is bypassing your CONFIG.SYS and AUTOEXEC.BAT files.',0Dh,0Ah,'$'
46314 000051FA 504320444F53206973- ; 17/12/2023
46314 00005203 20627970617373696E- db 'PC DOS is bypassing your CONFIG.SYS and AUTOEXEC.BAT files.',0Dh,0Ah,'$'
46314 0000520C 6720796F757220434F-
46314 00005215 4E4649472E53595320-
46314 0000521E 616E64204155544F45-
46314 00005227 5845432E4241542066-
46314 00005230 696C65732E0D0A24
46315
46316 _$InterMsg:
46317 ;db 'MS-DOS will prompt you to confirm each CONFIG.SYS command.',0Dh,0Ah,'$'
46318 00005238 504320444F53207769- ; 17/12/2023
46318 00005241 6C6C2070726F6D7074- db 'PC DOS will prompt you to confirm each CONFIG.SYS command.',0Dh,0Ah,'$'
46318 0000524A 20796F7520746F2063-
46318 00005253 6F6E6669726D206561-
46318 0000525C 636820434F4E464947-
46318 00005265 2E53595320636F6D6D-
46318 0000526E 616E642E0D0A24
46319
46320 00005275 0D0A _$MenuHeader:
46321 db 0Dh,0Ah
46322 ; 17/12/2023
46323 ;db ' MS-DOS 6.2 Startup Menu',0Dh,0Ah
46324 ;db '
46325 ;times 23 db (0Cdh) ; ALT 205 ; '===== ' ; 06/08/2023
46326 ;db 0Dh,0Ah,'$'
46327 00005277 2020504320444F5320- ; 04/08/2023 (PCDOS 7.10 - IBMBIO.COM SYSINIT:59A7h)
46327 00005280 372E31205374617274- db ' PC DOS 7.1 Startup Menu',0Dh,0Ah
46327 00005289 7570204D656E750D0A
46328 00005292 2020
46329 00005294 CD<rep 17h>
46330 000052AB 0D0A24 db '
46331
46332 000052AE 2020456E7465722061- _$MenuPrmpt:
46332 000052B7 2063686F6963653A20- db ' Enter a choice: '$
46332 000052C0 24
46333
46334 000052C1 46353D427970617373- _$StatusLine:
46334 000052CA 207374617274757020- db 'F5=Bypass startup files F8=Confirm each line of CONFIG.SYS '
46334 000052D3 66696C65732046383D-
46334 000052DC 436F6E6669726D2065-
46334 000052E5 616368206C696E6520-
46334 000052EE 6F6620434F4E464947-
46334 000052F7 2E53595320
46335 000052FC 616E64204155544F45- db 'and AUTOEXEC.BAT [ ]$'
46335 00005305 5845432E424154205B-
46335 0000530E 205D24
46336
46337 _$InterPrmpt:
46338 ;db '[Y,N]?$'
46339 ; 13/04/2024
46340 00005311 205B592C4E2C455343- ; 04/08/2023
46340 0000531A 5D3F24 db '[Y,N,ESC]?$' ; PCDOS 7.1 - IBMBIO.COM
46341 0000531D 59455324
46342 00005321 4E4F2024
46343
46344 00005325 54696D652072656D61- _$YES: db 'YES$'
46344 0000532E 696E696E673A2024 _$NO: db 'NO $'
46345
46346 _$TimeOut:
46347 db 'Time remaining: '$
46348 00005336 456E74657220636F72- badcomprompt:
46348 0000533F 72656374206E616D65- ;db 'Enter correct name of Command Interpreter (eg, C:\COMMAND.COM)'
46348 00005348 206F6620436F6D6D61- ; 13/04/2024 (PCDOS 7.1 IBMBIO.COM)
46348 00005351 6E6420496E74657270- db 'Enter correct name of Command Interpreter (for example, C:\COMMAND.COM)'
46348 0000535A 72657465722028666F-
46348 00005363 72206578616D706C65-
46348 0000536C 2C20433A5C434F4D4D-
46348 00005375 414E442E434F4D29
46349 0000537D 0D0A24 db 0Dh,0Ah,'$'
46350
46351 00005380 50726F636573732041- _$AutoPrmpt:
46351 00005389 55544F455845432E42- db 'Process AUTOEXEC.BAT [Y,N]?$'
46351 00005392 4154205B592C4E5D3F-
46351 0000539B 24
46352
46353 ;%endif ; 02/11/2022
46354
46355 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
46356 ; (SYSINIT:5840h)
46357
46358 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
46359 ; MSDOS 5.0 IO.SYS - SYSINIT:3CE0h
46360
46361 TooManyDrivesMsg:
46362 0000539C 5741524E494E472120- db 'WARNING! Logical drives past Z: exist and will be ignored',0Dh,0Ah,'$'
46362 000053A5 4C6F676963616C2064-
46362 000053AE 726976657320706173-
46362 000053B7 74205A3A2065786973-
46362 000053C0 7420616E642077696C-
46362 000053C9 6C2062652069676E6F-
46362 000053D2 7265640D0A24
46363
46364 ; MSDOS 6.21 IO.SYS - SYSINIT:587Ch

```

```

46365             ;db      'wrong DBLSPACE.BIN version',0Dh,0Ah,'$'
46366             ;db      7 dup(0)
46367
46368             ;times 7 db 0
46369             ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
46370 ; MSDOS 5.0 IO.SYS - SYSINIT:3D1Ch
46371             ; 09/12/2022
46372             ;times 4 db 0
46373
46374             ; 08/04/2024 - Retro DOS v5.0
46375 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:5B0Bh
46376 baddb1space:
46377 000053D8 526571756972656420-
46377 000053E1 73797374656D20636F-
46377 000053EA 6D706F6E656E742069-
46377 000053F3 73206E6F7420696E73-
46377 000053FC 74616C6C65640D0A24-
46377 00005405 00
46378
46379             ;db      7 dup(0)
46380
46381             ;-----
46382             ; 09/12/2022
46383             ;db 0
46384
46385 number3div equ ($-SYSINIT$)
46386 number3mod equ (number3div % 16)
46387
46388 %if number3mod>0 & number3mod<16
46388 00005406 00<rep Ah>         times (16-number3mod) db 0
46389 %endif
46390
46391             ;-----
46392             ; 09/12/2022 - MSDOS 5.0 IO.SYS:3D20h ;; SI_end = 3D20h for MSDOS 5.0 IO.SYS
46393             ;-----
46394
46395 ;MSDOS 6.21 IO.SYS - SYSINIT:5899h
46396
46397             ;-----
46398             ; 20/04/2019 - Retro DOS v4.0
46399
46400             ; 09/12/2022
46401             ;
46402 ;bss_start:
46403             ;
46404 ;ABSOLUTE bss_start
46405             ;
46406 ;alignb 16
46407
46408 SI_end:  ; SI_end equ $
46409
46410             ;-----
46411
46412 ;sysinitseg ends
46413
46414 ; *****
46415
46416 ; 04/01/2023 - MSDOS 6.21 SYSINIT:SI_end = SYSINIT:58A0h (IOSYS:9F46h)
46417 ; 09/12/2022 - MSDOS 5.0 SYSINIT:SI_end = SYSINIT:3D20h
46418
46419 SYSINITSIZE equ SI_end - SYSINIT$
46420 DOSLOADSEG equ SYSINITSEG+((SYSINITSIZE+15)/16)
46421
46422             ;-----
46423             ; End of Retro DOS 5.0 (PCDOS 7.1) IBMBIO.COM src by Erdogan Tan -21/04/2024-
46424             ;-----

```